

基于联合误差场景与分布鲁棒优化的微网购电及储能调度

摘 要

针对非孤岛式微网中光伏出力、负荷需求及外网电价变化引起的供需失配与购电成本不确定性,本文以 10 min 为统一调度尺度,严格遵循信息非前瞻原则,按照“确定性日前调度–源荷不确定性下的随机补救–多时点预报更新下的滚动调整–源荷价联合不确定性下的分布鲁棒调度”的递进思路,建立**微网购电与储能协同优化模型**。

问题一中,在负荷、电价和光伏预测已知的条件下,建立考虑能量平衡、储能状态转移及充放电互斥等约束的**混合整数线性规划模型**。最优方案全天购电量为 59482.70 kWh,购电费用为 35126.95 元,且无弃光发生,各项运行约束均得到满足。

问题二进一步考虑源荷预测误差,采用因果滚动预测构造**源荷联合样本外误差场景**,并建立带参数化非前瞻储能补救策略的随机优化模型。由 5 倍紧急购电机制可得净负荷低估与高估的边际损失比为 4:1,对应 80% 临界分位。2–12 月真实回放中,紧急购电量仅占实际外购电量的 0.60%,但费用占比达到 4.77%,表明供能不足具有显著的**非对称经济风险**。

问题三利用 0:00、6:00、12:00 和 18:00 的最新光伏预报及当前储能状态滚动修正后续策略,建立**四时点随机滚动调度模型**。相邻预报共同可比区间内,后三次更新使光伏预测 MAE 分别下降 47.94%、67.95% 和 31.50%。334 个评价日回放表明,四时点策略使总费用由 1419.63 万元降至 1338.77 万元,下降 5.70%,**紧急购电量下降 59.62%**,且 269 日实现费用降低,说明日内预报更新具有显著经济价值,其中 6:00 和 12:00 更新作用最为明显。

问题四进一步考虑未来实时电价不可提前获知,基于历史信息滚动预测电价,并利用**价格–净负荷联合样本外误差构造场景**及 Wasserstein 模糊集,分别建立波动电价下的日前与四时点**分布鲁棒调度模型**。真实回放中,四时点策略较日前策略使总结算费用降低 2.03%,紧急购电量和紧急购电费用分别降低 33.30% 和 33.18%,且 334 个运行日中有 230 日费用更低。结果表明,所建模型能够有效利用逐步到达的预测信息降低高倍补购风险,并兼顾调度经济性、供能安全性与运行可行性。

关键词: 微网经济调度; 储能优化; 随机规划; 滚动优化; 非前瞻决策; 分布鲁棒优化

一、问题重述

1.1 问题背景

在微网运行中，光伏出力与小区负荷随时间变化，二者在峰谷时段往往难以完全匹配，容易造成电能富余或供电不足。储能可以通过跨时段充放电调节供需，但受到容量、功率和效率等约束。因此，如何根据负荷、光伏和电价变化合理安排外网购电与储能运行，在满足用电需求的同时尽可能降低购电成本尤为重要。

1.2 问题要求

本题围绕非孤岛式微网中外网购电、光伏发电与储能运行的协同调度展开。四个问题在统一的源-荷-储运行框架下逐步增加现实因素：由确定条件下的日前调度，依次扩展到源荷变化下的紧急补购、多时点光伏预报更新下的日内调整，以及实时波动电价下的综合调度。

问题一：在不同日期采用相同的日内电价和小区负荷曲线、且当日光伏预测功率已知的条件下，于每天 0:00 制定全天计划购电策略。要求微网供能满足小区负荷，并满足储能容量、充放电功率、效率以及日初与日末储电量相等运行约束，在此基础上确定计划购电量和储能充放电方案，使全天购电费用尽可能低。

问题二：在问题一基础上，小区负荷和光伏实际发电功率随日期和时段变化，每天仍需在 0:00 制定当日计划购电策略。若实际供能不足，缺口部分需按交易时刻正常电价的 5 倍紧急购电，其余购电费用仍按计划购电量计算。据此制定相应的计划购电、储能运行和紧急购电策略。

问题三：在问题二基础上，每天 0:00、6:00、12:00 和 18:00 可获得未来 24h 的光伏发电功率预报。根据 0:00 预报制定全天计划，并可依据其余时点的新预报调整后续购电策略；计划购电量高于调整购电量的部分按交易电价的 50% 计违约费用，调整购电量高于计划购电量的部分按交易电价的 1.5 倍计费，同时仍需考虑供能不足产生的紧急购电。据此建立日内调整策略，并分析是否有必要利用后续预报修正原计划。

问题四：进一步考虑外网电价随日期和时段实时波动。在问题二和问题三原有源荷变化及光伏预报更新条件下，引入波动电价重新制定相应的购电与储能调度策略，并分别给出波动电价下问题二和问题三的计算结果。

二、模型假设与符号说明

2.1 模型假设

1) 各决策时点仅使用当时已经获得的历史信息与题目规定的预报信息，不提前使用尚未实现的真实负荷、光伏功率及实时电价。

- 2) 题目未规定微网向外网售电，故不考虑反向售电；负荷及储能无法消纳的剩余光伏允许弃用，且不产生收益。
- 3) 储能充、放电量均按微网母线侧计量，将题设 90% 的充放电效率解释为充电效率和放电效率均为 0.9；忽略自放电、寿命衰减及线路损耗。
- 4) 对问题三及问题四中的多次购电调整，按各时段最终生效购电量相对当日 0:00 原计划进行一次结算，不重复计算中间调整费用。

2.2 通用符号说明

符号	符号含义及说明
d	日期索引
t	单个自然日内的 10 min 时段索引
τ	滚动优化窗口内的相对时段索引
k	日内预报与决策节点
s	随机场景编号
p	外网交易电价，单位为元/kWh
E	储能设备的储电量，单位为 kWh
η_c, η_d	储能充电效率和放电效率
$E_{\min}, E_{\max}$	储能设备允许的最小、最大储电量，单位为 kWh
$Q_{\max}$	单个调度时段允许的最大充电量或放电量，单位为 kWh
n	净负荷电量，单位为 kWh
π_s	随机场景 s 的概率
e^L, e^{PV}, e^N	负荷、光伏功率及净负荷的样本外预测误差

2.3 数据统一口径

为保持各问在时间尺度、时段归属和计量单位上的一致性，本文统一采用 10 min 作为调度与运行分析的基本时间步长，记

$$\Delta t = \frac{1}{6} \text{ h}, \quad t = 1, 2, \dots, 144. \quad (1)$$

其中， t 表示单个自然日内的时段索引；跨日滚动优化仍保持相同时间步长，并按时间顺序延伸索引。

对于 10 min 时序数据，统一采用时段右端点进行标记。例如，00:10 对应区间 [00:00, 00:10)；

0:00 + 1 表示次日 0:00，即当日 24:00，对应区间 [23:50, 24:00)。在按日统计、历史信息划分及滚动回测中，该时段仍归属于原自然日。

小区负荷与光伏发电等源荷数据以功率 kW 表示。预测、误差分析及其他统计环节保留功率单位；进入调度模型时，按

$$Q = P\Delta t \quad (2)$$

转换为相应时段电量。计划购电量、调整购电量、紧急购电量、储能充放电量及储电量均以 kWh 计，电价单位统一为元/kWh。

下文不再重复上述时间、索引和单位定义，仅说明各附件特有的数据重构、时间对齐及信息处理步骤。

三、问题一的模型建立与求解

3.1 问题分析

问题一中，日内电价、小区负荷和光伏预测功率在决策时均作为已知参数，因此本问属于确定条件下的日前经济调度问题，无需进一步刻画预测误差及随机性。各时段的供需平衡可由外网购电、光伏发电与储能充放电共同协调，而储电量状态又将相邻时段的决策相互耦合，使购电与储能运行不能按单个时段独立优化。考虑到购电成本、能量平衡、储能状态转移及运行边界均可线性表示，同时充、放电互斥关系需要引入 0-1 状态变量，本文建立源-荷-储协同日前调度的混合整数线性规划模型（MILP）。

3.2 基于 MILP 的微网源-荷-储协同调度模型

设第 t 个调度时段的外网购电量、母线侧储能充电量、储能向母线侧输出的放电量及弃光量分别为 G_t 、 C_t 、 D_t 、 W_t ，时段末储电量为 E_t ，并引入二元变量 u_t 刻画充、放电互斥关系。鉴于题目未规定微网向外网反向售电的交易机制，本文假定不考虑售电，并以 W_t 表示未被利用的光伏电量。以全天外网购电费用最小为目标，有

$$\min J = \sum_{t=1}^{144} p_t G_t, \quad (3)$$

其中， p_t 为第 t 个时段的外网电价，单位为元/kWh。各时段母线侧满足能量守恒：

$$G_t + (S_t - W_t) + D_t = L_t + C_t, \quad t = 1, \dots, 144. \quad (4)$$

其中， $S_t - W_t$ 为实际参与微网调度的光伏电量。式 (4) 统一刻画了外网、光伏、负荷与储能之间的电量流动关系。储能通过电量状态将相邻调度时段耦合。按照前述母线侧充、放电量定义，第 t 个时段末的储电量满足

$$E_t = E_{t-1} + \eta_c C_t - \frac{D_t}{\eta_d}, \quad t = 1, \dots, 144. \quad (5)$$

其中， η_c 和 η_d 分别为充电效率和放电效率。本文主模型将题设“充放电效率为 90%”解释为充、放电单程效率均为 0.9，即 $\eta_c = \eta_d = 0.9$ 。在该计量口径下，母线侧输入 C_t 后实际存入电池的能量为 $\eta_c C_t$ ，而储能向母线输出 D_t 时需消耗 D_t/η_d 的内部电量。针对效率口径可能存在的解释差异，后文进一步进行敏感性检验。根据储电量安全运行范围、最大充放电能力以及充、放电互斥要求，对 $t = 1, \dots, 144$ 有

$$\begin{cases} 1200 \leq E_t \leq 10800, \\ 0 \leq C_t \leq Q_{\max} u_t, \\ 0 \leq D_t \leq Q_{\max}(1 - u_t), \\ 0 \leq W_t \leq S_t, \\ G_t \geq 0, \\ u_t \in \{0, 1\}. \end{cases} \quad (6)$$

其中，

$$Q_{\max} = P_{\max} \Delta t = \frac{5000}{6} \approx 833.33 \text{ kWh},$$

为单个 10 min 调度时段允许的最大充、放电量。由二元变量 u_t 与充、放电量上界共同构成的互斥约束，严格排除了同一时段同时充、放电的非物理状态，同时允许 $C_t = D_t = 0$ 的待机状态。此外，题目要求储能能在日初与日末保持相同储电量。结合初始储电量 6000 kWh，设置周期边界条件

$$E_0 = E_{144} = 6000 \text{ kWh}. \quad (7)$$

该约束使储能能在调度周期末恢复至初始状态，从而消除终端储能状态差异对当日购电成本的偏置。综上，目标函数以及能量平衡、储电量状态转移、安全运行边界、充放电功率、弃光和周期边界约束均关于决策变量线性。其中 G_t 、 C_t 、 D_t 、 E_t 、 W_t 为连续决策变量， u_t 为二元决策变量，因此该问题构成混合整数线性规划（MILP）。

3.3 最优调度结果与运行机理分析

基于附件 1 数据求解上述 MILP 模型，得到全天 144 个 10 min 时段的日前调度方案，完整计划购电策略按题目要求写入 `result1.xlsx`。题目指定时段的购电结果及全天购电量、购电费用见表 1，各指定时间区间的储能充放电量以及日初、日末储电量见表 2。为进一步展示最优方案的日内运行特征，图 1 给出了全天储能充放电及储电量变化轨迹。全天计划购电量为 59482.70 kWh，购电费用为 35126.95 元，且 0:00 与 24:00 储电量均为 6000 kWh。最优方案全天弃光量为 0，说明在该算例下无需通过弃光维持能量平衡，预测光伏均由负荷直接利用或经储能吸收。

表 1 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	480.41	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天计划购电量		59482.70	全天计划购电费		35126.95

表 2 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4500.00	0.00	4:00–8:00	833.33	6365.84
8:00–12:00	4787.96	1703.00	12:00–16:00	5286.04	91.10
16:00–20:00	0.00	5780.13	20:00–24:00	5333.33	2859.87
0:00 储电量		6000.00	24:00 储电量		6000.00

从日内轨迹看，储能主要呈现两类运行特征：一是吸收富余光伏，二是在部分低价时段充电、在高价时段集中放电。凌晨低价阶段，储电量逐步升至 10800 kWh 上限，随后在早间高价阶段释放；午后储电量再次升至上限，并在晚间高价阶段前保持高位。18:40–20:40 储能集中放电，储电量降至 1200 kWh 下限；此后电价回落，储能重新充电，并于日末恢复至 6000 kWh，满足日周期边界条件。

由于主模型取 $\eta_c = \eta_d = 0.9$ ，考虑将时段 t 的部分外购电用于充电，并在未来时段 s 放电的可行微小能量转移。在储电量与功率约束允许、且未来放电能够实际替代外购电的条件下，若要使该能量转移严格降低购电费用，则需满足

$$p_s \eta_c \eta_d > p_t, \quad \frac{p_s}{p_t} > \frac{1}{\eta_c \eta_d} = \frac{1}{0.81} \approx 1.2346. \quad (8)$$

因此，低电价并不足以单独决定充电行为，储能调度还受到未来时段电价、效率损耗以及储电量与功率边界的共同影响。式 (8) 仅用于解释局部价格驱动机制，并非完整 MILP 的最优性判据。

典型时段进一步体现了模型的跨时段协调性。12:00–12:10 时，光伏电量较负荷富余 352.92 kWh；该时段电价为 0.4411 元/kWh，且此时无弃光、无放电，最优方案额外外购 480.41 kWh 用于储能充电，与光伏富余量共同形成 833.33 kWh 的充电量，恰好达到该时段充电上限。

至 16:00–16:10，储电量维持在 10800 kWh 上限，最优方案仍外购 445.43 kWh 补足光伏缺口而不提前放电；这一调度与后续高价时段集中放电的轨迹相一致，体现了全天成本最小化目标下对储能电量跨时段使用时机的权衡。

至 20:00–20:10，电价升至 1.3513 元/kWh，储能放电 713.70 kWh，使外网购电量降为 0。由此可见，所得方案并非依据单时段电价或当期净负荷进行局部决策，而是在全

天范围内统筹分时电价、光伏供需关系、储能损耗及运行边界，通过跨时段调节降低全天购电费用。

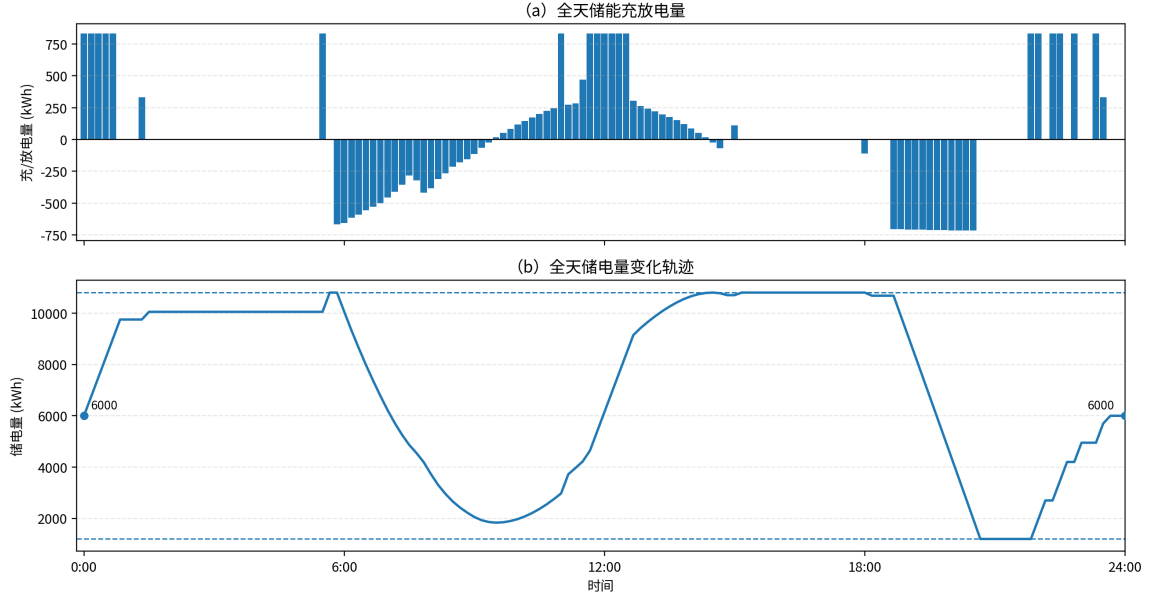


图 1 全天储能充放电及储电量变化轨迹

3.4 模型检验与敏感性分析

首先检验所得解的数值可行性。将最优解逐时回代能量平衡式和储电量递推式，144 个时段的最大绝对残差分别为 2.27×10^{-13} kWh 和 8.31×10^{-13} kWh；充、放电量上界、储电量边界及充放电互斥约束均满足，且 $E_0 = E_{144} = 6000$ kWh。对输出结果回读后重新计算购电费用，与求解器目标值的误差仅为 7.28×10^{-12} 元，验证了结果输出的一致性。最优性方面，求解器返回 OPTIMAL 状态，最终 MIP Gap 为 0，因此所得方案在求解器设定的数值容差范围内达到所建 MILP 的全局最优。

进一步考察关键参数变化对最优购电费用 J 的影响。首先将充、放电单程效率同时取为 0.85、0.90 和 0.95；随后采用单因素扰动法，在其余参数保持基准值不变的条件下，分别将全天负荷和光伏出力整体调整 $\pm 5\%$ ，并保持各自日内曲线形状不变，即

$$L_t^\pm = (1 \pm 0.05)L_t, \quad S_t^\pm = (1 \pm 0.05)S_t. \quad (9)$$

各扰动情形下的求解结果见表 3。

由表 3 可见，在 $\eta \in [0.85, 0.95]$ 的考察范围内，单程效率提高对应的最优购电费用下降。为进一步比较负荷与光伏整体规模变化对购电费用的相对影响，定义中心差分相对敏感系数

$$S_x = \frac{J_x^+ - J_x^-}{0.1J_0}, \quad x \in \{L, PV\}, \quad (10)$$

其中， J_0 为基准最优费用， J_x^+ 、 J_x^- 分别表示相应因素整体提高和降低 5% 时的最优费用。计算得

$$S_L = 2.073, \quad S_{PV} = -0.873. \quad (11)$$

表 3 关键参数敏感性分析结果

扰动因素	参数水平	最优购电费用/元	较基准变化
单程效率 η	0.85	36603.41	+4.20%
	0.90	35126.95	基准
	0.95	33767.03	-3.87%
负荷规模	-5%	—	-10.18%
	+5%	—	+10.55%
光伏规模	-5%	—	+4.47%
	+5%	—	-4.26%

其中 $S_L > 0$ 、 $S_{PV} < 0$ ，表明负荷规模增加将推高最优购电费用，而光伏出力增加则降低外网购电需求及相应费用；又有

$$|S_L| = 2.073 > |S_{PV}| = 0.873, \quad (12)$$

故在上述局部扰动范围内，最优购电费用对负荷规模的相对敏感程度高于光伏出力。

3.5 问题一小结

针对日内电价、负荷及光伏预测功率均已知的日前调度情形，本文在统一功率-电量口径的基础上，建立了以全天购电费用最小为目标的源-荷-储协同调度 MILP 模型，并通过能量平衡、储电量状态转移、充放电互斥及日周期边界等约束保证调度方案的物理可行性。求解结果表明，全天计划购电量为 59482.70 kWh，购电费用为 35126.95 元，全天无弃光，且储能日末恢复至 6000 kWh 的初始电量。最优调度通过在低电价或光伏富余时段储能、在高电价时段释放电量，实现了光伏消纳与分时电价利用的跨时段协调。

数值检验表明，所得方案满足全部模型约束，并在求解器设定的数值容差范围内达到所建 MILP 的全局最优；敏感性分析进一步表明，在所考察的局部扰动范围内，购电费用随单程储能效率提高而下降，对负荷规模的相对敏感程度高于光伏出力。由此，所建模型能够在题设确定性条件下有效刻画微网源-荷-储之间的时序耦合关系，并给出满足运行约束的日前经济调度方案。

四、问题二的模型建立与求解

4.1 问题分析与数据处理

问题二中，微网仍需于每日 0:00 制定全天计划购电策略，但当日实际负荷与光伏功率在决策时尚未实现；若实际供能不足，缺口部分需按交易时刻正常电价的 5 倍紧急购

电。相较于问题一的确定性调度，本问进一步引入源荷不确定性，且净负荷低估导致的高价补购损失显著高于计划购电过量产生的成本。因此，每日 0:00 需依据历史信息预测未来源荷，在不确定性场景下联合确定计划购电量与储能响应参数，日内再根据已实现的供需状态执行预先确定的储能反馈规则。故本文将问题二归结为源荷不确定性下具有非对称补购损失的滚动随机调度问题。

附件 2 分别以“日期 $\times$ 日内时段”的形式给出全年小区负荷与光伏实际功率。记第 d 日第 t 时段的负荷功率和光伏功率分别为 $L_{d,t}$ 和 $P_{d,t}^{PV}$ ，二者均以 kW 计。将两张工作表按日期和时段展开，以 (d, t) 为共同索引逐时配对，并将附件 1 的日内电价映射至相应时段；同时核查时间索引、缺失与重复记录以及源荷取值的物理合理性。时间尺度及功率-电量转换均沿用前文统一口径。

为保证历史回测与实际决策具有一致的信息边界，第 d 日 0:00 的预测、历史误差统计、场景构造及调度决策仅使用此前已经实现的信息，即 $\mathcal{I}_d = \{L_{j,t}, P_{j,t}^{PV} : j < d\}$ 。第 d 日及之后的真实源荷仅用于事后运行回放与结果评价。1 月用于积累预测历史和样本外误差，自 2 月 1 日起进行正式滚动评价。

4.2 负荷与光伏预测及联合随机场景构造

4.2.1 负荷与光伏功率预测

随机调度既需要未来源荷的中心预测，也需要严格样本外预测误差以刻画不确定性。本文分别对负荷和光伏进行逐日滚动预测，以下用 $\hat{\cdot}$ 表示预测值；多日前瞻中尚未实现的滞后特征均由前序预测递推生成，不使用未来真实观测。

负荷具有明显的日内及周周期特征，所以将周季节朴素模型 $\hat{L}_{d,t}^{\text{SNaive}} = L_{d-7,t}$ 作为基准，并采用 LightGBM 刻画非线性变化。模型输入包括日内时段、星期与工作日属性，前 1、2、7 日同刻负荷，以及过去 7 日同刻负荷的均值和标准差。LightGBM 候选超参数依据 1 月 15–31 日首日前瞻的样本外负荷 MAE 确定，自 2 月起固定；各日仅利用决策时已经获得的历史数据重新拟合。

光伏功率具有稳定的昼夜周期，但日际幅值波动较明显。以过去 7 日同刻均值为基准，并利用最近 28 日历史建立动态谐波回归-ARMA 模型。将 10 min 序列连续编号为 ℓ ，有 $P_\ell^{PV} = \beta^T \mathbf{h}_\ell + \varepsilon_\ell$ ，其中， $\mathbf{h}_\ell$ 为包含线性趋势、日周期谐波及其趋势交互项的特征向量， β 为回归系数， ε_ℓ 由低阶 ARMA 过程描述。谐波阶数在 3 阶和 6 阶中选择，AR、MA 阶数均在 $\{0, 1\}$ 中搜索，并以 BIC 为主要准则、AIC 为辅助准则确定模型，同时对残差相关性和白噪声特征进行诊断。

为使预测评价体现紧急购电造成的非对称经济后果，以下记 $[x]_+ = \max\{x, 0\}$ ，并定义实际与预测净外部需求电量分别为

$$Z_{d,t} = [\Delta t (L_{d,t} - P_{d,t}^{PV})]_+, \quad \hat{Z}_{d,t} = [\Delta t (\hat{L}_{d,t} - \hat{P}_{d,t}^{PV})]_+.$$

。计划少购 1 kWh 时，相较于正常提前购电将增加 $4p_t$ 的增量费用；计划多购 1 kWh 且

剩余电量无法回售时，损失为 p_t 。据此定义辅助经济损失

$$\mathcal{L}_{\text{econ}} = \frac{1}{N_{\text{eval}}} \sum_{d,t} \left[p_t \left(\hat{Z}_{d,t} - Z_{d,t} \right)_+ + 4p_t \left(Z_{d,t} - \hat{Z}_{d,t} \right)_+ \right], \quad (13)$$

其中 N_{eval} 为参与评价的样本数。该指标仅用于衡量预测误差的非对称经济后果，不进入后续随机调度目标，也不等同于考虑储能跨时段耦合后的实际运行费用。

记负荷与光伏模型组成的联合预测方案为 $\kappa_d = (\kappa_d^L, \kappa_d^{PV})$ 。各候选方案在相同的历史样本外区间上比较，以 $\mathcal{L}_{\text{econ}}$ 为首要指标；对处于当前最优值 1% 相对容差内的候选，再依次比较净负荷、负荷和光伏的 MAE、RMSE。1 月 1–14 日用于积累初始历史，自 1 月 15 日起逐日生成并保存严格样本外预测及误差，为 2 月起的随机场景构造提供历史依据。

4.2.2 基于样本外误差的联合场景构造与尾部校准

为保留负荷、光伏预测误差之间及连续时段间的相关性，本文以同一历史预测起点下完整的源荷样本外误差轨迹作为场景构造单元。记前瞻长度为 K 日， $T = 144K$ 。历史起点 j 的负荷、光伏误差分别为

$$e_{j,h,t}^L = L_{j+h,t} - \hat{L}_{j+h|j,t}, \quad e_{j,h,t}^{PV} = P_{j+h,t}^{PV} - \hat{P}_{j+h|j,t}^{PV}.$$

仅使用在第 d 日决策前已完整实现的历史轨迹，并优先选取近期样本构造经验场景池。

记场景 s 对应历史起点为 j_s 。将其误差叠加至当前点预测，并令 $\tau = 144h + t$ ，得到

$$\begin{aligned} L_{d+h,t}^{(s)} &= \left[\hat{L}_{d+h|d,t} + e_{j_s,h,t}^L \right]_+, \\ P_{d+h,t}^{PV,(s)} &= \left[\hat{P}_{d+h|d,t}^{PV} + e_{j_s,h,t}^{PV} \right]_+, \\ n_{\tau|d}^{(s)} &= \Delta t \left(L_{d+h,t}^{(s)} - P_{d+h,t}^{PV,(s)} \right). \end{aligned} \quad (14)$$

为避免近期样本遗漏连续净负荷低估的高风险轨迹，定义净负荷误差 $e_{j,h,t}^N = \Delta t(e_{j,h,t}^L - e_{j,h,t}^{PV})$ 及累计供能压力

$$A_j = \max_{1 \leq m \leq T} \left[\sum_{u=1}^m e_{j,u}^N \right]_+. \quad (15)$$

按 A_j 排序，将最高的 $\rho_{\text{tail}} = 20\%$ 历史轨迹划为尾部层，其余作为主体层。

当历史场景较多时，分别对主体与尾部的标准化联合误差轨迹采用 k -medoids 缩减；场景规模依据目标值、首日购电策略及尾部风险的稳定性确定，必要时回退至完整经验场景池。设尾部经验概率质量为 $\omega_d = |\mathcal{T}_d|/|\mathcal{J}_d^{\text{all}}|$ ，则两层内部按经验概率质量均匀赋权：

$$\pi_s = \begin{cases} \frac{1 - \omega_d}{S_d^{\text{body}}}, & s \in \mathcal{S}_d^{\text{body}}, \\ \frac{\omega_d}{S_d^{\text{tail}}}, & s \in \mathcal{S}_d^{\text{tail}}, \end{cases} \quad \sum_{s=1}^{S_d} \pi_s = 1. \quad (16)$$

由此在控制场景规模的同时，保留源荷误差的联合依赖、时间相关性及其高供能压力尾部。

4.3 不对称补购损失下的滚动随机调度模型

4.3.1 5 倍紧急购电机制下的报童模型分析

为揭示题设价格机制对应的非对称损失结构，暂忽略储能及跨时段耦合，考虑无储能单时段退化情形。记该时段计划购电量为 g_t ，净外部需求电量为 Z_t 。计划少购 1 kWh 时，相较于正常提前购电产生的增量缺货损失为

$$C_u = 5p_t - p_t = 4p_t;$$

计划多购 1 kWh 且剩余电量无法回售时，过量损失为

$$C_o = p_t.$$

记 $Q_\alpha(X)$ 为随机变量 X 的 α 分位数，则经典报童模型的临界分位水平为

$$\alpha^* = \frac{C_u}{C_u + C_o} = 0.8, \quad g_t^* = Q_{0.8}(Z_t). \quad (17)$$

因此，0.8 分位由题设 5 倍紧急购电价格直接确定。引入储能后，各时段通过储电量状态相互耦合，式 (17) 不再是逐时最优购电规则，但可作为完整随机调度中非对称补购风险的解析基准。

4.3.2 滚动时域与日前策略

若仅优化未来 24 h，日末储电量的后续价值无法充分体现，容易产生有限时域下的短视放电。因此，本文比较 48 h 与 72 h 两种滚动前瞻窗口。以 72 h 结果为参照，若二者首日计划购电量和期望费用的相对差异分别不超过 2% 和 1%，则采用计算量较低的 48 h 窗口；否则采用 72 h，必要时回退至 24 h。无论前瞻长度如何，每次仅实际执行首日 144 个时段，其余时段仅用于刻画日末储电量的继续价值。

第 d 日 0:00 在实际源荷实现前，预先确定场景共享的日前策略 $x_d = \{g_{\tau|d}, c_{\tau|d}^p, r_{\tau|d}^p\}_{\tau=1}^T$ ，其中 $g_{\tau|d}$ 为计划外网购电量， $c_{\tau|d}^p$ 、 $r_{\tau|d}^p$ 分别为充、放电响应上限，并满足 $g_{\tau|d} \geq 0$ 、 $0 \leq c_{\tau|d}^p, r_{\tau|d}^p \leq Q_{\max}$ 。上述策略参数在场景实现前确定并由所有场景共享，实际充、放电方向及调用量则根据日内已经实现的供需状态确定，从而保证决策的非前瞻性。

4.3.3 基于历史误差的动态储电安全裕度

连续出现净负荷低估时，储能可能在前部时段过早放电，从而削弱后续高需求阶段的补救能力。为此，本文在不改变储能物理下限的前提下，根据历史样本外预测误差设置时变安全裕度。

记 $\mathcal{J}_d^R$ 为截至第 d 日 0:00 已经完整形成首日前瞻误差、且与当前预测方案一致的历史预测起点集合。对 $j \in \mathcal{J}_d^R$ ，定义从第 t 个时段起至当日结束可能经历的最大累计净负荷低估量

$$H_{j,t} = \max_{t \leq m \leq 144} \left[\sum_{u=t}^m e_{j,0,u}^N \right]_+, \quad t = 1, \dots, 144. \quad (18)$$

据此设置第 t 个时段的储电安全裕度

$$R_{t|d} = \min \left\{ E_{\max} - E_{\min}, \frac{1}{\eta_d} Q_{0.8} (\{H_{j,t} : j \in \mathcal{J}_d^R\}) \right\}. \quad (19)$$

其中, $1/\eta_d$ 用于将母线侧备用电量折算为电池侧储电量。这里采用的 0.8 分位源于前述非对称补购损失, 仅作为经济风险启发下的安全裕度参数, 不解释为多时段储能问题的理论最优分位。

$R_{t|d}$ 在每日 0:00 根据当时已实现历史一次确定, 并仅作用于滚动窗口中实际执行的首日。为统一相对时段记号, 令

$$\tilde{R}_{\tau|d} = \begin{cases} R_{\tau|d}, & 1 \leq \tau \leq 144, \\ 0, & 144 < \tau \leq T. \end{cases}$$

4.3.4 负荷优先的非前瞻储能响应

场景逐时实现后, 定义计划购电后的即时净缺口

$$\delta_{\tau|d}^{(s)} = n_{\tau|d}^{(s)} - g_{\tau|d}.$$

记场景 s 下实际充电量、实际放电量、紧急购电量和最终未利用电量分别为 $c_{\tau|d}^{(s)}$ 、 $r_{\tau|d}^{(s)}$ 、 $q_{\tau|d}^{(s)}$ 和 $w_{\tau|d}^{(s)}$ 。

当 $\delta_{\tau|d}^{(s)} > 0$ 时, 计划购电不足, 优先在安全裕度允许范围内调用储能, 其余缺口由紧急购电补足:

$$\begin{cases} c_{\tau|d}^{(s)} = w_{\tau|d}^{(s)} = 0, \\ r_{\tau|d}^{(s)} = \min \left\{ r_{\tau|d}^p, \delta_{\tau|d}^{(s)}, \eta_d \left[E_{\tau-1|d}^{(s)} - E_{\min} - \tilde{R}_{\tau|d} \right]_+ \right\}, \\ q_{\tau|d}^{(s)} = \delta_{\tau|d}^{(s)} - r_{\tau|d}^{(s)}. \end{cases} \quad (20)$$

当 $\delta_{\tau|d}^{(s)} < 0$ 时, 满足负荷后仍存在剩余电量, 优先在储能容量和预设响应上限允许范围内充电:

$$\begin{cases} r_{\tau|d}^{(s)} = q_{\tau|d}^{(s)} = 0, \\ c_{\tau|d}^{(s)} = \min \left\{ c_{\tau|d}^p, -\delta_{\tau|d}^{(s)}, \frac{E_{\max} - E_{\tau-1|d}^{(s)}}{\eta_c} \right\}, \\ w_{\tau|d}^{(s)} = -\delta_{\tau|d}^{(s)} - c_{\tau|d}^{(s)}. \end{cases} \quad (21)$$

当 $\delta_{\tau|d}^{(s)} = 0$ 时, 上述四类响应量均为 0。因此, 紧急购电仅用于弥补当期实际供能缺口, 不用于储能充电。

上述反馈仅依赖当前已经实现的净负荷及上一时段储电状态, 不调用未来真实源荷, 因而满足非前瞻性。受附件数据 10 min 时间分辨率限制, 本文以区间聚合电量刻画该响应过程, 不假定在区间起点已获知整个区间的最终净负荷。

4.3.5 场景运行约束与随机优化目标

对于任一场景 s 和时段 τ ，母线侧能量平衡、储能状态转移及储电量边界分别满足

$$\begin{aligned} g_{\tau|d} + r_{\tau|d}^{(s)} + q_{\tau|d}^{(s)} &= n_{\tau|d}^{(s)} + c_{\tau|d}^{(s)} + w_{\tau|d}^{(s)}, \\ E_{\tau|d}^{(s)} &= E_{\tau-1|d}^{(s)} + \eta_c c_{\tau|d}^{(s)} - \frac{r_{\tau|d}^{(s)}}{\eta_d}, \\ E_{\min} &\leq E_{\tau|d}^{(s)} \leq E_{\max}. \end{aligned} \quad (22)$$

其中， $w_{\tau|d}^{(s)}$ 表示不存在回售收益的最终未利用电量，既可能来自富余光伏，也可能来自最终未被利用的计划购电，因此不简单解释为弃光量。储能参数沿用问题一，即 $E_{\min} = 1200 \text{ kWh}$ 、 $E_{\max} = 10800 \text{ kWh}$ 、 $\eta_c = \eta_d = 0.9$ 。

本文将 1 月作为预测历史和误差库的预热区间，期间储能不主动调度，且不计入正式评价费用。因此，结合题设初始储电量，正式评价开始时取

$$E_{\text{Feb } 1,0} = 6000 \text{ kWh}.$$

自 2 月 1 日起，各场景均从当日实际日初储电量出发，

$$E_{0|d}^{(s)} = E_{d,0},$$

并将上一日实际运行末态连续传递至下一日日初。

在上述联合随机场景及非前瞻反馈机制下，第 d 日滚动窗口以计划购电费用和期望紧急购电费用之和最小为目标：

$$\min_{x_d} J_d = \sum_{\tau=1}^T p_{\tau} g_{\tau|d} + \sum_{s=1}^{S_d} \pi_s \sum_{\tau=1}^T 5p_{\tau} q_{\tau|d}^{(s)}. \quad (23)$$

其中，计划购电始终按正常电价计费，未利用部分不产生退款或回售收益；紧急购电按对应时段正常电价的 5 倍计费。上述分段响应关系可通过辅助变量和指示约束进行等价线性化，从而得到有限场景下的混合整数线性规划模型。由于日内反馈映射的形式预先给定，所得解为上述参数化非前瞻策略类中的最优方案。

为与滚动决策机制保持一致，每日求解后仅实施窗口首日 144 个时段的 $g_{d,t}$ 、 $c_{d,t}^p$ 、 $r_{d,t}^p$ 及当日 0:00 已确定的安全裕度 $R_{d,t}$ ，并将当日实际负荷和光伏逐时输入同一反馈规则。记由此产生的实际紧急购电量为 $q_{d,t}^{\text{real}}$ ，则第 d 日实际购电费用按

$$C_d^{\text{real}} = \sum_{t=1}^{144} (p_t g_{d,t} + 5p_t q_{d,t}^{\text{real}}) \quad (24)$$

核算。当日 24:00 的实际储电量作为下一日日初状态。上述过程仅规定滚动执行与费用统计口径，具体运行轨迹及经济性结果在后文统一分析。““

4.4 调度结果与经济性分析

对 2025 年 2 月 1 日至 12 月 31 日进行逐日滚动求解，并以当日实际负荷与光伏功率进行回放。四个指定日期的计划购电、储能运行及紧急购电结果分别见表 4-6。

表 4 指定日期计划购电结果

日期	10:00-10:10	12:00-12:10	14:00-14:10	16:00-16:10	18:00-18:10	20:00-20:10	全天购电量	购电费
3.20	30.77	647.12	0.00	491.08	678.98	47.68	65354.44	41469.88
6.21	0.00	0.00	0.00	266.24	489.62	13.81	45357.23	27013.07
9.23	0.00	0.00	0.00	550.68	751.57	57.08	68930.26	44583.94
12.21	174.61	975.14	6.81	882.64	745.52	8.16	102661.44	67637.24

注：购电量单位为 kWh，购电费单位为元。

表 5 指定日期储能运行结果

时段	3.20		6.21		9.23		12.21	
	充电	放电	充电	放电	充电	放电	充电	放电
0:00-4:00	1704.88	78.66	1757.12	0.00	1897.73	0.00	2393.00	0.00
4:00-8:00	1113.39	4525.67	822.94	2877.36	1713.90	3205.68	939.07	1265.42
8:00-12:00	6041.43	2338.27	4777.70	0.00	2703.45	1896.62	5958.49	4546.23
12:00-16:00	2460.12	386.54	0.00	0.00	3031.42	403.62	2989.26	2196.68
16:00-20:00	257.99	5216.80	0.00	4683.45	47.84	5616.23	0.00	5232.62
20:00-24:00	3259.65	2866.88	8487.28	2445.85	7866.80	2869.97	7791.67	2742.77
$E_{0:00}$	8609.12		7375.09		8505.83		8646.30	
$E_{24:00}$	4837.48		10517.11		8494.07		8950.96	

注：充、放电量及储电量单位均为 kWh。

表 6 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
15:50-16:10	28.13	-	0	17:30-17:40	12.00	-	0
21:20-21:30	12.13	-	-	19:20-19:30	0.98	-	-
23:30-23:40	13.31	-	-	20:20-20:30	0.88	-	-
-	-	-	-	20:50-21:00	26.76	-	-

注：紧急购电量单位为 kWh，“-”表示无对应紧急购电时段。

图 2 表明，全年负荷具有较稳定的日内周期，光伏出力集中于白昼并随季节变化；二者共同作用下，净负荷在午间明显降低，而早晚时段整体较高，且相同时刻仍存在显著日际波动。这一特征说明源荷过程同时具有可预测的周期结构与不可忽略的随机扰动，与本文采用滚动预测和随机场景刻画不确定性的建模思路一致。

评价期内计划购电量累计为 2.1654×10^7 kWh，紧急购电量为 1.3026×10^5 kWh，仅占实际外购电量的 0.60%；但紧急购电费用为 67.72 万元，占总购电费用 1419.63 万元的 4.77%。四个指定日期中，仅 3 月 20 日和 9 月 23 日出现少量紧急购电。结果表明，所建策略能够将供能缺口控制在较小范围；同时，紧急购电量占比较低而费用占比明显较高，说明 5 倍补购机制下净负荷低估具有显著的非对称经济代价。

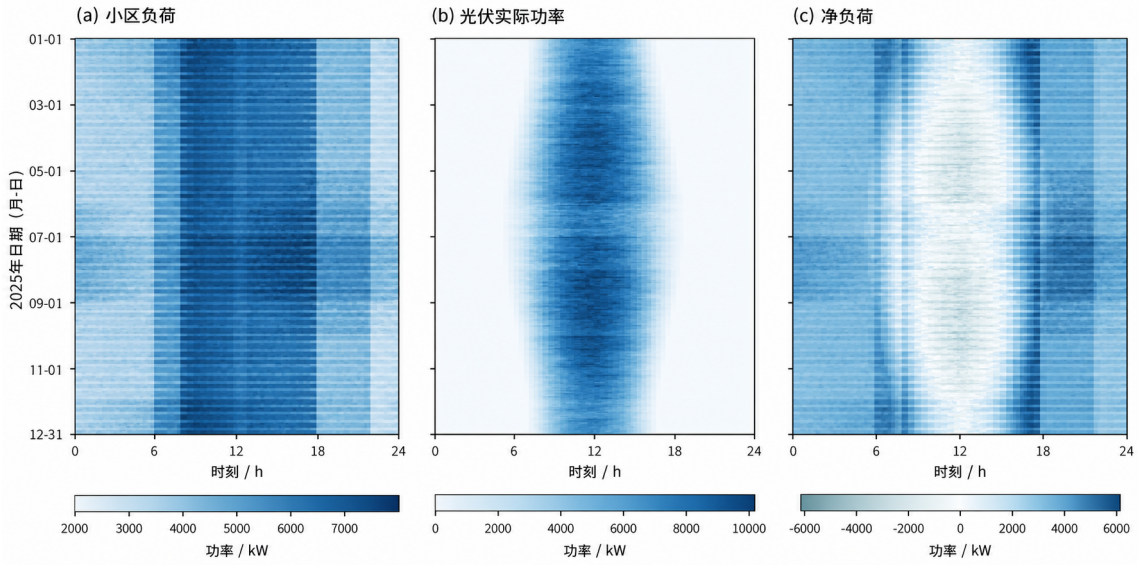


图2 2025年小区负荷、光伏实际功率与净负荷日内分布

4.5 参数敏感性分析

选取常规日、高预测误差日、高紧急购电日和高系统压力日进行单因素敏感性实验。为考察动态储电安全裕度的影响，在基准安全裕度 $R_{t|d}$ 上引入无量纲倍率 β_R ，令

$$R_{t|d}^{(\beta)} = \beta_R R_{t|d},$$

其中基准情形取 $\beta_R = 1$ 。同时改变尾部场景比例 ρ_{tail} ，分别考察安全裕度强度与尾部场景配置对实际运行结果的影响，结果见表7。

表7 问题二参数敏感性分析结果

参数	取值	成本变化/%	紧急购电量/kWh	最低储电量/kWh
β_R	0.8	0.399 [-1.406, 6.262]	4460.32 [20.29, 13899.00]	1521.92 [1288.96, 1972.20]
	1.0	0	4345.46 [13.97, 12344.95]	1545.00 [1311.20, 1968.73]
	1.2	-0.123 [-1.053, 0.257]	4274.12 [20.29, 12436.10]	1588.95 [1333.44, 2032.61]
ρ_{tail}	10%	-0.737 [-2.745, 7.567]	4397.11 [11.96, 14167.85]	1403.48 [1311.20, 1784.57]
	20%	0	4345.46 [13.97, 12344.95]	1545.00 [1311.20, 1968.73]
	30%	2.134 [-1.296, 8.387]	4428.57 [10.65, 12868.47]	1483.55 [1311.20, 1728.25]

注：表中为4个代表日的“中位数 [最小值, 最大值]”，成本变化均相对于同日基准方案。

由表7可见， β_R 提高至1.2时成本变化中位数仅为-0.123%，表明模型对安全裕度参数总体较稳定；降低 β_R 则在部分高风险工况下明显增加补购代价。

相比之下， ρ_{tail} 更为敏感：提高至30%后成本变化中位数增至2.134%，最大增幅达8.387%。因此，基准设置 $\beta_R = 1.0$ 、 $\rho_{\text{tail}} = 20\%$ 在代表工况下能够较好兼顾经济性与风险控制。

五、问题三的模型建立与求解

5.1 问题分析与数据预处理

问题三在问题二基础上增加 0:00、6:00、12:00 和 18:00 四个预报与决策节点。新预报到达后，已执行决策保持不变，并以当前实际储电量为新的初始状态重新优化未来策略。由于增购、减购及紧急购电对应不同结算代价，只有当预报更新带来的预期运行成本下降超过调整代价时，策略调整才产生净收益。考虑储能的跨时段耦合，本文建立基于负荷-光伏联合预测误差场景的四时点随机滚动经济调度模型。

负荷预测沿用问题二每日 0:00 生成的因果预测，日内仅更新附件 3 的光伏预报。四个节点均采用前向 24h 优化窗口，仅下一滚动决策时点前的决策实际执行，之后部分作为前瞻信息评估后续运行成本。整点光伏预报按发布时间和预测提前量映射至目标时刻后插值至 10min 尺度。历史场景由相同决策节点下、同一历史日期且对应相同目标时段的负荷与光伏预测误差轨迹配对构成，并仅选取整段目标区间均已实现的历史轨迹。1 月按相同滚动调度与结算机制连续推进储电量；尚无可用历史误差轨迹时采用确定性滚动，随历史样本积累逐步启用随机场景模型，自 2 月 1 日起进行正式评价。

5.2 基于联合历史误差的随机场景构造

负荷与光伏预测误差共同决定净负荷的不确定性。若对二者及各时段分别独立抽样，将破坏源荷误差之间及各时段之间的依赖结构。因此，本文以同一历史日期中与当前节点对应的前向 24h 源荷联合误差轨迹作为场景构造单元。

设当前为第 d 日节点

$$k \in \{0, 6, 12, 18\},$$

记 $h = 1, \dots, H$ 为自节点 k 起的相对 10min 目标步，其中 $H = 144$ 。候选历史起点 $j < d$ 须同时满足负荷样本外预测可得，且对应前向 24h 真实轨迹在当前节点前已完整实现。

固定第 d 日选定的负荷预测器，采用该预测器在历史日期 j 上仅利用当时可用信息得到的样本外预测；光伏误差则由与当前节点相同发布时间下的历史预报计算。定义

$$e_{j,k,h}^L = L_{j,k,h}^{\text{real}} - \hat{L}_{j,k,h}^0, \quad e_{j,k,h}^{PV} = P_{j,k,h}^{PV,\text{real}} - \hat{P}_{j,k,h|k}^{PV}. \quad (25)$$

其中，对负荷误差而言， k 仅用于确定前向 24h 误差轨迹的截取起点，其预测发布时间仍固定为当日 0:00。两类误差按同一历史日期及相同相对目标时段对齐，其中负荷误差来自该日 0:00 的预测，光伏误差来自节点 k 发布的预报；当 $k > 0$ 时，相应轨迹可自然跨越至次日。

将历史联合误差叠加至当前预测，得到场景 s 下

$$L_{h,s}^{(d,k)} = \max \left\{ 0, \hat{L}_{d,k,h}^0 + e_{j,k,h}^L \right\}, \quad (26)$$

$$P_{h,s}^{PV,(d,k)} = \max \left\{ 0, \hat{P}_{d,k,h|k}^{PV} + e_{j,k,h}^{PV} \right\}, \quad (27)$$

$$n_{h,s}^{(d,k)} = \Delta t \left(L_{h,s}^{(d,k)} - P_{h,s}^{PV,(d,k)} \right). \quad (28)$$

为兼顾历史样本的时效性与场景数量，候选池依次采用最近 28、56、84 日及全部可用历史，并选取首个满足场景规模要求的时间窗口。进一步按前向 24 h 预测光伏总电量

$$Z_{j,k} = \Delta t \sum_{h=1}^H \hat{P}_{j,k,h|k}^{PV} \quad (29)$$

的三分位数划分低、中、高三个区间，从当前 $Z_{d,k}$ 所在区间开始筛选；若样本不足，则依次并入相邻区间。记条件化后的候选轨迹数为 $M_{d,k}$ 。

若 $M_{d,k} > S^*$ ，在固定发布时间 k 下，分别对各相对目标步上的净负荷误差进行标准化，并以标准化轨迹间的欧氏距离进行场景缩减。定义净负荷误差

$$e_{j,k,h}^N = e_{j,k,h}^L - e_{j,k,h}^{PV}, \quad (30)$$

并考虑较大的正向净负荷预测误差更易造成供电缺口，引入仅用于尾部筛选的价格加权代理评分

$$R_{j,k} = \sum_{h=1}^H p_{d,k,h} [e_{j,k,h}^N]_+ \Delta t, \quad (31)$$

其中， $p_{d,k,h}$ 为当前节点后第 h 个目标时段的电价。 $R_{j,k}$ 不等同于实际紧急购电费用；由于紧急购电价格中的系数 5 对所有轨迹的排序均相同，故在该评分中省略。

正式场景数取

$$S^* = 20.$$

当 $M_{d,k} > S^*$ 时，首先保留 $R_{j,k}$ 最大的两条尾部轨迹，再对其余样本采用 k -medoids 进行缩减；当 $M_{d,k} \leq S^*$ 时，则保留全部候选轨迹。

将所有候选轨迹归入距离最近的代表轨迹。若代表场景 s 覆盖 m_s 条候选轨迹，则赋予概率

$$\pi_s = \frac{m_s}{M_{d,k}}, \quad \sum_s \pi_s = 1. \quad (32)$$

场景缩减仅用于选择代表历史轨迹并确定其概率，最终输入随机优化模型的仍为相应历史日期的原始配对源荷误差轨迹。由此，在控制计算规模的同时，能够保留源荷误差的联合依赖、时序相关结构及高缺口尾部信息。

5.3 四时点随机经济滚动优化模型

5.3.1 滚动时域与承诺执行机制

问题三在 0:00、6:00、12:00、18:00 四个节点获得新的光伏预报。本文沿用当日 0:00 基于当时可用信息生成的 48 h 负荷预测，日内不再更新，以覆盖最晚至次日 18:00 的滚

动时域；各节点结合最新光伏预报、当前实际储电量及第 5.2 节构造的随机场景，对未来 24h 重新优化。

沿用相对步长 $h = 1, \dots, H$ ，其中 $H = 144$ 。对 $k \in \{6, 12, 18\}$ ，记当前节点至当日 24:00 的剩余时段数为 $H_k^{\text{day}} = 6(24 - k)$ ，并划分

$$\mathcal{B} = \{1, \dots, 36\}, \quad \mathcal{P}_k = \{37, \dots, H_k^{\text{day}}\}, \quad \mathcal{C}_k = \{H_k^{\text{day}} + 1, \dots, H\}. \quad (33)$$

其中， $\mathcal{B}$ 为下一预报到达前的承诺执行区间， $\mathcal{P}_k$ 为当日后续前瞻区间， $\mathcal{C}_k$ 为跨日继续区间；当 $k = 18$ 时， $\mathcal{P}_k = \emptyset$ 。仅 $\mathcal{B}$ 内的决策在当前节点锁定，其余部分仅用于后续经济价值评估。

对于题目未明确的多次调整结算方式，本文采用最终生效购电量相对当日 0:00 原计划进行一次结算的解释，后续滚动优化与实际费用核算均统一采用该口径。

5.3.2 参数化因果储能策略与物理约束

记 $g_{d,t}^0$ 为第 d 日 0:00 制定的原计划，并以

$$g_{d,k,h}^0 = g_{d,6k+h}^0$$

表示节点 k 后第 h 个当日目标时段对应的原计划量，其中 $h \leq H_k^{\text{day}}$ 。

设 $g_{d,k,h}^{(k)}$ 为承诺区间的调整后绝对购电量， $\tilde{g}_{d,k,h}^{(k)}$ 为后续前瞻代理量，统一记

$$\gamma_h^{(d,k)} = \begin{cases} g_{d,k,h}^{(k)}, & h \in \mathcal{B}, \\ \tilde{g}_{d,k,h}^{(k)}, & h \in \mathcal{P}_k \cup \mathcal{C}_k. \end{cases} \quad (34)$$

其中 $k = 0$ 时取 $\gamma_h^{(d,0)} = g_{d,h}^0$ ，且

$$0 \leq \gamma_h^{(d,k)} \leq \Delta t \max_s L_{h,s}^{(d,k)} + Q_{\max}.$$

为避免不同场景利用尚未实现的未来信息， $\gamma_h^{(d,k)}$ 、 $\bar{c}_h^{(d,k)}$ 和 $\bar{r}_h^{(d,k)}$ 均由节点 k 预先确定并由各场景共享，其中

$$0 \leq \bar{c}_h^{(d,k)}, \bar{r}_h^{(d,k)} \leq Q_{\max}.$$

令场景净缺口

$$x_{h,s}^{(d,k)} = n_{h,s}^{(d,k)} - \gamma_h^{(d,k)},$$

则储能响应为

$$\begin{aligned} r_{h,s}^{(d,k)} &= \min \left\{ \bar{r}_h^{(d,k)}, \left[x_{h,s}^{(d,k)} \right]_+, \eta_d \left(E_{h-1,s}^{(d,k)} - E_{\min} \right) \right\}, \\ c_{h,s}^{(d,k)} &= \min \left\{ \bar{c}_h^{(d,k)}, \left[-x_{h,s}^{(d,k)} \right]_+, \frac{E_{\max} - E_{h-1,s}^{(d,k)}}{\eta_c} \right\}. \end{aligned} \quad (35)$$

进一步令

$$y_{h,s}^{(d,k)} = n_{h,s}^{(d,k)} + c_{h,s}^{(d,k)} - \gamma_h^{(d,k)} - r_{h,s}^{(d,k)},$$

则紧急购电量和未利用剩余电量分别为

$$q_{h,s}^{\text{em},(d,k)} = \left[y_{h,s}^{(d,k)} \right]_+, \quad u_{h,s}^{(d,k)} = \left[-y_{h,s}^{(d,k)} \right]_+,$$

从而自动满足能量平衡。

这里 u 可能同时包含光伏富余和计划购电偏高产生的剩余电量，故不将其简单解释为弃光量。

储电量满足

$$\begin{aligned} E_{h,s}^{(d,k)} &= E_{h-1,s}^{(d,k)} + \eta_c C_{h,s}^{(d,k)} - \frac{r_{h,s}^{(d,k)}}{\eta_d}, \\ E_{\min} &\leq E_{h,s}^{(d,k)} \leq E_{\max}, \quad E_{0,s}^{(d,k)} = E_{d,k}^{\text{real}}. \end{aligned} \quad (36)$$

上述反馈关系表示区间内实时因果控制在 10 min 尺度上的聚合近似。

5.3.3 四时点随机经济优化目标

记 $p_{d,k,h}$ 为节点 k 向前第 h 个目标时段对应的电价。在 0:00 节点尚不存在购电调整，采用风险中性目标

$$\min J_{d,0} = \sum_{h=1}^H p_{d,0,h} g_{d,h}^0 + \sum_s \pi_s \left(\sum_{h=1}^H 5 p_{d,0,h} q_{h,s}^{\text{em},(d,0)} - \lambda_{E,0}^{(d)} E_{H,s}^{(d,0)} \right). \quad (37)$$

其中， $\lambda_{E,k}^{(d)}$ 为窗口末端单位储电量的继续价值，由当前日期以前最近至多 28 个已完整实现的同节点历史窗口进行储能边际价值有限差分估计并取中位数，以减弱有限时域导致的末端过度放电。

g^0 为 0:00 滚动随机优化得到的基准计划；由于未显式构造后三次预报更新对应的完整多阶段场景树，因此不将其解释为完整四阶段随机规划的一阶段最优策略。

对 $k \in \{6, 12, 18\}$ ，承诺区间内相对原计划的增量调整费用定义为

$$A_h^{\text{adj},(d,k)} = p_{d,k,h} \left[1.5 \left(g_{d,k,h}^{(k)} - g_{d,k,h}^0 \right)_+ - 0.5 \left(g_{d,k,h}^0 - g_{d,k,h}^{(k)} \right)_+ \right]. \quad (38)$$

当日后续区间令 $\tilde{A}_h^{\text{sur},(d,k)}$ 表示将上式中的 $g_{d,k,h}^{(k)}$ 替换为 $\tilde{g}_{d,k,h}^{(k)}$ 后得到的同形式代理费用；跨日区间按正常购电费用估计。于是节点 k 的滚动目标为

$$\begin{aligned} \min J_{d,k} &= \sum_{h \in \mathcal{B}} A_h^{\text{adj},(d,k)} + \sum_{h \in \mathcal{P}_k} \tilde{A}_h^{\text{sur},(d,k)} + \sum_{h \in \mathcal{C}_k} p_{d,k,h} \tilde{g}_{d,k,h}^{(k)} \\ &\quad + \sum_s \pi_s \left[\sum_{h=1}^H 5 p_{d,k,h} q_{h,s}^{\text{em},(d,k)} - \lambda_{E,k}^{(d)} E_{H,s}^{(d,k)} \right]. \end{aligned} \quad (39)$$

其中， $\mathcal{B}$ 中的费用对应当前实际调整， $\mathcal{P}_k$ 与 $\mathcal{C}_k$ 中的变量仅用于评估后续经济影响，不直接计入当前真实结算。

5.3.4 实际执行与费用核算

令

$$\begin{aligned}\mathcal{T}_0 &= \{1, \dots, 36\}, & \mathcal{T}_6 &= \{37, \dots, 72\}, \\ \mathcal{T}_{12} &= \{73, \dots, 108\}, & \mathcal{T}_{18} &= \{109, \dots, 144\},\end{aligned}$$

并约定

$$g_{d,t}^{(0)} = g_{d,t}^0.$$

则当天最终生效购电量统一表示为

$$g_{d,t}^{\text{eff}} = g_{d,t}^{(k)}, \quad t \in \mathcal{T}_k, \quad k \in \{0, 6, 12, 18\}. \quad (40)$$

实际运行按附件 2 的真实轨迹及对应承诺区间冻结的储能反馈参数逐 10 min 回放, 记所得真实紧急购电量为 $q_{d,t}^{\text{em,real}}$; 日末实际储电量直接传递为次日初始状态。

在本文采用的单次结算口径下, 第 d 日实际总费用可统一写为

$$C_d^{\text{total}} = \sum_{t=1}^{144} p_t \left[g_{d,t}^0 + 1.5 (g_{d,t}^{\text{eff}} - g_{d,t}^0)_+ - 0.5 (g_{d,t}^0 - g_{d,t}^{\text{eff}})_+ + 5q_{d,t}^{\text{em,real}} \right]. \quad (41)$$

前瞻代理费用与终端价值仅参与滚动决策, 不计入实际费用。上述分段线性关系可通过辅助变量与 0–1 指示变量转化为有限场景下的确定性等价 MILP, 并采用整数规划求解器求解。求解所得策略进一步通过真实轨迹逐时回放, 对能量平衡、储能状态及运行边界进行复核。

5.4 调度结果与多时点预报价值分析

对 2025 年 2–12 月共 334 个评价日进行逐日滚动回放。按照题目要求, 选取 2025.3.20、2025.6.21、2025.9.23 和 2025.12.21 四个指定日期, 其购电、储能运行及紧急购电结果分别见表 8–表 10, 完整购电策略保存于 result3.xlsx。

表 8 指定日期的购电量及全天购电结果

日期	10:00–10:10	12:00–12:10	14:00–14:10	16:00–16:10	18:00–18:10	20:00–20:10	全天购电量	全天购电费
	最终有效购电量/kWh						/kWh	/元
2025.3.20	0.00	566.62	0.00	522.26	698.98	0.00	65903.58	42648.52
2025.6.21	0.00	0.00	0.00	186.37	425.50	0.00	35641.56	21866.93
2025.9.23	0.00	0.00	0.00	553.88	650.13	64.19	66878.06	44795.01
2025.12.21	0.00	1003.27	0.00	883.43	687.10	0.00	94774.99	62048.55

表 9 指定日期储能设备的充放电量及日初、日末储电量

日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	$E_{0:00}$	$E_{24:00}$
	充电量/放电量 (kWh)						储电量 (kWh)	
2025.3.20	133.35/508.92	901.75/5890.14	4378.90/2655.22	5787.54/314.20	258.97/5219.82	9711.84/2939.04	10328.72	9909.02
2025.6.21	203.99/3015.96	395.14/4836.61	9782.58/0.00	0.00/0.00	0.00/4223.79	8233.35/2485.90	10181.53	10754.81
2025.9.23	4640.26/0.00	971.29/5277.50	5123.57/1896.62	4037.29/410.90	66.66/6072.21	5426.35/2624.57	5910.21	6058.21
2025.12.21	431.33/389.22	1939.55/1813.82	5061.21/6733.62	7291.73/2397.78	35.81/5611.76	9713.67/2908.49	10142.37	10107.58

表 10 指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量 /kWh	时间段	购电量 /kWh	时间段	购电量 /kWh	时间段	购电量 /kWh
09:30–10:00	122.16	–	–	20:50–21:10	40.85	12:50–13:00	9.60
20:00–20:10	12.61	–	–	21:20–21:40	36.87	19:50–20:00	16.43
21:20–21:30	7.76	–	–	–	–	–	–
21:40–21:50	18.23	–	–	–	–	–	–

由表 8–表 10 可见，滚动策略能够依据新预报持续修正购电量，并通过储能充放电吸收源荷偏差；多数偏差可由购电调整与储能共同消化，仅在调节余量不足时产生紧急购电，其中 2025.6.21 全天未发生紧急购电。为进一步展示预报更新对实际决策的作用过程，选取题目指定日期 2025.3.20 的完整运行轨迹，如图 3 所示。

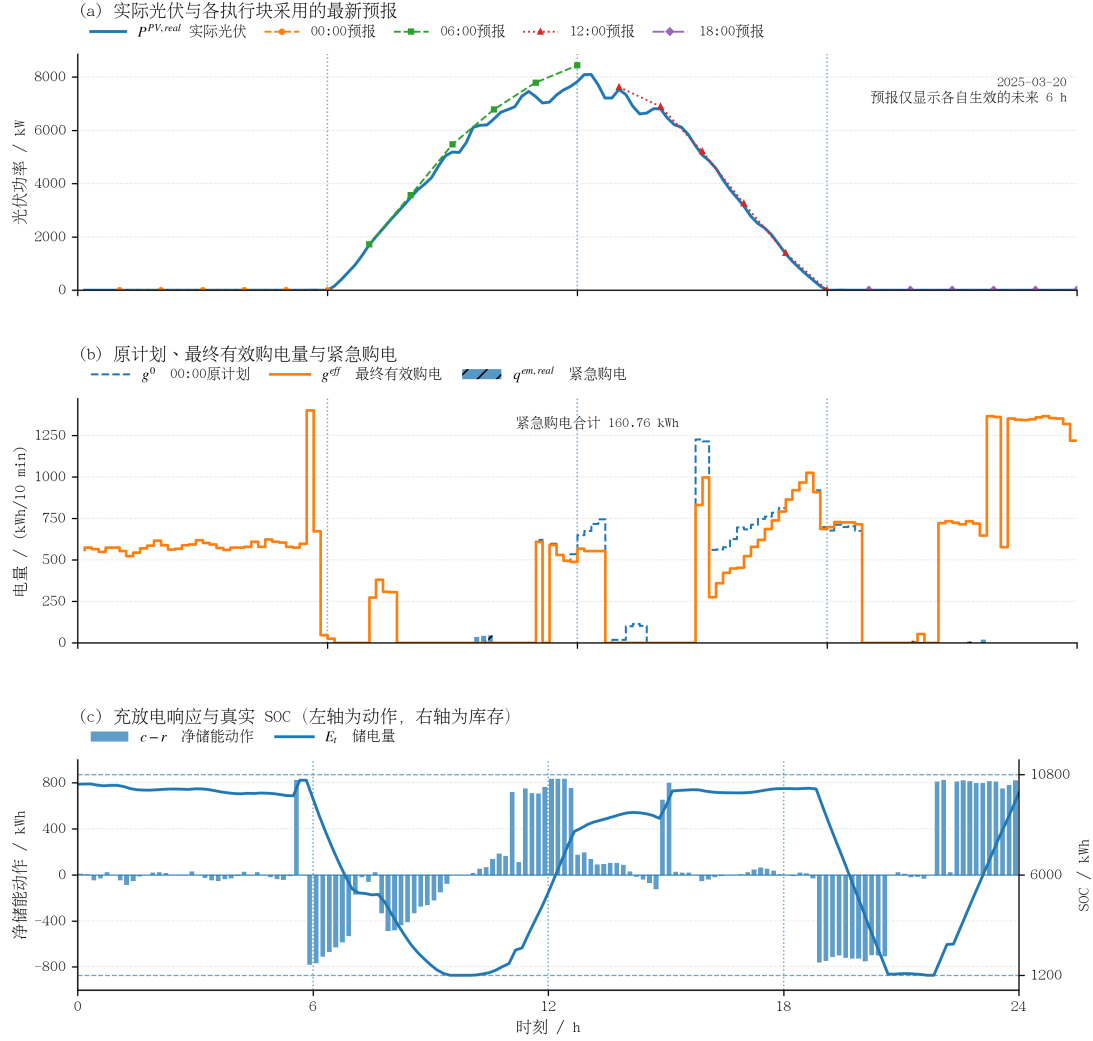


图 3 2025.3.20 多时点预报更新下的购电与储能调度轨迹

图 3 表明, 新预报到达后, 后续购电计划随之修正, 储能通过充放电协同吸收源荷偏差; 代表日仅发生 160.76 kWh 紧急购电, 且全过程满足储能运行约束。

在相邻预报共同覆盖的 18 h 区间内, 6:00、12:00 和 18:00 更新后的光伏预测 MAE 分别下降 47.94%、67.95% 和 31.50%, 其中 12:00 的信息增益最明显。全年回放中, 四时点滚动策略使实际总费用由 1419.63 万元降至 1338.77 万元, 降低 5.70%; 紧急购电量由 13.03 万 kWh 降至 5.26 万 kWh, 降低 59.62%, 且 334 个评价日中有 269 日费用更低。因此, 日内预报更新具有实际应用价值, 其中 6:00 和 12:00 作用较为显著; 18:00 的当日直接作用较弱, 但考虑储能跨时段耦合仍予以保留。上述成本改善为预报更新、状态反馈与滚动优化共同作用的综合收益。

5.5 模型检验与敏感性分析

将调度结果逐时回代后, 能量平衡、储能容量及充放电功率约束均满足。进一步选取代表工况, 对场景数、尾部场景保留数及终端价值进行单因素扰动。对于第 5.3.3 节

由历史数据估计得到的基准终端价值 $\lambda_{E,k}^{(d)}$ ，引入无量纲倍率 β_E ，令

$$\lambda_{E,k}^{(d,\beta)} = \beta_E \lambda_{E,k}^{(d)},$$

其中基准情形取 $\beta_E = 1$ 。各参数扰动结果见表 11。

表 11 问题三关键参数局部敏感性分析

参数设置	ΔC 中位数/%	ΔC 范围/%	ΔQ_{em} 中位数/kWh	E_{24} 中位数/kWh
基准: $S = 20, S_{tail} = 2, \beta_E = 1.0$	0.00	[0.00, 0.00]	0.00	8194.72
$S = 10$	+2.50	[-1.44, +6.16]	+435.91	8782.59
$S = 28$	-2.31	[-5.32, -0.12]	-130.65	5335.32
$S_{tail} = 0$	+0.30	[-0.77, +1.02]	+1.43	8191.06
$S_{tail} = 4$	+0.29	[+0.08, +0.67]	+100.05	8271.17
$\beta_E = 0.8$	-6.05	[-9.28, -1.91]	+2.99	1236.51
$\beta_E = 1.2$	+0.33	[-0.43, +0.90]	-85.77	8835.78

注：结果基于常规、高预测更新、高调整成本和高风险四类代表日； ΔC 、 ΔQ_{em} 分别表示相对基准方案的实际总费用和紧急购电量变化。

由表 11 可见，尾部场景保留数变化对实际费用影响较小，而场景规模变化会引起一定费用波动。当 $\beta_E = 0.8$ 时，窗口末端储电量明显下降，说明终端价值项能够有效抑制有限滚动时域下的短视放电。总体而言，模型在所考察代表工况下具有较好的局部稳定性。

六、问题四的模型建立与求解

6.1 问题分析与数据处理

问题四在问题二、问题三的基础上进一步考虑外网电价的实时波动。问题 4-2 与问题 4-3 分别沿用前文的源荷预测、储能运行及购电调整机制，并进一步引入未来电价的不确定性。由于计划购电、调整购电和紧急购电均按对应交易时刻电价结算，电价波动不仅改变各时段的购电成本，也会改变源荷预测偏差所对应的经济损失，并影响储能跨时段转移电量的价值。

尤其当净负荷偏高与电价偏高同时出现时，供能不足将对应更高的紧急补购成本；若将价格与净负荷分别独立处理，可能低估此类联合不利状态的运行风险。因此，本文在前述调度框架上进一步刻画价格与净负荷的联合不确定性，并据此分别建立波动电价下的日前分布鲁棒调度模型与四时点分布鲁棒滚动调度模型。

附件 4 以“日期 $\times$ 日内时段”的宽表形式给出全年实时电价。本文按日期和时段展开价格序列，以 (d, t) 为索引与附件 2 的源荷实际轨迹、前文生成的净负荷预测以及附件 3 的四时点光伏预报逐时对齐。核查结果表明，附件 4 共包含 365 日、每日 144 个时段的完整记录，未发现缺失或重复数据。按照前述非前瞻原则，每一决策节点仅保留

此前已经实现的电价作为历史样本，节点之后的真实电价在实现前不进入预测、场景构造及优化模型。

为确定后续不确定性描述方式，进一步考察价格的时序结构及其与净负荷的联合变化。电价的 1 日、7 日和 14 日同刻相关系数分别为 0.9335、0.9820 和 0.9745，显示出较强的周周期结构。采用 7 日同刻差分削弱共同周期成分后，价格与净负荷异常量的相关系数仍为 0.6695；当净负荷异常进入上 10% 时，价格异常同步进入上 10% 的条件概率为 0.5369，明显高于近似独立情形下的 0.1。据此，后文采用滚动电价预测，并以同日价格–净负荷样本外预测误差构造联合场景。

6.2 波动电价预测与联合场景构造

6.2.1 基于 Elastic Net 的滚动电价预测

前述分析表明，实时电价具有显著的周周期结构，因此以第 $d - 7$ 日同刻电价作为季节基准，并利用 Elastic Net 对周周期之外的价格偏移进行修正。定义周际价格残差

$$r_{d,t}^p = p_{d,t} - p_{d-7,t}, \quad \hat{p}_{d,t} = p_{d-7,t} + \hat{r}_{d,t}^p. \quad (42)$$

针对 $r_{d,t}^p$ 构造 17 维因果特征向量 $\mathbf{z}_{d,t}$ ，主要包含同刻多尺度滞后、近期周际变化、局部价格变化以及日内和星期周期特征。考虑到上述滞后变量间存在较强相关性，采用 Elastic Net 对冗余特征进行收缩并估计价格残差，全部 144 个日内时段共享同一组回归参数。

以 1 月作为模型预热与参数选择区间，通过时间有序训练–验证确定 $\lambda = 10^{-4}$ 、 $\gamma = 0.9$ ；自 2 月起固定该组超参数，每日 0:00 仅使用此前已经实现的历史数据重新完成特征标准化与模型拟合。对于跨日预测中尚未实现的滞后变量，均由前序预测递推生成，从而保证预测过程满足非前瞻性。

严格滚动样本外检验得到 $\text{MAE} = 0.04193$ 元/kWh， $\text{RMSE} = 0.05623$ 元/kWh，均优于 7 日同刻季节基线。因此，后续采用该模型生成实时电价的中心预测及样本外预测误差，并用于价格–净负荷联合场景构造。

6.2.2 价格–净负荷联合样本外误差场景

为保持价格与净负荷误差在同一历史状态下的联合关系，本文以严格滚动预测产生的同日样本外误差轨迹作为场景构造单元。记第 j 日价格误差及节点 k 下的净负荷误差分别为

$$e_{j,t}^p = p_{j,t}^{\text{real}} - \hat{p}_{j,t}^{\text{OOS}}, \quad e_{j,t|k}^N = n_{j,t}^{\text{real}} - \hat{n}_{j,t|k}^{\text{OOS}}, \quad (43)$$

其中问题 4-2 取 $k = 0$ ，问题 4-3 取 $k \in \{0, 6, 12, 18\}$ 。将同一历史日期、相同目标时段的两类误差配对，形成联合误差轨迹

$$\boldsymbol{\xi}_{j|k} = (\mathbf{e}_{j|k}^N, \mathbf{e}_j^p), \quad (44)$$

从而保留二者的同步变化、日内相关性及联合尾部特征。

考虑误差分布随季节变化，第 d 日节点 k 仅采用此前已经完整实现的最近 56 日联合轨迹；历史不足 56 日时使用全部可用样本。将联合误差叠加至当前中心预测，得到场景 j 下未来净负荷和电价

$$n_{d,t,j|k} = \hat{n}_{d,t|k} + e_{j,t|k}^N, \quad p_{d,t,j|k} = [\hat{p}_{d,t} + e_{j,t}^p]_+, \quad t > k, \quad (45)$$

其中价格下截断用于排除残差叠加产生的负值。

为控制后续鲁棒优化规模，对联合轨迹按历史误差尺度标准化并进行代表场景缩减，正式场景数不超过 20；同时单独保留联合高风险尾部轨迹，代表场景概率按其覆盖的原始历史轨迹经验频数确定。因此，场景压缩在降低计算规模的同时，仍保留价格—净负荷的联合依赖及高成本尾部状态。

6.3 价格—净负荷联合分布鲁棒调度模型

6.3.1 Wasserstein 模糊集构造与确定性等价

前述联合场景给出了价格—净负荷误差的经验分布，但有限历史样本下场景概率仍存在估计偏差。因此，本文不直接以经验概率作为真实分布，而在其邻域内考虑最不利的概率配置。记节点 k 下缩减后的 S 个联合场景为 $\{\xi_{i|k}\}_{i=1}^S$ ，相应经验概率为 $\hat{\pi}_i$ ，则

$$\hat{\mathbb{P}}_{d,k} = \sum_{i=1}^S \hat{\pi}_i \delta_{\xi_{i|k}}. \quad (46)$$

由于净负荷误差与电价误差的量纲和波动尺度不同，直接计算轨迹距离会使距离度量受到数值尺度影响。因此，分别按当前可用历史样本在各目标时段上的稳健尺度进行标准化。记节点 k 当前优化窗口内的目标时段集合为 $\mathcal{H}_k$ ，净负荷与电价误差对应的 1.4826 倍中位绝对偏差尺度分别为 $s_{t|k}^N$ 和 s_t^p 。对任意两个联合场景 i, j ，定义

$$\begin{aligned} d_{ij}^N &= \frac{1}{|\mathcal{H}_k|} \sum_{t \in \mathcal{H}_k} \frac{|e_{i,t|k}^N - e_{j,t|k}^N|}{s_{t|k}^N}, \\ d_{ij}^p &= \frac{1}{|\mathcal{H}_k|} \sum_{t \in \mathcal{H}_k} \frac{|e_{i,t}^p - e_{j,t}^p|}{s_t^p}, \\ d_{ij} &= \omega_N d_{ij}^N + \omega_p d_{ij}^p, \quad \omega_N + \omega_p = 1. \end{aligned} \quad (47)$$

其中， d_{ij}^N 和 d_{ij}^p 分别衡量净负荷与电价误差轨迹的差异。基准模型取 $(\omega_N, \omega_p) = (0.5, 0.5)$ ，即两类误差等权；后文进一步考察联合距离权重变化对调度结果的影响。

以 Γ_{ij} 表示由经验场景 i 向场景 j 转移的概率质量，构造有限支持 Wasserstein 模糊

集

$$\mathcal{P}_{d,k}(\rho_{d,k}) = \left\{ \mathbf{q} \left| \begin{array}{l} \Gamma_{ij} \geq 0, \quad \sum_j \Gamma_{ij} = \hat{\pi}_i, \\ \sum_i \Gamma_{ij} = q_j, \quad \sum_{i,j} d_{ij} \Gamma_{ij} \leq \rho_{d,k} \end{array} \right. \right\}. \quad (48)$$

该集合仅允许在已保留的联合轨迹之间重新分配概率质量，因而不会人为生成新的价格-净负荷组合；联合高风险轨迹则由前述尾部保持机制纳入支持集。

为使鲁棒程度随当前历史样本自适应变化，在每个决策节点仅利用当时可得的联合历史轨迹进行 $B = 100$ 次 bootstrap 重采样，并取经验 Wasserstein 距离的 95% 分位数作为半径

$$\rho_{d,k} = Q_{0.95} \left(\left\{ W(\hat{\mathbb{P}}_{d,k}^{*(b)}, \hat{\mathbb{P}}_{d,k}) \right\}_{b=1}^B \right). \quad (49)$$

设决策 $\mathbf{x}$ 在联合场景 j 下的运行成本为 $\Phi_j(\mathbf{x})$ 。则分布鲁棒目标中的最坏期望成本为

$$\min_{\mathbf{x} \in \mathcal{X}} \sup_{\mathbb{P} \in \mathcal{P}_{d,k}(\rho_{d,k})} \mathbb{E}_{\mathbb{P}}[\Phi(\mathbf{x}, \boldsymbol{\xi})], \quad (50)$$

其中 $\mathcal{X}$ 表示后文给出的购电与储能可行域。由于内层问题为有限场景上的概率运输线性规划，由强对偶可将其等价改写为

$$\begin{aligned} \min_{\mathbf{x}, \lambda^W, \alpha} \quad & \lambda^W \rho_{d,k} + \sum_{i=1}^S \hat{\pi}_i \alpha_i, \\ \text{s.t.} \quad & \alpha_i \geq \Phi_j(\mathbf{x}) - \lambda^W d_{ij}, \quad \forall i, j, \\ & \lambda^W \geq 0, \quad \mathbf{x} \in \mathcal{X}. \end{aligned} \quad (51)$$

由此，无需直接优化无限多个候选概率分布，即可在有限联合场景上求得最坏分布下的鲁棒调度策略。

6.3.2 问题 4-2 的日前分布鲁棒调度

问题 4-2 每日 0:00 制定一次正式调度方案。为减弱有限时域造成的日末储能偏差，采用 48 h 前瞻窗口，记 $\tau = 1, \dots, T$ ，其中 $T = 288$ 。前 144 个时段为当日执行区间，后 144 个时段仅用于刻画储能的跨日延续价值，并在次日重新优化。

为满足非前瞻性，计划购电量 g_τ 、充电量 c_τ 、放电量 r_τ 及储电量 E_τ 对所有联合场景共享；场景 j 下仅实际调用量 $x_{\tau j}$ 、紧急购电量 $h_{\tau j}$ 和弃电量 $w_{\tau j}$ 随不确定状态变化。其中 g_τ 为计划购电结算量，正常购电费用始终按 g_τ 计取，而非按 $x_{\tau j}$ 计取。

各场景满足

$$\begin{cases} E_\tau = E_{\tau-1} + \eta_c c_\tau - \frac{r_\tau}{\eta_d}, & E_{\min} \leq E_\tau \leq E_{\max}, \\ c_\tau + r_\tau \leq Q_{\max}, & 0 \leq x_{\tau j} \leq g_\tau, \\ x_{\tau j} + r_\tau + h_{\tau j} = n_{\tau j} + c_\tau + w_{\tau j}, \\ g_\tau, c_\tau, r_\tau, x_{\tau j}, h_{\tau j}, w_{\tau j} \geq 0. \end{cases} \quad (52)$$

其中 $E_0 = E_{d,0}$ ；正式回测首日取 $E_{d,0} = 6000 \text{ kWh}$ ，此后由上一日实际运行末态传递。

对联合场景 j ，定义前瞻成本

$$\phi_j = \sum_{\tau=1}^T p_{\tau j} (g_{\tau} + 5h_{\tau j}). \quad (53)$$

其中后 144 个时段的费用仅参与当前优化，不计入当日实际结算。代入前述有限支持 Wasserstein 对偶模型，得到

$$\begin{aligned} \min \quad & \lambda^W \rho_{d,0} + \sum_{i=1}^S \hat{\pi}_i \alpha_i + \varepsilon_{\text{th}} \sum_{\tau=1}^T (c_{\tau} + r_{\tau}), \\ \text{s.t.} \quad & \alpha_i \geq \phi_j - \lambda^W d_{ij}, \quad \forall i, j, \\ & \lambda^W \geq 0, \quad \text{式 (52) 成立.} \end{aligned} \quad (54)$$

其中 $\varepsilon_{\text{th}} = 10^{-7}$ ，仅用于消除无经济意义的充放电退化。模型全部采用连续变量，可直接转化为线性规划求解。

求解后仅执行当前日的前 144 个时段方案；实际运行按真实净负荷逐时完成能量平衡，按附件 4 的真实电价核算费用，日末实际储电量传递至次日。

6.3.3 问题 4-3 的四时点分布鲁棒滚动调度

问题 4-3 在问题 4-2 基础上增加 0:00、6:00、12:00 和 18:00 四个信息更新节点。每个节点以当前实际储电量为初始状态，结合截至该时刻已实现的源荷、电价信息及最新光伏预报，重新优化未来 24 h；其中仅下一节点到达前的 6 h 决策锁定执行，其余部分仅用于前瞻评价并在下一节点重新计算。

为利用日内新增信息，对 $k \in \{6, 12, 18\}$ ，根据当前已实现状态与历史日对应前缀的标准化距离 $D_{j,k}$ ，更新历史联合误差轨迹的经验概率：

$$\hat{\pi}_{j|k} = \frac{\exp(-D_{j,k}/h_k)}{\sum_{\ell} \exp(-D_{\ell,k}/h_k)}. \quad (55)$$

随后取同一历史日对应的节点 k 后净负荷与价格样本外误差构造未来联合场景。该过程仅使用节点前已实现信息，从而在保持价格-净负荷联合结构的同时实现条件场景更新。

各节点沿用问题 4-2 的连续分布鲁棒调度模型，仅将经验场景概率、当前储电状态及净负荷中心预测更新至节点 k ，并以相应 Wasserstein 半径 $\rho_{d,k}$ 重新求解。当天最终生效的购电量由四个承诺区间依次拼接：

$$g_{d,t}^{\text{eff}} = \begin{cases} g_{d,t}^{(0)}, & 1 \leq t \leq 36, \\ g_{d,t}^{(6)}, & 37 \leq t \leq 72, \\ g_{d,t}^{(12)}, & 73 \leq t \leq 108, \\ g_{d,t}^{(18)}, & 109 \leq t \leq 144. \end{cases} \quad (56)$$

记

$$\Delta_{d,t}^+ = (g_{d,t}^{\text{eff}} - g_{d,t}^0)_+, \quad \Delta_{d,t}^- = (g_{d,t}^0 - g_{d,t}^{\text{eff}})_+,$$

则多次调整统一按最终生效购电量相对 0:00 原计划一次结算。真实轨迹回放后，当日费用为

$$\begin{aligned} C_d^{(4-3)} = & \sum_{t=1}^{144} p_{d,t}^{\text{real}} g_{d,t}^0 \\ & + \sum_{t=1}^{144} p_{d,t}^{\text{real}} (1.5\Delta_{d,t}^+ - 0.5\Delta_{d,t}^- + 5h_{d,t}^{\text{real}}). \end{aligned} \quad (57)$$

跨入次日的前瞻决策不计入当日实际结算，日末实际储电量直接传递为次日初始状态。

6.4 模型检验与调度结果分析

为检验模型的数值可靠性，将所得策略置于真实负荷、光伏及波动电价序列中，按时间顺序逐 10 min 回放，并对求解状态、约束残差及运行边界进行复核，结果见表 12。

表 12 问题四模型求解与可行性检验

检验项目	检验结果
正式优化节点	1670/1670 均为 Optimal
最大原约束残差	6.90×10^{-10}
最大最优性证书残差	4.37×10^{-11}
能量平衡、储电量及充放电边界	全部满足

各正式优化节点均获得最优解，且约束残差处于较低数量级，能量平衡、储电状态及充放电边界均满足，表明所得策略具有良好的数值可行性。进一步将 Q4-2 日前策略与 Q4-3 四时点滚动策略进行真实回放比较，结果见表 13。

表 13 波动电价下两类调度策略的真实回放结果

指标	Q4-2 日前调度	Q4-3 四时点滚动	变化率
总结算费用/万元	1574.779	1542.780	-2.03%
计划购电费用/万元	1487.054	1510.091	+1.55%
紧急购电量/万 kWh	22.586	15.064	-33.30%
紧急购电费用/万元	87.725	58.615	-33.18%
日费用 $Q_{0.95}$ /万元	7.733	7.659	-0.96%

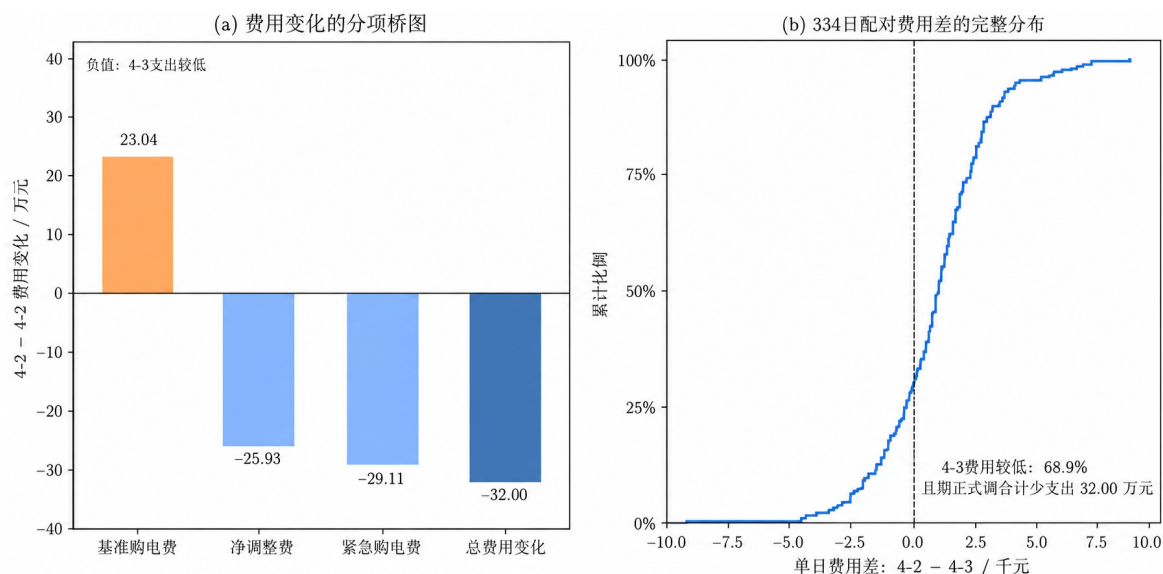


图 4 四时点滚动调度的费用变化分解及逐日费用差分布

由表 13 及图 4(a) 可见，Q4-3 的基准购电费用较 Q4-2 增加 23.04 万元，但净调整费用和紧急购电费用分别减少 25.93 万元和 29.11 万元，最终使全年总结算费用下降 32.00 万元。这表明滚动调度并非依靠压缩常规购电获得收益，而是通过日内信息更新及时修正购电与储能决策，以适度增加事前供能保障换取更低的调整损失和高倍紧急补购风险。

图 4(b) 进一步表明，在 334 个运行日中，Q4-3 有 230 日费用低于 Q4-2，占 68.86%，说明全年费用改善并非由少数极端日期驱动。与此同时，紧急购电量和紧急购电费用分别下降 33.30% 和 33.18%，远高于总费用 2.03% 的降幅，说明四时点滚动更新的主要价值在于降低供需偏差导致的高成本补购暴露。

6.5 敏感性分析

选取代表性工况，对 Wasserstein 半径及价格-净负荷联合距离权重进行单因素扰动，结果见表 14。

表 14 问题四关键参数敏感性分析

模型	参数设置	参数含义	费用变化/%	紧急购电量变化/kWh
Q4-2	$0.75\rho^*$	较弱鲁棒	0.669 [-0.113, 1.530]	285.22 [66.997, 568.939]
	ρ^*	基准设置	0	0
	$1.25\rho^*$	较强鲁棒	-0.862 [-2.173, 0.212]	-308.58 [-561.637, -41.385]
Q4-2	(0.7, 0.3)	净负荷权重较高	-0.035 [-1.489, 1.194]	15.79 [-382.110, 458.974]
	(0.5, 0.5)	等权基准	0	0
	(0.3, 0.7)	电价权重较高	-1.228 [-1.470, -0.215]	-260.48 [-379.707, -47.503]
Q4-3	$0.75\rho^*$	较弱鲁棒	1.935 [-0.152, 4.022]	437.29 [4.331, 870.247]
	ρ^*	基准设置	0	0
	$1.25\rho^*$	较强鲁棒	-0.781 [-1.808, 0.246]	-225.44 [-450.870, 0]

各扰动情形均保持最优可行，且能量平衡与储能运行约束均满足。Q4-2 对关键参数变化总体较稳定；Q4-3 在常规工况下费用变化较小，但高联合风险日最大变化达到 4.02%，说明极端联合状态下滚动策略对鲁棒参数更为敏感。

七、模型评价

7.1 模型的优点

- 1) 本文按照“确定性调度-源荷不确定性调度-多时点滚动调整-价格与净负荷联合分布鲁棒调度”的递进结构建立统一模型，各问之间具有较强的继承性与解释性。
- 2) 预测、场景构造与调度全过程严格遵守信息非前瞻原则，并利用样本外预测误差刻画不确定性，避免未来真实信息泄漏对策略评价造成偏差。
- 3) 模型同时考虑能量平衡、储能状态转移、充放电功率及购电结算规则，并通过滚动时域与 Wasserstein 分布鲁棒优化兼顾经济性、物理可行性及极端场景风险。

7.2 模型的缺点

- 1) 受题目数据与篇幅限制，模型未进一步考虑储能寿命衰减、自放电及线路损耗等长期运行因素，实际应用时需补充相应成本与物理参数。
- 2) 源荷及电价不确定性主要依据有限历史样本外误差刻画，对历史中未出现的突发极端状态仍可能存在描述不足。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码调试和文献检索辅助，详细使用情况见支撑材料。

参考文献

- [1] KE G, MENG Q, FINLEY T, et al. LightGBM: A highly efficient gradient boosting decision tree[C]// Advances in Neural Information Processing Systems. 2017, 30: 3146–3154.
- [2] KAUFMAN L, ROUSSEEUW P J. Finding Groups in Data: An Introduction to Cluster Analysis[M]. New York: John Wiley & Sons, 1990.
- [3] DUPAČOVÁ J, GRÖWE-KUSKA N, RÖMISCH W. Scenario reduction in stochastic programming[J]. Mathematical Programming, 2003, 95(3): 493–511. DOI:10.1007/s10107-002-0331-0.

附录 A 支撑材料文件列表

支撑材料位于 `approval` 目录，按下表列示。目录项包含该目录下的文件；完整逐文件清单见支撑材料中的文件清单。

序号	文件路径	内容说明
1	<code>src/q1/</code>	问题一源程序及配套文件，共 17 份
2	<code>src/q2/</code>	问题二源程序及配套文件，共 42 份
3	<code>src/q3/</code>	问题三源程序及配套文件，共 35 份
4	<code>src/q4/</code>	问题四源程序及配套文件，共 20 份
5	<code>src/sensitivity/</code>	敏感性分析程序及配套文件，共 11 份
6	<code>src/plots/</code>	绘图与数据整理程序及配套文件，共 26 份
7	<code>src/问题 1/</code>	问题一补充程序及配套文件，共 1 份
8	<code>docs/</code>	题目资料、模型文档与运行依据
9	<code>inputs/</code>	程序所需输入与价格缓存
10	<code>outputs/</code>	计算结果、冻结预测及实验数据
11	<code>参考文献/</code>	三项参考文献书目、两篇论文全文及获取说明
12	<code>README.md</code>	支撑材料总说明
13	<code>运行说明.md</code>	程序入口与运行步骤
14	<code>AI 使用报告.md</code>	AI 辅助使用情况说明
15	<code>文件清单.csv</code>	支撑材料逐文件清单
16	<code>校验结果.json</code>	支撑材料已有校验记录

附录 B 完整源程序

本附录收录支撑材料 `src` 目录中的全部 145 份源程序文件，按问题及用途分组，保留完整程序内容。论文工程中的对应副本位于 `code/src` 目录；依赖说明和配套数据文件随源码一并保存。

B.1 问题一源程序

`src/q1/analysis.py`

```
1 """强化版11.3 - 11.8: 同一模型的诊断、局部边际量与MILP重求敏感性。"""
2
3 import numpy as np
4
5 from export import read_csv
```



```

6 from model import (DT, PHYS_TOL_KWH, COST_TOL_YUAN, BINARY_TOL, ECONOMIC_VALUE_TOL, MIP_GAP_TOL,
7                     SCENARIOS, arrays, build_problem,
8                     check_solution, solve, solve_lp, solve_milp, summary, write_csv, write_json)
9
10 ANALYSIS_SETTINGS = {"active_tolerance_kwh": PHYS_TOL_KWH, "rhs_step_kwh": 0.01,
11                      "main_capacity_steps_kwh": [1.0, 10.0, 100.0],
12                      "main_power_steps_kw": [1.0, 10.0, 100.0],
13                      "all_scenario_boundary_steps": [1.0, 10.0, 100.0],
14                      "economic_significance_threshold": None,
15                      "economic_significance_note": "题目未给设备投资成本或经济显著性门槛；仅报告数值稳定性和节
16                      约",
17                      "load_deltas": [-0.05, 0.0, 0.05], "pv_deltas": [-0.05, 0.0, 0.05],
18                      "numerical_setting_note": "实现中的数值诊断设置；不改变主模型设备参数和目标"}
19
20 def verify_lp(problem, result):
21     """核查LP原始可行性和KKT；只服务LP诊断，不替代MILP最优性证明。"""
22     if result.status != 0 or not result.success:
23         raise ValueError(f"LP未最优: {result.message}")
24     n = len(problem["index"]["G"])
25     x, a, lo, hi = result.x, problem["a"], problem["lower"], problem["upper"]
26     y, z = result.eqlin.marginals, result.ineqlin.marginals
27     yl, yu = result.lower.marginals, result.upper.marginals
28     if not all(np.all(np.isfinite(v)) for v in (x, y, z, yl, yu)):
29         raise ValueError("LP原始或对偶变量非有限")
30     ubfinite = np.isfinite(hi)
31     slack = problem["ub"][2 * n:] - a[2 * n:] @ x
32     dual = y @ problem["ub"][:2 * n] + z @ problem["ub"][2 * n:]
33     dual += yl @ lo + yu[ubfinite] @ hi[ubfinite]
34     metrics = {
35         "equality_residual": float(np.max(np.abs(a[:2 * n] @ x - problem["ub"][:2 * n])),
36         "inequality_violation": float(max(0, -slack.min())),
37         "bound_violation": float(max(0, np.max(lo - x), np.max(x - hi))),
38         "stationarity_residual": float(np.max(np.abs(problem["objective"] - a[:2 * n].T @ y
39         - a[2 * n:].T @ z - yl - yu))),
40         "complementarity_residual": float(max(np.max(np.abs(z * slack)),
41         np.max(np.abs(yl * (x - lo))),
42         np.max(np.abs(yu[ubfinite] * (hi[ubfinite] - x[
43         ubfinite]))))),
44         "dual_sign_violation": float(max(0, z.max(), -yl.min(), yu.max())),
45         "objective_recalculation_error": float(abs(problem["objective"] @ x - result.fun)),
46         "primal_dual_gap": float(abs(result.fun - dual))
47     }
48     tolerances = {k: (COST_TOL_YUAN if k in ("complementarity_residual", "
49     objective_recalculation_error", "primal_dual_gap")
50     else ECONOMIC_VALUE_TOL if k in ("stationarity_residual", "dual_sign_violation
51     ")
52     else PHYS_TOL_KWH for k in metrics}
53     if any(not np.isfinite(v) or v > tolerances[k] for k, v in metrics.items()):
54         raise ValueError(f"LP检查失败: {metrics}")
55     return {"passed": True, "metrics": metrics, "metric_tolerances": tolerances}
56
57 def lp_record(problem, result):
58     return {"variables": {k: result.x[v].tolist() for k, v in problem["index"].items()},
59             "objective_yuan": float(result.fun), "status": int(result.status),
60             "equality_rhs_marginals": result.eqlin.marginals.tolist(),
61             "inequality_rhs_marginals": result.ineqlin.marginals.tolist(),

```

```

58         "lower_marginals": result.lower.marginals.tolist(),
59         "upper_marginals": result.upper.marginals.tolist()}
60
61
62 def active_sets(solution):
63     settings = solution["settings"]
64     return {key: (np.flatnonzero(np.abs(np.array(solution[var]) - limit) <= PHYS_TOL_KWH) + 1).
65             tolist()
66             for key, var, limit in (("E_min", "E", settings["e_min"]), ("E_max", "E", settings["
67             e_max"]),
68                                     ("charge", "C", settings["charge_kw"] * DT),
69                                     ("discharge", "D", settings["discharge_kw"] * DT))}
70
71 def scaled_data(data, dl, dpv):
72     return [{**r, "load_kw": r["load_kw"] * (1 + dl), "load_kwh": r["load_kwh"] * (1 + dl),
73             "pv_kw": r["pv_kw"] * (1 + dpv), "pv_kwh": r["pv_kwh"] * (1 + dpv)} for r in data]
74
75 def local_marginals(data, solution, scenario):
76     mode = np rint(solution["u"]).astype(int)
77     problem = build_problem(data, solution["eta_c"], solution["eta_d"], **solution["settings"],
78                             fixed_mode=mode)
79     result = solve_lp(problem)
80     checks = verify_lp(problem, result)
81     if abs(result.fun - solution["solver"]["objective_yuan"]) > COST_TOL_YUAN:
82         raise ValueError("固定最优模式LP费用与主MILP不一致")
83     n, ec, ed = len(data), solution["eta_c"], solution["eta_d"]
84     active = active_sets(solution)
85     # SciPy marginals=dJ/db. SOC RHS增加为内部注入, 价值lambda=-dJ/dh.
86     mu, lam = result.eqlin.marginals[:n], -result.eqlin.marginals[n:]
87     rows = []
88     for i, r in enumerate(data):
89         low, high = ec * lam[i], lam[i] / ed
90         gain_c, gain_d = low - mu[i], mu[i] - high
91         c, d, e = (solution[k][i] for k in ("C", "D", "E"))
92         if gain_c > ECONOMIC_VALUE_TOL:
93             tendency = "charge"
94         elif gain_d > ECONOMIC_VALUE_TOL:
95             tendency = "discharge"
96         elif min(abs(mu[i] - low), abs(mu[i] - high)) <= ECONOMIC_VALUE_TOL:
97             tendency = "indifferent_boundary"
98         else:
99             tendency = "idle_band"
100         action = "charge" if c > PHYS_TOL_KWH else "discharge" if d > PHYS_TOL_KWH else "idle"
101         if (c > PHYS_TOL_KWH and gain_c < -ECONOMIC_VALUE_TOL) or (d > PHYS_TOL_KWH and gain_d < -
102         ECONOMIC_VALUE_TOL):
103             raise ValueError("实际非零动作与允许方向的局部边际条件矛盾")
104         rows.append({"scenario": scenario, "t": i + 1, "interval": r["interval"],
105                     "price_yuan_per_kwh": r["price_yuan_per_kwh"], "mu_yuan_per_kwh": float(mu[i]),
106                     "lambda_yuan_per_kwh": float(lam[i]), "charge_threshold": float(low),
107                     "discharge_threshold": float(high), "charge_gain": float(gain_c),
108                     "discharge_gain": float(gain_d), "tendency": tendency, "actual_action": action,
109                     "fixed_u": int(mode[i]), "mode_blocks_tendency": int(
110                         (tendency == "charge" and mode[i] == 0) or (tendency == "discharge" and
111                         mode[i] == 1)),
112                     "E_min_active": int(i + 1 in active["E_min"]), "E_max_active": int(i + 1 in

```

```

110         active["E_max"]),
111         "charge_max_active": int(i + 1 in active["charge"]),
112         "discharge_max_active": int(i + 1 in active["discharge"])}))
113     return rows, problem, result, checks
114
115 def rhs_validation(problem, result, step=0.01):
116     """对所有母线需求/内部注入做双侧微扰；退化时校验影子价落在差分区间。"""
117     n = len(problem["index"]["G"])
118     records = []
119     for row, shadow in enumerate(result.eqlin.marginals):
120         slopes, statuses = {}, {}
121         for side, sign in (("minus", -1), ("plus", 1)):
122             changed = solve_lp(problem, (row, sign * step))
123             statuses[side] = int(changed.status)
124             if changed.status == 0:
125                 shifted = {**problem, "ub": problem["ub"].copy()}
126                 shifted["ub"][row] += sign * step
127                 verify_lp(shifted, changed)
128                 slopes[side] = float((changed.fun - result.fun) / (sign * step))
129             elif changed.status == 2:
130                 slopes[side] = None # 边界处方向不可行；绝不填造差分。
131             else:
132                 raise ValueError(f"RHS扰动求解异常: {changed.message}")
133             violation = max(0, (slopes["minus"] - shadow) if slopes["minus"] is not None else 0,
134                             (shadow - slopes["plus"]) if slopes["plus"] is not None else 0)
135             if violation > ECONOMIC_VALUE_TOL:
136                 raise ValueError(f"影子价格符号或差分区间错误: row={row}, {slopes}, shadow={shadow}")
137             records.append({"kind": "bus_demand" if row < n else "internal_injection",
138                             "t": row % n + 1, "step_kwh": step, "shadow_dJ_db": float(shadow),
139                             "minus_slope": slopes["minus"], "plus_slope": slopes["plus"],
140                             "minus_status": statuses["minus"], "plus_status": statuses["plus"],
141                             "interval_violation": float(violation),
142                             "two_sided_smooth": int(all(v is not None and abs(v - shadow) <=
143                             ECONOMIC_VALUE_TOL for v in slopes.values()))})
144     return records
145
146 def milp_rhs_validation(data, solution, fixed_rows, step=0.01):
147     """CR-02: 全部模式可重新选择；不将非凸MILP值函数强作凸函数。"""
148     problem = build_problem(data, solution["eta_c"], solution["eta_d"], **solution["settings"])
149     base = solution["solver"]["objective_yuan"]
150     records, raw = [], []
151     n = len(data)
152     for row, fixed in enumerate(fixed_rows):
153         item = {"kind": fixed["kind"], "t": fixed["t"], "step_kwh": step,
154                 "fixed_shadow_dJ_db": fixed["shadow_dJ_db"],
155                 "base_action": "idle" if max(solution["C"][row % n], solution["D"][row % n]) <=
156                 PHYS_TOL_KWH else "active"}
157         for side, sign in (("minus", -1), ("plus", 1)):
158             changed = {**problem, "lb": problem["lb"].copy(), "ub": problem["ub"].copy()}
159             changed["lb"][row] += sign * step
160             changed["ub"][row] += sign * step
161             result = solve_milp(changed)
162             item[side + "_status"] = int(result.status)
163             item[side + "_slope"] = None
164             if result.status == 2:

```

```

164         raw.append({"row": row, "side": side, "status": 2})
165         continue
166     if result.status != 0 or not result.success:
167         raise ValueError(f"原MILP扰动失败: {result.message}")
168     if not np.all(np.isfinite(result.x)) or not np.all(np.isfinite([result.fun, result.
mip_dual_bound, result.mip_gap])):
169         raise ValueError("原MILP扰动含非有限结果")
170     x, lhs = result.x, changed["a"] @ result.x
171     residual = float(max(0, np.max(changed["lb"] - lhs), np.max(lhs - changed["ub"]),
172                          np.max(changed["lower"] - x), np.max(x - changed["upper"])))
173     binary_error = float(np.max(np.abs(x[problem["index"]["u"]] - np rint(x[problem["index"
]]["u"]))))
174     cost_error = float(abs(problem["objective"] @ x - result.fun))
175     bound_gap = float(abs(result.fun - result.mip_dual_bound))
176     if (residual > PHYS_TOL_KWH or binary_error > BINARY_TOL or cost_error > COST_TOL_YUAN
177         or bound_gap > COST_TOL_YUAN or not np.isfinite(result.mip_gap) or result.
mip_gap > MIP_GAP_TOL):
178         raise ValueError("原MILP扰动的物理/整数/最优性复核失败")
179     slope = float((result.fun - base) / (sign * step))
180     item[side + "_slope"] = slope
181     # 固定模式可行域是原MILP子集, 只比较目标, 不假设两侧导数次序。
182     fixed_slope = fixed[side + "_slope"]
183     if fixed_slope is not None and result.fun > base + fixed_slope * sign * step +
COST_TOL_YUAN:
184         raise ValueError("原MILP最优费用高于可行固定模式LP")
185     raw.append({"row": row, "side": side, "status": 0,
186                "objective_yuan": float(result.fun), "dual_bound_yuan": float(result.
mip_dual_bound),
187                "mip_gap": float(result.mip_gap), "residual_kwh": residual,
188                "variables": {k: x[v].tolist() for k, v in problem["index"].items()})
189     item["fixed_shadow_matches_milp_both_sides"] = int(all(
190         item[side + "_slope"] is not None and
191         abs(item[side + "_slope"] - fixed["shadow_dj_db"]) <= ECONOMIC_VALUE_TOL
192         for side in ("minus", "plus")))
193     item["milp_two_sided_smooth"] = int(item["minus_slope"] is not None and item["plus_slope"]
is not None
194                                         and abs(item["minus_slope"] - item["plus_slope"])
<= ECONOMIC_VALUE_TOL)
195     # lambda = -dj/dh; 保留两侧来源, 避免将非光滑点压成唯一价格。
196     for side in ("minus", "plus"):
197         slope = item[side + "_slope"]
198         item[side + "_economic_value"] = None if slope is None else slope * (1 if row < n else
-1)
199     records.append(item)
200     return records, raw
201
202
203 def bottlenecks(data, solution, scenario, steps):
204     rows, records = [], {}
205     active = active_sets(solution)
206     base_cost = solution["solver"]["objective_yuan"]
207     for resource, parameter, direction, unit, active_key in (
208         ("storage_upper", "e_max", 1, "yuan/(kWh day)", "E_max"),
209         ("storage_lower", "e_min", -1, "yuan/(kWh day)", "E_min"),
210         ("charge_power", "charge_kw", 1, "yuan/(kW day)", "charge"),
211         ("discharge_power", "discharge_kw", 1, "yuan/(kW day)", "discharge")):
212         group = []

```

```

213     base = solution["settings"][parameter]
214     for step in steps:
215         settings = {**solution["settings"], parameter: base + direction * step}
216         changed = solve(data, solution["eta_c"], solution["eta_d"], **settings)
217         check = check_solution(data, changed)
218         saving = base_cost - changed["solver"]["objective_yuan"]
219         if saving < -COST_TOL_YUAN:
220             raise ValueError("放宽约束后最优费用增加, 违反嵌套可行域")
221         group.append({"scenario": scenario, "resource": resource, "base_limit": base,
222                     "increment": step, "parameter_change": direction * step,
223                     "increment_period_kwh": step if parameter in ("e_min", "e_max") else step
224 * DT,
225                     "base_cost_yuan": base_cost, "relaxed_cost_yuan": changed["solver"]["
226 objective_yuan"],
227                     "saving_yuan": saving, "value_per_unit_day": saving / step, "unit": unit,
228                     "active_count": len(active[active_key]),
229                     "active_periods": ";".join(map(str, active[active_key])),
230                     "global_gap": changed["solver"]["mip_gap"]})
231         records[f"{resource}_{step:g}"] = {"solution": changed, "validation": check}
232         values = [r["value_per_unit_day"] for r in group]
233         stable = len(values) >= 3 and max(values) - min(values) <= ECONOMIC_VALUE_TOL
234         for r in group:
235             r.update({"multistep_value_spread": max(values) - min(values), "multistep_stable": int(
236 stable),
237                     "stable_positive_value": int(stable and min(values) > ECONOMIC_VALUE_TOL),
238                     "numerical_bottleneck": int(bool(active[active_key]) and stable and min(values
239 ) > ECONOMIC_VALUE_TOL),
240                     "economic_bottleneck": None,
241                     "economic_significance": "not_assessed_no_economic_threshold"})
242         rows.extend(group)
243     return rows, records
244
245 def checked_analysis_csv(path, rows):
246     if any(isinstance(v, (float, np.floating)) and not np.isfinite(v) for r in rows for v in r.
247 values()):
248         raise ValueError(f"分析CSV含非有限数值: {path}")
249     write_csv(path, rows)
250     actual = read_csv(path)
251     expected = [{"k": " " if v is None else str(v) for k, v in r.items()} for r in rows]
252     if actual != expected:
253         raise ValueError(f"分析CSV关闭回读不一致: {path}")
254     return {"passed": True, "rows": len(rows), "columns": list(rows[0])}
255
256 def run_analysis(data, solutions, output_raw, output):
257     """覆盖三档单程效率和往返口径×负荷三档×光伏三档, 共36情景。"""
258     main = solutions["main"]
259     raw, checks, csv_checks = {}, {}, {}
260     baseline_rows = []
261     for name in ("no_ess", "pv_only", "full"):
262         s = main if name == "full" else solve(data, 0.9, baseline=name)
263         checks[name] = check_solution(data, s)
264         raw[name] = s
265         baseline_rows.append({"baseline": name, **summary(data, s)})
266     j0, jpv, jm = (r["cost_yuan"] for r in baseline_rows)
267     if not jm <= jpv + COST_TOL_YUAN <= j0 + 2 * COST_TOL_YUAN:

```

```

265         raise ValueError("嵌套基准成本次序错误")
266     p, load, pv = arrays(data)
267     if abs(j0 - p @ np.maximum(load - pv, 0)) > COST_TOL_YUAN:
268         raise ValueError("No-ESS与解析基准费用不符")
269     benefits = {"pv_increment_yuan": j0 - jpv, "flex_increment_yuan": jpv - jm,
270               "total_saving_yuan": j0 - jm,
271               "interpretation": "嵌套可行域增量节约; 不是唯一光伏/套利因果分解"}
272     problem = build_problem(data, 0.9)
273     relaxed = solve_lp(problem)
274     checks["lp_relaxation"] = verify_lp(problem, relaxed)
275     raw["lp_relaxation"] = lp_record(problem, relaxed)
276     u = relaxed.x[problem["index"]["u"]]
277     c, d = (relaxed.x[problem["index"][v]] for v in ("C", "D"))
278     diagnosis = {"milp_cost_yuan": jm, "lp_cost_yuan": float(relaxed.fun),
279               "integer_gap": float((jm - relaxed.fun) / max(1, abs(jm))),
280               "fractional_modes": int(np.sum((u > BINARY_TOL) & (u < 1 - BINARY_TOL))),
281               "simultaneous_periods": int(np.sum((c > PHYS_TOL_KWH) & (d > PHYS_TOL_KWH)))}
282     if relaxed.fun > jm + COST_TOL_YUAN:
283         raise ValueError("连续松弛下界大于MILP费用")
284     scenario_rows, marginal_rows, capacity_rows, all_schedules, active_rows = [], [], [], [], []
285     raw["scenarios"] = {}
286     rhs_rows, milp_rhs_rows = [], []
287     for efficiency_name, efficiency in SCENARIOS.items():
288         for dl in ANALYSIS_SETTINGS["load_deltas"]:
289             for dpv in ANALYSIS_SETTINGS["pv_deltas"]:
290                 key = f"{efficiency_name}_l{round(100 * (1 + dl))}_pv{round(100 * (1 + dpv))}"
291                 current = scaled_data(data, dl, dpv)
292                 s = solutions[efficiency_name] if dl == dpv == 0 else solve(current, efficiency)
293                 checks[key] = check_solution(current, s)
294                 local, fixed_problem, fixed_result, lp_check = local_marginals(current, s, key)
295                 checks[key + "_fixed_lp"] = lp_check
296                 marginal_rows.extend(local)
297                 active = active_sets(s)
298                 is_main = efficiency_name == "main" and dl == dpv == 0
299                 values, perturbed = bottlenecks(current, s, key, ANALYSIS_SETTINGS["
all_scenario_boundary_steps"])
300                 capacity_rows.extend(values)
301                 if is_main:
302                     rhs_rows = rhs_validation(fixed_problem, fixed_result, ANALYSIS_SETTINGS["
rhs_step_kwh"])
303                     milp_rhs_rows, raw["milp_rhs_solutions"] = milp_rhs_validation(current, s,
rhs_rows, ANALYSIS_SETTINGS["rhs_step_kwh"])
304                     values_one = {r["resource"]: r["value_per_unit_day"] for r in values if r["increment
"] == 1}
305                     aggregates = summary(current, s)
306                     scenario_rows.append({"scenario": key, "efficiency_name": efficiency_name,
307                                           "load_delta": dl, "pv_delta": dpv, **aggregates,
308                                           "arbitrage_price_multiplier": 1 / efficiency ** 2,
309                                           **{k + "_active_count": len(v) for k, v in active.items()},
310                                           **{k + "_value": v for k, v in values_one.items()},
311                                           "mode_blocks_tendency_count": sum(r["mode_blocks_tendency"]
for r in local),
312                                           "action_changed_periods": sum(
313                                             r["actual_action"] != ("charge" if main["C"][i] >
PHYS_TOL_KWH else "discharge" if main["D"][i] > PHYS_TOL_KWH else "idle")
314                                             for i, r in enumerate(local)),
315                                           "max_soc_change_from_main_kwh": float(np.max(np.abs(np.array(s

```

```

["E"]) - main["E"]))))))
316         for i, r in enumerate(current):
317             all_schedules.append({"scenario": key, **r,
318                                   **{v: s[v][i] for v in ("G", "C", "D", "E", "W", "u")}})
319             active_rows.append({"scenario": key, "t": i + 1, "interval": r["interval"],
320                                **{k + "_active": int(i + 1 in v) for k, v in active.items()
321                                }})
322             raw["scenarios"][key] = {"solution": s, "active_sets": active,
323                                     "fixed_lp": lp_record(fixed_problem, fixed_result), "
perturbations": perturbed}
324             print(f"分析 {len(scenario_rows)}/36 {key}: {aggregates['cost_yuan']:.6f} 元; 原MILP与
固定模式LP检查通过", flush=True)
325         sensitivity = []
326         for ename in SCENARIOS:
327             subset = [r for r in scenario_rows if r["efficiency_name"] == ename]
328             base = next(r for r in subset if r["load_delta"] == r["pv_delta"] == 0)
329             for factor, held in (("load_delta", "pv_delta"), ("pv_delta", "load_delta")):
330                 minus = next(r for r in subset if r[factor] == -0.05 and r[held] == 0)
331                 plus = next(r for r in subset if r[factor] == 0.05 and r[held] == 0)
332                 sensitivity.append({"efficiency_name": ename, "factor": factor, "delta": 0.05,
333                                   "minus_cost_yuan": minus["cost_yuan"], "base_cost_yuan": base["
cost_yuan"],
334                                   "plus_cost_yuan": plus["cost_yuan"],
335                                   "normalized_sensitivity": (plus["cost_yuan"] - minus["cost_yuan"]) /
(0.1 * base["cost_yuan"])}))
336 # 11.6: 只考虑当天未来时段, 理论价差条件不代替带边界的可执行计划。
337 threshold_rows = []
338 for ename, efficiency in SCENARIOS.items():
339     for i, r in enumerate(data):
340         future = p[i + 1:]
341         threshold_rows.append({"efficiency_name": ename, "t": i + 1, "interval": r["interval"],
342                                "current_price": float(p[i]), "future_price_threshold": float(p[i
] / efficiency ** 2),
343                                "future_max_price": float(future.max()) if len(future) else None,
344                                "future_profitable_periods": int(np.sum(future * efficiency ** 2
> p[i] + ECONOMIC_VALUE_TOL)),
345                                "pv_surplus_kwh": float(max(pv[i] - load[i], 0))})
346 tables = {"baseline_comparison.csv": baseline_rows, "lp_diagnosis.csv": [diagnosis],
347           "incremental_benefits.csv": [benefits], "marginal_values.csv": marginal_rows,
348           "marginal_rhs_validation.csv": rhs_rows, "milp_rhs_validation.csv": milp_rhs_rows, "
bottleneck_values.csv": capacity_rows,
349           "sensitivity_scenarios.csv": scenario_rows, "normalized_sensitivity.csv": sensitivity,
350           "active_constraints.csv": active_rows, "arbitrage_thresholds.csv": threshold_rows}
351 for name, rows in tables.items():
352     csv_checks[name] = checked_analysis_csv(output / name, rows)
353 csv_checks["analysis_schedules.csv"] = checked_analysis_csv(output_raw / "analysis_schedules.csv
", all_schedules)
354 result = {"settings": ANALYSIS_SETTINGS, "lp_diagnosis": diagnosis, "benefits": benefits,
355           "baseline_rows": baseline_rows, "scenarios": scenario_rows, "bottlenecks":
capacity_rows,
356           "normalized_sensitivity": sensitivity,
357           "rhs_nonsmooth_or_one_sided_count": sum(not r["two_sided_smooth"] for r in rhs_rows),
358           "milp_rhs_mismatch_count": sum(not r["fixed_shadow_matches_milp_both_sides"] for r in
milp_rhs_rows),
359           "milp_rhs_nonsmooth_count": sum(not r["milp_two_sided_smooth"] for r in milp_rhs_rows)
,
           "all_checks_passed": True}

```



```

360 # 主情景在遍历中的位置由标签定位，不依赖列表顺序。
361 result["main_marginals"] = [r for r in marginal_rows if r["scenario"] == "main_l100_pv100"]
362 write_json(output_raw / "analysis_solutions.json", raw)
363 write_json(output_raw / "analysis_validation.json", {"passed": True, "checks": checks,
364                                                    "csv_readback": csv_checks, "rhs_checks":
365                                                    rhs_rows, "milp_rhs_checks": milp_rhs_rows})
366 write_json(output / "analysis_summary.json", result)
367 return result

```

src/q1/analysis_report.py

```

1  """新版边际价值分析的科研图、可阅读报告和完整CSV入口。"""
2
3  import html
4  import re
5
6  from model import PHYS_TOL_KWH, ECONOMIC_VALUE_TOL
7  from report import html_table
8
9
10 def analysis_figures(a, output):
11     """兼容原入口；绘图实现统一维护于src/plots/q1_legacy.py。"""
12     import sys
13     from pathlib import Path
14     sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
15     from src.plots.q1_legacy import analysis_figures as render
16     return render(a, output)
17
18
19 def interpretations(a):
20     b, diag = a["benefits"], a["lp_diagnosis"]
21     main = next(r for r in a["scenarios"] if r["scenario"] == "main_l100_pv100")
22     local = a["main_marginals"]
23     texts = [
24         f"无储能费用为{a['baseline_rows'][0]['cost_yuan']:.6f}元，仅光伏消纳储能为{a['baseline_rows']
25         '[1]['cost_yuan']:.6f}元，完整MILP为{diag['milp_cost_yuan']:.6f}元。",
26         f"两步增量节约分别为{b['pv_increment_yuan']:.6f}元和{b['flex_increment_yuan']:.6f}元，总计{b['
27         total_saving_yuan']:.6f}元；它们依赖基准设置，不是唯一的光伏/套利因果分解。",
28         f"连续松弛费用{diag['lp_cost_yuan']:.9f}元，整数松弛间隙{diag['integer_gap']:.3e}；分数模式{diag['
29         fractional_modes']}个，同时充放电{diag['simultaneous_periods']}个。",
30         "仅说明本算例的松弛紧度，不能推出MILP与LP普遍等价，也不使用松弛解替换正式调度。",
31         "固定MILP最优模式后，用母线等式  $G-C+D-W=L-S+b$  的右端项导数定义 $\mu$ ；用状态等式  $E_t-E_{t-1}-\eta C+D/\eta=h$  的负导数定
32         义 $\lambda$ 。",
33         "h表示在第t时段末额外注入内部电量，初始储量和日末目标保持不变；这与直接抬高 $E_t$ 下限或日末目标不是同一扰动。",
34         " $\mu$ 、 $\lambda$ 仅适用于所选最优模式附近，模式切换、活跃边界和对偶退化都可能改变它们。",
35         "充电倾向条件为 $\mu < \eta\lambda$ ，待机区间为 $\eta\lambda \leq \mu \leq \lambda/\eta$ ，放电倾向条件为 $\mu > \lambda/\eta$ 。",
36         "等号表示边际无差异，允许出现非零充放电；固定模式不允许的方向不能仅凭阈值执行。",
37         f"本次主方案有{sum(r['mode_blocks_tendency'] for r in local)}个时段的严格阈值倾向受固定模式限制。",
38         f"对固定模式LP的144个母线需求和144个内部注入方向分别作 $\pm 0.01\text{kWh}$ 重求，其中{a['
39         rhs_nonsmooth_or_one_sided_count']}项为非光滑或单侧情况，",
40         "保留两侧差分与不可行状态，并检验影子价格位于可行方向的差分区间内；不伪称每个点都有唯一导数。",
41         f"主模型的未来电价须严格超过当前电价的{main['arbitrage_price_multiplier']:.9f}倍，忽略其他约束时才有正
42         的外网跨期收益。",
43         "等价阈值 $p_s > p_t / (\eta c_{nd})$ 仅是经济条件，不保证计划可执行；原本将被弃置的光伏不适用外网购电倍率。",
44     ]
45     texts.append(f"另对原MILP的全部288项RHS作双侧扰动，共576次整数重求，允许全部模式改变。")

```



```

40         f"其中{a['milp_rhs_mismatch_count']}项固定模式影子价格不能同时匹配原MILP两侧差分，"
41         f"{a['milp_rhs_nonsmooth_count']}项原MILP出现非光滑或单侧情况。"
42         "对应点只报告两侧有限差分，不将固定模式 $\mu$ 、 $\lambda$ 解释为原MILP的唯一边际价格，也不据此生成全局充停放指令。
    ")
43     for resource, label in (("storage_upper", "安全储能上限提高"), ("storage_lower", "安全储能下限降低"),
44                             ("charge_power", "充电功率提高"), ("discharge_power", "放电功率提高")):
45         rows = [r for r in a["bottlenecks"] if r["scenario"] == main["scenario"] and r["resource"]
46 == resource]
47         one = rows[0]
48         values = ", ".join(f"{r['value_per_unit_day']:.9f}" for r in rows)
49         texts.append(f"{label}基准活跃{one['active_count']}个时段：分别放宽1、10、100个单位后的单位日价值为{
50 values} ({one['unit']})。"
51 + ("该边界活跃且多步长单位价值稳定为正，存在稳定的数值节约。" if one["
52 numerical_bottleneck"] else
53 "所测步长单位价值在数值容差内为零。" if all(abs(r["value_per_unit_day"]) <=
54 ECONOMIC_VALUE_TOL for r in rows) else
55 "所测步长的单位价值未满足稳定正值判据，应按有限增量逐项解释。")
56 + "题目未给经济显著性门槛或设备成本，不据此认定投资意义上的经济瓶颈。")
57     texts.append("储能空间与功率价值的单位不同，不能直接按数值大小推导设备投资优先级；安全上限放宽是分析实验，原正式
58 结果仍采用10800kWh与5000kW。")
59     central = {r["efficiency_name"]: r for r in a["scenarios"] if r["load_delta"] == r["pv_delta"]
60 == 0}
61     lo, hi = central["eta085"], central["eta095"]
62     texts.append(f"效率由0.95降至0.85时，套利倍率由{hi['arbitrage_price_multiplier']:.6f}升至{lo['
63 arbitrage_price_multiplier']:.6f}，"
64 + f"充电总量由{hi['charge_kwh']:.6f}变为{lo['charge_kwh']:.6f}kWh，放电总量由{hi['
65 discharge_kwh']:.6f}变为{lo['discharge_kwh']:.6f}kWh。"
66 + "本算例效率降低后充电量增加、放电量下降，不支持二者同时下降的预期。")
67     for factor, label, other in (("load_delta", "负荷", "pv_delta"), ("pv_delta", "光伏", "load_delta
68 ")):
69         minus, plus = [next(r for r in a["scenarios"] if r["efficiency_name"] == "main" and r[factor
70 ] == v and r[other] == 0) for v in (-.05, .05)]
71         norm = next(r for r in a["normalized_sensitivity"] if r["efficiency_name"] == "main" and r["
72 factor"] == factor)
73         texts.append(f"{label}从-5%增加至+5%：费用{minus['cost_yuan']:.6f}→{plus['cost_yuan']:.6f}元，
74 购电{minus['grid_kwh']:.6f}→{plus['grid_kwh']:.6f}kWh，"
75 + f"弃光{minus['curtail_kwh']:.6f}→{plus['curtail_kwh']:.6f}kWh；SOC下界活跃{minus
76 ['E_min_active_count']→{plus['E_min_active_count']}个时段，"
77 + f"空间价值{minus['storage_upper_value']:.6f}→{plus['storage_upper_value']:.6f}，
78 放电功率价值{minus['discharge_power_value']:.6f}→{plus['discharge_power_value']:.6f}。"
79 + f"归一化成本敏感度为{norm['normalized_sensitivity']:.9f}。")
80     rt = central["roundtrip090"]
81     texts.append(f"改用对称往返0.90口径后，成本{rt['cost_yuan']:.6f}元，套利倍率{rt['
82 arbitrage_price_multiplier']:.9f}；"
83 + f"空间/充电/放电功率单位日价值分别为{rt['storage_upper_value']:.9f}、{rt['
84 charge_power_value']:.9f}、{rt['discharge_power_value']:.9f}。"
85 + "与主口径逐项对照，正式CSV仍只交付双单程0.9解。")
86     texts.append("本次主效率口径下，负荷增加未提高放电功率边际价值；光伏增加未导致弃光或储能空间单位价值上升。"
87 + "因此这些预期趋势不应写成实证结论，费用及活跃程度的实际变化以上述数值为准。")
88     order_ok = sum(r["charge_weighted_price"] < r["discharge_weighted_price"] for r in a["scenarios"
89 ])
90     pv_ok = sum(abs(r["pv_surplus_kwh"] - r["pv_surplus_absorbed_kwh"]) <= PHYS_TOL_KWH for r in a["
91 scenarios"])
92     texts.append(f"36个组合中，{order_ok}组充电加权电价低于放电加权电价，{pv_ok}组吸收全部光伏净富余；"
93 + "全部所选模式均通过非零动作与允许方向边际条件检查。"
94 + "情景CSV同时列出动作变化时段数、最大储能轨迹差异和活跃边界计数；最优调度可能不唯一，动作差异不能单
95 独解释为模型不稳健。")

```

```

77     return texts
78
79
80 def create_analysis_report(a, output, documents, figure_status):
81     texts = interpretations(a)
82     setting = ("沿用右端点、ZOH、母线侧电量、双单程0.9、不售电与允许弃光；Qmax按5000/6准确计算，按用户要求仅用CSV
    交付。"
83             "容差按物理、费用、整数、Gap、边际价值分别设置；36情景的储能上下限/充放功率均取步长1、10、100 kWh/
    kW；固定LP与原MILP的RHS步长均为±0.01 kWh。"
84             "这些是数值分析设置，模型原稿和设备基准参数未改写。")
85     links = {"baseline_comparison.csv": "三种基准", "incremental_benefits.csv": "增量节约", "
    lp_diagnosis.csv": "LP松弛诊断",
86             "marginal_values.csv": "全部5184行局部边际量", "marginal_rhs_validation.csv": "288项双侧扰
    动",
87             "milp_rhs_validation.csv": "288项原MILP双侧扰动",
88             "bottleneck_values.csv": "432项容量上下限/功率放宽", "sensitivity_scenarios.csv": "36组情景
    汇总",
89             "normalized_sensitivity.csv": "归一化敏感度", "active_constraints.csv": "全部活跃约束",
90             "arbitrage_thresholds.csv": "跨期价格阈值", " ../q1/raw/analysis_schedules.csv": "36组完
    整输入与输出"}
91     body = '<h1>q1 · 边际价值、经济瓶颈与稳健性</h1><p><a href="report.html">返回主结果与输入输出</a></p>'
92     body += f'<p class="note">假设与数值设置: {html.escape(setting)}</p>'
93     body += '<p>' + ' · '.join(f'<a href="{k}">{v}</a>' for k,v in links.items()) + '</p>'
94     body += '<h2>结果解释</h2>' + ' · '.join(f'<p>{html.escape(t)}</p>' for t in texts)
95     body += '<img src
    ="bottleneck_analysis.png" alt="容量、充电功率和放电功率三种步长的日价值">'
96     body += '<h2>基准费用与增量节约</h2>' + html_table(["基准", "费用 元", "购电 kWh", "充电 kWh", "放电
    kWh", "弃光 kWh"], [[r[k] for k in ("baseline", "cost_yuan", "grid_kwh", "charge_kwh", "
    discharge_kwh", "curtail_kwh")] for r in a["baseline_rows"]])
97     body += '<details open><summary>主方案144行边际价值与充停放解释</summary>' + html_table(
98         ["时间段", "μ 元/kWh", "λ 元/kWh", "ηλ", "λ/η", "阈值倾向", "实际动作", "固定u", "模式阻止倾向"],
99         [[r[k] for k in ("interval", "mu_yuan_per_kwh", "lambda_yuan_per_kwh", "charge_threshold", "
    discharge_threshold", "tendency", "actual_action", "fixed_u", "mode_blocks_tendency")] for r in a["
    main_marginals"]]) + '</details>'
100    body += '<p>charge=充电, discharge=放电, idle=待机, idle_band=待机区间, indifferent_boundary=边界无差
    异; 模式阻止倾向1表示该方向在固定LP中不允许。</p>'
101    body += '<details open><summary>全部36个情景汇总与经济瓶颈</summary>' + html_table(
102        ["情景", "费用 元", "购电 kWh", "充电 kWh", "放电 kWh", "弃光 kWh", "储能空间 元/(kWh·日)", "充电功
    率 元/(kW·日)", "放电功率 元/(kW·日)", "动作变化时段"],
103        [[r[k] for k in ("scenario", "cost_yuan", "grid_kwh", "charge_kwh", "discharge_kwh", "curtail_kwh
    ", "storage_upper_value", "charge_power_value", "discharge_power_value", "action_changed_periods")]
    for r in a["scenarios"]]) + '</details>'
104    body += '<details><summary>主方案十二项容量与功率步长对照</summary>' + html_table(
105        ["放宽边界", "增加量", "节约 元", "单位日价值", "单位", "活跃时段", "稳定正值且活跃(数值判据)"],
106        [[r[k] for k in ("resource", "increment", "saving_yuan", "value_per_unit_day", "unit", "
    active_count", "numerical_bottleneck")] for r in a["bottlenecks"]] if r["scenario"]=="
    main_ll100_pv100"]) + '</details>'
107    body += '<h2>核验与复现</h2><p>每个扰动都重新求解保留整数互斥的原MILP; 固定模式LP与原MILP目标一致, LP原始/对
    偶数值检查、分析CSV逐单元回读均通过。</p>'
108    body += '<p><a href=" ../q1/raw/analysis_validation.json">分析验收JSON</a> · <a href=" ../q1/
    raw/analysis_solutions.json">原始解与影子价格</a> · <a href=" ../q1/docs/1/deliverables/
    q1_implementation_acceptance.md">逐句验收与审查</a></p>'
109    body += '<p>影子价格符号依据 <a href="https://docs.scipy.org/doc/scipy/reference/optimize.linprog-
    highs-ds.html">SciPy 官方文档</a>, 并由本次双侧扰动校验。</p>'
110    base = (output/'report.html').read_text(encoding='utf-8')
111    style = base.split('<style>',1)[1].split('</style>',1)[0]
112    if figure_status['status'] != 'passed':

```

```

113     body = re.sub(r'<img[^>]+>', '', body)
114     body += '<p class="note">绘图状态: ' + html.escape(figure_status['message']) + '</p>'
115     page = '<!doctype html><html lang="zh-CN"><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-scale=1"><title>Q1 边际价值强化版</title><style>' + style + '</style><main>' + body + '</main></html>'
116     (output/'marginal_report.html').write_text(page,encoding='utf-8')
117     base = base.replace('<h2>输入: 来源、时间与单位</h2>', '<p class="note"><a href="marginal_report.html">查看新版完整分析: 边际价值、经济瓶颈、36组敏感性情景及全部CSV</a></p><h2>输入: 来源、时间与单位</h2>')
118     (output/'report.html').write_text(base,encoding='utf-8')
119     document = documents/'q1_marginal_analysis.md'
120     content = '# Q1 边际价值强化版分析结果\n\n假设与数值设置: ' + setting + '\n\n'
121     content += '\n\n'.join(texts) + '\n\n'
122     content += '\n\n'.join(f'- [{v} CSV](../../outputs/processed/q1/{k})' for k,v in links.items())
123     content += '\n\n[完整可视分析](../../outputs/processed/q1/marginal_report.html)\n\n'
124     content += '影子价格的右端项导数与符号参见[SciPy官方文档](https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs-ds.html), 并已作实际扰动复核.\n'
125     document.write_text(content,encoding='utf-8')
126     result_doc = documents/'q1_results.md'
127     with result_doc.open('a',encoding='utf-8') as f:
128         f.write('\n新版第11.3 - 11.8节完整分析见[边际价值与经济瓶颈](q1_marginal_analysis.md)。\n')

```

src/q1/device_sensitivity.py

```

1  """论文设备敏感性: 每个参数点重求原MILP, 保存CSV与压缩原始解。"""
2  import hashlib
3  import json
4  from pathlib import Path
5  import shutil
6  import tempfile
7
8  import numpy as np
9
10 from src.q1.model import ROOT, COST_TOL_YUAN, check_solution, solve, write_csv, write_json
11
12 # 明示的数值实验范围; 仅改变指定边界, 不推断实际扩容或投资回报。
13 ENERGY_GRID = list(range(6000, 30001, 2400))
14 POWER_GRID = list(range(1000, 10001, 1000))
15 POWER_SWEEP = list(range(0, 10001, 1000))
16
17
18 def digest(path):
19     return hashlib.sha256(path.read_bytes()).hexdigest()
20
21
22 def prepare(data, generation):
23     sources = {str(p.relative_to(ROOT)): digest(p) for p in
24               [Path(__file__), ROOT / 'src/q1/model.py', generation / 'inputs/processed/timeseries.csv']}
25     key = hashlib.sha256(json.dumps(sources, sort_keys=True).encode('utf-8')).hexdigest()[:12]
26     parent = ROOT / 'outputs/q1/figure_analysis'
27     parent.mkdir(parents=True, exist_ok=True)
28     target = parent / key
29     if target.exists():
30         manifest = json.loads((target / 'manifest.json').read_text(encoding='utf-8'))
31         if not manifest['passed'] or any(digest(target / p) != h for p, h in manifest['output_sha256'].items()):

```

```

32         raise ValueError('设备敏感性缓存校验失败')
33     verify(data, target)
34     return target
35 staging = Path(tempfile.mkdtemp(prefix='.pending-', dir=parent))
36 try:
37     solutions, metadata, sweeps, grid = {}, [], [], []
38
39     def point(emax, charge, discharge):
40         name = f'E{emax}_C{charge}_D{discharge}'
41         if name not in solutions:
42             solution = solve(data, .9, e_max=emax, charge_kw=charge, discharge_kw=discharge)
43             checked = check_solution(data, solution)
44             solutions[name] = solution
45             metadata.append({'point': name, 'e_max_kwh': emax, 'charge_kw': charge,
46                             'discharge_kw': discharge, 'cost_yuan': solution['solver']['
objective_yuan'],
47                             'dual_bound_yuan': solution['solver']['dual_bound_yuan'],
48                             'mip_gap': solution['solver']['mip_gap'], 'status': solution['
solver']['status'],
49                             'max_physical_residual_kwh': max(v for k, v in checked['metrics'].
items() if k.endswith('_kwh'))})
50             if len(solutions) % 20 == 0:
51                 print(f'设备敏感性: 已求解并验收 {len(solutions)} 个不同参数点', flush=True)
52             return {'point': name, 'cost_yuan': solutions[name]['solver']['objective_yuan']}
53
54     for power in POWER_GRID:
55         for energy in ENERGY_GRID:
56             grid.append({'e_max_kwh': energy, 'common_power_kw': power, **point(energy, power,
power)})
57         for resource, values in [(('e_max', sorted([*ENERGY_GRID, 10900])),
58                                   ('charge_kw', sorted([*POWER_SWEEP, 5100])),
59                                   ('discharge_kw', sorted([*POWER_SWEEP, 5100]))):
60             for value in values:
61                 settings = {'e_max': 10800, 'charge_kw': 5000, 'discharge_kw': 5000, resource: value
62 }
63                 sweeps.append({'resource': resource, 'limit': value,
64                                **point(settings['e_max'], settings['charge_kw'], settings['
discharge_kw'])})
65             base = solutions['E10800_C5000_D5000']['solver']['objective_yuan']
66             original = json.loads((generation / 'raw/solution_main.json').read_text(encoding='utf-8'))
67             if abs(base - original['solver']['objective_yuan']) > COST_TOL_YUAN:
68                 raise ValueError('敏感性基准与已验收主目标不一致')
69             for row in [*grid, *sweeps]:
70                 row['saving_from_base_yuan'] = base - row['cost_yuan']
71             write_csv(staging / 'joint_grid.csv', grid)
72             write_csv(staging / 'one_dimensional.csv', sweeps)
73             write_csv(staging / 'solver_checks.csv', metadata)
74             np.savez_compressed(staging / 'solutions.npz', **{
75                 key: np.asarray([s[k] for k in ('G', 'C', 'D', 'E', 'W', 'u')]) for key, s in solutions.
items()})
76             verification = verify(data, staging)
77             write_json(staging / 'manifest.json', {
78                 'passed': True, 'source_sha256': sources, 'generation': generation.name,
79                 'assumptions': ['双单程效率0.9; Emin=1200; E0=E144=6000; 其余模型与输入不变',
80                                'Emax是储电上限; 超过原额定容量的点仅为数学约束放宽实验, 不能直接作为设备设计',
81                                '二维纵轴同时改变充放电功率; 一维功率曲线只改变一种功率',
82                                '等值线在实际求解网格间作线性绘图插值, 插值点不声称已求解',

```

```

82         '网络范围为绘图实验选择，不代表题目给定设备候选集：没有投资成本或经济显著性门槛'],
83         'energy_grid_kwh': ENERGY_GRID, 'common_power_grid_kw': POWER_GRID,
84         **verification, 'output_sha256': {p.name: digest(p) for p in staging.iterdir() if p.
is_file()})
85         staging.rename(target)
86     except BaseException:
87         shutil.rmtree(staging)
88         raise
89     return target
90
91
92 def verify(data, folder):
93     """从磁盘原始数组独立复核每个解，以及可行域嵌套的费用单调性。"""
94     import csv
95     def rows(name):
96         with (folder / name).open(encoding='utf-8', newline='') as stream:
97             return list(csv.DictReader(stream))
98     checks, grid, curves = rows('solver_checks.csv'), rows('joint_grid.csv'), rows('one_dimensional.
.csv')
99     with np.load(folder / 'solutions.npz', allow_pickle=False) as saved:
100         if set(saved.files) != {r['point'] for r in checks}:
101             raise ValueError('参数点与原始解不一一对应')
102         for row in checks:
103             solution = {k: a.tolist() for k, a in zip(('G', 'C', 'D', 'E', 'W', 'u'), saved[row['
point']])}
104             solution.update(E0=6000, eta_c=.9, eta_d=.9,
105                             settings={ 'e_min': 1200, 'e_max': float(row['e_max_kwh']),
106                                         'charge_kw': float(row['charge_kw']), 'discharge_kw': float(
row['discharge_kw'])},
107                             solver={ 'status': int(row['status']), 'objective_yuan': float(row['
cost_yuan']),
108                                     'dual_bound_yuan': float(row['dual_bound_yuan']), 'mip_gap':
float(row['mip_gap'])})
109             check_solution(data, solution)
110         lookup = {r['point']: float(r['cost_yuan']) for r in checks}
111         for row in [*grid, *curves]:
112             if abs(float(row['cost_yuan']) - lookup[row['point']]) > COST_TOL_YUAN:
113                 raise ValueError('绘图CSV费用与原始解不一致')
114         matrix = np.array([float(r['cost_yuan']) for r in grid]).reshape(len(POWER_GRID), len(
ENERGY_GRID))
115         differences = [np.diff(matrix, axis=0).max(), np.diff(matrix, axis=1).max()]
116         for resource in ('e_max', 'charge_kw', 'discharge_kw'):
117             differences.append(np.diff([float(r['cost_yuan']) for r in curves if r['resource'] ==
resource]).max())
118         if max(differences) > COST_TOL_YUAN:
119             raise ValueError('可行域放宽后费用反而升高')
120         return {'unique_solves': len(checks), 'joint_points': len(grid), 'one_dimensional_points': len(
curves),
121                 'all_saved_solutions_rechecked': True, 'monotonicity_passed': True,
122                 'max_mip_gap': max(float(r['mip_gap']) for r in checks)}

```

src/q1/export.py

```

1 """用户指定 CSV 的写入一关闭一读取检查。"""
2
3 import csv

```

```

4 import tempfile
5 from pathlib import Path
6
7 import numpy as np
8
9 from model import PHYS_TOL_KWH, COST_TOL_YUAN, BINARY_TOL, DIMENSIONLESS_TOL, TOLERANCES, T, arrays,
    block_rows, check_solution, schedule_rows, summary, write_csv
10
11
12 def read_csv(path):
13     with path.open(encoding="utf-8", newline="") as stream:
14         return list(csv.DictReader(stream))
15
16
17 def check_csv_output(directory, data, solution):
18     rows = read_csv(directory / "result1.csv")
19     blocks = read_csv(directory / "table2_storage.csv")
20     daily = read_csv(directory / "daily_summary.csv")
21     table1 = read_csv(directory / "table1_purchase.csv")
22     if len(rows) != T or [r["interval"] for r in rows] != [r["interval"] for r in data]:
23         raise ValueError("CSV必须具有144个正确的右端点映射时间段")
24     expected_rows = schedule_rows(data, solution)
25     if list(rows[0]) != list(expected_rows[0]):
26         raise ValueError("CSV字段缺失或顺序不符")
27     for actual, expected in zip(rows, expected_rows):
28         for key in data[0]:
29             if actual[key] != str(expected[key]):
30                 raise ValueError(f"CSV输入来源字段不一致: {key}")
31     reread = {**solution}
32     try:
33         for name in ("G", "C", "D", "E", "W"):
34             reread[name] = [float(row[f"{name}_kwh"]) for row in rows]
35             reread["u"] = [float(row["u"]) for row in rows]
36             reread["E0"] = float(rows[0]["E_start_kwh"])
37             g = np.array(reread["G"])
38             if len(daily) != 1 or list(daily[0]) != list(summary(data, solution)):
39                 raise ValueError("日汇总表字段不符")
40             day = {k: float(v) if v else None for k, v in daily[0].items()}
41             expected_day = summary(data, solution)
42             if any(day[k] is None or not np.isfinite(day[k]) for k, v in expected_day.items() if v is
not None):
43                 raise ValueError("日汇总含缺失或非有限结果")
44             if any(not np.isfinite(float(r[k])) for r in rows
45                 for k in ("E_start_kwh", "soc_fraction", "cost_yuan")):
46                 raise ValueError("逐时派生结果含非有限值")
47             values = np.array([[float(b["charge_kwh"]), float(b["discharge_kwh"])] for b in blocks])
48             if not all(np.isfinite(float(row["grid_kwh"])) for row in table1):
49                 raise ValueError("表1含非有限值")
50     except (KeyError, TypeError, ValueError) as exc:
51         raise ValueError("CSV含缺失或非数值结果") from exc
52     expected_blocks = block_rows(solution)
53     if [r["interval"] for r in blocks] != [r["interval"] for r in expected_blocks]:
54         raise ValueError("4小时汇总区间不符")
55     if [r["interval"] for r in table1] != [data[h * 6]["interval"] for h in (10, 12, 14, 16, 18, 20)
56         ]:
57         raise ValueError("表1指定区间不符")
58     physical = check_solution(data, reread)

```



```

58 p, _, _ = arrays(data)
59 metrics = {
60     "12.1_count_error": abs(len(rows) - T),
61     "12.2_all_variables_max_abs_error": max(float(np.max(np.abs(np.array(reread[k]) - solution[k
62 ]))) for k in ("G", "C", "D", "E", "W", "u")),
63     "12.3_grid_day_error_kwh": float(abs(g.sum() - day["grid_kwh"])),
64     "12.4_cost_error_yuan": float(abs(p @ g - day["cost_yuan"])),
65     "12.5_block_max_abs_error_kwh": float(np.max(np.abs(values - [[b["charge_kwh"], b["
66 discharge_kwh"]] for b in expected_blocks))),
67     "12.6_endpoint_max_abs_error_kwh": max(abs(day["initial_kwh"] - solution["E0"]), abs(day["
68 terminal_kwh"] - solution["E"][-1])),
69     "daily_summary_max_abs_error": max(abs(day[k] - v) for k, v in expected_day.items() if v is
70 not None),
71     "table1_max_abs_error_kwh": max(abs(float(row["grid_kwh"]) - solution["G"][h * 6]) for row,
72 h in zip(table1, (10, 12, 14, 16, 18, 20))),
73     "start_state_max_abs_error_kwh": max(abs(float(r["E_start_kwh"]) - e["E_start_kwh"]) for r,
74 e in zip(rows, expected_rows)),
75     "soc_fraction_max_abs_error": max(abs(float(r["soc_fraction"]) - e["soc_fraction"]) for r, e
76 in zip(rows, expected_rows)),
77     "period_cost_max_abs_error_yuan": max(abs(float(r["cost_yuan"]) - e["cost_yuan"]) for r, e
78 in zip(rows, expected_rows)),
79 }
80 tolerances = {k: COST_TOL_YUAN if k.endswith("_yuan") else
81 DIMENSIONLESS_TOL if k == "soc_fraction_max_abs_error" else PHYS_TOL_KWH for k in
82 metrics}
83 # 分开检查不同单位的字段，不让汇总最大值掩盖更严格的二元/比率容差。
84 for actual, expected in zip(rows, expected_rows):
85     for key, tolerance in (("u", BINARY_TOL), ("soc_fraction", DIMENSIONLESS_TOL), ("cost_yuan",
86 COST_TOL_YUAN)):
87         if abs(float(actual[key]) - expected[key]) > tolerance:
88             raise ValueError(f"CSV字段超出量纲容差: {key}")
89 for key, value in expected_day.items():
90     tol = (COST_TOL_YUAN if key == "cost_yuan" else DIMENSIONLESS_TOL if key in
91 ("eta_c", "eta_d", "roundtrip_efficiency") else PHYS_TOL_KWH)
92     if value is not None and abs(day[key] - value) > tol:
93         raise ValueError(f"日汇总字段超出量纲容差: {key}")
94 if any(not np.isfinite(v) or v > tolerances[k] for k, v in metrics.items()):
95     raise ValueError(f"CSV回读一致性检查失败: {metrics}")
96 return {"passed": True, "tolerances": TOLERANCES, "metric_tolerances": tolerances, "metrics":
97 metrics, "reread_physical": physical,
98     "reread_grid_kwh": float(g.sum()), "reread_cost_yuan": float(p @ g),
99     "reread_initial_kwh": reread["E0"], "reread_terminal_kwh": reread["E"][-1]}
100
101 def export_csv_output(directory, data, solution):
102     # 暂存CSV通过关闭后的回读核验，才替换本次正式结果。
103     with tempfile.TemporaryDirectory(prefix=".ql-csv-", dir=directory.parent) as temporary:
104         staging = Path(temporary)
105         write_csv(staging / "result1.csv", schedule_rows(data, solution))
106         write_csv(staging / "daily_summary.csv", [summary(data, solution)])
107         write_csv(staging / "table2_storage.csv", block_rows(solution))
108         write_csv(staging / "table1_purchase.csv", [
109             {"interval": data[h * 6]["interval"], "grid_kwh": solution["G"][h * 6]}
110             for h in (10, 12, 14, 16, 18, 20)])
111         check_csv_output(staging, data, solution)
112         for path in staging.iterdir():
113             path.replace(directory / path.name)

```

```
104     return check_csv_output(directory, data, solution)
```

src/q1/export_submission.mjs

```
1 // Run from the project root with the bundled @oai/artifact-tool dependency.
2 import fs from 'node:fs/promises';
3 import path from 'node:path';
4 import { createHash } from 'node:crypto';
5 import { FileBlob, SpreadsheetFile, Workbook } from '@oai/artifact-tool';
6
7 const root = process.cwd();
8 const qa = path.join(root, 'docs/final_submission');
9 const template = path.join(root, 'docs/CUMCM2026Problems/C题/附件/附件5/result1.xlsx');
10 const source = path.join(root, 'outputs/.q1-runs/run-ie6fkaf/processed');
11 const workbook = await SpreadsheetFile.importXlsx(await FileBlob.load(template));
12 await fs.mkdir(qa, { recursive: true });
13
14 async function render(name, sheetName, range) {
15     const blob = await workbook.render({ sheetName, range, scale: 2, format: 'png' });
16     await fs.writeFile(path.join(qa, `${name}.png`), new Uint8Array(await blob.arrayBuffer()));
17 }
18
19 if (process.argv.includes('--preview-only')) {
20     console.log((await workbook.inspect({ kind: 'sheet', include: 'id,name' })).ndjson);
21     await render('template_plan', '计划购电量', 'A1:B14');
22     await render('template_storage', '充放电量', 'A1:E7');
23 } else {
24     const csv = await Workbook.fromCSV(await fs.readFile(path.join(source, 'result1.csv'), 'utf8'), {
25         sheetName: 'Source' });
26     const [headers, ...records] = csv.worksheets.getItem('Source').getUsedRange().values;
27     const rows = records.filter(row => row[0] !== null && row[0] !== '');
28     if (rows.length !== 144) throw new Error(`Expected 144 intervals, got ${rows.length}`);
29     const index = name => {
30         const i = headers.indexOf(name);
31         if (i < 0) throw new Error(`Missing source field ${name}`);
32         return i;
33     };
34     const num = (row, name) => {
35         const value = Number(row[index(name)]);
36         if (!Number.isFinite(value)) throw new Error(`Non-finite ${name}`);
37         return value;
38     };
39     const stamp = minutes => `${String(Math.floor(minutes / 60)).padStart(2, '0')}:${String(minutes % 60).padStart(2, '0')}`;
40     const planValues = rows.map((row, i) => {
41         const interval = `${stamp(i * 10)}-${stamp((i + 1) * 10)}`;
42         if (row[index('interval')] !== interval || num(row, 't') !== i + 1) throw new Error(`Interval mismatch at ${i + 1}`);
43         return [interval, num(row, 'G_kwh')];
44     });
45     const plan = workbook.worksheets.getItem('计划购电量');
46     const originalIntervals = plan.getRange('A2:A145').values.flat();
47     plan.getRange('A2:B145').values = planValues;
48     plan.getRange('A1:A145').format.columnWidth = 20;
49     plan.getRange('B1:B145').format.columnWidth = 18;
50     plan.getRange('B2:B145').setNumberFormat('0.0000');
```



```

50
51 const storage = workbook.worksheets.getItem('充放电量');
52 const blocks = Array.from({ length: 6 }, (_, i) => rows.slice(i * 24, (i + 1) * 24));
53 storage.getRange('B2:C7').values = blocks.map(block => ['C_kwh', 'D_kwh'].map(key => block.reduce
    ((sum, row) => sum + num(row, key), 0)));
54 storage.getRange('E2:E3').values = [[num(rows[0], 'E_start_kwh')], [num(rows[143], 'E_kwh')]];
55 storage.getRange('A1:A7').format.columnWidth = 19;
56 storage.getRange('B1:C7').format.columnWidth = 17;
57 storage.getRange('E1:E7').format.columnWidth = 17;
58 storage.getRange('B2:C7').setNumberFormat('0.0000');
59 storage.getRange('E2:E3').setNumberFormat('0.0000');
60
61 const inspection = await workbook.inspect({ kind: 'table', range: '充放电量!A1:E7', include: '
    values,formulas', tableMaxRows: 7, tableMaxCols: 5 });
62 const errors = await workbook.inspect({ kind: 'match', searchTerm: '#REF!|#DIV/0!|#VALUE!|#NAME
    \?|#N/A|#NUM!|#NULL!|#SPILL!|#CALC!', options: { useRegex: true, maxResults: 20 } });
63 await fs.writeFile(path.join(qa, 'artifact_inspection.ndjson'), `${inspection.ndjson}\n${errors.
    ndjson}\n`, 'utf8');
64 await render('result1_plan_start', '计划购电量', 'A1:B15');
65 await render('result1_plan_middle', '计划购电量', 'A57:B75');
66 await render('result1_plan_end', '计划购电量', 'A132:B145');
67 await render('result1_storage', '充放电量', 'A1:E7');
68 const output = path.join(root, 'outputs/final/result1.xlsx');
69 await fs.mkdir(path.dirname(output), { recursive: true });
70 await (await SpreadsheetFile.exportXlsx(workbook)).save(output);
71 const sidecar = `${output}.inspect.ndjson`;
72 if (await fs.stat(sidecar).catch(() => null)) await fs.rename(sidecar, path.join(qa, 'result1.xlsx
    .inspect.ndjson'));
73 const hashes = {};
74 for (const file of [template, ...['result1.csv', 'table2_storage.csv', 'table1_purchase.csv', '
    daily_summary.csv'].map(name => path.join(source, name))]) {
75     hashes[path.relative(root, file)] = createHash('sha256').update(await fs.readFile(file)).digest
        ('hex');
76 }
77 await fs.writeFile(path.join(qa, 'result1_export.json'), JSON.stringify({ template: path.relative(
    root, template), source: path.relative(root, source), output: path.relative(root, output),
    originalIntervals, correctedIntervals: planValues.map(row => row[0]), reason: 'Template labels
    are shifted by ten minutes; align with the verified 144 source intervals from 00:00 to 24:00, as
    in the other four official outputs.', source_sha256: hashes }, null, 2) + '\n', 'utf8'));
78 console.log(`Exported ${output}`);
79 console.log(inspection.ndjson);
80 console.log(errors.ndjson);
81 }

```

src/q1/model.py

```

1 """Q1 最终稿第 2-11 节：输入、六类变量 MILP 和独立物理复核。"""
2
3 import csv
4 import datetime as dt
5 import json
6 import os
7 import warnings
8 from pathlib import Path
9
10 import numpy as np

```

```

11 from openpyxl import load_workbook
12 from scipy.optimize import Bounds, LinearConstraint, linprog, milp
13 from scipy.sparse import csc_matrix
14
15 ROOT = Path(__file__).resolve().parents[2]
16 T = 144
17 DT = 1 / 6 # 最终稿第 2、3 节: 10 min, 以 h 为单位。
18 Q_MAX = 5000 * DT # 用户确认精确公式: 833.3333 仅作参考。
19 E_MIN, E_MAX, E_INITIAL = 1200.0, 10800.0, 6000.0
20 # CR-07: 每类验收独立指定量纲; 相同数值不代表可混用。
21 PHYS_TOL_KWH = 1e-6
22 COST_TOL_YUAN = 1e-6
23 BINARY_TOL = 1e-8
24 MIP_GAP_TOL = 1e-6
25 ECONOMIC_VALUE_TOL = 1e-6
26 DIMENSIONLESS_TOL = 1e-8
27 TOLERANCES = {"physical_kwh": PHYS_TOL_KWH, "cost_yuan": COST_TOL_YUAN,
28               "binary": BINARY_TOL, "mip_gap": MIP_GAP_TOL,
29               "economic_value_per_unit": ECONOMIC_VALUE_TOL,
30               "dimensionless": DIMENSIONLESS_TOL}
31 # 测试进程指向本次staging; 正常读取者固定一次generation, 避免跨代读取。
32 ARTIFACT_ROOT = Path(os.environ.get("Q1_ARTIFACT_ROOT", ROOT / "outputs/q1_current")).resolve()
33 SOLVER_OPTIONS = {"mip_rel_gap": 0.0, "mip_feasibility_tolerance": 1e-9,
34                  "primal_feasibility_tolerance": 1e-9, "dual_feasibility_tolerance": 1e-9}
35 SCENARIOS = {"main": 0.90, "eta085": 0.85, "eta095": 0.95,
36              "roundtrip090": float(np.sqrt(0.90))}
37
38
39 def write_json(path, value):
40     path.write_text(json.dumps(value, ensure_ascii=False, indent=2,
41                                allow_nan=False) + "\n", encoding="utf-8")
42
43
44 def write_csv(path, rows):
45     with path.open("w", encoding="utf-8", newline="") as stream:
46         writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
47         writer.writeheader()
48         writer.writerows(rows)
49
50
51 def clock_label(minutes):
52     return f"{minutes // 60:02d}:{minutes % 60:02d}"
53
54
55 def endpoint_minutes(value):
56     """新版2.1: 将Excel时间、日分数、时长及文本统一为整分钟。"""
57     if isinstance(value, dt.timedelta):
58         minutes = value.total_seconds() / 60
59     elif isinstance(value, (int, float)) and not isinstance(value, bool):
60         minutes = float(value) * 1440
61     else:
62         minutes = None
63     if minutes is not None:
64         if np.isfinite(minutes) and 0 < minutes <= 1440 and abs(minutes - round(minutes)) < 1e-7:
65             return round(minutes)
66         raise ValueError(f"无效Excel时间数值: {value!r}")
67     if isinstance(value, dt.time):

```

```

68         if value.second or value.microsecond:
69             raise ValueError(f"非整分钟时间: {value}")
70         return 60 * value.hour + value.minute
71     if isinstance(value, str):
72         value = value.strip()
73         if value in ("0:00+1", "00:00+1", "24:00"):
74             return 1440
75         parts = value.split(":")
76         if len(parts) == 2 and all(part.isdigit() for part in parts):
77             hour, minute = map(int, parts)
78             if 0 <= hour < 24 and 0 <= minute < 60:
79                 return hour * 60 + minute
80         raise ValueError(f"无效时间标签: {value!r}")
81
82
83 def read_inputs(path):
84     wb = load_workbook(path, read_only=True, data_only=True)
85     try:
86         rows = list(wb.active.values)
87     finally:
88         wb.close()
89     expected_header = ("时间", "电价", "小区负载", "光伏发电预测功率")
90     if tuple(rows[0]) != expected_header or len(rows) != T + 1:
91         raise ValueError("附件1必须具有规定四列及144条数据, 不允许静默截断或补值")
92     result = []
93     for t, row in enumerate(rows[1:], 1):
94         endpoint = endpoint_minutes(row[0])
95         if endpoint != t * 10:
96             raise ValueError(f"第{t}时段右端点不符: {row[0]}, 应为{clock_label(t * 10)}")
97         if any(isinstance(x, bool) or not isinstance(x, (int, float)) for x in row[1:]):
98             raise ValueError(f"第{t}时段含缺失值或非数值")
99         p, pl, ppv = map(float, row[1:])
100         if not np.all(np.isfinite([p, pl, ppv])) or min(p, pl, ppv) < 0:
101             raise ValueError(f"第{t}时段含非有限值、负电价或负功率")
102         result.append({"t": t, "source_excel_row": t + 1,
103                       "source_endpoint": str(row[0]),
104                       "interval": f"{clock_label(endpoint - 10)}-{clock_label(endpoint)}",
105                       "start_minute": endpoint - 10, "end_minute": endpoint,
106                       "price_yuan_per_kwh": p, "load_kw": pl, "pv_kw": ppv,
107                       "load_kwh": pl * DT, "pv_kwh": ppv * DT})
108     return result
109
110
111 def arrays(data):
112     return tuple(np.array([r[key] for r in data], dtype=float)
113                 for key in ("price_yuan_per_kwh", "load_kwh", "pv_kwh"))
114
115
116 def build_problem(data, eta_c, eta_d=None, *, e_min=E_MIN, e_max=E_MAX, charge_kw=5000.0,
117                  discharge_kw=5000.0, baseline="full", fixed_mode=None):
118     """共享第10节原式: 11.3 - 11.8只改变明确指定的边界或整数性。"""
119     eta_d = eta_c if eta_d is None else eta_d
120     p, load, pv = arrays(data)
121     if not np.all(np.isfinite([p, load, pv])) or min(p.min(), load.min(), pv.min()) < 0:
122         raise ValueError("电价、负荷、光伏必须有限且非负")
123     if not (0 < eta_c <= 1 and 0 < eta_d <= 1) or not np.all(np.isfinite([e_min, e_max, charge_kw,
124                                     discharge_kw])):

```

```

124     raise ValueError("效率或设备边界无效")
125 if not 0 <= e_min <= E_INITIAL <= e_max or min(charge_kw, discharge_kw) < 0:
126     raise ValueError("设备边界与初始状态不兼容")
127 if baseline not in ("full", "no_ess", "pv_only"):
128     raise ValueError("未知基准")
129 qc, qd = charge_kw * DT, discharge_kw * DT
130 n = len(data)
131 names = ("G", "C", "D", "E", "W", "u")
132 index = {name: np.arange(k * n, (k + 1) * n) for k, name in enumerate(names)}
133 size = len(names) * n
134 objective = np.zeros(size)
135 objective[index["G"]] = p
136 lower, upper = np.zeros(size), np.full(size, np.inf)
137 lower[index["E"]], upper[index["E"]] = e_min, e_max
138 lower[index["E"][-1]] = upper[index["E"][-1]] = E_INITIAL
139 upper[index["W"]] = pv
140 upper[index["u"]] = 1
141 if fixed_mode is not None:
142     mode = np.asarray(fixed_mode)
143     if mode.shape != (n,) or not np.all(np.isin(mode, [0, 1])):
144         raise ValueError("固定模式必须是逐时0/1整数")
145     lower[index["u"]] = upper[index["u"]] = mode
146 if baseline == "no_ess":
147     upper[index["C"]] = upper[index["D"]] = 0
148 elif baseline == "pv_only":
149     upper[index["C"]] = np.maximum(pv - load, 0)
150     upper[index["D"]] = np.maximum(load - pv, 0)
151 # CR-11: 富余光伏时禁止购电, 两层基准共同采用, 保持可行域嵌套。
152 if baseline in ("no_ess", "pv_only"):
153     upper[index["G"]][pv > load] = 0
154 integer = np.zeros(size, dtype=int)
155 integer[index["u"]] = 1
156 a = np.zeros((4 * n, size))
157 lb, ub = np.full(4 * n, -np.inf), np.zeros(4 * n)
158 for t in range(n):
159     # 第7.2节:  $G - C + D - W = L - S$ 。
160     for name, coefficient in (("G", 1), ("C", -1), ("D", 1), ("W", -1)):
161         a[t, index[name][t]] = coefficient
162     lb[t] = ub[t] = load[t] - pv[t]
163     # 第8节:  $E_t - E_{(t-1)} - \eta_c C + D/\eta_d = 0$ 。
164     a[n + t, index["E"][t]] = 1
165     a[n + t, index["C"][t]] = -eta_c
166     a[n + t, index["D"][t]] = 1 / eta_d
167     if t:
168         a[n + t, index["E"][t - 1]] = -1
169     lb[n + t] = ub[n + t] = E_INITIAL if t == 0 else 0
170     # 第9节:  $C \leq Q$ ;  $D \leq Q(1-u)$ 。
171     a[2 * n + t, index["C"][t]] = 1
172     a[2 * n + t, index["u"][t]] = -qc
173     a[3 * n + t, index["D"][t]] = 1
174     a[3 * n + t, index["u"][t]] = qd
175     ub[3 * n + t] = qd
176 return {"objective": objective, "index": index, "integer": integer,
177        "lower": lower, "upper": upper, "a": csc_matrix(a), "lb": lb, "ub": ub,
178        "eta_c": eta_c, "eta_d": eta_d,
179        "settings": {"e_min": e_min, "e_max": e_max, "charge_kw": charge_kw,
180                    "discharge_kw": discharge_kw, "baseline": baseline}}

```

```

181
182
183 def solve_milp(problem):
184     """对给定矩阵求原整数问题，保留不可行状态供双侧扰动诊断。"""
185     # 三个容差是HiGHS原生选项；SciPy按其提示原样传递。只屏蔽这一转发通知。
186     with warnings.catch_warnings():
187         warnings.filterwarnings("ignore", category=RuntimeWarning,
188                                 message=r"Unrecognized options detected: .*These will be passed to
189                                     HiGHS verbatim\.")
190         result = milp(problem["objective"], integrality=problem["integer"],
191                       bounds=Bounds(problem["lower"], problem["upper"]),
192                       constraints=LinearConstraint(problem["a"], problem["lb"], problem["ub"]),
193                       options=SOLVER_OPTIONS)
194     return result
195
196 def solve(data, eta_c, eta_d=None, **settings):
197     """MILP全局求解；分别参数化两种效率，默认相等兼容既有调用。"""
198     problem = build_problem(data, eta_c, eta_d, **settings)
199     result = solve_milp(problem)
200     if not result.success or result.status != 0:
201         raise RuntimeError(f"MILP未获得最优解，拒绝导出: {result.message}")
202     solution = {name: result.x[index].tolist() for name, index in problem["index"].items()}
203     solution.update({"E0": E_INITIAL, "eta_c": problem["eta_c"], "eta_d": problem["eta_d"],
204                    "settings": problem["settings"],
205                    "solver": {"status": int(result.status), "message": result.message,
206                             "objective_yuan": float(result.fun),
207                             "dual_bound_yuan": float(result.mip_dual_bound),
208                             "mip_gap": float(result.mip_gap),
209                             "node_count": int(result.mip_node_count),
210                             "variables": 6 * len(data), "binary_variables": len(data),
211                             "linear_constraint_rows": 4 * len(data),
212                             "options": SOLVER_OPTIONS}})
213     return solution
214
215
216 def solve_lp(problem, rhs_shift=None):
217     """保留所有原约束的连续LP；rhs_shift仅用于固定模式局部数值校验。"""
218     n = len(problem["index"]["G"])
219     rhs = problem["ub"][:2 * n].copy()
220     if rhs_shift is not None:
221         row, amount = rhs_shift
222         rhs[row] += amount
223     return linprog(problem["objective"], A_eq=problem["a"][:2 * n], b_eq=rhs,
224                   A_ub=problem["a"][2 * n:], b_ub=problem["ub"][2 * n:],
225                   bounds=list(zip(problem["lower"], problem["upper"])), method="highs-ds",
226                   options={"primal_feasibility_tolerance": 1e-9,
227                           "dual_feasibility_tolerance": 1e-9})
228
229
230 def check_solution(data, solution):
231     """独立从物理等式复算，不依赖求解器的约束矩阵；失败即中止输出。"""
232     p, load, pv = arrays(data)
233     g, c, d, e, w, u = (np.asarray(solution[k], dtype=float)
234                          for k in ("G", "C", "D", "E", "W", "u"))
235     if any(a.shape != p.shape or not np.all(np.isfinite(a)) for a in (g, c, d, e, w, u)):
236         raise ValueError("变量长度不符或含非有限值")

```

```

237 e0, ec, ed = solution["E0"], solution["eta_c"], solution["eta_d"]
238 settings = solution.get("settings", {})
239 emin, emax = settings.get("e_min", E_MIN), settings.get("e_max", E_MAX)
240 qc, qd = settings.get("charge_kw", 5000) * DT, settings.get("discharge_kw", 5000) * DT
241 prior = np.r_[e0, e[:-1]]
242 metrics = {
243     "11.1_balance_max_abs_kwh": float(np.max(np.abs(g + pv - w + d - load - c))),
244     "11.2_soc_max_abs_kwh": float(np.max(np.abs(e - prior - ec * c + d / ed))),
245     "11.3_soc_bound_violation_kwh": float(max(0, np.max(emin - e), np.max(e - emax))),
246     "11.4_power_bound_violation_kwh": float(max(0, np.max(c - qc), np.max(d - qd))),
247     "11.5_simultaneous_kwh": float(max(0, np.max(np.minimum(c, d)))),
248     "11.6_nonnegative_violation_kwh": float(max(0, -np.min([g, c, d, w]))),
249     "11.7_curtail_bound_violation_kwh": float(max(0, -w.min(), np.max(w - pv))),
250     "11.8_initial_error_kwh": float(abs(e0 - E_INITIAL)),
251     "11.8_terminal_error_kwh": float(abs(e[-1] - E_INITIAL)),
252     "11.9_objective_recalculation_error_yuan": float(abs(p @ g - solution["solver"]["
objective_yuan"])),
253     "9_binary_distance": float(np.max(np.abs(u - np rint(u)))),
254     "9_binary_bound_violation": float(max(0, -u.min(), u.max() - 1)),
255     "9_binary_coupling_violation_kwh": float(max(0, np.max(c - qc * u), np.max(d - qd * (1 - u)
))),
256     "10_primal_dual_gap_yuan": float(abs(p @ g - solution["solver"]["dual_bound_yuan"])),
257     "day_energy_identity_error_kwh": float(abs(g.sum() + pv.sum() - w.sum() - load.sum()
- (1 - ec) * c.sum() - (1 / ed - 1) * d.sum()
- (e[-1] - e0))),
260 }
261 if settings.get("baseline") == "no_ess":
262     metrics["11.4_no_ess_activity_kwh"] = float(max(np.max(np.abs(c)), np.max(np.abs(d))))
263 if settings.get("baseline") == "pv_only":
264     metrics["11.4_pv_only_bound_violation_kwh"] = float(max(
265         0, np.max(c - np.maximum(pv - load, 0)), np.max(d - np.maximum(load - pv, 0)))
266 if settings.get("baseline") in ("no_ess", "pv_only"):
267     metrics["11.4_surplus_grid_source_kwh"] = float(np.max(np.abs(g[pv > load]), initial=0))
268 metrics["11.2_relative_global_gap"] = float(abs(p @ g - solution["solver"]["dual_bound_yuan"])
/ max(1, abs(p @ g)))
269
270 metric_tolerances = {key: COST_TOL_YUAN if key.endswith("_yuan") else
271     MIP_GAP_TOL if key.endswith("relative_global_gap") else
272     BINARY_TOL if key in ("9_binary_distance", "9_binary_bound_violation") else
273     PHYS_TOL_KWH for key in metrics}
274 failed = {key: val for key, val in metrics.items()
275     if not np.isfinite(val) or val > metric_tolerances[key]}
276 if (solution["solver"]["status"] != 0 or not np.isfinite(solution["solver"]["mip_gap"])
277     or solution["solver"]["mip_gap"] > MIP_GAP_TOL):
278     failed["solver_status_or_gap"] = solution["solver"]
279 if failed:
280     raise ValueError(f"物理/最优性检查失败: {failed}")
281 return {"passed": True, "tolerances": TOLERANCES, "metric_tolerances": metric_tolerances, "
metrics": metrics}
282
283
284 def schedule_rows(data, solution):
285     rows = []
286     for t, record in enumerate(data):
287         row = dict(record)
288         row.update({f"{name}_kwh": solution[name][t] for name in ("G", "C", "D", "E", "W")})
289         row["u"] = solution["u"][t]
290         row["E_start_kwh"] = solution["E0"] if t == 0 else solution["E"][t - 1]

```

```

291     row["soc_fraction"] = solution["E"][t] / 12000
292     row["cost_yuan"] = record["price_yuan_per_kwh"] * solution["G"][t]
293     rows.append(row)
294     return rows
295
296
297 def summary(data, solution):
298     p, load, pv = arrays(data)
299     g, c, d, e, w = (np.asarray(solution[k]) for k in ("G", "C", "D", "E", "W"))
300     return {"eta_c": solution["eta_c"], "eta_d": solution["eta_d"],
301            "roundtrip_efficiency": solution["eta_c"] * solution["eta_d"],
302            "cost_yuan": float(p @ g), "grid_kwh": float(g.sum()),
303            "curtail_kwh": float(w.sum()), "charge_kwh": float(c.sum()),
304            "discharge_kwh": float(d.sum()), "load_kwh": float(load.sum()),
305            "pv_kwh": float(pv.sum()), "soc_min_kwh": float(e.min()),
306            "soc_max_kwh": float(e.max()), "initial_kwh": solution["E0"], "terminal_kwh": float(e
307            [-1]),
308            "loss_kwh": float((1 - solution["eta_c"]) * c.sum() + (1 / solution["eta_d"] - 1) * d.
309            sum()),
310            "charge_weighted_price": float(p @ c / c.sum()) if c.sum() > PHYS_TOL_KWH else None,
311            "discharge_weighted_price": float(p @ d / d.sum()) if d.sum() > PHYS_TOL_KWH else None,
312            "pv_surplus_kwh": float(np.maximum(pv - load, 0).sum()),
313            "pv_surplus_absorbed_kwh": float(np.minimum(c, np.maximum(pv - load, 0)).sum())}
314
315 def block_rows(solution):
316     return [{"interval": f"{clock_label(h * 60)}-{clock_label((h + 4) * 60)}",
317            "charge_kwh": float(sum(solution["C"][h * 6:(h + 4) * 6])),
318            "discharge_kwh": float(sum(solution["D"][h * 6:(h + 4) * 6]))}
319            for h in range(0, 24, 4)]

```

src/q1/pipeline.py

```

1  """CR-03/04: 数值与图形解耦；验证完整generation后原子切换唯一指针。"""
2
3  import os
4  import shutil
5  import subprocess
6  import sys
7  from pathlib import Path
8
9  from model import ROOT, write_json
10
11 # 原有对外路径保持可用；所有本次运行文件都通过同一个current指针访问。
12 ALIASES = {"inputs/q1": "inputs",
13            "docs/1/deliverables/q1_source_line_traceability.csv": "docs/q1_source_line_traceability.
14            csv", "outputs/q1/raw": "raw",
15            "outputs/processed/q1": "processed",
16            "docs/1/deliverables/q1_results.md": "docs/q1_results.md",
17            "docs/1/deliverables/q1_marginal_analysis.md": "docs/q1_marginal_analysis.md"}
18
19 def run_tests(staging):
20     environment = {**os.environ, "Q1_ARTIFACT_ROOT": str(staging)}
21     completed = subprocess.run([sys.executable, "-m", "unittest", "discover", "-s", "src/q1", "-p",
22                                "test_*.py", "-v"],

```

```

22         cwd=R00T, env=environment, capture_output=True, text=True, encoding="
    utf-8")
23 log = completed.stdout + completed.stderr
24 (staging / "raw/test_log.txt").write_text(log, encoding="utf-8")
25 write_json(staging / "raw/tests.json", {"passed": completed.returncode == 0, "returncode":
    completed.returncode,
26                                     "command": "python -m unittest discover -s src/q1 -p
    test_*.py -v", "log": log})
27 if completed.returncode:
28     raise RuntimeError(f"本次staging测试未全部通过, 拒绝发布: \n{log}")
29
30
31 def render_figures(data, solutions, summaries, analysis, staging, enabled=True):
32     """图形全部在独立临时目录产生; 失败可见且不会留下部分图形。"""
33     directory = staging / "figures_tmp"
34     directory.mkdir()
35     try:
36         if not enabled:
37             return {"status": "skipped", "message": "按--no-plots仅执行数值、报告和交付文件"}
38         from report import create_figures
39         from analysis_report import analysis_figures
40         create_figures(data, solutions, summaries, directory)
41         analysis_figures(analysis, directory)
42         for path in directory.iterdir():
43             path.replace(staging / "processed" / path.name)
44         return {"status": "passed", "message": "全部四组图形PNG/SVG生成成功"}
45     except Exception as exc:
46         return {"status": "failed", "message": f"{type(exc).__name__}: {exc}"}
47     finally:
48         shutil.rmtree(directory)
49         if "matplotlib.pyplot" in sys.modules:
50             sys.modules["matplotlib.pyplot"].close("all")
51
52
53 def publish_generation(staging, root=R00T):
54     """staging须已通过所有数值/交付/测试门槛; 最终只调用一次os.replace切换。"""
55     root = Path(root).resolve()
56     current = root / "outputs/q1_current"
57     if current.exists() and not current.is_symlink():
58         raise ValueError("q1_current必须为generation符号链接")
59     # 首次迁移保留原目录, 失败逐一恢复。后续运行不会再修改别名。
60     if not current.is_symlink():
61         legacy = staging.parent / "legacy_before_atomic"
62         legacy.mkdir()
63         current.symlink_to(os.path.relpath(legacy, current.parent), target_is_directory=True)
64         moved = []
65         try:
66             for relative, target in ALIASES.items():
67                 alias, old = root / relative, legacy / target
68                 old.parent.mkdir(parents=True, exist_ok=True)
69                 alias.parent.mkdir(parents=True, exist_ok=True)
70                 had_old = alias.exists()
71                 if had_old:
72                     alias.rename(old)
73                 moved.append((alias, old, had_old))
74                 alias.symlink_to(os.path.relpath(current / target, alias.parent),
75                                target_is_directory=old.is_dir())

```



```

75     except Exception:
76         for alias, old, had_old in reversed(moved):
77             if alias.is_symlink():
78                 alias.unlink()
79             if had_old:
80                 old.rename(alias)
81         current.unlink()
82         shutil.rmtree(legacy)
83         raise
84     for relative, target in ALIASES.items():
85         alias = root / relative
86         expected = os.path.relpath(current / target, alias.parent)
87         if not alias.is_symlink() or os.readlink(alias) != expected:
88             raise ValueError(f"正式路径未指向统一generation: {relative}")
89     pending = current.with_name(".q1_current_next")
90     try:
91         pending.symlink_to(os.path.relpath(staging, current.parent), target_is_directory=True)
92         os.replace(pending, current) # 同一文件系统的唯一发布提交点。
93     finally:
94         if pending.is_symlink():
95             pending.unlink()

```

src/q1/prepare_paper_materials.py

```

1  """从已验收的Q1运行代整理论文CSV与图表；不重新优化或改写原结果。"""
2  from pathlib import Path
3  import csv
4  import hashlib
5  import html
6  import json
7  import math
8  import shutil
9  import tempfile
10 import zipfile
11
12 ROOT = Path(__file__).resolve().parents[2]
13
14
15 def digest(path):
16     return hashlib.sha256(path.read_bytes()).hexdigest()
17
18
19 def read_csv(path):
20     with path.open(encoding="utf-8", newline="") as stream:
21         return list(csv.DictReader(stream))
22
23
24 def write_csv(path, rows):
25     with path.open("w", encoding="utf-8", newline="") as stream:
26         csv.writer(stream).writerows(rows)
27     with path.open(encoding="utf-8", newline="") as stream:
28         reread = list(csv.reader(stream))
29     expected = [[str(value) for value in row] for row in rows]
30     if reread != expected:
31         raise ValueError(f"CSV关闭回读不一致: {path.name}")
32

```

```

33
34 def close(actual, expected, tolerance=1e-6):
35     if not math.isfinite(actual) or abs(actual - expected) > tolerance:
36         raise ValueError(f"论文数据核验失败: {actual} != {expected}")
37
38
39 def table_html(rows, caption):
40     def cell(value):
41         if str(value).isdigit():
42             return str(value)
43         try:
44             number = float(value)
45             # HTML统一显示六位; CSV不截断精度, 时间段与中文文本原样显示。
46             return f"{number:.6f}" if math.isfinite(number) else str(value)
47         except (ValueError, TypeError):
48             return str(value)
49     header = "".join(f"<th>{html.escape(str(v))}</th>" for v in rows[0])
50     body = "".join("<tr>" + "".join(f"<td>{html.escape(cell(v))}</td>" for v in row) + "</tr>" for
51         row in rows[1:])
52     return f"<div class='scroll'><table><caption>{html.escape(caption)}</caption><thead><tr>{header}</tr></thead><tbody>{body}</tbody></table></div>"
53
54 def main():
55     generation = (ROOT / "outputs/q1_current").resolve(strict=True)
56     source = generation / "processed"
57     manifest = json.loads((generation / "raw/run_manifest.json").read_text(encoding="utf-8"))
58     if not all(manifest[k] for k in ("all_physical_csv_checks_passed", "analysis_checks_passed", "
59         tests_passed")):
60         raise ValueError("来源运行没有通过完整验收")
61     for rel, value in manifest["output_sha256"].items():
62         if digest(generation / rel) != value:
63             raise ValueError(f"来源产物哈希变化: {rel}")
64     data = read_csv(source / "result1.csv")
65     summary = read_csv(source / "daily_summary.csv")[0]
66     parameters = json.loads((generation / "inputs/processed/parameters.json").read_text(encoding="
67         utf-8"))
68     value = lambda row, key: float(row[key])
69     if len(data) != 144:
70         raise ValueError("主规划表必须包含144时段")
71     for t, row in enumerate(data, 1):
72         if int(row["t"]) != t or int(row["start_minute"]) != (t - 1) * 10 or int(row["end_minute"])
73             != t * 10:
74             raise ValueError(f"时段缺失或错位: {t}")
75         g, c, d, e, w = (value(row, k) for k in ("G_kwh", "C_kwh", "D_kwh", "E_kwh", "W_kwh"))
76         close(g + value(row, "pv_kwh") - w + d, value(row, "load_kwh") + c)
77         close(e, value(row, "E_start_kwh") + value(summary, "eta_c") * c - d / value(summary, "eta_d
78             "))
79         close(value(row, "cost_yuan"), value(row, "price_yuan_per_kwh") * g)
80         close(value(row, "soc_fraction"), e / parameters["rated_capacity_kwh"], 1e-8)
81         if t > 1:
82             close(value(row, "E_start_kwh"), value(data[t - 2], "E_kwh"))
83         if min(c, d) > 1e-6 or min(g, c, d, w) < -1e-6:
84             raise ValueError("负电量或同时充放电")
85         if not 1200 - 1e-6 <= e <= 10800 + 1e-6 or max(c, d) > 5000 / 6 + 1e-6:
86             raise ValueError("SOC或功率越界")
87     for column, key in [("G_kwh", "grid_kwh"), ("C_kwh", "charge_kwh"), ("D_kwh", "discharge_kwh"),

```

```

145 ("cost_yuan", "cost_yuan"), ("W_kwh", "curtail_kwh"]):
146     close(math.fsum(value(r, column) for r in data), value(summary, key))
147 close(value(data[0], "E_start_kwh"), 6000)
148 close(value(data[-1], "E_kwh"), 6000)
149
150 tables = {}
151 plan_columns = [("时段序号", "t"), ("时间段", "interval"), ("电价(元/kWh)", "price_yuan_per_kwh"),
152 ("负载电量(kWh)", "load_kwh"), ("光伏预测电量(kWh)", "pv_kwh"), ("计划购电量(kWh)", "G_kwh"), ("充电量
153 (kWh)", "C_kwh"), ("放电量(kWh)", "D_kwh"), ("期初储电量(kWh)", "E_start_kwh"), ("期末储电量(kWh)",
154 "E_kwh"), ("弃光电量(kWh)", "W_kwh"), ("购电费(元)", "cost_yuan")]
155 plan = [[label for label, _ in plan_columns] + ["期末SOC(%)", "实际运行状态"]]
156 for row in data:
157     action = "充电" if value(row, "C_kwh") > 1e-6 else "放电" if value(row, "D_kwh") > 1e-6 else
158 "待机"
159     plan.append([row[key] for _, key in plan_columns] + [100 * value(row, "soc_fraction"),
160 action])
161 tables["table_144_dispatch.csv"] = ("完整最优方案: 144个10分钟时段", "附录或完整结果附件", plan)
162
163 t1 = read_csv(source / "table1_purchase.csv")
164 lookup = {r["interval"]: r for r in data}
165 expected_times = [f"{h:02d}:00-{h:02d}:10" for h in (10, 12, 14, 16, 18, 20)]
166 if [r["interval"] for r in t1] != expected_times:
167     raise ValueError("题目表1时段不符")
168 for row in t1:
169     close(value(row, "grid_kwh"), value(lookup[row["interval"]], "G_kwh"))
170 table1 = [[["时间段", "购电量(kWh)"] * 3]
171 for group in (t1[:3], t1[3:]):
172     table1.append([x for row in group for x in (row["interval"], row["grid_kwh"])]])
173 table1.append(["全天购电量(kWh)", "", summary["grid_kwh"], "全天购电费(元)", "", summary["cost_yuan"
174 ]])
175 tables["table1_purchase_paper.csv"] = ("表1 微网指定时段及全天购电结果", "题目要求: 论文正文", table1)
176
177 t2 = read_csv(source / "table2_storage.csv")
178 if len(t2) != 6:
179     raise ValueError("题目表2须为六个4小时区间")
180 for block, row in enumerate(t2):
181     if row["interval"] != f"{block * 4:02d}:00-{(block + 1) * 4:02d}:00":
182         raise ValueError("题目表2时段不符")
183     for column, key in [("C_kwh", "charge_kwh"), ("D_kwh", "discharge_kwh")]:
184         close(math.fsum(value(r, column) for r in data[24 * block:24 * (block + 1)]), value(row,
185 key))
186 table2 = [[["时间段", "充电量(kWh)", "放电量(kWh)"] * 2]
187 for i in (0, 2, 4):
188     table2.append([x for row in t2[i:i + 2] for x in (row["interval"], row["charge_kwh"], row["
189 discharge_kwh"])]])
190 table2.append(["0:00储电量(kWh)", "", summary["initial_kwh"], "24:00储电量(kWh)", "", summary["
191 terminal_kwh"]])
192 tables["table2_storage_paper.csv"] = ("表2 储能分区间充放电量及首尾储电量", "题目要求: 论文正文", table2
193 )
194
195 baselines = read_csv(source / "baseline_comparison.csv")
196 base_cost = value(next(r for r in baselines if r["baseline"] == "no_ess"), "cost_yuan")
197 saving = base_cost - value(summary, "cost_yuan")
198 saving_pct = 100 * saving / base_cost
199 indicators = [[["指标", "数值", "单位"], *[label, summary[key], unit] for label, key, unit in [
200 ("全天购电费用", "cost_yuan", "元"), ("全天购电量", "grid_kwh", "kWh"), ("全天负载电量", "load_kwh
201 ", "kWh"), ("全天光伏预测电量", "pv_kwh", "kWh"), ("全天充电量(母线侧)", "charge_kwh", "kWh"), ("全天

```

```

放电量(母线侧)", "discharge_kwh", "kWh"), ("全天弃光电量", "curtail_kwh", "kWh"), ("充放电能量损耗", "
loss_kwh", "kWh"), ("0:00储电量", "initial_kwh", "kWh"), ("24:00储电量", "terminal_kwh", "kWh"), (
"最低储电量", "soc_min_kwh", "kWh"), ("最高储电量", "soc_max_kwh", "kWh"), ("充电加权电价", "
charge_weighted_price", "元/kWh"), ("放电加权电价", "discharge_weighted_price", "元/kWh"))]]
129 indicators += [{"相对无储能节约费用", saving, "元/日"}, {"相对无储能费用降幅", saving_pct, "%"}, {"光伏
电量消纳率", 100 * (1 - value(summary, "curtail_kwh") / value(summary, "pv_kwh")), "%"}]
130 tables["daily_indicators.csv"] = ("主方案全日指标", "摘要、结果分析", indicators)
131 param_rows = [{"参数或口径", "取值", "单位或解释"}]
132 for label, key, unit in [{"时段数", "period_count", "个"}, {"时段长度", "dt_hours", "h"}, {"额定容量
", "rated_capacity_kwh", "kWh"}, {"储电量下限", "min_energy_kwh", "kWh"}, {"储电量上限", "
max_storage_kwh", "kWh"}, {"最大充放电功率", "max_power_kw", "kW"}, {"单时段充放电电量上限", "
max_energy_kwh", "kWh: 精确5000/6"}, {"首尾储电量", "initial_and_terminal_kwh", "kWh"}]:
133     param_rows.append([label, parameters[key], unit])
134 param_rows += [{"充电效率", summary["eta_c"], "沿用已确认的单程口径"}, {"放电效率", summary["eta_d"],
"沿用已确认的单程口径"}, {"往返效率", summary["roundtrip_efficiency"], "充电效率乘放电效率"}, {"时间标签"
, "右端点", "00:10对应00:00-00:10"}, {"充放电电量口径", "母线侧", "储电量为电池内部电量"}, {"售电", "不允许
", "允许弃光"}, {"目标", "最小化购电费", "无二级目标"}]
135 tables["model_parameters.csv"] = ("模型参数与计算口径", "符号说明、参数设置", param_rows)
136 labels = {"main": "主方案: 双单程0.90", "eta085": "敏感性: 双单程0.85", "eta095": "敏感性: 双单程0.95",
"roundtrip090": "口径对照: 对称往返0.90", "no_ess": "无储能", "pv_only": "仅光伏消纳储能", "full": "
完整最优策略"}
137 comparisons = [{"efficiency_comparison.csv", "efficiency_comparison_paper.csv", "scenario", "效率
口径比较"}, {"baseline_comparison.csv", "baseline_comparison_paper.csv", "baseline", "嵌套基准与节
约比较"}]
138 for filename, output, key, title in comparisons:
139     rows = [{"方案", "充电效率", "放电效率", "往返效率", "全天费用(元)", "购电量(kWh)", "充电量(kWh)", "
放电量(kWh)", "弃光量(kWh)", "相对无储能节约(元)", "相对无储能降幅(%)"}]
140     for r in read_csv(source / filename):
141         rows.append([labels[r[key]] + [r[k] for k in ("eta_c", "eta_d", "roundtrip_efficiency",
"cost_yuan", "grid_kwh", "charge_kwh", "discharge_kwh", "curtail_kwh")] + [base_cost - value(r,
"cost_yuan"), 100 * (base_cost - value(r, "cost_yuan")) / base_cost])
142         # 不同效率必须与同效率无储能基准相比: 无储能没有充放电, 费用不受效率影响。
143         tables[output] = (title, "结果分析或稳健性分析", rows)
144
145 resources = {"storage_upper": "提高储电量上限", "storage_lower": "降低储电量下限", "charge_power": "
提高充电功率", "discharge_power": "提高放电功率"}
146 boundary = [{"诊断方向", "基准边界", "放宽步长", "参数有符号变化", "费用节约(元/日)", "单位日价值", "单位",
"基准活跃时段数", "三步长价值稳定", "解释边界"}]
147 for r in read_csv(source / "bottleneck_values.csv"):
148     if r["scenario"] == "main_l100_pv100":
149         boundary.append([resources[r["resource"]] + [r[k] for k in ("base_limit", "increment",
"parameter_change", "saving_yuan", "value_per_unit_day")] + ["元/(kWh·日)" if r["resource"].
startswith("storage") else "元/(kW·日)", r["active_count"], "是" if r["multistep_stable"] == "1"
else "否", "仅边界诊断, 不改变主方案约束: 未判断投资经济显著性"]])
150 if len(boundary) != 13:
151     raise ValueError("主方案边界实验应为12项")
152 tables["boundary_values_paper.csv"] = ("主方案四类边界的三步长日价值", "边际价值分析(可选)", boundary)
153 sensitivity = [{"情景", "效率口径", "负荷变化(%)", "光伏变化(%)", "费用(元)", "购电量(kWh)", "充电量(kWh
)", "放电量(kWh)", "弃光量(kWh)", "最低储量(kWh)", "最高储量(kWh)"}]
154 for r in read_csv(source / "sensitivity_scenarios.csv"):
155     sensitivity.append([r["scenario"], labels[r["efficiency_name"]], 100 * value(r, "load_delta"
), 100 * value(r, "pv_delta")] + [r[k] for k in ("cost_yuan", "grid_kwh", "charge_kwh", "
discharge_kwh", "curtail_kwh", "soc_min_kwh", "soc_max_kwh")])
156 tables["sensitivity_36_paper.csv"] = ("36组效率、负荷及光伏情景", "稳健性分析或附录(可选)",
sensitivity)
157
158 figures = [{"dispatch", "主方案调度与储电轨迹", "输入、电价、购电与充放电量、SOC的时序关系: 正文优先使用"},

```

```
("efficiency", "效率口径对费用和储电轨迹的影响", "较低效率导致更高费用；主方案与往返效率口径须区分"), ("marginal_analysis", "基准比较、局部阈值及输入敏感性", "固定模式阈值只作局部诊断；25项原MILP双侧差分不匹配"), ("bottleneck_analysis", "四类边界放宽的单位日价值", "储能上限、下限、充电和放电功率单位不同；不据数值直接作投资排序")]
```

```
parent = R00T / "outputs/processed/q1_paper"
```

```
parent.mkdir(parents=True, exist_ok=True)
```

```
package_id = generation.name + "-" + digest(Path(__file__))[8:]
```

```
destination = parent / package_id
```

```
staging = Path(tempfile.mkdtemp(prefix=".staging-", dir=parent))
```

```
try:
```

```
    index = [["文件", "标题", "建议用途", "数据行数", "备注"]]
```

```
    for filename, (title, use, rows) in tables.items():
```

```
        write_csv(staging / filename, rows)
```

```
        index.append([filename, title, use, len(rows) - 1, "CSV保留原数值精度；表1/表2空白单元表示题目合并表头位置"])
```

```
    for stem, title, caption in figures:
```

```
        for extension in ("png", "svg"):
```

```
            filename = f"{stem}.{extension}"
```

```
            if (source / filename).exists():
```

```
                shutil.copyfile(source / filename, staging / filename)
```

```
                index.append([filename, title, "论文插图", "", caption])
```

```
    write_csv(staging / "materials_index.csv", index)
```

```
    guide = f'"# 第一问论文结果材料"
```

本包使用已验收运行`{generation.name}`，沿用双单程效率0.90、母线侧充放电、右端点时间标签和10分钟零阶保持；本次只整理现有最优解，不改变模型。结果表格均为UTF-8 CSV，不使用Office。CSV保留原始数值精度，HTML与下列示例显示六位小数。本文的光伏结果基于附件1预测输入，不代表实际发电保证。

直接取用顺序

1. [144时段完整规划表](table_144_dispatch.csv)：中文列名，含时间、电价、负荷与光伏电量、计划购电、充放电、首尾储量、SOC、弃光、费用与实际运行状态。完整表适合附录或结果附件，正文可引用题目指定表及调度图。
2. [题目表1](table1_purchase_paper.csv)：按题目三组“时间段/购电量”并排排列，末行含全天购电量与费用。
3. [题目表2](table2_storage_paper.csv)：按题目两组“时间段/充电量/放电量”并排排列，末行含0:00及24:00储电量。CSV不保存合并单元格，空白位置对应题目合并标题；导入排版软件后按题面合并即可。
4. [全日指标](daily_indicators.csv)和[模型参数](model_parameters.csv)：用于摘要、参数设置与结果概述。
5. [基准比较](baseline_comparison_paper.csv)、[效率比较](efficiency_comparison_paper.csv)、[边界价值](boundary_values_paper.csv)、[36情景表](sensitivity_36_paper.csv)：供结果分析及稳健性章节选择使用。
6. [图表目录与图注](materials_index.csv)：列出所有CSV和现有PNG/SVG；优先使用调度图，SVG可用于矢量排版。

[打开全部表格的浏览器预览](index.html)。文件独立齐全，整体目录移动后内部链接仍有效。

可用于论文的结果表述

在给定一天的电价、负荷及光伏预测数据下，将全天划分为144个10 min时段，以购电费用最小为目标求解混合整数线性规划。所得一组最优调度的全天购电量为{value(summary, 'grid_kwh'):.6f} kWh，购电费用为{value(summary, 'cost_yuan'):.6f}元；储能设备累计充电{value(summary, 'charge_kwh'):.6f} kWh、放电{value(summary, 'discharge_kwh'):.6f} kWh，均按微网母线侧计量。储电量保持在1200—10800 kWh，0:00与24:00均为6000 kWh，满足日周期约束。所用光伏预测电量全部被利用，弃光量为0。求解器报告MIP gap为0，该最优性结论针对当前模型和数值容差；调度本身可能不唯一。

相对于无储能基准的{base_cost:.6f}元，完整策略节约{saving:.6f}元，费用下降{saving_pct:.6f}%。此比较同时包含光伏消纳和跨时段购电调整的作用，不是对二者的唯一因果分解。全天充放电之差{value(summary, 'loss_kwh'):.6f} kWh对应储能能量损耗；首尾储量相同并不意味着母线侧充电量与放电量相等。

表格口径与解释边界

— 时段00:00—00:10的“期初储量”为0:00状态，“期末储量”为00:10状态。最后一行23:50—24:00属于本日。SOC以额定容量

```

12000 kWh为分母。
200 - “充电/放电/待机”由实际电量及 $1e-6$  kWh数值容差识别；不把空闲二元变量直接当作真实运行状态。
201 - 表1中部分时段购电量为0，是光伏与储能共同满足负荷的结果。表2同一4小时区间内充电量和放电量都为正，表示不同10分钟时段发
    生不同动作，不违反时段互斥。
202 - 主方案效率为充电0.9、放电0.9，往返0.81。对称往返0.90是另一个对照情景，不能替换主方案表1、表2或144规划表。
203 - 36个情景属于确定性参数扰动，不是独立随机样本；没有由这些行构造统计显著性或置信区间。
204 - 25项固定模式影子价不能同时匹配原MILP双侧扰动，局部阈值图只作诊断。边界日价值未计设备投资成本，不能直接作经济投资结
    论。
205 - 较低效率下本算例充电量增加、放电量减少；提高光伏在所测情景中未造成弃光。正文应保留这些实测方向，不能写成相反趋势。
206
207 ## 来源与复现
208
209 全部来源文件哈希、生成器哈希、独立物理与汇总核验记录见[材料清单JSON](provenance.json)。源码入口：src/q1/
    prepare_paper_materials.py；在项目根目录运行`python3 src/q1/prepare_paper_materials.py`可从当前已验
    收运行重新整理。此材料包不重新求解，后续若模型重算，须重新执行整理并采用新的运行代材料。
210
211 (staging / "README.md").write_text(guide, encoding="utf-8")
212 content = ["<!doctype html><html lang='zh-CN'><meta charset='utf-8'><meta name='viewport'
    content='width=device-width,initial-scale=1'><title>第一问论文结果材料</title><style>body{font:16px
    /1.65 system-ui,sans-serif;color:#182e3e;background:#f5f7f8;margin:0}main{max-width:1400px;
    margin:32px auto;padding:24px}h1{color:#174d5b}a{color:#126079}section{background:white;padding
    :24px;margin:24px 0;border-radius:8px}.scroll{overflow:auto;max-height:640px}table{border-
    collapse:collapse;width:100%;font-variant-numeric:tabular-nums}th,td{padding:8px 12px;border-
    bottom:1px solid #dce3e7;white-space:nowrap;text-align:right}th{background:#e9f1f3;position:
    sticky;top:0}caption{text-align:left;font-weight:600;margin:8px 0}td:first-child,th:first-child{
    text-align:left}img{max-width:100%;height:auto}summary{cursor:pointer;font-weight:600}@media
    print{.scroll{max-height:none;overflow:visible}main{margin:0;padding:0}section{break-inside:
    avoid}}</style><main><h1>第一问 · 论文结果材料</h1>"]
213 content.append(f"<p>来源：{generation.name}。表格均为CSV，原始数值精度保留；本页显示六位小数。主方案为
    双单程效率0.90。</p><p><a href='README.md'>使用说明与结果表述</a> · <a href='materials_index.csv'>全
    部材料索引CSV</a></p>")
214 for filename in ("table1_purchase_paper.csv", "table2_storage_paper.csv", "
    table_144_dispatch.csv", "daily_indicators.csv", "model_parameters.csv", "
    baseline_comparison_paper.csv", "efficiency_comparison_paper.csv", "boundary_values_paper.csv",
    "sensitivity_36_paper.csv"):
215     title, _, rows = tables[filename]
216     rendered = table_html(rows, title)
217     if filename == "table_144_dispatch.csv":
218         rendered = f"<details id='dispatch'><summary>展开144个时间段的完整规划表</summary>{
    rendered}</details>"
219     content.append(f"<section><h2>{html.escape(title)}</h2><p><a href='{filename}'>下载CSV: {
    html.escape(title)}</a></p>{rendered}</section>")
220 content.append("<p>表1、表2末行的空白单元对应页面合并标题。完整规划表按实际电量标记充电/放电/待机；4小时内
    可在不同10分钟时段分别充电和放电。</p>")
221 for stem, title, caption in figures:
222     if (staging / f"{stem}.png").exists():
223         content.append(f"<section><h2>{title}</h2><img src='{stem}.png' alt='{title}'><p>{
    caption}</p><a href='{stem}.svg'>SVG矢量图</a></section>")
224 content.append("<p>25项固定模式影子价与原MILP双侧差分不匹配；局部阈值不能直接当成通用调控规律。边界价值未
    作投资显著性判断。</p></main></html>")
225 (staging / "index.html").write_text("\n".join(content), encoding="utf-8")
226 provenance = {"source_generation": generation.name, "source_manifest_sha256": digest(
    generation / "raw/run_manifest.json"), "source_artifact_sha256": manifest["output_sha256"], "
    generator_sha256": digest(Path(__file__)), "checks": {"passed": True, "plan_periods": 144, "
    table1_intervals": 6, "table2_blocks": 6, "physical_balance_soc_limits_costs": True, "
    all_csv_closed_readback": True, "source_all_hashes_verified": True}, "package_sha256": {p.name:
    digest(p) for p in staging.iterdir()}}

```



```

227     (staging / "provenance.json").write_text(json.dumps(provenance, ensure_ascii=False, indent
228     =2) + "\n", encoding="utf-8")
229     if destination.exists():
230         for p in staging.iterdir():
231             if not (destination / p.name).exists() or p.read_bytes() != (destination / p.name).
232             read_bytes():
233                 raise ValueError("同一源代与生成器的材料目录已有不同内容，不覆盖")
234             shutil.rmtree(staging)
235         else:
236             staging.rename(destination)
237             archive = parent / f"{package_id}.zip"
238             with zipfile.ZipFile(archive, "w", zipfile.ZIP_DEFLATED) as bundle:
239                 for p in sorted(destination.iterdir()):
240                     bundle.write(p, arcname=f"q1_paper/{p.name}")
241             print(json.dumps({"directory": str(destination), "archive": str(archive), "csv_files": len(
242             tables) + 1, "plan_rows": len(plan) - 1, "saved_yuan": saving, "saving_percent": saving_pct, "
243             passed": True}, ensure_ascii=False, indent=2))
244         finally:
245             if staging.exists():
246                 shutil.rmtree(staging)
247
248 if __name__ == "__main__":
249     main()

```

src/q1/report.py

```

1  """从已验收结果生成可见输入/输出、题目表格及科研图，无额外优化假设。"""
2
3  import html
4  import re
5
6  import numpy as np
7
8  from model import PHYS_TOL_KWH, TOLERANCES, ROOT, arrays, block_rows, schedule_rows
9
10 LABELS = {"main": "单程 0.90（主模型）", "eta085": "单程 0.85",
11           "eta095": "单程 0.95", "roundtrip090": "往返 0.90（对称）"}
12
13
14 def create_figures(data, solutions, summaries, output):
15     """兼容原入口；绘图实现统一维护于src/plots/q1_legacy.py。"""
16     import sys
17     from pathlib import Path
18     sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
19     from src.plots.q1_legacy import create_figures as render
20     return render(data, solutions, summaries, output)
21
22
23 def html_table(headers, rows):
24     def cell(x):
25         return f"{x:,.6f}" if isinstance(x, (float, np.floating)) else html.escape(str(x))
26     return '<div class="table-wrap"><table><thead><tr>' + ''.join(f"<th>{html.escape(h)}</th>" for h
27         in headers) + \
28         '</tr></thead><tbody>' + ''.join('<tr>' + ''.join(f"<td>{cell(v)}</td>" for v in row) + '
29         </tr>' for row in rows) + '</tbody></table></div>'

```

```

28
29
30 def create_report(data, solutions, summaries, validation, output, documents, figure_status):
31     main, s = solutions["main"], summaries[0]
32     p, load, pv = arrays(data)
33     blocks = block_rows(main)
34     max_residual = max(max(v["metrics"].values()) for v in validation.values())
35     assumptions = ("依据最终版模型采用右端点时间映射与10分钟内零阶保持；C、D均为微网母线侧电量；"
36                   "主模型充电与放电效率各0.9；不向外网售电，允许弃光。"
37                   "经用户确认，功率上限使用5000×1/6的准确值，833.3333仅用于显示。"
38                   "仅最小化购电费，不添加二级目标；同成本最优解可能不唯一，空闲时段u也不唯一。")
39     reason = (f"全天负荷 {s['load_kwh']:, .6f} kWh、光伏 {s['pv_kwh']:, .6f} kWh、"
40              f"储能损耗 {s['loss_kwh']:, .6f} kWh。购电+光伏-弃光=负荷+损耗，首尾电量均为6000 kWh。"
41              f"充电量加权电价为 {s['charge_weighted_price']:.6f} 元/kWh，"
42              f"放电量加权电价为 {s['discharge_weighted_price']:.6f} 元/kWh；"
43              f"光伏超过负荷的 {s['pv_surplus_kwh']:, .6f} kWh 中，储能吸收 {s['pv_surplus_absorbed_kwh']:, .6f} kWh。"
44              "这些量来自实际解，分别用于检查低价充电、高价放电与富余光伏吸收规律。")
45     roundtrip = next(r for r in summaries if r["scenario"] == "roundtrip090")
46     delta = roundtrip["cost_yuan"] - s["cost_yuan"]
47     robustness = (f"另一效率口径的成本较主结果变化 {delta:,.6f} 元 ({delta / s['cost_yuan'] * 100:.4f}%"
48                  ")。"
49                  "下表列出每个情景的实际成本、能量与加权电价；状态轨迹和4小时分组数据用于对照阶段变化。"
50                  "效率口径会改变成本和局部充放电计划，不能将不同口径的数值混用于正式结果。")
51     if all(r["charge_weighted_price"] < r["discharge_weighted_price"] and
52           abs(r["pv_surplus_absorbed_kwh"] - r["pv_surplus_kwh"]) <= PHYS_TOL_KWH for r in
53           summaries):
54         robustness += ("本次四个情景均满足充电加权电价低于放电加权电价，且全部吸收富余光伏；"
55                        "随单程效率由0.85升至0.95，成本和外网购电量均下降。"
56                        "这些实算结果支持低价充电、高价放电、吸收富余光伏的整体规律对所检验效率参数稳健，"
57                        "不意味着逐时策略完全相同或对未检验参数也成立。")
58     report = f"""<!doctype html><html lang="zh-CN"><meta charset="UTF-8"><meta name="viewport"
59             content="width=device-width,initial-scale=1">
60             <title>Q1 输入与输出 · 最终模型计算结果</title><style>
61             body{{font:16px/1.7 -apple-system,BlinkMacSystemFont,'PingFang SC',sans-serif;color:#173047;
62                 background:#f4f6f8;margin:0}}
63             main{{max-width:1180px;margin:auto;padding:40px 24px}}h1{{font-size:32px;line-height:1.3}}h2{{margin
64                 -top:44px;font-size:24px}}
65             a{{color:#0072b2}}.intro{{color:#46586a}}.metrics{{display:flex;gap:20px;flex-wrap:wrap;margin:28px
66                 0}}
67             .metric{{background:white;padding:20px 24px;flex:1;min-width:200px;border-top:4px solid #0072b2}}.
68                 metric strong{{display:block;font-size:27px}}
69             .table-wrap{{overflow:auto;background:white;margin:18px 0;max-height:620px}}table{{border-collapse:
70                 collapse;width:100%;font-variant-numeric:tabular-nums;font-size:14px}}
71             th,td{{padding:10px 14px;border-bottom:1px solid #e1e7eb;text-align:right;white-space:nowrap}}th{{
72                 background:#e6eff5;position:sticky;top:0}}
73             td:first-child,th:first-child{{text-align:left}}img{{width:100%;height:auto;background:white}}
74             details{{margin:20px 0}}summary{{cursor:pointer;font-weight:600}}
75             .note{{background:#e9f2f3;padding:16px 20px;border-left:4px solid #009e73}}code{{background:#e6ebee;
76                 padding:2px 5px}}footer{{margin-top:45px;color:#526577;font-size:13px}}
77             </style><main><p>数学建模 / C题 / 问题1</p><h1>输入与输出：微网日前经济调度</h1>
78             <p class="intro">144 个时段 · 864 个决策变量（其中144个二元变量）· 单一购电费目标 · 四种效率情景</p>
79             <p class="note">假设与已确认的计算口径：{assumptions}</p>
80             <div class="metrics"><div class="metric">最低全天购电费<strong>{s['cost_yuan']:, .6f} 元</strong></div>
81             <div class="metric">全天外网购电量<strong>{s['grid_kwh']:, .6f} kWh</strong></div>
82             <div class="metric">全天弃光量<strong>{s['curtail_kwh']:, .6f} kWh</strong></div></div>
83             <p><a href="result1.csv">完整144时段输出 CSV</a> · <a href="daily_summary.csv">日汇总 CSV</a> ·

```



```

73 <a href="../../../inputs/q1/raw/attachment1.csv">原始输入 CSV</a> ·
74 <a href="../../../inputs/q1/processed/timeseries.csv">预处理输入 CSV</a> ·
75 <a href="../../../inputs/q1/processed/parameters.json">模型参数与来源</a></p>
76 <h2>输入：来源、时间与单位</h2>
77 <p>数据源为 C题附件1。{len(data)}条记录覆盖00:00 - 24:00。第一条附件标签00:10对应00:00 - 00:10，最后一条0:00+1对
    应23:50 - 24:00。
78 负荷和光伏的kW值乘1/6得到kWh；未进行插值、填补、平滑或四舍五入后求解。电价范围{p.min():.4f} - {p.max():.4f}元/kWh
    。</p>
79 <details open><summary>查看全部144行输入（可滚动）</summary>""
80 report += html_table(["t", "附件右端点", "调度区间", "电价 元/kWh", "负荷 kW", "光伏 kW", "负荷 kWh",
    "光伏 kWh"],
81                        [[r[k] for k in ("t", "source_endpoint", "interval", "price_yuan_per_kwh",
    "load_kw", "pv_kw", "load_kwh", "pv_kwh")] for r in data])
82 report += '</details><h2>输出：题目表1与表2</h2><p>表1 · 指定时段购电量及全天汇总</p>'
83 # 保持题目表1的六列、两行时段布局。
84 report += html_table(["时间段", "购电量 kWh", "时间段", "购电量 kWh", "时间段", "购电量 kWh"],
85                        [[v for h in hours for v in (data[h * 6]["interval"], main["G"][h * 6])]
86                         for hours in ((10, 12, 14), (16, 18, 20))])
87 report += html_table(["全天购电量 kWh", "全天购电费 元"], [[s["grid_kwh"], s["cost_yuan"]]])
88 report += '<p>表2 · 指定时段母线侧充放电量与首尾储电量</p>'
89 report += html_table(["时间段", "充电量 kWh", "放电量 kWh", "时间段", "充电量 kWh", "放电量 kWh"],
90                        [[v for b in blocks[i:i + 2] for v in b.values()] for i in (0, 2, 4)])
91 report += html_table(["0:00储电量 kWh", "24:00储电量 kWh"], [[main["E0"], main["E"][-1]]])
92 report += '<p>同一4小时区间的充电、放电量可以都为正，分别发生在不同10分钟时段。逐时互斥已核验。<br>图中功率为
    时段常数；储电量按时段边界连接。显示保留6位小数，文件内部保留求解精度。</p>'
93 report += f'<h2>结果合理性与最优性证据</h2><p>{reason}</p><p>HiGHS返回最优状态，MIP gap为{main["solver
    "]["mip_gap"]}; '
94 report += f'原始目标与最优性下界均为{main["solver"]["objective_yuan"]:.9f}元。全部物理与CSV回读检查通过，
    最大误差{max_residual:.3e}，容差为{PHYS_TOL_KWH:.0e}。</p>'
95 report += '<details><summary>查看全部144行输出（含二元变量、SOC与时段费用）</summary>'
96 report += html_table(["区间", "G kWh", "C kWh", "D kWh", "E结束 kWh", "W kWh", "u", "E开始 kWh", "
    SOC", "费用 元"],
97                        [[r[k] for k in ("interval", "G_kwh", "C_kwh", "D_kwh", "E_kwh", "W_kwh", "
    u", "E_start_kwh", "soc_fraction", "cost_yuan")]
98                         for r in schedule_rows(data, main)])
99 report += f'</details><h2>效率敏感性与口径稳健性</h2><p>{robustness}</p>'
100 report += html_table(["效率情景", "单程效率", "往返效率", "费用 元", "购电 kWh", "弃光 kWh", "充电 kWh",
    "放电 kWh", "充电加权电价", "放电加权电价"],
101                        [[LABELS[r["scenario"]]] + [r[k] for k in ("eta_c", "roundtrip_efficiency",
    "cost_yuan", "grid_kwh", "curtail_kwh", "charge_kwh", "discharge_kwh", "charge_weighted_price",
    "discharge_weighted_price")] for r in summaries])
102 report += ''
103 report += '<details><summary>查看四个情景的4小时充放电汇总</summary>'
104 report += html_table(["情景", "时间段", "充电 kWh", "放电 kWh"],
105                        [[LABELS[name], *b.values()] for name, solution in solutions.items() for b
    in block_rows(solution)])
106 report += '</details><h2>验证与复现</h2><p>按用户要求仅用CSV交付并回读复核，文件按最终模型右端点映射生成：原
    始附件保留。'
107 report += '只有主模型0.9效率结果写入result1.csv。每个效率情景均独立检查，未通过的解不用于正式输出。</p>'
108 report += html_table(["情景/文件", "检查项", "误差或违反量", "通过"],
109                        [[LABELS.get(name, name), key, f"{value:.3e}", "是"] for name, v in
    validation.items() for key, value in v["metrics"].items()])
110 report += '<p>在项目根目录运行：<br><code>uv run --with-requirements src/q1/requirements.txt python
    src/q1/run.py</code></p>'
111 report += '<p><a href="../../../q1/raw/validation.json">机器验收记录</a> · <a href="../../../q1/raw/
    run_manifest.json">来源哈希与运行环境</a> · <a href="../../../docs/1/deliverables/

```

```

112 q1_implementation_acceptance.md">逐条开发验收报告</a></p>'
report += '<footer>正式数值来自本次MILP实算。效率情景仅用于敏感性检验；数值最优性在所声明模型及容差内成立。</
113 footer></main></html>'
114 if figure_status["status"] != "passed":
115     report = re.sub(r'<img[^>]+>', '', report)
116     report = report.replace('<h2>输入：来源、时间与单位</h2>',
117                             '<p class="note">绘图状态： ' + html.escape(figure_status["message"]) + '</
118 p>'
119                             '<p class="note">交付口径：按用户要求仅输出CSV，购电时间范围为00:00 - 24:00；
120 review中的XLSX交付建议不采用。</p>'
121                             '<h2>输入：来源、时间与单位</h2>')
122 (output / "report.html").write_text(report, encoding="utf-8")
123 markdown = f"# Q1 最终模型计算结果\n\n假设与已确认口径：{assumptions}\n\n"
124 markdown += f"最低购电费 **{s['cost_yuan']:.9f} 元**；全天购电 **{s['grid_kwh']:.9f} kWh**；弃光 **{s
125 ['curtail_kwh']:.9f} kWh**.\n\n"
126 markdown += "| 时间段 | 购电量 kWh |\n|---|---:|\n" + "".join(f"| {data[h*6]['interval']} | {main
127 ['G'][h*6]:.9f} |\n" for h in (10,12,14,16,18,20))
128 markdown += "\n| 时间段 | 充电量 kWh | 放电量 kWh |\n|---|---:|---:|\n" + "".join(f"| {b['interval
129 ']} | {b['charge_kwh']:.9f} | {b['discharge_kwh']:.9f} |\n" for b in blocks)
130 markdown += f"\n0:00储电量: {main['E0']:.9f} kWh; 24:00储电量: {main['E'][-1]:.9f} kWh.\n\n{reason
131 }\n\n{robustness}\n\n"
132 markdown += "| 情景 | 费用 元 | 购电 kWh | 弃光 kWh | 充电 kWh | 放电 kWh |\n
133 |---|---:|---:|---:|---:|\n"
134 markdown += "".join(f"| {LABELS[r['scenario']]} | {r['cost_yuan']:.9f} | {r['grid_kwh']:.9f} | {
135 r['curtail_kwh']:.9f} | {r['charge_kwh']:.9f} | {r['discharge_kwh']:.9f} |\n" for r in summaries
136 )
137 markdown += f"\n全部物理、最优性及CSV回读检查通过；最大误差{max_residual:.3e}，按量纲分别验收，物理容差{
138 PHYS_TOL_KWH:.0e} kWh.\n\n"
139 markdown += f"[完整输入输出报告]({ROOT / 'outputs/processed/q1/report.html'}) • [正式结果]({ROOT /
140 'outputs/processed/q1/result1.csv'})\n"
141 markdown += "\nCSV逐格回读通过；144个购电时段、6个充放电区间、首尾储电量与总费用均已核验。"
142 markdown += "\n绘图状态：" + figure_status["message"] + "\n"
143 markdown += "\n分量纲容差：" + str(TOLERANCES) + "\n"
144 (documents / "q1_results.md").write_text(markdown, encoding="utf-8")

```

src/q1/review_compact_q1.py

```

1 """Independent numerical evidence for the compact Q1 manuscript review.
2
3 Run: uv run --with scipy --with openpyxl python src/q1/review_compact_q1.py
4 Input timestamps are provisionally interval endpoints; energies are bus-side.
5 These are audit experiments, not an official result1 submission.
6 """
7
8 import hashlib
9 import json
10 from pathlib import Path
11
12 import numpy as np
13 import scipy
14 from openpyxl import load_workbook
15 from scipy.optimize import Bounds, LinearConstraint, linprog, milp
16 from scipy.sparse import csc_matrix
17
18
19 ROOT = Path(__file__).resolve().parents[2]

```

```

20 BASE = ROOT / "docs/CUMCM2026Problems/C题/附件"
21 MANUSCRIPT = Path("/Users/slumpyfufu/Library/Containers/com.tencent.xinWeChat/Data/Documents/
    xwechat_files/wxid_upurjlc0hwme22_9950/temp/drag/Q1_精简修订版_公式兼容Obsidian.md")
22 # Captured on the first successful run, before WeChat's temporary file disappeared.
23 MANUSCRIPT_SHA256 = "84bac79a12fb2a3db018b230b9d93653fb3154ef1e48682ad7f209816a8634b3"
24
25
26 def solve(p, load, pv, physical=False, integer=False, ec=0.9, ed=0.9,
27           q=5000/6, lower=1200, upper=10800, initial=6000,
28           service="both", internal_power=False, epsilon=0, free_x=False,
29           range_index=None):
30     n = len(p)
31     names = ["gL", "gB", "vL", "vB", "z", "u"] if physical else ["x", "y", "z"]
32     index = {name: np.arange(k*n, (k+1)*n) for k, name in enumerate(names)}
33     index["E"] = np.arange(len(names)*n, (len(names)+1)*n+1)
34     size = (len(names)+1)*n+1
35     if integer:
36         index["b"] = np.arange(size, size+n)
37         size += n
38     lo, hi = np.zeros(size), np.full(size, np.inf)
39     lo[index["E"]], hi[index["E"]] = lower, upper
40     lo[index["E"]][0, -1]] = initial
41     hi[index["E"]][0, -1]] = initial
42     if integer:
43         hi[index["b"]] = 1
44     if free_x:
45         lo[index["x"]] = -np.inf
46     charge = ["gB", "vB"] if physical else ["x", "y"]
47     d, r = np.maximum(load-pv, 0), np.maximum(pv-load, 0)
48     if not physical:
49         hi[index["y"]], hi[index["z"]] = r, d
50         if service in ("grid", "none"):
51             hi[index["y"]] = 0
52         if service in ("pv", "none"):
53             hi[index["x"]] = 0
54         if service == "none":
55             hi[index["z"]] = 0
56     cost, throughput = np.zeros(size), np.zeros(size)
57     if physical:
58         cost[index["gL"]], cost[index["gB"]] = p, p
59         constant = 0
60     else:
61         cost[index["x"]], cost[index["z"]] = p, -p
62         constant = float(p @ d)
63     for name in charge:
64         throughput[index[name]] = ec
65     throughput[index["z"]] = 1/ed
66     eq, rhs, ub, limits = [], [], [], []
67
68     def row(terms):
69         result = np.zeros(size)
70         for column, value in terms:
71             result[column] = value
72         return result
73
74     for t in range(n):
75         if physical:

```

```

76     eq.append(row([(index[k][t], 1) for k in ["gL", "vL", "z"]]))
77     rhs.append(load[t])
78     eq.append(row([(index[k][t], 1) for k in ["vL", "vB", "u"]]))
79     rhs.append(pv[t])
80     eq.append(row([(index["E"][t+1], 1), (index["E"][t], -1),
81                    (index["z"][t], 1/ed)] + [(index[k][t], -ec) for k in charge]))
82     rhs.append(0)
83     qc, qd = (q/ec, q*ed) if internal_power else (q, q)
84     ub.append(row([(index[k][t], 1) for k in charge]
85                  + [(index["b"][t], -qc)] if integer else []))
86     limits.append(0 if integer else qc)
87     ub.append(row([(index["z"][t], 1)]
88                  + [(index["b"][t], qd)] if integer else []))
89     limits.append(qd)
90     eq, rhs, ub, limits = map(np.array, (eq, rhs, ub, limits))
91     integrality = np.zeros(size)
92     if integer:
93         integrality[index["b"]] = 1
94
95     def optimize(objective, extra_eq=None, extra_rhs=None, cost_cap=None):
96         aeq = eq if extra_eq is None else np.vstack([eq, extra_eq])
97         beq = rhs if extra_rhs is None else np.r_[rhs, extra_rhs]
98         aub = ub if cost_cap is None else np.vstack([ub, cost])
99         bub = limits if cost_cap is None else np.r_[limits, cost_cap]
100        if integer:
101            result = milp(objective, integrality=integrality, bounds=Bounds(lo, hi),
102                          constraints=[LinearConstraint(csc_matrix(aeq), beq, beq),
103                                       LinearConstraint(csc_matrix(aub), -np.inf, bub)],
104                          options={"mip_rel_gap": 1e-10, "time_limit": 30})
105        else:
106            result = linprog(objective, A_eq=aeq, b_eq=beq, A_ub=aub, b_ub=bub,
107                             bounds=list(zip(lo, hi)), method="highs")
108        if not result.success:
109            raise RuntimeError(result.message)
110        return result
111
112    first = optimize(cost)
113    second = optimize(throughput, cost, first.fun) if epsilon == 0 else optimize(throughput,
114    cost_cap=first.fun+epsilon)
115    solution = second.x
116    c = sum(solution[index[k]] for k in charge)
117    z, e = solution[index["z"]], solution[index["E"]]
118    grid = solution[index["gL"]]+solution[index["gB"]] if physical else d-z+solution[index["x"]]
119    curt = solution[index["u"]] if physical else r-solution[index["y"]]
120    summary = {"primary_cost": float(first.fun+constant), "cost": float(cost @ solution+constant),
121              "internal_throughput": float(throughput @ solution), "grid_kwh": float(grid.sum()),
122              "charge_kwh": float(c.sum()), "discharge_kwh": float(z.sum()),
123              "curtail_kwh": float(curt.sum()), "state_min": float(e.min()), "state_max": float(e.
124    max()),
125              "state_initial": float(e[0]), "state_final": float(e[-1]),
126              "max_equation_residual": float(np.max(np.abs(eq @ solution-rhs))),
127              "max_inequality_violation": float(max(0, np.max(ub @ solution-limits))),
128              "max_bound_violation": float(max(0, np.max(lo-solution), np.max(solution-hi))),
129              "max_simultaneous_kwh": float(np.minimum(c,z).max()),
130              "whole_day_energy_residual": float(abs(grid.sum()+pv.sum()-curt.sum()-load.sum()-(1-
131    ec)*c.sum()-(1/ed-1)*z.sum()))}
132    if integer:

```

```

130     summary["primary_mip_gap"] = float(first.mip_gap)
131 if range_index is not None:
132     name, t = range_index
133     objective = row([(index[name][t], 1)])
134     a = np.vstack([cost, throughput])
135     b = np.array([first.fun, second.fun])
136     summary["optimal_face_range"] = [float(optimize(objective, a, b).fun),
137                                     float(-optimize(-objective, a, b).fun)]
138 return summary, {"grid": grid.tolist(), "charge": c.tolist(), "discharge": z.tolist(),
139                 "state": e.tolist(), "curtail": curt.tolist()}
140
141
142 def main():
143     records = list(load_workbook(BASE/"附件1.xlsx", read_only=True, data_only=True).active.values
144                    [1:])
145     a = np.array([r[1:] for r in records])
146     p, load, pv = a[:, 0], a[:, 1]/6, a[:, 2]/6
147     report = {"scipy_version": scipy.__version__, "assumptions": __doc__,
148              "manuscript_sha256_at_first_read": {str(MANUSCRIPT): MANUSCRIPT_SHA256},
149              "sha256": {str(f): hashlib.sha256(f.read_bytes()).hexdigest()
150                          for f in [BASE/"附件1.xlsx", BASE/"附件5/result1.xlsx"]},
151              "input": {"n": len(p), "price_min": float(p.min()), "price_max": float(p.max()),
152                        "surplus_kwh": float(np.maximum(pv-load, 0).sum())}, "cases": {}, "schedules
153              ": {}}
154     surplus_slot = int(np.argmax(pv-load))
155     cases = {"physical_milp": {"physical": True, "integer": True},
156             "canonical_milp": {"integer": True}, "canonical_lp": {"range_index": ("y", surplus_slot
157             )},
158             "no_storage": {"service": "none"}, "pv_only": {"service": "pv"},
159             "grid_only": {"service": "grid"}, "roundtrip90_symmetric": {"ec": np.sqrt(.9), "ed": np
160             .sqrt(.9)},
161             "roundtrip90_c90_d100": {"ec": .9, "ed": 1}, "roundtrip90_c100_d90": {"ec": 1, "ed":
162             .9},
163             "power_internal": {"internal_power": True}, "epsilon_1e_6_yuan": {"epsilon": 1e-6},
164             "missing_x_lower_bound": {"free_x": True}}
165     for name, options in cases.items():
166         report["cases"][name], report["schedules"][name] = solve(p, load, pv, **options)
167     report["cases"]["shift_all_10min"], _ = solve(np.roll(p,1), np.roll(load,1), np.roll(pv,1))
168     report["range_interval_endpoint"] = str(records[surplus_slot][0])
169     rng = np.random.default_rng(20260910)
170     stress = []
171     for trial in range(30):
172         sp = rng.choice([0., .2, 1.], 6)
173         sl, sv = rng.uniform(0, 8, (2, 6))
174         opts = {"q": 4, "lower": 1, "upper": 10, "initial": float(rng.choice([1, 5, 10]))}
175         physical, _ = solve(sp, sl, sv, physical=True, integer=True, **opts)
176         canonical, _ = solve(sp, sl, sv, **opts)
177         stress.append({"trial": trial, "prices": sp.tolist(), "load": sl.tolist(), "pv": sv.tolist()
178         ,
179                       "options": opts, "cost_gap": abs(physical["cost"]-canonical["cost"]),
180                       "throughput_gap": abs(physical["internal_throughput"]-canonical["
181                       internal_throughput"])}))
182     report["stress_cases"] = stress
183     report["synthetic_nonunique"], _ = solve(
184         np.array([.1, .1, 1.]), np.array([0., 0., 1.]), np.zeros(3),
185         q=2, lower=0, upper=2, initial=0, range_index=("x", 0))
186     target = ROOT/"outputs/q1/raw/q1_compact_review_evidence.json"

```

```

180     target.parent.mkdir(parents=True, exist_ok=True)
181     target.write_text(json.dumps(report, ensure_ascii=False, indent=2)+"\n", encoding="utf-8")
182     print(json.dumps({"cases": report["cases"], "input": report["input"],
183                     "range_interval_endpoint": report["range_interval_endpoint"],
184                     "stress_count": len(stress),
185                     "max_stress_cost_gap": max(s["cost_gap"] for s in stress),
186                     "max_stress_throughput_gap": max(s["throughput_gap"] for s in stress)}),
187           ensure_ascii=False, indent=2))
188
189
190 if __name__ == "__main__":
191     main()

```

src/q1/run.py

```

1  """一条命令: 附件→MILP→分析→CSV→回读→当前代码测试→原子发布。"""
2
3  import argparse
4  import datetime as dt
5  import hashlib
6  import platform
7  import shutil
8  import sys
9  import tempfile
10 from pathlib import Path
11
12 import numpy as np
13 import openpyxl
14 import scipy
15
16 from export import export_csv_output
17 from model import (DT, TOLERANCES, E_INITIAL, E_MAX, E_MIN, Q_MAX, ROOT, SCENARIOS,
18                   arrays, check_solution, read_inputs, schedule_rows,
19                   solve, summary, write_csv, write_json)
20 from report import create_report
21 from analysis import ANALYSIS_SETTINGS, run_analysis
22 from analysis_report import create_analysis_report
23 from traceability import generate_traceability
24 from pipeline import publish_generation, render_figures, run_tests
25
26
27 def sha256(path):
28     return hashlib.sha256(path.read_bytes()).hexdigest()
29
30
31 def run_pipeline(root=ROOT, plots=True):
32     root = Path(root).resolve()
33     started = dt.datetime.now(dt.timezone.utc).isoformat()
34     source = root / "docs/CUMCM2026Problems/C题/附件"
35     generations = root / "outputs/.q1-runs"
36     generations.mkdir(parents=True, exist_ok=True)
37     staging = Path(tempfile.mkdtemp(prefix="run-", dir=generations))
38     try:
39         input_raw, input_processed = staging / "inputs/raw", staging / "inputs/processed"
40         output_raw, output, documents = staging / "raw", staging / "processed", staging / "docs"
41         for directory in (input_raw, input_processed, output_raw, output, documents):

```

```

42     directory.mkdir(parents=True)
43     provenance = []
44     for name, original in {"attachment1.xlsx": source / "附件1.xlsx"}.items():
45         copied = input_raw / name
46         shutil.copy2(original, copied)
47         if sha256(original) != sha256(copied):
48             raise ValueError("原始输入副本校验失败")
49         provenance.append({"source": str(original.relative_to(root)),
50                           "copy": f"inputs/q1/raw/{name}", "sha256": sha256(copied)})
51     data = read_inputs(input_raw / "attachment1.xlsx")
52     write_csv(input_raw / "attachment1.csv", [{k: r[k] for k in
53         ("source_endpoint", "price_yuan_per_kwh", "load_kw", "pv_kw")} for r in data])
54     write_csv(input_processed / "timeseries.csv", data)
55     params = {"model_source": "docs/1/final/第一问_MILP_边际价值强化版.md", "period_count": 144,
56              "dt_hours": DT, "max_power_kw": 5000, "max_energy_kwh": Q_MAX,
57              "max_energy_precision_decision": "5000×1/6准确计算及验收; 833.3333仅用于显示",
58              "rated_capacity_kwh": 12000, "min_energy_kwh": E_MIN, "max_storage_kwh": E_MAX,
59              "initial_and_terminal_kwh": E_INITIAL, "tolerances": TOLERANCES,
60              "efficiency_scenarios": SCENARIOS, "delivery_format": "用户明确要求仅CSV; review中的
XLSX建议不采用",
61              "analysis_settings": ANALYSIS_SETTINGS,
62              "assumptions": ["右端点标记", "10分钟内零阶保持", "充放电量为微网母线侧",
63                             "主模型双单程效率均0.9; 函数支持独立效率", "不售电且允许弃光",
64                             "只最小化购电费, 无二级目标", "无经济显著性门槛时不判断投资意义"]}
65     write_json(input_processed / "parameters.json", params)
66     write_json(input_raw / "manifest.json", provenance)
67     solutions, validation, summaries = {}, {}, []
68     for name, efficiency in SCENARIOS.items():
69         solution = solve(data, efficiency)
70         validation[name] = check_solution(data, solution)
71         solutions[name] = solution
72         row = {"scenario": name, **summary(data, solution)}
73         summaries.append(row)
74         write_json(output_raw / f"solution_{name}.json", solution)
75         write_csv(output_raw / f"schedule_{name}.csv", schedule_rows(data, solution))
76         print(f"{name}: 费用={row['cost_yuan']:.9f}元; MIP gap={solution['solver']['mip_gap']}; 物
理检查通过", flush=True)
77     analysis = run_analysis(data, solutions, output_raw, output)
78     validation["csv"] = export_csv_output(output, data, solutions["main"])
79     write_json(output_raw / "validation.json", validation)
80     write_csv(output / "efficiency_comparison.csv", summaries)
81     write_json(output / "summary.json", summaries[0])
82     figure_status = render_figures(data, solutions, summaries, analysis, staging, enabled=plots)
83     write_json(output_raw / "figure_status.json", figure_status)
84     create_report(data, solutions, summaries, validation, output, documents, figure_status)
85     create_analysis_report(analysis, output, documents, figure_status)
86     # 旧稿审查证据不属于本次求解; 原样保留并在manifest标明, 避免历史链接失效。
87     historical = []
88     for path in sorted((root / "outputs/q1/raw").glob("*review*.json")):
89         shutil.copy2(path, output_raw / path.name)
90         historical.append({"file": path.name, "sha256": sha256(path), "role": "
historical_review_only"})
91     run_tests(staging)
92     p, load, pv = arrays(data)
93     record = {"started_utc": started, "finished_utc": dt.datetime.now(dt.timezone.utc).isoformat
94     (),
95             "generation": staging.name, "python": sys.version, "platform": platform.platform())

```



```

95         "command": "uv run --with-requirements src/q1/requirements.txt python src/q1/run.
py",
96         "versions": {"numpy": np.__version__, "scipy": scipy.__version__, "openpyxl":
openpyxl.__version__},
97         "inputs": provenance, "model_sha256": sha256(root / params["model_source"]),
98         "code_sha256": {str(f.relative_to(ROOT)): sha256(f) for f in sorted([*(ROOT / "src
/q1").glob("*.py"), ROOT / "src/plots/q1_legacy.py"])},
99         "input_statistics": {"records": len(data), "price_min": float(p.min()), "price_max
": float(p.max()),
100                                "load_kwh": float(load.sum()), "pv_kwh": float(pv.sum())},
101         "output_sha256": {str(f.relative_to(staging)): sha256(f) for f in sorted(staging.
rglob("*")) if f.is_file()},
102         "historical_artifacts": historical, "all_physical_csv_checks_passed": True,
103         "analysis_checks_passed": analysis["all_checks_passed"], "analysis_scenarios": len
(analysis["scenarios"]),
104         "tests_passed": True, "figure_status": figure_status,
105         "publication": "single atomic q1_current symlink replacement after all numerical
gates"}
106     write_json(output_raw / "run_manifest.json", record)
107     generate_traceability(staging)
108     record["output_sha256"] = {str(f.relative_to(staging)): sha256(f) for f in sorted(staging.
rglob("*"))
109                                if f.is_file() and f != output_raw / "run_manifest.json"}
110     write_json(output_raw / "run_manifest.json", record)
111     publish_generation(staging, root)
112 except BaseException:
113     # 从未发布的generation移除; 上一次正式结果保持可用。
114     if (root / "outputs/q1_current").resolve() != staging:
115         shutil.rmtree(staging)
116     raise
117 print(f"CSV回读和当前代码测试通过, 已发布: {root / 'outputs/processed/q1/result1.csv'}", flush=True)
118 print(f"绘图状态: {figure_status['status']}; {figure_status['message']}", flush=True)
119 return staging
120
121
122 if __name__ == "__main__":
123     parser = argparse.ArgumentParser()
124     parser.add_argument("--no-plots", action="store_true", help="只生成数值、可读报告和提交文件")
125     args = parser.parse_args()
126     run_pipeline(plots=not args.no_plots)

```

src/q1/test_analysis.py

```

1  """新版分析验收: 解析边际量、真实情景、独立重算和反向篡改。"""
2
3  import copy
4  import itertools
5  import json
6  import tempfile
7  import unittest
8  from pathlib import Path
9
10 import numpy as np
11
12 from analysis import (active_sets, checked_analysis_csv, local_marginals,

```



```

13         rhs_validation, scaled_data, verify_lp)
14 from export import read_csv
15 from model import (DT, PHYS_TOL_KWH, COST_TOL_YUAN, ARTIFACT_ROOT, E_MAX, ROOT, SCENARIOS, arrays,
16                     build_problem,
17                     check_solution, read_inputs, solve, solve_lp, summary)
18
19 class MarginalAcceptance(unittest.TestCase):
20     @classmethod
21     def setUpClass(cls):
22         cls.data = read_inputs(ARTIFACT_ROOT / 'inputs/raw/attachment1.xlsx')
23         cls.raw = json.loads((ARTIFACT_ROOT / 'raw/analysis_solutions.json').read_text(encoding='utf
24 -8'))
25         cls.a = json.loads((ARTIFACT_ROOT / 'processed/analysis_summary.json').read_text(encoding='
26 utf-8'))
27
28     def test_all_36_scenarios_and_432_relaxations(self):
29         expected = set(itertools.product(SCENARIOS, [-.05,0,.05], [-.05,0,.05]))
30         actual = {(r['efficiency_name'],r['load_delta'],r['pv_delta']) for r in self.a['scenarios']}
31         self.assertEqual(actual, expected)
32         count = 0
33         for row in self.a['scenarios']:
34             sraw = self.raw['scenarios'][row['scenario']]
35             data = scaled_data(self.data,row['load_delta'],row['pv_delta'])
36             s = sraw['solution']
37             self.assertTrue(check_solution(data,s)['passed'])
38             self.assertEqual(active_sets(s), sraw['active_sets'])
39             for key,value in summary(data,s).items():
40                 self.assertAlmostEqual(row[key], value, delta=COST_TOL_YUAN if key == "cost_yuan"
41 else PHYS_TOL_KWH)
42             for name,record in sraw['perturbations'].items():
43                 changed = record['solution']
44                 self.assertTrue(check_solution(data,changed)['passed'])
45                 original = s['settings']
46                 settings = changed['settings']
47                 changed_keys = [k for k in original if settings[k] != original[k]]
48                 self.assertEqual(len(changed_keys),1)
49                 parameter = changed_keys[0]
50                 self.assertIn(parameter,['e_min','e_max','charge_kw','discharge_kw'])
51                 step = abs(settings[parameter] - original[parameter])
52                 values = next(v for v in self.a['bottlenecks'] if v['scenario']==row['scenario']
53 and v['resource']==name.rsplit('_',1)[0] and v['increment']==step)
54                 saving = s['solver']['objective_yuan'] - changed['solver']['objective_yuan']
55                 self.assertAlmostEqual(values['value_per_unit_day'],saving/step,delta=PHYS_TOL_KWH)
56                 self.assertGreaterEqual(saving,-PHYS_TOL_KWH)
57                 self.assertAlmostEqual(values['increment_period_kwh'],step if parameter in ('e_min',
58 'e_max') else step/6,delta=PHYS_TOL_KWH)
59                 count += 1
60         self.assertEqual(count,432)
61
62     def test_nested_baselines_and_analytic_no_ess(self):
63         p, load, pv = arrays(self.data)
64         s0, spv, sfull = (self.raw[k] for k in ('no_ess','pv_only','full'))
65         for s in (s0,spv,sfull):
66             self.assertTrue(check_solution(self.data,s)['passed'])
67             self.assertAlmostEqual(s0['solver']['objective_yuan'],p @ np.maximum(load-pv,0),delta=
68 PHYS_TOL_KWH)

```

```

64     self.assertTrue(np.all(np.array(spv['C'])<=np.maximum(pv-load,0)+PHYS_TOL_KWH))
65     self.assertTrue(np.all(np.array(spv['D'])<=np.maximum(load-pv,0)+PHYS_TOL_KWH))
66     self.assertLessEqual(sfull['solver']['objective_yuan'],spv['solver']['objective_yuan']+
PHYS_TOL_KWH)
67     b=self.a['benefits']
68     self.assertAlmostEqual(b['pv_increment_yuan']+b['flex_increment_yuan'],b['total_saving_yuan']
],delta=PHYS_TOL_KWH)
69
70 def test_lp_relaxation_keeps_big_m_coupling(self):
71     problem=build_problem(self.data,.9)
72     s=self.raw['lp_relaxation']
73     x=np.concatenate([s['variables'][v] for v in problem['index']])
74     lhs=problem['a']@x
75     self.assertTrue(np.all(lhs>=problem['lb']-PHYS_TOL_KWH))
76     self.assertTrue(np.all(lhs<=problem['ub']+PHYS_TOL_KWH))
77     self.assertTrue(np.all(x>=problem['lower']-PHYS_TOL_KWH))
78     self.assertTrue(np.all(x<=problem['upper']+PHYS_TOL_KWH))
79     self.assertAlmostEqual(problem['objective']@x,s['objective_yuan'],delta=PHYS_TOL_KWH)
80     u=np.array(s['variables']['u']); c=np.array(s['variables']['C']); d=np.array(s['variables']['D'])
81     self.assertEqual(self.a['lp_diagnosis']['fractional_modes'],np.sum((u>PHYS_TOL_KWH)&(u<1-
PHYS_TOL_KWH)))
82     self.assertEqual(self.a['lp_diagnosis']['simultaneous_periods'],np.sum((c>PHYS_TOL_KWH)&(d>
PHYS_TOL_KWH)))
83     self.assertLessEqual(s['objective_yuan'],self.a['lp_diagnosis']['milp_cost_yuan']+
PHYS_TOL_KWH)
84
85 def test_analytic_shadow_sign_and_price_threshold(self):
86     # 两时段、首段低价、末段100kWh负荷。初末均6000，充放边界不活跃。
87     data=[{'price_yuan_per_kwh':.1,'load_kwh':0.,'pv_kwh':0.,'interval':'00:00-00:10'},
88           {'price_yuan_per_kwh':1.,'load_kwh':100.,'pv_kwh':0.,'interval':'00:10-00:20'}]
89     s=solve(data,.9)
90     rows,problem,result,checks=local_marginals(data,s,'analytic')
91     self.assertAlmostEqual(result.fun,100*.1/.81,delta=PHYS_TOL_KWH)
92     self.assertAlmostEqual(rows[0]['mu_yuan_per_kwh'],.1,delta=PHYS_TOL_KWH)
93     self.assertAlmostEqual(rows[1]['mu_yuan_per_kwh'],.1/.81,delta=PHYS_TOL_KWH)
94     self.assertAlmostEqual(rows[0]['lambda_yuan_per_kwh'],.1/.9,delta=PHYS_TOL_KWH)
95     self.assertAlmostEqual(rows[1]['lambda_yuan_per_kwh'],.1/.9,delta=PHYS_TOL_KWH)
96     self.assertEqual(len(rhs_validation(problem,result)),4)
97     bad=copy.deepcopy(result)
98     bad.eqlin.marginals *=-1
99     with self.assertRaises(ValueError):
100         verify_lp(problem,bad)
101     # 无效率套利收益时不能强行充放：0.9*.81小于当前价1。
102     data[0]['price_yuan_per_kwh']=1.; data[1]['price_yuan_per_kwh']=.9
103     no_gain=solve(data,.9)
104     self.assertAlmostEqual(sum(no_gain['C']),0,delta=PHYS_TOL_KWH)
105     self.assertAlmostEqual(sum(no_gain['D']),0,delta=PHYS_TOL_KWH)
106
107 def test_actual_rhs_slopes_and_local_thresholds(self):
108     rows=read_csv(ARTIFACT_ROOT/'processed/marginal_rhs_validation.csv')
109     self.assertEqual(len(rows),288)
110     self.assertEqual({(r['kind'],int(r['t'])) for r in rows},set(itertools.product(['bus_demand'
,'internal_injection'],range(1,145))))
111     for r in rows:
112         shadow=float(r['shadow_dJ_db'])
113         if r['minus_slope']:

```

```

114         self.assertLessEqual(float(r['minus_slope']), shadow+PHYS_TOL_KWH)
115         if r['plus_slope']:
116             self.assertLessEqual(shadow, float(r['plus_slope'])+PHYS_TOL_KWH)
117         local=read_csv(ARTIFACT_ROOT/'processed/marginal_values.csv')
118         self.assertEqual(len(local), 36*144)
119         by_key={r['scenario']:r for r in self.a['scenarios']}
120         for r in local:
121             eta=by_key[r['scenario']]['eta_c']
122             self.assertAlmostEqual(float(r['charge_threshold']), eta*float(r['lambda_yuan_per_kwh']),
123                                     delta=PHYS_TOL_KWH)
124             self.assertAlmostEqual(float(r['discharge_threshold']), float(r['lambda_yuan_per_kwh'])/
125                                     eta, delta=PHYS_TOL_KWH)
126             if r['actual_action']=='charge':
127                 self.assertGreaterEqual(float(r['charge_gain']), -PHYS_TOL_KWH)
128             if r['actual_action']=='discharge':
129                 self.assertGreaterEqual(float(r['discharge_gain']), -PHYS_TOL_KWH)
130
131     def test_full_input_output_csv_and_normalized_sensitivity(self):
132         rows=read_csv(ARTIFACT_ROOT/'raw/analysis_schedules.csv')
133         self.assertEqual(len(rows), 36*144)
134         grouped={key:[] for key in self.raw['scenarios']}
135         for row in rows:
136             grouped[row['scenario']].append(row)
137         for case in self.a['scenarios']:
138             data=scaled_data(self.data, case['load_delta'], case['pv_delta'])
139             s=self.raw['scenarios'][case['scenario']]['solution']
140             for i, r in enumerate(grouped[case['scenario']]):
141                 for k in data[i]:
142                     self.assertEqual(r[k], str(data[i][k]))
143                     for k in ('G', 'C', 'D', 'E', 'W', 'u'):
144                         self.assertAlmostEqual(float(r[k]), s[k][i], delta=PHYS_TOL_KWH)
145             for r in self.a['normalized_sensitivity']:
146                 self.assertAlmostEqual(r['normalized_sensitivity'],
147                                         (r['plus_cost_yuan']-r['minus_cost_yuan'])/(.1*r['base_cost_yuan']
148                                         )), delta=PHYS_TOL_KWH)
149
150     def test_independent_power_limits_and_csv_nonfinite_rejected(self):
151         a=build_problem(self.data, .9, charge_kw=5001)
152         b=build_problem(self.data, .9, discharge_kw=5001)
153         n=len(self.data); u=a['index']['u'][0]
154         self.assertAlmostEqual(a['a'][2*n, u], -5001/6, delta=PHYS_TOL_KWH)
155         self.assertAlmostEqual(a['a'][3*n, u], 5000/6, delta=PHYS_TOL_KWH)
156         self.assertAlmostEqual(b['a'][2*n, u], -5000/6, delta=PHYS_TOL_KWH)
157         self.assertAlmostEqual(b['a'][3*n, u], 5001/6, delta=PHYS_TOL_KWH)
158         with tempfile.TemporaryDirectory() as tmp:
159             for value in (float('nan'), float('inf')):
160                 with self.assertRaises(ValueError):
161                     checked_analysis_csv(Path(tmp)/'bad.csv', [{'value':value}])
162
163 if __name__ == "__main__":
164     unittest.main(verbosity=2)

```

src/q1/test_q1.py

```
1 """针对时间错位、约束遗漏及输出篡改的独立验收：合成数据仅用于测试。"""
```

```

2
3 import copy
4 import csv
5 import datetime as dt
6 import json
7 import tempfile
8 import unittest
9 from pathlib import Path
10
11 import numpy as np
12 from openpyxl import load_workbook
13
14 from export import check_csv_output, read_csv
15 from model import (PHYS_TOL_KWH, COST_TOL_YUAN, ARTIFACT_ROOT, ROOT, SCENARIOS, arrays,
16                    check_solution, endpoint_minutes,
17                    read_inputs, solve, write_csv)
18
19 class Q1Acceptance(unittest.TestCase):
20     @classmethod
21     def setUpClass(cls):
22         cls.input_path = ARTIFACT_ROOT / "inputs/raw/attachment1.xlsx"
23         cls.data = read_inputs(cls.input_path)
24         cls.solution = solve(cls.data, .9) # 当前代码重求, CSV必须与本轮结果对应。
25         cls.output = ARTIFACT_ROOT / "processed"
26
27     def test_all_actual_scenarios(self):
28         for key, efficiency in SCENARIOS.items():
29             with self.subTest(scenario=key):
30                 s = json.loads((ARTIFACT_ROOT / f"raw/solution_{key}.json").read_text(encoding="utf
31 -8"))
32                 self.assertEqual(s["eta_c"], efficiency)
33                 self.assertEqual(s["eta_d"], efficiency)
34                 self.assertTrue(check_solution(self.data, s)["passed"])
35
36     def test_time_and_unit_examples(self):
37         self.assertEqual(self.data[0]["interval"], "00:00-00:10")
38         self.assertEqual(self.data[-1]["interval"], "23:50-24:00")
39         self.assertEqual(self.data[60]["interval"], "10:00-10:10")
40         self.assertEqual(endpoint_minutes(self.data[60]["source_endpoint"]), 610)
41         self.assertAlmostEqual(self.data[0]["load_kwh"], 3439.8466 / 6, places=10)
42         self.assertEqual(len([r["interval"] for r in self.data]), 144)
43         for value in (dt.time(0, 10), dt.timedelta(minutes=10), 10/1440, "00:10"):
44             self.assertEqual(endpoint_minutes(value), 10)
45         for value in (1.0, dt.timedelta(days=1), "0:00+1", "24:00"):
46             self.assertEqual(endpoint_minutes(value), 1440)
47
48     def test_corrupted_inputs_rejected(self):
49         mutations = {"shifted_endpoint": ("A2", "00:20"), "duplicate_endpoint": ("A3", "00:10"),
50                     "missing_load": ("C2", None), "negative_pv": ("D2", -1),
51                     "negative_price": ("B2", -1), "numeric_text": ("B2", "0.4248"),
52                     "invalid_minute": ("A2", "00:70")}
53         with tempfile.TemporaryDirectory() as tmp:
54             for name, (cell, value) in mutations.items():
55                 with self.subTest(case=name):
56                     wb = load_workbook(self.input_path)
57                     wb.active[cell] = value

```

```

57         path = Path(tmp) / f"{name}.xlsx"
58         wb.save(path)
59         wb.close()
60         with self.assertRaises(ValueError):
61             read_inputs(path)
62     wb = load_workbook(self.input_path)
63     wb.active.delete_rows(145)
64     path = Path(tmp) / "missing_row.xlsx"
65     wb.save(path)
66     wb.close()
67     with self.assertRaises(ValueError):
68         read_inputs(path)
69
70     def test_corrupted_schedules_rejected(self):
71         cases = [(("G", 0, -1), ("C", 0, 1000), ("D", 0, 1), ("E", 0, 12001),
72                  ("W", 0, 1), ("u", 0, 0.5), ("E", 143, 5999), ("G", 10, float("nan")))]
73     for name, index, value in cases:
74         with self.subTest(variable=name, index=index):
75             s = copy.deepcopy(self.solution)
76             s[name][index] = value
77             with self.assertRaises(ValueError):
78                 check_solution(self.data, s)
79     for key, value in (("objective_yuan", 0), ("dual_bound_yuan", 0), ("status", 1),
80                       ("mip_gap", float("nan"))):
81         with self.subTest(solver_field=key):
82             s = copy.deepcopy(self.solution)
83             s["solver"][key] = value
84             with self.assertRaises(ValueError):
85                 check_solution(self.data, s)
86     s = copy.deepcopy(self.solution)
87     s["E0"] = 5000
88     with self.assertRaises(ValueError):
89         check_solution(self.data, s)
90
91     def test_csv_roundtrip_and_tamper_rejection(self):
92         self.assertTrue(check_csv_output(self.output, self.data, self.solution)["passed"])
93         cases = [(("result1.csv", 0, "interval", "00:10-00:20"),
94                  ("result1.csv", 0, "G_kwh", -1), ("result1.csv", 0, "C_kwh", 1000),
95                  ("result1.csv", 0, "G_kwh", "nan"), ("result1.csv", 0, "E_start_kwh", 5999),
96                  ("result1.csv", 2, "soc_fraction", 0), ("result1.csv", 0, "cost_yuan", 0),
97                  ("table2_storage.csv", 0, "charge_kwh", 1),
98                  ("daily_summary.csv", 0, "terminal_kwh", 5999),
99                  ("daily_summary.csv", 0, "load_kwh", "nan"),
100                 ("result1.csv", 0, "cost_yuan", "nan"),
101                 ("table1_purchase.csv", 0, "grid_kwh", 1)]
102         cases.append(("table1_purchase.csv", 3, "grid_kwh", "nan"))
103         files = ["result1.csv", "table1_purchase.csv", "table2_storage.csv", "daily_summary.csv"]
104         with tempfile.TemporaryDirectory() as tmp:
105             directory = Path(tmp)
106             for file, index, field, value in cases:
107                 with self.subTest(file=file, field=field):
108                     for name in files:
109                         write_csv(directory / name, read_csv(self.output / name))
110                     rows = read_csv(directory / file)
111                     rows[index][field] = value
112                     write_csv(directory / file, rows)
113                     with self.assertRaises(ValueError):

```

```

114         check_csv_output(directory, self.data, self.solution)
115         # 保留总和, 交换两个时段的购电量, 也必须被发现。
116         for name in files:
117             write_csv(directory / name, read_csv(self.output / name))
118         rows = read_csv(directory / "result1.csv")
119         rows[0]["G_kwh"], rows[18]["G_kwh"] = rows[18]["G_kwh"], rows[0]["G_kwh"]
120         write_csv(directory / "result1.csv", rows)
121         with self.assertRaises(ValueError):
122             check_csv_output(directory, self.data, self.solution)
123
124     def test_analytical_arbitrage_case(self):
125         # 专用合成测试: 首段0.1元, 其余1元; 末段负荷100kWh, 无光伏。
126         # 最优需首段购电100/(0.9^2), 末段放电100; 准确费用1000/81。
127         synthetic = [{"price_yuan_per_kwh": 0.1 if t == 0 else 1.0,
128                      "load_kwh": 100.0 if t == 143 else 0.0, "pv_kwh": 0.0} for t in range(144)]
129         s = solve(synthetic, 0.9)
130         self.assertTrue(check_solution(synthetic, s)["passed"])
131         self.assertAlmostEqual(s["solver"]["objective_yuan"], 1000 / 81, delta=PHYS_TOL_KWH)
132         self.assertAlmostEqual(sum(s["C"]), 100 / 0.81, delta=PHYS_TOL_KWH)
133         self.assertAlmostEqual(sum(s["D"]), 100, delta=PHYS_TOL_KWH)
134
135     def test_curtailment_is_available_when_absorption_is_limited(self):
136         # 专用压力测试: 每段2000kWh光伏且零负荷。日周期+互斥禁止无限吸收。
137         synthetic = [{"price_yuan_per_kwh": 1.0, "load_kwh": 0.0, "pv_kwh": 2000.0} for _ in range
138                      (144)]
139         s = solve(synthetic, 0.9)
140         self.assertTrue(check_solution(synthetic, s)["passed"])
141         self.assertAlmostEqual(sum(s["G"]), 0, delta=PHYS_TOL_KWH)
142         self.assertGreater(sum(s["W"]), 0)
143
144     def test_export_csv_matches_all_variables_and_inputs(self):
145         with (ARTIFACT_ROOT / "processed/result1.csv").open(encoding="utf-8", newline="") as stream:
146             rows = list(csv.DictReader(stream))
147             self.assertEqual(len(rows), 144)
148             p, _, _ = arrays(self.data)
149             for key in ("G", "C", "D", "E", "W"):
150                 np.testing.assert_allclose([float(r[f"{key}_kwh"]) for r in rows], self.solution[key],
151                 rtol=0, atol=PHYS_TOL_KWH)
152             np.testing.assert_allclose([float(r["u"]) for r in rows], self.solution["u"], rtol=0, atol=
153             PHYS_TOL_KWH)
154             self.assertAlmostEqual(sum(float(r["cost_yuan"]) for r in rows), p @ self.solution["G"],
155             delta=PHYS_TOL_KWH)
156             with (ARTIFACT_ROOT / "inputs/processed/timeseries.csv").open(encoding="utf-8", newline="")
157             as stream:
158                 inputs = list(csv.DictReader(stream))
159                 self.assertEqual(len(inputs), 144)
160                 for i, record in enumerate(inputs):
161                     for key, value in self.data[i].items():
162                         self.assertEqual(record[key], str(value))
163
164 if __name__ == "__main__":
165     unittest.main(verbosity=2)

```

src/q1/test_review.py

```

1  """CR-01 - 13回归：新鲜实算、解析解、文件篡改与故障注入。"""
2
3  import copy
4  import hashlib
5  import json
6  import os
7  import shutil
8  import tempfile
9  import unittest
10 from pathlib import Path
11 from unittest.mock import patch
12
13 import numpy as np
14
15 from analysis import active_sets, bottlenecks, local_marginals
16 from export import check_csv_output, export_csv_output
17 from model import (ARTIFACT_ROOT, ROOT, DT, PHYS_TOL_KWH, COST_TOL_YUAN, BINARY_TOL,
18                   ECONOMIC_VALUE_TOL, arrays, build_problem, check_solution, read_inputs, solve,
19                   solve_milp)
20 from pipeline import ALIASES, publish_generation, render_figures
21 import run
22
23 class ReviewAcceptance(unittest.TestCase):
24     @classmethod
25     def setUpClass(cls):
26         cls.source = ROOT / 'docs/CUMCM2026Problems/C题/附件/附件1.xlsx'
27         cls.data = read_inputs(cls.source)
28         cls.solution = solve(cls.data, .9)
29
30     def test_official_attachment_fresh_solve_golden_and_csv(self):
31         golden = json.loads((ROOT / 'src/q1/golden_main.json').read_text(encoding='utf-8'))
32         self.assertEqual(hashlib.sha256(self.source.read_bytes()).hexdigest(), golden['source_sha256'])
33         self.assertTrue(check_solution(self.data, self.solution)['passed'])
34         self.assertAlmostEqual(self.solution['solver']['objective_yuan'], golden['objective_yuan'],
35                                delta=COST_TOL_YUAN)
36         with tempfile.TemporaryDirectory() as temporary:
37             directory = Path(temporary)
38             check = export_csv_output(directory, self.data, self.solution)
39             self.assertTrue(check['passed'])
40             self.assertFalse(list(directory.glob('*.xlsx')))
41             self.assertAlmostEqual(check['reread_cost_yuan'], golden['objective_yuan'], delta=
42                                    COST_TOL_YUAN)
43
44     def test_independent_efficiencies_analytical_case(self):
45         data = [{'price_yuan_per_kwh': .1, 'load_kwh': 0., 'pv_kwh': 0., 'interval': 'first'},
46                 {'price_yuan_per_kwh': 1., 'load_kwh': 100., 'pv_kwh': 0., 'interval': 'last'}]
47         solution = solve(data, eta_c=.8, eta_d=.95)
48         self.assertTrue(check_solution(data, solution)['passed'])
49         self.assertAlmostEqual(solution['solver']['objective_yuan'], 10 / (.8 * .95), delta=
50                                COST_TOL_YUAN)
51         self.assertAlmostEqual(sum(solution['C']), 100 / (.8 * .95), delta=PHYS_TOL_KWH)
52         rows, _, _, _ = local_marginals(data, solution, 'asymmetric')
53         for row in rows:
54             self.assertAlmostEqual(row['charge_threshold'], .8 * row['lambda_yuan_per_kwh'], delta=

```

```

ECONOMIC_VALUE_TOL)
52     self.assertAlmostEqual(row['discharge_threshold'], row['lambda_yuan_per_kwh'] / .95,
delta=ECONOMIC_VALUE_TOL)
53     for kwargs in ({'eta_c': 0}, {'eta_c': .9, 'eta_d': 1.01}, {'eta_c': .9, 'e_min': 6001}):
54         with self.assertRaises(ValueError):
55             build_problem(data, **kwargs)
56
57 def test_active_sets_use_current_limits_and_lower_space(self):
58     solution = solve(self.data, .9, e_min=5900, e_max=6100, charge_kw=100, discharge_kw=150)
59     active = active_sets(solution)
60     for key, variable, limit in [('E_min', 'E', 5900), ('E_max', 'E', 6100),
61                                ('charge', 'C', 100 * DT), ('discharge', 'D', 150 * DT)]:
62         expected = [i + 1 for i, value in enumerate(solution[variable]) if abs(value - limit) <=
PHYS_TOL_KWH]
63         self.assertEqual(active[key], expected)
64     rows, raw = bottlenecks(self.data, solution, 'custom', [1., 10., 100.])
65     self.assertEqual(len(rows), 12)
66     for row in rows:
67         self.assertIsNone(row['economic_bottleneck'])
68         if row['resource'] == 'storage_lower':
69             self.assertEqual(row['parameter_change'], -row['increment'])
70             changed = raw[f"storage_lower_{row['increment']:g}"]['solution']
71             self.assertEqual(changed['settings']['e_min'], 5900 - row['increment'])
72             self.assertEqual(changed['settings']['e_max'], 6100)
73
74 def test_pv_only_source_with_zero_price_and_surplus(self):
75     data = [{'price_yuan_per_kwh': 0., 'load_kwh': 50., 'pv_kwh': 200.},
76             {'price_yuan_per_kwh': 1., 'load_kwh': 100., 'pv_kwh': 0.}]
77     for baseline in ('pv_only', 'no_ess'):
78         solution = solve(data, .9, baseline=baseline)
79         self.assertTrue(check_solution(data, solution)['passed'])
80         self.assertAlmostEqual(solution['G'][0], 0., delta=PHYS_TOL_KWH)
81         problem = build_problem(data, .9, baseline=baseline)
82         self.assertEqual(problem['upper'][problem['index']['G'][0]], 0)
83         # 强迫富余光伏时外网购电应不可行，不能仅依赖正电价抑制它。
84         problem['lower'][problem['index']['G'][0]] = 1
85         self.assertEqual(solve_milp(problem).status, 2)
86
87 def test_binary_tolerance_is_independent_of_physical(self):
88     solution = copy.deepcopy(self.solution)
89     idle = next(i for i in range(144) if max(solution['C'][i], solution['D'][i]) <= PHYS_TOL_KWH
)
90     solution['u'][idle] = 20 * BINARY_TOL
91     with self.assertRaises(ValueError):
92         check_solution(self.data, solution)
93
94 def test_original_milp_rhs_all_576_results(self):
95     raw = json.loads((ARTIFACT_ROOT / 'raw/analysis_solutions.json').read_text(encoding='utf-8')
)
96     checks = json.loads((ARTIFACT_ROOT / 'raw/analysis_validation.json').read_text(encoding='utf
-8'))['milp_rhs_checks']
97     problem = build_problem(self.data, .9)
98     self.assertEqual(len(raw['milp_rhs_solutions']), 576)
99     self.assertEqual(len(checks), 288)
100    for record in raw['milp_rhs_solutions']:
101        index = record['row']
102        if record['status'] == 2:

```



```

103         self.assertEqual(checks[index][record['side'] + '_status'], 2)
104         continue
105     x = np.concatenate([record['variables'][key] for key in problem['index']])
106     expected = problem['ub'][:288].copy()
107     step = -.01 if record['side'] == 'minus' else .01
108     expected[index] += step
109     np.testing.assert_allclose(problem['a'][:288] @ x, expected, atol=PHYS_TOL_KWH, rtol=0)
110     self.assertTrue(np.all(problem['a'][:288] @ x <= problem['ub'][:288] + PHYS_TOL_KWH))
111     self.assertTrue(np.all(x >= problem['lower'] - PHYS_TOL_KWH))
112     self.assertTrue(np.all(x <= problem['upper'] + PHYS_TOL_KWH))
113     u = np.array(record['variables']['u'])
114     np.testing.assert_allclose(u, np.rint(u), atol=BINARY_TOL, rtol=0)
115     objective = arrays(self.data)[0] @ record['variables']['G']
116     self.assertAlmostEqual(objective, record['dual_bound_yuan'], delta=COST_TOL_YUAN)
117     slope = (objective - self.solution['solver']['objective_yuan']) / step
118     self.assertAlmostEqual(slope, checks[index][record['side'] + '_slope'], delta=
ECONOMIC_VALUE_TOL)
119     self.assertTrue(any(r['base_action'] == 'idle' for r in checks))
120
121
122 class PipelineAcceptance(unittest.TestCase):
123     def test_atomic_pointer_and_failed_commit_preserve_old_generation(self):
124         with tempfile.TemporaryDirectory() as temporary:
125             root = Path(temporary)
126             for relative in ALIASES:
127                 alias = root / relative
128                 if relative.endswith(('md', 'csv')):
129                     alias.parent.mkdir(parents=True, exist_ok=True)
130                     alias.write_text('old', encoding='utf-8')
131                 else:
132                     alias.mkdir(parents=True, exist_ok=True)
133                     (alias / 'version.txt').write_text('old', encoding='utf-8')
134             def generation(name):
135                 directory = root / 'outputs/.ql-runs' / name
136                 for target in ALIASES.values():
137                     path = directory / target
138                     if target.endswith(('md', 'csv')):
139                         path.parent.mkdir(parents=True, exist_ok=True)
140                         path.write_text(name, encoding='utf-8')
141                     else:
142                         path.mkdir(parents=True, exist_ok=True)
143                         (path / 'version.txt').write_text(name, encoding='utf-8')
144             return directory
145             first, second = generation('first'), generation('second')
146             publish_generation(first, root)
147             before = os.readlink(root / 'outputs/ql_current')
148             with patch('pipeline.os.replace', side_effect=OSError('injected commit failure')):
149                 with self.assertRaises(OSError):
150                     publish_generation(second, root)
151             self.assertEqual(os.readlink(root / 'outputs/ql_current'), before)
152             for relative in ALIASES:
153                 path = root / relative
154                 actual = path if relative.endswith(('md', 'csv')) else path / 'version.txt'
155                 self.assertEqual(actual.read_text(encoding='utf-8'), 'first')
156             publish_generation(second, root)
157             self.assertEqual((root / 'outputs/ql_current').resolve(), second.resolve())
158

```

```

159 def test_missing_font_does_not_leave_partial_figures(self):
160     with tempfile.TemporaryDirectory() as temporary:
161         staging = Path(temporary)
162         (staging / 'processed').mkdir()
163         def broken(*args):
164             (args[-1] / 'partial.png').write_bytes(b'partial')
165             raise RuntimeError('没有可用中文字体')
166         with patch('report.create_figures', side_effect=broken):
167             status = render_figures(None, None, None, None, staging)
168             self.assertEqual(status['status'], 'failed')
169             self.assertIn('中文字体', status['message'])
170             self.assertEqual(list((staging / 'processed').iterdir()), [])
171
172 def test_pipeline_analysis_and_test_failure_do_not_publish(self):
173     # 真实读取/求解/导出; 只在分析入口或测试门槛注入错误。
174     for target in ('run.run_analysis', 'run.run_tests'):
175         with self.subTest(stage=target), tempfile.TemporaryDirectory() as temporary:
176             root = Path(temporary)
177             source = root / 'docs/CUMCM2026Problems/C题/附件'
178             source.mkdir(parents=True)
179             shutil.copy2(ROOT / 'docs/CUMCM2026Problems/C题/附件/附件1.xlsx', source / '附件1.xlsx')
180
181             old = root / 'outputs/processed/q1'
182             old.mkdir(parents=True)
183             (old / 'sentinel.txt').write_text('previous', encoding='utf-8')
184             with patch(target, side_effect=RuntimeError('injected numerical gate failure')):
185                 with self.assertRaisesRegex(RuntimeError, 'injected numerical gate'):
186                     run.run_pipeline(root, plots=False)
187             self.assertEqual((old / 'sentinel.txt').read_text(encoding='utf-8'), 'previous')
188             self.assertFalse((root / 'outputs/q1_current').is_symlink())
189             self.assertEqual(list((root / 'outputs/.q1-runs').iterdir()), [])
190
191 def test_font_failure_still_publishes_valid_csv_and_visible_status(self):
192     with tempfile.TemporaryDirectory() as temporary:
193         root = Path(temporary)
194         source = root / 'docs/CUMCM2026Problems/C题/附件'
195         source.mkdir(parents=True)
196         for rel in ('附件1.xlsx',):
197             shutil.copy2(ROOT / 'docs/CUMCM2026Problems/C题/附件' / rel, source / rel)
198         model = root / 'docs/1/final/第一问_MILP_边际价值强化版.md'
199         model.parent.mkdir(parents=True)
200         shutil.copy2(ROOT / 'docs/1/final/第一问_MILP_边际价值强化版.md', model)
201         # 外层正在执行真实测试套件; 嵌套流水线不递归调用同一套件。
202         def nested_test_gate(staging):
203             self.assertTrue((staging / 'processed/result1.csv').is_file())
204             with patch('report.create_figures', side_effect=RuntimeError('没有可用中文字体')), \
205                 patch('run.run_tests', side_effect=nested_test_gate):
206                 generation = run.run_pipeline(root)
207                 data = read_inputs(source / '附件1.xlsx')
208                 solution = json.loads((generation / 'raw/solution_main.json').read_text(encoding='utf-8'))
209
210             self.assertTrue(check_csv_output(generation / 'processed', data, solution)['passed'])
211             self.assertFalse(list((generation / 'processed').glob('*.xlsx'))))
212             self.assertEqual((root / 'outputs/q1_current').resolve(), generation)
213             for name in ('report.html', 'marginal_report.html'):
214                 page = (generation / 'processed' / name).read_text(encoding='utf-8')
215                 self.assertIn('没有可用中文字体', page)

```

```

214         self.assertNotIn('<img', page)
215         self.assertEqual(list((generation / 'processed').glob('*.png')), [])

```

src/q1/traceability.py

```

1  """CR-13: 每个展示公式/新增关键行内公式→具体赋值行→测试→本代证据。"""
2
3  import hashlib
4  from pathlib import Path
5
6  from model import ROOT, write_csv, write_json
7
8  PAPER = 'docs/1/final/第一问_MILP_边际价值强化版.md'
9
10 # 展示公式按源文出现顺序编号；相同公式的重复展示共用同一精确实现。
11 # 纯术语/能流示意也标注其作用，不伪称它们是新增约束。
12 GROUPS = [
13     ('1 2 40 41 42 47 48 49 50', 'model.py', 'for name, coefficient in', 'test_q1.py', '
        test_corrupted_schedules_rejected', 'raw/validation.json#main/metrics/11.1_balance_max_abs_kwh',
        '母线来源、去向与平衡'),
14     ('3 4', 'model.py', 'result = milp(', 'test_review.py', '
        test_official_attachment_fresh_solve_golden_and_csv', 'raw/solution_main.json#solver', '模型类型
        说明，由整数求解调用落实'),
15     ('5 22', 'model.py', 'DT = 1 / 6', 'test_q1.py', 'test_time_and_unit_examples', 'inputs/processed/
        parameters.json#dt_hours', '时长'),
16     ('6 7 14 44 51 53 59', 'model.py', 'T = 144', 'test_q1.py', 'test_time_and_unit_examples', 'inputs/
        processed/timeseries.csv', '时段数和索引域'),
17     ('8 9 10 11 12 13', 'model.py', 'if endpoint != t * 10:', 'test_q1.py', '
        test_time_and_unit_examples', 'inputs/processed/timeseries.csv', '时间映射'),
18     ('15 16 17 18 19 20 21', 'model.py', '"load_kwh": pl * DT, "pv_kwh": ppv * DT', 'test_q1.py', '
        test_time_and_unit_examples', 'inputs/processed/timeseries.csv', 'ZOH等效常量与单位转换'),
19     ('23 24 25 26 64 83 136', 'model.py', 'Q_MAX = 5000 * DT', 'test_analysis.py', '
        test_independent_power_limits_and_csv_nonfinite_rejected', 'inputs/processed/parameters.json#
        max_energy_kwh', '紧物理功率上界'),
20     ('27 28 30 32 43 78', 'model.py', 'names = ("G", "C", "D", "E", "W", "u")', 'test_q1.py', '
        test_export_csv_matches_all_variables_and_inputs', 'raw/solution_main.json', '变量定义与输出列'),
21     ('29 31 33 34 36 52 55', 'model.py', 'a[n + t, index["C"][t]] = -eta_c', 'test_review.py', '
        test_independent_efficiencies_analytical_case', 'raw/validation.json#main/metrics/11.2
        _soc_max_abs_kwh', '充电效率与状态方程'),
22     ('29 31 33 34 36 52 55', 'model.py', 'a[n + t, index["D"][t]] = 1 / eta_d', 'test_review.py', '
        test_independent_efficiencies_analytical_case', 'raw/validation.json#main/metrics/11.2
        _soc_max_abs_kwh', '独立放电效率与状态方程'),
23     ('35 54 84 126 147', 'model.py', 'SCENARIOS = {"main": 0.90', 'test_q1.py', '
        test_all_actual_scenarios', 'raw/solution_main.json#eta_c', '主效率口径'),
24     ('37 39', 'model.py', 'upper[index["W"]] = pv', 'test_q1.py', '
        test_curtailment_is_available_when_absorption_is_limited', 'raw/validation.json#main/metrics
        /11.7_curtail_bound_violation_kwh', '弃光范围'),
25     ('38', 'model.py', 'g + pv - w + d - load - c', 'test_q1.py', 'test_corrupted_schedules_rejected',
        'raw/validation.json#main/metrics/11.1_balance_max_abs_kwh', '有效光伏'),
26     ('45', 'model.py', 'lower, upper = np.zeros(size)', 'test_q1.py', '
        test_corrupted_schedules_rejected', 'raw/validation.json#main/metrics/11.6
        _nonnegative_violation_kwh', '变量非负性'),
27     ('46 68 96', 'model.py', 'integer[index["u"]] = 1', 'test_review.py', '
        test_binary_tolerance_is_independent_of_physical', 'raw/validation.json#main/metrics/9
        _binary_distance', '二元域'),
28     ('56', 'model.py', 'row["soc_fraction"] = solution["E"][t] / 12000', 'test_q1.py', '

```

```

test_csv_roundtrip_and_tamper_rejection', 'processed/result1.csv#soc_fraction', '额定容量用于比例显
示, 不替代安全上限'),
29 ('57 58', 'model.py', 'lower[index["E"]], upper[index["E"]] = e_min, e_max', 'test_review.py', '
test_active_sets_use_current_limits_and_lower_space', 'raw/validation.json#main/metrics/11.3
_soc_bound_violation_kwh', '安全范围'),
30 ('60 61', 'model.py', 'lb[n + t] = ub[n + t] = E_INITIAL if t == 0 else 0', 'test_q1.py', '
test_corrupted_schedules_rejected', 'raw/validation.json#main/metrics/11.8_initial_error_kwh', '
初始时刻与6000kWh状态'),
31 ('62 63', 'model.py', 'lower[index["E"][-1]] = upper[index["E"][-1]] = E_INITIAL', 'test_q1.py', '
test_corrupted_schedules_rejected', 'raw/validation.json#main/metrics/11.8_terminal_error_kwh', '
终端时刻与6000kWh状态'),
32 ('65 69 73 74', 'model.py', 'a[2 * n + t, index["u"][t]] = -qc', 'test_analysis.py', '
test_independent_power_limits_and_csv_nonfinite_rejected', 'raw/validation.json#main/metrics/9
_binary_coupling_violation_kwh', '充电Big-M及模式含义'),
33 ('66 70 71 72', 'model.py', 'a[3 * n + t, index["u"][t]] = qd', 'test_analysis.py', '
test_independent_power_limits_and_csv_nonfinite_rejected', 'raw/validation.json#main/metrics/9
_binary_coupling_violation_kwh', '放电Big-M及模式含义'),
34 ('67 75', 'model.py', '"11.5_simultaneous_kwh"', 'test_q1.py', 'test_corrupted_schedules_rejected',
'raw/validation.json#main/metrics/11.5_simultaneous_kwh', '67为被排除反例; 75为互斥结果'),
35 ('76 77', 'model.py', 'p, pl, ppv = map(float, row[1: ])', 'test_q1.py', '
test_corrupted_inputs_rejected', 'inputs/processed/timeseries.csv#price_yuan_per_kwh', '电价与单
位'),
36 ('79', 'model.py', 'row["cost_yuan"] = record["price_yuan_per_kwh"] * solution["G"][t]', 'test_q1.
py', 'test_export_csv_matches_all_variables_and_inputs', 'processed/result1.csv#cost_yuan', '逐
时费用'),
37 ('80 81', 'model.py', 'objective[index["G"]] = p', 'test_review.py', '
test_official_attachment_fresh_solve_golden_and_csv', 'raw/solution_main.json#solver/
objective_yuan', '单一目标'),
38 ('82', 'model.py', 'objective[index["G"]] = p', 'test_review.py', '
test_official_attachment_fresh_solve_golden_and_csv', 'raw/validation.json#main', '完整MILP: 目
标, 约束分别见同表49/52/58/60/63/69/70/39/45/46'),
39 ('85', 'model.py', 'TOLERANCES = {', 'test_review.py', '
test_binary_tolerance_is_independent_of_physical', 'inputs/processed/parameters.json#tolerances'
, '分量纲数值容差'),
40 ('86 88', 'model.py', '"11.1_balance_max_abs_kwh"', 'test_q1.py', '
test_corrupted_schedules_rejected', 'raw/validation.json#main/metrics/11.1_balance_max_abs_kwh',
'母线残差'),
41 ('87 88', 'model.py', '"11.2_soc_max_abs_kwh"', 'test_q1.py', 'test_corrupted_schedules_rejected',
'raw/validation.json#main/metrics/11.2_soc_max_abs_kwh', '状态残差'),
42 ('89', 'model.py', '"11.3_soc_bound_violation_kwh"', 'test_q1.py', '
test_corrupted_schedules_rejected', 'raw/validation.json#main/metrics/11.3
_soc_bound_violation_kwh', '安全界验收'),
43 ('90', 'model.py', '"11.4_power_bound_violation_kwh"', 'test_q1.py', '
test_corrupted_schedules_rejected', 'raw/validation.json#main/metrics/11.4
_power_bound_violation_kwh', '功率界验收'),
44 ('91', 'model.py', '"11.8_terminal_error_kwh"', 'test_q1.py', 'test_corrupted_schedules_rejected',
'raw/validation.json#main/metrics/11.8_terminal_error_kwh', '首尾状态验收'),
45 ('92 93', 'model.py', '"11.9_objective_recalculation_error_yuan"', 'test_q1.py', '
test_corrupted_schedules_rejected', 'raw/validation.json#main/metrics/11.9
_objective_recalculation_error_yuan', '费用独立复算'),
46 ('94 95', 'model.py', 'metrics["11.2_relative_global_gap"]', 'test_q1.py', '
test_corrupted_schedules_rejected', 'raw/validation.json#main/metrics/11.2_relative_global_gap',
'全局Gap'),
47 ('97 98 99 100', 'analysis.py', '"integer_gap":', 'test_analysis.py', '
test_lp_relaxation_keeps_big_m_coupling', 'processed/lp_diagnosis.csv', 'LP诊断及松弛间隙'),
48 ('101', 'analysis.py', '"fractional_modes":', 'test_analysis.py', '
test_lp_relaxation_keeps_big_m_coupling', 'processed/lp_diagnosis.csv#fractional_modes', '分数模

```

```

式'),
49 ('102', 'analysis.py', '"simultaneous_periods":', 'test_analysis.py', '
    test_lp_relaxation_keeps_big_m_coupling', 'processed/lp_diagnosis.csv#simultaneous_periods', '同
    时动作'),
50 ('103 104', 'model.py', 'upper[index["C"]] = upper[index["D"]] = 0', 'test_analysis.py', '
    test_nested_baselines_and_analytic_no_ess', 'processed/baseline_comparison.csv', 'No-ESS'),
51 ('105', 'model.py', 'upper[index["C"]] = np.maximum(pv - load, 0)', 'test_review.py', '
    test_pv_only_source_with_zero_price_and_surplus', 'raw/analysis_validation.json#checks/pv_only',
    'PV-only充电界'),
52 ('106 107', 'model.py', 'upper[index["D"]] = np.maximum(load - pv, 0)', 'test_review.py', '
    test_pv_only_source_with_zero_price_and_surplus', 'raw/analysis_validation.json#checks/pv_only',
    'PV-only放电界'),
53 ('108 109 110', 'analysis.py', 'if not jm <= jpv + COST_TOL_YUAN', 'test_analysis.py', '
    test_nested_baselines_and_analytic_no_ess', 'processed/baseline_comparison.csv', '基准嵌套及成本次
    序'),
54 ('111 112 113 114', 'analysis.py', 'benefits = {', 'test_analysis.py', '
    test_nested_baselines_and_analytic_no_ess', 'processed/incremental_benefits.csv', '依赖基准的增量
    节约'),
55 ('115', 'model.py', 'lower[index["u"]] = upper[index["u"]] = mode', 'test_analysis.py', '
    test_analytic_shadow_sign_and_price_threshold', 'raw/analysis_solutions.json#scenarios', '固定模
    式'),
56 ('116 117', 'analysis.py', 'mu, lam = result.eqlin.marginals', 'test_analysis.py', '
    test_analytic_shadow_sign_and_price_threshold', 'processed/marginal_values.csv', '局部影子价及符号
    '),
57 ('118 119 120', 'analysis.py', 'low, high = ec * lam[i], lam[i] / ed', 'test_review.py', '
    test_independent_efficiencies_analytical_case', 'processed/marginal_values.csv', '阈值与独立效率')
    ,
58 ('121 122', 'model.py', '"roundtrip_efficiency":', 'test_review.py', '
    test_independent_efficiencies_analytical_case', 'processed/efficiency_comparison.csv', '母线与内
    部能量效率'),
59 ('123 124 125 127 151', 'analysis.py', '"future_price_threshold":', 'test_analysis.py', '
    test_analytic_shadow_sign_and_price_threshold', 'processed/arbitrage_thresholds.csv', '跨期价差阈
    值'),
60 ('128 129 130 133', 'analysis.py', 'for key, var, limit in', 'test_review.py', '
    test_active_sets_use_current_limits_and_lower_space', 'processed/active_constraints.csv', '按当前
    储能上下限识别活跃界'),
61 ('131', 'analysis.py', '("charge", "C", settings["charge_kw"] * DT)', 'test_review.py', '
    test_active_sets_use_current_limits_and_lower_space', 'processed/active_constraints.csv', '按当前
    充电功率识别活跃界'),
62 ('132', 'analysis.py', '("discharge", "D", settings["discharge_kw"] * DT)', 'test_review.py', '
    test_active_sets_use_current_limits_and_lower_space', 'processed/active_constraints.csv', '按当前
    放电功率识别活跃界'),
63 ('134 135 138 139 152', 'analysis.py', '"value_per_unit_day": saving / step', 'test_analysis.py', '
    test_all_36_scenarios_and_432_relaxations', 'processed/bottleneck_values.csv', '四方向有限增量价值'
    ),
64 ('137', 'analysis.py', '"increment_period_kwh": step if parameter', 'test_analysis.py', '
    test_all_36_scenarios_and_432_relaxations', 'processed/bottleneck_values.csv#
    increment_period_kwh', '功率扰动单位转换'),
65 ('140 141', 'analysis.py', '"economic_significance": "not_assessed_no_economic_threshold"', '
    test_review.py', 'test_active_sets_use_current_limits_and_lower_space', 'processed/
    bottleneck_values.csv#economic_significance', '概念比较: 经济显著性未擅自设门槛'),
66 ('142 148', 'model.py', '"roundtrip090": float(np.sqrt(0.90))', 'test_q1.py', '
    test_all_actual_scenarios', 'processed/efficiency_comparison.csv', '效率情景'),
67 ('143', 'analysis.py', 'return [{**r, "load_kw":', 'test_analysis.py', '
    test_full_input_output_csv_and_normalized_sensitivity', 'raw/analysis_schedules.csv', '输入缩放')
    ,
68 ('144', 'analysis.py', '"load_deltas":', 'test_analysis.py', '

```

```

    test_all_36_scenarios_and_432_relaxations', 'processed/sensitivity_scenarios.csv', '扰动网格'),
69 ('I45', 'model.py', '"cost_yuan": float(p @ g)', 'test_analysis.py', '
    test_all_36_scenarios_and_432_relaxations', 'processed/sensitivity_scenarios.csv', '汇总电量和费用
    '),
70 ('I46', 'analysis.py', '"normalized_sensitivity": (plus', 'test_analysis.py', '
    test_full_input_output_csv_and_normalized_sensitivity', 'processed/normalized_sensitivity.csv',
    '归一化敏感度'),
71 ('I49', 'export.py', '"12.4_cost_error_yuan":', 'test_review.py', '
    test_official_attachment_fresh_solve_golden_and_csv', 'raw/validation.json#csv', '按用户要求改CSV
    关闭回读'),
72 ('I50 I53 I54', 'run.py', 'analysis = run_analysis(', 'test_review.py', '
    test_pipeline_analysis_and_test_failure_do_not_publish', 'raw/run_manifest.json', '流程/章节小结:
    各计算公式见单独映射'),
73 ]
74
75
76 INLINE = [
77 ('I01', '进一步要求 $S_t > L_t$', 'model.py', 'upper[index["G"][pv > load]] = 0', '
    test_pv_only_source_with_zero_price_and_surplus', 'raw/analysis_validation.json#checks/pv_only',
    'PV来源约束: 两层基准共同施加'),
78 ('I02', '二者首先由固定模式 LP', 'analysis.py', 'mu, lam = result.eqlin.marginals', '
    test_independent_efficiencies_analytical_case', 'processed/marginal_values.csv', '母线RHS及状态注
    入RHS的导数符号'),
79 ('I03', '直接固定求解器给出的空闲模式', 'analysis.py', 'changed["lb"][row] += sign * step', '
    test_original_milp_rhs_all_576_results', 'processed/milp_rhs_validation.csv', '原MILP全部模式可重
    新选择的双侧扰动'),
80 ('I04', '当三步单位价值的极差', 'analysis.py', 'stable = len(values) >= 3', '
    test_active_sets_use_current_limits_and_lower_space', 'processed/bottleneck_values.csv', '多步长
    稳定性, 不替代经济门槛'),
81 ('I05', '上述数值容差不代表经济显著性.', 'analysis.py', '"economic_significance": "
    not_assessed_no_economic_threshold"', 'test_active_sets_use_current_limits_and_lower_space', '
    processed/bottleneck_values.csv', '无经济门槛不作显著性结论'),
82 ]
83
84
85 def unique_line(path, needle):
86     lines = path.read_text(encoding='utf-8').splitlines()
87     found = [(i, text) for i, text in enumerate(lines, 1) if needle in text]
88     if len(found) != 1:
89         raise ValueError(f'精确代码定位不唯一或已失效: {path}:{needle} ({len(found)})')
90     return found[0]
91
92
93 def generate_traceability(staging):
94     lines = (ROOT / PAPER).read_text(encoding='utf-8').splitlines()
95     blocks, start = [], None
96     for i, line in enumerate(lines, 1):
97         if line.strip() == '$$':
98             if start is None:
99                 start = i
100             else:
101                 blocks.append((start, i, '\n'.join(lines[start:i - 1])))
102                 start = None
103     if start is not None or len(blocks) != 154:
104         raise ValueError('论文公式数量改变, 必须逐公式重新审查映射')
105     rows, covered = [], set()
106     for ids, code, needle, test, test_name, evidence, meaning in GROUPS:

```

```

107 line, statement = unique_line(ROOT / 'src/q1' / code, needle)
108 test_line, _ = unique_line(ROOT / 'src/q1' / test, 'def ' + test_name + '(')
109 if not (staging / evidence.split('#')[0]).exists():
110     raise ValueError(f'缺失本轮运行证据: {evidence}')
111 for number in map(int, ids.split()):
112     first, last, formula = blocks[number - 1]
113     covered.add(number)
114     rows.append({'formula_id': f'F{number:03}', 'paper_path': PAPER, 'paper_start_line':
first,
115                 'paper_end_line': last, 'formula': formula, 'meaning': meaning,
116                 'code_path': f'src/q1/{code}', 'code_line': line, 'code_statement':
statement.strip(),
117                 'unit_test_path': f'src/q1/{test}', 'unit_test_line': test_line,
118                 'unit_test': test_name, 'runtime_evidence_in_generation': evidence})
119 if covered != set(range(1, len(blocks) + 1)):
120     raise ValueError(f'存在未映射展示公式: {set(range(1, len(blocks) + 1)) - covered}')
121 for number, paper_needle, code, needle, test_name, evidence, meaning in INLINE:
122     source_line, formula = unique_line(ROOT / PAPER, paper_needle)
123     code_line, statement = unique_line(ROOT / 'src/q1' / code, needle)
124     test_line, _ = unique_line(ROOT / 'src/q1/test_review.py', 'def ' + test_name + '(')
125     if not (staging / evidence.split('#')[0]).exists():
126         raise ValueError(f'缺失本轮运行证据: {evidence}')
127     rows.append({'formula_id': number, 'paper_path': PAPER, 'paper_start_line': source_line,
128                 'paper_end_line': source_line, 'formula': formula, 'meaning': meaning,
129                 'code_path': f'src/q1/{code}', 'code_line': code_line, 'code_statement':
statement.strip(),
130                 'unit_test_path': 'src/q1/test_review.py', 'unit_test_line': test_line,
131                 'unit_test': test_name, 'runtime_evidence_in_generation': evidence})
132 write_csv(staging / 'docs/q1_source_line_traceability.csv', rows)
133 write_json(staging / 'raw/traceability_validation.json', {'passed': True, 'display_formulas':
len(blocks),
134                 'inline_items': len(INLINE), 'mapping_rows': len(rows), 'source_sha256': hashlib.
sha256((ROOT / PAPER).read_bytes()).hexdigest(),
135                 'coverage': '154个展示公式及5项关键行内公式/判据: 纯说明公式注明作用'})

```

B.2 问题二源程序

src/q2/backtest.py

```

1 """2.1.12 - 2.1.13: 严格日前信息接口和真实评价; 不替方案指定预测/调度器。"""
2
3 import numpy as np
4 import pandas as pd
5
6 from data import (CALENDAR, DT_HOURS, END, FORECAST_START, SERIES, SLOTS,
7                  calendar_for_day, history_before)
8
9
10 def rolling_folds(frame, start=FORECAST_START, end=END):
11     rows = []
12     for origin in pd.date_range(start, end, freq="D"):
13         history = history_before(frame, origin)
14         actual = frame.loc[frame.date == origin]
15         if history.empty or len(actual) != SLOTS or not actual.timestamp.equals(
16             pd.Series(calendar_for_day(origin).timestamp.to_numpy(), index=actual.index)):
17             raise ValueError(f"{origin.date()} 历史为空或目标日不完整/顺序不正确")

```



```

18     rows.append({"origin": origin, "history_start": history.timestamp.iloc[0],
19                 "history_end_exclusive": origin, "history_last_timestamp": history.timestamp.
20                 iloc[-1],
21                 "history_points": len(history), "horizon_points": SLOTS,
22                 "target_first": actual.timestamp.iloc[0], "target_last": actual.timestamp.iloc
23                 [-1],
24                 "rule": "timestamp < origin", "window": "expanding", "unit": "kW"})
25
26     return pd.DataFrame(rows)
27
28 def rolling_predict(frame, model_factory, start=FORECAST_START, end=END):
29     """factory(series) 每日返回全新预测器；只获该变量历史和未来日历，不获真值。
30
31     预测器为 callable(history[timestamp,value_kw], calendar) -> 长度144的功率数组。
32     参数估计、缩放、调参和特征选择必须完全在传入历史内部进行。
33     """
34     rows = []
35     folds = rolling_folds(frame, start, end)
36     for fold in folds.itertuples(index=False):
37         history = history_before(frame, fold.origin)
38         prediction = calendar_for_day(fold.origin)
39         prediction["origin"] = fold.origin
40         for variable in ("load_kw", "pv_kw"):
41             model = model_factory(variable)
42             past = history[["timestamp", variable]].rename(columns={variable: "value_kw"}).copy(deep=True)
43             values = np.asarray(model(past, prediction[list(CALENDAR)].copy(deep=True)), dtype=float)
44
45             if values.shape != (SLOTS,) or not np.isfinite(values).all() or (values < 0).any():
46                 raise ValueError(f"{variable} 预测必须为 144 个有限非负 kW 值；不自动裁剪")
47             prediction[f"predicted_{variable}"] = values
48             prediction["predicted_net_load_kw"] = prediction.predicted_load_kw - prediction.predicted_pv_kw
49
50             # 只有全部预测完成后才提取真值，不能给模型调用传递 actual。
51             actual = frame.loc[frame.date == fold.origin, list(SERIES)]
52             for variable in SERIES:
53                 prediction[f"actual_{variable}"] = actual[variable].to_numpy()
54             rows.append(prediction)
55     return pd.concat(rows, ignore_index=True)
56
57 def forecast_metrics(predictions):
58     """三类功率 MAE/RMSE；按所有时段等权计算，不能平均每日 RMSE。"""
59     if predictions.empty or predictions.timestamp.duplicated().any():
60         raise ValueError("预测结果为空或时间重复")
61     if predictions.date.isna().any() or predictions.timestamp.isna().any():
62         raise ValueError("预测日期或时间戳不能缺失")
63     for date, part in predictions.groupby("date"):
64         expected_time = calendar_for_day(date).timestamp.to_numpy()
65         if len(part) != SLOTS or not np.array_equal(pd.to_datetime(part.timestamp).to_numpy(),
66             expected_time):
67             raise ValueError("预测评价必须逐日按序包含完整144个目标时间戳")
68     expected = list(SERIES)
69     for prefix in ("predicted", "actual"):
70         values = predictions[[f"{prefix}_{v}" for v in expected]].to_numpy(dtype=float)
71         if not np.isfinite(values).all() or (values[:, :2] < 0).any():
72             raise ValueError("预测/真实功率必须有限，负荷与光伏不能为负")

```



```

69         if not np.allclose(values[:, 0] - values[:, 1], values[:, 2], rtol=1e-12, atol=1e-10):
70             raise ValueError("净负荷必须由负荷减光伏得到")
71     rows = []
72     for label, group in [("all", predictions), *list(predictions.groupby("date"))]:
73         for variable in SERIES:
74             error = group[f

"predicted_{variable}"

] - group[f

actual_{variable}"

]
75             rows.append({"period": str(label), "series": variable, "n": len(error),
76                         "mae_kw": float(error.abs().mean()), "rmse_kw": float(np.sqrt(np.mean(error
77 **2)))})
78     return pd.DataFrame(rows)
79
80 def dispatch_metrics(dispatch):
81     """输入事后调度明细；只计费，不由净负荷误差猜测紧急电量。
82
83     planned_grid_kwh 为 0:00 已制定计划；actual_supply_before_emergency_kw 是已执行供给
84     （计及实际光伏、储能与弃电后的母线供给）。储能可行性须由后续调度模型验证。
85     """
86     required = ("timestamp", "date", "planned_grid_kwh", "price_yuan_per_kwh", "actual_load_kw",
87               "actual_supply_before_emergency_kw")
88     if dispatch.empty or any(c not in dispatch for c in required):
89         raise ValueError("缺少事后调度明细字段")
90     dispatch = dispatch.copy().sort_values("timestamp")
91     time = pd.to_datetime(dispatch.timestamp)
92     dates = pd.to_datetime(dispatch.date)
93     if time.isna().any() or dates.isna().any() or time.duplicated().any() or not time.diff().dropna
94        (.eq(pd.Timedelta(minutes=10)).all():
95         raise ValueError("调度时间必须唯一、连续且间隔 10 min")
96     if not dates.eq((time - pd.Timedelta(minutes=10)).dt.normalize()).all():
97         raise ValueError("调度日期必须为区间所属日")
98     if not dispatch.groupby("date").size().eq(SLOTS).all():
99         raise ValueError("调度评价必须覆盖完整的每日144时段")
100     values = dispatch[list(required[2:])].to_numpy(dtype=float)
101     if not np.isfinite(values).all() or (values[:, :3] < 0).any():
102         raise ValueError("调度字段必须有限，计划电量、电价与负荷必须非负；实际净供给可为负")
103     emergency = np.maximum(values[:, 2] - values[:, 3], 0) * DT_HOURS
104     planned_cost = values[:, 0] * values[:, 1]
105     emergency_cost = emergency * values[:, 1] * 5 # C题明确为交易时刻电价的5倍。
106     active = emergency > 0
107     event_start = active & ~np.r_[False, active[:-1]]
108     return {"start": str(time.iloc[0]), "end": str(time.iloc[-1]), "days": int(dates.nunique()),
109           "emergency_kwh": float(emergency.sum()), "emergency_intervals": int(active.sum()),
110           "emergency_interval_rate": float(active.mean()), "emergency_events": int(event_start.sum
111           ()),
112           "days_with_emergency": int(dates[active].nunique()), "planned_cost_yuan": float(
113           planned_cost.sum()),
114           "emergency_cost_yuan": float(emergency_cost.sum()), "total_cost_yuan": float((
115           planned_cost + emergency_cost).sum()),
116           "cost_period_label": "full_calendar_year" if time.iloc[0] == pd.Timestamp("2025-01-01
117           00:10") and
118           time.iloc[-1] == pd.Timestamp("2026-01-01") else "evaluated_period_only"}

```

src/q2/baseline_reuse.py

```

1 """仅复用已经完整验收、与指定基准输入相同的主轨迹。"""
2

```

```

3 import json
4 from pathlib import Path
5 import shutil
6 import tempfile
7
8 import numpy as np
9
10 from checkpoint import TABLES, atomic_json, read_checkpoint
11 from data import sha256
12 from export_dispatch import write_tables
13 from shadows import choose_day
14
15
16 def main_baselines(initial):
17     return {"initial_{initial:g}", "window_28"}
18
19
20 def window_inputs_match(store, frozen, actual, dates, prices, main, start, end):
21     """包括未选中的预测流程评分；不能只因最终选中窗口为28就认定等价。"""
22     for day in range(start, end):
23         audit = json.loads((main/f"daily_audit/{dates[day].date()}.json").read_text(encoding="utf-8"
24         ))
25         for key, item in audit.items():
26             if not key.isdigit() or "selection" not in item:
27                 continue
28             k = int(key)
29             automatic = choose_day(store, frozen, actual[:day], day, k, prices)
30             try:
31                 fixed = choose_day(store, frozen, actual[:day], day, k, prices, window=28)
32             except ValueError:
33                 return False
34             if (not np.array_equal(automatic[0], fixed[0]) or not np.array_equal(automatic[1], fixed
35             [1])
36                 or automatic[2] != fixed[2] or fixed[2] != item["selection"]):
37                 return False
38     return True
39
40
41 def reuse_main_baseline(run, name, store, frozen, actual, dates, prices, initial):
42     """失败或不匹配则交给正常实验，绝不覆盖已有独立实验存档。"""
43     run = Path(run)
44     if name not in main_baselines(initial):
45         return False
46     main = run/"processed/main"
47     status = json.loads((main/"run_status.json").read_text(encoding="utf-8"))
48     if not status.get("complete") or status.get("settings") != {}:
49         return False
50     if status.get("start_index") != 31 or status.get("end_index") != 365:
51         return False
52     target = run/"processed/experiments"/name
53     if target.exists():
54         return False
55     source_manifest = (main/"checkpoint.json").read_bytes()
56     frame, _, _ = read_checkpoint(main, actual, dates, prices, initial, 31, 365, {})
57     if name == "window_28" and not window_inputs_match(store, frozen, actual, dates, prices, main,
58     31, 365):
59         return False

```

```

57 target.parent.mkdir(parents=True, exist_ok=True)
58 staging = Path(tempfile.mkdtemp(prefix=".baseline_reuse_", dir=target.parent))
59 try:
60     for filename in (*TABLES, "checkpoint.json", "run_status.json"):
61         shutil.copyfile(main/filename, staging/filename)
62     for folder in ("daily_audit", "frozen_plans"):
63         shutil.copypath(main/folder, staging/folder)
64     reused_status = dict(status)
65     reused_status["settings"] = {"window": 28} if name == "window_28" else {}
66     atomic_json(staging/"run_status.json", reused_status)
67     write_tables(frame, staging)
68     # 核验复制结果及跨日物理响应：源主轨迹必须保持同一完整版本。
69     read_checkpoint(staging, actual, dates, prices, initial, 31, 365, reused_status["settings"])
70     if (main/"checkpoint.json").read_bytes() != source_manifest:
71         raise ValueError("复用期间主轨迹版本改变")
72     atomic_json(staging/"baseline_reuse.json", {
73         "complete": True, "source": "processed/main", "days": 334,
74         "source_checkpoint_sha256": sha256(main/"checkpoint.json"),
75         "source_status_sha256": sha256(main/"run_status.json"),
76         "source_execution": status.get("execution", {}),
77         "source_execution_history": status.get("execution_history", []),
78         "reason": "identical_initial_baseline" if name != "window_28" else "
all_daily_predictor_scores_points_and_blocks_identical",
79         "interpretation": "shared_baseline_not_an_independent_rerun", "extra_MILP_solves": 0})
80     staging.rename(target)
81 finally:
82     if staging.exists():
83         shutil.rmtree(staging)
84 return True

```

src/q2/benchmarks/horizon_for_figure.py

```

1  """图5-5经用户确认的局部K比较：共同72h场景前缀，同机同预算，非全年。"""
2  from pathlib import Path
3  import sys, json, time, hashlib
4  import numpy as np
5  import pandas as pd
6  ROOT=Path(__file__).resolve().parents[3]
7  sys.path.insert(0, str(ROOT/'src/q2'))
8  from rolling import _solve, dispatch_frame
9  from checkpoint import atomic_json
10
11 OUT=ROOT/'outputs/q2/benchmarks/horizon_figure_20260913'
12 INPUT=ROOT/'outputs/sensitivity_q2_q3/raw/local_20260913/inputs/q2'
13 DATES=['2025-09-17', '2025-06-01', '2025-06-02', '2025-09-28']
14
15
16 def main():
17     OUT.mkdir(parents=True, exist_ok=True); rows=[]
18     for date in DATES:
19         data=np.load(INPUT/date/'frozen.npz'); meta=json.loads((INPUT/date/'metadata.json').read_text())
20         for k in (1,2,3):
21             directory=OUT/f'{date}_K{k}'; directory.mkdir(exist_ok=True)
22             if (directory/'result.json').exists(): rows.append(json.loads((directory/'result.json').read_text())); continue

```

```

23     count=k*144;net=data['net'][:, :count].copy();weights=data['weights'].copy();prices=data[
'prices'][:, :count].copy();reserve=data['reserve'][:, :count].copy()
24     atomic_json(OUT/'status.json',{ 'phase': 'solving', 'date':date, 'K':k, 'completed':len(rows)
, 'total':12})
25     start=time.perf_counter();cache={};opts=dict(reserve=reserve, solver_gap=meta['gap'],
solver_threads=meta['threads'], refine_seconds=meta['refine_seconds'], solve_cache=cache, cache_key
='local_K')
26     policy,_,info=_solve(net,prices,weights,float(data['initial']),solver_seconds=meta['
seconds'],**opts)
27     if policy is None or not info['reliable']:
28         atomic_json(directory/'first_attempt.json',info)
29         policy,_,info=_solve(net,prices,weights,float(data['initial']),solver_seconds=meta['
rescue_seconds'],**opts)
30     elapsed=time.perf_counter()-start
31     if policy is None:raise RuntimeError(f'{date} K{k}没有可执行策略')
32     frame=dispatch_frame(pd.Timestamp(date),policy,data['real_net'],float(data['initial']),
prices[:144])
33     frame.to_csv(directory/'dispatch.csv',index=False,encoding='utf-8',float_format='%.17g')
34     atomic_json(directory/'audit.json',info)
35     row={'date':date, 'K':k, 'horizon_hours':k*24, 'S':len(weights), 'initial_soc':float(data['
initial']),
36         'realized_day_cost':float((frame.planned_cost+frame.emergency_cost).sum()), '
emergency_kwh':float(frame.emergency.sum()),
37         'end_soc':float(frame.soc.iloc[-1]), 'solve_seconds':elapsed, 'gap':info.get('mip_gap
',info.get('gap')), 'reliable':bool(info['reliable']),
38         'requested_gap':meta['gap'], 'threads':meta['threads'], 'shared_72h_input_sha256':
hashlib.sha256((INPUT/date/'frozen.npz').read_bytes()).hexdigest()}
39     atomic_json(directory/'result.json',row);rows.append(row);print(date,k,row['reliable'],
round(elapsed,2),flush=True)
40     pd.DataFrame(rows).to_csv(OUT/'cases.csv',index=False,encoding='utf-8',float_format='
%.17g')
41     frame=pd.DataFrame(rows);assert len(frame)==12
42     frame.groupby('K',as_index=False).agg(horizon_hours=('horizon_hours', 'first'), mean_day_cost=('
realized_day_cost', 'mean'),
43         mean_solve_seconds=('solve_seconds', 'mean'), mean_end_soc=('end_soc', 'mean'),
mean_emergency_kwh=('emergency_kwh', 'mean'),
44         certified_cases=('reliable', 'sum'), cases=('date', 'count')).to_csv(OUT/'summary.csv',index=
False,encoding='utf-8',float_format='%.17g')
45     atomic_json(OUT/'status.json',{ 'phase': 'complete', 'completed':12, 'total':12, 'all_certified':bool
(frame.reliable.all()), 'scope': 'four matched days, shared scenario prefixes, same machine, not
annual'})
46
47
48 if __name__=='__main__':main()

```

src/q2/benchmarks/precision_pair.py

```

1  """两真实日各重复两次的1%/3%配对实验：完整日模型、统一算法、正式执行只读。"""
2
3  import datetime as dt
4  import hashlib
5  import importlib.metadata
6  import io
7  import json
8  import os
9  from pathlib import Path

```

```

10 import platform
11 import subprocess
12 import sys
13 import time
14
15 import numpy as np
16 import pandas as pd
17
18 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
19 import rolling
20 from data import ROOT, sha256, write_csv, write_json
21 from dispatch import load_prepared
22 from linear_policy import solve_linear_policy
23 from policy import DT, validate_replay
24
25
26 def main():
27     run = ROOT / "outputs/q2/dispatch_runs/20260911_224501"
28     output = ROOT / "outputs/q2/precision_comparison" / dt.datetime.now().strftime("%Y%m%d_%H%M%S")
29     raw_dir, processed = output / "raw", output / "processed"
30     raw_dir.mkdir(parents=True); processed.mkdir()
31     print("RESULT_DIRECTORY", output, flush=True)
32     protected = list((ROOT / "src/q2").glob("*.py")) + [run / "raw/solver_precision_revision.json",
33     run / "raw/prepared_manifest.json"]
34     hashes = {str(p.relative_to(ROOT)): sha256(p) for p in protected}
35     load_started = time.perf_counter()
36     store, frozen, actual, dates, prices = load_prepared(run)
37     load_seconds = time.perf_counter() - load_started
38     directory = run / "processed/main"
39     for _ in range(10):
40         checkpoint = (directory / "checkpoint.json").read_bytes()
41         daily_bytes = (directory / "daily.csv").read_bytes()
42         if (checkpoint == (directory / "checkpoint.json").read_bytes()
43             and hashlib.sha256(daily_bytes).hexdigest() == json.loads(checkpoint)["sha256"] ["
44             daily.csv"]):
45             break
46     else:
47         raise RuntimeError("未取得完整已提交日表")
48     daily = pd.read_csv(io.BytesIO(daily_bytes), float_precision="round_trip")
49     selected_dates = ["2025-02-01", "2025-03-20"]
50     write_csv(raw_dir / "initial_states.csv", daily[daily.date.isin(selected_dates)])
51     metadata = {
52         "status": "running", "created_at": dt.datetime.now(dt.timezone.utc).isoformat(),
53         "dates": selected_dates, "date_rule": "
54         evaluation_first_day_and_first_protocol_specified_date",
55         "repeats": 2, "requested_gaps": [.01, .03], "candidate_time_limit_seconds": 120,
56         "supplemental_retries": False, "run_directory": str(run),
57         "same_algorithm": "rolling_LP_certificate_then_existing_equivalent_HiGHS_MILP",
58         "one_percent_route": "isolated_native_fallback_redirected_to_current_equivalent_HiGHS;
59         production_unchanged",
60         "timing_scope": "make_day_including_all_K_selection_solve_validation;
61         excludes_common_library_load_and_actual_replay",
62         "shared_load_seconds": load_seconds, "source_sha256": hashes,
63         "benchmark_sha256": sha256(Path(__file__)), "daily_snapshot_sha256": hashlib.sha256(
64         daily_bytes).hexdigest(),
65         "machine": platform.platform(), "processor": platform.processor(), "logical_cpus": os.
66         cpu_count(),

```

```

60     "versions": {name: importlib.metadata.version(name) for name in ("numpy", "scipy", "pandas",
61     "pysciopopt")},
62     "thread_environment": {key: os.environ.get(key) for key in ("OPENBLAS_NUM_THREADS", "
OMP_NUM_THREADS")},
63     "background_processes_start": subprocess.check_output(["ps", "-axo", "pid,etime,%cpu,command
"], text=True).splitlines(),
64     "records": [], "input_hashes": {},
65 }
66 metadata["background_processes_start"] = [line for line in metadata["background_processes_start"
] if "src/q2/full_run.py" in line]
67 original_solve, original_native = rolling._solve, rolling.solve_policy
68 calls, summaries, candidates = [], [], []
69 context = {}
70
71 def same_linear(net, price, weights, initial, *, gap, time_limit, seed_policy, known_lower_bound
):
72     return solve_linear_policy(net, price, weights, initial, gap=gap, time_limit=time_limit,
seed_policy=seed_policy)
73
74 def timed_solve(net, price, weights, initial, **kwargs):
75     k = len(price) // 144
76     key = f"{context['date']}_K{k}"
77     digest = hashlib.sha256()
78     for value in (net, price, weights, np.array([initial])):
79         value = np.ascontiguousarray(value, dtype=np.float64)
80         digest.update(str(value.shape).encode("utf-8")); digest.update(value.tobytes())
81     fingerprint = digest.hexdigest()
82     assert metadata["input_hashes"].setdefault(key, fingerprint) == fingerprint
83     if not (raw_dir / f"input_{key}.npz").exists():
84         np.savez_compressed(raw_dir / f"input_{key}.npz", net=net, prices=price, weights=weights
, initial_soc=initial)
85     started = time.perf_counter()
86     policy, responses, info = original_solve(net, price, weights, initial, **kwargs)
87     elapsed = time.perf_counter() - started
88     item = {**context, "K": k, "scenarios": len(net), "input_sha256": fingerprint,
"wall_seconds": elapsed, "solver": info}
89     if policy is not None:
90         for path, response in zip(net, responses):
91             validate_replay(policy.grid, path, initial, response)
92             policy_file = f"policy_{context['trial']}_K{k}.npz"
93             np.savez_compressed(raw_dir / policy_file, grid=policy.grid, charge_cap=policy.
charge_cap, discharge_cap=policy.discharge_cap)
94             item["policy_file"] = policy_file
95             if info["reliable"]:
96                 assert policy is not None and info["mip_gap"] <= kwargs["solver_gap"] + 1e-8
97             calls.append(item)
98             print(json.dumps({**context, "K": k, "seconds": elapsed, "gap": info.get("mip_gap"), "
accepted": info["reliable"]}), flush=True)
99             return policy, responses, info
100
101 rolling._solve, rolling.solve_policy = timed_solve, same_linear
102 try:
103     for date_index, date in enumerate(selected_dates):
104         day = int(dates.get_loc(pd.Timestamp(date)))
105         soc = float(daily.loc[daily.date == date, "initial_soc"].iloc[0])
106         for repeat in (1, 2):
107             order = (1, 3) if (date_index + repeat) % 2 else (3, 1)

```

```

108         for percent in order:
109             trial = f"{date}_r{repeat}_{percent}pct"
110             context = {"date": date, "repeat": repeat, "gap_percent": percent, "trial":
trial}
111             calls = []
112             write_json(raw_dir / "active.json", context)
113             started = time.perf_counter()
114             policy = None
115             try:
116                 policy, _, _, _, audit = rolling.make_day(store, frozen, actual[:day], day,
prices, soc, solver_gap=percent / 100)
117                 except rolling.ComputationalLimit as error:
118                     audit = error.details
119                     elapsed = time.perf_counter() - started
120                     assert [item["K"] for item in calls] == [1, 2, 3]
121                     row = {**context, "initial_soc": soc, "day_wall_seconds": elapsed,
"accepted_candidates": sum(item["solver"]["reliable"] for item in calls),
122                             "total_candidates": len(calls), "selected_K": audit.get("selected_K")}
123                     if policy is not None:
124                         frame = rolling.dispatch_frame(dates[day], policy, DT * (actual[day, :, 0] -
actual[day, :, 1]), soc, prices)
125                         write_csv(raw_dir / f"dispatch_{trial}.csv", frame)
126                         row.update(planned_cost=float(frame.planned_cost.sum()), emergency_cost=
float(frame.emergency_cost.sum()),
127                                 actual_cost=float(frame.planned_cost.sum() + frame.emergency_cost
.sum()),
128                                 grid_kwh=float(frame.grid.sum()), emergency_kwh=float(frame.
emergency.sum()),
129                                 final_soc=float(frame.soc.iloc[-1]),
130                                 first_day_expected_cost=audit[str(audit["selected_K"])]["
first_day_expected_cost"])
131                     summaries.append(row)
132                     for item in calls:
133                         info = item["solver"]
134                         candidates.append({**context, "K": item["K"], "scenarios": item["scenarios"]
},
135                                           "wall_seconds": item["wall_seconds"], "achieved_gap":
info.get("mip_gap"),
136                                           "accepted": info["reliable"], "status": info["status"],
137                                           "objective": info.get("objective"), "dual_bound": info.
get("dual_bound"),
138                                           "input_sha256": item["input_sha256"]})
139                     metadata["records"].append({**row, "calls": calls, "audit": audit})
140                     write_json(raw_dir / "benchmark.json", metadata)
141                     write_csv(processed / "summary.csv", pd.DataFrame(summaries))
142                     write_csv(processed / "candidates.csv", pd.DataFrame(candidates))
143                     print("COMPLETE", json.dumps(row), flush=True)
144             finally:
145                 rolling._solve, rolling.solve_policy = original_solve, original_native
146                 assert all(sha256(ROOT / name) == digest for name, digest in hashes.items()), "正式源码或冻结配置在
实验中改变"
147                 metadata.update(status="completed", finished_at=dt.datetime.now(dt.timezone.utc).isoformat(),
protected_files_unchanged=True)
148                 write_json(raw_dir / "benchmark.json", metadata)
149                 (raw_dir / "active.json").unlink()
150
151
152

```

```

153 if __name__ == "__main__":
154     main()

```

src/q2/benchmarks/precision_pair_report.py

```

1  """回读配对实验的全部输入、策略与费用，生成自包含HTML对照表。"""
2
3  import hashlib
4  import html
5  import json
6  import os
7  from pathlib import Path
8  import sys
9
10 import numpy as np
11 import pandas as pd
12
13 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
14 from data import R00T, sha256, write_csv, write_json
15 from dispatch import load_prepared
16 from policy import DT, Policy, replay, validate_replay
17 from scenarios import construct
18 from shadows import choose_day
19
20
21 def table(headers, rows):
22     return '<div class="table-wrap"><table><thead><tr>' + ''.join(f'<th>{html.escape(str(v))}</th>'
23     for v in headers) + '</tr></thead><tbody>' + ''.join('<tr>' + ''.join(f'<td>{html.escape(str(v))}
24     </td>' for v in row) + '</tr>' for row in rows) + '</tbody></table></div>'
25
26
27 def n(value, digits=2):
28     return f"{value:.,{digits}f}"
29
30
31 def tail_evidence(run, output):
32     """冻结的历史48日只说明长尾负担，不用旧算法耗时估算当前精度效应。"""
33     source = run / "processed/timing_audit"
34     verification = json.loads((source / "verification.json").read_text(encoding="utf-8"))
35     names = ("daily_timing.csv", "summary.json", "snapshot.json")
36     hashes = {name: sha256(source / name) for name in names}
37     assert all(digest == verification["artifact_sha256"][name] for name, digest in hashes.items())
38     old = json.loads((source / "summary.json").read_text(encoding="utf-8"))
39     frame = pd.read_csv(source / "daily_timing.csv", float_precision="round_trip")
40     days = frame[(frame.job == "main") & frame.overnight].copy()
41     assert len(days) == 48 and days.date.nunique() == 48
42     assert list(days.date) == list(pd.date_range("2025-03-10", "2025-04-26").strftime("%Y-%m-%d"))
43     seconds = days.audit_wall_since_previous_seconds
44     assert seconds.notna().all() and (seconds >= 0).all()
45     elapsed = (pd.Timestamp(old["completed_window_local"][1]) - pd.Timestamp(old["
46     completed_window_local"][0])).total_seconds()
47     np.testing.assert_allclose(seconds.sum(), elapsed, atol=.01, rtol=0)
48     rescue = days[days.supplemental]
49     regular = days[~days.supplemental]
50     assert sorted(rescue.date) == sorted(old["supplemental_dates"])
51     share = rescue.audit_wall_since_previous_seconds.sum() / seconds.sum()

```



```

49 np.testing.assert_allclose(share, old["supplemental_share_of_main_advancement_wall"], atol=1e
    -10, rtol=0)
50 rows = []
51 for label, group in (("未触发整日补算", regular), ("触发整日补算", rescue), ("全部已推进日", days)):
52     values = group.audit_wall_since_previous_seconds
53     rows.append({"group": label, "days": len(group), "day_share_percent": 100 * len(group) / len
        (days),
54                 "wall_hours": values.sum() / 3600, "wall_share_percent": 100 * values.sum() /
        seconds.sum(),
55                 "mean_minutes": values.mean() / 60, "median_minutes": values.median() / 60,
56                 "p95_minutes": values.quantile(.95) / 60, "max_minutes": values.max() / 60})
57 write_csv(output / "processed/tail_groups.csv", pd.DataFrame(rows))
58 write_csv(output / "processed/tail_days.csv", days)
59 info = {"source_directory": str(source), "source_sha256": hashes, "historical_window": old["
    completed_window_local"],
60         "supplemental_day_share_percent": 100 * len(rescue) / len(days), "
    supplemental_time_share_percent": 100 * share,
61         "max_to_non_supplemental_mean_ratio": seconds.max() / regular.
    audit_wall_since_previous_seconds.mean(),
62         "mean_to_median_ratio": seconds.mean() / seconds.median(), "groups": rows,
63         "historical_only_not_current_gap_speedup": True, "
    current_unfinished_days_not_in_snapshot": True,
64         "scope": "
    audit_completion_intervals_including_retries_repairs_waits_not_pure_solver_seconds"}
65 write_json(output / "raw/tail_review.json", info)
66 grid = table(["历史分组", "日期数", "日期占比", "累计推进时间 / 小时", "时间占比", "平均 / 分钟", "P95 /
    分钟", "最长 / 分钟"],
67              [[r["group"], r["days"], f'{r["day_share_percent"]:.1f}%', n(r["wall_hours"]), f'{r
    ["wall_share_percent"]:.1f}%',
68                n(r["mean_minutes"]), n(r["p95_minutes"]), n(r["max_minutes"])] for r in rows])
69 markup = f'''<section><h2>长尾核查: 少数日期占据大部分时间</h2>
70 <p class="note">历史主任务连续推进区间: 模拟 2025-03-10 至 04-26, 共 48 日; 实际记录窗口 2026-09-12 01:02—09
    :05。按是否触发整日追加预算分组, 使用完整区间, 未按最终耗时挑选日期。</p>{grid}
71 <div class="callout caution">7 个补算日仅占日期数的 {100*len(rescue)/len(days):.1f}%, 却占推进时间的 {100*
    share:.1f}%。最长日为 121.22 分钟, 约为未触发补算日平均值的 {info['max_to_non_supplemental_mean_ratio'
    ]:.1f} 倍。忽略这部分日期会严重低估总耗时。</div>
72 <p class="note">这里的“推进时间”是相邻日审计提交间隔, 包括准备、失败重试、修复和等待。它来自历史 1% 阶段及当时求解
    路线, 不能与下方当前算法的单次秒数相除来推算精度加速。未触发整日补算也可能有个别 K 失败; 此分组不等于完整难度分类。
    该快照只含当时已推进日期, 未计入现在尚未完成的阻塞日, 不能当作全年分布。P95 为本组经验分位数的线性插值, 7 日组的尾
    部分位数仍不稳定。</p>
73 <p class="links"><a href="tail_groups.csv" download>下载长尾分组 CSV</a><a href="tail_days.csv"
    download>下载历史 48 日明细</a><a href="../raw/tail_review.json">长尾核查来源与验证</a></p>
74 <details><summary>后续精度比较应怎样覆盖困难日</summary>
75 <p>按基准执行的求解路径分层: LP 证书直接通过、进入 MILP 后在首轮达标、追加预算后达标、追加预算仍未达标。困难层应多抽样
    或全部纳入; 各档使用相同实例、原始 SOC、完整 K 候选、求解路线及重试规则。最终按各层日期数量加权汇总时间, 不能把多
    抽的困难日或少抽的普通日直接等权平均。</p>
76 <p>同时报告达标日数、各 K 通过率、首轮与补算总时间、P50/P90/P95/最大值。到统一上限仍不达标的日期记为“成功时间至少超
    过已尝试预算”, 不删掉, 也不把截止时间当作完成时间。真实费用在双方均有合格解的相同日期比较, 并列出了未配对日期和未
    SOC; 经济差异仍需连续回放检验。</p>
77 <p class="note">以上是修正后的抽样与评价要求, 尚未完成分层 1%/3% 实测, 因此当前不能给出全任务加速百分比。本轮只核查
    已有记录, 没有启动额外长时间求解。</p></details></section>'''
78 return markup
79
80
81 def main(output):
82     raw, processed = output / "raw", output / "processed"
83     data = json.loads((raw / "benchmark.json").read_text(encoding="utf-8"))

```

```

84 assert data["status"] == "completed" and len(data["records"]) == 8
85 assert all(sha256(ROOT / name) == digest for name, digest in data["source_sha256"].items())
86 run = Path(data["run_directory"])
87 store, frozen, actual, dates, prices = load_prepared(run)
88 summary = pd.read_csv(processed / "summary.csv", float_precision="round_trip")
89 candidates = pd.read_csv(processed / "candidates.csv", float_precision="round_trip")
90 assert len(summary) == 8 and len(candidates) == 24
91 verified_inputs, selections, objective_checks = {}, {}, []
92 for record in data["records"]:
93     date, soc = record["date"], record["initial_soc"]
94     day = int(dates.get_loc(pd.Timestamp(date)))
95     for key, value in record.items():
96         if key in summary.columns:
97             saved = summary.loc[summary.trial == record["trial"], key].iloc[0]
98             assert saved == value, (key, saved, value)
99     assert record["day_wall_seconds"] >= sum(c["wall_seconds"] for c in record["calls"])
100    for call in record["calls"]:
101        k = call["K"]
102        key = f"{date}_K{k}"
103        with np.load(raw / f"input_{key}.npz") as f:
104            net, price, weights, initial = [f[x].copy() for x in ("net", "prices", "weights", "
initial_soc")]
105            if key not in verified_inputs:
106                point, blocks, _ = choose_day(store, frozen, actual[:day], day, k, prices)
107                expected_net, expected_weights, _ = construct(point, blocks, np.tile(prices, k), len
(net))
108                np.testing.assert_array_equal(expected_net, net)
109                np.testing.assert_array_equal(expected_weights, weights)
110                np.testing.assert_array_equal(price, np.tile(prices, k))
111                assert initial == soc
112                digest = hashlib.sha256()
113                for value in (net, price, weights, np.array([soc])):
114                    value = np.ascontiguousarray(value, dtype=np.float64)
115                    digest.update(str(value.shape).encode("utf-8")); digest.update(value.tobytes())
116                assert digest.hexdigest() == data["input_hashes"][key]
117                verified_inputs[key] = digest.hexdigest()
118            assert call["input_sha256"] == verified_inputs[key]
119            selection = json.dumps(record["audit"][str(k)]["selection"], sort_keys=True)
120            assert selections.setdefault(key, selection) == selection
121            info = call["solver"]
122            assert info["reliable"] and info["mip_gap"] <= record["gap_percent"] / 100 + 1e-8
123            with np.load(raw / call["policy_file"]) as f:
124                policy = Policy(*[f[x] for x in ("grid", "charge_cap", "discharge_cap")])
125                responses = [replay(policy, path, soc) for path in net]
126                objective = float(price @ policy.grid + sum(5 * w * (price @ r["emergency"]) for w, r in
zip(weights, responses)))
127                np.testing.assert_allclose(objective, info["objective"], atol=1e-5, rtol=0)
128                gap = max(objective - info["dual_bound"], 0) / max(abs(info["dual_bound"]), 1e-8)
129                np.testing.assert_allclose(gap, info["mip_gap"], atol=1e-10, rtol=0)
130                candidate_row = candidates[(candidates.trial == record["trial"]) & (candidates.K == k)].
iloc[0]
131                assert candidate_row.accepted and candidate_row.input_sha256 == verified_inputs[key]
132                for column, expected in (("objective", objective), ("dual_bound", info["dual_bound"]), ("
achieved_gap", gap), ("wall_seconds", call["wall_seconds"])):
133                    np.testing.assert_allclose(candidate_row[column], expected, atol=1e-8, rtol=0)
134                objective_checks.append({"trial": record["trial"], "K": k, "objective": objective, "gap"
: gap})

```

```

135         if k == record["selected_K"]:
136             response = replay(policy.first(), DT * (actual[day, :, 0] - actual[day, :, 1]), soc)
137             frame = pd.read_csv(raw / f"dispatch_{record['trial']}.csv", float_precision="
round_trip")
138             assert len(frame) == 144
139             np.testing.assert_array_equal(frame.net_kwh, DT * (actual[day, :, 0] - actual[day,
:, 1]))
140             np.testing.assert_array_equal(frame.price, prices)
141             np.testing.assert_allclose(frame.grid, policy.grid[:144], atol=1e-8, rtol=0)
142             for name, values in response.items():
143                 np.testing.assert_allclose(frame[name], values, atol=1e-8, rtol=0)
144             validate_replay(frame.grid.to_numpy(), frame.net_kwh.to_numpy(), soc, response)
145             np.testing.assert_allclose(frame.planned_cost, prices * frame.grid, atol=1e-8, rtol
=0)
146             np.testing.assert_allclose(frame.emergency_cost, 5 * prices * frame.emergency, atol
=1e-8, rtol=0)
147             checks = {"actual_cost": (frame.planned_cost + frame.emergency_cost).sum(),
148                       "planned_cost": frame.planned_cost.sum(), "emergency_cost": frame.
emergency_cost.sum(),
149                       "grid_kwh": frame.grid.sum(), "emergency_kwh": frame.emergency.sum(), "
final_soc": frame.soc.iloc[-1]}
150             for name, value in checks.items():
151                 np.testing.assert_allclose(value, record[name], atol=1e-8, rtol=0)
152
153         archive = run / "raw/solver_diagnostics/precision_sweep_20260912_112216"
154         old = json.loads((archive / "benchmark.json").read_text(encoding="utf-8"))
155         old_records = {r["gap_percent"]: r for r in old["records"] if r["gap_percent"] in (1, 3)}
156         assert old["status"] == "completed" and old_records[1]["accepted_candidates"] == 0 and
old_records[3]["accepted_candidates"] == 3
157         for k, call1, call3 in zip((1, 2, 3), old_records[1]["calls"], old_records[3]["calls"]):
158             assert call1["input_sha256"] == call3["input_sha256"]
159             assert call1["K"] == call3["K"] == k
160             for call in (call1, call3):
161                 info = call["solver"]
162                 np.testing.assert_allclose(max(info["objective"] - info["dual_bound"], 0) / abs(info["
dual_bound"]), info["mip_gap"], atol=1e-10, rtol=0)
163         old_policy = pd.read_csv(archive / "policy_3pct.csv", float_precision="round_trip")
164         policy = Policy(*[old_policy[key].to_numpy() for key in ("grid", "charge_cap", "discharge_cap")
])
165         day = int(dates.get_loc(pd.Timestamp(old["date"])))
166         response = replay(policy.first(), DT * (actual[day, :, 0] - actual[day, :, 1]), old["initial_soc
"])
167         np.testing.assert_allclose(prices @ (policy.grid[:144] + 5 * response["emergency"]), old_records
[3]["actual_day_cost"], atol=1e-8, rtol=0)
168         np.testing.assert_allclose(response["soc"][-1], old_records[3]["final_soc"], atol=1e-8, rtol=0)
169         review = {"passed": True, "input_instances": len(verified_inputs), "candidate_policies": len(
objective_checks),
170                  "actual_replay_days": len(summary), "real_replay_rows": 144 * len(summary),
171                  "all_candidate_gaps_and_physics_passed": True, "formal_files_unchanged": True,
172                  "archived_3pct_actual_replay_passed": True, "archived_1pct_has_no_accepted_solution":
True,
173                  "objective_checks": objective_checks}
174         write_json(raw / "independent_review.json", review)
175
176         means = summary.groupby(["date", "gap_percent"]).mean(numeric_only=True)
177         rows, comparison = [], []
178         for date in data["dates"]:

```

```

179     a, b = means.loc[(date, 1)], means.loc[(date, 3)]
180     saving = 100 * (1 - b.day_wall_seconds / a.day_wall_seconds)
181     delta = b.actual_cost - a.actual_cost
182     rows.append([date, n(a.day_wall_seconds), n(b.day_wall_seconds), f"{saving:.1f}%",
183                 n(a.actual_cost), n(b.actual_cost), f"{delta:+.2f} ({100 * delta / a.
actual_cost:+.3f}%)"])
184     comparison.append({"date": date, "seconds_1pct": a.day_wall_seconds, "seconds_3pct": b.
day_wall_seconds,
185                      "time_saved_percent": saving, "actual_cost_1pct": a.actual_cost, "
actual_cost_3pct": b.actual_cost,
186                      "actual_cost_change_yuan": delta, "actual_cost_change_percent": 100 *
delta / a.actual_cost})
187 write_csv(processed / "comparison.csv", pd.DataFrame(comparison))
188 primary = table(["日期", "1% 用时 / 秒", "3% 用时 / 秒", "3% 节省时间", "1% 实际费 / 元", "3% 实际费 /
元", "费用变化: 3% - 1%"], rows)
189 state_rows = []
190 for date in data["dates"]:
191     for gap in (1, 3):
192         a = means.loc[(date, gap)]
193         ks = "/".join(str(k) for k in sorted(summary[(summary.date == date) & (summary.
gap_percent == gap)].selected_K.unique()))
194         state_rows.append([date, f"{gap}%", f"K={ks}", n(a.initial_soc), n(a.final_soc), n(a.
grid_kwh), n(a.emergency_kwh), n(a.planned_cost), n(a.emergency_cost)])
195 states = table(["日期", "精度", "选中时域", "初始储电 / kWh", "末储电 / kWh", "计划购电 / kWh", "紧急购
电 / kWh", "计划费用 / 元", "紧急费用 / 元"], state_rows)
196 repeats = table(["日期", "轮次", "精度", "完整选策用时 / 秒", "合格候选", "真实回放费用 / 元"],
197                 [[r.date, r.repeat, f"{r.gap_percent}%", n(r.day_wall_seconds, 3), f"{r.
accepted_candidates}/3", n(r.actual_cost)] for r in summary.sort_values(["date", "repeat", "
gap_percent"])])
198 candidate_rows = []
199 for date in data["dates"]:
200     for k in (1, 2, 3):
201         a = candidates[(candidates.date == date) & (candidates.K == k) & (candidates.gap_percent
== 1)]
202         b = candidates[(candidates.date == date) & (candidates.K == k) & (candidates.gap_percent
== 3)]
203         candidate_rows.append([date, k, int(a.scenarios.iloc[0]), n(a.objective.mean()), n(b.
objective.mean()),
204                                f"{100*(b.objective.mean()/a.objective.mean()-1):+.3f}%", f"{100*
a.achieved_gap.max():.4f}%", f"{100*b.achieved_gap.max():.4f}%"])
205 objectives = table(["日期", "K / 天", "场景数", "1% 场景目标 / 元", "3% 场景目标 / 元", "目标变化", "1%
实际 gap", "3% 实际 gap"], candidate_rows)
206 hard = table(["精度", "原实测用时 / 秒", "合格候选", "真实回放费用 / 元", "说明"],
207              [[["1%", n(old_records[1]["day_wall_seconds"]), "0/3", "—", "限时失败: 不是成功求解时间"
],
208                 ["3%", n(old_records[3]["day_wall_seconds"]), "3/3", n(old_records[3]["
actual_day_cost"]), "合格解: K=2"]]])
209 hard_objectives = table(["时域", "1% 未验收上界 / 元", "3% 已验收上界 / 元", "1% 实际 gap", "3% 实际
gap"],
210                        [[f"K={a['K']}", n(a['solver']['objective']), n(b['solver']['objective'
]), f"{100*a['solver']['mip_gap']:.4f}%", f"{100*b['solver']['mip_gap']:.4f}%"] for a, b in zip(
old_records[1]["calls"], old_records[3]["calls"])])
211 average = summary.groupby("gap_percent").day_wall_seconds.mean()
212 overall_saved = 100 * (1 - average.loc[3] / average.loc[1])
213 tail = tail_evidence(run, output)
214 archive_link = html.escape(os.path.relpath(archive / "benchmark.json", processed))
215 page = f'<!doctype html>

```

```

216 <html lang="zh-CN"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,
    initial-scale=1">
217 <title>第二问 • 1% 与 3% 精度实测</title>
218 <style>
219 :root{--ink:#152a31;--muted:#52656a;--paper:#f7f6f1;--line:#d6dfda;--green:#17664f;--gold:#a76720}}
220 *{box-sizing:border-box}body{margin:0;background:var(--paper);color:var(--ink);font-family:"
    PingFang SC","Noto Sans CJK SC",sans-serif;line-height:1.7}}
221 main{max-width:1400px;margin:auto;padding:42px 38px 70px}header{{border-top:5px solid var(--green)
    ;padding-top:20px}}.eyebrow{{font-size:12px;letter-spacing:.17em;color:var(--green);font-weight
    :700}}h1{{font-family:"Songti SC",serif;font-size:clamp(30px,4vw,50px);margin:10px 0;line-height
    :1.25;letter-spacing:-.025em}}h2{{font-size:21px;margin:0 0 10px}}p{{margin:10px 0}}.sub,.note{{
    color:var(--muted);font-size:13px}}.lead{{font-size:17px;max-width:960px}}.stats{{display:grid;
    grid-template-columns:repeat(3,1fr);border-top:1px solid var(--line);border-bottom:1px solid var
    (--line);margin:26px 0}}.stat{{padding:17px 22px;border-right:1px solid var(--line)}}.stat:first
    -child{{padding-left:0}}.stat:last-child{{border:0}}.stat strong{{font-family:"Georgia",serif;
    font-size:35px;display:block;color:var(--green);line-height:1.3}}.stat span{{font-size:13px;
    color:var(--muted)}}section{{margin-top:35px}}.table-wrap{{overflow-x:auto;border:1px solid var
    (--line);background:#fff}}table{{border-collapse:collapse;width:100%;font-size:13px;font-variant
    -numeric:tabular-nums;white-space:nowrap}}th{{background:#e9efea;text-align:right;font-weight
    :600;font-size:12px;padding:13px 15px}}td{{padding:13px 15px;border-top:1px solid #e4e9e5;text-
    align:right}}th:first-child,td:first-child{{text-align:left}}tbody tr: hover{{background:#f4f8f4
    }}.callout{{padding:15px 20px;background:#eaf1eb;border-left:3px solid var(--green);font-size:14
    px;margin:16px 0}}.caution{{background:#f5eddf;border-color:var(--gold)}}details{{margin-top:20
    px}}summary{{cursor:pointer;font-weight:600;padding:12px 0}}a{{color:var(--green);text-decoration
    -offset:3px}}.links{{display:flex;flex-wrap:wrap;gap:10px 25px}}footer{{margin-top:35px;border-
    top:1px solid var(--line);padding-top:17px;font-size:12px;color:var(--muted)}}code{{font-size:12
    px;overflow-wrap:anywhere}}@media(max-width:650px){{main{{padding:22px 16px 40px}}.stats{{grid-
    template-columns:1fr}}.stat,.stat:first-child{{padding:12px 0;border-right:0;border-bottom:1px
    solid var(--line)}}.stat strong{{font-size:28px}}th,td{{padding:11px}}.lead{{font-size:15px}}}}
    @media print{{main{{padding:0}}details{{display:block}}.table-wrap{{overflow:visible}}table{{
    font-size:10px}}th,td{{padding:6px}}}}
222 </style></head><body><main>
223 <header><div class="eyebrow">Q2 / PRECISION BENCHMARK • 2026.09.12</div><h1>1% 与 3%，实际差多少？</h1
    >
224 <p class="sub">长尾核查修订 • 历史 48 日耗时分布 + 2 个普通日期配对实测 + 1 个既有困难日</p>
225 <p class="lead"><b>现有样本不足以估计全任务的提速幅度。</b>历史 48 日中，7 个需补算日期占约 68% 的推进时间。新增两
    日样本全部通过较快的 LP 证书流程，没有覆盖 MILP 和补算瓶颈；下方的 {overall_saved:.1f}% 仅是这两个普通样本的
    局部结果。</p>
226 <p class="note">实验约定：精度采用项目定义 <b>gap = (可行费用 U - 全局下界 L) / |L|</b>。小量测试缩减日期数量，
    保留每个日期的全部场景、10 分钟时段和 K=1/2/3 选择规则。</p></header>
227 {tail}
228 <div class="stats"><div class="stat"><strong>{average.loc[1]:.2f} s</strong><span>两个普通样本 • 1% 平
    均用时</span></div><div class="stat"><strong>{average.loc[3]:.2f} s</strong><span>两个普通样本 •
    3% 平均用时</span></div><div class="stat"><strong>24 / 24</strong><span>普通样本候选全部通过 LP 证书
    </span></div></div>
229 <section><h2>两个普通样本的实际费用与时间</h2><p class="note">2 个日期 × 2 档精度 × 2 次重复。每个日期使用相同
    历史信息、预测、场景权重及初始储电量；数值为两次重复均值。费用变化为 3% 减 1%，负值表示 3% 当日回放费用较低。重复
    两次只观察计时波动，不增加独立日期的覆盖。</p>{primary}
230 <div class="callout">“实际费用”是冻结日前策略后，用附件中的真实负荷与光伏逐时回放得到的仿真电费，等于计划购电费 +
    紧急购电费（5 倍电价）。不是场景期望目标，也不是现场账单。</div></section>
231 <section><h2>为什么费用不能单独看</h2>{states}<p class="note">K=1/2/3 分别代表 24/48/72 小时规划，最终只执行
    首日。2 月 1 日的精度变化触发了不同的时域选择。每个测试日从原正式轨迹的同一个 SOC 出发，两个日期不串联；未储电量
    有差异，未折算残值，因此不能据此推断全年哪档更省钱。</p></section>
232 <section><h2>同一时域的优化目标</h2><p class="note">以下目标覆盖 K 天、按场景概率加权，只在一行横向比较。目标为
    两次均值，实际 gap 列取两次最大值；达到 3% 门槛不表示误差恰好为 3%。</p>{objectives}</section>
233 <section><h2>慢日补充：5 月 5 日的既有实测</h2><p class="note">复用此前同机、同输入、统一算法的单次记录，本次没
    有重跑。每个候选 MILP 的时限为 120 秒，表中包含 LP、建模与选策开销，因此整日可以超过 360 秒。</p>{hard}

```

```

234 <div class="callout caution">这一天的 1% 没有合格结果，不能填写两档的实际电费差，也不能把 433 秒当作成功耗时。1%
    尝试与 3% 成功记录的同 K 可行费用上界相同，额外时间主要用于收紧最优性证明；日志上界相同不等于已验证两份策略逐项
    相同。</div>
235 <details><summary>查看慢日的上界与证书</summary>{hard_objectives}</details></section>
236 <section><h2>实验口径与复核</h2><p>新测试日期固定为评价期首日 2 月 1 日和方案第一个指定日 3 月 20 日；每档重复两
    次，按 1%→3%、3%→1% 交替执行。统一使用现有 LP 下界证书，必要时进入等价 HiGHS MILP；各次重新求解，不跨精度复用
    候选策略或求解界。</p>
237 <p class="note">只在隔离测试进程中统一了 1% 的后备求解入口，正式源码、2% 配置及冻结准备库摘要保持不变。计时包括完整
    make_day 的选择、各 K 求解、内部验收及测试记录的轻量存档开销，排除共同数据加载（{data['shared_load_seconds'
    ]:.2f} 秒）和最终真实回放。每候选 MILP 时限 120 秒，无 900 秒补算。后台正式任务仍在运行，重复仅两次；本表不作
    统计显著性或全年提速保证。</p>
238 <p class="note">独立回读确认：6 组输入与原准备库逐值一致；24 份候选策略的费用、gap 和物理回放通过；8 个日结果共
    1,152 行与真实输入、电价、SOC 和费用逐值核对通过。所有新候选均由现有 LP 下界证书通过，未进入后备 MILP。慢日另核
    对原日志与 3% 实际回放。</p>
239 <details><summary>展开 8 次原始测量</summary>{repeats}</details>
240 <details><summary>查看软件与机器信息</summary><p class="note">{html.escape(data['machine'])} · {data['
    logical_cpus']} 逻辑 CPU · {html.escape(str(data['versions']))}。BLAS / OMP 均为单线程设置，后台进程
    快照见原始 JSON。</p></details>
241 <p class="links"><a href="comparison.csv" download>下载对照表 CSV</a><a href="summary.csv" download>8
    次原始测量 CSV</a><a href="candidates.csv" download>24 个候选 CSV</a><a href="../raw/benchmark.
    json">完整实验记录</a><a href="../raw/independent_review.json">独立验收记录</a><a href="{
    archive_link}">慢日原始记录</a></p></section>
242 <footer>结论范围：历史 48 日证明长尾负担明显；两个普通日及一个既有难例不足以估计全任务精度效应。分层困难日比较与连续全
    年费用差尚未得到。正式运行精度仍为 2%。</footer>
243 </main></body></html>'''
244     assert all(not c["solver"].get("branch_and_bound_required", True) for r in data["records"] for c
    in r["calls"])
245     (processed / "report.html").write_text(page, encoding="utf-8")
246     print(json.dumps({"report": str(processed / "report.html"), "average_1pct_seconds": average.loc
    [1], "average_3pct_seconds": average.loc[3], "time_saved_percent": overall_saved, "comparisons":
    comparison}, ensure_ascii=False, indent=2))
247
248
249 if __name__ == "__main__":
250     main(Path(sys.argv[1]).resolve())

```

src/q2/benchmarks/precision_sweep.py

```

1  """同一真实日、相同求解路线的1%—5%限时比较；不修改正式执行。"""
2
3  import datetime as dt
4  import hashlib
5  import io
6  import json
7  from pathlib import Path
8  import sys
9  import time
10
11 import numpy as np
12 import pandas as pd
13
14 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
15 import rolling
16 from data import ROOT, sha256, write_csv, write_json
17 from dispatch import load_prepared
18 from linear_policy import solve_linear_policy

```



```

19 from policy import DT, validate_replay
20
21
22 def main():
23     run = ROOT / "outputs/q2/dispatch_runs/20260911_224501"
24     output = run / "raw/solver_diagnostics" / dt.datetime.now().strftime("precision_sweep_%Y%m%d_%H%M%S")
25     output.mkdir()
26     date = "2025-05-05"
27     sources = {str(p.relative_to(ROOT)): sha256(p) for p in (ROOT / "src/q2").glob("*.py")}
28     precision_hash = sha256(run / "raw/solver_precision_revision.json")
29     store, frozen, actual, dates, prices = load_prepared(run)
30     directory = run / "processed/main"
31     for _ in range(10):
32         checkpoint = (directory / "checkpoint.json").read_bytes()
33         raw = (directory / "daily.csv").read_bytes()
34         if (checkpoint == (directory / "checkpoint.json").read_bytes()
35             and hashlib.sha256(raw).hexdigest() == json.loads(checkpoint)["sha256"]["daily.csv"]
36         ):
37             break
38     else:
39         raise RuntimeError("未取得稳定的已提交逐日表")
40     daily = pd.read_csv(io.BytesIO(raw), float_precision="round_trip")
41     old = json.loads((directory / f"daily_audit/{date}.json").read_text(encoding="utf-8"))
42     soc = float(daily.loc[daily.date == date, "initial_soc"].iloc[0])
43     day = int(dates.get_loc(pd.Timestamp(date)))
44     records, summary, candidate_rows, input_hashes = [], [], [], {}
45     metadata = {
46         "status": "running", "date": date, "initial_soc": soc,
47         "created_at": dt.datetime.now(dt.timezone.utc).isoformat(),
48         "candidate_time_limit_seconds": 120, "supplemental_retries": False,
49         "gap_order_percent": [3, 4, 5, 2, 1], "official_gap_unchanged": .02,
50         "source_sha256": sources, "benchmark_source_sha256": sha256(Path(__file__)),
51         "official_precision_sha256": precision_hash,
52         "timing_scope": "
make_day_K1_K2_K3_including_selection_and_validation_excluding_shared_library_load_and_final_actual_replay
",
53         "same_algorithm": "original_LP_certificate_then_existing_equivalent_HiGHS_MILP_for_all_gaps"
54     ,
55         "one_percent_difference_from_legacy": "
isolated_native_fallback_redirected_to_same_current_equivalent_MILP;
production_lpct_route_not_changed",
56         "limitations": "one_day_one_trial_per_gap_with_two_live_official_workers;
censored_failures_are_not_time_to_success; not_an_annual_ETA",
57         "records": records,
58     }
59     write_json(output / "benchmark.json", metadata)
60     original_solve, original_native = rolling.solve, rolling.solve_policy
61     calls = []
62
63     def same_linear(net, price, weights, initial, *, gap, time_limit, seed_policy, known_lower_bound):
64         # 原1%入口会调用native: 仅在本隔离进程统一算法, 避免把算法差异当精度效应。
65         return solve_linear_policy(net, price, weights, initial, gap=gap,
66                                     time_limit=time_limit, seed_policy=seed_policy)
67
68     def timed_solve(net, price, weights, initial, **kwargs):

```

```

67     k = len(price) // 144
68     digest = hashlib.sha256()
69     for value in (net, price, weights, np.array([initial])):
70         array = np.ascontiguousarray(value, dtype=np.float64)
71         digest.update(str(array.shape).encode("utf-8")); digest.update(array.tobytes())
72     fingerprint = digest.hexdigest()
73     assert fingerprint == input_hashes.setdefault(k, fingerprint), "不同精度输入不一致"
74     started = time.perf_counter()
75     policy, responses, info = original_solve(net, price, weights, initial, **kwargs)
76     elapsed = time.perf_counter() - started
77     if policy is not None:
78         for path, response in zip(net, responses):
79             validate_replay(policy.grid, path, initial, response)
80     if info["reliable"]:
81         assert policy is not None and info["mip_gap"] <= kwargs["solver_gap"] + 1e-8
82         call = {"K": k, "input_sha256": fingerprint, "wall_seconds": elapsed, "solver": info}
83         calls.append(call)
84         print(json.dumps({"gap_percent": 100 * kwargs["solver_gap"], "K": k,
85             "seconds": elapsed, "reliable": info["reliable"], "gap": info.get("mip_gap
86 "))), flush=True)
87     return policy, responses, info
88
89 rolling.solve_policy, rolling._solve = same_linear, timed_solve
90 try:
91     for percent in metadata["gap_order_percent"]:
92         calls = []
93         started = time.perf_counter()
94         policy = None
95         try:
96             policy, _, _, audit = rolling.make_day(
97                 store, frozen, actual[:day], day, prices, soc, solver_gap=percent / 100)
98         except rolling.ComputationalLimit as error:
99             audit = error.details
100         wall = time.perf_counter() - started
101         assert len(calls) == 3 and [c["K"] for c in calls] == [1, 2, 3]
102         for k in ("1", "2", "3"):
103             assert audit[k]["selection"] == old[k]["selection"]
104             new_k, old_k = audit[k].get("error", audit[k]), old[k].get("error", old[k])
105             assert new_k["M"] == old_k["M"]
106             for count, entry in new_k["candidates"].items():
107                 assert entry["scenarios"] == old_k["candidates"][count]["scenarios"]
108         accepted = sum(c["solver"]["reliable"] for c in calls)
109         row = {"gap_percent": percent, "day_wall_seconds": wall,
110             "accepted_candidates": accepted, "total_candidates": 3,
111             "all_candidates_passed": accepted == 3, "selected_K": audit.get("selected_K"),
112             "K1_seconds": calls[0]["wall_seconds"], "K2_seconds": calls[1]["wall_seconds"],
113             "K3_seconds": calls[2]["wall_seconds"]}
114         for call in calls:
115             info = call["solver"]
116             candidate_rows.append({"gap_percent": percent, "K": call["K"],
117                 "wall_seconds": call["wall_seconds"], "solver_seconds": info[
118 "solve_seconds"],
119                 "achieved_gap": info.get("mip_gap"), "accepted": info["
120 reliable"],
121                 "status": info["status"], "objective": info.get("objective"),
122                 "dual_bound": info.get("dual_bound")})
123         if policy is not None:

```



```

121         frame = rolling.dispatch_frame(dates[day], policy, DT * (actual[day, :, 0] - actual[
day, :, 1]), soc, prices)
122         row.update(actual_day_cost=float(frame.planned_cost.sum() + frame.emergency_cost.sum
()),
123                 final_soc=float(frame.soc.iloc[-1]))
124         write_csv(output / f"policy_{percent}pct.csv", pd.DataFrame({
125             "grid": policy.grid, "charge_cap": policy.charge_cap, "discharge_cap": policy.
discharge_cap}))
126         summary.append(row); records.append(**row, "calls": calls, "audit": audit})
127         write_json(output / "benchmark.json", metadata)
128         write_csv(output / "summary.csv", pd.DataFrame(summary).sort_values("gap_percent"))
129         write_csv(output / "candidates.csv", pd.DataFrame(candidate_rows).sort_values(["
gap_percent", "K"]))
130         print("COMPLETE", json.dumps(row), flush=True)
131     finally:
132         rolling._solve, rolling.solve_policy = original_solve, original_native
133         assert all(sha256(ROOT / name) == digest for name, digest in sources.items())
134         assert sha256(run / "raw/solver_precision_revision.json") == precision_hash
135         metadata.update(status="completed", finished_at=dt.datetime.now(dt.timezone.utc).isoformat(),
136                       input_sha256_by_K=input_hashes)
137         write_json(output / "benchmark.json", metadata)
138         print("RESULT_DIRECTORY", output, flush=True)
139
140
141 if __name__ == "__main__":
142     main()

```

src/q2/checkpoint.py

```

1  """完整日前缀验收：数值损坏、跨日断链及未提交的CSV均禁止续算。"""
2
3  import json
4  import os
5  import shutil
6
7  import numpy as np
8  import pandas as pd
9
10 from data import sha256, write_csv, write_json
11 from export_dispatch import validate_dispatch
12 from policy import DT, PHYSICAL_TOL, Policy, replay
13
14 TABLES = ("dispatch.csv", "daily.csv", "calibration.csv")
15
16
17 def atomic_json(path, value):
18     path.parent.mkdir(parents=True, exist_ok=True)
19     pending = path.with_name(path.name+".pending")
20     write_json(pending, value)
21     with pending.open("r+b") as stream:
22         os.fsync(stream.fileno())
23     pending.replace(path)
24
25
26 def _matches(output, manifest):
27     return (set(manifest.get("sha256", {})) == set(TABLES))

```

```

28         and all((output/name).exists() and sha256(output/name) == digest
29                 for name, digest in manifest["sha256"].items()))
30
31
32 def recover_checkpoint(output):
33     """仅恢复有提交中断标记的事务：无标记的摘要损坏仍拒绝，不静默重封存。"""
34     marker = output/"checkpoint_transaction.json"
35     if not marker.exists():
36         return
37     transaction = json.loads(marker.read_text(encoding="utf-8"))
38     manifest = output/"checkpoint.json"
39     if (manifest.exists() and json.loads(manifest.read_text(encoding="utf-8")) == transaction["
40         target"])
41         and _matches(output, transaction["target"]):
42             marker.unlink()
43             return
44     base = transaction["base"]
45     if base is None:
46         for name in (*TABLES, "checkpoint.json"):
47             (output/name).unlink(missing_ok=True)
48     else:
49         backup = output/"checkpoint_previous"
50         if not _matches(backup, base):
51             raise ValueError("提交中断且上一完整检查点备份摘要不符，禁止自动覆盖")
52         for name in TABLES:
53             pending = output/(name+".recovery")
54             shutil.copyfile(backup/name, pending)
55             with pending.open("r+b") as stream:
56                 os.fsync(stream.fileno())
57             pending.replace(output/name)
58             atomic_json(manifest, base)
59     marker.unlink()
60     atomic_json(output/"checkpoint_recovery.json", {
61         "restored_days": base["days"] if base else 0,
62         "discarded_uncommitted_days": transaction["target"]["days"],
63         "reason": "interrupted_multi_csv_commit_restored_last_verified_checkpoint"})
64
65 def save_checkpoint(output, frame, daily, calibration):
66     recover_checkpoint(output)
67     manifest = output/"checkpoint.json"
68     base = json.loads(manifest.read_text(encoding="utf-8")) if manifest.exists() else None
69     if base is not None:
70         if not _matches(output, base):
71             raise ValueError("保存前检查点摘要不符，禁止覆盖已提交结果")
72         backup = output/"checkpoint_previous"
73         backup.mkdir(exist_ok=True)
74         for name in TABLES:
75             shutil.copyfile(output/name, backup/name)
76             with (backup/name).open("r+b") as stream:
77                 os.fsync(stream.fileno())
78             atomic_json(backup/"checkpoint.json", base)
79     for name, table in zip(TABLES, (frame, pd.DataFrame(daily), pd.DataFrame(calibration))):
80         pending = output/(name+".pending")
81         write_csv(pending, table)
82         with pending.open("r+b") as stream:
83             os.fsync(stream.fileno())

```

```

84     target = {"days": len(daily), "sha256": {name: sha256(output/(name+".pending")) for name in
TABLES}}
85     atomic_json(output/"checkpoint_transaction.json", {"base": base, "target": target})
86     for name in TABLES:
87         (output/(name+".pending")).replace(output/name)
88     atomic_json(manifest, target)
89     (output/"checkpoint_transaction.json").unlink()
90
91
92 def read_checkpoint(output, actual, dates, prices, initial_soc, start, end, settings):
93     recover_checkpoint(output)
94     status = json.loads((output/"run_status.json").read_text(encoding="utf-8"))
95     if status.get("settings") != settings or status.get("start_index") != start or status.get("
end_index") != end:
96         raise ValueError("已有回放与当前实验设置不同, 拒绝覆盖")
97     manifest = output/"checkpoint.json"
98     if manifest.exists():
99         hashes = json.loads(manifest.read_text(encoding="utf-8"))["sha256"]
100         if any(sha256(output/name) != digest for name, digest in hashes.items()):
101             raise ValueError("检查点摘要不符或CSV事务尚未完整提交")
102     previous = pd.read_csv(output/"dispatch.csv", parse_dates=["date", "timestamp"], float_precision
="round_trip")
103     completed = previous.date.nunique()
104     if completed > end-start or not np.array_equal(previous.date.unique(), dates[start:start+
completed].to_numpy()):
105         raise ValueError("回放检查点不是完整连续前缀")
106     rigid = settings.get("kind") == "rigid"
107     validate_dispatch(previous, rigid=rigid)
108     np.testing.assert_allclose(previous.initial_soc.iloc[0], initial_soc, atol=PHYSICAL_TOL, rtol=0)
109     expected_net = DT*(actual[start:start+completed, :, 0]-actual[start:start+completed, :, 1]).
ravel()
110     np.testing.assert_allclose(previous.net_kwh, expected_net, atol=PHYSICAL_TOL, rtol=0)
111     np.testing.assert_array_equal(previous.price, np.tile(prices, completed))
112     daily = pd.read_csv(output/"daily.csv", float_precision="round_trip")
113     calibration = pd.read_csv(output/"calibration.csv", float_precision="round_trip")
114     if len(daily) != completed or daily.date.tolist() != [str(d.date()) for d in dates[start:start+
completed]]:
115         raise ValueError("逐日摘要与检查点日期不一致")
116     for i, (date, day) in enumerate(previous.groupby("date", sort=False)):
117         audit = json.loads((output/f"daily_audit/{date.date()}.json").read_text(encoding="utf-8"))
118         frozen = pd.read_csv(output/f"frozen_plans/{date.date()}.csv", float_precision="round_trip")
119         k = audit["selected_K"]
120         if len(frozen) != k*144 or not audit[str(k)]["selected_solver"]["reliable"]:
121             raise ValueError("冻结策略长度或求解验收状态不符")
122         for key in ("grid", "charge_cap", "discharge_cap"):
123             np.testing.assert_array_equal(frozen[key].iloc[:144], day[key])
124         reserve = (frozen.soc_reserve_kwh.iloc[:144].to_numpy()
125                     if "soc_reserve_kwh" in frozen else np.zeros(144))
126         if "soc_reserve_kwh" in day:
127             np.testing.assert_array_equal(reserve, day.soc_reserve_kwh)
128         if not rigid:
129             policy = Policy(day.grid, day.charge_cap, day.discharge_cap, reserve)
130             response = replay(policy, day.net_kwh.to_numpy(), float(day.initial_soc.iloc[0]))
131             for key, values in response.items():
132                 np.testing.assert_allclose(day[key], values, atol=PHYSICAL_TOL, rtol=0)
133             row = daily.iloc[i]
134             np.testing.assert_allclose([row.initial_soc, row.final_soc, row.cost, row.emergency_kwh],

```

```

135         [day.initial_soc.iloc[0], day.soc.iloc[-1],
136          (day.planned_cost+day.emergency_cost).sum(), day.emergency.sum()
137 ],
138         atol=PHYSICAL_TOL, rtol=0)
139     observed = calibration[calibration.origin == str(date.date())]
140     expected = {(h, nominal) for h in range(min(k, len(dates)-start-i)) for nominal in (.8, .9)}
141     if len(observed) != len(expected) or set(zip(observed.horizon, observed.nominal)) !=
142     expected:
143         raise ValueError("预测步长覆盖率记录缺失或重复")
144     return previous, daily.to_dict("records"), calibration.to_dict("records")

```

src/q2/comparators.py

```

1  """2.10: 刚性场景计划、完全信息及fixed-g Oracle: 均与主政策入口分离。"""
2
3  import time
4
5  import numpy as np
6  from scipy.optimize import linprog
7  from scipy.sparse import coo_matrix
8
9  from policy import CAP, E_MIN, E_MAX, ETA_C, ETA_D, PHYSICAL_TOL, Policy, replay, validate_replay
10
11
12  def rigid_schedule(net, prices, weights, initial_soc, *, fixed_grid=None):
13      """共同c/r轨迹的精确LP。
14
15      无回售、非负能价、可弃电且无终端等式时，LP同时充放电可消去：令
16       $a = \min(c, r/(\eta c \eta d))$ ,  $c' = c - a$ ,  $r' = r - \eta c \eta d \cdot a$ 。SOC不变，母线需求减少，
17      仅减少q/增加w，故费用不增；投影后与LP下界一致才接受为精确解。
18      """
19      started = time.perf_counter()
20      net, prices, weights = map(lambda x: np.asarray(x, dtype=float), (net, prices, weights))
21      s, t = net.shape
22      if prices.shape != (t,) or weights.shape != (s,) or not np.isclose(weights.sum(), 1) or min(
23          weights) <= 0 or min(prices) <= 0:
24          raise ValueError("对照输入维度/概率/电价不符")
25      if not np.isfinite(net).all() or not E_MIN <= initial_soc <= E_MAX:
26          raise ValueError("对照净负荷/初态无效")
27      size = 4*t+2*s*t
28      g, c, r, e = [np.arange(k*t, (k+1)*t) for k in range(4)]
29      q = np.arange(4*t, 4*t+s*t).reshape(s, t)
30      w = np.arange(4*t+s*t, size).reshape(s, t)
31      objective = np.zeros(size)
32      objective[g] = prices
33      objective[q] = 5*weights[:, None]*prices
34      bounds = [(0, None)]*size
35      for indices, bound in ((c, (0, CAP)), (r, (0, CAP)), (e, (E_MIN, E_MAX))):
36          for i in indices:
37              bounds[i] = bound
38      if fixed_grid is not None:
39          fixed_grid = np.asarray(fixed_grid)
40          if fixed_grid.shape != (t,) or not np.isfinite(fixed_grid).all() or (fixed_grid < 0).any():
41              raise ValueError("Oracle固定购电输入不符")
42          for i, value in zip(g, fixed_grid):
43              bounds[i] = (float(value), float(value))

```

```

43 rows, cols, coefficients, rhs = [], [], [], []
44 def equation(terms, value):
45     row = len(rhs)
46     for col, coefficient in terms:
47         rows.append(row); cols.append(int(col)); coefficients.append(coefficient)
48     rhs.append(value)
49 for i in range(t):
50     terms = [(e[i], 1), (c[i], -ETA_C), (r[i], 1/ETA_D)]
51     if i:
52         terms.append((e[i-1], -1))
53     equation(terms, initial_soc if i == 0 else 0)
54 for j in range(s):
55     for i in range(t):
56         equation([(g[i], 1), (r[i], 1), (q[j, i], 1), (c[i], -1), (w[j, i], -1)], net[j, i])
57 matrix = coo_matrix((coefficients, (rows, cols)), shape=(len(rhs), size)).tocsc()
58 solved = linprog(objective, A_eq=matrix, b_eq=rhs, bounds=bounds, method="highs")
59 if not solved.success:
60     raise RuntimeError(f"对照/Oracle LP失败: {solved.message}")
61 grid, charge, discharge = np.maximum(solved.x[g], 0), np.maximum(solved.x[c], 0), np.maximum(
62     solved.x[r], 0)
63 removed = np.minimum(charge, discharge/(ETA_C*ETA_D))
64 charge, discharge = charge-removed, discharge-ETA_C*ETA_D*removed
65 soc = initial_soc+np.cumsum(ETA_C*charge-discharge/ETA_D)
66 responses = []
67 for path in net:
68     gap = path+charge-grid-discharge
69     result = {"charge": charge, "discharge": discharge, "emergency": np.maximum(gap, 0),
70             "unused": np.maximum(-gap, 0), "soc": soc}
71     validate_replay(grid, path, initial_soc, result, rigid=True)
72     responses.append(result)
73 exact_cost = float(prices@grid + sum(5*p*(prices@response["emergency"]) for p, response in zip(
74     weights, responses)))
75 if not np.isclose(exact_cost, solved.fun, rtol=1e-8, atol=1e-5):
76     raise ValueError("去除充放电循环后费用未与LP下界一致")
77 info = {"objective": exact_cost, "lower_bound": float(solved.fun), "reliable": True,
78        "solve_seconds": time.perf_counter()-started, "status": "optimal",
79        "continuous_variables": size, "binary_variables": 0, "constraints": len(rhs),
80        "cycle_removed_kwh": float(removed.sum()), "solver": "scipy.linprog/HiGHS",
81        "mip_gap": 0.0, "role": "rigid_shared_storage" if s > 1 else "perfect_information"}
82 return grid, responses, info
83
84 def deterministic_policy(net, prices, initial_soc, reserve=None):
85     """确定性退化：若LP下界被同一反馈映射达到，即取得可验证的全局最优证书。
86
87     LP只是lower-bound计算器：只有原模型回放费用等于该下界才返回可靠策略。
88     无法闭合时返回None，由调用方转入完整indicator模型，绝不松弛主模型。
89     """
89     net, prices = np.asarray(net), np.asarray(prices)
90     grid, responses, info = rigid_schedule(net[None], prices, [1.], initial_soc)
91     policy = Policy(grid, responses[0]["charge"], responses[0]["discharge"], reserve)
92     actual = replay(policy, net, initial_soc)
93     value = float(prices@grid+5*actual["emergency"])
94     closed = bool(np.isclose(value, info["lower_bound"], rtol=1e-9, atol=PHYSICAL_TOL))
95     return (policy if closed else None), [actual], {**info, "objective": value,
96         "role": "deterministic_feedback_lower_bound_certificate", "reliable": closed,
97         "certificate_gap_yuan": value-info["lower_bound"]}

```

```

98
99
100 def oracle_pair(real_net, prices, initial_soc, planned_grid, actual_cost):
101     """两类Oracle仅接收已经完成的真实回放；调用方不得回流至预测/选模。"""
102     _, _, perfect = rigid_schedule(np.asarray(real_net)[None], prices, [1.], initial_soc)
103     _, _, response = rigid_schedule(np.asarray(real_net)[None], prices, [1.], initial_soc,
104                                     fixed_grid=planned_grid)
105     pi, fixed = perfect["objective"], response["objective"]
106     if pi > fixed+PHYSICAL_TOL or fixed > actual_cost+PHYSICAL_TOL:
107         raise ValueError("Oracle下界嵌套失败")
108     return {"perfect_information": perfect, "fixed_grid_information": response,
109            "actual_cost": actual_cost, "Gap_PI": (actual_cost-pi)/pi if pi else None,
110            "Gap_resp": (actual_cost-fixed)/actual_cost if actual_cost else None,
111            "interpretation": "
information_value_and_policy_restriction_not_pure_parameterization_loss"}

```

src/q2/data.py

```

1  """2.1.1-2.1.5: 保留原值，核验宽表，建立右端点连续功率序列。"""
2
3  import datetime as dt
4  import hashlib
5  import json
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10 from openpyxl import load_workbook
11
12 ROOT = Path(__file__).resolve().parents[2]
13 SLOTS = 144
14 DT_HOURS = 1 / 6
15 START = pd.Timestamp("2025-01-01")
16 END = pd.Timestamp("2025-12-31")
17 FORECAST_START = pd.Timestamp("2025-02-01")
18 SERIES = ("load_kw", "pv_kw", "net_load_kw")
19 CALENDAR = ("date", "interval", "timestamp", "slot", "weekday", "weekday_name", "month")
20 SHEETS = {"load_kw": "小区负载", "pv_kw": "光伏发电实际功率"}
21
22
23 def sha256(path):
24     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
25
26
27 def write_json(path, value):
28     Path(path).parent.mkdir(parents=True, exist_ok=True)
29     Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding
30                             ="utf-8")
31
32
33 def write_csv(path, frame):
34     Path(path).parent.mkdir(parents=True, exist_ok=True)
35     frame.to_csv(path, index=False, encoding="utf-8", float_format="%.17g")
36
37 def endpoint_minutes(value):

```

```

38     """附件时间的唯一含义为区间右端点：不把裸 0:00 猜成 24:00。"""
39     if isinstance(value, dt.time):
40         if value.second or value.microsecond:
41             raise ValueError("时间存在秒数")
42         minutes = value.hour * 60 + value.minute
43     elif isinstance(value, str):
44         if value.strip() in ("0:00+1", "00:00+1", "24:00"):
45             return 1440
46         hour, minute = map(int, value.strip().split(":"))
47         if not (0 <= hour < 24 and 0 <= minute < 60):
48             raise ValueError("非法时刻")
49         minutes = hour * 60 + minute
50     elif isinstance(value, (int, float)) and not isinstance(value, bool):
51         if not np.isfinite(value) or not np.isclose(value * 1440, round(value * 1440), atol=1e-8,
52             rtol=0):
53             raise ValueError("非法 Excel 时间日分数")
54         minutes = round(value * 1440)
55     else:
56         raise ValueError("无法识别的时间格式")
57     if minutes not in range(10, 1441, 10):
58         raise ValueError("右端点必须为 00:10 至 24:00 的 10 分钟倍数")
59     return minutes
60
61 def calendar_for_day(date):
62     date = pd.Timestamp(date).normalize()
63     minute = np.arange(1, SLOTS + 1) * 10
64     def clock(m):
65         return f"{m // 60:02d}:{m % 60:02d}"
66     return pd.DataFrame({
67         "date": date, "interval": [f"{clock(m - 10)}-{clock(m)}" for m in minute],
68         "timestamp": date + pd.to_timedelta(minute, unit="min"), "slot": np.arange(1, SLOTS + 1),
69         "weekday": date.dayofweek + 1, "weekday_name": "星期" + "一二三四五六日"[date.dayofweek],
70         "month": date.month,
71     })
72
73
74 def read_attachment(path, qc_path):
75     """硬错误记录后拒绝继续：不存在统计截尾、插值或逐点改值。"""
76     checks, arrays = [], {}
77     def check(name, passed, detail):
78         checks.append({"check": name, "status": "pass" if passed else "fail", "detail": str(detail)})
79     workbook = load_workbook(path, read_only=True, data_only=True)
80     dates = pd.date_range(START, END, freq="D")
81     try:
82         check("required_sheets", set(SHEETS.values()).issubset(workbook.sheetnames), workbook.
83             sheetnames)
84         for variable, sheet_name in SHEETS.items():
85             if sheet_name not in workbook.sheetnames:
86                 continue
87             rows = list(workbook[sheet_name].iter_rows(values_only=True))
88             check(f"{variable}:shape", len(rows) == 366 and all(len(r) == 145 for r in rows),
89                 f"rows={len(rows)}, columns={len(rows[0]) if rows else 0}")
90             if len(rows) != 366 or any(len(r) != 145 for r in rows):
91                 continue
92     try:

```

```

92         minutes = [endpoint_minutes(v) for v in rows[0][1:]]
93         check(f"{variable}:144_unique_endpoints", sorted(minutes) == list(range(10, 1441,
104         observed_dates = pd.DatetimeIndex(pd.to_datetime([r[0] for r in rows[1:]], errors="
105         raise"))
106         check(f"{variable}:365_unique_dates", observed_dates.is_unique and observed_dates.
107         sort_values().equals(dates),
108         f"unique={observed_dates.nunique()}, first={observed_dates.min()}, last={
109         observed_dates.max()}")
110         values = np.array([r[1:] for r in rows[1:]], dtype=float)
111         booleans = sum(isinstance(v, bool) for r in rows[1:] for v in r[1:])
112         check(f"{variable}:numeric_not_boolean", booleans == 0, f"boolean_count={booleans}")
113         check(f"{variable}:finite_no_missing", np.isfinite(values).all(),
114         f"nonfinite_count={np.count_nonzero(~np.isfinite(values))}")
115         check(f"{variable}:nonnegative", (values >= 0).all(), f"negative_count={np.
116         count_nonzero(values < 0)}")
117         arrays[variable] = values[np.argsort(observed_dates)][:, np.argsort(minutes)]
118         except (ValueError, TypeError, OverflowError) as error:
119             check(f"{variable}:format", False, error)
120
121     finally:
122         workbook.close()
123     write_csv(qc_path, pd.DataFrame(checks))
124     if any(c["status"] == "fail" for c in checks):
125         raise ValueError(f"附件核验失败, 原始值未修改; 见 {qc_path}")
126     frame = pd.concat([calendar_for_day(d) for d in dates], ignore_index=True)
127     for variable in SHEETS:
128         frame[variable] = arrays[variable].ravel()
129     frame["net_load_kw"] = frame.load_kw - frame.pv_kw
130     check("52560_continuous_unique_timestamps", len(frame) == 52560 and frame.timestamp.is_unique
131     and
132         frame.timestamp.diff().dropna().eq(pd.Timedelta(minutes=10)).all(),
133         f"{frame.timestamp.iloc[0]} to {frame.timestamp.iloc[-1]}")
134     check("144_periods_each_source_date", frame.groupby("date").size().eq(SLOTS).all(), "按原始日期归
135     属, 非右端点自然日分组")
136     check("zero_pv_preserved", int((frame.pv_kw == 0).sum()) == int((arrays["pv_kw"] == 0).sum()),
137         f"zero_count={int((frame.pv_kw == 0).sum())}; 零值合法, 未指定或推断固定夜间时段")
138     write_csv(qc_path, pd.DataFrame(checks))
139     if any(c["status"] == "fail" for c in checks):
140         raise ValueError("长表连续性检查失败")
141     return frame, pd.DataFrame(checks)
142
143 def history_before(frame, origin):
144     """用户确认: 严格小于; 上一日末段即使已结束也排除。"""
145     origin = pd.Timestamp(origin)
146     if origin != origin.normalize():
147         raise ValueError("预测时点必须为 0:00")
148     return frame.loc[frame.timestamp < origin].copy(deep=True)
149
150 def jump_context(frame):
151     """列出所有局部转折点, 按偏离相邻值程度排序; 仅供人工复核, 不判异常。"""
152     rows = []
153     for variable in SHEETS:
154         x = frame[variable]
155         previous, following = x.shift(1), x.shift(-1)
156         reversal = ((x - previous) * (following - x) < 0)

```



```

142     view = frame.loc[reversal, ["date", "timestamp", "slot", "weekday"]].copy()
143     view["series"] = variable
144     view["value_kw"] = x[reversal]
145     view["previous_kw"] = previous[reversal]
146     view["following_kw"] = following[reversal]
147     view["neighbor_deviation_kw"] = (x - (previous + following) / 2)[reversal]
148     view["previous_day_kw"] = x.shift(144)[reversal]
149     view["previous_week_kw"] = x.shift(1008)[reversal]
150     groups = frame.groupby(["weekday", "slot"])[variable]
151     for method in ("min", "median", "max", "count"):
152         historic = groups.transform(lambda v: getattr(v.shift(1).expanding(), method)())
153         view[f"past_same_weekday_{method}"] = historic[reversal]
154     view["action"] = "retain_review_only"
155     view = view.loc[view.neighbor_deviation_kw.abs().sort_values(ascending=False).index]
156     view["review_rank"] = np.arange(1, len(view) + 1)
157     rows.append(view)
158     return pd.concat(rows, ignore_index=True)

```

src/q2/deliverables/export_partial.py

```

1  """只读导出已验收的前94日快照；不调用全年result2发布入口。"""
2
3  import datetime as dt
4  import hashlib
5  import html
6  import json
7  from pathlib import Path
8  import shutil
9  import sys
10 import tempfile
11 import zipfile
12
13 import numpy as np
14 import pandas as pd
15 from openpyxl import Workbook, load_workbook
16 from openpyxl.styles import Alignment, Font, PatternFill
17 from openpyxl.utils import get_column_letter
18
19 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
20 from checkpoint import read_checkpoint, TABLES
21 from data import ROOT, sha256, write_csv, write_json
22 from dispatch import load_prepared
23 from export_dispatch import aggregate_storage, emergency_events
24
25
26 def main():
27     run = ROOT / "outputs/q2/dispatch_runs/20260911_224501"
28     source = run / "processed/main"
29     output = run / "deliverables" / dt.datetime.now().strftime("partial_94days_%Y%m%d_%H%M%S")
30     output.mkdir(parents=True)
31     _, _, actual, dates, prices = load_prepared(run)
32     # 活动原目录绝不调用恢复函数；只复制完整提交后在临时目录验收。
33     with tempfile.TemporaryDirectory() as temp:
34         copy = Path(temp)
35         for _ in range(20):
36             checkpoint = (source / "checkpoint.json").read_bytes()

```

```

37     manifest = json.loads(checkpoint)
38     tables = {name: (source / name).read_bytes() for name in TABLES}
39     if (checkpoint == (source / "checkpoint.json").read_bytes()
40         and all(hashlib.sha256(raw).hexdigest() == manifest["sha256"][name]
41                 for name, raw in tables.items())):
42         break
43     else:
44         raise RuntimeError("没有读到稳定的已提交存档")
45     assert manifest["days"] >= 94
46     for name, raw in tables.items():
47         (copy / name).write_bytes(raw)
48     (copy / "checkpoint.json").write_bytes(checkpoint)
49     status = json.loads((source / "run_status.json").read_text(encoding="utf-8"))
50     write_json(copy / "run_status.json", status)
51     saved_days = pd.read_csv(copy / "daily.csv", float_precision="round_trip")
52     for folder, extension in (("daily_audit", ".json"), ("frozen_plans", ".csv")):
53         (copy / folder).mkdir()
54         for date in saved_days.date:
55             shutil.copyfile(source / folder / (date + extension), copy / folder / (date +
extension))
56         frame, daily, calibration = read_checkpoint(
57             copy, actual, dates, prices, float(saved_days.initial_soc.iloc[0]),
58             status["start_index"], status["end_index"], status["settings"])
59         frame = frame.iloc[:94 * 144].copy()
60         daily = pd.DataFrame(daily).iloc[:94].copy()
61         selected_dates = set(daily.date)
62         calibration = pd.DataFrame(calibration)
63         calibration = calibration[calibration.origin.isin(selected_dates)].copy()
64         audits = []
65         for date in daily.date:
66             audit = json.loads((copy / f"daily_audit/{date}.json").read_text(encoding="utf-8"))
67             solver = audit[str(audit["selected_K"])]["selected_solver"]
68             audits.append({"date": date, "requested_gap": solver.get("requested_gap", .01),
69                           "achieved_gap": solver["mip_gap"], "selected_reliable": solver["reliable"]
70                           },
71                           "computational_limited": audit["computational_limited"],
72                           "audit_sha256": sha256(copy / f"daily_audit/{date}.json"))
73         audit_table = pd.DataFrame(audits)
74     assert len(frame) == 13536 and len(daily) == 94
75     assert daily.date.iloc[0] == "2025-02-01" and daily.date.iloc[-1] == "2025-05-05"
76     assert audit_table.selected_reliable.all()
77     assert (audit_table.achieved_gap <= audit_table.requested_gap + 1e-8).all()
78     tables = {"dispatch": frame, "daily": daily, "calibration": calibration,
79              "storage_blocks": aggregate_storage(frame), "emergency_events": emergency_events(frame)
80              },
81              "solver_acceptance": audit_table}
82     for name, table in tables.items():
83         write_csv(output / f"{name}.csv", table)
84     explanation = [
85         ("文件性质", "第二问已完成94天的阶段结果: 不是全年最终result2"),
86         ("日期范围", "2025-02-01 至 2025-05-05"),
87         ("已保存模拟日", 94), ("10分钟调度记录", 13536),
88         ("精度记录", "前85天采用1%; 后9天采用2%; 逐日门槛与实际gap见求解验收"),
89         ("验收范围", "原始输入、电价、连续日期、跨日SOC、原反馈物理与费用逐值核验通过"),
90         ("候选限制", "选中策略达标; 部分未选候选曾超时, computational_limited标识如实保留"),
91         ("数值性质", "冻结仿真输出, 数值逐值导入CSV; 不是可编辑的Excel计算模型, 无Excel公式"),
92         ("单位", "电量和SOC为kWh, 负荷与光伏功率为kW, 费用为元; 单价为元/kWh"),

```

```

91     ("时间定义", "timestamp为10分钟区间右端点: 24:00归属前一源日期"),
92     ("剩余工作", "其余240个模拟日及尚未完成实验不在本文件内; 正式计算继续"),
93     ("来源", str(source.relative_to(ROOT))),
94     ("快照生成时间", dt.datetime.now().astimezone().isoformat()),
95 ]
96 labels = {"date": "日期", "K": "预测跨度(天)", "M": "历史块数", "S": "场景数",
97           "initial_soc": "日初电量(kWh)", "final_soc": "日末电量(kWh)", "cost": "实际日费用(元)",
98           "emergency_kwh": "紧急购电(kWh)", "computational_limited": "存在候选计算受限",
99           "interval": "时段", "timestamp": "区间右端点", "grid": "计划购电(kWh)",
100          "charge_cap": "充电响应上限(kWh)", "discharge_cap": "放电响应上限(kWh)",
101          "net_kwh": "净负荷(kWh)", "price": "电价(元每kWh)", "charge": "实际充电(kWh)",
102          "discharge": "实际放电(kWh)", "emergency": "紧急购电(kWh)", "unused": "未利用电量(kWh)"
103          },
104          "soc": "时段末电量(kWh)", "planned_cost": "计划购电费用(元)", "emergency_cost": "紧急购电
105          费用(元)",
106          "predicted_load_kw": "预测负荷(kW)", "predicted_pv_kw": "预测光伏(kW)" }
107 workbook = Workbook()
108 workbook.remove(workbook.active)
109 contents = {"说明": pd.DataFrame(explanation, columns=["项目", "说明"]), "每日汇总": daily,
110            "10分钟调度明细": frame, "4小时充放电": tables["storage_blocks"],
111            "紧急购电区间": tables["emergency_events"], "求解验收": audit_table}
112 for title, table in contents.items():
113     sheet = workbook.create_sheet(title)
114     sheet.append([labels.get(key, key) for key in table.columns])
115     for row in table.itertuples(index=False, name=None):
116         sheet.append([v.to_pydatetime() if isinstance(v, pd.Timestamp) else v for v in row])
117     for row in sheet:
118         for cell in row:
119             cell.font = Font(name="Arial", size=10)
120             if isinstance(cell.value, (float, int)) and not isinstance(cell.value, bool):
121                 cell.number_format = "#,##0.000000"
122             if isinstance(cell.value, dt.datetime):
123                 cell.number_format = "yyyy-mm-dd hh:mm"
124     for cell in sheet[1]:
125         cell.fill = PatternFill("solid", fgColor="163D52")
126         cell.font = Font(name="Arial", size=10, bold=True, color="FFFFFF")
127     for column in range(1, len(table.columns) + 1):
128         sheet.column_dimensions[get_column_letter(column)].width = 25
129     sheet.freeze_panes = "C2" if title == "10分钟调度明细" else "A2"
130     sheet.auto_filter.ref = sheet.dimensions
131     sheet.sheet_view.showGridLines = False
132 workbook["说明"].column_dimensions["B"].width = 105
133 for row in workbook["说明"].iter_rows(min_row=2):
134     row[1].alignment = Alignment(wrap_text=True, vertical="center")
135     workbook["说明"].row_dimensions[row[0].row].height = 34
136 filename = output / "第二问_94天阶段结果_20250201-20250505.xlsx"
137 workbook.save(filename); workbook.close()
138 check = load_workbook(filename, read_only=True, data_only=True)
139 for title, original in (("每日汇总", daily), ("10分钟调度明细", frame)):
140     rows = list(check[title].values)
141     assert len(rows) == len(original) + 1
142     restored = pd.DataFrame(rows[1:], columns=original.columns)
143     numeric = original.select_dtypes(include="number").columns
144     np.testing.assert_allclose(restored[numeric].to_numpy(dtype=float), original[numeric].
145                                to_numpy(dtype=float),
146                                atol=1e-8, rtol=1e-12)
147 assert not any(cell.data_type == "e" for sheet in check for row in sheet for cell in row)

```

```

145     check.close()
146     metadata = {"partial": True, "days": 94, "records": len(frame), "start": daily.date.iloc[0], "
147               "end": daily.date.iloc[-1],
148               "source_checkpoint": manifest, "original_data_physics_cost_and_soc_validated": True,
149               "excel_roundtrip_passed": True, "workbook_sha256": sha256(filename),
150               "csv_sha256": {name: sha256(output / f"{name}.csv") for name in tables}}
151     write_json(output / "manifest.json", metadata)
152     preview = "<!doctype html><meta charset='utf-8'><title>第二问94天阶段结果</title><style>body{font
153               :16px Arial;max-width:1100px;margin:36px auto;color:#163d52}td,th{padding:8px;border-bottom:1px
154               solid #ddd}table{border-collapse:collapse}a{color:#066e85}</style>"
155     preview += f"<h1>第二问: 已完成94天的阶段结果</h1><p>2025-02-01 至 2025-05-05 • 13,536条10分钟记录 •
156               存档核验与Excel逐值回读通过</p><p><a href='{html.escape(filename.name)}'>下载94天结果 Excel</a></p><p>
157               >这是阶段快照, 全年尚未算完。前85天采用1%, 后9天采用2%, 历史候选受限记录保留。</p>"
158     preview += "<h2>前5日结果预览</h2>" + daily.head().rename(columns=labels).to_html(index=False)
159     (output / "index.html").write_text(preview, encoding="utf-8")
160     with zipfile.ZipFile(output / "第二问_94天结果及CSV.zip", "w", zipfile.ZIP_DEFLATED) as bundle:
161         for path in output.iterdir():
162             if path.suffix != ".zip":
163                 bundle.write(path, path.name)
164     print(json.dumps({"directory": str(output), "xlsx": str(filename), **metadata}, ensure_ascii=
165               False), flush=True)
166
167 if __name__ == "__main__":
168     main()

```

src/q2/diagnostics.py

```

1  """2.1.5 - 2.1.11: 描述、时域/频域、稳定性与季节差分; 不替代预测验证。"""
2
3  import warnings
4
5  import numpy as np
6  import pandas as pd
7  from scipy.signal import periodogram
8  from statsmodels.tsa.stattools import acf, adfuller, kpss, pacf_burg
9  from threadpoolctl import threadpool_limits
10
11  from data import SERIES, SHEETS
12
13  LAGS = (1, 6, 144, 288, 1008, 2016)
14  MAX_LAG = 2016
15  SETTINGS = {
16      "scope": "full_year 为事后描述; initial_history 仅含首次回测前严格可见的观测",
17      "continuous_curve_start": "2025-01-01 (附件起点, 非择优选取)",
18      "key_lags": LAGS, "acf_pacf_max_lag": MAX_LAG,
19      "acf": "FFT, adjusted=False; 常数序列相关无定义",
20      "pacf": "Burg, demean=True; 直接计算至2016, 不跳过周滞后",
21      "correlation_interpretation": "报告效应量, 不把高频相关当独立样本显著性; 7日相关可能仅为日周期重复",
22      "spectrum": "periodogram fs=144/day, boxcar, detrend=constant, density; 保留所有正频率, 不插值",
23      "spectral_peak": "目标最近离散频点及其左右紧邻频点; 局部峰不等于统计显著",
24      "stability": "按原始月份; 滞后两端均限制于当月; std ddof=1; 不设稳定/不稳定硬阈值",
25      "stationarity": "raw, diff144, diff1008, diff144_then1008; ADF默认Schwert上限+ BIC选滞后; KPSS auto",
26      "stationarity_regression": "每种变换分别检验c (水平平稳) 与ct (趋势平稳); 记录1%、5%、10%临界值",
27      "stationarity_caveat": "ADF拒绝单位根不证明季节/方差稳定; KPSS表外p值按上下界标记; 联合差分或过度差分",

```

```

28 "outlier_policy": "所有局部转折点提供前后值与既往同星期同槽位上下文；只排序，不删除或平滑",
29 "calendar": "weekday=1..7 按源日期；周末仅指周六周日，不推断法定节假日或调休",
30 "pv_night": "统计零值占比与全零槽位；没有日出日落元数据，不能预设固定夜间窗口",
31 "numeric_setting_role": "以上均为明确披露的计算/显示方法，不是新增题设或预测模型参数",
32 "candidate_priority": "仅初始历史：日频局部峰、去日曲线后周频局部峰、残差周相关大于日相关三项一致时优先比较
DHR_AR: 仅比较证据，不等同统计显著性或最终选型",
33 }
34
35
36 def correlation(x, y):
37     x, y = np.asarray(x), np.asarray(y)
38     if len(x) < 2 or np.std(x) == 0 or np.std(y) == 0:
39         return np.nan
40     return float(np.corrcoef(x, y)[0, 1])
41
42
43 def describe(frame):
44     profiles = []
45     for variable in SERIES:
46         for grouping in ("month", "weekday"):
47             part = frame.groupby([grouping, "slot"])[variable].agg(["mean", "std", "count"]).
reset_index()
48             part = part.rename(columns={grouping: "group", "mean": "mean_kw", "std": "std_kw", "
count": "n"})
49             part["grouping"], part["series"] = grouping, variable
50             profiles.append(part)
51             part = frame.assign(day_type=np.where(frame.weekday <= 5, "weekday", "weekend"))
52             part = part.groupby(["day_type", "slot"])[variable].agg(["mean", "std", "count"]).
reset_index()
53             part = part.rename(columns={"day_type": "group", "mean": "mean_kw", "std": "std_kw", "count"
: "n"})
54             part["grouping"], part["series"] = "day_type", variable
55             profiles.append(part)
56     daily = []
57     for date, group in frame.groupby("date", sort=True):
58         for variable in SERIES:
59             x = group[variable]
60             peaks = group.loc[x == x.max()]
61             daily.append({"date": date, "month": date.month, "weekday": date.dayofweek + 1,
62                 "series": variable, "n": len(group), "mean_kw": x.mean(), "std_kw": x.std
63                 (),
64                 "min_kw": x.min(), "max_kw": x.max(), "amplitude_kw": x.max() - x.min(),
65                 "peak_first_slot": int(peaks.slot.iloc[0]), "peak_tie_count": len(peaks),
66                 "zero_points": int((x == 0).sum()), "all_zero": bool((x == 0).all())})
67     summary = pd.DataFrame([{"series": v, "n": len(frame), "mean_kw": frame[v].mean(),
68         "std_kw": frame[v].std(), "min_kw": frame[v].min(), "max_kw": frame[v].
69         max(),
70         "zero_count": int((frame[v] == 0).sum())} for v in SERIES])
71     zeros = frame.groupby("slot").pv_kw.agg(n="size", zero_points=lambda x: int((x == 0).sum())).
reset_index()
72     zeros["zero_fraction"] = zeros.zero_points / zeros.n
73     return pd.concat(profiles, ignore_index=True), pd.DataFrame(daily), summary, zeros
74
75 def seasonal_transforms(x):
76     x = pd.Series(np.asarray(x, dtype=float))
77     return {"raw": x, "diff144": x.diff(144), "diff1008": x.diff(1008),

```

```

77         "diff144_then1008": x.diff(144).diff(1008)}
78
79
80 def lag_analysis(frame, scope):
81     keys, curves = [], []
82     for variable in SHEETS:
83         x = frame[variable].to_numpy()
84         # 以本scope的平均日曲线去除日内形状, 仅供本scope描述; 不传入预测器。
85         residual = x - frame.groupby("slot")[variable].transform("mean").to_numpy()
86         for form, values in (("raw", x), ("daily_profile_residual", residual)):
87             for lag in LAGS:
88                 keys.append({"scope": scope, "series": variable, "form": form, "lag": lag,
89                             "lag_days": lag / 144, "n_pairs": max(len(x) - lag, 0),
90                             "pearson_r": correlation(values[lag:], values[:-lag])})
91             limit = min(MAX_LAG, len(x) // 2 - 1)
92             if np.std(x) == 0:
93                 a, p = np.full(limit + 1, np.nan), np.full(limit + 1, np.nan)
94             else:
95                 a = acf(x, nlags=limit, fft=True, adjusted=False, missing="raise")
96                 with threadpool_limits(limits=1):
97                     p, _ = pacf_burg(x, nlags=limit, demean=True)
98                 curves.extend({"scope": scope, "series": variable, "lag": lag, "lag_days": lag / 144,
99                               "acf": a[lag], "pacf": p[lag], "n": len(x)} for lag in range(limit + 1))
100     return pd.DataFrame(keys), pd.DataFrame(curves)
101
102
103 def spectral_analysis(frame, scope):
104     curves, targets = [], []
105     for variable in SHEETS:
106         x = frame[variable].to_numpy()
107         for form, values in (("raw", x), ("daily_profile_residual", x - frame.groupby("slot")[
108             variable].transform("mean").to_numpy())):
109             frequency, density = periodogram(values, fs=144, window="boxcar", detrend="constant",
110             scaling="density")
111             total_variance = density.sum() * (frequency[1] - frequency[0])
112             curves.extend({"scope": scope, "series": variable, "form": form, "frequency_per_day": f,
113                           "period_days": 1 / f, "psd_kw2_per_cycle_per_day": d}
114                           for f, d in zip(frequency[1:], density[1:]))
115             for days in (1, 7):
116                 i = int(np.argmax(abs(frequency - 1 / days)))
117                 local = bool(density[i] > density[i - 1] and density[i] > density[i + 1])
118                 targets.append({"scope": scope, "series": variable, "form": form, "
119 target_period_days": days,
120                                "target_frequency_per_day": 1 / days, "frequency_resolution_per_day"
121                                : frequency[1],
122                                "nearest_frequency_per_day": frequency[i], "nearest_period_days": 1
123                                / frequency[i],
124                                "target_psd": density[i], "left_psd": density[i - 1], "right_psd":
125                                density[i + 1],
126                                "local_peak": local, "bin_variance_fraction": density[i] * frequency
127                                [1] / total_variance if total_variance > 0 else np.nan,
128                                "integrated_psd_kw2": total_variance, "population_variance_kw2": np.
129                                var(values),
130                                "interpretation": "descriptive_not_significance_test"})
131     return pd.DataFrame(curves), pd.DataFrame(targets)
132
133
134
135

```

```

126 def stability_analysis(frame, scope):
127     rows = []
128     for month, part in frame.groupby("month"):
129         for variable in SERIES:
130             x = part[variable].to_numpy()
131             daily = part.groupby("date")[variable].agg(["max", "min"])
132             average_profile = part.groupby("slot")[variable].mean()
133             rows.append({"scope": scope, "month": month, "series": variable, "n": len(x),
134                         "mean_kw": np.mean(x), "std_kw": np.std(x, ddof=1),
135                         "daily_lag_r": correlation(x[144:], x[:-144]),
136                         "weekly_lag_r": correlation(x[1008:], x[:-1008]),
137                         "daily_pairs": max(0, len(x) - 144), "weekly_pairs": max(0, len(x) - 1008),
138                         "mean_daily_amplitude_kw": (daily["max"] - daily["min"]).mean(),
139                         "mean_profile_amplitude_kw": average_profile.max() - average_profile.min(),
140                         "cv": np.std(x, ddof=1) / abs(np.mean(x)) if np.mean(x) != 0 else np.nan})
141     return pd.DataFrame(rows)
142
143
144 def stationarity_analysis(frame, scope):
145     tests, moments, transformed = [], [], []
146     with threadpool_limits(limits=1):
147         for variable in SHEETS:
148             for form, values in seasonal_transforms(frame[variable]).items():
149                 mask = values.notna().to_numpy()
150                 x = values.dropna().to_numpy()
151                 view = frame.loc[mask, ["date", "timestamp", "month"]].copy()
152                 view["value_kw"], view["series"], view["form"], view["scope"] = x, variable, form,
153                 scope
154                 transformed.append(view)
155                 for month, part in view.groupby("month"):
156                     moments.append({"scope": scope, "series": variable, "form": form, "month": month
157                                     ,
158                                     "n": len(part), "mean_kw": part.value_kw.mean(), "std_kw": part.
159                                     value_kw.std()})
160                     for regression in ("c", "ct"):
161                         for test in ("ADF", "KPSS"):
162                             base = {"scope": scope, "series": variable, "form": form, "test": test,
163                                     "regression": regression, "input_n": len(x)}
164                             if len(x) < 10 or np.std(x) == 0:
165                                 tests.append(**base, "status": "undefined_constant_or_short", "warning"
166                                             : "不强行检验"})
167                             continue
168                             with warnings.catch_warnings(record=True) as caught:
169                                 warnings.simplefilter("always")
170                                 if test == "ADF":
171                                     stat, p, lag, nob, critical, bic = adfuller(x, regression=
172                                     regression, autolag="BIC")
173                                     bound = "approximate"
174                                 else:
175                                     stat, p, lag, critical = kpss(x, regression=regression, nlags="auto"
176                                     )
177                                     nob, bic = len(x), np.nan
178                                     bound = "less_than" if p == 0.01 else "greater_than" if p == 0.1
179                             else "interpolated"
180                             tests.append(**base, "status": "ok", "statistic": stat, "p_value": p, "
181                                     p_value_type": bound,
182                                     "selected_lags": lag, "test_n": nob, "bic": bic,

```

```

175         "critical_1pct": critical["1%"], "critical_5pct": critical["5%
176     ],
177         "critical_10pct": critical["10%"],
178         "reject_null_1pct": bool(stat < critical["1%"] if test == "ADF
179     " else stat > critical["1%"]),
180         "reject_null_5pct": bool(stat < critical["5%"] if test == "ADF
181     " else stat > critical["5%"]),
182         "reject_null_10pct": bool(stat < critical["10%"] if test == "
183     ADF" else stat > critical["10%"]),
184         "warning": " | ".join(str(w.message) for w in caught)})
185     return pd.DataFrame(tests), pd.DataFrame(moments), pd.concat(transformed, ignore_index=True)
186
187 def analyze_scope(frame, scope):
188     print(f"{scope}: {len(frame)} 点, 描述与相关诊断", flush=True)
189     profiles, daily, summary, zeros = describe(frame)
190     lags, correlation_curves = lag_analysis(frame, scope)
191     spectrum, spectral_targets = spectral_analysis(frame, scope)
192     stability = stability_analysis(frame, scope)
193     print(f"{scope}: ADF/KPSS 原始、日差分、周差分、联合差分", flush=True)
194     tests, moments, transforms = stationarity_analysis(frame, scope)
195     return {"profiles": profiles, "daily_statistics": daily, "summary": summary, "pv_zero_profile":
        zeros,
        "lag_correlations": lags, "acf_pacf": correlation_curves, "spectrum": spectrum,
        "spectral_targets": spectral_targets, "monthly_stability": stability,
        "stationarity_tests": tests, "transformed_monthly_moments": moments, "transformed_series
        ": transforms}

```

src/q2/dispatch.py

```

1  """第二问最终方案运行入口；旧run.py继续承担数据诊断。"""
2
3  import argparse
4  import datetime as dt
5  import hashlib
6  import importlib.metadata
7  import json
8  import shutil
9  import tempfile
10 from pathlib import Path
11
12 import numpy as np
13 import pandas as pd
14 from openpyxl import load_workbook
15
16 from data import ROOT, endpoint_minutes, read_attachment, sha256, write_csv, write_json
17 from export_dispatch import export_result, write_final_artifacts, write_tables
18 from protocol import PLAN, TASK, PROTOCOL, signature
19 from rolling import ComputationalLimit, run_rollout, warm_start
20 from shadows import build_shadow_library, open_library
21
22 FORECAST_FILES = ("data.py", "forecast.py", "shadows.py", "requirements.txt")
23 FORECAST_PROTOCOL_KEYS = (
24     "history_boundary", "forecast_horizons", "shadow_start", "lightgbm_candidates",
25     "lightgbm_training_window", "lightgbm_freeze_metric", "dhr_harmonics",
26     "dhr_ar_orders", "dhr_ma_orders", "dhr_history_days", "dhr_order_rule",

```



```

27     "dhr_max_iterations", "dhr_difference_order")
28
29
30 def input_data(directory):
31     directory.mkdir(parents=True, exist_ok=True)
32     attachment2 = R00T/"docs/CUMCM2026Problems/C题/附件/附件2.xlsx"
33     attachment1 = R00T/"docs/CUMCM2026Problems/C题/附件/附件1.xlsx"
34     frame, _ = read_attachment(attachment2, directory/"quality_checks.csv")
35     frame["day_of_year"] = pd.to_datetime(frame.date).dt.dayofyear
36     workbook = load_workbook(attachment1, read_only=True, data_only=True)
37     rows = list(workbook.active.values); workbook.close()
38     if len(rows) != 145 or rows[0][1] != "电价":
39         raise ValueError("附件1电价表结构不符")
40     if [endpoint_minutes(row[0]) for row in rows[1:]] != list(range(10, 1441, 10)):
41         raise ValueError("附件1电价右端点序列不符")
42     prices = np.array([row[1] for row in rows[1:]], dtype=float)
43     if not np.isfinite(prices).all() or (prices <= 0).any():
44         raise ValueError("电价必须有限且为正")
45     write_csv(directory/"prices.csv", pd.DataFrame({"slot": np.arange(1, 145), "price_yuan_per_kwh":
46         prices}))
47     write_csv(directory/"timeseries.csv", frame)
48     write_json(directory/"source_manifest.json", {str(path.relative_to(R00T)): sha256(path) for path
49         in (attachment1, attachment2)})
50     dates = pd.DatetimeIndex(frame.date.unique())
51     actual = frame[["load_kw", "pv_kw"]].to_numpy().reshape(365, 144, 2)
52     return actual, dates, prices
53
54 def forecast_hashes():
55     def normalized(path):
56         text = path.read_text(encoding="utf-8").replace("\r\n", "\n")
57         return hashlib.sha256(text.encode("utf-8")).hexdigest()
58     return {name: normalized(R00T/"src/q2"/name) for name in FORECAST_FILES}
59
60 def validate_prepared_protocol(protocol):
61     """模型增强不使预测缓存失效：只核验真正参与影子预测的源码和冻结参数。"""
62     stored = protocol.get("forecast_source_sha256", {})
63     current = forecast_hashes()
64     if any(stored.get(name) != digest for name, digest in current.items()):
65         raise ValueError("准备库与当前预测源码不一致：禁止混用运行代")
66     if protocol.get("protocol_sha256") == signature() and stored == current:
67         return
68     changed = [key for key in FORECAST_PROTOCOL_KEYS if protocol.get(key) != PROTOCOL[key]]
69     if changed:
70         raise ValueError("准备库的预测冻结参数已改变：+", ".join(changed))
71
72 def prepared_hashes(path):
73     files = [p for directory in (path/"inputs/processed", path/"raw/shadows")
74         for p in directory.rglob("*") if p.is_file()]
75     return {p.relative_to(path).as_posix(): sha256(p) for p in sorted(files)}
76
77 def seal_prepared(path):
78     """旧准备库迁移：重新读取附件逐值核对，并从此刻冻结生成文件摘要。"""
79     path = Path(path)

```

```

82 if (path/"raw/prepared_manifest.json").exists():
83     expected = json.loads((path/"raw/prepared_manifest.json").read_text(encoding="utf-8"))["
sha256"]
84     if prepared_hashes(path) != expected:
85         raise ValueError("已封存准备库改变，禁止重新封存掩盖差异")
86     return
87 protocol = json.loads((path/"raw/protocol.json").read_text(encoding="utf-8"))
88 validate_prepared_protocol(protocol)
89 with tempfile.TemporaryDirectory() as directory:
90     actual, dates, prices = input_data(Path(directory))
91     saved = pd.read_csv(path/"inputs/processed/timeseries.csv", float_precision="round_trip")
92     np.testing.assert_array_equal(saved[["load_kw", "pv_kw"]].to_numpy().reshape(actual.shape),
actual)
93     np.testing.assert_array_equal(pd.to_datetime(saved.date.unique()), dates)
94     np.testing.assert_array_equal(pd.read_csv(path/"inputs/processed/prices.csv", float_precision="
round_trip").price_yuan_per_kwh, prices)
95     open_library(path/"raw/shadows")
96     write_json(path/"raw/prepared_manifest.json", {"sha256": prepared_hashes(path),
"sealed_at": dt.datetime.now(dt.timezone.utc).
97         isoformat(),
98         "origin": "
post_generation_seal_original_inputs_independently_rechecked"})
99
100
101 def reuse_prepared(source, destination):
102     """逐字节复用已验收输入与影子预测；新模型运行目录保持独立。"""
103     source, destination = Path(source).resolve(), Path(destination).resolve()
104     load_prepared(source)
105     if destination.exists() and any(destination.iterdir()):
106         raise ValueError("目标运行目录非空，拒绝覆盖")
107     destination.mkdir(parents=True, exist_ok=True)
108     shutil.copytree(source/"inputs/processed", destination/"inputs/processed")
109     shutil.copytree(source/"raw/shadows", destination/"raw/shadows")
110     versions = {name: importlib.metadata.version(name) for name in
("numpy", "scipy", "pandas", "lightgbm", "scikit-learn",
111         "statsmodels", "pysciplot", "openpyxl")}
112     write_json(destination/"raw/protocol.json", {
113         **PROTOCOL, "protocol_sha256": signature(), "plan_sha256": sha256(PPLAN),
114         "task_sha256": sha256(TASK), "forecast_source_sha256": forecast_hashes(),
115         "versions": versions, "prepared_cache_reused_from": str(source),
116         "prepared_cache_source_manifest_sha256": sha256(source/"raw/prepared_manifest.json")})
117     write_json(destination/"raw/prepare_status.json", {
118         "complete": True, "forecast_source_sha256": forecast_hashes(),
119         "cache_reused": True, "source": str(source)})
120     source_revision = source/"raw/solver_precision_revision.json"
121     if source_revision.exists():
122         previous = json.loads(source_revision.read_text(encoding="utf-8"))
123         write_json(destination/"raw/solver_precision_revision.json", {
124             "version": "q2-tail-reserve-inherit-current-gap-20260912",
125             "created_at": dt.datetime.now(dt.timezone.utc).isoformat(),
126             "solver_gap": previous["solver_gap"],
127             "parent_protocol_sha256": signature(),
128             "authorization": "承接用户此前明确选择的当前正式MIP gap策略；本轮任务书要求保持当前solver与MIP gap
策略不变。",
129             "source_revision": str(source_revision),
130             "source_revision_sha256": sha256(source_revision),
131             "no_gap_experiment_rerun": True})
132

```

```

133     write_json(destination/"raw/prepared_manifest.json", {
134         "sha256": prepared_hashes(destination),
135         "origin": "verified_existing_forecast_cache_byte_copy",
136         "source": str(source)})
137     load_prepared(destination)
138     print(f"已复用预测缓存: {source} -> {destination}", flush=True)
139     return destination
140
141
142 def prepare(run_path=None):
143     if PROTOCOL["history_boundary"] != "source_date_before_origin":
144         raise ValueError("新方案完整日信息边界未确定")
145     path = Path(run_path) if run_path else ROOT/"outputs/q2/dispatch_runs"/dt.datetime.now().
        strftime("%Y%m%d_%H%M%S")
146     if path.exists() and any(path.iterdir()):
147         raise ValueError("准备目录非空, 拒绝覆盖已有运行记录")
148     path.mkdir(parents=True, exist_ok=True)
149     hashes = forecast_hashes()
150     write_json(path/"raw/protocol.json", {**PROTOCOL, "protocol_sha256": signature(),
151         "plan_sha256": sha256(PLAN), "task_sha256": sha256(TASK),
152         "forecast_source_sha256": hashes,
153         "versions": {name: importlib.metadata.version(name) for
154             name in
155                 ("numpy", "scipy", "pandas", "lightgbm", "
156                 scikit-learn", "statsmodels", "pyscioppt", "openpyxl")})})
157     print(f"运行目录: {path.resolve()}", flush=True)
158     actual, dates, prices = input_data(path/"inputs/processed")
159     try:
160         build_shadow_library(actual, dates, path/"raw/shadows", boundary=PROTOCOL["history_boundary"]
161         ])
162         if hashes != forecast_hashes():
163             raise ValueError("影子库生成期间预测源码改变, 请在新目录重算")
164         write_json(path/"raw/prepare_status.json", {"complete": True, "forecast_source_sha256":
165             hashes})
166         write_json(path/"raw/prepared_manifest.json", {"sha256": prepared_hashes(path), "origin": "
167             at_preparation_completion"})
168     except BaseException as error:
169         write_json(path/"raw/prepare_status.json", {"complete": False, "error": str(error)})
170         raise
171     print(f"影子库完整生成: {path.resolve()}", flush=True)
172     return path
173
174
175 def load_prepared(path):
176     path = Path(path)
177     protocol = json.loads((path/"raw/protocol.json").read_text(encoding="utf-8"))
178     validate_prepared_protocol(protocol)
179     manifest = json.loads((path/"inputs/processed/source_manifest.json").read_text(encoding="utf-8")
180     )
181     if any(sha256(ROOT/name) != digest for name, digest in manifest.items()):
182         raise ValueError("原始输入已改变")
183     sealed = path/"raw/prepared_manifest.json"
184     if not sealed.exists():
185         raise ValueError("准备库尚未封存: 旧运行须先执行seal-prepared独立核对输入")
186     if prepared_hashes(path) != json.loads(sealed.read_text(encoding="utf-8"))["sha256"]:
187         raise ValueError("处理输入/影子预测库摘要改变, 禁止继续求解")
188     frame = pd.read_csv(path/"inputs/processed/timeseries.csv", parse_dates=["date", "timestamp"],

```

```

float_precision="round_trip")
183 actual = frame[["load_kw", "pv_kw"]].to_numpy().reshape(365, 144, 2)
184 prices = pd.read_csv(path/"inputs/processed/prices.csv", float_precision="round_trip").
price_yuan_per_kwh.to_numpy()
185 store, frozen = open_library(path/"raw/shadows")
186 return store, frozen, actual, pd.DatetimeIndex(frame.date.unique()), prices
187
188
189 def run_main(path, *, rescue_seconds=None, solver_gap=PROTOCOL["solver_gap"],
190             execution_revision=None, resume=False):
191     path = Path(path)
192     store, frozen, actual, dates, prices = load_prepared(path)
193     output = path/"processed"
194     output.mkdir(parents=True, exist_ok=True)
195     if (output/"main/dispatch.csv").exists() and not resume:
196         raise ValueError("运行目录已有逐日结果; 仅可显式使用--resume从最后完整日期续算")
197     sources = {p.name: sha256(p) for p in (ROOT/"src/q2").glob("*.py")}
198     write_json(path/f"raw/dispatch_sources_{dt.datetime.now().strftime('%Y%m%d_%H%M%S')}.json",
sources)
199     write_json(path/"raw/dispatch_sources.json", sources)
200     try:
201         initial = warm_start(actual, dates, prices, output)
202         result, _ = run_rollout(store, frozen, actual, dates, prices, initial, output/"main",
rescue_seconds=rescue_seconds,
203                               solver_gap=solver_gap, execution_revision=execution_revision)
204         write_tables(result, output/"main")
205         from final_report import verify_code
206         verify_code(path)
207         export_result(result, ROOT/"docs/CUMCM2026Problems/C题/附件/附件5/result2.xlsx", output/"main"
)
208         write_final_artifacts(result, actual, output/"main")
209     except Exception as error:
210         write_json(path/"raw/dispatch_status.json", {"complete": False, "reason": str(error), "
details": getattr(error, "details", None)})
211         raise
212     write_json(path/"raw/dispatch_status.json", {"complete": True, "main_days": 334,
"mandatory_experiments_complete": False,
213 "execution": json.loads((output/"main/run_status.
214 json").read_text(encoding="utf-8"))["execution"],
"supplemental_days": sum("
215 supplemental_solver_seconds" in json.loads(p.read_text(encoding="utf-8"))
for p in (output/"main/
216 daily_audit").glob("*.json")),
"source_sha256_at_start": sources})
217     print(f"主回放与论文必需离线验收完成: {output/'main'}", flush=True)
218
219
220
221 def main():
222     parser = argparse.ArgumentParser(description=__doc__)
223     sub = parser.add_subparsers(dest="command", required=True)
224     command = sub.add_parser("prepare", help="核验输入并逐日生成所有候选的因果影子库")
225     command.add_argument("--run-dir", type=Path)
226     command = sub.add_parser("reuse-prepared", help="从既有运行代复用已验收输入与影子预测")
227     command.add_argument("--source-run", type=Path, required=True)
228     command.add_argument("--run-dir", type=Path, required=True)
229     for name, help_text in (("run", "执行因果预热及334日主模型"), ("experiments", "运行方案2.10的诊断/敏感
性"),

```

```

230         ("seal-prepared", "独立核对旧准备库并冻结生成文件摘要")):
231     command = sub.add_parser(name, help=help_text)
232     command.add_argument("--run-dir", type=Path, required=True)
233     if name in ("run", "experiments"):
234         command.add_argument("--rescue-seconds", type=float,
235                               help="原预算整日全候选失败后追加的每次求解秒数：保留原失败记录，不放松gap")
236     if name == "run":
237         command.add_argument("--resume", action="store_true",
238                               help="从最后一个已验收完整日继续：未指定时拒绝覆盖已有逐日结果")
239     if name == "experiments":
240         command.add_argument("--only", help="只运行一个具名实验，便于独立续算与资源控制")
241     args = parser.parse_args()
242     if args.command == "prepare":
243         prepare(args.run_dir)
244     elif args.command == "reuse-prepared":
245         reuse_prepared(args.source_run, args.run_dir)
246     elif args.command == "run":
247         from execution import execution_settings
248         run_main(args.run_dir, rescue_seconds=args.rescue_seconds, resume=args.resume,
249                 **execution_settings(args.run_dir))
250     elif args.command == "experiments":
251         from experiments import run_experiments
252         status = run_experiments(args.run_dir, only=args.only, rescue_seconds=args.rescue_seconds)
253         if any(not entry.get("complete") for entry in status.values()):
254             raise SystemExit(2)
255     elif args.command == "seal-prepared":
256         seal_prepared(args.run_dir)
257
258
259 if __name__ == "__main__":
260     main()

```

src/q2/execution.py

```

1  """求解精度的显式执行修订：不改变已冻结的预测协议和准备库。"""
2
3  import hashlib
4  import json
5  import math
6  from pathlib import Path
7
8  from protocol import PROTOCOL, signature
9
10
11 def execution_settings(run):
12     path = Path(run)/"raw/solver_precision_revision.json"
13     if not path.exists():
14         return {}
15     raw = path.read_bytes()
16     revision = json.loads(raw)
17     gap = revision["solver_gap"]
18     if (revision["parent_protocol_sha256"] != signature() or not revision.get("authorization")
19         or isinstance(gap, bool) or not isinstance(gap, (int, float))
20         or not math.isfinite(gap) or not PROTOCOL["solver_gap"] <= gap < 1):
21         raise ValueError("求解精度修订缺少授权、父协议不符或门槛无效")
22     return {"solver_gap": float(gap), "execution_revision": hashlib.sha256(raw).hexdigest()}

```

src/q2/experiments.py

```
1 """最终方案2.10的可复现检验：未运行/计算受限的实验显式留状态而不填零。"""
2
3 import json
4 import hashlib
5 from pathlib import Path
6 import time
7
8 import numpy as np
9 import pandas as pd
10
11 from comparators import oracle_pair
12 from data import sha256, write_csv, write_json
13 from checkpoint import atomic_json
14 from baseline_reuse import main_baselines, reuse_main_baseline
15 from dispatch import load_prepared
16 from export_dispatch import emergency_events, summary, validate_dispatch, write_tables
17 from forecast import economic_metrics
18 from optimization import certify_policy, solve_policy
19 from incumbent import refine
20 from relaxation import recourse_bound
21 from policy import DT, ETA_C, ETA_D, PHYSICAL_TOL, Policy, replay
22 from protocol import PROTOCOL
23 from rolling import ComputationalLimit, _solve, dispatch_frame, run_rollout, warm_start
24 from scenarios import construct, eligible_origins, weighted_quantile
25 from shadows import combined, choose_day
26
27
28 def forecast_audit(store, frozen, actual, dates, prices, output):
29     metrics, distribution, dependencies = [], [], []
30     for name, prediction in store.items():
31         variable = 0 if name.startswith("load") else 1
32         for h in range(3):
33             origins = np.arange(31, len(dates)-h)
34             origins = origins[np.isfinite(prediction[origins, h]).all(axis=1)]
35             if not len(origins):
36                 continue
37             errors = actual[origins+h, :, variable]-prediction[origins, h]
38             flat = errors.ravel()
39             metrics.append({"model": name, "horizon": h, "n": flat.size, "mae_kw": float(np.abs(flat)
40 ).mean()),
41                             "rmse_kw": float(np.sqrt(np.mean(flat**2))), "role": "
42 posthoc_never_online_selection"})
43             distribution.append({"model": name, "horizon": h, "origins": len(origins), "mean_kw":
44 float(flat.mean()),
45                             "std_kw": float(flat.std(ddof=1)), **{f"q{q:g}_kw": float(np.
46 quantile(flat, q)) for q in (.1, .5, .8, .9)}})
47             for lag in (1, 6, 12):
48                 dependencies.append({"pipeline": name, "horizon": h, "kind": f"within_day_lag_{lag}"
49 ,
50                                     "correlation": _corr(errors[:, :-lag], errors[:, lag:])})
51     for pair in (("load_week", "pv_mean7"), ("load_week", "pv_dhr"),
52                 (f"load_lgb_{frozen}", "pv_mean7"), (f"load_lgb_{frozen}", "pv_dhr")):
53         prediction = combined(store, pair)
54         origins = eligible_origins(prediction, len(actual), 3, minimum=1)
55         origins = origins[origins >= 31]
```

```

51     if not len(origins):
52         continue
53     blocks = np.stack([actual[j:j+3]-prediction[j] for j in origins])
54     for h in range(3):
55         metrics.append({"model": "|".join(pair), "horizon": h, "n": len(origins)*144,
56                        **economic_metrics(actual[origins+h], prediction[origins, h], prices)})
57         dependencies.append({"pipeline": "|".join(pair), "horizon": h, "kind": "load_pv_joint",
58                             "correlation": _corr(blocks[:, h, :, 0], blocks[:, h, :, 1])})
59         if h < 2:
60             net_error = blocks[:, 0]-blocks[:, 1]
61             dependencies.append({"pipeline": "|".join(pair), "horizon": h, "kind": "
62 same_origin_cross_day",
63                                "correlation": _corr(net_error[:, h], net_error[:, h+1])})
64     rows = []
65     for d in range(31, len(actual)):
66         direct = actual[d-7, :, 0]-actual[d-7, :, 1]
67         separate = store["load_week"][d, 0]-store["pv_mean7"][d, 0]
68         real = actual[d, :, 0]-actual[d, :, 1]
69         for name, predicted in (("direct_net_week", direct), ("separate_strong_baselines", separate)
70 ):
71             demand, forecast = np.maximum(real*DT, 0), np.maximum(predicted*DT, 0)
72             loss = prices*(np.maximum(forecast-demand, 0)+4*np.maximum(demand-forecast, 0))
73             rows.append({"date": str(dates[d].date()), "path": name, "n": 144,
74                         "absolute_error_sum_kw": float(np.abs(predicted-real).sum()),
75                         "squared_error_sum_kw2": float(np.sum((predicted-real)**2)), "econ_loss_sum
76 ": float(loss.sum())})
77     write_csv(output/"forecast_metrics.csv", pd.DataFrame(metrics))
78     write_csv(output/"horizon_error_statistics.csv", pd.DataFrame(distribution))
79     write_csv(output/"error_dependencies.csv", pd.DataFrame(dependencies))
80     write_csv(output/"direct_net_forecast_comparison.csv", pd.DataFrame(rows))
81
82 def _corr(a, b):
83     a, b = np.asarray(a).ravel(), np.asarray(b).ravel()
84     return float(np.corrcoef(a, b)[0, 1]) if a.std() > 0 and b.std() > 0 else None
85
86 def scenario_audit(store, actual, dates, prices, directory):
87     """从冻结预测、原origin块及实际medoid索引复现分布；不重新选模或求解。"""
88     if not (directory/"dispatch.csv").exists() or not (directory/"daily_audit").exists():
89         return
90     executed = pd.read_csv(directory/"dispatch.csv", parse_dates=["date"], float_precision="
91 round_trip")
92     rigid = json.loads((directory/"run_status.json").read_text(encoding="utf-8")).get("settings",
93 {}).get("kind") == "rigid"
94     rows, profiles, states = [], [], []
95     observed_dates = pd.DatetimeIndex(executed.date.unique())
96     representative = {observed_dates[0], observed_dates[-1]} | set(pd.Timestamp(d) for d in PROTOCOL
97 ["specified_dates"])
98     for date in observed_dates:
99         audit = json.loads((directory/f"daily_audit/{date.date()}.json").read_text(encoding="utf-8")
100 )
101         k = audit["selected_K"]; entry = audit[str(k)]; selection = entry["selection"]
102         d = dates.get_loc(date); origins = selection["origin_indices"]
103         if any(j+k > d for j in origins):
104             raise ValueError("事后审计发现场景误差块使用了当时尚未实现的数据")
105         forecasts = combined(store, selection["pipeline"])

```



```

101     point = forecasts[d, :k]
102     blocks = np.stack([actual[j:j+k]-forecasts[j, :k] for j in origins])
103     all_origins = selection.get("all_origin_indices", origins)
104     all_blocks = np.stack([actual[j:j+k]-forecasts[j, :k] for j in all_origins])
105     full, full_weights, _ = construct(
106         point, blocks, np.tile(prices, k), origins=origins,
107         all_blocks=all_blocks, all_origins=all_origins,
108         tail_fraction=PROTOCOL["tail_fraction"])
109     scenario = entry.get("scenarios", entry.get("candidates", {}).get(str(entry["S"]), {}).get("
scenarios", {}))
110     weights = np.asarray(scenario.get("weights", full_weights))
111     selected_origins = scenario.get("selected_origin_indices")
112     if selected_origins is None:
113         net = full[scenario.get("indices", list(range(len(full))))]
114     else:
115         selected_blocks = np.stack([actual[j:j+k]-forecasts[j, :k] for j in selected_origins])
116         values = np.maximum(point[None]+selected_blocks, 0)
117         net = DT*(values[..., 0]-values[..., 1]).reshape(len(selected_origins), -1)
118     if entry["S"] == 1 and len(full) > 1:
119         net = DT*(point[..., 0]-point[..., 1]).reshape(1, -1); weights = np.ones(1)
120     mean = np.average(net, weights=weights, axis=0)
121     std = np.sqrt(np.average((net-mean)**2, weights=weights, axis=0))
122     full80, full90 = [weighted_quantile(full, full_weights, q) for q in (.8, .9)]
123     q80, q90 = [weighted_quantile(net, weights, q) for q in (.8, .9)]
124     for h in range(k):
125         cut = slice(144*h, 144*(h+1))
126         rows.append({"origin": str(date.date()), "horizon": h, "M": len(full), "S": len(net),
127             "mean_net_kwh": float(mean[cut].mean()), "mean_timewise_variance_kwh2":
128             float((std[cut]**2).mean()),
129             "full_Q80_mean_kwh": float(full80[cut].mean()), "reduced_Q80_mean_kwh":
130             float(q80[cut].mean()),
131             "full_Q90_mean_kwh": float(full90[cut].mean()), "reduced_Q90_mean_kwh":
132             float(q90[cut].mean()),
133             "positive_Q90_tail_underestimate_kwh": float(np.maximum(full90[cut]-q90[cut
134             ], 0).mean()),
135             "expected_objective_yuan_all_horizons": entry["selected_solver"]["objective
136             "])})
137     profiles.append(pd.DataFrame({"origin": str(date.date()), "step": np.arange(144*k), "horizon
138     ": np.arange(144*k)//144,
139     "mean_net_kwh": mean, "std_net_kwh": std, "full_Q80_kwh":
140     full80,
141     "reduced_Q80_kwh": q80, "full_Q90_kwh": full90, "
142     reduced_Q90_kwh": q90}))
143     if date in representative:
144         frozen = pd.read_csv(directory/f"frozen_plans/{date.date()}.csv", float_precision="
145         round_trip")
146         reserve = frozen.soc_reserve_kwh if "soc_reserve_kwh" in frozen else None
147         policy = Policy(frozen.grid, frozen.charge_cap, frozen.discharge_cap, reserve)
148         initial = float(executed[executed.date == date].initial_soc.iloc[0])
149         if rigid:
150             fixed_soc = initial+np.cumsum(ETA_C*policy.charge_cap-policy.discharge_cap/ETA_D)
151             scenario_soc = np.tile(fixed_soc, (len(net), 1))
152         else:
153             scenario_soc = np.array([replay(policy, path, initial)["soc"] for path in net])
154         states.append(pd.DataFrame({"origin": str(date.date()), "step": np.arange(k*144),
155             "mean_scenario_soc_kwh": np.average(scenario_soc, weights=
156             weights, axis=0),

```



```

147         "scenario_soc_Q10_kwh": weighted_quantile(scenario_soc,
weights, .1),
148         "scenario_soc_Q90_kwh": weighted_quantile(scenario_soc,
weights, .9)))
149 write_csv(directory/"scenario_distribution.csv", pd.DataFrame(rows))
150 write_csv(directory/"scenario_profiles.csv", pd.concat(profiles, ignore_index=True))
151 write_csv(directory/"scenario_soc_representative.csv", pd.concat(states, ignore_index=True))
152
153
154 def no_storage_quantile(store, frozen, actual, dates, prices, output):
155     records = []
156     for d in range(31, len(dates)):
157         point, blocks, _ = choose_day(store, frozen, actual[:d], d, 1, prices)
158         net, weights, _ = construct(point, blocks, prices)
159         demand = np.maximum(net, 0)
160         policies = {"point_forecast": np.maximum(DT*(point[0, :, 0]-point[0, :, 1]), 0),
161                    "conditional_empirical_mean": np.average(demand, weights=weights, axis=0),
162                    "empirical_Q80": weighted_quantile(demand, weights, .8)}
163         real = np.maximum(DT*(actual[d, :, 0]-actual[d, :, 1]), 0)
164         for name, grid in policies.items():
165             records.append({"date": str(dates[d].date()), "method": name,
166                            "actual_cost_yuan": float(prices@grid+5*np.maximum(real-grid, 0)),
167                            "empirical_expected_cost_yuan": float(prices@grid+sum(5*p*(prices@np.
168                            maximum(path-grid, 0)) for p, path in zip(weights, demand))),
169                            "emergency_kwh": float(np.maximum(real-grid, 0).sum())})
169 write_csv(output/"no_storage_quantile.csv", pd.DataFrame(records))
170
171
172 def structural_rollout(actual, dates, prices, initial_soc, output, *, direct):
173     """结构对照在相同K=2、同一确定性反馈调度器和初态下仅改变净负荷预测路径。"""
174     from forecast import seasonal
175     if (output/"summary.json").exists():
176         saved = pd.read_csv(output/"dispatch.csv", parse_dates=["date", "timestamp"],
float_precision="round_trip")
177         design = json.loads((output/"design.json").read_text(encoding="utf-8"))
178         validate_dispatch(saved, full_period=True)
179         if design != {"direct_net": direct, "K": 2, "uncertainty": "
held_deterministic_for_structure_isolation", "initial_soc": initial_soc}:
180             raise ValueError("结构对照缓存条件不符")
181         np.testing.assert_allclose(saved.net_kwh, DT*(actual[31:, :, 0]-actual[31:, :, 1]).ravel(),
atol=1e-5, rtol=0)
182         return saved
183     soc, frames = initial_soc, []
184     for d in range(31, len(dates)):
185         if direct:
186             point_net = np.concatenate([actual[d-7+h, :, 0]-actual[d-7+h, :, 1] for h in range(2)])*
DT
187         else:
188             point = seasonal(actual[:d], 2)
189             point_net = DT*(point[... , 0]-point[... , 1]).ravel()
190             policy, _, info = _solve(point_net[None], np.tile(prices, 2), np.ones(1), soc)
191             if policy is None or not info["reliable"]:
192                 raise ComputationalLimit({"date": str(dates[d]), "solver": info})
193             frame = dispatch_frame(dates[d], policy, DT*(actual[d, :, 0]-actual[d, :, 1]), soc, prices)
194             frames.append(frame); soc = float(frame.soc.iloc[-1])
195     result = pd.concat(frames, ignore_index=True)
196     write_csv(output/"dispatch.csv", result)

```

```

197 write_tables(result, output)
198 write_json(output/"design.json", {"direct_net": direct, "K": 2, "uncertainty": "
held_deterministic_for_structure_isolation", "initial_soc": initial_soc})
199 return result
200
201
202 def richer_context(net, weights, prices, soc, audit_path, plan_path):
203     digest = hashlib.sha256()
204     for values in (net, weights, prices, [soc]):
205         array = np.asarray(values, dtype="<f8")
206         digest.update(str(array.shape).encode("utf-8")); digest.update(array.tobytes())
207     return {"scenario_input_sha256": digest.hexdigest(), "main_audit_sha256": sha256(audit_path),
208           "main_policy_sha256": sha256(plan_path),
209           "solver_sources": {name: sha256(Path(__file__).parent/name)
210                             for name in ("policy.py", "optimization.py", "incumbent.py", "
relaxation.py")}}
211
212
213 def save_richer_policy(output, record, policy, context):
214     """策略CSV先以唯一文件名写完，最后原子提交逐日期收据，旧收据不会指向半份文件。"""
215     date = record["date"]
216     path = output/f"richer_soc/{date}_{time.time_ns()}_policy.csv"
217     columns = {"grid": policy.grid}
218     for key, caps in (("charge_cap", policy.charge_cap), ("discharge_cap", policy.discharge_cap)):
219         columns.update({f"{key}_{k}": caps[:, k] for k in range(3)})
220     write_csv(path, pd.DataFrame(columns))
221     saved = {**record, "context": context, "policy_file": str(path.relative_to(output)), "
policy_sha256": sha256(path)}
222     atomic_json(output/f"richer_soc/{date}.json", saved)
223     return saved
224
225
226 def load_richer_policy(output, date, context, net, prices, weights, soc, *, solver_gap=PROTOCOL["
solver_gap"]):
227     path = output/f"richer_soc/{date}.json"
228     if not path.exists():
229         return None
230     saved = json.loads(path.read_text(encoding="utf-8"))
231     if not saved["solver"]["reliable"]:
232         return None
233     if saved["context"] != context:
234         raise ValueError("已完成的SOC分段对照与当前主策略/场景/源码不一致，禁止混用")
235     policy_path = output/saved["policy_file"]
236     if sha256(policy_path) != saved["policy_sha256"]:
237         raise ValueError("SOC分段策略存档摘要不符，禁止复用损坏策略")
238     frame = pd.read_csv(policy_path, float_precision="round_trip")
239     if len(frame) != len(prices):
240         raise ValueError("SOC分段策略的完整前瞻长度不符")
241     policy = Policy(frame.grid, frame[[f"charge_cap_{k}" for k in range(3)]].to_numpy(),
242                    frame[[f"discharge_cap_{k}" for k in range(3)]].to_numpy())
243     responses = [replay(policy, row, soc) for row in net]
244     cost = float(prices@policy.grid+sum(5*p*(prices@row["emergency"]) for p, row in zip(weights,
responses)))
245     np.testing.assert_allclose(cost, saved["solver"]["objective"], atol=PHYSICAL_TOL, rtol=0)
246     lower = saved["solver"]["dual_bound"]
247     if lower > cost+PHYSICAL_TOL or max(cost-lower, 0)/max(abs(lower), 1e-8) > solver_gap+1e-8:
248         raise ValueError("SOC分段策略存档的费用与当前精度证书不一致")

```

```

249     return policy, saved
250
251
252 def richer_real_metrics(policy, net, prices, soc, main_day):
253     response = replay(policy.first(), net, soc)
254     actual_frame = pd.DataFrame({"timestamp": main_day.timestamp.to_numpy(), "emergency": response["
emergency"]},
255                                "emergency_cost": 5*prices*response["emergency"]})
256     return {"actual_cost": float(prices@(policy.grid[:144]+5*response["emergency"])),
257            "emergency_kwh": float(response["emergency"].sum()), "final_soc": float(response["soc"
][-1]),
258            "emergency_events": len(emergency_events(actual_frame)),
259            "main_actual_cost": float((main_day.planned_cost+main_day.emergency_cost).sum()),
260            "main_emergency_kwh": float(main_day.emergency.sum()),
261            "main_emergency_events": len(emergency_events(main_day)), "main_final_soc": float(
main_day.soc.iloc[-1])}
262
263
264 def richer_policy_diagnostic(store, frozen, actual, dates, prices, main, output, *, rescue_seconds=
None, solver_gap=PROTOCOL["solver_gap"], execution_revision=None):
265     frame = pd.read_csv(main/"dispatch.csv", parse_dates=["date", "timestamp"], float_precision="
round_trip")
266     volatility = np.std(np.diff(actual[31:, :, 0]-actual[31:, :, 1], axis=1), axis=1)
267     varied = dates[31:][np.argsort(-volatility, kind="stable")[:PROTOCOL["high_variation_dates_count
"]]]
268     targets = sorted(set(pd.Timestamp(d) for d in PROTOCOL["specified_dates"]) | set(varied))
269     records = []
270     for date in targets:
271         d = dates.get_loc(date)
272         audit_path = main/f"daily_audit/{date.date()}.json"
273         plan_path = main/f"frozen_plans/{date.date()}.csv"
274         reference = json.loads(audit_path.read_text(encoding="utf-8"))
275         k = reference["selected_K"]; selected = reference[str(k)]
276         soc = float(frame[frame.date == date].initial_soc.iloc[0])
277         point, blocks, _ = choose_day(store, frozen, actual[:d], d, k, prices,
278                                     fixed_pair=selected["selection"]["pipeline"])
279         net, weights, _ = construct(point, blocks, np.tile(prices, k), selected["S"])
280         context = richer_context(net, weights, np.tile(prices, k), soc, audit_path, plan_path)
281         main_day = frame[frame.date == date]
282         cached = load_richer_policy(output, str(date.date()), context, net, np.tile(prices, k),
weights, soc, solver_gap=solver_gap)
283         if cached is not None:
284             policy, record = cached
285             actual_metrics = richer_real_metrics(policy, DT*(actual[d, :, 0]-actual[d, :, 1]),
prices, soc, main_day)
286             for key, value in actual_metrics.items():
287                 np.testing.assert_allclose(value, record[key], atol=PHYSICAL_TOL, rtol=0)
288             records.append(record)
289             atomic_json(output/"richer_soc_policy.json", {"records": records, "complete": len(
records) == len(targets),
290                                                         "reliable_all": all(v["solver"]["reliable"]
for v in records)})
291             continue
292         frozen_plan = pd.read_csv(plan_path, float_precision="round_trip")
293         base = Policy(frozen_plan.grid, frozen_plan.charge_cap, frozen_plan.discharge_cap)
294         rich_seed = Policy(base.grid, np.repeat(base.charge_cap[:, None], 3, axis=1),
295                           np.repeat(base.discharge_cap[:, None], 3, axis=1))

```

```

296     rich, refinement = refine(rich_seed, net, np.tile(prices, k), weights, soc)
297     _, _, bound = recourse_bound(net, np.tile(prices, k), weights, soc, target_gap=solver_gap)
298     responses = [replay(rich, path, soc) for path in net]
299     cost = float(np.tile(prices, k)@rich.grid+sum(5*p*(np.tile(prices, k)@r["emergency"]) for p,
r in zip(weights, responses)))
300     bound.update(incumbent_cost=cost, certified_gap=max(cost-bound["lower_bound"], 0)/max(abs(
bound["lower_bound"]), 1e-8),
301                 rich_refinement=refinement, seconds=bound["seconds"]+refinement["seconds"])
302     if bound["certified_gap"] <= solver_gap:
303         policy, _, solver = certify_policy(net, np.tile(prices, k), weights, soc, rich,
responses, bound, solver_gap)
304     else:
305         policy, _, solver = solve_policy(net, np.tile(prices, k), weights, soc, seed_policy=rich
,
306                                     known_lower_bound=bound["lower_bound"],
307                                     gap=solver_gap, time_limit=PROTOCOL["solver_seconds"],
segments=3)
308         if not solver["reliable"] and rescue_seconds is not None:
309             original = solver
310             atomic_json(output/f"richer_soc_attempts/{date.date()}_base_{time.time_ns()}.json",
original)
311             policy, _, solver = solve_policy(net, np.tile(prices, k), weights, soc,
312                                         seed_policy=policy if policy is not None else rich,
313                                         known_lower_bound=bound["lower_bound"],
314                                         gap=solver_gap, time_limit=rescue_seconds, segments
=3)
315             solver["original_budget_failure"] = original
316             record = {"date": str(date.date()), "K": k, "S": len(weights), "solver": solver,
317                     "same_information": True, "used_to_change_main": False,
318                     "execution": {"gap": solver_gap, "precision_revision_sha256": execution_revision}}
319             if policy is not None:
320                 record.update(richer_real_metrics(policy, DT*(actual[d, :, 0]-actual[d, :, 1]), prices,
soc, main_day))
321                 record = save_richer_policy(output, record, policy, context)
322                 records.append(record)
323                 atomic_json(output/"richer_soc_policy.json", {"records": records, "complete": len(records)
== len(targets),
324                                     "reliable_all": all(v["solver"]["reliable"] for
v in records)})
325             if not all(v["solver"]["reliable"] for v in records):
326                 raise ComputationalLimit({"experiment": "richer_soc", "records": records})
327
328
329 def run_experiments(run_path, only=None, *, rescue_seconds=None, solver_gap=PROTOCOL["solver_gap"],
execution_revision=None):
330     run_path = Path(run_path)
331     store, frozen, actual, dates, prices = load_prepared(run_path)
332     root = run_path/"processed"
333     output = root/"experiments"
334     output.mkdir(parents=True, exist_ok=True)
335     if only is None or only == "forecast":
336         forecast_audit(store, frozen, actual, dates, prices, output)
337         no_storage_quantile(store, frozen, actual, dates, prices, output)
338         if only == "forecast":
339             return {"forecast": {"complete": True}}
340     warm_file = root/"warm_start.json"
341     initial = json.loads(warm_file.read_text(encoding="utf-8"))["feb1_baseline_soc"] if warm_file.

```

```

exists() else warm_start(actual, dates, prices, root)
342 status = {}
343 def record(name, function):
344     if only is not None and name != only:
345         return
346     try:
347         function()
348         status[name] = {"complete": True}
349     except (RuntimeError, ValueError, AssertionError) as error:
350         status[name] = {"complete": False, "reason": str(error), "details": getattr(error, "
details", None)}
351     write_json(output/f"status/{name}.json", status[name])
352 for direct in (False, True):
353     name = "structure_direct_net" if direct else "structure_separate"
354     record(name, lambda direct=direct, name=name: structural_rollout(actual, dates, prices,
initial, output/name, direct=direct))
355 main = root/"main"
356 main_state = json.loads((main/"run_status.json").read_text(encoding="utf-8")) if (main/"
run_status.json").exists() else {}
357 frame = pd.read_csv(main/"dispatch.csv", parse_dates=["date", "timestamp"], float_precision="
round_trip") if main_state.get("complete") else None
358 specifications = {
359     "deterministic": {"kind": "deterministic", "reference_audits": main/"daily_audit"},
360     "rigid": {"kind": "rigid", "reference_audits": main/"daily_audit"},
361     **{f"predictor_{load}_{pv}": {"fixed_pair": ["load_week" if load == "week" else f"load_lgb_{
frozen}"],
362                                     "pv_mean7" if pv == "mean7" else "pv_dhr"]}}
363     for load in ("week", "lgb") for pv in ("mean7", "dhr")},
364     **{f"window_{w or 'expanding'}": {"window": w} for w in (28, 56, 84, None)},
365     **{f"horizon_{k}": {"horizon": k} for k in (1, 2, 3)},
366     **{f"scenario_{s}": {"window": None, "forced_count": s} for s in (10, 20, 40)},
367     **{f"initial_{soc:g}": {"initial_override": soc} for soc in sorted({1200., 6000., initial,
10800.})},
368 }
369 if frame is not None:
370     baseline = combined(store, ("load_week", "pv_mean7"))
371     common_start = next(d for d in range(31, len(dates)) if len(eligible_origins(baseline, d, 3,
28)))
372     common_soc = float(frame[frame.date == dates[common_start]].initial_soc.iloc[0])
373     for threshold in (14, 21, 28):
374         specifications[f"minimum_{threshold}"] = {"minimum": threshold, "start": common_start, "
initial_override": common_soc}
375 for name, settings in specifications.items():
376     if (name in ("deterministic", "rigid") or name in main_baselines(initial)) and frame is None
:
377         if only is None or only == name:
378             status[name] = {"complete": False, "reason": "主回放尚未完整通过, 缺少共同预测与场景审计"}
379             write_json(output/f"status/{name}.json", status[name])
380             continue
381     current = dict(settings); soc = current.pop("initial_override", initial)
382     def execute(current=current, soc=soc, name=name):
383         if reuse_main_baseline(run_path, name, store, frozen, actual, dates, prices, initial):
384             return
385         result, _ = run_rollout(store, frozen, actual, dates, prices, soc, output/name,
386                                 rescue_seconds=rescue_seconds, solver_gap=solver_gap,
387                                 execution_revision=execution_revision, **current)
388         validate_dispatch(result, rigid=current.get("kind") == "rigid")

```

```

389         write_tables(result, output/name)
390         record(name, execute)
391     if frame is not None:
392         cost = float(frame.planned_cost.sum()+frame.emergency_cost.sum())
393         record("oracles", lambda: write_json(output/"oracles.json", oracle_pair(frame.net_kwh.
to_numpy(), np.tile(prices, 334),
394                                     initial, frame.grid.
to_numpy(), cost)))
395         record("richer_soc", lambda: richer_policy_diagnostic(store, frozen, actual, dates, prices,
main, output,
396                                     rescue_seconds=rescue_seconds,
solver_gap=solver_gap,
397                                     execution_revision=execution_revision))
398     else:
399         for name in ("minimum_14", "minimum_21", "minimum_28", "oracles", "richer_soc"):
400             if only is None or only == name:
401                 status[name] = {"complete": False, "reason": "主回放尚未完整通过，不能杜撰共同SOC或Oracle"}
402             write_json(output/f"status/{name}.json", status[name])
403     if only is not None and not status:
404         raise ValueError(f"未知实验: {only}")
405     return status

```

src/q2/export_dispatch.py

```

1  """方案2.9.3: 真实回放CSV、连续紧急区间及result2原模板回填/逐值回读。"""
2
3  from copy import copy
4  from pathlib import Path
5
6  import numpy as np
7  import pandas as pd
8  from openpyxl import load_workbook
9  from openpyxl.comments import Comment
10
11  from data import calendar_for_day, write_csv, write_json
12  from policy import CAP, E_MIN, E_MAX, ETA_D, PHYSICAL_TOL, validate_replay
13  from protocol import PROTOCOL
14
15
16  def validate_dispatch(frame, *, full_period=False, rigid=False):
17      if frame.empty:
18          raise ValueError("真实回放为空")
19      required = ["grid", "net_kwh", "initial_soc", "charge", "discharge", "emergency", "unused", "soc",
20                  "price", "planned_cost", "emergency_cost"]
21      if not np.isfinite(frame[required].to_numpy(dtype=float)).all() or (frame.grid < 0).any() or (
22          frame.price <= 0).any():
23          raise ValueError("回放包含非有限数值、负购电或非正电价")
24      dates = pd.to_datetime(frame.date)
25      times = pd.to_datetime(frame.timestamp)
26      if times.duplicated().any() or not times.diff().dropna().eq(pd.Timedelta(minutes=10)).all():
27          raise ValueError("真实回放时间必须连续唯一")
28      if not dates.eq((times-pd.Timedelta(minutes=10)).dt.normalize()).all():
29          raise ValueError("源日期与右端点不符")
30      previous = None
31      for date, day in frame.groupby(dates, sort=False):

```

```

30     if len(day) != 144 or not np.array_equal(pd.to_datetime(day.timestamp).to_numpy(),
calendar_for_day(date).timestamp.to_numpy()):
31         raise ValueError("日期不是完整144时段")
32     if previous is not None and abs(float(day.initial_soc.iloc[0])-previous) > PHYSICAL_TOL:
33         raise ValueError("跨日SOC不连续")
34     if not np.allclose(day.initial_soc, day.initial_soc.iloc[0], atol=PHYSICAL_TOL, rtol=0):
35         raise ValueError("同一天的0:00初态不一致")
36     reserve = day.soc_reserve_kwh.to_numpy() if "soc_reserve_kwh" in day else np.zeros(len(day))
37     validate_replay(day.grid.to_numpy(), day.net_kwh.to_numpy(), float(day.initial_soc.iloc[0]),
38                     {k: day[k].to_numpy() for k in ("charge", "discharge", "emergency", "unused"
, "soc")}),
39                     rigid=rigid, reserve=reserve)
40     np.testing.assert_allclose(day.planned_cost, day.price*day.grid, atol=PHYSICAL_TOL, rtol=0)
41     np.testing.assert_allclose(day.emergency_cost, 5*day.price*day.emergency, atol=PHYSICAL_TOL,
rtol=0)
42     previous = float(day.soc.iloc[-1])
43 if full_period and (dates.min() != pd.Timestamp("2025-02-01") or dates.max() != pd.Timestamp("
2025-12-31") or len(frame) != 334*144):
44     raise ValueError("result2必须完整覆盖2025年2—12月，不能发布部分日期")
45
46
47 def emergency_events(frame):
48     """q>0严格定义：连续区间跨午夜继续合并，电量与费用逐段累加。"""
49     columns = ["date", "start", "end", "interval", "emergency_kwh", "cost_yuan", "intervals"]
50     rows, current = [], None
51     for row in frame.itertuples():
52         end = pd.Timestamp(row.timestamp)
53         start = end-pd.Timedelta(minutes=10)
54         if row.emergency > 0:
55             if current is None or start != current["end"]:
56                 if current is not None:
57                     rows.append(current)
58                     current = {"date": start.normalize(), "start": start, "end": end,
59                               "emergency_kwh": 0.0, "cost_yuan": 0.0, "intervals": 0}
60                 current["end"] = end
61                 current["emergency_kwh"] += row.emergency
62                 current["cost_yuan"] += row.emergency_cost
63                 current["intervals"] += 1
64             elif current is not None:
65                 rows.append(current); current = None
66         if current is not None:
67             rows.append(current)
68     for row in rows:
69         days = (row["end"].normalize()-row["start"].normalize()).days
70         suffix = "" if days == 0 else f"+{days}日"
71         row["interval"] = f'{row["start"]:%H:%M}-{row["end"]:%H:%M}-{suffix}'
72     return pd.DataFrame(rows, columns=columns)
73
74
75 def aggregate_storage(frame):
76     rows = []
77     for date, day in frame.groupby("date", sort=False):
78         for block in range(6):
79             part = day.iloc[24*block:24*(block+1)]
80             rows.append({"date": str(pd.Timestamp(date).date()), "interval": f"{block*4:02d}:00-((
block+1)*4:02d):00",
81                         "charge_kwh": float(part.charge.sum()), "discharge_kwh": float(part.

```



```

discharge.sum()),
82         "soc_at_0000": float(day.initial_soc.iloc[0]), "soc_at_2400": float(day.soc
    .iloc[-1]))
83 return pd.DataFrame(rows)
84
85
86 def summary(frame):
87     events = emergency_events(frame)
88     return {"start": str(pd.Timestamp(frame.date.iloc[0]).date()), "end": str(pd.Timestamp(frame.
    date.iloc[-1]).date()),
89           "days": int(frame.date.nunique()), "period": "evaluation_period_not_calendar_year",
90           "planned_cost_yuan": float(frame.planned_cost.sum()), "emergency_cost_yuan": float(frame
    .emergency_cost.sum()),
91           "total_cost_yuan": float(frame.planned_cost.sum()+frame.emergency_cost.sum()),
92           "planned_grid_kwh": float(frame.grid.sum()), "emergency_kwh": float(frame.emergency.sum
    ()),
93           "emergency_intervals": int((frame.emergency > 0).sum()), "emergency_events": len(events)
    ,
94           "intervals_above_numerical_tolerance": int((frame.emergency > PHYSICAL_TOL).sum()),
95           "unused_kwh": float(frame.unused.sum()), "charge_kwh": float(frame.charge.sum()),
96           "discharge_kwh": float(frame.discharge.sum()), "final_soc": float(frame.soc.iloc[-1]),
97           "charge_with_emergency_intervals": int(((frame.charge > 0)&(frame.emergency > 0)).sum())
    }
98
99
100 def write_tables(frame, output):
101     write_csv(output/"emergency_events.csv", emergency_events(frame))
102     storage = aggregate_storage(frame)
103     write_csv(output/"storage_blocks.csv", storage)
104     specified = frame[pd.to_datetime(frame.date).dt.strftime("%Y-%m-%d").isin(PROTOCOL["
    specified_dates"])]
105     rows = []
106     for date, day in specified.groupby("date"):
107         for slot in (61, 73, 85, 97, 109, 121):
108             point = day[day.slot == slot].iloc[0]
109             rows.append({"date": str(pd.Timestamp(date).date()), "interval": point.interval,
110                         "planned_kwh": float(point.grid), "daily_planned_kwh": float(day.grid.sum()
    ),
111                         "daily_planned_cost_yuan": float(day.planned_cost.sum()),
112                         "daily_actual_cost_yuan": float(day.planned_cost.sum()+day.emergency_cost.
    sum())})
113     write_csv(output/"paper_table1.csv", pd.DataFrame(rows, columns=["date", "interval", "
    planned_kwh", "daily_planned_kwh", "daily_planned_cost_yuan", "daily_actual_cost_yuan"]))
114     write_csv(output/"paper_table2.csv", storage[storage.date.isin(PROTOCOL["specified_dates"])]
115     # 指定日期表按日截取区间; 全局事件表另外保留跨日连续事件定义。
116     events = [table for _, day in specified.groupby("date")
117               if not (table := emergency_events(day)).empty]
118     write_csv(output/"paper_table3.csv", pd.concat(events, ignore_index=True) if events else
    emergency_events(specified))
119     write_json(output/"summary.json", summary(frame))
120
121
122 def final_validation(frame):
123     previous = np.r_[frame.initial_soc.iloc[0], frame.soc.to_numpy()[:-1]]
124     balance = (frame.grid+frame.discharge+frame.emergency
125               -frame.net_kwh-frame.charge-frame.unused)
126     soc_violation = (frame.soc < E_MIN-PHYSICAL_TOL) | (frame.soc > E_MAX+PHYSICAL_TOL)

```



```

127 simultaneous = (frame.charge > PHYSICAL_TOL) & (frame.discharge > PHYSICAL_TOL)
128 power = (frame.charge > CAP+PHYSICAL_TOL) | (frame.discharge > CAP+PHYSICAL_TOL)
129 emergency_charge = (frame.emergency > PHYSICAL_TOL) & (frame.charge > PHYSICAL_TOL)
130 reserve = frame.soc_reserve_kwh.to_numpy() if "soc_reserve_kwh" in frame else np.zeros(len(frame))
131 reserve_discharge = frame.discharge.to_numpy() > (
132     ETA_D*np.maximum(previous-E_MIN-reserve, 0)+PHYSICAL_TOL)
133 result = {
134     "soc_min": float(frame.soc.min()), "soc_max": float(frame.soc.max()),
135     "soc_violation_count": int(soc_violation.sum()),
136     "simultaneous_charge_discharge_count": int(simultaneous.sum()),
137     "power_violation_count": int(power.sum()),
138     "max_energy_balance_residual": float(np.abs(balance).max()),
139     "emergency_and_charge_count": int(emergency_charge.sum()),
140     "reserve_discharge_violation_count": int(reserve_discharge.sum()),
141     "intervals_checked": len(frame)}
142 result["overall_pass"] = bool(
143     result["soc_violation_count"] == 0
144     and result["simultaneous_charge_discharge_count"] == 0
145     and result["power_violation_count"] == 0
146     and result["max_energy_balance_residual"] <= PHYSICAL_TOL
147     and result["emergency_and_charge_count"] == 0
148     and result["reserve_discharge_violation_count"] == 0)
149 return result
150
151
152 def final_calibration(frame, actual, calibration):
153     day_indices = pd.DatetimeIndex(pd.to_datetime(frame.date).unique()).dayofyear.to_numpy()-1
154     realized = np.asarray(actual)[day_indices]
155     predicted_load = frame.predicted_load_kw.to_numpy()
156     predicted_pv = frame.predicted_pv_kw.to_numpy()
157     actual_load, actual_pv = realized[..., 0].ravel(), realized[..., 1].ravel()
158     series = {
159         "load": (actual_load, predicted_load),
160         "pv": (actual_pv, predicted_pv),
161         "net_load": (actual_load-actual_pv, predicted_load-predicted_pv)}
162     rows, payload = [], {}
163     for name, (observed, predicted) in series.items():
164         error = observed-predicted
165         values = {"mae_kw": float(np.abs(error).mean()),
166                 "rmse_kw": float(np.sqrt(np.mean(error**2))), "n": len(error)}
167         payload[name] = values
168         rows.extend([
169             {"metric": f"{name}_MAE", "value": values["mae_kw"], "unit": "kW", "n": len(error)},
170             {"metric": f"{name}_RMSE", "value": values["rmse_kw"], "unit": "kW", "n": len(error)}])
171     calibration = calibration[calibration.horizon == 0]
172     coverage = {}
173     for nominal, label in ((.8, "PICP80"), (.9, "PICP90")):
174         selected = calibration[np.isclose(calibration.nominal, nominal)]
175         covered, n = int(selected.covered_intervals.sum()), int(selected.n.sum())
176         value = covered/n
177         coverage[label] = {"value": value, "covered_intervals": covered, "n": n,
178                           "scope": "executed_first_day_scenarios"}
179     rows.append({"metric": label, "value": value, "unit": "fraction", "n": n})
180     payload["coverage"] = coverage
181     return pd.DataFrame(rows), payload
182

```

```

183
184 def solver_statistics(daily):
185     runtime = daily.runtime_seconds.to_numpy(dtype=float)
186     achieved = daily.achieved_mip_gap.to_numpy(dtype=float)
187     requested = daily.requested_mip_gap.to_numpy(dtype=float)
188     if not (np.isfinite(runtime).all() and np.isfinite(achieved).all()
189            and np.isfinite(requested).all()):
190         raise ValueError("solver逐日统计包含非有限数值")
191     return {"total_runtime": float(runtime.sum()),
192            "mean_daily_runtime": float(runtime.mean()),
193            "median_daily_runtime": float(np.median(runtime)),
194            "max_daily_runtime": float(runtime.max()),
195            "requested_mip_gap": float(requested.max()),
196            "achieved_mip_gap_mean": float(achieved.mean()),
197            "achieved_mip_gap_max": float(achieved.max()),
198            "days_completed": len(daily)}
199
200
201 def write_final_artifacts(frame, actual, output):
202     output = Path(output)
203     validate_dispatch(frame, full_period=True)
204     validation = final_validation(frame)
205     write_json(output/"final_validation.json", validation)
206     if not validation["overall_pass"]:
207         raise ValueError("最终物理一致性验收失败")
208     calibration = pd.read_csv(output/"calibration.csv", float_precision="round_trip")
209     calibration_table, calibration_payload = final_calibration(frame, actual, calibration)
210     write_csv(output/"final_calibration.csv", calibration_table)
211     write_json(output/"final_calibration.json", calibration_payload)
212     daily = pd.read_csv(output/"daily.csv", float_precision="round_trip")
213     solver = solver_statistics(daily)
214     write_json(output/"solver_statistics.json", solver)
215     write_json(output/"summary.json", {
216         **summary(frame),
217         "model_configuration": {
218             "tail_stratification": PROTOCOL["tail_stratification"],
219             "tail_fraction": PROTOCOL["tail_fraction"],
220             "dynamic_soc_reserve": PROTOCOL["dynamic_soc_reserve"],
221             "reserve_quantile": PROTOCOL["reserve_quantile"],
222             "evaluation_initial_soc_kwh": PROTOCOL["evaluation_initial_soc_kwh"]},
223         "solver_statistics": solver})
224
225
226 def export_result(frame, template, output):
227     validate_dispatch(frame, full_period=True)
228     output = Path(output)
229     output.mkdir(parents=True, exist_ok=True)
230     workbook = load_workbook(template)
231     expected = ["计划购电量", "充放电量", "紧急购电量"]
232     if workbook.sheetnames != expected:
233         raise ValueError("result2模板工作表结构不符")
234     plans, storage, emergencies = [workbook[name] for name in expected]
235     original_headers = [cell.value for cell in plans[1]]
236     styles = {sheet.title: [copy(c_style) for c in sheet[2]] for sheet in workbook}
237     for sheet in workbook:
238         sheet.delete_rows(2, sheet.max_row)
239     intervals = calendar_for_day("2025-02-01").interval.tolist()

```

```

240     for col, label in enumerate(intervals, 2):
241         plans.cell(1, col, label)
242     plans.cell(1, 2).comment = Comment("原模板144列整体偏移10分钟; 按方案2.2.2统一为00:00—24:00的144个区
    间。原标签已保存至template_audit.json。", "Q2模型")
243     for row, (date, day) in enumerate(frame.groupby("date", sort=False), 2):
244         plans.cell(row, 1, pd.Timestamp(date).to_pydatetime())
245         for col, value in enumerate(day.grid, 2):
246             plans.cell(row, col, float(value))
247         # 模板为冻结仿真结果的数值输入区, 费用由优化/回放引擎计算, 非可编辑Excel计算模型。
248         plans.cell(row, 146, float(day.grid.sum()))
249         plans.cell(row, 147, float(day.planned_cost.sum()))
250     blocks = aggregate_storage(frame)
251     for i, item in enumerate(blocks.itertuples(), 2):
252         block = (i-2)%6
253         for col, value in enumerate((pd.Timestamp(item.date).to_pydatetime() if block == 0 else None
    ,
254                                     item.interval, item.charge_kwh, item.discharge_kwh,
255                                     "0:00" if block == 0 else "24:00" if block == 1 else None,
256                                     item.soc_at_0000 if block == 0 else item.soc_at_2400 if block
    == 1 else None), 1):
257             storage.cell(i, col, value)
258     for i, item in enumerate(emergency_events(frame).itertuples(), 2):
259         emergencies.cell(i, 1, pd.Timestamp(item.date).to_pydatetime())
260         emergencies.cell(i, 2, item.interval)
261         emergencies.cell(i, 3, item.emergency_kwh)
262     for sheet in workbook:
263         sheet.freeze_panes = "B2" if sheet == plans else "A2"
264         for row in sheet.iter_rows(min_row=2):
265             for cell in row:
266                 cell.style = copy(styles[sheet.title][cell.column-1])
267                 if isinstance(cell.value, (float, int)):
268                     cell.number_format = "0.000000"
269     pending = output/"result2.pending.xlsx"
270     workbook.save(pending); workbook.close()
271     check = load_workbook(pending, read_only=True, data_only=True)
272     try:
273         saved = np.asarray([row[1:145] for row in list(check["计划购电量"].values)[1:]], dtype=float)
274         np.testing.assert_allclose(saved.ravel(), frame.grid, atol=1e-9, rtol=1e-12)
275         totals = np.asarray([row[145:147] for row in list(check["计划购电量"].values)[1:]], dtype=
    float)
276         np.testing.assert_allclose(totals, frame.groupby("date", sort=False)[["grid", "planned_cost"
    ]].sum(), atol=1e-8)
277         if check["充放电量"].max_row != 1+334*6:
278             raise ValueError("充放电表不是334天×6个区间")
279         for i, row in enumerate(list(check["充放电量"].values)[1:]):
280             np.testing.assert_allclose(row[2:4], blocks.iloc[i][["charge_kwh", "discharge_kwh"]].
    to_numpy(dtype=float), atol=1e-9)
281             if i%6 in (0, 1):
282                 expected_soc = blocks.iloc[i][["soc_at_0000" if i%6 == 0 else "soc_at_2400"]]
283                 np.testing.assert_allclose(row[5], expected_soc, atol=1e-9)
284             total_emergency = sum(row[2] for row in list(check["紧急购电量"].values)[1:])
285             if not np.isclose(total_emergency, frame.emergency.sum(), rtol=1e-12, atol=1e-8):
286                 raise ValueError("紧急区间汇总不守恒")
287     finally:
288         check.close()
289     pending.replace(output/"result2.xlsx")
290     write_json(output/"template_audit.json", {"original_plan_headers": original_headers,

```

```

291         "corrected_interval_headers": intervals,
292         "reason": "
template_shift_conflicts_with_144_source_day_intervals",
293         "full_period_rows": len(frame), "numeric_roundtrip":
True})
294 write_tables(frame, output)

```

src/q2/figures.py

```

1  """兼容Q2原运行入口；全部绘图实现集中在src/plots/q2.py。"""
2  import sys
3  from pathlib import Path
4
5  sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
6  from src.plots.q2 import LABELS
7
8  def create_figures(frame, diagnostics, output):
9      from src.plots.q2 import create_figures as render
10     return render(frame, diagnostics, output)

```

src/q2/final_diagnostics.py

```

1  """新方案2.3的事后修正：局部PV日曲线残差及同评价区间的季节差分方差。"""
2
3  import numpy as np
4  import pandas as pd
5  import json
6
7  from data import write_csv
8  from forecast import _harmonic_design
9  from protocol import PROTOCOL
10
11
12  def descriptive_corrections(frame, output):
13      local, differences = [], []
14      for month, group in frame.groupby("month"):
15          pv = group.pv_kw.to_numpy()
16          mean_curve = group.groupby("slot").pv_kw.transform("mean").to_numpy()
17          residual = pv - mean_curve
18          for lag in (144, 1008):
19              for name, values in (("raw_pv", pv), ("within_month_profile_residual", residual)):
20                  left, right = values[:-lag], values[lag:]
21                  correlation = float(np.corrcoef(left, right)[0, 1]) if left.std() > 0 and right.std() > 0 else np.nan
22                  local.append({"month": month, "series": name, "lag": lag, "pairs": len(left),
23                              "correlation": correlation, "scope": "
posthoc_month_never_online_features"})
24      for variable in ("load_kw", "pv_kw", "net_load_kw"):
25          values = frame[variable].to_numpy()
26          for lag in (144, 1008):
27              original, difference = values[lag:], values[lag:] - values[:-lag]
28              raw_variance, variance = np.var(original, ddof=1), np.var(difference, ddof=1)
29              differences.append({"series": variable, "lag": lag, "n": len(difference),
30                              "raw_variance_kw2_same_target_times": raw_variance,
31                              "difference_variance_kw2": variance,

```

```

32         "variance_ratio": variance/raw_variance if raw_variance else np.nan,
33         "scope": "posthoc_only_not_model_order_selection"})
34 write_csv(output/"pv_local_residual_correlations.csv", pd.DataFrame(local))
35 write_csv(output/"seasonal_difference_variance.csv", pd.DataFrame(differences))
36
37
38 def dhr_residual_stability(frame, shadows, output):
39     """各origin当时窗口的均值漂移和白噪声诊断，不修改影子预测。"""
40     actual = frame.pv_kw.to_numpy().reshape(365, 144)
41     dates = pd.DatetimeIndex(pd.to_datetime(frame.date).unique())
42     rows = []
43     for d in range(14, len(dates)):
44         record = json.loads((shadows/f"dhr/{dates[d].date()}.json").read_text(encoding="utf-8"))
45         if record["status"] != "ok":
46             continue
47         values = actual[max(d-PROTOCOL["dhr_history_days"], 0):d].ravel()
48         n = len(values)
49         design = _harmonic_design(np.arange(n), record["selected"]["harmonics"], (n-1)/2, n)
50         residual = values-design@np.linalg.lstsq(design, values, rcond=None)[0]
51         rows.append({"origin": str(dates[d].date()), "history_days": n//144,
52                     "first_half_mean_kw": float(residual[:n//2].mean()),
53                     "second_half_mean_kw": float(residual[n//2:].mean()),
54                     "first_half_std_kw": float(residual[:n//2].std(ddof=1)),
55                     "second_half_std_kw": float(residual[n//2:].std(ddof=1)),
56                     **{f"innovation_ljung_box_p_lag_{r['lag']}": r["lb_pvalue"] for r in record["ljung_box"]}},
57                     "use": "posthoc_diagnostic_only_does_not_change_orders_or_forecasts"})
58     write_csv(output/"dhr_residual_stability.csv", pd.DataFrame(rows))

```

src/q2/final_report.py

```

1     """已验证结果与待完成项分开展示；支持长回放期间生成明确标注的进度快照。"""
2
3     import argparse
4     import html
5     import json
6     from pathlib import Path
7     import sys
8     import os
9     import subprocess
10
11     import numpy as np
12     import pandas as pd
13
14     from data import ROOT, sha256, write_csv, write_json
15     from dispatch import load_prepared
16     from export_dispatch import summary, validate_dispatch
17     from final_diagnostics import descriptive_corrections, dhr_residual_stability
18     from final_traceability import build as build_traceability
19     from protocol import PROTOCOL
20     from execution import execution_settings
21
22
23     def read_json(path, default=None):
24         return json.loads(path.read_text(encoding="utf-8")) if path.exists() else ({ if default is None
25                                         else default})

```

```

25
26
27 def visual_review_current(output):
28     review = read_json(output/"figures/visual_acceptance.json")
29     names = read_json(output/"figures/manifest.json").get("figures", [])
30     hashes = review.get("reviewed_png_sha256", {})
31     return bool(names and review.get("source_sha256") == sha256(ROOT/"src/plots/q2_dispatch.py")
32                 and set(names) == set(hashes)
33                 and all((output/f"figures/{name}.png").exists()
34                         and sha256(output/f"figures/{name}.png") == hashes[name] for name in names))
35
36
37 def validation_sources():
38     paths = list((ROOT/"src/q2").glob("*.py"))+[ROOT/"src/plots/q2_dispatch.py", ROOT/"src/plots/
39             common.py"]
40     return {str(p.relative_to(ROOT)): sha256(p) for p in paths}
41
42 def verify_code(run_path):
43     before = validation_sources()
44     log = run_path/"raw/final_tests.txt"
45     generation = ROOT/"outputs/q2_current"
46     if generation.is_file():
47         generation = (generation.parent/generation.read_text(encoding="utf-8").strip()).resolve()
48     environment = {**os.environ, "Q2_GENERATION": str(generation),
49                   "OPENBLAS_NUM_THREADS": "1", "OMP_NUM_THREADS": "1", "MPLBACKEND": "Agg"}
50     with log.open("w", encoding="utf-8") as stream:
51         tested = subprocess.run([sys.executable, "-m", "unittest", "discover", "-s", "src/q2", "-p",
52                                "test_*.py", "-v"],
53                                cwd=ROOT, env=environment, stdout=stream, stderr=subprocess.STDOUT)
54     complete = tested.returncode == 0 and before == validation_sources()
55     write_json(run_path/"raw/test_manifest.json", {"complete": complete, "source_sha256": before,
56                                                  "exit_code": tested.returncode, "log_sha256":
57                                                  sha256(log)})
58     (ROOT/"docs/2/revision_work/tests.txt").write_text(log.read_text(encoding="utf-8"), encoding="
59     utf-8")
60     if not complete:
61         raise ValueError("当前源码测试失败或测试期间源码改变，不能发布正式结果")
62
63 def solver_table(directory):
64     rows = []
65     records = []
66     for p in sorted((directory/"daily_audit").glob("*.json")):
67         audit = read_json(p)
68         if "original_budget_failure" in audit:
69             records.append((p, p.stem, audit["original_budget_failure"], "original_budget_failed"))
70             records.append((p, p.stem, audit, "supplemental" if "supplemental_solver_seconds" in audit
71                             else "original"))
72         failed = read_json(directory/"run_status.json")
73         if failed.get("failed_date") and failed.get("solver_details"):
74             records.append((directory/"run_status.json", failed["failed_date"][:10], failed["
75                             solver_details"], "failed_attempt"))
76     for path, date, audit, budget in records:
77         for k in ("1", "2", "3"):
78             if k not in audit:
79                 continue

```

```

76     entry = audit[k]
77     entry = entry.get("error", entry)
78     candidates = entry.get("candidates", {str(entry.get("S", 1)): {"solver": entry.get("
selected_solver", {})}}})
79     for s, candidate in candidates.items():
80         solver = candidate["solver"]
81         before = solver.get("before_presolve") or {}
82         after = solver.get("after_presolve") or {}
83         rows.append({"date": date, "execution_budget": budget, "K": int(k), "M": entry.get("
M"), "S": int(s),
84                     "selected": int(k) == audit.get("selected_K") and int(s) == entry.get("
S"),
85                     "status": solver.get("status"), "reliable": solver.get("reliable",
False),
86                     "objective_yuan": solver.get("objective"), "lower_bound_yuan": solver.
get("dual_bound", solver.get("lower_bound")),
87                     "gap": solver.get("mip_gap"), "requested_gap": solver.get("
requested_gap", PROTOCOL["solver_gap"]),
88                     "seconds": solver.get("solve_seconds"),
89                     "continuous_before": before.get("continuous", solver.get("
continuous_variables")),
90                     "binary_before": before.get("binary", solver.get("binary_variables")),
91                     "constraints_before": before.get("constraints", solver.get("constraints
")),
92                     "continuous_after": after.get("continuous"), "binary_after": after.get(
"binary"),
93                     "constraints_after": after.get("constraints"), "solver": solver.get("
solver"),
94                     "version": solver.get("solver_version"), "audit_file": str(path.
relative_to(ROOT))})
95     return pd.DataFrame(rows)
96
97
98 def collect(run_path):
99     _, _, _, _, _ = load_prepared(run_path)
100     output, main = run_path/"processed", run_path/"processed/main"
101     # 只读取已提交CSV快照，正在写入的未提交事务不会被当作验收依据。
102     manifest_before = (main/"checkpoint.json").read_bytes() if (main/"checkpoint.json").exists()
else None
103     frame = pd.read_csv(main/"dispatch.csv", parse_dates=["date", "timestamp"], float_precision="
round_trip") if (main/"dispatch.csv").exists() else None
104     main_state = read_json(main/"run_status.json")
105     complete = bool(main_state.get("complete"))
106     real = summary(frame) if frame is not None else {"days": 0}
107     if frame is not None:
108         validate_dispatch(frame, full_period=complete)
109     if manifest_before is not None and manifest_before != (main/"checkpoint.json").read_bytes():
110         raise ValueError("读取期间检查点更新，请重新生成报告")
111     solves = solver_table(main)
112     if not solves.empty:
113         write_csv(output/"solver_records.csv", solves)
114         stats = solves.groupby(["K", "S"], as_index=False, dropna=False).agg(
115             attempted=("reliable", "size"), reliable=("reliable", "sum"),
116             mean_seconds=("seconds", "mean"), max_seconds=("seconds", "max"),
117             max_gap=("gap", "max"), mean_continuous_before=("continuous_before", "mean"),
118             max_binary_before=("binary_before", "max"), max_constraints_before=("constraints_before
", "max"),

```



```

119         max_binary_after=("binary_after", "max"), max_constraints_after=("constraints_after", "
max"))
120         write_csv(output/"solver_resource_summary.csv", stats)
121     daily = pd.read_csv(main/"daily.csv") if (main/"daily.csv").exists() else pd.DataFrame()
122     limits = int(daily.computational_limited.sum()) if not daily.empty else 0
123     supplemental_days = sum("supplemental_solver_seconds" in read_json(p) for p in (main/"
daily_audit").glob("*.json"))
124     calibration_path = main/"calibration.csv"
125     if calibration_path.exists():
126         calibration = pd.read_csv(calibration_path)
127         calibration["month"] = pd.to_datetime(calibration.origin).dt.month
128         groups = calibration.groupby(["horizon", "nominal", "month"], as_index=False)[["
covered_intervals", "n"]].sum()
129         groups["picp"] = groups.covered_intervals/groups.n
130         write_csv(output/"scenario_calibration_summary.csv", groups)
131     initial = read_json(output/"warm_start.json").get("febl_baseline_soc")
132     # 最终任务书第6节取消所有额外优化实验；正式验收只依赖主回放及其后处理。
133     required = []
134     experiments = []
135     queue_jobs = read_json(run_path/"raw/full_run_state.json").get("jobs", {})
136     for name in required:
137         path = output/"experiments"/name
138         status = read_json(output/f"experiments/status/{name}.json")
139         record = read_json(path/"summary.json")
140         finished = bool(status.get("complete"))
141         if name.startswith("structure_") and record.get("days") == 334:
142             finished = True
143         partial_days = len(pd.read_csv(path/"daily.csv")) if (path/"daily.csv").exists() else 0
144         reason = status.get("reason", "已完成" if finished else "尚未完成真实验收")
145         reuse = read_json(path/"baseline_reuse.json")
146         if finished and reuse.get("complete"):
147             reason = "已核验并复用相同主基准；未独立重复求解"
148         phase = queue_jobs.get(name, {}).get("status")
149         if not finished and phase in ("running", "pending"):
150             reason = "计算中" if phase == "running" else "排队等待工作位或主结果依赖"
151         if status.get("reason"):
152             reason += "；历史记录："+status["reason"]
153         experiments.append({"experiment": name, "complete": finished, "completed_days": record.get("
days", partial_days),
154                             "total_cost_yuan": record.get("total_cost_yuan") if finished else None,
155                             "emergency_kwh": record.get("emergency_kwh") if finished else None,
156                             "reason": reason})
157     write_csv(output/"experiment_comparison.csv", pd.DataFrame(experiments))
158     accepted_experiments = {r["experiment"] for r in experiments if r["complete"]}
159     def state(accepted, reason, *artifacts):
160         return {"accepted": bool(accepted), "reason": reason, "artifacts": list(artifacts)}
161     tested = read_json(run_path/"raw/test_manifest.json")
162     current_tests = tested.get("complete", False) and tested.get("source_sha256") ==
validation_sources()
163     shadow_status = read_json(run_path/"raw/shadows/progress.json")
164     evidence = {
165         "prepare": state(True, "原始附件逐值回读及完整准备库摘要核验通过", "raw/prepared_manifest.json", "
inputs/processed/quality_checks.csv"),
166         "forecast": state(shadow_status.get("complete", False), "已复用并逐摘要核验完整影子预测库；本轮未重
训预测器", "raw/shadows/frozen_lightgbm.json", "raw/prepared_manifest.json"),
167         "warm": state(initial == PROTOCOL["evaluation_initial_soc_kwh"], "1月仅积累预测历史，储能不主动调
度；2月1日SOC固定为6000kWh", "processed/warm_start.json"),

```



```

168     "main": state(complete, f"主回放已完成{real['days']}/334日; 计算受限{limits}日", "processed/main
169 /run_status.json", "processed/main/dispatch.csv"),
170     "compute": state(complete, f"334日均有可靠入选策略; 另有候选受限或采用补充预算的日期{limits}日", "
171 processed/main/daily_audit", "processed/main/solver_statistics.json"),
172     "export": state(complete and (main/"result2.xlsx").exists(), "仅完整334日通过物理/模板回读后导出
173 result2: 合成模板测试不代表正式结果", "processed/main/template_audit.json"),
174     "diagnostic": state(visual_review_current(output), "新增图必须与实际目视版本的PNG摘要匹配; 重绘后的
175 不同图不能沿用旧验收", "processed/pv_local_residual_correlations.csv", "processed/
176 seasonal_difference_variance.csv", "processed/figures/manifest.json"),
177     "scenarios": state(complete, "联合origin×horizon历史块及物理投影已测试; 全年场景审计随主回放完成", "
178 processed/main/daily_audit", "processed/main/calibration.csv"),
179     "calibration": state(complete, "覆盖率按horizon和月份汇总; 低覆盖必须如实报告, 不能视作预设覆盖保证",
180 "processed/scenario_calibration_summary.csv"),
181     "quantile": state(current_tests, "80%临界分位由5倍紧急购电解析推导及单元测试验收; 取消额外全年实验", "
182 raw/test_manifest.json"),
183     "optional_cvar": state(False, "可选扩展有求解入口, 未执行; 不进入风险中性主结果"),
184 }
185 groups = {"structure": [], "predictor_economics": [], "scenario_sensitivity": [],
186           "minimums": [], "initials": [], "horizons": [], "ablations": [],
187           "richer": [], "oracles": [], "experiments": required}
188 for key, names in groups.items():
189     if not names:
190         evidence[key] = state(
191             True, "按最终任务书取消; 未运行且不作为论文验收门槛",
192             "processed/experiment_comparison.csv")
193         continue
194     pending = [name for name in names if name not in accepted_experiments]
195     evidence[key] = state(not pending, "真实实验已完成" if not pending else "待完成: "+"、".join(
196 pending), "processed/experiment_comparison.csv")
197     evidence["warm_and_initials"] = evidence["initials"]
198     evidence["forecast_and_main"] = evidence["main"]
199     evidence["tests"] = state(current_tests, "当前源码测试摘要通过" if current_tests else "当前源码测试摘
200 要尚待生成或已改变", "raw/test_manifest.json", "raw/final_tests.txt")
201     evidence["all"] = state(complete and len(accepted_experiments) == len(required)
202                             and current_tests and evidence["diagnostic"]["accepted"],
203                             "所有必需条件同时验收; 当前不得用局部结果宣称整体通过", "processed/acceptance.
204 json")
205 audit = {"main": real, "main_complete": complete, "computational_limited_days": limits,
206          "supplemental_days": supplemental_days, "execution": main_state.get("execution", {}),
207          "original_budget_all_passed": complete and not limits and not supplemental_days,
208          "failed_date": main_state.get("failed_date"), "failure_reason": main_state.get("reason")
209 ),
210          "main_full_year_success_rate": real["days"]/334 if complete else None,
211          "observed_candidate_success_rate": float(solves.reliable.mean()) if not solves.empty
212 else None,
213          "observed_attempted_dates": int(solves.date.nunique()) if not solves.empty else 0,
214          "full_year_all_candidates_reliable_day_rate": (334-limits)/334 if complete else None,
215          "selected_policy_physics_valid": frame is not None, "mandatory_experiments_complete":
216 len(accepted_experiments) == len(required),
217          "completed_experiments": len(accepted_experiments), "required_experiments": len(
218 required), "evidence": evidence,
219          "all_accepted": evidence["all"]["accepted"]}
220 write_json(output/"acceptance.json", audit)
221 build_traceability(output, evidence)
222 return audit, experiments

```

```

210 def render(run_path, audit, experiments):
211     output = run_path/"processed"
212     main, done = audit["main"], audit["main_complete"]
213     state = "主回放完整；其他验收见下表" if done else (
214         f"主回放在{audit['failed_date'][:10]}因计算限制停止 · 不构成完整正式结果"
215         if audit.get("failed_date") else "计算中 · 不构成完整正式结果")
216     cards = [{"真实回放", f"{main['days']} / 334 日"}, ("必需实验", f"{audit['completed_experiments']} / {audit['required_experiments']}"),
217              ("计算受限日", str(audit["computational_limited_days"]))]
218     content = ''.join(f'<div class="card"><span>{label}</span><strong>{value}</strong></div>' for
219                       label, value in cards)
220     body = '<p>执行依据为用户指定的新论文终版与代码任务书。预测缓存及在线选模保持不变；场景总数保持原设置，并按历史累
221           计净负荷低估压力分为约80%主体层与20%尾部层。当天144段动态SOC安全裕度采用历史误差未来累计压力的80%分位，0:00冻
222           结，代理日为0。1月储能不主动调度，2月1日SOC固定为6000kWh。</p>'
223     precision = execution_settings(run_path)
224     current_gap = precision.get("solver_gap", PROTOCOL["solver_gap"])
225     if precision:
226         body += f'<p><strong>用户授权精度修订：</strong>后续单次优化gap门槛为{current_gap:.0%}，此前各精度阶
227         段已完成的结果按原策略和末SOC保留。各实验切换日期及旧执行设置单独记录；不是从第一天采用新停止规则的重跑，亦不代表全
228         年电费或预测误差为{current_gap:.0%}。物理约束、场景及稳定性门槛不变。<a href="../../raw/
229         solver_precision_revision.json">修订与选择依据</a></p>'
230     execution = read_json(run_path/"raw/full_run_execution.json")
231     if execution:
232         budget = execution["execution"]["rescue_seconds"]
233         body += f'<p><strong>补充执行预算：</strong>用户授权整日候选全部受限时追加每次{budget:g}秒计算，当前
234         gap为{current_gap:.0%}；主回放已使用补算{audit["supplemental_days"]}日。原120秒失败记录保留，补算成功不改
235         写为原预算通过。<a href="../../raw/full_run_execution.json">执行记录</a> · <a href="progress.html">实时
236         进度</a></p>'
237     if (run_path/"raw/supplemental_solver_revision.json").exists():
238         body += '<p>后续补算采用经独立等价检查的HiGHS物理边界线性MILP，并逐步回放核对min/max响应。求解路线按当次
239         执行设置选择；两种编码目标、策略类、场景相同，均遵循当次执行精度。<a href="../../raw/
240         supplemental_solver_revision.json">补算求解实现记录</a></p>'
241     if (run_path/"raw/base_solver_revision.json").exists():
242         body += f'<p>当前{current_gap:.0%}执行的自由单段策略在LP证书不足时，首次120秒直接采用已验证的等价HiGHS
243         MILP；整日所有候选仍失败时才补充900秒。固定策略与三段SOC仍保留原生入口。历史失败不改写。<a href="../../raw/
244         base_solver_revision.json">首轮求解路线与验证</a></p>'
245     if (run_path/"raw/horizon_compute_revision.json").exists():
246         body += '<p>提速验证未支持全年固定48小时，仍按原阈值比较48与72小时。只有两者都未获可靠解才求24小时兜底；主
247         动跳过记录不冒充求解成功。基准初值和28日窗口仅在与主结果完全一致时复用，并保留来源及核验记录。<a href="../../raw/
248         horizon_compute_revision.json">提速验证与执行记录</a></p>'
249     if (run_path/"raw/charge_dominance_revision.json").exists():
250         body += '<p>单段补算还采用充电上限支配性预处理：在本模型中，提高充电上限使逐步SOC不减、紧急购电不减，因此可
251         固定为题设上限而保持最优费用不变。固定策略回放和三段SOC诊断保持各自原参数。<a href="../../raw/
252         charge_dominance_revision.json">证明与验证记录</a></p>'
253     if (run_path/"raw/branch_cuts_revision.json").exists():
254         body += '<p>精确反馈线性编码增加原分支已隐含的四条约束，以加强求解下界；所有原可行策略仍保留，目标、场景不
255         变，精度按当次执行设置。<a href="../../raw/branch_cuts_revision.json">分支约束证明与验证</a></p>'
256     if (run_path/"raw/monotone_charge_revision.json").exists():
257         body += '<p>自由单段策略的后续补算采用单调充电下界：允许场景轨迹少充电，但放电仍遵守原反馈；执行时用同一共享
258         策略恢复原最大允许充电响应。原反馈逐步SOC不减、紧急购电和总费用不减，因此该模型与原模型最优费用相同，其全局下界用于
259         当次精度证书。实际执行始终采用原min/max反馈，少充轨迹从不执行；原轨迹差异、线性可行性、整数性和单调投影分别核验。
260         固定策略与三段SOC仍用精确反馈。<a href="../../raw/monotone_charge_revision.json">证明及正式实例验证</a></p>'
261     body += '<p>精确反馈策略的可行上界与自由响应LP全局下界共同证明误差门槛；LP自由响应从不执行。必要时继续求解原生
262           indicator MILP。三段SOC扩展也用真实半开区间回放核验，不能用完美信息Oracle替代同信息策略比较。</p>'
263     if main['days']:
264         body += f'<p>已完成区间：{main['start']}—{main['end']}。该区间真实费用 {main['total_cost_yuan']}</p>'

```

```

    ']:,.2f} 元, 紧急购电 {main['emergency_kwh']:,.2f} kWh; 紧急购电同时充电的时段数为 {main['
charge_with_emergency_intervals']}. </p>"
243 figures = read_json(output/"figures/manifest.json").get("figures", [])
244 images = ''.join(f'<figure><figcaption>{html.escape(name)} • <a href="figures/{html.escape(name)}.svg">SVG</a></
figcaption></figure>' for name in figures)
245 descriptive = []
246 original_figures = ROOT/"outputs/processed/q2/figures"
247 for item in read_json(ROOT/"outputs/q2/raw/figure_manifest.json", []):
248     name = item["name"]
249     if name in ("seasonal_moments", "daily_peaks"):
250         continue
251     path = original_figures/f"{name}.png"
252     if path.exists():
253         link = os.path.relpath(path, output)
254         descriptive.append(f'<figure><
figcaption>{html.escape(item["caption"])} • 全年事后描述, 不用于历史选模</figcaption></figure>')
255 images += '<h2>保留的数据结构诊断</h2><p>原始光伏高相关主要反映昼夜形状, 不能直接当作随机幅度可预测性的证明。星
期分组不强加光伏星期驱动; 频谱不用于反向选择早期模型阶数。季节差分的方差压缩见上方新图。</p>'+''.join(
descriptive)
256 links = [("逐源行/句子追溯CSV", "source_sentence_traceability.csv"), ("求解器逐候选记录", "
solver_records.csv"),
257         ("计算规模与时间", "solver_resource_summary.csv"), ("场景覆盖率", "
scenario_calibration_summary.csv"),
258         ("最终预测与覆盖统计", "main/final_calibration.csv"), ("最终物理验收", "main/
final_validation.json"),
259         ("solver运行统计", "main/solver_statistics.json"), ("真实执行CSV", "main/dispatch.csv")]
260 nav = ''.join(f'<a href="{path}">{label}</a>' for label, path in links if (output/path).exists()
)
261 table = pd.DataFrame(experiments).rename(columns={"experiment": "实验", "complete": "已完成", "
completed_days": "完成日数", "total_cost_yuan": "全期费用/元", "emergency_kwh": "紧急购电/kWh", "reason
": "状态说明"}).to_html(index=False, na_rep="待完成", border=0, float_format=lambda v: f"{v:,.2f}")
262 document = f'<!doctype html><html lang="zh-CN"><meta charset="utf-8"><meta name="viewport"
content="width=device-width,initial-scale=1"><title>第二问 • 最终方案执行与验收</title><style>
263 body{{margin:0;background:#f3f5f7;color:#172c3b;font:16px/1.75 system-ui,-apple-system,sans-serif}}
main{{max-width:1160px;margin:40px auto;padding:0 24px}}h1{{font-size:30px;margin-bottom:8px}}.
state{{color:#8a4812}}.cards{{display:grid;grid-template-columns:repeat(3,1fr);gap:16px;margin
:24px 0}}.card{{background:white;padding:20px;border:1px solid #db2e8b;border-radius:12px}}.card
span{{display:block;color:#516777}}.card strong{{display:block;font-size:28px;color:#155b7a}}
nav{{display:flex;flex-wrap:wrap;gap:12px 22px;padding:20px 0}}a{{color:#126588}}figure{{margin
:24px 0;padding:16px;background:white;border-radius:12px}}img{{width:100%;height:auto}}
figcaption{{color:#637786;font-size:14px}}.table{{overflow:auto;background:white;padding:16px;
border-radius:12px}}table{{border-collapse:collapse;font-size:14px;width:100%;white-space:nowrap
}}th,td{{padding:9px 14px;text-align:left;border-bottom:1px solid #e3e8ec}}footer{{font-size:14
px;color:#627381;margin:30px 0}}@media(max-width:600px){{main{{margin:20px auto;padding:0 14px}}
h1{{font-size:25px}}.cards{{grid-template-columns:1fr}}figure{{padding:4px}}}}
264 </style><main><h1>第二问 • 最终方案执行与验收</h1><div class="state">{state}</div><div class="cards">{
content}</div>{body}<nav>{nav}</nav><h2>必需实验的真实状态</h2><div class="table">{table}</div><h2>
预测、场景与真实回放</h2><images><footer>全部全年预测统计均为事后评价, 不回流历史决策。未完成指标显示“待完
成”, 不填零、不推算全年费用。完整验收条件见 acceptance.json; 新方案原文保持不变。</footer></main></html>'
265 (output/"report.html").write_text(document, encoding="utf-8")
266 pending = [r["experiment"] for r in experiments if not r["complete"]]
267 markdown = f"# 第二问最终方案执行状态\n\n{state}。主回放{main['days']/334日, 计算受限{audit['
computational_limited_days']}日; 必需实验{audit['completed_experiments']}/{audit['
required_experiments']}}。\\n\\n假设与预注册: 完整日j<d、因果预热初态、10分钟聚合运行近似; 初始数值容差/求解预
算见raw/protocol.json; 已启用的用户精度修订见raw/solver_precision_revision.json。可行策略与全局LP下界证

```

书不改变策略类。\\n\\n未完成实验: {'、'.join(pending) or '无'}。\\n\\n逐句追溯:
source_sentence_traceability.csv。真实费用仅在相同完整区间比较, 不将部分日期外推为全年。详见report.html与
acceptance.json。\\n"
(output/"report.md").write_text(markdown, encoding="utf-8")

```

271 def generate(run_path):
272     run_path = Path(run_path).resolve()
273     frame = pd.read_csv(run_path/"inputs/processed/timeseries.csv")
274     descriptive_corrections(frame, run_path/"processed")
275     dhr_residual_stability(frame, run_path/"raw/shadows", run_path/"processed")
276     from experiments import scenario_audit
277     store, _, actual, dates, prices = load_prepared(run_path)
278     experiments = run_path/"processed/experiments"
279     directories = sorted(experiments.iterdir()) if experiments.exists() else []
280     for directory in [run_path/"processed/main", *directories]:
281         if directory.is_dir():
282             scenario_audit(store, actual, dates, prices, directory)
283     sys.path.insert(0, str(ROOT/"src/plots"))
284     from q2_dispatch import create
285     create(run_path)
286     audit, experiments = collect(run_path)
287     render(run_path, audit, experiments)
288     print(json.dumps({key: value for key, value in audit.items() if key != "evidence"}, ensure_ascii=False), flush=True)
289     return audit
290
291 if __name__ == "__main__":
292     parser = argparse.ArgumentParser(description=__doc__)
293     parser.add_argument("--run-dir", type=Path, required=True)
294     parser.add_argument("--verify", action="store_true", help="运行当前源码测试并保存源码摘要门禁")
295     args = parser.parse_args()
296     if args.verify:
297         verify_code(args.run_dir.resolve())
298     generate(args.run_dir)
299 
```

src/q2/final_traceability.py

```

1  """最终方案逐源行/句子追溯: 代码定位与真实验收分列, 文件存在不等于模型已通过。"""
2
3  import ast
4  import re
5  from functools import lru_cache
6  from pathlib import Path
7
8  import pandas as pd
9
10 from data import ROOT, sha256, write_csv, write_json
11 from protocol import PLAN
12
13 # 逐节审查后的语义对应; 展示公式每行保留原文, 禁止沿用旧方案的265句验收。
14 RULES = {
15     "2.1": ("rolling.py#run_rollout;policy.py#respond;optimization.py#build_model", "test_dispatch.py#test_load_priority_and_soc_clipping", "main"),
16     "2.2": ("dispatch.py#input_data", "test_q2.py#Q2Tests", "prepare"),

```

```

17 "2.2.1": ("data.py#read_attachment;dispatch.py#input_data", "test_q2.py#Q2Tests", "prepare"),
18 "2.2.2": ("data.py#calendar_for_day;policy.py#respond", "test_dispatch.py#
test_load_priority_and_soc_clipping", "prepare"),
19 "2.2.3": ("shadows.py#build_shadow_library;rolling.py#run_rollout;shadows.py#choose_day", "
test_forecast_dispatch.py#test_execution_cannot_call_optimizer_and_prefix_cannot_see_future", "
forecast"),
20 "2.2.4": ("data.py#read_attachment;data.py#jump_context", "test_q2.py#Q2Tests", "prepare"),
21 "2.2.5": ("rolling.py#warm_start;experiments.py#run_experiments", "test_forecast_dispatch.py#
test_deterministic_certificate_matches_exact_model", "warm_and_initials"),
22 "2.3": ("final_diagnostics.py#descriptive_corrections", "test_q2.py#Q2Tests", "diagnostic"),
23 "2.3.1": ("../plots/q2.py#create_figures;diagnostics.py#describe", "test_q2.py#Q2Tests", "
diagnostic"),
24 "2.3.2": ("diagnostics.py#lag_analysis;../plots/q2.py#create_figures", "test_q2.py#Q2Tests", "
diagnostic"),
25 "2.3.3": ("final_diagnostics.py#descriptive_corrections;forecast.py#harmonic_forecast", "
test_forecast_dispatch.py#test_dhr_orders_and_diagnostics_use_fixed_dictionary", "diagnostic"),
26 "2.3.4": ("diagnostics.py#spectral_analysis;../plots/q2.py#create_figures", "test_q2.py#
test_spectrum_known_daily_weekly_peaks_and_parseval", "diagnostic"),
27 "2.3.5": ("diagnostics.py#stability_analysis;final_diagnostics.py#descriptive_corrections", "
test_q2.py#Q2Tests", "diagnostic"),
28 "2.3.6": ("final_diagnostics.py#descriptive_corrections;../plots/q2_dispatch.py#create", "
test_q2.py#test_transforms_match_four_term_identity", "diagnostic"),
29 "2.4": ("forecast.py#seasonal;experiments.py#forecast_audit", "test_forecast_dispatch.py#
ForecastTests", "forecast"),
30 "2.4.1": ("forecast.py#seasonal;experiments.py#structural_rollout;experiments.py#forecast_audit"
, "test_forecast_dispatch.py#test_recursive_seasonal_uses_its_own_previous_prediction", "
structure"),
31 "2.4.2": ("forecast.py#load_features;forecast.py#lightgbm_forecast;shadows.py#freeze_lightgbm",
"test_forecast_dispatch.py#test_exact_eight_load_features_and_actual_candidate;
test_forecast_dispatch.py#test_freezing_cannot_use_february_hyperparameter_performance", "
forecast"),
32 "2.4.3": ("forecast.py#harmonic_forecast;shadows.py#choose_day;protocol.py#PROTOCOL;experiments.
py#run_experiments", "test_forecast_dispatch.py#
test_dhr_orders_and_diagnostics_use_fixed_dictionary", "predictor_economics"),
33 "2.4.4": ("shadows.py#choose_day;forecast.py#economic_metrics;forecast.py#select_pipeline", "
test_forecast_dispatch.py#test_four_to_one_loss_and_lexicographic_selection;
test_forecast_dispatch.py#test_real_selection_metadata_serializes_numpy_origin_indices", "
forecast_and_main"),
34 "2.5": ("shadows.py#build_shadow_library;scenarios.py#joint_blocks", "test_dispatch.py#
ScenarioTests", "scenarios"),
35 "2.5.1": ("shadows.py#build_shadow_library;shadows.py#choose_day;scenarios.py#joint_blocks", "
test_dispatch.py#test_complete_origin_horizon_and_window_fallback", "scenarios"),
36 "2.5.2": ("scenarios.py#eligible_origins;scenarios.py#history_window", "test_dispatch.py#
test_complete_origin_horizon_and_window_fallback", "scenarios"),
37 "2.5.3": ("scenarios.py#history_window;shadows.py#choose_day", "test_dispatch.py#
test_complete_origin_horizon_and_window_fallback", "scenarios"),
38 "2.5.4": ("scenarios.py#joint_blocks;scenarios.py#construct", "test_dispatch.py#
test_no_bootstrap_below_forty_and_physical_projection", "scenarios"),
39 "2.5.5": ("scenarios.py#cumulative_pressure;scenarios.py#construct;rolling.py#select_resolution"
, "test_dispatch.py#test_tail_strata_preserve_count_origins_and_probability_mass;test_dispatch.
py#test_tail_stratification_does_not_increase_reduced_scenario_count", "scenarios"),
40 "2.5.6": ("scenarios.py#construct;scenarios.py#weighted_quantile", "test_dispatch.py#
test_tail_strata_preserve_count_origins_and_probability_mass;test_dispatch.py#
test_weighted_quantile_uses_discrete_probability", "scenarios"),
41 "2.6": ("forecast.py#economic_metrics;scenarios.py#weighted_quantile;experiments.py#
no_storage_quantile", "test_dispatch.py#test_no_storage_optimizer_is_eighty_percent_quantile;
test_dispatch.py#test_weighted_quantile_uses_discrete_probability", "quantile"),

```



```

42 "2.7": ("policy.py#Policy;policy.py#respond;optimization.py#build_model", "test_dispatch.py#
PolicyTests", "main"),
43 "2.7.1": ("rolling.py#make_day;rolling.py#run_rollout", "test_forecast_dispatch.py#RollingTests"
, "horizons"),
44 "2.7.2": ("policy.py#Policy;optimization.py#build_model;rolling.py#run_rollout", "test_dispatch.
py#test_frozen_parameters_and_causal_prefix", "main"),
45 "2.7.3": ("policy.py#respond;optimization.py#build_model", "test_dispatch.py#
test_load_priority_and_soc_clipping;test_dispatch.py#
test_exact_indicator_mapping_fixed_random_paths", "main"),
46 "2.7.4": ("scenarios.py#dynamic_reserve;policy.py#respond;policy.py#validate_replay;optimization
.py#build_model", "test_dispatch.py#test_cumulative_pressure_and_dynamic_reserve_use_kwh_paths;
test_dispatch.py#test_indicator_model_and_replay_share_dynamic_reserve;test_linear_policy.py#
test_dynamic_reserve_linear_native_and_replay_are_equivalent", "main"),
47 "2.7.5": ("policy.py#respond;policy.py#validate_replay;optimization.py#build_model", "
test_dispatch.py#test_load_priority_and_soc_clipping", "main"),
48 "2.7.6": ("optimization.py#build_model;optimization.py#solve_policy;optimization.py#
certify_policy;relaxation.py#recourse_bound;linear_policy.py#build_linear_policy;linear_policy.
py#solve_linear_policy", "test_dispatch.py#test_unfixed_policy_replay_matches_solver;
test_forecast_dispatch.py#test_lower_bound_is_not_executed_and_oracles_are_nested;
test_linear_policy.py#LinearPolicyTests", "compute"),
49 "2.7.7": ("optimization.py#build_model;optimization.py#solve_policy;linear_policy.py#
build_linear_policy;rolling.py#make_day", "test_dispatch.py#
test_indicator_model_and_replay_share_dynamic_reserve;test_linear_policy.py#
test_dynamic_reserve_linear_native_and_replay_are_equivalent", "main"),
50 "2.8": ("optimization.py#build_model;optimization.py#solve_policy", "test_dispatch.py#
test_unfixed_policy_replay_matches_solver", "main"),
51 "2.8.1": ("optimization.py#build_model;rolling.py#make_day;linear_policy.py#build_linear_policy"
, "test_dispatch.py#test_unfixed_policy_replay_matches_solver;test_linear_policy.py#
test_maximum_charge_cap_dominates_for_each_realized_step", "main"),
52 "2.8.2": ("optimization.py#solve_policy", "", "optional_cvar"),
53 "2.9": ("rolling.py#run_rollout;export_dispatch.py#export_result", "test_forecast_dispatch.py#
ExportTests;test_forecast_dispatch.py#RollingTests", "main"),
54 "2.9.1": ("rolling.py#warm_start;shadows.py#build_shadow_library", "test_forecast_dispatch.py#
test_deterministic_certificate_matches_exact_model", "warm"),
55 "2.9.2": ("rolling.py#make_day;rolling.py#run_rollout;checkpoint.py#read_checkpoint", "
test_forecast_dispatch.py#RollingTests", "main"),
56 "2.9.3": ("rolling.py#dispatch_frame;export_dispatch.py#emergency_events;export_dispatch.py#
export_result", "test_forecast_dispatch.py#
test_complete_template_roundtrip_and_partial_rejection;test_forecast_dispatch.py#
test_cross_midnight_events_and_empty_events", "export"),
57 "2.10": ("experiments.py#run_experiments", "test_forecast_dispatch.py#ComparatorTests", "
experiments"),
58 "2.10.1": ("experiments.py#forecast_audit;shadows.py#choose_day", "test_forecast_dispatch.py#
ForecastTests;test_forecast_dispatch.py#RollingTests", "forecast_and_main"),
59 "2.10.2": ("experiments.py#forecast_audit;rolling.py#run_rollout", "test_dispatch.py#
test_complete_origin_horizon_and_window_fallback", "calibration"),
60 "2.10.3": ("rolling.py#select_resolution;scenarios.py#quantile_comparison;experiments.py#
scenario_audit;experiments.py#run_experiments", "test_dispatch.py#
test_tail_protection_and_cluster_probability", "scenario_sensitivity"),
61 "2.10.4": ("experiments.py#run_experiments;scenarios.py#eligible_origins", "test_dispatch.py#
test_complete_origin_horizon_and_window_fallback", "minimums"),
62 "2.10.5": ("rolling.py#dispatch_frame;comparators.py#rigid_schedule;experiments.py#
run_experiments", "test_forecast_dispatch.py#
test_execution_cannot_call_optimizer_and_prefix_cannot_see_future", "ablations"),
63 "2.10.6": ("rolling.py#warm_start;experiments.py#run_experiments", "test_forecast_dispatch.py#
test_resume_replays_frozen_policy_and_rejects_corrupted_checkpoint", "initials"),
64 "2.10.7": ("rolling.py#make_day;experiments.py#run_experiments", "test_forecast_dispatch.py#

```

```

65     test_real_selection_metadata_serializes_numpy_origin_indices", "horizons"),
66     "2.10.8": ("experiments.py#no_storage_quantile;scenarios.py#weighted_quantile", "test_dispatch.
py#test_no_storage_optimizer_is_eighty_percent_quantile", "quantile"),
67     "2.10.9": ("experiments.py#run_experiments;comparators.py#rigid_schedule", "
test_forecast_dispatch.py#ComparatorTests", "ablations"),
68     "2.10.10": ("experiments.py#richer_policy_diagnostic;optimization.py#build_model;policy.py#
respond", "test_forecast_dispatch.py#test_richer_policy_contains_main_policy_at_soc_boundaries",
"richer"),
69     "2.10.11": ("comparators.py#oracle_pair;experiments.py#run_experiments", "test_forecast_dispatch
.py#test_lower_bound_is_not_executed_and_oracles_are_nested", "oracles"),
70     "2.11": ("dispatch.py#run_main;experiments.py#run_experiments;policy.py#respond", "test_dispatch
.py#PolicyTests;test_forecast_dispatch.py#RollingTests", "all"),
71 }
72
73 @lru_cache(maxsize=None)
74 def locate(references):
75     result = []
76     for item in filter(None, references.split(";")):
77         filename, symbol = item.split("#")
78         path = (ROOT/"src/q2"/filename).resolve()
79         tree = ast.parse(path.read_text(encoding="utf-8"))
80         matches = [node for node in ast.walk(tree) if isinstance(node, (ast.FunctionDef, ast.
ClassDef)) and node.name == symbol]
81         if not matches:
82             matches = [node for node in ast.walk(tree) if isinstance(node, ast.Assign) and any(
isinstance(t, ast.Name) and t.id == symbol for t in node.targets)]
83         if len(matches) != 1:
84             raise ValueError(f"追溯符号不存在或不唯一: {item}")
85         result.append(f"{path.relative_to(ROOT)}:{matches[0].lineno}#{symbol}")
86     return ";".join(result)
87
88
89 def build(output, evidence):
90     rows = []
91     section, math = "2.1", False
92     for number, raw in enumerate(PLAN.read_text(encoding="utf-8").splitlines(), 1):
93         line = raw.strip()
94         if not line:
95             continue
96         heading = re.match(r"^#{2,3}\s+(2\.\d+(?:\.\d+)?)\s", line)
97         if heading:
98             section = heading[1]
99         if line == "$$":
100             math = not math
101         markup = line.startswith("#") or line in ("$$", "---")
102         kind = "markup" if markup else "formula_line" if math else "sentence"
103         sentences = [line] if kind != "sentence" else [s.strip() for s in re.split(r"(?<=[。! ? ; ])"
, line) if s.strip()]
104         code, tests, criterion = RULES[section]
105         state = evidence.get(criterion, {"accepted": False, "reason": "待真实运行验收", "artifacts":
[]})
106         for i, sentence in enumerate(sentences, 1):
107             rows.append({"id": f"Q2F-L{number:04d}-S{i}", "source_line": number, "section": section,
108 "kind": kind, "source_text": sentence, "code_locations": locate(code),
109 "test_locations": locate(tests), "criterion": criterion,
110 "implementation_linked": True, "real_acceptance": bool(state["accepted"]),

```

```

111         "evidence": ";".join(state.get("artifacts", [])), "acceptance_note": state[
    "reason"]})
112     frame = pd.DataFrame(rows)
113     required = {i for i, line in enumerate(PLAN.read_text(encoding="utf-8").splitlines(), 1) if line
    .strip()}
114     if set(frame.source_line) != required or frame.id.duplicated().any():
115         raise ValueError("逐句矩阵未覆盖全部非空源行或存在重复编号")
116     write_csv(output/"source_sentence_traceability.csv", frame)
117     write_json(output/"traceability_audit.json", {"plan_sha256": sha256(PLAN), "
    nonempty_source_lines": len(required),
118                                             "records": len(frame), "source_coverage": 1.0,
119                                             "unaccepted_records": int((~frame.real_acceptance)
    .sum()),
120                                             "scope": "
    code_links_and_evidence_status_do_not_imply_unrun_experiments_passed"})
121     return frame

```

src/q2/forecast.py

```

1  """最终方案2.4：有限候选、仅历史拟合、forecast-origin内递推。"""
2
3  import itertools
4  import warnings
5
6  import numpy as np
7  import pandas as pd
8  from lightgbm import LGBMRegressor
9  from statsmodels.tsa.statespace.sarimax import SARIMAX
10 from statsmodels.tsa.stattools import acf, pacf
11 from statsmodels.stats.diagnostic import acorr_ljungbox
12
13 from protocol import PROTOCOL
14 from policy import DT
15
16
17 def seasonal(history, horizon):
18     """history仅含源日期早于origin的完整日；未来PV均值递推使用自身预测。"""
19     history = np.asarray(history, dtype=float)
20     if history.ndim != 3 or history.shape[1:] != (144, 2) or len(history) < 7:
21         raise ValueError("季节基线需要至少7个完整历史日")
22     if not np.isfinite(history).all() or (history < 0).any():
23         raise ValueError("负荷/光伏历史无效")
24     buffer = list(history.copy())
25     output = []
26     for _ in range(horizon):
27         day = np.column_stack((buffer[-7][:, 0], np.mean([v[:, 1] for v in buffer[-7:]], axis=0)))
28         buffer.append(day); output.append(day)
29     return np.asarray(output)
30
31
32 def load_features(past_load, target_date):
33     if len(past_load) < 7:
34         raise ValueError("负荷特征缺少7日历史")
35     recent = np.asarray(past_load[-7:])
36     weekday = pd.Timestamp(target_date).dayofweek
37     return np.column_stack((np.arange(1, 145), np.full(144, weekday), np.full(144, weekday < 5),

```



```

38         past_load[-1], past_load[-2], past_load[-7], recent.mean(0), recent.std
    (0, ddof=1)))
39
40
41 def lightgbm_forecast(history, origin, horizon, configuration):
42     """固定八个主特征；训练样本及未来递推均不能访问当日/未来真实量。"""
43     past = np.asarray(history)[: , : , 0]
44     first_date = pd.Timestamp(origin) - pd.Timedelta(days=len(past))
45     if len(past) < 8:
46         raise ValueError("LightGBM没有带完整滞后的训练日")
47     x = np.concatenate([load_features(past[:d], first_date+pd.Timedelta(days=d)) for d in range(7,
    len(past))])
48     y = past[7:].ravel()
49     model = LGBMRegressor(**configuration, objective="regression", n_jobs=1, random_state=0,
50                             deterministic=True, force_col_wise=True, verbosity=-1)
51     model.fit(x, y, categorical_feature=[1, 2])
52     buffer, output = list(past.copy()), []
53     for h in range(horizon):
54         features = load_features(buffer, pd.Timestamp(origin)+pd.Timedelta(days=h))
55         prediction = np.maximum(model.predict(features, validate_features=False), 0)
56         output.append(prediction); buffer.append(prediction)
57     return np.asarray(output)
58
59
60 def _harmonic_design(indices, harmonics, center, scale):
61     indices = np.asarray(indices)
62     trend = (indices-center)/scale
63     columns = [np.ones(len(indices)), trend]
64     for k in range(1, harmonics+1):
65         for wave in (np.sin(2*np.pi*k*indices/144), np.cos(2*np.pi*k*indices/144)):
66             columns.extend((wave, trend*wave))
67     return np.column_stack(columns)
68
69
70 def harmonic_forecast(history, horizon):
71     """局部日内谐波及缓慢线性振幅漂移；无年度傅里叶/未来额定功率上界。"""
72     values = np.asarray(history)[-PROTOCOL["dhr_history_days"]:, : , 1].ravel()
73     n = len(values)
74     indices = np.arange(n)
75     future = np.arange(n, n+144*horizon)
76     candidates, best = [], None
77     for harmonic in PROTOCOL["dhr_harmonics"]:
78         design = _harmonic_design(indices, harmonic, (n-1)/2, n)
79         beta = np.linalg.lstsq(design, values, rcond=None)[0]
80         residual = values-design@beta
81         for p, q in itertools.product(PROTOCOL["dhr_ar_orders"], PROTOCOL["dhr_ma_orders"]):
82             with warnings.catch_warnings(record=True) as caught:
83                 warnings.simplefilter("always")
84                 fit = SARIMAX(residual, order=(p, 0, q), trend="n",
85                             enforce_stationarity=True, enforce_invertibility=True).fit(
86                     disp=False, maxiter=PROTOCOL["dhr_max_iterations"])
87                 converged = bool(fit.mle_retvals.get("converged", False))
88                 bic = float(fit.bic + design.shape[1]*np.log(n))
89                 aic = float(fit.aic + 2*design.shape[1])
90                 record = {"harmonics": harmonic, "p": p, "q": q, "bic": bic, "aic": aic,
91                         "converged": converged, "warnings": "; ".join(str(w.message) for w in caught)}
92                 candidates.append(record)

```

```

93     key = (bic, aic, harmonic+p+q)
94     if converged and np.isfinite([bic, aic]).all() and (best is None or key < best[0]):
95         mean = _harmonic_design(future, harmonic, (n-1)/2, n)@beta
96         forecast = np.maximum(mean+np.asarray(fit.forecast(len(future))), 0).reshape(horizon
, 144)
97         best = (key, forecast, record, residual, np.asarray(fit.resid))
98     if best is None:
99         return None, {"status": "all_order_fits_failed", "candidates": candidates}, None
100     _, forecast, chosen, residual, innovations = best
101     max_lag = min(144, n//2-1)
102     residual_acf, residual_pacf = acf(residual, nlags=max_lag, fft=True), pacf(residual, nlags=
max_lag, method="burg")
103     white = acorr_ljungbox(innovations, lags=[6, 144], model_df=chosen["p"]+chosen["q"], return_df=
True)
104     diagnostics = {"status": "ok", "selected": chosen, "candidates": candidates,
105                   "residual_mean_kw": float(residual.mean()), "residual_std_kw": float(residual.std
()),
106                   "first_half_residual_std_kw": float(residual[:n//2].std()),
107                   "second_half_residual_std_kw": float(residual[n//2:].std()),
108                   "ljung_box": white.reset_index(names="lag").to_dict("records"),
109                   "annual_harmonics": False, "adf": "not_invoked_fixed_stationary_error_candidate"}
110     correlations = pd.DataFrame({"lag": np.arange(max_lag+1), "acf": residual_acf, "pacf":
residual_pacf})
111     return forecast, diagnostics, correlations
112
113
114 def economic_metrics(actual, prediction, prices):
115     """统计及决策代理损失：不使用调度费用选模。actual/prediction最后轴为L、PV。"""
116     error = np.asarray(prediction)-np.asarray(actual)
117     net_error = error[... , 0]-error[... , 1]
118     actual_d = np.maximum((actual[... , 0]-actual[... , 1])*DT, 0)
119     predicted_d = np.maximum((prediction[... , 0]-prediction[... , 1])*DT, 0)
120     delta = predicted_d-actual_d
121     return {"econ": float(np.mean(prices*(np.maximum(delta, 0)+4*np.maximum(-delta, 0)))),
122           "net_mae": float(np.abs(net_error).mean()), "net_rmse": float(np.sqrt(np.mean(net_error
**2))),
123           "load_mae": float(np.abs(error[... , 0]).mean()), "load_rmse": float(np.sqrt(np.mean(
error[... , 0]**2))),
124           "pv_mae": float(np.abs(error[... , 1]).mean()), "pv_rmse": float(np.sqrt(np.mean(error
[... , 1]**2)))}
125
126
127 def select_pipeline(scores, tolerance=None):
128     """预注册逐级容差筛选：基线排在首位，近似平局保留简单流程。"""
129     tolerance = PROTOCOL["selection_relative_tolerance"] if tolerance is None else tolerance
130     remaining = list(scores)
131     for metric in ("econ", "net_mae", "net_rmse", "load_mae", "load_rmse", "pv_mae", "pv_rmse"):
132         best = min(scores[key][metric] for key in remaining)
133         remaining = [key for key in remaining if scores[key][metric] <= best + tolerance*max(abs(
best), 1e-8)]
134         if len(remaining) == 1:
135             break
136     return remaining[0]

```

src/q2/full_run.py

```

1  """全量队列：两个独立工作进程、持久收据、依赖调度和逐日断点恢复。"""
2
3  import argparse
4  import datetime as dt
5  import html
6  import json
7  import os
8  from pathlib import Path
9  import signal
10 import subprocess
11 import sys
12 import time
13 import traceback
14 import uuid
15
16 from checkpoint import atomic_json, read_checkpoint
17 from data import ROOT
18 from protocol import PROTOCOL
19 from execution import execution_settings
20 from baseline_reuse import main_baselines
21 from rolling import _exclusive_run_lock
22
23 DEPENDENT = {"deterministic", "rigid", "minimum_14", "minimum_21", "minimum_28", "oracles", "
    richer_soc"}
24 REQUIRED_FOR_PAPER = ("main",)
25 OPTIONAL_DIAGNOSTICS = ("forecast",)
26 LEGACY_OR_DEVELOPMENT_ONLY = (
27     "structure_separate", "structure_direct_net",
28     "horizon_1", "horizon_2", "horizon_3",
29     "predictor_week_mean7", "predictor_week_dhr",
30     "predictor_lgb_mean7", "predictor_lgb_dhr",
31     "window_28", "window_56", "window_84", "window_expanding",
32     "scenario_10", "scenario_20", "scenario_40",
33     "initial_1200", "initial_6000", "initial_10800",
34     "deterministic", "rigid", "minimum_14", "minimum_21", "minimum_28",
35     "oracles", "richer_soc")
36 RESCUE_SECONDS = 900
37 WORKERS = 2
38 MAX_WORKER_RSS_KIB = 10*1024*1024 # 16GiB Mac为系统和其他应用保留约6GiB。
39
40
41 def read(path, default=None):
42     return json.loads(path.read_text(encoding="utf-8")) if path.exists() else ({} if default is None
        else default)
43
44
45 def now():
46     return dt.datetime.now(dt.timezone.utc).isoformat()
47
48
49 def job_names(initial, *, include_optional=False, include_legacy=False):
50     """默认只运行论文必需主结果：旧334日实验必须显式选择且不再作为论文门槛。"""
51     names = list(REQUIRED_FOR_PAPER)
52     if include_optional:
53         names.extend(OPTIONAL_DIAGNOSTICS)
54     if include_legacy:

```

```

55     names.extend(LEGACY_OR_DEVELOPMENT_ONLY)
56     legacy_initial = f"initial_{initial:g}"
57     if legacy_initial not in names:
58         names.append(legacy_initial)
59     return list(dict.fromkeys(names))
60
61
62 def experiment_category(name):
63     if name in REQUIRED_FOR_PAPER:
64         return "required_for_paper"
65     if name in OPTIONAL_DIAGNOSTICS:
66         return "optional_diagnostics"
67     return "legacy_or_development_only"
68
69
70 def output_for(run, name):
71     return run/"processed/main" if name == "main" else run/"processed/experiments"/name
72
73
74 def eligible(jobs):
75     """主结果依赖不满足时，仍继续所有独立实验。"""
76     return [name for name, job in jobs.items() if job["status"] == "pending"
77            and ((name not in DEPENDENT and not job.get("requires_main")) or jobs["main"]["status"]
78               == "complete")]
79
80 def owned_process(job):
81     """PID可能复用，必须同时匹配脚本及唯一尝试标识。"""
82     pid = job.get("pid")
83     if not pid:
84         return False
85     result = subprocess.run(["ps", "-p", str(pid), "-o", "command="], capture_output=True, text=True)
86     return result.returncode == 0 and "full_run.py" in result.stdout and job["attempt"] in result.stdout
87
88
89 def terminate_worker(pid):
90     if hasattr(os, "killpg"):
91         os.killpg(pid, signal.SIGTERM)
92     else:
93         os.kill(pid, signal.SIGTERM)
94
95
96 def reconcile(run, jobs):
97     for name, job in jobs.items():
98         if job["status"] != "running":
99             continue
100         receipt = read(run/job["receipt"])
101         if receipt:
102             if receipt.get("attempt") != job["attempt"] or receipt.get("job") != name:
103                 raise ValueError("工作进程收据与任务不符")
104             job.update(status="complete" if receipt["complete"] else "failed", finished=receipt["finished"],
105                       error=receipt.get("error"), exit_code=receipt["exit_code"])
106         elif not owned_process(job):
107             # 进程被外部中断时重做未提交日，不把中断当计算失败。

```

```

108         job.setdefault("interruptions", []).append({"attempt": job["attempt"], "observed": now()
109     })
110     job["status"] = "pending" if len(job["interruptions"]) < 3 else "failed"
111     if job["status"] == "failed":
112         job["error"] = "连续三次进程退出且未形成收据，保留存档并等待自动排查"
113
114 def guard_memory(run, jobs):
115     """仅当本队列工作进程合计超出10GiB时停止最大任务，避免整机内存耗尽。"""
116     measured = []
117     for name, job in jobs.items():
118         if job["status"] != "running" or not owned_process(job):
119             continue
120         result = subprocess.run(["ps", "-p", str(job["pid"]), "-o", "rss="], capture_output=True,
121             text=True)
122         if result.returncode == 0 and result.stdout.strip():
123             job["rss_kib"] = int(result.stdout.strip())
124             measured.append((job["rss_kib"], name, job))
125     if sum(row[0] for row in measured) <= MAX_WORKER_RSS_KIB:
126         return
127     rss, name, job = max(measured, key=lambda row: row[0])
128     if not owned_process(job) or (run/job["receipt"]).exists():
129         return
130     try:
131         terminate_worker(job["pid"])
132     except ProcessLookupError:
133         return # 正常结束恰好发生在身份核验之后，交给完成收据处理。
134     atomic_json(run/job["receipt"], {"job": name, "attempt": job["attempt"], "complete": False,
135         "finished": now(), "exit_code": -int(signal.SIGTERM),
136         "error": "队列工作进程总内存超过10GiB；仅中止最大任务的未提交日，已存档日
137         保留",
138         "worker_rss_kib": rss, "queue_rss_kib": sum(row[0] for row in
139         measured),
140         "resource_limit_kib": MAX_WORKER_RSS_KIB})
141
142 def monitor(run, state):
143     rows = []
144     labels = {"pending": "排队", "running": "计算中", "complete": "完成", "failed": "待自动排查"}
145     for name, job in state["jobs"].items():
146         directory = output_for(run, name)
147         checkpoint = read(directory/"checkpoint.json")
148         active = read(directory/"active_solve.json")
149         days = checkpoint.get("days", read(directory/"summary.json").get("days", "-"))
150         job["completed_days"] = days
151         activity = ""
152         if job["status"] == "running" and active.get("phase") == "solving":
153             activity = f"{active.get('date')} · K={active.get('K')} · 单次上限{active.get('
154             solver_seconds')}秒"
155         if (name in DEPENDENT or job.get("requires_main")) and job["status"] == "pending" and state[
156             "jobs"][name]["status"] != "complete":
157             activity = "等待主回放完整结果"
158         reuse = read(directory/"baseline_reuse.json")
159         if job["status"] == "complete" and reuse.get("complete"):
160             activity = "复用经核实的相同主基准；未重复求解"
161         log = f'<a href="{html.escape(job["log"])}">日志</a>' if job.get("log") else ""
162         rows.append(f'<tr><td>{html.escape(name)}</td><td>{labels[job["status"]]}</td><td>{days}</td>

```

```

>'
159         f'<td>{html.escape(activity)}</td><td>{log}</td></tr>')
160     main_days = state["jobs"]["main"]["completed_days"]
161     gap = state.get("execution", {}).get("gap", PROTOCOL["solver_gap"])
162     document = f'''<!doctype html><html lang="zh-CN"><meta charset="utf-8"><meta name="viewport"
        content="width=device-width,initial-scale=1">
163 <meta http-equiv="refresh" content="15"><title>第二问全量计算进度</title><style>
164 body{{font:16px/1.7 system-ui;background:#f3f5f7;color:#173747;margin:0}}main{{max-width:1100px;
        margin:32px auto;padding:0 20px}}
165 .cards{{display:flex;gap:20px;flex-wrap:wrap}}.card{{background:white;border-radius:12px;padding:20
        px;flex:1;min-width:190px}}
166 strong{{display:block;font-size:28px;color:#126587}}.table{{overflow:auto;background:white;padding
        :16px;margin-top:24px;border-radius:12px}}
167 table{{border-collapse:collapse;width:100%;white-space:nowrap}}td,th{{padding:9px;text-align:left;
        border-bottom:1px solid #dce6e9}}
168 a{{color:#126587}}small{{color:#526b78}}</style><main><h1>第二问全量计算进度</h1>
169 <div class="cards"><div class="card">主回放<strong>{main_days} / 334 日</strong></div><div class="
        card">额外全年实验<strong>不默认运行</strong></div></div>
170 <p>论文必需验收由正式主回放及其离线预测覆盖、物理一致性和经济后处理组成。开发阶段的27项实验不再是论文完成门槛；如需诊
        断，只能显式选择小型代表日任务。</p>
171 <p>每个完整日独立存档，两个实验同时计算。任务中断后从最后完整日续算。原120秒预算整日无可靠候选时，追加每次900秒计算，当
        前gap门槛为{gap:.0%}，已完成的更严格结果保留；历史预算失败保留，不算作原预算通过。</p>
172 <p>精度指单次优化费用证书，不是全年费用或预测误差。<a href="../raw/solver_precision_revision.json">精度修订记
        录（如已启用）</a></p>
173 <p><a href="report.html">验收报告（生成时的快照）</a> • <a href="../raw/full_run_state.json">完整运行状态
        </a></p>
174 <div class="table"><table><thead><tr><th>任务</th><th>状态</th><th>已存档日数</th><th>当前计算</th><th>记
        录</th></tr></thead><tbody>{''.join(rows)}</tbody></table></div>
175 <p><small>自动刷新15秒。更新时间: {state['updated']}; 队列状态: {html.escape(state['status'])}。完成主回放后
        才发布result2.xlsx，全部实验验收另行记录。</small></p></main></html>'''
176     pending = run/"processed/progress.html.pending"
177     pending.write_text(document, encoding="utf-8")
178     pending.replace(run/"processed/progress.html")
179
180
181 def worker(run, name, attempt, receipt):
182     result = {"job": name, "attempt": attempt, "started": now(), "pid": os.getpid()}
183     code = 0
184     try:
185         precision = execution_settings(run)
186         result["precision_execution"] = precision
187         if name == "main":
188             from dispatch import run_main
189             run_main(run, rescue_seconds=RESCUE_SECONDS, resume=True, **precision)
190         else:
191             from experiments import run_experiments
192             statuses = run_experiments(run, only=name, rescue_seconds=None, **precision)
193             if not statuses.get(name, {}).get("complete"):
194                 raise RuntimeError(f"实验尚未完成: {statuses}")
195             result["complete"] = True
196     except Exception as error:
197         result.update(complete=False, error=str(error))
198         traceback.print_exc()
199         code = 2
200     finally:
201         result.update(finished=now(), exit_code=code)
202         atomic_json(run/receipt, result)

```

```

203     return code
204
205
206 def preflight(run):
207     from dispatch import load_prepared
208     from final_report import verify_code
209     _, _, actual, dates, prices = load_prepared(run)
210     verify_code(run)
211     initial = read(run/"processed/warm_start.json")["febl_baseline_soc"]
212     checked = []
213     for directory in [run/"processed/main", *sorted((run/"processed/experiments").glob("*"))]:
214         if not (directory/"checkpoint.json").exists():
215             continue
216         status = read(directory/"run_status.json")
217         import pandas as pd
218         first = pd.read_csv(directory/"dispatch.csv", nrows=1, float_precision="round_trip")
219         frame, _, _ = read_checkpoint(directory, actual, dates, prices, float(first.initial_soc.iloc
220 [0]),
221                                     status["start_index"], status["end_index"], status["settings"
222 ])
223         checked.append({"directory": str(directory.relative_to(run)), "days": frame.date.nunique()})
224     atomic_json(run/"raw/full_run_preflight.json", {"complete": True, "at": now(), "checkpoints":
225 checked})
226     return initial
227
228
229 def supervise(run):
230     state_path = run/"raw/full_run_state.json"
231     state = read(state_path)
232     # 若监督进程重启但工作进程仍在，不在写入期间读取或恢复它的检查点。
233     if state:
234         reconcile(run, state["jobs"])
235         guard_memory(run, state["jobs"])
236         reconcile(run, state["jobs"])
237     if not state or not any(v["status"] == "running" for v in state["jobs"].values()):
238         initial = preflight(run)
239     else:
240         initial = read(run/"processed/warm_start.json")["febl_baseline_soc"]
241     precision = execution_settings(run)
242     protocol = {"workers": WORKERS, "base_solver_seconds": PROTOCOL["solver_seconds"], "
243 rescue_seconds": RESCUE_SECONDS,
244               "gap": precision.get("solver_gap", PROTOCOL["solver_gap"]), "trigger": "
245 all_day_candidates_failed", "model_changed": False,
246               "interpretation": "supplementary_compute_not_retroactive_original_budget_success"}
247     protocol_path = run/"raw/full_run_execution.json"
248     if precision:
249         protocol["precision_revision_sha256"] = precision["execution_revision"]
250         protocol_path = run/f"raw/full_run_execution_{precision['execution_revision'][:12]}.json"
251     if protocol_path.exists():
252         if read(protocol_path)["execution"] != protocol:
253             raise ValueError("队列执行设置改变，须显式记录新执行版本，禁止静默混用")
254     else:
255         atomic_json(protocol_path, {"execution": protocol, "frozen_at": now(), "authorization": "用户
256 明确授权Mac自主全量计算与验收"})
257     if not state:
258         state = {"started": now(), "jobs": {name: {"status": "pending"} for name in job_names(
259 initial)}}

```

```

253     for name in main_baselines(initial):
254         state["jobs"][name]["requires_main"] = True
255     state.update(status="running", supervisor_pid=os.getpid(), execution=protocol)
256     state["resource_policy"] = {"queue_worker_rss_limit_kib": MAX_WORKER_RSS_KIB,
257                                "memory_action": "stop_largest_owned_worker_preserve_committed_days"
258                                ,
259                                "unreported_process_exit_attempts": 3}
260     children = {}
261     while True:
262         for pid, child in list(children.items()):
263             if child.poll() is not None:
264                 del children[pid]
265             reconcile(run, state["jobs"])
266             guard_memory(run, state["jobs"])
267             reconcile(run, state["jobs"])
268             active = sum(v["status"] == "running" for v in state["jobs"].values())
269             for name in eligible(state["jobs"])[max(0, WORKERS-active)]:
270                 attempt = uuid.uuid4().hex
271                 log = f"raw/full_run_logs/{name}_{attempt}.log"
272                 receipt = f"raw/full_run_receipts/{name}_{attempt}.json"
273                 (run/log).parent.mkdir(parents=True, exist_ok=True)
274                 job = state["jobs"][name]
275                 job.update(status="running", attempt=attempt, log=log, receipt=receipt, started=now())
276                 command = [sys.executable, str(Path(__file__).resolve()), "--run-dir", str(run),
277                            "--job", name, "--attempt", attempt, "--receipt", receipt]
278                 with (run/log).open("w", encoding="utf-8") as stream:
279                     child = subprocess.Popen(command, cwd=ROOT, stdout=stream, stderr=subprocess.STDOUT,
280                                             env={**os.environ, "OPENBLAS_NUM_THREADS": "1", "
281                                             OMP_NUM_THREADS": "1", "MPLBACKEND": "Agg"},
282                                             start_new_session=True)
283                 job["pid"] = child.pid
284                 children[child.pid] = child
285                 # 每个启动立刻提交PID, 重启时可识别尚在运行的工作进程。
286                 atomic_json(state_path, state)
287                 print(f"{now()} 启动 {name} PID={child.pid}", flush=True)
288                 active = any(v["status"] == "running" for v in state["jobs"].values())
289                 state["updated"] = now()
290                 if not active and not eligible(state["jobs"]):
291                     state["status"] = "complete" if all(v["status"] == "complete" for v in state["jobs"].
292 values()) else "needs_repair"
293                 monitor(run, state)
294                 atomic_json(state_path, state)
295                 if not active and not eligible(state["jobs"]):
296                     break
297                 time.sleep(10)
298             from final_report import generate, verify_code
299             verify_code(run)
300             generate(run)
301             return 0 if state["status"] == "complete" else 2
302
303 def main():
304     parser = argparse.ArgumentParser(description=__doc__)
305     parser.add_argument("--run-dir", type=Path, required=True)
306     parser.add_argument("--job")
307     parser.add_argument("--attempt")
308     parser.add_argument("--receipt")

```



```

307     args = parser.parse_args()
308     run = args.run_dir.resolve()
309     if args.job:
310         if not args.attempt or not args.receipt:
311             parser.error("worker必须提供唯一attempt和receipt")
312         return worker(run, args.job, args.attempt, args.receipt)
313     with _exclusive_run_lock(run/"raw/full_run.lock"):
314         return supervise(run)
315
316
317 if __name__ == "__main__":
318     raise SystemExit(main())

```

src/q2/incumbent.py

```

1  """仅为原MILP寻找更好的共享参数初始解；不提供全局最优性声明。"""
2
3  import numpy as np
4  from scipy.optimize import minimize
5  import time
6
7  from policy import CAP, E_MIN, E_MAX, ETA_C, ETA_D, Policy
8
9
10 def value_gradient(vector, net, prices, weights, initial_soc, reserve=None):
11     """对原文逐步min/max映射作分段导数；最终策略另由独立回放及LP下界验收。"""
12     t = len(prices)
13     segments = (len(vector)//t-1)//2
14     grid = vector[:t]
15     cp, rp = vector[t:].reshape(2, t, segments)
16     reserve = np.zeros(t) if reserve is None else np.asarray(reserve, dtype=float)
17     if reserve.shape != (t,):
18         raise ValueError("动态SOC安全裕度长度不符")
19     state = np.full(len(net), initial_soc, dtype=float)
20     value = float(prices@grid)
21     derivatives = []
22     for i in range(t):
23         x = net[:, i]-grid[i]
24         band = np.minimum((segments*(state-E_MIN)/(E_MAX-E_MIN)).astype(int), segments-1)
25         band = np.maximum(band, 0)
26         c_terms = np.array([cp[i, band], np.maximum(-x, 0), np.maximum((E_MAX-state)/ETA_C, 0)])
27         reserve_active = state > E_MIN+reserve[i]
28         r_terms = np.array([rp[i, band], np.maximum(x, 0),
29                             np.maximum(ETA_D*(state-E_MIN-reserve[i]), 0)])
30         c_index, r_index = c_terms.argmin(0), r_terms.argmin(0)
31         charge, discharge = c_terms.min(0), r_terms.min(0)
32         shortage = x-discharge > 0
33         value += float(5*prices[i]*(np.maximum(x-discharge, 0)@weights))
34         cg = (x < 0)&(c_index == 1)
35         cc = (x < 0)&(c_index == 0)
36         ce = -((x < 0)&(c_index == 2)).astype(float)/ETA_C
37         rg = -((x > 0)&(r_index == 1)).astype(float)
38         rr = (x > 0)&(r_index == 0)
39         re = ((x > 0)&(r_index == 2)&reserve_active).astype(float)*ETA_D
40         derivatives.append((cg, cc, ce, rg, rr, re, shortage, band))
41         state += ETA_C*charge-discharge/ETA_D

```

```

42 gg = np.zeros(t); gc, gr = np.zeros((2, t, segments)); adjoint = np.zeros(len(net))
43 for i in range(t-1, -1, -1):
44     cg, cc, ce, rg, rr, re, shortage, band = derivatives[i]
45     emergency_price = 5*prices[i]*weights*shortage
46     gg[i] = prices[i]+np.sum(emergency_price*(-1-rg)+adjoint*(ETA_C*cg-rg/ETA_D))
47     np.add.at(gc[i], band, adjoint*ETA_C*cc)
48     np.add.at(gr[i], band, -emergency_price*rr-adjoint*rr/ETA_D)
49     adjoint = -emergency_price*re+adjoint*(1+ETA_C*ce-re/ETA_D)
50 return value, np.r_[gg, gc.ravel(), gr.ravel()]
51
52
53 def refine(policy, net, prices, weights, initial_soc, *, target_cost=None, seconds=90):
54     initial = np.r_[policy.grid, policy.charge_cap.ravel(), policy.discharge_cap.ravel()]
55     best = [value_gradient(initial, net, prices, weights, initial_soc, policy.reserve)[0], initial.
56             copy()]
57     started = time.perf_counter()
58     evaluations = 0
59     class Finished(Exception):
60         pass
61     def objective(vector):
62         nonlocal evaluations
63         evaluations += 1
64         value, gradient = value_gradient(vector, net, prices, weights, initial_soc, policy.reserve)
65         if value < best[0]:
66             best[:] = [value, vector.copy()]
67         if target_cost is not None and best[0] <= target_cost:
68             raise Finished("feasible_upper_bound_reached_requested_gap")
69         if time.perf_counter()-started >= seconds:
70             raise Finished("incumbent_search_time_budget")
71         return value, gradient
72     bounds = [(0, float(v)) for v in np.maximum(net.max(0), 0)+CAP]+[(0, CAP)]*(2*policy.charge_cap.
73     size)
74     try:
75         solved = minimize(objective, initial, jac=True, method="L-BFGS-B", bounds=bounds,
76                           options={"maxiter": 1000, "maxls": 30, "ftol": 1e-10})
77         iterations, termination = int(solved.nit), str(solved.message)
78     except Finished as error:
79         iterations, termination = None, str(error)
80     grid = best[1][:len(prices)]
81     caps = best[1][len(prices):].reshape((2,)+policy.charge_cap.shape)
82     return Policy(grid, *caps, policy.reserve), {"method": "piecewise_gradient_incumbent_only", "
83         iterations": iterations,
84         "evaluations": evaluations, "seconds": time.perf_counter()-started,
85         "termination": termination, "candidate_cost": best[0],
86         "global_optimality_claim": False}

```

src/q2/linear_policy.py

```

1 """原反馈策略的物理边界MILP：自由单段充电用单调投影的等值下界证书。"""
2
3 import time
4
5 import numpy as np
6 from scipy.optimize import Bounds, LinearConstraint, linprog
7 from scipy.optimize._highspy import _core as highs_core
8 from scipy.sparse import coo_matrix

```

```

9
10 from policy import CAP, E_MIN, E_MAX, ETA_C, ETA_D, PHYSICAL_TOL, Policy, replay
11
12
13 def build_linear_policy(net, prices, weights, initial_soc, *, fixed_policy=None, reserve=None):
14     net, prices, weights = [np.asarray(v, dtype=float) for v in (net, prices, weights)]
15     if net.ndim != 2 or prices.shape != (net.shape[1],) or weights.shape != (net.shape[0],):
16         raise ValueError("场景/电价/概率维度不符")
17     if not all(np.isfinite(v).all() for v in (net, prices, weights)) or (prices <= 0).any() or (
18         weights <= 0).any():
19         raise ValueError("输入必须有限且电价/概率为正")
20     if not np.isclose(weights.sum(), 1, atol=1e-12) or not E_MIN <= initial_soc <= E_MAX:
21         raise ValueError("概率或初态不符")
22     s, t = net.shape
23     if reserve is None and fixed_policy is not None:
24         reserve = fixed_policy.reserve
25     reserve = np.zeros(t) if reserve is None else np.asarray(reserve, dtype=float)
26     if reserve.shape != (t,) or not np.isfinite(reserve).all() or (reserve < 0).any() or (reserve >
27         E_MAX-E_MIN).any():
28         raise ValueError("动态SOC安全裕度长度或数值不符")
29     if fixed_policy is not None and not np.array_equal(fixed_policy.reserve, reserve):
30         raise ValueError("固定策略与线性MILP动态SOC安全裕度不一致")
31     g, cp, rp = [np.arange(k*t, (k+1)*t) for k in range(3)]
32     blocks = [np.arange(3*t+k*s*t, 3*t+(k+1)*s*t).reshape(s, t) for k in range(13)]
33     c, r, q, w, e, z, reserve_active, *selectors = blocks
34     lr, lc = selectors[:3], selectors[3:]
35     size = 3*t+13*s*t
36     objective, lo, hi, integer = np.zeros(size), np.zeros(size), np.full(size, np.inf), np.zeros(
37         size)
38     grid_upper = np.maximum(net.max(0), 0)+CAP
39     objective[g] = prices; objective[q] = 5*weights[:, None]*prices
40     hi[g], hi[cp], hi[rp] = grid_upper, CAP, CAP
41     hi[c], hi[r], hi[q], hi[w] = CAP, CAP, np.maximum(net, 0), np.maximum(grid_upper-net, 0)
42     lo[e], hi[e] = E_MIN, E_MAX
43     for block in (z, reserve_active, *lr, *lc):
44         hi[block] = 1; integer[block] = 1
45     hi[z[net < 0]] = 0
46     free_charge = fixed_policy is None
47     if free_charge:
48         # z=sum(lr), 且lr为二元变量、z在[0,1]; z的整数性已由等式隐含。
49         integer[z] = 0
50         for block in lc:
51             hi[block] = 0; integer[block] = 0
52     if fixed_policy is None:
53         # 单段反馈下提高充电上限使逐步SOC不减、紧急购电不减; 见等价性文档的支配性证明。
54         # 仅优化时消去被支配的选择; 固定策略回放必须保留调用者指定的上限。
55         lo[cp] = CAP
56     else:
57         if fixed_policy.charge_cap.shape != (t,) or fixed_policy.grid.shape != (t,):
58             raise ValueError("线性编码只支持单段策略")
59         for indices, values in ((g, fixed_policy.grid), (cp, fixed_policy.charge_cap), (rp,
60             fixed_policy.discharge_cap)):
61             if (values < lo[indices]-1e-8).any() or (values > hi[indices]+1e-8).any():
62                 raise ValueError("固定策略超出线性编码的物理上界")
63             values = np.clip(values, lo[indices], hi[indices])
64             lo[indices] = values; hi[indices] = values
65     rows, cols, coefficients, lower, upper = [], [], [], [], []

```

```

62 def add(terms, rhs, equal=False):
63     index = len(upper)
64     for column, value in terms:
65         rows.append(index); cols.append(int(column)); coefficients.append(value)
66     lower.append(rhs if equal else -np.inf); upper.append(rhs)
67 for j in range(s):
68     for i in range(t):
69         n = float(net[j, i]); ci, ri, qi, wi, ei, zi = [int(v[j, i]) for v in (c, r, q, w, e, z)
70 ]
71         previous = int(e[j, i-1]) if i else None
72         threshold = E_MIN+float(reserve[i])
73         positive_available_max = ETA_D*(E_MAX-threshold)
74         negative_available_max = ETA_D*float(reserve[i])
75         reserve_binary = int(reserve_active[j, i])
76         if i == 0:
77             active = int(initial_soc >= threshold)
78             lo[reserve_binary] = active
79             hi[reserve_binary] = active
80             add([(g[i], 1), (ri, 1), (qi, 1), (ci, -1), (wi, -1)], n, True)
81             add([(ei, 1), (ci, -ETA_C), (ri, 1/ETA_D)]+[(previous, -1)] if i else [], 0 if i else
initial_soc, True)
82             add([(ci, 1), (cp[i], -1)], 0); add([(ri, 1), (rp[i], -1)], 0)
83             room_max = (E_MAX-E_MIN)/ETA_C if i else (E_MAX-initial_soc)/ETA_C
84             add([(ci, 1)]+[(previous, 1/ETA_C)] if i else [], E_MAX/ETA_C if i else room_max)
85             add([(ri, 1), (reserve_binary, -CAP)], 0)
86             if i:
87                 add([(previous, ETA_D), (reserve_binary, -positive_available_max)], ETA_D*threshold)
88                 add([(previous, -ETA_D), (reserve_binary, negative_available_max)], -ETA_D*E_MIN)
89                 add([(ri, 1), (previous, -ETA_D), (reserve_binary, negative_available_max)], -ETA_D*
E_MIN)
90             else:
91                 add([(ri, 1), (reserve_binary, negative_available_max)], ETA_D*(initial_soc-E_MIN))
92                 add([(ci, 1), (zi, CAP)], CAP); add([(ri, 1), (zi, -CAP)], 0)
93                 add([(qi, 1), (zi, -max(n, 0))], 0)
94                 surplus_max = max(float(grid_upper[i])-n, 0)
95                 add([(wi, 1), (zi, surplus_max)], surplus_max)
96                 add([(int(v[j, i]), 1) for v in lr]+[(zi, -1)], 0, True)
97                 if not free_charge:
98                     add([(int(v[j, i]), 1) for v in lc]+[(zi, 1)], 1, True)
99                 # 分支激活常数分别为: CAP、max(n,0)、可用电量上界, 以及CAP、Gmax-n、容量空间上界。
100                 add([(rp[i], 1), (ri, -1), (lr[0][j, i], CAP)], CAP)
101                 deficit_max = max(n, 0)
102                 add([(g[i], -1), (ri, -1), (lr[1][j, i], deficit_max)], deficit_max-n)
103                 add([(ri, -1), (lr[2][j, i], positive_available_max)]+
([(previous, ETA_D)] if i else []),
104                     positive_available_max+ETA_D*threshold if i
105                     else positive_available_max-ETA_D*(initial_soc-threshold))
106                 if not free_charge:
107                     add([(cp[i], 1), (ci, -1), (lc[0][j, i], CAP)], CAP)
108                     add([(g[i], 1), (ci, -1), (lc[1][j, i], surplus_max)], surplus_max+n)
109                     add([(ci, -1), (lc[2][j, i], room_max)]+[(previous, -1/ETA_C)] if i else []),
110                     room_max-E_MAX/ETA_C if i else 0)
111                 # 原整数反馈分支隐含的有效约束; 仅加强松弛, 不删除任何可行反馈策略。
112                 add([(qi, 1), (lr[0][j, i], -deficit_max), (lr[2][j, i], -deficit_max)], 0)
113                 if not free_charge:
114                     add([(wi, 1), (lc[0][j, i], -surplus_max), (lc[2][j, i], -surplus_max)], 0)
115                 add([(ei, 1), (lr[2][j, i], E_MAX-threshold)], E_MAX)

```

```

116         if not free_charge:
117             add([(ei, -1), (lc[2][j, i], E_MAX-E_MIN)], -E_MIN)
118             # 同一方向的响应总量受净缺口/剩余上界约束，不能将分量上界重复相加。
119             add([(ri, 1), (qi, 1), (zi, -deficit_max)], 0)
120             add([(ci, 1), (wi, 1), (zi, surplus_max)], surplus_max)
121             if i:
122                 # 当储能边界项为最小项，上一SOC必须在一时段功率可达的边界邻域。
123                 add([(previous, 1), (lr[2][j, i], E_MAX-threshold-CAP/ETA_D)], E_MAX)
124                 if not free_charge:
125                     add([(previous, -1), (lc[2][j, i], E_MAX-E_MIN-ETA_C*CAP)], -E_MIN)
126 matrix = coo_matrix((coefficients, (rows, cols)), shape=(len(upper), size)).tocsr()
127 return objective, Bounds(lo, hi), integer, LinearConstraint(matrix, lower, upper), {"g": g, "cp"
128 : cp, "rp": rp,
129                                     "responses": (c,
130 r, q, w, e), "z": z,
131                                     "reserve_active":
132 reserve_active, "lr": lr,
133                                     "lc": lc, "
134 free_charge": free_charge,
135                                     "reserve":
136 reserve}
137
138 def linear_seed(policy, net, initial_soc, indices, size):
139     vector = np.zeros(size)
140     for key, values in (("g", policy.grid), ("cp", policy.charge_cap), ("rp", policy.discharge_cap))
141 :
142         vector[indices[key]] = values
143     for j, path in enumerate(net):
144         response = replay(policy, path, initial_soc)
145         for columns, key in zip(indices["responses"], ("charge", "discharge", "emergency", "unused",
146 "soc")):
147             vector[columns[j]] = response[key]
148         previous = initial_soc
149         for i, n in enumerate(path):
150             x = n-policy.grid[i]; deficit = x >= 0
151             vector[indices["z"][j, i]] = int(deficit)
152             available = max(ETA_D*(previous-E_MIN-policy.reserve[i]), 0)
153             vector[indices["reserve_active"][j, i]] = int(previous >= E_MIN+policy.reserve[i])
154             rmin = np.argmin([policy.discharge_cap[i], max(x, 0), available])
155             cmin = np.argmin([policy.charge_cap[i], max(-x, 0), (E_MAX-previous)/ETA_C])
156             if deficit:
157                 vector[indices["lr"][rmin][j, i]] = 1
158             elif not indices["free_charge"]:
159                 vector[indices["lc"][cmin][j, i]] = 1
160             previous = response["soc"][i]
161     return vector
162
163 def polish_linear_seed(policy, net, initial_soc, objective, bounds, integer, constraints, indices,
164 *, seconds=30):
165     """固定当前响应分支求局部LP，仅改善可行上界；它的最优值从不作全局下界。"""
166     started = time.perf_counter()
167     vector = linear_seed(policy, net, initial_soc, indices, len(objective))
168     initial_cost = float(objective@vector)
169     equality = constraints.lb == constraints.ub
170     rounds = 0

```

```

165     for _ in range(1000):
166         remaining = seconds-(time.perf_counter()-started)
167         if remaining <= 0:
168             break
169         lo, hi = bounds.lb.copy(), bounds.ub.copy()
170         lo[integer == 1] = hi[integer == 1] = vector[integer == 1]
171         solved = linprog(objective, A_eq=constraints.A[equality], b_eq=constraints.ub[equality],
172                         A_ub=constraints.A[~equality], b_ub=constraints.ub[~equality], bounds=np.
column_stack([lo, hi]),
173                         method="highs", options={"time_limit": remaining, "
primal_feasibility_tolerance": 1e-8,
174                                         "dual_feasibility_tolerance": 1e-8})
175         if not solved.success:
176             break
177         candidate = Policy(*[np.maximum(solved.x[indices[key]], 0) for key in ("g", "cp", "rp")],
178                           indices["reserve"])
179         updated = linear_seed(candidate, net, initial_soc, indices, len(objective))
180         if float(objective@updated) >= float(objective@vector)-1e-5:
181             break
182         policy, vector = candidate, updated
183         rounds += 1
184     return vector, {"rounds": rounds, "seconds": time.perf_counter()-started, "initial_cost":
initial_cost,
185                   "candidate_cost": float(objective@vector), "role": "
local_region_feasible_upper_bound_only"}
186
187
188 def solve_linear_policy(net, prices, weights, initial_soc, *, gap=.01, time_limit=900,
189                       fixed_policy=None, seed_policy=None, log=False, reserve=None,
190                       threads=1):
191     started = time.perf_counter()
192     objective, bounds, integer, constraints, indices = build_linear_policy(
193         net, prices, weights, initial_soc, fixed_policy=fixed_policy, reserve=reserve)
194     refinement = None
195     if seed_policy is not None:
196         polish_started = time.perf_counter()
197         before_dominance = None
198         if fixed_policy is None:
199             seed_policy = Policy(seed_policy.grid, np.full(len(prices), CAP),
200                               seed_policy.discharge_cap, indices["reserve"])
201             # 先在原策略类改善初值，避免过早固定cp使局部分支LP停在较差初值。
202             seed_lower = bounds.lb.copy()
203             seed_lower[indices["cp"]] = 0
204             vector, before_dominance = polish_linear_seed(
205                 seed_policy, net, initial_soc, objective, Bounds(seed_lower, bounds.ub.copy()),
206                 integer, constraints, indices)
207             seed_policy = Policy(vector[indices["g"]], np.full(len(prices), CAP),
208                               vector[indices["rp"]], indices["reserve"])
209             vector, refinement = polish_linear_seed(
210                 seed_policy, net, initial_soc, objective, bounds, integer, constraints, indices,
211                 seconds=max(30-(time.perf_counter()-polish_started), 0))
212         if before_dominance is not None:
213             refinement = {"before_dominance": before_dominance, "after_dominance": refinement,
214                           "rounds": before_dominance["rounds"]+refinement["rounds"],
215                           "seconds": time.perf_counter()-polish_started,
216                           "initial_cost": before_dominance["initial_cost"], "candidate_cost": float(
objective@vector),

```

```

217         "role": "local_region_feasible_upper_bound_only"}
218 # 使用已锁定SciPy 1.16.3内置的HiGHS 1.8绑定, 以传入经过逐行检查的完整可行初值。
219 highs_core._Highs.resetGlobalScheduler(True)
220 solver = highs_core._Highs()
221 # HiGHS以可行上界U归一化gap, 本项目以全局下界L归一化; 换算保证(U-L)/L仍不超过gap。
222 internal_gap = gap/(1+gap)
223 for key, value in {"mip_rel_gap": internal_gap, "time_limit": max(time_limit-(time.perf_counter
224                        (-started), .01), "output_flag": log, "threads": int(threads),
225                        "mip_feasibility_tolerance": 1e-8, "primal_feasibility_tolerance": 1e-8}.
226 items():
227     if solver.setOptionValue(key, value) == highs_core.HighsStatus.kError:
228         raise ValueError(f"HiGHS参数不受当前锁定版本支持: {key}")
229 lp, matrix = highs_core.HighsLp(), constraints.A
230 lp.num_col_, lp.num_row_ = len(objective), matrix.shape[0]
231 lp.a_matrix_.num_col_, lp.a_matrix_.num_row_ = lp.num_col_, lp.num_row_
232 lp.a_matrix_.format_ = highs_core.MatrixFormat.kColwise
233 lp.col_cost_, lp.col_lower_, lp.col_upper_ = objective, bounds.lb, bounds.ub
234 lp.row_lower_, lp.row_upper_ = constraints.lb, constraints.ub
235 lp.a_matrix_.start_, lp.a_matrix_.index_, lp.a_matrix_.value_ = matrix.indptr, matrix.indices,
236 matrix.data
237 lp.integrality_ = [highs_core.HighsVarType(int(v)) for v in integer]
238 if solver.passModel(lp) == highs_core.HighsStatus.kError:
239     raise ValueError("HiGHS未能加载物理边界线性模型")
240 if seed_policy is not None:
241     activity = matrix@vector
242     violation = max(float(np.max(bounds.lb-vector)), float(np.max(vector-bounds.ub)),
243                    float(np.max(constraints.lb-activity)), float(np.max(activity-constraints.ub
244                        )))
245     if violation > PHYSICAL_TOL:
246         raise ValueError(f"完整可行初值未通过线性编码逐行核验: {violation}")
247     seed = highs_core.HighsSolution(); seed.col_value = vector; seed.value_valid = True
248     if solver.setSolution(seed) == highs_core.HighsStatus.kError:
249         raise ValueError("HiGHS拒绝已校验初值")
250 if solver.run() == highs_core.HighsStatus.kError:
251     raise ValueError("HiGHS求解器错误")
252 solved = solver.getInfo()
253 lower = float(solved.mip_dual_bound)
254 info = {"status": solver.modelStatusToString(solver.getModelStatus()), "solver": "
255         HiGHS_monotone_charge_bound_MILP" if fixed_policy is None else "HiGHS_physical_bound_linear_MILP
256         ",
257         "solver_version": solver.version(), "time_limit": time_limit,
258         "requested_gap": gap, "solve_seconds": time.perf_counter()-started, "reliable": False,
259         "continuous_variables": int((integer == 0).sum()), "binary_variables": int(integer.sum()
260             )
261         ),
262         "constraints": int(constraints.A.shape[0]), "nodes": int(solved.mip_node_count),
263         "dual_bound": lower if np.isfinite(lower) else None, "seed_verified": seed_policy is not
264         None,
265         "seed_refinement": refinement, "highs_primal_relative_gap_target": internal_gap,
266         "maximum_charge_cap_dominance_presolve": fixed_policy is None, "
267         response_branch_valid_cuts": True}
268 if solved.primal_solution_status != highs_core.kSolutionStatusFeasible:
269     return None, None, info
270 vector = np.asarray(solver.getSolution().col_value)
271 policy = Policy(*[np.maximum(vector[indices[key]], 0) for key in ("g", "cp", "rp")],
272                indices["reserve"])
273 responses = [replay(policy, row, initial_soc) for row in net]
274 raw = np.stack([vector[v] for v in indices["responses"]], axis=-1)

```

```

265 exact = np.array([np.array([row[key] for key in ("charge", "discharge", "emergency", "unused", "
soc"))]).T for row in responses])
266 raw_difference = float(np.max(np.abs(raw-exact)))
267 activity = constraints.A@vector
268 feasibility_error = max(float(np.max(bounds.lb-vector)), float(np.max(vector-bounds.ub)),
269 float(np.max(constraints.lb-activity)), float(np.max(activity-
constraints.ub)), 0.)
270 integrality_error = float(np.max(np.abs(vector[integer == 1]-np rint(vector[integer == 1])))
271 projection_error = max(float(np.max(raw[... , 4]-exact[... , 4])),
272 float(np.max(exact[... , 2]-raw[... , 2])), 0.) if fixed_policy is None
else None
273 value = float(prices@policy.grid+sum(5*p*(prices@row["emergency"]) for p, row in zip(weights,
responses)))
274 raw_objective = float(solved.objective_function_value)
275 cost_nonincrease = bool(value <= raw_objective+PHYSICAL_TOL)
276 certified_gap = max(value-lower, 0)/max(abs(lower), 1e-8) if np.isfinite(lower) else None
277 error = projection_error if fixed_policy is None else raw_difference
278 info.update(objective=value, optimizer_objective=raw_objective, mip_gap=certified_gap,
279 monotone_charge_bound=fixed_policy is None, raw_response_difference_kwh=
raw_difference,
280 mapping_error_kwh=raw_difference if fixed_policy is not None else None,
281 monotone_projection_error_kwh=projection_error, linear_feasibility_error=
feasibility_error,
282 integrality_error=integrality_error, mapped_policy_cost_not_above_raw_objective=
cost_nonincrease,
283 certificate_error_definition="SOC_dominance_and_emergency_nonincrease" if
fixed_policy is None else "exact_mapping",
284 reliable=bool(certified_gap is not None and error <= PHYSICAL_TOL
285 and feasibility_error <= PHYSICAL_TOL and integrality_error <= 1e-8
286 and lower <= value+PHYSICAL_TOL and certified_gap <= gap+1e-8 and
cost_nonincrease))
287 if error > PHYSICAL_TOL or feasibility_error > PHYSICAL_TOL or integrality_error > 1e-8 or not
cost_nonincrease:
288     raise ValueError("线性MILP可行性或原反馈投影证书不符：不得执行原始场景轨迹")
289 return policy, responses, info

```

src/q2/optimization.py

```

1 """最终方案2.7.6: SCIP原生指示约束，精确绑定场景响应而非自由recourse."""
2
3 import time
4
5 import numpy as np
6 from pyscipopt import Model, SCIP_PARAMSETTING, quicksum
7
8 from policy import (CAP, E_INITIAL, E_MIN, E_MAX, ETA_C, ETA_D,
9 PHYSICAL_TOL, Policy, replay)
10
11
12 def _size(model, transformed=False):
13     variables = model.getVars(transformed=transformed)
14     return {"continuous": sum(v.vtype() == "CONTINUOUS" for v in variables),
15           "binary": sum(v.vtype() == "BINARY" for v in variables),
16           "variables": len(variables), "constraints": model.getNConss()}
17
18

```



```

19 def build_model(net, prices, weights, initial_soc=E_INITIAL, *, fixed_policy=None, segments=1,
20                reserve=None):
21     net, prices, weights = map(lambda v: np.asarray(v, dtype=float), (net, prices, weights))
22     if net.ndim != 2 or prices.shape != (net.shape[1],) or weights.shape != (net.shape[0],):
23         raise ValueError("场景/电价/概率维度不符")
24     if not all(np.isfinite(v).all() for v in (net, prices, weights)) or (prices <= 0).any() or (
25         weights <= 0).any():
26         raise ValueError("模型输入必须有限, 电价及场景概率必须为正")
27     if not np.isclose(weights.sum(), 1, atol=1e-12) or not E_MIN <= initial_soc <= E_MAX:
28         raise ValueError("概率不守恒或初态越界")
29     scenarios, periods = net.shape
30     if reserve is None and fixed_policy is not None:
31         reserve = fixed_policy.reserve
32     reserve = np.zeros(periods) if reserve is None else np.asarray(reserve, dtype=float)
33     if reserve.shape != (periods,) or not np.isfinite(reserve).all() or (reserve < 0).any() or (
34         reserve > E_MAX-E_MIN).any():
35         raise ValueError("动态SOC安全裕度长度或数值不符")
36     if fixed_policy is not None and not np.array_equal(fixed_policy.reserve, reserve):
37         raise ValueError("固定策略与MILP动态SOC安全裕度不一致")
38     model = Model("q2_precommitted_feedback")
39     model.hideOutput()
40     grid_upper = np.maximum(net.max(axis=0), 0) + CAP
41     g = [model.addVar(name=f"g_{t}", ub=float(grid_upper[t])) for t in range(periods)]
42     if segments not in (1, 3):
43         raise ValueError("仅实现方案指定的单一/三段SOC策略")
44     cp = [[model.addVar(name=f"cp_{t}_{k}", ub=CAP) for k in range(segments)] for t in range(periods)]
45     rp = [[model.addVar(name=f"rp_{t}_{k}", ub=CAP) for k in range(segments)] for t in range(periods)]
46     if fixed_policy is not None:
47         if fixed_policy.grid.shape != (periods,) or fixed_policy.charge_cap.size != periods*segments:
48             raise ValueError("固定策略长度或维度不符")
49         for variables, values in ((g, fixed_policy.grid), (np.array(cp).ravel(), fixed_policy.charge_cap.ravel()),
50                                 (np.array(rp).ravel(), fixed_policy.discharge_cap.ravel())):
51             for variable, value in zip(variables, values):
52                 model.addCons(variable == float(value))
53     responses, branches = [], []
54     for s in range(scenarios):
55         row, selectors = [], []
56         previous = float(initial_soc)
57         for t in range(periods):
58             n = float(net[s, t])
59             c = model.addVar(name=f"c_{s}_{t}", ub=CAP)
60             r = model.addVar(name=f"r_{s}_{t}", ub=CAP)
61             q = model.addVar(name=f"q_{s}_{t}", ub=max(n, 0))
62             w = model.addVar(name=f"w_{s}_{t}", ub=max(float(grid_upper[t])-n, 0))
63             e = model.addVar(name=f"e_{s}_{t}", lb=E_MIN, ub=E_MAX)
64             z = model.addVar(name=f"z_{s}_{t}", vtype="B", ub=0 if n < 0 else 1)
65             threshold = E_MIN+float(reserve[t])
66             if t == 0:
67                 active = int(initial_soc >= threshold)
68                 reserve_active = model.addVar(name=f"reserve_active_{s}_{t}", vtype="B",
69                                                 lb=active, ub=active)
70             else:
71                 reserve_active = model.addVar(name=f"reserve_active_{s}_{t}", vtype="B")

```

```

70     lr = [model.addVar(vtype="B", name=f"lr_{s}_{t}_{k}") for k in range(3)]
71     lc = [model.addVar(vtype="B", name=f"lc_{s}_{t}_{k}") for k in range(3)]
72     x = n - g[t]
73     available = ETA_D*(previous-E_MIN-float(reserve[t]))
74     room = (E_MAX-previous)/ETA_C
75     if segments == 1:
76         active_cp, active_rp = cp[t][0], rp[t][0]
77     else:
78         active_cp = model.addVar(name=f"active_cp_{s}_{t}", ub=CAP)
79         active_rp = model.addVar(name=f"active_rp_{s}_{t}", ub=CAP)
80         bands = [model.addVar(vtype="B", name=f"band_{s}_{t}_{k}") for k in range(3)]
81         model.addCons(quicksum(bands) == 1)
82         for k, band in enumerate(bands):
83             lo, hi = E_MIN+k*(E_MAX-E_MIN)/3, E_MIN+(k+1)*(E_MAX-E_MIN)/3
84             # 闭区间给出半开区间策略的下界松弛；独立半开区间回放+gap证书决定能否接受。
85             if t == 0:
86                 initial_band = min(int(3*(initial_soc-E_MIN)/(E_MAX-E_MIN)), 2)
87                 model.addCons(band == int(k == initial_band))
88             else:
89                 model.addConsIndicator(lo-previous <= 0, binvar=band)
90                 model.addConsIndicator(previous-hi <= 0, binvar=band)
91                 for active, cap in ((active_cp, cp[t][k]), (active_rp, rp[t][k])):
92                     model.addConsIndicator(active-cap <= 0, binvar=band)
93                     model.addConsIndicator(cap-active <= 0, binvar=band)
94         model.addCons(e == previous + ETA_C*c-r/ETA_D)
95         model.addCons(g[t]+r+q == n+c+w)
96         model.addCons(c <= active_cp); model.addCons(c <= room)
97         model.addCons(r <= active_rp)
98         model.addCons(r <= CAP*reserve_active)
99         if t == 0:
100             model.addCons(r <= max(float(available), 0))
101         else:
102             model.addConsIndicator(-available <= 0, binvar=reserve_active)
103             model.addConsIndicator(available <= 0, binvar=reserve_active, activeone=False)
104             model.addConsIndicator(r-available <= 0, binvar=reserve_active)
105         model.addCons(c <= CAP*(1-z)); model.addCons(r <= CAP*z)
106         model.addCons(q <= max(n, 0)*z)
107         model.addCons(w <= max(float(grid_upper[t])-n, 0)*(1-z))
108         model.addConsIndicator(-x <= 0, binvar=z)
109         model.addConsIndicator(x <= 0, binvar=z, activeone=False)
110         model.addCons(quicksum(lr) == z)
111         model.addCons(quicksum(lc) == 1-z)
112         # 平衡+q/w非负给出对应分支的r<=x、c<=-x；仅需绑定一个最小项。
113         for k, bound in enumerate((active_rp, x, available)):
114             model.addConsIndicator(bound-r <= 0, binvar=lr[k])
115         for k, bound in enumerate((active_cp, -x, room)):
116             model.addConsIndicator(bound-c <= 0, binvar=lc[k])
117         row.append((c, r, q, w, e)); selectors.append((z, lr, lc, reserve_active))
118         previous = e
119     responses.append(row); branches.append(selectors)
120 plan_cost = quicksum(float(prices[t])*g[t] for t in range(periods))
121 scenario_costs = [quicksum(5*float(prices[t])*responses[s][t][2] for t in range(periods)) for s
122 in range(scenarios)]
123 objective = plan_cost + quicksum(float(weights[s])*scenario_costs[s] for s in range(scenarios))
124 model.setObjective(objective, "minimize")
125 return model, {"g": g, "cp": cp if segments == 3 else [v[0] for v in cp],
                "rp": rp if segments == 3 else [v[0] for v in rp], "response": responses, "

```

```

126         "branches": branches,
127         "objective": objective, "scenario_costs": scenario_costs, "reserve": reserve}
128
129 def _seed(model, variables, net, initial_soc, policy):
130     """可行预承诺策略作为求解器初始解，最终仍须满足预注册gap；不是替代求解。"""
131     solution = model.createPartialSol()
132     for key, values in (("g", policy.grid), ("cp", policy.charge_cap), ("rp", policy.discharge_cap)):
133         :
134         for variable, value in zip(np.array(variables[key], dtype=object).ravel(), values.ravel()):
135             model.setSolVal(solution, variable, float(value))
136     for s, path in enumerate(net):
137         response = replay(policy, path, initial_soc)
138         previous = initial_soc
139         for t in range(len(path)):
140             for variable, key in zip(variables["response"][s][t], ("charge", "discharge", "emergency", "unused", "soc")):
141                 model.setSolVal(solution, variable, float(response[key][t]))
142                 z, lr, lc, reserve_active = variables["branches"][s][t]
143                 x = path[t]-policy.grid[t]
144                 deficit = x >= 0
145                 model.setSolVal(solution, z, int(deficit))
146                 band = min(max(int(3*(previous-E_MIN)/(E_MAX-E_MIN)), 0), 2)
147                 cp = policy.charge_cap[t] if policy.charge_cap.ndim == 1 else policy.charge_cap[t, band]
148                 rp = policy.discharge_cap[t] if policy.discharge_cap.ndim == 1 else policy.discharge_cap[t, band]
149                 available = max(ETA_D*(previous-E_MIN-policy.reserve[t]), 0)
150                 rmin = int(np.argmin([rp, max(x, 0), available]))
151                 cmin = int(np.argmin([cp, max(-x, 0), (E_MAX-previous)/ETA_C]))
152                 for k in range(3):
153                     model.setSolVal(solution, lr[k], int(deficit and k == rmin))
154                     model.setSolVal(solution, lc[k], int(not deficit and k == cmin))
155                     model.setSolVal(solution, reserve_active, int(previous >= E_MIN+policy.reserve[t]))
156                     previous = response["soc"][t]
157     model.addSol(solution)
158
159 def solve_policy(net, prices, weights, initial_soc=E_INITIAL, *, gap=0.01, time_limit=120,
160                 fixed_policy=None, seed_policy=None, log_path=None, cvar=None, segments=1,
161                 known_lower_bound=None, reserve=None, threads=1):
162     started = time.perf_counter()
163     model, variables = build_model(net, prices, weights, initial_soc, fixed_policy=fixed_policy,
164                                   segments=segments, reserve=reserve)
165     model.setPresolve(SCIP_PARAMSETTING.FAST)
166     model.setIntParam("presolving/maxrounds", 1)
167     model.setIntParam("misc/usesymmetry", 0)
168     model.setRealParam("limits/gap", gap)
169     model.setRealParam("limits/time", time_limit)
170     model.setIntParam("parallel/maxnthreads", int(threads))
171     model.setRealParam("numerics/feastol", 1e-8)
172     model.setIntParam("randomization/randomseedshift", 0)
173     if log_path:
174         model.setLogfile(str(log_path))
175     if cvar is not None:
176         alpha, epsilon, reference = cvar
177         if not 0 < alpha < 1 or epsilon < 0 or reference < 0:
178             raise ValueError("CVaR扩展参数不合法")

```

```

179     eta = model.addVar(name="cvar_eta", lb=0)
180     xi = [model.addVar(name=f"cvar_xi_{s}") for s in range(len(weights))]
181     for excess, scenario_cost in zip(xi, variables["scenario_costs"]):
182         model.addCons(excess >= scenario_cost-eta)
183         model.addCons(variables["objective"] <= (1+epsilon)*reference)
184         model.setObjective(eta + quicksum(float(p)*v for p, v in zip(weights, xi))/(1-alpha))
185 before = _size(model)
186 if seed_policy is not None:
187     _seed(model, variables, np.asarray(net), initial_soc, seed_policy)
188 model.presolve()
189 after = _size(model, transformed=True)
190 model.optimize()
191 status = str(model.getStatus())
192 info = {"status": status, "solve_seconds": model.getSolvingTime(),
193        "wall_seconds": time.perf_counter()-started, "before_presolve": before,
194        "after_presolve": after, "solver": "SCIP", "solver_version": ".".join(map(str, (
195            model.getMajorVersion(), model.getMinorVersion(), model.getTechVersion()))),
196        "mip_gap": float(model.getGap()) if model.getNSols() else None,
197        "dual_bound": float(model.getDualbound()), "solutions": model.getNSols(),
198        "requested_gap": gap, "time_limit": time_limit}
199 if not model.getNSols():
200     model.freeProb()
201     return None, None, {**info, "reliable": False}
202 sol = model.getBestSol()
203 def values(key):
204     array = np.array(variables[key], dtype=object)
205     return np.maximum([model.getSolVal(sol, v) for v in array.ravel()], 0).reshape(array.shape)
206 policy = Policy(values("g"), values("cp"), values("rp"), variables["reserve"])
207 responses = [replay(policy, path, initial_soc) for path in net]
208 raw = np.array([[model.getSolVal(sol, v) for v in entry] for entry in row] for row in variables
209                ["response"]])
210 exact = np.array([np.array([r[k] for k in ("charge", "discharge", "emergency", "unused", "soc")
211                               ]).T for r in responses])
212 mapping_error = float(np.max(np.abs(exact-raw)))
213 objective = float(np.dot(prices, policy.grid) + sum(float(p)*5*np.dot(prices, r["emergency"])
214               for p, r in zip(weights, responses)))
215 certificate_gap = max(objective-info["dual_bound"], 0)/max(abs(info["dual_bound"]), 1e-8)
216 info.update(objective=objective, optimizer_objective=float(model.getObjVal()),
217            mapping_error_kwh=mapping_error,
218            reliable=bool(info["mip_gap"] <= gap+1e-8 and mapping_error <= PHYSICAL_TOL))
219 if known_lower_bound is not None and cvar is None:
220     lower = max(info["dual_bound"], known_lower_bound)
221     certificate_gap = max(objective-lower, 0)/max(abs(lower), 1e-8)
222     info.update(scip_mip_gap=info["mip_gap"], mip_gap=certificate_gap,
223                dual_bound=lower, external_lp_lower_bound=known_lower_bound,
224                reliable=bool(certificate_gap <= gap+1e-8 and mapping_error <= PHYSICAL_TOL))
225 if segments == 3:
226     info.update(half_open_policy_certified_gap=certificate_gap,
227                soc_boundary_formulation="
228 closed_interval_lower_bound_and_exact_half_open_replay_certificate",
229                reliable=bool(certificate_gap <= gap+1e-8))
230 if mapping_error > PHYSICAL_TOL and segments == 1:
231     model.freeProb()
232     raise ValueError(f"MILP与单步因果映射不等价: {mapping_error}kWh")
233 model.freeProb()
234 return policy, responses, info

```

```

232
233 def certify_policy(net, prices, weights, initial_soc, policy, responses, bound, gap,
234                   *, collect_model_statistics=False):
235     """可行策略+全局LP下界直接给出gap证书；默认不再构建无需求解的SCIP模型。"""
236     started = time.perf_counter()
237     segments = 1 if policy.charge_cap.ndim == 1 else policy.charge_cap.shape[1]
238     model = None
239     before = after = None
240     solver_version = None
241     if collect_model_statistics:
242         model, _ = build_model(net, prices, weights, initial_soc, segments=segments,
243                               reserve=policy.reserve)
244         before = _size(model)
245         model.setPresolve(SCIP_PARAMSETTING.FAST)
246         model.setRealParam("numerics/feastol", 1e-8)
247         model.setIntParam("presolving/maxrounds", 1)
248         model.setIntParam("misc/usesymmetry", 0)
249         model.setRealParam("limits/time", 120)
250         model.presolve()
251         after = _size(model, transformed=True)
252         solver_version = ".".join(map(str, (
253             model.getMajorVersion(), model.getMinorVersion(), model.getTechVersion()))))
254     actual = [replay(policy, path, initial_soc) for path in net]
255     cost = float(np.dot(prices, policy.grid)+sum(5*p*np.dot(prices, r["emergency"]) for p, r in zip(
256         weights, actual)))
257     lower = bound["lower_bound"]
258     certified_gap = max(cost-lower, 0)/max(abs(lower), 1e-8)
259     info = {"status": "certified_by_feasible_policy_and_global_lp_lower_bound",
260           "solver": "LP_bound_and_exact_policy_replay", "solver_version": solver_version,
261           "before_presolve": before, "after_presolve": after,
262           "mip_gap": certified_gap, "requested_gap": gap, "objective": cost, "dual_bound": lower,
263           "solve_seconds": time.perf_counter()-started+bound["seconds"],
264           "mapping_error_kwh": 0.0, "mapping_evidence": "
265           all_responses_generated_by_same_exact_frozen_policy",
266           "reliable": bool(certified_gap <= gap and lower <= cost+PHYSICAL_TOL),
267           "certificate": bound, "branch_and_bound_required": False,
268           "certificate_source": "LP_BOUND", "upper_bound": cost, "lower_bound": lower,
269           "scip_build_skipped": not collect_model_statistics}
270     info["presolve_scope"] = ("SCIP_FAST_one_round_statistics_only" if collect_model_statistics
271                             else "skipped_after_LP_certificate")
272     if model is not None:
273         model.freeProb()
274     return policy, actual, info

```

src/q2/policy.py

```

1  """最终方案2.7.3/2.9.3：冻结策略、单步因果响应与独立物理验收。"""
2
3  from dataclasses import dataclass
4
5  import numpy as np
6
7  DT = 1 / 6
8  E_MIN, E_MAX, E_INITIAL = 1200.0, 10800.0, 6000.0
9  ETA_C = ETA_D = 0.9
10 CAP = 5000 * DT

```

```

11 PHYSICAL_TOL = 1e-5 # kWh, 仅数值验收容差, 不参与经济策略或紧急事件定义。
12
13
14 @dataclass(frozen=True)
15 class Policy:
16     grid: np.ndarray
17     charge_cap: np.ndarray
18     discharge_cap: np.ndarray
19     reserve: np.ndarray | None = None
20
21     def __post_init__(self):
22         for name in ("grid", "charge_cap", "discharge_cap"):
23             value = np.array(getattr(self, name), dtype=float, copy=True)
24             if not np.isfinite(value).all() or (value < 0).any():
25                 raise ValueError("策略必须有限、非负")
26             value.setflags(write=False)
27             object.__setattr__(self, name, value)
28         if self.grid.ndim != 1 or not len(self.grid):
29             raise ValueError("购电策略应为非空一维数组")
30         reserve = np.zeros(len(self.grid)) if self.reserve is None else np.array(self.reserve, dtype
= float, copy=True)
31         if reserve.shape != (len(self.grid),) or not np.isfinite(reserve).all():
32             raise ValueError("动态SOC安全裕度必须是与策略等长的有限一维数组")
33         if (reserve < 0).any() or (reserve > E_MAX-E_MIN+PHYSICAL_TOL).any():
34             raise ValueError("动态SOC安全裕度超出可用容量")
35         reserve.setflags(write=False)
36         object.__setattr__(self, "reserve", reserve)
37         shapes = ((len(self.grid),), (len(self.grid), 3))
38         if self.charge_cap.shape not in shapes or self.discharge_cap.shape != self.charge_cap.shape:
39             raise ValueError("响应上限应为每时段一组或三组SOC分段参数")
40         if max(self.charge_cap.max(), self.discharge_cap.max()) > CAP + PHYSICAL_TOL:
41             raise ValueError("响应上限超过物理功率")
42
43     def first(self, periods=144):
44         return Policy(self.grid[:periods], self.charge_cap[:periods], self.discharge_cap[:periods],
45             self.reserve[:periods])
46
47
48 def respond(net, previous_soc, grid, charge_cap, discharge_cap, reserve=0.0):
49     """只接收本时段观测; 没有未来数组、场景库或优化器接口。单位全部kWh。"""
50     if not np.isfinite([net, previous_soc, grid, charge_cap, discharge_cap, reserve]).all():
51         raise ValueError("非有限单步输入")
52     if not E_MIN - PHYSICAL_TOL <= previous_soc <= E_MAX + PHYSICAL_TOL:
53         raise ValueError("上一时段SOC越界")
54     if min(grid, charge_cap, discharge_cap) < 0 or max(charge_cap, discharge_cap) > CAP +
55         PHYSICAL_TOL:
56         raise ValueError("单步策略边界不合法")
57     if not 0 <= reserve <= E_MAX-E_MIN+PHYSICAL_TOL:
58         raise ValueError("动态SOC安全裕度不合法")
59     deficit = net - grid
60     charge = min(charge_cap, max(-deficit, 0), max((E_MAX - previous_soc) / ETA_C, 0))
61     discharge = min(discharge_cap, max(deficit, 0),
62         max(ETA_D * (previous_soc - E_MIN - reserve), 0))
63     emergency = max(deficit - discharge, 0)
64     unused = max(-deficit - charge, 0)
65     soc = previous_soc + ETA_C * charge - discharge / ETA_D
66     return charge, discharge, emergency, unused, soc

```

```

66
67
68 def replay(policy, net, initial_soc):
69     """顺序聚合回放：第t步只向respond传入net[t]及已实现状态。"""
70     net = np.asarray(net, dtype=float)
71     if net.shape != policy.grid.shape or not np.isfinite(net).all():
72         raise ValueError("真实净负荷与冻结策略长度不一致")
73     state = float(initial_soc)
74     output = np.empty((len(net), 5))
75     for t, current_net in enumerate(net):
76         if policy.charge_cap.ndim == 2:
77             segment = min(int(3 * max(state - E_MIN, 0) / (E_MAX - E_MIN)), 2)
78             cp, rp = policy.charge_cap[t, segment], policy.discharge_cap[t, segment]
79         else:
80             cp, rp = policy.charge_cap[t], policy.discharge_cap[t]
81         output[t] = respond(current_net, state, policy.grid[t], cp, rp, policy.reserve[t])
82         state = output[t, 4]
83     result = dict(zip(("charge", "discharge", "emergency", "unused", "soc"), output.T))
84     validate_replay(policy.grid, net, initial_soc, result, reserve=policy.reserve)
85     return result
86
87
88 def validate_replay(grid, net, initial_soc, result, *, rigid=False, reserve=None):
89     c, r, q, w, e = [np.asarray(result[k]) for k in ("charge", "discharge", "emergency", "unused", "soc")]
90     np.testing.assert_allclose(grid + r + q, net + c + w, atol=PHYSICAL_TOL, rtol=0)
91     np.testing.assert_allclose(e, np.r_[initial_soc, e[:-1]] + ETA_C*c - r/ETA_D,
92                                atol=PHYSICAL_TOL, rtol=0)
93     reserve = np.zeros(len(e)) if reserve is None else np.asarray(reserve, dtype=float)
94     if reserve.shape != e.shape or not np.isfinite(reserve).all() or (reserve < 0).any():
95         raise ValueError("回放动态SOC安全裕度不合法")
96     previous = np.r_[initial_soc, e[:-1]]
97     available = ETA_D*np.maximum(previous-E_MIN-reserve, 0)
98     if not rigid and np.any(r > available+PHYSICAL_TOL):
99         raise ValueError("回放放电超过动态SOC安全裕度后的可用电量")
100     if min(c.min(), r.min(), q.min(), w.min()) < -PHYSICAL_TOL or e.min() < E_MIN-PHYSICAL_TOL or e.max() > E_MAX+PHYSICAL_TOL:
101         raise ValueError("回放能量/SOC边界失败")
102     if max(c.max(), r.max()) > CAP + PHYSICAL_TOL or np.any((c > PHYSICAL_TOL) & (r > PHYSICAL_TOL)):
103         :
104         raise ValueError("回放功率/充放电互斥失败")
105     if not rigid and np.any((q > PHYSICAL_TOL) & (c > PHYSICAL_TOL)):
106         raise ValueError("负荷优先失败：紧急购电同时充电")
107
108 def cost(policy, response, prices):
109     return float(np.dot(prices, policy.grid + 5 * response["emergency"]))

```

src/q2/protocol.py

```

1 """正式数据计算前固定的工程协议：题设参数仅定义在policy.py。"""
2
3 from pathlib import Path
4 import hashlib
5 import json
6

```



```

7 ROOT = Path(__file__).resolve().parents[2]
8 PLAN = ROOT / "docs/2/final/第二问_最终建模思路_论文精简终版.md"
9 TASK = ROOT / "docs/2/final/Q2_Codex_最终模型修改任务书.md"
10 ACCELERATION_TASK = ROOT / "docs/2/final/Codex_问题二求解加速任务书.md"
11 # 以下为方案要求预注册但没有指定数值的数值计算设置，不是题设事实。
12 PROTOCOL = {
13     "version": "q2-tail-reserve-final-20260912",
14     "history_boundary": "source_date_before_origin",
15     "boundary_authority": "current_user_requests_exclusive_execution_of_new_plan_section_2.2.3_j_less_than_d",
16     "forecast_horizons": [1, 2, 3], "warm_horizon": 2,
17     "shadow_start": "2025-01-15", "evaluation_start": "2025-02-01",
18     "minimum_blocks": 14, "history_windows_days": [28, 56, 84, None],
19     "scenario_max": 40, "scenario_candidates": [10, 20, 40],
20     "tail_stratification": True, "tail_fraction": 0.20,
21     "dynamic_soc_reserve": True, "reserve_quantile": 0.80,
22     "evaluation_initial_soc_kwh": 6000.0,
23     "january_storage_operation": "inactive",
24     "solver_gap": 0.01, "solver_seconds": 120.0,
25     "refine_seconds": 5.0, "solver_threads": 1,
26     "stability_objective_relative": 0.01, "stability_grid_relative": 0.02,
27     "tail_quantile_relative": 0.05, "full_pool_emergency_relative": 0.05,
28     "selection_relative_tolerance": 0.01,
29     "lightgbm_candidates": [
30         {"num_leaves": 7, "n_estimators": 100, "learning_rate": 0.05,
31          "max_depth": -1, "min_child_samples": 20, "reg_lambda": 1.0},
32         {"num_leaves": 15, "n_estimators": 100, "learning_rate": 0.05,
33          "max_depth": -1, "min_child_samples": 20, "reg_lambda": 1.0}],
34     "lightgbm_training_window": "expanding", "lightgbm_freeze_metric": "January_h0_load_MAE",
35     "dhr_harmonics": [3, 6], "dhr_ar_orders": [0, 1], "dhr_ma_orders": [0, 1],
36     "dhr_history_days": 28, "dhr_order_rule": "BIC_then_AIC_then_lower_complexity",
37     "dhr_max_iterations": 100, "dhr_difference_order": 0,
38     "selection_history": "common_complete_origins_in_selected_window",
39     "soc_segments": [0, 1/3, 2/3, 1],
40     "specified_dates": ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"],
41     "high_variation_dates_count": 3,
42     "numerical_settings_origin": "
    implementation_preregistered_before_formal_backtest_not_problem_facts",
43 }
44
45
46 def signature():
47     payload = (json.dumps(PROTOCOL, sort_keys=True).encode("utf-8")
48               + PLAN.read_bytes() + TASK.read_bytes() + ACCELERATION_TASK.read_bytes())
49     return hashlib.sha256(payload).hexdigest()

```

src/q2/relaxation.py

```

1 """主MILP的自由场景响应LP下界；只能加速/证明gap，不能冒充实际策略。"""
2
3 import time
4
5 import numpy as np
6 from scipy.optimize import linprog
7 from scipy.sparse import coo_matrix
8

```



```

9 from policy import CAP, E_MIN, E_MAX, ETA_C, ETA_D, Policy, replay
10 from scenarios import weighted_quantile
11 from incumbent import refine
12
13
14 def _candidate_costs(net, prices, weights, grid, charges, discharges, initial_soc, reserve):
15     """同时评价多个合法参数候选：每一步仍仅依赖当前场景值和上一状态。"""
16     count, scenarios = len(charges), len(net)
17     state = np.full((count, scenarios), initial_soc, dtype=float)
18     costs = np.full(count, float(prices@grid))
19     for i in range(len(prices)):
20         x = net[:, i]-grid[i]
21         charge = np.minimum(np.minimum(charges[:, i, None], np.maximum(-x, 0)), np.maximum((E_MAX-
22             state)/ETA_C, 0))
23         discharge = np.minimum(np.minimum(discharges[:, i, None], np.maximum(x, 0)),
24             np.maximum(ETA_D*(state-E_MIN-reserve[i]), 0))
25         state += ETA_C*charge-discharge/ETA_D
26         costs += 5*prices[i]*(np.maximum(x-discharge, 0)*weights)
27     return costs
28
29 def recourse_bound(net, prices, weights, initial_soc, *, target_gap=.01, reserve=None,
30     refine_seconds=5.):
31     started = time.perf_counter()
32     s, t = net.shape
33     reserve = np.zeros(t) if reserve is None else np.asarray(reserve, dtype=float)
34     if reserve.shape != (t,) or not np.isfinite(refine_seconds) or refine_seconds < 0:
35         raise ValueError("动态SOC安全裕度长度不符")
36     size = t+5*s*t
37     grid = np.arange(t)
38     charge, discharge, emergency, unused, soc = [np.arange(t+k*s*t, t+(k+1)*s*t).reshape(s, t) for k
39         in range(5)]
40     objective = np.zeros(size)
41     objective[grid] = prices
42     objective[emergency] = 5*weights[:, None]*prices
43     bounds = np.zeros((size, 2)); bounds[:, 1] = np.inf
44     bounds[grid, 1] = np.maximum(net.max(0), 0)+CAP
45     bounds[charge, 1] = bounds[discharge, 1] = CAP
46     bounds[soc, 0], bounds[soc, 1] = E_MIN, E_MAX
47     rows, cols, coefficients, rhs = [], [], [], []
48     def equation(terms, value):
49         row = len(rhs)
50         for column, coefficient in terms:
51             rows.append(row); cols.append(int(column)); coefficients.append(coefficient)
52             rhs.append(float(value))
53     for j in range(s):
54         for i in range(t):
55             equation([(grid[i], 1), (discharge[j, i], 1), (emergency[j, i], 1),
56                 (charge[j, i], -1), (unused[j, i], -1)], net[j, i])
57             terms = [(soc[j, i], 1), (charge[j, i], -ETA_C), (discharge[j, i], 1/ETA_D)]
58             if i:
59                 terms.append((soc[j, i-1], -1))
60             equation(terms, initial_soc if i == 0 else 0)
61     matrix = coo_matrix((coefficients, (rows, cols)), shape=(len(rhs), size)).tocsc()
62     solved = linprog(objective, A_eq=matrix, b_eq=rhs, bounds=bounds, method="highs")
63     if not solved.success:
64         raise RuntimeError(f"主MILP的LP下界求解失败: {solved.message}")

```

```

64 # 这些水平只是MILP可行初始解搜索网格，不是新的风险目标或预测超参数。
65 caps = []
66 for index in (charge, discharge):
67     actions = np.maximum(solved.x[index], 0)
68     caps.append([np.average(actions, weights=weights, axis=0),
69                 *[weighted_quantile(actions, weights, q) for q in (0., .25, .5, .75, 1.)],
70                 np.full(t, CAP)])
71 pairs = [(c, r) for c in caps[0] for r in caps[1]]
72 charges, discharges = np.asarray([c for c, _ in pairs]), np.asarray([r for _, r in pairs])
73 planned = np.maximum(solved.x[grid], 0)
74 costs = _candidate_costs(net, prices, weights, planned, charges, discharges, initial_soc,
75                          reserve)
76 best = int(np.argmin(costs))
77 policy = Policy(planned, charges[best], discharges[best], reserve)
78 refinement = None
79 if (costs[best]-solved.fun)/max(abs(solved.fun), 1e-8) > target_gap and refine_seconds > 0:
80     policy, refinement = refine(policy, net, prices, weights, initial_soc,
81                                target_cost=solved.fun+target_gap*max(abs(solved.fun), 1e-8),
82                                seconds=refine_seconds)
83 responses = [replay(policy, path, initial_soc) for path in net]
84 value = float(prices@policy.grid + sum(5*p*(prices@r["emergency"]) for p, r in zip(weights,
85 responses)))
86 if refinement is None:
87     np.testing.assert_allclose(value, costs[best], atol=1e-5, rtol=1e-12)
88 else:
89     np.testing.assert_allclose(value, refinement["candidate_cost"], atol=1e-5, rtol=1e-12)
90 # 由可行域包含关系: LP下界 <= 原参数化策略最优值 <= 经独立回放的可行策略费用。
91 lower = float(solved.fun)
92 gap = max(value-lower, 0)/max(abs(lower), 1e-8)
93 return policy, responses, {"lower_bound": lower, "incumbent_cost": value, "certified_gap": gap,
94                           "seconds": time.perf_counter()-started, "variables": size,
95                           "constraints": len(rhs), "refinement": refinement,
96                           "refine_seconds_budget": refine_seconds,
97                           "role": "lower_bound_only_never_free_recourse_execution"}

```

src/q2/report.py

```

1 """2.1.14 - 2.1.15: 由实算诊断形成候选依据与可见报告，明确尚未建模的部分。"""
2
3 import html
4
5 import pandas as pd
6
7 from data import SERIES, write_csv
8 from figures import LABELS
9
10
11 def candidate_models(diagnostics):
12     """仅接受 initial_history 诊断，避免用全年证据回填首次决策。"""
13     lags, spectra = diagnostics["lag_correlations"], diagnostics["spectral_targets"]
14     if set(lags.scope) != {"initial_history"} or set(spectra.scope) != {"initial_history"}:
15         raise ValueError("候选集必须来自首次预测前的可见历史诊断")
16     rows = []
17     for variable in SERIES[:2]:
18         def r(lag, form="raw"):
19             return float(lags.loc[(lags.series == variable) & (lags.lag == lag) & (lags.form == form)

```

```

), "pearson_r"].iloc[0])
20     daily = spectra.loc[(spectra.series == variable) & (spectra.form == "raw") & (spectra.
target_period_days == 1)].iloc[0]
21     weekly = spectra.loc[(spectra.series == variable) & (spectra.form == "daily_profile_residual
") & (spectra.target_period_days == 7)].iloc[0]
22     evidence = f"r144={r(144):.6f}; r1008={r(1008):.6f}; 去平均日曲线r1008={r(1008, '
daily_profile_residual'):.6f}; 日频局部峰={daily.local_peak}; 残差周频局部峰={weekly.local_peak}"
23     # 无人为强弱阈值: 周频局部峰还须与去日曲线后周相关强于日相关相互印证。
24     # 未通过只表示证据不足, 绝不同于证明没有周效应。
25     multi = bool(daily.local_peak and weekly.local_peak and r(1008, "daily_profile_residual") >
r(144, "daily_profile_residual"))
26     for model, status, reason in (
27         ("SARIMA", "基础候选, 须历史内部验证", "日季节项以144为候选; 固定参数是否足够不能由单个月份定论, 阶数
不能由图形直接定死。"),
28         ("DHR_AR", "优先比较的多季节候选" if multi else "条件保留, 不预设周效应",
29         "日/周频峰与相关共同支持比较多季节动态谐波回归—AR误差; 仍非统计显著证明。" if multi else
30         "尚未形成独立周周期的完整证据; 若后续可见历史出现周证据, 再比较多季节模型。"),
31         ("Prophet", "条件对照", "仅在历史趋势与季节分解足够平滑时作为对照; 当前未开展分解拟合及内层回测。"),
32         ("RF_or_GBDT", "暂不引入", "须先有传统模型无法解释非线性残差的回测证据, 不能仅凭原始曲线判定。"),
33     ):
34         rows.append({"series": variable, "model": model, "status": status, "reason": reason,
35         "evidence": evidence, "evidence_scope": "initial_history", "fitted": False,
36         "selection_rule": "所有估计/参数选择仅用各预测日可见历史; 不能读取full_year诊断"})
37     return pd.DataFrame(rows)
38
39
40 def result_text(frame, diagnostics, candidates, folds):
41     full = diagnostics["full_year"]
42     lags = full["lag_correlations"]
43     def value(variable, lag):
44         return float(lags.loc[(lags.series == variable) & (lags.form == "raw") & (lags.lag == lag),
"pearson_r"].iloc[0])
45     lines = [ "# C题第二问: 数据预处理与时间序列结构诊断结果", "",
46             "## 实施口径与信息边界", "",
47             "本结果完整覆盖 dierwen1.md 的2.1阶段。用户确认训练数据严格 timestamp < 当日0:00, 等于0:00的上一
日末段也不使用。",
48             "预测阶段全程使用kW; 仅在事后调度指标中按1/6小时转为kWh。全文统计不改变原始功率。",
49             "全年诊断仅作事后描述; 首次候选模型依据单独使用2025-02-01之前的4463个可见点。",
50             "算法设置为实现口径而非新增题设: 相关滞后至2016、Burg PACF、原长FFT周期图、ADF默认上限/BIC滞滞后、
KPSS自动带宽; 检验同时给出1%、5%、10%临界值。",
51             "工作日分组仅为周一至周五, 不推定法定节假日; 夜间缺少天文/天气标注, 只统计零光伏而不自定夜间范围。", ""
52     ,
53     "## 原始输入核验", "",
54     f"52560个连续10分钟观测, 365个源日期, 每日144段; 首点{frame.timestamp.iloc[0]}, 末点{frame.
timestamp.iloc[-1]}。",
55     f"负荷与光伏无缺失、无非有限值、无负值。光伏零值{int((frame.pv_kw == 0).sum())}个, 全部保留; 净负
荷为负的时段{int((frame.net_load_kw < 0).sum())}个, 属于光伏超过负荷的合法状态。",
56     "局部转折点全部导出相邻时刻、前日、前周与既往同星期同槽位范围, 仅按偏离邻点幅度排列人工复核顺序, 没有设异
常截断阈值, 没有删除或平滑。", "",
57     "## 周期证据与结果合理性", "",
58     "| 变量 | 1日滞后r | 7日滞后r | 14日滞后r |", "|---|---|---|---|"]
59     for variable in SERIES[:2]:
60         lines.append(f"| {LABELS[variable]} | {value(variable,144):.6f} | {value(variable,1008):.6f}
| {value(variable,2016):.6f} |")
61     lines += [ "", "负荷的周滞后相关高于日滞后相关, 应结合星期平均曲线及周频峰考虑多季节结构。光伏的高7日相关本身不证
明星期效应, 须检查去除平均日曲线后的周频局部峰。",
"周期图同时保留原始与日曲线残差结果, 目标1/7周期/日通常不恰落在离散FFT格点上, CSV公开实际频率及分辨

```

```

率；局部峰不是显著性检验。", "" ]
62 weekday_means = frame.groupby("weekday").load_kw.mean()
63 lines += [ "### 与通常工作日/周末预期不一致的观测", "",
64           "按附件真实2025年日期映射，低负荷主要出现在星期五、星期六；星期日多数时段接近其他高负荷日。"
65           "原始日期、星期和末段归属已独立核对，不能为了符合通常的周末预期而平移日期或修改功率。"
66           "这一现象仅是附件数据的统计特征，数据未提供成因，不能据此断言居民作息或数据生成原因。", "",
67           "| 星期 | 全日平均负荷 / kW |", "|---|---:|" ]
68 lines += [ f"| 星期{'一二三四五六日'[int(day)-1]} | {value:.3f} |" for day, value in weekday_means.
69             items()]
70 lines += [ "", "因此星期一至星期日应分别分析；周一至周五/周六日的分组图仅为描述对照，不能作为已证实的行为分类。",
71           "" ]
72 for variable in SERIES[:2]:
73     stability = full["monthly_stability"].loc[lambda x: x.series == variable]
74     lines += [ f"{LABELS[variable]}月均功率范围{stability.mean_kw.min():.3f} - {stability.mean_kw.
75               max():.3f} kW; "
76               f"月内1日相关范围{stability.daily_lag_r.min():.6f} - {stability.daily_lag_r.max():.6f}
77               , "
78               f"7日相关范围{stability.weekly_lag_r.min():.6f} - {stability.weekly_lag_r.max():.6f}
79               。"
80               "这些差异须用于后续检验参数和季节性是否随时间变化，不能仅据全年高相关固定参数。", "" ]
81 lines += [ "## 平稳性检验的解释", "",
82           "ADF原假设为存在单位根，KPSS原假设为水平(c)或趋势(ct)平稳；原始、日差分、周差分、日后周差分分别检验。
83           ",
84           "KPSS表外p值只给上/下界，警告原文保留；ADF输出0是数值近似，不能解释为精确零概率。拒绝单位根不证明消除
85           了季节性或异方差；同时查看各变换的月均值和标准差。日后周差分包含重复季节因子，可能过度差分，只用于诊断，未据此规定
86           最终模型。", "",
87           "| 变量 | 变换 | ADF(c) p | KPSS(c) p口径 |", "|---|---|---:|---|" ]
88 tests = full["stationarity_tests"]
89 for variable in SERIES[:2]:
90     for form in ("raw", "diff144", "diff1008", "diff144_then1008"):
91         part = tests.loc[(tests.series == variable) & (tests.form == form) & (tests.regression
92 == "c")].set_index("test")
93         lines.append(f"| {LABELS[variable]} | {form} | {part.loc['ADF', 'p_value']:.5g} | {part.
94 loc['KPSS', 'p_value_type']} {part.loc['KPSS', 'p_value']:.5g} |")
95 lines += [ "", "## 候选模型与回测规则", "",
96           "下表仅据首次回测前的可见历史。全年图形不能回流为2月1日的模型选择信息。", "",
97           "| 变量 | 候选 | 状态 |", "|---|---|---|" ]
98 lines += [ f"| {LABELS[r.series]} | {r.model} | {r.status} |" for r in candidates.itertuples(
99 index=False)]
100 first, last = folds.iloc[0], folds.iloc[-1]
101 lines += [ "", f"回测起止为{first.origin.date()}至{last.origin.date()}，共{len(folds)}日，每日0:00一次
102 预测未来144段。",
103           f"首日历史{first.history_points}点，最后可见时刻为{first.history_last_timestamp}；末日预测最
104 后一个时段的右端点为{last.target_last}。",
105           "负荷与光伏分别调用新的预测器，输入仅为该变量历史与未来日历；两者相减得到净负荷。不得随机划分，拟合、缩
106 放、选阶与调参均限历史内部。",
107           "三类预测均实现MAE/RMSE；总RMSE从全部时段平方误差计算。事后调度指标实现紧急购电电量、时段数/比例、连
108 续事件数、发生天数与总费用。",
109           "紧急电量来自事后实际供给缺口×1/6小时，不能拿净负荷预测误差直接代替。费用=计划电量×正常电价+紧急电量×
110 当期电价×5；未用电的计划电量仍计费。",
111           "2-12月费用只可称评价期总费用；没有1月策略就不能伪称全年费用。接口只有收到365个完整日的调度明细后才标
112 记完整自然年。", "",
113           "## 本阶段完成状态与后续边界", "",
114           "2.1.1-2.1.15所要求的数据重构、诊断图表/统计、候选依据、回测规则与评价接口已实现。",
115           "原方案未给出预测模型阶数/拟合策略、调度优化和场景风险模型，因此真实预测误差、紧急购电与费用目前未计
116 算；没有填零、模拟或以代理指标冒充真实结果。",
117           "下一建模阶段应在上述候选中用历史内层时间回测确定模型，并提供调度策略后运行已实现的预测与调度评价接口。

```

```

100     " ", " ",
101         "## 数值方法依据", " ",
102         "- [Burg PACF](https://www.statsmodels.org/v0.14.6/generated/statsmodels.tsa.stattools.pacf_burg.html)",
103         "- [ADF](https://www.statsmodels.org/v0.14.6/generated/statsmodels.tsa.stattools.adfuller.html) 与 [KPSS](https://www.statsmodels.org/v0.14.6/generated/statsmodels.tsa.stattools.kpss.html)",
104         "- [SciPy周期图及功率谱单位](https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.periodogram.html)"]
105     return "\n".join(lines) + "\n"
106
107 def create_report(frame, diagnostics, candidates, folds, figures, output):
108     text = result_text(frame, diagnostics, candidates, folds)
109     (output / "results.md").write_text(text, encoding="utf-8")
110     write_csv(output / "model_candidates.csv", candidates)
111     write_csv(output / "rolling_folds.csv", folds)
112     status = pd.DataFrame([{"metric": m, "status": "not_computed_model_not_specified", "value": None
113         ,
114         "reason": "本方案止于预测前诊断与评价规则，未指定拟合预测器/调度模型"}
115         for m in ("load_mae_kw", "load_rmse_kw", "pv_mae_kw", "pv_rmse_kw", "
116         net_load_mae_kw",
117         "net_load_rmse_kw", "emergency_kwh", "emergency_frequency", "
118         total_cost_yuan")])
119     write_csv(output / "evaluation_status.csv", status)
120     cards = []
121     for figure in figures:
122         links = " • ".join(f'<a href="{html.escape(s)}">{html.escape(s)}</a>' for s in figure["source_csv"])
123         cards.append(f'<figure id="{figure["name"]}"><h2>{html.escape(figure["name"])}</h2>'
124             f'{html.escape(figure["caption"])}</figcaption>'
126             f'<a href="figures/{figure["name"]}.svg">SVG</a> • {links}</figcaption></figure>')
127     tables = []
128     for scope, results in diagnostics.items():
129         for name in ("summary", "lag_correlations", "spectral_targets", "monthly_stability", "stationarity_tests"):
130             table = results[name].to_html(index=False, border=0, float_format=lambda v: f"{v:.6g}", na_rep="无定义/不适用")
131             tables.append(f'<details><summary>{scope} / {name}</summary><a href="{scope}_{name}.csv">完整CSV</a><div class="table">{table}</div></details>')
132     downloads = "\n".join(f'<li><a href="{p.name}">{p.name}</a></li>' for p in sorted(output.glob("*.csv")))
133     navigation = " • ".join(f'<a href="#{f["name"]}">{f["name"]}</a>' for f in figures)
134     page = f'<!doctype html><html lang="zh-CN"><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1">
135     <title>C题第二问 • 时间序列结构诊断</title><style>
136     body{{font:16px/1.65 system-ui,sans-serif;color:#172b3a;background:#f5f7fa;margin:0;overflow-wrap:
137     anywhere}}main{{max-width:1280px;margin:auto;padding:28px}}
138     h1{{font-size:30px}}h2{{font-size:21px}}a{{color:#07588c}}a:focus,summary:focus{{outline:3px solid #d55e00}}
139     nav{{padding:12px 0}}figure,details,.intro{{background:white;border:1px solid #d8e1e8;border-radius:8px;padding:18px;margin:20px 0}}
140     figure img{{width:100%;height:auto}}figcaption{{font-size:14px}}summary{{cursor:pointer;font-weight:650}}.table{{overflow-x:auto}}

```

```

137 table{{border-collapse:collapse;font-size:13px;white-space:nowrap}}td,th{{padding:7px;border-bottom
    :1px solid #d8e1e8;text-align:right}}
138 pre{{white-space:pre-wrap;overflow-wrap:anywhere;font:inherit}}.badge{{color:#704600;background:#
    fff4d7;padding:10px}}
139 </style><main><h1>C题第二问 · 时间序列结构诊断</h1>
140 <p>365天 · 52,560点 · 10分钟 · 区间右端点 · 原始功率kW</p>
141 <p class="badge">本方案为2.1预测前阶段。真实预测与购电策略尚未指定，评价数值未计算；全年图表为事后诊断。</p>
142 <nav>{navigation} · <a href="#data">CSV与指标</a></nav>
143 <section class="intro"><p><strong>附件2核验通过：</strong>365天、每日144段，无缺失、负值、重复或时间断点；原始
    负荷与光伏逐值保留。</p>
144 <p>负荷的周周期证据强于日滞后；光伏以日周期为主。训练严格排除等于0:00的观测。下方11张图均提供PNG、SVG与对应数据。</p>
145 <details><summary>展开完整结果、实施口径与候选依据</summary><pre>{html.escape(text)}</pre></details>
146 <a href="results.md">下载结果说明</a> · <a href="../../docs/2/implementation_acceptance.md">逐句验收
    报告</a></section>
147 {'.join(cards)}<section id="data"><h2>实算统计与下载</h2>{''.join(tables)}
148 <details><summary>334日严格滚动切分</summary><div class="table">{folds.to_html(index=False,border=0)
    }</div></details>
149 <details><summary>候选模型证据</summary><div class="table">{candidates.to_html(index=False,border=0)
    }</div></details>
150 <details><summary>后续评价数值状态</summary><div class="table">{status.assign(status="尚未指定实际预测/调
    度模型").fillna("未计算").rename(columns={"metric":"指标","status":"状态","value":"值","reason":"说
    明"})</div></details>
151 <ul>{downloads}</ul></section></main></html>'''
152 (output / "report.html").write_text(page, encoding="utf-8")

```

src/q2/rolling.py

```

1 """最终方案2.9/2.10: 先冻结首日策略，再逐步真实回放；求解受限显式记录。"""
2
3 import numpy as np
4 import pandas as pd
5 import hashlib
6 import json
7 import os
8 import time
9 import copy
10 from contextlib import contextmanager
11 from pathlib import Path
12
13 from comparators import deterministic_policy, rigid_schedule
14 from checkpoint import atomic_json, read_checkpoint, recover_checkpoint, save_checkpoint
15 from data import calendar_for_day, sha256, write_csv, write_json
16 from optimization import certify_policy, solve_policy
17 from relaxation import recourse_bound
18 from policy import DT, E_INITIAL, Policy, replay, validate_replay
19 from protocol import PROTOCOL
20 from scenarios import (construct, dynamic_reserve, eligible_origins, joint_blocks,
21                       quantile_comparison, weighted_quantile)
22 from shadows import choose_day, combined
23
24
25 class ComputationalLimit(RuntimeError):
26     def __init__(self, details):
27         super().__init__("未获得满足预注册MIP gap及映射验收的策略；不得当正式解发布")
28         self.details = details
29

```



```

30
31 def _solve(net, prices, weights, soc, *, reserve=None, solver_seconds=None,
32           solver_gap=PROTOCOL["solver_gap"], refine_seconds=PROTOCOL["refine_seconds"],
33           certificate_fast_path=True, solver_threads=PROTOCOL["solver_threads"],
34           solve_cache=None, cache_key=None, **options):
35     if not np.isfinite(solver_gap) or not 0 < solver_gap < 1:
36         raise ValueError("求解精度门槛无效")
37     reserve = np.zeros(np.asarray(net).shape[1]) if reserve is None else np.asarray(reserve)
38     cache_entry = None
39     signature = hashlib.sha256(
40         np.ascontiguousarray(np.asarray(net)).view(np.uint8).tobytes()
41         + np.ascontiguousarray(np.asarray(prices)).view(np.uint8).tobytes()
42         + np.ascontiguousarray(np.asarray(weights)).view(np.uint8).tobytes()
43         + np.ascontiguousarray(reserve).view(np.uint8).tobytes()
44         + np.asarray([soc], dtype=float).view(np.uint8).tobytes()).hexdigest()
45     if solve_cache is not None and cache_key is not None:
46         cache_entry = solve_cache.setdefault(("solve", cache_key), {"signature": signature})
47         if cache_entry["signature"] != signature:
48             raise ValueError("同日rescue缓存与当前场景、概率、SOC或reserve不一致")
49     if len(net) == 1 and cache_entry is None:
50         policy, responses, info = deterministic_policy(net[0], prices, soc, reserve=reserve)
51         if policy is not None:
52             return policy, responses, info
53     reused_bound = bool(cache_entry and "bound" in cache_entry)
54     if reused_bound:
55         bound_policy, bound_responses, bound = (
56             cache_entry["bound_policy"], cache_entry["bound_responses"], cache_entry["bound"])
57         seed = cache_entry.get("seed_policy", bound_policy)
58     else:
59         mean = np.average(net, weights=weights, axis=0)
60         seed, _, _ = deterministic_policy(mean, prices, soc, reserve=reserve)
61         bound_policy, bound_responses, bound = recourse_bound(
62             net, prices, weights, soc, target_gap=solver_gap, reserve=reserve,
63             refine_seconds=refine_seconds)
64         if cache_entry is not None:
65             cache_entry.update(bound_policy=bound_policy, bound_responses=bound_responses,
66                               bound=bound, seed_policy=bound_policy)
67     if bound["certified_gap"] <= solver_gap:
68         policy, responses, info = certify_policy(
69             net, prices, weights, soc, bound_policy, bound_responses, bound, solver_gap,
70             collect_model_statistics=not certificate_fast_path)
71         info.update(reused_recourse_bound=reused_bound, refine_seconds=refine_seconds)
72         return policy, responses, info
73     if seed is None or sum(p*(prices@(seed.grid+5*replay(seed, path, soc)["emergency"])) for p, path
74                          in zip(weights, net)) > bound["incumbent_cost"]:
75         seed = bound_policy
76     supplemental = solver_seconds is not None and solver_seconds > PROTOCOL["solver_seconds"]
77     if not options and (supplemental or solver_gap > PROTOCOL["solver_gap"]):
78         from linear_policy import solve_linear_policy
79         policy, responses, info = solve_linear_policy(net, prices, weights, soc, gap=solver_gap,
80                                                       time_limit=solver_seconds or PROTOCOL["
81 solver_seconds"],
82                                                       seed_policy=seed, reserve=reserve,
83                                                       threads=solver_threads)
84         info["execution_role"] = ("
85 supplemental_equivalent_physical_bound_MILP_after_original_budget_failure" if supplemental
86 else "

```

```

original_budget_equivalent_MILP_under_authorized_precision_revision")
84 else:
85     policy, responses, info = solve_policy(
86         net, prices, weights, soc, gap=solver_gap,
87         time_limit=solver_seconds or PROTOCOL["solver_seconds"], seed_policy=seed,
88         known_lower_bound=bound["lower_bound"], reserve=reserve,
89         threads=solver_threads, **options)
90     if cache_entry is not None and policy is not None:
91         cache_entry["seed_policy"] = policy
92     info.update(reused_recourse_bound=reused_bound,
93               reused_incumbent=bool(reused_bound and seed is not bound_policy),
94               pre_solver_seconds=0. if reused_bound else bound.get("seconds", 0.),
95               refine_seconds=refine_seconds, solver_threads=solver_threads)
96     return policy, responses, info
97
98
99 def _pool_cost(policy, net, prices, soc):
100     return float(np.mean([5*np.dot(prices[:144], replay(policy.first(), path[:144], soc)["emergency"
101         ]) for path in net]))
102
103 def select_resolution(point, blocks, prices, soc, *, origins=None, all_blocks=None,
104                     all_origins=None, reserve=None, forced_count=None,
105                     solver_seconds=None, solver_gap=PROTOCOL["solver_gap"],
106                     refine_seconds=PROTOCOL["refine_seconds"],
107                     certificate_fast_path=True, solver_threads=PROTOCOL["solver_threads"],
108                     solve_cache=None, cache_prefix=None):
109     construction = {"origins": origins, "all_blocks": all_blocks, "all_origins": all_origins,
110                    "tail_fraction": PROTOCOL["tail_fraction"]}
111     def scenarios(count):
112         key = ("scenarios", cache_prefix, count)
113         if solve_cache is not None and key in solve_cache:
114             return solve_cache[key]
115         value = construct(point, blocks, prices, count, **construction)
116         if solve_cache is not None:
117             solve_cache[key] = value
118         return value
119     full_net, full_weights, _ = scenarios(None)
120     m = len(blocks)
121     counts = [m] if m <= 40 else [10, 20, 40]
122     if forced_count is not None and m > 40:
123         counts = [min(m, forced_count)]
124     results, audit = {}, {"M": m, "candidates": {}, "computational_limited": False}
125     for count in counts:
126         net, weights, scenario = scenarios(count)
127         policy, responses, info = _solve(net, prices, weights, soc, reserve=reserve,
128                                         solver_seconds=solver_seconds, solver_gap=solver_gap,
129                                         refine_seconds=refine_seconds,
130                                         certificate_fast_path=certificate_fast_path,
131                                         solver_threads=solver_threads, solve_cache=solve_cache,
132                                         cache_key=(cache_prefix, count))
133         item = {"scenarios": scenario, "solver": info}
134         if policy is not None:
135             item["full_pool_first_day_emergency_cost"] = _pool_cost(policy, full_net, prices, soc)
136             item["scenario_first_day_emergency_cost"] = float(sum(5*p*np.dot(prices[:144], r["
137 emergency"][:144]) for p, r in zip(weights, responses)))
138             item["tail_quantiles"] = quantile_comparison(full_net, net, weights)

```



```

138     audit["candidates"][str(count)] = item
139     if policy is not None and info["reliable"]:
140         results[count] = (policy, responses, weights, net)
141     if not results:
142         raise ComputationalLimit(audit)
143     selected = max(results)
144     if m > 40 and forced_count is None:
145         if 20 in results and 40 in results:
146             j20, j40 = [audit["candidates"][str(s)]["solver"]["objective"] for s in (20, 40)]
147             g20, g40 = results[20][0].grid[:144], results[40][0].grid[:144]
148             dj = abs(j40-j20)/max(abs(j40), 1e-8)
149             dg = float(np.abs(g40-g20).sum()/max(g40.sum(), 1e-8))
150             audit.update(delta_J_20_40=dj, delta_g_20_40=dg)
151             if dj <= PROTOCOL["stability_objective_relative"] and dg <= PROTOCOL["
stability_grid_relative"]:
152                 selected = 20
153         else:
154             audit["computational_limited"] = True
155         def tail_bad(count):
156             entry = audit["candidates"][str(count)]
157             quantile_bad = max(v["normalized_error"] for v in entry["tail_quantiles"].values()) >
PROTOCOL["tail_quantile_relative"]
158             reference = audit["candidates"].get("40", entry).get("full_pool_first_day_emergency_cost
", 0)
159             deterioration = (entry["full_pool_first_day_emergency_cost"]-reference)/max(reference, 1
e-8)
160             return quantile_bad or deterioration > PROTOCOL["full_pool_emergency_relative"]
161         if selected == 20 and tail_bad(20) and 40 in results:
162             selected = 40
163         if selected == 40 and tail_bad(40):
164             policy, responses, info = _solve(full_net, prices, full_weights, soc, reserve=reserve,
solver_seconds=solver_seconds, solver_gap=solver_gap,
165             refine_seconds=refine_seconds,
166             certificate_fast_path=certificate_fast_path,
167             solver_threads=solver_threads, solve_cache=solve_cache,
168             cache_key=(cache_prefix, m))
169             audit["candidates"][str(m)] = {"solver": info, "scenarios": {"M": m, "S": m, "reduced":
False}, "trigger": "tail_check_failed_at_40"}
170         if policy is not None and info["reliable"]:
171             results[m] = (policy, responses, full_weights, full_net)
172             selected = m
173         else:
174             audit["computational_limited"] = True
175             audit["tail_requirement_unresolved"] = True
176     policy, responses, weights, net = results[selected]
177     audit.update(S=selected, selected_solver=audit["candidates"][str(selected)]["solver"])
178     return policy, responses, weights, net, audit
179
180
181
182 def make_day(store, frozen, history, d, prices, soc, *, horizon="auto", minimum=14,
183             window="auto", forced_count=None, fixed_pair=None, kind="feedback",
184             solver_seconds=None, progress_path=None, solver_gap=PROTOCOL["solver_gap"],
185             refine_seconds=PROTOCOL["refine_seconds"], certificate_fast_path=True,
186             solver_threads=PROTOCOL["solver_threads"], solve_cache=None):
187     started = time.perf_counter()
188     solve_cache = {} if solve_cache is None else solve_cache
189     candidates = (2, 3, 1) if horizon == "auto" else (int(horizon),)

```

```

190 results, audits = {}, {}
191 for k in candidates:
192     # K1只能在K2/K3全部失败时被选中; 已有可靠长时域时不重复求K1。
193     if horizon == "auto" and k == 1 and results:
194         audits["skipped_K1"] = {"reason": "K2_or_K3_reliable_K1_cannot_change_selection"}
195         continue
196     if progress_path is not None:
197         atomic_json(progress_path, {"day_index": d, "date": str((pd.Timestamp("2025-01-01")+pd.
198             Timedelta(days=d)).date()),
199             "K": k, "solver_gap": solver_gap, "solver_seconds":
200 solver_seconds or PROTOCOL["solver_seconds"],
201             "started_at": time.time(), "phase": "solving"})
202     asset_key = ("day_assets", k, minimum, str(window), tuple(fixed_pair or ()))
203     if asset_key in solve_cache:
204         point, blocks, selection, origins, all_blocks, all_origins, reserve = solve_cache[
205 asset_key]
206         selection = copy.deepcopy(selection)
207         selection["day_assets_reused"] = True
208     else:
209         point, blocks, selection = choose_day(
210             store, frozen, history, d, k, prices, minimum=minimum,
211             window=window, fixed_pair=fixed_pair)
212         selection["common_scored_origins"] = [int(j) for j in selection["common_scored_origins"
213 ]]
214     if blocks is None:
215         origins = all_origins = None
216         all_blocks = None
217         reserve_day = np.zeros(144)
218         reserve_meta = {"quantile": PROTOCOL["reserve_quantile"], "historical_blocks": 0,
219             "max_reserve_kwh": 0., "mean_reserve_kwh": 0.}
220     else:
221         origins = np.asarray(selection["origin_indices"], dtype=int)
222         forecasts = combined(store, selection["pipeline"])
223         all_origins = eligible_origins(forecasts, d, k, minimum=minimum, window=None)
224         all_blocks = joint_blocks(history, forecasts, all_origins, k)
225         reserve_day, reserve_meta = dynamic_reserve(all_blocks, PROTOCOL["reserve_quantile"
226 ])
227         selection["all_origin_indices"] = all_origins.tolist()
228         reserve = np.r_[reserve_day, np.zeros(144*(k-1))]
229         selection["dynamic_soc_reserve"] = reserve_meta
230         selection["day_assets_reused"] = False
231         solve_cache[asset_key] = (point, blocks, copy.deepcopy(selection), origins,
232             all_blocks, all_origins, reserve)
233     price_horizon = np.tile(prices, k)
234     try:
235         if kind == "deterministic":
236             net = DT*(point[..., 0]-point[..., 1]).reshape(1, -1)
237             policy, responses, solver = _solve(net, price_horizon, np.array([1.]), soc,
238                 reserve=reserve, solver_seconds=solver_seconds,
239                 solver_gap=solver_gap, refine_seconds=
240 refine_seconds,
241                 certificate_fast_path=certificate_fast_path,
242                 solver_threads=solver_threads, solve_cache=
243 solve_cache,
244                 cache_key=(k, "deterministic"))
245         if policy is None or not solver["reliable"]:
246             raise ComputationalLimit(solver)

```

```

240         weights = np.array([1.]); audit = {"M": len(blocks), "S": 1, "selected_solver":
solver}
241     elif kind == "rigid":
242         if forced_count is None:
243             _, _, weights, net, audit = select_resolution(
244                 point, blocks, price_horizon, soc, origins=origins, all_blocks=all_blocks,
245                 all_origins=all_origins, reserve=reserve, solver_seconds=solver_seconds,
246                 solver_gap=solver_gap, refine_seconds=refine_seconds,
247                 certificate_fast_path=certificate_fast_path,
248                 solver_threads=solver_threads, solve_cache=solve_cache,
249                 cache_prefix=k)
250         else:
251             net, weights, scenarios = construct(
252                 point, blocks, price_horizon, forced_count, origins=origins,
253                 all_blocks=all_blocks, all_origins=all_origins,
254                 tail_fraction=PROTOCOL["tail_fraction"])
255             audit = {"M": len(blocks), "S": len(weights), "scenarios": scenarios}
256             grid, responses, solver = rigid_schedule(net, price_horizon, weights, soc)
257             policy = Policy(grid, responses[0]["charge"], responses[0]["discharge"])
258             audit["selected_solver"] = solver
259         else:
260             policy, responses, weights, net, audit = select_resolution(point, blocks,
price_horizon, soc,
261
262
263
264
265
266
267
268
269
270
271             origins=origins,
272             all_blocks=all_blocks,
273             all_origins=all_origins,
274             reserve=reserve,
275             forced_count=forced_count,
276             solver_seconds=
277             solver_gap=solver_gap,
278             refine_seconds=
279             certificate_fast_path=
280             solver_threads=
281             solve_cache=solve_cache,
282             cache_prefix=k)
283     except ComputationalLimit as error:
284         audits[str(k)] = {"selection": selection, "error": error.details}
285         continue
286     day_cost = float(prices@policy.grid[:144] + sum(5*p*(prices@r["emergency"][:144]) for p, r
in zip(weights, responses)))
287     audit.update(selection=selection, first_day_expected_cost=day_cost)
288     audits[str(k)] = audit
289     results[k] = (policy, weights, net, point)
290 if not results:
291     raise ComputationalLimit(audits)
292 selected = max(results)
293 if horizon == "auto" and 2 in results and 3 in results:
294     g2, g3 = results[2][0].grid[:144], results[3][0].grid[:144]
295     dg = float(np.abs(g2-g3).sum()/max(g3.sum(), 1e-8))
296     c2, c3 = audits["2"]["first_day_expected_cost"], audits["3"]["first_day_expected_cost"]
297     dc = abs(c2-c3)/max(abs(c3), 1e-8)
298     audits["stability_48_72"] = {"grid_relative": dg, "first_day_expected_cost_relative": dc,
"interpretation": "open_loop_first_day_stability_only"}

```

```

288         if dg <= PROTOCOL["stability_grid_relative"] and dc <= PROTOCOL["
stability_objective_relative"]:
289             selected = 2
290             audits["selected_K"] = selected
291             audits["computational_limited"] = any(str(k) in audits and "error" in audits[str(k)] for k in
candidates) or any(v.get("computational_limited", False) for v in audits.values() if isinstance(
v, dict))
292             audits["runtime_seconds"] = time.perf_counter()-started
293             return *results[selected], audits
294
295
296 def dispatch_frame(date, policy, net, soc, prices, *, rigid=False):
297     first = policy.first()
298     if rigid:
299         charge, discharge = first.charge_cap, first.discharge_cap
300         gap = net+charge-first.grid-discharge
301         response = {"charge": charge, "discharge": discharge, "emergency": np.maximum(gap, 0),
302                     "unused": np.maximum(-gap, 0), "soc": soc+np.cumsum(.9*charge-discharge/.9)}
303         validate_replay(first.grid, net, soc, response, rigid=True)
304     else:
305         response = replay(first, net, soc)
306         frame = calendar_for_day(date)
307         frame["day_of_year"] = pd.Timestamp(date).dayofyear
308         frame["grid"] = first.grid
309         frame["charge_cap"] = first.charge_cap
310         frame["discharge_cap"] = first.discharge_cap
311         frame["soc_reserve_kwh"] = first.reserve
312         frame["net_kwh"] = net
313         frame["initial_soc"] = soc
314         frame["price"] = prices
315         for key, value in response.items():
316             frame[key] = value
317         frame["planned_cost"] = prices*first.grid
318         frame["emergency_cost"] = 5*prices*response["emergency"]
319         return frame
320
321
322 def warm_start(actual, dates, prices, output):
323     """1月仅积累预测/误差历史，储能不主动调度，2月1日SOC固定为6000kWh。"""
324     if np.asarray(actual).shape[:2] != (365, 144) or len(dates) != 365 or len(prices) != 144:
325         raise ValueError("1月历史预热输入不完整")
326     write_csv(output/"warm_dispatch.csv", pd.DataFrame(columns=[
327         "date", "charge", "discharge", "soc"]))
328     write_json(output/"warm_start.json", {"initial_soc_jan1": E_INITIAL,
329                                           "january_charge_kwh": 0.,
330                                           "january_discharge_kwh": 0.,
331                                           "prediction_history_days": 31,
332                                           "febl_baseline_soc": E_INITIAL,
333                                           "interpretation": "
paper_final_fixed_initialization_january_storage_inactive"})
334     return E_INITIAL
335
336
337 @contextmanager
338 def _exclusive_run_lock(path):
339     path.parent.mkdir(parents=True, exist_ok=True)
340     if os.name == "nt":

```

```

341     import msvcrt
342     with path.open("a+b") as lock:
343         if lock.seek(0, os.SEEK_END) == 0:
344             lock.write(b"0"); lock.flush()
345         lock.seek(0)
346         try:
347             msvcrt.locking(lock.fileno(), msvcrt.LK_NBLCK, 1)
348         except OSError as error:
349             raise RuntimeError("该实验已有写入进程，拒绝并发覆盖检查点") from error
350         try:
351             yield
352         finally:
353             lock.seek(0)
354             msvcrt.locking(lock.fileno(), msvcrt.LK_UNLCK, 1)
355     else:
356         import fcntl
357         with path.open("a", encoding="utf-8") as lock:
358             try:
359                 fcntl.flock(lock, fcntl.LOCK_EX | fcntl.LOCK_NB)
360             except BlockingIOError as error:
361                 raise RuntimeError("该实验已有写入进程，拒绝并发覆盖检查点") from error
362             try:
363                 yield
364             finally:
365                 fcntl.flock(lock, fcntl.LOCK_UN)
366
367
368 def run_rollout(store, frozen, actual, dates, prices, initial_soc, output, **settings):
369     """每个实验仅允许一个写入者；不同实验可在独立目录并行。"""
370     output = Path(output)
371     output.mkdir(parents=True, exist_ok=True)
372     with _exclusive_run_lock(output/"run.lock"):
373         return _run_rollout(store, frozen, actual, dates, prices, initial_soc, output, **settings)
374
375
376 def _run_rollout(store, frozen, actual, dates, prices, initial_soc, output, *, start=31, end=None,
377                 reference_audits=None, rescue_seconds=None, solver_gap=PROTOCOL["solver_gap"],
378                 execution_revision=None, **settings):
379     if not np.isfinite(solver_gap) or not 0 < solver_gap < 1:
380         raise ValueError("求解精度门槛无效")
381     output.mkdir(parents=True, exist_ok=True)
382     recover_checkpoint(output)
383     if rescue_seconds is not None and rescue_seconds <= PROTOCOL["solver_seconds"]:
384         raise ValueError("追加计算预算必须大于原120秒，精度门槛保持不变")
385     end = len(dates) if end is None else end
386     source_hashes = {p.name: sha256(p) for p in Path(__file__).parent.glob("*.py")}
387     soc, frames, daily, calibration = initial_soc, [], [], []
388     resume_at = start
389     if (output/"dispatch.csv").exists():
390         previous, daily, calibration = read_checkpoint(output, actual, dates, prices, initial_soc,
391             start, end, settings)
392         completed = previous.date.nunique()
393         frames = [previous]
394         soc = float(previous.soc.iloc[-1]); resume_at += completed
395         print(f"从已验收的{completed}日连续回放检查点续算", flush=True)
396     previous_status = {}
397     if (output/"run_status.json").exists():

```

```

397     previous_status = json.loads((output/"run_status.json").read_text(encoding="utf-8"))
398     if previous_status.get("failed_date"):
399         atomic_json(output/f"failed_attempts/previous_status_{time.time_ns()}.json",
previous_status)
400     execution = {"base_solver_seconds": PROTOCOL["solver_seconds"], "rescue_seconds": rescue_seconds
,
401                 "rescue_trigger": "all_day_candidates_failed", "gap": solver_gap,
402                 "interpretation": "supplementary_compute_not_retroactive_original_budget_success",
403                 "tail_stratification": PROTOCOL["tail_stratification"],
404                 "tail_fraction": PROTOCOL["tail_fraction"],
405                 "dynamic_soc_reserve": PROTOCOL["dynamic_soc_reserve"],
406                 "reserve_quantile": PROTOCOL["reserve_quantile"],
407                 "refine_seconds": PROTOCOL["refine_seconds"],
408                 "lp_certificate_fast_path": True,
409                 "solver_threads": PROTOCOL["solver_threads"],
410                 "rescue_reuses_day_assets_bound_and_incumbent": True,
411                 "evaluation_initial_soc_kwh": initial_soc}
412     if solver_gap != PROTOCOL["solver_gap"] and not execution_revision:
413         raise ValueError("非原精度续算必须携带显式执行修订摘要")
414     if execution_revision:
415         execution["precision_revision_sha256"] = execution_revision
416     if solver_gap > PROTOCOL["solver_gap"]:
417         execution["base_policy_solver"] = "equivalent_MILP_after_global_LP_certificate_attempt"
418     if settings.get("horizon", "auto") == "auto" and reference_audits is None:
419         execution["horizon_evaluation"] = "K2_K3_then_K1_only_if_both_fail"
420     old_execution = previous_status.get("execution", {})
421     if daily and old_execution.get("gap", PROTOCOL["solver_gap"]) > solver_gap:
422         raise ValueError("更严格精度不能直接沿用较宽门槛的历史, 需独立复核")
423     history = previous_status.get("execution_history", [])
424     if old_execution and old_execution != execution:
425         history = [*history, {"execution": old_execution, "completed_days": len(daily),
426                               "next_date": str(dates[resume_at].date()) if resume_at < end else None
}}]
427     status_base = {"settings": settings, "start_index": start, "end_index": end,
428                   "execution": execution, "execution_history": history}
429     atomic_json(output/"run_status.json", {"complete": False, **status_base})
430     for d in range(resume_at, end):
431         daily_settings = dict(settings)
432         day_solve_cache = {}
433         if reference_audits is not None:
434             reference = json.loads((reference_audits/f"{pd.Timestamp(dates[d]).date()}.json").
read_text(encoding="utf-8"))
435             chosen = reference[str(reference["selected_K"])]
436             daily_settings.update(horizon=reference["selected_K"], forced_count=chosen["S"],
437                                   fixed_pair=chosen["selection"]["pipeline"])
438         try:
439             try:
440                 policy, weights, net_scenarios, point, audit = make_day(
441                     store, frozen, actual[:d], d, prices, soc,
442                     progress_path=output/"active_solve.json", solver_gap=solver_gap,
443                     solve_cache=day_solve_cache, **daily_settings)
444             except ComputationalLimit as original:
445                 atomic_json(output/f"failed_attempts/{dates[d].date()}_base_{time.time_ns()}.json",
446                               {"date": str(dates[d]), "solver_seconds": PROTOCOL["solver_seconds"], "
details": original.details})
447                 if rescue_seconds is None:
448                     raise

```

```

449         print(f"{dates[d].date()} 原预算全候选受限, 追加{rescue_seconds:g}秒预算, gap为{
solver_gap:.0%}", flush=True)
450         policy, weights, net_scenarios, point, audit = make_day(
451             store, frozen, actual[:d], d, prices, soc, solver_seconds=rescue_seconds,
452             progress_path=output/"active_solve.json", solver_gap=solver_gap,
453             solve_cache=day_solve_cache, **daily_settings)
454         audit.update(original_budget_failure=original.details, supplemental_solver_seconds=
rescue_seconds,
455                     computational_limited=True)
456     except ComputationalLimit as error:
457         atomic_json(output/"run_status.json", {"complete": False, **status_base,
458                                             "failed_date": str(dates[d]), "reason": str(error)
, "solver_details": error.details})
459         atomic_json(output/"active_solve.json", {"date": str(dates[d].date()), "phase": "failed"
})
460         raise
461         audit["execution"] = execution
462         audit["source_sha256_at_rollout_start"] = source_hashes
463         atomic_json(output/f"daily_audit/{pd.Timestamp(dates[d]).date()}.json", audit)
464         write_csv(output/f"frozen_plans/{pd.Timestamp(dates[d]).date()}.csv", pd.DataFrame({
465             "step": np.arange(len(policy.grid)), "grid": policy.grid,
466             "charge_cap": policy.charge_cap, "discharge_cap": policy.discharge_cap,
467             "soc_reserve_kwh": policy.reserve,
468             "role": np.where(np.arange(len(policy.grid)) < 144, "first_day_frozen", "
open_loop_proxy_only"))))
469         # 到此之前当日actual[d]未进入任何选模、场景或求解调用。
470         net = DT*(actual[d, :, 0]-actual[d, :, 1])
471         frame = dispatch_frame(dates[d], policy, net, soc, prices, rigid=settings.get("kind") == "
rigid")
472         frame["predicted_load_kw"] = point[0, :, 0]
473         frame["predicted_pv_kw"] = point[0, :, 1]
474         frames.append(frame)
475         selected_k = audit["selected_K"]
476         selected = audit[str(selected_k)]
477         selected_count = selected.get("S", len(weights))
478         scenario_meta = selected.get("candidates", {}).get(
479             str(selected_count), {}).get("scenarios", {})
480         selected_solver = selected["selected_solver"]
481         daily.append({"date": str(pd.Timestamp(dates[d]).date()), "K": selected_k,
482                     "M": selected["M"], "S": len(weights),
483                     "initial_soc": soc, "final_soc": float(frame.soc.iloc[-1]),
484                     "min_soc": float(frame.soc.min()),
485                     "planned_grid_kwh": float(frame.grid.sum()),
486                     "planned_cost": float(frame.planned_cost.sum()),
487                     "emergency_cost": float(frame.emergency_cost.sum()),
488                     "cost": float(frame.planned_cost.sum()+frame.emergency_cost.sum()),
489                     "emergency_kwh": float(frame.emergency.sum()),
490                     "unused_kwh": float(frame.unused.sum()),
491                     "charge_kwh": float(frame.charge.sum()),
492                     "discharge_kwh": float(frame.discharge.sum()),
493                     "tail_scenario_count": scenario_meta.get("S_tail", 0),
494                     "body_scenario_count": scenario_meta.get("S_body", len(weights)),
495                     "max_reserve_kwh": float(policy.reserve[:144].max()),
496                     "mean_reserve_kwh": float(policy.reserve[:144].mean()),
497                     "requested_mip_gap": selected_solver.get("requested_gap", solver_gap),
498                     "achieved_mip_gap": selected_solver.get("mip_gap", 0.),
499                     "runtime_seconds": audit.get("runtime_seconds", 0.),

```

```

500         "solver_status": selected_solver.get("status", "test_double"),
501         "computational_limited": audit["computational_limited"]})
502     for h in range(min(selected_k, len(dates)-d)):
503         realized = DT*(actual[d+h, :, 0]-actual[d+h, :, 1])
504         scenario = net_scenarios[:, h*144:(h+1)*144]
505         for nominal, low, high in ((.8, .1, .9), (.9, .05, .95)):
506             lower, upper = weighted_quantile(scenario, weights, low), weighted_quantile(scenario
, weights, high)
507             calibration.append({"origin": str(pd.Timestamp(dates[d]).date()), "horizon": h, "
nominal": nominal,
508                               "covered_intervals": int(((realized >= lower)&(realized <= upper)
)).sum()),
509                               "n": 144, "picp": float(((realized >= lower)&(realized <= upper)
).mean())})
510         soc = float(frame.soc.iloc[-1])
511         save_checkpoint(output, pd.concat(frames, ignore_index=True), daily, calibration)
512         print(f"真实回放 {pd.Timestamp(dates[d]).date()} K={selected_k} S={len(weights)} SOC={soc:.3f
}", flush=True)
513     atomic_json(output/"run_status.json", {"complete": True, **status_base, "days": len(daily),
514                                           "initial_soc": initial_soc, "final_soc": soc,
515                                           "full_evaluation_period": start == 31 and end == 365})
516     atomic_json(output/"active_solve.json", {"phase": "complete", "days": len(daily)})
517     return pd.concat(frames, ignore_index=True), pd.DataFrame(daily)

```

src/q2/run.py

```

1  """一条命令: 附件2→严格核验→全年/历史诊断→CSV/图→测试→发布。"""
2
3  import datetime as dt
4  import importlib.metadata
5  import os
6  from pathlib import Path
7  import shutil
8  import subprocess
9  import sys
10 import tempfile
11
12 import numpy as np
13 import pandas as pd
14
15 from backtest import rolling_folds
16 from data import (FORECAST_START, ROOT, SERIES, history_before, jump_context, read_attachment,
17                  sha256, write_csv, write_json)
18 from diagnostics import SETTINGS, analyze_scope
19 from figures import create_figures
20 from report import candidate_models, create_report
21 from traceability import generate_traceability
22
23
24 def validate_outputs(staging, frame, diagnostics):
25     """磁盘回读、源值一致与频谱积分等独立验收; 失败不能发布。"""
26     counts = {}
27     for path in staging.rglob("*.csv"):
28         raw = path.read_bytes()
29         if raw.startswith(b"\xef\xbb\xbf"):
30             raise ValueError(f"CSV含BOM: {path}")

```



```

31     raw.decode("utf-8")
32     counts[str(path.relative_to(staging))] = len(pd.read_csv(path, float_precision="round_trip")
33 )
34 saved = pd.read_csv(staging / "inputs/processed/timeseries.csv", float_precision="round_trip")
35 for variable in SERIES:
36     np.testing.assert_array_equal(saved[variable], frame[variable])
37 for scope, results in diagnostics.items():
38     targets = results["spectral_targets"]
39     np.testing.assert_allclose(targets.integrated_psd_kw2, targets.population_variance_kw2, rtol
40 =1e-12, atol=1e-10)
41     if len(results["stationarity_tests"]) != 32 or not results["stationarity_tests"].status.eq("
42 ok").all():
43         raise ValueError(f"{scope}平稳性检验不完整")
44     return {"csv_rows": counts, "source_values_exact": True, "psd_parseval_pass": True,
45           "utf8_no_bom": True, "stationarity_tests": 64}
46
47 def publish(staging):
48     """沿用项目当前结果指针结构；输入/输出统一切换，失败不替换已有结果。"""
49     current = ROOT / "outputs/q2_current"
50     for alias, target in ((ROOT / "inputs/q2", current / "inputs"),
51                           (ROOT / "outputs/q2/raw", current / "raw"),
52                           (ROOT / "outputs/processed/q2", current / "processed")):
53         alias.parent.mkdir(parents=True, exist_ok=True)
54         relative = os.path.relpath(target, alias.parent)
55         if alias.is_symlink():
56             if os.readlink(alias) != relative:
57                 raise ValueError(f"结果别名指向其他位置，拒绝覆盖: {alias}")
58             elif alias.exists():
59                 raise ValueError(f"结果位置已有实体目录，拒绝覆盖: {alias}")
60             else:
61                 alias.symlink_to(relative, target_is_directory=True)
62         pending = current.with_name(".q2_current_pending")
63         pending.symlink_to(os.path.relpath(staging, current.parent), target_is_directory=True)
64         try:
65             os.replace(pending, current)
66         finally:
67             pending.unlink(missing_ok=True)
68
69 def run_pipeline():
70     generations = ROOT / "outputs/.q2-runs"
71     generations.mkdir(parents=True, exist_ok=True)
72     staging = Path(tempfile.mkdtemp(prefix="run-", dir=generations))
73     started = dt.datetime.now(dt.timezone.utc).isoformat()
74     raw, output = staging / "raw", staging / "processed"
75     input_raw, input_processed = staging / "inputs/raw", staging / "inputs/processed"
76     for path in (raw, output, input_raw, input_processed):
77         path.mkdir(parents=True)
78     code_hashes = {str(p.relative_to(ROOT)): sha256(p) for p in [(ROOT / "src/q2").glob("*.py"),
79 ROOT / "src/plots/q2.py"]}
80     plan_digest = sha256(ROOT / "docs/2/dierwen1.md")
81     try:
82         original = ROOT / "docs/CUMCM2026Problems/C题/附件/附件2.xlsx"
83         copied = input_raw / "attachment2.xlsx"
84         shutil.copy2(original, copied)
85         if sha256(original) != sha256(copied):

```

```

84         raise ValueError("原始附件副本哈希不一致")
85         source_digest = sha256(copied)
86         write_json(input_raw / "manifest.json", {"source": str(original.relative_to(ROOT)), "sha256"
87 : sha256(original),
88                                     "copy": "inputs/q2/raw/attachment2.xlsx", "role": "
immutable_raw_observations"})
89         frame, quality = read_attachment(copied, raw / "quality_checks.csv")
90         write_csv(input_processed / "timeseries.csv", frame)
91         shutil.copy2(input_processed / "timeseries.csv", output / "timeseries.csv")
92         shutil.copy2(raw / "quality_checks.csv", output / "quality_checks.csv")
93         write_csv(output / "jump_context.csv", jump_context(frame))
94         write_json(input_processed / "settings.json", {**SETTINGS, "forecast_start": "2025-02-01",
95                                     "cutoff": "timestamp < origin", "
boundary_authority": "user_explicit_confirmation",
96                                     "forecast_horizon": 144, "forecast_unit": "kW
", "dt_hours_dispatch_only": 1/6})
97         diagnostics = {}
98         for scope, view in (("initial_history", history_before(frame, FORECAST_START)), ("full_year"
, frame)):
99             diagnostics[scope] = analyze_scope(view, scope)
100             for name, values in diagnostics[scope].items():
101                 write_csv(output / f"{scope}_{name}.csv", values)
102             candidates = candidate_models(diagnostics["initial_history"])
103             folds = rolling_folds(frame)
104             print("生成11张科研分析图，每张PNG与SVG", flush=True)
105             figures = create_figures(frame, diagnostics["full_year"], output / "figures")
106             write_json(raw / "figure_manifest.json", figures)
107             create_report(frame, diagnostics, candidates, folds, figures, output)
108             trace = generate_traceability(output)
109             validation = validate_outputs(staging, frame, diagnostics)
110             print("运行当前源码测试与真实CSV回读验收", flush=True)
111             environment = {**os.environ, "Q2_GENERATION": str(staging), "PYTHONIOENCODING": "utf-8"}
112             tests = subprocess.run([sys.executable, "-m", "unittest", "discover", "-s", str(ROOT / "src/
q2"),
113                                     "-p", "test_*.py", "-v"], env=environment, capture_output=True, text
=True, encoding="utf-8")
114             (raw / "test_results.txt").write_text(tests.stdout + tests.stderr, encoding="utf-8")
115             if tests.returncode:
116                 raise ValueError(f"测试失败，正式结果未更新: {raw / 'test_results.txt'}")
117             if code_hashes != {str(p.relative_to(ROOT)): sha256(p) for p in [(ROOT / "src/q2").glob("*.
py"), ROOT / "src/plots/q2.py"]}:
118                 raise ValueError("运行中源码发生修改，需重新完整运行")
119             if source_digest != sha256(original) or plan_digest != sha256(ROOT / "docs/2/dierwen1.md"):
120                 raise ValueError("运行中原始附件或方案发生修改，需重新完整运行")
121             write_json(raw / "validation.json", {**validation, "traceability": trace, "tests_passed":
True,
122                                     "figure_count": len(figures), "ui_smoke": "
requires_actual_browser_review"})
123             manifest = {"started_utc": started, "finished_utc": dt.datetime.now(dt.timezone.utc).
isoformat(),
124                         "source_sha256": source_digest, "plan_sha256": plan_digest,
125                         "code_sha256": code_hashes, "python": sys.version,
126                         "versions": {name: importlib.metadata.version(name) for name in
("numpy", "scipy", "pandas", "statsmodels", "matplotlib", "openpyxl
", "threadpoolctl")},
127                         "output_sha256": {str(p.relative_to(staging)): sha256(p) for p in staging.rglob(
"*) if p.is_file()},

```

```

128         "real_forecast_or_dispatch_computed": False, "all_stage_2_1_checks_passed": True
129     }
130     write_json(raw / "run_manifest.json", manifest)
131     publish(staging)
132     except BaseException as error:
133         (raw / "failure.txt").write_text(str(error), encoding="utf-8")
134         raise
135     print(f"已发布: {ROOT / 'outputs/processed/q2/report.html'}", flush=True)
136     return staging
137
138 if __name__ == "__main__":
139     run_pipeline()

```

src/q2/scenarios.py

```

1  """最终方案2.5：完整联合误差块、近期窗口、尾部保护medoids与概率守恒。"""
2
3  import numpy as np
4  from scipy.spatial.distance import cdist
5
6  from policy import DT, E_MIN, E_MAX, ETA_D
7
8
9  def eligible_origins(shadows, current, horizon, minimum=14, window=None):
10     """shadow的索引是forecast origin: 只有整个horizon都已实现才有误差块。"""
11     origins = np.arange(len(shadows))
12     ready = (origins+horizon <= current) & np.isfinite(shadows[:, :horizon]).all(axis=(1, 2, 3))
13     if window is not None:
14         ready &= origins >= current-window
15     result = origins[ready]
16     return result if len(result) >= minimum else np.empty(0, dtype=int)
17
18
19  def history_window(shadows, current, horizon, minimum=14, forced_window="auto"):
20     windows = [28, 56, 84, None] if forced_window == "auto" else [forced_window]
21     for window in windows:
22         origins = eligible_origins(shadows, current, horizon, minimum, window)
23         if len(origins):
24             return origins, window
25     raise ValueError("该联合预测流程没有足够完整误差块：不得重采样制造样本")
26
27
28  def joint_blocks(history, shadows, origins, horizon):
29     """在同一个origin下相减：保持L/PV、日内以及跨日前瞻次序。"""
30     blocks = np.stack([history[j:j+horizon]-shadows[j, :horizon] for j in origins])
31     if blocks.shape[1:] != (horizon, 144, 2) or not np.isfinite(blocks).all():
32         raise ValueError("联合误差块不完整")
33     return blocks
34
35
36  def net_error_energy(blocks):
37     """负荷/PV联合误差块转换为母线侧净负荷误差，单位kWh。"""
38     blocks = np.asarray(blocks, dtype=float)
39     if blocks.ndim != 4 or blocks.shape[2:] != (144, 2) or not np.isfinite(blocks).all():
40         raise ValueError("联合误差块维度或数值不合法")

```

```

41     return (DT*(blocks[... , 0]-blocks[... , 1])).reshape(len(blocks), -1)
42
43
44 def cumulative_pressure(blocks):
45     """S_j=max_k[cumsum(e_net)]_+, 保留完整预测窗口的时序压力。"""
46     errors = net_error_energy(blocks)
47     return np.maximum(np.cumsum(errors, axis=1).max(axis=1), 0)
48
49
50 def dynamic_reserve(blocks, quantile=.8):
51     """由已完整实现的历史首日误差计算并冻结144段SOC安全裕度。"""
52     if not 0 <= quantile <= 1:
53         raise ValueError("动态SOC安全裕度分位数不合法")
54     errors = net_error_energy(blocks)[: , :144]
55     future_pressure = np.empty_like(errors)
56     running = np.zeros(len(errors))
57     for t in range(143, -1, -1):
58         running = np.maximum(errors[:, t]+running, 0)
59         future_pressure[:, t] = running
60     weights = np.ones(len(errors))/len(errors)
61     bus_reserve = weighted_quantile(future_pressure, weights, quantile)
62     reserve = np.minimum((E_MAX-E_MIN), bus_reserve/ETA_D)
63     return reserve, {"quantile": quantile, "historical_blocks": len(blocks),
64                     "max_reserve_kwh": float(reserve.max()),
65                     "mean_reserve_kwh": float(reserve.mean())}
66
67
68 def weighted_quantile(values, weights, quantile):
69     """离散经验分布左分位, 满足 $F(g^-) \leq \alpha \leq F(g)$ ; 不作线性插值。"""
70     values, weights = np.asarray(values), np.asarray(weights)
71     if not 0 <= quantile <= 1 or values.shape[0] != len(weights) or (weights <= 0).any() or not np.
72         isclose(weights.sum(), 1):
73         raise ValueError("无效分位概率")
74     order = np.argsort(values, axis=0, kind="stable")
75     sorted_values = np.take_along_axis(values, order, axis=0)
76     sorted_weights = np.broadcast_to(weights.reshape((-1,)+(1,)*(values.ndim-1)), values.shape)
77     cumulative = np.cumsum(np.take_along_axis(sorted_weights, order, axis=0), axis=0)
78     index = np.argmax(cumulative >= quantile-1e-12, axis=0)
79     return np.take_along_axis(sorted_values, index[None], axis=0)[0]
80
81
82 def medoids(blocks, prices, count):
83     """确定性PAM交换, 强制最严重价格加权正净误差块, 真实块代表且簇质量赋权。"""
84     size = len(blocks)
85     if not 1 <= count <= size:
86         raise ValueError("medoid数量必须介于1和真实块数之间")
87     scales = np.std(blocks, axis=(0, 1, 2), ddof=1)
88     normalized = blocks/np.where(scales > 0, scales, 1)
89     points = normalized.reshape(size, -1)
90     distances = cdist(points, points)
91     positive = np.maximum(DT*(blocks[... , 0]-blocks[... , 1]), 0)
92     protected = int(np.argmax(np.sum(positive*np.asarray(prices).reshape(blocks.shape[1:3]), axis
93         =(1, 2))))
94     selected = [protected]
95     while len(selected) < count:
96         nearest = distances[:, selected].min(axis=1)
97         remaining = [j for j in range(size) if j not in selected]

```

```

96     costs = [np.minimum(nearest, distances[:, j]).sum() for j in remaining]
97     selected.append(remaining[int(np.argmin(costs))])
98     # PAM以严格下降为终止条件, 不引入随机样本或任意迭代次数。
99     while True:
100         current = float(distances[:, selected].min(axis=1).sum())
101         best, replacement = current, None
102         for position in range(1, count):
103             others = selected[:position]+selected[position+1:]
104             nearest = distances[:, others].min(axis=1)
105             for candidate in range(size):
106                 if candidate in selected:
107                     continue
108                 score = float(np.minimum(nearest, distances[:, candidate]).sum())
109                 if score < best-1e-10:
110                     best, replacement = score, (position, candidate)
111             if replacement is None:
112                 break
113             selected[replacement[0]] = replacement[1]
114     selected = np.asarray(selected)
115     assignment = np.argmax(distances[:, selected], axis=1)
116     # 重复误差块也按实际origin保留概率; 每个medoid自身归入自己的簇。
117     assignment[selected] = np.arange(count)
118     weights = np.bincount(assignment, minlength=count)/size
119     return selected, weights, {"protected_index": protected, "cluster_assignment": assignment.tolist(),
120                               "scales_kw": scales.tolist(), "distance_objective": float(distances[
121                               np.arange(size), selected[assignment]].sum())}
122
123 def _representatives(blocks, prices, count):
124     if count == 0:
125         return np.empty(0, dtype=int), {}
126     if count >= len(blocks):
127         return np.arange(len(blocks)), {"reduced": False}
128     indices, _, meta = medoids(blocks, prices, count)
129     return indices, {**meta, "reduced": True}
130
131
132 def construct(point, blocks, prices, count=None, *, origins=None, all_blocks=None,
133              all_origins=None, tail_fraction=.2):
134     """总场景数不变的body/tail分层: 长期尾部与近期主体分别保留经验概率质量。"""
135     blocks = np.asarray(blocks, dtype=float)
136     all_blocks = blocks if all_blocks is None else np.asarray(all_blocks, dtype=float)
137     size = len(blocks)
138     if not size or not len(all_blocks) or not 0 < tail_fraction < 1:
139         raise ValueError("场景池或尾部分层比例不合法")
140     origins = np.arange(size) if origins is None else np.asarray(origins, dtype=int)
141     all_origins = origins if all_origins is None else np.asarray(all_origins, dtype=int)
142     if len(origins) != size or len(all_origins) != len(all_blocks):
143         raise ValueError("场景误差块与forecast-origin索引不一致")
144     scenario_count = size if size <= 40 or count is None or count == size else min(size, int(count))
145     tail_pool_count = max(1, int(np.floor(tail_fraction*len(all_blocks)+.5)))
146     pressure = cumulative_pressure(all_blocks)
147     tail_pool_positions = np.argsort(pressure, kind="stable")[-tail_pool_count:]
148     tail_origin_set = set(int(v) for v in all_origins[tail_pool_positions])
149     body_pool_positions = np.array([i for i, origin in enumerate(origins)
150                                   if int(origin) not in tail_origin_set], dtype=int)

```

```

151 tail_count = max(1, int(np.floor(tail_fraction*scenario_count+.5)))
152 body_count = scenario_count-tail_count
153 if len(body_pool_positions) < body_count:
154     fallback = [i for i in range(size) if i not in set(body_pool_positions.tolist())]
155     body_pool_positions = np.r_[body_pool_positions, fallback]
156 body_choice, body_meta = _representatives(blocks[body_pool_positions], prices, body_count)
157 tail_choice, tail_meta = _representatives(all_blocks[tail_pool_positions], prices, tail_count)
158 selected_body_positions = body_pool_positions[body_choice]
159 selected_tail_positions = tail_pool_positions[tail_choice]
160 selected_blocks = np.concatenate((blocks[selected_body_positions],
161                                   all_blocks[selected_tail_positions]), axis=0)
162 tail_mass = tail_pool_count/len(all_blocks)
163 weights = np.r_[np.full(body_count, (1-tail_mass)/body_count) if body_count else np.empty(0),
164                 np.full(tail_count, tail_mass/tail_count)]
165 weights /= weights.sum()
166 values = np.maximum(np.asarray(point)[None]+selected_blocks, 0)
167 net = DT*(values[..., 0]-values[..., 1]).reshape(scenario_count, -1)
168 selected_origins = np.r_[origins[selected_body_positions], all_origins[selected_tail_positions]]
169 meta = {"reduced": bool(scenario_count < size), "tail_stratification": True,
170         "M": size, "M_all": len(all_blocks), "S": scenario_count,
171         "S_body": body_count, "S_tail": tail_count,
172         "tail_fraction": tail_fraction, "tail_probability_mass": tail_mass,
173         "body_probability_mass": 1-tail_mass,
174         "tail_pool_count": tail_pool_count,
175         "body_origin_indices": origins[selected_body_positions].tolist(),
176         "tail_origin_indices": all_origins[selected_tail_positions].tolist(),
177         "selected_origin_indices": selected_origins.tolist(),
178         "indices": list(range(scenario_count)), "weights": weights.tolist(),
179         "body_selection": body_meta, "tail_selection": tail_meta,
180         "tail_pressure_min_kwh": float(pressure[tail_pool_positions].min()),
181         "tail_pressure_max_kwh": float(pressure[tail_pool_positions].max())}
182 return net, weights, meta
183
184
185 def quantile_comparison(full_net, reduced_net, weights):
186     rows = {}
187     scale = max(float(np.mean(np.abs(full_net))), 1e-8)
188     for q in (.8, .9):
189         full = weighted_quantile(full_net, np.ones(len(full_net))/len(full_net), q)
190         reduced = weighted_quantile(reduced_net, weights, q)
191         rows[str(q)] = {"mean_absolute_error_kwh": float(np.abs(full-reduced).mean()),
192                        "normalized_error": float(np.abs(full-reduced).mean()/scale),
193                        "positive_tail_underestimate_kwh": float(np.maximum(full-reduced, 0).mean())}
194     return rows

```

src/q2/shadows.py

```

1 """预测流程条件化的影子库；持久保存origin×horizon×slot预测，不重算历史预测。"""
2
3 import itertools
4 from pathlib import Path
5
6 import numpy as np
7 import pandas as pd
8
9 from data import write_csv, write_json

```

```

10 from forecast import (economic_metrics, harmonic_forecast, lightgbm_forecast, seasonal,
11                        select_pipeline)
12 from protocol import PROTOCOL
13 from scenarios import history_window, joint_blocks
14
15 MODEL_IDS = ("load_week", "load_lgb_0", "load_lgb_1", "pv_mean7", "pv_dhr")
16
17
18 def fit_day(history, origin, active_load_ids):
19     baseline = seasonal(history, 3)
20     output = {"load_week": baseline[...], 0}, {"pv_mean7": baseline[...], 1}
21     for index in active_load_ids:
22         output[f"load_lgb_{index}"] = lightgbm_forecast(history, origin, 3, PROTOCOL["
23             lightgbm_candidates"][index])
24     pv, diagnostics, correlation = harmonic_forecast(history, 3)
25     if pv is not None:
26         output["pv_dhr"] = pv
27     return output, diagnostics, correlation
28
29 def freeze_lightgbm(store, actual):
30     """只使用1月影子h=0真值；保留每个配置自身的历史影子预测以保证流程一致。"""
31     rows = []
32     for index in range(len(PROTOCOL["lightgbm_candidates"])):
33         prediction = store[f"load_lgb_{index}"][14:31, 0]
34         valid = np.isfinite(prediction).all(axis=1)
35         if not valid.all():
36             raise ValueError("1月LightGBM候选影子不完整，不能冻结或用2月真值选参")
37         error = prediction - actual[14:31, :, 0]
38         rows.append({"configuration": index, "mae_kw": float(np.abs(error).mean()),
39                     "rmse_kw": float(np.sqrt(np.mean(error**2))), "origins": 17,
40                     "first_origin": "2025-01-15", "last_origin": "2025-01-31"})
41     chosen = min(rows, key=lambda r: (r["mae_kw"], r["rmse_kw"], r["configuration"]))["configuration"]
42     return chosen, rows
43
44
45 def build_shadow_library(actual, dates, output, *, boundary):
46     """调用预测器时仅传入actual[:d]副本；存储三维原始float64 DAT及显式形状。"""
47     if boundary != "source_date_before_origin":
48         raise ValueError("完整日边界尚未确认；不能擅自对缺失末段作填补")
49     output = Path(output)
50     output.mkdir(parents=True, exist_ok=True)
51     shape = (len(dates), 3, 144)
52     store = {}
53     for name in MODEL_IDS:
54         store[name] = np.memmap(output/f"{name}.dat", mode="w+", dtype="<f8", shape=shape)
55         store[name][:] = np.nan
56     write_json(output/"shape.json", {"shape": shape, "dtype": "<f8", "axes": ["origin", "horizon", "
57         slot"],
58         "first_origin_date": str(dates[0]), "model_ids": MODEL_IDS,
59         "boundary": boundary, "missing": "NaN=no_forecast_not_zero"})
60     chosen = None
61     for d in range(14, len(dates)):
62         if d == 31:
63             chosen, scores = freeze_lightgbm(store, actual[:31])
64             write_json(output/"frozen_lightgbm.json", {"index": chosen, "configuration": PROTOCOL["

```

```

lightgbm_candidates"][chosen],
64                                     "freeze_origin": "2025-02-01", "scores":
scores})
65     active = list(range(len(PROTOCOL["lightgbm_candidates"]))) if d < 31 else [chosen]
66     predictions, diagnostic, correlations = fit_day(actual[:d].copy(), dates[d], active)
67     for name, prediction in predictions.items():
68         store[name][d] = prediction
69         store[name].flush()
70     write_json(output/f"dhr/{pd.Timestamp(dates[d]).date()}.json", diagnostic)
71     if correlations is not None:
72         write_csv(output/f"dhr/{pd.Timestamp(dates[d]).date()}_correlations.csv", correlations)
73     write_json(output/"progress.json", {"last_origin": str(dates[d]), "days_done": d-13,
74                                         "complete": d == len(dates)-1})
75     print(f"影子预测 {pd.Timestamp(dates[d]).date()}, 仅使用此前{d}个完整日", flush=True)
76     return store, chosen
77
78
79 def open_library(output):
80     import json
81     output = Path(output)
82     meta = json.loads((output/"shape.json").read_text(encoding="utf-8"))
83     progress = json.loads((output/"progress.json").read_text(encoding="utf-8"))
84     if not progress["complete"]:
85         raise ValueError("影子库未完整计算")
86     store = {name: np.memmap(output/f"{name}.dat", mode="r", dtype=meta["dtype"], shape=tuple(meta["
87     shape"]))) for name in MODEL_IDS}
88     frozen = json.loads((output/"frozen_lightgbm.json").read_text(encoding="utf-8"))
89     return store, frozen["index"]
90
91 def combined(store, pair):
92     return np.stack([store[pair[0]], store[pair[1]]], axis=-1)
93
94
95 def choose_day(store, frozen, history, current, horizon, prices, *, minimum=14, window="auto",
96               fixed_pair=None):
97     """所有联合候选在共同已实现origin上比较代理损失，禁止使用反事实调度费用。"""
98     candidates = list(itertools.product(("load_week", f"load_lgb_{frozen}"), ("pv_mean7", "pv_dhr")))
99     if fixed_pair is not None:
100         candidates = [tuple(fixed_pair)]
101     usable = {}
102     for pair in candidates:
103         forecasts = combined(store, pair)
104         if not np.isfinite(forecasts[current, :horizon]).all():
105             continue
106         try:
107             origins, chosen_window = history_window(forecasts, current, horizon, minimum, window)
108         except ValueError:
109             continue
110         usable[pair] = (forecasts, origins, chosen_window)
111     if not usable:
112         raise ValueError("没有满足最低完整历史块门槛的预测流程：需要基线回退期")
113     common = sorted(set.intersection(*(set(v[1]) for v in usable.values()))))
114     if not common:
115         raise ValueError("在线联合候选没有共同已实现样本外评价区间")
116     scores = {pair: economic_metrics(history[common], values[0][common, 0], prices) for pair, values

```



```

116         in usable.items()}
117     pair = select_pipeline(scores)
118     forecasts, origins, chosen_window = usable[pair]
119     blocks = joint_blocks(history, forecasts, origins, horizon)
120     metadata = {"pipeline": list(pair), "origin_indices": origins.tolist(), "window_days":
        chosen_window,
121                "common_scored_origins": common, "scores": {"|".join(k): v for k, v in scores.items
        ()},
122                "selection_uses_dispatch_cost": False, "minimum_blocks": minimum}
123     return forecasts[current, :horizon].copy(), blocks, metadata

```

src/q2/test_baseline_reuse.py

```

1  """基准复用的完整轨迹、输入等价与非覆盖验收。"""
2
3  import json
4  from pathlib import Path
5  import tempfile
6  import unittest
7  from unittest.mock import patch
8
9  import numpy as np
10 import pandas as pd
11
12 from baseline_reuse import reuse_main_baseline, window_inputs_match
13 from checkpoint import atomic_json, save_checkpoint
14 from policy import Policy
15 from rolling import dispatch_frame
16
17
18 class BaselineReuseTests(unittest.TestCase):
19     def test_full_year_initial_baseline_copies_exact_committed_results(self):
20         dates = pd.date_range("2025-01-01", periods=365)
21         actual = np.zeros((365, 144, 2)); prices = np.ones(144)
22         policy = Policy(np.zeros(288), np.zeros(288), np.zeros(288))
23         with tempfile.TemporaryDirectory() as tmp:
24             run = Path(tmp); main = run/"processed/main"; main.mkdir(parents=True)
25             (main/"daily_audit").mkdir(); (main/"frozen_plans").mkdir()
26             frames, daily, calibration = [], [], []
27             for d in range(31, 365):
28                 date = str(dates[d].date())
29                 frames.append(dispatch_frame(dates[d], policy, np.zeros(144), 6000, prices))
30                 daily.append({"date": date, "initial_soc": 6000., "final_soc": 6000., "cost": 0., "
emergency_kwh": 0.})
31                 for h in range(min(2, 365-d)):
32                     for nominal in (.8, .9):
33                         calibration.append({"origin": date, "horizon": h, "nominal": nominal, "
covered_intervals": 144, "n": 144})
34                 atomic_json(main/f"daily_audit/{date}.json", {"selected_K": 2, "2": {"
selected_solver": {"reliable": True}}})
35                 pd.DataFrame({"grid": policy.grid, "charge_cap": policy.charge_cap, "discharge_cap":
policy.discharge_cap}).to_csv(
36                     main/f"frozen_plans/{date}.csv", index=False, encoding="utf-8")
37                 atomic_json(main/"run_status.json", {"complete": True, "settings": {}, "start_index":
31, "end_index": 365,
38                     "execution": {"gap": .03}, "execution_history": [{"

```

```

execution": {"gap": .02}}}}))
39         save_checkpoint(main, pd.concat(frames, ignore_index=True), daily, calibration)
40         self.assertTrue(reuse_main_baseline(run, "initial_6000", {}, 0, actual, dates, prices,
6000))
41         target = run/"processed/experiments/initial_6000"
42         for name in ("dispatch.csv", "daily.csv", "calibration.csv", "checkpoint.json", "
run_status.json",
43                     "daily_audit/2025-02-01.json", "frozen_plans/2025-12-31.csv"):
44             self.assertEqual((main/name).read_bytes(), (target/name).read_bytes())
45         provenance = json.loads((target/"baseline_reuse.json").read_text(encoding="utf-8"))
46         self.assertEqual(provenance["days"], 334)
47         self.assertEqual(provenance["extra_MILP_solves"], 0)
48         self.assertFalse(reuse_main_baseline(run, "initial_6000", {}, 0, actual, dates, prices,
6000))
49         self.assertFalse(reuse_main_baseline(run, "initial_1200", {}, 0, actual, dates, prices,
6000))
50
51     def test_window_reuse_checks_all_pipeline_scores_and_causal_history(self):
52         dates = pd.date_range("2025-01-01", periods=34)
53         selection = {"pipeline": ["chosen"], "window_days": 28, "scores": {"chosen": 1., "other":
2.}}
54         with tempfile.TemporaryDirectory() as tmp:
55             main = Path(tmp)
56             atomic_json(main/"daily_audit/2025-02-01.json", {"2": {"selection": selection}})
57             for changed in (False, True):
58                 def choose(store, frozen, history, day, k, prices, **kwargs):
59                     self.assertEqual(len(history), day)
60                     metadata = json.loads(json.dumps(selection))
61                     if changed and kwargs.get("window") == 28:
62                         metadata["scores"]["other"] = 3.
63                     return np.zeros((k, 144, 2)), np.zeros((1, k, 144, 2)), metadata
64                 with patch("baseline_reuse.choose_day", side_effect=choose):
65                     self.assertEqual(window_inputs_match({}, 0, np.zeros((34, 144, 2)), dates,
66                                     np.ones(144), main, 31, 32), not changed)
67
68     def test_incomplete_or_different_main_settings_cannot_be_reused(self):
69         with tempfile.TemporaryDirectory() as tmp:
70             run = Path(tmp)
71             for value in ({"complete": False, "settings": {}}, {"complete": True, "settings": {"
horizon": 3}}):
72                 atomic_json(run/"processed/main/run_status.json", value)
73                 self.assertFalse(reuse_main_baseline(run, "initial_6000", {}, 0, None, None, None,
6000))
74
75
76 if __name__ == "__main__":
77     unittest.main()

```

src/q2/test_dispatch.py

```

1  """最终方案核心验收：手算、未来扰动、MILP精确映射、经验概率和信息边界。"""
2
3  import unittest
4
5  import numpy as np
6

```

```

7 from optimization import build_model, solve_policy
8 from policy import CAP, E_MIN, E_MAX, Policy, replay, respond
9 from scenarios import (construct, cumulative_pressure, dynamic_reserve, eligible_origins,
10                        history_window, joint_blocks, medoids, weighted_quantile)
11
12
13 class PolicyTests(unittest.TestCase):
14     def test_load_priority_and_soc_clipping(self):
15         c, r, q, w, e = respond(1000, E_MIN+100, 0, CAP, CAP)
16         np.testing.assert_allclose((c, r, q, w, e), (0, 90, 910, 0, E_MIN))
17         np.testing.assert_allclose(respond(-1000, E_MAX-90, 0, CAP, CAP), (100, 0, 0, 900, E_MAX))
18         np.testing.assert_allclose(respond(200, 6000, 200, CAP, CAP), (0, 0, 0, 0, 6000))
19
20     def test_frozen_parameters_and_causal_prefix(self):
21         policy = Policy([0, 20, 40, 0], [CAP]*4, [CAP]*4)
22         with self.assertRaises(ValueError):
23             policy.grid[0] = 500
24         first = replay(policy, [500, -200, 600, 800], 6000)
25         changed = replay(policy, [500, -200, -10000, -10000], 6000)
26         for key in first:
27             np.testing.assert_array_equal(first[key][:2], changed[key][:2])
28
29     def test_dynamic_reserve_only_blocks_discharge_and_never_recharges(self):
30         reserve = 200.
31         actual = respond(500, E_MIN+100, 0, CAP, CAP, reserve)
32         np.testing.assert_allclose(actual, (0, 0, 500, 0, E_MIN+100))
33         policy = Policy([0], [CAP], [CAP], [reserve])
34         replayed = replay(policy, [500], E_MIN+100)
35         self.assertEqual(replayed["charge"][0], 0)
36         self.assertEqual(replayed["discharge"][0], 0)
37         self.assertEqual(replayed["emergency"][0], 500)
38
39     def test_indicator_model_and_replay_share_dynamic_reserve(self):
40         policy = Policy([0., 0.], [0., 0.], [CAP, CAP], [200., 0.])
41         net = np.array([[500., 500.]])
42         _, responses, info = solve_policy(
43             net, np.ones(2), np.ones(1), E_MIN+100,
44             fixed_policy=policy, gap=1e-8, time_limit=15)
45         self.assertTrue(info["reliable"], info)
46         expected = replay(policy, net[0], E_MIN+100)
47         for key in expected:
48             np.testing.assert_allclose(responses[0][key], expected[key], atol=1e-5)
49         np.testing.assert_allclose(responses[0]["emergency"], [500., 410.])
50
51     def test_soc_piecewise_execution_boundary(self):
52         cp = np.array([[0, 100, 200]])
53         policy = Policy([0], cp, cp)
54         self.assertEqual(replay(policy, [-300], 4400)["charge"][0], 100)
55         self.assertEqual(replay(policy, [-300], 7600)["charge"][0], 200)
56
57     def test_exact_indicator_mapping_fixed_random_paths(self):
58         random = np.random.default_rng(23)
59         net = random.uniform(-700, 1400, (3, 12))
60         policy = Policy(random.uniform(0, 400, 12), random.uniform(0, CAP, 12), random.uniform(0,
61 CAP, 12))
62         for soc in (E_MIN, 6000, E_MAX):
63             solved, responses, info = solve_policy(net, np.ones(12), np.ones(3)/3, soc,

```

```

63         fixed_policy=policy, gap=1e-8, time_limit=15)
64     self.assertTrue(info["reliable"], info)
65     self.assertLess(info["mapping_error_kwh"], 1e-5)
66     for path, actual in zip(net, responses):
67         expected = replay(policy, path, soc)
68         for key in expected:
69             np.testing.assert_allclose(actual[key], expected[key], atol=1e-5)
70
71     def test_no_storage_optimizer_is_eighty_percent_quantile(self):
72         net = np.array([[0.], [10.], [30.], [100.], [300.]])
73         weights = np.array([.1, .3, .3, .2, .1])
74         model, variables = build_model(net, [1.], weights)
75         model.addCons(variables["cp"][0] == 0)
76         model.addCons(variables["rp"][0] == 0)
77         model.optimize()
78         self.assertEqual(str(model.getStatus()), "optimal")
79         grid = model.getVal(variables["g"][0])
80         self.assertAlmostEqual(grid, 100)
81         self.assertAlmostEqual(model.getObjVal(), 200)
82         model.freeProb()
83
84     def test_unfixed_policy_replay_matches_solver(self):
85         n = np.array([[100, 1000, -500, 300], [150, 500, -100, 400.]])
86         _, _, info = solve_policy(n, [.4, 1, .4, 1], [.5, .5], time_limit=15, gap=1e-8)
87         self.assertTrue(info["reliable"])
88         self.assertAlmostEqual(info["objective"], 1000-CAP)
89         self.assertAlmostEqual(info["objective"], info["optimizer_objective"])
90
91     class ScenarioTests(unittest.TestCase):
92     def test_cumulative_pressure_and_dynamic_reserve_use_kwh_paths(self):
93         blocks = np.zeros((5, 1, 144, 2))
94         blocks[:, 0, 0, 0] = 6*np.arange(5)
95         np.testing.assert_allclose(cumulative_pressure(blocks), np.arange(5))
96         reserve, meta = dynamic_reserve(blocks, .8)
97         self.assertAlmostEqual(reserve[0], 3/.9)
98         np.testing.assert_array_equal(reserve[1:], np.zeros(143))
99         self.assertEqual(meta["historical_blocks"], 5)
100
101     def test_tail_strata_preserve_count_origins_and_probability_mass(self):
102         local = np.zeros((5, 1, 144, 2))
103         all_blocks = np.zeros((10, 1, 144, 2))
104         all_blocks[:, 0, 0, 0] = 6*np.arange(10)
105         net, weights, meta = construct(
106             np.zeros((1, 144, 2)), local, np.ones(144),
107             origins=np.arange(100, 105), all_blocks=all_blocks,
108             all_origins=np.arange(100, 110))
109         self.assertEqual((len(net), meta["S_body"], meta["S_tail"]), (5, 4, 1))
110         self.assertEqual(meta["tail_origin_indices"], [109])
111         self.assertAlmostEqual(weights[-1], .2)
112         self.assertAlmostEqual(weights[:-1].sum(), .8)
113         self.assertAlmostEqual(weights.sum(), 1)
114
115     def test_no_bootstrap_below_forty_and_physical_projection(self):
116         blocks = np.zeros((14, 2, 144, 2))
117         blocks[:, :, :, 0] = np.arange(14)[: , None, None]-20
118         blocks[:, :, :, 1] = 60

```

```

120     net, weights, meta = construct(np.zeros((2, 144, 2)), blocks, np.ones(288), count=10)
121     self.assertEqual(meta["S"], 14)
122     self.assertFalse(meta["reduced"])
123     np.testing.assert_allclose(weights, np.ones(14)/14)
124     np.testing.assert_allclose(net, -10)
125
126     def test_weighted_quantile_uses_discrete_probability(self):
127         self.assertEqual(weighted_quantile(np.array([0., 10, 20]), [.1, .6, .3], .8), 20)
128
129     def test_tail_protection_and_cluster_probability(self):
130         random = np.random.default_rng(19)
131         blocks = random.normal(size=(45, 1, 144, 2))
132         blocks[-1, :, :, 0] = 1000
133         indices, weights, meta = medoids(blocks, np.ones(144), 10)
134         self.assertIn(44, indices)
135         self.assertEqual(len(set(indices)), 10)
136         np.testing.assert_allclose(weights*45, np.bincount(meta["cluster_assignment"]), atol=1e-12,
137 rtol=0)
137         self.assertAlmostEqual(weights.sum(), 1)
138
139     def test_tail_stratification_does_not_increase_reduced_scenario_count(self):
140         rng = np.random.default_rng(20260912)
141         blocks = rng.normal(size=(45, 1, 144, 2))
142         net, weights, meta = construct(np.zeros((1, 144, 2)), blocks, np.ones(144), count=10)
143         self.assertEqual((len(net), len(weights), meta["S"]), (10, 10, 10))
144         self.assertEqual((meta["S_body"], meta["S_tail"]), (8, 2))
145
146     def test_complete_origin_horizon_and_window_fallback(self):
147         shadows = np.full((80, 3, 144, 2), np.nan)
148         shadows[14:45] = 0
149         origins, window = history_window(shadows, 65, 3)
150         self.assertEqual(window, 56)
151         self.assertTrue(np.all(origins+3 <= 65))
152         self.assertEqual(eligible_origins(shadows, 31, 3).tolist(), list(range(14, 29)))
153         history = np.broadcast_to(np.arange(80)[: , None, None], (80, 144, 2)).copy()
154         blocks = joint_blocks(history, shadows, [14, 15], 3)
155         np.testing.assert_array_equal(blocks[0, :, 0, 0], [14, 15, 16])
156
157
158 if __name__ == "__main__":
159     unittest.main()

```

src/q2/test_execution.py

```

1  """精度修订的传递、断点保留和错误协议隔离。"""
2
3  import json
4  from pathlib import Path
5  import tempfile
6  import unittest
7  from unittest.mock import patch
8
9  import numpy as np
10 import pandas as pd
11
12 import optimization

```

```

13 from execution import execution_settings
14 from full_run import worker
15 from policy import Policy, replay
16 from protocol import PROTOCOL, signature
17 from rolling import ComputationalLimit, _solve, run_rollout
18
19
20 class ExecutionTests(unittest.TestCase):
21     def revision(self, root, **changes):
22         value = {"solver_gap": .02, "parent_protocol_sha256": signature(), "authorization": "test"}
23         path = root/"raw/solver_precision_revision.json"
24         path.parent.mkdir(parents=True, exist_ok=True)
25         path.write_text(json.dumps(**value, **changes), encoding="utf-8")
26
27     def test_revision_does_not_mutate_frozen_protocol_and_reaches_worker(self):
28         original = signature()
29         with tempfile.TemporaryDirectory() as tmp:
30             root = Path(tmp)
31             self.assertEqual(execution_settings(root), {})
32             for gap in (.02, .03):
33                 self.revision(root, solver_gap=gap)
34                 settings = execution_settings(root)
35                 with patch("experiments.run_experiments", return_value={"horizon_1": {"complete":
True}}) as run:
36                     self.assertEqual(worker(root, "horizon_1", "a", "receipt.json"), 0)
37                     self.assertEqual(run.call_args.kwargs["solver_gap"], gap)
38                     self.assertEqual(run.call_args.kwargs["execution_revision"], settings["
execution_revision"])
39                 self.assertEqual(PROTOCOL["solver_gap"], .01)
40                 self.assertEqual(signature(), original)
41
42     def test_mismatched_unauthorized_or_nonfinite_revision_is_rejected(self):
43         with tempfile.TemporaryDirectory() as tmp:
44             root = Path(tmp)
45             for change in ({ "parent_protocol_sha256": "wrong"}, {"authorization": ""},
46                             {"solver_gap": float("nan")}, {"solver_gap": .001}, {"solver_gap": True})
47             :
48                 with self.subTest(change=change):
49                     self.revision(root, **change)
50                     with self.assertRaises(ValueError):
51                         execution_settings(root)
52
53     def test_gap_reaches_both_solvers_and_certificate_without_changing_scenarios(self):
54         net = np.ones((2, 3)); prices = np.ones(3); weights = np.array([.4, .6])
55         policy = Policy(np.ones(3), np.zeros(3), np.zeros(3))
56         bound = {"certified_gap": .025, "incumbent_cost": 3., "lower_bound": 2.9}
57         with patch("rolling.deterministic_policy", return_value=(policy, [], {})), \
58             patch("rolling.recourse_bound", return_value=(policy, [], bound)) as lower, \
59             patch("rolling.solve_policy", return_value=(policy, [], {})) as native, \
60             patch("linear_policy.solve_linear_policy", return_value=(policy, [], {})) as linear, \
61             patch("rolling.certify_policy", return_value=(policy, [], {})) as certificate:
62             _solve(net, prices, weights, 6000, solver_gap=.01)
63             self.assertEqual(native.call_args.kwargs["gap"], .01)
64             self.assertIs(native.call_args.args[0], net)
65             _, _, info = _solve(net, prices, weights, 6000, solver_gap=.02)
66             self.assertEqual(linear.call_args.kwargs["time_limit"], 120)

```

```

66         self.assertEqual(info["execution_role"], "
original_budget_equivalent_MILP_under_authorized_precision_revision")
67         _solve(net, prices, weights, 6000, solver_gap=.02, solver_seconds=900)
68         self.assertEqual(linear.call_args.kwargs["gap"], .02)
69         self.assertEqual(linear.call_args.kwargs["time_limit"], 900)
70         self.assertIs(linear.call_args.args[2], weights)
71         _solve(net, prices, weights, 6000, solver_gap=.03)
72         self.assertEqual(lower.call_args.kwargs["target_gap"], .03)
73         self.assertEqual(certificate.call_args.args[-1], .03)
74
75     def test_lp_certificate_fast_path_skips_scip_build_and_keeps_audit(self):
76         net, prices, weights = np.zeros((2, 3)), np.ones(3), np.array([.4, .6])
77         policy = Policy(np.ones(3), np.zeros(3), np.zeros(3))
78         responses = [replay(policy, row, 6000) for row in net]
79         bound = {"certified_gap": .02, "incumbent_cost": 3., "lower_bound": 2.95,
80                 "seconds": .1, "variables": 1, "constraints": 1,
81                 "refinement": None, "refine_seconds_budget": 5.,
82                 "role": "lower_bound_only_never_free_recourse_execution"}
83         with (patch("rolling.deterministic_policy", return_value=(None, None, {})),
84               patch("rolling.recourse_bound", return_value=(policy, responses, bound)),
85                 patch("object.optimization", "build_model", side_effect=AssertionError("SCIP不得构建"))):
86             solved, actual, info = _solve(net, prices, weights, 6000, solver_gap=.03)
87             self.assertIs(solved, policy)
88             self.assertEqual(len(actual), 2)
89             self.assertTrue(info["reliable"] and info["scip_build_skipped"])
90             self.assertEqual(info["certificate_source"], "LP_BOUND")
91             self.assertEqual((info["upper_bound"], info["lower_bound"]), (3., 2.95))
92
93     def test_rescue_reuses_base_bound_and_incumbent(self):
94         net, prices, weights = np.ones((2, 3)), np.ones(3), np.array([.4, .6])
95         bound_policy = Policy(np.ones(3), np.zeros(3), np.zeros(3))
96         candidate = Policy(np.ones(3), np.zeros(3), np.zeros(3))
97         bound = {"certified_gap": .5, "incumbent_cost": 3., "lower_bound": 2.,
98                 "seconds": 5., "refinement": None}
99         cache = {}
100         first_info = {"reliable": False, "status": "time limit", "objective": 3.}
101         second_info = {"reliable": True, "status": "gap reached", "objective": 3.}
102         with (patch("rolling.deterministic_policy", return_value=(None, None, {})),
103               patch("rolling.recourse_bound", return_value=(bound_policy, [], bound)) as lower,
104                 patch("linear_policy.solve_linear_policy",
105                       side_effect=[(candidate, [], first_info), (candidate, [], second_info)]) as
106 solve):
107             _solve(net, prices, weights, 6000, solver_gap=.03,
108                   solve_cache=cache, cache_key="same-day")
109             _, _, rescued = _solve(net, prices, weights, 6000, solver_gap=.03,
110                                   solver_seconds=900, solve_cache=cache,
111                                   cache_key="same-day")
112             self.assertEqual(lower.call_count, 1)
113             self.assertIs(solve.call_args_list[1].kwargs["seed_policy"], candidate)
114             self.assertTrue(rescued["reused_recourse_bound"])
115             self.assertTrue(rescued["reused_incumbent"])
116             self.assertEqual(rescued["pre_solver_seconds"], 0.)
117
118     def test_fixed_and_segmented_policies_retain_native_solver(self):
119         policy = Policy(np.ones(3), np.zeros(3), np.zeros(3))
120         bound = {"certified_gap": .03, "incumbent_cost": 3., "lower_bound": 2.9}
121         with patch("rolling.deterministic_policy", return_value=(policy, [], {})), \

```

```

121         patch("rolling.recourse_bound", return_value=(policy, [], bound)), \
122         patch("rolling.solve_policy", return_value=(policy, [], {})) as native, \
123         patch("linear_policy.solve_linear_policy") as linear:
124     for options in ({"segments": 3}, {"fixed_policy": policy}):
125         _solve(np.ones((2, 3)), np.ones(3), np.array([.4, .6]), 6000,
126                solver_gap=.02, solver_seconds=900, **options)
127     for key, value in options.items():
128         self.assertIs(native.call_args.kwargs[key], value)
129     linear.assert_not_called()
130
131 def test_relaxed_resume_preserves_old_day_and_records_new_gap_and_failures(self):
132     actual = np.zeros((34, 144, 2)); dates = pd.date_range("2025-01-01", periods=34)
133     seen = []
134     def solve(store, frozen, history, d, prices, soc, **settings):
135         gap = settings["solver_gap"]; seconds = settings.get("solver_seconds", 120)
136         seen.append((d, len(history), gap, seconds))
137         if d == 32 and (gap == .01 or seconds == 120):
138             raise ComputationalLimit({"failed_gap": gap})
139         return Policy(np.zeros(144), np.zeros(144), np.zeros(144), np.ones(1), np.zeros((1,
140         144)), np.zeros((1, 144, 2)), {
141             "selected_K": 1, "computational_limited": False,
142             "1": {"M": 14, "selected_solver": {"reliable": True, "requested_gap": gap, "mip_gap"
143 : 0.}}}
144     with tempfile.TemporaryDirectory() as tmp, patch("rolling.make_day", side_effect=solve):
145         root = Path(tmp)
146         with self.assertRaises(ComputationalLimit):
147             run_rollout({}, 0, actual, dates, np.ones(144), 6000, root, rescue_seconds=900)
148         old_audit = (root/"daily_audit/2025-02-01.json").read_bytes()
149         old_plan = (root/"frozen_plans/2025-02-01.csv").read_bytes()
150         seen.clear()
151         frame, _ = run_rollout({}, 0, actual, dates, np.ones(144), 6000, root,
152                                rescue_seconds=900, solver_gap=.02, execution_revision="verified"
153 )
154     self.assertEqual(seen, [(32, 32, .02, 120), (32, 32, .02, 900), (33, 33, .02, 120)])
155     self.assertEqual(frame.date.nunique(), 3)
156     self.assertEqual(old_audit, (root/"daily_audit/2025-02-01.json").read_bytes())
157     self.assertEqual(old_plan, (root/"frozen_plans/2025-02-01.csv").read_bytes())
158     status = json.loads((root/"run_status.json").read_text(encoding="utf-8"))
159     self.assertEqual(status["execution"]["gap"], .02)
160     self.assertEqual(status["execution_history"][0]["execution"]["gap"], .01)
161     self.assertEqual(status["execution_history"][0]["completed_days"], 1)
162     new = json.loads((root/"daily_audit/2025-02-02.json").read_text(encoding="utf-8"))
163     self.assertEqual(new["original_budget_failure"], {"failed_gap": .02})
164     self.assertTrue(list((root/"failed_attempts").glob("previous_status_*.json")))
165     with self.assertRaisesRegex(ValueError, "更严格精度"):
166         run_rollout({}, 0, actual, dates, np.ones(144), 6000, root, rescue_seconds=900)
167
168 def test_second_precision_transition_keeps_both_prior_prefixes(self):
169     actual = np.zeros((34, 144, 2)); dates = pd.date_range("2025-01-01", periods=34)
170     def solve(store, frozen, history, day, prices, soc, **settings):
171         gap = settings["solver_gap"]
172         if (day >= 32 and gap < .02) or (day >= 33 and gap < .03):
173             raise ComputationalLimit({"failed_gap": gap})
174         return Policy(np.zeros(144), np.zeros(144), np.zeros(144), np.ones(1), np.zeros((1,
175         144)), np.zeros((1, 144, 2)), {
176             "selected_K": 1, "computational_limited": False,
177             "1": {"M": 14, "selected_solver": {"reliable": True, "requested_gap": gap, "mip_gap"

```



```

174         : 0.}}}}
175         with tempfile.TemporaryDirectory() as tmp, patch("rolling.make_day", side_effect=solve):
176             root = Path(tmp)
177             for gap in (.01, .02):
178                 with self.assertRaises(ComputationalLimit):
179                     run_rollout({}, 0, actual, dates, np.ones(144), 6000, root,
180                                 solver_gap=gap, execution_revision=str(gap))
181             preserved = {str(path.relative_to(root)): path.read_bytes()
182                          for folder in ("daily_audit", "frozen_plans") for path in (root/folder).
183                          iterdir())
184             frame, _ = run_rollout({}, 0, actual, dates, np.ones(144), 6000, root,
185                                 solver_gap=.03, execution_revision="three-percent")
186             self.assertEqual(frame.date.nunique(), 3)
187             for name, raw in preserved.items():
188                 self.assertEqual((root/name).read_bytes(), raw)
189                 status = json.loads((root/"run_status.json").read_text(encoding="utf-8"))
190                 self.assertEqual(status["execution"]["gap"], .03)
191                 self.assertEqual([x["execution"]["gap"] for x in status["execution_history"]], [.01,
192                                                                                               .02])
193                 self.assertEqual([x["completed_days"] for x in status["execution_history"]], [1, 2])
194
195 if __name__ == "__main__":
196     unittest.main()

```

src/q2/test_forecast_dispatch.py

```

1  """新版预测与外部输出的独立验收：所有合成数据只存在于测试临时目录。"""
2
3  import json
4  from pathlib import Path
5  import tempfile
6  import unittest
7  from unittest.mock import patch
8
9  import numpy as np
10 import pandas as pd
11 from openpyxl import load_workbook
12
13 from comparators import deterministic_policy, oracle_pair, rigid_schedule
14 from data import R00T
15 from export_dispatch import (emergency_events, export_result, final_calibration,
16                              final_validation, solver_statistics, validate_dispatch)
17 from forecast import economic_metrics, harmonic_forecast, lightgbm_forecast, load_features, seasonal
18 , select_pipeline
19 from optimization import solve_policy
20 from policy import CAP, E_MIN, Policy, replay
21 from protocol import PROTOCOL
22 from relaxation import recourse_bound
23 from incumbent import refine
24 from rolling import (ComputationalLimit, _exclusive_run_lock, dispatch_frame, make_day,
25                     run_rollout, warm_start)
26 from shadows import freeze_lightgbm
27 from incumbent import value_gradient
28 from checkpoint import read_checkpoint, recover_checkpoint, save_checkpoint
29 from dispatch import (forecast_hashes, load_prepared, prepared_hashes,

```

```

29         validate_prepared_protocol)
30 from data import write_json
31 from protocol import signature
32 from experiments import load_richer_policy, richer_context, save_richer_policy
33 from final_report import visual_review_current
34 from data import sha256
35
36
37 def history_fixture(days=14):
38     slot = np.arange(144)
39     history = np.zeros((days, 144, 2))
40     for d in range(days):
41         history[d, :, 0] = 1000+100*(d%7)+200*np.cos(2*np.pi*slot/144)
42         history[d, :, 1] = (300+d)*np.maximum(np.sin(2*np.pi*(slot-36)/144), 0)
43     return history
44
45
46 class RicherCheckpointTests(unittest.TestCase):
47     def fixture(self, output):
48         audit, plan = output/"reference.json", output/"reference.csv"
49         audit.write_text("{} ", encoding="utf-8"); plan.write_text("reference", encoding="utf-8")
50         net, weights, prices = np.zeros((2, 144)), np.array([.5, .5]), np.ones(144)
51         policy = Policy(np.zeros(144), np.tile([100., 200., 300.], (144, 1)),
52                        np.tile([300., 200., 100.], (144, 1)))
53         context = richer_context(net, weights, prices, 6000, audit, plan)
54         record = {"date": "2025-03-20", "K": 1, "S": 2,
55                 "solver": {"reliable": True, "objective": 0., "dual_bound": 0.}}
56         return policy, record, context, net, prices, weights
57
58     def test_richer_policy_roundtrip_replays_and_checks_global_certificate(self):
59         with tempfile.TemporaryDirectory() as tmp:
60             output = Path(tmp)
61             policy, record, context, net, prices, weights = self.fixture(output)
62             save_richer_policy(output, record, policy, context)
63             recovered, saved = load_richer_policy(output, record["date"], context, net, prices,
64             weights, 6000)
65             np.testing.assert_array_equal(recovered.charge_cap, policy.charge_cap)
66             np.testing.assert_array_equal(recovered.discharge_cap, policy.discharge_cap)
67             self.assertEqual(saved["solver"]["objective"], 0.)
68             saved["solver"]["objective"] = 10.
69             write_json(output/f"richer_soc/{record['date']}.json", saved)
70             with self.assertRaises(AssertionError):
71                 load_richer_policy(output, record["date"], context, net, prices, weights, 6000)
72
73     def test_richer_cache_rejects_wrong_scenarios_and_changed_policy_bytes(self):
74         with tempfile.TemporaryDirectory() as tmp:
75             output = Path(tmp)
76             policy, record, context, net, prices, weights = self.fixture(output)
77             saved = save_richer_policy(output, record, policy, context)
78             with self.assertRaisesRegex(ValueError, "不一致"):
79                 load_richer_policy(output, record["date"], {**context, "scenario_input_sha256": "
80 changed"}, net, prices, weights, 6000)
81             path = output/saved["policy_file"]
82             path.write_text(path.read_text(encoding="utf-8")+"\n", encoding="utf-8")
83             with self.assertRaisesRegex(ValueError, "摘要不符"):
84                 load_richer_policy(output, record["date"], context, net, prices, weights, 6000)

```

```

84     def test_richer_interrupted_new_receipt_preserves_previous_complete_policy(self):
85         with tempfile.TemporaryDirectory() as tmp:
86             output = Path(tmp)
87             policy, record, context, net, prices, weights = self.fixture(output)
88             original = save_richer_policy(output, record, policy, context)
89             with patch("experiments.atomic_json", side_effect=OSError("simulated interrupted commit"
90 ))):
91                 with self.assertRaises(OSError):
92                     save_richer_policy(output, record, policy, context)
93             _, restored = load_richer_policy(output, record["date"], context, net, prices, weights,
94 6000)
95             self.assertEqual(restored["policy_file"], original["policy_file"])
96
97     class ArtifactAcceptanceTests(unittest.TestCase):
98         def test_regenerated_plot_cannot_inherit_previous_visual_approval(self):
99             with tempfile.TemporaryDirectory() as tmp:
100                 output = Path(tmp)
101                 (output/"figures").mkdir()
102                 png = output/"figures/real_rollout.png"
103                 png.write_bytes(b"synthetic previously reviewed image fixture")
104                 write_json(output/"figures/manifest.json", {"figures": [{"real_rollout"}]})
105                 write_json(output/"figures/visual_acceptance.json", {
106                     "source_sha256": sha256(ROOT/"src/plots/q2_dispatch.py"),
107                     "reviewed_png_sha256": {"real_rollout": sha256(png)}})
108                 self.assertTrue(visual_review_current(output))
109                 png.write_bytes(b"synthetic regenerated different image fixture")
110                 self.assertFalse(visual_review_current(output))
111
112     class ForecastTests(unittest.TestCase):
113         def test_recursive_seasonal_uses_its_own_previous_prediction(self):
114             history = history_fixture()
115             predicted = seasonal(history, 3)
116             np.testing.assert_array_equal(predicted[0, :, 0], history[-7, :, 0])
117             np.testing.assert_allclose(predicted[1, :, 1], np.mean(np.r_[history[-6:, :, 1], predicted
118 [0:1, :, 1]], axis=0))
119             np.testing.assert_allclose(predicted[2, :, 1], np.mean(np.r_[history[-5:, :, 1], predicted
120 [:2, :, 1]], axis=0))
121
122         def test_exact_eight_load_features_and_actual_candidate(self):
123             history = history_fixture()
124             features = load_features(history[:, :, 0], "2025-01-15")
125             self.assertEqual(features.shape, (144, 8))
126             np.testing.assert_array_equal(features[:, 3:6], history[[-1, -2, -7], :, 0].T)
127             expected = history.copy()
128             prediction = lightgbm_forecast(history, "2025-01-15", 3, PROTOCOL["lightgbm_candidates"][0])
129             self.assertEqual(prediction.shape, (3, 144))
130             self.assertTrue(np.isfinite(prediction).all())
131             np.testing.assert_array_equal(history, expected)
132
133         def test_dhr_orders_and_diagnostics_use_fixed_dictionary(self):
134             prediction, diagnostics, correlation = harmonic_forecast(history_fixture(), 3)
135             self.assertEqual(prediction.shape, (3, 144))
136             self.assertTrue((prediction >= 0).all())
137             self.assertFalse(diagnostics["annual_harmonics"])
138             self.assertEqual(len(diagnostics["candidates"]), 8)

```

```

137     self.assertEqual(len(correlation), 145)
138     self.assertEqual([v["lag"] for v in diagnostics["ljung_box"]], [6, 144])
139
140     def test_freezing_cannot_use_february_hyperparameter_performance(self):
141         actual = np.zeros((40, 144, 2))
142         store = {"load_lgb_0": np.zeros((40, 3, 144)), "load_lgb_1": np.ones((40, 3, 144))}
143         store["load_lgb_0"][31:] = 1e9
144         self.assertEqual(freeze_lightgbm(store, actual)[0], 0)
145         actual[31:] = 1e12
146         self.assertEqual(freeze_lightgbm(store, actual)[0], 0)
147
148     def test_january_storage_is_inactive_and_february_soc_is_6000(self):
149         actual = np.zeros((365, 144, 2))
150         dates = pd.date_range("2025-01-01", periods=365)
151         with tempfile.TemporaryDirectory() as directory:
152             initial = warm_start(actual, dates, np.ones(144), Path(directory))
153             record = json.loads((Path(directory)/"warm_start.json").read_text(encoding="utf-8"))
154             self.assertEqual(initial, 6000.)
155             self.assertEqual(record["febl_baseline_soc"], 6000.)
156             self.assertEqual(record["january_charge_kwh"], 0.)
157             self.assertEqual(record["january_discharge_kwh"], 0.)
158
159     def test_four_to_one_loss_and_lexicographic_selection(self):
160         actual = np.array([[6., 0.], [6., 0.]])
161         prediction = np.array([[0., 0.], [12., 0.]])
162         result = economic_metrics(actual, prediction, np.ones(2))
163         self.assertEqual(result["econ"], 2.5)
164         scores = {"baseline": dict(result), "complex": {**result, "econ": 3., "net_mae": 0}}
165         self.assertEqual(select_pipeline(scores), "baseline")
166
167
168     class ComparatorTests(unittest.TestCase):
169         def test_incumbent_gradient_matches_finite_differences_away_from_kinks(self):
170             net = np.array([[100., 400, 450, 100], [90., 350, 400, 120]])
171             vector = np.r_[np.full(4, 200.), np.full(4, 80.), np.full(4, 30.)]
172             prices, weights = np.array([.4, 1, .8, .5]), np.array([.5, .5])
173             _, gradient = value_gradient(vector, net, prices, weights, 5000)
174             finite = [(value_gradient(vector+step*1e-4, net, prices, weights, 5000)[0]-
175                       value_gradient(vector-step*1e-4, net, prices, weights, 5000)[0])/2e-4 for step in
176                       np.eye(len(vector))]
177             np.testing.assert_allclose(gradient, finite, atol=1e-6, rtol=1e-6)
178
179         def test_richer_gradient_uses_actual_half_open_soc_band(self):
180             net = np.array([[700., -900, 1000, -500], [600., -600, 900, -400]])
181             grid = np.full(4, 200.)
182             cp = np.tile([120., 400., 500.], (4, 1))
183             rp = np.tile([180., 450., 600.], (4, 1))
184             vector = np.r_[grid, cp.ravel(), rp.ravel()]
185             prices, weights = np.array([.4, 1, .8, .5]), np.array([.5, .5])
186             value, gradient = value_gradient(vector, net, prices, weights, 4480)
187             exact = prices@grid+sum(5*p*(prices@replay(Policy(grid, cp, rp), path, 4480)["emergency"])
188             for p, path in zip(weights, net))
189             self.assertEqual(value, exact)
190             finite = [(value_gradient(vector+step*1e-4, net, prices, weights, 4480)[0]-
191                       value_gradient(vector-step*1e-4, net, prices, weights, 4480)[0])/2e-4 for step in
192                       np.eye(len(vector))]
193             np.testing.assert_allclose(gradient, finite, atol=1e-5, rtol=1e-5)

```

```

191
192 def test_deterministic_certificate_matches_exact_model(self):
193     net, prices = np.array([1000., -900, 700, 200]), np.array([1., .4, 1., .4])
194     _, _, certificate = deterministic_policy(net, prices, E_MIN)
195     _, _, exact = solve_policy(net[None], prices, np.array([1.]), E_MIN, time_limit=15, gap=1e
196     -8)
197     self.assertTrue(certificate["reliable"])
198     self.assertAlmostEqual(certificate["objective"], exact["objective"], places=5)
199
200 def test_lower_bound_is_not_executed_and_oracles_are_nested(self):
201     net = np.array([[1000., -900, 700, 200], [1300, -400, 900, 100]])
202     prices, weights = np.array([1., .4, 1., .4]), np.array([.4, .6])
203     policy, responses, bound = recourse_bound(net, prices, weights, E_MIN)
204     _, _, exact = solve_policy(net, prices, weights, E_MIN, time_limit=15, gap=1e-8)
205     self.assertLessEqual(bound["lower_bound"], exact["objective"]+1e-5)
206     self.assertLessEqual(exact["objective"], bound["incumbent_cost"]+1e-5)
207     real_cost = float(prices@(policy.grid+5*responses[0]["emergency"]))
208     oracle = oracle_pair(net[0], prices, E_MIN, policy.grid, real_cost)
209     self.assertGreaterEqual(oracle["Gap_resp"], -1e-8)
210
211 def test_refine_budget_is_explicitly_forwarded(self):
212     rng = np.random.default_rng(314)
213     net = rng.uniform(-900, 1500, (3, 8))
214     prices, weights = np.ones(8), np.ones(3)/3
215     with (patch("relaxation.candidate_costs",
216               side_effect=lambda *args: np.full(len(args[4]), 1e9)),
217          patch("relaxation.refine", wraps=refine) as improve):
218         recourse_bound(net, prices, weights, 6000, target_gap=0,
219                       refine_seconds=.05)
220     self.assertTrue(improve.called)
221     self.assertEqual(improve.call_args.kwargs["seconds"], .05)
222
223 def test_richer_policy_contains_main_policy_at_soc_boundaries(self):
224     base = Policy([20, 0, 0], [300]*3, [200]*3)
225     expanded = Policy(base.grid, np.repeat(base.charge_cap[:, None], 3, axis=1),
226                      np.repeat(base.discharge_cap[:, None], 3, axis=1))
227     net = np.array([[300., -200, 700], [200, -300, 800]])
228     _, responses, info = solve_policy(net, np.ones(3), [.5, .5], 4400,
229                                     segments=3, fixed_policy=expanded, time_limit=15)
230     self.assertTrue(info["reliable"])
231     for path, actual in zip(net, responses):
232         expected = replay(base, path, 4400)
233         for key in expected:
234             np.testing.assert_allclose(actual[key], expected[key], atol=1e-5)
235
236 class ExportTests(unittest.TestCase):
237     def test_final_validation_calibration_and_solver_statistics(self):
238         policy = Policy(np.zeros(144), np.zeros(144), np.zeros(144), np.full(144, 100.))
239         frame = dispatch_frame("2025-02-01", policy, np.zeros(144), 6000, np.ones(144))
240         frame["predicted_load_kw"] = 0.
241         frame["predicted_pv_kw"] = 0.
242         validation = final_validation(frame)
243         self.assertTrue(validation["overall_pass"])
244         self.assertEqual(validation["reserve_discharge_violation_count"], 0)
245         actual = np.zeros((365, 144, 2))
246         coverage = pd.DataFrame([

```

```

247         {"horizon": 0, "nominal": .8, "covered_intervals": 120, "n": 144},
248         {"horizon": 0, "nominal": .9, "covered_intervals": 130, "n": 144})
249     table, payload = final_calibration(frame, actual, coverage)
250     self.assertEqual(len(table), 8)
251     self.assertAlmostEqual(payload["coverage"]["PICP80"]["value"], 120/144)
252     daily = pd.DataFrame({"runtime_seconds": [2., 4.], "achieved_mip_gap": [.01, .02],
253                          "requested_mip_gap": [.03, .03]})
254     stats = solver_statistics(daily)
255     self.assertEqual(stats["total_runtime"], 6.)
256     self.assertEqual(stats["days_completed"], 2)
257
258     def test_cross_midnight_events_and_empty_events(self):
259         times = pd.date_range("2025-02-01 23:50", periods=4, freq="10min")
260         frame = pd.DataFrame({"timestamp": times, "emergency": [0., 1., 2., 0.], "emergency_cost":
261                               [0., 5., 10., 0.]})
262         events = emergency_events(frame)
263         self.assertEqual(len(events), 1)
264         self.assertEqual(events.emergency_kwh.iloc[0], 3)
265         self.assertEqual(events.interval.iloc[0], "23:50-00:10+1日")
266         frame["emergency"] = 0
267         self.assertTrue(emergency_events(frame).empty)
268
269     def test_complete_template_roundtrip_and_partial_rejection(self):
270         dates = pd.date_range("2025-02-01", "2025-12-31")
271         # 全期合成零负荷用来核验布局/日期/单位, 绝不写入正式运行目录。
272         policy = Policy(np.zeros(144), np.zeros(144), np.zeros(144))
273         frame = pd.concat([dispatch_frame(d, policy, np.zeros(144), 6000, np.ones(144)) for d in
274                             dates], ignore_index=True)
275         with self.assertRaises(ValueError):
276             validate_dispatch(frame.iloc[:144], full_period=True)
277         with tempfile.TemporaryDirectory() as directory:
278             output = Path(directory)
279             export_result(frame, ROOT/"docs/CUMCM2026Problems/C题/附件/附件5/result2.xlsx", output)
280             wb = load_workbook(output/"result2.xlsx", read_only=True, data_only=True)
281             self.assertEqual(wb["计划购电量"].max_row, 335)
282             self.assertEqual(wb["计划购电量"]["B1"].value, "00:00-00:10")
283             self.assertEqual(wb["计划购电量"].cell(1, 145).value, "23:50-24:00")
284             self.assertEqual(wb["充放电量"].max_row, 2005)
285             wb.close()
286             self.assertEqual(json.loads((output/"template_audit.json").read_text(encoding="utf-8"))["
287                                 numeric_roundtrip"])
288
289     class RollingTests(unittest.TestCase):
290     def test_interrupted_csv_commit_recovers_complete_old_or_new_checkpoint(self):
291         for failed_file in ("dispatch.csv", "daily.csv", "calibration.csv", "checkpoint.json"):
292             with self.subTest(failed_file=failed_file), tempfile.TemporaryDirectory() as directory:
293                 output = Path(directory)
294                 first = pd.DataFrame({"value": [1]})
295                 second = pd.DataFrame({"value": [1, 2]})
296                 save_checkpoint(output, first, [{"day": 1}], [{"score": 1}])
297                 replace = Path.replace
298                 def interrupted(path, target):
299                     result = replace(path, target)
300                     if target == output/failed_file:
301                         raise InterruptedError("模拟在文件替换后进程中断")
302                     return result

```

```

301         with patch.object(Path, "replace", interrupted), self.assertRaises(InterruptedError)
302         :
303             save_checkpoint(output, second, [{"day": 1}, {"day": 2}], [{"score": 1}, {"score
304             ": 2}])
305             recover_checkpoint(output)
306             expected = 2 if failed_file == "checkpoint.json" else 1
307             manifest = json.loads((output/"checkpoint.json").read_text(encoding="utf-8"))
308             self.assertEqual(manifest["days"], expected)
309             for name in ("dispatch.csv", "daily.csv", "calibration.csv"):
310                 self.assertEqual(len(pd.read_csv(output/name)), expected)
311             self.assertFalse((output/"checkpoint_transaction.json").exists())
312
313     def test_interrupted_first_day_leaves_no_false_checkpoint(self):
314         with tempfile.TemporaryDirectory() as directory:
315             output = Path(directory)
316             replace = Path.replace
317             def interrupted(path, target):
318                 result = replace(path, target)
319                 if target == output/"daily.csv":
320                     raise InterruptedError("首次提交中断")
321                 return result
322             with patch.object(Path, "replace", interrupted), self.assertRaises(InterruptedError):
323                 save_checkpoint(output, pd.DataFrame({"value": [1]}), [{"day": 1}], [{"score": 1}])
324                 recover_checkpoint(output)
325                 self.assertFalse((output/"dispatch.csv").exists())
326                 self.assertFalse((output/"checkpoint.json").exists())
327
328     def test_supplementary_compute_only_after_entire_day_failed(self):
329         actual = np.zeros((33, 144, 2)); dates = pd.date_range("2025-01-01", periods=33)
330         seen = []
331         def solver(store, frozen, history, d, prices, soc, **settings):
332             budget = settings.get("solver_seconds", 120)
333             seen.append((d, len(history), budget))
334             if d == 32 and budget == 120:
335                 raise ComputationalLimit({"original_failed": True})
336             return Policy(np.zeros(144), np.zeros(144), np.zeros(144)), np.ones(1), np.zeros((1,
337             144)), np.zeros((1, 144, 2)), {
338                 "selected_K": 1, "computational_limited": False,
339                 "l": {"M": 14, "selected_solver": {"reliable": True}}}
340         with tempfile.TemporaryDirectory() as directory:
341             output = Path(directory)
342             with patch("rolling.make_day", side_effect=solver):
343                 result, _ = run_rollout({}, 0, actual, dates, np.ones(144), 6000, output,
344                 rescue_seconds=900)
345             self.assertEqual(seen, [(31, 31, 120), (32, 32, 120), (32, 32, 900)])
346             self.assertEqual(result.date.nunique(), 2)
347             audit = json.loads((output/"daily_audit/2025-02-02.json").read_text(encoding="utf-8"))
348             self.assertTrue(audit["computational_limited"])
349             self.assertEqual(audit["original_budget_failure"], {"original_failed": True})
350             self.assertTrue(list((output/"failed_attempts").glob("*.json")))
351
352     def test_failed_supplement_does_not_skip_date_or_erase_previous_day(self):
353         actual = np.zeros((34, 144, 2)); dates = pd.date_range("2025-01-01", periods=34)
354         seen = []
355         def solver(store, frozen, history, d, prices, soc, **settings):
356             seen.append(d)
357             if d != 31:

```



```

354         raise ComputationalLimit({"all_candidates_failed": True})
355     return Policy(np.zeros(144), np.zeros(144), np.zeros(144)), np.ones(1), np.zeros((1,
144)), np.zeros((1, 144, 2)), {
356         "selected_K": 1, "computational_limited": False,
357         "1": {"M": 14, "selected_solver": {"reliable": True}}
358     with tempfile.TemporaryDirectory() as directory:
359         output = Path(directory)
360         with patch("rolling.make_day", side_effect=solver), self.assertRaises(ComputationalLimit
):
361             run_rollout({}, 0, actual, dates, np.ones(144), 6000, output, rescue_seconds=900)
362             self.assertEqual(seen, [31, 32, 32])
363             self.assertEqual(len(pd.read_csv(output/"dispatch.csv")), 144)
364             self.assertEqual(json.loads((output/"checkpoint.json").read_text(encoding="utf-8"))["
days"], 1)
365
366     def test_same_experiment_cannot_have_two_writers(self):
367         with tempfile.TemporaryDirectory() as directory:
368             output = Path(directory)
369             with _exclusive_run_lock(output/"run.lock"):
370                 with self.assertRaisesRegex(RuntimeError, "已有写入进程"):
371                     run_rollout({}, 0, None, None, None, None, output)
372
373     def test_prepared_generated_data_tampering_is_rejected_before_loading(self):
374         with tempfile.TemporaryDirectory() as directory:
375             output = Path(directory)
376             write_json(output/"raw/protocol.json", {"protocol_sha256": signature(), "
forecast_source_sha256": forecast_hashes()})
377             write_json(output/"inputs/processed/source_manifest.json", {})
378             data = output/"inputs/processed/timeseries.csv"
379             data.write_text("load_kw,pv_kw\n1,2\n", encoding="utf-8")
380             write_json(output/"raw/prepared_manifest.json", {"sha256": prepared_hashes(output)})
381             data.write_text("load_kw,pv_kw\n9,2\n", encoding="utf-8")
382             with self.assertRaisesRegex(ValueError, "摘要改变"):
383                 load_prepared(output)
384
385     def test_prepared_forecasts_remain_reusable_when_only_dispatch_model_changes(self):
386         old = {"protocol_sha256": "old-model-signature",
387             "forecast_source_sha256": {**forecast_hashes(), "scenarios.py": "old",
388                 "policy.py": "old", "protocol.py": "old"},
389             **{key: PROTOCOL[key] for key in
390                 ("history_boundary", "forecast_horizons", "shadow_start",
391                     "lightgbm_candidates", "lightgbm_training_window",
392                     "lightgbm_freeze_metric", "dhr_harmonics", "dhr_ar_orders",
393                     "dhr_ma_orders", "dhr_history_days", "dhr_order_rule",
394                     "dhr_max_iterations", "dhr_difference_order")}}
395         validate_prepared_protocol(old)
396         old["dhr_history_days"] += 1
397         with self.assertRaisesRegex(ValueError, "预测冻结参数"):
398             validate_prepared_protocol(old)
399
400     def test_real_selection_metadata_serializes_numpy_origin_indices(self):
401         actual = np.zeros((34, 144, 2))
402         store = {key: np.zeros((34, 3, 144)) for key in
403             ("load_week", "load_lgb_0", "pv_mean7", "pv_dhr")}
404         for values in store.values():
405             values[:14] = np.nan
406         _, _, _, _, audit = make_day(store, 0, actual[:31], 31, np.ones(144), 6000)

```



```

407     serialized = json.loads(json.dumps(audit))
408     self.assertEqual(serialized["selected_K"], 2)
409     self.assertTrue(all(j < 31 for j in serialized["2"]["selection"]["common_scored_origins"]))
410
411     def test_resume_replays_frozen_policy_and_rejects_corrupted_checkpoint(self):
412         actual = np.zeros((33, 144, 2)); dates = pd.date_range("2025-01-01", periods=33)
413         seen = []
414         def day_solver(store, frozen, history, d, prices, soc, **settings):
415             seen.append((d, len(history)))
416             policy = Policy(np.zeros(144), np.zeros(144), np.zeros(144), np.full(144, 100.))
417             return policy, np.ones(1), np.zeros((1, 144)), np.zeros((1, 144, 2)), {
418                 "selected_K": 1, "computational_limited": False,
419                 "1": {"M": 14, "selected_solver": {"reliable": True}}
420         with tempfile.TemporaryDirectory() as directory:
421             output = Path(directory)
422             with patch("rolling.make_day", side_effect=day_solver):
423                 first, _ = run_rollout({}, 0, actual, dates, np.ones(144), 6000, output)
424                 self.assertEqual(seen, [(31, 31), (32, 32)])
425                 with patch("rolling.make_day", side_effect=AssertionError("已完成前缀不得重新优化")):
426                     resumed, _ = run_rollout({}, 0, actual, dates, np.ones(144), 6000, output)
427                     np.testing.assert_array_equal(resumed.grid, first.grid)
428                     np.testing.assert_array_equal(resumed.soc_reserve_kwh, first.soc_reserve_kwh)
429                     changed = pd.read_csv(output/"dispatch.csv")
430                     changed.loc[144:, "initial_soc"] = 7000
431                     changed.to_csv(output/"dispatch.csv", index=False, encoding="utf-8")
432                     with self.assertRaisesRegex(ValueError, "摘要不符"):
433                         read_checkpoint(output, actual, dates, np.ones(144), 6000, 31, 33, {})
434                     (output/"checkpoint.json").unlink()
435                     with self.assertRaisesRegex(ValueError, "跨日SOC"):
436                         read_checkpoint(output, actual, dates, np.ones(144), 6000, 31, 33, {})
437
438     def test_execution_cannot_call_optimizer_and_prefix_cannot_see_future(self):
439         policy = Policy(np.zeros(144), np.full(144, 100.), np.full(144, 100.))
440         net = np.zeros(144); net[:2] = [200, -200]
441         with patch("rolling.solve", side_effect=AssertionError("日内不能求解")):
442             first = dispatch_frame("2025-02-01", policy, net, 6000, np.ones(144))
443             net[:2] = 1e9
444             second = dispatch_frame("2025-02-01", policy, net, 6000, np.ones(144))
445             pd.testing.assert_frame_equal(first.iloc[:2], second.iloc[:2])
446
447
448 if __name__ == "__main__":
449     unittest.main()

```

src/q2/test_full_run.py

```

1  """全量队列的故障隔离、重启识别与完成收据；合成数据只在临时目录。"""
2
3  import tempfile
4  import unittest
5  from pathlib import Path
6  from unittest.mock import patch
7
8  from checkpoint import atomic_json
9  from full_run import (DEPENDENT, LEGACY_OR_DEVELOPMENT_ONLY, eligible,
10                        experiment_category, guard_memory, job_names, read, reconcile, worker)

```

```

11
12
13 class FullRunTests(unittest.TestCase):
14     def test_repeated_crash_isolated_after_three_attempts(self):
15         with tempfile.TemporaryDirectory() as tmp:
16             job = {"status": "running", "attempt": "crash", "receipt": "absent.json", "pid": 123}
17             with patch("full_run.owned_process", return_value=False):
18                 for i in range(3):
19                     job["status"] = "running"
20                     reconcile(Path(tmp), {"main": job})
21                     self.assertEqual(job["status"], "pending" if i < 2 else "failed")
22
23     def test_memory_guard_only_stops_largest_verified_worker(self):
24         import subprocess
25         with tempfile.TemporaryDirectory() as tmp:
26             jobs = {name: {"status": "running", "attempt": name, "receipt": name+".json", "pid": i
27 +100}
28                     for i, name in enumerate(("small", "large", "unrelated"))}
29             with patch("full_run.owned_process", side_effect=lambda job: job["attempt"] != "
30 unrelated"), \
31                 patch("full_run.subprocess.run", side_effect=[subprocess.CompletedProcess([], 0,
32 "5000000"),
33                                     subprocess.CompletedProcess([], 0,
34 "6000000")]), \
35                 patch("full_run.terminate_worker") as kill:
36                 guard_memory(Path(tmp), jobs)
37                 self.assertEqual(kill.call_args.args[0], 101)
38                 self.assertEqual(kill.call_count, 1)
39                 self.assertFalse((Path(tmp)/"small.json").exists())
40                 self.assertFalse(read(Path(tmp)/"large.json")["complete"])
41
42     def test_failed_main_keeps_independent_jobs_and_blocks_dependencies(self):
43         self.assertEqual(job_names(6000.), ["main"])
44         self.assertEqual(experiment_category("main"), "required_for_paper")
45         self.assertEqual(experiment_category("forecast"), "optional_diagnostics")
46         self.assertEqual(experiment_category("horizon_1"), "legacy_or_development_only")
47         jobs = {name: {"status": "pending"} for name in job_names(
48 6000., include_optional=True, include_legacy=True)}
49         self.assertTrue(set(LEGACY_OR_DEVELOPMENT_ONLY).issubset(jobs))
50         jobs["main"]["status"] = "failed"
51         jobs["window_28"]["requires_main"] = True
52         jobs["initial_6000"]["requires_main"] = True
53         self.assertTrue(eligible(jobs))
54         self.assertFalse(DEPENDENT.intersection(eligible(jobs)))
55         self.assertNotIn("window_28", eligible(jobs))
56         self.assertNotIn("initial_6000", eligible(jobs))
57         self.assertIn("initial_1200", eligible(jobs))
58         jobs["main"]["status"] = "complete"
59         self.assertTrue(DEPENDENT.issubset(eligible(jobs)))
60         self.assertIn("window_28", eligible(jobs))
61
62     def test_restart_preserves_live_worker_and_requeues_only_interrupted_job(self):
63         with tempfile.TemporaryDirectory() as tmp:
64             jobs = {name: {"status": "running", "attempt": name, "receipt": name+".json", "pid": i
65 +1}
66                     for i, name in enumerate(("live", "interrupted", "failed", "done"))}
67             for name, complete in (("failed", False), ("done", True)):

```

```

63         atomic_json(Path(tmp)/(name+".json"), {"attempt": name, "job": name, "complete":
complete,
64                                     "finished": "test", "exit_code": 0 if
complete else 2})
65         with patch("full_run.owned_process", side_effect=lambda job: job["attempt"] == "live"):
66             reconcile(Path(tmp), jobs)
67             self.assertEqual([jobs[name]["status"] for name in jobs], ["running", "pending", "failed
", "complete"])
68
69     def test_failure_receipt_does_not_claim_success_or_prevent_next_job(self):
70         with tempfile.TemporaryDirectory() as tmp:
71             path = Path(tmp)
72             with patch("experiments.run_experiments", return_value={"horizon_1": {"complete": False
}}):
73                 self.assertEqual(worker(path, "horizon_1", "a", "receipt/a.json"), 2)
74                 self.assertFalse(read(path/"receipt/a.json")["complete"])
75                 with patch("experiments.run_experiments", return_value={"horizon_2": {"complete": True
}}) as run:
76                     self.assertEqual(worker(path, "horizon_2", "b", "receipt/b.json"), 0)
77                     self.assertIsNone(run.call_args.kwargs["rescue_seconds"])
78                     self.assertTrue(read(path/"receipt/b.json")["complete"])
79
80
81 if __name__ == "__main__":
82     unittest.main()

```

src/q2/test_horizon_execution.py

```

1     """延迟K1必须保留全部可选结果下的旧选择规则与失败隔离。"""
2
3     import itertools
4     import unittest
5     from unittest.mock import patch
6
7     import numpy as np
8
9     from policy import Policy
10    from rolling import ComputationalLimit, make_day
11
12
13    class HorizonExecutionTests(unittest.TestCase):
14        def test_every_candidate_availability_and_stability_case(self):
15            for flags in itertools.product((False, True), repeat=3):
16                for stable in (False, True):
17                    available = {k for k, reliable in enumerate(flags, 1) if reliable}
18                    called = []
19                    policies = {k: Policy(np.full(k*144, 1. if k != 2 or stable else 2.),
20                                         np.zeros(k*144), np.zeros(k*144)) for k in (1, 2, 3)}
21
22                    def choose(store, frozen, history, day, k, prices, **settings):
23                        return np.zeros((k, 144, 2)), None, {"common_scored_origins": []}
24
25                    def solve(point, blocks, prices, soc, **settings):
26                        k = len(point); called.append(k)
27                        if k not in available:
28                            raise ComputationalLimit({"K": k})

```

```

29         return policies[k], [{"emergency": np.zeros(k*144)}], np.ones(1), np.zeros((1, k
    *144)), {
30             "M": 14, "selected_solver": {"reliable": True}, "computational_limited":
    False}
31
32     with self.subTest(available=available, stable=stable), \
33         patch("rolling.choose_day", side_effect=choose), \
34         patch("rolling.select_resolution", side_effect=solve):
35         if not available:
36             with self.assertRaises(ComputationalLimit):
37                 make_day({}, 0, None, 31, np.ones(144), 6000)
38         else:
39             policy, _, _, _, audit = make_day({}, 0, None, 31, np.ones(144), 6000)
40             expected = 2 if {2, 3}.issubset(available) and stable else max(available)
41             self.assertIs(policy, policies[expected])
42             self.assertEqual(audit["selected_K"], expected)
43             self.assertEqual(audit["computational_limited"], not {2, 3}.issubset(
    available))
44             self.assertEqual("skipped_K1" in audit, bool(available.intersection({2, 3}))
    )
45             self.assertEqual(called, [2, 3] if available.intersection({2, 3}) else [2, 3,
    1])
46
47     def test_fixed_K1_is_still_executed_for_sensitivity(self):
48         policy = Policy(np.ones(144), np.zeros(144), np.zeros(144))
49         with patch("rolling.choose_day", return_value=(np.zeros((1, 144, 2)), None, {"
    common_scored_origins": []})) as choose, \
50             patch("rolling.select_resolution", return_value=(policy, [{"emergency": np.zeros
    (144)}],
51                 np.ones(1), np.zeros((1, 144)), {"M": 14, "selected_solver": {"reliable": True
    }}})):
52             result = make_day({}, 0, None, 31, np.ones(144), 6000, horizon=1)
53             self.assertEqual(choose.call_args.args[4], 1)
54             self.assertIs(result[0], policy)
55             self.assertNotIn("skipped_K1", result[-1])
56
57
58 if __name__ == "__main__":
59     unittest.main()

```

src/q2/test_linear_policy.py

```

1  """有界线性MILP与原生indicator模型及原始min/max映射的独立等价验收。"""
2
3  import unittest
4
5  import numpy as np
6
7  from linear_policy import build_linear_policy, linear_seed, solve_linear_policy
8  from optimization import solve_policy
9  from policy import CAP, E_MIN, E_MAX, ETA_C, ETA_D, Policy, replay
10
11
12 class LinearPolicyTests(unittest.TestCase):
13     def test_undercharging_matrix_paths_project_to_original_feedback_without_cost_increase(self):
14         rng = np.random.default_rng(2719)

```

```

15     difference = 0.
16     for soc in (E_MIN, 1538.0615848778116, 6000., E_MAX):
17         for _ in range(8):
18             net, prices = rng.uniform(-2200, 2500, (3, 144)), rng.uniform(.2, 1.2, 144)
19             policy = Policy(rng.uniform(0, CAP, 144), np.full(144, CAP), rng.uniform(0, CAP,
144))
20             objective, bounds, integer, constraints, indices = build_linear_policy(
21                 net, prices, np.full(3, 1/3), soc)
22             vector = np.zeros(len(objective))
23             for key, values in (("g", policy.grid), ("cp", policy.charge_cap), ("rp", policy.
discharge_cap)):
24                 vector[indices[key]] = values
25             for j, path in enumerate(net):
26                 previous, raw = soc, []
27                 for i, n in enumerate(path):
28                     x = n-policy.grid[i]
29                     vector[indices["reserve_active"][j, i]] = int(previous >= E_MIN+policy.
reserve[i])
30                     if x >= 0:
31                         terms = [policy.discharge_cap[i], x, ETA_D*(previous-E_MIN)]
32                         c, r = 0., min(terms)
33                         q, w = x-r, 0.
34                         vector[indices["z"][j, i]] = 1
35                         vector[indices["lr"][int(np.argmax(terms))][j, i]] = 1
36                     else:
37                         c = rng.uniform(0, 1)*min(policy.charge_cap[i], -x, (E_MAX-previous)/
ETA_C)
38                         r, q, w = 0., 0., -x-c
39                         previous += ETA_C*c-r/ETA_D
40                         raw.append([c, r, q, w, previous])
41                 raw = np.asarray(raw)
42                 for columns, values in zip(indices["responses"], raw.T):
43                     vector[columns[j]] = values
44                 exact = replay(policy, path, soc)
45                 self.assertTrue(np.all(exact["soc"] >= raw[:, 4]-1e-5))
46                 self.assertTrue(np.all(exact["emergency"] <= raw[:, 2]+1e-5))
47                 self.assertLessEqual(prices@exact["emergency"], prices@raw[:, 2]+1e-5)
48                 difference = max(difference, float(np.max(np.abs(exact["soc"]-raw[:, 4]))))
49                 activity = constraints.A@vector
50                 self.assertTrue(np.all(activity <= constraints.ub+1e-5))
51                 self.assertTrue(np.all(activity >= constraints.lb-1e-5))
52                 self.assertTrue(np.all(vector >= bounds.lb-1e-5))
53                 self.assertTrue(np.all(vector <= bounds.ub+1e-5))
54                 np.testing.assert_array_equal(vector[integer == 1], np rint(vector[integer == 1]))
55                 self.assertGreater(difference, 1.)
56
57     def test_dynamic_reserve_linear_native_and_replay_are_equivalent(self):
58         net = np.array([[500., 300., 900., -200.], [700., 100., 600., -300.]])
59         prices, weights = np.array([.4, .9, .7, .3]), np.array([.4, .6])
60         reserve = np.array([300., 200., 100., 0.])
61         initial = E_MIN+250
62         linear_policy, linear_responses, linear = solve_linear_policy(
63             net, prices, weights, initial, reserve=reserve, gap=1e-7, time_limit=15)
64         native_policy, native_responses, native = solve_policy(
65             net, prices, weights, initial, reserve=reserve, gap=1e-7, time_limit=15)
66         self.assertTrue(linear["reliable"] and native["reliable"])
67         np.testing.assert_allclose(linear["objective"], native["objective"], atol=1e-4, rtol=1e-7)

```

```

68     np.testing.assert_array_equal(linear_policy.reserve, reserve)
69     np.testing.assert_array_equal(native_policy.reserve, reserve)
70     for policy, responses in ((linear_policy, linear_responses), (native_policy,
native_responses)):
71         for path, response in zip(net, responses):
72             expected = replay(policy, path, initial)
73             for key in expected:
74                 np.testing.assert_allclose(response[key], expected[key], atol=1e-5)
75
76 def test_monotone_certificate_is_distinct_from_fixed_policy_exact_mapping(self):
77     net = np.array([[-600., 900., 400.], [-300., 1100., 200.]])
78     prices, weights = np.array([.3, .9, .7]), np.array([.4, .6])
79     policy, _, free = solve_linear_policy(net, prices, weights, E_MIN, time_limit=15)
80     self.assertTrue(free["reliable"] and free["mapped_policy_cost_not_above_raw_objective"])
81     self.assertTrue(free["monotone_charge_bound"])
82     self.assertIsNone(free["mapping_error_kwh"])
83     for key in ("monotone_projection_error_kwh", "linear_feasibility_error", "integrality_error"
):
84         self.assertLessEqual(free[key], 1e-5)
85     _, _, fixed = solve_linear_policy(net, prices, weights, E_MIN, fixed_policy=policy,
time_limit=15)
86     self.assertTrue(fixed["reliable"])
87     self.assertFalse(fixed["monotone_charge_bound"])
88     self.assertIsNone(fixed["monotone_projection_error_kwh"])
89     self.assertLessEqual(fixed["mapping_error_kwh"], 1e-5)
90
91 def test_branch_cuts_preserve_complete_fixed_policy_trajectories(self):
92     rng = np.random.default_rng(7216)
93     for soc in (E_MIN, 1538.0615848778116, 6000., E_MAX):
94         for _ in range(8):
95             net = rng.uniform(-2200, 2500, (3, 144))
96             policy = Policy(*[rng.uniform(0, CAP, 144) for _ in range(3)])
97             objective, bounds, _, constraints, indices = build_linear_policy(
98                 net, np.ones(144), np.full(3, 1/3), soc, fixed_policy=policy)
99             vector = linear_seed(policy, net, soc, indices, len(objective))
100             activity = constraints.A@vector
101             self.assertTrue(np.all(activity <= constraints.ub+1e-5))
102             self.assertTrue(np.all(activity >= constraints.lb-1e-5))
103             self.assertTrue(np.all(vector >= bounds.lb-1e-5))
104             self.assertTrue(np.all(vector <= bounds.ub+1e-5))
105
106 def test_maximum_charge_cap_dominates_for_each_realized_step(self):
107     rng = np.random.default_rng(20260912)
108     for soc in (E_MIN, 1462.9246853535594, 6000., E_MAX):
109         for _ in range(8):
110             net, grid = rng.uniform(-2000, 2500, 144), rng.uniform(0, 1200, 144)
111             original = Policy(grid, rng.uniform(0, CAP, 144), rng.uniform(0, CAP, 144))
112             raised = Policy(grid, np.full(144, CAP), original.discharge_cap)
113             before, after = replay(original, net, soc), replay(raised, net, soc)
114             self.assertTrue(np.all(after["soc"] >= before["soc"]-1e-5))
115             self.assertTrue(np.all(after["emergency"] <= before["emergency"]+1e-5))
116
117 def test_dominance_presolve_transforms_seed_but_preserves_fixed_policy(self):
118     net = np.array([[-600., 900., 400.], [-300., 1100., 200.]])
119     prices, weights = np.array([.3, .9, .7]), np.array([.4, .6])
120     seed = Policy(np.zeros(3), np.zeros(3), np.full(3, CAP))
121     optimized, _, info = solve_linear_policy(net, prices, weights, E_MIN,

```

```

122         seed_policy=seed, time_limit=15)
123     self.assertTrue(info["reliable"] and info["maximum_charge_cap_dominance_presolve"])
124     np.testing.assert_array_equal(optimized.charge_cap, np.full(3, CAP))
125     fixed, responses, fixed_info = solve_linear_policy(net, prices, weights, E_MIN,
126                                                         fixed_policy=seed, seed_policy=seed,
time_limit=15)
127     self.assertTrue(fixed_info["reliable"])
128     self.assertFalse(fixed_info["maximum_charge_cap_dominance_presolve"])
129     np.testing.assert_array_equal(fixed.charge_cap, seed.charge_cap)
130     for path, response in zip(net, responses):
131         np.testing.assert_allclose(response["emergency"], replay(seed, path, E_MIN)["emergency"],
132                                     atol=1e-5)
133
134     def test_optima_match_indicator_model_at_three_initial_states(self):
135         rng = np.random.default_rng(31)
136         prices, weights = np.array([.3, .9, .8, .2, .7, .4]), np.array([.2, .3, .5])
137         for soc in (E_MIN, 6000., E_MAX):
138             with self.subTest(initial_soc=soc):
139                 net = rng.uniform(-900, 1500, (3, 6))
140                 _, _, linear = solve_linear_policy(net, prices, weights, soc, gap=1e-7, time_limit
141                                                     =15)
142                 _, _, native = solve_policy(net, prices, weights, soc, gap=1e-7, time_limit=15)
143                 self.assertTrue(linear["reliable"] and native["reliable"])
144                 np.testing.assert_allclose(linear["objective"], native["objective"], atol=1e-4, rtol
145                                             =1e-7)
146
147     def test_complete_causal_seed_satisfies_every_linear_row(self):
148         net = np.array([[-600., 900., 0., 1200.], [400., -300., -900., 500.]])
149         seed = Policy(np.array([50., 100., 0., 200.]), np.full(4, 400.), np.full(4, 700.))
150         _, _, info = solve_linear_policy(net, np.array([.3, .9, .2, .7]), np.array([.4, .6]), E_MAX,
151                                         seed_policy=seed, time_limit=15)
152         self.assertTrue(info["seed_verified"] and info["reliable"])
153         self.assertEqual(info["solver_version"], "1.8.0")
154         _, _, native = solve_policy(net, np.array([.3, .9, .2, .7]), np.array([.4, .6]), E_MAX,
155                                     time_limit=15)
156         np.testing.assert_allclose(info["objective"], native["objective"], atol=1e-5, rtol=0)
157
158     def test_fixed_policy_exactly_matches_clipping_and_has_no_emergency_charging(self):
159         net = np.array([[-600., -100., 400., 1100., 200., -900.], [200., 400., -300., 700., -1200.,
160 0.]])
161         policy = Policy(np.array([50., 70., 200., 300., 50., 200.]),
162                        np.array([200., 0., 400., 200., 800., 600.]), np.array([150., 300., 700.,
163 150., 0., 700.]))
164         for soc in (E_MIN, E_MAX):
165             _, responses, info = solve_linear_policy(net, np.ones(6), np.array([.5, .5]), soc,
166                                                 fixed_policy=policy, gap=1e-7, time_limit=15)
167             self.assertTrue(info["reliable"])
168             for path, response in zip(net, responses):
169                 expected = replay(policy, path, soc)
170                 for key in expected:
171                     np.testing.assert_allclose(response[key], expected[key], atol=1e-5, rtol=0)
172                 self.assertFalse((response["emergency"] > 1e-5)&(response["charge"] > 1e-5)).any())
173                 self.assertLess(info["mapping_error_kwh"], 1e-5)
174
175     def test_fixed_policy_cannot_invalidate_derived_activation_bounds(self):
176         policy = Policy(np.full(2, 10000.), np.zeros(2), np.zeros(2))
177         with self.assertRaisesRegex(ValueError, "物理上界"):

```

```

172         solve_linear_policy(np.zeros((1, 2)), np.ones(2), np.ones(1), 6000., fixed_policy=policy
173     )
174
175 if __name__ == "__main__":
176     unittest.main()

```

src/q2/test_q2.py

```

1  """真实附件验收 + 手算/合成数据检验；合成结果不作为实际预测交付。"""
2
3  import os
4  from pathlib import Path
5  import tempfile
6  import unittest
7  from unittest.mock import patch
8
9  import numpy as np
10 import pandas as pd
11 from openpyxl import load_workbook
12
13 import data
14 from data import (CALENDAR, FORECAST_START, ROOT, SERIES, SHEETS, calendar_for_day,
15                  endpoint_minutes, history_before, jump_context, read_attachment)
16 from backtest import dispatch_metrics, forecast_metrics, rolling_folds, rolling_predict
17 from diagnostics import (correlation, describe, lag_analysis, seasonal_transforms, spectral_analysis
18                          ,
19                          stability_analysis, stationarity_analysis)
20 from report import candidate_models
21
22 class Q2Tests(unittest.TestCase):
23     @classmethod
24     def setUpClass(cls):
25         cls.tmp = tempfile.TemporaryDirectory()
26         cls.path = ROOT / "docs/CUMCM2026Problems/C题/附件/附件2.xlsx"
27         cls.frame, cls.qc = read_attachment(cls.path, Path(cls.tmp.name) / "quality.csv")
28         workbook = load_workbook(cls.path, read_only=True, data_only=True)
29         cls.rows = {s.title: list(s.iter_rows(values_only=True)) for s in workbook}
30         workbook.close()
31
32     @classmethod
33     def tearDownClass(cls):
34         cls.tmp.cleanup()
35
36     def test_original_values_and_quality(self):
37         self.assertTrue(self.qc.status.eq("pass").all())
38         self.assertEqual(len(self.frame), 52560)
39         for variable, name in SHEETS.items():
40             original = np.array([row[1:] for row in self.rows[name][1:]], dtype=float).ravel()
41             np.testing.assert_array_equal(original, self.frame[variable])
42             np.testing.assert_array_equal(self.frame.net_load_kw, self.frame.load_kw - self.frame.pv_kw)
43             self.assertGreater(int((self.frame.pv_kw == 0).sum()), 0)
44             self.assertGreater(int((self.frame.net_load_kw < 0).sum()), 0)
45
46     def test_right_endpoint_and_calendar(self):

```



```

47     first, last = self.frame.iloc[0], self.frame.iloc[-1]
48     self.assertEqual(first.timestamp, pd.Timestamp("2025-01-01 00:10"))
49     self.assertEqual(last.timestamp, pd.Timestamp("2026-01-01"))
50     self.assertEqual(last.date, pd.Timestamp("2025-12-31"))
51     self.assertEqual(last.interval, "23:50-24:00")
52     self.assertEqual(self.frame.iloc[143].weekday, 3)
53     self.assertTrue(self.frame.timestamp.diff().dropna().eq(pd.Timedelta(minutes=10)).all())
54     for value in ("0:00+1", "24:00", 1.0):
55         self.assertEqual(endpoint_minutes(value), 1440)
56     self.assertEqual(endpoint_minutes(1/144), 10)
57     for value in ("0:00", "24:10", "0:11", float("nan"), True):
58         with self.assertRaises((ValueError, TypeError)):
59             endpoint_minutes(value)
60
61     def assert_bad_workbook(self, mutate):
62         rows = {name: [list(row) for row in values] for name, values in self.rows.items()}
63         mutate(rows)
64         class Sheet:
65             def __init__(self, values):
66                 self.values = values
67             def iter_rows(self, **kwargs):
68                 return iter(self.values)
69         class Book:
70             sheetnames = list(rows)
71             def __getitem__(self, key):
72                 return Sheet(rows[key])
73             def close(self):
74                 pass
75         with patch.object(data, "load_workbook", return_value=Book()):
76             with self.assertRaises(ValueError):
77                 read_attachment(self.path, Path(self.tmp.name) / "bad.csv")
78             self.assertIn("fail", pd.read_csv(Path(self.tmp.name) / "bad.csv").status.values)
79
80     def test_missing_and_nonfinite_rejected(self):
81         for invalid in (None, float("nan"), float("inf"), "unreadable", True):
82             with self.subTest(invalid=invalid):
83                 self.assert_bad_workbook(lambda rows: rows["小区负载"][10].__setitem__(8, invalid))
84
85     def test_negative_physical_values_rejected(self):
86         for sheet in SHEETS.values():
87             self.assert_bad_workbook(lambda rows: rows[sheet][10].__setitem__(8, -1))
88
89     def test_duplicate_and_missing_dates_rejected(self):
90         self.assert_bad_workbook(lambda rows: rows["小区负载"][3].__setitem__(0, rows["小区负载"]
91         ][2][0]))
92         self.assert_bad_workbook(lambda rows: rows["小区负载"].pop())
93
94     def test_duplicate_missing_and_wrong_endpoints_rejected(self):
95         self.assert_bad_workbook(lambda rows: rows["光伏发电实际功率"][0].__setitem__(5, rows["光伏发电
96         实际功率"][0][4]))
97         self.assert_bad_workbook(lambda rows: rows["光伏发电实际功率"][0].__setitem__(144, "0:00"))
98
99     def test_missing_sheet_rejected(self):
100         self.assert_bad_workbook(lambda rows: rows.pop("光伏发电实际功率"))
101
102     def test_jump_review_preserves_spikes_and_uses_past_peers(self):
103         part = self.frame.iloc[:3000].copy()

```

```

102     part.loc[2016, "load_kw"] = 1000000.
103     before = part.copy(deep=True)
104     context = jump_context(part)
105     pd.testing.assert_frame_equal(part, before)
106     row = context.loc[(context.series == "load_kw") & (context.timestamp == part.loc[2016, "
timestamp"])].iloc[0]
107     self.assertEqual(row.value_kw, 1000000.)
108     self.assertEqual(row.previous_week_kw, part.loc[1008, "load_kw"])
109     self.assertEqual(row.past_same_weekday_count, 2)
110     self.assertEqual(row.past_same_weekday_max, max(part.loc[0, "load_kw"], part.loc[1008, "
load_kw"]))
111     self.assertEqual(row.action, "retain_review_only")
112
113     def test_strict_midnight_and_334_folds(self):
114         folds = rolling_folds(self.frame)
115         self.assertEqual(len(folds), 334)
116         self.assertEqual(folds.iloc[0].history_points, 4463)
117         self.assertEqual(folds.iloc[0].history_last_timestamp, pd.Timestamp("2025-01-31 23:50"))
118         self.assertTrue((folds.history_last_timestamp < folds.origin).all())
119         self.assertTrue(folds.horizon_points.eq(144).all())
120         self.assertEqual(folds.iloc[-1].target_last, pd.Timestamp("2026-01-01"))
121         with self.assertRaises(ValueError):
122             history_before(self.frame, "2025-02-01 00:10")
123         with self.assertRaises(ValueError):
124             rolling_folds(self.frame.iloc[:-1])
125
126     @staticmethod
127     def mean_factory(variable):
128         def predict(history, future):
129             assert set(history) == {"timestamp", "value_kw"}
130             assert tuple(future.columns) == CALENDAR
131             assert history.timestamp.max() < future.date.iloc[0]
132             return np.full(144, history.value_kw.mean())
133         return predict
134
135     def test_future_and_midnight_perturbation_cannot_change_prediction(self):
136         a = rolling_predict(self.frame, self.mean_factory, end=FORECAST_START)
137         altered = self.frame.copy()
138         mask = altered.timestamp >= FORECAST_START
139         altered.loc[mask, "load_kw"] += 10000
140         altered.loc[mask, "pv_kw"] += 20000
141         altered["net_load_kw"] = altered.load_kw - altered.pv_kw
142         b = rolling_predict(altered, self.mean_factory, end=FORECAST_START)
143         for v in SERIES:
144             np.testing.assert_array_equal(a[f"predicted_{v}"], b[f"predicted_{v}"])
145         self.assertFalse(a.actual_load_kw.equals(b.actual_load_kw))
146         # 反向检查: 可见历史改变, 预测应改变; 避免恒定假预测使防泄漏测试虚假通过。
147         altered.loc[altered.timestamp < FORECAST_START, "load_kw"] += 500
148         c = rolling_predict(altered, self.mean_factory, end=FORECAST_START)
149         np.testing.assert_allclose(c.predicted_load_kw - a.predicted_load_kw, 500)
150
151     def test_new_model_per_variable_per_day(self):
152         calls = []
153         def factory(variable):
154             calls.append(variable)
155             return self.mean_factory(variable)
156         result = rolling_predict(self.frame, factory, end="2025-02-02")

```

```

157     self.assertEqual(calls, ["load_kw", "pv_kw"] * 2)
158     self.assertEqual(len(result), 288)
159     np.testing.assert_array_equal(result.predicted_net_load_kw, result.predicted_load_kw -
    result.predicted_pv_kw)
160
161     def test_invalid_predictions_rejected(self):
162         for values in (np.ones(143), np.full(144, np.nan), np.full(144, -1)):
163             with self.assertRaises(ValueError):
164                 rolling_predict(self.frame, lambda _: lambda past, future: values, end=
    FORECAST_START)
165
166     def test_forecast_metrics_hand_calculation(self):
167         frame = calendar_for_day("2025-02-01")
168         for v, actual, predicted in (("load_kw", 10, np.tile([12, 8], 72)),
169                                     ("pv_kw", 4, np.tile([5, 2], 72))):
170             frame[f"actual_{v}"] = actual
171             frame[f"predicted_{v}"] = predicted
172         frame["actual_net_load_kw"] = 6
173         frame["predicted_net_load_kw"] = frame.predicted_load_kw - frame.predicted_pv_kw
174         all_metrics = forecast_metrics(frame).query("period == 'all'").set_index("series")
175         self.assertEqual(all_metrics.loc["load_kw", "mae_kw"], 2)
176         self.assertEqual(all_metrics.loc["pv_kw", "mae_kw"], 1.5)
177         self.assertAlmostEqual(all_metrics.loc["pv_kw", "rmse_kw"], np.sqrt(2.5))
178         self.assertEqual(all_metrics.loc["net_load_kw", "mae_kw"], .5)
179         self.assertAlmostEqual(all_metrics.loc["net_load_kw", "rmse_kw"], np.sqrt(.5))
180         frame.loc[0, "predicted_net_load_kw"] = 999
181         with self.assertRaises(ValueError):
182             forecast_metrics(frame)
183
184     def test_dispatch_metrics_plan_billing_and_fivefold_emergency(self):
185         frame = calendar_for_day("2025-02-01")
186         frame["planned_grid_kwh"], frame["price_yuan_per_kwh"] = 2., .5
187         frame["actual_load_kw"], frame["actual_supply_before_emergency_kw"] = 12., 12.
188         frame.loc[[0, 1, 3], "actual_supply_before_emergency_kw"] = [0., 6., 0.]
189         m = dispatch_metrics(frame)
190         self.assertEqual(m["emergency_kwh"], 5)
191         self.assertEqual(m["emergency_intervals"], 3)
192         self.assertEqual(m["emergency_events"], 2)
193         self.assertEqual(m["days_with_emergency"], 1)
194         self.assertEqual(m["planned_cost_yuan"], 144)
195         self.assertEqual(m["emergency_cost_yuan"], 12.5)
196         self.assertEqual(m["total_cost_yuan"], 156.5)
197         self.assertEqual(m["cost_period_label"], "evaluated_period_only")
198         frame.actual_supply_before_emergency_kw = 999
199         self.assertEqual(dispatch_metrics(frame)["total_cost_yuan"], 144) # 多余供给不能退款。
200         frame.loc[0, "actual_supply_before_emergency_kw"] = -6.
201         self.assertEqual(dispatch_metrics(frame)["emergency_kwh"], 3.) # 充电可能令紧急补足前净供给为
    负。
202         with self.assertRaises(ValueError):
203             dispatch_metrics(frame.iloc[:-1])
204
205     def test_cross_midnight_emergency_is_one_contiguous_event(self):
206         frame = pd.concat([calendar_for_day(d) for d in ("2025-02-01", "2025-02-02")], ignore_index=
    True)
207         frame["planned_grid_kwh"], frame["price_yuan_per_kwh"] = 0., 1.
208         frame["actual_load_kw"], frame["actual_supply_before_emergency_kw"] = 0., 0.
209         frame.loc[[143, 144], "actual_load_kw"] = 6.

```

```

210     m = dispatch_metrics(frame)
211     self.assertEqual(m["emergency_events"], 1)
212     self.assertEqual(m["days_with_emergency"], 2)
213
214     def test_transforms_match_four_term_identity(self):
215         x = np.arange(3000, dtype=float) ** 2
216         transforms = seasonal_transforms(x)
217         self.assertEqual(transforms["diff144"].notna().sum(), 3000-144)
218         np.testing.assert_array_equal(transforms["diff144_then1008"].dropna(),
219                                     x[1152:] - x[1008:-144] - x[144:-1008] + x[:-1152])
220
221     def test_spectrum_known_daily_weekly_peaks_and_parseval(self):
222         frame = pd.concat([calendar_for_day(d) for d in pd.date_range("2025-01-01", periods=28)],
223                           ignore_index=True)
224         t = np.arange(len(frame))/144
225         frame["load_kw"] = 10 + 2*np.cos(2*np.pi*t) + np.cos(2*np.pi*t/7)
226         frame["pv_kw"] = 5 + 2*np.cos(2*np.pi*t)
227         _, targets = spectral_analysis(frame, "test")
228         load = targets.query("series == 'load_kw' and form == 'raw'")
229         self.assertTrue(load.local_peak.all())
230         np.testing.assert_allclose(load.integrated_psd_kw2, 2.5, atol=1e-12)
231         np.testing.assert_allclose(targets.integrated_psd_kw2, targets.population_variance_kw2, atol=1e-12)
232
233     def test_acf_and_pacf_computed_through_weekly_lags(self):
234         part = history_before(self.frame, FORECAST_START)
235         keys, curves = lag_analysis(part, "initial_history")
236         self.assertEqual(curves.groupby("series").lag.max().tolist(), [2016, 2016])
237         x = part.load_kw.to_numpy()
238         centered = x-x.mean()
239         actual = curves.query("series == 'load_kw' and lag == 1008").acf.iloc[0]
240         self.assertEqual(actual, np.dot(centered[1008:], centered[:-1008])/np.dot(centered,
241                                     centered))
242         self.assertTrue(np.isfinite(curves.pacf).all())
243         r = keys.query("series == 'load_kw' and form == 'raw' and lag == 144").pearson_r.iloc[0]
244         self.assertEqual(r, np.corrcoef(x[144:], x[:-144])[0, 1])
245
246     def test_monthly_and_weekday_profiles_and_stability(self):
247         profiles, daily, summary, zeros = describe(self.frame)
248         self.assertEqual(len(profiles.query("grouping == 'month'")), 12*144*3)
249         self.assertEqual(len(profiles.query("grouping == 'weekday'")), 7*144*3)
250         self.assertEqual(len(daily), 365*3)
251         january = self.frame.loc[self.frame.month == 1]
252         stability = stability_analysis(self.frame, "full_year")
253         row = stability.query("month == 1 and series == 'load_kw'").iloc[0]
254         self.assertEqual(row.weekly_pairs, 31*144-1008)
255         self.assertEqual(row.weekly_lag_r, correlation(january.load_kw.to_numpy()[1008:],
256                                     january.load_kw.to_numpy()[:-1008]))
257         self.assertEqual(zeros.zero_points.sum(), int((self.frame.pv_kw == 0).sum()))
258
259     def test_constant_series_not_reported_as_significant(self):
260         part = self.frame.iloc[:2000].copy()
261         part.load_kw, part.pv_kw = 1., 0.
262         tests, _, _ = stationarity_analysis(part, "test")
263         self.assertTrue(tests.status.eq("undefined_constant_or_short").all())
264         self.assertTrue(np.isnan(correlation([1, 1], [2, 2])))

```

```

263 def test_candidate_guard_rejects_full_year_information(self):
264     with self.assertRaises(ValueError):
265         candidate_models({"lag_correlations": pd.DataFrame({"scope": ["full_year"]}),
266                           "spectral_targets": pd.DataFrame({"scope": ["full_year"]})})
267
268 def test_small_weekly_peak_alone_does_not_prioritize_multiseason_model(self):
269     history = history_before(self.frame, FORECAST_START)
270     lags, _ = lag_analysis(history, "initial_history")
271     _, targets = spectral_analysis(history, "initial_history")
272     candidates = candidate_models({"lag_correlations": lags, "spectral_targets": targets})
273     pv = candidates.query("series == 'pv_kw' and model == 'DHR_AR'").iloc[0]
274     self.assertEqual(pv.status, "条件保留, 不预设周效应")
275     self.assertTrue(candidates.fitted.eq(False).all())
276
277 @unittest.skipIf(os.name == "nt", "旧2.1发布者依赖目录符号链接; 新最终模型使用独立运行目录")
278 def test_failed_publish_keeps_current_generation(self):
279     import run
280     root = Path(self.tmp.name) / "atomic"
281     previous, following = root / "outputs/.q2-runs/previous", root / "outputs/.q2-runs/following"
282
283     for p in (previous, following):
284         p.mkdir(parents=True, exist_ok=True)
285     with patch.object(run, "ROOT", root):
286         run.publish(previous)
287         with patch.object(run.os, "replace", side_effect=OSError("injected publish failure")):
288             with self.assertRaises(OSError):
289                 run.publish(following)
290     self.assertEqual((root / "outputs/q2_current").resolve(), previous.resolve())
291     with patch.object(run, "ROOT", root):
292         run.publish(following)
293     self.assertEqual((root / "outputs/q2_current").resolve(), following.resolve())
294
295 def test_generated_artifacts_roundtrip(self):
296     generation = os.environ.get("Q2_GENERATION")
297     if not generation:
298         self.skipTest("仅完整流水线设置Q2_GENERATION后执行产物回读; 其余测试独立运行")
299     path = Path(generation)
300     saved = pd.read_csv(path / "inputs/processed/timeseries.csv", float_precision="round_trip")
301     for variable in SERIES:
302         np.testing.assert_array_equal(saved[variable], self.frame[variable])
303     keys = pd.read_csv(path / "processed/full_year_lag_correlations.csv")
304     self.assertEqual(len(keys), 24)
305     self.assertEqual(len(pd.read_csv(path / "processed/rolling_folds.csv")), 334)
306     for scope in ("full_year", "initial_history"):
307         tests = pd.read_csv(path / f"processed/{scope}_stationarity_tests.csv")
308         self.assertEqual(len(tests), 32)
309         self.assertTrue(tests.status.eq("ok").all())
310     self.assertEqual(len(list((path / "processed/figures").glob("*.png"))), 11)
311     self.assertEqual(len(list((path / "processed/figures").glob("*.svg"))), 11)
312     trace = pd.read_csv(path / "processed/source_traceability.csv")
313     source = (ROOT / "docs/2/dierwen1.md").read_text(encoding="utf-8").splitlines()
314     self.assertEqual(set(trace.source_line), {i for i, line in enumerate(source, 1) if line.
315 strip()})
316
317 if __name__ == "__main__":
318     unittest.main(verbosity=2)

```

src/q2/traceability.py

```
1 """原方案每个非空内容源行/句子均关联实际函数位置、测试和产物。"""
2
3 import ast
4 import re
5
6 import pandas as pd
7
8 from data import ROOT, write_csv
9
10 MAPPING = {
11     "2.1": ("run.py:run_pipeline", "test_generated_artifacts_roundtrip", "report.html;results.md"),
12     "2.1.1": ("data.py:history_before;backtest.py:rolling_predict", "
13     test_future_and_midnight_perturbation_cannot_change_prediction", "rolling_folds.csv"),
14     "2.1.2": ("data.py:read_attachment;data.py:calendar_for_day", "test_original_values_and_quality",
15     "timeseries.csv"),
16     "2.1.3": ("data.py:endpoint_minutes;data.py:calendar_for_day;backtest.py:dispatch_metrics", "
17     test_right_endpoint_and_calendar", "timeseries.csv;rolling_folds.csv"),
18     "2.1.4": ("data.py:read_attachment;data.py:jump_context", "test_original_values_and_quality;
19     test_missing_and_nonfinite_rejected;test_negative_physical_values_rejected;
20     test_duplicate_and_missing_dates_rejected;test_duplicate_missing_and_wrong_endpoints_rejected",
21     "quality_checks.csv;jump_context.csv;full_year_pv_zero_profile.csv"),
22     "2.1.5": ("data.py:read_attachment;diagnostics.py:describe;backtest.py:forecast_metrics", "
23     test_original_values_and_quality;test_forecast_metrics_hand_calculation", "timeseries.csv;
24     full_year_daily_statistics.csv;full_year_profiles.csv"),
25     "2.1.6": ("diagnostics.py:describe;../plots/q2.py:create_figures", "
26     test_monthly_and_weekday_profiles_and_stability;test_generated_artifacts_roundtrip", "figures/
27     timeseries_7d.png;figures/timeseries_30d.png;figures/annual_heatmaps.png;figures/
28     monthly_profiles.png;figures/weekday_profiles.png;figures/weekday_weekend.png"),
29     "2.1.7": ("diagnostics.py:lag_analysis", "test_acf_and_pacf_computed_through_weekly_lags", "
30     full_year_lag_correlations.csv;initial_history_lag_correlations.csv"),
31     "2.1.8": ("diagnostics.py:lag_analysis;../plots/q2.py:create_figures", "
32     test_acf_and_pacf_computed_through_weekly_lags", "full_year_acf_pacf.csv;figures/acf_pacf.png"),
33     "2.1.9": ("diagnostics.py:spectral_analysis;../plots/q2.py:create_figures", "
34     test_spectrum_known_daily_weekly_peaks_and_parseval", "full_year_spectrum.csv;
35     full_year_spectral_targets.csv;figures/spectra.png"),
36     "2.1.10": ("diagnostics.py:stability_analysis;../plots/q2.py:create_figures", "
37     test_monthly_and_weekday_profiles_and_stability", "full_year_monthly_stability.csv;figures/
38     monthly_stability.png"),
39     "2.1.11": ("diagnostics.py:seasonal_transforms;diagnostics.py:stationarity_analysis;../plots/q2.
40     py:create_figures", "test_transforms_match_four_term_identity;
41     test_constant_series_not_reported_as_significant;test_generated_artifacts_roundtrip", "
42     full_year_stationarity_tests.csv;full_year_transformed_series.csv;
43     full_year_transformed_monthly_moments.csv;figures/seasonal_moments.png"),
44     "2.1.12": ("backtest.py:rolling_folds;backtest.py:rolling_predict", "
45     test_strict_midnight_and_334_folds;test_new_model_per_variable_per_day;
46     test_invalid_predictions_rejected;test_future_and_midnight_perturbation_cannot_change_prediction",
47     "rolling_folds.csv"),
48     "2.1.13": ("backtest.py:forecast_metrics;backtest.py:dispatch_metrics;report.py:create_report", "
49     test_forecast_metrics_hand_calculation;
50     test_dispatch_metrics_plan_billing_and_fifefold_emergency;
51     test_cross_midnight_emergency_is_one_contiguous_event", "evaluation_status.csv;report.html"),
52     "2.1.14": ("report.py:candidate_models;report.py:result_text", "
53     test_candidate_guard_rejects_full_year_information", "model_candidates.csv;results.md"),
54     "2.1.15": ("run.py:run_pipeline", "test_generated_artifacts_roundtrip", "report.html;results.md"
55     ),
56 }
```

```

27 }
28
29
30 def generate_traceability(output):
31     positions = {}
32     for path in [(ROOT / "src/q2").glob("*.py"), ROOT / "src/plots/q2.py"]:
33         tree = ast.parse(path.read_text(encoding="utf-8"))
34         for node in ast.walk(tree):
35             if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
36                 relative = str(path.relative_to(ROOT / "src"))
37                 key = "../plots/q2.py" if relative == "plots/q2.py" else path.name
38                 positions[f"{key}:{node.name}"] = f"src/{relative}:{node.lineno}-{node.end_lineno}"
39 rows, section = [], None
40 for line_number, line in enumerate((ROOT / "docs/2/dierwen1.md").read_text(encoding="utf-8").
splitlines(), 1):
41     match = re.search(r"^\#{2,3}\s+(2\.1(?:\.\d+)?)", line)
42     if match:
43         section = match.group(1)
44     if not line.strip():
45         continue
46     if section not in MAPPING:
47         raise ValueError(f"源行{line_number}无逐句映射")
48     functions, tests, artifacts = MAPPING[section]
49     source_refs = ";".join(positions[f] for f in functions.split(";"))
50     test_refs = ";".join(positions[f"test_q2.py:{t}"] for t in tests.split(";"))
51     for artifact in artifacts.split(";"):
52         if not (output / artifact).is_file():
53             raise ValueError(f"验收证据缺失: {artifact}")
54     for sentence in filter(str.strip, re.split(r"(?<=[。：！？])", line)):
55         rows.append({"source_file": "docs/2/dierwen1.md", "source_line": line_number,
56                     "section": section, "source_text": sentence, "implementation": source_refs,
57                     "tests": test_refs, "runtime_evidence": artifacts,
58                     "status": "protocol_and_interfaces_complete_real_metrics_pending_model" if
59 section == "2.1.13" else
60                     "conditional_candidates_no_fitted_model" if section == "2.1.14" else "
61 implemented_and_checked"})
62 write_csv(output / "source_traceability.csv", pd.DataFrame(rows))
63 return {"mapped_source_lines": len([r["source_line"] for r in rows]), "mapped_sentence_rows":
64 len(rows), "sections": len(MAPPING)-1}

```

B.3 问题三源程序

src/q3/__init__.py

```

1 """C题第三问：v4四时点随机经济滚动优化。"""

```

src/q3/analysis.py

```

1 """3.22—3.25: 预测诊断、paired统计、事后实验及科研图。"""
2 from dataclasses import replace
3 from pathlib import Path
4 import json
5 import numpy as np
6 import pandas as pd
7 from .config import Config, DT, write_csv, write_json

```



```

8 from .data import Inputs
9 from .rolling import date_of
10 from .terminal import TerminalValues
11 from .multistage import tree_inputs, solve_tree, replay_tree
12 from .export import summaries, write_tables
13
14
15 def forecast_diagnostics(data: Inputs, output: Path) -> dict:
16     errors, updates, corr = [], [], []
17     actual = data.actual.reshape(-1, 2)
18     for hour in (0, 6, 12, 18):
19         trajectories = []
20         for day in range(365):
21             trajectory = []
22             for h in range(1, 25):
23                 slot = day*144+(hour+h)*6-1
24                 if slot >= len(actual):
25                     continue
26                 truth = actual[slot, 1]
27                 error = data.hourly[day, hour//6, h-1]-truth
28                 errors.append({'date': str(date_of(day).date()), 'issue_hour': hour, 'horizon': h,
29                             'target': date_of(day)+pd.Timedelta(hours=hour+h), 'error_kw': error
30 })
31                 trajectory.append(error)
32                 if hour and h <= 18:
33                     old = data.hourly[day, hour//6-1, h+5]
34                     updates.append({'date': str(date_of(day).date()), 'hour': hour, 'horizon': h,
35                                 'block': h <= 6, 'old_absolute_error': abs(old-truth),
36                                 'new_absolute_error': abs(error)})
37             if len(trajectory) == 24:
38                 trajectories.append(trajectory)
39         x = np.asarray(trajectories)
40         for lag in range(1, 24):
41             left, right = x[:, :-lag].ravel(), x[:, lag:].ravel()
42             value = float(np.corrcoef(left, right)[0, 1]) if min(left.std(), right.std()) > 0 else
43             None
44             corr.append({'issue_hour': hour, 'lag_hours': lag, 'correlation': value, 'pairs': len(
45 left)})
46         frame = pd.DataFrame(errors); rows = []
47         for (hour, horizon), group in frame.groupby(['issue_hour', 'horizon']):
48             e = group.error_kw.to_numpy()
49             rows.append({'issue_hour': hour, 'horizon': horizon, 'n': len(e), 'MAE': np.abs(e).mean(),
50                         'RMSE': np.sqrt(np.mean(e**2)), 'Bias': e.mean()})
51         metrics, comparison = pd.DataFrame(rows), pd.DataFrame(updates)
52         write_csv(output/'forecast_errors.csv', frame)
53         write_csv(output/'forecast_metrics.csv', metrics)
54         write_csv(output/'forecast_revision_pairs.csv', comparison)
55         write_csv(output/'forecast_error_autocorrelation.csv', pd.DataFrame(corr))
56         block = comparison[comparison.block].groupby('hour')[['old_absolute_error', 'new_absolute_error'
57 ]].mean()
58         write_csv(output/'forecast_block_mae.csv', block.reset_index())
59         # 已实现真值只在独立诊断模块读取，绝不写回主配置或选模。
60         from src.plots.q3 import forecast_heatmaps
61         font = forecast_heatmaps(metrics, output/'figures')
62         return {'hourly_block_mae': block.reset_index().to_dict('records'), 'font': font,
63                 'year_end_missing_targets': 365*4*24-len(frame), 'future_targets_filled': False}

```



```

61
62 def paired_summary(values: np.ndarray) -> dict:
63     values = np.asarray(values, dtype=float)
64     if len(values) == 0:
65         raise ValueError('配对统计没有共同日期')
66     return {'days': len(values), 'total_saving': float(values.sum()), 'mean': float(values.mean()),
67            'median': float(np.median(values)), 'IQR': float(np.quantile(values, .75)-np.quantile(
68            values, .25)),
69            'positive_day_fraction': float((values > 0).mean()), 'min': float(values.min()), 'max':
70            float(values.max())}
71
72 def information_summary(paths: dict[str, Path], output: Path) -> pd.DataFrame:
73     sets = ['K0', 'K06', 'K0612', 'K061218']
74     costs = {}
75     for key in sets:
76         frame = pd.read_csv(paths[key]/'dispatch.csv')
77         costs[key] = frame.groupby('date').total_cost.sum()
78     rows, paired = [], []
79     for old, new, hour in zip(sets[:-1], sets[1:], (6, 12, 18)):
80         comparison = pd.concat([costs[old], costs[new]], axis=1, keys=['old', 'new'])
81         if comparison.isna().any().any():
82             raise ValueError('OUV分支评价日期不一致')
83         difference = comparison.old-comparison.new
84         rows.append({'metric': 'OUV', 'hour': hour, **paired_summary(difference.to_numpy())})
85         paired.extend({'date': str(date), 'hour': hour, 'saving': float(value)} for date, value in
86         difference.items())
87     information = []
88     for path in sorted((paths['K061218']/'information').glob('*.json')):
89         information.extend(json.loads(path.read_text(encoding='utf-8')))
90     for hour in (6, 12, 18):
91         group = [r for r in information if r['hour'] == hour]
92         if group:
93             rows.append({'metric': 'FIV_block', 'hour': hour, **paired_summary(np.array([r['
94             FIV_block'] for r in group]))})
95             if hour == 18:
96                 for metric in ('FIV_day', 'FIV_cmp', 'FIV_cont'):
97                     valid = [r[metric] for r in group if r[metric] is not None]
98                     if valid:
99                         rows.append({'metric': metric, 'hour': hour, **paired_summary(np.array(valid))})
100     result = pd.DataFrame(rows)
101     write_csv(output/'information_value_summary.csv', result)
102     write_csv(output/'OUV_daily_pairs.csv', pd.DataFrame(paired))
103     write_json(output/'FIV_detail.json', information)
104     return result
105
106 def variants(base: Config) -> dict[str, Config]:
107     """预先固定的单因素事后对照，不通过其结果改写base。"""
108     return {'S10': replace(base, scenarios=10), 'S30': replace(base, scenarios=30),
109            'tail0': replace(base, tail=0), 'tail4': replace(base, tail=4),
110            'all_history': replace(base, unconditional=True),
111            'terminal08': replace(base, terminal_scale=.8), 'terminal12': replace(base,
112            terminal_scale=1.2),
113            'pchip': replace(base, interpolation='pchip'), 'lag': replace(base, feedback='lag')}

```

```

113 def stress_dates(data: Inputs, main: pd.DataFrame) -> pd.DataFrame:
114     daily, _ = summaries(main)
115     rows = []
116     features = []
117     for row in daily.itertuples():
118         d = (pd.Timestamp(row.date)-pd.Timestamp('2025-01-01')).days
119         # 不选年末无法回放完整18h反事实的日期。
120         if d >= 364:
121             continue
122         actual = data.actual[d]
123         features.append({'day': d, 'pv_energy': float(actual[:, 1].sum()*DT),
124             'pv_error': float(np.abs(data.pv(d, 0)-actual[:, 1]).mean()),
125             'peak_net': float((actual[:, 0]-actual[:, 1]).max()), 'emergency': row.emergency})
126     features = pd.DataFrame(features)
127     for q, label in ((1/6, 'low_pv'), (.5, 'middle_pv'), (5/6, 'high_pv')):
128         target = features.pv_energy.quantile(q)
129         d = int(features.loc[(features.pv_energy-target).abs().idxmin(), 'day'])
130         rows.append({'day': d, 'stratum': label})
131     for feature in ('pv_error', 'peak_net', 'emergency'):
132         rows.append({'day': int(features.loc[features[feature].idxmax(), 'day']), 'stratum': 'high_'
133             +feature})
134     return pd.DataFrame(rows).merge(features, on='day')
135
136 def run_experiments(data: Inputs, base: Config, root: Path, main: pd.DataFrame, *, workers: int = 1)
137     -> None:
138     """独立SOC链有界并行；每条链仍严格按日期推进。"""
139     from .parallel import run_rollout_jobs
140     configs = {'baseline': base, **variants(base)}
141     jobs = []
142     for name, config in configs.items():
143         for label, nodes in zip(('K0', 'K06', 'K0612', 'K061218'), ((0,), (0, 6), (0, 6, 12), (0, 6,
144             12, 18))):
145             if name == 'baseline' and len(nodes) == 4:
146                 continue
147             jobs.append(dict(data=data, config=replace(config, nodes=nodes),
148                 output=root/'experiments'/name/label, with_fiv=len(nodes) == 4))
149     jobs.append(dict(data=data, config=replace(base, deterministic=True), output=root/'experiments/'
150         M0', with_fiv=False))
151     run_rollout_jobs(jobs, workers, root/'parallel_experiments.json')
152     all_results, info_rows, differences = [], [], []
153     for name, config in configs.items():
154         paths = {}
155         for label, nodes in zip(('K0', 'K06', 'K0612', 'K061218'), ((0,), (0, 6), (0, 6, 12), (0, 6,
156             12, 18))):
157             path = root/'main' if name == 'baseline' and len(nodes) == 4 else root/'experiments/'
158             name/label
159             frame = main if name == 'baseline' and len(nodes) == 4 else pd.read_csv(path/'dispatch.
160                 csv')
161             paths[label] = path
162             daily, _ = summaries(frame)
163             all_results.append({'variant': name, 'nodes': label, 'cost': float(daily.total_cost.sum
164                 ()),
165                 **{key: float(daily[key].sum()) for key in ('planned_cost', '
166                 adjustment_cost', 'emergency_cost',
167                 'emergency', 'emergency_events', 'emergency_slots', 'throughput
168                 ')}},

```

```

160         'last_soc': float(daily.soc_end.iloc[-1]))
161     if len(nodes) == 4:
162         # 首块决策差和求解耗时按实际已提交审计计算。
163         merged = main[['date', 'slot', 'grid']].merge(frame[['date', 'slot', 'grid']], on=['
date', 'slot'], suffixes=('_base', '_variant'))
164         first = merged[merged.slot < 36]
165         seconds = sum(node['solver']['seconds'] for path_audit in (path/'audit').glob('*.
json'))
166         for node in json.loads(path_audit.read_text(encoding='utf-8')).get('
nodes', []):
167             differences.append({'variant': name, 'first_block_grid_L1': float((first.grid_base-
first.grid_variant).abs().sum()),
168                                 'solve_seconds': seconds})
169         info = information_summary(paths, root/'experiments'/name)
170         info['variant'] = name; info_rows.append(info)
171     m0 = pd.read_csv(root/'experiments/M0/dispatch.csv')
172     write_tables(m0, root/'experiments/M0')
173     daily0, _ = summaries(m0)
174     all_results.append({'variant': 'M0', 'nodes': 'K061218', 'cost': float(daily0.total_cost.sum()),
175                         **{key: float(daily0[key].sum()) for key in ('planned_cost', '
adjustment_cost', 'emergency_cost',
176                             'emergency', 'emergency_events', 'emergency_slots', 'throughput')},
177                         'last_soc': float(daily0.soc_end.iloc[-1]))
178     write_csv(root/'experiment_comparison.csv', pd.DataFrame(all_results))
179     write_csv(root/'scenario_decision_timing.csv', pd.DataFrame(differences))
180     combined = pd.concat(info_rows, ignore_index=True)
181     combined['rank_by_saving'] = combined.groupby(['variant', 'metric']).total_saving.rank(ascending
=False, method='min')
182     write_csv(root/'information_robustness.csv', combined)
183     selected = stress_dates(data, main); write_csv(root/'stress_dates.csv', selected)
184     from .parallel import process_pool
185     requests = []
186     for day in sorted(selected.day.unique()):
187         baseline_day = main[pd.to_datetime(main.date) == date_of(int(day))]
188         requests.append((data, base, root, int(day), float(baseline_day.initial_soc.iloc[0]),
189                         float(baseline_day.total_cost.sum())))
190     with process_pool(workers, base.solver_threads) as executor:
191         results = list(executor.map(m2_job, requests)) if executor is not None else list(map(m2_job,
requests))
192     write_csv(root/'M2_comparison.csv', pd.DataFrame(results))
193     values = np.array([r['relative_loss'] for r in results])
194     write_json(root/'M2_summary.json', {**paired_summary(values), 'q90': float(np.quantile(values,
.9)),
195                                     'max_absolute_gap': float(np.max(np.abs(values))), 'interpretation': '
stratified_stress_sample_not_annual_unbiased_estimate'})
196     with process_pool(workers, base.solver_threads) as executor:
197         groups = executor.map(settlement_job, requests) if executor is not None else map(
settlement_job, requests)
198         sequential_rows = [row for group in groups for row in group]
199     write_csv(root/'settlement_stress_comparison.csv', pd.DataFrame(sequential_rows))
200
201
202 def m2_job(request: tuple) -> dict:
203     data, base, root, day, initial, baseline_cost = request
204     terminal = TerminalValues(data, base)
205     net, labels, nodes, audit = tree_inputs(data, day, base)
206     coefficient, calibration = terminal.value(day, 0)

```

```

207 from .optimization import LimitedSolve
208 try:
209     solution = solve_tree(net, labels, nodes, np.array(audit['load_upper']), data.prices,
        initial, coefficient, base)
210 except LimitedSolve as error:
211     write_json(root/f'M2/{date_of(day).date()}_failure.json', error.audit)
212     raise
213 frame = replay_tree(data, day, initial, nodes, solution, base)
214 write_csv(root/f'M2/{date_of(day).date()}.csv', frame)
215 write_json(root/f'M2/{date_of(day).date()}.json', {'scenarios': audit, 'nodes': nodes, 'labels':
        labels.tolist(),
216             'solver': solution['audit'], 'calibration': calibration})
217 return {'date': str(date_of(day).date()), 'M1_cost': baseline_cost,
218         'M2_cost': float(frame.total_cost.sum()),
219         'relative_loss': float((baseline_cost-frame.total_cost.sum())/frame.total_cost.sum())}
220
221
222 def settlement_job(request: tuple) -> List[dict]:
223     # 每个stress日独立; 同日各更新时间集从相同主轨迹初态开始。
224     from .rolling import run_day
225     data, base, root, day, initial, _ = request
226     terminal = TerminalValues(data, base)
227     sequential_rows = []
228     for settlement in ('final', 'sequential'):
229         costs = []
230         for nodes in ((0,), (0, 6), (0, 6, 12), (0, 6, 12, 18)):
231             config = replace(base, settlement=settlement, nodes=nodes)
232             frame, audit, info = run_day(data, terminal, int(day), initial, config, with_fiv=len(
                nodes) == 4)
233             path = root/f'settlement_stress/{settlement}/{day}_K{len(nodes)}'
234             write_csv(path/'dispatch.csv', frame); write_json(path/'audit.json', audit); write_json(
                path/'information.json', info)
235             costs.append(float(frame.total_cost.sum()))
236             sequential_rows.append({'day': int(day), 'settlement': settlement, 'nodes': len(nodes),
237                                     'cost': costs[-1], 'adjustment_cost': float(frame.adjustment_cost.sum()),
238                                     'emergency_cost': float(frame.emergency_cost.sum()), 'scope': '
239             paired_stress_day_same_initial_state',
240                                     'FIV6': next((r['FIV_block'] for r in info if r['hour'] == 6), None),
241                                     'FIV12': next((r['FIV_block'] for r in info if r['hour'] == 12), None),
242                                     'FIV18': next((r['FIV_block'] for r in info if r['hour'] == 18), None)})
243         for row in sequential_rows[-4:]:
244             row.update(0UV6=costs[0]-costs[1], 0UV12=costs[1]-costs[2], 0UV18=costs[2]-costs[3])
245     return sequential_rows

```

src/q3/checkpoint.py

```

1 """单写入者锁、输入摘要、带校验的节点求解快照。"""
2 from contextlib import contextmanager
3 import fcntl
4 import hashlib
5 import json
6 import os
7 from pathlib import Path
8 from typing import Iterator
9 import numpy as np
10 from .config import Config, write_json

```

```

11 from .physics import Policy
12
13
14 @contextmanager
15 def run_lock(path: Path) -> Iterator[None]:
16     path.parent.mkdir(parents=True, exist_ok=True)
17     with path.open('a+', encoding='utf-8') as stream:
18         try:
19             fcntl.flock(stream.fileno(), fcntl.LOCK_EX | fcntl.LOCK_NB)
20         except BlockingIOError as error:
21             raise RuntimeError(f'已有进程占用{path}, 禁止重复运行') from error
22         stream.seek(0); stream.truncate(); stream.write(str(os.getpid())); stream.flush(); os.fsync(
23             stream.fileno())
24         try:
25             yield
26         finally:
27             fcntl.flock(stream.fileno(), fcntl.LOCK_UN)
28
29 def request_digest(config: Config, *values: object) -> str:
30     digest = hashlib.sha256(config.signature().encode('utf-8'))
31     for value in values:
32         if value is None:
33             digest.update(b'None'); continue
34         array = np.asarray(value, dtype='<f8')
35         digest.update(str(array.shape).encode('utf-8')); digest.update(array.tobytes())
36     return digest.hexdigest()
37
38
39 def save_snapshot(path: Path, signature: str, policy: Policy, audit: dict) -> None:
40     content = {'signature': signature, 'policy': {k: getattr(policy, k).tolist() for k in ('grid', '
41         charge_cap', 'discharge_cap')},
42         'audit': audit}
43     payload = json.dumps(content, sort_keys=True, ensure_ascii=False, allow_nan=False).encode('utf-8
44         ')
45     write_json(path, {'sha256': hashlib.sha256(payload).hexdigest(), 'content': content})
46
47 def load_snapshot(path: Path, signature: str) -> tuple[Policy, dict] | None:
48     if not path.exists():
49         return None
50     wrapper = json.loads(path.read_text(encoding='utf-8')); content = wrapper['content']
51     payload = json.dumps(content, sort_keys=True, ensure_ascii=False, allow_nan=False).encode('utf-8
52         ')
53     if hashlib.sha256(payload).hexdigest() != wrapper['sha256'] or content['signature'] != signature
54         :
55         raise ValueError('节点快照损坏或输入/配置/代码不同, 拒绝复用策略和下界')
56     return Policy(*(np.asarray(content['policy'][k], dtype=float) for k in ('grid', 'charge_cap', '
57         discharge_cap'))), content['audit'])

```

src/q3/config.py

```

1 """题设参数、v4冻结协议及用户确认的终端标定口径。"""
2 from dataclasses import asdict, dataclass
3 from pathlib import Path
4 import hashlib

```

```

5 import json
6 import os
7
8 CODE_ROOT = Path(__file__).resolve().parent
9 ROOT = Path(os.environ.get('Q3_WORKSPACE_ROOT', str(CODE_ROOT.parents[1]))).resolve()
10 SOURCE = ROOT / 'docs/3/Q3_第三问_最终建模_严格修订终稿_v4.md'
11 ATTACHMENTS = ROOT / 'docs/CUMCM2026Problems/C题/附件'
12 DT, CAP, E_MIN, E_MAX, E_INITIAL, ETA = 1/6, 5000/6, 1200., 10800., 6000., .9
13 NODES = (0, 6, 12, 18)
14 TOL = 1e-5
15
16
17 @dataclass(frozen=True)
18 class Config:
19     scenarios: int = 20
20     tail: int = 2
21     unconditional: bool = False
22     terminal_scale: float = 1.
23     interpolation: str = 'linear'
24     feedback: str = 'aggregate'
25     settlement: str = 'final'
26     nodes: tuple[int, ...] = NODES
27     deterministic: bool = False
28     # 数值预算不改变数学模型；正式默认求证至数值最优，限时不通过则停止。
29     seconds: float = 120.
30     gap: float = 0.
31     solver_threads: int = 1
32     solver_backend: str = 'highs'
33     formulation: str = 'v2_free'
34     solver_focus: str = 'default'
35     production_rescue: bool = False
36     hard_06_seconds: float = 35.
37     hard_other_seconds: float = 8.
38     fd_delta: float = 100.
39     fd_base: float = E_INITIAL
40     fd_days: int = 28
41
42     def __post_init__(self) -> None:
43         if not isinstance(self.solver_threads, int) or self.solver_threads < 1:
44             raise ValueError('求解线程数必须为正整数')
45         if self.solver_backend not in ('auto', 'highs', 'gurobi') or self.formulation not in ('legacy', 'v2_free'):
46             raise ValueError('求解后端或精确重构版本非法')
47         if self.solver_focus not in ('default', 'bound'):
48             raise ValueError('求解搜索重点非法')
49         if self.hard_06_seconds <= 0 or self.hard_other_seconds <= 0:
50             raise ValueError('节点硬时限必须为正数')
51         if self.scenarios < 1 or not 0 <= self.tail <= self.scenarios:
52             raise ValueError('场景/尾部数非法')
53         if self.nodes not in ((0,), (0, 6), (0, 6, 12), NODES):
54             raise ValueError('更新时间集合必须为方案四种之一')
55         if self.interpolation not in ('linear', 'pchip') or self.feedback not in ('aggregate', 'lag'):
56             raise ValueError('插值或反馈口径非法')
57         if self.settlement not in ('final', 'sequential') or self.seconds <= 0 or not 0 <= self.gap < 1:
58             raise ValueError('结算或求解设置非法')

```

```

59     if not 0 < self.fd_delta <= E_MAX-self.fd_base or self.terminal_scale <= 0:
60         raise ValueError('终端标定参数非法')
61
62     def signature(self) -> str:
63         payload = SOURCE.read_bytes()+json.dumps(asdict(self), sort_keys=True).encode('utf-8')
64         for source in sorted(CODE_ROOT.glob('*.py')):
65             if not source.name.startswith('test_'):
66                 payload += source.name.encode('utf-8')+source.read_bytes()
67         return hashlib.sha256(payload).hexdigest()
68
69
70 def validate_production_rescue(config: Config) -> None:
71     frozen=(config.production_rescue and config.scenarios==20 and config.tail==2 and not config.
72         unconditional
73         and config.terminal_scale==1. and config.interpolation=='linear' and config.feedback=='
74         aggregate'
75         and config.settlement=='final' and config.nodes==NODES and not config.deterministic and
76         config.seconds==120.
77         and config.gap==.03 and config.solver_threads==4 and config.solver_backend=='highs'
78         and config.formulation=='legacy' and config.solver_focus=='default' and config.fd_delta
79         ==100.
80         and config.fd_base==E_INITIAL and config.fd_days==28 and config.hard_06_seconds==35.
81         and config.hard_other_seconds==8.)
82     if not frozen:
83         raise ValueError('production rescue冻结S20/tail2/四节点/24h模型/3%优先/HiGHS4线程/legacy fast/35
84         与8秒')
85
86
87 def write_json(path: Path, value: object) -> None:
88     path.parent.mkdir(parents=True, exist_ok=True)
89     temporary = path.with_suffix(path.suffix+f'_{os.getpid()}.pending')
90     with temporary.open('w', encoding='utf-8') as stream:
91         json.dump(value, stream, ensure_ascii=False, indent=2, allow_nan=False)
92         stream.flush(); os.fsync(stream.fileno())
93     temporary.replace(path)
94     sync_directory(path.parent)
95
96
97 def write_csv(path: Path, frame: object) -> None:
98     path.parent.mkdir(parents=True, exist_ok=True)
99     temporary = path.with_suffix(path.suffix+f'_{os.getpid()}.pending')
100     with temporary.open('w', encoding='utf-8', newline='') as stream:
101         frame.to_csv(stream, index=False, float_format='%.17g')
102         stream.flush(); os.fsync(stream.fileno())
103     temporary.replace(path)
104     sync_directory(path.parent)
105
106
107 def sync_directory(path: Path) -> None:
108     descriptor = os.open(path, os.O_RDONLY)
109     try:
110         os.fsync(descriptor)
111     finally:
112         os.close(descriptor)

```

src/q3/data.py

```

1  """3.3—3.8: 原始附件校验、Q2只读继承与严格可用时间边界。"""
2  from dataclasses import dataclass
3  import hashlib
4  import json
5  from pathlib import Path
6  import sys
7  import numpy as np
8  import pandas as pd
9  from openpyxl import load_workbook
10 from scipy.interpolate import PchipInterpolator
11 from .config import ROOT, ATTACHMENTS, NODES, DT, write_csv, write_json
12
13
14 @dataclass
15 class Inputs:
16     actual: np.ndarray # day × 144 × (load,pv), kW
17     hourly: np.ndarray # day × four origins × 24 horizons, kW
18     prices: np.ndarray
19     load_store: dict[str, np.ndarray]
20     selected: list[str]
21     provenance: dict
22
23     def load(self, day: int, name: str | None = None) -> np.ndarray:
24         if day < 7:
25             raise ValueError('沿用Q2 1月1—7日无负荷预测、储能停机冷启动')
26         name = name or self.selected[day]
27         result = self.load_store[name][day, :2].reshape(-1).copy()
28         if not np.isfinite(result).all():
29             raise ValueError(f'Q2因果48h预测缺失: {day}/{name}')
30         return result
31
32     def observation(self, day: int, hour: int) -> float:
33         endpoint = day*144+hour*6
34         if endpoint == 0:
35             raise ValueError('附件没有2025-01-01 00:00观测, 不能把00:10真值当左端点')
36         return float(self.actual.reshape(-1, 2)[endpoint-1, 1])
37
38     def pv(self, day: int, hour: int, method: str = 'linear') -> np.ndarray:
39         values = np.r_[self.observation(day, hour), self.hourly[day, hour//6]]
40         points = np.arange(1, 145)/6
41         if method == 'linear':
42             return np.interp(points, np.arange(25), values)
43         if method == 'pchip':
44             return np.maximum(PchipInterpolator(np.arange(25), values)(points), 0)
45         raise ValueError('未知插值方式')
46
47     def history(self, day: int, hour: int, name: str, *, method: str = 'linear',
48                 forecast_hour: int | None = None, length: int = 144) -> tuple[np.ndarray, np.ndarray,
49                                     np.ndarray, np.ndarray]:
50         """候选日j<d且所用轨迹终点<=当前节点; 按Q2当前predictor取同名历史样本外残差。"""
51         origin = hour if forecast_hour is None else forecast_hour
52         offset = (hour-origin)*6
53         if offset < 0 or offset+length > 144:
54             raise ValueError('请求超过官方预报共同支持集')
55         ids, load_errors, pv_errors, scales = [], [], [], []
56         cutoff = day*144+hour*6

```



```

56     flattened = self.actual.reshape(-1, 2)
57     for j in range(7, day):
58         start = j*144+hour*6
59         if start+length > cutoff or start+length > len(flattened):
60             continue
61         forecast = self.load_store[name][j, :2].reshape(-1)[hour*6:hour*6+length]
62         if not np.isfinite(forecast).all():
63             continue
64         pv_full = self.pv(j, origin, method)
65         observed = flattened[start:start+length]
66         ids.append(j)
67         load_errors.append(observed[:, 0]-forecast)
68         pv_errors.append(observed[:, 1]-pv_full[offset:offset+length])
69         scales.append(pv_full.sum()*DT)
70     return (np.asarray(ids, dtype=int), np.asarray(load_errors).reshape(-1, length),
71            np.asarray(pv_errors).reshape(-1, length), np.asarray(scales))
72
73
74 def read_inputs(q2_run: Path, processed: Path | None = None) -> Inputs:
75     # Q2模块采用脚本式导入；只读导入其已经验证的附件解析与季节预测函数。
76     sys.path.insert(0, str(ROOT/'src/q2'))
77     from data import read_attachment, endpoint_minutes
78     from forecast import seasonal
79     shadow = q2_run/'raw/shadows'
80     meta = json.loads((shadow/'shape.json').read_text(encoding='utf-8'))
81     if meta['boundary'] != 'source_date_before_origin' or tuple(meta['shape']) != (365, 3, 144):
82         raise ValueError('Q2预测库来源/形状不符')
83     folder = processed or ROOT/'inputs/q3/processed'
84     frame, _ = read_attachment(ATTACHMENTS/'附件2.xlsx', folder/'actual_quality.csv')
85     actual = frame[['load_kw', 'pv_kw']].to_numpy().reshape(365, 144, 2)
86     wb = load_workbook(ATTACHMENTS/'附件1.xlsx', read_only=True, data_only=True)
87     rows = list(wb.active.values)[1:]; wb.close()
88     if [endpoint_minutes(r[0]) for r in rows] != list(range(10, 1441, 10)):
89         raise ValueError('附件1电价右端点错位')
90     prices = np.array([r[1] for r in rows], dtype=float)
91     wb = load_workbook(ATTACHMENTS/'附件3.xlsx', read_only=True, data_only=True)
92     rows = list(wb.active.values); wb.close()
93     if len(rows) != 1461 or rows[0][2:] != tuple(f'预报{h}小时' for h in range(1, 25)):
94         raise ValueError('附件3表头/形状不符')
95     hourly = np.full((365, 4, 24), np.nan)
96     day = None
97     long_rows = []
98     for row in rows[1:]:
99         if row[0]:
100             day = pd.Timestamp(row[0]).normalize()
101             d = (day-pd.Timestamp('2025-01-01')).days
102             hour = int(str(row[1]).split(':')[0])
103             if hour not in NODES or not 0 <= d < 365 or np.isfinite(hourly[d, hour//6]).any():
104                 raise ValueError('附件3日期/发布时间重复或越界')
105             hourly[d, hour//6] = np.array(row[2:], dtype=float)
106             issue = day+pd.Timedelta(hours=hour)
107             long_rows.extend({'issue': issue, 'target': issue+pd.Timedelta(hours=h),
108                             'horizon_hours': h, 'forecast_kw': float(value)} for h, value in enumerate
109                             (row[2:], 1))
110     if not np.isfinite(hourly).all() or (hourly < 0).any() or not np.isfinite(prices).all() or (
111         prices <= 0).any():
112         raise ValueError('原始预报或电价非法，不擅自填补')

```

```

111 store = {name: np.array(np.memmap(shadow/f'{name}.dat', mode='r', dtype=meta['dtype'], shape=
    tuple(meta['shape'])))
112         for name in meta['model_ids'] if name.startswith('load_')}
113 # Q2冷启动只用季节负荷; 保留其原始影子预测, 并补齐7—13日同一因果流程。
114 for d in range(7, 14):
115     store['load_week'][d] = seasonal(actual[:d].copy(), 3)[..., 0]
116 selected, decisions = ['unavailable']*7+['load_week']*24, []
117 for d in range(31, 365):
118     date = pd.Timestamp('2025-01-01')+pd.Timedelta(days=d)
119     path = q2_run/f'processed/main/daily_audit/{date.date()}.json'
120     audit = json.loads(path.read_text(encoding='utf-8'))
121     name = audit[str(audit['selected_K'])]['selection']['pipeline'][0]
122     selected.append(name)
123     decisions.append({'date': str(date.date()), 'load_predictor': name, 'q2_audit': str(path),
124                     'sha256': hashlib.sha256(path.read_bytes()).hexdigest()})
125 sources = [ATTACHMENTS/f'附件{i}.xlsx' for i in (1, 2, 3)]
126 sources += [shadow/'shape.json', shadow/'frozen_lightgbm.json']+ [shadow/f'{name}.dat' for name
    in store]
127 provenance = {'q2_run': str(q2_run), 'q2_reselection': False, 'q2_decisions': decisions,
128               'files': {str(p): hashlib.sha256(p.read_bytes()).hexdigest() for p in sources},
129               'cold_start': 'Jan1-Jan7 no load forecast/storage; Jan8-Jan31 seasonal; Q2 daily
    load identity thereafter',
130               'units': 'power kW; all optimization and replay energies kWh',
131               'right_endpoint': True, 'jan1_0000_pv': 'missing; never filled with future
    observation'}
132 if processed is not None:
133     write_csv(folder/'pv_forecasts.csv', pd.DataFrame(long_rows))
134     write_csv(folder/'prices.csv', pd.DataFrame({'slot': np.arange(144), 'price': prices}))
135     write_csv(folder/'q2_load_selection.csv', pd.DataFrame(decisions))
136     write_json(folder/'source_manifest.json', provenance)
137 return Inputs(actual, hourly, prices, store, selected, provenance)

```

src/q3/export.py

```

1 """3.21/3.27: 真实费用表、原模板回填与逐值回读; 禁止部分结果冒充完整结果。"""
2 from copy import copy
3 from pathlib import Path
4 import os
5 import numpy as np
6 import pandas as pd
7 from openpyxl import load_workbook
8 from openpyxl.comments import Comment
9 from .config import ATTACHMENTS, Config, write_csv, write_json, sync_directory
10 from .rolling import validate_frame
11
12
13 def intervals() -> list[str]:
14     return [f'{t//6:02d}:{t%6*10:02d}-{(t+1)//6:02d}:{(t+1)%6*10:02d}' for t in range(144)]
15
16
17 def emergency_events(frame: pd.DataFrame) -> pd.DataFrame:
18     rows = []
19     current = None
20     for row in frame.itertuples():
21         end = pd.Timestamp(row.timestamp); start = end-pd.Timedelta(minutes=10)
22         if row.emergency > 0:

```

```

23         if current is None or current['end'] != start:
24             if current is not None:
25                 rows.append(current)
26                 current = {'date': str(start.date()), 'start': start, 'end': end, 'energy_kwh': 0.,
27                             'cost_yuan': 0.}
28                 current['end'] = end
29                 current['energy_kwh'] += row.emergency
30                 current['cost_yuan'] += row.emergency_cost
31             elif current is not None:
32                 rows.append(current); current = None
33         if current is not None:
34             rows.append(current)
35         for row in rows:
36             offset = (row['end'].normalize()-row['start'].normalize()).days
37             row['interval'] = f'{row["start"]:%H:%M}-{row["end"]:%H:%M}'+(f'+{offset}日' if offset else
38                             '')
39         return pd.DataFrame(rows, columns=['date', 'start', 'end', 'interval', 'energy_kwh', 'cost_yuan'
40                                           ])
41
42 def summaries(frame: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
43     daily, blocks = [], []
44     for date, day in frame.groupby('date', sort=False):
45         events = emergency_events(day)
46         daily.append({'date': date, **{key: float(day[key].sum()) for key in
47                                     ('g0', 'grid', 'planned_cost', 'adjustment_cost', 'emergency_cost', 'total_cost',
48                                     'emergency', 'charge', 'discharge', 'spill')},
49                     'emergency_events': len(events), 'emergency_slots': int((day.emergency > 0).sum()),
50                     'throughput': float(day.charge.sum()+day.discharge.sum()),
51                     'soc_start': float(day.initial_soc.iloc[0]), 'soc_end': float(day.soc.iloc[-1])})
52     for block in range(6):
53         part = day.iloc[block*24:(block+1)*24]
54         blocks.append({'date': date, 'interval': f'{block*4:02d}:00-{(block+1)*4:02d}:00',
55                       'charge': float(part.charge.sum()), 'discharge': float(part.discharge.sum()),
56                       'soc_start': float(day.initial_soc.iloc[0]), 'soc_end': float(day.soc.iloc[-1])})
57     return pd.DataFrame(daily), pd.DataFrame(blocks)
58
59 def write_tables(frame: pd.DataFrame, output: Path) -> None:
60     daily, blocks = summaries(frame)
61     write_csv(output/'daily.csv', daily)
62     write_csv(output/'storage_blocks.csv', blocks)
63     write_csv(output/'emergency_events.csv', emergency_events(frame))
64     dates = ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']
65     chosen = frame[frame.date.astype(str).isin(dates)]
66     table1 = chosen[chosen.slot.isin([60, 72, 84, 96, 108, 120])].copy()
67     table1['interval'] = [intervals()[int(t)] for t in table1.slot]
68     write_csv(output/'paper_table1.csv', table1[['date', 'interval', 'g0', 'grid', 'planned_cost',
69                                                  'adjustment_cost', 'emergency_cost', 'total_cost']])
70     .merge(daily[['date', 'grid', 'total_cost']], on='date', suffixes=('', '_daily'))
71     write_csv(output/'paper_table2.csv', blocks[blocks.date.astype(str).isin(dates)])
72     events = [emergency_events(part) for _, part in chosen.groupby('date')]
73     write_csv(output/'paper_table3.csv', pd.concat(events, ignore_index=True) if events else
74               emergency_events(chosen))
75
76 def export_workbook(frame: pd.DataFrame, output: Path, config: Config, *, smoke: bool = False) ->

```

```

dict:
74 validate_frame(frame, config, full=not smoke)
75 output.mkdir(parents=True, exist_ok=True)
76 workbook = load_workbook(ATTACHMENTS/'附件5/result3.xlsx')
77 expected = ['计划购电量', '调整购电量', '充放电量', '紧急购电量']
78 if workbook.sheetnames != expected:
79     raise ValueError('原模板工作表结构发生改变')
80 originals = {s.title: [c.value for c in s[1]] for s in list(workbook)[:2]}
81 styles = {s.title: [copy(c_style) for c in s[2]] for s in workbook}
82 for sheet in workbook:
83     for merged in list(sheet.merged_cells.ranges):
84         sheet.unmerge_cells(str(merged))
85     sheet.delete_rows(2, sheet.max_row)
86 plans, adjusted, storage, emergencies = [workbook[name] for name in expected]
87 for sheet in (plans, adjusted):
88     for col, label in enumerate(intervals(), 2):
89         sheet.cell(1, col, label)
90         sheet.cell(1, 2).comment = Comment('原模板时段整体偏移10分钟。按附件右端点统一回填00:00—24:00共144
          段，原标签记录于template_audit.json。', 'Q3')
91 adjusted.cell(1, 2).comment = Comment('调整购电量为最终生效绝对量；0—6点保持原计划。差额和逐节点proxy分别
          存CSV及节点审计，不混入真实费用。原时段标签错位已修正。', 'Q3')
92 for i, (date, day) in enumerate(frame.groupby('date', sort=False), 2):
93     for sheet, key, cost in ((plans, 'g0', 'planned_cost'), (adjusted, 'grid', 'adjustment_cost'
          )):
94         sheet.cell(i, 1, pd.Timestamp(date).to_pydatetime())
95         for col, value in enumerate(day[key], 2):
96             sheet.cell(i, col, float(value))
97         # 原比赛模板为仿真结果的数值输入区，沿用模板及Q2惯例，非用户可编辑计算模型。
98         sheet.cell(i, 146, float(day[key].sum()))
99         sheet.cell(i, 147, float(day[cost].sum()))
100 adjusted.cell(1, 147).comment = Comment('调整相关净费用=Σ[1.5p增量-0.5p取消量]；负数表示相对原计划费用减
          少。总真实费用另存daily.csv。', 'Q3')
101 _, blocks = summaries(frame)
102 for i, row in enumerate(blocks.itertuples(index=False), 2):
103     block = (i-2)%6
104     values = (pd.Timestamp(row.date).to_pydatetime() if block == 0 else None, row.interval, row.
          charge, row.discharge,
105               '0:00' if block == 0 else '24:00' if block == 1 else None,
106               row.soc_start if block == 0 else row.soc_end if block == 1 else None)
107     for col, value in enumerate(values, 1):
108         storage.cell(i, col, value)
109 events = emergency_events(frame)
110 for i, row in enumerate(events.itertuples(index=False), 2):
111     for col, value in enumerate((pd.Timestamp(row.date).to_pydatetime(), row.interval, row.
          energy_kwh), 1):
112         emergencies.cell(i, col, value)
113 for sheet in workbook:
114     sheet.freeze_panes = 'B2' if sheet in (plans, adjusted) else 'A2'
115     for row in sheet.iter_rows(min_row=2):
116         for cell in row:
117             cell.style = copy(styles[sheet.title][cell.column-1])
118             if isinstance(cell.value, (int, float)):
119                 cell.number_format = '0.000000'
120 filename = 'smoke_result3.xlsx' if smoke else 'result3.xlsx'
121 temporary = output/(filename+'.pending.xlsx')
122 workbook.save(temporary); workbook.close()
123 check = load_workbook(temporary, read_only=True, data_only=True)

```

```

124     try:
125         for sheet, key, cost in (('计划购电量', 'g0', 'planned_cost'), ('调整购电量', 'grid', '
adjustment_cost')):
126             saved = list(check[sheet].values)[1:]
127             np.testing.assert_allclose(np.array([r[1:145] for r in saved], dtype=float).ravel(),
frame[key], atol=1e-8)
128             np.testing.assert_allclose(np.array([r[145:147] for r in saved], dtype=float), frame.
groupby('date', sort=False)[[key, cost]].sum(), atol=1e-8)
129             saved = list(check['充放电量'].values)[1:]
130             np.testing.assert_allclose(np.array([r[2:4] for r in saved], dtype=float), blocks[['charge',
'discharge']], atol=1e-8)
131             for i, row in enumerate(saved):
132                 if i%6 in (0, 1):
133                     np.testing.assert_allclose(row[5], blocks.iloc[i]['soc_start' if i%6 == 0 else '
soc_end'], atol=1e-8)
134                     saved_events = list(check['紧急购电量'].values)[1:]
135                     np.testing.assert_allclose(np.asarray([r[2] for r in saved_events], dtype=float), events.
energy_kwh.to_numpy(dtype=float), atol=1e-8)
136             finally:
137                 check.close()
138             with temporary.open('rb') as stream:
139                 os.fsync(stream.fileno())
140                 temporary.replace(output/filename)
141                 sync_directory(output)
142                 audit = {'smoke_only': smoke, 'days': int(frame.date.nunique()), 'roundtrip_all_sheets': True,
'default_headers': originals, 'intervals': intervals(), 'adjusted_quantity': '
final_effective_absolute',
143                         'storage_from': 'actual_replay', 'full_period': not smoke}
144                 write_json(output/'template_audit.json', audit)
145             return audit
146

```

src/q3/fast_solver.py

```

1  """Q3单调充电等价MILP：复用Q2验证结构，加入调整成本与正终端残值。"""
2  import hashlib
3  import time
4  import numpy as np
5  from scipy.optimize import Bounds, LinearConstraint
6  from scipy.sparse import coo_matrix, hstack, vstack
7  from highspy import Highs as HighsAPI
8  from highspy import _core as highs_core
9  from .config import CAP, E_MIN, E_MAX, ETA, TOL, Config
10 from .physics import Policy, replay, adjustment
11 from .scenarios import Scenarios
12 ETA_C = ETA_D = ETA
13
14
15 def matrix_digest(objective, bounds, integer, constraints) -> str:
16     """对完整数值矩阵建立跨后端稳定摘要：不包含求解器或热启动。"""
17     matrix=constraints.A.tocsc(copy=True);matrix.sum_duplicates();matrix.sort_indices()
18     digest=hashlib.sha256(b'Q3_MATRIX_CSC_V1')
19     arrays=((('objective',objective,'<f8'),('bounds_lb',bounds.lb,'<f8'),('bounds_ub',bounds.ub,'<f8'
),
20             ('integer',integer,'<i1'),('row_lb',constraints.lb,'<f8'),('row_ub',constraints.ub,'<f8'
),
21             ('indptr',matrix.indptr,'<i8'),('indices',matrix.indices,'<i8'),('data',matrix.data,'<f8

```

```

    '))
22     for name, values, dtype in arrays:
23         array = np.ascontiguousarray(np.asarray(values, dtype=dtype))
24         digest.update(name.encode('ascii')); digest.update(str(array.shape).encode('ascii')); digest.
            update(array.tobytes())
25     return digest.hexdigest()
26
27
28 def reachable_bounds(net: np.ndarray, grid_upper: np.ndarray, initial: float) -> tuple[np.ndarray
    , ...]:
29     """仅用当前场景递推原饱和和反馈的安全 SOC/充放电包络。"""
30     net = np.asarray(net, dtype=float); grid_upper = np.asarray(grid_upper, dtype=float)
31     s, t = net.shape
32     e_lower = np.empty((s, t)); e_upper = np.empty((s, t)); c_upper = np.empty((s, t)); r_upper = np.empty((s, t))
33     for j, path in enumerate(net):
34         low = high = float(initial)
35         for i, n in enumerate(path):
36             deficit = min(CAP, max(n, 0.)); surplus = min(CAP, max(float(grid_upper[i]) - n, 0.))
37             c_upper[j, i] = min(surplus, max((E_MAX - low) / ETA_C, 0.))
38             r_upper[j, i] = min(deficit, max(ETA_D * (high - E_MIN), 0.))
39             # 这些界由旧 fast LP 已有的 c<=surplus、r<=deficit、SOC 方程直接蕴含。
40             # 因而它们包含旧 formulation 的全部 LP/MIP 可行点，不依赖某个 incumbent。
41             low = max(E_MIN, low - deficit / ETA_D)
42             high = min(E_MAX, high + ETA_C * surplus)
43             e_lower[j, i] = max(E_MIN, np.nextafter(low, -np.inf))
44             e_upper[j, i] = min(E_MAX, np.nextafter(high, np.inf))
45             c_upper[j, i] = min(CAP, np.nextafter(c_upper[j, i], np.inf))
46             r_upper[j, i] = min(CAP, np.nextafter(r_upper[j, i], np.inf))
47     return e_lower, e_upper, c_upper, r_upper
48
49 def build_base(net, prices, weights, initial_soc, *, grid_upper, fixed_policy=None):
50     net, prices, weights = [np.asarray(v, dtype=float) for v in (net, prices, weights)]
51     if net.ndim != 2 or prices.shape != (net.shape[1],) or weights.shape != (net.shape[0],):
52         raise ValueError("场景/电价/概率维度不符")
53     if not all(np.isfinite(v).all() for v in (net, prices, weights)) or (prices <= 0).any() or (
        weights <= 0).any():
54         raise ValueError("输入必须有限且电价/概率为正")
55     if not np.isclose(weights.sum(), 1, atol=1e-12) or not E_MIN <= initial_soc <= E_MAX:
56         raise ValueError("概率或初态不符")
57     s, t = net.shape
58     g, cp, rp = [np.arange(k*t, (k+1)*t) for k in range(3)]
59     blocks = [np.arange(3*t+k*s*t, 3*t+(k+1)*s*t).reshape(s, t) for k in range(12)]
60     c, r, q, w, e, z, *selectors = blocks
61     lr, lc = selectors[:3], selectors[3:]
62     size = 3*t+12*s*t
63     objective, lo, hi, integer = np.zeros(size), np.zeros(size), np.full(size, np.inf), np.zeros(
        size)
64     grid_upper = np.asarray(grid_upper, dtype=float)
65     objective[g] = prices; objective[q] = 5*weights[:, None]*prices
66     hi[g], hi[cp], hi[rp] = grid_upper, CAP, CAP
67     hi[c], hi[r], hi[q], hi[w] = CAP, CAP, np.maximum(net, 0), np.maximum(grid_upper-net, 0)
68     lo[e], hi[e] = E_MIN, E_MAX
69     for block in (z, *lr, *lc):
70         hi[block] = 1; integer[block] = 1
71     hi[z[net < 0]] = 0
72     free_charge = fixed_policy is None
73     if free_charge:

```

```

74     # z=sum(lr), 且lr为二元变量、z在[0,1]; z的整数性已由等式隐含。
75     integer[z] = 0
76     for block in lc:
77         hi[block] = 0; integer[block] = 0
78 if fixed_policy is None:
79     # 单段反馈下提高充电上限使逐步SOC不减、紧急购电不增; 见等价性文档的支配性证明。
80     # 仅优化时消去被支配的选择; 固定策略回放必须保留调用者指定的上限。
81     lo[cp] = CAP
82 else:
83     if fixed_policy.charge_cap.shape != (t,) or fixed_policy.grid.shape != (t,):
84         raise ValueError("线性编码只支持单段策略")
85     for indices, values in ((g, fixed_policy.grid), (cp, fixed_policy.charge_cap), (rp,
fixed_policy.discharge_cap)):
86         if (values < lo[indices]-1e-8).any() or (values > hi[indices]+1e-8).any():
87             raise ValueError("固定策略超出线性编码的物理上界")
88         values = np.clip(values, lo[indices], hi[indices])
89         lo[indices] = values; hi[indices] = values
90 rows, cols, coefficients, lower, upper = [], [], [], [], []
91 def add(terms, rhs, equal=False):
92     index = len(upper)
93     for column, value in terms:
94         rows.append(index); cols.append(int(column)); coefficients.append(value)
95         lower.append(rhs if equal else -np.inf); upper.append(rhs)
96 for j in range(s):
97     for i in range(t):
98         n = float(net[j, i]); ci, ri, qi, wi, ei, zi = [int(v[j, i]) for v in (c, r, q, w, e, z)
99 ]
100         previous = int(e[j, i-1]) if i else None
101         add([(g[i], 1), (ri, 1), (qi, 1), (ci, -1), (wi, -1)], n, True)
102         add([(ei, 1), (ci, -ETA_C), (ri, 1/ETA_D)]+[(previous, -1)] if i else [], 0 if i else
initial_soc, True)
103         add([(ci, 1), (cp[i], -1)], 0); add([(ri, 1), (rp[i], -1)], 0)
104         room_max = (E_MAX-E_MIN)/ETA_C if i else (E_MAX-initial_soc)/ETA_C
105         available_max = ETA_D*(E_MAX-E_MIN) if i else ETA_D*(initial_soc-E_MIN)
106         add([(ci, 1)]+[(previous, 1/ETA_C)] if i else [], E_MAX/ETA_C if i else room_max)
107         add([(ri, 1)]+[(previous, -ETA_D)] if i else [], -ETA_D*E_MIN if i else available_max)
108         add([(ci, 1), (zi, CAP)], CAP); add([(ri, 1), (zi, -CAP)], 0)
109         add([(qi, 1), (zi, -max(n, 0))], 0)
110         surplus_max = max(float(grid_upper[i])-n, 0)
111         add([(wi, 1), (zi, surplus_max)], surplus_max)
112         add([(int(v[j, i]), 1) for v in lr]+[(zi, -1)], 0, True)
113         if not free_charge:
114             add([(int(v[j, i]), 1) for v in lc]+[(zi, 1)], 1, True)
115         # 分支激活常数分别为: CAP、max(n,0)、可用电量上界, 以及CAP、Gmax-n、容量空间上界。
116         add([(rp[i], 1), (ri, -1), (lr[0][j, i], CAP)], CAP)
117         deficit_max = max(n, 0)
118         add([(g[i], -1), (ri, -1), (lr[1][j, i], deficit_max)], deficit_max-n)
119         add([(ri, -1), (lr[2][j, i], available_max)]+[(previous, ETA_D)] if i else [],
available_max+ETA_D*E_MIN if i else 0)
120         if not free_charge:
121             add([(cp[i], 1), (ci, -1), (lc[0][j, i], CAP)], CAP)
122             add([(g[i], 1), (ci, -1), (lc[1][j, i], surplus_max)], surplus_max+n)
123             add([(ci, -1), (lc[2][j, i], room_max)]+[(previous, -1/ETA_C)] if i else [],
room_max-E_MAX/ETA_C if i else 0)
124         # 原整数反馈分支隐含的有效约束; 仅加强松弛, 不删除任何可行反馈策略。
125         add([(qi, 1), (lr[0][j, i], -deficit_max), (lr[2][j, i], -deficit_max)], 0)
126         if not free_charge:

```



```

128         add([(wi, 1), (lc[0][j, i], -surplus_max), (lc[2][j, i], -surplus_max)], 0)
129     add([(ei, 1), (lr[2][j, i], E_MAX-E_MIN)], E_MAX)
130     if not free_charge:
131         add([(ei, -1), (lc[2][j, i], E_MAX-E_MIN)], -E_MIN)
132     # 同一方向的响应总量受净缺口/剩余上界约束, 不能将分量上界重复相加。
133     add([(ri, 1), (qi, 1), (zi, -deficit_max)], 0)
134     add([(ci, 1), (wi, 1), (zi, surplus_max)], surplus_max)
135     if i:
136         # 当储能边界项为最小项, 上一SOC必须在一时段功率可达的边界邻域。
137         add([(previous, 1), (lr[2][j, i], E_MAX-E_MIN-CAP/ETA_D)], E_MAX)
138         if not free_charge:
139             add([(previous, -1), (lc[2][j, i], E_MAX-E_MIN-ETA_C*CAP)], -E_MIN)
140     matrix = coo_matrix((coefficients, (rows, cols)), shape=(len(upper), size)).tocsc()
141     return objective, Bounds(lo, hi), integer, LinearConstraint(matrix, lower, upper), {"g": g, "cp": cp, "rp": rp,
142                                                                                                     "responses": (c,
143                                                                                                     r, q, w, e), "z": z, "lr": lr, "lc": lc, "free_charge": free_charge}
144
145 def build_base_v2_free(net, prices, weights, initial_soc, *, grid_upper):
146     """free aggregate 专用精确消元: 不创建固定 cp/lc, 并以 sum(lr) 代换 z。"""
147     net, prices, weights = [np.asarray(v, dtype=float) for v in (net, prices, weights)]
148     if net.ndim != 2 or prices.shape != (net.shape[1],) or weights.shape != (net.shape[0],):
149         raise ValueError('场景/电价/概率维度不符')
150     if not all(np.isfinite(v).all() for v in (net, prices, weights)) or (prices <= 0).any() or (weights <= 0).any():
151         raise ValueError('输入必须有限且电价/概率为正')
152     if not np.isclose(weights.sum(), 1, atol=1e-12) or not E_MIN <= initial_soc <= E_MAX:
153         raise ValueError('概率或初态不符')
154     s, t = net.shape; grid_upper = np.asarray(grid_upper, dtype=float)
155     g = np.arange(t); rp = np.arange(t, 2*t)
156     blocks = [np.arange(2*t+k*s*t, 2*t+(k+1)*s*t).reshape(s, t) for k in range(8)]
157     c, r, q, w, e, *lr = blocks; size = 2*t+8*s*t
158     objective = np.zeros(size); lo = np.zeros(size); hi = np.full(size, np.inf); integer = np.zeros(size)
159     objective[g] = prices; objective[q] = 5*weights[:, None]*prices
160     hi[g], hi[rp] = grid_upper, CAP
161     e_lo, e_hi, c_hi, r_hi = reachable_bounds(net, grid_upper, initial_soc)
162     hi[c], hi[r] = c_hi, r_hi; hi[q] = np.maximum(net, 0); hi[w] = np.maximum(grid_upper-net, 0)
163     lo[e], hi[e] = e_lo, e_hi
164     for selector in lr:
165         hi[selector] = 1; integer[selector] = 1; hi[selector][net < 0] = 0
166     rows = []; cols = []; coefficients = []; lower = []; upper = []
167     def add(terms, rhs, equal=False):
168         row = len(upper)
169         for column, value in terms:
170             rows.append(row); cols.append(int(column)); coefficients.append(float(value))
171             lower.append(rhs if equal else -np.inf); upper.append(rhs)
172     for j in range(s):
173         for i in range(t):
174             n = float(net[j, i]); ci, ri, qi, wi, ei = [int(v[j, i]) for v in (c, r, q, w, e)]
175             previous = int(e[j, i-1]) if i else None
176             selectors = [int(v[j, i]) for v in lr]; z = [(v, 1.) for v in selectors]
177             add([(g[i], 1), (ri, 1), (qi, 1), (ci, -1), (wi, -1)], n, True)
178             add([(ei, 1), (ci, -ETA_C), (ri, 1/ETA_D)] + [(previous, -1)] if i else [], 0 if i else
179             initial_soc, True)
180             add([(ri, 1), (rp[i], -1)], 0)
181             room_max = (E_MAX-E_MIN)/ETA_C if i else (E_MAX-initial_soc)/ETA_C

```



```

181     available_max=ETA_D*(E_MAX-E_MIN) if i else ETA_D*(initial_soc-E_MIN)
182     add([(ci,1)]+[(previous,1/ETA_C)] if i else [],E_MAX/ETA_C if i else room_max)
183     add([(ri,1)]+[(previous,-ETA_D)] if i else [],-ETA_D*E_MIN if i else available_max)
184     add([(ci,1)]+[(v,CAP) for v in selectors],CAP)
185     add([(ri,1)]+[(v,-CAP) for v in selectors],0)
186     deficit_max=max(n,0.);surplus_max=max(float(grid_upper[i])-n,0.)
187     add([(qi,1)]+[(v,-deficit_max) for v in selectors],0)
188     add([(wi,1)]+[(v,surplus_max) for v in selectors],surplus_max)
189     # z∈[0,1] 与 z=sum(lr) 机械代换; n<0 时 selector 列界已固定为0。
190     add(z,1.)
191     add([(rp[i],1),(ri,-1),(selectors[0],CAP)],CAP)
192     add([(g[i],-1),(ri,-1),(selectors[1],deficit_max)],deficit_max-n)
193     add([(ri,-1),(selectors[2],available_max)]+[(previous,ETA_D)] if i else []),
194         available_max+ETA_D*E_MIN if i else 0)
195     add([(qi,1),(selectors[0],-deficit_max),(selectors[2],-deficit_max)],0)
196     add([(ei,1),(selectors[2],E_MAX-E_MIN)],E_MAX)
197     add([(ri,1),(qi,1)]+[(v,-deficit_max) for v in selectors],0)
198     add([(ci,1),(wi,1)]+[(v,surplus_max) for v in selectors],surplus_max)
199     if i:
200         add([(previous,1),(selectors[2],E_MAX-E_MIN-CAP/ETA_D)],E_MAX)
201 matrix=coo_matrix((coefficients,(rows,cols)),shape=(len(upper),size)).tocsc()
202 indices={'g':g,'cp':np.array([],dtype=int),'rp':rp,'responses':(c,r,q,w,e),
203         'z':np.array([],dtype=int),'lr':lr,'lc':(),'free_charge':True,'formulation':'v2_free')}
204 return objective,Bounds(lo,hi),integer,LinearConstraint(matrix,lower,upper),indices
205
206
207 def build_base_v2_2bit(net, prices, weights, initial_soc, *, grid_upper):
208     """四种物理模式的 2-bit 精确编码; 仅作参考通过 LB 基准后才可生产的候选。"""
209     net,prices,weights=[np.asarray(v,dtype=float) for v in (net,prices,weights)]
210     if net.ndim!=2 or prices.shape!=(net.shape[1],) or weights.shape!=(net.shape[0],):
211         raise ValueError('场景/电价/概率维度不符')
212     if not all(np.isfinite(v).all() for v in (net,prices,weights)) or (prices<=0).any() or (weights
213         <=0).any():
214         raise ValueError('输入必须有限且电价/概率为正')
215     if not np.isclose(weights.sum(),1,atol=1e-12) or not E_MIN<=initial_soc<=E_MAX:
216         raise ValueError('概率或初态不符')
217     s,t=net.shape;grid_upper=np.asarray(grid_upper,dtype=float)
218     g=np.arange(t);rp=np.arange(t,2*t)
219     blocks=[np.arange(2*t+k*s*t,2*t+(k+1)*s*t).reshape(s,t) for k in range(7)]
220     c,r,q,w,e,b0,b1=blocks;size=2*t+7*s*t
221     objective=np.zeros(size);lo=np.zeros(size);hi=np.full(size,np.inf);integer=np.zeros(size)
222     objective[g]=prices;objective[q]=5*weights[:,None]*prices;hi[g],hi[rp]=grid_upper,CAP
223     e_lo,e_hi,c_hi,r_hi=reachable_bounds(net,grid_upper,initial_soc)
224     hi[c],hi[r]=c_hi,r_hi;hi[q]=np.maximum(net,0);hi[w]=np.maximum(grid_upper-net,0)
225     lo[e],hi[e]=e_lo,e_hi
226     for bit in (b0,b1):
227         hi[bit]=1;integer[bit]=1;hi[bit][net<0]=0
228     rows=[];cols=[];coefficients=[];lower=[];upper=[]
229     def add(terms,rhs,equal=False):
230         row=len(upper)
231         for column,value in terms:
232             rows.append(row);cols.append(int(column));coefficients.append(float(value))
233             lower.append(rhs if equal else -np.inf);upper.append(rhs)
234     for j in range(s):
235         for i in range(t):
236             n=float(net[j,i]);ci,ri,qi,wi,ei,first,second=[int(v[j,i]) for v in (c,r,q,w,e,b0,b1)]
237             previous=int(e[j,i-1]) if i else None

```

```

237 deficit=max(n,0.);surplus=max(float(grid_upper[i])-n,0.)
238 previous_high=initial_soc if i==0 else e_hi[j,i-1]
239 available=max(0.,ETA_D*(previous_high-E_MIN))
240 add([(g[i],1),(ri,1),(qi,1),(ci,-1),(wi,-1)],n,True)
241 add([(ei,1),(ci,-ETA_C),(ri,1/ETA_D)]+([(previous,-1)] if i else []),0 if i else
initial_soc,True)
242 add([(ri,1),(rp[i],-1)],0)
243 room=(E_MAX-E_MIN)/ETA_C if i else (E_MAX-initial_soc)/ETA_C
244 add([(ci,1)]+([(previous,1/ETA_C)] if i else []),E_MAX/ETA_C if i else room)
245 add([(ri,1)]+([(previous,-ETA_D)] if i else []),-ETA_D*E_MIN if i else available)
246 # 00=充电, 01=rp最小, 10=deficit最小, 11=available最小。
247 add([(ci,1),(first,CAP)],CAP);add([(ci,1),(second,CAP)],CAP)
248 add([(ri,1),(first,-CAP),(second,-CAP)],0)
249 add([(qi,1),(first,-deficit),(second,-deficit)],0)
250 add([(wi,1),(first,surplus)],surplus);add([(wi,1),(second,surplus)],surplus)
251 # 被编码模式的候选值必须等于 r; 上界 r<=三个候选由共享约束/平衡给出。
252 add([(rp[i],1),(ri,-1),(first,-CAP),(second,CAP)],CAP)
253 add([(g[i],-1),(ri,-1),(first,deficit),(second,-deficit)],deficit-n)
254 add([(ri,-1),(first,available),(second,available)]+([(previous,ETA_D)] if i else []),
255 2*available+ETA_D*E_MIN if i else 2*available-ETA_D*(initial_soc-E_MIN))
256 # q只可能在rp或available为最小的编码(第二位=1)中为正。
257 add([(qi,1),(second,-deficit)],0)
258 span=E_MAX-E_MIN
259 add([(ei,1),(first,span),(second,span)],E_MIN+2*span)
260 add([(ri,1),(qi,1)],deficit);add([(ci,1),(wi,1)],surplus)
261 if i:
262     threshold=E_MIN+CAP/ETA_D;big=max(0.,E_MAX-threshold)
263     add([(previous,1),(first,big),(second,big)],threshold+2*big)
264 matrix=coo_matrix((coefficients,(rows,cols)),shape=(len(upper),size)).tocsc()
265 indices={'g':g,'cp':np.array([],dtype=int),'rp':rp,'responses':(c,r,q,w,e),
266         'z':np.array([],dtype=int),'lr':(),'lc':(),'bits':(b0,b1),'free_charge':True,
267         'formulation':'v2_2bit'}
268 return objective,Bounds(lo,hi),integer,LinearConstraint(matrix,lower,upper),indices
269
270
271 def _finish_matrix(objective,bounds,integer,constraints,indices,scenes,prices,*,reference,today,
terminal):
272     """旧版与 v2 共用终端项、有效前缀序和非对称调整成本。"""
273     objective[indices['responses'][4][:,-1]]=-terminal*scenes.weights
274     rows=[];cols=[];vals=[];rhs=[]
275     e=indices['responses'][4];q=indices['responses'][2];w=indices['responses'][3];net=scenes.net
276     for a in range(len(net)):
277         for b in range(len(net)):
278             if a==b:continue
279             ordered=np.flatnonzero(np.maximum.accumulate(net[a]-net[b])<=0.)
280             for t in ordered:
281                 for block,sign in ((e,-1.),(q,1.),(w,-1.)):
282                     row=len(rhs);rows.extend([row,row]);cols.extend([block[a,t],block[b,t]])
283                     vals.extend([sign,-sign]);rhs.append(0.)
284     if rhs:
285         addition=coo_matrix((vals,(rows,cols)),shape=(len(rhs),len(objective))).tocsc()
286         constraints=LinearConstraint(vstack([constraints.A,addition]).tocsc(),
287                                     np.r_[constraints.lb,np.full(len(rhs),-np.inf)],np.r_[constraints.ub,rhs])
288     today=min(today,len(prices));indices['adjustment']=np.array([],dtype=int)
289     if reference is not None:
290         count=len(objective);extra=np.arange(count,count+today);indices['adjustment']=extra
291         objective[indices['g'][:today]]=0.;objective=np.r_[objective,np.ones(today)]

```

```

292     upper=np.maximum(scenes.net.max(0),0)+CAP
293     lo=-.5*prices[:today]*reference[:today];delta=upper[:today]-reference[:today]
294     hi=prices[:today]*np.maximum(.5*delta,1.5*delta)
295     bounds=Bounds(np.r_[bounds.lb,lo],np.r_[bounds.ub,hi]);integer=np.r_[integer,np.zeros(today)
    ]
296     rows=[];cols=[];vals=[];rhs=[]
297     for t in range(today):
298         for slope in (.5,1.5):
299             row=len(rhs);rows.extend([row,row]);cols.extend([indices['g'][t],extra[t]])
300             vals.extend([slope*prices[t],-1.]);rhs.append(slope*prices[t]*reference[t])
301             additional=coo_matrix((vals,(rows,cols)),shape=(len(rhs),len(objective))).tocsc()
302             matrix=vstack([hstack([constraints.A,coo_matrix((constraints.A.shape[0],today))],additional
    ]).tocsc()
303             constraints=LinearConstraint(matrix,np.r_[constraints.lb,np.full(len(rhs),-np.inf)],np.r_[
    constraints.ub,rhs])
304     return objective,bounds,integer,constraints,indices
305
306
307 def linear_seed(policy, net, initial_soc, indices, size):
308     vector = np.zeros(size)
309     for key, values in (("g", policy.grid), ("cp", policy.charge_cap), ("rp", policy.discharge_cap))
    :
310         vector[indices[key]] = values
311     for j, path in enumerate(net):
312         response = replay(policy, path, initial_soc)
313         for columns, key in zip(indices["responses"], ("charge", "discharge", "emergency", "spill",
    "soc")):
314             vector[columns[j]] = response[key]
315         previous = initial_soc
316         for i, n in enumerate(path):
317             x = n-policy.grid[i]; deficit = x >= 0
318             vector[indices["z"][j, i]] = int(deficit)
319             rmin = np.argmin([policy.discharge_cap[i], max(x, 0), ETA_D*(previous-E_MIN)])
320             cmin = np.argmin([policy.charge_cap[i], max(-x, 0), (E_MAX-previous)/ETA_C])
321             if deficit:
322                 vector[indices["lr"][rmin][j, i]] = 1
323             elif not indices["free_charge"]:
324                 vector[indices["lc"][cmin][j, i]] = 1
325             previous = response["soc"][i]
326     return vector
327
328
329
330 def build_matrix(scenes: Scenarios, prices: np.ndarray, initial: float, *, reference=None,
331                 today: int=144, terminal: float=0., fixed_policy: Policy | None=None) -> tuple:
332     if terminal < 0:
333         raise ValueError('单调充电证明仅适用于非负终端残值')
334     # 自由优化时削去必定成为spill的购电: 超过max(n,0)+CAP不改变任何场景的储能动作。
335     upper = scenes.load.max(0)/6+CAP if fixed_policy is not None else np.maximum(scenes.net.max(0)
    ,0)+CAP
336     objective, bounds, integer, constraints, indices = build_base(scenes.net, prices, scenes.weights
    , initial,
337                           grid_upper=upper, fixed_policy=fixed_policy)
338     objective[indices["responses"][4][:,-1]] = -terminal*scenes.weights
339     if fixed_policy is None:
340         # 已知净负荷前缀逐点支配蕴含原反馈SOC/q/spill次序; 只加入原模型必然满足的有效约束。
341         rows=[];cols=[];vals=[];rhs=[]

```

```

342     e=indices['responses'][4];q=indices['responses'][2];w=indices['responses'][3]
343     net=scenes.net
344     for a in range(len(net)):
345         for b in range(len(net)):
346             if a==b:continue
347             ordered=np.flatnonzero(np.maximum.accumulate(net[a]-net[b])<=0.)
348             for t in ordered:
349                 for block,sign in ((e,-1.),(q,1.),(w,-1.)):
350                     row=len(rhs);rows.extend([row,row]);cols.extend([block[a,t],block[b,t]])
351                     vals.extend([sign,-sign]);rhs.append(0.)
352             if rhs:
353                 addition=coo_matrix((vals,(rows,cols)),shape=(len(rhs),len(objective))).tocsc()
354                 constraints=LinearConstraint(vstack([constraints.A,addition]).tocsc(),
355                     np.r_[constraints.lb,np.full(len(rhs),-np.inf)],np.r_[constraints.ub,rhs])
356 today = min(today, len(prices))
357 indices['adjustment'] = np.array([],dtype=int)
358 if reference is not None:
359     count=len(objective);extra=np.arange(count,count+today);indices['adjustment']=extra
360     objective[indices['g'][:today]]=0.
361     objective=np.r_[objective,np.ones(today)]
362     lo=-.5*prices[:today]*reference[:today]
363     delta=upper[:today]-reference[:today]
364     hi=prices[:today]*np.maximum(.5*delta,1.5*delta)
365     bounds=Bounds(np.r_[bounds.lb,lo],np.r_[bounds.ub,hi])
366     integer=np.r_[integer,np.zeros(today)]
367     rows=[];cols=[];vals=[];rhs=[]
368     for t in range(today):
369         for slope in (.5,1.5):
370             row=len(rhs);rows.extend([row,row]);cols.extend([indices['g'][t],extra[t]])
371             vals.extend([slope*prices[t],-1.]);rhs.append(slope*prices[t]*reference[t])
372             additional=coo_matrix((vals,(rows,cols)),shape=(len(rhs),len(objective))).tocsc()
373             matrix=vstack([hstack([constraints.A,coo_matrix((constraints.A.shape[0],today))]),additional
374 ]).tocsc()
375             constraints=LinearConstraint(matrix,np.r_[constraints.lb,np.full(len(rhs),-np.inf)],np.r_[
376 constraints.ub,rhs])
377 return objective,bounds,integer,constraints,indices
378
379 def build_matrix_v2_free(scenes: Scenarios, prices: np.ndarray, initial: float, *, reference=None,
380     today: int=144, terminal: float=0.) -> tuple:
381     """仅用于自由 aggregate 策略: 旧 build_matrix 保留作固定策略与回归基准。"""
382     if terminal<0:raise ValueError('单调充电证明仅适用于非负终端残值')
383     upper=np.maximum(scenes.net.max(0),0)+CAP
384     result=build_base_v2_free(scenes.net,prices,scenes.weights,initial,grid_upper=upper)
385     return _finish_matrix(*result,scenes,prices,reference=reference,today=today,terminal=terminal)
386
387 def build_matrix_v2_2bit(scenes: Scenarios, prices: np.ndarray, initial: float, *, reference=None,
388     today: int=144, terminal: float=0.) -> tuple:
389     if terminal<0:raise ValueError('单调充电证明仅适用于非负终端残值')
390     upper=np.maximum(scenes.net.max(0),0)+CAP
391     result=build_base_v2_2bit(scenes.net,prices,scenes.weights,initial,grid_upper=upper)
392     return _finish_matrix(*result,scenes,prices,reference=reference,today=today,terminal=terminal)
393
394 def linear_seed_v2_free(policy: Policy, net: np.ndarray, initial: float, indices: dict, size: int)
395     -> np.ndarray:

```

```

396 vector=np.zeros(size);vector[indices['g']]=policy.grid;vector[indices['rp']]=policy.
    discharge_cap
397 for j,path in enumerate(net):
398     response=replay(policy,path,initial)
399     for columns,key in zip(indices['responses'],('charge','discharge','emergency','spill','soc')
):
400         vector[columns[j]]=response[key]
401     previous=initial
402     for i,n in enumerate(path):
403         x=n-policy.grid[i]
404         if x>=0:
405             branch=int(np.argmin([policy.discharge_cap[i],max(x,0),ETA_D*(previous-E_MIN)]))
406             vector[indices['lr']][branch][j,i]=1
407             previous=response['soc'][i]
408     return vector
409
410
411 def linear_seed_v2_2bit(policy: Policy, net: np.ndarray, initial: float, indices: dict, size: int)
    -> np.ndarray:
412     vector=np.zeros(size);vector[indices['g']]=policy.grid;vector[indices['rp']]=policy.
        discharge_cap
413     b0,b1=indices['bits']
414     for j,path in enumerate(net):
415         response=replay(policy,path,initial)
416         for columns,key in zip(indices['responses'],('charge','discharge','emergency','spill','soc')
):
417             vector[columns[j]]=response[key]
418             previous=initial
419             for i,n in enumerate(path):
420                 x=n-policy.grid[i]
421                 if x>=0:
422                     branch=int(np.argmin([policy.discharge_cap[i],max(x,0),ETA_D*(previous-E_MIN)]))
423                     vector[b0[j,i]]=float(branch in (1,2));vector[b1[j,i]]=float(branch in (0,2))
424                     previous=response['soc'][i]
425             return vector
426
427
428 def seed_vector(policy: Policy, scenes: Scenarios, prices: np.ndarray, initial: float,
429     indices: dict, size: int, reference: np.ndarray | None) -> np.ndarray:
430     formulation=indices.get('formulation')
431     vector=(linear_seed_v2_free(policy,scenes.net,initial,indices,size) if formulation=='v2_free'
432         else linear_seed_v2_2bit(policy,scenes.net,initial,indices,size) if formulation=='
433             v2_2bit'
434         else linear_seed(policy,scenes.net,initial,indices,size))
435     if len(indices['adjustment']):
436         n=len(indices['adjustment']);vector[indices['adjustment']]=adjustment(policy.grid[:n],
437             reference[:n],prices[:n])
438     return vector
439
440
441 def matrix_error(vector, bounds, integer, constraints) -> tuple[float,float]:
442     if not np.isfinite(vector).all():return float('inf'),float('inf')
443     bound_error=max(0.,float(np.max(bounds.lb-vector)),float(np.max(vector-bounds.ub)))
444     if bound_error>TOL:return bound_error,float('inf')
445     activity=constraints.A@vector
446     if not np.isfinite(activity).all():return float('inf'),float('inf')
447     feasibility=max(0.,float(np.max(bounds.lb-vector)),float(np.max(vector-bounds.ub))),

```

```

446         float(np.max(constraints.lb-activity)),float(np.max(activity-constraints.ub)))
447     integrality=float(np.max(abs(vector[integer==1]-np rint(vector[integer==1]))))
448     return feasibility,integrality
449
450
451 def solve_fast(scenes: Scenarios, prices: np.ndarray, initial: float, config: Config, *, reference=
    None,
452             today: int=144, terminal: float=0., seed: Policy | None=None, known_lower: float |
    None=None,
453             fixed_policy: Policy | None=None, seconds: float | None=None, progress=None,
454             formulation: str | None=None, known_lower_provenance: dict | None=None,
455             deadline: float | None=None) -> tuple:
456     from .optimization import objective as policy_objective
457     if config.feedback!='aggregate':
458         raise ValueError('Lag反馈不能使用单调充电简化')
459     started=time.perf_counter()
460     formulation=config.formulation if formulation is None else formulation
461     if config.solver_backend not in ('auto','highs'):
462         from .gurobi_backend import GurobiUnavailable
463         raise GurobiUnavailable('正式Gurobi后端未启用: 当前许可证不能承载Q3完整矩阵')
464     if formulation not in ('legacy','v2_free','v2_2bit'):raise ValueError('未知 fast formulation')
465     if fixed_policy is not None:formulation='legacy'
466     builder=build_matrix_v2_free if formulation=='v2_free' else build_matrix_v2_2bit if formulation
    == 'v2_2bit' else build_matrix
467     objective,bounds,integer,constraints,indices=builder(scenes,prices,initial,reference=reference,
468     today=today,terminal=terminal,**({'fixed_policy':fixed_policy} if formulation=='legacy' else
    {}))
469     if fixed_policy is not None:seed=fixed_policy
470     elif seed is not None:seed=Policy(np.minimum(seed.grid,np.maximum(scenes.net.max(0),0)+CAP),np.
    full(len(prices),CAP),seed.discharge_cap.copy())
471     vector=seed_vector(seed,scenes,prices,initial,indices,len(objective),reference) if seed else
    None
472     if vector is not None:
473         feasibility,integrality=matrix_error(vector,bounds,integer,constraints)
474         if feasibility>TOL or integrality>1e-8:raise ValueError(f'热启动向量不可行: {feasibility}/{
    integrality}')
475     highs_core._Highs.resetGlobalScheduler(True);solver=HighsAPI()
476     requested_solver_seconds=seconds or config.seconds
477     options={'threads':config.solver_threads,'parallel':'on' if config.solver_threads>1 else 'off',
478             # 原 replay 目标可能小于内部松弛目标, 停止条件只能由下方 assess 回调判定。
479             # MIP logging callback is the periodic durability clock; suppress console/file text
    separately.
480             'output_flag':True,'log_to_console':False,'mip_rel_gap':0.,
481             'mip_min_logging_interval':1.,
482             'mip_feasibility_tolerance':1e-8,'primal_feasibility_tolerance':1e-8}
483     if config.solver_focus=='bound':
484         options.update(mip_heuristic_effort=0.,mip_heuristic_run_feasibility_jump=False,
485             mip_heuristic_run_rens=False,mip_heuristic_run_rins=False,
486             mip_heuristic_run_root_reduced_cost=False,mip_heuristic_run_shifting=False,
487             mip_heuristic_run_zi_round=False)
488     for key,value in options.items():
489         if solver.setOptionValue(key,value)==highs_core.HighsStatus.kError:raise ValueError(f'HiGHS
    不接受参数{key}')
490     lp=highs_core.HighsLp();matrix=constraints.A
491     lp.num_col_,lp.num_row_=len(objective),matrix.shape[0]
492     lp.col_cost_,lp.col_lower_,lp.col_upper_=objective,bounds.lb,bounds.ub
493     lp.row_lower_,lp.row_upper_=constraints.lb,constraints.ub

```

```

494 lp.a_matrix_.num_col_,lp.a_matrix_.num_row_=lp.num_col_,lp.num_row_
495 lp.a_matrix_.format_=highs_core.MatrixFormat.kColwise
496 lp.a_matrix_.start_,lp.a_matrix_.index_,lp.a_matrix_.value_=matrix.indptr,matrix.indices,matrix.
    data
497 lp.integrality_[highs_core.HighsVarType(int(v)) for v in integer]
498 if solver.passModel(lp)==highs_core.HighsStatus.kError:raise ValueError('HiGHS模型加载失败')
499 solver_seconds=requested_solver_seconds
500 if deadline is not None:solver_seconds=max(.01,min(solver_seconds,deadline-time.perf_counter()))
501 if solver.setOptionValue('time_limit',solver_seconds)==highs_core.HighsStatus.kError:
502     raise ValueError('HiGHS不接受剩余hard wall')
503 if vector is not None:
504     hs=highs_core.HighsSolution();hs.col_value=vector;hs.value_valid=True
505     if solver.setSolution(hs)==highs_core.HighsStatus.kError:raise ValueError('HiGHS拒绝已验收的初
        值')
506 digest=matrix_digest(objective,bounds,integer,constraints)
507 if (known_lower is not None and known_lower_provenance and known_lower_provenance.get('kind')==
    'checkpoint'
508     and known_lower_provenance.get('matrix_digest')!=digest):
509     raise ValueError('不同 formulation/matrix 的检查点下界禁止复用')
510 bound_sources=[{'kind':'solver_global_bound','matrix_digest':digest}]
511 if known_lower is not None:
512     bound_sources.append(known_lower_provenance or {'kind':'unspecified_valid_bound'})
513 callback_counts={}
514 base_audit={'status':'searching','solver':'HiGHS_Q3_monotone_equivalent_MILP',
515     'solver_version':solver.version(),'binaries':int(integer.sum()),'variables':len(objective),
516     'constraints':matrix.shape[0],'scenario_count':len(scenes.weights),'requested_gap':config.
    gap,
517     'terminal':terminal,'solver_threads':config.solver_threads,'formulation_version':formulation
    ,
518     'matrix_digest':digest,'objective_offset':0.,'bound_sources':bound_sources,
519     'solver_backend_requested':config.solver_backend,'solver_focus':config.solver_focus,
520     'solver_time_limit':solver_seconds,
521     'callback_counts':callback_counts,'reliable':False}
522 def assess(vector: np.ndarray, lower: float) -> tuple:
523     policy=Policy(np.maximum(vector[indices['g']],0),
524         np.full(len(prices),CAP) if formulation!='legacy' else np.maximum(vector[indices['cp'
    ]],0),
525         np.maximum(vector[indices['rp']],0))
526     responses=[replay(policy,path,initial) for path in scenes.net]
527     raw=np.stack([vector[v] for v in indices['responses']],axis=-1)
528     exact=np.array([np.array([row[k] for k in ('charge','discharge','emergency','spill','soc')])
    .T for row in responses])
529     feasibility,integrality=matrix_error(vector,bounds,integer,constraints)
530     projection=max(0.,float(np.max(raw[... ,4]-exact[... ,4])),float(np.max(exact[... ,2]-raw
    [... ,2])))
531     difference=float(np.max(abs(raw-exact)))
532     value=policy_objective(policy,responses,scenes.weights,prices,reference,min(today,len(prices
    )),terminal)
533     raw_value=float(objective@vector)
534     gap=max(0.,value-lower)/max(abs(value),1e-10)
535     valid=(feasibility<=TOL and integrality<=1e-8 and value<=raw_value+TOL and lower<=value+TOL
    and (difference if fixed_policy is not None else projection)<=TOL)
536     audit={**base_audit}
537     audit.update(objective=value,upper_bound=value,optimizer_objective=raw_value,gap=gap,
    linear_feasibility_error=feasibility,
538         integrality_error=integrality,monotone_projection_error=projection,
    raw_response_difference=difference,

```



```

540         mapping_error=difference if fixed_policy is not None else None,
541         mapped_cost_not_above_optimizer=bool(value<=raw_value+TOL),
542         reliable=bool(valid and value-lower<=max(TOL,config.gap*abs(value)))
543         if not valid:raise ValueError(f'单调等价模型证书不符: {audit}')
544         return policy,responses,audit
545     cached_vector=vector.copy() if vector is not None else None
546     current_lower=known_lower if known_lower is not None else -float('inf')
547     last_save=0.;last_archive=0;cached_assessed_value=None;callback_error=[]
548     if cached_vector is not None and np.isfinite(current_lower):
549         _,_,initial_audit=assess(cached_vector,current_lower);cached_assessed_value=initial_audit['
objective']
550     def callback(event):
551         nonlocal cached_vector,current_lower,last_save,last_archive,cached_assessed_value
552         kind=int(event.callback_type);output=event.data_out;improved=False
553         callback_counts[str(kind)]=callback_counts.get(str(kind),0)+1
554         if np.isfinite(output.mip_dual_bound):current_lower=max(current_lower,float(output.
mip_dual_bound))
555         if kind in (int(highs_core.cb.kCallbackMipImprovingSolution),int(highs_core.cb.
kCallbackMipSolution)):
556             candidate=np.asarray(output.mip_solution).copy()
557             if len(candidate)==len(objective) and np.isfinite(candidate).all():
558                 feasible,integral=matrix_error(candidate,bounds,integer,constraints)
559                 if feasible<=TOL and integral<=1e-8 and (cached_vector is None or
objective@candidate<objective@cached_vector):
560                     cached_vector=candidate.copy();cached_assessed_value=None;improved=True
561                     elapsed=time.perf_counter()-started;archive=int(elapsed//120)
562                     if improved and np.isfinite(current_lower):
563                         try:
564                             policy,responses,audit=assess(cached_vector,current_lower);cached_assessed_value=
audit['objective']
565                             if audit['reliable']:
566                                 audit.update(seconds=elapsed,lower_bound=current_lower,solver_seconds=float(
output.running_time),
567                                     node_count=int(output.mip_node_count),simplex_iterations=int(output.
simplex_iteration_count),
568                                     barrier_iterations=int(output.ipm_iteration_count))
569                                 if progress is not None:progress(policy,responses,audit)
570                                 last_save=elapsed;last_archive=archive;event.interrupt();return
571                         except Exception as error:
572                             callback_error.append(error);event.interrupt();return
573                     certificate_due=(cached_assessed_value is not None and
574                         cached_assessed_value-current_lower<=max(TOL,config.gap*abs(cached_assessed_value)))
575                     durable_due=elapsed-last_save>=30 or archive>last_archive
576                     if progress is not None and cached_vector is not None and np.isfinite(current_lower) and (
certificate_due or durable_due):
577                         try:
578                             policy,responses,audit=assess(cached_vector,current_lower)
579                             audit.update(seconds=elapsed,lower_bound=current_lower)
580                             audit.update(solver_seconds=float(output.running_time),node_count=int(output.
mip_node_count),
581                                     simplex_iterations=int(output.simplex_iteration_count),
582                                     barrier_iterations=int(output.ipm_iteration_count))
583                             progress(policy,responses,audit)
584                             last_save=elapsed;last_archive=archive;cached_assessed_value=audit['objective']
585                             if audit['reliable']:event.interrupt()
586                         except Exception as error:
587                             callback_error.append(error);event.interrupt()

```



```

588     if progress is not None:
589         solver.cbMipImprovingSolution+=callback;solver.cbMipSolution+=callback
590         solver.cbMipLogging+=callback;solver.cbMipInterrupt+=callback
591     if solver.run()==highs_core.HighsStatus.kError:raise ValueError('HiGHS运行失败')
592     if callback_error:raise callback_error[0]
593     info=solver.getInfo();lower=float(info.mip_dual_bound)
594     if known_lower is not None:lower=max(lower,known_lower)
595     lower=max(lower,current_lower)
596     base_audit.update(status=solver.modelStatusToString(solver.getModelStatus()),seconds=time.
597         perf_counter()-started,
598         solver_seconds=float(solver.getRuntime()),lower_bound=lower if np.isfinite(
599             lower) else None,
600         node_count=int(info.mip_node_count),simplex_iterations=int(info.
601             simplex_iteration_count),
602         barrier_iterations=int(info.ipm_iteration_count))
603     if info.primal_solution_status==highs_core.kSolutionStatusFeasible:
604         final=np.asarray(solver.getSolution().col_value)
605         if cached_vector is None or objective@final<objective@cached_vector:cached_vector=final
606     if cached_vector is None:return None,[],base_audit
607     policy,responses,audit=assess(cached_vector,lower)
608     if progress is not None:progress(policy,responses,audit)
609     return policy,responses,audit

```

src/q3/full_run.py

```

1  """主轨迹优先的全量监督：分片续算、异常重启、主表先导出、再做少量实验。"""
2  import argparse
3  from dataclasses import asdict
4  from datetime import datetime, timezone
5  import json
6  import os
7  from pathlib import Path
8  import signal
9  import subprocess
10 import sys
11 import time
12 import traceback
13 import threading
14 from .config import ROOT, CODE_ROOT, Config, write_json, validate_production_rescue
15 from .checkpoint import run_lock
16 from .parallel import limit_native_libraries, validate_parallelism
17
18
19 def now() -> str:
20     return datetime.now(timezone.utc).isoformat()
21
22
23 def worker(root: Path, q2_run: Path, workers: int, config: Config, end: int, samples: bool) -> None:
24     from .data import read_inputs
25     from .rolling import run_period
26     from .export import write_tables, export_workbook
27     from .sampled import run_samples
28     write_json(root/'phase.json', {'phase': 'main', 'started': now()})
29     data = read_inputs(q2_run, ROOT/'inputs/q3/processed')
30     frame = run_period(data, config, root/'main', end=end, with_fiv=False, workers=workers,
31         max_slices=None)

```

```

31     if end < 365:
32         write_json(root/'phase.json', {'phase': 'bounded_test_complete', 'end_day': end, 'finished':
now()})
33         return
34     write_tables(frame, root/'main')
35     export_workbook(frame, root/'main', config)
36     rescue=[]
37     if config.production_rescue:
38         for path in sorted((root/'main/audit').glob('*.json')):
39             day=json.loads(path.read_text(encoding='utf-8'))
40             for node in day.get('nodes',[]):
41                 audit=node['solver'];rescue.append({'date':day['date'],'hour':node['hour'],
42                 'accepted_by':audit.get('accepted_by'),'certified_3pct' if audit.get('reliable')
else 'unknown'},
43                 'objective':audit.get('objective'),'upper_bound':audit.get('upper_bound',audit.
get('objective')),
44                 'lower_bound':audit.get('lower_bound'),'gap':audit.get('gap')})
45             finite=[row for row in rescue if row['gap'] is not None]
46             write_json(root/'main/rescue_summary.json',{'nodes':len(rescue),
47                 'certified_3pct':sum(row['accepted_by']=='certified_3pct' for row in rescue),
48                 'hard_limit_feasible':sum(row['accepted_by']=='hard_limit_feasible' for row in rescue),
49                 'max_recorded_gap':max((row['gap'] for row in finite),default=None),'records':rescue})
50     write_json(root/'main_complete.json', {'rows': len(frame), 'gap_target': config.gap,
51         'result': str(root/'main/result3.xlsx'), 'finished': now(), '
52         sample_experiments_not_needed_for_main_export': True,
53         'production_rescue':config.production_rescue,'all_nodes_certified_3pct':not config.
production_rescue or all(
54             row['accepted_by']=='certified_3pct' for row in rescue)})
55     if samples:
56         write_json(root/'phase.json', {'phase': 'sampled_experiments', 'started': now()})
57         run_samples(data, config, frame, root/'samples', root/'main', workers)
58         write_json(root/'phase.json', {'phase': 'complete', 'finished': now()})
59
60 def progress(root: Path, process_pid: int | None, status: str, attempt: int) -> dict:
61     def read(path: Path) -> dict:
62         return json.loads(path.read_text(encoding='utf-8')) if path.exists() else {}
63     state = read(root/'main/state.json'); active = read(root/'main/active.json')
64     phase=read(root/'phase.json');execution=read(root/'execution.json')
65     rescue=bool(execution.get('config',{}).get('production_rescue'))
66     samples=bool(execution.get('sampled_experiments'))
67     date = active.get('date')
68     candidates = list((root/'main/nodes'/date).glob('*/latest.json')) if date else []
69     latest = max(candidates, key=lambda p: p.stat().st_mtime) if candidates else None
70     node = read(latest).get('content', {}).get('audit', {}) if latest else {}
71     info = {'status': status, 'phase': phase.get('phase', 'starting'), 'worker_pid': process_pid,
72         'attempt': attempt, 'completed_days_including_warmup': len(state.get('days', [])),
73         'formal_days': max(0, len(state.get('days', []))-31), 'total_formal_days': 334,
74         'active_date': date, 'active_hour': latest.parent.name if latest else None,
75         'last_solver_gap': node.get('gap'), 'completed_slices': node.get('completed_slices'),
76         'node_reliable': node.get('reliable'), 'node_accepted':node.get('accepted'),
77         'accepted_by':node.get('accepted_by'),'hard_limit_seconds':node.get('hard_limit_seconds'),
78         'checkpoint': str(latest) if latest else None,
79         'soc': state.get('soc'), 'updated_at': now(), 'main_result_ready': (root/'main_complete.json'
).exists(),
80         'production_rescue':rescue,'completed_sample_jobs':len(list((root/'samples').glob('*/*/*
complete.json'))),

```

```

81     'sample_jobs_total':36 if samples else 0}
82 write_json(root/'progress.json', info)
83 gap = '-' if info['last_solver_gap'] is None else f'{info["last_solver_gap"]:.3%}'
84 body = '<!doctype html><html lang="zh"><meta charset="utf-8"><meta http-equiv="refresh" content
      ="10"><meta name="viewport" content="width=device-width,initial-scale=1"><title>第三问全量计算进度
      </title><style>body{max-width:1000px;margin:40px auto;padding:0 20px;font:17px/1.7 -apple-system
      ,sans-serif;color:#263b50}strong{color:#0072b2}table{border-collapse:collapse}td{padding:8px 18
      px;border-bottom:1px solid #ccd6df}</style><h1>第三问全年主计算</h1>'
85 body += f'<p>状态 <strong>{status}</strong>; 阶段 {info["phase"]}; 正式完成 <strong>{info["
      formal_days"]}/334 天</strong>, 含预热完成 {info["completed_days_including_warmup"]} 天。</p>'
86 body += f'<table><tr><td>当前日期/节点</td><td>{date or "-"} / {info["active_hour"] or "-"}</td></
      tr><tr><td>最新求解间隙</td><td>{gap}</td></tr><tr><td>已完成求解切片</td><td>{info["
      completed_slices"] or 0}</td></tr><tr><td>最近日未SOC</td><td>{info["soc"]}</td></tr><tr><td>监督
      启动次数</td><td>{attempt}</td></tr></table>'
87 body += ('<p>比赛生产救援: 3%证书优先; 06:00最多35秒, 其余节点最多8秒。到限后仅接受通过原物理和完整矩阵验证的可
      行策略, 并如实保存UB、LB和gap。当前只运行主轨迹。</p>' if rescue else
88         '<p>主计算按用户批准的3%最优性间隙推进。每节点/每6小时块/每日存档; 求解有切片及事件快照。</p>')
89 if info['main_result_ready']:
90     body += '<p><a href="main/result3.xlsx">下载主结果 result3.xlsx</a></p>'
91     if info['sample_jobs_total']:
92         body += f'<p>抽样辅助任务 {info["completed_sample_jobs"]}/{info["sample_jobs_total"]}</p>
93 >'
94 body += '<p><a href="progress.json">进度记录</a> • <a href="supervisor.json">运行与重启记录</a> •
95 <a href="worker.log">计算日志</a></p></html>'
96 (root/'progress.html').write_text(body, encoding='utf-8')
97 return info
98
99 def stop_group(child: subprocess.Popen, force: bool = False) -> None:
100     try:
101         os.killpg(child.pid, signal.SIGKILL if force else signal.SIGTERM)
102     except ProcessLookupError:
103         pass
104
105 def last_worker_activity(root: Path) -> float:
106     paths = [root/'phase.json', root/'main/state.json']
107     paths += list((root/'main').rglob('latest.json'))+list((root/'samples').rglob('latest.json'))
108     paths += list(root.rglob('terminal_cache.json'))
109     return max((path.stat().st_mtime for path in paths if path.exists()), default=0.)
110
111 def supervise(root: Path, q2_run: Path, workers: int, config: Config, end: int, samples: bool) ->
112     None:
113     root.mkdir(parents=True, exist_ok=True)
114     with run_lock(root/'supervisor.lock'):
115         frozen = {'config': asdict(config), 'signature': config.signature(), 'q2_run': str(q2_run),
116                 'end_day': end, 'sampled_experiments': samples, 'workers': workers,
117                 'authorization': ('用户授权七小时production rescue: 3%优先, hard limit后仅接受原物理验证
118 可行解'
119                                 if config.production_rescue else '用户允许主计算gap≤3%, 少量辅助实
120 验, 准备完成后自主启动全年')}}
121         path = root/'execution.json'
122         if path.exists() and json.loads(path.read_text(encoding='utf-8')) != json.loads(json.dumps(
            frozen)):
            raise ValueError('已存在运行的代码/配置不同, 禁止混入旧检查点')
        write_json(path, frozen)

```

```

123     commands = [sys.executable, '-m', 'src.q3.full_run', '--worker', '--run-dir', str(root),
124                 '--q2-run', str(q2_run), '--workers', str(workers), '--seconds', str(config.seconds),
125                 '--gap', str(config.gap), '--solver-threads', str(config.solver_threads),
126                 '--solver-backend', config.solver_backend, '--formulation', config.formulation,
127                 '--solver-focus', config.solver_focus,
128                 '--hard-06-seconds', str(config.hard_06_seconds), '--hard-other-seconds', str(config.
hard_other_seconds),
129                 '--end-day', str(end), '--supervisor-pid', str(os.getpid())]
130     if not samples:
131         commands.append('--main-only')
132     if config.production_rescue:
133         commands.append('--production-rescue')
134     existing = root/'supervisor.json'
135     history = json.loads(existing.read_text(encoding='utf-8')).get('history', []) if existing.
exists() else []
136     caffeinate = subprocess.Popen(['caffeinate', '-i', '-w', str(os.getpid())]) if sys.platform
== 'darwin' else None
137     child = None
138     def stop(signum, frame) -> None:
139         if child is not None and child.poll() is None:
140             stop_group(child)
141             raise SystemExit(128+signum)
142     previous_handlers = {s: signal.signal(s, stop) for s in (signal.SIGTERM, signal.SIGINT)}
143     try:
144         for retry in range(1, 4):
145             attempt = len(history)+1
146             with (root/'worker.log').open('ab') as log:
147                 child = subprocess.Popen(commands, cwd=CODE_ROOT.parent.parent, stdout=log,
stderr=subprocess.STDOUT, start_new_session=True)
148                 launched = time.time()
149                 history.append({'attempt': attempt, 'pid': child.pid, 'started_at': now()})
150                 write_json(root/'supervisor.json', {'status': 'running', 'supervisor_pid': os.
getpid(), 'history': history})
151                 while child.poll() is None:
152                     progress(root, child.pid, 'running', attempt)
153                     if time.time()-max(launched, last_worker_activity(root)) > max(600., 5*
config.seconds):
154                         history[-1]['watchdog_timeout'] = True
155                         stop_group(child, force=True)
156                         time.sleep(5)
157                     history[-1].update(return_code=child.returncode, ended_at=now())
158                     stop_group(child, force=True) # 回收异常退出worker遗留的池进程，不影响别的运行组。
159                     if child.returncode == 0:
160                         write_json(root/'supervisor.json', {'status': 'complete', 'history': history})
161                         progress(root, None, 'complete', attempt)
162                         return
163                     write_json(root/'supervisor.json', {'status': 'retrying' if retry < 3 else 'failed',
'history': history})
164                     progress(root, None, 'failed', attempt)
165                     raise RuntimeError('连续三次异常退出：已有检查点保留，停止重复失败')
166     finally:
167         if child is not None and child.poll() is None:
168             stop_group(child)
169         try:
170             child.wait(timeout=10)
171         except subprocess.TimeoutExpired:
172             stop_group(child, force=True); child.wait()

```

```

173         for signum, handler in previous_handlers.items():
174             signal.signal(signum, handler)
175         if caffeinate is not None:
176             caffeinate.terminate(); caffeinate.wait()
177
178
179 def main() -> None:
180     parser = argparse.ArgumentParser(description=__doc__)
181     parser.add_argument('--run-dir', type=Path, required=True)
182     parser.add_argument('--q2-run', type=Path, default=ROOT/'outputs/q2/dispatch_runs/20260911_224501')
183     parser.add_argument('--workers', type=int, default=4)
184     parser.add_argument('--solver-threads', type=int, default=1)
185     parser.add_argument('--solver-backend', choices=('auto', 'highs', 'gurobi'), default='highs')
186     parser.add_argument('--formulation', choices=('legacy', 'v2_free'), default='v2_free')
187     parser.add_argument('--solver-focus', choices=('default', 'bound'), default='default')
188     parser.add_argument('--production-rescue', action='store_true')
189     parser.add_argument('--hard-06-seconds', type=float, default=35.)
190     parser.add_argument('--hard-other-seconds', type=float, default=8.)
191     parser.add_argument('--seconds', type=float, default=120.)
192     parser.add_argument('--gap', type=float, default=.03)
193     parser.add_argument('--end-day', type=int, default=365, help='仅用于有界验收，正式365')
194     parser.add_argument('--main-only', action='store_true')
195     parser.add_argument('--worker', action='store_true')
196     parser.add_argument('--supervisor-pid', type=int)
197     args = parser.parse_args(); limit_native_libraries(); validate_parallelism(args.workers, args.solver_threads)
198     if not 1 <= args.end_day <= 365:
199         parser.error('end-day必须在1—365')
200     if args.production_rescue and not args.main_only:
201         parser.error('production rescue只允许--main-only，禁止辅助实验')
202     config = Config(seconds=args.seconds, gap=args.gap, solver_threads=args.solver_threads,
203                    solver_backend=args.solver_backend, formulation=args.formulation,
204                    solver_focus=args.solver_focus, production_rescue=args.production_rescue,
205                    hard_06_seconds=args.hard_06_seconds, hard_other_seconds=args.hard_other_seconds)
206     if config.production_rescue: validate_production_rescue(config)
207     function = worker if args.worker else supervise
208     finished = threading.Event()
209     if args.worker and args.supervisor_pid:
210         def watch_owner() -> None:
211             while not finished.wait(2):
212                 if os.getppid() != args.supervisor_pid:
213                     if os.getpgrp() == os.getpid():
214                         os.killpg(os.getpgrp(), signal.SIGTERM)
215                     else:
216                         os.kill(os.getpid(), signal.SIGTERM)
217             threading.Thread(target=watch_owner, daemon=True).start()
218     try:
219         function(args.run_dir.resolve(), args.q2_run.resolve(), args.workers, config, args.end_day,
220                not args.main_only)
221     except Exception as error:
222         write_json(args.run_dir.resolve()/('worker_error.json' if args.worker else 'supervisor_error.json'),
223                  {'type': type(error).__name__, 'error': str(error), 'traceback': traceback.format_exc(), 'time': now()})
224         raise
225     finally:

```

```

225         finished.set()
226
227
228 if __name__ == '__main__':
229     main()

```

src/q3/gurobi_backend.py

```

1  """同一 SciPy 矩阵的可选 Gurobi 后端; 许可证不足时明确失败, 不静默改模型。"""
2  from __future__ import annotations
3  import time
4  from typing import Callable
5  import numpy as np
6  from .config import TOL
7
8
9  class GurobiUnavailable(RuntimeError):
10     pass
11
12
13  def license_probe() -> dict:
14     try:
15         import gurobipy as gp
16         from gurobipy import GRB
17     except ImportError as error:
18         return {'installed': False, 'usable': False, 'reason': 'not_installed', 'error': str(error)}
19     try:
20         model=gp.Model('q3_license_probe');model.Params.OutputFlag=0
21         value=model.addVar(vtype=GRB.BINARY);model.setObjective(value,GRB.MAXIMIZE);model.optimize()
22         return {'installed': True, 'usable': model.Status==GRB.OPTIMAL, 'version': list(gp.gurobi.version
            ()),
23                 'tiny_status': int(model.Status), 'tiny_objective': float(model.ObjVal)}
24     except gp.GurobiError as error:
25         return {'installed': True, 'usable': False, 'version': list(gp.gurobi.version()),
26                 'reason': 'license_error', 'error_code': int(error.errno), 'error': str(error)}
27
28
29  def solve_gurobi_matrix(objective, bounds, integer, constraints, *, seed_vector: np.ndarray,
30                          gap: float, seconds: float, threads: int, assess: Callable,
31                          known_lower: float | None=None, progress: Callable | None=None,
32                          mip_focus: int=0) -> tuple[np.ndarray, float, dict]:
33     """解完全相同的稀疏矩阵; 返回向量和 Gurobi 全局下界, 语义验收由调用者的 assess 完成。"""
34     try:
35         import gurobipy as gp
36         from gurobipy import GRB
37     except ImportError as error:
38         raise GurobiUnavailable('gurobipy未安装') from error
39     objective=np.asarray(objective,dtype=float);seed_vector=np.asarray(seed_vector,dtype=float)
40     matrix=constraints.A.tocsr();started=time.perf_counter()
41     model=gp.Model('q3_same_matrix');model.Params.OutputFlag=0;model.Params.MIPGap=gap
42     model.Params.TimeLimit=seconds;model.Params.Threads=threads;model.Params.FeasibilityTol=1e-8
43     model.Params.IntFeasTol=1e-8;model.Params.MIPFocus=mip_focus
44     vtypes=np.where(np.asarray(integer)==1,GRB.BINARY,GRB.CONTINUOUS)
45     variables=model.addMVar(len(objective),lb=bounds.lb,ub=bounds.ub,obj=objective,vtype=vtypes)
46     equal=constraints.lb==constraints.ub
47     if equal.any():model.addMConstr(matrix[equal],variables,'=',constraints.ub[equal])

```

```

48 lower=np.isfinite(constraints.lb)&~equal
49 if lower.any():model.addMConstr(matrix[lower],variables,'>',constraints.lb[lower])
50 upper=np.isfinite(constraints.ub)&~equal
51 if upper.any():model.addMConstr(matrix[upper],variables,'<',constraints.ub[upper])
52 variables.Start=seed_vector
53 state={'vector':seed_vector.copy(), 'raw':float(objective@seed_vector),
54        'lower':float(known_lower) if known_lower is not None else -float('inf'),
55        'last_save':0., 'last_archive':0, 'assessed_value':None, 'records':{}}
56 def callback(active,where):
57     if where==GRB.Callback.MIPSOL:
58         candidate=np.asarray(active.cbGetSolution(variables),dtype=float)
59         if np.isfinite(candidate).all() and float(objective@candidate)<state['raw']-TOL:
60             state['vector']=candidate.copy();state['raw']=float(objective@candidate)
61             state['lower']=max(state['lower'],float(active.cbGet(GRB.Callback.MIPSOL_OBJBND)))
62     elif where==GRB.Callback.MIP:
63         state['lower']=max(state['lower'],float(active.cbGet(GRB.Callback.MIP_OBJBND)))
64     else:
65         return
66     elapsed=time.perf_counter()-started;archive=int(elapsed//120)
67     certificate_due=(state['assessed_value'] is not None and
68                    state['assessed_value']-state['lower']<=max(TOL,gap*abs(state['assessed_value'])))
69     if progress is not None and (certificate_due or elapsed-state['last_save']>=30 or archive>
state['last_archive']):
70         policy,responses,audit=assess(state['vector'],state['lower'])
71         audit.update(seconds=elapsed,lower_bound=state['lower'],solver='Gurobi_same_matrix',
72                    solver_version='.'.join(map(str, gp.gurobi.version())),solver_threads=
threads,
73                    mip_focus=mip_focus)
74         progress(policy,responses,audit);state['last_save']=elapsed;state['last_archive']=
archive
75         state['assessed_value']=audit['objective']
76         state['records'].append({'seconds':elapsed, 'objective':audit['objective'],
77                                'lower_bound':state['lower'], 'gap':audit['gap'], 'reliable':
audit['reliable']})
78         if audit['reliable']:active.terminate()
79     try:
80         model.optimize(callback)
81     except gp.GurobiError as error:
82         reason='size_limited_license' if 'too large' in str(error).lower() else 'gurobi_error'
83         raise GurobiUnavailable(f'{reason}: {error}') from error
84     if model.SolCount:
85         candidate=np.asarray(variables.X,dtype=float)
86         if float(objective@candidate)<state['raw']-TOL:state['vector']=candidate;state['raw']=float(
objective@candidate)
87     final_lower=max(state['lower'],float(model.ObjBound))
88     metadata={'status':int(model.Status), 'solver':'Gurobi_same_matrix',
89              'solver_version':.'.'.join(map(str, gp.gurobi.version())), 'solver_seconds':float(model.
Runtime),
90              'node_count':float(model.NodeCount), 'simplex_iterations':float(model.IterCount),
91              'barrier_iterations':float(model.BarIterCount), 'solver_threads':threads, 'mip_focus':
mip_focus,
92              'trajectory':state['records']}
93     return state['vector'],final_lower,metadata

```

src/q3/incumbent.py


```

1  """只改进MILP初始可行策略：分段梯度不提供任何全局最优性声明。"""
2  import time
3  import numpy as np
4  from scipy.optimize import minimize
5  from .config import CAP, E_MIN, E_MAX, ETA
6  from .physics import Policy, adjustment
7
8
9  def value_gradient(vector: np.ndarray, net: np.ndarray, prices: np.ndarray, weights: np.ndarray,
10                    initial: float, reference: np.ndarray | None, today: int, terminal: float) ->
11      tuple[float, np.ndarray]:
12      t = len(prices); g, cp, rp = vector.reshape(3, t)
13      state = np.full(len(net), initial)
14      if reference is None:
15          value, price_gradient = float(prices@g), prices.copy()
16      else:
17          value = float(adjustment(g[:today], reference[:today], prices[:today]).sum()+prices[today:]
18                        @g[today:])
19          price_gradient = prices.copy(); price_gradient[:today] *= np.where(g[:today] >= reference[:
20                                today], 1.5, .5)
21      derivatives = []
22      for i in range(t):
23          x = net[:, i]-g[i]
24          cterms = np.array([np.full(len(net), cp[i]), np.maximum(-x, 0), np.maximum((E_MAX-state)/ETA
25                                , 0)])
26          rterms = np.array([np.full(len(net), rp[i]), np.maximum(x, 0), np.maximum(ETA*(state-E_MIN),
27                                0)])
28          ci, ri = cterms.argmin(0), rterms.argmin(0)
29          c, r = cterms.min(0), rterms.min(0)
30          shortage = x-r > 0
31          value += float(5*prices[i]*(np.maximum(x-r, 0)@weights))
32          derivatives.append(((x < 0)&(ci == 1), (x < 0)&(ci == 0), -((x < 0)&(ci == 2)).astype(float)
33                                /ETA,
34                                -((x > 0)&(ri == 1)).astype(float), (x > 0)&(ri == 0), ((x > 0)&(ri ==
35                                2)).astype(float)*ETA, shortage))
36          state += ETA*c-r/ETA
37      value -= float(terminal*(weights@state))
38      gradient = np.zeros((3, t)); adjoint = -terminal*weights
39      for i in range(t-1, -1, -1):
40          cg, cc, ce, rg, rr, re, shortage = derivatives[i]
41          emergency_price = 5*prices[i]*weights*shortage
42          gradient[0, i] = price_gradient[i]+np.sum(emergency_price*(-1-rg)+adjoint*(ETA*cg-rg/ETA))
43          gradient[1, i] = np.sum(adjoint*ETA*cc)
44          gradient[2, i] = np.sum(-emergency_price*rr-adjoint*rr/ETA)
45          adjoint = -emergency_price*re+adjoint*(1+ETA*ce-re/ETA)
46      return value, gradient.ravel()
47
48
49 def refine(policy: Policy, net: np.ndarray, prices: np.ndarray, weights: np.ndarray, initial: float,
50           reference: np.ndarray | None, today: int, terminal: float, upper: np.ndarray,
51           seconds: float) -> Policy:
52     started = time.perf_counter()
53     vector = np.r_[policy.grid, policy.charge_cap, policy.discharge_cap]
54     best = [value_gradient(vector, net, prices, weights, initial, reference, today, terminal)[0],
55            vector.copy()]

```



```

49 class BudgetReached(Exception):
50     pass
51
52 def evaluate(v: np.ndarray) -> tuple[float, np.ndarray]:
53     value, gradient = value_gradient(v, net, prices, weights, initial, reference, today,
54     terminal)
55     if value < best[0]:
56         best[:] = [value, v.copy()]
57     if time.perf_counter()-started > seconds:
58         raise BudgetReached
59     return value, gradient
60
61 try:
62     minimize(evaluate, vector, jac=True, method='L-BFGS-B',
63             bounds=[(0., float(v)) for v in upper]+[(0., CAP)]*(2*len(prices)),
64             options={'maxiter': 100, 'maxls': 20, 'ftol': 1e-10})
65 except BudgetReached:
66     pass
67
68 return Policy(*best[1].reshape(3, len(prices)))

```

src/q3/migrate_rescue.py

```

1  """把已停止的严格运行迁移为限时可行 production rescue; 旧目录始终保留。"""
2  import argparse
3  from dataclasses import asdict, replace
4  import hashlib
5  import json
6  from pathlib import Path
7  import shutil
8  import shlex
9  from unittest.mock import patch
10 import numpy as np
11 import pandas as pd
12 from .config import ROOT, CODE_ROOT, Config, E_INITIAL, TOL, write_json, validate_production_rescue
13 from .checkpoint import request_digest, load_snapshot, save_snapshot
14 from .data import read_inputs
15 from .scenarios import construct
16 from .physics import replay
17 from .optimization import objective, accept_feasible_incumbent
18 from .rolling import previous_net, validate_frame
19 from .status_server import worker_alive
20
21
22 def migrate(old: Path, new: Path, hard_06: float, hard_other: float) -> dict:
23     old=old.resolve();new=new.resolve();progress=json.loads((old/'progress.json').read_text(encoding
24     ='utf-8'))
25     if worker_alive(progress.get('worker_pid'),old):raise RuntimeError('旧主仍在运行, 拒绝复制活动现场')
26     if new.exists():raise FileExistsError(new)
27     original=json.loads((old/'execution.json').read_text(encoding='utf-8'));values=dict(original['
28     config'])
29     values['nodes']=tuple(values['nodes']);values.setdefault('solver_backend','highs');values.
30     setdefault('formulation','legacy')
31     values.setdefault('solver_focus','default');values.setdefault('production_rescue',False)
32     values.setdefault('hard_06_seconds',hard_06);values.setdefault('hard_other_seconds',hard_other)
33     before=Config(**values);after=replace(before,solver_threads=4,solver_backend='highs',formulation
34     ='legacy',

```

```

31     solver_focus='default',production_rescue=True,hard_06_seconds=hard_06,hard_other_seconds=
    hard_other)
32 validate_production_rescue(after)
33 frozen=json.loads((old/'runtime_sources.json').read_text(encoding='utf-8'))
34 for name,digest in frozen['files'].items():
35     if hashlib.sha256((old/'runtime/src/q3'/name).read_bytes()).hexdigest()!=digest:
36         raise ValueError(f'旧冻结源码摘要不符: {name}')
37 requirements=(old/'runtime/requirements.txt').read_bytes()
38 if hashlib.sha256(requirements).hexdigest()!=frozen['requirements_sha256'] or b'highspy==1.15.1'
    not in requirements:
39     raise ValueError('旧冻结依赖摘要或HiGHS版本不符')
40 runtime=new/'runtime/src/q3';runtime.mkdir(parents=True)
41 for source in CODE_ROOT.glob('*.py'):
42     if not source.name.startswith('test_'):shutil.copy2(source,runtime/source.name)
43 (new/'runtime/src/__init__.py').write_text('',encoding='utf-8')
44 (new/'runtime/requirements.txt').write_bytes(requirements)
45 manifest={path.name:hashlib.sha256(path.read_bytes()).hexdigest() for path in runtime.glob('*.py
    ')}
46 signature_payload=(ROOT/'docs/3/Q3_第三问_最终建模_严格修订终稿_v4.md').read_bytes()
47 signature_payload+=json.dumps(asdict(after),sort_keys=True).encode()
48 for path in sorted(runtime.glob('*.py')):
49     signature_payload+=path.name.encode()+path.read_bytes()
50 frozen_signature=hashlib.sha256(signature_payload).hexdigest()
51 if frozen_signature!=after.signature():raise ValueError('复制期间源码变化, 冻结runtime签名不一致')
52 write_json(new/'runtime_sources.json',{'files':manifest,
53     'requirements_sha256':hashlib.sha256(requirements).hexdigest(),'previous_run':str(old),
54     'workspace':str(ROOT),'solver':'HiGHS 1.15.1','formulation':'legacy fast exact-equivalent',
55     'production_rescue':True,'config_signature':frozen_signature})
56 shutil.copytree(old/'main',new/'main')
57 for legacy_slice in (new/'main/nodes').glob('*/*/slice_*.json'):legacy_slice.unlink()
58 data=read_inputs(Path(original['q2_run']),new/'migration_prepared')
59 state=json.loads((new/'main/state.json').read_text(encoding='utf-8'))
60 sources=hashlib.sha256(json.dumps(data.provenance,sort_keys=True).encode()).hexdigest()
61 if sources!=state['sources']:raise ValueError('原始数据或Q2来源变化')
62 if len(state.get('days',[]))!=25 or state.get('with_fiv') is not False:
63     raise ValueError('只接受已核实的25日main-only前缀')
64 frames=[]
65 for receipt in state['days']:
66     date=str((pd.Timestamp('2025-01-01')+pd.Timedelta(days=receipt['day'])).date())
67     files={'dispatch':f'days/{date}.csv','audit':f'audit/{date}.json','information':f'
    information/{date}.json'}
68     for key,relative in files.items():
69         copied=new/'main'/relative;source=old/'main'/relative
70         if copied.read_bytes()!=source.read_bytes() or hashlib.sha256(copied.read_bytes()).
    hexdigest()!=receipt['hashes'][key]:
71             raise ValueError(f'已提交日期字节或收据不符: {date}/{key}')
72     frames.append(pd.read_csv(new/f'main/days/{date}.csv',float_precision='round_trip'))
73 validate_frame(pd.concat(frames,ignore_index=True),before)
74 if abs(float(frames[-1].soc.iloc[-1])-float(state['soc']))>TOL:raise ValueError('state SOC与
    Jan25日末不符')
75 records=[];rescued=[]
76 for source_path in sorted((old/'main/nodes').glob('*/*/latest.json')):
77     date=source_path.parent.parent.name;hour=int(source_path.parent.name)
78     day=(pd.Timestamp(date)-pd.Timestamp('2025-01-01')).days;folder=old/'main/nodes'/date
79     scenes=construct(data,day,hour,before);saved=json.loads(source_path.read_text(encoding='utf
    -8'))['content']
80     terminal=float(saved['audit']['terminal'])

```

```

81     if hour:
82         initial=float(pd.read_csv(folder/f'{hour-6:02d}/executed_block.csv',float_precision='
round_trip').soc.iloc[-1])
83         midnight=json.loads((folder/'00/latest.json').read_text(encoding='utf-8'))['content']['
policy']['grid']
84         reference=np.asarray(midnight,dtype=float)[hour*6:]
85     else:
86         prior=str((pd.Timestamp(date)-pd.Timedelta(days=1)).date())
87         initial=float(pd.read_csv(oid/f'main/days/{prior}.csv',float_precision='round_trip').soc
.iloc[-1]) if day else E_INITIAL
88         reference=None
89         prices=data.prices[(hour*6+np.arange(144))%144];today=144-hour*6
90         inputs=(scenes.load,scenes.pv,scenes.weights,prices,initial,reference,today,terminal,
91             previous_net(data,day,hour),None,None,None)
92         with patch.object(Config,'signature',return_value=original['signature']):old_request=
request_digest(before,*inputs)
93         policy,audit=load_snapshot(source_path,old_request)
94         responses=[replay(policy,path,initial) for path in scenes.net]
95         value=objective(policy,responses,scenes.weights,prices,reference,today,terminal)
96         lower=audit.get('lower_bound')
97         if abs(value-audit['objective'])>TOL or (lower is not None and lower>value+TOL):
98             raise ValueError(f'旧节点重算失败: {date}/{hour:02d}')
99         if audit.get('reliable'):
100             if value-float(lower)>max(TOL,before.gap*abs(value)):raise ValueError('旧3%证书不符')
101             migrated={**audit,'upper_bound':value,'accepted':True,'certificate_met':True,'
accepted_by':'certified_3pct',
102                 'migrated_from_request_signature':old_request,'migration_replay_validated':True}
103         else:
104             hard=hard_06 if hour==6 else hard_other
105             accepted=accept_feasible_incumbent(policy,scenes,prices,initial,after,reference=
reference,
106                 today=today,terminal=terminal,previous_net=previous_net(data,day,hour),audit=audit,
107                 hard_seconds=hard,source='saved_incumbent_migrated_at_rescue_start')
108             migrated={**accepted.audit,'migrated_from_request_signature':old_request,
109                 'legacy_elapsed_before_rescue':audit.get('seconds'),'hard_limit_reached':False
,
110                 'accepted_from_preexisting_checkpoint':True,
111                 'lower_bound_origin':'preexisting_HiGHS_1.15.1_same_legacy_matrix'}
112             rescued.append({'date':date,'hour':hour,'upper_bound':migrated['upper_bound'],
113                 'lower_bound':migrated['lower_bound'],'gap':migrated['gap'],'objective':migrated['
objective']})
114             new_request=request_digest(after,*inputs);save_snapshot(new/source_path.relative_to(oid),
new_request,policy,migrated)
115             records.append({'date':date,'hour':hour,'old_request':old_request,'new_request':new_request,
116                 'accepted_by':migrated['accepted_by']})
117         if [(row['date'],row['hour']) for row in rescued]!=(('2025-01-26',6)):
118             raise ValueError(f'未可靠节点集合不是唯一Jan26/06: {rescued}')
119         state['signature']=after.signature();state.setdefault('execution_history',[]).append(
120             {'source':str(oid),'signature':original['signature'],'days':len(state['days']),'mode':
'production_rescue'})
121         write_json(new/'main/state.json',state)
122         cache_path=new/'main/terminal_cache.json';wrapper=json.loads(cache_path.read_text(encoding='utf
-8'));content=wrapper['content']
123         payload=json.dumps(content,sort_keys=True,allow_nan=False).encode()
124         if wrapper['sha256']!=hashlib.sha256(payload).hexdigest():raise ValueError('FD缓存摘要不符')
125         expected_old=hashlib.sha256((original['signature']+json.dumps(data.provenance,sort_keys=True)).
encode()).hexdigest()

```

```

126 if content.get('signature')!=expected_old:raise ValueError('FD缓存与旧运行签名不符')
127 rows_digest=hashlib.sha256(json.dumps(content['rows'],sort_keys=True,allow_nan=False).encode()).
    hexdigest()
128 calibration_path=new/'main/terminal_calibration.csv';calibration_sha=hashlib.sha256(
    calibration_path.read_bytes()).hexdigest()
129 calibration=pd.read_csv(calibration_path,float_precision='round_trip')
130 cache_keys={(int(row['history_day']),int(row['hour']),float(row['delta'])):row for row in
    content['rows']}
131 for row in calibration.to_dict('records'):
132     key=(int(row['history_day']),int(row['hour']),float(row['delta']))
133     if key not in cache_keys or abs(float(row['value'])-float(cache_keys[key]['value']))>TOL:
134         raise ValueError('CSV终端标定不是durable JSON缓存子集')
135 content['signature']=hashlib.sha256((after.signature()+json.dumps(data.provenance,sort_keys=True
    )).encode()).hexdigest()
136 payload=json.dumps(content,sort_keys=True,allow_nan=False).encode();write_json(cache_path,{
    'sha256':hashlib.sha256(payload).hexdigest(),'content':content})
137 uv=shutil.which('uv') or '/opt/homebrew/bin/uv'
138 command=[uv,'run','--offline','--with-requirements',str(new/'runtime/requirements.txt'),'python'
    ,'-m','src.q3.full_run',
139     '--run-dir',str(new),'--q2-run',str(Path(original['q2_run'])), '--gap','0.03','--seconds','120
    ',
140     '--workers','2','--solver-threads','4','--solver-backend','highs','--formulation','legacy',
141     '--solver-focus','default','--production-rescue','--hard-06-seconds','35','--hard-other-
    seconds','8','--main-only']
142 resume='#!/bin/sh\nset -eu\ncd '+shlex.quote(str(new/'runtime'))+'\nexport Q3_WORKSPACE_ROOT='+
    shlex.quote(str(ROOT))+'\nexport OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 MKL_NUM_THREADS=1
    VECLIB_MAXIMUM_THREADS=1 NUMEXPR_NUM_THREADS=1\nexec '+shlex.join(command)+'\n'
143 (new/'resume.sh').write_text(resume,encoding='utf-8'); (new/'resume.sh').chmod(0o755)
144 report={'old_run':str(old),'new_run':str(new),'old_signature':original['signature'],'
    new_signature':after.signature(),
145     'days_byte_identical':len(state['days']), 'nodes_revalidated':len(records), '
    rescued_unfinished_nodes':rescued,
146     'hard_06_seconds':hard_06, 'hard_other_seconds':hard_other, 'solver':'HiGHS 1.15.1', 'threads'
    :4,
147     'formulation':'legacy fast exact-equivalent', 'model_parameters_unchanged':True, '
    legacy_slices_copied':False,
148     'old_directory_preserved':True, 'FD_rows':len(content['rows']), 'FD_rows_sha256':rows_digest,
149     'terminal_calibration_rows':len(calibration), 'terminal_calibration_sha256':calibration_sha,
150     'terminal_calibration_is_cache_subset':True, 'resume_command':command, 'records':records}
151 write_json(new/'migration.json',report);return report
152
153
154 if __name__=='__main__':
155     parser=argparse.ArgumentParser();parser.add_argument('--old',type=Path,required=True);parser.
    add_argument('--new',type=Path,required=True)
156     parser.add_argument('--hard-06',type=float,default=35.);parser.add_argument('--hard-other',type=
    float,default=8.)
157     args=parser.parse_args();result=migrate(args.old,args.new,args.hard_06,args.hard_other)
158     print(json.dumps({'days':result['days_byte_identical'],'nodes':result['nodes_revalidated'],
    'rescued':result['rescued_unfinished_nodes'],'signature':result['new_signature']
    '}}))

```

src/q3/migrate_solver.py

```

1 """等价求解器升级时逐节点核移并迁移；仅对已停止的旧运行执行。"""
2 import argparse

```

```

3 from dataclasses import replace
4 import hashlib
5 import json
6 from pathlib import Path
7 import shutil
8 from unittest.mock import patch
9 import numpy as np
10 import pandas as pd
11 from .config import ROOT, CODE_ROOT, Config, E_INITIAL, TOL, write_json
12 from .checkpoint import request_digest, load_snapshot, save_snapshot
13 from .data import read_inputs
14 from .scenarios import construct
15 from .physics import replay
16 from .optimization import objective
17 from .fast_solver import build_matrix_v2_free, matrix_digest
18 from .rolling import previous_net, validate_frame
19 from .status_server import worker_alive
20
21
22 def migrate(old: Path, new: Path, threads: int) -> dict:
23     old=old.resolve();new=new.resolve()
24     progress=json.loads((old/'progress.json').read_text(encoding='utf-8'))
25     if worker_alive(progress.get('worker_pid'),old):raise RuntimeError('必须先停止旧运行，避免复制活动提交')
26     if new.exists():raise FileExistsError(new)
27     original=json.loads((old/'execution.json').read_text(encoding='utf-8'))
28     values=dict(original['config']);values['nodes']=tuple(values['nodes'])
29     values.setdefault('solver_backend','highs');values.setdefault('formulation','legacy')
30     before=Config(**values)
31     after=replace(before,solver_threads=threads,solver_backend='highs',formulation='v2_free')
32     for name,digest in json.loads((old/'runtime_sources.json').read_text(encoding='utf-8'))['files'].items():
33         if hashlib.sha256((old/'runtime/src/q3'/name).read_bytes()).hexdigest()!=digest:raise
34         ValueError('旧冻结源码摘要不符')
35         runtime=new/'runtime/src/q3';runtime.mkdir(parents=True)
36         for p in CODE_ROOT.glob('*.py'):
37             if not p.name.startswith('test_'):shutil.copy2(p,runtime/p.name)
38             (new/'runtime/src/__init__.py').write_text('',encoding='utf-8')
39             requirements=(ROOT/'src/q2/requirements.txt').read_text(encoding='utf-8')+'\nhighspy==1.15.1\n'
40             (new/'runtime/requirements.txt').write_text(requirements,encoding='utf-8')
41             manifest={p.name:hashlib.sha256(p.read_bytes()).hexdigest() for p in runtime.glob('*.py')}
42             write_json(new/'runtime_sources.json',{ 'files':manifest,'requirements_sha256':hashlib.sha256(
43                 requirements.encode()).hexdigest(),
44                 'previous_run':str(old),'workspace':str(ROOT),'solver':'highspy==1.15.1','basis':'unchanged
45                 build_matrix, physics and model; solver implementation upgrade'})
46             shutil.copytree(old/'main',new/'main')
47             for legacy_slice in (new/'main/nodes').glob('*/*/slice_*.json'):
48                 legacy_slice.unlink()
49             data=read_inputs(Path(original['q2_run']),new/'migration_prepared')
50             state=json.loads((new/'main/state.json').read_text(encoding='utf-8'))
51             sources=hashlib.sha256(json.dumps(data.provenance,sort_keys=True).encode('utf-8')).hexdigest()
52             if sources!=state['sources']:raise ValueError('原始输入/Q2来源改变，拒绝迁移')
53             frames=[]
54             for row in state['days']:
55                 date=str((pd.Timestamp('2025-01-01')+pd.Timedelta(days=row['day'])).date())
56                 for key,path in {'dispatch':f'days/{date}.csv','audit':f'audit/{date}.json','information':f'
57                     information/{date}.json'}.items():

```

```

54     p=new/'main'/path
55     if p.read_bytes()!=(old/'main'/path).read_bytes() or hashlib.sha256(p.read_bytes()).
hexdigest() !=row['hashes'][key]:raise ValueError('原日收据/字节不符')
56     frames.append(pd.read_csv(new/f'main/days/{date}.csv',float_precision='round_trip'))
57     validate_frame(pd.concat(frames,ignore_index=True),before)
58     records=[]
59     for p in sorted((old/'main/nodes').glob('*/*/latest.json')):
60         date=p.parent.parent.name;hour=int(p.parent.name);day=(pd.Timestamp(date)-pd.Timestamp('
2025-01-01')).days
61         folder=old/'main/nodes'/date;scenes=construct(data,day,hour,before)
62         saved=json.loads(p.read_text(encoding='utf-8'))['content'];terminal=saved['audit']['terminal
']
63         if hour:
64             initial=float(pd.read_csv(folder/f'{hour-6:02d}/executed_block.csv',float_precision='
round_trip').soc.iloc[-1])
65             midnight=json.loads((folder/'00/latest.json').read_text(encoding='utf-8'))['content']['
policy']['grid']
66             reference=np.asarray(midnight)[hour*6:]
67         else:
68             prior=str((pd.Timestamp(date)-pd.Timedelta(days=1)).date())
69             initial=float(pd.read_csv(old/f'main/days/{prior}.csv',float_precision='round_trip').soc
.iloc[-1]) if day else E_INITIAL
70             reference=None
71             prices=data.prices[(hour*6+np.arange(144))%144];today=144-hour*6
72             inputs=(scenes.load,scenes.pv,scenes.weights,prices,initial,reference,today,terminal,
previous_net(data,day,hour),None,None,None)
73             with patch.object(Config,'signature',return_value=original['signature']):old_request=
request_digest(before,*inputs)
74             policy,audit=load_snapshot(p,old_request)
75             response=[replay(policy,n,initial) for n in scenes.net]
76             cost=objective(policy,response,scenes.weights,prices,reference,today,terminal)
77             if abs(cost-audit['objective'])>TOL or audit['lower_bound']>cost+TOL:raise ValueError('原节点
政策/下界重算不符')
78             if audit['reliable'] and cost-audit['lower_bound']>max(TOL,after.gap*abs(cost)):raise
ValueError('原节点精度不符')
79             request=request_digest(after,*inputs)
80             if audit.get('reliable'):
81                 migrated_audit={**audit,'migrated_from_request_signature':old_request,
82                                 'certificate_origin_formulation':audit.get('formulation_version','legacy'),
83                                 'migration_replay_objective_verified':True}
84             else:
85                 matrix=build_matrix_v2_free(scenes,prices,initial,reference=reference,today=today,
terminal=terminal)
86                 migrated_audit={'status':'migrated_policy_only_bound_reset','objective':cost,'
lower_bound':None,
87                                 'gap':None,'reliable':False,'completed_slices':0,'seconds':0.,'solver_threads':
threads,
88                                 'solver':'pending_HiGHS','solver_backend_requested':'highs','formulation_version':
v2_free',
89                                 'matrix_digest':matrix_digest(*matrix[:4]),'bound_sources':[],
90                                 'migrated_from_request_signature':old_request,'legacy_bound_discarded':audit.get('
lower_bound'),
91                                 'migration_replay_objective_verified':True}
92                 save_snapshot(new/p.relative_to(old),request,policy,migrated_audit)
93                 records.append({'date':date,'hour':hour,'old_request':old_request,'new_request':request,'
reliable':audit['reliable']})
94                 state['signature']=after.signature()

```

```

95     state.setdefault('execution_history', []).append({'source':str(old), 'signature':original['
signature'], 'days':len(state['days'])})
96     write_json(new/'main/state.json', state)
97     cache_path=new/'main/terminal_cache.json'; cache=json.loads(cache_path.read_text(encoding='utf-8'
)); content=cache['content']
98     if cache['sha256']!=hashlib.sha256(json.dumps(content, sort_keys=True, allow_nan=False).encode()).
hexdigest(): raise ValueError('FD快照校验失败')
99     content['signature']=hashlib.sha256((after.signature()+json.dumps(data.provenance, sort_keys=True
)).encode()).hexdigest()
100    write_json(cache_path, {'sha256':hashlib.sha256(json.dumps(content, sort_keys=True, allow_nan=False
).encode()).hexdigest(), 'content':content})
101    report={'old_run':str(old), 'old_signature':original['signature'], 'new_signature':after.signature
()},
102           'days_byte_identical':len(state['days']), 'node_records':records, '
all_old_request_signatures_verified':True, 'precision':after.gap,
103           'solver_threads':threads, 'all_committed_days_physics_verified':True, 'FD_values_unchanged':
True}
104    report.update(solver_backend='highs', solver_version='1.15.1', formulation_version='v2_free',
105                 legacy_search_slices_copied=False, unreliable_node_bounds_reset=True)
106    write_json(new/'migration.json', report)
107    return report
108
109
110 if __name__=='__main__':
111     parser=argparse.ArgumentParser(); parser.add_argument('--old', type=Path, required=True); parser.
add_argument('--new', type=Path, required=True)
112     parser.add_argument('--threads', type=int, default=4); args=parser.parse_args()
113     result=migrate(args.old, args.new, args.threads)
114     print(json.dumps({'days':result['days_byte_identical'], 'nodes':len(result['node_records']), '
signature':result['new_signature']}))

```

src/q3/multistage.py

```

1  """3.23: 阶段条件聚类四阶段树与实际求解，严格限于ex-post诊断。"""
2  import numpy as np
3  import pandas as pd
4  import time
5  from pyscipopt import quicksum
6  from .config import CAP, Config, DT, TOL
7  from .data import Inputs
8  from .scenarios import medoids, pool_indices
9  from .physics import Policy, replay, adjustment
10 from .optimization import make_model, optimize_model, add_path, scheduled_cost, LimitedSolve
11 from .rolling import frame_for, validate_frame
12
13
14 def conditional_tree(revisions: np.ndarray) -> tuple[np.ndarray, list[dict]]:
15     """每父节点至少4条且每子节点至少2条才能分支，避免叶节点揭示唯一未来轨迹。"""
16     n = len(revisions)
17     if revisions.ndim != 3 or revisions.shape[1] != 3 or n == 0:
18         raise ValueError('需要history×三阶段×共同目标修正向量')
19     labels = np.zeros((n, 4), dtype=int)
20     nodes = [{'id': 0, 'stage': 0, 'parent': None, 'members': list(range(n)), 'probability': 1., '
conditional_probability': 1.}]
21     for stage in range(1, 4):
22         for parent in [node for node in nodes if node['stage'] == stage-1]:

```



```

23     ids = np.asarray(parent['members'])
24     values = revisions[ids, stage-1]
25     center, scale = values.mean(0), values.std(0)+np.finfo(float).eps
26     standardized = (values-center)/scale
27     count = 2 if len(ids) >= 4 and np.unique(values, axis=0).shape[0] >= 2 else 1
28     reps, assignment = medoids(standardized, count)
29     if count == 2 and np.bincount(assignment, minlength=2).min() < 2:
30         reps, assignment = medoids(standardized, 1)
31     for cluster, representative in enumerate(reps):
32         members = ids[assignment == cluster]
33         node_id = len(nodes)
34         labels[members, stage] = node_id
35         nodes.append({'id': node_id, 'stage': stage, 'parent': parent['id'], 'members':
members.tolist(),
36                     'probability': len(members)/n, 'conditional_probability': len(members)/len(ids),
37                     'medoid_revision': values[representative].tolist(), 'center': center.tolist(), '
scale': scale.tolist()})
38     return labels, nodes
39
40
41 def tree_inputs(data: Inputs, day: int, config: Config) -> tuple[np.ndarray, np.ndarray, list[dict],
dict]:
42     name = data.selected[day]
43     ids, el, ep, scales = data.history(day, 0, name, method=config.interpolation)
44     # 三次修正的共同18h支持直到次日12: 所有历史修正/最终24h轨迹均需已实现。
45     valid = ids*144+216 <= day*144
46     ids, el, ep, scales = ids[valid], el[valid], ep[valid], scales[valid]
47     selected, pool = pool_indices(ids, scales, day, data.pv(day, 0, config.interpolation).sum()*DT,
config)
48     ids, el, ep = ids[selected], el[selected], ep[selected]
49     if len(ids) < 2:
50         raise ValueError('M2历史样本不足以保留最终实现不确定性')
51     revisions = np.asarray([[data.pv(int(j), k, config.interpolation)[:108]-data.pv(int(j), k-6,
config.interpolation)[36:]
52                             for k in (6, 12, 18)] for j in ids])
53     labels, nodes = conditional_tree(revisions)
54     load = np.maximum(data.load(day)[:144]+el, 0)
55     pv = np.maximum(data.pv(day, 0, config.interpolation)+ep, 0)
56     return (load-pv)*DT, labels, nodes, {'history_days': ids.tolist(), 'load_upper': (load.max(0)*DT
+CAP).tolist(),
57                                         'pool': pool, 'role': 'ex_post_only_never_select_M1',
58                                         'minimum_child_samples': 2, 'maximum_branches': [2, 2, 2]}
59
60
61 def solve_tree(net: np.ndarray, labels: np.ndarray, nodes: list[dict], upper: np.ndarray,
62               prices: np.ndarray, initial: float, terminal: float, config: Config,
63               *, require: bool = True) -> dict:
64     started = time.perf_counter()
65     n, slots = net.shape
66     if slots%4:
67         raise ValueError('四阶段树时域需可均分为四块')
68     block = slots//4
69     model = make_model(config.seconds, config.gap, config.solver_threads)
70     seed_grid = np.maximum(net.max(0), 0)
71     g0 = [model.addVar(lb=0, ub=float(v)) for v in upper]
72     entries = list(zip(g0, seed_grid))
73     policies = {}

```



```

74 obj = quicksum(float(p)*v for p, v in zip(prices, g0))
75 for node in nodes:
76     stage = node['stage']; start, end = stage*block, (stage+1)*block
77     g = g0[:block] if stage == 0 else [model.addVar(lb=0, ub=float(v)) for v in upper[start:end
78 ]]
79     cp, rp = [[model.addVar(lb=0, ub=CAP) for _ in range(block)] for _ in range(2)]
80     if stage:
81         entries.extend(zip(g, seed_grid[start:end]))
82         obj += node['probability']*scheduled_cost(model, g, prices[start:end], g0[start:end],
83 block,
84         entries, seed_grid[start:end], seed_grid[start:end])
85         entries.extend((v, 0.) for v in cp+rp)
86         policies[node['id']] = (g, cp, rp)
87     paths = []
88     for s, demand in enumerate(net):
89         g, cp, rp = ([v for stage in range(4) for v in policies[int(labels[s, stage])][kind]] for
90 kind in range(3))
91         path = add_path(model, g, cp, rp, demand, upper, initial, exact=True, feedback='aggregate',
92 previous_net=0., initial_lower=initial, initial_upper=initial,
93 seed_policy=Policy(seed_grid, np.zeros(slots), np.zeros(slots)), seed_initial=initial,
94 seed_entries=entries)
95         paths.append(path)
96         obj += (quicksum(5*float(p)*q for p, q in zip(prices, path['emergency']))-terminal*path['soc
97 '][-1])/n
98 model.setObjective(obj)
99 seed = model.createSol()
100 for variable, value in entries:
101     model.setSolVal(seed, variable, float(value))
102 seed_accepted = bool(model.addSol(seed))
103 size = {'variables': model.getNVars(), 'constraints': model.getNConss(), 'binaries': model.
104 getNBinVars()}
105 optimize_model(model, config.solver_threads)
106 audit = {'status': str(model.getStatus()), 'seconds': time.perf_counter()-started, '
107 seed_accepted': seed_accepted,
108 'scenario_count': n, 'nodes': len(nodes), 'role': 'ex_post_only',
109 'solver_threads': config.solver_threads, **size}
110 if not model.getNSols():
111     audit.update(reliable=False, gap=None)
112     if require:
113         raise LimitedSolve(audit)
114     return {'audit': audit}
115 solved = {node_id: Policy(*(np.maximum(np.array([model.getVal(v) for v in a]), 0) for a in
116 arrays)) for node_id, arrays in policies.items()}
117 mapping = 0.
118 for s, demand in enumerate(net):
119     policy = Policy(*(np.concatenate([getattr(solved[int(labels[s, stage])], key) for stage in
120 range(4)])
121 for key in ('grid', 'charge_cap', 'discharge_cap')))
122     response = replay(policy, demand, initial)
123     mapping = max(mapping, max(abs(float(model.getVal(v))-response[key][t]) for key in paths[s]
124 for t, v in enumerate(paths[s][key])))
125 value, bound = float(model.getObjVal()), float(model.getDualbound())
126 gap = max(0., value-bound)/max(abs(value), 1e-10)
127 audit.update(objective=value, lower_bound=bound, gap=gap, mapping_error=mapping,
128 reliable=mapping <= TOL and value-bound <= max(TOL, config.gap*abs(value)))
129 if require and not audit['reliable']:
130     raise LimitedSolve(audit)

```

```

121     return {'audit': audit, 'g0': np.maximum(np.array([model.getVal(v) for v in g0]), 0), 'policies'
122           : solved}
123
124 def replay_tree(data: Inputs, day: int, initial: float, nodes: list[dict], solution: dict, config:
125               Config) -> pd.DataFrame:
126     current, soc, frames = 0, initial, []
127     for stage in range(4):
128         hour = stage*6
129         if stage:
130             revision = data.pv(day, hour, config.interpolation)[:108]-data.pv(day, hour-6, config.
131             interpolation)[36:]
132             children = [node for node in nodes if node['parent'] == current]
133             selected = min(children, key=lambda node: (np.linalg.norm((revision-np.array(node['
134             medoid_revision']))/np.array(node['scale'])), node['id']))
135             current = selected['id']
136             policy = solution['policies'][current]
137             frame = frame_for(data, day, hour, policy, soc, config)
138             frame['g0'] = solution['g0'][hour*6:hour*6+36]
139             frame['initial_soc'] = initial
140             frame['tree_node'] = current
141             frames.append(frame); soc = float(frame.soc.iloc[-1])
142             frame = pd.concat(frames, ignore_index=True)
143             frame['planned_cost'] = frame.price*frame.g0
144             frame['adjustment_cost'] = adjustment(frame.grid.to_numpy(), frame.g0.to_numpy(), frame.price.
145             to_numpy())
146             frame['emergency_cost'] = 5*frame.price*frame.emergency
147             frame['total_cost'] = frame.planned_cost+frame.adjustment_cost+frame.emergency_cost
148             validate_frame(frame, config)
149     return frame

```

src/q3/optimization.py

```

1  """3.13—3.17: 有限域精确反馈MILP: LP只提供下界和可行策略初值."""
2  from dataclasses import dataclass
3  import time
4  import numpy as np
5  from pyscipopt import Model, quicksum, Eventhdlr, SCIP_EVENTTYPE, SCIP_STAGE
6  from pathlib import Path
7  from .config import Config, CAP, E_MIN, E_MAX, ETA, TOL
8  from .physics import Policy, replay, adjustment
9  from .scenarios import Scenarios
10
11
12 @dataclass
13 class Solution:
14     policy: Policy | None
15     responses: list[dict]
16     audit: dict
17
18
19 class LimitedSolve(RuntimeError):
20     def __init__(self, audit: dict):
21         super().__init__('限定时内未取得所要求的最优性证书; 保留诊断, 不发布正式结果')
22         self.audit = audit
23

```

```

24     def __reduce__(self) -> tuple:
25         return (LimitedSolve, (self.audit,))
26
27
28 def accept_feasible_incumbent(policy: Policy | None, scenarios: Scenarios, prices: np.ndarray,
29                               initial: float, config: Config, *, reference: np.ndarray | None,
30                               today: int, terminal: float, previous_net: float, audit: dict,
31                               hard_seconds: float, source: str) -> Solution:
32     """比赛救援仅接受受原 replay 与完整 legacy 矩阵复核的当前可行策略。"""
33     if policy is None:
34         raise LimitedSolve(**audit, 'accepted': False, 'acceptance_rejection': 'no_feasible_incumbent'
35                               })
36     net=scenarios.net;prices=np.asarray(prices,dtype=float);count=len(prices)
37     upper=scenarios.load.max(0)/6+CAP
38     domain_error=max(float(np.max(-policy.grid)),float(np.max(policy.grid-upper)),
39                      float(np.max(-policy.charge_cap)),float(np.max(policy.charge_cap-CAP)),
40                      float(np.max(-policy.discharge_cap)),float(np.max(policy.discharge_cap-CAP))
41                      ,0.)
42     if any(values.shape!=(count,) or not np.isfinite(values).all()
43           for values in (policy.grid,policy.charge_cap,policy.discharge_cap)) or domain_error>TOL:
44         raise LimitedSolve(**audit, 'accepted': False, 'acceptance_rejection': 'policy_domain_invalid',
45                               'policy_domain_error': domain_error})
46     responses=[replay(policy,path,initial,mode=config.feedback,previous_net=previous_net) for path
47                in net]
48     value=objective(policy,responses,scenarios.weights,prices,reference,today,terminal)
49     from .fast_solver import build_matrix,seed_vector,matrix_error,matrix_digest
50     matrix=build_matrix(scenarios,prices,initial,reference=reference,today=today,
51                        terminal=terminal,fixed_policy=policy if audit.get('fixed_policy') else None
52                        )
53     coefficients,bounds,integer,constraints,indices=matrix
54     vector=seed_vector(policy,scenarios,prices,initial,indices,len(coefficients),reference)
55     feasibility,integrality=matrix_error(vector,bounds,integer,constraints)
56     raw_value=float(coefficients@vector)
57     lower=audit.get('lower_bound');lower=float(lower) if lower is not None and np.isfinite(lower)
58     else None
59     if lower is None:
60         raise LimitedSolve(**audit, 'accepted': False, 'acceptance_rejection': '
61                               missing_valid_global_lower_bound'})
62     if feasibility>TOL or integrality>1e-8 or abs(raw_value-value)>TOL or lower>value+TOL:
63         raise LimitedSolve(**audit, 'accepted': False, 'acceptance_rejection': '
64                               independent_validation_failed',
65                               'objective': value, 'optimizer_objective': raw_value, 'lower_bound': lower,
66                               'linear_feasibility_error': feasibility, 'integrality_error': integrality})
67     gap=max(0.,value-lower)/max(abs(value),1e-10)
68     accepted={**audit, 'status': 'hard_limit_feasible', 'objective': value, 'upper_bound': value, '
69                optimizer_objective': raw_value,
70                'lower_bound': lower, 'gap': gap, 'reliable': False, 'certificate_met': False, 'accepted': True,
71                'accepted_by': 'hard_limit_feasible', 'hard_limit_reached': True, 'hard_limit_seconds':
72                hard_seconds,
73                'acceptance_source': source, 'original_physics_replay_validated': True,
74                'policy_domain_error': domain_error, 'linear_feasibility_error': feasibility,
75                'integrality_error': integrality, 'matrix_digest': matrix_digest(*matrix[:4]),
76                'energy_balance_soc_power_validated_by': 'physics.replay'}
77     return Solution(policy,responses,accepted)
78
79 def reached_hard_limit(audit: dict, elapsed: float, budget: float) -> bool:

```

```

72     return 'time limit' in str(audit.get('status', '')).lower() or elapsed >= budget - .05
73
74
75 def objective(policy: Policy, responses: list[dict], weights: np.ndarray, prices: np.ndarray,
76             reference: np.ndarray | None, today: int, terminal: float) -> float:
77     schedule = np.dot(prices, policy.grid) if reference is None else (
78         adjustment(policy.grid[:today], reference[:today], prices[:today]).sum()
79         + np.dot(prices[today:], policy.grid[today:]))
80     return float(schedule + sum(w * (5 * np.dot(prices, r['emergency']) - terminal * r['soc'])[-1])
81                        for w, r in zip(weights, responses))
82
83
84 def make_model(seconds: float, gap: float = 0., threads: int = 1) -> Model:
85     model = Model('q3_exact_causal_policy')
86     model.hideOutput()
87     model.setRealParam('limits/time', seconds)
88     model.setRealParam('limits/gap', gap)
89     model.setIntParam('parallel/minnthreads', threads)
90     model.setIntParam('parallel/maxnthreads', threads)
91     model.setIntParam('randomization/randomseedshift', 0)
92     return model
93
94
95 def optimize_model(model: Model, threads: int) -> None:
96     if threads > 1:
97         model.solveConcurrent()
98     else:
99         model.optimize()
100
101
102 def scheduled_cost(model: Model, grid: list, prices: np.ndarray, reference: list | np.ndarray | None
103 ,
104                 today: int, seed: list | None = None, seed_grid: np.ndarray | None = None,
105                 seed_reference: np.ndarray | None = None) -> object:
106     terms = []
107     for t, (g, price) in enumerate(zip(grid, prices)):
108         if reference is None or t >= today:
109             terms.append(price * g)
110         else:
111             # convex slopes 0.5p and 1.5p; negative value is cancellation net saving.
112             value = model.addVar(lb=-model.infinity(), name=f'adjust_{model.getNVars()}')
113             for slope in (.5, 1.5):
114                 model.addCons(value >= slope * price * (g - reference[t]))
115                 terms.append(value)
116             if seed is not None:
117                 delta = seed_grid[t] - seed_reference[t]
118                 seed.append((value, price * (1.5 * max(delta, 0) - .5 * max(-delta, 0))))
119     return quicksum(terms)
120
121 def add_path(model: Model, grid: list, cp: list, rp: list, net: np.ndarray,
122             upper: np.ndarray, initial: object, *, exact: bool, feedback: str,
123             previous_net: float, initial_lower: float = E_MIN, initial_upper: float = E_MAX,
124             seed_policy: Policy | None = None, seed_initial: float | None = None,
125             seed_entries: list | None = None) -> dict:
126     """允许initial为树父状态; 所有响应由共享节点参数和已观测状态决定。"""
127     responses = {key: [] for key in ('charge', 'discharge', 'emergency', 'spill', 'soc')}

```

```

128 previous = initial
129 seeded = replay(seed_policy, net, seed_initial, mode=feedback, previous_net=previous_net) if
seed_policy else None
130
131 def minimum(bounds: list, maxima: list[float], label: str, seed_bounds: list | None) -> object:
132     value = model.addVar(lb=0, ub=min(maxima), name=label)
133     switches = [model.addVar(vtype='B') for _ in bounds]
134     model.addCons(quicksum(switches) == 1)
135     for bound, maximum, switch in zip(bounds, maxima, switches):
136         model.addCons(value <= bound)
137         model.addCons(value >= bound-maximum*(1-switch))
138     if seed_bounds is not None:
139         selected = int(np.argmin(seed_bounds))
140         seed_entries.append((value, min(seed_bounds)))
141         seed_entries.extend((v, float(i == selected)) for i, v in enumerate(switches))
142     return value
143
144 for t, demand in enumerate(net):
145     low = max(E_MIN, initial_lower-t*CAP/ETA)
146     high = min(E_MAX, initial_upper+t*ETA*CAP)
147     observed = demand if feedback == 'aggregate' else (previous_net if t == 0 else net[t-1])
148     previous_seed = seed_initial if t == 0 else (seeded['soc'][t-1] if seeded else None)
149     if exact:
150         plus_bound, minus_bound = max(observed, 0), max(upper[t]-observed, 0)
151         zp = model.addVar(lb=0, ub=plus_bound)
152         zm = model.addVar(lb=0, ub=minus_bound)
153         sign = model.addVar(vtype='B')
154         model.addCons(observed-grid[t] == zp-zm)
155         model.addCons(zp <= plus_bound*sign)
156         model.addCons(zm <= minus_bound*(1-sign))
157         if seeded:
158             delta = observed-seed_policy.grid[t]
159             seed_entries.extend(((zp, max(delta, 0)), (zm, max(-delta, 0)), (sign, float(delta
160 >= 0))))
161         cb = ([seed_policy.charge_cap[t], max(-delta, 0), max((E_MAX-previous_seed)/ETA, 0)] if
seeded else None)
162         rb = ([seed_policy.discharge_cap[t], max(delta, 0), max(ETA*(previous_seed-E_MIN), 0)]
if seeded else None)
163         c = minimum([cp[t], zm, (E_MAX-previous)/ETA], [CAP, minus_bound, (E_MAX-low)/ETA], 'c',
cb)
164         r = minimum([rp[t], zp, ETA*(previous-E_MIN)], [CAP, plus_bound, ETA*(high-E_MIN)], 'r',
rb)
165     else:
166         c, r = (model.addVar(lb=0, ub=CAP) for _ in range(2))
167         q = model.addVar(lb=0, ub=max(demand, 0)+CAP)
168         u = model.addVar(lb=0, ub=max(upper[t]-demand, 0)+CAP)
169         e = model.addVar(lb=max(E_MIN, low-CAP/ETA), ub=min(E_MAX, high+ETA*CAP))
170         model.addCons(e == previous+ETA*c-r/ETA)
171         model.addCons(grid[t]+r+q == demand+c+u)
172         if exact and feedback == 'aggregate':
173             model.addCons(q == zp-r)
174             model.addCons(u == zm-c)
175         # lag: q/u positive part is exact at optimum since q cost strictly positive and u free.
176         for key, variable in zip(responses, (c, r, q, u, e)):
177             responses[key].append(variable)
178             if seeded and key not in ('charge', 'discharge'):
179                 seed_entries.append((variable, seeded[key][t]))

```

```

179     previous = e
180     return responses
181
182
183 def solve(scenarios: Scenarios, prices: np.ndarray, initial: float, config: Config, *,
184          reference: np.ndarray | None = None, today: int = 144, terminal: float = 0.,
185          previous_net: float = 0., require: bool = True, fixed: Policy | None = None,
186          checkpoint: Path | None = None, max_slices: int | None = 1,
187          hard_seconds: float | None = None) -> Solution:
188     started = time.perf_counter()
189     budget=hard_seconds if config.production_rescue else None
190     if config.production_rescue and (budget is None or budget<=0):
191         raise ValueError('production rescue必须显式传入节点总hard wall')
192     def remaining() -> float:
193         return max(.01,(budget-(time.perf_counter()-started)) if budget is not None else config.
194             seconds)
195     net, weights = scenarios.net, scenarios.weights
196     prices = np.asarray(prices, dtype=float)
197     s, count = net.shape
198     if len(prices) != count or scenarios.load.shape != net.shape or (prices <= 0).any():
199         raise ValueError('场景/价格维度非法')
200     if not all(np.isfinite(v).all() for v in (net, scenarios.load, weights, prices)) or (weights <=
201         0).any() or not np.isclose(weights.sum(), 1):
202         raise ValueError('场景/概率非法')
203     if not E_MIN <= initial <= E_MAX or terminal < 0 or not np.isfinite(terminal):
204         raise ValueError('初态或终端残值非法')
205     upper = scenarios.load.max(0)/6+CAP # 3.13的负荷上界，不能以净负荷替换。
206     today = min(today, count)
207     if reference is not None and (len(reference) < today or not np.isfinite(reference[:today]).all()
208         ):
209         raise ValueError('当天原计划不完整')
210
211     if max_slices is not None and max_slices < 1:
212         raise ValueError('切片次数必须为正或None')
213     from .checkpoint import request_digest, load_snapshot, save_snapshot
214     signature = request_digest(config, scenarios.load, scenarios.pv, weights, prices, initial,
215         reference, today, terminal, previous_net,
216         *(getattr(fixed, key) if fixed else None for key in ('grid', 'charge_cap', 'discharge_cap'))
217     ) if checkpoint else ''
218     restored = load_snapshot(checkpoint/'latest.json', signature) if checkpoint else None
219     restored_lower = -float('inf')
220     if restored:
221         restored_policy, restored_audit = restored
222         restored_responses = [replay(restored_policy, n, initial, mode=config.feedback, previous_net
223             =previous_net) for n in net]
224         restored_value = objective(restored_policy, restored_responses, weights, prices, reference,
225             today, terminal)
226         if (restored_policy.grid > upper+TOL).any() or not np.isclose(restored_value, restored_audit
227             ['objective'], rtol=0, atol=TOL):
228             raise ValueError('节点快照策略物理上界或重算目标不符')
229         restored_lower = restored_audit.get('lower_bound')
230         restored_lower = -float('inf') if restored_lower is None else float(restored_lower)
231         if restored_lower > restored_value+TOL:
232             raise ValueError('节点快照下界超过可行上界')
233         if restored_audit.get('reliable'):
234             if restored_value-restored_lower > max(TOL, config.gap*abs(restored_value)):
235                 raise ValueError('已完成节点快照的最优性证书不符')

```

```

229         return Solution(restored_policy, restored_responses, {**restored_audit, 'accepted': True,
230             'upper_bound': restored_value,
231             'certificate_met': True, 'accepted_by': 'certified_3pct', 'hard_limit_reached': False,
232             'restored_complete_node': True})
233     if config.production_rescue and restored_audit.get('accepted'):
234         accepted=accept_feasible_incumbent(restored_policy, scenarios, prices, initial, config,
235             reference=reference, today=today, terminal=terminal, previous_net=previous_net,
236             audit=restored_audit, hard_seconds=budget, source='restored_rescue_checkpoint')
237     if restored_audit.get('accepted_from_preexisting_checkpoint'):
238         accepted.audit.update(hard_limit_reached=False, accepted_from_preexisting_checkpoint=
True,
239             acceptance_source='reused_preexisting_over_budget_incumbent')
240     return accepted
241     if not require and max_slices == 1 and restored_audit.get('completed_slices', 0) >= 1:
242         return Solution(restored_policy, restored_responses, {**restored_audit, '
restored_budgeted_sample': True})
243
244 # 自由场景recourse仅为可证明下界，不作为实际策略。
245 relaxed = make_model(min(remaining(), max(.01, (budget/4 if budget is not None else config.seconds
/4))))
246 rg = [relaxed.addVar(lb=0, ub=float(v)) for v in upper]
247 paths = [add_path(relaxed, rg, [], [], n, upper, initial, exact=False, feedback=config.feedback,
248     previous_net=previous_net, initial_lower=initial, initial_upper=initial) for n
in net]
249 expression = scheduled_cost(relaxed, rg, prices, reference, today)
250 expression += quicksum(float(w)*(quicksum(5*float(p)*q for p, q in zip(prices, path['emergency'
])))
251     -terminal*path['soc'][-1]) for w, path in zip(weights, paths))
252 relaxed.setObjective(expression); relaxed.optimize()
253 relaxed_lower=float(relaxed.getDualbound())
254 if restored_lower>relaxed_lower:
255     lower=restored_lower
256     if restored_audit.get('matrix_digest'):
257         lower_provenance={'kind': 'checkpoint', 'matrix_digest': restored_audit['matrix_digest'],
258             'formulation_version': restored_audit.get('formulation_version'),
259             'source_audit': restored_audit.get('bound_sources')}
260     else:
261         sources=restored_audit.get('bound_sources') or []
262         lower_provenance=(sources[-1] if sources else {'kind': '
legacy_valid_bound_without_matrix_digest',
263             'value': lower, 'source_status': restored_audit.get('status')})
264 else:
265     lower=relaxed_lower
266     lower_provenance={'kind': 'external_original_recourse_LP', 'solver': 'SCIP', 'value': lower,
267         'valid_for': 'original_Q3_policy_problem'}
268 candidates = []
269 if fixed is not None:
270     candidates.append(fixed)
271 elif relaxed.getNSols():
272     g = np.clip(np.array([relaxed.getVal(v) for v in rg]), 0, upper)
273     candidates.append(Policy(g, np.full(count, CAP), np.full(count, CAP)))
274     rc = np.array([relaxed.getVal(v) for v in p['charge']] for p in paths)
275     rr = np.array([relaxed.getVal(v) for v in p['discharge']] for p in paths)
276     candidates.append(Policy(g, np.clip(rc.max(0), 0, CAP), np.clip(rr.max(0), 0, CAP)))
277 else:
278     candidates.append(Policy(np.maximum(net.max(0), 0), np.zeros(count), np.zeros(count)))
279 if restored and fixed is None:

```



```

280     candidates.append(restored_policy)
281     evaluated = []
282     for candidate in candidates:
283         responses = [replay(candidate, n, initial, mode=config.feedback, previous_net=previous_net)
284                       for n in net]
285         value = objective(candidate, responses, weights, prices, reference, today, terminal)
286         evaluated.append((value, candidate, responses))
287     value, seed, responses = min(evaluated, key=lambda item: item[0])
288     certified = max(0., value-lower)/max(abs(value), 1e-10) if np.isfinite(lower) else None
289     if fixed is None and config.feedback == 'aggregate' and value-lower > max(TOL, config.gap*abs(
290         value)):
291         from .incumbent import refine
292         candidate = refine(seed, net, prices, weights, initial, reference, today, terminal, upper,
293                             min(1., remaining()), (budget/10 if budget is not None else config.seconds
294 /10)))
295         candidate_responses = [replay(candidate, n, initial) for n in net]
296         candidate_value = objective(candidate, candidate_responses, weights, prices, reference,
297 today, terminal)
298         if candidate_value < value:
299             value, seed, responses = candidate_value, candidate, candidate_responses
300             certified = max(0., value-lower)/max(abs(value), 1e-10)
301         if fixed is None and certified is not None and value-lower <= max(TOL, config.gap*abs(value)):
302             audit = {'status': 'certified_policy_from_LP_bound', 'objective': value, 'upper_bound':value
303 ,
304                     'lower_bound': lower, 'gap': certified, 'reliable': True, 'seconds': time.perf_counter()
305 -started,
306                     'scenario_count': s, 'slots': count, 'variables': relaxed.getNVars(), 'binaries': 0,
307                     'mapping_error': 0., 'terminal': terminal, 'relaxation_is_not_execution': True,
308                     'solver_threads': 1, 'requested_solver_threads': config.solver_threads, 'solve_method':
309 'LP_certificate',
310                     'accepted':True,'certificate_met':True,'accepted_by':'certified_3pct',
311 hard_limit_reached':False,
312                     'hard_limit_seconds':budget if budget is not None else None}
313             if checkpoint:
314                 save_snapshot(checkpoint/'latest.json', signature, seed, audit)
315             return Solution(seed, responses, audit)
316         if fixed is None and config.feedback=='aggregate' and config.gap>0:
317             from .fast_solver import solve_fast
318             attempt=time.time_ns();last_slice=[0]
319             if checkpoint:
320                 save_snapshot(checkpoint/'latest.json',signature,seed,{
321                     'status':'initial_incumbent_saved', 'objective':value, 'lower_bound':lower,
322                     'gap':certified, 'reliable':False, 'completed_slices':0, 'seconds':time.perf_counter()-
323 started,
324                     'bound_sources':[lower_provenance], 'formulation_version':config.formulation})
325             slice_seconds=budget if budget is not None else config.seconds
326             def persist_fast(policy: Policy, responses: list, audit: dict) -> None:
327                 elapsed=time.perf_counter()-started
328                 completed=int(elapsed//slice_seconds)
329                 if audit['status']!='searching':completed=max(1,int(np.ceil(elapsed/slice_seconds)))
330                 audit.update(completed_slices=completed,slice_seconds=slice_seconds,seconds=elapsed,
331                             restored_incumbent=restored is not None,
332                             certificate='original_feedback_cost_with_valid_monotone_MILP_lower_bound')
333             if checkpoint:
334                 save_snapshot(checkpoint/'latest.json',signature,policy,audit)
335             if completed>last_slice[0]:
336                 save_snapshot(checkpoint/f'slice_{attempt}_{completed:05d}.json',signature,

```



```

policy,audit)
328     last_slice[0]=completed
329     policy,responses,audit=solve_fast(scenarios,prices,initial,config,reference=reference,today=
today,
330     terminal=terminal,seed=seed,known_lower=lower,
331     seconds=remaining() if budget is not None else (1e20 if max_slices is None else config.
seconds*max_slices),
332     progress=persist_fast if checkpoint else None,formulation=config.formulation,
333     known_lower_provenance=lower_provenance,
334     deadline=started+budget if budget is not None else None)
335     if policy is not None:persist_fast(policy,responses,audit)
336     if audit.get('reliable'):
337         audit.update(accepted=True,certificate_met=True,accepted_by='certified_3pct',
338                     hard_limit_reached=False,hard_limit_seconds=budget if budget is not None
else None)
339         return Solution(policy,responses,audit)
340     if config.production_rescue:
341         elapsed=time.perf_counter()-started
342         if not reached_hard_limit(audit,elapsed,budget):
343             raise LimitedSolve(**audit,'accepted':False,
344                                'acceptance_rejection':'solver_ended_before_hard_limit','elapsed_seconds':
elapsed})
345         accepted=accept_feasible_incumbent(policy or seed,scenarios,prices,initial,config,
346         reference=reference,today=today,terminal=terminal,previous_net=previous_net,
347         audit=audit,hard_seconds=budget,source='best_incumbent_at_hard_limit')
348         if checkpoint:save_snapshot(checkpoint/'latest.json',signature,accepted.policy,accepted.
audit)
349         return accepted
350         if require:raise LimitedSolve(audit)
351         return Solution(policy,responses,audit)
352     model = make_model(max(.01, config.seconds-(time.perf_counter()-started)), config.gap, config.
solver_threads)
353     entries = []
354     arrays = []
355     for bounds, values in ((upper, seed.grid), (np.full(count, CAP), seed.charge_cap), (np.full(
count, CAP), seed.discharge_cap)):
356         variables = [model.addVar(lb=float(values[t]) if fixed else 0., ub=float(values[t]) if fixed
else float(b))
357                      for t, b in enumerate(bounds)]
358         entries.extend(zip(variables, values)); arrays.append(variables)
359     g, cp, rp = arrays
360     paths = [add_path(model, g, cp, rp, n, upper, initial, exact=True, feedback=config.feedback,
361                     previous_net=previous_net, initial_lower=initial, initial_upper=initial,
362                     seed_policy=seed, seed_initial=initial, seed_entries=entries) for n in net]
363     expression = scheduled_cost(model, g, prices, reference, today, entries, seed.grid, reference)
364     expression += quicksum(float(w)*(quicksum(5*float(p)*q for p, q in zip(prices, path['emergency'
])))
365                      -terminal*path['soc'][-1]) for w, path in zip(weights, paths))
366     model.setObjective(expression)
367     seed_solution = model.createSol()
368     for variable, seed_value in entries:
369         model.setSolVal(seed_solution, variable, float(seed_value))
370     seed_accepted = bool(model.addSol(seed_solution))
371     dimensions = {'variables': model.getNVars(), 'constraints': model.getNConss(), 'binaries': model
.getNBinVars()}
372     completed_slices = 0
373     attempt_id = time.time_ns()

```

```

374 def extract() -> Solution:
375     nonlocal lower
376     audit = {'status': str(model.getStatus()), 'seconds': time.perf_counter()-started,
377             'solver_seconds': float(model.getSolvingTime()), 'scenario_count': s, 'slots':
count,
378             'requested_gap': config.gap, 'seed_accepted': seed_accepted, 'terminal': terminal,
379             'solver_threads': config.solver_threads,
380             'solve_method': 'SCIP_solveConcurrent' if config.solver_threads > 1 else '
SCIP_optimize', **dimensions}
381     if model.getNSols():
382         policy = Policy(*(np.clip(np.array([model.getVal(v) for v in a]), 0, b) for a, b in zip(
arrays, (upper, CAP, CAP))))
383         responses = [replay(policy, n, initial, mode=config.feedback, previous_net=previous_net)
for n in net]
384         value = objective(policy, responses, weights, prices, reference, today, terminal)
385         mapping = max(abs(float(model.getVal(v))-float(response[key][t])) for path, response in
zip(paths, responses)
386                     for key in path for t, v in enumerate(path[key]))
387         lower = max(lower, float(model.getDualbound()))
388         gap = max(0., value-lower)/max(abs(value), 1e-10)
389         reliable = mapping <= TOL and value-lower <= max(TOL, config.gap*abs(value))
390         audit.update(objective=value, lower_bound=lower, gap=gap, mapping_error=mapping,
reliable=reliable)
391     else:
392         policy, responses = None, []
393         audit.update(objective=None, lower_bound=lower if abs(lower) < 1e19 else None, gap=None,
reliable=False)
394         audit['completed_slices'] = completed_slices
395         audit['slice_seconds'] = config.seconds
396         audit['restored_incumbent'] = restored is not None
397         return Solution(policy, responses, audit)
398
399 if checkpoint:
400     seed_audit = {'status': 'initial_incumbent_saved', 'objective': value,
401                 'lower_bound': lower if abs(lower) < 1e19 else None, 'gap': certified,
402                 'reliable': False, 'seconds': time.perf_counter()-started, 'completed_slices': 0,
403                 'mapping_error': 0., 'source': 'original_feedback_feasible_seed'}
404     save_snapshot(checkpoint/'latest.json', signature, seed, seed_audit)
405     event = ProgressEvent(lambda: extract(), checkpoint, signature, Solution(seed, responses,
seed_audit))
406     model.includeEventhdlr(event, 'q3_durable_incumbent', 'periodic feasible policy and valid
bound checkpoint')
407 else:
408     event = None
409 while True:
410     if completed_slices:
411         model.setRealParam('limits/time', float(model.getSolvingTime())+config.seconds)
412         optimize_model(model, config.solver_threads)
413         completed_slices += 1
414         if event is not None and event.failure is not None:
415             raise event.failure
416         result = extract()
417         policy, responses, audit = result.policy, result.responses, result.audit
418         if checkpoint and policy is not None:
419             save_snapshot(checkpoint/'latest.json', signature, policy, audit)
420             save_snapshot(checkpoint/f'slice_{attempt_id}_{completed_slices:05d}.json', signature,
policy, audit)

```

```

421         if audit['reliable'] or (max_slices is not None and completed_slices >= max_slices):
422             break
423         if str(model.getStatus()) not in ('timelimit', 'gaplimit') or policy is None:
424             break
425         print(f'节点继续切片{completed_slices+1}, gap={audit["gap"]:.6%}', flush=True)
426     if event is not None:
427         event.extract = None # 断开事件闭包对SCIP模型的引用, 避免跨日累计保留模型。
428     if require and not audit['reliable']:
429         raise LimitedSolve(audit)
430     return Solution(policy, responses, audit)
431
432
433 class ProgressEvent(Eventhdlr):
434     """约10秒保存一次; 仅原模型可行策略及有效全局下界, 不宣称序列化搜索树。"""
435     def __init__(self, extract, directory: Path, signature: str, fallback: Solution):
436         self.extract, self.directory, self.signature = extract, directory, signature
437         self.fallback = fallback
438         self.last = time.monotonic()
439         self.failure: Exception | None = None
440
441     def eventinit(self) -> None:
442         self.model.catchEvent(SCIP_EVENTTYPE.BESTSOLFOUND | SCIP_EVENTTYPE.NODESOLVED |
443                               SCIP_EVENTTYPE.LPSOLVED | SCIP_EVENTTYPE.PRESOLVEROUND, self)
444
445     def eventexit(self) -> None:
446         self.model.dropEvent(SCIP_EVENTTYPE.BESTSOLFOUND | SCIP_EVENTTYPE.NODESOLVED |
447                              SCIP_EVENTTYPE.LPSOLVED | SCIP_EVENTTYPE.PRESOLVEROUND, self)
448
449     def eventexec(self, event) -> None:
450         if time.monotonic()-self.last < 10 or self.model.getNSols() == 0:
451             return
452         self.last = time.monotonic()
453         try:
454             from .checkpoint import save_snapshot
455             result = self.extract() if self.model.getStage() >= SCIP_STAGE.SOLVING else self.fallback
456             if result.policy is not None and result.audit['mapping_error'] <= TOL:
457                 save_snapshot(self.directory/'latest.json', self.signature, result.policy,
458                               {**result.audit, 'checkpoint_event': True, 'checkpoint_stage': int(
459 self.model.getStage())})
460             except Exception as error:
461                 self.failure = error
462                 self.model.interruptSolve()

```

src/q3/parallel.py

```

1  """有界spawn进程池: 只提交彼此独立、输入已冻结的计算。"""
2  from concurrent.futures import ProcessPoolExecutor
3  from contextlib import contextmanager
4  import multiprocessing as mp
5  import os
6  import time
7  from pathlib import Path
8  from typing import Iterator
9  from threadpoolctl import threadpool_limits
10 from .optimization import Solution, solve

```

```

11
12
13 def limit_native_libraries() -> None:
14     for name in ('OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS', 'MKL_NUM_THREADS', '
15         VECLIB_MAXIMUM_THREADS', 'NUMEXPR_NUM_THREADS'):
16         os.environ[name] = '1'
17     threadpool_limits(limits=1)
18
19 def validate_parallelism(workers: int, solver_threads: int) -> None:
20     if not isinstance(workers, int) or not isinstance(solver_threads, int) or min(workers,
21         solver_threads) < 1:
22         raise ValueError('进程数和求解线程数必须为正整数')
23     available = len(os.sched_getaffinity(0)) if hasattr(os, 'sched_getaffinity') else os.cpu_count()
24     if available is not None and workers*solver_threads > available:
25         raise ValueError(f'{workers}进程×{solver_threads}求解线程超过当前可见CPU数{available}')
26
27 @contextmanager
28 def process_pool(workers: int, solver_threads: int = 1) -> Iterator[ProcessPoolExecutor | None]:
29     validate_parallelism(workers, solver_threads)
30     if workers == 1:
31         yield None
32         return
33     # fork继承已初始化的SCIP/BLAS状态可能死锁；macOS显式使用spawn。
34     limit_native_libraries()
35     with ProcessPoolExecutor(max_workers=workers, mp_context=mp.get_context('spawn'),
36         initializer=limit_native_libraries) as executor:
37         yield executor
38
39
40 def solve_job(request: dict) -> Solution:
41     return solve(**request)
42
43
44 def rollout_job(request: dict) -> dict:
45     """每条完整SOC链在一个worker中顺序执行；不再启动子进程池。"""
46     from .rolling import run_period
47     started = time.perf_counter()
48     try:
49         frame = run_period(**request, workers=1)
50         return {'path': str(request['output']), 'status': 'complete', 'rows': len(frame),
51             'seconds': time.perf_counter()-started, 'pid': os.getpid()}
52     except Exception as error:
53         # 父进程保存所有独立分支完成/失败状态后统一拒绝出具完整对照结果。
54         return {'path': str(request['output']), 'status': 'failed', 'error_type': type(error).
55             __name__,
56             'error': str(error), 'seconds': time.perf_counter()-started, 'pid': os.getpid()}
57
58 def run_rollout_jobs(requests: list[dict], workers: int, manifest: Path) -> list[dict]:
59     from .config import write_json
60     if len({str(job['output']).resolve() for job in requests}) != len(requests):
61         raise ValueError('并行任务输出目录重叠，禁止并发写同一检查点')
62     threads = max((job['config'].solver_threads for job in requests), default=1)
63     results = []
64     with process_pool(workers, threads) as executor:

```

```

65     iterator = executor.map(rollout_job, requests) if executor is not None else map(rollout_job,
66     requests)
67     for result in iterator:
68         results.append(result)
69         write_json(manifest, {'workers': workers, 'threads_per_worker': threads,
70         'nested_process_pools': False, 'jobs': results, 'expected_jobs': len(requests)})
71     failed = [result for result in results if result['status'] != 'complete']
72     if failed:
73         raise RuntimeError(f'{len(failed)}条独立实验链未完成, 见{manifest}; 已完成结果保留, 未生成完整比较')
74     return results

```

src/q3/parallel_benchmark.py

```

1  """同一冻结输入的串行/进程并行及SCIP线程对照, 不触发全年运行。"""
2  from dataclasses import replace
3  import json
4  import os
5  from pathlib import Path
6  import resource
7  import sys
8  import time
9  import numpy as np
10 import pandas as pd
11 from .config import Config, ROOT, write_csv, write_json
12 from .scenarios import Scenarios
13 from .optimization import solve
14 from .parallel import process_pool, limit_native_libraries
15 from .data import read_inputs
16 from .terminal import TerminalValues
17
18
19 def measured_solve(request: dict) -> dict:
20     started, cpu_start = time.perf_counter(), time.process_time()
21     solution = solve(**request)
22     cpu = time.process_time() - cpu_start
23     rss = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss / (1024 * 2 if sys.platform == 'darwin'
24     else 1024)
25     return {'solution.audit', 'wall_seconds': time.perf_counter() - started, 'cpu_seconds': cpu,
26     'peak_rss_mib': rss, 'pid': os.getpid(),
27     'grid': solution.policy.grid.tolist() if solution.policy else None}
28
29 def frozen_requests(seconds: float) -> list[dict]:
30     folder = ROOT / 'outputs/q3/raw/extended_timing_20260912'
31     requests = []
32     for name in ('node_171_12', 'node_354_6'):
33         audit = json.loads((folder / (name + '.json')).read_text(encoding='utf-8'))
34         frame = pd.read_csv(folder / (name + '.csv'), float_precision='round_trip')
35         load = frame.pivot(index='scenario', columns='slot', values='load_kw').to_numpy()
36         net = frame.pivot(index='scenario', columns='slot', values='net_kwh').to_numpy()
37         scenes = Scenarios(load, np.maximum(load - 6 * net, 0), np.asarray(audit['weights']), audit['
38         scenarios'])
39         requests.append(dict(scenarios=scenes, prices=np.asarray(audit['prices']), initial=6000.,
40         config=Config(seconds=seconds), reference=np.asarray(audit['reference']),
41         today=144 - audit['audit']['hour'] * 6, terminal=audit['terminal'], require=False))
42     return requests

```

```

42
43
44 def benchmark(output: Path, *, seconds: float = 8., workers: int = 4) -> dict:
45     limit_native_libraries()
46     output.mkdir(parents=True, exist_ok=True)
47     requests = frozen_requests(seconds)
48     # 不因结果选模型；以下只比较相同精度/相同输入的执行资源分配。
49     native = []
50     for threads in (1, 2, 4):
51         for index, job in enumerate(requests):
52             result = measured_solve(**job, 'config': replace(job['config'], solver_threads=threads)
53             })
54             native.append({'case': index, 'threads': threads, **{key: value for key, value in result
55             .items() if key != 'grid'}})
56             print(f'SCIP线程={threads} case={index} wall={result["wall_seconds"]:.2f}s gap={result["
57             gap"]:.4%}', flush=True)
58             write_csv(output/'native_threads.csv', pd.DataFrame(native))
59     jobs = requests*2
60     started = time.perf_counter()
61     serial = [measured_solve(job) for job in jobs]
62     serial_seconds = time.perf_counter()-started
63     started = time.perf_counter()
64     with process_pool(workers) as executor:
65         parallel = list(executor.map(measured_solve, jobs)) if executor else [measured_solve(job)
66         for job in jobs]
67         parallel_seconds = time.perf_counter()-started
68         # 将已热启动进程池复用用于许多短辅助问题，避免每个FD单独建池。
69         data = read_inputs(R00T/'outputs/q2/dispatch_runs/20260911_224501')
70         config = Config(seconds=15.)
71         serial_fd = TerminalValues(data, config)
72         fd_start = time.perf_counter()
73         first = [serial_fd.value(31, hour)[0] for hour in (0, 6, 12, 18)]
74         serial_fd_seconds = time.perf_counter()-fd_start
75         parallel_fd = TerminalValues(data, config, executor)
76         fd_start = time.perf_counter()
77         second = [parallel_fd.value(31, hour)[0] for hour in (0, 6, 12, 18)]
78         parallel_fd_seconds = time.perf_counter()-fd_start
79         np.testing.assert_allclose(first, second, rtol=0, atol=1e-10)
80     rows = []
81     for mode, group in (('serial', serial), ('parallel', parallel)):
82         for index, result in enumerate(group):
83             rows.append({'mode': mode, 'case': index%2, 'replicate': index//2,
84             **{key: value for key, value in result.items() if key != 'grid'}})
85     write_csv(output/'process_comparison.csv', pd.DataFrame(rows))
86     write_json(output/'policies.json', {'serial': serial, 'parallel': parallel})
87     equal = all(all(np.isclose(left[key], right[key], atol=1e-8, rtol=0) for key in ('objective', '
88     lower_bound', 'gap'))
89     and np.allclose(left['grid'], right['grid'], atol=1e-8, rtol=0) for left, right in
90     zip(serial, parallel))
91     report = {'full_run_started': False, 'workers': workers, 'solver_threads_per_process': 1,
92     'jobs': len(jobs), 'serial_wall_seconds': serial_seconds, '
93     parallel_wall_seconds_including_startup': parallel_seconds,
94     'throughput_speedup': serial_seconds/parallel_seconds, 'worker_pids': sorted({row['pid'] for
95     row in parallel}),
96     'serial_FD_seconds': serial_fd_seconds, 'parallel_FD_seconds_warm_pool': parallel_fd_seconds
97     ,
98     'FD_speedup': serial_fd_seconds/parallel_fd_seconds, 'FD_coefficients_equal': True,

```

```

90     'max_worker_peak_rss_mib': max(row['peak_rss_mib'] for row in parallel),
91     'all_replay_mapping_valid': all(row['mapping_error'] <= 1e-5 for row in serial+parallel),
92     'observed_objective_bound_gap_grid_equal': equal,
93     'memory_note': 'ru_maxrss is process lifetime high-water mark; serial parent previously ran
native4-thread trials, not per-job memory',
94     'precision': 'gap=0 remains unchanged; timed-out policies are diagnostic only',
95     'interpretation': 'throughput of four identical bounded jobs; not proof of equal final
policies or annual optimal completion time'}
96 write_json(output/'report.json', report)
97 body = '<!doctype html><html lang="zh"><meta charset="utf-8"><meta name="viewport" content="
width=device-width,initial-scale=1"><title>第三问并行计算验收</title><style>body{font:16px/1.6 -
apple-system,sans-serif;max-width:1100px;margin:32px auto;padding:0 18px;color:#263b50}.table{
overflow-x:auto}table{border-collapse:collapse;white-space:nowrap}td,th{padding:7px;border-
bottom:1px solid #ccd6df}th{background:#eaf1f5}strong{color:#0072b2}</style><h1>第三问并行计算验收
</h1>'
98 body += f'<p>4个冻结输入实例：串行 <strong>{serial_seconds:.2f}s</strong> → {workers}进程 <strong>{
parallel_seconds:.2f}s</strong>，吞吐加速 <strong>{report["throughput_speedup"]:.2f}x</strong>（含
启动）。</p>'
99 body += '<p>仍未启动全量。场景、模型、结算与gap=0不变；限时可行解不代表严格最优。跨日SOC链保持顺序。</p><h2>
SCIP内部多线程</h2><div class="table">'
100 body += pd.DataFrame(native)[['case', 'threads', 'wall_seconds', 'cpu_seconds', 'gap', '
mapping_error']].to_html(index=False)+'</div>'
101 body += '<h2>独立任务进程并行</h2><div class="table">'+pd.DataFrame(rows)[['mode', 'case', '
replicate', 'wall_seconds', 'gap', 'peak_rss_mib']].to_html(index=False)+'</div>'
102 body += f'<p>历史FD：串行{serial_fd_seconds:.3f}s、复用进程池{parallel_fd_seconds:.3f}s；四节点系数逐
值一致。实测单worker峰值{report["max_worker_peak_rss_mib"]:.1f}MiB。</p>'
103 body += '<p>内存列是进程生命期高水位；串行父进程此前已做4线程试验，不能用串并行两列断言单次内存节省。</p>'
104 body += '<p><a href="native_threads.csv">线程对照CSV</a> • <a href="process_comparison.csv">进程对
照CSV</a> • <a href="report.json">完整记录</a></p></html>'
105 (output/'report.html').write_text(body, encoding='utf-8')
106 return report

```

src/q3/physics.py

```

1  """3.9、3.14—3.21：同一冻结反馈用于场景和真实回放。"""
2  from dataclasses import dataclass
3  import numpy as np
4  from .config import CAP, E_MIN, E_MAX, ETA, TOL
5
6
7  @dataclass
8  class Policy:
9      grid: np.ndarray
10     charge_cap: np.ndarray
11     discharge_cap: np.ndarray
12
13     def part(self, start: int, end: int) -> 'Policy':
14         return Policy(*(v[start:end].copy() for v in (self.grid, self.charge_cap, self.discharge_cap
15         )))
16
17 def adjustment(grid: np.ndarray, reference: np.ndarray, prices: np.ndarray) -> np.ndarray:
18     difference = grid-reference
19     return prices*(1.5*np.maximum(difference, 0)-.5*np.maximum(-difference, 0))
20
21

```

```

22 def replay(policy: Policy, net: np.ndarray, initial: float, *, mode: str = 'aggregate',
23           previous_net: float | None = None) -> dict[str, np.ndarray]:
24     net = np.asarray(net, dtype=float)
25     if not E_MIN-TOL <= initial <= E_MAX+TOL or mode not in ('aggregate', 'lag'):
26         raise ValueError('SOC或反馈口径非法')
27     if any(v.shape != net.shape or not np.isfinite(v).all() for v in (policy.grid, policy.charge_cap,
28                               policy.discharge_cap)):
29         raise ValueError('冻结策略维度或数值非法')
30     if not np.isfinite(net).all() or min(policy.grid.min(), policy.charge_cap.min(), policy.
31         discharge_cap.min()) < -TOL:
32         raise ValueError('策略/净负荷非法')
33     if max(policy.charge_cap.max(), policy.discharge_cap.max()) > CAP+TOL:
34         raise ValueError('响应上限超过物理限值')
35     if mode == 'lag' and previous_net is None:
36         raise ValueError('lag反馈需要已实现上一时段净负荷，不得读取当前值填补')
37     result = {key: np.zeros(len(net)) for key in ('charge', 'discharge', 'emergency', 'spill', 'soc'
38         )}
39     soc = float(initial)
40     for t, demand in enumerate(net):
41         observed = demand if mode == 'aggregate' else (previous_net if t == 0 else net[t-1])
42         x = observed-policy.grid[t]
43         c = min(policy.charge_cap[t], max(-x, 0), max((E_MAX-soc)/ETA, 0))
44         r = min(policy.discharge_cap[t], max(x, 0), max(ETA*(soc-E_MIN), 0))
45         remaining = demand-policy.grid[t]+c-r
46         q, u = max(remaining, 0), max(-remaining, 0)
47         soc += ETA*c-r/ETA
48         for key, value in zip(result, (c, r, q, u, soc)):
49             result[key][t] = value
50     validate(policy.grid, net, initial, result)
51     return result
52
53 def validate(grid: np.ndarray, net: np.ndarray, initial: float, result: dict) -> None:
54     c, r, q, u, e = (result[key] for key in ('charge', 'discharge', 'emergency', 'spill', 'soc'))
55     if not all(np.isfinite(v).all() for v in (grid, net, c, r, q, u, e)):
56         raise ValueError('回放出现非有限数值')
57     if min(c.min(), r.min(), q.min(), u.min(), grid.min()) < -TOL:
58         raise ValueError('负电量')
59     if max(c.max(), r.max()) > CAP+TOL or np.minimum(c, r).max() > TOL:
60         raise ValueError('功率/互斥违规')
61     if e.min() < E_MIN-TOL or e.max() > E_MAX+TOL:
62         raise ValueError('SOC越界')
63     np.testing.assert_allclose(grid+r+q-net-c-u, 0, atol=TOL, rtol=0)
64     np.testing.assert_allclose(e-np.r_[initial, e[:-1]]-ETA*c+r/ETA, 0, atol=TOL, rtol=0)
65
66 def no_storage_decision(demand: np.ndarray, reference: float, weights: np.ndarray) -> float:
67     """3.10离散经验分布退化模型；只供机制核实验，不约束完整储能优化。"""
68     choices = np.unique(np.r_[0., reference, demand[demand >= 0]])
69     costs = [reference+float(adjustment(np.array([q]), np.array([reference]), np.ones(1))[0])
70             +5*np.dot(weights, np.maximum(demand-q, 0)) for q in choices]
71     return float(choices[np.argmin(costs)])

```

src/q3/rolling.py

```

1 """3.11—3.22、3.28：冻结节点、真实逐槽执行、SOC连续和成对FIV。"""

```



```

2 import hashlib
3 import json
4 from concurrent.futures import Executor
5 from pathlib import Path
6 import numpy as np
7 import pandas as pd
8 from .config import Config, DT, E_INITIAL, TOL, write_csv, write_json
9 from .data import Inputs
10 from .physics import Policy, replay, adjustment, validate
11 from .scenarios import construct
12 from .terminal import TerminalValues
13 from .optimization import solve, LimitedSolve
14 from .parallel import process_pool, solve_job
15
16
17 def date_of(day: int) -> pd.Timestamp:
18     return pd.Timestamp('2025-01-01')+pd.Timedelta(days=day)
19
20
21 def previous_net(data: Inputs, day: int, hour: int) -> float:
22     slot = day*144+hour*6
23     if slot == 0:
24         raise ValueError('没有上一时段观测')
25     return float(np.diff(data.actual.reshape(-1, 2)[slot-1][::-1])[0]*DT)
26
27
28 def frame_for(data: Inputs, day: int, hour: int, policy: Policy, initial: float, config: Config) ->
    pd.DataFrame:
29     start, end = hour*6, hour*6+len(policy.grid)
30     actual = data.actual[day, start:end]
31     if len(actual) != len(policy.grid):
32         raise ValueError('真实执行不得跨越尚未建立下一日正式计划的午夜')
33     net = (actual[:, 0]-actual[:, 1])*DT
34     response = replay(policy, net, initial, mode=config.feedback, previous_net=previous_net(data,
    day, hour))
35     return pd.DataFrame({'date': str(date_of(day).date()), 'slot': np.arange(start, end),
36                          'timestamp': date_of(day)+pd.to_timedelta((np.arange(start, end)+1)*10, unit='min'),
37                          'node_hour': hour, 'net_kwh': net, 'load_kw': actual[:, 0], 'pv_kw': actual[:, 1],
38                          'grid': policy.grid, 'charge_cap': policy.charge_cap, 'discharge_cap': policy.discharge_cap,
39                          'lag_observation': np.r_[previous_net(data, day, hour), net[:-1]],
40                          'price': data.prices[start:end], **response})
41
42
43 def fiv(data: Inputs, terminal: TerminalValues, day: int, hour: int, soc: float,
44         reference: np.ndarray, config: Config, *, checkpoint_root: Path | None = None,
45         require: bool = True, max_slices: int | None = 1) -> tuple[dict, list[dict]]:
46     length, today = 108, min(144-hour*6, 108)
47     new = construct(data, day, hour, config, length=length)
48     old = construct(data, day, hour, config, origin=hour-6, length=length,
49                    paired_days=np.asarray(new.audit['history_days'], dtype=int) if not new.audit['
50 M0'] else None,
51                    paired_weights=new.weights)
52     if new.audit['M0'] != old.audit['M0'] or new.audit['history_days'] != old.audit['history_days']:
53         raise ValueError('stale/updated配对历史支持不一致')
54     coefficient, calibration = terminal.value(day, hour+18)
55     prices = data.prices[(hour*6+np.arange(length))%144]
56     records, costs, audits = [], [], []

```

```

56 requests = [dict(scenarios=scenes, prices=prices, initial=soc, config=config,
57                 reference=reference, today=today, terminal=coefficient,
58                 previous_net=previous_net(data, day, hour), require=require, max_slices=
max_slices,
59                 checkpoint=checkpoint_root/name if checkpoint_root else None)
60                 for name, scenes in (('stale', old), ('updated', new))]
61 solutions = list(terminal.executor.map(solve_job, requests)) if terminal.executor is not None
else [solve_job(job) for job in requests]
62 for (name, scenes), solution in zip((('stale', old), ('updated', new)), solutions):
63     if solution.policy is None:
64         raise LimitedSolve(solution.audit)
65     # 直接价值只执行当前6h; 跨日继续价值在相同18h冻结反事实策略上回放, 代理交易不计入主轨迹。
66     start = day*144+hour*6
67     realized = data.actual.reshape(-1, 2)[start:start+length]
68     if len(realized) < length:
69         # 年末共同支持集未来实际量缺失, 仍报告可见当日直接价值, 不补造次年真值。
70         valid_length = len(realized)
71     else:
72         valid_length = length
73     policy = solution.policy.part(0, valid_length)
74     response = replay(policy, (realized[:, 0]-realized[:, 1])*DT, soc,
75                          mode=config.feedback, previous_net=previous_net(data, day, hour))
76     schedule = prices[:valid_length]*policy.grid
77     boundary = min(today, valid_length)
78     schedule[:boundary] = prices[:boundary]*reference[:boundary]+adjustment(policy.grid[:
boundary], reference[:boundary], prices[:boundary])
79     cost = schedule+5*prices[:valid_length]*response['emergency']
80     costs.append(cost)
81     audits.append({'branch': name, 'solver': solution.audit, 'scenarios': scenes.audit})
82     records.append({'branch': name, 'grid': policy.grid.tolist(), 'charge_cap': policy.
charge_cap.tolist(),
83                   'discharge_cap': policy.discharge_cap.tolist(), 'real_cost': cost.tolist(),
84                   'soc': response['soc'].tolist()})
85     direct = float(costs[0][:36].sum()-costs[1][:36].sum())
86     complete = all(len(cost) == length for cost in costs)
87     total = float(costs[0].sum()-costs[1].sum()) if complete else None
88     row = {'date': str(date_of(day).date()), 'hour': hour, 'initial_soc': soc,
89           'FIV_block': direct, 'FIV_day': direct if hour == 18 else None,
90           'FIV_cmp': total if hour == 18 else None,
91           'FIV_cont': total-direct if hour == 18 and complete else None,
92           'comparison_complete': complete, 'calibration': calibration,
93           'interpretation': '18h matched-support frozen counterfactual; no state mutation to main',
94           'audits': audits}
95     return row, records
96
97 def run_day(data: Inputs, terminal: TerminalValues, day: int, initial: float, config: Config,
98            *, with_fiv: bool = False, checkpoint_root: Path | None = None,
99            max_slices: int | None = 1, require: bool = True) -> tuple[pd.DataFrame, dict, list[dict
]]:
100     if day < 7:
101         # Q2既定预热协议: 首7日不制定可用负荷计划、储能停机; 真实缺口只记紧急购电。
102         net = (data.actual[day, :, 0]-data.actual[day, :, 1])*DT
103         zero = np.zeros(144)
104         response = replay(Policy(zero, zero, zero), net, initial)
105         frame = pd.DataFrame({'date': str(date_of(day).date()), 'slot': np.arange(144),
106                              'timestamp': date_of(day)+pd.to_timedelta((np.arange(144)+1)*10, unit='min'),

```

```

107         'node_hour': 0, 'net_kwh': net, 'load_kw': data.actual[day, :, 0], 'pv_kw': data.actual[
108         day, :, 1],
109         'grid': zero, 'charge_cap': zero, 'discharge_cap': zero, 'price': data.prices, **
110         response,
111         'g0': zero, 'initial_soc': initial, 'adjustment_cost': zero,
112         'lag_observation': np.r_[0. if day == 0 else previous_net(data, day, 0), net[:-1]])
113         frame['planned_cost'] = 0.
114         frame['emergency_cost'] = 5*frame.price*frame.emergency
115         frame['total_cost'] = frame.emergency_cost
116         return frame, {'cold_start': True, 'reason': 'Q2 Jan1-Jan7 stationary storage, unavailable
117         load forecast'}, []
118     soc, g0, version = initial, None, None
119     frames, nodes, information = [], [], []
120     node_list = list(config.nodes)
121     for index, hour in enumerate(node_list):
122         next_hour = node_list[index+1] if index+1 < len(node_list) else 24
123         length = (next_hour-hour)*6
124         scenes = construct(data, day, hour, config)
125         coefficient, calibration = terminal.value(day, hour)
126         prices = data.prices[(hour*6+np.arange(144))%144]
127         reference = None if hour == 0 else (version if config.settlement == 'sequential' else g0)[
128         hour*6:]
129         hard_seconds=(config.hard_06_seconds if hour==6 else config.hard_other_seconds) if config.
130         production_rescue else None
131         solution = solve(scenes, prices, soc, config, reference=reference, today=144-hour*6,
132         terminal=coefficient, previous_net=previous_net(data, day, hour), require=
133         require,
134         checkpoint=checkpoint_root/f'{hour:02d}' if checkpoint_root else None,
135         max_slices=max_slices,hard_seconds=hard_seconds)
136         if solution.policy is None:
137             raise LimitedSolve(solution.audit)
138         policy = solution.policy
139         if hour == 0:
140             g0 = policy.grid.copy(); version = g0.copy()
141             booked_adjustment = np.zeros(144)
142         elif config.settlement == 'sequential':
143             # sequential交易语义: 当天所有剩余计划在每次节点成为新有效版并真实记账; 跨日仍非交易。
144             count = 144-hour*6
145             booked_adjustment[hour*6:] += adjustment(policy.grid[:count], version[hour*6:], data.
146             prices[hour*6:])
147             version[hour*6:] = policy.grid[:count]
148             if with_fiv and hour and day >= 31:
149                 paired_reference = (reference.copy() if config.settlement == 'sequential' else g0[hour
150                 *6:].copy())
151                 item, counterfactuals = fiv(data, terminal, day, hour, soc, paired_reference, config,
152                 checkpoint_root=checkpoint_root/f'FIV_{hour}' if checkpoint_root else None, require=
153                 require, max_slices=max_slices)
154                 information.append(item)
155             else:
156                 counterfactuals = []
157             executed = frame_for(data, day, hour, policy.part(0, length), soc, config)
158             executed['g0'] = g0[hour*6:next_hour*6]
159             executed['initial_soc'] = initial
160             if checkpoint_root:
161                 write_csv(checkpoint_root/f'{hour:02d}'/'executed_block.csv', executed)
162             nodes.append({'hour': hour, 'initial_soc': soc, 'committed_slots': length,
163             'solver': solution.audit, 'scenarios': scenes.audit, 'calibration': calibration,

```

```

155         'grid': policy.grid.tolist(), 'charge_cap': policy.charge_cap.tolist(),
156         'discharge_cap': policy.discharge_cap.tolist(),
157         'roles': ['committed' if t < length else 'today_proxy' if t < 144-hour*6 else '
continuation_proxy' for t in range(144)],
158         'delta_from_g0': (policy.grid[:144-hour*6]-g0[hour*6:]).tolist(), 'fiv_branches':
counterfactuals})
159     frames.append(executed); soc = float(executed.soc.iloc[-1])
160     frame = pd.concat(frames, ignore_index=True)
161     frame['planned_cost'] = frame.price*frame.g0
162     frame['adjustment_cost'] = (booked_adjustment if config.settlement == 'sequential' else
163         adjustment(frame.grid.to_numpy(), frame.g0.to_numpy(), frame.price.to_numpy()))
164     frame['emergency_cost'] = 5*frame.price*frame.emergency
165     frame['total_cost'] = frame.planned_cost+frame.adjustment_cost+frame.emergency_cost
166     validate_frame(frame, config)
167     return frame, {'date': str(date_of(day).date()), 'nodes': nodes, 'settlement': config.settlement
}, information
168
169
170 def validate_frame(frame: pd.DataFrame, config: Config, *, full: bool = False) -> None:
171     dates, times = pd.to_datetime(frame.date), pd.to_datetime(frame.timestamp)
172     if frame.empty or times.duplicated().any() or not times.diff().dropna().eq(pd.Timedelta(minutes
=10)).all():
173         raise ValueError('回放不连续')
174     if not dates.eq((times-pd.Timedelta(minutes=10)).dt.normalize()).all():
175         raise ValueError('右端点归属日期错误')
176     np.testing.assert_allclose(frame.net_kwh, (frame.load_kw-frame.pv_kw)*DT, atol=TOL, rtol=0)
177     last = None
178     for _, day in frame.groupby('date', sort=False):
179         if len(day) != 144 or not np.array_equal(day.slot, np.arange(144)):
180             raise ValueError('每日必须144个完整时段')
181         initial = float(day.initial_soc.iloc[0])
182         if last is not None and abs(initial-last) > TOL:
183             raise ValueError('跨日SOC断裂')
184         validate(day.grid.to_numpy(), day.net_kwh.to_numpy(), initial,
185             {k: day[k].to_numpy() for k in ('charge', 'discharge', 'emergency', 'spill', 'soc')
})
186         recovered = replay(Policy(day.grid.to_numpy(), day.charge_cap.to_numpy(), day.discharge_cap.
to_numpy()),
187             day.net_kwh.to_numpy(), initial, mode=config.feedback, previous_net=float(day.
lag_observation.iloc[0]))
188         for key in recovered:
189             np.testing.assert_allclose(day[key], recovered[key], atol=TOL, rtol=0)
190             np.testing.assert_allclose(day.lag_observation.iloc[1:], day.net_kwh.iloc[:-1], atol=TOL,
rtol=0)
191         if config.settlement == 'final':
192             np.testing.assert_allclose(day.adjustment_cost, adjustment(day.grid.to_numpy(), day.g0.
to_numpy(), day.price.to_numpy()), atol=TOL)
193             np.testing.assert_allclose(day.planned_cost, day.price*day.g0, atol=TOL)
194             np.testing.assert_allclose(day.emergency_cost, 5*day.price*day.emergency, atol=TOL)
195             np.testing.assert_allclose(day.total_cost, day.planned_cost+day.adjustment_cost+day.
emergency_cost, atol=TOL)
196             last = float(day.soc.iloc[-1])
197     if full and (len(frame) != 334*144 or dates.min() != pd.Timestamp('2025-02-01') or dates.max()
!= pd.Timestamp('2025-12-31')):
198         raise ValueError('拒绝把部分日期发布为完整result3.xlsx')
199
200

```

```

201 def run_period(data: Inputs, config: Config, output: Path, *, end: int = 365, with_fiv: bool = True,
202               workers: int = 1, max_slices: int | None = 1) -> pd.DataFrame:
203     from .checkpoint import run_lock
204     with run_lock(output/'run.lock'), process_pool(workers, config.solver_threads) as executor:
205         return _run_period(data, config, output, end=end, with_fiv=with_fiv, executor=executor,
206                           max_slices=max_slices)
207
208 def _run_period(data: Inputs, config: Config, output: Path, *, end: int, with_fiv: bool,
209               executor: Executor | None, max_slices: int | None) -> pd.DataFrame:
210     """只由显式full命令调用全年; 每日CSV+审计先落盘, 再原子提交receipt, 可安全续算。"""
211     output.mkdir(parents=True, exist_ok=True)
212     signature = config.signature()
213     sources = hashlib.sha256(json.dumps(data.provenance, sort_keys=True).encode('utf-8')).hexdigest()
214     state_path = output/'state.json'
215     state = json.loads(state_path.read_text(encoding='utf-8')) if state_path.exists() else {
216         'signature': signature, 'sources': sources, 'days': [], 'soc': E_INITIAL, 'with_fiv':
217         with_fiv}
218     if state['signature'] != signature or state['sources'] != sources or state['with_fiv'] !=
219     with_fiv:
220         raise ValueError('续算配置/模型/来源/FIV设置变化, 禁止混用前缀')
221     terminal = TerminalValues(data, config, executor, output/'terminal_cache.json')
222     calibration_path = output/'terminal_calibration.csv'
223     if calibration_path.exists() and not terminal.cache:
224         for row in pd.read_csv(calibration_path, float_precision='round_trip').to_dict('records'):
225             terminal.cache[(int(row['history_day']), int(row['hour']), float(row['delta']))] = row
226     all_frames = []
227     for day in range(end):
228         date = str(date_of(day).date()); path = output/f'days/{date}.csv'
229         if day < len(state['days']):
230             receipt = state['days'][day]
231             files = {'dispatch': path, 'audit': output/f'audit/{date}.json', 'information': output/f
232             'information/{date}.json'}
233             if receipt['day'] != day or any(hashlib.sha256(p.read_bytes()).hexdigest() != receipt['
234             hashes'][key] for key, p in files.items()):
235                 raise ValueError('已提交日期或数据摘要不一致')
236             frame = pd.read_csv(path, float_precision='round_trip')
237         else:
238             write_json(output/'active.json', {'date': date, 'status': 'solving', 'completed_days':
239             len(state['days'])})
240         try:
241             frame, audit, information = run_day(data, terminal, day, state['soc'], config,
242             with_fiv=with_fiv,
243             checkpoint_root=output/'nodes'/date, max_slices=max_slices)
244         except LimitedSolve as error:
245             write_json(output/f'failures/{date}.json', error.audit)
246             raise
247         write_csv(path, frame)
248         write_json(output/f'audit/{date}.json', audit)
249         write_json(output/f'information/{date}.json', information)
250         state['soc'] = float(frame.soc.iloc[-1])
251         files = {'dispatch': path, 'audit': output/f'audit/{date}.json', 'information': output/f
252         'information/{date}.json'}
253         state['days'].append({'day': day, 'hashes': {key: hashlib.sha256(p.read_bytes()).
254         hexdigest() for key, p in files.items()}})
255         write_json(state_path, state)

```

```

248         if terminal.cache:
249             write_csv(calibration_path, pd.DataFrame(terminal.cache.values()))
250             print(f'{output.name} {date} SOC={state["soc"]:.3f} 已提交', flush=True)
251         all_frames.append(frame)
252     result = pd.concat(all_frames, ignore_index=True)
253     validate_frame(result, config)
254     formal = result[pd.to_datetime(result.date) >= pd.Timestamp('2025-02-01')].copy()
255     write_csv(output/'warmup.csv', result[pd.to_datetime(result.date) < pd.Timestamp('2025-02-01')])
256     write_csv(output/'dispatch.csv', formal)
257     write_json(output/'active.json', {'status': 'complete', 'completed_days': end})
258     return formal

```

src/q3/run.py

```

1  """python -m src.q3.run: 默认安全计时；全量仅通过显式full子命令。"""
2  import argparse
3  from dataclasses import asdict, replace
4  from datetime import datetime
5  import json
6  from pathlib import Path
7  import time
8  import numpy as np
9  import pandas as pd
10 from .config import Config, ROOT, E_INITIAL, write_csv, write_json
11 from .data import read_inputs
12 from .scenarios import construct
13 from .terminal import TerminalValues
14 from .optimization import solve
15 from .physics import replay
16 from .rolling import run_period, run_day, previous_net
17 from .analysis import forecast_diagnostics, run_experiments
18 from .export import export_workbook, write_tables
19
20
21 def benchmark(q2_run: Path, output: Path, seconds: float = 20., *, solver_threads: int = 1) -> dict:
22     started = time.perf_counter()
23     config = Config(seconds=seconds, solver_threads=solver_threads)
24     data = read_inputs(q2_run, ROOT/'inputs/q3/processed')
25     preparation_seconds = time.perf_counter()-started
26     terminal = TerminalValues(data, config)
27     rows, node_details = [], []
28     # 日期在读取测试结果前固定；初态6000仅用于性能试验，不声称是真实1月预热后的状态。
29     for day in (31, 171, 354):
30         soc, reference = E_INITIAL, None
31         for hour in (0, 6, 12, 18):
32             node_initial = soc
33             node_reference = None if hour == 0 else reference[hour*6:].copy()
34             stage_start = time.perf_counter()
35             scenes = construct(data, day, hour, config)
36             construction_seconds = time.perf_counter()-stage_start
37             fd_start = time.perf_counter()
38             coefficient, calibration = terminal.value(day, hour)
39             fd_seconds = time.perf_counter()-fd_start
40             prices = data.prices[(hour*6+np.arange(144))%144]
41             solution = solve(scenes, prices, soc, config, reference=None if hour == 0 else reference
[hour*6:],

```

```

42         today=144-hour*6, terminal=coefficient, previous_net=previous_net(data, day, hour),
require=False)
43     if solution.policy is not None:
44         if hour == 0:
45             reference = solution.policy.grid.copy()
46             actual = data.actual[day, hour*6:hour*6+36]
47             response = replay(solution.policy.part(0, 36), (actual[:, 0]-actual[:, 1])/6, soc)
48             soc = float(response['soc'][-1])
49             rows.append({'date': str((pd.Timestamp('2025-01-01')+pd.Timedelta(days=day)).date()), '
hour': hour,
50             'scenario_seconds': construction_seconds, 'terminal_seconds': fd_seconds, **solution
.audit})
51             node_details.append({'day': day, 'hour': hour, 'initial_soc': node_initial,
52             'reference': None if node_reference is None else node_reference.tolist(),
53             'policy': None if solution.policy is None else {key: getattr(solution.policy, key).
tolist() for key in ('grid', 'charge_cap', 'discharge_cap')},
54             'scenarios': scenes.audit, 'terminal': calibration})
55             write_csv(output/'timing_samples.csv', pd.DataFrame(rows))
56             write_json(output/'sample_details.json', node_details)
57             print(f'小样本 day={day} hour={hour} S={len(scenes.weights)} {solution.audit["seconds
"]:.2f}s gap={solution.audit.get("gap")} {solution.audit["status"]}', flush=True)
58             if reference is None:
59                 break
60         frame = pd.DataFrame(rows)
61         write_csv(output/'terminal_calibration.csv', pd.DataFrame(terminal.cache.values()))
62         write_csv(output/'terminal_delta_stability.csv', pd.DataFrame(terminal.stability(29, 0)))
63         diag = forecast_diagnostics(data, output/'diagnostics')
64         m0_frame, m0_audit, _ = run_day(data, terminal, 31, E_INITIAL, replace(config, deterministic=
True))
65         write_csv(output/'M0_timing_dispatch.csv', m0_frame)
66         write_json(output/'M0_timing_audit.json', m0_audit)
67         m0_durations = np.array([node['solver']['seconds'] for node in m0_audit['nodes']])
68         durations = frame.seconds.to_numpy()
69         # 主模型预热Jan8-31为96次; 正式334*4=1336; 首7天冷启动无MILP。
70         jobs = {'main_warmup_and_formal': 24*4+334*4, 'FIV_18h_comparisons': 334*6,
71             'OUV_extra_three_update_sets': (24+334)*(1+2+3), 'M0': (24+334)*4,
72             'nine_sensitivities_with_FIV_and_OUV': 9*((24+334)*10+334*6)}
73         projections = []
74         for name, count in jobs.items():
75             measured = m0_durations if name == 'M0' else durations
76             projections.append({'scope': name, 'solve_count': count, 'median_hours': float(np.median(
measured)*count/3600),
77                 'p90_hours': float(np.quantile(measured, .9)*count/3600), 'max_sample_hours': float(
measured.max()*count/3600)})
78         write_csv(output/'runtime_projection.csv', pd.DataFrame(projections))
79         report = {'full_run_started': False, 'config': asdict(config), 'preparation_seconds':
preparation_seconds,
80             'elapsed_seconds': time.perf_counter()-started, 'samples': len(rows),
81             'certified_samples': int(frame.reliable.sum()), 'limited_samples': int((~frame.reliable).sum
()),
82             'median_node_seconds': float(np.median(durations)), 'p90_node_seconds': float(np.quantile(
durations, .9)),
83             'projections': projections, 'diagnostics': diag,
84             'initial_state': 'Each timing day starts at6000; intra-day SOC continuous; not formal warmup
state',
85             'caveat': 'Censored solves make exact-optimum completion time unknown. Projections are
observed-budget CPU planning, not guaranteed completion; FD/preparation/M2/stress and IO are

```



```

extra.',
86     'hardware_contention': f'Sequential Q3 sample jobs, SCIP threads={solver_threads}; other
machine workload may contend'}
87 write_json(output/'timing_report.json', report)
88 body = '<!doctype html><html lang="zh"><meta charset="utf-8"><title>第三问代码与计时验收</title><
style>body{font:16px -apple-system,sans-serif;max-width:1180px;margin:40px auto;color:#263445;
padding:0 20px}table{border-collapse:collapse;font-size:13px;width:100%;margin:20px 0}td,th{
border-bottom:1px solid #d5dce3;padding:8px;text-align:right}th{background:#ecf3f7}img{max-width
:100%}.status{padding:18px;background:#e7f0f4;border-left:4px solid #0072b2}</style><h1>第三问 •
小样本计时验收</h1>'
89 body += '<p class="status">未启动全量。正式 S=20、尾部2。以下为真实附件的性能试验，不能作为正式 result3。</
p>'
90 body += f'<p>样本 {len(rows)} 个；获得所要求证书 {int(frame.reliable.sum())} 个；其余仅保存可行解与间
隙。单节点中位数 {np.median(durations):.2f} 秒。</p>'
91 body += frame[['date', 'hour', 'scenario_count', 'seconds', 'gap', 'reliable', 'status']].
to_html(index=False)
92 body += '<h2>按实测预算外推</h2><p>限时未达最优的样本被截断，不能据此承诺严格最优全年完成时间。下表未含辅助标
定、M2和附加结算对照。</p>'
93 body += pd.DataFrame(projections).to_html(index=False)
94 body += '<h2>预报原始数据诊断</h2>'
95 body += '<p><a href="timing_samples.csv">计时CSV</a> • <a href="timing_report.json">完整计时记录</
a> • <a href="terminal_delta_stability.csv">终端扰动核验收</a></p></html>'
96 body = body.replace('<meta charset="utf-8">', '<meta charset="utf-8"><meta name="viewport"
content="width=device-width,initial-scale=1">')
97 body = body.replace('<style>', '<style>.table-scroll{overflow-x:auto;max-width:100%}table{min-
width:700px}')
98 body = body.replace('<table>', '<div class="table-scroll"><table>').replace('</table>', '</table
></div>')
99 (output/'report.html').write_text(body, encoding='utf-8')
100 return report
101
102
103 def main() -> None:
104     parser = argparse.ArgumentParser(description=__doc__)
105     parser.add_argument('command', choices=['benchmark', 'parallel-benchmark', 'prepare', 'full'],
nargs='?', default='parallel-benchmark')
106     parser.add_argument('--q2-run', type=Path, default=R00T/'outputs/q2/dispatch_runs/20260911
_224501')
107     parser.add_argument('--output', type=Path)
108     parser.add_argument('--seconds', type=float, default=None)
109     parser.add_argument('--gap', type=float, default=0., help='正式默认0，不继承Q2历史3%授权')
110     parser.add_argument('--workers', type=int, default=4, help='独立计算进程上限：子实验内不再建池')
111     parser.add_argument('--solver-threads', type=int, default=1, help='每个MILP的SCIP线程数：>1显式调用
solveConcurrent')
112     parser.add_argument('--experiments', action='store_true', help='显式full后执行所有M0/M2/FIV/OUV和敏
感性')
113     args = parser.parse_args()
114     from .parallel import limit_native_libraries, validate_parallelism
115     validate_parallelism(args.workers, args.solver_threads)
116     limit_native_libraries()
117     output = args.output or R00T/'outputs/q3/raw'/datetime.now().strftime('%Y%m%d_%H%M%S')
118     if args.command == 'parallel-benchmark':
119         from .parallel_benchmark import benchmark as parallel_benchmark
120         report = parallel_benchmark(output, seconds=args.seconds or 8., workers=args.workers)
121         print(json.dumps({key: report[key] for key in ('workers', 'throughput_speedup', '
full_run_started')}, ensure_ascii=False))

```



```

122 elif args.command == 'benchmark':
123     report = benchmark(args.q2_run.resolve(), output, args.seconds or 20., solver_threads=args.
        solver_threads)
124     print(json.dumps({k: report[k] for k in ('samples', 'certified_samples', '
        median_node_seconds', 'full_run_started')}, ensure_ascii=False))
125 elif args.command == 'prepare':
126     data = read_inputs(args.q2_run.resolve(), ROOT/'inputs/q3/processed')
127     write_json(output/'diagnostics.json', forecast_diagnostics(data, output/'diagnostics'))
128 else:
129     config = Config(seconds=args.seconds or 120., gap=args.gap, solver_threads=args.
        solver_threads)
130     data = read_inputs(args.q2_run.resolve(), ROOT/'inputs/q3/processed')
131     write_json(output/'frozen_config.json', {'config': asdict(config), 'signature': config.
        signature(),
132         'assumption_authority': '2026-09-12 user confirmed FD historical24h E6000/6100
        median last28'})
133     frame = run_period(data, config, output/'main', with_fiv=True, workers=args.workers)
134     write_tables(frame, output/'main')
135     export_workbook(frame, output/'main', config)
136     forecast_diagnostics(data, output/'diagnostics')
137     if args.experiments:
138         run_experiments(data, config, output, frame, workers=args.workers)
139
140
141 if __name__ == '__main__':
142     main()

```

src/q3/sampled.py

```

1 """用户授权的少量日期/取值辅助表；不再启动全年对照轨迹。"""
2 from dataclasses import replace
3 import json
4 import hashlib
5 from pathlib import Path
6 import numpy as np
7 import pandas as pd
8 from .config import Config, write_json, write_csv
9 from .data import Inputs
10 from .rolling import run_day, fiv, date_of
11 from .terminal import TerminalValues
12 from .parallel import process_pool
13 from .export import summaries
14
15 DATES = ('2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21')
16
17
18 def sample_configs(base: Config) -> dict[str, Config]:
19     return {'S10': replace(base, scenarios=10), 'S30': replace(base, scenarios=30),
20         'terminal08': replace(base, terminal_scale=.8), 'terminal12': replace(base,
        terminal_scale=1.2),
21         'M0': replace(base, deterministic=True), 'K0': replace(base, nodes=(0,)),
22         'K06': replace(base, nodes=(0, 6)), 'K0612': replace(base, nodes=(0, 6, 12))}
23
24
25 def sample_job(request: tuple) -> dict:
26     data, config, day, initial, label, output, main_directory = request

```

```

27 signature = config.signature()
28 receipt = output/'complete.json'
29 if receipt.exists():
30     saved = json.loads(receipt.read_text(encoding='utf-8'))
31     if saved['signature'] != signature:
32         raise ValueError('抽样任务已有收据与当前配置/源码不同')
33     if any(hashlib.sha256((output/name).read_bytes()).hexdigest() != sha for name, sha in saved['hashes'].items()):
34         raise ValueError('抽样任务结果文件损坏或缺失')
35     return saved['result']
36 terminal = TerminalValues(data, config, cache_path=output/'terminal_cache.json')
37 if label == 'FIV':
38     audit = json.loads((main_directory/f'audit/{date_of(day).date()}.json').read_text(encoding='utf-8'))
39     original = np.array(audit['nodes'][0]['grid'])
40     rows = []
41     for node in audit['nodes'][1:]:
42         hour = node['hour']
43         part = output/f'FIV_{hour:02d}.json'
44         if part.exists():
45             saved = json.loads(part.read_text(encoding='utf-8'))
46             if saved['signature'] != signature:
47                 raise ValueError('已保存FIV分片配置不同')
48             row = saved['row']
49         else:
50             row, branches = fiv(data, terminal, day, hour, node['initial_soc'], original[hour*6:], config,
51                                checkpoint_root=output/f'{hour:02d}', require=False)
52             row['sample_scope'] = 'four_predeclared_dates_only'
53             write_json(part, {'signature': signature, 'row': row, 'branches': branches})
54             rows.append(row)
55             result = {'date': str(date_of(day).date()), 'variant': label, 'FIV': rows}
56     else:
57         frame, audit, _ = run_day(data, terminal, day, initial, config, checkpoint_root=output/'nodes',
58                                   max_slices=1, require=False)
59         write_csv(output/'dispatch.csv', frame); write_json(output/'audit.json', audit)
60         summary = summaries(frame)[0].iloc[0].to_dict()
61         result = {**summary, 'variant': label, 'sample_scope': 'single_day_same_main_initial_SOC',
62                  'certified_all_nodes': all(node['solver']['reliable'] for node in audit['nodes']),
63                  'max_solver_gap': max(node['solver']['gap'] for node in audit['nodes']),
64                  'nodes_count': len(audit['nodes'])}
65     names = [f'FIV_{hour:02d}.json' for hour in (6, 12, 18)] if label == 'FIV' else ['dispatch.csv', 'audit.json']
66     write_json(receipt, {'signature': signature, 'result': result,
67                        'hashes': {name: hashlib.sha256((output/name).read_bytes()).hexdigest() for name in names}})
68     return result
69
70
71 def run_samples(data: Inputs, base: Config, main: pd.DataFrame, output: Path, main_directory: Path,
72                workers: int) -> None:
73     output.mkdir(parents=True, exist_ok=True)
74     rows, jobs = [], []
75     for date in DATES:
76         day = (pd.Timestamp(date)-pd.Timestamp('2025-01-01')).days
77         frame = main[pd.to_datetime(main.date) == pd.Timestamp(date)]

```

```

78     if len(frame) != 144:
79         raise ValueError('主结果尚不完整，不能先抢算抽样对照')
80     initial = float(frame.initial_soc.iloc[0])
81     summary = summaries(frame)[0].iloc[0].to_dict()
82     main_audit = json.loads((main_directory/f'audit/{date}.json').read_text(encoding='utf-8'))
83     rows.append(**summary, 'variant': 'M1', 'sample_scope': 'single_day_same_main_initial_SOC',
84                 'certified_all_nodes': all(node['solver']['reliable'] for node in main_audit['
nodes']),
85                 'max_solver_gap': max(node['solver']['gap'] for node in main_audit['nodes']), '
nodes_count': 4))
86     for label, config in **sample_configs(base), 'FIV': base}.items():
87         jobs.append((data, config, day, initial, label, output/date/label, main_directory))
88     write_json(output/'design.json', {'dates': list(DATES), 'variants': list(sample_configs(base)),
89                                     'jobs': len(jobs), 'annual_comparators': False, 'selection_uses_observed_results': False,
90                                     'authority': '用户2026-09-12要求辅助实验只取少量值做表，优先全年主计算',
91                                     'each_auxiliary_solve_slices': 1, 'slice_seconds': base.seconds,
92                                     'interpretation': '限时可行解可列真实费用及gap，不冒充已达标最优或全年稳健结论'})
93     information = []
94     with process_pool(workers, base.solver_threads) as executor:
95         iterator = executor.map(sample_job, jobs) if executor else map(sample_job, jobs)
96         for result in iterator:
97             if result['variant'] == 'FIV':
98                 information.extend(result['FIV'])
99             else:
100                 rows.append(result)
101                 write_csv(output/'sensitivity_table.csv', pd.DataFrame(rows))
102                 write_json(output/'FIV_samples.json', information)
103     frame = pd.DataFrame(rows)
104     differences = []
105     for date in DATES:
106         costs = frame[frame.date == date].set_index('variant').total_cost
107         differences.extend({'date': date, 'hour': hour, 'sample_OUV': float(costs[old]-costs[new]),
108                             'scope': 'single_day_same_initial_state_not_annual_OUV'})
109         for hour, old, new in ((6, 'K0', 'K06'), (12, 'K06', 'K0612'), (18, 'K0612', 'M1')):
110             write_csv(output/'OUV_samples.csv', pd.DataFrame(differences))
111             write_csv(output/'FIV_samples.csv', pd.DataFrame([{'key': row[key] for key in
112                 ('date', 'hour', 'FIV_block', 'FIV_day', 'FIV_cmp', 'FIV_cont', 'comparison_complete')} for
row in information]))
113     write_json(output/'all_complete.json', {'dates': list(DATES), 'rows': len(rows), 'FIV_rows': len
(information),
114                                     'annual_conclusion_claimed': False})

```

src/q3/scenarios.py

```

1  """3.8.1: 近期量级条件池、尾部保持PAM k-medoids与配对场景。"""
2  from dataclasses import dataclass
3  import numpy as np
4  from scipy.spatial.distance import cdist
5  from .config import Config, DT
6  from .data import Inputs
7
8
9  def medoids(values: np.ndarray, count: int) -> tuple[np.ndarray, np.ndarray]:
10     """确定性PAM BUILD+SWAP，稳定并列按索引；重复轨迹不复制伪样本。"""
11     n = len(values)
12     if not 1 <= count <= n:

```

```

13     raise ValueError('medoid数无效')
14     distance = cdist(values, values)
15     chosen = [int(np.argmin(distance.sum(axis=0)))]
16     while len(chosen) < count:
17         candidates = [j for j in range(n) if j not in chosen]
18         chosen.append(min(candidates, key=lambda j: (np.min(distance[:, chosen+[j]], axis=1).sum(),
19 j)))
19     while True:
20         cost = np.min(distance[:, chosen], axis=1).sum()
21         best = (cost, None)
22         for position in range(count):
23             for candidate in range(n):
24                 if candidate in chosen:
25                     continue
26                 trial = chosen.copy(); trial[position] = candidate
27                 objective = np.min(distance[:, trial], axis=1).sum()
28                 if objective < best[0]-1e-10:
29                     best = objective, trial
30         if best[1] is None:
31             break
32         chosen = best[1]
33     chosen = np.asarray(chosen, dtype=int)
34     labels = np.argmin(distance[:, chosen], axis=1)
35     labels[chosen] = np.arange(count) # 每个真实代表至少代表其自身，零距离并列可任意分配。
36     return chosen, labels
37
38
39 def pool_indices(days: np.ndarray, scales: np.ndarray, day: int, current_scale: float,
40                 config: Config) -> tuple[np.ndarray, dict]:
41     if len(days) == 0:
42         return np.array([], dtype=int), {'window': None, 'candidate_count': 0}
43     indices = np.arange(len(days)); window = None
44     if not config.unconditional:
45         for window in (28, 56, 84, None):
46             indices = np.flatnonzero(day-days <= window) if window is not None else np.arange(len(
47 days))
48             if len(indices) >= config.scenarios or window is None:
49                 break
50             cuts = np.quantile(scales[indices], [1/3, 2/3])
51             bins = np.searchsorted(cuts, scales[indices], side='right')
52             current_bin = int(np.searchsorted(cuts, current_scale, side='right'))
53             selected = []
54             for b in sorted(range(3), key=lambda b: (abs(b-current_bin), b)):
55                 selected.extend(indices[bins == b].tolist())
56             if len(selected) >= config.scenarios:
57                 break
58             indices = np.asarray(sorted(selected), dtype=int)
59     return indices, {'window': window, 'candidate_count': len(indices), 'unconditional': config.
60 unconditional}
61
62 def reduce_errors(errors: np.ndarray, prices: np.ndarray, config: Config) -> tuple[np.ndarray, np.
63 ndarray, dict]:
64     n = len(errors)
65     count = min(config.scenarios, n)
66     if n <= count:
67         return np.arange(n), np.full(n, 1/n), {'tail_indices': [], 'reduced': False}

```

```

66     standardized = (errors-errors.mean(0))/(errors.std(0)+np.finfo(float).eps)
67     risk = np.maximum(errors, 0)*prices*DT
68     tail = np.argsort(-risk, kind='stable')[:config.tail]
69     rest = np.array([j for j in range(n) if j not in tail])
70     ordinary, _ = medoids(standardized[rest], count=len(tail)) if count > len(tail) else (np.array
71     ([], dtype=int), None)
72     representatives = np.r_[tail, rest[ordinary]]
73     labels = np.argmin(cdist(standardized, standardized[representatives]), axis=1)
74     labels[representatives] = np.arange(count)
75     weights = np.bincount(labels, minlength=count)/n
76     return representatives, weights, {'tail_indices': tail.tolist(), 'risk_scores': risk.tolist(), '
77     reduced': True,
78     'represented_counts': np.bincount(labels, minlength=count).
79     tolist()}
80
81 @dataclass
82 class Scenarios:
83     load: np.ndarray
84     pv: np.ndarray
85     weights: np.ndarray
86     audit: dict
87
88     @property
89     def net(self) -> np.ndarray:
90         return (self.load-self.pv)*DT
91
92 def construct(data: Inputs, day: int, hour: int, config: Config, *, origin: int | None = None,
93             length: int = 144, paired_days: np.ndarray | None = None,
94             paired_weights: np.ndarray | None = None) -> Scenarios:
95     origin = hour if origin is None else origin
96     offset = (hour-origin)*6
97     name = data.selected[day]
98     point_load = data.load(day)[hour*6:hour*6+length]
99     full_pv = data.pv(day, origin, config.interpolation)
100     point_pv = full_pv[offset:offset+length]
101     if len(point_pv) != length or len(point_load) != length:
102         raise ValueError('预测支持不足, 禁止填未来真值')
103     days, el, ep, scales = data.history(day, hour, name, method=config.interpolation,
104                                         forecast_hour=origin, length=length)
105     audit = {'day': day, 'hour': hour, 'forecast_origin': origin, 'length': length, 'predictor':
106             name,
107             'history_boundary_slot': day*144+hour*6}
108     if config.deterministic or len(days) == 0:
109         return Scenarios(point_load[None], point_pv[None], np.ones(1), {**audit, 'history_days': [],
110             'S': 1, 'M0': True})
111     if paired_days is None:
112         indices, pool = pool_indices(days, scales, day, full_pv.sum()*DT, config)
113         reps, weights, reduction = reduce_errors(el[indices]-ep[indices],
114                                                  data.prices[(hour*6+np.arange(length))%144], config)
115         selected = indices[reps]
116         audit.update(pool, **reduction)
117     else:
118         lookup = {int(j): i for i, j in enumerate(days)}
119         if any(int(j) not in lookup for j in paired_days):
120             raise ValueError('配对场景包含当前尚未完整实现或predictor不匹配轨迹')

```

```

118         selected = np.array([lookup[int(j)] for j in paired_days])
119         weights = np.asarray(paired_weights, dtype=float)
120         audit.update(history_days=days[selected].tolist(), S=len(selected), weights=weights.tolist(), M0
121                        =False)
122     return Scenarios(np.maximum(point_load+el[selected], 0), np.maximum(point_pv+ep[selected], 0),
123                     weights, audit)

```

src/q3/solver_probe.py

```

1  """固定真实节点的求解器对比；只写隔离证据，不改活动运行。"""
2  import argparse
3  from dataclasses import replace
4  import hashlib
5  import json
6  from pathlib import Path
7  import time
8  from unittest.mock import patch
9  import numpy as np
10 import pandas as pd
11 from scipy.optimize._highspy import _core as highs
12 from scipy.optimize import LinearConstraint, Bounds
13 from scipy.sparse import vstack, csr_matrix, hstack, coo_matrix
14 from pyscipopt import Model, quicksum
15 from .config import ROOT, Config, CAP, TOL, ETA, E_MIN, E_MAX, write_json
16 from .data import read_inputs
17 from .scenarios import Scenarios, construct
18 from .physics import Policy, replay
19 from .optimization import objective
20 from .fast_solver import (build_matrix, build_matrix_v2_free, build_matrix_v2_2bit, matrix_digest,
21                           seed_vector, matrix_error, solve_fast)
22 from .checkpoint import request_digest
23 from .rolling import previous_net
24
25
26 def discharge_hull(objective, bounds, integer, constraints, indices, scenes, initial):
27     """四个反馈分支的透视凸包：整数时与原紧凑矩阵相同，连续时加强下界。"""
28     net=scenes.net;s,t=net.shape;size=len(objective)
29     split=np.arange(size,size+s*t*12).reshape(s,t,3,4)
30     indices['hull_split']=split
31     lower=np.r_[bounds.lb,np.zeros(s*t*12)];upper=np.r_[bounds.ub,np.full(s*t*12,np.inf)]
32     rows=[];cols=[];vals=[];lbs=[];ubs=[]
33     def add(terms,lo=-np.inf,hi=0.):
34         row=len(ubs)
35         for col,value in terms:
36             rows.append(row);cols.append(int(col));vals.append(float(value))
37             lbs.append(lo);ubs.append(hi)
38     c,r,q,w,e=indices['responses'];lr=indices['lr'];z=indices['z']
39     for j in range(s):
40         emin=emax=initial
41         for i in range(t):
42             n=float(net[j,i]);gu=float(bounds.ub[indices['g'][i]])
43             gb,pb,eb=split[j,i]
44             # lambda_0=1-z; lambda_1..3为原有的三个最小值选择变量。
45             selectors=[(int(z[j,i]),-1,1.)+[(int(a[j,i]),1,0.) for a in lr]
46                        for block,total in ((gb,indices['g'][i]),(pb,indices['rp'][i]))):
47                 add([(x,1) for x in block]+[(total,-1)],0.,0.)

```

```

48         add([(x,1) for x in eb]+([(e[j,i-1],-1)]),initial if i==0 else 0.,
initial if i==0 else 0.)
49         for branch,(selector,sign,constant) in enumerate(selectors):
50             for column,lo,hi in ((gb[branch],0.,gu),(pb[branch],0.,CAP),(eb[branch],emin,emax)):
51                 add([(column,1),(selector,-hi*sign)],hi=hi*constant)
52                 add([(column,-1),(selector,lo*sign)],hi=-lo*constant)
53             # 充电分支g>=n. 且只允许该分支充电。
54             selector,sign,constant=selectors[0]
55             add([(c[j,i],1),(selector,-CAP*sign)],hi=CAP*constant)
56             add([(c[j,i],1),(gb[0],-1),(selector,n*sign)],hi=-n*constant)
57             add([(c[j,i],ETA),(eb[0],1),(selector,-E_MAX*sign)],hi=E_MAX*constant)
58             add([(w[j,i],1),(gb[0],-1),(c[j,i],1),(selector,n*sign)],-n*constant,-n*constant)
59             # 放电上限分支: rp <= deficit, available.
60             l1,l2,l3=[a[j,i] for a in lr]
61             add([(pb[1],1),(gb[1],1),(l1,-n)])
62             add([(pb[1],1),(eb[1],-ETA),(l1,ETA*E_MIN)])
63             # 净缺口分支: 0 <= deficit <= rp, available.
64             add([(gb[2],1),(l2,-n)])
65             add([(gb[2],-1),(l2,n),(pb[2],-1)])
66             add([(gb[2],-1),(l2,n+ETA*E_MIN),(eb[2],-ETA)])
67             # 剩余储能分支: available <= rp, deficit.
68             add([(eb[3],ETA),(l3,-ETA*E_MIN),(pb[3],-1)])
69             add([(eb[3],ETA),(l3,-ETA*E_MIN-n),(gb[3],1)])
70             # 汇总三分支放电: q由能量平衡自动得到, 避免重复分量。
71             add([(r[j,i],1),(pb[1],-1),(l2,-n),(gb[2],1),(eb[3],-ETA),(l3,ETA*E_MIN)],0.,0.)
72             # 可达储能界覆盖原cp=CAP反馈: 不改变任何原策略。
73             emin=max(E_MIN,emin-min(CAP,max(n,0))/ETA)
74             emax=min(E_MAX,emax+ETA*min(CAP,max(gu-n,0)))
75             extra=coo_matrix((vals,(rows,cols)),shape=(len(ubs),len(lower))).tocsc()
76             matrix=vstack([hstack([constraints.A,coo_matrix((constraints.A.shape[0],len(lower)-size))],
extra)].tocsc()
77             return np.r_[objective,np.zeros(len(lower)-size)],Bounds(lower,upper),np.r_[integer,np.zeros(len
(lower)-size)],LinearConstraint(matrix,np.r_[constraints.lb,lbs],np.r_[constraints.ub,ubs]),
indices
78
79
80 def hull_seed(vector, indices, initial):
81     """把经原反馈验证的完整初值投影到分支拆分变量。"""
82     split=indices['hull_split'];s,t=split.shape[:2]
83     e=indices['responses'][4]
84     for j in range(s):
85         for i in range(t):
86             branch=0 if vector[indices['z'][j,i]]<.5 else 1+int(np.argmax([vector[a[j,i]] for a in
indices['lr']]))
87             vector[split[j,i,0,branch]]=vector[indices['g'][i]]
88             vector[split[j,i,1,branch]]=vector[indices['rp'][i]]
89             vector[split[j,i,2,branch]]=initial if i==0 else vector[e[j,i-1]]
90     return vector
91
92
93
94 def distance_cuts(f,b,integer,c,idx,scenes):
95     """同一因果反馈对输入路径的非扩张界, 包含全部场景对。"""
96     from scipy.sparse import coo_matrix
97     from .config import ETA,E_MAX,E_MIN
98     rows=[];cols=[];values=[];rhs=[]
99     e=idx['responses'][4];q=idx['responses'][2]

```

```

100     for a in range(len(scenes.weights)):
101         for d in range(a+1,len(scenes.weights)):
102             lo=hi=0.
103             for t,delta in enumerate(scenes.net[a]-scenes.net[d]):
104                 qhi=max(delta,0)+ETA*max(-lo,0);qlo=-max(-delta,0)-ETA*max(hi,0)
105                 lo=max(-(E_MAX-E_MIN),min(lo,0)-max(delta,0)/ETA)
106                 hi=min(E_MAX-E_MIN,max(hi,0)+max(-delta,0)/ETA)
107                 for block,l,u in ((e,lo,hi),(q,qlo,qhi)):
108                     for sign,bound in ((1,u),(-1,-l)):
109                         if bound>=b.ub[block[a,t]]-b.lb[block[d,t]] and sign==1:continue
110                         if bound>=b.ub[block[d,t]]-b.lb[block[a,t]] and sign==-1:continue
111                         k=len(rhs);rows.extend([k,k]);cols.extend([block[a,t],block[d,t]]);values.
112 extend([sign,-sign]);rhs.append(bound)
113 matrix=coo_matrix((values,(rows,cols)),shape=(len(rhs),len(f))).tocsc()
114 return f,b,integer,LinearConstraint(vstack([c.A,matrix]).tocsc(),np.r_[c.lb,np.full(len(rhs),-np
115 .inf)],np.r_[c.ub,rhs]),idx)
116
117 def capture(run: Path, date: str, output: Path) -> None:
118     output.mkdir(parents=True, exist_ok=True)
119     execution=json.loads((run/'execution.json').read_text(encoding='utf-8'))
120     values=execution['config'];values['nodes']=tuple(values['nodes']);config=Config(**values)
121     day=(pd.Timestamp(date)-pd.Timestamp('2025-01-01')).days
122     data=read_inputs(Path(execution['q2_run']),output/'prepared')
123     scenes=construct(data,day,6,config)
124     folder=run/'main/nodes'/date
125     saved=json.loads((folder/'06/latest.json').read_text(encoding='utf-8'))['content']
126     midnight=json.loads((folder/'00/latest.json').read_text(encoding='utf-8'))['content']
127     initial=float(pd.read_csv(folder/'00/executed_block.csv',float_precision='round_trip').soc.iloc
128 [-1])
129     reference=np.asarray(midnight['policy']['grid'])[36:]
130     today=108;prices=data.prices[(36+np.arange(144))%144];terminal=saved['audit']['terminal']
131     with patch.object(Config,'signature',return_value=execution['signature']):
132         signature=request_digest(config,scenes.load,scenes.pv,scenes.weights,prices,initial,
133         reference,today,terminal,previous_net(data,day,6),None,None,None)
134     if signature!=saved['signature']:raise ValueError('固定节点输入签名重建不符')
135     with (output/'input.dat').open('wb') as stream:
136         np.savez_compressed(stream,load=scenes.load,pv=scenes.pv,weights=scenes.weights,net=scenes.
137 net,prices=prices,
138         initial=initial,reference=reference,today=today,terminal=terminal,
139         grid=saved['policy']['grid'],charge_cap=saved['policy']['charge_cap'],discharge_cap=
140 saved['policy']['discharge_cap'])
141     policy_sha=hashlib.sha256(json.dumps(saved['policy'],sort_keys=True,separators=(',',':')).encode
142 ()).hexdigest()
143     matrices={}
144     for name,builder in (('legacy',build_matrix),('v2_free',build_matrix_v2_free),('v2_2bit',
145 build_matrix_v2_2bit)):
146         matrix=builder(scenes,prices,initial,reference=reference,today=today,terminal=terminal)
147         matrices[name]={ 'digest':matrix_digest(*matrix[:4]), 'variables':len(matrix[0]),
148         'constraints':int(matrix[3].A.shape[0]), 'binaries':int(matrix[2].sum())}
149     input_sha=hashlib.sha256((output/'input.dat').read_bytes()).hexdigest()
150     record={'run':str(run),'date':date,'hour':6,'signature':signature,'today':today,
151         'source_audit':saved['audit'],'scenario_audit':scenes.audit,'policy_sha256':policy_sha,
152         'input_sha256':input_sha,'matrices':matrices}
153     write_json(output/'input.json',record)
154     manifest_path=output.parent/'matrix_manifest.json'
155     manifest=json.loads(manifest_path.read_text(encoding='utf-8')) if manifest_path.exists() else {'

```



```

150     nodes':{}}
151     manifest['nodes'][date]=record;write_json(manifest_path,manifest)
152
153 def policy_sha(policy: Policy) -> str:
154     payload={key:getattr(policy,key).tolist() for key in ('grid','charge_cap','discharge_cap')}
155     return hashlib.sha256(json.dumps(payload,sort_keys=True,separators=(',',':')).encode()).
156         hexdigest()
157
158 def benchmark(folder: Path, formulation: str, seconds: float, threads: int, output: Path,
159             solver_focus: str='default') -> dict:
160     data=np.load(folder/'input.dat');scenes=Scenarios(data['load'],data['pv'],data['weights'],{})
161     if 'net' in data.files:np.testing.assert_array_equal(data['net'],scenes.net)
162     initial=float(data['initial']);terminal=float(data['terminal']);prices=data['prices'];ref=data['
163         reference']
164     today=int(data['today']) if 'today' in data.files else 108
165     seed=Policy(data['grid'],np.full(len(prices),CAP),data['discharge_cap'])
166     cfg=Config(gap=.03,seconds=seconds,solver_threads=threads,solver_backend='highs',
167         formulation=formulation,solver_focus=solver_focus)
168     records=[];started=time.perf_counter()
169     seed_responses=[replay(seed,path,initial) for path in scenes.net]
170     records.append({'seconds':0.,'objective':objective(seed,seed_responses,scenes.weights,prices,ref
171         ,today,terminal),
172         'lower_bound':None,'gap':None,'reliable':False,'policy_sha256':policy_sha(seed)
173     })
174     def observe(policy, responses, audit):
175         records.append({'seconds':audit.get('seconds',time.perf_counter()-started),'objective':audit
176             ['objective'],
177             'optimizer_objective':audit.get('optimizer_objective'),'lower_bound':audit.get('
178             lower_bound'),
179             'gap':audit.get('gap'),'reliable':audit.get('reliable'),'node_count':audit.get('
180             node_count'),
181             'simplex_iterations':audit.get('simplex_iterations'),'barrier_iterations':audit.get('
182             barrier_iterations'),
183             'policy_sha256':policy_sha(policy)})
184     policy, responses, audit=solve_fast(scenes,prices,initial,cfg,reference=ref,today=today,terminal=
185     terminal,
186     seed=seed,seconds=seconds,progress=observe,formulation=formulation)
187     audit.update({'wall_seconds':time.perf_counter()-started,'backend':'highs',
188         'formulation_version':formulation,'policy_sha256':policy_sha(policy),
189         'solver_focus':solver_focus,'trajectory':records,'fixture':folder.name,'
190         original_replay_verified':True})
191     output.parent.mkdir(parents=True,exist_ok=True);write_json(output,audit)
192     return audit
193
194 def probe(folder: Path, method: str, seconds: float) -> dict:
195     started=time.perf_counter()
196     data=np.load(folder/'input.dat');scenes=Scenarios(data['load'],data['pv'],data['weights'],{})
197     initial=float(data['initial']);terminal=float(data['terminal']);prices=data['prices'];ref=data['
198         reference']
199     today=int(data['today']) if 'today' in data.files else 108
200     f,b,integer,c,idx=build_matrix(scenes,prices,initial,reference=ref,today=today,terminal=terminal
201     )
202     if method.startswith('distance'):f,b,integer,c,idx=distance_cuts(f,b,integer,c,idx,scenes)
203     if method.startswith('hull'):

```

```

194     f,b,integer,c,idx=discharge_hull(f,b,integer,c,idx,scenes,initial)
195     p=Policy(np.minimum(data['grid'],np.maximum(scenes.net.max(0),0)+CAP),np.full(len(prices),CAP),
196             data['discharge_cap'])
197     x=seed_vector(p,scenes,prices,initial,idx,len(f),ref)
198     if 'hull_split' in idx:x=hull_seed(x,idx,initial)
199     if max(matrix_error(x,b,integer,c))>TOL:raise ValueError('对比初值不可行')
200     original_constraints=c;cutoff=None
201     if method.startswith('cut'):
202         cutoff=float(f@x-.03*abs(f@x))
203         c=LinearConstraint(vstack([c.A,csr_matrix(f.reshape(1,-1))]).tocsc(),np.r_[c.lb,-np.inf],np.
204                             r_[c.ub,cutoff])
205     threads=int(method[-1]);kind=method[:-1]
206     if kind in ('highs','hull','cut','bound','seed','new','distance'):
207         api=highs
208         if kind=='new':
209             from highs import _core as api
210             api._Highs.resetGlobalScheduler(True);m=api._Highs()
211             options={'threads':threads,'parallel':'on' if threads>1 else 'off','output_flag':False,
212                     'mip_rel_gap':.03,'time_limit':seconds,'mip_feasibility_tolerance':1e-8,
213                     'primal_feasibility_tolerance':1e-8}
214             if kind=='bound':options['mip_heuristic_effort']=0.
215             if kind=='seed':options['random_seed']=17
216             for key,value in options.items():
217                 if m.setOptionValue(key,value)==api.HighsStatus.kError:raise ValueError(key)
218             lp=api.HighsLp();a=c.A
219             lp.num_col_,lp.num_row_=len(f),a.shape[0];lp.col_cost_,lp.col_lower_,lp.col_upper_=f,b.lb,b.
220             ub
221             lp.row_lower_,lp.row_upper_=c.lb,c.ub;lp.a_matrix_.num_col_,lp.a_matrix_.num_row_=len(f),a.
222             shape[0]
223             lp.a_matrix_.format_=api.MatrixFormat.kColwise
224             lp.a_matrix_.start_,lp.a_matrix_.index_,lp.a_matrix_.value_=a.indptr,a.indices,a.data
225             lp.integrality_[api.HighsVarType(int(v)) for v in integer];m.passModel(lp)
226             if cutoff is None:
227                 seed=api.HighsSolution();seed.col_value=x;seed.value_valid=True;m.setSolution(seed)
228                 m.run();info=m.getInfo();lower=float(info.mip_dual_bound)
229                 if info.primal_solution_status==api.kSolutionStatusFeasible:x=np.array(m.getSolution().
230                 col_value)
231                 status=m.modelStatusToString(m.getModelStatus())
232                 if cutoff is not None:
233                     lower=cutoff if status=='Infeasible' else min(lower,cutoff)
234             else:
235                 m=Model('q3_compact_probe');m.hideOutput();m.setRealParam('limits/time',seconds)
236                 m.setRealParam('limits/gap',.03/(1+.03));m.setRealParam('numerics/feastol',1e-8)
237                 m.setIntParam('parallel/minnthreads',threads);m.setIntParam('parallel/maxnthreads',threads)
238                 variables=[m.addVar(lb=float(lo),ub=None if np.isposinf(hi) else float(hi),vtype='B' if i
239                 else 'C',obj=float(cost))
240                             for lo,hi,i,cost in zip(b.lb,b.ub,integer,f)]
241                 a=c.A.tocsr()
242                 for row in range(a.shape[0]):
243                     start,end=a.indptr[row:row+2];expr=quicksum(float(value)*variables[col] for col,value in
244                     zip(a.indices[start:end],a.data[start:end]))
245                     if c.lb[row]==c.ub[row]:m.addCons(expr==float(c.ub[row]))
246                     else:
247                         if np.isfinite(c.lb[row]):m.addCons(expr>=float(c.lb[row]))
248                         if np.isfinite(c.ub[row]):m.addCons(expr<=float(c.ub[row]))
249                 seed=m.createSol()
250                 for var,value in zip(variables,x):m.setSolVal(seed,var,float(value))

```

```

243     m.addSol(seed)
244     if threads>1:m.solveConcurrent()
245     else:m.optimize()
246     lower=float(m.getDualbound());status=str(m.getStatus())
247     if m.getNSols():x=np.array([m.getVal(v) for v in variables])
248     policy=Policy(*[np.maximum(x[idx[k]],0) for k in ('g','cp','rp')])
249     responses=[replay(policy,n,initial) for n in scenes.net]
250     value=objective(policy,responses,scenes.weights,prices,ref,today,terminal)
251     feasibility,integrality=matrix_error(x,b,integer,original_constraints)
252     raw_soc=x[idx['responses']][4];raw_q=x[idx['responses']][2]
253     projection=max(0.,float(np.max(raw_soc-np.array([r['soc'] for r in responses])),float(np.max(np
        .array([r['emergency'] for r in responses])-raw_q)))
254     if max(feasibility,projection)>TOL or integrality>1e-8 or value>f@x+TOL or lower>value+TOL:raise
        ValueError('独立物理/费用/界核验不通过')
255     audit={'method':method,'seconds':time.perf_counter()-started,'status':status,'objective':value,'
        lower_bound':lower,
256         'gap':max(0.,value-lower)/max(abs(value),1e-10),'reliable':bool(value-lower<=max(TOL,.03*abs
        (value))),
257         'feasibility':feasibility,'integrality':integrality,'projection':projection,'binaries':int(
        integer.sum()),'threads':threads}
258     write_json(folder/f'{method}.json',audit)
259     write_json(folder/f'{method}_policy.json',{k:getattr(policy,k).tolist() for k in ('grid','
        charge_cap','discharge_cap')})
260     return audit
261
262
263 if __name__=='__main__':
264     parser=argparse.ArgumentParser();parser.add_argument('--capture',type=Path);parser.add_argument(
        '--date')
265     parser.add_argument('--folder',type=Path,required=True);parser.add_argument('--method',choices=[
        'highs1','highs4','scip1','scip4','hull1','cut1','bound1','seed1','new1','new4','distance1'])
266     parser.add_argument('--seconds',type=float,default=60.);parser.add_argument('--benchmark-output'
        ,type=Path)
267     parser.add_argument('--formulation',choices=('legacy','v2_free'),default='v2_free');parser.
        add_argument('--threads',type=int,default=4)
268     parser.add_argument('--solver-focus',choices=('default','bound'),default='default')
269     args=parser.parse_args()
270     if args.capture:capture(args.capture,args.date,args.folder)
271     elif args.benchmark_output:print(json.dumps(benchmark(args.folder,args.formulation,args.seconds,
        args.threads,args.benchmark_output,args.solver_focus)))
272     elif args.method:print(json.dumps(probe(args.folder,args.method,args.seconds)))
273     else:parser.error('必须指定capture、benchmark-output或method')

```

src/q3/status_server.py

```

1  """本地实时看板：核对进程存活，不把旧progress.json当作运行证据。"""
2  from http.server import ThreadingHTTPServer, SimpleHTTPRequestHandler
3  from pathlib import Path
4  import json
5  import shlex
6  import subprocess
7  import time
8  from urllib.parse import urlparse
9
10 ROOT=Path(__file__).resolve().parents[2]
11 WEB=ROOT/'outputs/processed/q3_monitor'

```

```

12 POINTER=ROOT/'outputs/q3/active_run.json'
13
14
15 def read(path: Path) -> dict:
16     return json.loads(path.read_text(encoding='utf-8')) if path.exists() else {}
17
18
19 def worker_alive(pid: int | None, run: Path) -> bool:
20     if not pid: return False
21     result=subprocess.run(['ps', '-p', str(pid), '-o', 'command='], capture_output=True, text=True, timeout=
22     =2)
23     args=shlex.split(result.stdout.strip()) if result.returncode==0 else []
24     return 'src.q3.full_run' in args and '--worker' in args and str(run) in args
25
26 def current() -> dict:
27     pointer=read(POINTER)
28     run=Path(pointer.get('run_directory', ROOT/'outputs/q3/raw/full_priority_20260912')).resolve()
29     if not run.is_relative_to(ROOT/'outputs/q3/raw'): raise ValueError('非Q3运行路径')
30     progress=read(run/'progress.json'); state=read(run/'main/state.json'); active=read(run/'main/
31     active.json')
32     execution=read(run/'execution.json'); rescue=bool(execution.get('config', {}).get('
33     production_rescue'))
34     date=active.get('date') or progress.get('active_date')
35     candidates=list((run/'main/nodes'/date).glob('*/latest.json')) if date else []
36     latest=max(candidates, key=lambda p: p.stat().st_mtime) if candidates else None
37     audit=read(latest).get('content', {}).get('audit', {}) if latest else {}
38     alive=worker_alive(progress.get('worker_pid'), run)
39     cpu=None
40     if alive:
41         result=subprocess.run(['ps', '-p', str(progress['worker_pid']), '-o', '%cpu='], capture_output=
42         True, text=True, timeout=2)
43         if result.returncode==0 and result.stdout.strip(): cpu=float(result.stdout.strip())
44     ready=(run/'main_complete.json').exists()
45     status='running' if alive else 'complete' if ready else pointer.get('transition', 'stopped')
46     if status=='running' and not alive: status='stopped'
47     saved=len(state.get('days', [])); formal=max(0, saved-31)
48     log=''
49     if (run/'worker.log').exists():
50         with (run/'worker.log').open('rb') as f:
51             f.seek(max(0, (run/'worker.log').stat().st_size-5000)); log=f.read().decode('utf-8', errors
52             ='replace')
53     val=read(ROOT/'outputs/q3/raw/validation_20260912/status.json')
54     certificate=read(ROOT/'outputs/q3/raw/compute_rescue_20260912/jan14_06_strengthened.json')
55     return {'status': status, 'worker_alive': alive, 'worker_pid': progress.get('worker_pid') if alive
56     else None,
57     'run_name': run.name, 'formal_days': formal, 'formal_total': 334, 'warm_days': min(saved, 31), '
58     warm_total': 31,
59     'active_date': date, 'active_hour': latest.parent.name if latest else progress.get('active_hour
60     '),
61     'gap': audit.get('gap'), 'target_gap': .03, 'node_reliable': audit.get('reliable', False),
62     'node_accepted': audit.get('accepted', False), 'accepted_by': audit.get('accepted_by'),
63     'hard_limit_seconds': audit.get('hard_limit_seconds'), 'production_rescue': rescue,
64     'slices': audit.get('completed_slices', 0), 'node_seconds': audit.get('seconds'),
65     'saved_ago_seconds': time.time()-latest.stat().st_mtime if latest else None,
66     'soc': state.get('soc'), 'solver': audit.get('solver', audit.get('solve_method', '-')),
67     'solver_threads': audit.get('solver_threads'), 'solver_version': audit.get('solver_version'), '

```

```

61     'cpu_percent':cpu,
62     'lower_bound':audit.get('lower_bound'),'objective':audit.get('objective'),
63     'validation_phase':'disabled_main_only' if rescue else val.get('phase','未启动'),
64     'validation_scope':'disabled' if rescue else val.get('scope','pending'),
65     'rescue_certificate':{'seconds':certificate.get('seconds'),'gap':certificate.get('gap'),'
passed':certificate.get('reliable')},
66     'main_result_ready':ready,'log_lines':log.splitlines()[-12:], 'observed_at':time.time()}
67
68 class Handler(SimpleHTTPRequestHandler):
69     def __init__(self,*args,**kwargs):super().__init__(*args,directory=str(WEB),**kwargs)
70     def log_message(self,format,*args):pass
71     def do_GET(self):
72         if urlparse(self.path).path=='/api/status':
73             try:payload=json.dumps(current(),ensure_ascii=False,allow_nan=False).encode('utf-8');
code=200
74             except Exception as error:payload=json.dumps({'error':str(error)},ensure_ascii=False).
encode('utf-8');code=503
75             self.send_response(code);self.send_header('Content-Type','application/json; charset=utf
-8')
76             self.send_header('Cache-Control','no-store');self.send_header('Content-Length',str(len(
payload)));self.end_headers();self.wfile.write(payload)
77             else:super().do_GET()
78
79
80 if __name__=='__main__':ThreadingHTTPServer(('127.0.0.1',8764),Handler).serve_forever()

```

src/q3/terminal.py

```

1  """3.3.4: 用户确认的历史确定性继续价值有限差分标定。"""
2  from dataclasses import replace
3  from concurrent.futures import Executor
4  import json
5  import hashlib
6  from pathlib import Path
7  import numpy as np
8  from .config import Config, TOL, write_json
9  from .data import Inputs
10 from .scenarios import Scenarios
11 from .optimization import solve
12
13
14 class TerminalValues:
15     def __init__(self, data: Inputs, config: Config, executor: Executor | None = None, cache_path:
Path | None = None):
16         self.data, self.config, self.executor = data, config, executor
17         self.cache_path = cache_path
18         self.cache_signature = hashlib.sha256((config.signature()+json.dumps(data.provenance,
sort_keys=True)).encode('utf-8')).hexdigest()
19         self.cache: dict[tuple, dict] = {}
20         if cache_path and cache_path.exists():
21             stored = json.loads(cache_path.read_text(encoding='utf-8'))
22             content = stored['content']
23             payload = json.dumps(content, sort_keys=True, allow_nan=False).encode('utf-8')
24             if stored['sha256'] != hashlib.sha256(payload).hexdigest() or content['signature'] !=
self.cache_signature:

```

```

25         raise ValueError('FD持久缓存与当前配置/源码不一致')
26     for row in content['rows']:
27         self.cache[(int(row['history_day']), int(row['hour']), float(row['delta']))] = row
28
29     def remember(self, row: dict) -> None:
30         self.cache[(row['history_day'], row['hour'], row['delta'])] = row
31         if self.cache_path:
32             content = {'signature': self.cache_signature, 'rows': list(self.cache.values())}
33             payload = json.dumps(content, sort_keys=True, allow_nan=False).encode('utf-8')
34             write_json(self.cache_path, {'sha256': hashlib.sha256(payload).hexdigest(), 'content':
content})
35
36     def request(self, day: int, hour: int, delta: float) -> tuple:
37         start = day*144+hour*6
38         return (day, hour, delta, self.data.actual.reshape(-1, 2)[start:start+144].copy(),
39                 self.data.prices[(hour*6+np.arange(144))%144].copy(), self.config)
40
41     def marginal(self, day: int, hour: int, delta: float) -> dict:
42         key = (day, hour, delta)
43         if key in self.cache:
44             return self.cache[key]
45         row = marginal_job(self.request(day, hour, delta))
46         self.remember(row)
47         return row
48
49     def value(self, day: int, endpoint_hour: int) -> tuple[float, dict]:
50         hour = endpoint_hour%24
51         # 模型要求仅当前日期以前完全实现；比当前节点截止更严格，不纳入当天实现样本。
52         history = [j for j in range(day) if j*144+hour*6+144 <= day*144]
53         history = history[-self.config.fd_days:]
54         if not history:
55             raise ValueError('没有完整历史辅助问题，不能人为指定终端价值')
56         missing = [j for j in history if (j, hour, self.config.fd_delta) not in self.cache]
57         if self.executor is not None and len(missing) > 1:
58             requests = [self.request(j, hour, self.config.fd_delta) for j in missing]
59             for row in self.executor.map(marginal_job, requests):
60                 self.remember(row)
61         rows = [self.marginal(j, hour, self.config.fd_delta) for j in history]
62         coefficient = float(np.median([row['value'] for row in rows]))
63         if coefficient <= 0:
64             raise ValueError(f'历史中位数为零(day={day}, hour={hour})，与v4严格正终端价值冲突，需确认，禁止填
任意正数')
65         return coefficient*self.config.terminal_scale, {
66             'endpoint_hour': hour, 'history_days': history, 'median_unscaled': coefficient,
67             'scale': self.config.terminal_scale, 'source_cutoff_slot': day*144,
68             'delta': self.config.fd_delta, 'initial_soc': self.config.fd_base,
69             'auxiliary_information': 'fully_realized_historical_24h_deterministic;
terminal_value_only'}
70
71     def stability(self, day: int, hour: int) -> list[dict]:
72         """方案要求小范围扰动检查：50/100/200是delta=100的半倍/原值/双倍。"""
73         return [self.marginal(day, hour, self.config.fd_delta*factor) for factor in (.5, 1., 2.)]
74
75
76     def marginal_job(request: tuple) -> dict:
77         day, hour, delta, actual, price, config = request
78         start = day*144+hour*6

```

```

79     if len(actual) != 144:
80         raise ValueError('辅助继续价值问题需要完整历史24h')
81     scenarios = Scenarios(actual[None, :, 0], actual[None, :, 1], np.ones(1), {})
82     # 辅助问题两侧必须同口径、同预算、零终端残值；以最优目标差而非真实回放成本差标定。
83     config = replace(config, gap=0., feedback='aggregate', solver_threads=1, production_rescue=False)
84     first = solve(scenarios, price, config.fd_base, config)
85     second = solve(scenarios, price, config.fd_base+delta, config)
86     value = (first.audit['objective']-second.audit['objective'])/delta
87     if value < -TOL:
88         raise ValueError(f'历史边际价值为负({value})，需检查辅助问题证书')
89     value = max(0., value) # 仅消除浮点舍入负零；单个历史日零价值是合法的。
90     row = {'history_day': day, 'hour': hour, 'complete_at_slot': start+144,
91           'initial_soc': config.fd_base, 'delta': delta, 'value': value,
92           'J_E': first.audit['objective'], 'J_E_plus_delta': second.audit['objective'],
93           'seconds': first.audit['seconds']+second.audit['seconds']}
94     return row

```

src/q3/test_fast_solver.py

```

1  """Q3单调等价模型：终端值/调整成本、原反馈投影、原模型最优值对照。"""
2  import unittest
3  import numpy as np
4  from .config import Config, CAP, E_MIN, E_MAX, ETA
5  from .physics import Policy, replay, adjustment
6  from .scenarios import Scenarios
7  from .optimization import solve, objective
8  from .fast_solver import solve_fast, build_matrix, seed_vector, matrix_error
9
10
11  class FastSolverTests(unittest.TestCase):
12      def test_positive_terminal_preserves_monotone_projection(self) -> None:
13          rng=np.random.default_rng(48)
14          for initial in (E_MIN, 6000., E_MAX):
15              for _ in range(24):
16                  net=rng.uniform(-900, 1900, (3, 24)); prices=rng.uniform(.4, 1.4, 24); weights=np.full
17                  (3, 1/3)
18                  p=Policy(rng.uniform(0, 1400, 24), np.full(24, CAP), rng.uniform(0, CAP, 24))
19                  reference=rng.uniform(0, 1400, 18); terminal=float(rng.uniform(.2, 1.5))
20                  raw=[]
21                  for path in net:
22                      e=initial; row={k:[] for k in ('charge', 'discharge', 'emergency', 'spill', 'soc')}
23                      for i, n in enumerate(path):
24                          x=n-p.grid[i]
25                          r=min(p.discharge_cap[i], max(x, 0), ETA*(e-E_MIN))
26                          c=rng.uniform(0, 1)*min(CAP, max(-x, 0), (E_MAX-e)/ETA)
27                          e+=ETA*c-r/ETA
28                          for k, v in zip(row, (c, r, max(x-r, 0), max(-x-c, 0), e)): row[k].append(v)
29                      raw.append({k: np.array(v) for k, v in row.items()})
30                  exact=[replay(p, n, initial) for n in net]
31                  for a, b in zip(raw, exact):
32                      self.assertTrue((b['soc']>=a['soc']-1e-8).all())
33                      self.assertTrue((b['emergency']<=a['emergency']+1e-8).all())
34                      self.assertLessEqual(objective(p, exact, weights, prices, reference, 18, terminal),
35                                           objective(p, raw, weights, prices, reference, 18, terminal)+1e-7)
36
37      def test_exact_original_and_compact_optimal_values(self) -> None:

```



```

37     rng=np.random.default_rng(512)
38     for initial in (1200.,6000.,10000.):
39         load=rng.uniform(0,11000,(3,5));pv=rng.uniform(0,7000,(3,5));prices=rng.uniform
40         (.4,1.4,5)
41         scenes=Scenarios(load,pv,np.array([.2,.3,.5]),{});reference=np.full(3,500.)
42         cfg=Config(gap=0.,seconds=20.)
43         original=solve(scenes,prices,initial,cfg,reference=reference,today=3,terminal=.6)
44         _,_,fast=solve_fast(scenes,prices,initial,cfg,reference=reference,today=3,terminal=.6)
45         self.assertTrue(fast['reliable']);self.assertAlmostEqual(original.audit['objective'],
46         fast['objective'],places=5)
47
48     def test_fixed_policy_still_matches_exact_response(self) -> None:
49         scenes=Scenarios(np.array([[0.,12000.,400.,10000.],[6000.,300.,10000.,200.]]),
50         np.array([[3000.,0.,5000.,0.],[0.,6000.,0.,800.]]),np.array([.4,.6]),{})
51         p=Policy(np.array([200.,400.,200.,100.]),np.full(4,100.),np.full(4,200.))
52         _,_,info=solve_fast(scenes,np.ones(4),1250.,Config(gap=0,seconds=15),fixed_policy=p,
53         reference=np.full(4,300.),today=2,terminal=.6)
54         self.assertTrue(info['reliable']);self.assertLess(info['raw_response_difference'],1e-5)
55
56     def test_full_matrix_accepts_exact_feedback_and_prefix_orders(self) -> None:
57         rng=np.random.default_rng(20)
58         load=rng.uniform(100,8000,(1,24))+np.arange(5)[:None]*100
59         pv=np.broadcast_to(rng.uniform(0,6000,(1,24)),load.shape).copy()
60         scenes=Scenarios(load,pv,np.full(5,.2),{});prices=np.ones(24);ref=np.full(18,500.)
61         f,b,integer,c,idx=build_matrix(scenes,prices,6000.,reference=ref,today=18,terminal=.8)
62         upper=np.maximum(scenes.net.max(0),0)+CAP
63         p=Policy(rng.uniform(0,1,24)*upper,np.full(24,CAP),rng.uniform(0,CAP,24))
64         x=seed_vector(p,scenes,prices,6000.,idx,len(f),ref)
65         feasibility,integrality=matrix_error(x,b,integer,c)
66         self.assertLess(feasibility,1e-5);self.assertEqual(integrality,0.)
67
68     def test_grid_dominance_does_not_change_scenario_soc(self) -> None:
69         rng=np.random.default_rng(5);net=rng.uniform(-600,1500,(4,24))
70         safe=np.maximum(net.max(0),0)+CAP
71         p=Policy(safe+rng.uniform(0,1000,24),rng.uniform(0,CAP,24),rng.uniform(0,CAP,24))
72         clipped=Policy(safe,p.charge_cap,p.discharge_cap)
73         for path in net:
74             old,new=replay(p,path,6000.),replay(clipped,path,6000.)
75             np.testing.assert_allclose(old['soc'],new['soc'],atol=1e-8)
76             np.testing.assert_allclose(old['emergency'],new['emergency'],atol=1e-8)
77
78     def test_parallel_solver_callback_preserves_negative_objective_certificate(self) -> None:
79         rng=np.random.default_rng(421)
80         scenes=Scenarios(rng.uniform(500,11000,(5,16)),rng.uniform(0,7000,(5,16)),np.full(5,.2),{})
81         prices=rng.uniform(.4,1.4,16);ref=np.full(12,500.);observed=[]
82         policy,_,audit=solve_fast(scenes,prices,6000.,Config(gap=.03,seconds=5,solver_threads=4),
83         reference=ref,today=12,terminal=1.4,progress=lambda p,r,a:observed.append(a.copy()))
84         self.assertIsNotNone(policy);self.assertTrue(observed)
85         self.assertEqual(audit['solver_version'],'1.15.1');self.assertEqual(audit['solver_threads']
86         ,4)
87         self.assertLess(audit['objective'],0.)
88         self.assertLessEqual(audit['lower_bound'],audit['objective']+1e-5)
89         self.assertLessEqual(audit['linear_feasibility_error'],1e-5)
90         self.assertTrue(audit['reliable'])
91
92     def test_invalid_candidate_bounds_rejected_before_matrix_product(self) -> None:
93         scenes=Scenarios(np.full((1,3),1000.),np.zeros((1,3)),np.ones(1),{})

```



```

90         f,b,i,c,_=build_matrix(scenes,np.ones(3),6000.)
91         for value in (np.inf,np.nan,1e308):
92             errors=matrix_error(np.full(len(f),value),b,i,c)
93             self.assertGreater(errors[0],1e-5)
94
95
96 if __name__=='__main__':unittest.main(verbosity=2)

```

src/q3/test_full_run.py

```

1  """切片、缓存命中、真实SIGKILL恢复、原子发布及抽样任务范围。"""
2  from dataclasses import replace
3  import hashlib
4  import json
5  import os
6  import signal
7  from pathlib import Path
8  import subprocess
9  import sys
10 import tempfile
11 import time
12 import unittest
13 from unittest.mock import patch
14 import numpy as np
15 from .config import ROOT, Config, write_json
16 from .checkpoint import run_lock
17 from .optimization import solve
18 from .scenarios import Scenarios
19 from .data import read_inputs
20 from .terminal import TerminalValues
21 from .rolling import run_day
22 from .parallel_benchmark import frozen_requests
23 from .sampled import DATES, sample_configs
24
25
26 class CheckpointTests(unittest.TestCase):
27     def test_completed_node_replays_without_optimizer(self) -> None:
28         scenes = Scenarios(np.full((1, 8), 1000.), np.zeros((1, 8)), np.ones(1), {})
29         with tempfile.TemporaryDirectory() as folder:
30             request = dict(scenarios=scenes, prices=np.ones(8), initial=6000., config=Config(),
31                             checkpoint=Path(folder))
32             first = solve(**request)
33             with patch('src.q3.optimization.make_model', side_effect=AssertionError('should not
34 solve')):
35                 second = solve(**request)
36                 self.assertTrue(second.audit['restored_complete_node'])
37                 np.testing.assert_array_equal(first.policy.grid, second.policy.grid)
38                 with self.assertRaises(ValueError): solve(**{**request, 'initial': 6001.})
39
40     def test_atomic_json_preserves_previous_on_replace_failure(self) -> None:
41         with tempfile.TemporaryDirectory() as folder:
42             path = Path(folder)/'state.json'; write_json(path, {'valid': 1})
43             with patch('pathlib.Path.replace', side_effect=OSError('simulated interrupted rename')):
44                 with self.assertRaises(OSError): write_json(path, {'valid': 2})
45             self.assertEqual(json.loads(path.read_text(encoding='utf-8')), {'valid': 1})

```

```

45 def test_lock_rejects_second_writer(self) -> None:
46     with tempfile.TemporaryDirectory() as folder:
47         with run_lock(Path(folder)/'run.lock'):
48             with self.assertRaises(RuntimeError):
49                 with run_lock(Path(folder)/'run.lock'): pass
50
51 def test_real_solver_slices_and_snapshot_validation(self) -> None:
52     request = frozen_requests(1.)[0]
53     with tempfile.TemporaryDirectory() as folder:
54         path = Path(folder)
55         result = solve(**request, checkpoint=path, max_slices=2)
56         self.assertEqual(result.audit['completed_slices'], 2)
57         self.assertEqual(len(list(path.glob('slice_*.json'))), 2)
58         saved = json.loads((path/'latest.json').read_text(encoding='utf-8'))
59         saved['content']['policy']['grid'][0] += 1.
60         (path/'latest.json').write_text(json.dumps(saved), encoding='utf-8')
61         with self.assertRaises(ValueError): solve(**request, checkpoint=path)
62
63 def test_event_checkpoint_during_solving(self) -> None:
64     from .optimization import ProgressEvent
65     from .checkpoint import save_snapshot
66     original = ProgressEvent.__init__; saved = []
67     def start_due(self, *args):
68         original(self, *args); self.last -= 11
69     def observe(path, signature, policy, audit):
70         saved.append(audit.copy()); save_snapshot(path, signature, policy, audit)
71     with tempfile.TemporaryDirectory() as folder, patch.object(ProgressEvent, '__init__',
72 start_due), patch('src.q3.checkpoint.save_snapshot', side_effect=observe):
73         solve(**frozen_requests(2.)[0], checkpoint=Path(folder))
74         self.assertTrue(any(row.get('checkpoint_event') for row in saved))
75
76 def test_sigkill_retains_and_restores_incumbent(self) -> None:
77     with tempfile.TemporaryDirectory() as folder:
78         path = Path(folder)
79         code = "from pathlib import Path; import sys; from src.q3.parallel_benchmark import
80 frozen_requests; from src.q3.optimization import solve; solve(**frozen_requests(2.)[0],
81 checkpoint=Path(sys.argv[1]), max_slices=None)"
82         child = subprocess.Popen([sys.executable, '-c', code, str(path)], cwd=ROOT,
83             stdout=subprocess.DEVNULL, stderr=subprocess.PIPE)
84         try:
85             deadline = time.monotonic()+20
86             while not (path/'latest.json').exists() and time.monotonic() < deadline and child.
87 poll() is None:
88                 time.sleep(.05)
89             self.assertTrue((path/'latest.json').exists(), 'worker must save feasible seed
90 before kill')
91             child.kill(); child.wait(timeout=10)
92             saved = json.loads((path/'latest.json').read_text(encoding='utf-8'))['content']['
93 audit']
94             result = solve(**frozen_requests(2.)[0], checkpoint=path)
95             self.assertTrue(result.audit['restored_incumbent'])
96             self.assertLessEqual(result.audit['objective'], saved['objective']+1e-5)
97         finally:
98             if child.poll() is None: child.kill(); child.wait()
99             child.stderr.close()
100
101 def test_parallel_highs_sigkill_preserves_valid_bound(self) -> None:

```

```

96     with tempfile.TemporaryDirectory() as folder:
97         path=Path(folder)
98         code="from pathlib import Path; from dataclasses import replace; import sys; from src.q3
.parallel_benchmark import frozen_requests; from src.q3.optimization import solve; r=
frozen_requests(1.)[0]; r['config']=replace(r['config'], gap=.03, solver_threads=4); solve(**r,
checkpoint=Path(sys.argv[1]), max_slices=None)"
99         child=subprocess.Popen([sys.executable, '-c', code, str(path)], cwd=R00T, stdout=subprocess.
DEVNULL, stderr=subprocess.PIPE)
100         try:
101             deadline=time.monotonic()+25; saved=None
102             while time.monotonic()<deadline and child.poll() is None:
103                 if (path/'latest.json').exists():
104                     saved=json.loads((path/'latest.json').read_text(encoding='utf-8'))['content'
][['audit']]
105                     if saved.get('solver_version')== '1.15.1':break
106                     time.sleep(.05)
107                     self.assertIsNone(child.poll()); self.assertIsNotNone(saved)
108                     self.assertEqual(saved.get('solver_version'), '1.15.1')
109                     child.kill(); child.wait(timeout=10)
110                     request=frozen_requests(1.)[0]; request['config']=replace(request['config'], gap=.03,
solver_threads=4)
111                     request['require']=False
112                     resumed=solve(**request, checkpoint=path, max_slices=1)
113                     self.assertLessEqual(resumed.audit['objective'], saved['objective']+1e-5)
114                     self.assertGreaterEqual(resumed.audit['lower_bound'], saved['lower_bound']-1e-5)
115                     self.assertTrue(resumed.audit['restored_incumbent'])
116             finally:
117                 if child.poll() is None: child.kill(); child.wait(timeout=10)
118                 child.stderr.close()
119
120     def test_experiments_are_four_dates_and_few_values(self) -> None:
121         self.assertEqual(len(DATES), 4)
122         values = sample_configs(Config(gap=.03))
123         self.assertEqual(set(values), {'S10', 'S30', 'terminal08', 'terminal12', 'M0', 'K0', 'K06',
'K0612'})
124         self.assertTrue(all(config.gap == .03 for config in values.values()))
125
126
127     class NodeResumeTests(unittest.TestCase):
128         @classmethod
129         def setUpClass(cls) -> None:
130             cls.data = read_inputs(R00T/'outputs/q2/dispatch_runs/20260911_224501')
131
132     def test_day_resumes_after_second_node_crash(self) -> None:
133         config = Config(gap=.03)
134         with tempfile.TemporaryDirectory() as folder:
135             path = Path(folder); calls = []
136             def fail_second(*args, **kwargs):
137                 calls.append(1)
138                 if len(calls) == 2: raise RuntimeError('simulated node crash')
139                 return solve(*args, **kwargs)
140             with patch('src.q3.rolling.solve', side_effect=fail_second):
141                 with self.assertRaisesRegex(RuntimeError, 'simulated node crash'):
142                     run_day(self.data, TerminalValues(self.data, config), 7, 6000., config,
checkpoint_root=path)
143             before = hashlib.sha256((path/'00/executed_block.csv').read_bytes()).hexdigest()
144             frame, audit, _ = run_day(self.data, TerminalValues(self.data, config), 7, 6000., config

```

```

145         , checkpoint_root=path)
146         self.assertTrue(audit['nodes'][0]['solver']['restored_complete_node'])
147         self.assertEqual(before, hashlib.sha256((path/'00/executed_block.csv').read_bytes()).
148             hexdigest())
149         self.assertEqual(len(frame), 144)
150
151     def test_terminal_one_history_job_persisted(self) -> None:
152         config = Config(gap=.03)
153         with tempfile.TemporaryDirectory() as folder:
154             path = Path(folder)/'cache.json'
155             first = TerminalValues(self.data, config, cache_path=path).marginal(6, 0, 100.)
156             with patch('src.q3.terminal.marginal_job', side_effect=AssertionError('already cached'))
157             :
158                 second = TerminalValues(self.data, config, cache_path=path).marginal(6, 0, 100.)
159                 self.assertEqual(first, second)
160
161     def test_supervisor_restarts_killed_worker(self) -> None:
162         with tempfile.TemporaryDirectory() as folder:
163             root = Path(folder)
164             master = subprocess.Popen([sys.executable, '-m', 'src.q3.full_run', '--run-dir', str(
165                 root),
166                 '--end-day', '9', '--main-only', '--workers', '1', '--seconds', '3'], cwd=ROOT,
167                 stdout=subprocess.DEVNULL, stderr=subprocess.PIPE)
168             try:
169                 deadline = time.monotonic()+30
170                 state = root/'main/state.json'; supervisor = root/'supervisor.json'
171                 while time.monotonic() < deadline:
172                     if state.exists() and len(json.loads(state.read_text(encoding='utf-8'))['days'])
173                     >= 7:
174                         break
175                     if master.poll() is not None: break
176                     time.sleep(.01)
177                 self.assertTrue(state.exists())
178                 first_pid = json.loads(supervisor.read_text(encoding='utf-8'))['history'][0]['pid']
179                 prefix = (root/'main/days/2025-01-01.csv').read_bytes()
180                 os.kill(first_pid, signal.SIGKILL)
181                 self.assertEqual(master.wait(timeout=40), 0)
182                 history = json.loads(supervisor.read_text(encoding='utf-8'))['history']
183                 self.assertGreaterEqual(len(history), 2)
184                 self.assertEqual(history[0]['return_code'], -signal.SIGKILL)
185                 self.assertEqual(prefix, (root/'main/days/2025-01-01.csv').read_bytes())
186                 self.assertEqual(len(json.loads(state.read_text(encoding='utf-8'))['days']), 9)
187             finally:
188                 if master.poll() is None: master.terminate(); master.wait(timeout=15)
189                 master.stderr.close()
190
191 if __name__ == '__main__':
192     unittest.main(verbosity=2)

```

src/q3/test_gurobi_backend.py

```

1 """同矩阵 Gurobi 后端；未安装或许可证不可用时明确跳过。"""
2 import importlib.util
3 import unittest
4 import numpy as np

```

```

5 from .config import Config, CAP, TOL
6 from .fast_solver import build_matrix, build_matrix_v2_free, matrix_digest, matrix_error,
  seed_vector
7 from .gurobi_backend import license_probe, solve_gurobi_matrix
8 from .optimization import solve, objective
9 from .physics import Policy, replay
10 from .scenarios import Scenarios
11
12
13 PROBE=license_probe() if importlib.util.find_spec('gurobipy') else {'usable':False,'reason':
  not_installed'}
14
15
16 @unittest.skipUnless(PROBE.get('usable'),f'Gurobi不可用: {PROBE}')
17 class GurobiBackendTests(unittest.TestCase):
18     def _case(self,initial: float,terminal: float,reference: bool) -> None:
19         rng=np.random.default_rng(int(initial+terminal*10))
20         scenes=Scenarios(rng.uniform(300,9000,(3,5)),rng.uniform(0,6500,(3,5)),np.array([.2,.3,.5])
  ,{})
21         prices=rng.uniform(.4,1.4,5);ref=np.full(3,400.) if reference else None;today=3
22         cfg=Config(gap=0.,seconds=15.,formulation='legacy')
23         original=solve(scenes,prices,initial,cfg,reference=ref,today=today,terminal=terminal)
24         matrix=build_matrix(scenes,prices,initial,reference=ref,today=today,terminal=terminal)
25         f,b,integer,c,indices=matrix
26         seed=seed_vector(original.policy,scenes,prices,initial,indices,len(f),ref)
27         self.assertLessEqual(max(matrix_error(seed,b,integer,c)),TOL)
28         before=matrix_digest(f,b,integer,c)
29         vector,lower,meta=solve_gurobi_matrix(f,b,integer,c,seed_vector=seed,gap=0.,seconds=15.,
  threads=1,
30         assess=lambda vector,bound:(None,None,{}))
31         self.assertEqual(before,matrix_digest(f,b,integer,c))
32         policy=Policy(*[np.maximum(vector[indices[key]],0) for key in ('g','cp','rp')])
33         responses=[replay(policy,path,initial) for path in scenes.net]
34         value=objective(policy,responses,scenes.weights,prices,ref,today,terminal)
35         self.assertLessEqual(max(matrix_error(vector,b,integer,c)),1e-5)
36         self.assertLessEqual(lower,value+1e-5)
37         self.assertAlmostEqual(value,original.audit['objective'],places=5)
38         self.assertEqual(meta['solver'],'Gurobi_same_matrix')
39
40     def test_gap_zero_positive_terminal_adjustment_and_initial_states(self) -> None:
41         for initial in (1200.,6000.,10800.):
42             self._case(initial,.7,True)
43             self._case(6000.,0.,False)
44
45     def test_fixed_policy_uses_legacy_matrix_and_seed(self) -> None:
46         scenes=Scenarios(np.array([[1000.,9000.],[5000.,3000.]]),np.array([[3000.,0.],[0.,4000.]]),
  np.array([.4,.6]),{})
47         policy=Policy(np.array([200.,400.]),np.array([100.,120.]),np.array([150.,180.]))
48         f,b,integer,c,indices=build_matrix(scenes,np.ones(2),6000.,fixed_policy=policy)
49         self.assertFalse(indices['free_charge'])
50         self.assertGreater(len(indices['cp']),0);self.assertGreater(sum(len(x.ravel()) for x in
  indices['lc']),0)
51         seed=seed_vector(policy,scenes,np.ones(2),6000.,indices,len(f),None)
52         self.assertLessEqual(max(matrix_error(seed,b,integer,c)),TOL)
53         with self.assertRaises(TypeError):build_matrix_v2_free(scenes,np.ones(2),6000.,fixed_policy=
  policy)
54

```

```

55
56 if __name__ == '__main__': unittest.main(verbosity=2)

```

src/q3/test_parallel.py

```

1  """串并数值一致性、因果FD、FIV配对、独立SOC链和异常传播。"""
2  from dataclasses import replace
3  from pathlib import Path
4  import json
5  import tempfile
6  import unittest
7  from unittest.mock import patch
8  import numpy as np
9  import pandas as pd
10 from .config import ROOT, Config
11 from .parallel import process_pool, run_rollout_jobs, validate_parallelism
12 from .optimization import LimitedSolve, solve
13 from .physics import Policy
14 from .scenarios import Scenarios
15 from .data import read_inputs
16 from .terminal import TerminalValues
17 from .rolling import fiv, run_period
18
19
20 def raise_limited(_: int) -> None:
21     raise LimitedSolve({'status': 'deliberate_test_limit', 'reliable': False})
22
23
24 class ParallelMathTests(unittest.TestCase):
25     def test_cpu_budget_rejects_oversubscription(self) -> None:
26         with self.assertRaises(ValueError): validate_parallelism(0, 1)
27         with self.assertRaises(ValueError): validate_parallelism(2, 0)
28         with self.assertRaises(ValueError): validate_parallelism(100000, 2)
29
30     def test_native_concurrent_exact_mapping(self) -> None:
31         load = np.array([[0., 12000., 400., 10000.], [6000., 300., 10000., 200.]])
32         pv = np.array([[3000., 0., 5000., 0.], [0., 6000., 0., 800.]])
33         scenes = Scenarios(load, pv, np.array([.4, .6]), {})
34         policy = Policy(np.array([200., 400., 200., 100.]), np.full(4, 100.), np.full(4, 200.))
35         results = [solve(scenes, np.ones(4), 1250., Config(seconds=15, solver_threads=t),
36                         fixed=policy, terminal=.2) for t in (1, 2)]
37         self.assertEqual(results[1].audit['solve_method'], 'SCIP_solveConcurrent')
38         self.assertTrue(all(r.audit['reliable'] for r in results))
39         self.assertAlmostEqual(results[0].audit['objective'], results[1].audit['objective'], places
40                                =6)
41         for left, right in zip(results[0].responses, results[1].responses):
42             for key in left: np.testing.assert_allclose(left[key], right[key], atol=1e-6)
43
44     def test_process_exception_preserves_original_audit(self) -> None:
45         with process_pool(2) as executor:
46             with self.assertRaises(LimitedSolve) as result:
47                 list(executor.map(raise_limited, [1]))
48             self.assertEqual(result.exception.audit['status'], 'deliberate_test_limit')
49
50     def test_duplicate_checkpoint_paths_rejected(self) -> None:
51         with tempfile.TemporaryDirectory() as folder:

```

```

51     path = Path(folder)
52     with self.assertRaises(ValueError):
53         run_rollout_jobs([{'output': path}, {'output': path}], 2, path/'status.json')
54
55
56 class ParallelRealTests(unittest.TestCase):
57     @classmethod
58     def setUpClass(cls) -> None:
59         cls.data = read_inputs(ROOT/'outputs/q2/dispatch_runs/20260911_224501')
60
61     def test_fd_causal_batch_and_fiv_pair_equal(self) -> None:
62         config = Config(seconds=15, deterministic=True)
63         sequential = TerminalValues(self.data, config)
64         expected, expected_audit = sequential.value(31, 6)
65         reference = np.ones(108)*500
66         old, old_branches = fiv(self.data, sequential, 31, 6, 6000., reference, config)
67         with process_pool(2) as executor:
68             concurrent = TerminalValues(self.data, config, executor)
69             actual, actual_audit = concurrent.value(31, 6)
70             new, new_branches = fiv(self.data, concurrent, 31, 6, 6000., reference, config)
71             self.assertEqual(expected_audit, actual_audit)
72             self.assertAlmostEqual(expected, actual, places=10)
73             self.assertTrue(all(row['complete_at_slot'] <= 31*144 for row in concurrent.cache.values()))
74             self.assertAlmostEqual(old['FIV_block'], new['FIV_block'], places=6)
75             self.assertEqual([r['branch'] for r in new_branches], ['stale', 'updated'])
76             for left, right in zip(old_branches, new_branches):
77                 np.testing.assert_allclose(left['real_cost'], right['real_cost'], atol=1e-6)
78                 np.testing.assert_allclose(left['soc'], right['soc'], atol=1e-6)
79
80     def test_parallel_independent_soc_chains_equal(self) -> None:
81         config = Config(seconds=15, deterministic=True)
82         with tempfile.TemporaryDirectory() as folder:
83             root = Path(folder)
84             jobs = [dict(data=self.data, config=replace(config, nodes=nodes), output=root/f'parallel
85 {index}',
86                         end=9, with_fiv=False) for index, nodes in enumerate(((0,), (0, 6, 12, 18))
87 )]
88
89             results = run_rollout_jobs(jobs, 2, root/'jobs.json')
90             self.assertTrue(all(row['status'] == 'complete' for row in results))
91             self.assertEqual(len({row['pid'] for row in results}), 2)
92             for index, job in enumerate(jobs):
93                 path = root/f'serial{index}'
94                 run_period(self.data, job['config'], path, end=9, with_fiv=False, workers=2 if index
95 == 0 else 1)
96                 expected = pd.read_csv(path/'warmup.csv')
97                 actual = pd.read_csv(job['output']/'warmup.csv')
98                 pd.testing.assert_frame_equal(expected, actual, atol=1e-7, rtol=0)
99                 manifest = json.loads((root/'jobs.json').read_text(encoding='utf-8'))
100                 self.assertFalse(manifest['nested_process_pools'])
101
102     def test_parallel_failures_do_not_publish_complete_comparison(self) -> None:
103         with tempfile.TemporaryDirectory() as folder:
104             root = Path(folder)
105             # 故意给空日期范围，验证worker错误记录和父进程拒绝完整比较。
106             jobs = [dict(data=self.data, config=Config(), output=root/'bad', end=0, with_fiv=False)]
107             with self.assertRaises(RuntimeError): run_rollout_jobs(jobs, 2, root/'jobs.json')
108             value = json.loads((root/'jobs.json').read_text(encoding='utf-8'))

```

```

105         self.assertEqual(value['jobs'][0]['status'], 'failed')
106
107     def test_all_required_experiment_chains_are_dispatched(self) -> None:
108         from .analysis import run_experiments
109         with tempfile.TemporaryDirectory() as folder:
110             with patch('src.q3.parallel.run_rollout_jobs', side_effect=RuntimeError('
stop_after_dispatch')) as dispatcher:
111                 with self.assertRaisesRegex(RuntimeError, 'stop_after_dispatch'):
112                     run_experiments(self.data, Config(), Path(folder), pd.DataFrame(), workers=4)
113             jobs, workers, _ = dispatcher.call_args.args
114             self.assertEqual(workers, 4)
115             self.assertEqual(len(jobs), 40) # 9×4敏感性链+3个额外OUV链+M0; 已有主链不重复。
116             self.assertEqual(len({str(job['output']) for job in jobs}), 40)
117             self.assertEqual(sum(job['config'].deterministic for job in jobs), 1)
118             self.assertEqual(sum(job['with_fiv'] for job in jobs), 9)
119
120     def test_stress_day_process_preserves_settlement_values(self) -> None:
121         from .analysis import settlement_job
122         config = Config(seconds=15, deterministic=True)
123         with tempfile.TemporaryDirectory() as folder:
124             root = Path(folder)
125             sequential = settlement_job((self.data, config, root/'serial', 31, 6000., 0.))
126             with process_pool(2) as executor:
127                 concurrent = list(executor.map(settlement_job, [(self.data, config, root/'parallel',
31, 6000., 0.)])[0]
128             pd.testing.assert_frame_equal(pd.DataFrame(sequential), pd.DataFrame(concurrent), atol=1
e-6, rtol=0)
129
130
131 if __name__ == '__main__':
132     unittest.main(verbosity=2)

```

src/q3/test_production_rescue.py

```

1  """七小时生产救援：节点硬墙、可行门禁、预算映射和恢复。"""
2  import tempfile
3  import time
4  import unittest
5  from pathlib import Path
6  from unittest.mock import patch
7  import numpy as np
8  from dataclasses import replace
9  from .config import Config, CAP, validate_production_rescue
10 from .data import Inputs
11 from .optimization import Solution, LimitedSolve, accept_feasible_incumbent, objective,
reached_hard_limit, solve
12 from .checkpoint import request_digest, save_snapshot
13 from .physics import Policy, replay
14 from .rolling import run_day
15 from .scenarios import Scenarios
16
17
18 def small_case() -> tuple[Scenarios, np.ndarray, Policy]:
19     load=np.array([[6000.,8000.,3000.,9000.],[7000.,4000.,8500.,2000.]])
20     pv=np.array([[1000.,0.,4000.,0.],[0.,3000.,0.,3500.]])
21     scenes=Scenarios(load,pv,np.array([.4,.6]),{})

```



```

22 policy=Policy(np.array([500.,600.,300.,700.]),np.full(4,CAP),np.full(4,CAP))
23 return scenes,np.array([.5,.8,1.1,.6]),policy
24
25
26 class ProductionRescueTests(unittest.TestCase):
27     def test_valid_incumbent_accepted_without_faking_certificate(self) -> None:
28         scenes,prices,policy=small_case();responses=[replay(policy,path,6000.) for path in scenes.
29 net]
30         value=objective(policy,responses,scenes.weights,prices,None,4,.6)
31         result=accept_feasible_incumbent(policy,scenes,prices,6000.,Config(production_rescue=True,
32 formulation='legacy'),
33         reference=None,today=4,terminal=.6,previous_net=0.,audit={'objective':value,'lower_bound
34 ':value-100.,'gap':1.},
35         hard_seconds=8.,source='test')
36         self.assertTrue(result.audit['accepted']);self.assertFalse(result.audit['reliable'])
37         self.assertFalse(result.audit['certificate_met']);self.assertEqual(result.audit['accepted_by
38 ','hard_limit_feasible'])
39         self.assertAlmostEqual(result.audit['gap'],100/abs(value))
40
41     def test_invalid_or_missing_incumbent_is_rejected(self) -> None:
42         scenes,prices,policy=small_case();config=Config(production_rescue=True,formulation='legacy')
43         with self.assertRaises(LimitedSolve):
44             accept_feasible_incumbent(None,scenes,prices,6000.,config,reference=None,today=4,
45 terminal=.6,
46             previous_net=0.,audit={},hard_seconds=8.,source='test')
47         with self.assertRaises(LimitedSolve):
48             accept_feasible_incumbent(policy,scenes,prices,6000.,config,reference=None,today=4,
49 terminal=.6,
50             previous_net=0.,audit={'objective':0.,'lower_bound':None},
51             hard_seconds=8.,source='test')
52         bad=Policy(policy.grid.copy(),policy.charge_cap.copy(),policy.discharge_cap.copy());bad.grid
53 [0]=-1.
54         with self.assertRaises(LimitedSolve):
55             accept_feasible_incumbent(bad,scenes,prices,6000.,config,reference=None,today=4,terminal
56 =.6,
57             previous_net=0.,audit={'objective':0.},hard_seconds=8.,source=
58 'test')
59
60     def test_actual_hard_wall_returns_valid_policy(self) -> None:
61         scenes,prices,_=small_case();config=Config(production_rescue=True,formulation='legacy',
62 gap=1e-9,solver_threads=1,hard_06_seconds=.3,hard_other_seconds=.2)
63         started=time.perf_counter();result=solve(scenes,prices,6000.,config,terminal=.6,hard_seconds
64 =.3)
65         self.assertTrue(result.audit['accepted']);self.assertIn(result.audit['accepted_by'],('
66 certified_3pct','hard_limit_feasible'))
67         self.assertLess(time.perf_counter()-started,3.)
68         if 'solver_time_limit' in result.audit:self.assertLessEqual(result.audit['solver_time_limit'
69 ],.3)
70         for response in result.responses:self.assertEqual(len(response['soc']),4)
71
72     def test_restored_hard_limit_node_skips_optimizer(self) -> None:
73         scenes,prices,_=small_case();config=Config(production_rescue=True,formulation='legacy',gap=1
74 e-9,
75             hard_06_seconds=.2,hard_other_seconds=.2)
76         with tempfile.TemporaryDirectory() as folder:
77             checkpoint=Path(folder);policy=small_case()[2]
78             responses=[replay(policy,path,6000.) for path in scenes.net]

```

```

65         value=objective(policy, responses, scenes.weights, prices, None, 4, .6)
66         accepted=accept_feasible_incumbent(policy, scenes, prices, 6000., config, reference=None,
today=4,
67             terminal=.6, previous_net=0., audit={'objective': value, 'lower_bound': value-100.},
68             hard_seconds=.2, source='initial_test')
69         signature=request_digest(config, scenes.load, scenes.pv, scenes.weights, prices, 6000., None
, 4, .6, 0., None, None, None)
70         save_snapshot(checkpoint/'latest.json', signature, accepted.policy, accepted.audit)
71         with patch('src.q3.optimization.make_model', side_effect=AssertionError('不得重新求解已接受
节点')):
72             second=solve(scenes, prices, 6000., config, terminal=.6, hard_seconds=.2,
73                 checkpoint=checkpoint, max_slices=None)
74             self.assertTrue(second.audit['accepted']); self.assertEqual(second.audit['
acceptance_source'], 'restored_rescue_checkpoint')
75
76     def test_run_day_maps_35_and_8_second_limits(self) -> None:
77         actual=np.zeros((365, 144, 2)); prices=np.ones(144)
78         data=Inputs(actual, np.zeros((365, 4, 24)), prices, {}, [], {})
79         scenes=Scenarios(np.zeros((1, 144)), np.zeros((1, 144)), np.ones(1), {})
80         calls=[]
81         def fake_solve(*args, **kwargs):
82             calls.append(kwargs['hard_seconds']); policy=Policy(np.zeros(144), np.zeros(144), np.zeros
(144))
83             return Solution(policy, [replay(policy, np.zeros(144), args[2])],
84                 {'accepted': True, 'reliable': False, 'accepted_by': 'hard_limit_feasible', '
gap': None})
85         terminal=type('Terminal', (), {'value': lambda self, day, hour: (.5, {})})( )
86         config=Config(production_rescue=True, formulation='legacy', hard_06_seconds=35.,
hard_other_seconds=8.)
87         with patch('src.q3.rolling.construct', return_value=scenes), patch('src.q3.rolling.solve',
side_effect=fake_solve):
88             frame, audit, _=run_day(data, terminal, 7, 6000., config)
89             self.assertEqual(calls, [8., 35., 8., 8.]); self.assertEqual(len(frame), 144); self.assertEqual(len
(audit['nodes']), 4)
90
91     def test_rescue_config_freezes_model_invariants(self) -> None:
92         config=Config(production_rescue=True, formulation='legacy', solver_threads=4, gap=.03,
93             hard_06_seconds=35., hard_other_seconds=8.)
94         validate_production_rescue(config)
95         self.assertEqual((config.scenarios, config.tail, config.nodes), (20, 2, (0, 6, 12, 18)))
96         self.assertEqual(config.gap, .03); self.assertEqual(config.formulation, 'legacy')
97         with self.assertRaises(ValueError): validate_production_rescue(replace(config, scenarios=10))
98
99     def test_only_real_timeout_can_accept_uncertified_policy(self) -> None:
100         self.assertTrue(reached_hard_limit({'status': 'Time limit reached'}, .2, 35.))
101         self.assertTrue(reached_hard_limit({'status': 'searching'}, 34.96, 35.))
102         self.assertFalse(reached_hard_limit({'status': 'searching'}, 34.4, 35.))
103         self.assertFalse(reached_hard_limit({'status': 'Interrupted by error'}, 2., 35.))
104
105
106 if __name__ == '__main__': unittest.main(verbosity=2)

```

src/q3/test_q3.py

```

1 """需求导向核：物理恒等式、精确min、未来哨兵、结算、树非前瞻、模板。"""
2 from dataclasses import replace

```

```

3 import json
4 from pathlib import Path
5 import tempfile
6 import unittest
7 import numpy as np
8 import pandas as pd
9 from .config import Config, ROOT, CAP, E_MIN, E_MAX, E_INITIAL
10 from .data import read_inputs
11 from .physics import Policy, replay, adjustment, no_storage_decision
12 from .scenarios import Scenarios, construct, medoids, reduce_errors, pool_indices
13 from .optimization import solve, LimitedSolve
14 from .terminal import TerminalValues
15 from .rolling import run_day, run_period, fiv, validate_frame, previous_net
16 from .multistage import conditional_tree, solve_tree, tree_inputs
17 from .export import export_workbook, summaries, emergency_events
18
19
20 class MathTests(unittest.TestCase):
21     def test_incumbent_gradient_matches_finite_difference(self) -> None:
22         from .incumbent import value_gradient
23         rng = np.random.default_rng(2)
24         net = rng.uniform(-500, 1500, (4, 8)); prices = rng.uniform(.4, 1., 8); weights = np.full(4,
25         .25)
26         vector = rng.uniform(20, 700, 24); reference = rng.uniform(200, 500, 4)
27         value, gradient = value_gradient(vector, net, prices, weights, 6000., reference, 4, .6)
28         for i in range(24):
29             positive = vector.copy(); positive[i] += 1e-4
30             negative = vector.copy(); negative[i] -= 1e-4
31             numerical = (value_gradient(positive, net, prices, weights, 6000., reference, 4, .6)[0] -
32             value_gradient(negative, net, prices, weights, 6000., reference, 4, .6)[0])/2e-4
33             self.assertAlmostEqual(gradient[i], numerical, places=5)
34
35     def test_energy_balance_saturation(self) -> None:
36         rng = np.random.default_rng(87)
37         for initial in (E_MIN, E_INITIAL, E_MAX):
38             for mode in ('aggregate', 'lag'):
39                 policy = Policy(rng.uniform(0, 1500, 144), rng.uniform(0, CAP, 144), rng.uniform(0,
40                 CAP, 144))
41                 result = replay(policy, rng.uniform(-1000, 2000, 144), initial, mode=mode,
42                 previous_net=10.)
43                 self.assertTrue(np.all(np.minimum(result['charge'], result['discharge']) == 0))
44
45     def test_lag_does_not_observe_current(self) -> None:
46         policy = Policy(np.array([0., 0.]), np.full(2, CAP), np.full(2, CAP))
47         a = replay(policy, np.array([200., -200.]), 6000., mode='lag', previous_net=-100.)
48         b = replay(policy, np.array([500., -900.]), 6000., mode='lag', previous_net=-100.)
49         self.assertEqual(a['charge'][0], b['charge'][0])
50         self.assertEqual(a['discharge'][0], 0)
51         self.assertGreater(a['emergency'][0], 200) # 上一负荷触发充电, 当期缺口由紧急购电同时平衡。
52
53     def test_settlement_cancellation_and_repeated_proxy(self) -> None:
54         g0 = np.array([100., 100., 100.]); p = np.ones(3)
55         effective = np.array([80., 100., 140.])
56         np.testing.assert_allclose(adjustment(effective, g0, p), [-10., 0., 60.])
57         old = np.array([200., 50., 100.])
58         final = adjustment(effective, g0, p)
59         sequential = adjustment(old, g0, p)+adjustment(effective, old, p)

```

```

56         self.assertFalse(np.allclose(final, sequential))
57
58     def test_no_storage_deadband(self) -> None:
59         demand, weights = np.arange(1., 101.), np.full(100, .01)
60         self.assertEqual(no_storage_decision(demand, 80., weights), 80.)
61         self.assertIn(no_storage_decision(demand, 1., weights), (70., 71.))
62         self.assertIn(no_storage_decision(demand, 100., weights), (90., 91.))
63
64     def test_reduction_probability_tail_original_support(self) -> None:
65         errors = np.random.default_rng(1).normal(size=(30, 12))
66         errors[29] += 40; errors[28] += 20
67         indices, weights, audit = reduce_errors(errors, np.ones(12), Config(scenarios=10, tail=2))
68         self.assertTrue({28, 29}.issubset(indices)); self.assertEqual(len(indices), 10)
69         self.assertEqual(len(np.unique(indices)), 10)
70         self.assertAlmostEqual(weights.sum(), 1.)
71         self.assertTrue((weights > 0).all())
72         self.assertEqual(sum(audit['represented_counts']), 30)
73
74     def test_no_artificial_samples(self) -> None:
75         indices, weights, _ = reduce_errors(np.ones((3, 12)), np.ones(12), Config())
76         np.testing.assert_array_equal(indices, [0, 1, 2]); np.testing.assert_allclose(weights,
77 [1/3]*3)
78
79     def test_recent_then_expand_and_condition(self) -> None:
80         days = np.arange(80)
81         indices, audit = pool_indices(days, np.arange(80.), 100, 79., Config())
82         self.assertEqual(audit['window'], 56)
83         self.assertTrue((days[indices] >= 44).all())
84         self.assertGreaterEqual(len(indices), 20)
85
86     def test_exact_milp_matches_fixed_min(self) -> None:
87         # 同时覆盖不同净缺口符号、SOC容量约束和每种响应上限：不可用自由场景recourse蒙混。
88         load = np.array([[0., 12000., 400., 10000.], [6000., 300., 10000., 200.]])
89         pv = np.array([[3000., 0., 5000., 0.], [0., 6000., 0., 800.]])
90         scene = Scenarios(load, pv, np.array([.4, .6]), {})
91         fixed = Policy(np.array([200., 400., 200., 100.]), np.array([100., 500., 300., 700.]), np.
92 array([700., 200., 800., 50.]))
93         for mode in ('aggregate', 'lag'):
94             solved = solve(scene, np.array([.5, .8, 1., .3]), 1250., Config(seconds=15, feedback=
95 mode),
96 fixed=fixed, reference=np.full(4, 300.), today=2, terminal=.2,
97 previous_net=-100.)
98             self.assertLessEqual(solved.audit['mapping_error'], 1e-5)
99             self.assertTrue(solved.audit['reliable'])
100
101     def test_solution_objective_uses_residual_not_real_cost(self) -> None:
102         scene = Scenarios(np.full((1, 8), 1000.), np.zeros((1, 8)), np.ones(1), {})
103         result = solve(scene, np.ones(8), E_INITIAL, Config(seconds=5), terminal=.5)
104         real_cost = np.sum(result.policy.grid*5*result.responses[0]['emergency'])
105         self.assertAlmostEqual(result.audit['objective'], real_cost-.5*result.responses[0]['soc'
106 ][-1], places=5)
107
108     def test_tree_conditioning_and_uncertainty(self) -> None:
109         revisions = np.random.default_rng(4).normal(size=(24, 3, 18))
110         labels, nodes = conditional_tree(revisions)
111         self.assertTrue((labels[:, 0] == 0).all())
112         for node in nodes[1:]:

```

```

108         parent = nodes[node['parent']]
109         self.assertTrue(set(node['members']).issubset(parent['members']))
110         self.assertGreaterEqual(len(node['members']), 2)
111         self.assertLessEqual(len([x for x in nodes if x['parent'] == node['parent']], 2)
112         for stage in range(4):
113             self.assertAlmostEqual(sum(node['probability'] for node in nodes if node['stage'] ==
stage), 1.)
114
115     def test_actual_small_multistage_solve(self) -> None:
116         revisions = np.random.default_rng(3).normal(size=(8, 3, 3))
117         labels, nodes = conditional_tree(revisions)
118         net = np.random.default_rng(5).uniform(10, 100, size=(8, 8))
119         result = solve_tree(net, labels, nodes, net.max(0)+CAP, np.ones(8), 6000., .8, Config(
seconds=20))
120         self.assertTrue(result['audit']['reliable'])
121         self.assertLessEqual(result['audit']['mapping_error'], 1e-5)
122
123
124     class RealDataTests(unittest.TestCase):
125         @classmethod
126         def setUpClass(cls) -> None:
127             cls.data = read_inputs(ROOT/'outputs/q2/dispatch_runs/20260911_224501')
128
129         def test_origin_alignment_and_nodes(self) -> None:
130             for hour in (0, 6, 12, 18):
131                 p = self.data.pv(31, hour)
132                 np.testing.assert_allclose(p[5::6], self.data.hourly[31, hour//6])
133                 self.assertAlmostEqual(p[0], 5/6*self.data.observation(31, hour)+self.data.hourly[31,
hour//6, 0]/6)
134                 with self.assertRaises(ValueError):
135                     self.data.observation(0, 0)
136
137         def test_48h_predictor_identity_frozen(self) -> None:
138             before = self.data.load(31)
139             for hour in (0, 6, 12, 18):
140                 scenes = construct(self.data, 31, hour, Config())
141                 self.assertEqual(scenes.audit['predictor'], self.data.selected[31])
142                 np.testing.assert_array_equal(self.data.load(31), before)
143                 self.assertEqual(len(before), 288)
144
145         def test_future_poison_not_used_by_node(self) -> None:
146             original = self.data.actual.copy()
147             before = construct(self.data, 70, 12, Config())
148             try:
149                 self.data.actual.reshape(-1, 2)[70*144+72:] = 1e10
150                 after = construct(self.data, 70, 12, Config())
151                 np.testing.assert_array_equal(before.net, after.net)
152                 self.assertEqual(before.audit, after.audit)
153             finally:
154                 self.data.actual[:] = original
155
156         def test_completed_trajectory_horizon(self) -> None:
157             for hour in (0, 6, 12, 18):
158                 scenes = construct(self.data, 31, hour, Config())
159                 for j in scenes.audit['history_days']:
160                     self.assertLess(j, 31)
161                     self.assertLessEqual(j*144+hour*6+144, 31*144+hour*6)

```

```

162         self.assertEqual(len(scenes.weights), 20)
163
164     def test_pchip_preserves_hourly_knots(self) -> None:
165         np.testing.assert_allclose(self.data.pv(171, 6, 'pchip')[5::6], self.data.hourly[171, 1])
166         self.assertTrue((self.data.pv(171, 6, 'pchip') >= 0).all())
167
168     def test_terminal_causal_and_delta_stability(self) -> None:
169         value = TerminalValues(self.data, Config(seconds=10))
170         coefficient, audit = value.value(8, 0)
171         self.assertGreater(coefficient, 0)
172         self.assertTrue(all(row['complete_at_slot'] <= 8*144 for row in value.cache.values()))
173         rows = value.stability(6, 0)
174         self.assertTrue(all(row['value'] >= 0 for row in rows))
175
176     def test_real_day_rollout_and_template(self) -> None:
177         config = Config(seconds=15, deterministic=True)
178         terminal = TerminalValues(self.data, config)
179         first, audit, _ = run_day(self.data, terminal, 7, E_INITIAL, config)
180         second, _, _ = run_day(self.data, terminal, 8, float(first.soc.iloc[-1]), config)
181         frame = pd.concat([first, second], ignore_index=True)
182         validate_frame(frame, config)
183         self.assertEqual([node['hour'] for node in audit['nodes']], [0, 6, 12, 18])
184         self.assertTrue(all(len(node['grid']) == 144 for node in audit['nodes']))
185         with tempfile.TemporaryDirectory() as folder:
186             result = export_workbook(frame, Path(folder), config, smoke=True)
187             self.assertTrue(result['roundtrip_all_sheets'])
188             with self.assertRaises(ValueError):
189                 export_workbook(frame, Path(folder), config)
190             # 零紧急事件是合法结果，空事件表仍须可回填与回读。
191             net = first.net_kwh.to_numpy()
192             first['grid'] = first['g0'] = np.maximum(net, 0)
193             first['charge_cap'] = first['discharge_cap'] = 0.
194             recovered = replay(Policy(first.grid.to_numpy(), np.zeros(144), np.zeros(144)), net,
195                                E_INITIAL)
196             for key, values in recovered.items():
197                 first[key] = values
198             first['planned_cost'] = first.price*first.g0
199             first['adjustment_cost'] = first['emergency_cost'] = 0.
200             first['total_cost'] = first.planned_cost
201             self.assertEqual(len(emergency_events(first)), 0)
202             self.assertTrue(export_workbook(first, Path(folder), config, smoke=True)['
203                                roundtrip_all_sheets'])
204
205     def test_cold_start_no_future_or_soc_reset(self) -> None:
206         config = Config()
207         terminal = TerminalValues(self.data, config)
208         frame, audit, _ = run_day(self.data, terminal, 0, E_INITIAL, config)
209         self.assertTrue(audit['cold_start'])
210         np.testing.assert_array_equal(frame.soc, np.full(144, E_INITIAL))
211         self.assertEqual(frame.charge.sum()+frame.discharge.sum()+frame.g0.sum(), 0.)
212
213     def test_fiv_common_support_and_state_independence(self) -> None:
214         config = Config(seconds=15, deterministic=True)
215         terminal = TerminalValues(self.data, config)
216         reference = np.ones(108)*500
217         row, records = fiv(self.data, terminal, 31, 6, 6000., reference, config)
218         self.assertEqual(row['audits'][0]['scenarios']['length'], 108)

```

```

217     self.assertEqual(row['audits'][1]['scenarios']['length'], 108)
218     self.assertEqual(row['initial_soc'], 6000.)
219     self.assertAlmostEqual(row['FIV_block'], sum(records[0]['real_cost'][:36]) - sum(records[1]['
real_cost'][:36]))
220     np.testing.assert_array_equal(reference, np.ones(108)*500)
221
222     def test_resume_keeps_prefix_and_rejects_tamper(self) -> None:
223         config = Config(seconds=15, deterministic=True)
224         with tempfile.TemporaryDirectory() as folder:
225             root = Path(folder)
226             run_period(self.data, config, root, end=9, with_fiv=False)
227             before = (root/'days/2025-01-08.csv').read_bytes()
228             run_period(self.data, config, root, end=9, with_fiv=False)
229             self.assertEqual(before, (root/'days/2025-01-08.csv').read_bytes())
230             (root/'audit/2025-01-08.json').write_text('{}', encoding='utf-8')
231             with self.assertRaises(ValueError):
232                 run_period(self.data, config, root, end=9, with_fiv=False)
233
234     def test_sequential_reoptimization_and_ouv_sets(self) -> None:
235         for settlement in ('final', 'sequential'):
236             for nodes in ((0,), (0, 6), (0, 6, 12), (0, 6, 12, 18)):
237                 config = Config(seconds=15, deterministic=True, nodes=nodes, settlement=settlement)
238                 frame, audit, _ = run_day(self.data, TerminalValues(self.data, config), 8, 6000.,
config)
239                 self.assertEqual(len(audit['nodes']), len(nodes))
240                 self.assertEqual(sum(node['committed_slots'] for node in audit['nodes']), 144)
241                 validate_frame(frame, config)
242                 if settlement == 'sequential':
243                     version = frame.g0.to_numpy().copy(); cost = np.zeros(144)
244                     for node in audit['nodes'][1:]:
245                         start = node['hour']*6; current = np.array(node['grid'][:144-start])
246                         cost[start:] += adjustment(current, version[start:], self.data.prices[start
:]))
247                     version[start:] = current
248                     np.testing.assert_allclose(frame.adjustment_cost, cost, atol=1e-5)
249
250
251 if __name__ == '__main__':
252     unittest.main(verbosity=2)

```

src/q3/test_solver_acceleration.py

```

1  """v2 精确列消元、可达界和矩阵摘要回归。"""
2  import unittest
3  import numpy as np
4  from .config import Config, CAP, E_MIN, E_MAX, TOL
5  from .fast_solver import (build_matrix, build_matrix_v2_free, build_matrix_v2_2bit, matrix_digest,
matrix_error,
6                           reachable_bounds, seed_vector, solve_fast)
7  from .optimization import solve
8  from .physics import Policy, replay
9  from .scenarios import Scenarios
10
11
12 class SolverAccelerationTests(unittest.TestCase):
13     def test_v2_gap_zero_matches_legacy_on_random_cases(self) -> None:

```

```

14     rng=np.random.default_rng(9301)
15     for case in range(36):
16         s=2+case%3;t=3+case%4
17         scenes=Scenarios(rng.uniform(0,11000,(s,t)),rng.uniform(0,8000,(s,t)),np.full(s,1/s),{})
18         prices=rng.uniform(.35,1.5,t);initial=(E_MIN,6000.,E_MAX)[case%3]
19         today=max(1,t-2);reference=rng.uniform(0,800,today);terminal=float(rng.uniform(0,1.4))
20         cfg=Config(gap=0.,seconds=20.)
21         _,_,legacy=solve_fast(scenes,prices,initial,cfg,reference=reference,today=today,
22                               terminal=terminal,formulation='legacy')
23         _,_,v2=solve_fast(scenes,prices,initial,cfg,reference=reference,today=today,
24                           terminal=terminal,formulation='v2_free')
25         self.assertTrue(legacy['reliable']);self.assertTrue(v2['reliable'])
26         self.assertAlmostEqual(legacy['objective'],v2['objective'],places=5)
27
28     def test_v2_removes_only_fixed_columns_and_keeps_binary_count(self) -> None:
29         rng=np.random.default_rng(44);s,t=5,12
30         scenes=Scenarios(rng.uniform(0,10000,(s,t)),rng.uniform(0,7000,(s,t)),np.full(s,1/s),{})
31         old=build_matrix(scenes,np.ones(t),6000.);new=build_matrix_v2_free(scenes,np.ones(t),6000.)
32         self.assertEqual(len(old[0])-len(new[0]),t+4*s*t)
33         self.assertEqual(int(old[2].sum()),int(new[2].sum()))
34         # 删除 c<=cp 的 ST 行; z 等式由 selector 上界行机械替代, 故其行数净变化为0。
35         self.assertEqual(old[3].A.shape[0]-new[3].A.shape[0],s*t)
36         self.assertEqual(len(new[4]['cp']),0);self.assertEqual(len(new[4]['z']),0);self.assertEqual(
37             new[4]['lc'],())
38
39     def test_reachable_bounds_contain_many_original_replays(self) -> None:
40         rng=np.random.default_rng(208);checked=0
41         for initial in (E_MIN,6000.,E_MAX):
42             for _ in range(80):
43                 net=rng.uniform(-1200,1800,(4,144));upper=np.maximum(net.max(0),0)+CAP
44                 lo,hi,cu,ru=reachable_bounds(net,upper,initial)
45                 for _ in range(4):
46                     policy=Policy(rng.uniform(0,1,144)*upper,np.full(144,CAP),rng.uniform(0,CAP,144))
47                     result=replay(policy,path,initial)
48                     self.assertTrue((result['soc']>=lo[j]-TOL).all());self.assertTrue((result['soc']<=hi[j]+TOL).all())
49                     self.assertTrue((result['charge']<=cu[j]+TOL).all());self.assertTrue((result['discharge']<=ru[j]+TOL).all())
50                     checked+=144
51                 self.assertEqual(checked,552960)
52
53     def test_seed_and_digest_are_stable_for_edge_cases(self) -> None:
54         net=np.array([[0.,100.,-100.,CAP],[CAP,0.,-CAP,100.]])
55         load=np.maximum(net,0)*6;pv=np.maximum(-net,0)*6;scenes=Scenarios(load,pv,np.array([.4,.6]),{})
56         for initial in (E_MIN,E_MAX):
57             matrix=build_matrix_v2_free(scenes,np.ones(4),initial)
58             f,b,integer,c,indices=matrix;upper=np.maximum(net.max(0),0)+CAP
59             policy=Policy(upper.copy(),np.full(4,CAP),np.full(4,CAP))
60             vector=seed_vector(policy,scenes,np.ones(4),initial,indices,len(f),None)
61             self.assertLessEqual(max(matrix_error(vector,b,integer,c)),TOL)
62             self.assertEqual(matrix_digest(*matrix[:4]),matrix_digest(*matrix[:4]))
63
64     def test_original_exact_small_problem_matches_v2(self) -> None:
65         rng=np.random.default_rng(99);scenes=Scenarios(rng.uniform(0,9000,(3,5)),rng.uniform

```



```

(0,6000,(3,5)),np.array([.2,.3,.5]),{ })
66     prices=rng.uniform(.4,1.3,5);reference=np.full(3,400.);cfg=Config(gap=0.,seconds=20.)
67     exact=solve(scenes,prices,6000.,cfg,reference=reference,today=3,terminal=.5)
68     _,v2=solve_fast(scenes,prices,6000.,cfg,reference=reference,today=3,terminal=.5,
formulation='v2_free')
69     self.assertAlmostEqual(exact.audit['objective'],v2['objective'],places=5)
70
71     def test_two_bit_gap_zero_matches_three_selector_v2(self) -> None:
72         rng=np.random.default_rng(772)
73         for case in range(36):
74             s=2+case%3;t=3+case%4
75             scenes=Scenarios(rng.uniform(0,10500,(s,t)),rng.uniform(0,7500,(s,t)),np.full(s,1/s),{ })
76             prices=rng.uniform(.35,1.45,t);initial=(E_MIN,6000.,E_MAX)[case%3]
77             today=max(1,t-2);reference=rng.uniform(0,700,today);terminal=float(rng.uniform(0,1.3));
cfg=Config(gap=0.,seconds=20.)
78             _,three=solve_fast(scenes,prices,initial,cfg,reference=reference,today=today,
79                               terminal=terminal,formulation='v2_free')
80             _,two=solve_fast(scenes,prices,initial,cfg,reference=reference,today=today,
81                              terminal=terminal,formulation='v2_2bit')
82             self.assertTrue(three['reliable']);self.assertTrue(two['reliable'])
83             self.assertAlmostEqual(three['objective'],two['objective'],places=5)
84
85     def test_two_bit_reduces_binary_count_and_accepts_exact_seed(self) -> None:
86         rng=np.random.default_rng(663);s,t=5,12
87         scenes=Scenarios(rng.uniform(0,10000,(s,t)),rng.uniform(0,7000,(s,t)),np.full(s,1/s),{ })
88         three=build_matrix_v2_free(scenes,np.ones(t),6000.);two=build_matrix_v2_2bit(scenes,np.ones(
t),6000.)
89         self.assertEqual(int(three[2].sum())-int(two[2].sum()),s*t)
90         self.assertEqual(len(three[0])-len(two[0]),s*t)
91         upper=np.maximum(scenes.net.max(0),0)+CAP
92         policy=Policy(rng.uniform(0,1,t)*upper,np.full(t,CAP),rng.uniform(0,CAP,t))
93         vector=seed_vector(policy,scenes,np.ones(t),6000.,two[4],len(two[0]),None)
94         self.assertLessEqual(max(matrix_error(vector,*two[1:4])),TOL)
95
96
97 if __name__=='__main__':unittest.main(verbosity=2)

```

src/q3/test_validation.py

```

1  """最终检验方案范围、优先级、真实反事实及原始主结果只读核验。"""
2  import hashlib
3  import json
4  from pathlib import Path
5  import tempfile
6  import unittest
7  import numpy as np
8  import pandas as pd
9  from .config import ROOT, Config, write_json, write_csv
10 from .data import read_inputs
11 from .rolling import run_day, date_of
12 from .terminal import TerminalValues
13 from .validation import variant_configs, plan_jobs, base_fiv_job, rollout_job, metric_tables,
audit_main
14
15
16 class ValidationTests(unittest.TestCase):

```

```

17 @classmethod
18 def setUpClass(cls) -> None:
19     cls.data=read_inputs(ROOT/'outputs/q2/dispatch_runs/20260911_224501')
20
21 def test_six_core_settings_and_three_assumptions(self) -> None:
22     variants=variant_configs(Config(gap=.03))
23     self.assertEqual(set(variants),{'M1','S10','S30','tail0','tail4','terminal08','terminal12','
pchip','lag','sequential'})
24     self.assertEqual(variants['sequential'].settlement,'sequential')
25     self.assertEqual(variants['lag'].feedback,'lag')
26     self.assertTrue(all(cfg.gap==.03 for cfg in variants.values()))
27
28 def test_queue_reuses_M1_and_prioritizes_information(self) -> None:
29     with tempfile.TemporaryDirectory() as folder:
30         root=Path(folder);main=root/'main';date='2025-02-01'
31         write_csv(main/f'days/{date}.csv',pd.DataFrame({'initial_soc':[6000.]})
32         jobs=plan_jobs(self.data,Config(gap=.03),main,root/'validation',[date],'sampled')
33         self.assertEqual(len(jobs),40)
34         self.assertEqual([job[2].parts[-2:] for job in jobs[:4]], [('M1','K0'),('M1','K06'),('M1
','K0612'),('M0','K061218')])
35         self.assertEqual([job[2].parts[-2] for job in jobs[4:28]], {'S10','S30','tail0','tail4',
'terminal08','terminal12'})
36         self.assertEqual([job[2].parts[-2] for job in jobs[28:]], {'pchip','lag','sequential'})
37         self.assertFalse(any(job[2].parts[-2:]=='M1','K061218') for job in jobs))
38
39 def test_real_single_date_information_and_comparison_tables(self) -> None:
40     cfg=Config(gap=.03,deterministic=True,seconds=15.)
41     with tempfile.TemporaryDirectory() as folder:
42         root=Path(folder); main=root/'main'; output=root/'validation'; date='2025-02-01'; d=31
43         frame,audit,_=run_day(self.data,TerminalValues(self.data,cfg),d,6000.,cfg)
44         write_csv(main/'dispatch.csv',frame);write_csv(main/f'days/{date}.csv',frame);write_json
(main/f'audit/{date}.json',audit)
45         before=hashlib.sha256((main/'dispatch.csv').read_bytes()).hexdigest()
46         for job in plan_jobs(self.data,cfg,main,output,[date],'sampled')[:4]:rollout_job(job)
47         base_fiv_job((self.data,cfg,main,output/'base_fiv',[date],'sampled'))
48         errors=pd.read_csv(ROOT/'outputs/q3/raw/timing_final_20260912/diagnostics/
forecast_revision_pairs.csv')
49         write_csv(output/'forecast/forecast_revision_pairs.csv',errors)
50         metric_tables(self.data,main,output,{'M1':cfg},[date],'sampled')
51         table=pd.read_csv(output/'information_value.csv')
52         self.assertEqual(len(table),3);self.assertTrue((table.scope=='sampled').all())
53         self.assertTrue((table.days==1).all());self.assertTrue(table.FIV18_cont.iloc[2]==table.
FIV18_cont.iloc[2])
54         self.assertEqual(len(pd.read_csv(output/'M0_M1_comparison.csv')),2)
55         self.assertEqual(before,hashlib.sha256((main/'dispatch.csv').read_bytes()).hexdigest())
56
57 def test_empty_formal_prefix_does_not_claim_pass(self) -> None:
58     with tempfile.TemporaryDirectory() as folder:
59         root=Path(folder);write_json(root/'main/state.json',{'days':[]})
60         result=audit_main(self.data,root,root/'validation')
61         self.assertTrue(result.value.isna().all())
62         self.assertTrue((result.status=='待正式数据').all())
63
64
65 if __name__=='__main__':unittest.main(verbosity=2)

```

src/q3/traceability.py

```
1 """逐条保留模型源文与代码/验收对应；结构覆盖不冒充全年数值验证。"""
2 import ast
3 import hashlib
4 import re
5 from pathlib import Path
6 import pandas as pd
7 from .config import ROOT, SOURCE, write_csv, write_json
8
9 # source section -> concrete implementation symbols, acceptance mechanism.
10 MAPPING = {
11     '3.1': ('rolling.run_day;optimization.solve;analysis.run_experiments', '四节点真实回放和分层实验入口'),
12     '3.2': ('optimization.solve;multistage.solve_tree;analysis.run_experiments', 'test_actual_small_multistage_solve'),
13     '3.3': ('data.Inputs;terminal.TerminalValues;rolling.run_period', 'test_future_poison_not_used_by_node;test_cold_start_no_future_or_soc_reset;test_terminal_causal_and_delta_stability'),
14     '3.4': ('config.Config;physics.replay', 'test_energy_balance_saturation'),
15     '3.5': ('data.read_inputs', 'test_origin_alignment_and_nodes'),
16     '3.6': ('data.Inputs', 'test_origin_alignment_and_nodes;test_pchip_preserves_hourly_knots'),
17     '3.7': ('data.Inputs;data.read_inputs', 'test_48h_predictor_identity_frozen;test_completed_trajectory_horizon'),
18     '3.8': ('scenarios.construct;scenarios.pool_indices;scenarios.reduce_errors;scenarios.medoids', 'test_recent_then_expand_and_condition;test_reduction_probability_tail_original_support;test_no_artificial_samples'),
19     '3.9': ('physics.adjustment;rolling.run_day', 'test_settlement_cancellation_and_repeated_proxy'),
20     '3.10': ('physics.no_storage_decision', 'test_no_storage_deadband; 只作退化解释，不施加于完整模型'),
21     '3.11': ('rolling.run_day', 'test_real_day_rollout_and_template'),
22     '3.12': ('rolling.run_day', 'test_real_day_rollout_and_template'),
23     '3.13': ('optimization.solve;physics.Policy', 'test_exact_milp_matches_fixed_min'),
24     '3.14': ('optimization.add_path;physics.replay', 'test_exact_milp_matches_fixed_min'),
25     '3.15': ('optimization.add_path;physics.validate;rolling.validate_frame', 'test_energy_balance_saturation'),
26     '3.16': ('optimization.solve;optimization.objective', 'test_solution_objective_uses_residual_not_real_cost'),
27     '3.17': ('optimization.scheduled_cost;optimization.add_path;optimization.solve', 'test_exact_milp_matches_fixed_min;test_settlement_cancellation_and_repeated_proxy'),
28     '3.18': ('rolling.run_day', 'test_real_day_rollout_and_template'),
29     '3.19': ('physics.replay;rolling.frame_for', 'test_energy_balance_saturation;test_real_day_rollout_and_template'),
30     '3.20': ('physics.replay;optimization.add_path', 'test_lag_does_not_observe_current;test_exact_milp_matches_fixed_min'),
31     '3.21': ('physics.adjustment;rolling.validate_frame;export.summaries', 'test_settlement_cancellation_and_repeated_proxy;test_solution_objective_uses_residual_not_real_cost'),
32     '3.22': ('rolling.fiv;analysis.information_summary;analysis.paired_summary', 'test_fiv_common_support_and_state_independence; 全年FIV/OUV按用户指令未执行'),
33     '3.23': ('multistage.conditional_tree;multistage.solve_tree;multistage.replay_tree;analysis.stress_dates;analysis.run_experiments', 'test_tree_conditioning_and_uncertainty;test_actual_small_multistage_solve; 正式stress集合未全跑'),
34     '3.24': ('analysis.forecast_diagnostics', 'diagnostics/forecast_metrics.csv、forecast_block_mae.csv及真实渲染热图'),
35     '3.25': ('analysis.variants;analysis.run_experiments', '所有配置入口已实现；完整事后实验按用户指令未执行'),
36 }
```

```

36 '3.26': ('rolling.validate_frame;physics.validate;config.Config;multistage.conditional_tree;data
    .Inputs', 'tests.txt: 未来污染、冻结策略、费用、SOC、模板、树结构实际核验'),
37 '3.27': ('export.export_workbook;export.emergency_events;export.write_tables', '
    test_real_day_rollout_and_template: 正式334天工作簿未生成'),
38 '3.28': ('rolling.run_period;run.main;analysis.run_experiments', '命令行默认benchmark: full显式隔
    离: test_real_day_rollout_and_template'),
39 '3.29': ('run.main;analysis.run_experiments', '整体结构总结, 数值结论等待授权全量后按实际结果生成'),
40 }
41
42
43 def generate() -> dict:
44     source_lines = SOURCE.read_text(encoding='utf-8').splitlines()
45     symbols = {}
46     for path in (ROOT/'src/q3').glob('*.py'):
47         for node in ast.walk(ast.parse(path.read_text(encoding='utf-8'))):
48             if isinstance(node, (ast.FunctionDef, ast.ClassDef)):
49                 symbols[f'{path.stem}.{node.name}'] = f'{path.relative_to(ROOT)}:{node.lineno}'
50     rows, section = [], '3.1'
51     for line_no, content in enumerate(source_lines, 1):
52         match = re.match(r'#+\s+(3\.\d+)', content)
53         if match:
54             section = match.group(1)
55         if not content.strip():
56             continue
57         implementations, validation = MAPPING[section]
58         references = [symbols[symbol] for symbol in implementations.split(';')]
59         rows.append({'requirement_id': f'Q3-L{line_no:04d}', 'section': section, 'source_line':
    line_no,
60                     'source_text': content, 'implementation_symbols': implementations,
61                     'code_locations': ';'.join(references), 'acceptance': validation,
62                     'status': 'implemented; bounded-tests-only; full-numerical-evaluation-not-run'
    })
63     frame = pd.DataFrame(rows)
64     write_csv(ROOT/'docs/3/source_line_acceptance.csv', frame)
65     files = sorted((ROOT/'src/q3').glob('*.py'))+[ROOT/'src/plots/q3.py']
66     audit = {'source_lines': len(source_lines), 'nonblank_lines_mapped': len(rows),
67             'all_nonblank_source_lines_covered': len(rows) == sum(bool(line.strip()) for line in
    source_lines),
68             'sections': sorted(set(frame.section), key=lambda s: int(s.split('.')[1])),
69             'all_symbols_resolve': True, 'full_run_executed': False,
70             'limits': '逐行映射提供定位: 公式语义由人工逐节复核及独立数学/真实小样本测试补足, 不宣称仅靠存在性证明
    正确。',
71             'source_sha256': hashlib.sha256(SOURCE.read_bytes()).hexdigest(),
72             'code_sha256': {str(path.relative_to(ROOT)): hashlib.sha256(path.read_bytes()).
    hexdigest() for path in files}
73     write_json(ROOT/'docs/3/traceability_audit.json', audit)
74     return audit
75
76
77 if __name__ == '__main__':
78     print(generate())

```

src/q3/validation.py

```

1 """最终检验方案的独立队列: 只读主结果, 计算任务等待主表完成."""
2 import argparse

```

```

3 from dataclasses import replace
4 import hashlib
5 import json
6 import os
7 import subprocess
8 import sys
9 from pathlib import Path
10 import time
11 import numpy as np
12 import pandas as pd
13 from .config import ROOT, Config, CAP, DT, E_MIN, E_MAX, ETA, TOL, write_csv, write_json
14 from .data import Inputs, read_inputs
15 from .rolling import date_of, run_period, run_day, fiv
16 from .physics import adjustment, replay, Policy
17 from .scenarios import construct, pool_indices
18 from .terminal import TerminalValues
19 from .export import summaries
20 from .analysis import forecast_diagnostics, paired_summary, stress_dates
21 from .parallel import process_pool, limit_native_libraries
22 from .checkpoint import run_lock
23 from .sampled import DATES
24
25 SPEC = ROOT/'docs/3/validation/Q3_第三问_模型检验与实验方案_最终版.md'
26 NODE_SETS = {'K0': (0,), 'K06': (0, 6), 'K0612': (0, 6, 12), 'K061218': (0, 6, 12, 18)}
27
28
29 def load_config(main_run: Path) -> Config:
30     values = json.loads((main_run/'execution.json').read_text(encoding='utf-8'))['config']
31     values['nodes'] = tuple(values['nodes'])
32     return Config(**values)
33
34
35 def variant_configs(base: Config) -> dict[str, Config]:
36     return {'M1': base, 'S10': replace(base, scenarios=10), 'S30': replace(base, scenarios=30),
37            'tail0': replace(base, tail=0), 'tail4': replace(base, tail=4),
38            'terminal08': replace(base, terminal_scale=.8), 'terminal12': replace(base, terminal_scale
39            =1.2),
40            'pchip': replace(base, interpolation='pchip'), 'lag': replace(base, feedback='lag'),
41            'sequential': replace(base, settlement='sequential')}
42
43 def audit_main(data: Inputs, main_run: Path, output: Path) -> pd.DataFrame:
44     """读取一次已提交日数据，未完成的节点/日不纳入正式审计。"""
45     main = main_run/'main'; state_path = main/'state.json'
46     if not state_path.exists():
47         return pd.DataFrame()
48     state = json.loads(state_path.read_text(encoding='utf-8'))
49     if [item['day'] for item in state['days']] != list(range(len(state['days']))):
50         raise ValueError('主轨迹提交日期不是从1月1日起的连续前缀')
51     metrics = {name: 0. for name in ('能量平衡最大残差', 'SOC递推最大残差', 'SOC越界次数', '功率越界次数',
52                                     '同时充放电次数', '跨日SOC最大残差', '费用重算最大残差', '场景权重异常次数',
53                                     '未来历史轨迹次数', '负荷预测器不一致次数', '已执行购电与节点策略最大残差', '来源数据最大残差', '未达求解精
54                                     度节点数')}
55     n, previous = 0, None
56     for receipt in state['days']:
57         d = int(receipt['day']); date = str(date_of(d).date())
58         paths = {'dispatch': main/f'days/{date}.csv', 'audit': main/f'audit/{date}.json', '

```

```

information': main/f'information/{date}.json'}
58     if any(hashlib.sha256(path.read_bytes()).hexdigest() != receipt['hashes'][key] for key, path
in paths.items()):
59         raise ValueError(f'主结果已提交文件摘要不符: {date}')
60     frame = pd.read_csv(paths['dispatch'], float_precision='round_trip')
61     expected=date_of(d)+pd.to_timedelta((np.arange(144)+1)*10,unit='min')
62     if len(frame)!=144 or not np.array_equal(pd.to_datetime(frame.timestamp).to_numpy(),expected
.to_numpy()):
63         raise ValueError('主轨迹日期或144时段不完整')
64     if d < 31:
65         previous = float(frame.soc.iloc[-1]); continue
66     n += 1
67     c, r, q, u, e = [frame[key].to_numpy() for key in ('charge', 'discharge', 'emergency', '
spill', 'soc')]
68     initial = float(frame.initial_soc.iloc[0]); net = frame.net_kwh.to_numpy(); g = frame.grid.
to_numpy(); price = frame.price.to_numpy()
69     metrics['能量平衡最大残差'] = max(metrics['能量平衡最大残差'], float(np.max(abs(g+r+q-net-c-u))))
70     metrics['SOC递推最大残差'] = max(metrics['SOC递推最大残差'], float(np.max(abs(e-np.r_[initial, e
[:-1]]-ETA*c+r/ETA))))
71     metrics['SOC越界次数'] += int(((e < E_MIN-TOL)|(e > E_MAX+TOL)).sum())
72     metrics['功率越界次数'] += int(((c < -TOL)|(r < -TOL)|(c > CAP+TOL)|(r > CAP+TOL)).sum())
73     metrics['同时充放电次数'] += int(((c > TOL)&(r > TOL)).sum())
74     if previous is not None: metrics['跨日SOC最大残差'] = max(metrics['跨日SOC最大残差'], abs(
initial-previous))
75     previous = float(e[-1])
76     expected = frame.g0.to_numpy()*price+adjustment(g, frame.g0.to_numpy(), price)+5*price*q
77     metrics['费用重算最大残差'] = max(metrics['费用重算最大残差'], float(np.max(abs(expected-frame.
total_cost.to_numpy()))))
78     metrics['来源数据最大残差'] = max(metrics['来源数据最大残差'], float(np.max(abs(frame[['load_kw',
'pv_kw']].to_numpy()-data.actual[d]))))
79     audit = json.loads(paths['audit'].read_text(encoding='utf-8'))
80     if [node['hour'] for node in audit['nodes']]!=[0,6,12,18]:
81         raise ValueError('主轨迹缺少四信息节点')
82     np.testing.assert_allclose(frame.g0,np.array(audit['nodes'][0]['grid']),atol=TOL,rtol=0)
83     for node in audit['nodes']:
84         hour = node['hour']; scenes = node['scenarios']; weights = np.array(scenes.get('weights'
, [1.]))
85         metrics['场景权重异常次数'] += int((weights < 0).any() or not np.isclose(weights.sum(), 1.,
atol=1e-12, rtol=0))
86         metrics['未来历史轨迹次数'] += sum(j >= d or j*144+hour*6+144 > d*144+hour*6 for j in
scenes['history_days'])
87         metrics['负荷预测器不一致次数'] += int(scenes['predictor'] != data.selected[d])
88         metrics['未达求解精度节点数'] += int(not node['solver']['reliable'])
89         metrics['已执行购电与节点策略最大残差'] = max(metrics['已执行购电与节点策略最大残差'],
float(np.max(abs(g[hour*6:hour*6+36]-np.array(node['grid'][:36])))))
90     result = pd.DataFrame([{'check': key, 'value': value if n else None,
91     'status': '待正式数据' if n == 0 else '通过' if value <= (0 if '次数' in key or '节点数' in key
else TOL) else '未通过',
92     'formal_days_checked': n, 'scope': 'full_334_days' if n == 334 else 'committed_prefix_only',
93     'tolerance': 0 if '次数' in key or '节点数' in key else TOL} for key, value in metrics.items()
])
94 write_csv(output/'audit_table.csv', result)
95 write_json(output/'audit_receipt.json', {'committed_days': len(state['days']), 'formal_days': n,
96     'input_manifest': data.provenance['files'], 'model_parameters_not_retuned': True,
97     'historical_load_errors': 'inherited_Q2_rolling_origin_library; source/code tests accompany
numeric audit',
98     'time_resolution': '10min aggregate causal feedback, not claimed as observed sub10min

```

```

100     control'}}
101     return result
102
103 def rollout_job(request: tuple) -> dict:
104     data, cfg, path, dates, initials, scope, with_fiv = request
105     signature = cfg.signature(); complete = path/'validation_complete.json'
106     if complete.exists():
107         saved = json.loads(complete.read_text(encoding='utf-8'))
108         if saved['signature'] != signature or saved['scope'] != scope:
109             raise ValueError('检验分支的已有收据与配置/范围不同')
110         for file, digest in saved['files'].items():
111             if hashlib.sha256((path/file).read_bytes()).hexdigest() != digest: raise ValueError('检验
112             结果摘要损坏')
113         return {'path': str(path), 'status': 'complete', 'reused': True}
114     if scope == 'full':
115         frame = run_period(data, cfg, path, workers=1, with_fiv=with_fiv, max_slices=None)
116     else:
117         terminal = TerminalValues(data, cfg, cache_path=path/'terminal_cache.json'); pieces = []
118         for date in dates:
119             d = (pd.Timestamp(date)-pd.Timestamp('2025-01-01')).days
120             daily, audit, info = run_day(data, terminal, d, initials[date], cfg, with_fiv=with_fiv,
121             checkpoint_root=path/'nodes'/date, require=False, max_slices=1)
122             write_csv(path/f'days/{date}.csv', daily); write_json(path/f'audit/{date}.json', audit)
123             write_json(path/f'information/{date}.json', info); pieces.append(daily)
124             frame = pd.concat(pieces, ignore_index=True); write_csv(path/'dispatch.csv', frame)
125         write_csv(path/'daily.csv', summaries(frame)[0])
126         names=['dispatch.csv', 'daily.csv']+['f'{kind}/{date}.json' for kind in ('audit', 'information')
127         for date in dates]
128         write_json(complete, {'signature': signature, 'scope': scope, 'dates': list(dates),
129         'files': {name: hashlib.sha256((path/name).read_bytes()).hexdigest() for
130         name in names}})
131         return {'path': str(path), 'status': 'complete', 'reused': False}
132
133 def base_fiv_job(request: tuple) -> dict:
134     data, cfg, main, output, dates, scope = request
135     for date in dates:
136         terminal = TerminalValues(data, cfg, cache_path=output/'nodes'/date/'terminal_cache.json')
137         done = output/f'information/{date}.json'
138         receipt=output/f'receipts/{date}.json'
139         if done.exists() and receipt.exists():
140             saved=json.loads(receipt.read_text(encoding='utf-8'))
141             if saved['signature']!=cfg.signature() or saved['sha256']!=hashlib.sha256(done.
142             read_bytes()).hexdigest():
143                 raise ValueError('已有FIV日期收据与结果不匹配')
144             rows=json.loads(done.read_text(encoding='utf-8'))
145             if scope!='full' or all(branch['solver']['reliable'] for row in rows for branch in row['
146             audits']): continue
147             d = (pd.Timestamp(date)-pd.Timestamp('2025-01-01')).days
148             audit = json.loads((main/f'audit/{date}.json').read_text(encoding='utf-8'))
149             grid = np.array(audit['nodes'][0]['grid']); rows = []
150             for node in audit['nodes'][1:]:
151                 hour = node['hour']
152                 row, branches = fiv(data, terminal, d, hour, node['initial_soc'], grid[hour*6:], cfg,
153                 checkpoint_root=output/'nodes'/date/str(hour), require=scope == 'full',
154                 max_slices=None if scope == 'full' else 1)

```



```

151     write_json(output/f'branches/{date}_{hour}.json', branches); rows.append(row)
152     write_json(done, rows)
153     write_json(receipt,{ 'signature':cfg.signature(), 'sha256':hashlib.sha256(done.read_bytes()).
    hexdigest()})
154     return {'path': str(output), 'status': 'complete'}
155
156
157 def metric_tables(data: Inputs, main: Path, output: Path, variants: dict[str, Config], dates: list[
    str], scope: str) -> None:
158     economics, sensitivity, model_comparison = [], [], []
159     errors = pd.read_csv(output/'forecast/forecast_revision_pairs.csv')
160     mae = errors[errors.date.isin(dates)&errors.block].groupby('hour')[['old_absolute_error', '
    new_absolute_error']].mean()
161     base_dispatch = pd.read_csv(main/'dispatch.csv')
162     base_dispatch = base_dispatch[base_dispatch.date.isin(dates)]
163     base_daily = summaries(base_dispatch)[0].set_index('date')
164     for label, cfg in variants.items():
165         paths = {key: output/'rollouts'/label/key for key in NODE_SETS}
166         costs = {}
167         for key, path in paths.items():
168             frame = base_dispatch if label == 'M1' and key == 'K061218' else pd.read_csv(path/'
    dispatch.csv')
169             costs[key] = frame.groupby('date').total_cost.sum().reindex(dates)
170             info_dir = output/'base_fiv/information' if label == 'M1' else paths['K061218']/'information
    '
171             info = [row for date in dates for row in json.loads((info_dir/f'{date}.json').read_text(
    encoding='utf-8'))]
172             for hour, old, new in ((6, 'K0', 'K06'), (12, 'K06', 'K0612'), (18, 'K0612', 'K061218')):
173                 values = np.array([row['FIV_block'] for row in info if row['hour'] == hour])
174                 op = (costs[old]-costs[new]).to_numpy()
175                 if len(values) != len(dates) or not np.isfinite(op).all(): raise ValueError('FIV/OUV配对
    日期不完整')
176                 records = [row for row in info if row['hour'] == hour]
177                 reliable = all(branch['solver']['reliable'] for row in records for branch in row['audits
    '])
178                 ouv_reliable = True
179                 for key in (old, new):
180                     for date in dates:
181                         audit_path = main/f'audit/{date}.json' if label=='M1' and key=='K061218' else
    paths[key]/f'audit/{date}.json'
182                         node_audit = json.loads(audit_path.read_text(encoding='utf-8'))
183                         ouv_reliable &= all(node['solver']['reliable'] for node in node_audit['nodes'])
184                         economics.append({'variant': label, 'hour': hour, 'scope': scope, 'days': len(dates),
    'old_MAE': mae.loc[hour, 'old_absolute_error'], 'new_MAE': mae.loc[hour, '
    new_absolute_error'],
185                         'FIV_sum': float(values.sum()), 'FIV_median': float(np.median(values)), '
    FIV_positive_fraction': float((values > 0).mean()),
186                         'OUV_sum': float(op.sum()), 'OUV_median': float(np.median(op)), '
    OUV_positive_fraction': float((op > 0).mean()),
187                         'FIV18_day': sum(row['FIV_day'] for row in records) if hour == 18 else None,
188                         'FIV18_cont': sum(row['FIV_cont'] for row in records) if row['FIV_cont'] is not None)
189                         if hour == 18 else None,
190                         'cross_day_valid_dates': sum(row['comparison_complete'] for row in records), '
    all_FIV_solves_certified': reliable,
191                         'all_OUV_solves_certified': bool(ouv_reliable),
192                         'interpretation': '求解未全部达标, 暂不作结论' if not (reliable and ouv_reliable) else '
    本范围内FIV与OUV均为正' if values.sum()>0 and op.sum()>0 else '本范围内收益有限或符号不一致'})

```



```

193     full = base_dispatch if label == 'M1' else pd.read_csv(paths['K061218']/dispatch.csv')
194     day = summaries(full)[0].set_index('date').reindex(dates)
195     terminal_soc, gaps, seconds, tail_retained = [], [], [], []
196     for date in dates:
197         d = (pd.Timestamp(date)-pd.Timestamp('2025-01-01')).days
198         audit_path = main/f'audit/{date}.json' if label == 'M1' else paths['K061218']/f'audit/{
date}.json'
199         audit = json.loads(audit_path.read_text(encoding='utf-8'))
200         for node in audit['nodes']:
201             hour = node['hour']; scenes = construct(data, d, hour, cfg)
202             if scenes.audit['history_days'] != node['scenarios']['history_days']: raise
ValueError('回读场景与原节点不一致')
203             policy = Policy(*(np.array(node[key]) for key in ('grid', 'charge_cap', '
discharge_cap'))))
204             previous = float((data.actual.reshape(-1, 2)[d*144+hour*6-1, 0]-data.actual.reshape
(-1, 2)[d*144+hour*6-1, 1])*DT)
205             e = [replay(policy, n, node['initial_soc'], mode=cfg.feedback, previous_net=previous
)]['soc'][-1] for n in scenes.net]
206             terminal_soc.append(float(scenes.weights@e)); gaps.append(node['solver']['gap']);
seconds.append(node['solver']['seconds'])
207             ids, el, ep, scales = data.history(d, hour, data.selected[d], method=cfg.
interpolation)
208             indices, _ = pool_indices(ids, scales, d, data.pv(d, hour, cfg.interpolation).sum()*
DT, cfg)
209             risk = np.maximum(el[indices]-ep[indices], 0)@data.prices[(hour*6+np.arange(144))
%144]*DT
210             top = ids[indices[np.argsort(-risk, kind='stable')[:min(2, len(indices))]]]
211             tail_retained.append(float(np.mean(np.isin(top, scenes.audit['history_days']))))
212             aligned = full[full.slot<36].merge(base_dispatch[base_dispatch.slot<36], on=['date', 'slot'],
suffixes=('_v', '_base'))
213             sensitivity.append({'variant': label, 'scope': scope, 'days': len(dates), 'total_cost':
float(day.total_cost.sum()),
214                               'cost_change_pct': float((day.total_cost.sum()/base_daily.total_cost.sum()-1)*100),
215                               'emergency_kwh': float(day.emergency.sum()), 'emergency_cost': float(day.emergency_cost.
sum()),
216                               'emergency_change_kwh': float(day.emergency.sum()-base_daily.emergency.sum()),
217                               'emergency_change_pct': float((day.emergency.sum()/base_daily.emergency.sum()-1)*100) if
base_daily.emergency.sum() else None,
218                               'adjustment_cost': float(day.adjustment_cost.sum()), 'throughput': float(day.throughput.
sum()),
219                               'first_block_grid_L1': float((aligned.grid_v-aligned.grid_base).abs().sum()),
220                               'mean_window_terminal_soc': float(np.mean(terminal_soc)), 'top2_risk_retained_fraction'
: float(np.mean(tail_retained)),
221                               'solver_seconds': float(sum(seconds)), 'max_solver_gap': float(max(gaps)),
222                               'all_nodes_certified': all(gap <= cfg.gap+1e-8 for gap in gaps)})
223     eco = pd.DataFrame(economics)
224     for metric in ('FIV_sum', 'OUV_sum'):
225         eco[metric+'_rank'] = eco.groupby('variant')[metric].rank(ascending=False, method='min')
226     write_csv(output/'information_value.csv', eco); write_csv(output/'sensitivity.csv', pd.DataFrame
(sensitivity))
227     m0 = pd.read_csv(output/'rollouts/M0/K061218/dispatch.csv'); m0 = m0[m0.date.isin(dates)]
228     daily0 = summaries(m0)[0].set_index('date').reindex(dates)
229     for label, day in (('M0', daily0), ('M1', base_daily)):
230         model_comparison.append({'model': label, 'scope': scope, 'days': len(dates),
231                                **{key: float(day[key].sum()) for key in ('total_cost', 'planned_cost', 'adjustment_cost',
'emergency_cost', 'emergency', 'throughput')},
232                                'emergency_slot_rate': float(day.emergency_slots.sum()/(len(dates)*144)),

```

```

233         'cost_improvement_pct_vs_M0': float((daily0.total_cost.sum()-day.total_cost.sum())/
234         daily0.total_cost.sum()*100))
235     difference = (daily0.total_cost-base_daily.total_cost).to_numpy()
236     write_csv(output/'M0_M1_comparison.csv', pd.DataFrame(model_comparison))
237     write_csv(output/'M0_M1_daily_pairs.csv', pd.DataFrame({'date': dates, 'saving_M1':difference}))
238     write_json(output/'M0_M1_paired_statistics.json', paired_summary(difference))
239     write_csv(output/'M0_M1_daily_SOC.csv',pd.DataFrame({'date':dates,
240         'M0_initial_soc':daily0.soc_start.to_numpy(),'M1_initial_soc':base_daily.soc_start.to_numpy
241         ()),
242         'M0_end_soc':daily0.soc_end.to_numpy(),'M1_end_soc':base_daily.soc_end.to_numpy()}))
243
244 def benchmark_job(request: tuple) -> dict:
245     from .multistage import tree_inputs, solve_tree, replay_tree
246     data, cfg, main, output, day, regime = request
247     date = str(date_of(day).date()); path = output/date
248     receipt = path/'complete.json'
249     if receipt.exists():
250         saved = json.loads(receipt.read_text(encoding='utf-8'))
251         if saved['signature'] != cfg.signature(): raise ValueError('M2已有结果配置不同')
252         if any(hashlib.sha256((path/name).read_bytes()).hexdigest()!=sha for name,sha in saved['
253         files'].items()):
254             raise ValueError('M2切片输出损坏或缺失')
255         return saved['result']
256     original = pd.read_csv(main/f'days/{date}.csv'); initial = float(original.initial_soc.iloc[0])
257     net, labels, nodes, audit = tree_inputs(data, day, cfg)
258     terminal = TerminalValues(data, cfg, cache_path=path/'terminal_cache.json')
259     coefficient, calibration = terminal.value(day, 0)
260     # 每个M2日期为一个有界求解切片；开始前保留实际输入和可行初始策略，结束后保存完整节点策略。
261     write_csv(path/'scenario_input.csv', pd.DataFrame({'scenario':np.repeat(np.arange(len(net)),144)
262     ,
263         'slot':np.tile(np.arange(144),len(net)), 'net_kwh':net.ravel()}))
264     seed = np.maximum(net.max(0), 0)
265     write_json(path/'initial_checkpoint.json', {'signature':cfg.signature(), 'initial_soc':initial,
266         'grid':seed.tolist(), 'charge_cap':0., 'discharge_cap':0., 'slice_seconds':cfg.seconds,
267         'objective':float(data.prices@seed-coefficient*initial), 'lower_bound':-coefficient*E_MAX,
268         'tree':nodes,'labels':labels.tolist(),'calibration':calibration,'source':audit})
269     solved = solve_tree(net, labels, nodes, np.array(audit['load_upper']), data.prices, initial,
270     coefficient, cfg, require=False)
271     write_json(path/'solver.json', solved['audit'])
272     if 'policies' not in solved:
273         result = {'date':date,'regime':regime,'status':'no_feasible_solution','M1_cost':float(
274         original.total_cost.sum()),
275             'M2_cost':None,'relative_loss':None,'gap':None,'certified':False}
276     else:
277         frame = replay_tree(data, day, initial, nodes, solved, cfg)
278         write_csv(path/'dispatch.csv', frame)
279         write_json(path/'policies.json', {'g0':solved['g0'].tolist(), 'policies':{'str(k):{field:
280         getattr(policy,field).tolist()
281         for field in ('grid','charge_cap','discharge_cap')} for k,policy in solved['policies'].
282         items()}})
283         cost = float(frame.total_cost.sum())
284         result = {'date':date,'regime':regime,'status':solved['audit']['status'],'M1_cost':float(
285         original.total_cost.sum()),
286             'M2_cost':cost,'relative_loss':float((original.total_cost.sum()-cost)/cost),
287             'gap':solved['audit']['gap'],'certified':solved['audit']['reliable']}
288     files=['initial_checkpoint.json','scenario_input.csv','solver.json']

```

```

281 if 'policies' in solved: files += ['dispatch.csv', 'policies.json']
282 write_json(receipt, {'signature':cfg.signature(), 'result':result,
283                    'files':{name:hashlib.sha256((path/name).read_bytes()).hexdigest() for name in files}})
284 return result
285
286
287 def plan_jobs(data: Inputs, base: Config, main: Path, output: Path, dates: list[str], scope: str) ->
    list[tuple]:
288     initial = {date:float(pd.read_csv(main/f'days/{date}.csv').initial_soc.iloc[0]) for date in
    dates}
289     jobs = []
290     for label, cfg in variant_configs(base).items():
291         for key, nodes in NODE_SETS.items():
292             if label == 'M1' and key == 'K061218': continue
293             jobs.append((data, replace(cfg,nodes=nodes), output/'rollouts'/label/key, dates, initial
    , scope, key=='K061218'))
294     jobs.insert(3, (data, replace(base,deterministic=True), output/'rollouts/M0/K061218', dates,
    initial, scope, False))
295     return jobs
296
297
298 def render_status(output: Path, status: dict) -> None:
299     write_json(output/'status.json', status)
300     body='<!doctype html><html lang="zh"><meta charset="utf-8"><meta http-equiv="refresh" content
    ="30"><meta name="viewport" content="width=device-width,initial-scale=1"><title>第三问独立检验</
    title><style>body{font:17px/1.7 -apple-system,sans-serif;max-width:980px;margin:40px auto;
    padding:0 20px;color:#263b50}table{border-collapse:collapse}td,th{padding:7px;border-bottom:1px
    solid #ccd6df}.table{overflow:auto}</style><h1>第三问独立检验</h1>'
301     body+=f'<p>阶段: {status["phase"]}; 检验范围: {status.get("scope","待确认")}; 主计算独立运行, 检验不作为
    主表导出的前置条件.</p>'
302     if (output/'audit_table.csv').exists():
303         body+=f'<h2>已提交主结果审计</h2><div class="table">'+pd.read_csv(output/'audit_table.csv').
    to_html(index=False)+'</div>'
304     body+=f'<p>优先顺序: 正确性审计 → 预测误差/FIV/OUV → M0/M1 → 六组核心敏感性 → 4—8日M2 → 紧凑假设对照。所有
    统计按实际范围标注, 不预设M1或M2一定更优.</p>'
305     body+=f'<p><a href="forecast/figures/forecast_error_heatmaps.png">预报误差图</a> • <a href="
    audit_table.csv">审计表</a> • <a href="plan.json">检验计划</a> • <a href="status.json">状态</a></
    p></html>'
306     (output/'report.html').write_text(body, encoding='utf-8')
307
308
309 def execute(data: Inputs, main_run: Path, output: Path, scope: str, workers: int) -> None:
310     base = load_config(main_run); main = main_run/'main'
311     state=json.loads((main/'state.json').read_text(encoding='utf-8'))
312     if state['sources']!=hashlib.sha256(json.dumps(data.provenance,sort_keys=True).encode('utf-8')).
    hexdigest():
313         raise ValueError('检验与正式主轨迹的原始输入/预测来源不一致')
314     checks=audit_main(data,main_run,output)
315     if (checks.status=='未通过').any() or not (checks.formal_days_checked==334).all():
316         raise ValueError('正式主结果审计未全部通过, 暂停生成经济结论: 不改写主计算')
317     frame = pd.read_csv(main/'dispatch.csv')
318     dates = sorted(frame.date.unique().tolist()) if scope=='full' else list(DATES)
319     variants = variant_configs(base)
320     jobs = plan_jobs(data, base, main, output, dates, scope)
321     write_json(output/'plan.json', {'scope':scope,'dates':dates,'model':'frozen_M1_not_retuned','gap
    ':base.gap,
322         'variants':list(variants),'rollout_jobs':len(jobs),'reuse_M1':True,'M2_days_range':[4,8],

```

```

323     'spec_sha256': hashlib.sha256(SPEC.read_bytes()).hexdigest(), 'original_main_sources': str(
    main_run/'runtime_sources.json'})})
324 # 最高优先的信息价值先做; 基准M1不重求, 仅用已提交节点状态构造反事实。
325 render_status(output, {'phase': 'base_FIV', 'scope': scope})
326 with process_pool(workers, base.solver_threads) as pool:
327     requests = [(data, base, main, output/'base_fiv', [date], scope) for date in dates]
328     completed = list(pool.map(base_fiv_job, requests)) if pool else list(map(base_fiv_job,
    requests))
329     write_json(output/'base_FIV_jobs.json', completed)
330 # 三条更新时间链和M0先于敏感性; 所有任务与主进程分组/输出隔离。
331 with process_pool(workers, base.solver_threads) as pool:
332     render_status(output, {'phase': 'OUV_and_M0', 'scope': scope})
333     leading = list(pool.map(rollout_job, jobs[:4])) if pool else list(map(rollout_job, jobs[:4]))
334     write_json(output/'leading_jobs.json', leading)
335     metric_tables(data, main, output, {'M1': base}, dates, scope)
336     render_status(output, {'phase': 'core_sensitivity', 'scope': scope})
337     results = list(pool.map(rollout_job, jobs[4:28])) if pool else list(map(rollout_job, jobs
    [4:28]))
338     write_json(output/'core_jobs.json', results)
339 selection = stress_dates(data, frame)
340 chosen = selection.groupby('day', sort=True).stratum.apply(lambda s: ';'.join(s)).to_dict()
341 for date in DATES:
342     if len(chosen) >= 4: break
343     chosen[(pd.Timestamp(date) - pd.Timestamp('2025-01-01')).days] = 'fixed_calendar_supplement'
344     chosen = dict(list(chosen.items())[:8]); write_json(output/'M2_selection.json', {str(k): v for k,
    v in chosen.items()})
345     render_status(output, {'phase': 'M2_benchmark', 'scope': scope})
346     requests = [(data, base, main, output/'M2', int(day), regime) for day, regime in chosen.items()]
347     with process_pool(workers, base.solver_threads) as pool:
348         records = list(pool.map(benchmark_job, requests)) if pool else list(map(benchmark_job, requests)
    )
349     write_csv(output/'M2_comparison.csv', pd.DataFrame(records))
350     values = np.array([row['relative_loss'] for row in records if row['relative_loss'] is not None])
351     write_json(output/'M2_summary.json', {'days': len(records), 'median_relative_loss': float(np.median(
    values)) if len(values) else None,
352         'max_absolute_relative_loss': float(np.max(abs(values))) if len(values) else None,
353         'all_certified': all(row['certified'] for row in records), 'scope': '
    stratified_stress_sample_not_unbiased_annual_estimate'})
354     render_status(output, {'phase': 'assumption_robustness', 'scope': scope})
355     with process_pool(workers, base.solver_threads) as pool:
356         records = list(pool.map(rollout_job, jobs[28:])) if pool else list(map(rollout_job, jobs[28:]
    )
357         write_json(output/'assumption_jobs.json', records)
358     metric_tables(data, main, output, variants, dates, scope)
359     audit_main(data, main_run, output)
360     render_status(output, {'phase': 'complete', 'scope': scope, 'main_not_modified': True})
361
362
363 def main() -> None:
364     parser = argparse.ArgumentParser(description=__doc__)
365     parser.add_argument('--main-run', type=Path, required=True); parser.add_argument('--output', type=
    Path, required=True)
366     parser.add_argument('--scope', choices=['sampled', 'full', 'pending'], default='pending')
367     parser.add_argument('--workers', type=int, default=2)
368     args = parser.parse_args(); limit_native_libraries(); os.nice(10)
369     output = args.output.resolve(); main_run = args.main_run.resolve(); output.mkdir(parents=True, exist_ok
    =True)

```

```

370 with run_lock(output/'validation.lock'):
371     request=output/'request.json'
372     if not request.exists():write_json(request,{'scope':args.scope,'workers':args.workers})
373     data=read_inputs(ROOT/'outputs/q2/dispatch_runs/20260911_224501',output/'prepared')
374     forecast_diagnostics(data,output/'forecast')
375     last_days=-1
376     while True:
377         settings=json.loads(request.read_text(encoding='utf-8'));scope=settings['scope']
378         state_path=main_run/'main/state.json'
379         days=len(json.loads(state_path.read_text(encoding='utf-8')).get('days',[])) if
state_path.exists() else 0
380         if days!=last_days:
381             audit_main(data,main_run,output);last_days=days
382             ready=(main_run/'main_complete.json').exists()
383             render_status(output,{'phase':'waiting_scope' if scope=='pending' else 'waiting_main' if
not ready else 'preparing',
384                                 'scope':scope,'committed_main_days':days})
385             if ready and scope in ('sampled','full'):break
386             time.sleep(30)
387             caffeine=subprocess.Popen(['caffeinate','-i','-w',str(os.getpid())] if sys.platform=='
darwin' else None
388             try:
389                 execute(data,main_run,output,scope,int(settings['workers']))
390             except Exception as error:
391                 render_status(output,{'phase':'failed_resume_available','scope':scope,'error':str(error)
})
392             raise
393         finally:
394             if caffeine is not None:
395                 caffeine.terminate();caffeine.wait()
396
397
398 if __name__=='__main__':
399     main()

```

B.4 问题四源程序

src/q4/__init__.py

```

1 """问题四：严格因果波动电价与价格—源荷联合鲁棒调度。"""

```

src/q4/config.py

```

1 """问题四冻结参数、原子写入与可复现配置。"""
2 from dataclasses import asdict, dataclass
3 from pathlib import Path
4 import hashlib
5 import json
6 import os
7
8 CODE_ROOT = Path(__file__).resolve().parent
9 ROOT = Path(os.environ.get("Q4_WORKSPACE_ROOT", str(CODE_ROOT.parents[1]))).resolve()
10 SOURCE = ROOT / "docs/4/Q4_问题四_严格因果波动电价_完整建模_最终修订版.md"
11 ATTACHMENTS = ROOT / "docs/CUMCM2026Problems/C题/附件"
12 Q2_RUN = ROOT / "outputs/q2/dispatch_runs/20260912_tail_reserve_final"

```

```

13
14 DT = 1 / 6
15 CAP = 5000 * DT
16 E_MIN, E_MAX, E_INITIAL = 1200.0, 10800.0, 6000.0
17 ETA_C = ETA_D = 0.9
18 NODES = (0, 6, 12, 18)
19 TOL = 1e-6
20 MODEL_VERSION = "q4_rescue_lp_v3"
21 THROUGHPUT_EPSILON = 1e-7
22
23
24 @dataclass(frozen=True)
25 class Config:
26     """未由方案定值的计算参数全部显式保留在审计中。"""
27
28     scenarios: int = 20
29     residual_window: int = 56
30     net_weight: float = 0.5
31     bootstrap_repetitions: int = 100
32     bootstrap_quantile: float = 0.95
33     dro_scale: float = 1.0
34     conditional: bool = True
35     joint: bool = True
36     min_ess: float = 10.0
37     nodes: tuple[int, ...] = NODES
38     seconds: float = 30.0
39     gap: float = 0.03
40     solver_threads: int = 1
41     hard_timeout: float = 30.0
42     seed: int = 20250913
43     allow_limited: bool = False
44
45     def __post_init__(self) -> None:
46         if self.scenarios < 1:
47             raise ValueError("场景数必须为正")
48         if self.residual_window < 1:
49             raise ValueError("残差窗口必须为正; 正式运行预先固定56日")
50         if not 0 < self.net_weight < 1:
51             raise ValueError("联合距离权重必须位于(0,1)")
52         if self.bootstrap_repetitions < 1 or not 0 < self.bootstrap_quantile < 1:
53             raise ValueError("bootstrap设置非法")
54         if self.dro_scale < 0 or self.min_ess <= 0:
55             raise ValueError("鲁棒半径倍率或ESS阈值非法")
56         if self.nodes not in ((0,), (0, 6), (0, 6, 12), NODES):
57             raise ValueError("更新时间集合必须为方案四种之一")
58         if self.hard_timeout <= 0 or self.seconds <= 0 or not 0 <= self.gap < 1 or self.solver_threads < 1:
59             raise ValueError("求解预算非法")
60
61     def signature(self) -> str:
62         payload = SOURCE.read_bytes() + json.dumps(asdict(self), sort_keys=True).encode("utf-8")
63         for source in production_sources():
64             if not source.name.startswith("test_"):
65                 payload += str(source.relative_to(CODE_ROOT.parent)).encode("utf-8") + source.read_bytes()
66         return hashlib.sha256(payload).hexdigest()
67

```

```

68
69 def production_sources() -> List[Path]:
70     """覆盖实际Q4模块及只读输入解析的传递依赖；不依赖Q3求解器。"""
71     return sorted(CODE_ROOT.glob("*.py")) + [
72         CODE_ROOT.parent / relative for relative in
73         ("q3/data.py", "q3/config.py", "q2/data.py", "q2/forecast.py", "q2/protocol.py", "q2/policy.
74         py")
75     ]
76
77 def write_json(path: Path, value: object) -> None:
78     path.parent.mkdir(parents=True, exist_ok=True)
79     temporary = path.with_suffix(path.suffix + f".{os.getpid()}.pending")
80     with temporary.open("w", encoding="utf-8") as stream:
81         json.dump(value, stream, ensure_ascii=False, indent=2, allow_nan=False)
82         stream.flush()
83         os.fsync(stream.fileno())
84     temporary.replace(path)
85
86
87 def write_csv(path: Path, frame: object) -> None:
88     path.parent.mkdir(parents=True, exist_ok=True)
89     temporary = path.with_suffix(path.suffix + f".{os.getpid()}.pending")
90     with temporary.open("w", encoding="utf-8", newline="") as stream:
91         frame.to_csv(stream, index=False, float_format="%.17g")
92         stream.flush()
93         os.fsync(stream.fileno())
94     temporary.replace(path)

```

src/q4/data.py

```

1  """7.2/7.4: 附件4校验、Q2/Q3预测复用与同日联合OOS残差。"""
2  from dataclasses import dataclass
3  from pathlib import Path
4  import hashlib
5  import numpy as np
6  import pandas as pd
7  from openpyxl import load_workbook
8  from src.q3.data import Inputs as Q3Inputs, read_inputs as read_q3_inputs
9  from .config import ATTACHMENTS, DT, Q2_RUN, ROOT, production_sources, write_csv, write_json
10 from . import price as price_module
11 from .price import Forecasts, rolling_forecasts
12
13
14 def _endpoint_minutes(value: object) -> int:
15     if isinstance(value, str) and value == "0:00+1":
16         return 1440
17     if hasattr(value, "hour") and hasattr(value, "minute"):
18         result = int(value.hour) * 60 + int(value.minute)
19         return 1440 if result == 0 else result
20     raise ValueError(f"无法识别附件4时刻: {value!r}")
21
22
23 def read_prices(path: Path) -> np.ndarray:
24     workbook = load_workbook(path, read_only=True, data_only=True)
25     try:

```



```

26     sheet = workbook.active
27     rows = list(sheet.values)
28     finally:
29         workbook.close()
30     if len(rows) != 366 or len(rows[0]) != 145:
31         raise ValueError("附件4必须为表头加365日、每日144时段")
32     if [_endpoint_minutes(value) for value in rows[0][1:]] != list(range(10, 1441, 10)):
33         raise ValueError("附件4右端点时刻错位")
34     dates = [pd.Timestamp(row[0]).normalize() for row in rows[1:]]
35     expected = list(pd.date_range("2025-01-01", periods=365, freq="D"))
36     if dates != expected:
37         raise ValueError("附件4日期不连续或不属于2025年")
38     values = np.asarray([row[1:] for row in rows[1:]], dtype=float)
39     if not np.isfinite(values).all() or (values <= 0).any():
40         raise ValueError("附件4电价存在缺失、非有限值或非正值")
41     return values
42
43
44 @dataclass
45 class Inputs:
46     q3: Q3Inputs
47     actual_prices: np.ndarray
48     price: Forecasts
49     provenance: dict
50
51     def point_net(self, day: int, hour: int, length: int) -> np.ndarray:
52         name = self.q3.selected[day]
53         load = self.q3.load(day, name)[hour * 6:hour * 6 + length]
54         if length == 144:
55             pv = self.q3.pv(day, hour)
56         elif length == 288 and hour == 0:
57             if day < 7:
58                 raise ValueError("48h递推光伏至少需要7日历史")
59             pv = np.r_[self.q3.pv(day, 0), self.q3.actual[day - 6, :, 1]]
60         else:
61             raise ValueError("仅支持四节点24h或0:00的48h预测")
62         if len(load) != length or len(pv) != length or not np.isfinite(load).all():
63             raise ValueError("因果负荷/光伏预测支持不足")
64         return (load - pv) * DT
65
66     def point_price(self, day: int, hour: int, length: int) -> np.ndarray:
67         result = self.price.values[day, hour * 6:hour * 6 + length]
68         if len(result) != length or not np.isfinite(result).all():
69             raise ValueError("严格因果价格预测支持不足")
70         return result.copy()
71
72     def _net_history(self, day: int, hour: int, length: int) -> tuple[np.ndarray, np.ndarray]:
73         name = self.q3.selected[day]
74         if length == 144:
75             days, load_error, pv_error, _ = self.q3.history(
76                 day, hour, name, forecast_hour=hour, length=length
77             )
78             return days, (load_error - pv_error) * DT
79         if length != 288 or hour != 0:
80             raise ValueError("历史残差支持与当前预测窗口不一致")
81         cutoff = day * 144
82         actual = self.q3.actual.reshape(-1, 2)

```



```

83     ids, errors = [], []
84     for history_day in range(7, day):
85         start = history_day * 144
86         if start + length > cutoff:
87             continue
88         load = self.q3.load_store[name][history_day, :2].reshape(-1)
89         pv = np.r_[self.q3.pv(history_day, 0), self.q3.actual[history_day - 6, :, 1]]
90         if len(load) != length or not np.isfinite(load).all():
91             continue
92         realized = (actual[start:start + length, 0] - actual[start:start + length, 1]) * DT
93         ids.append(history_day)
94         errors.append(realized - (load - pv) * DT)
95     return np.asarray(ids, dtype=int), np.asarray(errors, dtype=float).reshape(-1, length)
96
97 def price_residual_path(self, origin_day: int, hour: int, length: int, *, cutoff: int) -> np.
    ndarray:
98     """完整窗口已实现，且跨午夜始终使用同一次0:00预测。"""
99     start = origin_day * 144 + hour * 6
100    if origin_day < 7 or start + length > cutoff or start + length > self.actual_prices.size:
101        raise ValueError("残差窗口尚未完整实现或历史起点非法")
102    forecast = self.point_price(origin_day, hour, length)
103    actual = self.actual_prices.reshape(-1)[start:start + length]
104    return actual - forecast
105
106 def joint_history(self, day: int, hour: int, length: int) -> tuple[np.ndarray, np.ndarray, np.
    ndarray]:
107     days, net_errors = self._net_history(day, hour, length)
108     cutoff = day * 144 + hour * 6
109     selected_days, selected_net, selected_price = [], [], []
110     for history_day, net_error in zip(days, net_errors):
111         start = int(history_day) * 144 + hour * 6
112         if start + length > cutoff:
113             continue
114         price_error = self.price_residual_path(int(history_day), hour, length, cutoff=cutoff)
115         if len(price_error) == length and np.isfinite(price_error).all():
116             selected_days.append(int(history_day))
117             selected_net.append(net_error)
118             selected_price.append(price_error)
119     return (
120         np.asarray(selected_days, dtype=int),
121         np.asarray(selected_net, dtype=float).reshape(-1, length),
122         np.asarray(selected_price, dtype=float).reshape(-1, length),
123     )
124
125 def prefix(self, day: int, hour: int) -> np.ndarray:
126     if hour == 0:
127         return np.empty(0)
128     end = hour * 6
129     actual_net = (self.q3.actual[day, :end, 0] - self.q3.actual[day, :end, 1]) * DT
130     net_error = actual_net - self.point_net(day, 0, 144)[:end]
131     price_error = self.actual_prices[day, :end] - self.price.values[day, :end]
132     one = min(6, end)
133     three = min(18, end)
134     return np.asarray([
135         price_error.mean(),
136         net_error.mean(),
137         price_error[-one:].mean(),

```

```

138         net_error[-one:].mean(),
139         price_error[-three:].mean(),
140         net_error[-three:].mean(),
141         net_error.sum(),
142     ])
143
144
145 def read_inputs(processed: Path | None = None, q2_run: Path = Q2_RUN) -> Inputs:
146     q3 = read_q3_inputs(q2_run)
147     attachment = ATTACHMENTS / "附件4.xlsx"
148     actual_prices = read_prices(attachment)
149     folder = processed or ROOT / "inputs/q4/processed"
150     attachment_hash = hashlib.sha256(attachment.read_bytes()).hexdigest()
151     price_code_hash = hashlib.sha256(Path(price_module.__file__).read_bytes()).hexdigest()
152     manifest_path = folder / "source_manifest.json"
153     cached = None
154     if manifest_path.exists() and (folder / "price_forecasts.csv").exists() and (folder / "
155 price_residuals.csv").exists():
156         manifest = __import__("json").loads(manifest_path.read_text(encoding="utf-8"))
157         if (manifest.get("attachment4_sha256") == attachment_hash
158             and manifest.get("price_code_sha256") == price_code_hash):
159             values = pd.read_csv(folder / "price_forecasts.csv", float_precision="round_trip").
160 to_numpy(float)
161             residuals = pd.read_csv(folder / "price_residuals.csv", float_precision="round_trip").
162 to_numpy(float)
163             if values.shape == (365, 288) and residuals.shape == (365, 144):
164                 cached = Forecasts(values, residuals, manifest["price_model"])
165 forecasts = cached or rolling_forecasts(actual_prices)
166 provenance = {
167     "q2_run": str(q2_run.resolve()),
168     "q3_provenance": q3.provenance,
169     "production_code_sha256": {str(p.relative_to(Path(__file__).resolve().parents[1])): hashlib.
170 sha256(p.read_bytes()).hexdigest()
171     for p in production_sources() if not p.name.startswith("test_")},
172     "attachment4": str(attachment.resolve()),
173     "attachment4_sha256": attachment_hash,
174     "price_code_sha256": price_code_hash,
175     "price_model": forecasts.audit,
176     "price_forecasts_array_sha256": hashlib.sha256(forecasts.values.tobytes()).hexdigest(),
177     "q2_frozen_dispatch_sha256": hashlib.sha256((q2_run / "processed/main/dispatch.csv").
178 read_bytes()).hexdigest(),
179     "net_forecast": "Q2 selected rolling-00S load + attachment3 forecast-origin PV",
180     "q4_2_second_day_pv": "target-day minus 7 days actual PV, available at current 0:00",
181     "information_boundary": "future actual load/PV/price used only by replay and residuals after
182 realization",
183 }
184 if processed is not None and cached is None:
185     processed.mkdir(parents=True, exist_ok=True)
186     write_csv(processed / "actual_prices.csv", pd.DataFrame(actual_prices))
187     write_csv(processed / "price_forecasts.csv", pd.DataFrame(
188         forecasts.values.reshape(365, 288)
189     ))
190     write_csv(processed / "price_residuals.csv", pd.DataFrame(forecasts.residuals))
191     write_json(processed / "source_manifest.json", provenance)
192 if processed is not None:
193     write_json(processed / "source_manifest.json", provenance)
194 return Inputs(q3, actual_prices, forecasts, provenance)

```

src/q4/diagnostics.py

```
1 """7.2/7.3.4/7.4.1/7.8: 真实数据诊断、联合依赖与消融/敏感性入口。"""
2 from dataclasses import asdict, replace
3 from pathlib import Path
4 import numpy as np
5 import pandas as pd
6 from .config import Config, DT, write_csv, write_json
7 from .data import Inputs
8 from .price import rolling_lightgbm_comparison
9
10
11 def _dependence(net_error: np.ndarray, price_error: np.ndarray) -> dict:
12     net = np.asarray(net_error).ravel()
13     price = np.asarray(price_error).ravel()
14     correlation = float(np.corrcoef(net, price)[0, 1])
15     qn, qp = np.quantile(net, 0.9), np.quantile(price, 0.9)
16     high_net = net > qn
17     tail = float(np.mean(price[high_net] > qp)) if high_net.any() else None
18     return {"correlation": correlation, "net_q90": float(qn), "price_q90": float(qp),
19           "price_upper_tail_given_net_upper_tail": tail}
20
21
22 def diagnostics(data: Inputs, output: Path, *, bootstrap: int = 500, seed: int = 20250913,
23               with_lightgbm: bool = False) -> dict:
24     prices = data.actual_prices
25     flat = prices.ravel()
26     daily = prices.mean(axis=1)
27     trend = np.polyfit(np.arange(365), daily, 1)
28     fitted = np.polyval(trend, np.arange(365))
29     r2 = 1 - np.sum((daily - fitted) ** 2) / np.sum((daily - daily.mean()) ** 2)
30     profile = prices.mean(axis=0)
31     net_real = (data.q3.actual[:, :, 0] - data.q3.actual[:, :, 1]) * DT
32     week_price = prices[31:] - prices[24:-7]
33     week_net = net_real[31:] - net_real[24:-7]
34     week_dependence = _dependence(week_net, week_price)
35     generator = np.random.default_rng(seed)
36     bootstrap_rows = []
37     for _ in range(bootstrap):
38         chosen = generator.integers(0, len(week_price), len(week_price))
39         item = _dependence(week_net[chosen], week_price[chosen])
40         bootstrap_rows.append((item["correlation"], item["price_upper_tail_given_net_upper_tail"]))
41     bootstrap_values = np.asarray(bootstrap_rows)
42     pv_mae = {}
43     actual_flat = data.q3.actual.reshape(-1, 2)
44     for hour in (0, 6, 12, 18):
45         errors = []
46         for day in range(365):
47             for horizon in range(1, 7):
48                 slot = day * 144 + (hour + horizon) * 6 - 1
49                 if slot < len(actual_flat):
50                     errors.append(abs(data.q3.hourly[day, hour // 6, horizon - 1] - actual_flat[slot
51 , 1]))
52         pv_mae[str(hour)] = float(np.mean(errors))
53     oos_net, oos_price = [], []
54     for day in range(31, 365):
55         oos_net.append(net_real[day] - data.point_net(day, 0, 144))
```

```

55     oos_price.append(data.price.residuals[day])
56 oos_dependence = _dependence(np.asarray(oos_net), np.asarray(oos_price))
57 result = {
58     "price": {
59         "minimum": float(flat.min()), "maximum": float(flat.max()), "mean": float(flat.mean()),
60         "daily_mean_trend_r2": float(r2),
61         "lag1_same_slot_correlation": float(np.corrcoef(prices[1:].ravel(), prices[:-1].ravel())
62 [0, 1]),
63         "lag7_same_slot_correlation": float(np.corrcoef(prices[7:].ravel(), prices[:-7].ravel())
64 [0, 1]),
65         "lag14_same_slot_correlation": float(np.corrcoef(prices[14:].ravel(), prices[:-14].ravel()
66 ())[0, 1]),
67         "profile_min_slot": int(np.argmin(profile)), "profile_min_price": float(profile.min()),
68         "profile_max_slot": int(np.argmax(profile)), "profile_max_price": float(profile.max()),
69         "forecast": data.price.audit,
70     },
71     "seven_day_anomaly_dependence": week_dependence,
72     "block_bootstrap_95": {
73         "correlation": np.quantile(bootstrap_values[:, 0], [0.025, 0.975]).tolist(),
74         "upper_tail_conditional_probability": np.quantile(bootstrap_values[:, 1], [0.025,
75 0.975]).tolist(),
76         "repetitions": bootstrap,
77     },
78     "pv_committed_6h_mae_kw": pv_mae,
79     "oos_residual_dependence": oos_dependence,
80 }
81 if with_lightgbm:
82     result["price"]["lightgbm_comparison"] = rolling_lightgbm_comparison(prices)
83     output.mkdir(parents=True, exist_ok=True)
84     write_json(output / "diagnostics.json", result)
85     write_csv(output / "price_daily.csv", pd.DataFrame({"day": np.arange(365), "mean_price": daily})
86 )
87 return result
88
89
90 def experiment_variants(base: Config) -> dict[str, Config]:
91     """7.8规定的消融、半径/场景/权重/窗口敏感性与四套更新时间集合。"""
92     variants = {
93         "independent_empirical": replace(base, joint=False, dro_scale=0),
94         "joint_empirical": replace(base, joint=True, dro_scale=0),
95         "joint_dro": base,
96         "q43_unconditional": replace(base, conditional=False),
97         "q43_conditional": replace(base, conditional=True),
98     }
99     variants.update({f"radius_{scale:g}": replace(base, dro_scale=scale) for scale in (0.75, 1.0,
100 1.25)})
101     variants.update({f"scenarios_{count}": replace(base, scenarios=count) for count in (10, 20, 30)
102 })
103     variants.update({f"net_weight_{weight:g}": replace(base, net_weight=weight) for weight in (0.25,
104 0.5, 0.75)})
105     variants.update({f>window_{window}": replace(base, residual_window=window) for window in (56,
106 84, 112)})
107     variants.update({
108         "P0_0": replace(base, nodes=(0,)),
109         "P1_06": replace(base, nodes=(0, 6)),
110         "P2_0612": replace(base, nodes=(0, 6, 12)),
111         "P3_061218": replace(base, nodes=(0, 6, 12, 18)),
112     })

```

```

103     })
104     return variants
105
106
107 def write_experiment_plan(base: Config, output: Path) -> None:
108     write_json(output / "experiment_plan.json", {name: asdict(config) for name, config in
109         experiment_variants(base).items()})
110
111 def write_window_policy(output: Path) -> dict:
112     """56日为正式计算前预设，无窗口比较实验。"""
113     result = {"selected_window": 56, "selection": "preset_before_formal_run", "calibration_performed": False}
114     write_json(output / "window_policy.json", result)
115     return result
116
117
118 def baseline_metrics(data: Inputs) -> dict:
119     """冻结Q2实际策略按附件4逐时重新结算，不重选、不优化。"""
120     import hashlib
121     source = Path(data.provenance['q2_run']) / 'processed/main/dispatch.csv'
122     frame = pd.read_csv(source, float_precision='round_trip')
123     prices = data.actual_prices[31:].ravel()
124     expected = pd.date_range('2025-02-01 00:10', periods=334 * 144, freq='10min')
125     if len(frame) != len(prices) or not np.array_equal(pd.to_datetime(frame.timestamp), expected):
126         raise ValueError('Q2冻结基准时序不完整')
127     return dict(source=str(source), sha256=hashlib.sha256(source.read_bytes()).hexdigest(),
128                C_realized=float(np.sum(prices * (frame.grid + 5 * frame.emergency))),
129                E_emergency=float(frame.emergency.sum()), grid=float(frame.grid.sum()),
130                initial_soc=float(frame.initial_soc.iloc[0]), method='frozen_Q2_policy_repriced_attachment4')
131
132
133 def sanity_metrics(frame: pd.DataFrame, baseline_cost: float) -> dict:
134     from .config import E_MIN, E_MAX, TOL
135     cost = float(frame.total_cost.sum())
136     upper = float(np.mean(frame.soc >= E_MAX - TOL))
137     return dict(C_realized=cost, baseline_cost=baseline_cost, cost_ratio=cost / baseline_cost,
138                economic_pass=cost <= 1.15 * baseline_cost, soc_upper_fraction=upper,
139                soc_lower_fraction=float(np.mean(frame.soc <= E_MIN + TOL)),
140                storage_pass=upper < 0.99, soc_min=float(frame.soc.min()), soc_max=float(frame.soc.
141                max()),
142                soc_final=float(frame.soc.iloc[-1]), E_emergency=float(frame.emergency.sum()),
143                grid=float(frame.grid.sum()), called=float(frame.called.sum()), unused=float(frame.
144                unused.sum()),
145                unused_ratio=float(frame.unused.sum() / max(1.0, frame.grid.sum())),
146                charge=float(frame.charge.sum()), discharge=float(frame.discharge.sum()),
147                storage_throughput=float(frame.charge.sum() + frame.discharge.sum()),
148                spill=float(frame.spill.sum()))
149
150 def production_gates(data: Inputs, frame: pd.DataFrame, mode: str, config: Config, output: Path) -> dict:
151     import hashlib
152     import json
153     from .rolling import validate_frame
154     validate_frame(frame, mode, config, full=True)

```

```

154     if config.allow_limited or abs(float(frame.initial_soc.iloc[0]) - 6000) > 1e-6:
155         raise ValueError('正式解设置或初始SOC不符')
156     state = json.loads((output / 'state.json').read_text(encoding='utf-8'))
157     audits = []
158     if [item['day'] for item in state['days']] != list(range(31, 365)):
159         raise ValueError('正式334日receipt不完整')
160     for receipt in state['days']:
161         date = str((pd.Timestamp('2025-01-01') + pd.Timedelta(days=receipt['day'])).date())
162         for key, path in [('dispatch', output / 'days' / f'{date}.csv'), ('audit', output / 'audit'
163 / f'{date}.json')]:
164             if hashlib.sha256(path.read_bytes()).hexdigest() != receipt['hashes'][key]:
165                 raise ValueError('日文件SHA不匹配')
166             day_audit = json.loads((output / 'audit' / f'{date}.json').read_text(encoding='utf-8'))
167             audits.extend([day_audit['solver']] if mode == '4-2' else [n['solver'] for n in day_audit['
168 nodes']])
169     expected = 334 if mode == '4-2' else 1336
170     certified = sum(a['status'] == 'Optimal' and a['reliable'] and a['solution_source'] == '
171 optimal_solver'
172 and a['integer_variables'] == 0 for a in audits)
173     baseline = baseline_metrics(data)
174     metrics = sanity_metrics(frame, baseline['C_realized'])
175     result = dict(mode=mode, baseline=baseline, metrics=metrics, solver_nodes=len(audits),
176 certified_nodes=certified, limited_nodes=sum(a['status'] != 'Optimal' for a in
177 audits),
178 fallback_nodes=sum(a['solution_source'] != 'optimal_solver' for a in audits),
179 max_balance_error=float(abs(frame.called + frame.discharge + frame.emergency -
180 frame.net_kwh - frame.charge - frame.spill).max()),
181 max_solver_seconds=max(a['wall_seconds'] for a in audits),
182 mean_solver_seconds=float(np.mean([a['wall_seconds'] for a in audits])),
183 variables_range=[min(a['variables'] for a in audits), max(a['variables'] for a in
184 audits)],
185 constraints_range=[min(a['constraints'] for a in audits), max(a['constraints'] for
186 a in audits)],
187 dispatch_sha256=hashlib.sha256((output / 'dispatch.csv').read_bytes()).hexdigest()
188 )
189     result['passed'] = bool(certified == expected == len(audits) and metrics['economic_pass'] and
190 metrics['storage_pass'])
191     write_json(output / 'production_gates.json', result)
192     if not result['passed']:
193         write_json(output / 'active.json', {'status': 'publication_blocked', 'gates': result})
194         raise ValueError('正式发布门禁失败, 详见production_gates.json')
195     return result

```

src/q4/export.py

```

1  """7.8: 真实费用表和result4-2/result4-3模板回填、回读验收。"""
2  from copy import copy
3  from pathlib import Path
4  import os
5  import numpy as np
6  import pandas as pd
7  from openpyxl import load_workbook
8  from openpyxl.comments import Comment
9  from .config import ATTACHMENTS, Config, write_csv, write_json
10 from .rolling import validate_frame
11

```

```

12
13 def intervals() -> list[str]:
14     return [f"{slot // 6:02d}:{slot % 6 * 10:02d}-{(slot + 1) // 6:02d}:{(slot + 1) % 6 * 10:02d}"
15             for slot in range(144)]
16
17
18 def emergency_events(frame: pd.DataFrame) -> pd.DataFrame:
19     rows, current = [], None
20     for row in frame.itertuples():
21         end = pd.Timestamp(row.timestamp)
22         start = end - pd.Timedelta(minutes=10)
23         if row.emergency > 0:
24             if current is None or current["end"] != start or current["date"] != str(start.date()):
25                 if current is not None:
26                     rows.append(current)
27                 current = {"date": str(start.date()), "start": start, "end": end,
28                           "energy_kwh": 0.0, "cost_yuan": 0.0}
29             current["end"] = end
30             current["energy_kwh"] += row.emergency
31             current["cost_yuan"] += row.emergency_cost
32         elif current is not None:
33             rows.append(current)
34             current = None
35     if current is not None:
36         rows.append(current)
37     for row in rows:
38         row["interval"] = f"{row['start']:%H:%M}-{row['end']:%H:%M}"
39     return pd.DataFrame(rows, columns=["date", "start", "end", "interval", "energy_kwh", "cost_yuan"
40 ])
41
42 def summaries(frame: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
43     daily, blocks = [], []
44     for date, day in frame.groupby("date", sort=False):
45         daily.append({"date": date, **{key: float(day[key].sum()) for key in
46             ("g0", "grid", "planned_cost", "adjustment_cost", "emergency_cost", "total_cost",
47             "emergency", "charge", "discharge", "spill", "unused")},
48             "cost_q95_component": float(day.total_cost.sum()),
49             "soc_start": float(day.initial_soc.iloc[0]), "soc_end": float(day.soc.iloc
50 [-1])})
51         for block in range(6):
52             part = day.iloc[block * 24:(block + 1) * 24]
53             blocks.append({"date": date, "interval": f"{block * 4:02d}:00-{(block + 1) * 4:02d}:00",
54                 "charge": float(part.charge.sum()), "discharge": float(part.discharge.sum
55 ()),
56                 "soc_start": float(day.initial_soc.iloc[0]), "soc_end": float(day.soc.
57 iloc[-1])})
58     return pd.DataFrame(daily), pd.DataFrame(blocks)
59
60 def write_tables(frame: pd.DataFrame, output: Path) -> None:
61     daily, blocks = summaries(frame)
62     write_csv(output / "daily.csv", daily)
63     write_csv(output / "storage_blocks.csv", blocks)
64     write_csv(output / "emergency_events.csv", emergency_events(frame))
65     metrics = {

```

```

64     "C_realized": float(frame.total_cost.sum()),
65     "E_emergency": float(frame.emergency.sum()),
66     "Q_0.95_daily_cost": float(daily.total_cost.quantile(0.95)),
67     "adjustment_energy": float(abs(frame.grid - frame.g0).sum()),
68     "price_MAE": float(abs(frame.price - frame.price_forecast).mean()),
69     "price_RMSE": float(np.sqrt(np.mean((frame.price - frame.price_forecast) ** 2))),
70 }
71 write_json(output / "metrics.json", metrics)
72
73
74 def export_workbook(frame: pd.DataFrame, output: Path, mode: str, config: Config, *, smoke: bool =
    False) -> dict:
75     validate_frame(frame, mode, config, full=not smoke)
76     if not smoke:
77         import hashlib
78         import json
79         gate_path = output / 'production_gates.json'
80         if not gate_path.exists():
81             raise ValueError('正式Excel必须先通过生产门禁')
82         gates = json.loads(gate_path.read_text(encoding='utf-8'))
83         if not gates['passed'] or gates['mode'] != mode or gates['dispatch_sha256'] != hashlib.
            sha256((output / 'dispatch.csv').read_bytes()).hexdigest():
84             raise ValueError('生产门禁失败或验证后数据变动')
85         expected = ([ "计划购电量", "充放电量", "紧急购电量" ] if mode == "4-2" else
86                     [ "计划购电量", "调整购电量", "充放电量", "紧急购电量" ])
87         template = ATTACHMENTS / "附件5" / f"result{mode}.xlsx"
88         workbook = load_workbook(template)
89         if workbook.sheetnames != expected:
90             raise ValueError("问题四模板工作表结构发生变化")
91         styles = {sheet.title: [copy(cell._style) for cell in sheet[2]] for sheet in workbook}
92         for sheet in workbook:
93             for merged in list(sheet.merged_cells.ranges):
94                 sheet.unmerge_cells(str(merged))
95                 sheet.delete_rows(2, sheet.max_row)
96         quantity_sheets = [(workbook["计划购电量"], "grid" if mode == "4-2" else "g0", "planned_cost")]
97         if mode == "4-3":
98             quantity_sheets.append((workbook["调整购电量"], "grid", "adjustment_cost"))
99         for sheet, _, _ in quantity_sheets:
100             for column, label in enumerate(intervals(), 2):
101                 sheet.cell(1, column, label)
102                 sheet.cell(1, 2).comment = Comment("按10分钟右端点纠正为00:00—24:00共144段。", "Q4")
103             for row_index, (date, day) in enumerate(frame.groupby("date", sort=False), 2):
104                 for sheet, key, cost in quantity_sheets:
105                     sheet.cell(row_index, 1, pd.Timestamp(date).to_pydatetime())
106                     for column, value in enumerate(day[key], 2):
107                         sheet.cell(row_index, column, float(value))
108                     sheet.cell(row_index, 146, float(day[key].sum()))
109                     sheet.cell(row_index, 147, float(day[cost].sum()))
110             if mode == "4-3":
111                 workbook["调整购电量"].cell(1, 147).comment = Comment(
112                     "净调整费=Σp[1.5增购量-0.5减购量]：数量列为最终生效绝对购电量。", "Q4"
113                 )
114     _, blocks = summaries(frame)
115     storage = workbook["充放电量"]
116     for row_index, row in enumerate(blocks.itertuples(index=False), 2):
117         block = (row_index - 2) % 6
118         values = (pd.Timestamp(row.date).to_pydatetime() if block == 0 else None, row.interval,

```



```

119         row.charge, row.discharge, "0:00" if block == 0 else "24:00" if block == 1 else
None,
120         row.soc_start if block == 0 else row.soc_end if block == 1 else None)
121     for column, value in enumerate(values, 1):
122         storage.cell(row_index, column, value)
123 events = emergency_events(frame)
124 emergency = workbook["紧急购电量"]
125 for row_index, row in enumerate(events.itertuples(index=False), 2):
126     for column, value in enumerate((pd.Timestamp(row.date).to_pydatetime(), row.interval, row.
energy_kwh), 1):
127         emergency.cell(row_index, column, value)
128 for sheet in workbook:
129     sheet.freeze_panes = "B2" if sheet in [item[0] for item in quantity_sheets] else "A2"
130     style = styles[sheet.title]
131     for row in sheet.iter_rows(min_row=2):
132         for cell in row:
133             if cell.column <= len(style):
134                 cell._style = copy(style[cell.column - 1])
135             if isinstance(cell.value, (int, float)):
136                 cell.number_format = "0.000000"
137 output.mkdir(parents=True, exist_ok=True)
138 filename = f"smoke_result{mode}.xlsx" if smoke else f"result{mode}.xlsx"
139 temporary = output / (filename + ".pending.xlsx")
140 workbook.save(temporary)
141 workbook.close()
142 check = load_workbook(temporary, read_only=True, data_only=True)
143 try:
144     for sheet, key, cost in [(item[0].title, item[1], item[2]) for item in quantity_sheets]:
145         saved = list(check[sheet].values)[1:]
146         np.testing.assert_allclose(np.asarray([row[1:145] for row in saved], float).ravel(),
frame[key], atol=1e-8)
147         expected_totals = frame.groupby("date", sort=False)[[key, cost]].sum().to_numpy()
148         np.testing.assert_allclose(np.asarray([row[145:147] for row in saved], float),
expected_totals, atol=1e-8)
149         saved_blocks = list(check["充放电量"].values)[1:]
150         np.testing.assert_allclose(np.asarray([row[2:4] for row in saved_blocks], float),
blocks[["charge", "discharge"]], atol=1e-8)
151         saved_events = list(check["紧急购电量"].values)[1:]
152         np.testing.assert_allclose(np.asarray([row[2] for row in saved_events], float),
events.energy_kwh.to_numpy(float), atol=1e-8)
153
154 finally:
155     check.close()
156 with temporary.open("r+b") as stream:
157     os.fsync(stream.fileno())
158     temporary.replace(output / filename)
159 audit = {"mode": mode, "smoke_only": smoke, "days": int(frame.date.nunique()),
"roundtrip_all_sheets": True, "adjusted_quantity": "final_effective_absolute"}
160 write_json(output / f"template_audit_{mode}.json", audit)
161 if not smoke:
162     write_json(output / "active.json", {"status": "complete", "mode": mode, "completed_days":
334, "workbook": filename})
163 return audit
164
165

```

src/q4/optimization.py

```

1 """节点共享储能计划的连续LP：有限支持Wasserstein直接对偶，无MIP或fallback。"""

```

```

2 from dataclasses import dataclass
3 import hashlib
4 import multiprocessing as mp
5 import time
6 import traceback
7 import numpy as np
8 from highs import Highs, HighsLp, HighsModelStatus, HighsStatus, MatrixFormat
9 from scipy.optimize import Bounds, LinearConstraint, linprog
10 from scipy.sparse import coo_matrix
11 from .config import CAP, E_MIN, E_MAX, ETA_C, ETA_D, Config, TOL, THROUGHPUT_EPSILON
12 from .physics import DispatchPolicy, replay, scenario_costs
13 from .scenarios import Scenarios
14
15
16 class LimitedSolve(RuntimeError):
17     def __init__(self, audit: dict):
18         super().__init__(f"节点未取得可信LP最优解: {audit.get('status')}")
19         def finite_json(value):
20             if isinstance(value, dict):
21                 return {key: finite_json(item) for key, item in value.items()}
22             if isinstance(value, (list, tuple)):
23                 return [finite_json(item) for item in value]
24             if isinstance(value, (float, np.floating)) and not np.isfinite(value):
25                 return None
26             return value
27         self.audit = finite_json(audit)
28
29
30 @dataclass
31 class Solution:
32     policy: DispatchPolicy
33     responses: list[dict]
34     audit: dict
35
36
37 def worst_case_distribution(costs: np.ndarray, weights: np.ndarray, distance: np.ndarray,
38                             radius: float) -> tuple[np.ndarray, float]:
39     count = len(weights)
40     columns = np.arange(count * count)
41     equalities = coo_matrix((np.ones(count * count), (np.repeat(np.arange(count), count), columns)),
42                             shape=(count, count * count)).tocsr()
43     result = linprog(-np.tile(costs, count), A_ub=distance.ravel()[None], b_ub=[radius],
44                     A_eq=equalities, b_eq=weights, bounds=(0, None), method='highs')
45     if not result.success:
46         raise RuntimeError(f'最坏分布运输问题失败: {result.message}')
47     distribution = result.x.reshape(count, count).sum(axis=0)
48     return distribution, float(distribution @ costs)
49
50
51 def build_matrix(scenarios: Scenarios, initial: float, *, reference: np.ndarray | None = None,
52                 today: int | None = None) -> tuple:
53     net, prices, weights = scenarios.net, scenarios.prices, scenarios.weights
54     if net.ndim != 2 or prices.shape != net.shape or weights.shape != (len(net),):
55         raise ValueError('联合场景维度不符')
56     count, length = net.shape
57     if any(not np.isfinite(v).all() for v in (net, prices, weights, scenarios.distance)):
58         raise ValueError('联合场景含非有限值')

```

```

59     if (prices < 0).any() or (weights <= 0).any() or not np.isclose(weights.sum(), 1):
60         raise ValueError('价格或概率非法')
61     if not E_MIN - TOL <= initial <= E_MAX + TOL or scenarios.radius < 0 or not np.isfinite(
        scenarios.radius):
62         raise ValueError('初始SOC或半径非法')
63     if scenarios.distance.shape != (count, count) or (scenarios.distance < 0).any():
64         raise ValueError('距离矩阵非法')
65     today = length if today is None else min(int(today), length)
66     if reference is not None and (len(reference) < today or not np.isfinite(reference).all() or (
        reference < 0).any()):
67         raise ValueError('基准购电计划非法')
68     indices, lower, upper = {}, [], []
69
70     def block(name, shape, lo=0.0, hi=np.inf):
71         size = int(np.prod(shape))
72         indices[name] = np.arange(len(lower), len(lower) + size).reshape(shape)
73         lower.extend(np.broadcast_to(lo, shape).ravel())
74         upper.extend(np.broadcast_to(hi, shape).ravel())
75         return indices[name]
76
77     g = block('g', (length,), hi=np.maximum(net.max(axis=0), 0) + CAP)
78     c = block('c', (length,), hi=CAP)
79     r = block('r', (length,), hi=CAP)
80     e = block('E', (length,), lo=E_MIN, hi=E_MAX)
81     x = block('x', (count, length))
82     h = block('h', (count, length))
83     w = block('w', (count, length))
84     adjustment = block('adjustment', (today if reference is not None else 0),
85                        lo=-0.5 * reference[:today] if reference is not None else 0)
86     phi = block('phi', (count,), lo=-np.inf)
87     lam = block('wasserstein_lambda', (1,))
88     alpha = block('alpha', (count,), lo=-np.inf)
89     objective = np.zeros(len(lower))
90     objective[c] = objective[r] = THROUGHPUT_EPSILON
91     objective[lam] = scenarios.radius
92     objective[alpha] = weights
93     rows, columns, values, row_lower, row_upper = [], [], [], [], []
94
95     def add(terms, lo=-np.inf, hi=np.inf):
96         row = len(row_lower)
97         for column, value in terms:
98             if value != 0:
99                 rows.append(row); columns.append(int(column)); values.append(float(value))
100             row_lower.append(float(lo)); row_upper.append(float(hi))
101
102     for t in range(length):
103         add([(c[t], 1), (r[t], 1)], hi=CAP)
104         terms = [(e[t], 1), (c[t], -ETA_C), (r[t], 1 / ETA_D)]
105         if t:
106             terms.append((e[t - 1], -1))
107         rhs = initial if t == 0 else 0
108         add(terms, lo=rhs, hi=rhs)
109         for j in range(count):
110             add([(x[j, t], 1), (g[t], -1)], hi=0)
111             add([(x[j, t], 1), (r[t], 1), (h[j, t], 1), (c[t], -1), (w[j, t], -1)],
112                  lo=net[j, t], hi=net[j, t])
113     if reference is not None:

```

```

114         for t in range(today):
115             for factor in (0.5, 1.5):
116                 add([(g[t], factor), (adjustment[t], -1)], hi=factor * reference[t])
117     for j in range(count):
118         terms = [(phi[j], -1)] + [(h[j, t], 5 * prices[j, t]) for t in range(length)]
119         constant = 0.0
120         for t in range(length):
121             if reference is not None and t < today:
122                 terms.append((adjustment[t], prices[j, t]))
123                 constant += reference[t] * prices[j, t]
124             else:
125                 terms.append((g[t], prices[j, t]))
126         add(terms, hi=-constant)
127     for i in range(count):
128         for j in range(count):
129             add([(phi[j], 1), (alpha[i], -1), (lam[0], -scenarios.distance[i, j])], hi=0)
130     matrix = coo_matrix((values, (rows, columns)), shape=(len(row_lower), len(lower))).tocsc()
131     return (objective, Bounds(np.asarray(lower), np.asarray(upper)), np.zeros(len(lower), dtype=int)
132         ,
133         LinearConstraint(matrix, np.asarray(row_lower), np.asarray(row_upper)), indices, 0.0)
134
135 def dual_bound(row_dual, col_dual, constraints, bounds):
136     """有限界才进入对偶内积；无限界处只允许solver容差内的舍入残差。"""
137     value, invalid_multiplier = 0.0, 0.0
138     for multipliers, lo, hi in ((row_dual, constraints.lb, constraints.ub),
139                                (col_dual, bounds.lb, bounds.ub)):
140         positive, negative = multipliers > 0, multipliers < 0
141         invalid = (positive & ~np.isfinite(lo)) | (negative & ~np.isfinite(hi))
142         invalid_multiplier = max(invalid_multiplier, float(np.max(abs(multipliers[invalid]), initial=0)))
143         pos = positive & np.isfinite(lo)
144         neg = negative & np.isfinite(hi)
145         value += float(multipliers[pos] @ lo[pos] + multipliers[neg] @ hi[neg])
146     return value, invalid_multiplier
147
148
149 def solve_dual(scenarios: Scenarios, initial: float, config: Config, *, reference=None, today=None)
150     -> Solution:
151     """worker内求解；只返回Optimal且原矩阵、真实回放、DRO原对偶均通过的解。"""
152     started = time.perf_counter()
153     objective, bounds, integer, constraints, indices, offset = build_matrix(
154         scenarios, initial, reference=reference, today=today)
155     solver = Highs()
156     for key, value in dict(threads=config.solver_threads, output_flag=False, time_limit=config.seconds,
157                             primal_feasibility_tolerance=1e-8, dual_feasibility_tolerance=1e-8).items():
158         if solver.setOptionValue(key, value) == HighsStatus.kError:
159             raise ValueError(f'HiGHS不接受参数{key}')
160     matrix = constraints.A
161     model = HighsLp()
162     model.num_col_, model.num_row_ = len(objective), matrix.shape[0]
163     model.col_cost_, model.col_lower_, model.col_upper_ = objective, bounds.lb, bounds.ub
164     model.row_lower_, model.row_upper_ = constraints.lb, constraints.ub
165     model.a_matrix_.format_ = MatrixFormat.kColwise
166     model.a_matrix_.num_col_, model.a_matrix_.num_row_ = model.num_col_, model.num_row_

```

```

166 model.a_matrix_.start_, model.a_matrix_.index_, model.a_matrix_.value_ = matrix.indptr, matrix.
    indices, matrix.data
167 if solver.passModel(model) == HighsStatus.kError or solver.run() == HighsStatus.kError:
168     raise LimitedSolve(dict(status='solver_error', reliable=False, solution_source='none'))
169 status = solver.getModelStatus()
170 if status != HighsModelStatus.kOptimal:
171     raise LimitedSolve(dict(status=solver.modelStatusToString(status), reliable=False,
172                             solution_source='none', seconds=time.perf_counter() - started))
173 vector = np.asarray(solver.getSolution().col_value)
174 activity = matrix @ vector
175 feasibility = max(0.0, float(np.max(bounds.lb - vector)), float(np.max(vector - bounds.ub)),
176                   float(np.max(constraints.lb - activity)), float(np.max(activity - constraints.
    ub)))
177 policy = DispatchPolicy(*(np.maximum(vector[indices[key]], 0) for key in ('g', 'c', 'r')))
178 # 保持SOC完全相同，消除数值退化造成的同时充放电；需求只能减少。
179 overlap = np.minimum(policy.charge, policy.discharge / (ETA_C * ETA_D))
180 policy.charge -= overlap
181 policy.discharge -= ETA_C * ETA_D * overlap
182 responses = [replay(policy, path, initial) for path in scenarios.net]
183 costs = scenario_costs(policy, responses, scenarios.prices, reference, today)
184 worst_weights, robust_value = worst_case_distribution(costs, scenarios.weights, scenarios.
    distance, scenarios.radius)
185 raw_value = float(objective @ vector + offset)
186 value = robust_value + THROUGHPUT_EPSILON * float(np.sum(policy.charge + policy.discharge))
187 duality_error = abs(value - raw_value)
188 # 使用行、列对偶重构LP下界，而不是MILP专有字段。
189 dual = solver.getSolution()
190 row_dual, col_dual = np.asarray(dual.row_dual), np.asarray(dual.col_dual)
191 lower_bound, invalid_multiplier = dual_bound(row_dual, col_dual, constraints, bounds)
192 info = solver.getInfo()
193 dual_feasibility = max(invalid_multiplier, float(info.max_dual_infeasibility))
194 certificate_error = abs(raw_value - lower_bound)
195 valid = bool(feasibility <= TOL and duality_error <= 2e-4 and certificate_error <= 2e-4 and
    dual_feasibility <= 1e-8)
196 digest = hashlib.sha256()
197 for array in (objective, bounds.lb, bounds.ub, constraints.lb, constraints.ub, matrix.indptr,
    matrix.indices, matrix.data):
198     digest.update(np.asarray(array).tobytes())
199 audit = dict(status='Optimal', solver='HiGHS_finite_support_Wasserstein_LP', solver_version=
    solver.version(),
200               seconds=time.perf_counter() - started, solver_seconds=solver.getRunTime(),
201               solver_threads=config.solver_threads, solution_source='optimal_solver',
202               gap=max(0.0, value - lower_bound) / max(1.0, abs(value)), objective=robust_value,
203               optimizer_objective=raw_value, lower_bound=lower_bound, variables=len(objective),
204               constraints=matrix.shape[0], integer_variables=int(integer.sum()), binaries=0,
205               linear_feasibility_error=feasibility, replay_duality_error=duality_error,
206               lp_certificate_error=certificate_error, dual_feasibility_error=dual_feasibility,
207               native_primal_dual_objective_error=float(info.primal_dual_objective_error),
208               simultaneous_charge_removed=float(overlap.sum()),
209               storage_schedule='node_shared_fixed_committed_block',
210               dro_formulation='finite_support_direct_dual', dro_coupling_constraints=len(costs)
    **2,
211               throughput_epsilon=THROUGHPUT_EPSILON, scenario_costs=costs.tolist(),
212               empirical_weights=scenarios.weights.tolist(), worst_case_weights=worst_weights.
    tolist(),
213               radius=scenarios.radius, matrix_digest=digest.hexdigest(), valid=valid, reliable=
    valid)

```

```

214     if not valid:
215         raise LimitedSolve(audit)
216     return Solution(policy, responses, audit)
217
218
219 def _worker(connection, scenarios, initial, config, reference, today):
220     try:
221         connection.send(('ok', solve_dual(scenarios, initial, config, reference=reference, today=
today)))
222     except LimitedSolve as error:
223         connection.send(('failure', error.audit))
224     except Exception:
225         connection.send(('failure', dict(status='worker_exception', reliable=False, traceback=
traceback.format_exc()))))
226     finally:
227         connection.close()
228
229
230 def solve(scenarios: Scenarios, initial: float, config: Config, *, reference=None, today=None) ->
Solution:
231     """独立进程硬截止；任何失败都不能进入正式日文件。"""
232     context = mp.get_context('spawn')
233     parent, child = context.Pipe(duplex=False)
234     process = context.Process(target=_worker, args=(child, scenarios, initial, config, reference,
today))
235     started = time.perf_counter()
236     process.start()
237     child.close()
238     try:
239         if not parent.poll(config.hard_timeout):
240             raise LimitedSolve(dict(status='hard_timeout', reliable=False, solution_source='none',
hard_timeout=config.hard_timeout))
241
242         try:
243             status, result = parent.recv()
244         except EOFError as error:
245             raise LimitedSolve(dict(status='worker_exit', reliable=False, solution_source='none'))
246     from error
247     if status != 'ok':
248         raise LimitedSolve(result)
249     result.audit['wall_seconds'] = time.perf_counter() - started
250     result.audit['hard_timeout'] = config.hard_timeout
251     return result
252 finally:
253     parent.close()
254     process.join(timeout=0.2)
255     if process.is_alive():
256         process.kill()
257         process.join()

```

src/q4/physics.py

```

1 """节点预提交储能计划；结算额度、实际调用和紧急购电分别核算。"""
2 from dataclasses import dataclass
3 import numpy as np
4 from .config import CAP, E_MIN, E_MAX, ETA_C, ETA_D, TOL
5

```

```

6
7 @dataclass
8 class DispatchPolicy:
9     grid: np.ndarray
10    charge: np.ndarray
11    discharge: np.ndarray
12
13    def part(self, start: int, end: int) -> 'DispatchPolicy':
14        return DispatchPolicy(*(value[start:end].copy() for value in
15                                (self.grid, self.charge, self.discharge)))
16
17
18 def adjustment_quantity(grid: np.ndarray, reference: np.ndarray) -> np.ndarray:
19     difference = np.asarray(grid) - np.asarray(reference)
20     return 1.5 * np.maximum(difference, 0) - 0.5 * np.maximum(-difference, 0)
21
22
23 def replay(policy: DispatchPolicy, net: np.ndarray, initial: float) -> dict[str, np.ndarray]:
24     net = np.asarray(net, dtype=float)
25     grid, charge, discharge = (np.asarray(v, dtype=float) for v in
26                                (policy.grid, policy.charge, policy.discharge))
27     if any(v.shape != net.shape or not np.isfinite(v).all() for v in (grid, charge, discharge, net)):
28         raise ValueError('储能计划与净负荷维度或数值非法')
29     if min(grid.min(), charge.min(), discharge.min()) < -TOL or np.max(charge + discharge) > CAP +
30        TOL:
31         raise ValueError('购电非负约束或储能功率约束违反')
32     soc = initial + np.cumsum(ETA_C * charge - discharge / ETA_D)
33     if min(initial, soc.min()) < E_MIN - TOL or max(initial, soc.max()) > E_MAX + TOL:
34         raise ValueError('SOC越界')
35     balance_need = net + charge - discharge
36     called = np.minimum(grid, np.maximum(balance_need, 0))
37     unused = grid - called
38     emergency = np.maximum(balance_need - called, 0)
39     spill = np.maximum(called - balance_need, 0)
40     np.testing.assert_allclose(called + discharge + emergency, net + charge + spill, atol=TOL, rtol=0)
41     return dict(called=called, unused=unused, charge=charge.copy(), discharge=discharge.copy(),
42                emergency=emergency, spill=spill, soc=soc, total_surplus=unused + spill)
43
44 def scenario_costs(policy: DispatchPolicy, responses: list[dict], prices: np.ndarray,
45                   reference: np.ndarray | None = None, today: int | None = None) -> np.ndarray:
46     prices = np.asarray(prices, dtype=float)
47     today = prices.shape[1] if today is None else min(int(today), prices.shape[1])
48     quantity = policy.grid.copy()
49     if reference is not None:
50         quantity[:, today] = reference[:, today] + adjustment_quantity(policy.grid[:, today], reference[:,
51         today])
52     return np.asarray([np.sum(p * (quantity + 5 * response['emergency']))
53                        for p, response in zip(prices, responses)])

```

src/q4/price.py

```

1 """7.3: 共享参数 Elastic Net 周季节残差的严格滚动预测。"""
2 from dataclasses import dataclass

```

```

3 import numpy as np
4 from sklearn.linear_model import ElasticNet
5 from sklearn.pipeline import make_pipeline
6 from sklearn.preprocessing import StandardScaler
7
8
9 HYPERPARAMETERS = tuple(
10     (alpha, ratio)
11     for alpha in (1e-4, 1e-3, 1e-2)
12     for ratio in (0.1, 0.5, 0.9)
13 )
14
15
16 def feature_matrix(history: np.ndarray, day: int) -> np.ndarray:
17     """所有列在目标日0:00前已知；允许用已递推预测日构造第二天特征。"""
18     if day < 14:
19         raise ValueError("Elastic Net 特征至少需要14日历史")
20     p1, p7, p8, p14 = history[day - 1], history[day - 7], history[day - 8], history[day - 14]
21     slots = np.arange(144, dtype=float)
22     weekday = day % 7
23     columns = (
24         p1,
25         p7,
26         p14,
27         p7 - p14,
28         p1 - p8,
29         np.roll(p7, 1) - p7,
30         np.roll(p7, -1) - p7,
31         np.full(144, p1.mean()),
32         np.full(144, p7.mean()),
33         np.full(144, p1.std()),
34         np.full(144, p7.std()),
35         np.sin(2 * np.pi * slots / 144),
36         np.cos(2 * np.pi * slots / 144),
37         np.sin(4 * np.pi * slots / 144),
38         np.cos(4 * np.pi * slots / 144),
39         np.full(144, np.sin(2 * np.pi * weekday / 7)),
40         np.full(144, np.cos(2 * np.pi * weekday / 7)),
41     )
42     result = np.column_stack(columns)
43     if not np.isfinite(result).all():
44         raise ValueError("价格特征含非有限值")
45     return result
46
47
48 def training_rows(prices: np.ndarray, start: int, end: int) -> tuple[np.ndarray, np.ndarray]:
49     days = range(max(14, start), min(end, len(prices)))
50     matrices = [feature_matrix(prices, day) for day in days]
51     if not matrices:
52         return np.empty((0, 17)), np.empty(0)
53     return np.vstack(matrices), np.concatenate([prices[day] - prices[day - 7] for day in days])
54
55
56 def fit(prices: np.ndarray, end: int, alpha: float, ratio: float):
57     x, y = training_rows(prices, 14, end)
58     if len(y) == 0:
59         return None

```



```

60     model = make_pipeline(
61         StandardScaler(),
62         ElasticNet(alpha=alpha, l1_ratio=ratio, max_iter=20000, tol=1e-6, selection="cyclic"),
63     )
64     model.fit(x, y)
65     return model
66
67
68 def tune_january(prices: np.ndarray) -> tuple[tuple[float, float], list[dict]]:
69     """1月内时间有序训练(14—23日)/验证(24—30日)，并列取更简单的较大alpha。"""
70     x_train, y_train = training_rows(prices, 14, 24)
71     x_valid, y_valid = training_rows(prices, 24, 31)
72     records = []
73     for alpha, ratio in HYPERPARAMETERS:
74         model = make_pipeline(
75             StandardScaler(),
76             ElasticNet(alpha=alpha, l1_ratio=ratio, max_iter=20000, tol=1e-6, selection="cyclic"),
77         )
78         model.fit(x_train, y_train)
79         error = model.predict(x_valid) - y_valid
80         records.append({"alpha": alpha, "l1_ratio": ratio, "mae": float(np.mean(abs(error))),
81                        "rmse": float(np.sqrt(np.mean(error ** 2)))})
82     best = min(records, key=lambda row: (row["rmse"], row["mae"], -row["alpha"], row["l1_ratio"]))
83     return (float(best["alpha"]), float(best["l1_ratio"])), records
84
85
86 @dataclass
87 class Forecasts:
88     values: np.ndarray # origin day × next 288 slots
89     residuals: np.ndarray # realized daily price - OOS first-day forecast
90     audit: dict
91
92
93 def rolling_forecasts(prices: np.ndarray) -> Forecasts:
94     prices = np.asarray(prices, dtype=float)
95     if prices.shape != (365, 144) or not np.isfinite(prices).all() or (prices <= 0).any():
96         raise ValueError("附件4价格必须为365×144个正有限值")
97     (alpha, ratio), tuning = tune_january(prices)
98     values = np.full((365, 288), np.nan)
99     residuals = np.full_like(prices, np.nan)
100     for origin in range(7, 365):
101         # 1月仅用于预热和超参数时序验证；不得用月底验证结果反向污染1月SOC轨迹。
102         model = fit(prices, origin, alpha, ratio) if origin >= 31 else None
103         extended = np.full((367, 144), np.nan)
104         extended[:origin] = prices[:origin]
105         for step in range(2):
106             target = origin + step
107             if target < 14 or model is None:
108                 prediction = extended[target - 7].copy()
109             else:
110                 prediction = extended[target - 7] + model.predict(feature_matrix(extended, target))
111             extended[target] = np.maximum(prediction, 0.0)
112             values[origin, step * 144:(step + 1) * 144] = extended[target]
113         residuals[origin] = prices[origin] - values[origin, :144]
114     formal = np.arange(31, 365)
115     error = residuals[formal]
116     seasonal = prices[formal - 7] - prices[formal]

```

```

117     audit = {
118         "information_boundary": "daily origin uses only days strictly before origin; day+1 is
recursive",
119         "feature_count": 17,
120         "selected_alpha": alpha,
121         "selected_ll_ratio": ratio,
122         "tuning": tuning,
123         "elastic_net_mae": float(np.mean(abs(error))),
124         "elastic_net_rmse": float(np.sqrt(np.mean(error ** 2))),
125         "seasonal_mae": float(np.mean(abs(seasonal))),
126         "seasonal_rmse": float(np.sqrt(np.mean(seasonal ** 2))),
127     }
128     return Forecasts(values, residuals, audit)
129
130
131 def rolling_lightgbm_comparison(prices: np.ndarray) -> dict:
132     """7.3.4同一rolling-origin边界下的严格因果LightGBM比较器；不进入正式调度。"""
133     from lightgbm import LGBMRegressor
134     predictions = np.full_like(prices, np.nan)
135     for origin in range(31, 365):
136         x, y = training_rows(prices, 14, origin)
137         model = LGBMRegressor(n_estimators=120, learning_rate=0.05, num_leaves=15,
138                               min_child_samples=40, subsample=1.0, colsample_bytree=1.0,
139                               reg_lambda=1e-3, random_state=20250913, n_jobs=1, verbosity=-1)
140         model.fit(x, y)
141         predictions[origin] = np.maximum(prices[origin - 7] + model.predict(feature_matrix(prices,
142         origin)), 0)
143     error = predictions[31:] - prices[31:]
144     return {"mae": float(np.mean(abs(error))), "rmse": float(np.sqrt(np.mean(error ** 2))),
145           "information_boundary": "same features; train target days strictly before each origin",
146           "used_for_dispatch": False}

```

src/q4/rescue/check_apr18.py

```

1  from pathlib import Path
2  import sys,json,time
3  root=Path(__file__).resolve().parents[3];sys.path.insert(0,str(root))
4  from src.q4.data import read_inputs
5  from src.q4.config import Config
6  from src.q4.rolling import run_day_q43
7  if __name__=='__main__':
8      data=read_inputs(root/'inputs/q4/rescue_processed')
9      state=json.loads((root/'outputs/q4/raw/rescue_lp_q43_20260913_v2/state.json').read_text())
10     started=time.perf_counter();frame,audit=run_day_q43(data,107,state['soc'],Config())
11     result={'seconds':time.perf_counter()-started,'nodes':[n['solver'] for n in audit['nodes']], '
day_total_cost':float(frame.total_cost.sum())}
12     assert all(n['reliable'] for n in result['nodes'])
13     assert result['nodes'][-1]['matrix_digest']=='
a87d00484bf5057d0e3595ed2dff1b1aa7998d3725df158f238249c09431648d'
14     (root/'docs/4/rescue_lp/apr18_verified.json').write_text(json.dumps(result,ensure_ascii=False,
indent=2),encoding='utf-8')
15     print('Apr18 four nodes all Optimal; same failed-node matrix; seconds',result['seconds'])

```

src/q4/rescue/launch_full.py

```

1 """仅启动已通过测试的v2全量运行：新目录，独立日志，保留依赖与源码快照。"""
2 from pathlib import Path
3 import subprocess,os,json,hashlib,shutil,sys,shlex
4 root=Path(__file__).resolve().parents[3]
5 sys.path.insert(0,str(root))
6 from src.q4.config import Config,production_sources,SOURCE,MODEL_VERSION
7 cfg=Config();saved=root/'docs/4/rescue_lp/frozen_source_v2'
8 if 'OK' not in (root/'docs/4/rescue_lp/tests_all_v2.txt').read_text(encoding='utf-8').splitlines()
9     [-1]:
10     raise RuntimeError('Tests must finish successfully')
11 manifest={}
12 for p in production_sources()+[SOURCE]:
13     if p.name.startswith('test_'):continue
14     relative=p.relative_to(root);dest=saved/relative;dest.parent.mkdir(parents=True,exist_ok=True);
15     shutil.copy2(p,dest)
16     manifest[str(relative)]=hashlib.sha256(p.read_bytes()).hexdigest()
17 (saved/'manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding='utf-8'
18 )
19 req=subprocess.check_output(['uv','pip','freeze','--python',str(root/'.venv/bin/python')],text=True)
20 (saved/'requirements.txt').write_text(req,encoding='utf-8')
21 processes={}
22 for command in ['q42','q43']:
23     out=root/f'outputs/q4/raw/rescue_lp_{command}_20260913_v2';out.mkdir(parents=True,exist_ok=True)
24     if (out/'state.json').exists():raise RuntimeError('Fresh directory required')
25     args=[str(root/'.venv/bin/python'),'--u','-m','src.q4.run',command,'--output',str(out)]
26     env=dict(os.environ,OMP_NUM_THREADS='1',OPENBLAS_NUM_THREADS='1',VECLIB_MAXIMUM_THREADS='1',
27             MKL_NUM_THREADS='1',PYTHONUTF8='1')
28     with (out/'run.log').open('ab') as stream:
29         p=subprocess.Popen(args,cwd=root,env=env,stdout=stream,stderr=subprocess.STDOUT,
30                             start_new_session=True)
31         metadata={'pid':p.pid,'command':args,'signature':cfg.signature(),'model_version':MODEL_VERSION,'
32                   python':sys.version,'frozen_sources':str(saved)}
33         (out/'launch.json').write_text(json.dumps(metadata,ensure_ascii=False,indent=2),encoding='utf-8'
34 )
35         resume='#!/bin/zsh\nset -e\ncd '+shlex.quote(str(root))+'\nexport OMP_NUM_THREADS=1
36                 OPENBLAS_NUM_THREADS=1 VECLIB_MAXIMUM_THREADS=1 MKL_NUM_THREADS=1 PYTHONUTF8=1\nexec '+shlex.
37                 join(args)+'\n'
38         (out/'resume.sh').write_text(resume,encoding='utf-8'); (out/'resume.sh').chmod(0o755)
39         processes[command]=metadata
40 print(json.dumps(processes,ensure_ascii=False,indent=2))
41 (root/'docs/4/rescue_lp/launches_v2.json').write_text(json.dumps(processes,ensure_ascii=False,indent
42 =2),encoding='utf-8')

```

src/q4/rescue/launch_q43_v3.py

```

1 from pathlib import Path
2 import subprocess,os,json,hashlib,shutil,sys,shlex
3 root=Path(__file__).resolve().parents[3];sys.path.insert(0,str(root))
4 from src.q4.config import Config,production_sources,SOURCE,MODEL_VERSION
5 assert (root/'docs/4/rescue_lp/apr18_verified.json').exists()
6 cfg=Config();saved=root/'docs/4/rescue_lp/frozen_source_v3';manifest={}
7 for p in production_sources()+[SOURCE]:
8     if p.name.startswith('test_'):continue
9     relative=p.relative_to(root);dest=saved/relative;dest.parent.mkdir(parents=True,exist_ok=True);
10    shutil.copy2(p,dest)
11    manifest[str(relative)]=hashlib.sha256(p.read_bytes()).hexdigest()

```

```

11 (saved/'manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding='utf-8'
    )
12 shutil.copy2(root/'docs/4/rescue_lp/frozen_source_v2/requirements.txt',saved/'requirements.txt')
13 out=root/'outputs/q4/raw/rescue_lp_q43_20260913_v3';out.mkdir(parents=True,exist_ok=True)
14 assert not (out/'state.json').exists()
15 args=[str(root/'venv/bin/python'),'--u','--m','src.q4.run','q43','--output',str(out)]
16 env=dict(os.environ,OMP_NUM_THREADS='1',OPENBLAS_NUM_THREADS='1',VECLIB_MAXIMUM_THREADS='1',
    MKL_NUM_THREADS='1',PYTHONUTF8='1')
17 with (out/'run.log').open('ab') as stream:
18     p=subprocess.Popen(args,cwd=root,env=env,stdout=stream,stderr=subprocess.STDOUT,
    start_new_session=True)
19 metadata={'pid':p.pid,'command':args,'signature':cfg.signature(),'model_version':MODEL_VERSION,'
    python':sys.version,'frozen_sources':str(saved)}
20 (out/'launch.json').write_text(json.dumps(metadata,ensure_ascii=False,indent=2),encoding='utf-8')
21 resume='#!/bin/zsh\nset -e\ncd '+shlex.quote(str(root))+'\nexport OMP_NUM_THREADS=1
    OPENBLAS_NUM_THREADS=1 VECLIB_MAXIMUM_THREADS=1 MKL_NUM_THREADS=1 PYTHONUTF8=1\nexec '+shlex.
    join(args)+'\n'
22 (out/'resume.sh').write_text(resume,encoding='utf-8'); (out/'resume.sh').chmod(0o755)
23 (root/'docs/4/rescue_lp/launch_q43_v3.json').write_text(json.dumps(metadata,ensure_ascii=False,
    indent=2),encoding='utf-8')
24 print(json.dumps(metadata,ensure_ascii=False,indent=2))

```

src/q4/rescue/verify_full.py

```

1 """全年完成后的独立验收：逐节点/物理/SHA/七个Excel工作表与逐格日期和SOC。"""
2 from pathlib import Path
3 import sys,json,hashlib
4 import numpy as np
5 import pandas as pd
6 from openpyxl import load_workbook
7 ROOT=Path(__file__).resolve().parents[3]
8 sys.path.insert(0,str(ROOT))
9
10
11 def check_mode(mode):
12     short='q4'+mode[-1]
13     version='v2' if mode=='4-2' else 'v3'
14     folder=ROOT/f'outputs/q4/raw/rescue_lp_{short}_20260913_{version}'
15     state=json.loads((folder/'state.json').read_text(encoding='utf-8'))
16     assert len(state['days'])==334
17     assert [a['day'] for a in state['days']]==list(range(31,365))
18     dispatch=pd.read_csv(folder/'dispatch.csv',float_precision='round_trip')
19     expected_dates=pd.date_range('2025-02-01',periods=334)
20     expected_times=pd.date_range('2025-02-01 00:10',periods=334*144,freq='10min')
21     np.testing.assert_array_equal(pd.to_datetime(dispatch.timestamp),expected_times)
22     np.testing.assert_array_equal(dispatch.slot,np.tile(np.arange(144),334))
23     assert len(dispatch)==48096
24     c=dispatch.charge.to_numpy();r=dispatch.discharge.to_numpy()
25     soc=6000+np.cumsum(.9*c-r/.9)
26     soc_error=float(np.max(abs(soc-dispatch.soc)))
27     assert soc_error<=1e-6
28     assert dispatch.soc.min()>=1200-1e-6 and dispatch.soc.max()<=10800+1e-6
29     assert np.max(c+r)<=5000/6+1e-6
30     assert np.max(np.minimum(c,r))<=1e-6
31     balance=float(abs(dispatch.called+dispatch.discharge+dispatch.emergency-dispatch.net_kwh-
    dispatch.charge-dispatch.spill).max())

```

```

32 assert balance<=1e-6
33 difference=dispatch.grid-dispatch.g0
34 adjustment=np.where(difference>=0,1.5*difference,.5*difference) if mode=='4-3' else np.zeros(len
(dispatch))
35 np.testing.assert_allclose(dispatch.total_cost,dispatch.price*(dispatch.g0+adjustment+5*dispatch
.emergency),atol=1e-6,rtol=0)
36 audits=[]
37 for index,date in enumerate(expected_dates):
38     day=dispatch.iloc[index*144:(index+1)*144]
39     receipt=state['days'][index]
40     for key,sub,suffix in [('dispatch','days','.csv'),('audit','audit','.json')]:
41         path=folder/sub/(str(date.date())+suffix)
42         assert hashlib.sha256(path.read_bytes()).hexdigest()==receipt['hashes'][key]
43         audit=json.loads((folder/'audit'/f'{date.date()}.json').read_text(encoding='utf-8'))
44         nodes=[audit] if mode=='4-2' else audit['nodes']
45         if mode=='4-3':assert [n['hour'] for n in nodes]==[0,6,12,18]
46         for node in nodes:
47             hour=0 if mode=='4-2' else node['hour'];count=node['committed_slots'];start=hour*6
48             part=day.iloc[start:start+count]
49             for key in ['grid','charge','discharge']:
50                 np.testing.assert_allclose(part[key],node[key][:count],atol=1e-6,rtol=0)
51             initial=6000 if index==0 and hour==0 else float(dispatch.soc.iloc[index*144+start-1])
52             assert abs(node['initial_soc']-initial)<=1e-6
53             solver=node['solver'];scenario=node['scenarios'];audits.append(solver)
54             assert solver['status']=='Optimal' and solver['reliable'] and solver['valid']
55             assert solver['solution_source']=='optimal_solver' and solver['integer_variables']==0
56             assert solver['storage_schedule']=='node_shared_fixed_committed_block'
57             assert solver['dro_coupling_constraints']==scenario['S']**2
58             assert scenario['window']==56 and scenario['joint'] and scenario['bootstrap_repetitions'
]==100
59             assert scenario['S']<=20
60             assert scenario['tail_singletons_verified']
61             np.testing.assert_allclose(scenario['tail_prior_mass'],scenario['tail_weights'],atol=1e
-12,rtol=0)
62             assert all(j*144+hour*6+scenario['length']<=receipt['day']*144+hour*6 for j in scenario[
'history_days'])
63             assert all(j>=receipt['day']-56 for j in scenario['history_days'])
64         assert len(audits)==(334 if mode=='4-2' else 1336)
65         workbook_path=folder/f'result{mode}.xlsx'
66         wb=load_workbook(workbook_path,read_only=True,data_only=False)
67         expected_sheets=[ '计划购电量']+([ '调整购电量' if mode=='4-3' else [])+[ '充放电量','紧急购电量']
68         assert wb.sheetnames==expected_sheets
69         maximum_error=0.
70         def compare(a,b):
71             nonlocal maximum_error
72             a=np.asarray(a,dtype=float);b=np.asarray(b,dtype=float)
73             if a.size:maximum_error=max(maximum_error,float(np.max(abs(a-b))))
74             np.testing.assert_allclose(a,b,atol=1e-7,rtol=0)
75         for sheet,key,cost in [('计划购电量','grid' if mode=='4-2' else 'g0','planned_cost')+([ '调整购电
量','grid','adjustment_cost'] if mode=='4-3' else []):
76             values=list(wb[sheet].values)
77             assert len(values)==335
78             for i,row in enumerate(values[1:]):
79                 assert pd.Timestamp(row[0])==expected_dates[i]
80                 day=dispatch.iloc[i*144:(i+1)*144]
81                 compare(row[1:145],day[key])
82                 compare(row[145:147],[day[key].sum(),day[cost].sum()])

```

```

83     for slot,label in enumerate(values[0][1:145]):
84         start=slot*10;end=(slot+1)*10
85         assert label==f'{start//60:02d}:{start%60:02d}-{end//60:02d}:{end%60:02d}'
86 storage=list(wb['充放电量'].values)[1:]
87 assert len(storage)==334*6
88 for i,date in enumerate(expected_dates):
89     day=dispatch.iloc[i*144:(i+1)*144]
90     block=storage[i*6:(i+1)*6]
91     assert pd.Timestamp(block[0][0])==date
92     compare([block[0][5],block[1][5]],[day.initial_soc.iloc[0],day.soc.iloc[-1]])
93     assert block[0][4]=='0:00' and block[1][4]=='24:00'
94     for k,row in enumerate(block):
95         assert row[1]==f'{k*4:02d}:00-{(k+1)*4:02d}:00'
96         part=day.iloc[k*24:(k+1)*24]
97         compare(row[2:4],[part.charge.sum(),part.discharge.sum()])
98 # 独立分组逐日连续正紧急购电，避免使用被测export的event实现。
99 expected=[]
100 for date,day in dispatch.groupby('date',sort=False):
101     values=day.emergency.to_numpy();start=None
102     for i in range(145):
103         positive=i<144 and values[i]>0
104         if positive and start is None:start=i
105         if not positive and start is not None:
106             label=f'{start//6:02d}:{start%6*10:02d}-{i//6:02d}:{i%6*10:02d}'
107             # 导出当前采用strftime午夜00:00；保持题目右端点与实际导出规则。
108             if i==144:label=label[:-5]+'00:00'
109             expected.append((pd.Timestamp(date),label,float(values[start:i].sum())))
110             start=None
111 saved=list(wb['紧急购电量'].values)[1:]
112 assert len(saved)==len(expected)
113 for row,wanted in zip(saved,expected):
114     assert pd.Timestamp(row[0])==wanted[0] and row[1]==wanted[1]
115     compare([row[2]],[wanted[2]])
116 error_values={'#REF!','#DIV/0!','#VALUE!','#NAME?','#N/A','#NUM!','#NULL!','#SPILL!','#CALC!'}
117 for sheet in wb:
118     for row in sheet.iter_rows():
119         for cell in row:
120             assert cell.data_type not in ('e','f')
121             assert not isinstance(cell.value,str) or cell.value not in error_values
122 wb.close()
123 gates=json.loads((folder/'production_gates.json').read_text())
124 assert gates['passed']
125 assert gates['metrics']['C_realized']<14836427.918189546*1.15
126 # 完成态收据显式汇总已验证的实际版本，避免仅凭路径追溯。
127 from src.q4.config import write_json
128 source=json.loads((folder/'source_manifest.json').read_text(encoding='utf-8'))
129 solver_versions=sorted({a['solver_version'] for a in audits})
130 assert len(solver_versions)==1
131 state.update(solver_version=solver_versions[0], source_hashes=source['production_code_sha256'],
132             input_hashes={**source['q3_provenance']['files'], source['attachment4']:source['
133 attachment4_sha256']},
134             q2_frozen_source={'path':source['q2_run'],'dispatch_sha256':source['
135 q2_frozen_dispatch_sha256']},
136             price_forecast_version=source['price_code_sha256'],
137             price_forecasts_array_sha256=source['price_forecasts_array_sha256'],
138             W=state['config']['residual_window'], S=state['config']['scenarios'],
139             bootstrap_repetitions=state['config']['bootstrap_repetitions'])

```

```

138 write_json(folder/'state.json',state)
139 return dict(mode=mode,days=334,rows=48096,certified_nodes=len(audits),daily_hashes=668,
140             integer_variables=0,max_balance_error=balance,max_soc_chain_error=soc_error,
141             excel_all_cells_checked=True,excel_max_numeric_error=maximum_error,
142             emergency_events=len(expected),formula_error_cells=0,
143             also_passes_review_baseline_15pct=True,workbook=str(workbook_path),
144             workbook_sha256=hashlib.sha256(workbook_path.read_bytes()).hexdigest())
145
146
147 if __name__=='__main__':
148     requested=sys.argv[1:] or ['4-2','4-3']
149     for mode in requested:
150         result=check_mode(mode)
151         (ROOT/'docs/4/rescue_lp'/f'full_acceptance_{mode}.json').write_text(json.dumps(result,
152         ensure_ascii=False,indent=2),encoding='utf-8')
153     print(json.dumps(result,ensure_ascii=False),flush=True)

```

src/q4/rolling.py

```

1  """7.6—7.8: 4-2日前48h、4-3四节点24h滚动与真实价格逐槽回放。"""
2  from dataclasses import asdict
3  from contextlib import contextmanager
4  from pathlib import Path
5  import hashlib
6  import json
7  import os
8  import numpy as np
9  import pandas as pd
10 from .physics import DispatchPolicy as Policy
11 from .config import Config, DT, E_INITIAL, E_MIN, E_MAX, ETA_C, ETA_D, CAP, MODEL_VERSION, TOL,
12     write_csv, write_json
13 from .data import Inputs
14 from .optimization import LimitedSolve, solve
15 from .physics import adjustment_quantity, replay
16 from .scenarios import construct
17
18 @contextmanager
19 def run_lock(path: Path):
20     """跨平台单写入者文件锁：进程退出后由操作系统释放。"""
21     path.parent.mkdir(parents=True, exist_ok=True)
22     with path.open("a+b") as stream:
23         if stream.tell() == 0:
24             stream.write(b"0")
25             stream.flush()
26         if os.name == "nt":
27             import msvcrt
28             stream.seek(0)
29             try:
30                 msvcrt.locking(stream.fileno(), msvcrt.LK_NBLCK, 1)
31             except OSError as error:
32                 raise RuntimeError("同一问题四输出目录已有写入者") from error
33         else:
34             import fcntl
35             try:
36                 fcntl.flock(stream.fileno(), fcntl.LOCK_EX | fcntl.LOCK_NB)

```

```

37         except OSError as error:
38             raise RuntimeError("同一问题四输出目录已有写入者") from error
39     try:
40         yield
41     finally:
42         stream.seek(0)
43         if os.name == "nt":
44             msvcrt.locking(stream.fileno(), msvcrt.LK_UNLCK, 1)
45         else:
46             fcntl.flock(stream.fileno(), fcntl.LOCK_UN)
47
48
49 def date_of(day: int) -> pd.Timestamp:
50     return pd.Timestamp("2025-01-01") + pd.Timedelta(days=day)
51
52
53 def _execute(data: Inputs, day: int, hour: int, policy: Policy, initial: float) -> pd.DataFrame:
54     start, end = hour * 6, hour * 6 + len(policy.grid)
55     if end > 144:
56         raise ValueError("正式提交块不得跨过当日午夜")
57     actual = data.q3.actual[day, start:end]
58     net = (actual[:, 0] - actual[:, 1]) * DT
59     response = replay(policy, net, initial)
60     return pd.DataFrame({
61         "date": str(date_of(day).date()),
62         "slot": np.arange(start, end),
63         "timestamp": date_of(day) + pd.to_timedelta((np.arange(start, end) + 1) * 10, unit="min"),
64         "node_hour": hour,
65         "net_kwh": net,
66         "load_kw": actual[:, 0],
67         "pv_kw": actual[:, 1],
68         "price": data.actual_prices[day, start:end],
69         "price_forecast": data.price.values[day, start:end],
70         "grid": policy.grid,
71         "charge_plan": policy.charge,
72         "discharge_plan": policy.discharge,
73         **response,
74     })
75
76
77 def _cold_start(data: Inputs, day: int, initial: float, mode: str) -> tuple[pd.DataFrame, dict]:
78     zero = np.zeros(144)
79     policy = Policy(zero, zero, zero)
80     frame = _execute(data, day, 0, policy, initial)
81     frame["g0"] = zero
82     frame["initial_soc"] = initial
83     frame["planned_cost"] = 0.0
84     frame["adjustment_cost"] = 0.0
85     frame["emergency_cost"] = 5 * frame.price * frame.emergency
86     frame["total_cost"] = frame.emergency_cost
87     return frame, {"date": str(date_of(day).date()), "mode": mode, "cold_start": True,
88                   "reason": "inherit Q2/Q3 Jan1-Jan7 unavailable-load stationary-storage protocol"}
89
90
91 def run_day_q42(data: Inputs, day: int, initial: float, config: Config) -> tuple[pd.DataFrame, dict]:
92     if day < 7:

```



```

93     return _cold_start(data, day, initial, "4-2")
94     scenarios = construct(data, day, 0, 288, config)
95     solution = solve(scenarios, initial, config)
96     committed = solution.policy.part(0, 144)
97     frame = _execute(data, day, 0, committed, initial)
98     frame["g0"] = frame.grid
99     frame["initial_soc"] = initial
100    frame["planned_cost"] = frame.price * frame.grid
101    frame["adjustment_cost"] = 0.0
102    frame["emergency_cost"] = 5 * frame.price * frame.emergency
103    frame["total_cost"] = frame.planned_cost + frame.emergency_cost
104    audit = {
105        "date": str(date_of(day).date()),
106        "mode": "4-2",
107        "initial_soc": initial,
108        "committed_slots": 144,
109        "roles": ["current_formal"] * 144 + ["next_day_continuation"] * 144,
110        "solver": solution.audit,
111        "scenarios": scenarios.audit,
112        "grid": solution.policy.grid.tolist(),
113        "charge": solution.policy.charge.tolist(),
114        "discharge": solution.policy.discharge.tolist(),
115    }
116    validate_frame(frame, "4-2", config)
117    return frame, audit
118
119
120 def run_day_q43(data: Inputs, day: int, initial: float, config: Config) -> tuple[pd.DataFrame, dict
121     ]:
122     if day < 7:
123         return _cold_start(data, day, initial, "4-3")
124     soc = initial
125     g0 = None
126     frames, audits = [], []
127     nodes = list(config.nodes)
128     for position, hour in enumerate(nodes):
129         next_hour = nodes[position + 1] if position + 1 < len(nodes) else 24
130         committed_slots = (next_hour - hour) * 6
131         scenarios = construct(data, day, hour, 144, config)
132         reference = None if hour == 0 else g0[hour * 6:]
133         solution = solve(scenarios, soc, config, reference=reference, today=144 - hour * 6)
134         if hour == 0:
135             g0 = solution.policy.grid.copy()
136             committed = solution.policy.part(0, committed_slots)
137             executed = _execute(data, day, hour, committed, soc)
138             executed["g0"] = g0[hour * 6:next_hour * 6]
139             executed["initial_soc"] = initial
140             frames.append(executed)
141             audits.append({
142                 "hour": hour,
143                 "initial_soc": soc,
144                 "committed_slots": committed_slots,
145                 "roles": ["committed" if slot < committed_slots else
146                     "today_provisional" if slot < 144 - hour * 6 else "next_day_continuation"
147                     for slot in range(144)],
148                 "solver": solution.audit,
149                 "scenarios": scenarios.audit,

```

```

149         "grid": solution.policy.grid.tolist(),
150         "charge": solution.policy.charge.tolist(),
151         "discharge": solution.policy.discharge.tolist(),
152         "delta_from_g0": (solution.policy.grid[:144 - hour * 6] - g0[hour * 6:]).tolist(),
153     })
154     soc = float(executed.soc.iloc[-1])
155     frame = pd.concat(frames, ignore_index=True)
156     frame["planned_cost"] = frame.price * frame.g0
157     frame["adjustment_cost"] = frame.price * adjustment_quantity(frame.grid.to_numpy(), frame.g0.
        to_numpy())
158     frame["emergency_cost"] = 5 * frame.price * frame.emergency
159     frame["total_cost"] = frame.planned_cost + frame.adjustment_cost + frame.emergency_cost
160     audit = {"date": str(date_of(day).date()), "mode": "4-3", "initial_soc": initial,
161             "settlement": "final_effective_relative_to_00", "nodes": audits}
162     validate_frame(frame, "4-3", config)
163     return frame, audit
164
165
166 def validate_frame(frame: pd.DataFrame, mode: str, config: Config, *, full: bool = False) -> None:
167     if mode not in ("4-2", "4-3") or frame.empty:
168         raise ValueError("回放模式或数据为空")
169     times = pd.to_datetime(frame.timestamp)
170     dates = pd.to_datetime(frame.date)
171     if times.duplicated().any() or not times.diff().dropna().eq(pd.Timedelta(minutes=10)).all():
172         raise ValueError("真实回放时序不连续")
173     if not dates.eq((times - pd.Timedelta(minutes=10)).dt.normalize()).all():
174         raise ValueError("10分钟右端点日期归属错误")
175     last = None
176     for _, day in frame.groupby("date", sort=False):
177         if len(day) != 144 or not np.array_equal(day.slot, np.arange(144)):
178             raise ValueError("每日必须恰含144个时段")
179         initial = float(day.initial_soc.iloc[0])
180         if last is not None and abs(initial - last) > TOL:
181             raise ValueError("跨日SOC断裂")
182         np.testing.assert_allclose(day.net_kwh, (day.load_kw - day.pv_kw) * DT, atol=TOL, rtol=0)
183         np.testing.assert_allclose(day.called + day.discharge + day.emergency,
184                                     day.net_kwh + day.charge + day.spill, atol=TOL, rtol=0)
185         np.testing.assert_allclose(day.unused + day.spill, day.total_surplus, atol=TOL, rtol=0)
186         np.testing.assert_allclose(day.planned_cost, day.price * day.g0, atol=TOL, rtol=0)
187         expected_adjustment = (np.zeros(144) if mode == "4-2" else
188                                 day.price.to_numpy() * adjustment_quantity(day.grid.to_numpy(), day.
189                                     g0.to_numpy()))
190         np.testing.assert_allclose(day.adjustment_cost, expected_adjustment, atol=TOL, rtol=0)
191         np.testing.assert_allclose(day.emergency_cost, 5 * day.price * day.emergency, atol=TOL, rtol
192                                     =0)
193         np.testing.assert_allclose(day.total_cost,
194                                     day.planned_cost + day.adjustment_cost + day.emergency_cost, atol
195                                     =TOL, rtol=0)
196         if not np.isfinite(day.select_dtypes(include=[np.number]).to_numpy()).all():
197             raise ValueError("回放含非有限值")
198         for key in ('grid', 'called', 'unused', 'charge', 'discharge', 'emergency', 'spill'):
199             if day[key].min() < -TOL:
200                 raise ValueError(f"负值: {key}")
201             if (day.called > day.grid + TOL).any() or (day.charge + day.discharge > CAP + TOL).any():
202                 raise ValueError("调用额度或储能功率越界")
203             if day.soc.min() < E_MIN - TOL or day.soc.max() > E_MAX + TOL:
204                 raise ValueError("SOC越界")

```

```

202     np.testing.assert_allclose(day.soc, initial + np.cumsum(ETA_C * day.charge - day.discharge /
203     ETA_D), atol=TOL, rtol=0)
204     np.testing.assert_allclose(day.charge, day.charge_plan, atol=TOL, rtol=0)
205     np.testing.assert_allclose(day.discharge, day.discharge_plan, atol=TOL, rtol=0)
206     last = float(day.soc.iloc[-1])
207     if full and (len(frame) != 334 * 144 or dates.min() != pd.Timestamp("2025-02-01")
208     or dates.max() != pd.Timestamp("2025-12-31")):
209         raise ValueError("拒绝将部分日期发布为完整问题四结果")
210
211 def run_period(data: Inputs, mode: str, config: Config, output: Path, *, end: int = 365) -> pd.
    DataFrame:
212     if mode not in ("4-2", "4-3"):
213         raise ValueError("模式只能为4-2或4-3")
214     if config.allow_limited:
215         raise ValueError("正式q42/q43禁止allow_limited")
216     if not 31 < end <= 365:
217         raise ValueError("正式期从Feb1开始, end必须在32..365")
218     if mode == '4-3' and config.nodes != (0, 6, 12, 18):
219         raise ValueError("正式Q4-3必须使用全部四时点")
220     if config.residual_window != 56 or not config.joint or config.dro_scale <= 0:
221         raise ValueError("正式计算固定W=56并保留联合残差与Wasserstein")
222     output.mkdir(parents=True, exist_ok=True)
223     with run_lock(output / "run.lock"):
224         signature = config.signature()
225         sources = hashlib.sha256(json.dumps(data.provenance, sort_keys=True).encode("utf-8")).
            hexdigest()
226         state_path = output / "state.json"
227         state = json.loads(state_path.read_text(encoding="utf-8")) if state_path.exists() else {
228             "signature": signature, "sources": sources, "mode": mode, "days": [], "soc": E_INITIAL,
229             "config": asdict(config), "model_version": MODEL_VERSION, "formal_start_day": 31,
230             "integer_variables": 0, "initial_soc": E_INITIAL,
231         }
232         if state["signature"] != signature or state["sources"] != sources or state["mode"] != mode:
233             raise ValueError("续算模型、输入或模式变化, 禁止混用已有前缀")
234         frames = []
235         runner = run_day_q42 if mode == "4-2" else run_day_q43
236         write_json(output / "source_manifest.json", data.provenance)
237         for position, day in enumerate(range(31, end)):
238             date = str(date_of(day).date())
239             dispatch_path = output / "days" / f"{date}.csv"
240             audit_path = output / "audit" / f"{date}.json"
241             if position < len(state["days"]):
242                 receipt = state["days"][position]
243                 if receipt["day"] != day:
244                     raise ValueError("续算日期顺序损坏")
245                 for key, path in (("dispatch", dispatch_path), ("audit", audit_path)):
246                     if hashlib.sha256(path.read_bytes()).hexdigest() != receipt["hashes"][key]:
247                         raise ValueError("已提交日文件摘要变化")
248                 frame = pd.read_csv(dispatch_path, float_precision="round_trip")
249             else:
250                 write_json(output / "active.json", {"status": "solving", "mode": mode, "date": date,
251                 "completed_days": len(state["days"])})
252             try:
253                 frame, audit = runner(data, day, state["soc"], config)
254             except LimitedSolve as error:
255                 write_json(output / "failures" / f"{date}.json", error.audit)

```

```

256         write_json(output / "active.json", {"status": "failed", "date": date, "reason":
error.audit}))
257         raise
258         validate_frame(frame, mode, config)
259         solver_audits = [audit['solver']] if mode == '4-2' else [node['solver'] for node in
audit['nodes']]
260         if not all(item['status'] == 'Optimal' and item['reliable'] and
261                    item['solution_source'] == 'optimal_solver' and item['integer_variables']
== 0
262                    for item in solver_audits):
263             raise ValueError("拒绝未认证或fallback策略正式落盘")
264             write_csv(dispatch_path, frame)
265             write_json(audit_path, audit)
266             hashes = {"dispatch": hashlib.sha256(dispatch_path.read_bytes()).hexdigest(),
267                      "audit": hashlib.sha256(audit_path.read_bytes()).hexdigest()}
268             state["days"].append({"day": day, "hashes": hashes})
269             state["soc"] = float(frame.soc.iloc[-1])
270             write_json(state_path, state)
271             print(f"{mode} {date} SOC={state['soc']:.3f} committed", flush=True)
272             frames.append(frame)
273             result = pd.concat(frames, ignore_index=True)
274             validate_frame(result, mode, config)
275             formal = result[pd.to_datetime(result.date) >= pd.Timestamp("2025-02-01")].copy()
276             write_csv(output / "dispatch.csv", formal)
277             write_json(output / "active.json", {"status": "calculation_complete", "mode": mode, "
completed_days": end - 31})
278             return formal

```

src/q4/run.py

```

1  """问题四命令行入口。"""
2  import argparse
3  from pathlib import Path
4  import time
5  from .config import Config, ROOT
6  from .data import read_inputs
7  from .diagnostics import diagnostics, write_experiment_plan, write_window_policy, production_gates
8  from .export import export_workbook, write_tables
9  from .rolling import run_day_q42, run_day_q43, run_period
10
11
12  def settings(arguments: argparse.Namespace) -> Config:
13      if arguments.command in ("q42", "q43") and arguments.allow_limited:
14          raise ValueError("正式q42/q43禁止--allow-limited; 仅供smoke/benchmark参数兼容")
15      nodes = {"0": (0,), "06": (0, 6), "0612": (0, 6, 12), "all": (0, 6, 12, 18)}[arguments.nodes]
16      return Config(scenarios=arguments.scenarios, residual_window=arguments.window,
17                  bootstrap_repetitions=arguments.bootstrap, seconds=arguments.seconds,
18                  gap=arguments.gap, solver_threads=arguments.threads, nodes=nodes, hard_timeout=
arguments.hard_timeout,
19                  allow_limited=arguments.allow_limited)
20
21
22  def main() -> None:
23      parser = argparse.ArgumentParser(description="Q4 strict-causal volatile-price DRO dispatch")
24      parser.add_argument("command", choices=("prepare", "diagnostics", "smoke", "benchmark", "q42", "
q43"))

```

```

25 parser.add_argument("--output", type=Path, default=ROOT / "outputs/q4/raw/current")
26 parser.add_argument("--end", type=int, default=365)
27 parser.add_argument("--day", type=int, default=31)
28 parser.add_argument("--scenarios", type=int, default=20)
29 parser.add_argument("--window", type=int, default=56, choices=(56, 84, 112))
30 parser.add_argument("--bootstrap", type=int, default=100)
31 parser.add_argument("--seconds", type=float, default=30.0)
32 parser.add_argument("--hard-timeout", type=float, default=30.0)
33 parser.add_argument("--gap", type=float, default=0.03)
34 parser.add_argument("--threads", type=int, default=1)
35 parser.add_argument("--nodes", choices=("0", "06", "0612", "all"), default="all")
36 parser.add_argument("--smoke-mode", choices=("4-2", "4-3"), default="4-2")
37 parser.add_argument("--allow-limited", action="store_true")
38 parser.add_argument("--with-lightgbm", action="store_true")
39 arguments = parser.parse_args()
40 config = settings(arguments)
41 processed = ROOT / "inputs/q4/rescue_processed"
42 data = read_inputs(processed)
43 if arguments.command == "prepare":
44     print(data.price.audit)
45     return
46 if arguments.command == "diagnostics":
47     print(diagnostics(data, arguments.output, with_lightgbm=arguments.with_lightgbm))
48     write_experiment_plan(config, arguments.output)
49     write_window_policy(arguments.output)
50     return
51 if arguments.command in ("smoke", "benchmark"):
52     runner = run_day_q42 if arguments.command == "smoke" and arguments.smoke_mode == "4-2" else
run_day_q43
53     started = time.perf_counter()
54     frame, audit = runner(data, arguments.day, 6000.0, config)
55     solver_audits = ([audit["solver"]] if "solver" in audit else
56                     [node["solver"] for node in audit["nodes"]])
57     print({"seconds": time.perf_counter() - started, "mode": audit["mode"],
58           "date": audit["date"], "node_gaps": [item["gap"] for item in solver_audits],
59           "all_reliable": all(item["reliable"] for item in solver_audits)})
60     if arguments.command == "smoke":
61         mode = arguments.smoke_mode
62         write_tables(frame, arguments.output)
63         export_workbook(frame, arguments.output, mode, config, smoke=True)
64     return
65     mode = "4-2" if arguments.command == "q42" else "4-3"
66     frame = run_period(data, mode, config, arguments.output, end=arguments.end)
67     write_tables(frame, arguments.output)
68     if arguments.end == 365:
69         production_gates(data, frame, mode, config, arguments.output)
70     export_workbook(frame, arguments.output, mode, config, smoke=arguments.end < 365)
71
72
73 if __name__ == "__main__":
74     main()

```

src/q4/scenarios.py

```

1 """7.4/7.5/7.7: 联合残差、前缀条件化、尾部保持缩减和Wasserstein半径。"""
2 from dataclasses import dataclass

```

```

3 from functools import lru_cache
4 import time
5 import numpy as np
6 from scipy.optimize import linprog
7 from scipy.sparse import coo_matrix
8 from .config import Config
9 from .data import Inputs
10
11
12 def _scale(values: np.ndarray) -> np.ndarray:
13     median = np.median(values, axis=0)
14     scale = 1.4826 * np.median(abs(values - median), axis=0)
15     positive = scale[scale > 1e-10]
16     fallback = float(np.median(positive)) if len(positive) else 1.0
17     return np.where(scale > 1e-10, scale, fallback)
18
19
20 def joint_distance(net: np.ndarray, price: np.ndarray, net_weight: float) -> tuple[np.ndarray, np.
    ndarray, np.ndarray]:
21     sn, sp = _scale(net), _scale(price)
22     dn = np.mean(abs(net[:, None] - net[None, :]) / sn, axis=2)
23     dp = np.mean(abs(price[:, None] - price[None, :]) / sp, axis=2)
24     return net_weight * dn + (1 - net_weight) * dp, sn, sp
25
26
27 def medoids(distance: np.ndarray, count: int) -> np.ndarray:
28     n = len(distance)
29     if not 1 <= count <= n or distance.shape != (n, n):
30         raise ValueError("k-medoids距离或数量非法")
31     chosen = [int(np.argmin(distance.sum(axis=0)))]
32     while len(chosen) < count:
33         candidates = [i for i in range(n) if i not in chosen]
34         chosen.append(min(candidates, key=lambda i: (np.min(distance[:, chosen + [i]], axis=1).sum()
35             , i)))
36     while True:
37         current = float(np.min(distance[:, chosen], axis=1).sum())
38         best = (current, None)
39         for position in range(count):
40             for candidate in range(n):
41                 if candidate in chosen:
42                     continue
43                 trial = chosen.copy()
44                 trial[position] = candidate
45                 value = float(np.min(distance[:, trial], axis=1).sum())
46                 if value < best[0] - 1e-10:
47                     best = value, trial
48         if best[1] is None:
49             return np.asarray(chosen, dtype=int)
50         chosen = best[1]
51
52 @lru_cache(maxsize=8)
53 def _transport_equalities(count: int):
54     columns = np.arange(count * count)
55     rows = np.r_[np.repeat(np.arange(count), count), np.tile(np.arange(count), count) + count]
56     return coo_matrix((np.ones(2 * count * count), (rows, np.tile(columns, 2))),
57         shape=(2 * count, count * count)).tocsr()

```

```

58
59
60 def _transport_distance(first: np.ndarray, second: np.ndarray, distance: np.ndarray) -> float:
61     equalities = _transport_equalities(len(first))
62     result = linprog(distance.ravel(), A_eq=equalities, b_eq=np.r_[first, second],
63                     bounds=(0, None), method="highs")
64     if not result.success:
65         raise RuntimeError(f"Wasserstein运输问题失败: {result.message}")
66     return float(result.fun)
67
68
69 def _prefix_weights(data: Inputs, days: np.ndarray, day: int, hour: int, config: Config) -> tuple[np.
    .ndarray, dict]:
70     if hour == 0 or not config.conditional:
71         weights = np.full(len(days), 1 / len(days))
72         return weights, {"conditional": False, "bandwidth": None, "ESS": float(len(days))}
73     history = np.vstack([data.prefix(int(j), hour) for j in days])
74     observed = data.prefix(day, hour)
75     scale = history.std(axis=0)
76     scale[scale < 1e-10] = 1.0
77     distances = np.mean(abs(history - observed) / scale, axis=1)
78     positive = distances[distances > 1e-12]
79     bandwidth = float(np.median(positive)) if len(positive) else 1.0
80     target = min(config.min_ess, float(len(days)))
81     while True:
82         shifted = distances - distances.min()
83         raw = np.exp(-shifted / max(bandwidth, 1e-12))
84         weights = raw / raw.sum()
85         ess = float(1 / np.dot(weights, weights))
86         if ess >= target - 1e-9:
87             break
88         bandwidth *= 1.5
89     return weights, {"conditional": True, "bandwidth": bandwidth, "ESS": ess,
90                     "prefix_dimension": history.shape[1], "target_ESS": target}
91
92
93 def _reduce(net: np.ndarray, price: np.ndarray, prior: np.ndarray, config: Config) -> tuple[np.
    ndarray, np.ndarray, np.ndarray, np.ndarray, dict]:
94     distance, sn, sp = joint_distance(net, price, config.net_weight)
95     risk = np.mean(np.maximum(net / sn, 0) * np.maximum(price / sp, 0), axis=1)
96     tail = np.flatnonzero(risk > np.quantile(risk, 0.9))
97     count = min(config.scenarios, len(net))
98     if len(tail) > count:
99         raise ValueError("90%联合尾部轨迹多于场景上限, 禁止删除强制尾部")
100     if len(net) <= count:
101         representatives = np.arange(len(net))
102     else:
103         rest = np.asarray([i for i in range(len(net)) if i not in set(tail)], dtype=int)
104         ordinary_count = count - len(tail)
105         ordinary = medoids(distance[np.ix_(rest, rest)], ordinary_count) if ordinary_count else np.
            empty(0, dtype=int)
106         representatives = np.r_[tail, rest[ordinary]]
107     # 尾部为singleton, 普通质量只分配给普通medoids.
108     tail_set = set(tail)
109     ordinary_rows = np.asarray([i for i in range(len(net)) if i not in tail_set], dtype=int)
110     ordinary_columns = np.asarray([k for k, i in enumerate(representatives) if i not in tail_set],
        dtype=int)

```

```

111     if len(ordinary_rows) and not len(ordinary_columns):
112         raise ValueError("强制尾部之外至少需要一个普通代表")
113     labels = np.empty(len(net), dtype=int)
114     if len(ordinary_rows):
115         labels[ordinary_rows] = ordinary_columns[np.argmin(
116             distance[np.ix_(ordinary_rows, representatives[ordinary_columns])], axis=1)]
117     labels[representatives] = np.arange(len(representatives))
118     weights = np.bincount(labels, weights=prior, minlength=len(representatives))
119     np.testing.assert_allclose(weights.sum(), prior.sum(), atol=1e-12)
120     weights /= weights.sum()
121     tail_columns = [k for k, i in enumerate(representatives) if i in tail_set]
122     np.testing.assert_allclose(weights[tail_columns], prior[representatives[tail_columns]] / prior.
123                               sum(), atol=1e-12)
124     audit = {
125         "tail_indices_in_pool": tail.tolist(),
126         "tail_prior_mass": prior[tail].tolist(),
127         "tail_weights": weights[tail_columns].tolist(),
128         "tail_singletons_verified": True,
129         "tail_risk_threshold": float(np.quantile(risk, 0.9)),
130         "risk_scores": risk.tolist(),
131         "represented_counts": np.bincount(labels, minlength=len(representatives)).tolist(),
132         "representative_pool_indices": representatives.tolist(),
133     }
134     return representatives, labels, weights, distance[np.ix_(representatives, representatives)],
135     audit
136
137 @dataclass
138 class Scenarios:
139     net: np.ndarray
140     prices: np.ndarray
141     weights: np.ndarray
142     distance: np.ndarray
143     radius: float
144     audit: dict
145
146 def construct(data: Inputs, day: int, hour: int, length: int, config: Config) -> Scenarios:
147     started = time.perf_counter()
148     point_net = data.point_net(day, hour, length)
149     point_price = data.point_price(day, hour, length)
150     days, net_errors, price_errors = data.joint_history(day, hour, length)
151     recent = days >= day - config.residual_window
152     days, net_errors, price_errors = days[recent], net_errors[recent], price_errors[recent]
153     if len(days) == 0:
154         return Scenarios(point_net[None], point_price[None], np.ones(1), np.zeros((1, 1)), 0.0,
155                           {"day": day, "hour": hour, "length": length, "S": 1, "deterministic": True
156                           })
157     prior, conditioning = _prefix_weights(data, days, day, hour, config)
158     if not config.joint and len(days) > 1:
159         permutation = np.random.default_rng(config.seed + day * 10 + hour).permutation(len(days))
160         price_errors = price_errors[permutation]
161     representatives, labels, weights, distance, reduction = _reduce(net_errors, price_errors, prior,
162                             config)
163     reduced_net = net_errors[representatives]
164     reduced_price = price_errors[representatives]
165     radius = 0.0

```



```

164 bootstrap = []
165 if config.dro_scale > 0 and len(representatives) > 1:
166     generator = np.random.default_rng(config.seed + day * 100 + hour)
167     for _ in range(config.bootstrap_repetitions):
168         counts = generator.multinomial(len(days), prior)
169         sampled = np.bincount(labels, weights=counts, minlength=len(representatives)) / len(days
    )
170         bootstrap.append(_transport_distance(sampled, weights, distance))
171         radius = float(np.quantile(bootstrap, config.bootstrap_quantile)) * config.dro_scale
172 audit = {
173     "construction_seconds": time.perf_counter() - started,
174     "day": day,
175     "hour": hour,
176     "length": length,
177     "history_days": days.tolist(),
178     "window": config.residual_window,
179     "joint": config.joint,
180     "S": len(representatives),
181     "weights": weights.tolist(),
182     "distance_weight_net": config.net_weight,
183     "radius": radius,
184     "radius_scale": config.dro_scale,
185     "bootstrap_repetitions": config.bootstrap_repetitions,
186     "bootstrap_quantile": config.bootstrap_quantile,
187     "bootstrap_distances": bootstrap,
188     **conditioning,
189     **reduction,
190 }
191 return Scenarios(
192     point_net + reduced_net,
193     np.maximum(point_price + reduced_price, 0.0),
194     weights,
195     distance,
196     radius,
197     audit,
198 )

```

src/q4/test_q4.py

```

1 import tempfile
2 import unittest
3 from pathlib import Path
4 import numpy as np
5 from openpyxl import load_workbook
6 from .physics import DispatchPolicy as Policy
7 from .config import Config, ROOT
8 from .data import read_inputs, read_prices
9 from .diagnostics import experiment_variants
10 from .export import export_workbook
11 from .optimization import solve, solve_dual
12 from .physics import adjustment_quantity, replay
13 from .price import feature_matrix
14 from .rolling import _cold_start, validate_frame
15 from .scenarios import Scenarios, _reduce, construct
16 from .traceability import generate
17

```

```

18
19 class Q4Tests(unittest.TestCase):
20     @classmethod
21     def setUpClass(cls):
22         cls.data = read_inputs(ROOT / "inputs/q4/rescue_processed")
23
24     def test_actual_price_shape_alignment_and_reported_statistics(self):
25         prices = read_prices(ROOT / "docs/CUMCM2026Problems/C题/附件/附件4.xlsx")
26         self.assertEqual(prices.shape, (365, 144))
27         self.assertAlmostEqual(float(prices.min()), 0.0076)
28         self.assertAlmostEqual(float(prices.max()), 1.7936)
29         self.assertAlmostEqual(float(prices.mean()), 0.76620, places=5)
30
31     def test_price_forecast_metrics_and_causality(self):
32         audit = self.data.price.audit
33         self.assertAlmostEqual(audit["seasonal_mae"], 0.04715, places=5)
34         self.assertLess(audit["elastic_net_mae"], audit["seasonal_mae"])
35         history = self.data.actual_prices.copy()
36         original = feature_matrix(history, 100)
37         history[100:] = 999.0
38         np.testing.assert_array_equal(original, feature_matrix(history, 100))
39         self.assertTrue(np.isnan(self.data.price.residuals[:7]).all())
40
41     def test_prefix_uses_realized_slots_only(self):
42         day, hour = 100, 12
43         original = self.data.prefix(day, hour)
44         actual = self.data.q3.actual[day, hour * 6:].copy()
45         prices = self.data.actual_prices[day, hour * 6:].copy()
46         try:
47             self.data.q3.actual[day, hour * 6:] += 1e6
48             self.data.actual_prices[day, hour * 6:] += 1e6
49             np.testing.assert_array_equal(original, self.data.prefix(day, hour))
50         finally:
51             self.data.q3.actual[day, hour * 6:] = actual
52             self.data.actual_prices[day, hour * 6:] = prices
53
54     def test_scenarios_invariant_to_all_unavailable_future_data(self):
55         config=Config(bootstrap_repetitions=1)
56         day=100
57         for hour,length in ((0,288),(6,144),(12,144),(18,144)):
58             original=construct(self.data,day,hour,length,config)
59             cutoff=day*144+hour*6
60             actual=self.data.q3.actual.reshape(-1,2)
61             prices=self.data.actual_prices.reshape(-1)
62             saved_actual=actual[cutoff:].copy();saved_prices=prices[cutoff:].copy()
63             saved_forecasts=self.data.price.values[day+1:].copy()
64             saved_pv=self.data.q3.hourly[day,hour//6+1:].copy()
65             try:
66                 actual[cutoff:]+=1e6;prices[cutoff:]+=1e6
67                 self.data.price.values[day+1:]+=1e6
68                 self.data.q3.hourly[day,hour//6+1:]+=1e6
69                 checked=construct(self.data,day,hour,length,config)
70                 for key in ('net','prices','weights','distance'):
71                     np.testing.assert_array_equal(getattr(original,key),getattr(checked,key))
72                 self.assertEqual(original.radius,checked.radius)
73             finally:
74                 actual[cutoff:]=saved_actual;prices[cutoff:]=saved_prices

```

```

75         self.data.price.values[day+1:]=saved_forecasts
76         self.data.q3.hourly[day,hour//6+1:]=saved_pv
77
78     def test_resume_receipts_without_reoptimization_and_tamper_rejection(self):
79         import json
80         import shutil
81         from unittest.mock import patch
82         from .rolling import run_period
83         source=ROOT/'outputs/q4/raw/rescue_lp_q42_20260913_v2'
84         if not (source/'state.json').exists():
85             self.skipTest('requires the already-computed production prefix')
86         state=json.loads((source/'state.json').read_text(encoding='utf-8'))
87         if state['signature'] != Config().signature():
88             self.skipTest('completed v2 prefix must not be resumed with revised audit code; v2
recovery already passed')
89         state['days']=state['days'][:3]
90         state['soc']=10800.0
91         with tempfile.TemporaryDirectory() as d:
92             output=Path(d)
93             for folder in ('days','audit'):
94                 (output/folder).mkdir()
95                 for date in ('2025-02-01','2025-02-02','2025-02-03'):
96                     suffix='.csv' if folder=='days' else '.json'
97                     shutil.copy2(source/folder/(date+suffix),output/folder/(date+suffix))
98                 (output/'state.json').write_text(json.dumps(state),encoding='utf-8')
99                 with patch('src.q4.rolling.run_day_q42',side_effect=AssertionError('must reuse receipts
')):
100                     frame=run_period(self.data,'4-2',Config(),output,end=34)
101                     self.assertEqual(len(frame),432)
102                     with (output/'days/2025-02-01.csv').open('a',encoding='utf-8') as stream:
103                         stream.write('tampered')
104                     with self.assertRaisesRegex(ValueError,'摘要变化'):
105                         run_period(self.data,'4-2',Config(),output,end=34)
106
107     def test_current_q43_receipt_recovery(self):
108         import json
109         import shutil
110         from unittest.mock import patch
111         from .rolling import run_period
112         source=ROOT/'outputs/q4/raw/rescue_lp_q43_20260913_v3'
113         if not (source/'state.json').exists():
114             self.skipTest('requires computed Q43v3 prefix')
115         state=json.loads((source/'state.json').read_text(encoding='utf-8'))
116         self.assertEqual(state['signature'],Config().signature())
117         state['days']=state['days'][:3]
118         import pandas as pd
119         state['soc']=float(pd.read_csv(source/'days/2025-02-03.csv').soc.iloc[-1])
120         with tempfile.TemporaryDirectory() as d:
121             output=Path(d)
122             for folder in ('days','audit'):
123                 (output/folder).mkdir()
124                 for date in ('2025-02-01','2025-02-02','2025-02-03'):
125                     suffix='.csv' if folder=='days' else '.json'
126                     shutil.copy2(source/folder/(date+suffix),output/folder/(date+suffix))
127                 (output/'state.json').write_text(json.dumps(state),encoding='utf-8')
128                 with patch('src.q4.rolling.run_day_q43',side_effect=AssertionError('must reuse receipts
')):

```

```

129         frame=run_period(self.data, '4-3', Config(), output, end=34)
130         self.assertEqual(len(frame), 432)
131         with (output/'days/2025-02-01.csv').open('a', encoding='utf-8') as stream:
132             stream.write('tampered')
133         with self.assertRaisesRegex(ValueError, '摘要变化'):
134             run_period(self.data, '4-3', Config(), output, end=34)
135
136     def test_joint_history_and_tail_reduction(self):
137         config = Config(scenarios=10, bootstrap_repetitions=3, seconds=1, solver_threads=1)
138         scenarios = construct(self.data, 60, 6, 144, config)
139         self.assertEqual(scenarios.net.shape, scenarios.prices.shape)
140         self.assertEqual(scenarios.distance.shape, (len(scenarios.weights),) * 2)
141         np.testing.assert_allclose(np.diag(scenarios.distance), 0, atol=1e-12)
142         self.assertAlmostEqual(float(scenarios.weights.sum()), 1)
143         net = np.zeros((20, 4)); price = np.zeros((20, 4))
144         net[-1] = 100; price[-1] = 10
145         representatives, _, _, _, audit = _reduce(net, price, np.full(20, 0.05),
146                                                    Config(scenarios=5, bootstrap_repetitions=1))
147         self.assertIn(19, representatives)
148         self.assertIn(19, audit["tail_indices_in_pool"])
149
150     def test_physical_call_separation(self):
151         policy = Policy(np.array([100.0, 100.0]), np.zeros(2), np.zeros(2))
152         result = replay(policy, np.array([20.0, -10.0]), 6000.0)
153         np.testing.assert_allclose(result["called"], [20, 0])
154         np.testing.assert_allclose(result["unused"], [80, 100])
155         np.testing.assert_allclose(result["spill"], [0, 10])
156         np.testing.assert_allclose(result["unused"] + result["spill"], result["total_surplus"])
157
158     def test_adjustment_settlement(self):
159         grid = np.array([150.0, 50.0, 100.0])
160         reference = np.full(3, 100.0)
161         np.testing.assert_allclose(adjustment_quantity(grid, reference), [75.0, -25.0, 0.0])
162
163     def test_dual_and_watchdog_agree(self):
164         net = np.array([[1000., 1200, 800, 500], [1300., 1400, 900, 700]])
165         prices = np.array([[.5, .6, .8, 1.], [.6, .8, 1., 1.2]])
166         scenarios = Scenarios(net, prices, np.array([.6, .4]), np.array([[0., 1.], [1., 0.]]), .1,
167                               {})
168         config = Config(scenarios=2, bootstrap_repetitions=1, seconds=5, gap=0,
169                         solver_threads=1, allow_limited=False)
170         generated = solve(scenarios, 1200.0, config)
171         dual = solve_dual(scenarios, 1200.0, config)
172         self.assertAlmostEqual(generated.audit["objective"], dual.audit["objective"], places=5)
173         self.assertTrue(generated.audit["reliable"] and dual.audit["reliable"])
174
175     def test_experiment_matrix_covers_required_ablations(self):
176         variants = experiment_variants(Config(bootstrap_repetitions=1))
177         required = {"independent_empirical", "joint_empirical", "joint_dro", "q43_unconditional",
178                    "q43_conditional", "radius_0.75", "radius_1.25", "scenarios_10", "scenarios_30",
179                    "net_weight_0.25", "net_weight_0.75", "window_84", "window_112",
180                    "P0_0", "P1_06", "P2_0612", "P3_061218"}
181         self.assertTrue(required.issubset(variants))
182
183     def test_workbook_roundtrip_and_partial_guard(self):
184         frame, _ = _cold_start(self.data, 31, 6000.0, "4-2")
185         config = Config(bootstrap_repetitions=1)

```

```

185         with tempfile.TemporaryDirectory() as directory:
186             output = Path(directory)
187             audit = export_workbook(frame, output, "4-2", config, smoke=True)
188             self.assertTrue(audit["roundtrip_all_sheets"])
189             workbook = load_workbook(output / "smoke_result4-2.xlsx", read_only=True, data_only=True
190         )
191             self.assertEqual(workbook.sheetnames, ["计划购电量", "充放电量", "紧急购电量"])
192             workbook.close()
193         with self.assertRaises(ValueError):
194             validate_frame(frame, "4-2", config, full=True)
195
196     def test_every_nonempty_source_line_is_mapped(self):
197         with tempfile.TemporaryDirectory() as directory:
198             audit = generate(Path(directory))
199             self.assertTrue(audit["complete"])
200             self.assertEqual(audit["mapped_lines"], audit["nonempty_lines"])
201
202 if __name__ == "__main__":
203     unittest.main()

```

src/q4/test_rescue.py

```

1  """紧急修复的公式与失效路径回归测试：真实连续回放另由vertical_slice执行。"""
2  import argparse
3  import tempfile
4  import unittest
5  from pathlib import Path
6  from unittest.mock import patch
7  import numpy as np
8  import pandas as pd
9  from .config import Config, E_MIN, ROOT
10 from .data import Inputs
11 from .price import Forecasts
12 from .physics import DispatchPolicy, replay, scenario_costs
13 from .optimization import build_matrix, solve, solve_dual, LimitedSolve
14 from .scenarios import Scenarios, _reduce
15 from .rolling import run_period
16 from .run import settings
17 from .diagnostics import sanity_metrics, write_window_policy
18
19
20 class RescueTests(unittest.TestCase):
21     def scenario(self, radius=0):
22         return Scenarios(np.array([[100.,120,80,50],[130.,140,90,70]]),
23                         np.ones((2,4)), np.array([.6,.4]), np.array([[0.,1.],[1.,0.]]), radius,{})
24
25     def test_zero_integer_and_shared_storage_variables(self):
26         obj, bounds, integer, con, ids, _ = build_matrix(self.scenario(), E_MIN)
27         self.assertEqual(integer.sum(), 0)
28         for key in ('g','c','r','E'):
29             self.assertEqual(ids[key].shape,(4,))
30             self.assertEqual(ids['x'].shape,(2,4))
31             self.assertEqual(ids['alpha'].shape,(2,))
32
33     def test_soc_roundoff_at_both_boundaries(self):

```

```

34     for initial in (1200.-1e-10,10800.+1e-10):
35         result=solve_dual(self.scenario(),initial,Config())
36         self.assertTrue(result.audit['reliable'])
37     for initial in (1200.-1e-3,10800.+1e-3):
38         with self.assertRaises(ValueError):
39             build_matrix(self.scenario(),initial)
40
41 def test_infinite_bound_roundoff_reproducer(self):
42     from .optimization import dual_bound
43     from scipy.optimize import Bounds, LinearConstraint
44     from scipy.sparse import csr_matrix
45     saved=np.load(ROOT/'docs/4/rescue_lp/dual_roundoff_reproducer.npz')
46     bounds=Bounds(saved['col_lower'],saved['col_upper'])
47     con=LinearConstraint(csr_matrix((len(saved['row_lower']),len(saved['col_lower']))),saved['
row_lower'],saved['row_upper'])
48     value,error=dual_bound(saved['row_dual'],saved['col_dual'],con,bounds)
49     self.assertAlmostEqual(value,10769.494225926748,places=6)
50     self.assertLess(error,1e-8)
51     changed=saved['row_dual'].copy();changed[5212]=1e-3
52     _,invalid=dual_bound(changed,saved['col_dual'],con,bounds)
53     self.assertGreater(invalid,1e-8)
54
55 def test_failure_audit_remains_json_serializable(self):
56     import json
57     error=LimitedSolve({'status':'failed','lower_bound':-np.inf,'nested':[np.nan]})
58     self.assertIsNone(error.audit['lower_bound'])
59     json.dumps(error.audit,allow_nan=False)
60
61 def test_four_slot_hand_balance(self):
62     policy=DispatchPolicy(np.array([100.,100,100,0]), np.array([10.,0,0,0]),np.array
([0.,20,0,0]))
63     out=replay(policy,np.array([120.,50,-10,30]),6000)
64     np.testing.assert_allclose(out['called'],[100,30,0,0])
65     np.testing.assert_allclose(out['emergency'],[30,0,0,30])
66     np.testing.assert_allclose(out['spill'],[0,0,10,0])
67     np.testing.assert_allclose(out['soc'],6000+np.cumsum([9,-20/.9,0,0]))
68
69 def test_rho_zero_equals_hand_expected_cost(self):
70     # 恒价、初始下界，无跨时段套利：80%分位数逐时取高需求。
71     scenario=self.scenario()
72     solution=solve_dual(scenario,E_MIN,Config())
73     self.assertAlmostEqual(solution.audit['objective'],430.,places=5)
74     costs=scenario_costs(solution.policy,solution.responses,scenario.prices)
75     self.assertAlmostEqual(solution.audit['objective'],float(scenario.weights@costs),places=6)
76
77 def test_positive_radius_matches_two_point_hand_worst_case(self):
78     scenario=self.scenario(.1)
79     solution=solve_dual(scenario,E_MIN,Config())
80     costs=np.asarray(solution.audit['scenario_costs'])
81     empirical=float(scenario.weights@costs)
82     expected=empirical+.1*abs(costs[1]-costs[0])
83     self.assertAlmostEqual(solution.audit['objective'],expected,places=5)
84
85 def test_adjustment_matrix_three_cases_and_continuation(self):
86     # 强制g为指定增购/减购/不变，第四段跨日，手工结算350+40=390。
87     scenario=Scenarios(np.zeros((1,4)),np.ones((1,4)),np.ones(1),np.zeros((1,1)),0,{})
88     with patch('src.q4.optimization.build_matrix', wraps=build_matrix) as builder:

```

```

89         original=build_matrix
90         def fixed(*args,**kwargs):
91             obj,bounds,integer,con,ids,offset=original(*args,**kwargs)
92             bounds.lb[ids['g']]=[150,50,100,40];bounds.ub[ids['g']]=[150,50,100,40]
93             return obj,bounds,integer,con,ids,offset
94         builder.side_effect=fixed
95         sol=solve_dual(scenario,6000,Config(),reference=np.full(3,100.),today=3)
96         self.assertAlmostEqual(sol.audit['objective'],390.,places=5)
97
98     def test_residual_origin_48h_and_midnight(self):
99         actual=np.arange(365*144,dtype=float).reshape(365,144)
100         forecasts=np.arange(365*288,dtype=float).reshape(365,288)*.3
101         data=Inputs(None,actual,Forecasts(forecasts,np.zeros_like(actual),{}),{})
102         for hour,length in ((0,288),(6,144),(12,144),(18,144)):
103             start=20*144+hour*6
104             expected=actual.ravel()[start:start+length]-forecasts[20,hour*6:hour*6+length]
105             np.testing.assert_array_equal(data.price_residual_path(20,hour,length,cutoff=10000),
106             expected)
107             old=forecasts[21].copy();forecasts[21]+=99999
108             np.testing.assert_array_equal(data.price_residual_path(20,hour,length,cutoff=10000),
109             expected)
110             forecasts[21]=old
111             with self.assertRaises(ValueError):
112                 data.price_residual_path(20,0,288,cutoff=21*144)
113
114     def test_tail_never_absorbs_ordinary_mass(self):
115         net=np.tile(np.arange(20,dtype=float)[: ,None],(1,4));price=net.copy()
116         prior=np.arange(1,21,dtype=float);prior/=prior.sum()
117         reps,labels,weights,_,audit=_reduce(net,price,prior,Config(scenarios=5))
118         for tail in audit['tail_indices_in_pool']:
119             column=list(reps).index(tail)
120             self.assertAlmostEqual(weights[column],prior[tail])
121             self.assertEqual(int((labels==column).sum()),1)
122             self.assertAlmostEqual(weights.sum(),1)
123
124     def test_formal_limited_forbidden_before_inputs_and_output(self):
125         for mode in ('4-2','4-3'):
126             with tempfile.TemporaryDirectory() as d, self.assertRaisesRegex(ValueError,
127             'allow_limited'):
128                 run_period(None,mode,Config(allow_limited=True),Path(d))
129             with self.assertRaisesRegex(ValueError,'allow_limited'):
130                 settings(argparse.Namespace(command='q42',allow_limited=True))
131
132     def test_hard_timeout_has_no_fallback(self):
133         with self.assertRaises(LimitedSolve) as context:
134             solve(self.scenario(),6000,Config(hard_timeout=.001))
135         self.assertEqual(context.exception.audit['status'],'hard_timeout')
136         self.assertFalse(context.exception.audit['reliable'])
137
138     def test_limited_solver_rejected_even_with_allow_flag(self):
139         with self.assertRaises(LimitedSolve):
140             solve_dual(self.scenario(),6000,Config(seconds=1e-9,allow_limited=True))
141
142     def test_baseline_alarm_and_storage_alarm(self):
143         frame=pd.DataFrame({key:[1.] for key in ('grid','called','unused','emergency','charge',
144         'discharge','spill')})
145         frame['total_cost']=116.;frame['soc']=10800.

```

```

142     metrics=sanity_metrics(frame,100.)
143     self.assertFalse(metrics['economic_pass']);self.assertFalse(metrics['storage_pass'])
144     frame['total_cost']=100;frame['soc']=6000
145     self.assertTrue(sanity_metrics(frame,100.)['economic_pass'])
146
147     def test_window_is_declared_preset(self):
148         with tempfile.TemporaryDirectory() as d:
149             audit=write_window_policy(Path(d))
150             self.assertEqual(audit['selected_window'],56)
151             self.assertFalse(audit['calibration_performed'])
152
153
154 if __name__=='__main__':
155     unittest.main()

```

src/q4/traceability.py

```

1  """把方案每个非空源行映射到实现与可执行验收。"""
2  from pathlib import Path
3  import pandas as pd
4  from .config import SOURCE, ROOT, write_csv, write_json
5
6
7  MAP = {
8      "7.1": ("src/q4/run.py;src/q4/rolling.py", "real smoke and CLI acceptance"),
9      "7.2": ("src/q4/data.py;src/q4/diagnostics.py", "Q4Tests.
10 test_actual_price_shape_alignment_and_reported_statistics"),
11      "7.3": ("src/q4/price.py", "Q4Tests.test_price_forecast_metrics_and_causality"),
12      "7.4": ("src/q4/data.py;src/q4/scenarios.py", "Q4Tests.test_joint_history_and_tail_reduction"),
13      "7.5": ("src/q4/scenarios.py;src/q4/optimization.py", "RescueTests.
14 test_positive_radius_matches_two_point_hand_worst_case"),
15      "7.6": ("src/q4/physics.py;src/q4/optimization.py;src/q4/rolling.py", "Q4Tests.
16 test_physical_call_separation; real Q4-2 smoke"),
17      "7.7": ("src/q4/scenarios.py;src/q4/physics.py;src/q4/optimization.py;src/q4/rolling.py", "
18 Q4Tests.test_prefix_uses_realized_slots_only; real Q4-3 smoke"),
19      "7.8": ("src/q4/diagnostics.py;src/q4/rolling.py;src/q4/export.py", "Q4Tests.
20 test_workbook_roundtrip_and_partial_guard"),
21      "7.9": ("src/q4", "full test suite and traceability audit"),
22  }
23
24 def generate(output: Path | None = None) -> dict:
25     output = output or ROOT / "docs/4"
26     chapter = "7.1"
27     rows = []
28     for number, text in enumerate(SOURCE.read_text(encoding="utf-8").splitlines(), 1):
29         if text.startswith("## 7."):
30             chapter = text.split()[1]
31         if text.strip():
32             implementation, verification = MAP[chapter]
33             rows.append({"source_line": number, "section": chapter, "source_text": text,
34                         "implementation": implementation, "verification": verification, "status": "
35 mapped"})
36     frame = pd.DataFrame(rows)
37     write_csv(output / "source_line_acceptance.csv", frame)
38     # 章节映射仅是索引；必须实际运行公式与拒绝路径断言。

```



```

34 import io
35 import unittest
36 from .test_rescue import RescueTests
37 stream = io.StringIO()
38 suite = unittest.defaultTestLoader.loadTestsFromTestCase(RescueTests)
39 checked = unittest.TextTestRunner(stream=stream, verbosity=2).run(suite)
40 (output / 'formula_assertions.txt').write_text(stream.getvalue(), encoding='utf-8')
41 audit = {"formula_assertions_run": checked.testsRun, "formula_assertions_passed": checked.
42         wasSuccessful(), "source": str(SOURCE), "nonempty_lines": len(frame), "mapped_lines": int((frame
43         .status == "mapped").sum()),
44         "complete": bool(len(frame) and (frame.status == "mapped").all() and checked.
45         wasSuccessful())}
46 write_json(output / "traceability_audit.json", audit)
47 return audit
48
49 if __name__ == "__main__":
50     print(generate())

```

src/q4/vertical_slice.py

```

1  """ 正式设置下3日Q42、2日Q43连续执行，验证后提供全年时间估计。 """
2  import time
3  from .config import ROOT, Config, write_json
4  from .data import read_inputs
5  from .diagnostics import baseline_metrics, write_window_policy
6  from .rolling import run_period
7  from .export import export_workbook, write_tables
8
9
10 def main():
11     folder=ROOT/'outputs/q4/raw/rescue_lp_vertical_20260913'
12     data=read_inputs(ROOT/'inputs/q4/rescue_processed')
13     config=Config()
14     summary={}
15     for mode,end in [('4-2',34),('4-3',33)]:
16         started=time.perf_counter()
17         output=folder/mode
18         frame=run_period(data,mode,config,output,end=end)
19         elapsed=time.perf_counter()-started
20         write_tables(frame,output)
21         workbook=export_workbook(frame,output,mode,config,smoke=True)
22         daily_count=end-31
23         summary[mode]=dict(days=daily_count,seconds=elapsed,estimated_full_seconds=elapsed/
24         daily_count*334,
25
26                             cost=float(frame.total_cost.sum()),emergency=float(frame.emergency.sum())
27
28                             ,
29
30                             charge=float(frame.charge.sum()),discharge=float(frame.discharge.sum()),
31                             max_balance_error=float(abs(frame.called+frame.discharge+frame.emergency-
32         frame.net_kwh-frame.charge-frame.spill).max()),
33                             template=workbook)
34         print(summary[mode],flush=True)
35     summary['baseline']=baseline_metrics(data)
36     write_window_policy(folder)
37     write_json(folder/'vertical_summary.json',summary)
38     print(summary,flush=True)

```

```

33
34
35 if __name__ == '__main__':
36     main()

```

B.5 敏感性分析程序

src/sensitivity/common.py

```

1  """独立敏感性目录、原子状态与同日基准比较。"""
2  from pathlib import Path
3  import hashlib,json,os,time
4  import numpy as np
5  import pandas as pd
6  ROOT=Path(__file__).resolve().parents[2]
7  RUN=ROOT/'outputs/sensitivity_q2_q3/raw/local_20260913'
8  Q2_RUN=ROOT/'outputs/q2/dispatch_runs/20260912_tail_reserve_final'
9  Q2=Q2_RUN/'processed/main'
10 Q3_RUN=ROOT/'outputs/q3/raw/full_priority_rescue_7h_20260912'
11 Q3=Q3_RUN/'main'
12
13
14 def safe(value):
15     if isinstance(value,dict):return {str(k):safe(v) for k,v in value.items()}
16     if isinstance(value,(list,tuple)):return [safe(v) for v in value]
17     if isinstance(value,np.ndarray):return safe(value.tolist())
18     if isinstance(value,np.generic):return safe(value.item())
19     if isinstance(value,float) and not np.isfinite(value):return None
20     return value
21
22
23 def write_json(path,value):
24     path=Path(path);path.parent.mkdir(parents=True,exist_ok=True)
25     temporary=path.with_suffix(path.suffix+f'_{os.getpid()}.pending')
26     with temporary.open('w',encoding='utf-8') as stream:
27         json.dump(safe(value),stream,ensure_ascii=False,indent=2,allow_nan=False);stream.flush();os.
28         fsync(stream.fileno())
29     temporary.replace(path)
30
31 def digest(*arrays):
32     h=hashlib.sha256()
33     for value in arrays:
34         a=np.asarray(value);h.update(str(a.shape).encode());h.update(np.ascontiguousarray(a).tobytes
35         ())
36     return h.hexdigest()
37
38 def file_hash(path):return hashlib.sha256(Path(path).read_bytes()).hexdigest()
39
40
41 def active(question,job,phase,**extra):
42     write_json(RUN/f'{question}_active.json',{'phase':phase,'date':job['date'],'parameter':job['
43     parameter'],'value':job['value'],'started_at':job.get('started_at',time.time()),**extra})
44

```

```

45 def metrics(frame):
46     return dict(total_cost=float((frame.planned_cost+frame.emergency_cost+(frame.adjustment_cost if
47         'adjustment_cost' in frame else 0)).sum()),
48                 planned_cost=float(frame.planned_cost.sum()),adjustment_cost=float(frame.
49         adjustment_cost.sum()) if 'adjustment_cost' in frame else 0.,
50                 emergency_kwh=float(frame.emergency.sum()),emergency_cost=float(frame.emergency_cost
51         .sum()),
52                 min_soc=float(frame.soc.min()),final_soc=float(frame.soc.iloc[-1]),
53                 charge_kwh=float(frame.charge.sum()),discharge_kwh=float(frame.discharge.sum()),
54                 throughput_kwh=float(frame.charge.sum()+frame.discharge.sum()))
55
56 def comparison(variant,baseline):
57     return dict(delta_cost_pct=100*(variant['total_cost']-baseline['total_cost'])/baseline['
58         total_cost'],
59                 delta_emergency_kwh=variant['emergency_kwh']-baseline['emergency_kwh'],
60                 delta_emergency_pct=100*(variant['emergency_kwh']-baseline['emergency_kwh'])/
61         baseline['emergency_kwh'] if baseline['emergency_kwh']>0 else None,
62                 emergency_pct_defined=baseline['emergency_kwh']>0)

```

src/sensitivity/prepare.py

```

1  """仅按预注册指标选择代表日，并检查正式条件池是否支持OFAT。"""
2  from pathlib import Path
3  import sys,json,hashlib
4  import numpy as np
5  import pandas as pd
6  ROOT=Path(__file__).resolve().parents[2]
7  sys.path.insert(0,str(ROOT))
8  from src.q3.data import read_inputs
9  from src.q3.config import Config,DT
10 from src.q3.scenarios import pool_indices
11 OUT=ROOT/'outputs/sensitivity_q2_q3/raw/local_20260913'
12 Q2=ROOT/'outputs/q2/dispatch_runs/20260912_tail_reserve_final/processed/main'
13 Q3=ROOT/'outputs/q3/raw/full_priority_rescue_7h_20260912/main'
14
15
16 def select(frame, rules):
17     selected=[];used=set()
18     for role,columns,ascending,mask in rules:
19         ordered=frame.loc[mask].sort_values(columns+['date'],ascending=ascending+[True],kind='stable
20         ')
21         chosen=ordered[~ordered.date.isin(used)].iloc[0]
22         used.add(chosen.date);selected.append({'role':role,**chosen.to_dict()})
23     return selected
24
25 def main():
26     OUT.mkdir(parents=True,exist_ok=True)
27     f=pd.read_csv(Q2/'dispatch.csv',float_precision='round_trip')
28     f['predicted_net_kwh']=(f.predicted_load_kw-f.predicted_pv_kw)*DT
29     f['net_mae']=abs(f.net_kwh-f.predicted_net_kwh)
30     f['total_cost']=f.planned_cost+f.emergency_cost
31     metrics=f.groupby('date',sort=True).agg(total_cost=('total_cost','sum'),emergency=('emergency','
32     sum'),net_mae=('net_mae','mean'),peak_net=('net_kwh','max'),initial_soc=('initial_soc','first'))
33     .reset_index()

```

```

32 metrics['median_cost_distance']=abs(metrics.total_cost-metrics.total_cost.median())
33 low=metrics.emergency<=metrics.emergency.median()
34 q2=select(metrics,[('常规日',[ 'median_cost_distance'],[True],low),('高预测误差日',[ 'net_mae'],[
False],np.ones(len(metrics),bool)),('高紧急购电日',[ 'emergency'],[False],np.ones(len(metrics),bool
)),('高系统压力日',[ 'peak_net'],[False],np.ones(len(metrics),bool))])
35 metrics.to_csv(OUT/'q2_selection_metrics.csv',index=False,encoding='utf-8')
36 data=read_inputs(ROOT/'outputs/q2/dispatch_runs/20260911_224501')
37 f=pd.read_csv(Q3/'dispatch.csv',float_precision='round_trip')
38 metrics=f.groupby('date',sort=True).agg(total_cost=('total_cost','sum'),emergency=('emergency','
sum'),adjustment_cost=('adjustment_cost','sum'),peak_net=('net_kwh','max'),initial_soc=('
initial_soc','first')).reset_index()
39 changes=[]
40 for date in metrics.date:
41     day=(pd.Timestamp(date)-pd.Timestamp('2025-01-01')).days
42     old=data.pv(day,0)
43     changes.append(float(np.mean(np.r_[abs(data.pv(day,6)[:36]-old[36:72]),abs(data.pv(day,12)
[:36]-old[72:108])]))))
44 metrics['pv_update_mae_kw']=changes
45 metrics['median_cost_distance']=abs(metrics.total_cost-metrics.total_cost.median())
46 any_day=np.ones(len(metrics),bool)
47 q3=select(metrics,[('常规日',[ 'median_cost_distance'],[True],any_day),('高预测更新日',[ '
pv_update_mae_kw'],[False],any_day),('高调整成本日',[ 'adjustment_cost'],[False],any_day),('高风险日
',[ 'emergency'],[False],any_day)])
48 metrics.to_csv(OUT/'q3_selection_metrics.csv',index=False,encoding='utf-8')
49 capacity=[]
50 for row in q3:
51     date=row['date'];day=(pd.Timestamp(date)-pd.Timestamp('2025-01-01')).days
52     baseline=json.loads((Q3/'audit'/f'{date}.json').read_text())
53     for node in baseline['nodes']:
54         h=node['hour'];ids,el,ep,scales=data.history(day,h,data.selected[day])
55         pool,meta=pool_indices(ids,scales,day,data.pv(day,h).sum()*DT,Config())
56         assert len(pool)==node['scenarios']['candidate_count']
57         capacity.append({'date':date,'hour':h,'frozen_pool_size':len(pool),'supports_S30':len(
pool)>=30,'history_days':ids[pool].tolist()})
58 for row in q2:
59     a=json.loads((Q2/'daily_audit'/f'{row['date']}.json').read_text());selected=a[str(a['
selected_K'])]
60     row.update(K=a['selected_K'],S=selected['S'],M=selected['M'],solver_gap=selected['
selected_solver'].get('requested_gap'),reserve_max=selected['selection']['dynamic_soc_reserve'][:
'max_reserve_kwh'])
61 result={'q2':q2,'q3':q3,'q3_pool_capacity':capacity,'q2_low_emergency_threshold':float(pd.
read_csv(OUT/'q2_selection_metrics.csv').emergency.median()),'rules_frozen_before_experiments':
True,'tie_break':'next unused by ranking then ascending date'}
62 (OUT/'representatives.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding=
'utf-8')
63 print(json.dumps(result,ensure_ascii=False,indent=2))
64
65
66 if __name__=='__main__':main()

```

src/sensitivity/q2.py

```

1 """固定正式K/S/预测与历史块，仅扰动reserve倍率或tail比例。"""
2 import sys,json,time
3 from pathlib import Path
4 import numpy as np

```

```

5 import pandas as pd
6 from .common import ROOT, RUN, Q2, Q2_RUN, digest, file_hash, write_json, metrics, active
7 sys.path.insert(0, str(ROOT/'src/q2'))
8 from policy import Policy, DT, replay
9 from shadows import open_library, combined
10 from scenarios import joint_blocks, construct, dynamic_reserve
11 from rolling import _solve, dispatch_frame
12 from protocol import PROTOCOL
13
14
15 def prepare():
16     store, _ = open_library(Q2_RUN/'raw/shadows')
17     actual_frame = pd.read_csv(Q2_RUN/'inputs/processed/timeseries.csv', float_precision='round_trip')
18     actual = actual_frame[['load_kw', 'pv_kw']].to_numpy().reshape(365, 144, 2)
19     prices = pd.read_csv(Q2_RUN/'inputs/processed/prices.csv', float_precision='round_trip').
20     price_yuan_per_kwh.to_numpy()
21     executed = pd.read_csv(Q2/'dispatch.csv', float_precision='round_trip')
22     representatives = json.loads((RUN/'representatives.json').read_text())['q2']
23     for row in representatives:
24         date = row['date']; day = (pd.Timestamp(date) - pd.Timestamp('2025-01-01')).days
25         audit_path = Q2/'daily_audit'/f'{date}.json'; audit = json.loads(audit_path.read_text()); k = audit[
26         'selected_K']; selected = audit[str(k)]
27         selection = selected['selection']; forecasts = combined(store, selection['pipeline'])
28         origins = np.asarray(selection['origin_indices']); all_origins = np.asarray(selection[
29         'all_origin_indices'])
30         assert np.all(origins + k <= day) and np.all(all_origins + k <= day)
31         point = np.asarray(forecasts[day, :k]); blocks = joint_blocks(actual[:day], forecasts, origins, k);
32         all_blocks = joint_blocks(actual[:day], forecasts, all_origins, k)
33         plan = pd.read_csv(Q2/'frozen_plans'/f'{date}.csv', float_precision='round_trip')
34         policy = Policy(plan.grid, plan.charge_cap, plan.discharge_cap, plan.soc_reserve_kwh)
35         calculated, _ = dynamic_reserve(all_blocks, .8); np.testing.assert_allclose(policy.reserve[:144],
36         calculated, atol=1e-8, rtol=0)
37         p = np.tile(prices, k); net, w, scenes = construct(point, blocks, p, selected['S'], origins=origins,
38         all_blocks=all_blocks, all_origins=all_origins, tail_fraction=.2)
39         oldscenes = selected['candidates'][str(selected['S'])]['scenarios']
40         assert scenes['selected_origin_indices'] == oldscenes['selected_origin_indices']
41         np.testing.assert_allclose(w, oldscenes['weights'], atol=1e-12, rtol=0)
42         initial = row['initial_soc']; responses = [replay(policy, n, initial) for n in net]
43         value = float(p@policy.grid + sum(weight*(5*p@resp['emergency']) for weight, resp in zip(w,
44         responses)))
45         np.testing.assert_allclose(value, selected['selected_solver']['objective'], atol=1e-5, rtol=0)
46         base = executed[executed.date == date].copy()
47         real = (actual[day, :, 0] - actual[day, :, 1]) * DT
48         for key, value in replay(policy, first(), real, initial).items(): np.testing.assert_allclose(base
49         [key], value, atol=1e-7, rtol=0)
50         folder = RUN/'inputs/q2'/date; folder.mkdir(parents=True, exist_ok=True)
51         np.savez(folder/'frozen.npz', point=point, blocks=blocks, all_blocks=all_blocks, origins=origins,
52         all_origins=all_origins,
53         prices=p, net=net, weights=w, reserve=policy.reserve, real_net=real, initial=initial)
54         base.to_csv(folder/'baseline_dispatch.csv', index=False, encoding='utf-8', float_format='%.17g'
55         )
56         execution = audit['execution']
57         metadata = {'date': date, 'role': row['role'], 'K': k, 'S': selected['S'], 'initial_soc': initial,
58         'baseline': metrics(base),
59         'baseline_solver': selected['selected_solver'], 'gap': float(execution['gap']),
60         'seconds': float(execution['base_solver_seconds']),
61         'rescue_seconds': float(execution['rescue_seconds']), 'threads': int(execution.get('

```

```

solver_threads',1)), 'refine_seconds':float(execution.get('refine_seconds',5.)),
50     'fixed_history_sha256':digest(blocks,all_blocks,origins,all_origins), 'point_sha256'
    ':digest(point),
51     'baseline_scenarios_sha256':digest(net,w), 'baseline_reserve_sha256':digest(policy.
reserve),
52     'baseline_audit_sha256':file_hash(audit_path), 'baseline_plan_sha256':file_hash(Q2/
'frozen_plans'/f'{date}.csv'),
53     'baseline_replay_verified':True, 'scenario_reconstruction_verified':True}
54     write_json(folder/'metadata.json', metadata)
55
56
57 def run(job, folder):
58     frozen=RUN/'inputs/q2'/job['date']; a=np.load(frozen/'frozen.npz'); meta=json.loads((frozen/'
metadata.json').read_text())
59     reserve=a['reserve'].copy(); net=a['net']; weights=a['weights']; scenes={}
60     if job['parameter']=='beta_R': reserve*=job['value']
61     elif job['parameter']=='rho_tail':
62         net, weights, scenes=construct(a['point'], a['blocks'], a['prices'], meta['S'], origins=a['origins
'], all_blocks=a['all_blocks'], all_origins=a['all_origins'], tail_fraction=job['value'])
63     else: raise ValueError('未授权的Q2实验参数')
64     assert len(net)==meta['S']
65     assert digest(a['blocks'], a['all_blocks'], a['origins'], a['all_origins'])==meta['
fixed_history_sha256']
66     if job['parameter']=='rho_tail': assert digest(reserve)==meta['baseline_reserve_sha256']
67     else: assert digest(net, weights)==meta['baseline_scenarios_sha256']
68     np.savez(folder/'variant_inputs.npz', net=net, weights=weights, reserve=reserve)
69     active('q2', job, '求解正式策略扰动', hour=0, detail=f"K={meta['K']}, S={meta['S']}, gap={meta['gap
']:.0%}, 线程={meta['threads']}")
70     started=time.perf_counter(); cache={}
71     settings=dict(reserve=reserve, solver_gap=meta['gap'], solver_threads=meta['threads'],
refine_seconds=meta['refine_seconds'], solve_cache=cache, cache_key='variant')
72     policy, _, info=_solve(net, a['prices'], weights, float(a['initial']), solver_seconds=meta['seconds'
], **settings)
73     attempts=[info]
74     if policy is None or not info['reliable']:
75         active('q2', job, '原120秒预算未认证, 沿用正式900秒追加预算', hour=0, detail='精度与模型不变: 保存首次尝试
')
76         write_json(folder/'base_attempt.json', info)
77         policy, _, info=_solve(net, a['prices'], weights, float(a['initial']), solver_seconds=meta['
rescue_seconds'], **settings)
78         attempts.append(info)
79     if policy is None: raise RuntimeError('求解预算内无可行策略')
80     frame=dispatch_frame(pd.Timestamp(job['date']), policy, a['real_net'], float(a['initial']), a['
prices'][:144])
81     frame.to_csv(folder/'dispatch.csv', index=False, encoding='utf-8', float_format='%.17g')
82     pd.DataFrame({'grid':policy.grid, 'charge_cap':policy.charge_cap, 'discharge_cap':policy.
discharge_cap, 'reserve':policy.reserve}).to_csv(folder/'policy.csv', index=False, encoding='utf-8'
, float_format='%.17g')
83     write_json(folder/'audit.json', {'solver':info, 'attempts':attempts, 'scenarios':scenes, 'baseline':
meta, 'history_frozen':True})
84     return {**metrics(frame), 'seconds':time.perf_counter()-started, 'reliable':bool(info['reliable'])
, 'max_gap':info.get('mip_gap', info.get('gap', 0)),
85         'nodes':1, 'certified_nodes':int(info['reliable']), 'history_sha256':meta['
fixed_history_sha256'], 'physical_verified':True}, meta['baseline']

```

src/sensitivity/q3.py

```
1 """固定正式历史条件池与lambda基准，节点内仍沿用原Q3求解与真实反馈。"""
2 from dataclasses import replace
3 import json,time
4 import numpy as np
5 import pandas as pd
6 from .common import ROOT,RUN,Q3,Q3_RUN,digest,file_hash,write_json,metrics,active
7 from src.q3.config import Config,DT
8 from src.q3.data import read_inputs
9 from src.q3.physics import Policy,replay,adjustment
10 from src.q3.scenarios import pool_indices,reduce_errors,Scenarios
11 from src.q3.optimization import solve,objective
12 from src.q3.rolling import frame_for,validate_frame,previous_net
13
14
15 def base_config():
16     values=json.loads((Q3_RUN/'execution.json').read_text())['config'];values['nodes']=tuple(values[
17         'nodes'])
18     return Config(**values)
19
20 def inputs():
21     source=json.loads((Q3_RUN/'execution.json').read_text())['q2_run']
22     return read_inputs(__import__('pathlib').Path(source))
23
24
25 def prepare():
26     data=inputs();config=base_config();executed=pd.read_csv(Q3/'dispatch.csv',float_precision='
27         round_trip')
28     for row in json.loads((RUN/'representatives.json').read_text())['q3']:
29         date=row['date'];day=(pd.Timestamp(date)-pd.Timestamp('2025-01-01')).days
30         audit_path=Q3/'audit'/f'{date}.json';audit=json.loads(audit_path.read_text())
31         folder=RUN/'inputs/q3'/date;folder.mkdir(parents=True,exist_ok=True)
32         np.save(folder/'actual_day.npy',data.actual[day])
33         base=executed[executed.date==date].copy();g0=base.g0.to_numpy();node_meta=[]
34         for node in audit['nodes']:
35             hour=node['hour'];ids,el,ep,scales=data.history(day,hour,data.selected[day]);pv=data.pv(
36                 day,hour)
37             ix,pool=pool_indices(ids,scales,day,pv.sum()*DT,config);prices=data.prices[(hour*6+np.
38                 arange(144))%144]
39             reps,w,reduction=reduce_errors(el[ix]-ep[ix],prices,config)
40             assert ids[ix][reps].tolist()==node['scenarios']['history_days']
41             np.testing.assert_allclose(w,node['scenarios']['weights'],atol=1e-12,rtol=0)
42             point_load=data.load(day)[hour*6:hour*6+144];load=np.maximum(point_load+el[ix][reps],0);
43             photovoltaic=np.maximum(pv+ep[ix][reps],0)
44             net=(load-photovoltaic)*DT;policy=Policy(np.asarray(node['grid']),np.asarray(node['
45                 charge_cap']),np.asarray(node['discharge_cap']))
46             responses=[replay(policy,path,node['initial_soc']) for path in net]
47             coefficient=node['calibration']['median_unscaled']*node['calibration']['scale']
48             reference=None if hour==0 else g0[hour*6:]
49             val=objective(policy,responses,w,prices,reference,144-hour*6,coefficient)
50             np.testing.assert_allclose(val,node['solver']['objective'],atol=1e-5,rtol=0)
51             np.savez(folder/f'{hour:02d}.npz',point_load=point_load,point_pv=pv,load_errors=el[ix],
52                 pv_errors=ep[ix],history_days=ids[ix],
53                 prices=prices,load=load,pv=photovoltaic,weights=w,terminal=coefficient,
54                 previous_net=previous_net(data,day,hour))
```



```

48     node_meta.append({'hour':hour, 'pool_size':len(ix), 'history_sha256':digest(ids[ix],el[ix]
49 ],ep[ix]), 'point_sha256':digest(point_load,pv),
50     'baseline_scenarios_sha256':digest(load,photovoltaic,w), '
51     terminal_baseline':coefficient, 'calibration':node['calibration'],
52     'baseline_solver':node['solver']})
53     reconstructed=replay(Policy(base.grid.to_numpy(),base.charge_cap.to_numpy(),base.
54     discharge_cap.to_numpy(),base.net_kwh.to_numpy(),row['initial_soc']))
55     for key,value in reconstructed.items():np.testing.assert_allclose(base[key],value,atol=1e-7,
56     rtol=0)
57     base.to_csv(folder/'baseline_dispatch.csv',index=False,encoding='utf-8',float_format='%.17g'
58 )
59     write_json(folder/'metadata.json',{'date':date, 'role':row['role'], 'day_index':day, '
60     initial_soc':row['initial_soc'], 'baseline':metrics(base),
61     'nodes':node_meta, 'baseline_audit_sha256':file_hash(
62     audit_path), 'baseline_replay_verified':True, 'scenario_reconstruction_verified':True})
63
64
65
66 def scenarios_for(a,config):
67     reps,weights,reduction=reduce_errors(a['load_errors']-a['pv_errors'],a['prices'],config)
68     load=np.maximum(a['point_load']+a['load_errors'][reps],0);pv=np.maximum(a['point_pv']+a['
69     pv_errors'][reps],0)
70     assert len(weights)==config.scenarios
71     return Scenarios(load,pv,weights,{ 'S':len(weights), 'history_pool_frozen':True, 'pool_size':len(a[
72     'history_days']),
73     'history_days':a['history_days'][reps].tolist(), 'weights':
74     weights.tolist(),**reduction})
75
76
77
78 def run(job,folder,data=None):
79     data=inputs() if data is None else data
80     frozen=RUN/'inputs/q3'/job['date'];meta=json.loads((frozen/'metadata.json').read_text());base=
81     base_config()
82     if job['parameter']=='S':config=replace(base,scenarios=int(job['value']))
83     elif job['parameter']=='lambda_E':config=replace(base,terminal_scale=float(job['value']))
84     elif job['parameter']=='S_tail':config=replace(base,tail=int(job['value']))
85     else:raise ValueError('未授权的Q3参数')
86     np.testing.assert_array_equal(data.actual[meta['day_index']],np.load(frozen/'actual_day.npy'))
87     soc=meta['initial_soc'];frames=[];audits=[];g0=None;started=time.perf_counter()
88     for h,node_meta in zip((0,6,12,18),meta['nodes']):
89         a=np.load(frozen/f'{h:02d}.npz');assert digest(a['history_days'],a['load_errors'],a['
90         pv_errors'])==node_meta['history_sha256']
91         scenes=scenarios_for(a,config)
92         if job['parameter']=='lambda_E':assert digest(scenes.load,scenes.pv,scenes.weights)==
93         node_meta['baseline_scenarios_sha256']
94         coefficient=float(a['terminal'])*config.terminal_scale
95         reference=None if h==0 else g0[h*6:]
96         active('q3',job,'求解节点',hour=h,detail=f'S={config.scenarios}, tail={config.tail}, lambda倍率
97         ={config.terminal_scale:g}, gap=3%')
98         hard=config.hard_06_seconds if h==6 else config.hard_other_seconds
99         solution=solve(scenes,a['prices'],soc,config,reference=reference,today=144-h*6,terminal=
100         coefficient,
101         previous_net=float(a['previous_net']),checkpoint=folder/'nodes'/f'{h:02d}',
102         max_slices=1,require=True,hard_seconds=hard)
103         if solution.policy is None:raise RuntimeError('Q3节点未得到可执行策略')
104         if h==0:g0=solution.policy.grid.copy()
105         frame=frame_for(data,meta['day_index'],h,solution.policy.part(0,36),soc,config)
106         frame['g0']=g0[h*6:(h+6)*6];frame['initial_soc']=meta['initial_soc'];frames.append(frame)

```



```

89     audits.append({'hour':h,'initial_soc':soc,'solver':solution.audit,'scenarios':scenes.audit,'
terminal':coefficient,'terminal_baseline':float(a['terminal']),
90                 'history_sha256':node_meta['history_sha256'],'grid':solution.policy.grid.
tolist(),'charge_cap':solution.policy.charge_cap.tolist(),'discharge_cap':solution.policy.
discharge_cap.tolist()})
91     soc=float(frame.soc.iloc[-1]);write_json(folder/'partial_audit.json',{'nodes':audits})
92     frame=pd.concat(frames,ignore_index=True);frame['planned_cost']=frame.price*frame.g0
93     frame['adjustment_cost']=adjustment(frame.grid.to_numpy(),frame.g0.to_numpy(),frame.price.
to_numpy())
94     frame['emergency_cost']=5*frame.price*frame.emergency;frame['total_cost']=frame.planned_cost+
frame.adjustment_cost+frame.emergency_cost
95     validate_frame(frame,config)
96     frame.to_csv(folder/'dispatch.csv',index=False,encoding='utf-8',float_format='%.17g')
97     write_json(folder/'audit.json',{'nodes':audits,'baseline':meta,'frozen_pool':True,'
only_first_soc_anchored_later_soc_from_variant':True})
98     certified=sum(n['solver']['reliable'] for n in audits)
99     gaps=[n['solver'].get('gap') for n in audits if n['solver'].get('gap') is not None]
100    return {**metrics(frame),'seconds':time.perf_counter()-started,'reliable':certified==4,'max_gap'
:max(gaps) if gaps else None,
101           'nodes':4,'certified_nodes':certified,'physical_verified':True},meta['baseline']

```

src/sensitivity/q4_experiment.py

```

1  """Q4-S1/S2与四节点迁移：只复用基准，只做20个代表日OFAT实验。"""
2  from pathlib import Path
3  from dataclasses import asdict, replace
4  import argparse
5  import hashlib
6  import json
7  import os
8  import sys
9  import time
10 import numpy as np
11 import pandas as pd
12
13 ROOT = Path(__file__).resolve().parents[2]
14 RUN = ROOT / 'outputs/q4/raw/sensitivity_20260913'
15 DOC = ROOT / 'docs/sensitivity_q4'
16 BASE = {'4-2': ROOT/'outputs/q4/raw/rescue_lp_q42_20260913_v2',
17        '4-3': ROOT/'outputs/q4/raw/rescue_lp_q43_20260913_v3'}
18
19
20 def read_json(path):
21     return json.loads(Path(path).read_text(encoding='utf-8'))
22
23
24 def sha(path):
25     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
26
27
28 def digest(*arrays):
29     h = hashlib.sha256()
30     for a in arrays:
31         a = np.ascontiguousarray(a)
32         h.update(str((a.shape, a.dtype)).encode())
33         h.update(a.tobytes())

```

```

34     return h.hexdigest()
35
36
37 def configure(mode):
38     """从对应正式快照加载Q4; Q3/Q2只读依赖逐文件校验。"""
39     os.environ['Q4_WORKSPACE_ROOT'] = str(ROOT)
40     os.environ['Q3_WORKSPACE_ROOT'] = str(ROOT)
41     sys.path.insert(0, str(ROOT))
42     import src.q4
43     frozen = ROOT / ('docs/4/rescue_lp/frozen_source_v2' if mode == '4-2'
44                     else 'docs/4/rescue_lp/frozen_source_v3')
45     src.q4.__path__ = [str(frozen/'src/q4')]
46     state = read_json(BASE[mode]/'state.json')
47     for relative, expected in state['source_hashes'].items():
48         source = frozen/'src'/relative if relative.startswith('q4/') else ROOT/'src'/relative
49         assert sha(source) == expected, ('source changed', source)
50     for name, expected in state['input_hashes'].items():
51         assert sha(name) == expected, ('input changed', name)
52     from src.q4.config import Config
53     from highspy import Highs
54     assert Highs().version() == state['solver_version']
55     values = dict(state['config']); values['nodes'] = tuple(values['nodes'])
56     return Config(**values), state
57
58
59 def inputs(state):
60     from src.q3.data import read_inputs
61     from src.q4.data import Inputs, read_prices
62     from src.q4.price import Forecasts
63     from src.q4.config import ATTACHMENTS, Q2_RUN, write_json
64     q3 = read_inputs(Q2_RUN, RUN/'inputs/q3')
65     folder = ROOT/'inputs/q4/rescue_processed'
66     provenance = read_json(folder/'source_manifest.json')
67     values = pd.read_csv(folder/'price_forecasts.csv', float_precision='round_trip').to_numpy(float)
68     residuals = pd.read_csv(folder/'price_residuals.csv', float_precision='round_trip').to_numpy(float)
69     assert hashlib.sha256(values.tobytes()).hexdigest() == state['price_forecasts_array_sha256']
70     for decision in q3.provenance['q2_decisions']:
71         old = next(d for d in provenance['q3_provenance'] if d['q2_decisions'] == decision['date'])
72         assert decision['sha256'] == old['sha256']
73     write_json(RUN/'inputs/frozen_forecasts.json', {
74         'price_forecasts_sha256': state['price_forecasts_array_sha256'],
75         'load_pv_inputs': q3.provenance, 'price_model_retrained': False})
76     return Inputs(q3, read_prices(ATTACHMENTS/'附件4.xlsx'),
77                 Forecasts(values, residuals, provenance['price_model']), provenance)
78
79
80 def metrics(frame, audit):
81     nodes = audit.get('nodes', [audit])
82     solvers = [n['solver'] for n in nodes]
83     difference = (frame.grid - frame.g0).to_numpy()
84     return dict(total_cost=float(frame.total_cost.sum()), planned_cost=float(frame.planned_cost.sum()),
85               emergency_kwh=float(frame.emergency.sum()), emergency_cost=float(frame.emergency_cost.sum()),
86               min_soc=float(min(frame.initial_soc.iloc[0], frame.soc.min()))),

```

```

87         final_soc=float(frame.soc.iloc[-1]), initial_soc=float(frame.initial_soc.iloc[0]),
88         adjustment_up_kwh=float(np.maximum(difference, 0).sum()),
89         adjustment_down_kwh=float(np.maximum(-difference, 0).sum()),
90         adjustment_abs_kwh=float(abs(difference).sum()), adjustment_net_kwh=float(difference
    .sum()),
91         adjustment_cost=float(frame.adjustment_cost.sum()),
92         solve_seconds=float(sum(s['seconds'] for s in solvers)),
93         solver_native_seconds=float(sum(s['solver_seconds'] for s in solvers)),
94         solve_wall_seconds=float(sum(s['wall_seconds'] for s in solvers)),
95         scenario_seconds=float(sum(n['scenarios']['construction_seconds'] for n in nodes)),
96         nodes=len(nodes), all_optimal=all(s['status'] == 'Optimal' and s['reliable'] for s
    in solvers),
97         max_gap=float(max(s['gap'] for s in solvers)))
98
99
100 def baseline(mode, date):
101     frame = pd.read_csv(BASE[mode]/f'days/{date}.csv', float_precision='round_trip')
102     audit = read_json(BASE[mode]/f'audit/{date}.json')
103     return frame, audit
104
105
106 def prepare(data, config):
107     """先固定规则、候选指标和日期，完成后才能开始任何敏感性求解。"""
108     from src.q4.config import write_csv, write_json, DT
109     from src.q4.scenarios import _scale
110     selection_path = RUN/'representatives.json'
111     if selection_path.exists():
112         return read_json(selection_path)
113     rows = []
114     for day in range(151, 365):
115         date = str((pd.Timestamp('2025-01-01') + pd.Timedelta(days=day)).date())
116         days, en, ep = data.joint_history(day, 0, 288)
117         keep = days >= day-config.residual_window
118         sn, sp = _scale(en[keep, :144]), _scale(ep[keep, :144])
119         actual_net = (data.q3.actual[day, :, 0]-data.q3.actual[day, :, 1])*DT
120         net_error = actual_net-data.point_net(day, 0, 288)[:144]
121         price_error = data.actual_prices[day]-data.point_price(day, 0, 288)[:144]
122         risk = float(np.mean(np.maximum(net_error/sn, 0)*np.maximum(price_error/sp, 0)))
123         rows.append(dict(date=date, day=day, price_std=float(data.actual_prices[day].std()),
124             joint_risk=risk, peak_net_kwh=float(actual_net.max())))
125     risks = pd.DataFrame(rows)
126     selected = {}
127     for mode in BASE:
128         daily = pd.read_csv(BASE[mode]/'daily.csv', float_precision='round_trip')
129         annual_median = float(daily.total_cost.median())
130         table = risks.merge(daily[['date', 'total_cost', 'emergency', 'adjustment_cost']], on='date')
131         table['median_distance'] = abs(table.total_cost-annual_median)
132         rules = [('常规日', 'median_distance', True), ('高价格波动日', 'price_std', False),
133             ('高联合风险日', 'joint_risk', False), ('高运行压力日', 'emergency', False)]
134         if mode == '4-3':
135             rules = [rules[0], rules[2]]
136         used, chosen = set(), []
137         for role, key, ascending in rules:
138             row = table[~table.date.isin(used)].sort_values([key, 'date'], ascending=[ascending, True
139 ]).iloc[0]
140             used.add(row.date)
141             chosen.append(dict(role=role, date=row.date, day=int(row.day), metric=key,

```

```

141         metric_value=float(row[key]), annual_cost_median=annual_median))
142     selected[mode] = chosen
143     write_csv(RUN/f'selection_metrics_{mode}.csv',table)
144     write_json(selection_path, {'rules': 'June-Dec; ordinary closest to full formal-year median;
    volatility population std; joint risk mean positive standardized OOS net residual times positive
    standardized OOS price residual (scales from available 56-day history); pressure maximum
    baseline emergency kWh; ties date ascending; duplicates advance rank',
145         'selected_before_variants': True, 'modes': selected})
146     return read_json(selection_path)
147
148
149 def matrix_digest(scenarios, initial, reference=None, today=None):
150     from src.q4.optimization import build_matrix
151     objective, bounds, integer, constraints, indices, offset = build_matrix(scenarios, initial,
    reference=reference, today=today)
152     matrix = constraints.A
153     h = hashlib.sha256()
154     for a in (objective,bounds.lb,bounds.ub,constraints.lb,constraints.ub,matrix.indptr,matrix.
    indices,matrix.data):
155         h.update(np.asarray(a).tobytes())
156     return h.hexdigest()
157
158
159 def freeze(data, day, mode, config, frame, audit):
160     from src.q4.scenarios import construct
161     from src.q4.config import write_json
162     nodes = audit.get('nodes', [dict(audit, hour=0)])
163     cache = {}
164     for n in nodes:
165         hour = n['hour']; length = 288 if mode == '4-2' else 144
166         s = construct(data, day, hour, length, config)
167         old = n['scenarios']
168         for key in ['history_days','representative_pool_indices','tail_indices_in_pool','
    represented_counts']:
169             assert s.audit[key] == old[key], key
170             for key in ['weights','bootstrap_distances','radius']:
171                 np.testing.assert_allclose(s.audit[key], old[key], atol=1e-12,rtol=1e-12)
172             reference = None if hour == 0 else np.asarray(nodes[0]['grid'])[hour*6:]
173             today = None if mode == '4-2' else 144-hour*6
174             assert matrix_digest(s,n['initial_soc'],reference,today) == n['solver']['matrix_digest'], (
    mode,day,hour,'baseline matrix mismatch')
175             signature = digest(s.net,s.prices,s.weights,s.distance)
176             s.audit.update(scenario_fingerprint=signature, baseline_matrix_verified=True)
177             cache[hour] = s
178             directory = RUN/'frozen'/mode/audit['date']; directory.mkdir(parents=True,exist_ok=True)
179             if not (directory/f'{hour:02d}.json').exists():
180                 np.savez_compressed(directory/f'{hour:02d}.npz',net=s.net,prices=s.prices,weights=s.
    weights,
181                     distance=s.distance,radius=s.radius)
182             write_json(directory/f'{hour:02d}.json',s.audit)
183         else:
184             assert read_json(directory/f'{hour:02d}.json')['scenario_fingerprint'] == signature
185     return cache
186
187
188 def run(mode):
189     os.environ["Q4_SENS_MODE"] = mode

```

```

190 config, state = configure(mode)
191 from src.q4.config import write_csv, write_json
192 from src.q4 import rolling
193 from src.q4.scenarios import construct
194 from src.q4.optimization import LimitedSolve
195 data = inputs(state)
196 representatives = prepare(data, config)['modes'][mode]
197 # 每种模式首次读入时固定所有原始基准文件hash: 恢复时校验, 禁止混用。
198 source_files = [BASE[mode]/'state.json']
199 for r in representatives:
200     source_files += [BASE[mode]/f"days/{r['date']}.csv", BASE[mode]/f"audit/{r['date']}.json"]
201 manifest = {str(p):sha(p) for p in source_files}
202 annual = BASE[mode]/'dispatch.csv'
203 if annual.is_file():
204     manifest[str(annual)] = sha(annual)
205 else:
206     reference = BASE[mode]/'annual_dispatch_reference.json'
207     manifest[str(annual)] = read_json(reference)['sha256']
208     manifest[str(reference)] = sha(reference)
209 manifest_path = RUN/f'baseline_hashes_{mode}.json'
210 if manifest_path.exists():
211     assert read_json(manifest_path) == manifest
212 else:
213     write_json(manifest_path, manifest)
214 cached, baseline_rows = {}, []
215 for r in representatives:
216     f, a = baseline(mode, r['date'])
217     rolling.validate_frame(f, mode, config)
218     cached[r['date']] = freeze(data, r['day'], mode, config, f, a)
219     baseline_rows.append(dict(mode=mode, date=r['date'], role=r['role'], parameter='baseline',
220 setting='1.00 / 0.5, 0.5',
221
222                                     baseline_reused=True, **metrics(f, a), delta_cost_pct=0.,
223
224                                     delta_emergency_kwh=0.,
225
226                                     delta_emergency_pct=0. if f.emergency.sum()>0 else None,
227
228                                     delta_adjustment_cost=0.))
229 write_json(RUN/f'baseline_metrics_{mode}.json', baseline_rows)
230 stages = [('rho', 0.75), ('rho', 1.25)]
231 if mode == '4-2': stages += [('weight', 0.7), ('weight', 0.3)]
232 original = rolling.construct
233 for parameter, value in stages:
234     for r, base in zip(representatives, baseline_rows):
235         date = r['date']; case = f'{mode}_{parameter}_{value}_{date}'
236         directory = RUN/'cases'/case; directory.mkdir(parents=True, exist_ok=True)
237         receipt = directory/'result.json'
238         if receipt.exists():
239             saved = read_json(receipt)
240             assert saved['artifact_hashes'] == {name:sha(directory/name) for name in ['dispatch.
241 csv', 'audit.json']}
242             print('REUSED', case, flush=True); continue
243         test = replace(config, **({f'dro_scale':value} if parameter=='rho' else {f'net_weight':
244 value}))
245         assert [k for k in asdict(config) if asdict(config)[k]!=asdict(test)[k]] == [f'dro_scale
246 ' if parameter=='rho' else f'net_weight'])
247         write_json(RUN/'active.json', dict(status='running', case=case, started_at=time.time()))
248         print('START', case, flush=True)
249         def selected_scenarios(data_arg, day, hour, length, config_arg):
250             base_s = cached[date][hour]

```

```

241         if parameter == 'rho':
242             s = replace(base_s, radius=base_s.radius*value, audit=dict(base_s.audit,
243                               radius=base_s.radius*value, radius_scale=value, construction_reused=
True))
244             assert digest(s.net, s.prices, s.weights, s.distance) == base_s.audit['
scenario_fingerprint']
245         else:
246             s = construct(data_arg, day, hour, length, config_arg)
247             assert s.audit['history_days'] == base_s.audit['history_days']
248             assert s.audit['tail_indices_in_pool'] == base_s.audit['tail_indices_in_pool']
249             assert not np.array_equal(s.distance, base_s.distance)
250             s.audit.update(full_weight_pipeline_rebuilt=True, construction_reused=False,
scenario_fingerprint=digest(s.net, s.prices, s.weights, s.distance))
251             s.audit['base_radius'] = base_s.radius
252         return s
253     rolling.construct = selected_scenarios
254     started = time.perf_counter()
255     try:
256         runner = rolling.run_day_q42 if mode=='4-2' else rolling.run_day_q43
257         frame, audit = runner(data, r['day'], base['initial_soc'], test)
258     except LimitedSolve as error:
259         write_json(directory/'failure.json', error.audit)
260         raise
261     finally:
262         rolling.construct = original
263         row = metrics(frame, audit)
264         assert row['all_optimal']
265         row.update(mode=mode, date=date, role=r['role'], parameter=parameter, setting=str(value),
baseline_reused=False, day_elapsed_seconds=time.perf_counter()-started,
266         delta_cost_pct=100*(row['total_cost']-base['total_cost'])/base['total_cost'],
267         delta_emergency_kwh=row['emergency_kwh']-base['emergency_kwh'],
268         delta_emergency_pct=100*(row['emergency_kwh']-base['emergency_kwh'])/base['
emergency_kwh'] if base['emergency_kwh']>0 else None,
269         delta_adjustment_cost=row['adjustment_cost']-base['adjustment_cost'])
270         audit['experiment'] = dict(parameter=parameter, value=value, base_config=asdict(config),
test_config=asdict(test),
271         formal_source_hashes=state['source_hashes'],
baseline_reoptimized=False)
272         write_csv(directory/'dispatch.csv', frame); write_json(directory/'audit.json', audit)
273         write_json(receipt, dict(metrics=row, artifact_hashes={name:sha(directory/name) for name
in ['dispatch.csv', 'audit.json']}))
274         print('DONE', case, round(row['delta_cost_pct'], 4), '%', round(row['solve_seconds'], 2), 's',
flush=True)
275     assert manifest == {str(p):sha(p) for p in source_files}
276     write_json(RUN/f'completed_{mode}.json', dict(status='complete', days=len(representatives)*len(
stages), baseline_days_reused=len(representatives)))
277
278
279
280
281 if __name__ == '__mp_main__':
282     configure(os.environ['Q4_SENS_MODE'])
283
284
285 if __name__ == '__main__':
286     parser = argparse.ArgumentParser(); parser.add_argument('mode', choices=['4-2', '4-3'])
287     run(parser.parse_args().mode)

```

src/sensitivity/q4_report.py

```
1  """Q4敏感性实际结果独立验收、代表日汇总与UTF-8论文报告。"""
2  from pathlib import Path
3  import html
4  import json
5  import re
6  import sys
7  import numpy as np
8  import pandas as pd
9
10 ROOT = Path(__file__).resolve().parents[2]
11 sys.path.insert(0, str(ROOT))
12 from src.sensitivity.q4_experiment import RUN, DOC, BASE, read_json, sha, metrics
13 from src.q4.config import Config, write_csv, write_json
14 from src.q4.rolling import validate_frame
15 WEB = ROOT/'outputs/processed/sensitivity_q4'
16
17
18 def aggregate(rows):
19     fields = ['delta_cost_pct', 'total_cost', 'planned_cost', 'emergency_kwh', 'emergency_cost',
20              'delta_emergency_kwh', 'delta_emergency_pct', 'min_soc', 'final_soc', 'solve_seconds',
21              'solve_wall_seconds', 'adjustment_abs_kwh', 'adjustment_up_kwh', 'adjustment_down_kwh',
22              'adjustment_cost', 'delta_adjustment_cost']
23     return rows.groupby(['mode', 'parameter', 'setting'], sort=False)[fields].agg(['median', 'min', 'max'])
24
25
26 def verify():
27     baselines = []
28     for mode in BASE:
29         expected = 16 if mode == '4-2' else 4
30         assert read_json(RUN/f'completed_{mode}.json')['days'] == expected
31         for path, value in read_json(RUN/f'baseline_hashes_{mode}.json').items():
32             assert sha(path) == value
33         baselines.extend(read_json(RUN/f'baseline_metrics_{mode}.json'))
34     baseline_map = {(r['mode'], r['date']): r for r in baselines}
35     rows, quality, weights = [], [], []
36     for path in sorted((RUN/'cases').glob('*/*result.json')):
37         saved = read_json(path); row=saved['metrics']; folder=path.parent
38         for name, value in saved['artifact_hashes'].items(): assert sha(folder/name)==value
39         frame = pd.read_csv(folder/'dispatch.csv', float_precision='round_trip')
40         audit = read_json(folder/'audit.json'); experiment=audit['experiment']
41         values = dict(experiment['test_config']); values['nodes']=tuple(values['nodes'])
42         validate_frame(frame, row['mode'], Config(**values))
43         base = baseline_map[row['mode'], row['date']]
44         assert frame.initial_soc.iloc[0] == base['initial_soc']
45         source = pd.read_csv(BASE[row['mode']]/f"days/{row['date']}.csv", float_precision='round_trip')
46
47         for col in ['price', 'net_kwh', 'load_kw', 'pv_kw', 'price_forecast']:
48             np.testing.assert_array_equal(frame[col], source[col])
49             # 独立结算：减购退回原购电款，再支付50%违约，净变化为-0.5*p*减购量。
50             delta = frame.grid-frame.g0
51             expected_cost = frame.price*(frame.g0+1.5*np.maximum(delta, 0)-0.5*np.maximum(-delta, 0)+5*
52 frame.emergency)
53             np.testing.assert_allclose(frame.total_cost, expected_cost, rtol=0, atol=1e-6)
54             recalculated = metrics(frame, audit)
```

```

53     for key,value in recalculated.items():
54         assert np.isclose(value,row[key],atol=1e-8,rtol=1e-12),(key,path)
55     assert np.isclose(row['delta_cost_pct'],100*(row['total_cost']/base['total_cost']-1),atol=1e
-10)
56     if base['emergency_kwh']==0:assert row['delta_emergency_pct'] is None
57     nodes = audit.get('nodes',[dict(audit,hour=0)])
58     assert [n['hour'] for n in nodes] == ([0] if row['mode']=='4-2' else [0,6,12,18])
59     for i,n in enumerate(nodes):
60         if i:assert abs(n['initial_soc']-frame[frame.node_hour==nodes[i-1]['hour']].soc.iloc
[-1])<1e-6
61         s=n['solver'];sc=n['scenarios']
62         assert s['status']=='Optimal' and s['reliable'] and s['binaries']==0 and s['gap']<=0.03
63         assert sc['tail_singletons_verified']
64         assert max(sc['history_days'])*144+n['hour']*6+sc['length'] <= (pd.Timestamp(row['date'
])-pd.Timestamp('2025-01-01')).days*144+n['hour']*6
65         frozen=read_json(RUN/'frozen'/row['mode']/row['date']/f"{n['hour']:02d}.json")
66         assert sc['history_days']==frozen['history_days']
67         if row['parameter']=='rho':
68             assert sc['scenario_fingerprint']==frozen['scenario_fingerprint']
69             assert sc['weights']==frozen['weights']
70             assert np.isclose(sc['radius'],frozen['radius']*float(row['setting']),rtol=1e-14)
71         else:
72             assert sc['full_weight_pipeline_rebuilt']
73             assert sc['tail_indices_in_pool']==frozen['tail_indices_in_pool']
74             weights.append(dict(date=row['date'],net_weight=float(row['setting']),
75             representatives_changed=sc['representative_pool_indices']!=
frozen['representative_pool_indices'],
76             probabilities_changed=sc['weights']!=frozen['weights'],
77             radius_base=frozen['radius'],radius_variant=sc['radius']))
78             quality.append(dict(mode=row['mode'],date=row['date'],parameter=row['parameter'],setting
=row['setting'],hour=n['hour'],
79             status=s['status'],solver=s['solver'],version=s['solver_version'],
gap=s['gap'],
80             primal_error=s['linear_feasibility_error'],certificate_error=s['
lp_certificate_error'],
81             replay_duality_error=s['replay_duality_error'],radius=sc['radius']))
82         rows.append(row)
83     assert len(rows)==20 and len(quality)==32
84     assert len({(r['mode'],r['date'],r['parameter'],r['setting']) for r in rows})==20
85     # 每项参数的基准行只是从已有正式结果复制，未调用优化器。
86     expanded = rows.copy()
87     for row in baselines:
88         for parameter in (['rho','weight'] if row['mode']=='4-2' else ['rho']):
89             expanded.append(dict(row,parameter=parameter,setting='1.0' if parameter=='rho' else '0.5
'))
90     verification = dict(status='passed',new_day_cases=20,new_optimal_nodes=32,baseline_days_reused
=6,
91             baseline_nodes_matrix_verified=12,baseline_resolved=0,
92             baseline_source_files_unchanged=True,physical_and_settlement_checks=True,
93             strict_history_boundaries=True,rho_scenario_probability_frozen=True,
94             weights_full_pipeline_rebuilt=True,rolling_node_soc_chain=True,
95             optional_window_scenarios='not run under requested stopping rule',
96             max_primal_error=max(q['primal_error'] for q in quality),
97             max_certificate_error=max(q['certificate_error'] for q in quality))
98     write_json(DOC/'verification.json',verification)
99     write_csv(WEB/'daily_results.csv',pd.DataFrame(expanded))
100    write_csv(WEB/'solver_quality.csv',pd.DataFrame(quality))

```



```

101 write_csv(WEB/'weight_reconstruction.csv',pd.DataFrame(weights))
102 return pd.DataFrame(expanded),verification
103
104
105 def table_text(rows,mode):
106     fields=['delta_cost_pct','planned_cost','emergency_kwh','emergency_cost','min_soc','final_soc','
107             solve_seconds']
108     labels=['费用变化%', '计划购电费/元', '紧急量/kWh', '紧急费/元', '最低SOC/kWh', '末SOC/kWh', '求解秒']
109     if mode=='4-3':
110         fields=['delta_cost_pct','adjustment_cost','adjustment_abs_kwh','emergency_kwh','final_soc',
111                 'solve_seconds']
112         labels=['费用变化%', '净调整费/元', '调整总量/kWh', '紧急量/kWh', '末SOC/kWh', '求解秒']
113     result=['|参数|设置|'+'|'.join(labels)+'|', '|'+ '---|'*(len(fields)+2)]
114     for parameter,settings in [(('rho', ['0.75', '1.0', '1.25']), ('weight', ['0.7', '0.5', '0.3']))]:
115         if mode=='4-3' and parameter=='weight':continue
116         for setting in settings:
117             subset=rows[(rows['mode']==mode)&(rows.parameter==parameter)&(rows.setting==setting)]
118             cells=[]
119             for key in fields:
120                 v=subset[key];cells.append(f'{v.median():.3f} [{v.min():.3f}, {v.max():.3f}]')
121                 label=setting+'p*' if parameter=='rho' else f'{float(setting):.1f}/{1-float(setting):.1f}'
122             }'
123             result.append('|'+('p' if parameter=='rho' else 'wN/wp')+'|'+label+'|'+|'.join(cells)+'
124             |')
125     return '\n'.join(result)
126
127
128 def requirements():
129     source=DOC/'问题四敏感性分析实验方案_时间受限版.md'
130     entries=[];section=0
131     mapping={
132         0:('report::main','results.md','原方案标题完整保存'),
133         1:('run / freeze','results.md::实测讨论与论文表述','目标通过真实结果评估,不预设稳定性通过'),
134         2:('run::stages','daily_results.csv','rho三档/权重三档: W/S为可选且未执行'),
135         3:('configure / inputs','baseline_hashes; inputs/frozen_forecasts.json','原配置/预测/物理与结算
136         冻结'),
137         4:('run / freeze','cases/*/audit.json::experiment','仅一个配置字段变化: 基准直接复用'),
138         5:('prepare','representatives.json; selection_metrics_4-2.csv; selection_metrics_4-3.csv','
139         结果前固定规则, 6-12月、去重、4/2日'),
140         6:('freeze / run::selected_scenarios / metrics','solver_quality.csv; daily_results.csv','rho
141         乘0.75/1.25且场景概率指纹不变'),
142         7:('run::selected_scenarios','weight_reconstruction.csv; cases/4-2_weight*/audit.json','8项权
143         重全流程重建, 场景/概率均实际改变'),
144         8:('run::stages','results.md::验收','依第19节停止, 不执行可选W'),
145         9:('run::stages','results.md::验收','依第19节停止, 不执行可选S'),
146         10:('run::stages','completed_4-2.json; completed_4-3.json','Q42主体16变体, Q43四日变体16节点'),
147         11:('run / baseline','verification.json','新增20日32节点, 复用6日12节点'),
148         12:('run::stages','run_q42.log; run_q43.log','按rho、权重、Q43顺序实际执行'),
149         13:('run::stages','cases/*/audit.json::experiment','未混入消融, 只动dro_scale/net_weight'),
150         14:('metrics / q4_report::aggregate','summary.csv; daily_results.csv','逐日变化率后聚合median/
151         min/max; 零分母留空'),
152         15:('q4_report::table_text','results.md::Q4-2','Q42六组正式表, 含计划费用和全部指定指标'),
153         16:('q4_report::table_text','results.md::Q4-3','Q43三组正式表和调整费用/量'),
154         17:('q4_report::main','results.md::实测讨论与论文表述','费用稳定性局部支持: 调整量稳定不支持, 完整披露'
155         ),
156         18:('run::stages / q4_report::verify','verification.json; daily_results.csv','三项必做全完: 可
157         选W/S未做'),

```

```

147     19:('run::stages','completed_4-2.json; completed_4-3.json','三项必做完成即停止,未扩参'),
148     20:('q4_report::main','results.md::实测讨论与论文表述','不按测试成本挑参,未宣称全年/所有指标稳定'))}
149
150 for number,line in enumerate(source.read_text(encoding='utf-8').splitlines(),1):
151     text=line.strip()
152     if not text:continue
153     match=re.match(r'^(?:#+\s*)?(\\d+)\\. (?:实验目标|敏感参数|不进行|总体|代表日|Q4-S1|Q4-S2|Q4-S3|Q4-
154     S4|Q4-2 与|最小|推荐|敏感性分析与|统一|Q4-2 结果|Q4-3 结果|论文|最终执行|停止|最终原则)',text)
155     if match:
156         candidate=int(match.group(1))
157         if 1<=candidate<=20:section=candidate
158         optional=section in (8,9)
159         entries.append(dict(source_line=number,requirement=text,section=section,
160                             status='optional_not_run' if optional else 'verified_or_reported',
161                             implementation='src/sensitivity/q4_experiment.py; src/sensitivity/
162                             q4_report.py::'+mapping[section][0],
163                             evidence=mapping[section][1],acceptance=mapping[section][2]))
164     write_csv(DOC/'requirements_acceptance.csv',pd.DataFrame(entries))
165
166 def main():
167     WEB.mkdir(parents=True,exist_ok=True)
168     rows,checks=verify()
169     summary=aggregate(rows);summary.columns=['_'.join(c) for c in summary.columns]
170     write_csv(WEB/'summary.csv',summary.reset_index())
171     variants=rows[~rows.baseline_reused]
172     facts=[]
173     for mode,parameter in [ ('4-2','rho'),('4-2','weight'),('4-3','rho')]:
174         r=variants[(variants['mode']==mode)&(variants.parameter==parameter)]
175         facts.append(f'- Q{mode} {"半径" if parameter=="rho" else "权重"}: 真实总费用变化范围 {r.
176         delta_cost_pct.min():+.3f}%~{r.delta_cost_pct.max():+.3f}%; 紧急购电量绝对变化 {r.
177         delta_emergency_kwh.min():+.3f}%~{r.delta_emergency_kwh.max():+.3f}% kWh。')
178     reps=read_json(RUN/'representatives.json')
179     dates='\n'.join(f'- Q{mode}: ' + ' ':'.join(r['role']+' '+r['date'] for r in items)+'.' for mode,
180     items in reps['modes'].items())
181     text=f'''# 问题四代表日敏感性分析结果
182
183 假设与执行口径: 按用户时间受限方案第19节停止规则,仅完成三项必做实验。代表日局部扰动,每日初始SOC继承对应正式全年轨迹当
184 天的真实值,不重新串联全年SOC,不在测试集选最优参数。结果只支持已选工况下的局部稳定性,不能替代全年敏感性结论。
185
186 ## 实测结果
187
188 已完成20个新增日级实验、32个新增节点,全部为HiGHS 1.15.1认证Optimal。6个正式基准日直接复用,12个基准节点的完整优化
189 矩阵SHA逐一一致;没有重求解基准。
190
191 {chr(10).join(facts)}
192
193 ## 预先固定代表日
194
195 {dates}
196
197 所有候选为6—12月。常规日费用最接近全年正式334日费用中位数;高波动按实际电价日内总体标准差;联合风险按当前日价格与净负
198 荷正00S误差的标准化乘积均值,标准化尺度来自该日0点前可用56日历史(沿用模型MAD及退化回退);压力按基准紧急购电量降
199 序。并列日期升序,重复向下顺延。选日真实数据只用于事后工况选择,不传入优化输入。selection_metrics与
200 representatives.json在第一个扰动求解前保存。
201
202 ## Q4-2 (4日)

```

每格为中位数 [最小值, 最大值]; 费用百分比先逐日对同日基准计算, 再跨日汇总。SOC为电量kWh, 最低值含日初SOC。

```
{table_text(rows,'4-2')}
```

Q4-3 (2日、每日0/6/12/18四节点)

```
{table_text(rows,'4-3')}
```

实测讨论与论文表述

Q4-2半径从0.75p*增至1.25p*时, 四日计划购电费均单调增加、紧急购电量均单调下降, 说明常规购电与紧急风险之间存在预期权衡。相对p*, 强鲁棒紧急量下降2.03%~12.08%; 弱鲁棒增加3.29%~19.48%。但末SOC并不一致增加: 7月1日由1800降至1200 kWh, 12月14日由9676.159升至10050 kWh。因此不能把稳健化简单等同于更高的日末储能库存。

Q4-2八项权重实验均实际改变了代表场景和概率。费用变化在-1.489%~+1.194%, 紧急量变化在-26.107%~+10.457%。价格偏重的四日紧急量均下降; 净负荷偏重在7月6日和12月14日分别增加458.974与104.870 kWh。三种设置下SOC均满足物理边界, 支持经济结果对精确等权值不敏感; 风险指标仍有响应, 不能写成完全不变, 更不能据此选择测试成本最低权重。

Q4-3常规日费用变化仅-0.152%~+0.246%; 0.75p*下紧急量从7.889增至12.220 kWh, 百分比虽为+54.90%, 绝对只增加4.331 kWh。高联合风险日弱鲁棒费用+4.022%、紧急量+11.275% (+870.247 kWh); 强鲁棒费用-1.808%、紧急量-5.842% (-450.870 kWh)。强鲁棒下风险下降的方向可迁移到四节点调度, 但敏感幅度不能视作与Q4-2完全相同。

高联合风险日Q4-3调整绝对总量从基准700.210 kWh变为弱鲁棒363.517、强鲁棒1363.000 kWh; 净调整费用从213.798元变为70.270、412.925元。调整量存在明显参数响应, 因此“调整行为无明显变化”不获本实验支持。费用影响仍小于日总费用, 且四个节点全部最优、SOC合法, 没有发现求解失败或物理失控。

可直接用于论文的审慎表述: 在预先选定的代表日上, 对Wasserstein半径与联合距离权重进行单因素扰动后, Q4-2真实结算费用最大绝对变化为2.17%, 储能状态满足物理约束; 强鲁棒半径总体以增加计划购电费用换取较低紧急风险。Q4-3常规日费用变化不超过0.25%, 高联合风险日最大增幅为4.02%, 同时调整量呈现较明显响应。结果支持所选工况下经济结果的局部参数稳定性, 但不支持所有风险与调整指标均不敏感, 也不构成全年参数稳健性的证明。

审查等级: 未发现未解决的P0/P1实现问题; P2解释限制为仅4/2代表日且未事先给出“异常大幅变化”的数值阈值, Q4-3调整量不能宣布稳定; P3计时限制为基准复用历史耗时。W/S按停止规则未做, 不属于必做遗漏。所有审查结论直接对应实测, 未追加调参或扩大样本。

参数冻结与真实结算

Q4-2直接加载正式v2快照, Q4-3加载正式v3快照; 只读依赖、附件、预测数组和负荷模型选择均核验hash。半径为每个日/节点原标定p*的0.75/1.25倍, 联合历史池、20场景、经验概率、距离矩阵和预测完全固定; 每个节点仍接收本实验前一节点实际SOC。未重新训练Elastic Net, 也未增加统计bootstrap; 场景构造沿用正式模型100次半径标定bootstrap。

联合权重实验改变net_weight为0.7或0.3, 分别对应(wN,wp)=(0.7,0.3)/(0.3,0.7)。完整重新执行联合距离→强制尾部singleton+k-medoids→概率聚合→Wasserstein距离和原标定规则→DRO→真实回放。因距离尺度改变, p*按原规则随新距离重新标定, dro_scale固定1; 这属于同一权重流程, 不额外调参。详见weight_reconstruction.csv。

物理与结算沿用正式值: 容量上限10800/下限1200 kWh、功率5000 kW、充放效率0.9; 紧急费5p, 调整增购1.5p; 减购先退原购电款再付0.5p违约, 故相对初购费用的净调整项为-0.5p×减购量。调整量同时给增购、减购、绝对总量、净量; adjustment_cost为正式口径净调整费, 可为负。

solve_seconds为同口径节点求解与证书核验耗时之和; solve_wall_seconds包含独立求解进程启动。基准时间复用原审计, 受机器负载影响, 不能作为严格同机加速测试。半径实验复用已构造场景; scenario_seconds保留原构造时间用于追溯, 不是新增构造耗时。day_elapsed_seconds才是本次日级实际耗时。

紧急购电基准为0时百分比字段留空, 直接报告绝对变化; 跨日汇总不对空值造百分比。未执行W/S可选实验、全年重跑、消融或结果bootstrap。

```

229 ## 验收
230
231 所有真实10分钟回放通过成本重算、能量平衡、SOC上下界与链条、充放功率、四时点节点顺序及历史截止检查。最大LP原约束误差{
    checks['max_primal_error']:.3g}, 最大原对偶证书误差{checks['max_certificate_error']:.3g}。新增32节点
    均认证最优, 原正式结果文件hash未改变。完整数据见daily_results.csv (含10行复用参数基准、20行变体), 汇总见
    summary.csv, 节点证书见solver_quality.csv。
232
233 论文解释必须结合逐日风险和库存变化; 总费用小幅变化不能推出每项风险指标都不敏感, 也不能推出p越大越好。审查与论文建议表述
    见本文后续实测讨论。
234 '''
235 (DOC/'results.md').write_text(text,encoding='utf-8')
236 requirements()
237 # 浏览器验收页只展示同一实测表, 保持论文报告和UI数据一致。
238 table=rows[['mode','date','role','parameter','setting','delta_cost_pct','emergency_kwh','
    delta_emergency_kwh','min_soc','final_soc','adjustment_cost','solve_seconds','all_optimal']].
    rename(columns={'mode':'问题','date':'日期','role':'工况','parameter':'参数','setting':'设置','
    delta_cost_pct':'费用变化%','emergency_kwh':'紧急量kWh','delta_emergency_kwh':'紧急量变化kWh','
    min_soc':'最低SOC','final_soc':'末SOC','adjustment_cost':'净调整费/元','solve_seconds':'求解秒','
    all_optimal':'认证最优'}).to_html(index=False,float_format=lambda x:f'{x:,.3f}',border=0)
239 page=f'''<!doctype html><html lang="zh-CN"><meta charset="utf-8"><title>问题四敏感性实验结果</title>
    <style>body{{font:15px -apple-system,BlinkMacSystemFont,sans-serif;background:#f4f6fa;color
    :#183148;margin:36px}}main{{max-width:1500px;margin:auto}}h1{{font-size:29px}}.cards{{display:
    flex;gap:16px;margin:24px 0}}.card{{background:white;border-radius:12px;padding:22px;flex:1;
    border-top:4px solid #167c69}}b{{font-size:28px;display:block}}.scroll{{overflow:auto;background
    :white;border-radius:12px}}table{{border-collapse:collapse;font-size:12px;white-space:nowrap;
    width:100%}}td,th{{padding:10px;border-bottom:1px solid #e3e8ef;text-align:right}}th{{background
    :#e9eff5}}p,li{{line-height:1.7}}a{{color:#125b9b}}</style><main><h1>问题四 • 敏感性实验已完成</h1>
    <p>代表日局部OFAT • 正式基准复用 • 真实电价回放</p><div class="cards"><div class="card"><b>20 /
    20</b>新增日级实验完成</div><div class="card"><b>32 / 32</b>节点认证最优</div><div class="card"><b>6
    日</b>基准复用, 零重求解</div></div><ul>{''.join('<li>'+html.escape(f[2:])+</li>' for f in facts)
    }</ul><p>数据表: <a href="daily_results.csv">逐日结果</a> • <a href="summary.csv">中位数与范围</a>
    • <a href="solver_quality.csv">求解证书</a> • <a href="results.md">完整报告</a></p><div class="
    scroll"><table></div><p>Q4-3高风险日调整总量为基准700.21 → 弱鲁棒363.52 / 强鲁棒1363.00 kWh, 不能称为调
    整行为不敏感。仅支持代表日局部结论。W、S可选实验依停止规则未执行。</p></main></html>'''
240 (WEB/'index.html').write_text(page,encoding='utf-8')
241 (WEB/'results.md').write_text(text,encoding='utf-8')
242 print(json.dumps(checks,ensure_ascii=False,indent=2))
243 print(variants[['mode','date','parameter','setting','delta_cost_pct','delta_emergency_kwh','
    delta_emergency_pct','delta_adjustment_cost','final_soc']].to_string(index=False))
244
245
246 if __name__=='__main__':main()

```

src/sensitivity/report.py

```

1 """报告已有代表日局部变化, 不预设单调性或参数稳健结论。"""
2 import json
3 import numpy as np
4 import pandas as pd
5 from .common import ROOT,RUN,write_json,file_hash,comparison
6
7
8 def generate(records):
9     rows=[];seen=set()
10     for record in records:
11         row={k:v for k,v in record.items() if not isinstance(v,(dict,list))};rows.append(row)

```

```

12     if record.get('status') in ('complete','limited'):
13         key=(record['question'],record['parameter'],record['date'])
14         if key not in seen:
15             seen.add(key);base_value={'beta_R':1.,'rho_tail':.2,'S':20,'lambda_E':1.,'S_tail':
:2}[record['parameter']]
16             meta=json.loads((RUN/'inputs'/record['question']/record['date']/'metadata.json').
read_text())
17             solvers=[meta['baseline_solver']] if record['question']=='q2' else [n['
baseline_solver'] for n in meta['nodes']]
18             seconds=sum(a.get('solve_seconds',a.get('seconds',0)) for a in solvers)
19             certified=sum(bool(a.get('reliable')) for a in solvers)
20             rows.append({'id':'baseline_'+record['id'],'question':record['question'],'parameter'
:record['parameter'],'value':base_value,'date':record['date'],'role':record['role'],
21                         '**record['baseline'],**comparison(record['baseline'],record['baseline'
]),'seconds':seconds,'status':'baseline_reused',
22                         'reliable':certified==len(solvers),'nodes':len(solvers),'
certified_nodes':certified,'physical_verified':True})
23 detailed=pd.DataFrame(rows)
24 for index,row in detailed.iterrows():
25     path=(RUN/'inputs'/row.question/row.date/'baseline_dispatch.csv') if row.status=='
baseline_reused' else (RUN/'jobs'/row.id/'dispatch.csv')
26     if path.exists():
27         frame=pd.read_csv(path,float_precision='round_trip')
28         detailed.loc[index,'soc_upper_fraction']=float(np.mean(frame.soc>=10800-1e-5))
29         detailed.loc[index,'soc_lower_fraction']=float(np.mean(frame.soc<=1200+1e-5))
30 detailed.to_csv(RUN/'daily_results.csv',index=False,encoding='utf-8',float_format='%.17g')
31 valid=detailed[detailed.status.isin(['complete','limited','baseline_reused'])]
32 measures=['delta_cost_pct','delta_emergency_kwh','delta_emergency_pct','total_cost','
planned_cost','adjustment_cost','emergency_kwh','emergency_cost','min_soc','final_soc','seconds'
,'throughput_kwh','soc_upper_fraction','soc_lower_fraction']
33 aggregates=[]
34 for (question,parameter,value),group in valid.groupby(['question','parameter','value'],sort=
False):
35     row={'question':question,'parameter':parameter,'value':value,'days':len(group),'
certified_days':int(group.reliable.sum()),'contains_uncertified':not bool(group.reliable.all())}
36     for m in measures:
37         series=pd.to_numeric(group[m],errors='coerce').dropna()
38         row.update({m+'_median':float(series.median()) if len(series) else None,m+'_min':float(
series.min()) if len(series) else None,m+'_max':float(series.max()) if len(series) else None,m+'
_n':len(series)})
39     aggregates.append(row)
40 aggregate=pd.DataFrame(aggregates);aggregate.to_csv(RUN/'aggregate_results.csv',index=False,
encoding='utf-8',float_format='%.17g')
41 report=['# 问题二与问题三局部敏感性分析',' ', '4个代表日、单因素扰动；基准直接复用。本文仅描述选定工况下的局部变
化，不能外推为全年敏感性。',' ',
42         'Q3场景数经用户确认改为10/20/28，固定同一28条历史条件池。Q2正式场景数为26，安全裕度与尾部比例实验均固定
26。',' ',
43         '表格格式为中位数 【最小值，最大值】，只对4个代表日作描述统计，不作bootstrap或显著性检验。基准紧急量为0
时，百分比留空并报告绝对变化。',' ',
44         '重要精度口径：保持正式求解设置和3%目标；Q3沿用原35/8秒限时可行策略。未认证的基准或变体可能把求解误差混
入参数变化，因此不据此宣称最优解稳定或单调性。基准耗时为原正式运行记录，与当前机器的局部重求解时间不作速度比结论。'
, ' ']
45 def cell(row,key):
46     if pd.isna(row[key+'_median']):return '不适用'
47     return f"{row[key+'_median']:.3f} [{row[key+'_min']:.3f}, {row[key+'_max']:.3f}]"
48 for q in ['q2','q3']:
49     keys=['delta_cost_pct','planned_cost' if q=='q2' else 'adjustment_cost','emergency_kwh','

```

```

emergency_cost', 'min_soc', 'final_soc', 'seconds']
50     extra='计划费用/元' if q=='q2' else '调整费用/元'
51     report += ['## '+q.upper(),'',f'| 参数 | 参数值 | 成本变化/% | {extra} | 紧急量/kWh | 紧急费用/元
| 最低SOC/kWh | 末SOC/kWh | 耗时/s | 已认证日数 |','|---|---:|---|---|---|---|---|---|---:|']
52     for row in aggregates:
53         if row['question']==q:
54             value=f"{row['value']:.0%}" if row['parameter']=='rho_tail' else str(row['value'])
55             report.append('| '+row['parameter']+' | '+value+' | '+' | '.join(cell(row,k) for k
in keys)+f" | {row['certified_days']}/{row['days']} |")
56             variant=valid[(valid.question==q)&(valid.status!='baseline_reused')]
57             if len(variant):
58                 report+=[',',f"实际变体成本变化范围: {variant.delta_cost_pct.min():.3f}% 至 {variant.
delta_cost_pct.max():.3f}%; 紧急购电绝对变化范围: {variant.delta_emergency_kwh.min():.3f} 至 {
variant.delta_emergency_kwh.max():.3f} kWh。"]
59 # 定向响应只如实逐日计数，不把非单调结果改写为预期结论。
60 directions=[]
61 for q,param in [('q2','beta_R'),('q2','rho_tail'),('q3','lambda_E'),('q3','S')]:
62     for date,g in valid[(valid.question==q)&(valid.parameter==param)].groupby('date'):
63         g=g.sort_values('value')
64         directions.append({'question':q,'parameter':param,'date':date,'values':g.value.tolist(),
'costs':g.total_cost.tolist(),'emergency':g.emergency_kwh.tolist(),
65                             'emergency_nonincreasing':bool(np.all(np.diff(g.emergency_kwh)<=1e-6)
),'all_certified':bool(g.reliable.all())})
66 write_json(RUN/'direction_checks.json',directions)
67 report+=[',',## 对实验问题的实际回答','', '以下仅为相同日初状态、相同信息与原求解预算下的局部响应。3%间隙针对模
型内目标，并不是实际回放费用的±3%误差带，不能据此直接扣除或解释实际费用变化。']
68 for q,param in [('q2','beta_R'),('q2','rho_tail'),('q3','S'),('q3','lambda_E'),('q3','S_tail')]:
69     groups=valid[(valid.question==q)&(valid.parameter==param)]
70     changes=groups[groups.status!='baseline_reused']
71     if changes.empty:continue
72     counts={'increase':0,'decrease':0,'same':0};cost_deltas=[];risk_deltas=[]
73     for date,g in groups.groupby('date'):
74         g=g.sort_values('value')
75         if len(g)<3:continue
76         difference=float(g.emergency_kwh.iloc[-1]-g.emergency_kwh.iloc[0])
77         counts['increase' if difference>1e-5 else 'decrease' if difference<-1e-5 else 'same'
]+=1
78     maximum=float(abs(changes.delta_cost_pct).max())
79     report+=[',',f"- {q.upper()} {param}: 变体相对同日基准的最大绝对成本变化为{maximum:.3f}%: 从最低参数档
升至最高档，紧急量下降{counts['decrease']}日、上升{counts['increase']}日、不变{counts['same']}日。风险变
化并不预先假定单调。"]
80     if not bool(groups.reliable.all()):report.append(' 该组含未认证的基准或变体；所列变化混有求解预算影
响，不能判定为精确最优策略的参数效应。')
81 report+=[',',f"所有已获得可行轨迹的SOC位于[{valid.min_soc.min():.6f}, 10800]kWh的数值可行范围；未出现容量
/功率越界或能量平衡失败。上界停留比例与吞吐量详见daily_results.csv。", '',
82         '主要经济结论的边界：仅凭这些扰动不能证明动态SOC裕度或尾部机制相对无机制策略的全年净收益，也不能宣称某个
参数理论最优。若不同代表日响应方向相反，应报告工况依赖；未设置新的主观百分比阈值来把结果自动归类为“稳定”。']
83 quality=[]
84 for record in records:
85     if record['status'] not in ('complete','limited'):continue
86     audit=json.loads((RUN/'jobs'/record['id']/'audit.json').read_text())
87     nodes=[{'hour':0,'solver':audit['solver']}] if record['question']=='q2' else audit['nodes']
88     for node in nodes:
89         solver=node['solver'];upper=solver.get('objective');lower=solver.get('lower_bound',
solver.get('dual_bound'))
90         quality.append({'id':record['id'],'question':record['question'],'date':record['date'],'
parameter':record['parameter'],'value':record['value'],

```



```

91         'hour':node['hour'],'reliable':solver.get('reliable'), 'relative_gap':
solver.get('mip_gap',solver.get('gap')),
92         'upper_bound':upper,'lower_bound':lower,'absolute_model_gap_yuan':max(0,
upper-lower) if upper is not None and lower is not None else None,
93         'status':solver.get('status'),'accepted_by':solver.get('accepted_by'),'
solver_seconds':solver.get('solve_seconds',solver.get('seconds'))))
94 pd.DataFrame(quality).to_csv(RUN/'solver_quality.csv',index=False,encoding='utf-8',float_format=
'%.17g')
95
96 report+=[',## 完整数据与边界',',', 'daily_results.csv保留每个代表日及其基准; aggregate_results.csv还包
含计划费用、调整费用和吞吐量的完整中位数/最小/最大。每个任务保存输入、策略、实际回放和求解审计。',',',
97         f"完成扰动任务 {sum(r['status']=='complete' for r in records)}, 限时可行 {sum(r['status']
)=='limited' for r in records)}, 失败 {sum(r['status']=='failed' for r in records)}。未认证结果只
作实际预算下的策略诊断，不作全局最优敏感性证明。"]
98 if any(r['status']=='failed' for r in records):report+=[',', '失败项: ']+[f"- {r['id']}: {r.get('
error')}" for r in records if r['status']=='failed']
99 interpretation=ROOT/'docs/sensitivity_q2_q3/interpretation.md'
100 if interpretation.exists():report+=[',',interpretation.read_text(encoding='utf-8')]
101 (RUN/'report.md').write_text('\n'.join(report)+'\n',encoding='utf-8')
102 (ROOT/'docs/sensitivity_q2_q3/results.md').write_text('\n'.join(report)+'\n',encoding='utf-8')
103 receipts=[]
104 for record in records:
105     folder=RUN/'jobs'/record['id']
106     if record['status'] in ('complete','limited'):
107         assert file_hash(folder/'dispatch.csv')==record['dispatch_sha256'];assert file_hash(
folder/'audit.json')==record['audit_sha256']
108         assert record['physical_verified'];receipts.append(record['id'])
109 write_json(RUN/'acceptance.json',{ 'jobs':len(records), 'verified_dispatch_audit_pairs':len(
receipts), 'baseline_reused_no_optimization':True,
110         'no_full_year_rerun':True, 'no_bootstrap':True, '
all_core_jobs_have_outcome':len(records)>=32, 'failed':sum(r['status']=='failed' for r in records
),
111         'local_inference_only':True, 'q3_S28_user_approved':True})

```

src/sensitivity/run.py

```

1  """两问独立进程执行局部OFAT; 结果收据可恢复，基准从不重算。"""
2  import argparse,json,os,subprocess,sys,time,traceback,hashlib
3  from pathlib import Path
4  from .common import ROOT,RUN,write_json,file_hash,active,comparison
5
6
7  def design():
8      reps=json.loads((RUN/'representatives.json').read_text())
9      jobs=[]
10     for question,parameters in [ ('q2',[('beta_R',(0.8,1.2)),('rho_tail',(0.1,.3))],('q3',[('S'
,(10,28)),('lambda_E',(0.8,1.2))]))]:
11         for parameter,values in parameters:
12             for row in reps[question]:
13                 for value in values:
14                     jobs.append({'id':f'{question}_{parameter}_{value}_{row["date"]}','question':
question,'parameter':parameter,'value':value,'date':row['date'],'role':row['role'],'status':
'pending'})
15     return jobs
16
17

```

```

18 def signature(job):
19     payload={k:job[k] for k in ('id','question','parameter','value','date')}
20     inputs=RUN/'inputs'/job['question']/job['date']
21     payload['inputs']={p.name:file_hash(p) for p in sorted(inputs.glob('*')) if p.is_file()}
22     paths=[ROOT/'src/sensitivity'/name for name in ('common.py','q2.py','q3.py','run.py')]+[ROOT/'
src'/job['question']/name for name in
23         ([ 'rolling.py','scenarios.py','policy.py','optimization.py','relaxation.py','incumbent.
py','linear_policy.py','protocol.py'] if job['question']=='q2' else
24         [ 'rolling.py','scenarios.py','physics.py','optimization.py','incumbent.py','fast_solver
.py','config.py','checkpoint.py'])]
25     import importlib.metadata
26     payload['runtime_versions']={name:importlib.metadata.version(name) for name in ('numpy','scipy',
'pandas','pyscipopt','highspy')}
27     payload['sources']={str(p.relative_to(ROOT)):file_hash(p) for p in paths if not p.name.
startswith('test_')}
28     return hashlib.sha256(json.dumps(payload,sort_keys=True).encode()).hexdigest()
29
30
31 def worker(question,limit=None,optional=False):
32     from . import q2,q3
33     module=q2 if question=='q2' else q3
34     jobs=design() if not optional else optional_jobs()
35     jobs=[j for j in jobs if j['question']==question]
36     data=q3.inputs() if question=='q3' else None
37     performed=0
38     for job in jobs:
39         folder=RUN/'jobs'/job['id'];folder.mkdir(parents=True,exist_ok=True);receipt=folder/'result.
json';sig=signature(job)
40         if receipt.exists():
41             prior=json.loads(receipt.read_text())
42             if prior['signature']!=sig:raise ValueError('已有实验源码或冻结输入变化，禁止混用结果')
43             continue
44         if limit is not None and performed>=limit:break
45         job['started_at']=time.time();write_json(folder/'request.json',{**job,'signature':sig})
46         try:
47             result,baseline=module.run(job,folder,**({'data':data} if question=='q3' else {}))
48             status='complete' if result['reliable'] else 'limited'
49             record={**job,**result,**comparison(result,baseline),'baseline':baseline,'status':status
,
50                 'status_label':'完成 / 已认证' if status=='complete' else '限时可行 / 未认证','
signature':sig,
51                 'dispatch_sha256':file_hash(folder/'dispatch.csv'),'audit_sha256':file_hash(
folder/'audit.json')}
52         except Exception as error:
53             record={**job,'status':'failed','status_label':'失败，已保留诊断','signature':sig,'seconds'
:time.time()-job['started_at'],
54                 'error':str(error),'traceback':traceback.format_exc(),'details':getattr(error,
'audit',getattr(error,'details',None))}
55             write_json(receipt,record);print(f"{job['id']} {record['status']} {record.get('seconds',0)
:.2f}s",flush=True);performed+=1
56         active(question,jobs[-1],'本轮队列完成',hour=None,detail='已完成记录均已原子保存，基准结果未修改')
57
58
59 def optional_jobs():
60     reps=json.loads((RUN/'representatives.json').read_text())['q3']
61     return [{ 'id':f'q3_S_tail_{value}_{row["date"]}', 'question':'q3','parameter':'S_tail','value':
value,'date':row['date'],'role':row['role'],'status':'pending','optional':True} for row in reps

```



```

    for value in (0,4)]
62
63
64 def snapshot(jobs,phase,started,notes):
65     records=[]
66     for job in jobs:
67         folder=RUN/'jobs'/job['id'];receipt=folder/'result.json';request=folder/'request.json'
68         if receipt.exists():records.append(json.loads(receipt.read_text()))
69         elif request.exists():records.append(**job,'status':'running','status_label':'计算中')
70         else:records.append(job)
71     write_json(RUN/'status.json',{'phase':phase,'total_jobs':len(jobs),'jobs':records,'notes':notes,
72         'started_at':started})
73     return records
74
75 def launch(question,optional=False):
76     args=[sys.executable,'-u','-m','src.sensitivity.run','--worker',question]+(['--optional'] if
77         optional else [])
78     stream=(RUN/f'{question}{"_optional" if optional else ""}_worker.log').open('ab')
79     process=subprocess.Popen(args,cwd=ROOT,stdout=stream,stderr=subprocess.STDOUT,env=dict(os.
80         environ,OMP_NUM_THREADS='1',OPENBLAS_NUM_THREADS='1',VECLIB_MAXIMUM_THREADS='1',MKL_NUM_THREADS=
81         '1',PYTHONUTF8='1'))
82     stream.close();return process
83
84 def supervise():
85     started=time.time();jobs=design();notes=['Q2固定正式K=3、S=26; Q3采用已确认的10/20/28, 同一历史条件池。
86         ', '基准直接复用; Q3沿用3%优先及35/8秒限时策略, 未认证解会明确标记。']
87     processes=[launch('q2'),launch('q3')]
88     write_json(RUN/'launch.json',{'supervisor_pid':os.getpid(),'workers':[p.pid for p in processes],
89         'started_at':started,'python':sys.executable})
90     while any(p.poll() is None for p in processes):snapshot(jobs,'核心32项实验计算中',started,notes);
91         time.sleep(2)
92     records=snapshot(jobs,'核心实验计算结束, 正在核验',started,notes)
93     missing=[r for r in records if r['status'] in ('pending','running')]
94     if missing:raise RuntimeError(f'工作进程提前退出, {len(missing)}项无结果, 检查worker.log')
95     core_seconds=time.time()-started
96     # 在计算前声明可选门槛: 核心<=1h、无失败且18:00前仍有20min以上。
97     from datetime import datetime
98     now=datetime.now();remaining=(now.replace(hour=18,minute=0,second=0,microsecond=0)-now).
99     total_seconds()
100     optional=core_seconds<=3600 and remaining>=1200 and all(r['status']!='failed' for r in records)
101     write_json(RUN/'optional_decision.json',{'q2_S3':'not_applicable_formal_S26_not_S20','q3_S3_run':
102         optional,'core_seconds':core_seconds,'seconds_to_18':remaining,'rule':'core<=3600s, no failed
103         jobs, remaining>=1200s'})
104     if optional:
105         jobs+=optional_jobs();notes.append('核心实验及时完成, 按方案执行可选Q3尾部名额0/2/4; Q2可选场景数因正
106             式S非20不适用。')
107         process=launch('q3',True)
108         while process.poll() is None:snapshot(jobs,'核心已完成, Q3可选尾部名额实验计算中',started,notes);
109             time.sleep(2)
110     records=snapshot(jobs,'计算结束, 生成汇总与验收报告',started,notes)
111     from .report import generate
112     generate(records)
113     snapshot(jobs,'全部实验与汇总已完成' if all(r['status']!='failed' for r in records) else '实验结束,
114         存在失败项, 详见报告',started,notes)

```

```

105
106 if __name__ == '__main__':
107     parser=argparse.ArgumentParser();parser.add_argument('--worker',choices=['q2','q3']);parser.
        add_argument('--limit',type=int);parser.add_argument('--optional',action='store_true');args=
        parser.parse_args()
108     if args.worker:worker(args.worker,args.limit,args.optional)
109     else:supervise()

```

src/sensitivity/status_server.py

```

1 """敏感性实验只读进度服务，右侧页面从真实任务状态读取进度。"""
2 from http.server import ThreadingHTTPServer,SimpleHTTPRequestHandler
3 from pathlib import Path
4 import json,time
5 ROOT=Path(__file__).resolve().parents[2]
6 WEB=ROOT/'outputs/processed/sensitivity_q2_q3'
7 RUN=ROOT/'outputs/sensitivity_q2_q3/raw/local_20260913'
8 class Handler(SimpleHTTPRequestHandler):
9     def __init__(self,*args,**kwargs):super().__init__(*args,directory=str(WEB),**kwargs)
10    def log_message(self,*args):pass
11    def do_GET(self):
12        if self.path.split('?')[0]=='/api/status':
13            state=json.loads((RUN/'status.json').read_text(encoding='utf-8'))
14            state['observed_at']=time.time()
15            reps=RUN/'representatives.json'
16            if reps.exists():state['representatives']=json.loads(reps.read_text(encoding='utf-8'))
17            state['workers']={}
18            for name in ('q2','q3'):
19                p=RUN/f'{name}_active.json'
20                if p.exists():state['workers'][name]=json.loads(p.read_text(encoding='utf-8'))
21                data=json.dumps(state,ensure_ascii=False,allow_nan=False).encode('utf-8')
22                self.send_response(200);self.send_header('Content-Type','application/json; charset=utf-8')
                self.send_header('Cache-Control','no-store');self.send_header('Content-Length',str(len(data)))
                self.end_headers();self.wfile.write(data)
23            else:super().do_GET()
24 if __name__ == '__main__':ThreadingHTTPServer(('127.0.0.1',8766),Handler).serve_forever()

```

src/sensitivity/test_sensitivity.py

```

1 import unittest,json
2 import numpy as np
3 from dataclasses import replace
4 from .common import RUN,comparison,digest
5 from .q3 import base_config,scenarios_for
6 from .run import design
7
8 class Tests(unittest.TestCase):
9     def test_core_design_and_no_baseline_resolve(self):
10        jobs=design();self.assertEqual(len(jobs),32);self.assertEqual(len({j['id'] for j in jobs}),
        ,32)
11        self.assertEqual({j['value'] for j in jobs if j['parameter']=='S'},{10,28})
12        self.assertFalse(any(j['value']==1 for j in jobs if j['parameter'] in ('beta_R','lambda_E'))
        )
13    def test_zero_emergency_baseline_is_not_percent(self):
14        r=comparison({'total_cost':105,'emergency_kwh':2},{'total_cost':100,'emergency_kwh':0})

```

```

15     self.assertEqual(r['delta_cost_pct'],5);self.assertIsNone(r['delta_emergency_pct']);self.
    assertEquals(r['delta_emergency_kwh'],2)
16 def test_representatives_are_four_unique_and_frozen(self):
17     r=json.loads((RUN/'representatives.json').read_text())
18     for q in ('q2','q3'):self.assertEqual(len({x['date'] for x in r[q]}),4)
19     self.assertTrue(r['rules_frozen_before_experiments'])
20 def test_q2_scaling_and_tail_fixed_reserve(self):
21     from .q2 import construct
22     for folder in (RUN/'inputs/q2').iterdir():
23         a=np.load(folder/'frozen.npz');m=json.loads((folder/'metadata.json').read_text())
24         for b in [.8,1.2]:
25             r=a['reserve']*b;np.testing.assert_array_equal(r[144:],a['reserve'][144:]);self.
    assertTrue((r<9600).all())
26         for frac in [.1,.3]:
27             net,w,meta=construct(a['point'],a['blocks'],a['prices'],m['S'],origins=a['origins'],
    all_blocks=a['all_blocks'],all_origins=a['all_origins'],tail_fraction=frac)
28             self.assertEqual(len(net),26);self.assertEqual(meta['S_body']+meta['S_tail'],26);
    self.assertAlmostEqual(w.sum(),1)
29 def test_q3_scenario_size_does_not_change_pool(self):
30     config=base_config()
31     for folder in (RUN/'inputs/q3').iterdir():
32         for h in (0,6,12,18):
33             a=np.load(folder/f'{h:02d}.npz');before=digest(a['history_days'],a['load_errors'],a[
    'pv_errors'])
34             for n in (10,20,28):
35                 scenes=scenarios_for(a,replace(config,scenarios=n));self.assertEqual(len(scenes.
    weights),n)
36                 self.assertTrue(set(scenes.audit['history_days']).issubset(set(a['history_days'
    ])))
37                 self.assertAlmostEqual(scenes.weights.sum(),1)
38                 self.assertEqual(before,digest(a['history_days'],a['load_errors'],a['pv_errors']))
39 def test_terminal_only_perturbation_leaves_scenarios_unchanged(self):
40     config=base_config()
41     folder=next((RUN/'inputs/q3').iterdir());a=np.load(folder/'06.npz')
42     original=scenarios_for(a,config)
43     for scale in [.8,1.2]:
44         scenes=scenarios_for(a,replace(config,terminal_scale=scale))
45         for key in ['load','pv','weights']:np.testing.assert_array_equal(getattr(original,key),
    getattr(scenes,key))
46
47 if __name__=='__main__':unittest.main()

```

src/sensitivity/verify.py

```

1 """独立核查40个局部实验：原件、OFAT输入、实际物理/费用、基准不变与聚合。"""
2 import json
3 import numpy as np
4 import pandas as pd
5 from .common import ROOT,RUN,file_hash,digest,write_json
6 from .run import design,optional_jobs,signature
7 from . import q2,q3
8
9
10 def main():
11     expected=design()+(optional_jobs() if json.loads((RUN/'optional_decision.json').read_text())['
    q3_S3_run'] else [])

```

```

12 baseline=json.loads((RUN/'baseline_files_manifest.json').read_text())
13 assert all(file_hash(ROOT/path)==value for path,value in baseline.items())
14 assert file_hash(ROOT/'docs/sensitivity_q2_q3/问题2与问题3敏感性分析实验方案_时间受限版.md')=='866563
d9481f002ba4feba69b1b7e113027d92f3f8d94b4df9ccac3838a3696f'
15 rows=[];balance_errors=[];state_errors=[]
16 for job in expected:
17     folder=RUN/'jobs'/job['id'];r=json.loads((folder/'result.json').read_text())
18     assert r['status'] in ('complete','limited') and r['physical_verified']
19     assert r['signature']==signature(job)
20     assert file_hash(folder/'dispatch.csv')==r['dispatch_sha256'] and file_hash(folder/'audit.
json')==r['audit_sha256']
21     f=pd.read_csv(folder/'dispatch.csv',float_precision='round_trip');meta=json.loads((RUN/'
inputs'/job['question']/job['date']/'metadata.json').read_text())
22     assert len(f)==144 and set(f.date)=={job['date']}
23     assert abs(float(f.initial_soc.iloc[0])-meta['initial_soc'])<1e-7
24     soc=meta['initial_soc']+np.cumsum(.9*f.charge-f.discharge/.9)
25     state_error=float(abs(f.soc-soc).max());assert state_error<1e-5;state_errors.append(
state_error)
26     assert f.soc.min()>=1200-1e-5 and f.soc.max()<=10800+1e-5
27     assert max(f.charge.max(),f.discharge.max())<=5000/6+1e-5
28     assert min(f.charge.min(),f.discharge.min(),f.emergency.min())>=-1e-5
29     if job['question']=='q2':
30         a=np.load(RUN/'inputs/q2'/job['date']/'frozen.npz');v=np.load(folder/'variant_inputs.npz
')
31         if job['parameter']=='beta_R':
32             np.testing.assert_array_equal(v['reserve'],a['reserve']*job['value']);np.testing.
assert_array_equal(v['net'],a['net']);np.testing.assert_array_equal(v['weights'],a['weights'])
33         else:
34             np.testing.assert_array_equal(v['reserve'],a['reserve'])
35             assert len(v['weights'])==meta['S']==26
36             policy_frame=pd.read_csv(folder/'policy.csv',float_precision='round_trip')
37             policy=q2.Policy(policy_frame.grid,policy_frame.charge_cap,policy_frame.discharge_cap,
policy_frame.reserve)
38             response=q2.replay(policy.first(),a['real_net'],meta['initial_soc'])
39             for key,values in response.items():np.testing.assert_allclose(f[key],values,atol=1e-7,
rtol=0)
40             np.testing.assert_allclose(f.net_kwh,a['real_net'],atol=1e-9,rtol=0)
41             cost=float(np.sum(a['prices'][:144]*(f.grid+5*f.emergency)))
42             error=float(abs(f.grid+f.discharge+f.emergency-f.net_kwh-f.charge-f.unused).max())
43         else:
44             a=json.loads((folder/'audit.json').read_text());assert len(a['nodes'])==4
45             np.testing.assert_array_equal(f.node_hour,np.repeat([0,6,12,18],36))
46             for n in a['nodes']:
47                 h=n['hour'];frozen=np.load(RUN/'inputs/q3'/job['date']/f'{h:02d}.npz');b=meta['nodes
'][:h//6]
48                 assert n['history_sha256']==b['history_sha256']==digest(frozen['history_days'],
frozen['load_errors'],frozen['pv_errors'])
49                 expected_S=int(job['value']) if job['parameter']=='S' else 20
50                 assert n['scenarios']['S']==expected_S and n['scenarios']['pool_size']==28
51                 assert len(set(n['scenarios']['history_days']))==expected_S
52                 assert set(n['scenarios']['history_days']).issubset(set(frozen['history_days']))
53                 scale=job['value'] if job['parameter']=='lambda_E' else 1
54                 assert abs(n['terminal']-float(frozen['terminal'])*scale)<1e-12
55                 initial=meta['initial_soc'] if h==0 else float(f.soc.iloc[h*6-1]);assert abs(n['
initial_soc']-initial)<1e-7
56                 block=f.iloc[h*6:(h+6)*6]
57                 for key in ['grid','charge_cap','discharge_cap']:np.testing.assert_allclose(block[

```

```

key],n[key][:36],atol=1e-7,rtol=0)
58     delta=f.grid-f.g0
59     adjustment=f.price*(1.5*np.maximum(delta,0)-.5*np.maximum(-delta,0))
60     np.testing.assert_allclose(f.adjustment_cost,adjustment,atol=1e-7,rtol=0)
61     cost=float(np.sum(f.price*f.g0+adjustment+5*f.price*f.emergency))
62     error=float(abs(f.grid+f.discharge+f.emergency-f.net_kwh-f.charge-f.spill).max())
63     assert error<1e-5;balance_errors.append(error)
64     assert abs(cost-r['total_cost'])<1e-6
65     b=meta['baseline'];assert abs(r['delta_cost_pct']-100*(cost-b['total_cost'])/b['total_cost']
    ])<1e-8
66     assert abs(r['delta_emergency_kwh']-(f.emergency.sum()-b['emergency_kwh']))<1e-7
67     if b['emergency_kwh']==0:assert r['delta_emergency_pct'] is None
68     rows.append(r)
69 detail=pd.read_csv(RUN/'daily_results.csv');aggregate=pd.read_csv(RUN/'aggregate_results.csv')
70 assert len(detail)==len(expected)+20
71 assert len(aggregate)==15
72 for _,row in aggregate.iterrows():
73     g=detail[(detail.question==row.question)&(detail.parameter==row.parameter)&(detail.value==
    row.value)]
74     assert len(g)==4 and row.days==4
75     for key in ['total_cost','delta_cost_pct','emergency_kwh','emergency_cost','adjustment_cost'
    , 'min_soc','final_soc','seconds']:
76         for suffix,fn in [('median',np.median),('min',np.min),('max',np.max)]:
77             np.testing.assert_allclose(row[key+'_'+suffix],fn(g[key]),atol=1e-7,rtol=0)
78 result={'jobs_verified':len(rows),'core_jobs':32,'optional_jobs':len(rows)-32,'
    original_plan_bytes_verified':True,'q3_S28_user_approved':True,
79     'baseline_files_unchanged':len(baseline),'all_same_initial_soc':True,'
    OFAT_frozen_inputs_verified':True,
80     'max_balance_error_kwh':max(balance_errors),'max_soc_transition_error_kwh':max(
    state_errors),
81     'all_delta_and_aggregate_statistics_verified':True,'baseline_not_resolved':True,
82     'certified_jobs':sum(r['status']=='complete' for r in rows),'limited_feasible_jobs':sum(
    r['status']=='limited' for r in rows),'failed_jobs':0}
83 write_json(RUN/'independent_acceptance.json',result);print(json.dumps(result,ensure_ascii=False,
    indent=2))
84
85
86 if __name__=='__main__':main()

```

B.6 绘图与数据整理程序

src/plots/catalog.py

```

1  """全项目绘图清单：脚本、数据、成图路径和论文必要性集中登记。"""
2  import csv
3  import html
4  import os
5  from pathlib import Path
6
7  ROOT = Path(__file__).resolve().parents[2]
8
9
10 def main():
11     rows = []
12     new = ROOT / 'outputs/processed/figures/q1'
13     with (new / 'figure_index.csv').open(encoding='utf-8', newline='') as stream:

```

```

14     for row in csv.DictReader(stream):
15         rows.append({'question': 'q1', 'name': row['figure'], 'title': row['title'], 'necessity': row['
role'],
16                     'script': row['script'], 'data': ';'.join(str((new/p).relative_to(ROOT)) for p
in row['data_csv'].split(';')),
17                     'png': str((new/(row['figure']+'.png')).relative_to(ROOT)),
18                     'svg': str((new/(row['figure']+'.svg')).relative_to(ROOT))})
19     for name, title in [('dispatch', '原报告输入与运行组合图'), ('efficiency', '原报告效率图'),
20                        ('marginal_analysis', '原报告边际分析组合图'), ('bottleneck_analysis', '原报告局部边界
价值图')]:
21         rows.append({'question': 'q1', 'name': name, 'title': title, 'necessity': '历史报告, 正文优先使用新图',
22                     'script': 'src/plots/q1_legacy.py', 'data': 'outputs/q1_current/raw;outputs/
q1_current/processed',
23                     'png': f'outputs/processed/q1/{name}.png', 'svg': f'outputs/processed/q1/{name}.
svg'})
24     import json
25     q2 = json.loads((ROOT/'outputs/q2_current/raw/figure_manifest.json').read_text(encoding='utf-8')
)
26     titles={'timeseries_7d': '连续7天原始功率', 'timeseries_30d': '连续30天原始功率', 'annual_heatmaps': '全年
日期—时间热力图', 'monthly_profiles': '12个月平均日曲线', 'weekday_profiles': '星期平均日曲线',
weekday_weekend': '工作日与周末平均日曲线', 'acf_pacf': '自相关与Burg偏自相关', 'spectra': '日周周期频谱验证',
'monthly_stability': '月度周期稳定性', 'seasonal_moments': '季节处理后的月度均值与波动', 'daily_peaks': '每
日峰值与峰值时刻'}
27     for row in q2:
28         name=row['name']
29         folder=ROOT/'outputs/processed/figures/q2'
30         if not (folder/(name+'.png')).exists(): folder=ROOT/'outputs/processed/q2/figures'
31         rows.append({'question': 'q2', 'name': name, 'title': titles[name], 'necessity': '第二问数据诊断, 按其
章节选用',
32                     'script': 'src/plots/q2.py', 'data': ';'.join('outputs/processed/q2/'+p for p in
row['source_csv'])),
33                     'png': str((folder/(name+'.png')).relative_to(ROOT)), 'svg': str((folder/(name+'.
svg')).relative_to(ROOT))})
34     for row in rows:
35         for field in ['script', 'data', 'png', 'svg']:
36             for path in row[field].split(';'):
37                 if not (ROOT/path).exists(): raise ValueError(f'绘图登记路径缺失: {path}')
38     folder=ROOT/'src/plots'
39     with (folder/'figure_registry.csv').open('w', encoding='utf-8', newline='') as stream:
40         writer=csv.DictWriter(stream, fieldnames=list(rows[0])); writer.writeheader(); writer.
writerows(rows)
41     output=ROOT/'outputs/processed/figures'; output.mkdir(parents=True, exist_ok=True)
42     content='<!doctype html><html lang="zh-CN"><meta charset="UTF-8"><meta name="viewport" content="
width=device-width, initial-scale=1"><title>项目图集</title>'
43     content+='\n<style>body{font:16px/1.65 -apple-system,BlinkMacSystemFont,"PingFang SC",sans-serif;
color:#253d56;background:#f3f6f8;margin:0}main{max-width:1200px;margin:auto;padding:30px 20px}.
cards{display:grid;grid-template-columns:repeat(auto-fit,minmax(290px,1fr));gap:22px}article{
background:white;padding:18px;border-radius:8px}img{width:100%;height:230px;object-fit:contain}a
{color:#0072b2}h2{font-size:18px}p{font-size:14px}</style><main><h1>项目图集与绘图脚本</h1><p><a
href="q1/index.html">第一问论文图集与必要性说明</a> • <a href="../../src/plots/README.md">统一绘图
说明</a> • <a href="../../src/plots/figure_registry.csv">全图清单 CSV</a></p><div class="cards
">'
44     for row in rows:
45         link=lambda path:html.escape(os.path.relpath(ROOT/path,output))
46         content+=f'\n<article><p>{html.escape(row["question"].upper())} • {row["necessity"]}</p><h2
>{html.escape(row["title"])}</h2><a href="{link(row["png"])}"></a><p><a href="{link(row["svg"])}">SVG</a> • <a href="{link(row

```

```

13 ["script"])]">绘图脚本</a></p></article>
47 (output/'index.html').write_text(content+'</div></main></html>',encoding='utf-8')
48 print(f'已登记{len(rows)}组图: src/plots/figure_registry.csv')
49
50
51 def register_manifest(path):
52     """独立登记新图清单；不重绘或重验证其他历史图集。"""
53     import json
54     path=Path(path).resolve();items=json.loads(path.read_text(encoding='utf-8'))
55     registry=R00T/'src/plots/figure_registry.csv'
56     with registry.open(encoding='utf-8',newline='') as stream:
57         reader=csv.DictReader(stream);fields=reader.fieldnames;rows=list(reader)
58         added=[]
59         for item in items:
60             data=[path.parent/'data'/name for name in item['data']]
61             if not data:data=[R00T/'docs/plots/q2_paper_final/问题二_必须与强推荐图_最终版.md']
62             png=path.parent/item['png'];svg=path.parent/item['svg']
63             for p in data+[png,svg,R00T/item['script']]:
64                 if not p.is_file():raise ValueError(f'新图登记路径不存在: {p}')
65                 added.append({'question':item.get('question','q2'),'name':item['name'],'title':item['title'],
66                               'necessity':item.get('necessity','用户指定正文图或附加热图'),
67                               'script':item['script'],'data':';'.join(str(p.relative_to(R00T)) for p in data
68                               ),
69                               'png':str(png.relative_to(R00T)),'svg':str(svg.relative_to(R00T))})
70             keys={r['question'],r['name']} for r in added}
71             rows=[r for r in rows if (r['question'],r['name']) not in keys]+added
72             with registry.open('w',encoding='utf-8',newline='') as stream:
73                 writer=csv.DictWriter(stream,fieldnames=fields);writer.writeheader();writer.writerows(rows)
74             print(f'已独立登记{len(added)}张新图, 保留其他登记')
75
76 if __name__=='__main__':
77     import sys
78     if len(sys.argv)==3 and sys.argv[1]=='--register-manifest':register_manifest(sys.argv[2])
79     else:main()

```

src/plots/common.py

```

1  """论文图的中文字体、语义配色及PNG/SVG导出。"""
2  import csv
3  import hashlib
4  import warnings
5
6  COLORS = {'load': '#41464D', 'pv': '#E69F00', 'grid': '#0072B2',
7            'charge': '#009E88', 'discharge': '#8B5FBF', 'muted': '#7B8792', 'energy': '#253D56'}
8
9
10 def read_csv(path):
11     with path.open(encoding='utf-8', newline='') as stream:
12         return list(csv.DictReader(stream))
13
14
15 def digest(path):
16     return hashlib.sha256(path.read_bytes()).hexdigest()
17
18

```



```

19 def setup():
20     import matplotlib
21     matplotlib.use('Agg')
22     import matplotlib.pyplot as plt
23     from matplotlib import font_manager
24     fonts = {f.name for f in font_manager.fontManager.ttflist}
25     selected = next((f for f in ('PingFang SC', 'Heiti TC', 'Microsoft YaHei', 'SimHei',
26                                'SimSun', 'Arial Unicode MS', 'Noto Sans CJK SC',
27                                'Songti SC') if f in fonts), None)
28     if selected is None:
29         raise RuntimeError('缺少中文字体, 拒绝输出乱码图')
30     plt.rcParams.update({'font.family': selected, 'font.size': 10, 'axes.titlesize': 12,
31                          'axes.labelsize': 10, 'legend.fontsize': 9, 'axes.unicode_minus': False,
32                          'axes.spines.top': False, 'axes.spines.right': False, 'axes.grid': True,
33                          'axes.axisbelow': True, 'grid.alpha': .17, 'svg.fonttype': 'path',
34                          'figure.facecolor': 'white', 'savefig.facecolor': 'white'})
35     return plt, selected
36
37
38 def cost_palette():
39     from matplotlib.colors import LinearSegmentedColormap
40     return LinearSegmentedColormap.from_list('cost_ocean', ['#ECF5F2', '#8FC6CC', '#4384AC', '#223
41     B66'])
42
43 def save(fig, output, name):
44     import matplotlib.pyplot as plt
45     with warnings.catch_warnings():
46         warnings.filterwarnings('error', message='Glyph .* missing from font')
47         for suffix in ('png', 'svg'):
48             fig.savefig(output / f'{name}.{suffix}', dpi=300)
49     plt.close(fig)

```

src/plots/project_expansion.py

```

1  """十二张高密度补充图：无区域边框/阴影、纯色曲线和TeX中文。"""
2  from pathlib import Path
3  import argparse, json, warnings
4  import numpy as np
5  import pandas as pd
6  from matplotlib.colors import LinearSegmentedColormap, Normalize
7  from matplotlib.text import Text
8  from matplotlib.patches import Rectangle
9  from .q4_paper_final import fonts, STYLE, plt, INK, DARK, BLUE, PERI, ICE, LILAC, APRICOT, GRAY, SEQ
10 ROOT=Path(__file__).resolve().parents[2]; OUT=ROOT/'outputs/processed/figures/
11     project_expansion_20260913'; DATA=OUT/'data'
12 COLORS={'Q2':DARK, 'Q3':BLUE, 'Q4-2':PERI, 'Q4-3': '#8472B8'}
13 DIV=LinearSegmentedColormap.from_list('signed_flat', [DARK, ICE, '#FBFCFF', '#FFE8CA', APRICOT])
14 MANIFEST=[]
15
16 def read(name): return pd.read_csv(DATA/name, float_precision='round_trip')
17 def title(ax, s, pad=10): ax.set_title(s, loc='left', pad=pad)
18 def clean(ax): ax.grid(True, axis='y'); ax.grid(False, axis='x')
19 def timeaxis(ax): ax.set_xlim(0, 24); ax.set_xticks([0, 6, 12, 18, 24]); ax.set_xlabel('时刻 / h')
20 def noframe(ax):

```



```

21 ax.grid(False)
22 for spine in ax.spines.values():spine.set_visible(False)
23
24
25 def save(fig,num,name,caption,data):
26     for t in fig.findobj(Text):
27         s=t.get_text()
28         if '$' in s and any('\u4e00'<=c<='\u9fff' for c in s):t.set_fontfamily('FandolSong')
29     with warnings.catch_warnings():
30         warnings.filterwarnings('error',message='Glyph .* missing from font')
31         fig.canvas.draw()
32         for suffix in ['png','svg']:fig.savefig(OUT/f'S{num:02}_{name}.{suffix}',dpi=400,facecolor='
white',transparent=False)
33     status=json.loads((OUT/'status.json').read_text(encoding='utf-8'));item=status['items'][num-1]
34     item.update({'png':f'S{num:02}_{name}.png','svg':f'S{num:02}_{name}.svg','status':'已生成，待最终验
收'})
35     status['stage']=f'已成图 {len(MANIFEST)+1}/12 · 正在继续绘制与检查'
36     MANIFEST.append({'question':'cross','name':f'S{num:02}_{name}','title':f'补充图S{num:02} '+item['
title'],'caption':caption,'data':data,
37                     'png':item['png'],'svg':item['svg'],'width_inches':fig.get_figwidth(),'
height_inches':fig.get_figheight(),'dpi':400,
38                     'font':'FandolSong / Latin Modern Roman / Computer Modern','script':'src/plots/
project_expansion.py','necessity':'全项目补充图，按正文论证需要选用'})
39     temp=OUT/'status.pending';temp.write_text(json.dumps(status,ensure_ascii=False,indent=2),
encoding='utf-8');temp.replace(OUT/'status.json')
40     (OUT/'manifest.json').write_text(json.dumps(MANIFEST,ensure_ascii=False,indent=2),encoding='utf
-8')
41     plt.close(fig);print(f'S{num:02} '+name,flush=True)
42
43
44 def overview():
45     f=read('information_boundaries.csv');fig,(ax,bt)=plt.subplots(2,1,figsize=(12.1,7.7),gridspec_kw
={ 'height_ratios':[1.65,1]},layout='constrained');ax.axis('off')
46     tab=ax.table(cellText=f.values,colLabels=f.columns,cellLoc='center',bbox=(0,0,1,.93),colWidths
=[.12]+[.176]*5)
47     tab.auto_set_font_size(False);tab.set_fontsize(10.2)
48     for (i,j),cell in tab.get_celld().items():
49         cell.set_edgecolor('none');cell.set_linewidth(0)
50         cell.set_facecolor(ICE if i==0 else '#ECF0FA' if j==0 else '#F6F8FC' if i%2 else '#FFFFFF')
51         cell.get_text().set_color(INK)
52     title(ax,'(a) 同一物理系统中的信息边界与决策方式')
53     labels=['问题一','问题二','问题三','问题4-2','问题4-3'];H=[24,72,24,48,24];B=[24,24,6,24,6]
54     for i,(h,b) in enumerate(zip(H,B)):
55         bt.barh(i,h,color=ICE,height=.54,edgecolor='none',lw=0)
56         bt.barh(i,b,color=DARK if i%2==0 else BLUE,height=.54,edgecolor='none',lw=0)
57         bt.text(b/2,i,f'执行{b}h',ha='center',va='center',fontsize=9,color='white')
58         if h>b:bt.text(b+(h-b)/2,i,'前瞻代理',ha='center',va='center',fontsize=9,color=INK)
59         if i==1:bt.plot([48,48],[i-.34,i+.34],color=GRAY,ls=':',lw=1);bt.text(74,i,'按日选择48/72 h',
va='center',fontsize=9,color=GRAY)
60     bt.set_yticks(range(5),labels);bt.invert_yaxis();bt.set_xlim(0,95);bt.set_xticks([0,6,24,48,72])
;bt.set_xlabel('单次优化窗口中的相对时间 / h');noframe(bt)
61     title(bt,'(b) 优化前瞻与实际执行长度分开理解')
62     save(fig,1,'information_boundaries','五列分开描述不同题目的信息、策略和风险表示，不能用不同评价范围和电价口
径的总费用直接排名。问题二为当前正式实际48/72h选择，局部K=1实验不混入此表；问题三与4-3全天四节点，此处时间带仅表
示一次节点优化。问题三正式轨迹含限时可行节点。',[ 'information_boundaries.csv'])
63
64

```

```

65 def worst_distribution():
66     f=read('dro_example.csv').sort_values('cost');y=np.arange(len(f));fig,axes=plt.subplots(1,3,
        figsize=(11.3,6.5),sharey=True,layout='constrained')
67     ax=axes[0];ax.barh(y,f.cost/10000,color=ICE,edgecolor='none');ax.set_yticks(y,[f'S{n:02}' for n
        in f.scenario]);ax.invert_yaxis();ax.set_xlabel('同一策略的场景费用 / 万元');title(ax,'(a) 按损失排序
        的20个场景');clean(ax)
68     ax=axes[1]
69     for yy,p,w in zip(y,f.empirical_p,f.worst_p):ax.plot([p,w],[yy,yy],color=PERI,lw=1.6)
70     ax.scatter(f.empirical_p,y,color=PERI,s=32,marker='o',edgecolors='none',label='经验概率')
71     ax.scatter(f.worst_p,y,color=DARK,s=30,marker='D',edgecolors='none',label='最坏概率')
72     ax.set_xlim(-.009,max(f.empirical_p.max(),f.worst_p.max())*1.23);ax.set_xlabel('概率');title(ax,'
        (b) 概率重分配');ax.legend(loc='lower right',fontsize=9);clean(ax)
73     ax=axes[2];ax.barh(y,f.delta_contribution/1000,color=[BLUE if v>=0 else LILAC for v in f.
        delta_contribution],edgecolor='none');ax.axvline(0,color=GRAY,lw=.8)
74     ax.set_xlabel('概率变化带来的费用贡献 / 千元');title(ax,'(c) 贡献可正可负, 合计为溢价');clean(ax)
75     ep=np.dot(f.empirical_p,f.cost);wp=np.dot(f.worst_p,f.cost)
76     fig.suptitle(f'Q4-3 • 2025-07-01 00:00 经验期望 {ep:,.2f} 元 → 最坏期望 {wp:,.2f} 元',fontsize
        =12,color=INK)
77     save(fig,2,'worst_distribution','使用预先指定高联合风险日7月1日0点原审计的20个场景, S编号为原审计顺序, 纵向
        只按同一固定策略的场景费用排序。贡献=(最坏概率-经验概率)×场景费用, 各贡献之和等于最坏期望相对经验期望的风险溢价。
        连线连接同一个场景的概率值, 不表示已知的场景间运输路径; 风险溢价不是实际费用节约。',[ 'dro_example.csv'])
78
79
80 def dro_annual():
81     f=read('dro_nodes.csv');f=f[f.model=='Q4-3'];fig,axes=plt.subplots(1,3,figsize=(10.3,5.8),sharey
        =True,layout='constrained')
82     dates=sorted(f.date.unique());positions=[0,59,120,181,242,303,333]
83     for ax,key,label in zip(axes,['rho','ESS','premium_pct'],[(a) 标定Wasserstein半径','(b) 条件历史有
        效样本量','(c) 最坏期望风险溢价 / %']):
84         v=f.pivot(index='date',columns='hour',values=key).to_numpy();im=ax.imshow(v,aspect='auto',
        cmap=SEQ,interpolation='nearest',origin='upper',extent=(-.5,3.5,334,0),vmin=0)
85         ax.set_xticks(range(4),['00','06','12','18']);ax.set_yticks(np.array(positions)+.5,[dates[i
        ] [5:] for i in positions]);ax.set_xlabel('节点 / h');noframe(ax);title(ax,label)
86         fig.colorbar(im,ax=ax,pad=.03,shrink=.86,label={ 'rho': '距离单位', 'ESS': '等效历史轨迹数', '
        premium_pct': '%'}[key])
87     axes[0].set_ylabel('2025年正式日期')
88     save(fig,3,'dro_annual_diagnostics','Q4-3正式334日×4节点的原始半径、ESS和风险溢价, 逐节点显示, 不按月份平
        滑。ESS由历史条件概率计算; 风险溢价=100×(最坏期望/经验期望-1), 分母为同一策略的场景经验期望, 非真实日费; 零分母
        若出现则留空。三个面板色标单位不同。',[ 'dro_nodes.csv'])
89
90
91 def storage_pair(models,num):
92     den=read('soc_density.csv');pr=read('action_profiles.csv');daily=read('daily_states.csv');fig,
        axes=plt.subplots(2,3,figsize=(11.2,7),layout='constrained')
93     for i,model in enumerate(models):
94         ax=axes[i,0];g=den[den.model==model];v=g.pivot(index='soc_low',columns='slot0',values='
        probability').to_numpy()*100
95         im=ax.imshow(v,aspect='auto',origin='lower',extent=(0,24,1.2,10.8),cmap=SEQ,vmin=0,vmax=100,
        interpolation='nearest')
96         ax.set_yticks([1.2,6,10.8]);ax.set_ylabel('SOC / MWh');timeaxis(ax);noframe(ax);title(ax,f'
        ({chr(97+3*i)}) {model} 时刻—SOC占用')
97         fig.colorbar(im,ax=ax,pad=.02,shrink=.87,label='该时刻日期占比 / %')
98         ax=axes[i,1];g=pr[pr.model==model]
99         ax.stackplot(g.hour,g.charge_fraction*100,g.discharge_fraction*100,g.idle_fraction*100,
        colors=[PERI,ICE,'#F0EDF7'],edgecolor='none',linewidth=0,labels=['充电','放电','闲置'])
100        ax.set_ylim(0,100);ax.set_ylabel('动作日期比例 / %');timeaxis(ax);clean(ax);title(ax,f'({chr
        (98+3*i)}) {model} 日内动作结构');ax.legend(loc='lower left',bbox_to_anchor=(0,1.0),ncol=3,

```

```

fontsize=8.5);ax.set_title(f'({chr(98+3*i)}) {model} 日内动作结构',loc='left',pad=27)
101     ax=axes[i,2];v=np.sort(daily[daily.model==model].end_soc.to_numpy()/1000);p=np.arange(1,len(
v)+1)/len(v)
102     ax.step(v,p,where='post',color=COLORS[model],lw=1.8);ax.set_xlim(1.0,11);ax.set_ylim(0,1.02)
;ax.set_xlabel('日末SOC / MWh');ax.set_ylabel('累计日期比例');clean(ax)
103     title(ax,f'({chr(99+3*i)}) {model} 日末库存分布');ax.text(.98,.10,f'n=334日\n中位数 {np.median(
v):.3f} MWh',transform=ax.transAxes,ha='right',fontsize=9,color=GRAY)
104 save(fig,num,'storage_'+('_'.join(models)),'每个时刻均有334个正式日，SOC以400kWh分箱，每列概率合计100%
;只有浮点容差内的越界值归入边界箱，原始轨迹完整保存。充/放电状态以1e-5kWh阈值识别，余者为闲置，类别互斥。各图共
用0-100%色标和1200-10800kWh范围；末库存ECDF保留全部334日。这是运行方式描述，各模型预测来源、信息和策略不同，
不能据占用形态直接宣称经济优劣。',[ 'soc_density.csv','action_profiles.csv','daily_states.csv'])
105
106
107 def reserve():
108     f=read('reserve_day.csv');x=f.hour.to_numpy();fig,axes=plt.subplots(3,1,figsize=(10.1,7.5),
sharex=True,layout='constrained')
109     ax=axes[0];ax.plot(x,f.net_kwh,color=DARK,lw=1.7,label='实际净负荷');ax.plot(x,f.forecast_net_kwh,
color=PERI,ls='--',lw=1.5,label='点预测净负荷');ax.step(x,f.grid,where='pre',color=BLUE,ls=':',lw
=1.5,label='冻结购电 G')
110     ax.axhline(0,color=GRAY,lw=.65);ax.set_ylabel('电量 / (kWh/10 min)');title(ax,'(a) 预测误差与冻结购
电',pad=29);ax.legend(loc='lower left',bbox_to_anchor=(0,1.01),ncol=3,fontsize=9)
111     ax=axes[1];soc=np.r_[f.initial_soc.iloc[0],f.soc];xx=np.arange(145)/6
ax.plot(xx,soc,color=DARK,lw=1.8,label='真实SOC');ax.step(xx,np.r_[f.reserve_threshold,f.
reserve_threshold.iloc[-1]],where='post',color=BLUE,ls='--',lw=1.45,label=r'$E_{min}+R_t$ 放电保
留阈值')
112     ax.fill_between(xx,1200,np.r_[f.reserve_threshold,f.reserve_threshold.iloc[-1]],step='post',
color=ICE,alpha=.4,lw=0);ax.axhline(1200,color=GRAY,lw=.65,ls=':');ax.set_ylim(450,11400);ax.
set_ylabel('库存 / kWh');title(ax,'(b) 历史信息预先确定的安全裕度',pad=29);ax.legend(loc='lower left'
,bbox_to_anchor=(0,1.01),ncol=2,fontsize=9)
113     ax=axes[2];ax.bar(x-1/12,f.discharge,width=.15,color=BLUE,label='实际放电',edgecolor='none');ax.
bar(x-1/12,f.emergency,bottom=f.discharge,width=.15,color=APRICOT,label='紧急补购',edgecolor='
none')
114     ax.set_ylabel('缺口电量 / kWh');ax.set_ylim(bottom=0);title(ax,'(c) 储能后缺口的实际去向',pad=29);ax.
legend(loc='lower left',bbox_to_anchor=(0,1.01),ncol=2,fontsize=9)
115     for ax in axes:timeaxis(ax);clean(ax)
116     for ax in axes[:-1]:ax.set_xlabel('')
117     fig.suptitle(f'问题二 · {f.date.iloc[0]} · 正式期紧急量最大日，合计 {f.emergency.sum():.1f} kWh',
fontsize=12)
118     save(fig,6,'dynamic_reserve_and_risk','按正式334日紧急电量最大值客观选择2025-06-01。放电可用能量=.9*max
(Eprev-1200-R,0)，实际放电还受冻结上限和当期缺口限制；图中阈值不是额外SOC硬约束，不能把SOC低于当前变动阈值误判
为物理越界。下方放电与紧急堆叠合计等于正缺口。全部来自同一正式轨迹，不作无裕度反事实收益解释。',[ 'reserve_day.
csv'])
119
120
121
122 def concentration():
123     f=read('risk_concentration.csv');shares=read('risk_shares.csv');fig,axes=plt.subplots(2,2,
figsize=(10,6.9),layout='constrained')
124     for j,models in enumerate(['Q2','Q3'],['Q4-2','Q4-3']):
125         ax=axes[0,j]
126         for model,ls in zip(models,['-','--']):
127             g=f[f.model==model];ax.plot(g.day_fraction*100,g.cumulative_cost_fraction*100,color=
COLORS[model],ls=ls,lw=1.8,label=model)
128             ax.plot([0,100],[0,100],color=GRAY,ls=':',lw=.8);ax.set_xlim(0,100);ax.set_ylim(0,102);ax.
set_xlabel('从风险高到低纳入的日期比例 / %');ax.set_ylabel('累计紧急费用占比 / %');title(ax,f'({chr(97+
j)}) 各策略内部的风险集中程度');clean(ax);ax.legend(loc='lower right')
129     ax=axes[1,j];x=np.arange(3)
130     for i,model in enumerate(models):

```

```

131     g=shares[shares.model==model].sort_values('nominal_fraction');xx=x+(i-.5)*.32
132     ax.bar(xx,g.share*100,width=.31,color=ICE if i==0 else LILAC,edgecolor='none',label=
model)
133     for p,v in zip(xx,g.share*100):ax.text(p,v+1,f'{v:.1f}',ha='center',fontsize=9,color=
COLORS[model])
134     ax.set_ylim(0,100);ax.set_xticks(x,['前5%\n17日','前10%\n34日','前20%\n67日']);ax.set_ylabel('
紧急费用占比 / %');title(ax,f'({chr(99+j)}) 高风险日期的费用贡献');ax.legend(loc='upper left',ncol=2)
;clean(ax)
135 save(fig,7,'tail_risk_concentration','对每个模型分别按334日紧急费用从高到低排序,累积归一化分母为该模型自己
的全年紧急费用;不是把不同价格口径的绝对费用直接比较。5/10/20%采用ceil,实际纳入17/34/67日,各模型高风险日期集
合可不同。所有日期包含在累计曲线中。',[ 'risk_concentration.csv','risk_shares.csv'])
136
137
138 def event_geometry():
139     f=read('emergency_events.csv');fig,axes=plt.subplots(2,2,figsize=(10.3,7.2),sharex=True,sharey=
True,layout='constrained');norm=Normalize(f.weighted_price.min(),f.weighted_price.max())
140     for ax,model,label in zip(axes.ravel(),COLORS,['a','b','c','d']):
141         g=f[f.model==model];im=ax.scatter(g.duration_h,g.energy_kwh,c=g.weighted_price,cmap=SEQ,norm
=norm,s=12,alpha=.65,edgecolors='none')
142         ax.set_xscale('log');ax.set_yscale('log');ax.set_xlabel('连续事件持续时间 / h (对数)');ax.
set_ylabel('事件紧急电量 / kWh (对数)');clean(ax)
143         title(ax,f'({label}) {model} • {len(g):,}次事件');ax.text(.98,.04,f'最长 {g.duration_h.max()
:.2f} h\n最大 {g.energy_kwh.max():.1f} kWh',transform=ax.transAxes,ha='right',fontsize=9,color=
GRAY)
144     fig.colorbar(im,ax=axes.ravel().tolist(),pad=.02,shrink=.8,label='紧急电量加权实价 / (元/kWh)')
145     save(fig,8,'emergency_event_geometry','统一以单时段紧急量>1e-5kWh定义事件,连续144以外的跨午夜事件不人为
拆开;因此计数可与原按日重置的事件表不同。持续时间=时段数/6h,总量为逐槽紧急量之和,颜色= $\Sigma(pq)/\Sigma q$ 。全部事件为正
值并保留,双对数仅用于同时观察大小事件,没有删除极端值。各模型采用共同色标。',[ 'emergency_events.csv'])
146
147
148 def calibration():
149     f=read('interval_monthly.csv');fig,axes=plt.subplots(2,2,figsize=(10.5,7.1),layout='constrained'
);bound=max(abs(f.bias_pp).max(),1)
150     for ax,nom,label in zip(axes[0],[.8,.9],['a','b']):
151         g=f[np.isclose(f.nominal,nom)];v=g.pivot(index='horizon',columns='month',values='bias_pp').
reindex(index=[0,1,2],columns=range(2,13)).to_numpy()
152         cm=DIV.copy();cm.set_bad('#EEF0F3');im=ax.imshow(v,cmap=cm,vmin=-bound,vmax=bound,aspect='
auto',interpolation='nearest')
153         for y in range(3):
154             for x in range(11):ax.text(x,y,'—' if np.isnan(v[y,x]) else f'{v[y,x]:+.0f}',ha='center
',va='center',fontsize=8.5,color='white' if np.isfinite(v[y,x]) and v[y,x]<-bound*.65 else INK)
155         ax.set_xticks(range(11),range(2,13));ax.set_yticks(range(3),['h=0','h=1','h=2']);ax.
set_xlabel('预测起点月份');noframe(ax);title(ax,f'({label}) {nom:.0%}区间:覆盖率偏差 / 百分点')
156         fig.colorbar(im,ax=ax,pad=.02,shrink=.8)
157     ax=axes[1,0]
158     for h,c in zip([0,1,2],[DARK,BLUE,PERI]):
159         for nom,ls in [(.8,'-'),(.9,'--')]:
160             g=f[(f.horizon==h)&np.isclose(f.nominal,nom)].sort_values('month')
161             ax.plot(g.month,g.mean_width_kwh,color=c,ls=ls,marker='o' if nom==.8 else 's',ms=3,lw
=1.4,label=f'h={h} • {nom:.0%}')
162     ax.set_xticks(range(2,13));ax.set_xlabel('预测起点月份');ax.set_ylabel('平均区间宽度 / (kWh/10 min)')
;title(ax,'(c) 不能只看覆盖率,还需看区间宽度');ax.legend(loc='upper left',ncol=3,fontsize=8);clean(
ax)
163     ax=axes[1,1]
164     for h,c in zip([0,1,2],[DARK,BLUE,PERI]):
165         g=f[(f.horizon==h)&np.isclose(f.nominal,.8)].set_index('month').reindex(range(2,13))
166         ax.bar(np.arange(2,13)+(h-1)*.24,g.n.fillna(0)/144,width=.23,color=c,alpha=.65,edgecolor='

```

```

167     none', label=f'h={h}'))
168 ax.set_xticks(range(2,13));ax.set_xlabel('预测起点月份');ax.set_ylabel('实际有效起点数 / 日');ax.
    set_ylim(0,38);title(ax,'(d) 每格统计支持与缺失来源');ax.legend(loc='upper left',ncol=3);clean(ax)
169 save(fig,9,'seasonal_interval_calibration','原正式审计的selected_origin_indices与weights直接重建场景
    分位: 真实历史误差叠加当前冻结预测, 对L/PV分别截非负后算净负荷; 使用原离散左分位。1942条origin×h×nominal覆盖计
    数与正式calibration逐条一致, 无重新优化或聚类。月覆盖按覆盖时段数/总时段数加权; 宽度为原区间upper-lower均值。
    无h=2的月份显示缺失, 底部支持数为0; 不同h可用起点数不同, 不能解释成同起点严格配对的跨h比较。',[
170     interval_monthly.csv','interval_origin_stats.csv','original_calibration.csv'])
171
172 def resource_gains():
173     f=read('device_joint_grid.csv');m=f.pivot(index='e_max_kwh',columns='common_power_kw',values='
    cost_yuan').sort_index();v=m.to_numpy();ev=m.index.to_numpy()/1000;pv=m.columns.to_numpy()/1000
174     gain_e=(v[:-1]-v[1:])/np.diff(ev)[:,None];gain_p=(v[:,-1]-v[:,1:])/np.diff(pv)[None,:])
175     fig,axes=plt.subplots(1,3,figsize=(11.5,5.1),layout='constrained')
176     for ax,z,ys,xs,label,unit in [(axes[0],v/10000,ev,pv,'(a) 110个已求解配置','单日费用 / 万元'),(axes
    [1],gain_e,(ev[:-1]+ev[1:])/2,pv,'(b) 扩充容量的有限步长收益','元/日 / MWh'),(axes[2],gain_p,ev,(pv
   [:-1]+pv[1:])/2,'(c) 扩充功率的有限步长收益','元/日 / MW')]:
177         im=ax.imshow(z,origin='lower',aspect='auto',cmap=SEQ,interpolation='nearest');ix=np.unique(
    np.linspace(0,len(xs)-1,5).astype(int));iy=np.unique(np.linspace(0,len(ys)-1,6).astype(int))
178         ax.set_xticks(ix,[f'{xs[i]:g}' for i in ix]);ax.set_yticks(iy,[f'{ys[i]:g}' for i in iy]);ax
    .set_xlabel('共同充放电功率 / MW');ax.set_ylabel('储电上限 / MWh if ax is not axes[1] else '容量增量
    区间中点 / MWh');noframe(ax);title(ax,label)
179         fig.colorbar(im,ax=ax,pad=.02,shrink=.86,label=unit)
180     save(fig,10,'capacity_power_marginal_gains','仅使用Q1已完成的11容量×10功率共110个原MILP结果, 不做插值。
    容量收益=(当前费-下一容量费)/增加MWh, 步长2.4MWh; 功率收益按相邻1MW差计算, 第三面板横坐标是功率增量区间中点。有
    限步长差不是连续影子导数; 超过原额定上限的配置为数学约束放宽, 未计投资, 不是设备采购ROI。费用均为原单个典型日。',[
    'device_joint_grid.csv','device_one_dimensional.csv'])
181
182 def marginal_validation():
183     f=read('milp_rhs_checks.csv');m=read('fixed_mode_marginals.csv');g=f[f.kind=='internal_injection
    '];fig,axes=plt.subplots(2,2,figsize=(10.6,7),layout='constrained')
184     ax=axes[0,0]
185     for key,c,ls,label in [(('fixed_shadow_dJ_db',DARK,'-','固定模式影子斜率'),('minus_slope',PERI,'--',
    '原MILP负向扰动'),('plus_slope',BLUE,':','原MILP正向扰动'))]:ax.plot(g.t/6,g[key],color=c,ls=ls,lw
    =1.6,label=label)
186     ax.set_ylabel('内部注入的费用斜率 / (元/kWh)');timeaxis(ax);title(ax,'(a) 局部边际值随时刻变化');ax.
    legend(loc='lower right',fontsize=8);clean(ax)
187     ax=axes[0,1];keys=['E_min_active','E_max_active','charge_max_active','discharge_max_active'];v=m
    [keys].to_numpy().T
188     ax.imshow(v,aspect='auto',cmap=LinearSegmentedColormap.from_list('binding',['#F2F5FA',BLUE]),
    vmin=0,vmax=1,interpolation='nearest',extent=(0,24,3.5,-.5))
189     ax.set_yticks(range(4),['SOC下界','SOC上界','充电上限','放电上限']);timeaxis(ax);noframe(ax);title(
    ax,'(b) 同一最优解的约束活跃时段')
190     ax=axes[1,0]
191     for kind,c in [(('bus_demand',BLUE),('internal_injection',PERI))]:
192         g=f[f.kind==kind]
193         for key,marker in [(('minus_slope','o'),('plus_slope','^'))]:ax.scatter(g.fixed_shadow_dJ_db,g
    [key],s=18,color=c,marker=marker,alpha=.5,edgecolors='none')
194     lo=min(f.fixed_shadow_dJ_db.min(),f.minus_slope.min(),f.plus_slope.min());hi=max(f.
    fixed_shadow_dJ_db.max(),f.minus_slope.max(),f.plus_slope.max());ax.plot([lo,hi],[lo,hi],color=
    GRAY,ls='--',lw=.9)
195     ax.set_xlabel('固定模式斜率 / (元/kWh)');ax.set_ylabel('原MILP有限差分 / (元/kWh)');title(ax,'(c) 两
    类扰动的双侧核验 (576个值)');clean(ax)
196     ax=axes[1,1];group=f.groupby('kind').fixed_shadow_matches_milp_both_sides.agg(['sum','count']);
    order=['bus_demand','internal_injection'];ratios=group.loc[order,'sum']/group.loc[order,'count']

```



```

197 ax.bar([0,1],ratios*100,color=[ICE,LILAC],edgecolor='none',width=.55)
198 for i,k in enumerate(order):ax.text(i,ratios.iloc[i]*100+2,f'{int(group.loc[k,"sum"])}/{int(
199 group.loc[k,"count"])}',ha='center',fontsize=11,color=DARK)
200 ax.set_xticks([0,1],['母线需求扰动','内部注入扰动']);ax.set_ylim(0,115);ax.set_ylabel('双侧与固定模式吻
    合比例 / %');title(ax,'(d) 局部适用边界: 步长0.01 kWh');clean(ax)
201
202
203 def inventory_cost():
204     daily=read('daily_states.csv');a=daily[daily.model=='Q4-2'].set_index('date');b=daily[daily.
        model=='Q4-3'].set_index('date');f=read('q3_terminal_cases.csv')
205     fig,axes=plt.subplots(1,2,figsize=(10.4,5.3),layout='constrained');ax=axes[0];x=(b.end_soc-a.
        end_soc)/1000;y=(b.total_cost-a.total_cost)/1000
206     ax.scatter(x,y,color=BLUE,s=19,alpha=.35,edgecolors='none');ax.axhline(0,color=GRAY,lw=.7);ax.
        axvline(0,color=GRAY,lw=.7)
207     ax.set_xlabel('日末库存差: 4-3 - 4-2 / MWh');ax.set_ylabel('当日费用差: 4-3 - 4-2 / 千元');title(ax,'
        (a) 两条全年轨迹的334个同日配对');clean(ax)
208     ax=axes[1]
209     for (date,g),c,ls in zip(f.groupby('date',sort=True),[DARK,BLUE,PERI,'#8C75AD'],['-','--','-.','
        :']):
210         g=g.sort_values('value');base=g[np.isclose(g.value,1)].iloc[0];x=(g.final_soc-base.final_soc
        )/1000
211         ax.plot(x,g.delta_cost_pct,color=c,ls=ls,lw=1.5,label=date[5:])
212         for xx,row in zip(x,g.itertuples()):ax.plot(xx,row.delta_cost_pct,marker='o' if row.reliable
        else 'x',ms=6,color=c,ls='none')
213     ax.axhline(0,color=GRAY,lw=.7);ax.axvline(0,color=GRAY,lw=.7);ax.set_xlabel('相对同日基准的日末库存
        变化 / MWh');ax.set_ylabel('相对同日基准的费用变化 / %');title(ax,r'(b) Q3四代表日: $\lambda_E
        =0.8/1/1.2$');ax.legend(loc='lower right',ncol=2,fontsize=9);clean(ax)
214     save(fig,12,'inventory_cost_coupling','左图为Q4同日真实费用与日末SOC之差, 包含连续库存继承及整个策略的其他
        差异, 散点关联不估计库存因果价值。右图是已完成Q3四日14E局部OFAT实验, 同日初态固定, 每日1.0为自身基准; 线只连三个实
        测点。叉号表示该案例存在未认证节点, 圆点为全节点达到3%证书。日末库存明显不同, 因此不能单看低当日费用认定跨日经济性
        更优。',['daily_states.csv','q3_terminal_cases.csv'])
215
216
217 def main():
218     global OUT,DATA
219     parser=argparse.ArgumentParser();parser.add_argument('--data-dir',type=Path,default=DATA);parser
        .add_argument('--output-dir',type=Path,default=OUT);args=parser.parse_args();DATA=args.data_dir.
        resolve();OUT=args.output_dir.resolve();fonts()
220     with plt.rc_context(**STYLE,'patch.edgecolor':'none','patch.linewidth':0,'legend.fontsize':9):
221         overview();worst_distribution();dro_annual();storage_pair(['Q2','Q3'],4);storage_pair(['Q4-2
        ','Q4-3'],5);reserve();concentration();event_geometry();calibration();resource_gains();
        marginal_validation();inventory_cost()
222     (OUT/'captions.md').write_text('# 十二张补充图图注\n\n'+'\n\n'.join('## '+r['title']+'\n\n'+r['
        caption'] for r in MANIFEST)+'\n',encoding='utf-8')
223     s=json.loads((OUT/'status.json').read_text(encoding='utf-8'));s['stage']='12/12张已绘制, 正在完成数
        值与视觉验收';(OUT/'status.json').write_text(json.dumps(s,ensure_ascii=False,indent=2),encoding='
        utf-8')
224
225
226 if __name__=='__main__':main()

```

src/plots/project_expansion_data.py

```

1  """全项目补充图的只读统计，保留各问口径与来源。"""
2  from pathlib import Path
3  import json,hashlib,shutil
4  import numpy as np
5  import pandas as pd
6  ROOT=Path(__file__).resolve().parents[2]
7  OUT=ROOT/'outputs/processed/figures/project_expansion_20260913';DATA=OUT/'data'
8  RUNS={'Q2':ROOT/'outputs/q2/dispatch_runs/20260912_tail_reserve_final/processed/main','Q3':ROOT/'
    outputs/q3/raw/full_priority_rescue_7h_20260912/main',
9      'Q4-2':ROOT/'outputs/q4/raw/rescue_lp_q42_20260913_v2','Q4-3':ROOT/'outputs/q4/raw/
    rescue_lp_q43_20260913_v3'}
10
11
12 def write(name,f):f.to_csv(DATA/name,index=False,encoding='utf-8',float_format='%.17g')
13 def read(path):return pd.read_csv(path,float_precision='round_trip')
14
15
16 def prepare():
17     DATA.mkdir(parents=True,exist_ok=True);sources=[];daily=[];dens=[];profiles=[];events=[];frames
    ={};edges=np.linspace(1200,10800,25)
18     for model,path in RUNS.items():
19         src=path/'dispatch.csv';sources.append(src);f=read(src);assert len(f)==334*144
20         f['slot0']=f.groupby('date').cumcount();f['hour']=(f.slot0+1)/6
21         if 'total_cost' not in f:f['total_cost']=f.planned_cost+f.emergency_cost
22         frames[model]=f
23         columns=['date','slot0','hour','timestamp','grid','charge','discharge','emergency','soc','
    initial_soc','price','net_kwh','total_cost','emergency_cost']
24         write(f'trajectory_{model}.csv',f[columns])
25         act=np.where(f.charge.to_numpy()>1e-5,0,np.where(f.discharge.to_numpy()>1e-5,1,2)).reshape
    (334,144)
26         assert not ((f.charge>1e-5)&(f.discharge>1e-5)).any()
27         soc=np.clip(f.soc.to_numpy()).reshape(334,144),1200,10800) # only tolerance-level excursions;
    raw trajectory remains unaltered
28         assert f.soc.between(1200-1e-5,10800+1e-5).all()
29         for slot in range(144):
30             counts,_=np.histogram(soc[:,slot],edges)
31             assert counts.sum()==334
32             dens.extend({'model':model,'slot0':slot,'soc_low':edges[j],'soc_high':edges[j+1],'count'
    :int(n),'probability':n/334} for j,n in enumerate(counts))
33             profiles.append({'model':model,'slot0':slot,'hour':(slot+1)/6,'charge_fraction':np.mean(
    act[:,slot]==0),'discharge_fraction':np.mean(act[:,slot]==1),'idle_fraction':np.mean(act[:,slot
    ]==2)})
34         for date,g in f.groupby('date'):
35             daily.append({'model':model,'date':date,'total_cost':g.total_cost.sum(),'emergency_cost'
    :g.emergency_cost.sum(),'emergency_kwh':g.emergency.sum(),
36                 'end_soc':g.soc.iloc[-1],'initial_soc':g.initial_soc.iloc[0],'mean_soc':g.
    soc.mean(),
37                 'lower_bound_fraction':np.mean(g.soc<=1200+1e-5),'upper_bound_fraction':np
    .mean(g.soc>=10800-1e-5)})
38             mask=f.emergency.to_numpy()>1e-5;starts=np.flatnonzero(mask&~np.r_[False,mask[:-1]]);ends=np
    .flatnonzero(mask&~np.r_[mask[1:],False])+1
39             for start,end in zip(starts,ends):
40                 g=f.iloc[start:end];energy=g.emergency.sum()
41                 events.append({'model':model,'start':g.timestamp.iloc[0],'end':g.timestamp.iloc[-1],
    'slots':end-start,'duration_h':(end-start)/6,

```

```

42         'energy_kwh':energy, 'cost_yuan':g.emergency_cost.sum(), 'weighted_price':
np.dot(g.emergency,g.price)/energy})
43 daily=pd.DataFrame(daily);write('daily_states.csv',daily);write('soc_density.csv',pd.DataFrame(
dens));write('action_profiles.csv',pd.DataFrame(profiles));write('emergency_events.csv',pd.
DataFrame(events))
44 tails=[];shares=[]
45 for model,g in daily.groupby('model'):
46     g=g.sort_values(['emergency_cost','date'],ascending=[False,True]);values=g.emergency_cost.
to_numpy();total=values.sum()
47     tails.extend({'model':model, 'rank':i, 'day_fraction':i/334, 'cumulative_cost_fraction':v/total
} for i,v in enumerate(np.r_[0,np.cumsum(values)]))
48     for frac in [.05,.1,.2]:
49         n=int(np.ceil(frac*334));shares.append({'model':model, 'nominal_fraction':frac, '
actual_days':n, 'share':values[:n].sum()/total, 'total_emergency_cost':total})
50 write('risk_concentration.csv',pd.DataFrame(tails));write('risk_shares.csv',pd.DataFrame(shares)
)
51 f=frames['Q2'];selected=f.groupby('date').emergency.sum().sort_values(ascending=False,kind='
stable').index[0];day=f[f.date==selected].copy()
52 day['previous_soc']=np.r_[day.initial_soc.iloc[0],day.soc.to_numpy()[:-1]];day['
reserve_threshold']=1200+day.soc_reserve_kwh
53 day['forecast_net_kwh']=(day.predicted_load_kw-day.predicted_pv_kw)/6
54 day['deficit']=np.maximum(day.net_kwh-day.grid,0);day['available_discharge']=.9*np.maximum(day.
previous_soc-day.reserve_threshold,0)
55 np.testing.assert_allclose(day.discharge,np.minimum.reduce([day.discharge_cap.to_numpy(),day.
deficit.to_numpy(),day.available_discharge.to_numpy()]),atol=1e-6,rtol=0)
56 write('reserve_day.csv',day)
57 node_rows=[];example=[]
58 for model in ['Q4-2','Q4-3']:
59     for path in sorted((RUNS[model]+'/audit').glob('*.json')):
60         sources.append(path);a=json.loads(path.read_text(encoding='utf-8'))
61         for node in [a] if model=='Q4-2' else a['nodes']:
62             s=node['solver'];sc=node['scenarios'];assert s['status']=='Optimal'
63             c=np.array(s['scenario_costs']);p=np.array(s['empirical_weights']);w=np.array(s['
worst_case_weights'])
64             np.testing.assert_allclose([p.sum(),w.sum()], [1,1],atol=1e-8,rtol=0)
65             empirical=p@c;worst=w@c;assert worst>=empirical-1e-6
66             np.testing.assert_allclose(worst,s['objective'],atol=1e-5,rtol=0)
67             node_rows.append({'model':model, 'date':path.stem, 'hour':node.get('hour',0), '
empirical':empirical, 'worst':worst, 'premium_yuan':worst-empirical,
68                             'premium_pct':(worst/empirical-1)*100 if empirical else np.nan, 'rho
':sc['radius'], 'ESS':sc['ESS'], 'history_n':len(sc['history_days']), 'S':len(c)})
69             if model=='Q4-3' and path.stem=='2025-07-01' and node['hour']==0:
70                 example.extend({'scenario':i+1, 'cost':cost, 'empirical_p':pp, 'worst_p':ww, '
delta_contribution':(ww-pp)*cost} for i,(cost,pp,ww) in enumerate(zip(c,p,w)))
71 write('dro_nodes.csv',pd.DataFrame(node_rows));write('dro_example.csv',pd.DataFrame(example))
72 # 原审计的场景origin与概率直接恢复分位区间，不重新构造或选择场景。
73 q2=RUNS['Q2'].parents[1];actual_path=q2/'inputs/processed/timeseries.csv';sources.append(
actual_path)
74 a=read(actual_path)[['load_kw','pv_kw']].to_numpy().reshape(365,144,2);shadow=q2/'raw/shadows';
shape=json.loads((shadow/'shape.json').read_text(encoding='utf-8'));sources.append(shadow/'shape
.json');stores={}
75 calibration=read(RUNS['Q2']/'calibration.csv');sources.append(RUNS['Q2']/'calibration.csv');
write('original_calibration.csv',calibration)
76 interval=[]
77 for date in sorted(frames['Q2'].date.unique()):
78     path=RUNS['Q2']/'daily_audit'/f'{date}.json';sources.append(path);audit=json.loads(path.
read_text(encoding='utf-8'));K=audit['selected_K'];item=audit[str(K)]

```



```

79     sc=item['candidates'][str(item['S'])]['scenarios'];ids=np.array(sc['selected_origin_indices']
80 );weights=np.array(sc['weights']);pair=item['selection']['pipeline']
81     for name in pair:
82         if name not in stores:
83             p=shadow/(name+'.dat');sources.append(p);stores[name]=np.memmap(p,mode='r',dtype=
84 shape['dtype'],shape=tuple(shape['shape']))
85             d=(pd.Timestamp(date)-pd.Timestamp('2025-01-01')).days
86             point=np.stack([stores[name][d,:K] for name in pair],axis=-1)
87             blocks=np.stack([a[j:j+K]-np.stack([stores[name][j,:K] for name in pair],axis=-1) for j in
88 ids])
89             value=np.maximum(point[None]+blocks,0);net=(value[...,0]-value[...,1])/6
90             for h in range(min(K,365-d)):
91                 v=net[:,h,:];order=np.argsort(v,axis=0,kind='stable');sorted_v=np.take_along_axis(v,
92 order,axis=0)
93                 sw=np.take_along_axis(np.broadcast_to(weights[:,None],v.shape),order,axis=0);cdf=np.
94 cumsum(sw,axis=0)
95                 q=lambda level:sorted_v[np.argmax(cdf>=level-1e-12,axis=0),np.arange(144)]
96                 truth=(a[d+h,:,0]-a[d+h,:,1])/6
97                 for nominal,lo,hi in [(.8,.1,.9),(.9,.05,.95)]:
98                     lower,upper=q(lo),q(hi);covered=int(((truth>=lower)&(truth<=upper)).sum())
99                     orig=calibration[(calibration.origin==date)&(calibration.horizon==h)&(calibration.
100 nominal==nominal)].iloc[0]
101                     assert covered==orig.covered_intervals,(date,h,nominal,covered,orig.
102 covered_intervals)
103                     interval.append({'origin':date,'month':pd.Timestamp(date).month,'horizon':h,'nominal
104 ':nominal,'n':144,'covered':covered,'width_sum':float((upper-lower).sum())})
105                 interval=pd.DataFrame(interval);write('interval_origin_stats.csv',interval)
106                 monthly=interval.groupby(['month','horizon','nominal'],as_index=False)[['n','covered','width_sum
107 ']].sum();monthly['coverage']=monthly.covered/monthly.n;monthly['bias_pp']=(monthly.coverage-
108 monthly.nominal)*100;monthly['mean_width_kwh']=monthly.width_sum/monthly.n
109                 write('interval_monthly.csv',monthly)
110                 for name in ['device_joint_grid.csv','device_one_dimensional.csv','fixed_mode_marginals.csv','
111 milp_rhs_checks.csv']:
112                     src=R00T/'outputs/processed/figures/q1'/name;sources.append(src);shutil.copy2(src,DATA/name)
113                     sens=R00T/'outputs/sensitivity_q2_q3/raw/local_20260913/daily_results.csv';sources.append(sens);
114                     ss=read(sens);write('q3_terminal_cases.csv',ss[(ss.question=='q3')&(ss.parameter=='lambda_E')])
115                 model_table=pd.DataFrame({
116                     '维度':['评价范围','电价信息','优化前瞻','正式执行块','储能决策','风险表示','求解结构'],
117                     '问题一':['单个典型日','已知分时电价','24 h','24 h','完整日计划','确定性输入','MILP'],
118                     '问题二':['正式334日','已知分时电价','自适应48/72 h','24 h','冻结上限+实时反馈','联合误差+动态裕度','随
119 机策略优化'],
120                     '问题三':['正式334日','已知分时电价','每节点24 h','6 h','节点策略+实时反馈','条件联合误差','四节点随机优
121 化'],
122                     '问题4-2':['正式334日','0点48h预测','48 h','24 h','节点固定充放电','联合误差+DR0','连续LP'],
123                     '问题4-3':['正式334日','0点48h预测','每节点24 h','6 h','节点固定充放电','条件联合误差+DR0','四节点连续
124 LP'])
125                 write('information_boundaries.csv',model_table)
126                 sources.extend([R00T/'src/q2/policy.py',R00T/'src/q4/rolling.py',R00T/'docs/plots/
127 project_expansion/补充图方案.md'])
128                 meta={'language':'中文标注, 数学符号与单位英文; 用户已确认','models':list(RUNS),'formal_days':334,'
129 density_bin_edges':edges.tolist(),'emergency_event_threshold_kwh':1e-5,
130                     'reserve_date':selected,'reserve_role':'retained-discharge threshold, not an extra hard
131 SOC constraint',
132                     'calibration_origin_rows':len(interval),'DR0_nodes':len(node_rows),'sources':{str(p.
133 relative_to(R00T)):hashlib.sha256(p.read_bytes()).hexdigest() for p in set(sources)}}
134                 (DATA/'provenance.json').write_text(json.dumps(meta,ensure_ascii=False,indent=2),encoding='utf-8
135 ');print({'reserve_date':selected,'interval_rows':len(interval),'DR0_nodes':len(node_rows),

```

```

116     source_hashes':len(meta['sources'])}},flush=True)
117
118 if __name__=='__main__':prepare()

```

src/plots/q1_legacy.py

```

1  """Q1原有四组图的唯一实现；由原管线兼容入口调用。"""
2  import numpy as np
3  from src.q1.model import DT, E_MAX, E_MIN, arrays
4
5  LABELS = {"main": "单程 0.90（主模型）", "eta085": "单程 0.85",
6            "eta095": "单程 0.95", "roundtrip090": "往返 0.90（对称）"}
7  COLORS = {"main": "#0072B2", "eta085": "#D55E00", "eta095": "#009E73", "roundtrip090": "#CC79A7"}
8
9
10 def create_figures(data, solutions, summaries, output):
11     import matplotlib
12     matplotlib.use("Agg")
13     import matplotlib.pyplot as plt
14     from matplotlib import font_manager
15     fonts = {f.name for f in font_manager.fontManager.ttflist}
16     selected = next((f for f in ("PingFang SC", "Heiti TC", "Arial Unicode MS", "Noto Sans CJK SC",
17                                  "Songti SC", "Hiragino Sans GB") if f in fonts), None)
18     if selected is None:
19         raise RuntimeError("没有可用中文字体，拒绝输出乱码图表")
20     plt.rcParams.update({"font.family": selected, "axes.unicode_minus": False, "font.size": 10,
21                          "axes.spines.top": False, "axes.spines.right": False,
22                          "axes.grid": True, "grid.alpha": 0.17, "svg.fonttype": "none"})
23
24     p, load, pv = arrays(data)
25     edges = np.arange(145) * DT
26     center = (edges[1:] + edges[:-1]) / 2
27     main = solutions["main"]
28     fig, axes = plt.subplots(3, 1, figsize=(12, 10), sharex=True, layout="constrained")
29     axes[0].stairs(load / DT, edges, color="#0072B2", label="小区负荷", linewidth=1.5)
30     axes[0].stairs(pv / DT, edges, color="#E69F00", label="光伏预测", linewidth=1.5)
31     axes[0].set_ylabel("功率 / kW", title="A 输入 · 144 个 10 min 时段，区间内零阶保持")
32     tariff = axes[0].twinx()
33     tariff.stairs(p, edges, color="#666666", label="购电价格", linewidth=1, linestyle="--")
34     tariff.set_ylabel("电价 / (元/kWh)")
35     tariff.grid(False)
36     handles, labels = axes[0].get_legend_handles_labels()
37     h2, l2 = tariff.get_legend_handles_labels()
38     axes[0].legend(handles + h2, labels + l2, loc="upper left", ncols=3)
39     axes[1].stairs(main["G"], edges, color="#0072B2", label="购电量 G", linewidth=1.3)
40     axes[1].bar(center, main["C"], width=DT * 0.9, color="#009E73", label="充电量 C")
41     axes[1].bar(center, -np.asarray(main["D"]), width=DT * 0.9, color="#D55E00", label="放电量 D（向  
下）")
42     axes[1].set_ylabel("时段电量 / kWh", title="B 输出 · 母线侧购电与储能调度")
43     axes[1].legend(loc="upper left", ncols=3)
44     axes[2].plot(edges, [main["E0"], *main["E"]], color="#0072B2", linewidth=1.8, label="储电量 E")
45     axes[2].axhline(E_MIN, color="#777777", linestyle="--", linewidth=1, label="安全上下限")
46     axes[2].axhline(E_MAX, color="#777777", linestyle="--", linewidth=1)
47     axes[2].scatter([0, 24], [main["E0"], main["E"][-1]], color="#D55E00", s=28, zorder=3)
48     axes[2].set_ylabel("储电量 / kWh", xlabel="当天时刻 / h", title="C 状态 · 日初与日末均为 6000 kWh",
49                      ylim=(0, 12000), xticks=np.arange(0, 25, 2), xlim=(0, 24))

```

```

48 axes[2].legend(loc="upper left", ncols=2)
49 fig.suptitle(f"Q1 日前经济调度 | 最低购电费 {summaries[0]['cost_yuan']:, .6f} 元", fontsize=16)
50 for suffix in ("png", "svg"):
51     fig.savefig(output / f"dispatch.{suffix}", dpi=300, facecolor="white")
52 plt.close(fig)
53
54 order = ["eta085", "main", "roundtrip090", "eta095"]
55 rows = {r["scenario"]: r for r in summaries}
56 fig, axes = plt.subplots(2, 1, figsize=(12, 8), layout="constrained")
57 bars = axes[0].bar([LABELS[k] for k in order], [rows[k]["cost_yuan"] for k in order],
58                 color=[COLORS[k] for k in order], width=0.6)
59 axes[0].bar_label(bars, labels=[f"{rows[k]['cost_yuan']:, .2f}" for k in order], padding=4)
60 axes[0].set(ylabel="最低购电费 / 元", title="A 成本 · 每个效率情景独立求解同一 MILP")
61 axes[0].set_ylim(0, max(r["cost_yuan"] for r in summaries) * 1.16)
62 for i, key in enumerate(order):
63     axes[1].plot(edges, [solutions[key]["E0"], *solutions[key]["E"]],
64                 color=COLORS[key], label=LABELS[key], linewidth=1.4,
65                 linestyle=["--", "-", ":", "-."][i])
66 axes[1].axhline(E_MIN, color="#777777", linewidth=0.8, linestyle="--")
67 axes[1].axhline(E_MAX, color="#777777", linewidth=0.8, linestyle="--")
68 axes[1].set(xlabel="当天时刻 / h", ylabel="储电量 / kWh", xticks=np.arange(0, 25, 2),
69            xlim=(0, 24), ylim=(0, 12000), title="B 轨迹 · 对比阶段规律, 具体时段策略可以变化")
70 axes[1].legend(loc="upper left", ncols=2)
71 fig.suptitle("Q1 储能效率稳健性检验", fontsize=16)
72 for suffix in ("png", "svg"):
73     fig.savefig(output / f"efficiency.{suffix}", dpi=300, facecolor="white")
74 plt.close(fig)
75
76
77 def analysis_figures(a, output):
78     import matplotlib.pyplot as plt
79     def save(fig, name):
80         for suffix in ("png", "svg"):
81             fig.savefig(output / f"{name}.{suffix}", dpi=300, facecolor="white")
82         plt.close(fig)
83     rows = a["main_marginals"]
84     edges = np.arange(145) * DT
85     fig, axes = plt.subplots(3, 1, figsize=(12, 10), layout="constrained")
86     for field, label, color, style in (("mu_yuan_per_kwh", "母线边际成本  $\mu$ ", "#0072B2", "-"),
87                                       ("charge_threshold", "充电阈值  $\eta\lambda$ ", "#009E73", "--"),
88                                       ("discharge_threshold", "放电阈值  $\lambda/\eta$ ", "#D55E00", ":")):
89         axes[0].stairs([r[field] for r in rows], edges, label=label, color=color, linestyle=style)
90     axes[0].set(title="A 固定最优模式的局部阈值 · 等号处可有非零动作", ylabel="元/kWh", xlabel="时刻 / h",
91               xlim=(0, 24))
92     axes[0].legend(ncols=3, fontsize=9)
93     bars = axes[1].bar(["无储能", "仅光伏消纳储能", "完整 MILP"],
94                      [r["cost_yuan"] for r in a["baseline_rows"]], color=["#8296A6", "#E69F00", "#0072B2"])
95     axes[1].bar_label(bars, fmt="%.2f", padding=4)
96     axes[1].set(title="B 嵌套可行域增量节约 · 归因依赖中间基准", ylabel="全天购电费 / 元", ylim=(0, 55000))
97     scenarios = [r for r in a["scenarios"] if r["efficiency_name"] == "main"]
98     costs = np.array([r["cost_yuan"] for r in scenarios]).reshape(3, 3)
99     im = axes[2].imshow(costs, cmap="cividis", aspect="auto")
100    for i in range(3):
101        for j in range(3):
102            axes[2].text(j, i, f"{costs[i,j]:, .2f}", ha="center", va="center",

```

```

102         color="white" if costs[i,j] < (costs.min() + costs.max()) / 2 else "#12232e
    ")
103 axes[2].set(xticks=[0,1,2], xticklabels=["-5%", "原输入", "+5%"],
104             yticks=[0,1,2], yticklabels=["-5%", "原输入", "+5%"], xlabel="光伏规模", ylabel="负荷规
    模",
105             title="C 主效率口径下的九组负荷×光伏扰动")
106 axes[2].grid(False)
107 fig.colorbar(im, ax=axes[2], label="最低购电费 / 元")
108 fig.suptitle("Q1 边际机制、基准比较与输入敏感性", fontsize=16)
109 save(fig, "marginal_analysis")
110 fig, axes = plt.subplots(2, 2, figsize=(12, 9), layout="constrained")
111 for ax, resource, title, unit in zip(axes.flat, ("storage_upper", "storage_lower", "charge_power
    ", "discharge_power"),
112                                     ("安全储能上限提高", "安全储能下限降低", "充电功率上限提高", "放电功
    率上限提高"),
113                                     ("元/(kWh·日)", "元/(kWh·日)", "元/(kW·日)", "元/(kW·日)")):
114     selected = [r for r in a["bottlenecks"] if r["scenario"] == "main_l100_pv100" and r["
    resource"] == resource]
115     vals = [r["value_per_unit_day"] for r in selected]
116     bars = ax.bar([str(int(r["increment"])) for r in selected], vals, color="#0072B2")
117     ax.bar_label(bars, labels=[f"{v:.6f}" for v in vals], padding=4, fontsize=9)
118     ymax = max(max(vals) * 1.4, 0.01)
119     ax.set(title=f"{title}\n基准活跃 {selected[0]['active_count']} 个时段", ylabel=unit,
120           xlabel="增加量 / " + ("kWh" if resource.startswith("storage_") else "kW"), ylim=(-ymax
    * .04, ymax))
121 fig.suptitle("分别放宽单一边界后重求 MILP 有限增量日价值", fontsize=16)
122 save(fig, "bottleneck_analysis")

```

src/plots/q1_paper.py

```

1  """第一问完整论文图：读取验收结果，补算设备扫描，保存全部绘图数据与脚本来源。"""
2  import html
3  import json
4  from pathlib import Path
5  import shutil
6  import sys
7  import tempfile
8
9  import numpy as np
10
11  ROOT = Path(__file__).resolve().parents[2]
12  sys.path.insert(0, str(ROOT))
13  from src.plots.common import COLORS as C, cost_palette, digest, read_csv, save, setup
14  from src.q1.device_sensitivity import prepare
15  from src.q1.model import DT, check_solution, write_csv, write_json
16
17  FIGURES = [
18      ('01_data_features', '负荷、光伏与电价的日内变化特征', '可选：背景说明', 'data_features.csv',
19       '负荷与光伏采用144阶段梯线：浅绿区域仅表示光伏超过负荷的部分。电价独立成面板，不使用双纵轴或预测置信区间。'),
20      ('02_optimal_dispatch', '最优购电计划与储能运行轨迹', '必选：核心结果', 'dispatch_power.csv;storage_145
    .csv',
21       '无储能与最优购电功率共享纵轴；充电向上、放电向下，限值为±5000 kW；145个储电量点含日初状态。状态条依据实际C、D及
    1e-6 kWh容差判定。'),
22      ('03_cost_waterfall', '逐步放宽调度限制的购电费用变化', '建议：结果解释', 'cost_waterfall.csv;
    baseline_metrics.csv',
23       '中间基准为仅吸收富余光伏的储能方案；两项节约依赖这一嵌套顺序，不能解释为唯一的光伏收益与套利收益分解。'),

```

```

24 ('04_device_sensitivity', '设备边界与最优购电费用', '可选: 瓶颈分析', 'device_one_dimensional.csv',
25     '每条曲线只改变横轴对应边界, 其余设备参数保持基准。圆点全部为独立MILP求解值; 连接线仅帮助阅读。'),
26 ('05_joint_contour', '储电上限与共同功率上限的联合敏感性', '可选: 联合敏感性', 'device_joint_grid.csv',
27     '110个网格点分别求解MILP, 二维纵轴同时改变充、放电功率。等值线在网格之间作绘图插值; 储电上限放宽实验不等同于实际
    扩容投资分析。'),
28 ('06_efficiency', '储能效率变化下的费用与运行轨迹', '可选: 效率稳健性', 'efficiency_metrics.csv;
    efficiency_storage.csv',
29     '四种效率情景独立求解; 往返0.90采用两侧效率均为sqrt(0.90), 不得与单程0.90混用。'),
30 ('07_input_sensitivity', '负荷与光伏输入扰动下的购电费用', '可选: 输入稳健性', 'input_sensitivity.csv',
31     '固定两侧效率0.90, 只展示已计算的九组负荷、光伏-5%、0、+5%扰动。每格标注费用, 不冒充设备二维敏感性。'),
32 ('08_fixed_mode_diagnostic', '固定模式边际价格的局部诊断', '备查: 局部诊断, 不建议放正文', '
    fixed_mode_marginals.csv;milp_rhs_checks.csv',
33     '固定整数模式的LP影子价格属于局部诊断。与原MILP双侧扰动的对照数量及不匹配位置见图, 不能据此直接生成普适充停放策
    略。'),
34 ]
35
36
37 def numbers(rows, key):
38     return np.array([float(r[key]) for r in rows])
39
40
41 def time_axis(ax):
42     ax.set(xlim=(0, 24), xticks=np.arange(0, 25, 2), xlabel='当天时刻')
43     ax.set_xticklabels([f'{h:02}:00' for h in range(0, 25, 2)])
44
45
46 def draw(data, solutions, source, scan, output):
47     plt, font = setup()
48     main = solutions['main']
49     edges = np.arange(145) * DT
50     center = (edges[:-1] + edges[1:]) / 2
51     load, pv, price = (numbers(data, k) for k in ('load_kw', 'pv_kw', 'price_yuan_per_kwh'))
52     base_rows = read_csv(source / 'baseline_comparison.csv')
53     baseline = {r['baseline']: r for r in base_rows}
54     no_ess = np.maximum(load - pv, 0)
55     g, charge, discharge = (np.asarray(main[k]) / DT for k in ('G', 'C', 'D'))
56     energy = np.r_[main['E0'], main['E']]
57     states = np.where(np.asarray(main['C']) > 1e-6, 1, np.where(np.asarray(main['D']) > 1e-6, -1, 0)
58 )
59     if len(energy) != 145 or len(g) != 144 or abs(sum(no_ess) * DT - float(baseline['no_ess']['
60 grid_kwh']))) > 1e-6:
61         raise ValueError('绘图时段或无储能基准错误')
62     write_csv(output / 'data_features.csv', [{t: i+1, 'start_h': edges[i], 'end_h': edges[i+1],
63         'load_kw': load[i], 'pv_kw': pv[i], 'surplus_pv_kw': max(pv[i]-load[i], 0),
64         'price_yuan_per_kwh': price[i]} for i in range(144)])
65     write_csv(output / 'dispatch_power.csv', [{t: i+1, 'start_h': edges[i], 'end_h': edges[i+1],
66         'no_ess_grid_kw': no_ess[i], 'optimal_grid_kw': g[i], 'charge_kw': charge[i],
67         'discharge_kw': discharge[i], 'state': {1:'充电', 0:'待机', -1:'放电'}[int(states[i])]}
68     for i in range(144)])
69     write_csv(output / 'storage_145.csv', [{'boundary': i, 'time_h': edges[i], 'energy_kwh': energy[
70 i]} for i in range(145)])
71
72     fig, axes = plt.subplots(2, 1, figsize=(11.5, 6.4), sharex=True, height_ratios=[2, 1], layout='
    constrained')
73     axes[0].stairs(load, edges, baseline=None, color=C['load'], lw=1.6, zorder=3, label='小区负荷')
74     axes[0].stairs(pv, edges, baseline=None, color=C['pv'], lw=1.6, zorder=3, label='光伏预测')
75     x = np.column_stack([edges[:-1], edges[1:]]).ravel()

```

```

72 axes[0].fill_between(x, np.repeat(load, 2), np.repeat(pv, 2), where=np.repeat(pv > load, 2),
73                     color=C['charge'], alpha=.20, linewidth=0, label='富余光伏 (光伏高于负荷)')
74 axes[0].set(ylabel='功率 / kW', title='A 负荷与光伏', ylim=(0, max(load.max(), pv.max()) * 1.20))
75 axes[0].legend(loc='upper left', ncols=3)
76 axes[1].stairs(price, edges, baseline=None, color=C['grid'], lw=1.7)
77 axes[1].set(ylabel='电价 / (元/kWh)', title='B 购电价格', ylim=(0, price.max()*1.15))
78 time_axis(axes[-1]); fig.suptitle(FIGURES[0][1], fontsize=16)
79 save(fig, output, FIGURES[0][0])
80
81 fig, axes = plt.subplots(4, 1, figsize=(11.5, 9.8), sharex=True,
82                         height_ratios=[2, 1.65, 1.75, .28], layout='constrained')
83 axes[0].stairs(no_ess, edges, baseline=None, color=C['muted'], ls='--', lw=1.4, label='无储能购电
84 功率')
85 axes[0].stairs(g, edges, baseline=None, color=C['grid'], lw=1.6, label='MILP最优购电功率')
86 axes[0].set(ylabel='购电功率 / kW', title='A 外网购电', ylim=(0, max(g.max(), no_ess.max())*1.20))
87 axes[0].legend(loc='upper left', ncols=2)
88 axes[1].bar(center, charge, width=DT*.91, color=C['charge'], label='充电 (正)')
89 axes[1].bar(center, -discharge, width=DT*.91, color=C['discharge'], label='放电 (负)')
90 for value in (-5000, 5000):
91     axes[1].axhline(value, color=C['muted'], lw=.9, ls='--')
92 axes[1].axhline(0, color=C['load'], lw=.7)
93 axes[1].set(ylabel='储能功率 / kW', title='B 母线侧充放电 • 虚线为±5000 kW', ylim=(-6600, 7800),
94             yticks=[-5000, 0, 5000])
95 axes[1].legend(loc='upper left', ncols=2)
96 axes[2].plot(edges, energy, color=C['energy'], lw=1.8, label='储电量 (145个边界点)')
97 for value in (1200, 10800):
98     axes[2].axhline(value, color=C['muted'], ls='--', lw=.9)
99 axes[2].scatter([0, 24], [6000, 6000], s=35, color=C['pv'], zorder=4, label='首尾均为6000 kWh')
100 axes[2].set(ylabel='储电量 / kWh', title='C 储电状态 • 安全下限1200 / 上限10800 kWh',
101             ylim=(0, 13500), yticks=[1200, 6000, 10800])
102 axes[2].legend(loc='upper left', ncols=2)
103 axes[3].bar(center, np.ones(144), width=DT, color=[{1:C['charge'], 0:'#E3E7EA', -1:C['discharge']}][int(s)] for s in states])
104 axes[3].set(ylim=(0, 1), yticks=[], ylabel='状态')
105 axes[3].set_ylabel('状态\n灰=待机', fontsize=8, rotation=0, ha='right', va='center')
106 axes[3].grid(False); axes[3].spines['left'].set_visible(False)
107 time_axis(axes[-1]); fig.suptitle(FIGURES[1][1], fontsize=16)
108 save(fig, output, FIGURES[1][0])
109
110 costs = [float(baseline[k]['cost_yuan']) for k in ('no_ess', 'pv_only', 'full')]
111 gains = [costs[0]-costs[1], costs[1]-costs[2]]
112 write_csv(output / 'cost_waterfall.csv', [
113     {'position': i, 'role': role, 'bottom_yuan': bottom, 'height_yuan': height}
114     for i, role, bottom, height in [(0, '无储能', 0, costs[0]), (1, '仅光伏储能增量节约', costs[1], gains
115     [0]),
116     (2, 'PV-only', 0, costs[1]), (3, '放宽充电来源增量节约', costs[2], gains[1]), (4, '完整MILP', 0, costs
117     [2])])])
118 shutil.copy2(source / 'baseline_comparison.csv', output / 'baseline_metrics.csv')
119 fig, axes = plt.subplots(2, 1, figsize=(11.5, 6.8), height_ratios=[4, 1.05], layout='constrained')
120 ax = axes[0]
121 ax.grid(axis='x', visible=False)
122 bars = ax.bar(range(5), [costs[0], gains[0], costs[1], gains[1], costs[2]],
123               bottom=[0, costs[1], 0, costs[2], 0], width=.62,
124               color=[C['muted'], C['charge'], C['pv'], C['charge'], C['grid']])
125 for i, (bar, value) in enumerate(zip(bars, [costs[0], gains[0], costs[1], gains[1], costs[2]])):
126     ax.text(bar.get_x()+bar.get_width()/2, bar.get_y()+bar.get_height()+950,

```



```

123         ('-' if i in (1,3) else '')+f'{value:,.2f}', ha='center', va='bottom', fontsize=11)
124     for i, level in enumerate([costs[0], costs[1], costs[1], costs[2]]):
125         ax.plot([i+.31, i+1-.31], [level, level], color=C['muted'], lw=.9)
126     ax.set(xticks=range(5), xticklabels=['无储能', '仅光伏储能\n增量节约', 'PV-only', '放宽充电来源\n增量节约', '完整MILP'],
127           ylabel='全天购电费 / 元', ylim=(0, 56000))
128     ax.set_title(f'总节约 {sum(gains):,.2f} 元 / 日 ({sum(gains)/costs[0]*100:.2f}%)', loc='left',
129                 pad=14)
130     axes[1].axis('off')
131     table = axes[1].table(cellText=[[label]+[f'{float(baseline[k][field]):,.2f}' for field in ('
132         grid_kwh','curtail_kwh','cost_yuan')]]
133                                for label,k in [(('无储能','no_ess'),('PV-only','pv_only'),('完整
134         MILP','full'))]],
135                                collLabels=['调度方案','购电量 / kWh','弃光量 / kWh','购电费 / 元'], cellLoc='
136         center', loc='center')
137     table.auto_set_font_size(False); table.set_fontsize(10); table.scale(1, 1.6)
138     for (row, col), cell in table.get_celld().items():
139         cell.set_edgecolor('#DCE3E8'); cell.set_facecolor('#EDF3F6' if row == 0 else 'white')
140     fig.suptitle(FIGURES[2][1], fontsize=16)
141     save(fig, output, FIGURES[2][0])
142
143     curves = read_csv(scan / 'one_dimensional.csv'); grid = read_csv(scan / 'joint_grid.csv')
144     shutil.copy2(scan / 'one_dimensional.csv', output / 'device_one_dimensional.csv')
145     shutil.copy2(scan / 'joint_grid.csv', output / 'device_joint_grid.csv')
146     fig, axes = plt.subplots(1, 3, figsize=(12, 4.6), sharey=True, layout='constrained')
147     for ax, resource, label, base, color in zip(axes, ('e_max','charge_kw','discharge_kw'),
148         ('储电上限 / kWh','充电功率上限 / kW','放电功率上限 / kW'), (10800,5000,5000),
149         (C['energy'],C['charge'],C['discharge'])):
150         rows = [r for r in curves if r['resource'] == resource]
151         ax.plot(numbers(rows,'limit'), numbers(rows,'cost_yuan'), '-o', color=color, lw=1.7, ms=4)
152         ax.axvline(base, color=C['muted'], ls='--', lw=1)
153         ax.scatter([base], [costs[2]], color='black', s=28, zorder=4)
154         saving = costs[2]-float(next(r['cost_yuan'] for r in rows if float(r['limit']) == base+100))
155         unit = 'kWh' if resource == 'e_max' else 'kW'
156         ax.set(xlabel=label, title=f'基准 {base} {unit}\n提高100 {unit}节约 {saving:.2f} 元 / 日')
157         ax.ticklabel_format(style='plain', axis='both', useOffset=False)
158         ax.set_xticks([6000,10800,18000,24000,30000] if resource == 'e_max' else
159             [0,2500,5000,7500,10000])
160         ax.tick_params(axis='x', labelrotation=30)
161     axes[0].set_ylabel('最低购电费 / 元')
162     fig.suptitle(FIGURES[3][1]+' · 分别改变单一边界', fontsize=15)
163     save(fig, output, FIGURES[3][0])
164
165     xs = sorted({float(r['e_max_kwh']) for r in grid}); ys = sorted({float(r['common_power_kw']) for
166         r in grid})
167     lookup = {(float(r['e_max_kwh']),float(r['common_power_kw'])):float(r['cost_yuan']) for r in
168         grid}
169     z = np.array([[lookup[x,y] for x in xs] for y in ys])
170     fig, ax = plt.subplots(figsize=(11.5, 6.5), layout='constrained')
171     fill = ax.contourf(xs, ys, z, levels=np.linspace(26000,44000,37), cmap=cost_palette())
172     contours = ax.contour(xs, ys, z, levels=np.arange(28000,44000,2000), colors='#415D70',
173                          linewidths=.7)
174     positions = []
175     for segments in contours.allsegs:
176         vertices = max(segments, key=len)
177         inside = vertices[(vertices[:,0]>xs[0]+900)&(vertices[:,0]<xs[-1]-900)&
178             (vertices[:,1]>ys[0]+450)&(vertices[:,1]<ys[-1]-450)]

```

```

171     if len(inside): positions.append(inside[len(inside)//2])
172 labels = ax.clabel(contours, inline=True, fontsize=9, fmt='%.0f', manual=positions)
173 for label in labels:
174     label.set_color(C['energy'])
175     label.set_bbox({'facecolor': 'white', 'edgecolor': 'none', 'alpha': .88, 'pad': 1.5})
176 xx, yy = np.meshgrid(xs, ys); ax.scatter(xx, yy, color='#253D56', s=5, alpha=.28, label='实际求解网格点'
177 )
178 ax.scatter([10800], [5000], marker='*', s=180, c='white', edgecolor='#182F46', lw=1.2, zorder=5)
179 ax.annotate(f'基准 (10800, 5000)\n{costs[2]:.2f} 元', (10800, 5000), xytext=(14500, 6300),
180             arrowprops={'arrowstyle': '→', 'color': C['energy']}, color=C['energy'], fontsize=11,
181             bbox={'boxstyle': 'round, pad=.45', 'fc': 'white', 'ec': 'none', 'alpha': .94})
182 ax.set(xlabel='储电上限 / kWh', ylabel='共同充放电功率上限 / kW', xticks
183       =[6000, 10800, 18000, 24000, 30000],
184       yticks=[1000, 3000, 5000, 7000, 10000], title='110个网格点独立求解 · 线标注为等购电费用 / 元')
185 ax.grid(False); fig.colorbar(fill, ax=ax, label='最低购电费 / 元', pad=.025, ticks=range
186 (26000, 44001, 3000))
187 fig.suptitle(FIGURES[4][1], fontsize=16)
188 save(fig, output, FIGURES[4][0])
189
190 efficiency = {r['scenario']: r for r in read_csv(source / 'efficiency_comparison.csv')}
191 order = ['eta085', 'main', 'roundtrip090', 'eta095']; labels = ['单程0.85', '单程0.90 (基准)', '往返0
192 .90 (对称)', '单程0.95']
193 colors = [C['pv'], C['grid'], C['discharge'], C['charge']]
194 shutil.copy2(source / 'efficiency_comparison.csv', output / 'efficiency_metrics.csv')
195 write_csv(output / 'efficiency_storage.csv', [{ 'scenario': key, 'boundary': i, 'time_h': edges[i], '
196 energy_kwh': v }
197
198         for key in order for i, v in enumerate([solutions[key]['E0'], *solutions[key]['E']]))
199 fig, axes = plt.subplots(2, 1, figsize=(11.5, 7), layout='constrained')
200 bars = axes[0].bar(labels, [float(efficiency[k]['cost_yuan']) for k in order], color=colors, width
201 =.55)
202 axes[0].grid(axis='x', visible=False)
203 axes[0].bar_label(bars, fmt='%.2f', padding=5)
204 axes[0].set(ylabel='最低购电费 / 元', ylim=(0, 43000), title='A 每种效率情景独立求解')
205 for key, label, color, style in zip(order, labels, colors, ['--', '-', ':', '-.']):
206     axes[1].plot(edges, [solutions[key]['E0'], *solutions[key]['E']], color=color, ls=style, label=
207 label, lw=1.5)
208 for bound in [1200, 10800]: axes[1].axhline(bound, color=C['muted'], lw=.8, ls='--')
209 axes[1].set(ylabel='储电量 / kWh', ylim=(0, 14000), title='B 储电轨迹与安全边界')
210 axes[1].legend(ncols=2, loc='upper left'); time_axis(axes[1])
211 fig.suptitle(FIGURES[5][1], fontsize=16); save(fig, output, FIGURES[5][0])
212
213 scenarios = [r for r in read_csv(source / 'sensitivity_scenarios.csv') if r['efficiency_name']=='
214 main']
215 write_csv(output / 'input_sensitivity.csv', scenarios)
216 values = {(float(r['load_delta']), float(r['pv_delta'])): float(r['cost_yuan']) for r in scenarios}
217 z = np.array([values[l, p] for p in [-.05, 0, .05] for l in [-.05, 0, .05]])
218 fig, ax = plt.subplots(figsize=(8.2, 5.6), layout='constrained')
219 im = ax.imshow(z, cmap=cost_palette(), aspect='auto')
220 for i in range(3):
221     for j in range(3): ax.text(j, i, f'{z[i, j]:.2f}', ha='center', va='center', fontsize=13, color='
222 white' if z[i, j] > (z.min() + z.max()) / 2 else '#182F46')
223 ax.set(xticks=[0, 1, 2], xticklabels=['-5%', '原输入', '+5%'], yticks=[0, 1, 2], yticklabels=['-5%', '原输入
224 ', '+5%'],
225        xlabel='光伏规模变化', ylabel='负荷规模变化', title='双单程效率均0.90 · 九个单独求解的情景')
226 ax.grid(False); fig.colorbar(im, ax=ax, label='最低购电费 / 元'); fig.suptitle(FIGURES[6][1],
227        fontsize=15)
228 save(fig, output, FIGURES[6][0])

```



```

217 marginals=[r for r in read_csv(source/'marginal_values.csv') if r['scenario']=='main_l100_pv100'
218 ]
219 if len(marginals)!=144: raise ValueError('主情景边际诊断必须为144行')
220 write_csv(output/'fixed_mode_marginals.csv',marginals)
221 shutil.copy2(source/'milp_rhs_validation.csv',output/'milp_rhs_checks.csv')
222 checks=read_csv(source/'milp_rhs_validation.csv')
223 mismatch=sum(int(r['fixed_shadow_matches_milp_both_sides'])==0 for r in checks)
224 fig,axes=plt.subplots(2,1,figsize=(11.5,6.2),sharex=True,height_ratios=[3,1],layout='constrained
225 ')
226 for key,label,color,style in [('mu_yuan_per_kwh','母线边际成本  $\mu$ ',C['grid'],'-'),
227 ('charge_threshold','充电比较值  $\eta\lambda$ ',C['charge'],'--'),('discharge_threshold','放电比较值  $\lambda/\eta$ '
228 ,C['discharge'],':')]:
229 axes[0].stairs(numbers(marginals,key),edges,baseline=None,color=color,ls=style,lw=1.5,label=
230 label)
231 axes[0].legend(loc='upper left',ncols=3); axes[0].set(ylabel='元/kWh',ylim=(-.06,1.65),title='A
232 固定最优整数模式的局部LP结果')
233 for i,kind in enumerate(['bus_demand','internal_injection']):
234 bad=[r for r in checks if r['kind']==kind and int(r['fixed_shadow_matches_milp_both_sides'])
235 ==0]
236 axes[1].scatter([(int(r['t'])-.5)*DT for r in bad],[i]*len(bad),s=32,color=C['discharge'],
237 marker='x')
238 axes[1].set(yticks=[0,1],yticklabels=['母线RHS','SOC RHS'],ylim=(-.6,1.6),
239 title=f'B 原MILP双侧扰动不匹配位置 · {len(checks)*2}次重求 / {mismatch}项不匹配')
240 time_axis(axes[1]); fig.suptitle(FIGURES[7][1]+' · 不用于普通动作选择',fontsize=15)
241 save(fig,output,FIGURES[7][0])
242 return {'font':font,'intervals':144,'energy_points':145,'rhs_mismatches':mismatch,
243 'grid_points':len(grid),'figure_count':len(FIGURES),'no_ess_cost_yuan':costs[0],
244 'optimal_cost_yuan':costs[2],'total_saving_yuan':sum(gains)}
245
246 def main():
247 generation=(ROOT/'outputs/q1_current').resolve(strict=True)
248 manifest=json.loads((generation/'raw/run_manifest.json').read_text(encoding='utf-8'))
249 if not all(manifest[k] for k in ['all_physical_csv_checks_passed','analysis_checks_passed','
250 tests_passed']):
251 raise ValueError('Q1来源尚未验收')
252 for name,value in manifest['output_sha256'].items():
253 if digest(generation/name)!=value: raise ValueError(f'来源哈希不符: {name}')
254 data=read_csv(generation/'inputs/processed/timeseries.csv')
255 for row in data:
256 for key in ['price_yuan_per_kwh','load_kwh','pv_kwh','load_kw','pv_kw']: row[key]=float(row[
257 key])
258 solutions={key:json.loads((generation/f'raw/solution_{key}.json').read_text(encoding='utf-8'))
259 for key in ['main','eta085','eta095','roundtrip090']}
260 for s in solutions.values(): check_solution(data,s)
261 scan=prepare(data,generation)
262 parent=ROOT/'outputs/processed/figures'; parent.mkdir(parents=True,exist_ok=True)
263 staging=Path(tempfile.mkdtemp(prefix='q1-pending-',dir=parent))
264 try:
265 checks=draw(data,solutions,generation/'processed',scan,staging)
266 rows=[{'figure':name,'title':title,'role':role,'script':'src/plots/q1_paper.py',
267 'data_csv':sources,'caption':caption} for name,title,role,sources,caption in FIGURES]
268 write_csv(staging/'figure_index.csv',rows)
269 ordered=sorted(FIGURES,key=lambda r: {'必选':0,'建议':1,'可选':2,'备查':3}[r[2][:2]])
270 cards=''.join(f'<article id="{name}"><h2>{html.escape(title)}</h2><p>{role} · {html.escape(
271 caption)}</p>')

```

```

263         f'<p><a href="{name}.png">PNG</a> • <a href="{name}.svg">SVG</a> • '+
264         ' • '.join(f'<a href="{p}">{p}</a>' for p in sources.split(';'))+
265         f'</p></article>' for name,
title,role,sources,caption in ordered)
266     page+=<!doctype html><html lang="zh-CN"><meta charset="UTF-8"><meta name="viewport" content
="width=device-width,initial-scale=1">'
267     page+=<title>第一问论文图集</title><style>body{font:16px/1.7 -apple-system,BlinkMacSystemFont
,"PingFang SC",sans-serif;background:#f3f6f8;color:#253d56;margin:0}main{max-width:1160px;margin
:auto;padding:32px 20px}article{background:white;padding:24px;margin:30px 0;border-radius:8px}
img{width:100%;height:auto}a{color:#0072b2}p{overflow-wrap:anywhere}h1{font-size:30px}h2{font-
size:22px}</style><main>'
268     page+=<h1>第一问论文图集</h1><p>核心运行结果为必选，费用对比建议放正文，其余按论证需要选用。共8张图：PNG
为300 dpi，SVG中文转为矢量路径。数据表全部为CSV。<a href="figure_index.csv">图目索引</a> • <a href
="../../../../src/plots/README.md">集中管理的绘图脚本</a></p>'
269     page+=<nav>'+ ' • '.join(f'<a href="#{name}">{role}: {title}</a>' for name,title,role,_,_
in ordered)+</nav>'
270     page+=<p>实验口径：Emin=1200 kWh、首尾6000 kWh、两侧效率0.9。设备扫描仅改变指明的边界；Emax为储电上
限，超过原额定容量的值仅用于数学约束放宽实验，不作设备投资结论。</p>'+cards+</main></html>'
271     (staging/'index.html').write_text(page,encoding='utf-8')
272     code=[Path(__file__),ROOT/'src/plots/common.py',ROOT/'src/q1/device_sensitivity.py',ROOT/'
src/q1/model.py']
273     write_json(staging/'provenance.json',{'source_generation':str(generation.relative_to(ROOT)),
274             'source_manifest_sha256':digest(generation/'raw/run_manifest.json'),
275             'device_analysis':str(scan.relative_to(ROOT)), 'checks':checks,
276             'code_sha256':{str(p.relative_to(ROOT)):digest(p) for p in code},
277             'output_sha256':{p.name:digest(p) for p in staging.iterdir() if p.is_file()})
278     target=parent/'q1'; old=parent/'q1-previous'
279     if old.exists(): raise ValueError('发现未完成的图集替换，需先检查.q1-previous')
280     if target.exists(): target.rename(old)
281     try: staging.rename(target)
282     except BaseException:
283         if old.exists(): old.rename(target)
284         raise
285     if old.exists(): shutil.rmtree(old)
286     except BaseException:
287         if staging.exists(): shutil.rmtree(staging)
288         raise
289     print(json.dumps({'output':str(target),'checks':checks},ensure_ascii=False),flush=True)
290
291
292 if __name__=='__main__':
293     main()

```

src/plots/q2.py

```

1  """按科研绘图技能输出可独立导出的 PNG/SVG，所有图关联数值CSV。"""
2
3  import matplotlib
4  matplotlib.use("Agg")
5  import matplotlib.dates as mdates
6  import matplotlib.pyplot as plt
7  from matplotlib import font_manager
8  import numpy as np
9  import warnings
10
11  from src.q2.data import SERIES

```

```

12
13 LABELS = {"load_kw": "小区负荷", "pv_kw": "光伏实际功率", "net_load_kw": "净负荷"}
14 COLORS = ("0072B2", "#D55E00", "#009E73", "#AA3377", "#332288", "#117733", "#882255")
15 FORMS = {"raw": "原始", "diff144": "日差分", "diff1008": "周差分", "diff144_then1008": "日后周差分"}
16
17
18 def create_figures(frame, diagnostics, output):
19     output.mkdir(parents=True, exist_ok=True)
20     fonts = {f.name for f in font_manager.fontManager.ttflist}
21     selected = next((f for f in ("PingFang SC", "Heiti TC", "Arial Unicode MS", "Noto Sans CJK SC",
22     "Songti SC") if f in fonts), None)
23     if not selected:
24         raise RuntimeError("未找到中文字体：拒绝发布乱码图表")
25     figures = []
26     def save(fig, name, caption, sources):
27         for suffix in ("png", "svg"):
28             with warnings.catch_warnings():
29                 warnings.filterwarnings("error", message="Glyph .* missing from font")
30                 fig.savefig(output / f"{name}.{suffix}", dpi=220, facecolor="white")
31             plt.close(fig)
32             figures.append({"name": name, "caption": caption, "source_csv": sources, "font": selected,
33             "png_dpi": 220, "svg_text": "paths_for_portable_chinese", "scope": "
34             full_year_retrospective"})
35     with plt.rc_context({"font.family": selected, "font.size": 10, "axes.titlesize": 12,
36     "axes.unicode_minus": False, "axes.spines.top": False, "axes.spines.right":
37     False,
38     "axes.grid": True, "grid.alpha": .18, "svg.fonttype": "path"}):
39         for days in (7, 30):
40             part = frame.iloc[:days * 144]
41             fig, axes = plt.subplots(3, 1, figsize=(13, 8.5), sharex=True, layout="constrained")
42             for i, (ax, variable) in enumerate(zip(axes, SERIES)):
43                 ax.plot(part.timestamp, part[variable], lw=.8, color=COLORS[i])
44                 ax.set(ylabel="功率 / kW", title=f"chr(65+i) {LABELS[variable]}")
45                 ax.axhline(0, color="#777777", lw=.6)
46                 axes[-1].xaxis.set_major_formatter(mdates.DateFormatter("%m-%d"))
47                 axes[-1].set_xlabel("区间右端点（日期）")
48                 fig.suptitle(f"连续 {days} 天原始功率 · 从附件首日开始，无平滑", fontsize=15)
49                 save(fig, f"timeseries_{days}d", f"1月1日起连续{days}天负荷、光伏与净负荷：净负荷保留负值。", ["
50                 timeseries.csv"])
51
52             fig, axes = plt.subplots(1, 2, figsize=(13, 8), layout="constrained")
53             ticks = frame.drop_duplicates("month")["date"]
54             for ax, variable in zip(axes, SERIES[:2]):
55                 matrix = frame.pivot(index="date", columns="slot", values=variable)
56                 im = ax.imshow(matrix, aspect="auto", interpolation="nearest", cmap="cividis", origin="
57                 upper",
58                 extent=(0, 24, len(matrix), 0), vmin=0)
59                 ax.set(title=LABELS[variable], xlabel="区间内时刻 / h", ylabel="源日期（2025年）",
60                 xticks=np.arange(0, 25, 4), yticks=[(d-frame.date.iloc[0]).days+.5 for d in ticks
61                 ],
62                 yticklabels=[d.strftime("%m-01") for d in ticks])
63             ax.grid(False)
64             fig.colorbar(im, ax=ax, label="功率 / kW（各面板独立色标）", shrink=.8)
65             fig.suptitle("全年日期—时间热力图 · 每格为原始10分钟观测", fontsize=15)
66             save(fig, "annual_heatmaps", "日期纵轴、日内时间横轴，原始365×144值，无插值；各变量使用独立且标明的色
67             标。", ["timeseries.csv"])

```

```

62     profiles = diagnostics["profiles"]
63     for grouping, variables, name in (("month", SERIES, "monthly_profiles"),
64                                     ("weekday", SERIES[:2], "weekday_profiles"),
65                                     ("day_type", SERIES[:2], "weekday_weekend")):
66         fig, axes = plt.subplots(len(variables), 1, figsize=(13, 3.8*len(variables)), layout="
constrained")
67         for ax, variable in zip(axes, variables):
68             view = profiles.loc[(profiles.grouping == grouping) & (profiles.series == variable)]
69             for i, (group, part) in enumerate(view.groupby("group", sort=True)):
70                 label = (f"{int(group)}月" if grouping == "month" else "星期"+"一二三四五六日"[int(
group)-1]
71                 if grouping == "weekday" else "周一至周五" if group == "weekday" else "周
六至周日")
72                 ax.plot(part.slot/6, part.mean_kw, label=label, color=COLORS[i % 7],
73                         linestyle=("-", "--", "-.", ":")[i // 3 % 4], lw=1.3)
74                 ax.set(title=LABELS[variable], ylabel="平均功率 / kW", xlabel="区间右端点 / h",
75                        xticks=np.arange(0, 25, 2), xlim=(0, 24))
76                 ax.legend(ncols=2 if grouping == "month" else 1, fontsize=9, loc="upper left",
bbox_to_anchor=(1.01, 1))
77                 fig.suptitle({"month": "12个月平均日曲线", "weekday": "星期平均日曲线 · 光伏星期效应仅据数据观察
",
78                             "day_type": "周一至周五与周末平均日曲线 · 不包含法定节假日推断"}[grouping],
79                             fontsize=15)
79                 save(fig, name, "同类源日期同槽位算术平均; 原始功率未改值或填补, 分组样本量与标准差见CSV.", ["
full_year_profiles.csv"])
80
81     curves = diagnostics["acf_pacf"]
82     fig, axes = plt.subplots(4, 3, figsize=(14, 12), layout="constrained")
83     for row, (variable, statistic) in enumerate((v, s) for v in SERIES[:2] for s in ("acf", "
pacf")):
84         view = curves.loc[curves.series == variable]
85         for ax, maximum in zip(axes[row], (60, 432, 2016)):
86             part = view.loc[(view.lag > 0) & (view.lag <= maximum)]
87             ax.vlines(part.lag, 0, part[statistic], color=COLORS[row//2], lw=.7)
88             ax.axhline(0, color="#555555", lw=.6)
89             for lag in (144, 288, 432, 1008, 2016):
90                 if lag <= maximum:
91                     ax.axvline(lag, color="#AA3377", lw=.6, ls="--")
92                     ax.set(title=f"{LABELS[variable]} · {statistic.upper()} · 1-{maximum}",
93                           xlabel="滞后 / 10 min", ylabel="相关系数", xlim=(0, maximum+1))
94                     ax.set_xticks({60: [0, 12, 24, 36, 48, 60], 432: [0, 144, 288, 432],
95                                2016: [0, 288, 576, 1008, 1440, 2016]}[maximum])
96             fig.suptitle("自相关与Burg偏自相关 · 三种滞后尺度, 未把白噪声界限当季节显著性", fontsize=14)
97             save(fig, "acf_pacf", "计算全部1至2016阶; 短滞后、3日与14日视图展示衰减及日/周结构.", ["
full_year_acf_pacf.csv"])
98
99     spectra = diagnostics["spectrum"]
100    fig, axes = plt.subplots(2, 2, figsize=(13, 8), layout="constrained")
101    for row, variable in enumerate(SERIES[:2]):
102        for col, form in enumerate(("raw", "daily_profile_residual")):
103            part = spectra.loc[(spectra.series == variable) & (spectra.form == form)]
104            positive = part.psd_kw2_per_cycle_per_day > 0
105            ax = axes[row, col]
106            ax.loglog(part.loc[positive, "frequency_per_day"], part.loc[positive, "
psd_kw2_per_cycle_per_day"],
107                    color=COLORS[row], lw=.8)
108            for frequency, label in ((1, "1日"), (1/7, "7日")):

```

```

109         ax.axvline(frequency, color="#AA3377", ls="--", lw=.8, label=label)
110         ax.text(frequency, .97 if frequency == 1 else .87, label, transform=ax.
get_axis_transform(), va="top")
111         ax.set(title=f"{LABELS[variable]} · {'原始去均值' if col == 0 else '去除平均日曲线'}",
112               xlabel="频率 / (周期/日) (log)", ylabel="PSD / [kW²/(周期/日)] (log)")
113         fig.suptitle("频谱验证 · 全部正频率, 无补零或平滑; 目标频点数值另附CSV", fontsize=14)
114         save(fig, "spectra", "FFT周期图: 两轴对数, 仅正PSD可显示, 零频不表示周期; 完整频点与局部峰见CSV.", ["
full_year_spectrum.csv", "full_year_spectral_targets.csv"])
115
116         stability = diagnostics["monthly_stability"]
117         fig, axes = plt.subplots(2, 2, figsize=(13, 8), layout="constrained")
118         for i, variable in enumerate(SERIES[:2]):
119             part = stability.loc[stability.series == variable]
120             for lag, style in (("daily_lag_r", "-"), ("weekly_lag_r", "--")):
121                 axes[0, 0].plot(part.month, part[lag], color=COLORS[i], ls=style, marker="o" if i ==
0 else "s",
122                               label=f"{LABELS[variable]} · {'1日' if lag == 'daily_lag_r' else '7
日'}")
123             for ax, metric in ((axes[0, 1], "mean_daily_amplitude_kw"), (axes[1, 0], "std_kw"), (
axes[1, 1], "mean_kw")):
124                 ax.plot(part.month, part[metric], color=COLORS[i], marker="o" if i == 0 else "s",
label=LABELS[variable])
125             for ax, title, ylabel in zip(axes.flat, ("当日/周滞后相关", "平均日峰谷差", "月内波动", "月平均水
平"),
126                                             ("Pearson r", "功率差 / kW", "样本标准差 / kW", "平均功率 / kW")):
127                 ax.set(title=title, xlabel="源日期月份", ylabel=ylabel, xticks=np.arange(1, 13))
128                 ax.legend(fontsize=8)
129                 axes[0, 0].set_ylim(-1, 1)
130                 fig.suptitle("周期稳定性 · 滞后配对两端均在同月, 不以全年均值代替局部证据", fontsize=14)
131                 save(fig, "monthly_stability", "按月比较日/周相关、日峰谷差、标准差及均值; 没有预设稳定性阈值.", ["
full_year_monthly_stability.csv"])
132
133         moments = diagnostics["transformed_monthly_moments"]
134         fig, axes = plt.subplots(4, 2, figsize=(13, 12), layout="constrained")
135         for row, form in enumerate(FORMS):
136             for i, variable in enumerate(SERIES[:2]):
137                 view = moments.loc[(moments.form == form) & (moments.series == variable)]
138                 for ax, metric in zip(axes[row], ("mean_kw", "std_kw")):
139                     ax.plot(view.month, view[metric], color=COLORS[i], marker="o" if i == 0 else "s"
, label=LABELS[variable])
140                     ax.set(title=f"{FORMS[form]} · {'均值' if metric == 'mean_kw' else '样本标准差'}",
141                           xlabel="源日期月份", ylabel="功率 / kW", xticks=np.arange(1, 13))
142                     ax.axhline(0, color="#888888", lw=.6)
143             handles, labels = axes[0, 0].get_legend_handles_labels()
144             fig.legend(handles, labels, loc="outside lower center", ncols=2, fontsize=10)
145             fig.suptitle("季节处理后的月度均值与波动 · 联合差分仅作诊断, 可能过度差分", fontsize=14)
146             save(fig, "seasonal_moments", "原始、日差分、周差分与日后周差分均使用单侧历史运算; 均值稳定不能替代方差
稳定.", ["full_year_transformed_monthly_moments.csv", "full_year_stationarity_tests.csv"])
147
148         daily = diagnostics["daily_statistics"]
149         fig, axes = plt.subplots(2, 2, figsize=(13, 8), layout="constrained")
150         for row, variable in enumerate(SERIES[:2]):
151             part = daily.loc[daily.series == variable]
152             axes[row, 0].plot(part.date, part.max_kw, color=COLORS[row], lw=.7)
153             axes[row, 1].scatter(part.date, part.peak_first_slot/6, color=COLORS[row], s=5)
154             for ax, title, ylabel in zip(axes[row], ("每日峰值", "每日峰值首次出现时刻"), ("功率 / kW", "
右端点 / h")):

```

```

155         ax.set(title=f"{LABELS[variable]} · {title}", ylabel=ylabel, xlabel="日期")
156         ax.xaxis.set_major_formatter(mdates.DateFormatter("%m月"))
157         axes[row, 1].set(ylim=(-.5, 24.5), yticks=np.arange(0, 25, 4))
158         fig.suptitle("峰值水平与峰值时段迁移 · 并列峰值数量、连续低光伏见每日统计", fontsize=14)
159         save(fig, "daily_peaks", "每个日期原始峰值及其首次右端点：并列峰值保留数量，不把全零日人为改值。", ["
full_year_daily_statistics.csv"])
160     return figures

```

src/plots/q2_dispatch.py

```

1  """第二问最终方案检验图：只读取已产生的CSV/JSON，未完成指标不画成0。"""
2
3  from pathlib import Path
4  import json
5
6  import numpy as np
7  import pandas as pd
8
9  from common import COLORS, setup, save
10
11
12  def create(run_path):
13      run_path = Path(run_path)
14      output = run_path/"processed/figures"
15      output.mkdir(parents=True, exist_ok=True)
16      plt, font = setup()
17      generated = []
18      metrics_path = run_path/"processed/experiments/forecast_metrics.csv"
19      if metrics_path.exists():
20          metrics = pd.read_csv(metrics_path)
21          fig, axes = plt.subplots(1, 2, figsize=(11, 4.5), layout="constrained")
22          labels = {"load_week": "负荷周季节基线", "load_lgb_1": "负荷LightGBM", "load_lgb_0": "负荷
LightGBM", "pv_mean7": "光伏7日均值", "pv_dhr": "光伏谐波ARMA"}
23          for ax, prefix, color in ((axes[0], "load_", COLORS["grid"]), (axes[1], "pv_", COLORS["pv"]
)):
24              for i, name in enumerate(v for v in labels if v.startswith(prefix)):
25                  group = metrics[metrics.model == name].sort_values("horizon")
26                  if group.empty:
27                      continue
28                  ax.plot(group.horizon+1, group.mae_kw, marker="o", ls="--" if name.endswith(("week",
"mean7")) else "-",
29                          label=labels[name], color=color if name.endswith(("week", "mean7")) else
COLORS["discharge"])
30                  ax.set(xticks=[1, 2, 3], xlabel="前瞻日", ylabel="MAE / kW", title="负荷候选" if prefix ==
"load_" else "光伏候选")
31                  ax.legend(frameon=False)
32                  fig.suptitle("不同预测步长的样本外误差 · 2—12月事后评价", fontsize=14)
33                  save(fig, output, "forecast_horizons"); generated.append("forecast_horizons")
34      variance_path = run_path/"processed/seasonal_difference_variance.csv"
35      if variance_path.exists():
36          frame = pd.read_csv(variance_path)
37          fig, ax = plt.subplots(figsize=(8, 4.5), layout="constrained")
38          names = {"load_kw": "负荷", "pv_kw": "光伏", "net_load_kw": "净负荷"}
39          for i, (lag, color, label) in enumerate(((144, COLORS["grid"], "1日差分"), (1008, COLORS["
charge"], "7日差分"))):
40              part = frame[frame.lag == lag].set_index("series").loc[list(names)]

```

```

41     bars = ax.bar(np.arange(3)+(i-.5)*.34, part.variance_ratio, .32, label=label, color=
color)
42     ax.bar_label(bars, fmt="%.3f", padding=3, fontsize=9)
43     ax.axhline(1, color=COLORS["muted"], lw=1, ls="--")
44     ax.set(xticks=np.arange(3), xticklabels=list(names.values()), ylabel="差分方差 / 同区间原始方差"
,
45           title="季节差分对波动的压缩程度 • 事后诊断")
46     ax.legend(frameon=False)
47     save(fig, output, "seasonal_variance"); generated.append("seasonal_variance")
48     residual_files = [run_path/f"raw/shadows/dhr/{date}_correlations.csv" for date in ("2025-03-20",
"2025-06-21", "2025-09-23", "2025-12-21")]
49     if all(path.exists() for path in residual_files):
50         fig, axes = plt.subplots(4, 2, figsize=(12, 10), layout="constrained")
51         for row, path in enumerate(residual_files):
52             frame = pd.read_csv(path)
53             for col, key in enumerate(("acf", "pacf")):
54                 ax = axes[row, col]
55                 ax.plot(frame.lag[1:], frame[key][1:], color=COLORS["pv"] if col == 0 else COLORS["
grid"], lw=1)
56                 ax.axhline(0, color=COLORS["muted"], lw=.7)
57                 ax.set(title=f"{path.stem[:10]} • {key.upper()}", xticks=[1, 36, 72, 108, 144],
ylabel="相关系数")
58                 if row == 3:
59                     ax.set_xlabel("滞后 / 10 min")
60             fig.suptitle("光伏谐波回归残差诊断 • 各预测时点仅用已实现历史", fontsize=14)
61             save(fig, output, "dhr_residuals"); generated.append("dhr_residuals")
62     daily_path = run_path/"processed/main/daily.csv"
63     if daily_path.exists():
64         daily = pd.read_csv(daily_path, parse_dates=["date"])
65         if not daily.empty:
66             fig, axes = plt.subplots(2, 1, figsize=(11, 7), layout="constrained", sharex=True)
67             axes[0].plot(daily.date, daily.cost, color=COLORS["grid"])
68             axes[0].set(ylabel="真实费用 / 元", title=f"已完成的主模型回放: {len(daily)}日")
69             axes[1].plot(daily.date, daily.final_soc, color=COLORS["energy"])
70             axes[1].axhline(1200, color=COLORS["muted"], ls="--", lw=.7)
71             axes[1].axhline(10800, color=COLORS["muted"], ls="--", lw=.7)
72             axes[1].set(ylabel="日末SOC / kWh", xlabel="源日期")
73             save(fig, output, "real_rollout"); generated.append("real_rollout")
74     (output/"manifest.json").write_text(json.dumps({"font": font, "figures": generated,
"formats": ["png", "svg"], "
75           missing_metrics_not_zero": True},
76           ensure_ascii=False, indent=2), encoding="utf-8")
77     return generated
78
79
80 if __name__ == "__main__":
81     import argparse
82     parser = argparse.ArgumentParser(description=__doc__)
83     parser.add_argument("--run-dir", type=Path, required=True)
84     print(create(parser.parse_args()).run_dir)

```

src/plots/q2_paper_data.py

```

1 """Q2论文六图的数据层：保留原值、同预测起点和完整尾部。"""
2 from pathlib import Path
3 import json,hashlib

```



```

4 import numpy as np
5 import pandas as pd
6 from scipy.signal import correlate
7 from statsmodels.tsa.stattools import acf
8 ROOT=Path(__file__).resolve().parents[2]
9 RUN=ROOT/'outputs/q2/dispatch_runs/20260912_tail_reserve_final'
10 OUT=ROOT/'outputs/processed/figures/q2_paper_final_20260913'
11 DATA=OUT/'data'
12
13
14 def write(name,frame):frame.to_csv(DATA/name,index=False,encoding='utf-8',float_format='%.17g')
15 def sha(p):return hashlib.sha256(p.read_bytes()).hexdigest()
16
17
18 def prepare():
19     DATA.mkdir(parents=True,exist_ok=True)
20     source=RUN/'inputs/processed/timeseries.csv';raw=pd.read_csv(source,float_precision='round_trip'
21     )
22     load=raw.load_kw.to_numpy().reshape(365,144);pv=raw.pv_kw.to_numpy().reshape(365,144)
23     assert np.isfinite(load).all() and np.isfinite(pv).all()
24     dates=pd.date_range('2025-01-01',periods=365);hours=np.arange(1,145)/6
25     rows=[]
26     for label,mask in [('周一至周五',dates.dayofweek<5),('周六及周日',dates.dayofweek>=5)]:
27         values=load[mask];q=np.quantile(values,[.25,.75],axis=0)
28         rows.extend({'group':label,'hour':h,'mean_kw':m,'p25_kw':lo,'p75_kw':hi,'n_days':len(values)
29         } for h,m,lo,hi in zip(hours,values.mean(0),q[0],q[1]))
30     write('fig51_weekday_profiles.csv',pd.DataFrame(rows))
31     rows=[];mu=np.zeros_like(pv)
32     for month in range(1,13):
33         mask=dates.month==month;profile=pv[mask].mean(0);mu[mask]=profile
34         if month in (1,4,7,10):rows.extend({'month':month,'hour':h,'mean_pv_kw':value,'n_days':int(
35         mask.sum())} for h,value in zip(hours,profile))
36     write('fig51_seasonal_pv.csv',pd.DataFrame(rows))
37     # Pearson lag-correlation uses the paired subset's own mean, rather than biased ACF.
38     y=load.ravel()-load.mean();n=len(y);lags=np.arange(1009);pairs=n-lags
39     products=correlate(y,y,mode='full',method='fft')[n-1:n+1008]
40     sums=np.r_[0,np.cumsum(y)];squares=np.r_[0,np.cumsum(y*y)]
41     sa=sums[n]-sums[lags];sb=sums[n-lags]
42     covariance=products-sa*sb/pairs
43     variance_a=squares[n]-squares[lags]-sa*sa/pairs;variance_b=squares[n-lags]-sb*sb/pairs
44     rho=covariance/np.sqrt(variance_a*variance_b)
45     for lag in (1,144,288,1008):np.testing.assert_allclose(rho[lag],np.corrcoef(load.ravel()[lag:],
46     load.ravel()[::-lag])[0,1],atol=1e-10,rtol=0)
47     write('fig51_load_lag_correlation.csv',pd.DataFrame({'lag':lags,'rho':rho,'paired_samples':pairs
48     })))
49     residual=(pv-mu).ravel();auto,interval=acf(residual,nlags=288,fft=True,alpha=.05,
50     bartlett_confint=True)
51     write('fig51_pv_residual_acf.csv',pd.DataFrame({'lag':np.arange(289),'acf':auto,'reference_lower
52     ':interval[:,0]-auto,'reference_upper':interval[:,1]-auto}))
53     write('fig51_pv_deseasonalized.csv',pd.DataFrame({'date':np.repeat(dates.strftime('%Y-%m-%d')
54     ,144),'hour':np.tile(hours,365),'pv_kw':pv.ravel(),'month_slot_mean_kw':mu.ravel(),'residual_kw'
55     :residual}))
56     calibration_path=RUN/'processed/main/calibration.csv';cal=pd.read_csv(calibration_path,
57     float_precision='round_trip')
58     origins=sorted(origin for origin,g in cal.groupby('origin') if set(g.horizon)=={0,1,2})
59     paired=cal[cal.origin.isin(origins)].copy()
60     coverage=paired.groupby(['horizon','nominal'],as_index=False)[['covered_intervals','n']].sum();

```

```

coverage['picp']=coverage.covered_intervals/coverage.n
51 write('fig52_paired_coverage.csv',coverage)
52 all_coverage=cal.groupby(['horizon','nominal'],as_index=False)[['covered_intervals','n']].sum();
all_coverage['picp']=all_coverage.covered_intervals/all_coverage.n
53 write('fig52_all_available_coverage.csv',all_coverage)
54 shadow=RUN/'raw/shadows';shape=json.loads((shadow/'shape.json').read_text());store={};parts=[];
audits=[]
55 for origin in origins:
56     day=(pd.Timestamp(origin)-dates[0]).days;assert day+2<365
57     path=RUN/'processed/main/daily_audit'/f'{origin}.json';audit=json.loads(path.read_text());
assert audit['selected_K']==3
58     pair=audit['3']['selection']['pipeline'];audits.append(path)
59     for name in pair:
60         if name not in store:store[name]=np.memmap(shadow/f'{name}.dat',mode='r',dtype=shape['
dtype'],shape=tuple(shape['shape']))
61         for h in range(3):
62             forecast=(store[pair[0]][day,h]-store[pair[1]][day,h])/6
63             actual=(load[day+h]-pv[day+h])/6;error=actual-forecast
64             assert np.isfinite(error).all()
65             parts.append(pd.DataFrame({'origin':origin,'horizon':h,'slot':np.arange(1,145),'
forecast_net_kwh':forecast,'actual_net_kwh':actual,'error_kwh':error}))
66 errors=pd.concat(parts,ignore_index=True);write('fig52_paired_errors.csv',errors)
67 assert len(errors)==len(origins)*3*144 and coverage.n.nunique()==1
68 write('fig52_error_quantiles.csv',errors.groupby('horizon').error_kwh.quantile
([0,.01,.05,.25,.5,.75,.95,.99,1]).rename('error_kwh').reset_index())
69 dispatch_path=RUN/'processed/main/dispatch.csv';dispatch=pd.read_csv(dispatch_path,
float_precision='round_trip')
70 candidates=['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
71 sums=dispatch[dispatch.date.isin(candidates)].groupby('date').emergency.sum().sort_values(
ascending=False,kind='stable')
72 day=sums.index[0];frame=dispatch[dispatch.date==day].copy();assert len(frame)==144
73 frame['hour_right']=np.arange(1,145)/6;frame['hour_center']=(np.arange(144)+.5)/6
74 frame['Z_real_kwh']=np.maximum(frame.net_kwh,0);frame['Z_forecast_kwh']=np.maximum((frame.
predicted_load_kw-frame.predicted_pv_kw)/6,0)
75 write('fig54_real_dispatch.csv',frame)
76 write('fig54_candidate_dates.csv',sums.rename('emergency_kwh').reset_index())
77 energy=np.r_[frame.initial_soc.iloc[0],frame.soc.to_numpy()]
78 np.testing.assert_allclose(np.diff(energy),.9*frame.charge-frame.discharge/.9,atol=1e-7,rtol=0)
79 assert energy.min()>=1200-1e-5 and energy.max()<=10800+1e-5
80 write('fig54_storage_145.csv',pd.DataFrame({'hour':np.arange(145)/6,'energy_kwh':energy}))
81 np.savez(DATA/'fig51_annual_arrays.npz',load_kw=load,pv_kw=pv,net_kw=load-pv)
82 sources=[source,calibration_path,dispatch_path,shadow/'shape.json']+shadow/f'{name}.dat' for
name in store]+audits
83 audit={'formal_run':str(RUN),'paired_forecast_origins':origins,'paired_days':len(origins),'
samples_per_horizon':len(origins)*144,
84         'error_definition':'actual minus forecast, kWh per 10min; same-origin matched horizons',
85         'PV_deseasonalization':'monthly x time-of-day mean removed; retrospective diagnostics
only, never supplied to forecasts',
86         'ACF_interval':'pointwise 95% Bartlett reference interval around zero; not simultaneous
IID proof',
87         'load_lag':'paired-subset Pearson correlation, independently checked at lags1
/144/288/1008',
88         'weekday_definition':'Monday-Friday vs Saturday-Sunday, not statutory-holiday
classification',
89         'quarter_band':'25th-75th percentile across calendar days, not confidence interval',
90         'right_endpoint':'profiles begin00:10; axis00:00 is boundary, no invented sample',
91         'pressure_date':day,'pressure_emergency_kwh':float(sums.iloc[0]),'Z_definition':'positive

```

```

    part of (load-PV)/6, before storage',
92     'source_sha256':{str(p.relative_to(ROOT)):sha(p) for p in sources}}
93 (DATA/'provenance.json').write_text(json.dumps(audit,ensure_ascii=False,indent=2),encoding='utf
    -8')
94 print({'paired_days':len(origins),'samples_per_horizon':len(origins)*144,'pressure_date':day})
95 print(errors.groupby('horizon').error_kwh.quantile([0,.25,.5,.75,1]).to_string())
96 return audit
97
98
99 if __name__=='__main__':prepare()

```

src/plots/q2_paper_final.py

```

1  """用户图5-1..5-5及全年热图：科研技能风格、真实数据、PNG/SVG。"""
2  from pathlib import Path
3  import json,hashlib,shutil,warnings
4  import numpy as np
5  import pandas as pd
6  from .common import setup
7  from .q2_paper_data import ROOT,OUT,DATA
8
9  BLUE='#1263A0';NAVY='#103B62';TEAL='#3988C1';ORANGE='#4F97C8';PURPLE='#285785';RED='#B76667';GRAY='
    #62778A';LIGHT='#DDEAF3'
10 plt,Font=setup()
11 STYLE={'font.size':11,'axes.titlesize':11.5,'axes.titleweight':'medium','axes.labelsize':10.5,'xtick
    .labelsize':9.5,'ytick.labelsize':9.5,
12     'legend.fontsize':9,'legend.frameon':False,'axes.edgecolor':'#9CA9B5','axes.linewidth':.7,'
    axes.labelcolor':NAVY,'text.color':NAVY,
13     'xtick.color':GRAY,'ytick.color':GRAY,'grid.color':LIGHT,'grid.alpha':.8,'grid.linewidth'
    :.55,'lines.linewidth':1.65,
14     'mathtext.fontset':'dejavusans','svg.fonttype':'path','axes.spines.top':False,'axes.spines.
    right':False,
15     'figure.constrained_layout.h_pad':.065,'figure.constrained_layout.w_pad':.055,'savefig.
    facecolor':'white'}
16 MANIFEST=[]
17
18
19 def read(name):return pd.read_csv(DATA/name,float_precision='round_trip')
20 def title(ax,text):ax.set_title(text,loc='left',pad=10)
21 def clean(ax):ax.grid(True,axis='y');ax.grid(axis='x',visible=False);ax.tick_params(length=3,width
    =.6)
22 def time_axis(ax):ax.set_xlim(0,24);ax.set_xticks(np.arange(0,25,4));ax.set_xlabel('时刻 / h')
23
24
25 def save(fig,name,title_text,caption,data):
26     fig.canvas.draw()
27     with warnings.catch_warnings():
28         warnings.filterwarnings('error',message='Glyph .* missing from font')
29         for suffix in ('png','svg'):fig.savefig(OUT/f'{name}.{suffix}',dpi=400,facecolor='white',
    transparent=False)
30     MANIFEST.append({'name':name,'title':title_text,'caption':caption,'data':data,'png':name+'.png',
    'svg':name+'.svg',
31         'width_inches':float(fig.get_figwidth()),'height_inches':float(fig.
    get_figheight()),'dpi':400,'font':Font,'svg_text':'glyph paths',
32         'script':'src/plots/q2_paper_final.py'})
33     plt.close(fig)

```

```

34
35
36 def structure():
37     fig, axes = plt.subplots(2, 2, figsize=(8.8, 6.5), layout='constrained')
38     profiles = read('fig51_weekday_profiles.csv'); ax = axes[0, 0]
39     for (name, color, style) in [('周一至周五', BLUE, '-'), ('周六及周日', ORANGE, '--')]:
40         f = profiles[profiles.group == name]; x = f.hour.to_numpy()
41         ax.fill_between(x, f.p25_kw.to_numpy(), f.p75_kw.to_numpy(), color=color, alpha=.11, lw=0)
42         ax.plot(x, f.mean_kw, color=color, ls=style, label=name)
43     ax.set_ylabel('平均负荷功率 / kW'); time_axis(ax); title(ax, '(a) 工作日与周末负荷'); ax.legend(loc='
upper left', ncol=1)
44     ax.text(.98, .04, '阴影: 25% - 75%分位范围', transform=ax.transAxes, ha='right', va='bottom', fontsize=8,
color=GRAY)
45     seasonal = read('fig51_seasonal_pv.csv'); ax = axes[0, 1]
46     for month, color, style, label in [(1, NAVY, '--', '1月 · 冬'), (4, PURPLE, '-.', '4月 · 春'), (7, BLUE, '-',
'7月 · 夏'), (10, TEAL, ':', '10月 · 秋')]:
47         f = seasonal[seasonal.month == month]; ax.plot(f.hour, f.mean_pv_kw, color=color, ls=style, label=
label, lw=1.9)
48     ax.set_ylabel('平均光伏功率 / kW'); ax.set_ylim(bottom=0); time_axis(ax); title(ax, '(b) 代表月份光伏日曲
线'); ax.legend(loc='upper left', ncol=1, columnspacing=1, handlelength=2)
49     f = read('fig51_load_lag_correlation.csv'); ax = axes[1, 0]
50     ax.plot(f.lag, f.rho, color=BLUE, lw=1.35)
51     for lag, label in [(144, '144 = 1日'), (288, '288 = 2日'), (1008, '1008 = 7日')]:
52         ax.axvline(lag, color=GRAY, ls='--', lw=.8, alpha=.6)
53         ax.plot(lag, float(f.loc[f.lag == lag, 'rho'].iloc[0]), 'o', color=BLUE, ms=4)
54         ax.text(lag+9, .04, label, transform=ax.get_xaxis_transform(), rotation=90, va='bottom', fontsize
=8, color=GRAY)
55     ax.set_xlim(0, 1040); ax.set_ylim(float(f.rho.min())-.055, 1.05); ax.set_xticks
([0, 144, 288, 504, 720, 1008]); ax.set_xlabel('滞后时段 k (每段10分钟)'); ax.set_ylabel(r'$\rho_L(k)$');
title(ax, '(c) 负荷滞后相关结构')
56     f = read('fig51_pv_residual_acf.csv'); ax = axes[1, 1]
57     ax.fill_between(f.lag.to_numpy()[1:], f.reference_lower.to_numpy()[1:], f.reference_upper.to_numpy
()[1:], color=GRAY, alpha=.19, lw=0, label='95% Bartlett参考带')
58     ax.plot(f.lag, f.acf, color=BLUE, lw=1.4, label='去周期光伏残差')
59     ax.axhline(0, color=GRAY, lw=.65); ax.set_xlim(0, 288); ax.set_xticks([0, 48, 96, 144, 192, 240, 288]); ax.
set_ylim(min(float(f.acf.min()), float(f.reference_lower.min()))-.04, 1.05)
60     ax.set_xlabel('滞后时段 k (每段10分钟)'); ax.set_ylabel('残差自相关'); title(ax, '(d) 去除日周期后的光伏残
差'); ax.legend(loc='upper right', fontsize=8)
61     for ax in axes.ravel(): clean(ax)
62     save(fig, 'fig5_1_source_load_structure', '图5-1 源荷结构与预测特征依据组合图',
63         '全年2025年数据的事后结构展示。工作日指周一至周五；分位带是跨日25% - 75%范围。光伏以每月×时刻均值去周期，未
将全样本统计用于滚动预测。负荷为配对Pearson相关；残差带为逐点Bartlett参考带。',
64         ['fig51_weekday_profiles.csv', 'fig51_seasonal_pv.csv', 'fig51_load_lag_correlation.csv', '
fig51_pv_residual_acf.csv', 'fig51_pv_deseasonalized.csv'])
65
66
67 def forecast():
68     fig, axes = plt.subplots(1, 2, figsize=(8.8, 4.25), layout='constrained'); f = read('fig52_paired_errors.
csv'); colors = [ORANGE, BLUE, NAVY]
69     values = [f[f.horizon == h].error_kwh.to_numpy() for h in range(3)]
70     ax = axes[0]; bp = ax.boxplot(values, positions=[0, 1, 2], widths=.46, patch_artist=True, showfliers=True,
71         medianprops={'color': NAVY, 'lw': 1.6}, whiskerprops={'color': GRAY, 'lw': 1}, capprops={'color'
: GRAY, 'lw': 1},
72         flierprops={'marker': '.', 'markersize': 2, 'markeredgewidth': 0, 'markerfacecolor': GRAY, '
alpha': .14})
73     for patch, color in zip(bp['boxes'], colors): patch.set(facecolor=color, alpha=.50, edgecolor=color,
lw=1.3)

```

```

74 ax.axhline(0,color=GRAY,ls='--',lw=.85,alpha=.85)
75 ax.set_xticks([0,1,2],[f'h=0\n第1日前瞻',f'h=1\n第2日前瞻',f'h=2\n第3日前瞻']);ax.set_ylim(-425,600)
76 ax.set_ylabel('净负荷误差 / (kWh/10 min)');title(ax,'(a) 同源预测误差分布')
77 for h,v in enumerate(values):
78     med=np.median(v);iqr=np.quantile(v,.75)-np.quantile(v,.25)
79     ax.text(h,570,f'中位 {med:.2f}\nIQR {iqr:.2f}',ha='center',va='top',fontsize=8,color=colors[h
80 ])
81 ax.set_xlabel(f'共同304日; 每组 n={len(values[0])},\n误差=真实值-预测值; 保留全部尾部',fontsize=8.5,
82 color=GRAY)
83 ax=axes[1];coverage=read('fig52_paired_coverage.csv')
84 for nominal,color,marker,style in [(.8,BLUE,'o','--'),(.9,TEAL,'s','--')]:
85     g=coverage[np.isclose(coverage.nominal,nominal)].sort_values('horizon')
86     ax.plot(g.horizon,g.picp,color=color,marker=marker,ms=6,ls=style,lw=1.6,label=f'{nominal
87 :.0%}预测区间')
88     ax.axhline(nominal,color=color,ls=':',lw=1,alpha=.55)
89     ax.text(2.13,nominal,f'名义{nominal:.0%}',ha='right',va='bottom',fontsize=8,color=color)
90     for h,v in zip(g.horizon,g.picp):ax.annotate(f'{v:.1%}',(h,v),xytext=(0,-15 if nominal==.8
91 else 8),textcoords='offset points',ha='center',fontsize=9,color=color)
92 ax.set_xlim(-.22,2.18);ax.set_ylim(.60,1.0);ax.set_yticks(np.arange(.6,1.01,.1));ax.yaxis.
93 set_major_formatter(__import__('matplotlib').ticker.PercentFormatter(1))
94 ax.set_xticks([0,1,2],[f'h=0',f'h=1',f'h=2']);ax.set_xlabel('前瞻日索引');ax.set_ylabel('经验覆盖率
95 PICP');title(ax,'(b) 场景区间覆盖率校准');ax.legend(loc='upper left',fontsize=8.5)
96 for ax in axes:clean(ax)
97 save(fig,'fig5_2_forecast_and_calibration','图5-2 不同前瞻日的误差分布与场景校准',
98 '仅使用三个前瞻日均有正式场景且目标真值完整的304个共同预测起点, 每个h为43,776时段。误差为同forecast-
99 origin真实净负荷减点预测, 单位kWh/10min。箱线须为1.5IQR, 点保留全部离群值; PICP由覆盖数/样本数计算。',
100 ['fig52_paired_errors.csv','fig52_error_quantiles.csv','fig52_paired_coverage.csv','
101 fig52_all_available_coverage.csv'])
102
103 def timeline():
104     from matplotlib.patches import FancyBboxPatch,FancyArrowPatch,Rectangle,Circle
105     fig,ax=plt.subplots(figsize=(9.6,5.6));fig.subplots_adjust(left=.025,right=.98,bottom=.035,top
106 =.965);ax.set(xlim=(0,1),ylim=(0,1));ax.axis('off')
107     def box(x,y,w,h,text,face='#FF5F8',edge=BLUE,size=10,weight='normal'):
108         patch=FancyBboxPatch((x,y),w,h,boxstyle='round,pad=0.010,rounding_size=0.012',facecolor=face
109 ,edgecolor=edge,lw=1)
110         ax.add_patch(patch);ax.text(x+w/2,y+h/2,text,ha='center',va='center',fontsize=size,color=
111 NAVY,weight=weight)
112     def arrow(a,b,color=GRAY,style='-',connection='arc3,rad=0',lw=1.15):
113         ax.add_patch(FancyArrowPatch(a,b,arrowstyle='->',mutation_scale=11,color=color,lw=lw,ls=
114 style,connectionstyle=connection))
115     ax.text(.01,.96,'日前: 已知信息 → 预测与优化',fontsize=12,weight='bold',color=NAVY)
116     box(.015,.745,.15,.13,r'$\mathcal{I}_d$'+'\n'+r'$E_{d,0}$'+ ' '+r'$p_t$',size=11)
117     ax.text(.09,.71,r'第 $d$ 日 0:00',ha='center',fontsize=9,color=GRAY)
118     box(.215,.775,.14,.07,'源荷预测');box(.405,.775,.17,.07,'联合场景构造');box(.625,.775,.14,.07,'随机
119 优化')
120     for a,b in [((.17,.81),(.205,.81)),((.36,.81),(.395,.81)),((.58,.81),(.615,.81))]:arrow(a,b)
121     box(.215,.51,.55,.18,' ',face='#E7F1F9',edge=BLUE)
122     ax.text(.49,.665,'0:00 一次确定并冻结',ha='center',fontsize=10.5,weight='bold',color=TEAL)
123     for x,text in [(.25,r'$G_t$'),(.42,r'$C_t^p$'),(.59,r'$D_t^p$')]:box(x,.58,.115,.053,text,face='
124 white',edge=TEAL,size=13)
125     ax.text(.49,.532,r'安全裕度 $R_t$ 亦由历史信息预先确定',ha='center',fontsize=8.5,color=GRAY)
126     arrow((.697,.765),(.697,.70),color=TEAL)
127     ax.text(.015,.455,'日内: 只用当前观测执行冻结反馈',fontsize=11.5,weight='bold')
128     box(.02,.315,.33,.085,r'$L_t^{\mathrm{real}}$,\;P_t^{\mathrm{PV}},\mathrm{real}$,\;E_{t-1}$',size=12,face='
129 #F4F6F8',edge=GRAY)

```

```

116 box(.415,.315,.22,.085,'冻结反馈映射',face='#E7F1F9',edge=BLUE,weight='bold')
117 box(.71,.315,.22,.085,r'$C_t,\quad D_t,\quad B_t$',face='#FFF5F8',edge=BLUE,size=13)
118 arrow((.355,.357),(.405,.357));arrow((.642,.357),(.70,.357));arrow((.54,.498),(.54,.41),color=
    TEAL,style='--')
119 ax.text(.554,.462,'冻结参数',fontsize=8,color=TEAL)
120 y=.265;arrow((.03,y),(.935,y),color=NAVY,lw=1.4)
121 for x,text in [(.07,'0:10'),(.255,'0:20'),(.46,'...'),(.67,r'$t$'),(.88,'24:00')]:
122     ax.plot([x,x],[y-.008,y+.012],color=NAVY,lw=.8);ax.text(x,y-.024,text,ha='center',va='top',
        fontsize=10)
123 arrow((.17,.275),(.17,.305),color=GRAY)
124 masked=Rectangle((.03,.045),.64,.115,facecolor='#F1F2F4',edgecolor='#A4ACB4',ls='--',lw=1,alpha
    =.80);ax.add_patch(masked)
125 ax.add_patch(Circle((.07,.105),.023,fill=False,edgecolor=RED,lw=1.2));ax.plot
    ([.054,.086],[.089,.121],color=RED,lw=1.2)
126 ax.text(.385,.11,r'未来  $L_{t+1:T}^{\mathrm{real}}$ ;  $P_{t+1:T}^{\mathrm{PV}}$  不可见',ha='
    center',va='center',fontsize=10,color=GRAY)
127 ax.text(.385,.067,'仅用于事后回放,不进入当前决策',ha='center',fontsize=8,color=GRAY)
128 box(.735,.07,.205,.105,r'$E_{d,144}^{\mathrm{real}}$'+'\n真实日未储电量',face='#FFF5F8',edge=BLUE,
    size=10)
129 arrow((.88,.211),(.84,.183),color=BLUE)
130 box(.81,.81,.165,.105,r'$(d+1)$ 日 0:00'+'\n继承真实 SOC',face='#FFF5F8',edge=BLUE,size=9.5)
131 arrow((.95,.123),(.983,.843),color=BLUE,connection='angle3,angleA=0,angleB=90')
132 arrow((.806,.89),(.088,.89),color=BLUE,style='--',connection='arc3,rad=.0');arrow((.088,.89)
    ,(.088,.884),color=BLUE)
133 ax.text(.46,.911,'滚动到下一日,重新预测与优化',ha='center',fontsize=8.5,color=BLUE)
134 save(fig,'fig5_3_causal_information_timeline','图5-3 非前瞻滚动调度的信息时序',
135     '横向时间轴区分日前冻结与日内反馈。G为计划购电,Cp/Dp为事前反馈上限,R为历史给定安全裕度;C/D/B为实际充电/
    放电/紧急购电。当前映射不访问未来真实轨迹。真实日未SOC传至下一日,不重置。',[1])
136
137
138 def dispatch():
139     f=read('fig54_real_dispatch.csv');e=read('fig54_storage_145.csv');edges=np.arange(145)/6;x=f.
        hour_center.to_numpy();dt=1/6
140     fig,axes=plt.subplots(3,1,figsize=(8.8,7.7),sharex=True,layout='constrained',gridspec_kw={'
        height_ratios':[1.1,1,1]})
141     ax=axes[0];ax.stairs(f.Z_real_kwh,edges,color=NAVY,lw=1.45,label=r'实际净外部需求  $Z_t$ ');ax.
        stairs(f.grid,edges,color=BLUE,lw=1.7,label=r'计划购电  $G_t$ ')
142     ax.stairs(f.Z_forecast_kwh,edges,color=GRAY,lw=1,ls='--',alpha=.6,label=r'预测净外部需求  $\hat{Z}_t$ ')
143     ax.bar(x,f.emergency,width=dt*.95,color=RED,alpha=.85,label=r'紧急购电  $B_t$ ',zorder=4)
144     for t,value in zip(x,f.emergency):
145         if value>le-6:ax.axvspan(t-dt/2,t+dt/2,color=RED,alpha=.065,lw=0)
146     ax.set_ylim(bottom=0);ax.set_ylabel('电量 / (kWh/10 min)');title(ax,'(a) 实际需求、计划购电与紧急补购
        ')
147     ax.legend(loc='lower left',bbox_to_anchor=(0,1.01),ncol=4,fontsize=8.0,columnspacing=1);ax.
        set_title('(a) 实际需求、计划购电与紧急补购',loc='left',pad=31);ax.text(.62,.95,f'{f.date.iloc[0]}\n
        紧急购电合计 {f.emergency.sum():.2f} kWh',transform=ax.transAxes,ha='center',va='top',fontsize=9,
        color=RED)
148     ax=axes[1];ax.bar(x,f.charge,width=dt*.95,color=TEAL,label=r'$+C_t$ 充电',lw=0);ax.bar(x,-f.
        discharge,width=dt*.95,color=PURPLE,label=r'$-D_t$ 放电',lw=0)
149     ax.axhline(0,color=GRAY,lw=.7);ax.set_ylim(-980,980);ax.set_yticks([-800,-400,0,400,800]);ax.
        set_ylabel('储能动作 / kWh');ax.set_title('(b) 冻结策略的真实充放电响应',loc='left',pad=31);ax.legend
        (loc='lower left',bbox_to_anchor=(0,1.01),ncol=2)
150     ax=axes[2];ax.axhspan(1200,10800,color=BLUE,alpha=.035);ax.plot(e.hour,e.energy_kwh,color=NAVY,
        lw=1.8)
151     for level,label in [(1200,r'$E_{\min}=1200$'),(10800,r'$E_{\max}=10800$')]:
152         ax.axhline(level,color=GRAY,ls='--',lw=.9);ax.text(23.8,level+180,label,ha='right',va='

```



```

    bottom', fontsize=9, color=GRAY)
153 ax.plot([0, 24], [e.energy_kwh.iloc[0], e.energy_kwh.iloc[-1]], 'o', color=TEAL, ms=4, zorder=5)
154 ax.annotate(f"日初 {e.energy_kwh.iloc[0]:.0f}", (0, e.energy_kwh.iloc[0]), xytext=(10, -14),
    textcoords='offset points', fontsize=8.5, color=TEAL)
155 ax.set_ylim(450, 11650); ax.set_yticks([1200, 3600, 6000, 8400, 10800]); ax.set_ylabel('储电量 / kWh');
    title(ax, '(c) 实际储电量及运行边界')
156 for ax in axes: clean(ax); ax.set_xlim(0, 24); ax.set_xticks(np.arange(0, 25, 4))
157 axes[-1].set_xlabel('时刻 / h')
158 save(fig, 'fig5_4_real_dispatch_20250320', '图5-4 压力代表日真实购电与储能运行',
159       '2025-03-20在题目四个指定日期中紧急量最高 (53.567333kWh)。Z为储能之前净负荷正部, 单位kWh/10min; 红柱为
    真实紧急量, 淡红时段仅标事件位置、不放大能量。充电为正、放电为负, SOC含日初共145点。',
160       ['fig54_real_dispatch.csv', 'fig54_storage_145.csv', 'fig54_candidate_dates.csv'])
161
162
163 def horizon():
164     source=R00T/'outputs/q2/benchmarks/horizon_figure_20260913'
165     if (DATA/'fig55_horizon_summary.csv').exists():
166         f=read('fig55_horizon_summary.csv'); cases=read('fig55_horizon_cases.csv')
167     else:
168         f=pd.read_csv(source/'summary.csv'); cases=pd.read_csv(source/'cases.csv')
169     assert len(cases)==12 and cases.reliable.all() and (f.cases==4).all()
170     if not (DATA/'fig55_horizon_summary.csv').exists():
171         shutil.copy2(source/'summary.csv', DATA/'fig55_horizon_summary.csv'); shutil.copy2(source/'
    cases.csv', DATA/'fig55_horizon_cases.csv')
172     fig, ax=plt.subplots(figsize=(7.0, 4.4), layout='constrained')
173     for (_, row), color, marker in zip(f.iterrows(), [ORANGE, BLUE, NAVY], ['o', 's', '^']):
174         x=row.mean_solve_seconds; y=row.mean_day_cost/10000
175         ax.scatter(x, y, s=95, color=color, marker=marker, edgecolor='white', linewidth=1.2, zorder=4)
176         ax.annotate(f"{int(row.horizon_hours)} h", (x, y), xytext=(10, 8), textcoords='offset points',
    fontsize=11, weight='bold', color=color)
177         ax.annotate(f"{y:.3f} 万元/日", (x, y), xytext=(10, -11), textcoords='offset points', fontsize=8.5,
    color=GRAY)
178     xmax=f.mean_solve_seconds.max(); ymin=f.mean_day_cost.min()/10000; ymax=f.mean_day_cost.max()
    /10000
179     ax.set_xlim(0, xmax*1.35+1); ax.set_ylim(ymin-.10, max(ymax+.13, ymin+.3))
180     ax.set_xlabel('平均单日求解时间 / s'); ax.set_ylabel('代表日平均真实费用 / (万元/日)')
181     title(ax, '前瞻长度的局部费用—计算代价权衡'); ax.text(.99, .96, '4个代表日 • 同机同预算\n共同场景前缀 • 非全
    年结果', transform=ax.transAxes, ha='right', va='top', fontsize=9, color=GRAY)
182     clean(ax)
183     save(fig, 'fig5_5_local_horizon_tradeoff', '图5-5 前瞻长度的局部费用与耗时权衡',
184           '经用户确认改为当前模型4个代表日局部比较, 纵轴不是全年费用。K1/2/3采用相同S26联合72h场景的前缀、同一各日日
    初SOC/R和同机3%预算, 均独立计时。日末库存不同, 较低当日费不等于更优跨日经济性。',
185           ['fig55_horizon_summary.csv', 'fig55_horizon_cases.csv'])
186
187
188 def heatmaps():
189     from matplotlib.colors import LinearSegmentedColormap, TwoSlopeNorm
190     a=np.load(DATA/'fig51_annual_arrays.npz'); dates=pd.date_range('2025-01-01', periods=365)
191     positions=[0, 59, 120, 181, 243, 304, 364]; labels=[dates[i].strftime('%m-%d') for i in positions]
192     fig, axes=plt.subplots(1, 3, figsize=(9.6, 4.9), sharey=True, layout='constrained')
193     ocean=LinearSegmentedColormap.from_list('load_seq', ['#F2F7FC', '#BBD5E9', '#659FCA', '#28699E', '#0
    A355D'])
194     sun=LinearSegmentedColormap.from_list('pv_seq', ['#F5F9FD', '#C6DFF0', '#80B6DD', '#337EB6', '#104775
    '])
195     diverging=LinearSegmentedColormap.from_list('net_div', ['#789FA8', '#B4CCD2', '#E1EDF0', '#FAFCFE', '
    #BDDCEF', '#6CA8D1', '#104775'])
196     for ax, key, cmap, text in zip(axes, ['load_kw', 'pv_kw', 'net_kw'], [ocean, sun, diverging], [(a) 小区负

```



```

197     荷', '(b) 光伏实际功率', '(c) 净负荷']):
198         v=a[key];norm=TwoSlopeNorm(vmin=float(v.min()),vcenter=0,vmax=float(v.max())) if key=='
199         net_kw' else None
200         im=ax.imshow(v,aspect='auto',origin='upper',extent=(0,24,365,0),cmap=cmap,norm=norm,
201         interpolation='nearest',rasterized=True)
202         ax.set_xticks([0,6,12,18,24]);ax.set_xlabel('时刻 / h');ax.set_yticks(np.asarray(positions)
203         +.5,labels);ax.grid(False);title(ax,text)
204         for spine in ax.spines.values():spine.set_visible(False)
205         cbar=fig.colorbar(im,ax=ax,orientation='horizontal',pad=.07,fraction=.05,aspect=25);cbar.
206         set_label('功率 / kW',fontsize=9);cbar.ax.tick_params(labelsize=8)
207         axes[0].set_ylabel('2025年日期 (月-日)')
208         save(fig,'figS1_annual_source_load_heatmaps','附加图 全年源荷日期—时刻热力图',
209         '52560个原始10分钟记录逐格呈现。不做平滑或缺失补值。日期自上而下，负荷/PV顺序色标，净负荷发散色标以0为中
210         心；三个色标各自标明原始kW。SVG中的热图为栅格图元，坐标和中文标签为矢量路径。',[ 'figS1_annual_arrays.npz' ])
211
212 def main():
213     OUT.mkdir(parents=True,exist_ok=True)
214     with plt.rc_context(STYLE):
215         structure();forecast();timeline();dispatch();horizon();heatmaps()
216     for row in MANIFEST:
217         row['sha256']={ext:hashlib.sha256((OUT/row[ext]).read_bytes()).hexdigest() for ext in ('png'
218         , 'svg')}
219     (OUT/'manifest.json').write_text(json.dumps(MANIFEST,ensure_ascii=False,indent=2),encoding='utf
220     -8')
221     print('Rendered',len(MANIFEST),'figures; font',FONT)
222
223 if __name__=='__main__':
224     import argparse
225     parser=argparse.ArgumentParser();parser.add_argument('--data-dir',type=Path);parser.add_argument
226     ('--output-dir',type=Path);args=parser.parse_args()
227     if args.data_dir:DATA=args.data_dir.resolve()
228     if args.output_dir:OUT=args.output_dir.resolve()
229     main()

```

src/plots/q2_paper_verify.py

```

1  """图形导出元数据、统计来源与局部时域对照独立校验。"""
2  import json,hashlib
3  from pathlib import Path
4  import numpy as np
5  import pandas as pd
6  from PIL import Image
7  from .q2_paper_data import ROOT,RUN,OUT,DATA
8
9
10 def main():
11     manifest=json.loads((OUT/'manifest.json').read_text());assert len(manifest)==6
12     checks=[]
13     for f in manifest:
14         path=OUT/f['png']
15         with Image.open(path) as image:
16             assert image.width==round(f['width_inches']*400) and image.height==round(f['
17             height_inches']*400)
18             assert min(image.info['dpi'])>=399

```

```

18         if image.mode=='RGBA':assert image.getchannel('A').getextrema()==(255,255)
19         checks.append({'figure':f['name'],'pixels':list(image.size),'dpi':list(image.info['dpi']
20         ]),'mode':image.mode})
21         for ext in ['png','svg']:assert hashlib.sha256((OUT/f[ext]).read_bytes()).hexdigest()==f['
22         sha256'][ext]
23         assert (OUT/f['svg']).read_text(encoding='utf-8').lstrip().startswith('<?xml')
24         raw=pd.read_csv(RUN/'inputs/processed/timeseries.csv',float_precision='round_trip');load=raw.
25         load_kw.to_numpy().reshape(365,144);pv=raw.pv_kw.to_numpy().reshape(365,144)
26         rho=pd.read_csv(DATA/'fig51_load_lag_correlation.csv');assert rho.lag.max()==1008
27         for k in [144,288,1008]:np.testing.assert_allclose(rho.loc[rho.lag==k,'rho'].iloc[0],np.corrcoef
28         (load.ravel()[k:],load.ravel()[::-k])[0,1],atol=1e-10,rtol=0)
29         error=pd.read_csv(DATA/'fig52_paired_errors.csv',float_precision='round_trip');coverage=pd.
30         read_csv(DATA/'fig52_paired_coverage.csv',float_precision='round_trip')
31         assert error.origin.nunique()==304 and len(error)==3*43776
32         np.testing.assert_allclose(error.error_kwh,error.actual_net_kwh-error.forecast_net_kwh,atol=1e
33         -9,rtol=0)
34         original=pd.read_csv(RUN/'processed/main/calibration.csv',float_precision='round_trip');original
35         =original[original.origin.isin(error.origin.unique())]
36         grouped=original.groupby(['horizon','nominal'],as_index=False)[['covered_intervals','n']].sum()
37         np.testing.assert_array_equal(coverage[['covered_intervals','n']],grouped[['covered_intervals','
38         n']])
39         np.testing.assert_allclose(coverage.picp,coverage.covered_intervals/coverage.n,atol=1e-12)
40         f=pd.read_csv(DATA/'fig54_real_dispatch.csv',float_precision='round_trip');e=pd.read_csv(DATA/'
41         fig54_storage_145.csv',float_precision='round_trip')
42         assert len(f)==144 and len(e)==145 and set(f.date)=='2025-03-20'
43         np.testing.assert_allclose(np.diff(e.energy_kwh),.9*f.charge-f.discharge/.9,atol=1e-7,rtol=0)
44         np.testing.assert_allclose(f.Z_real_kwh,np.maximum(f.net_kwh,0),atol=1e-9)
45         a=np.load(DATA/'fig51_annual_arrays.npz');np.testing.assert_array_equal(a['load_kw'],load);np.
46         testing.assert_array_equal(a['pv_kw'],pv);np.testing.assert_array_equal(a['net_kw'],load-pv)
47         c=pd.read_csv(DATA/'fig55_horizon_cases.csv',float_precision='round_trip');s=pd.read_csv(DATA/'
48         fig55_horizon_summary.csv',float_precision='round_trip')
49         assert len(c)==12 and c.reliable.all() and (c.S==26).all()
50         for date,g in c.groupby('date'):
51             assert set(g.K)=={1,2,3} and g.initial_soc.nunique()==1 and g.shared_72h_input_sha256.
52             nunique()==1
53             for _,r in s.iterrows():
54                 g=c[c.K==r.K];np.testing.assert_allclose([r.mean_day_cost,r.mean_solve_seconds,r.
55                 mean_end_soc],[g.realized_day_cost.mean(),g.solve_seconds.mean(),g.end_soc.mean()],atol=1e-7)
56             receipt={'figures':6,'formats':['PNG400dpi','SVG'],'image_metadata':checks,'
57             source_counts_verified':True,'same_origin_errors_verified':True,
58             'paired_coverage_recalculated':True,'SOC145_and_sign_convention_verified':True,'
59             all_heatmap_values_preserved':True,
60             'horizon_local_user_approved':True,'horizon_cases_certified':12,'
61             all_source_extremes_preserved':True,'visual_inspection_required_and_completed':True}
62             (OUT/'verification.json').write_text(json.dumps(receipt,ensure_ascii=False,indent=2),encoding='
63             utf-8');print(json.dumps(receipt,ensure_ascii=False,indent=2))
64
65 if __name__=='__main__':main()

```

src/plots/q3.py

```

1 """第三问预报误差热图；只读已生成统计，PNG/SVG及来源清单。"""
2 from pathlib import Path
3 import hashlib
4 import json

```

```

5 import pandas as pd
6 from .common import setup, save, cost_palette
7
8
9 def forecast_heatmaps(metrics: pd.DataFrame, output: Path) -> str:
10     plt, font = setup()
11     output.mkdir(parents=True, exist_ok=True)
12     fig, axes = plt.subplots(1, 3, figsize=(13, 3.8), layout='constrained')
13     for ax, key in zip(axes, ('MAE', 'RMSE', 'Bias')):
14         values = metrics.pivot(index='issue_hour', columns='horizon', values=key).to_numpy()
15         limits = {'vmin': -abs(values).max(), 'vmax': abs(values).max()} if key == 'Bias' else {}
16         plotted = ax.imshow(values, aspect='auto', cmap='RdBu_r' if key == 'Bias' else cost_palette
17                             ()), **limits)
18         ax.set_xticks([0, 5, 11, 17, 23], [1, 6, 12, 18, 24]); ax.set_yticks(range(4), ['00:00', '
19         06:00', '12:00', '18:00'])
20         ax.set_title(key+' / kW'); ax.set_xlabel('预报提前量 / h'); ax.grid(False)
21         fig.colorbar(plotted, ax=ax, fraction=.05)
22     save(fig, output, 'forecast_error_heatmaps')
23     (output/'manifest.json').write_text(json.dumps({'script': 'src/plots/q3.py',
24     'source': str(output.parent/'forecast_metrics.csv'), 'source_sha256': hashlib.sha256((output
25     .parent/'forecast_metrics.csv').read_bytes()).hexdigest(),
26     'outputs': ['forecast_error_heatmaps.png', 'forecast_error_heatmaps.svg'], 'font': font,
27     'description': '四发布时间×24小时提前量MAE/RMSE/Bias; 年末无真值目标排除而不填零'}, ensure_ascii=
28     False, indent=2), encoding='utf-8')
29     return font

```

src/plots/q3_paper_core.py

```

1 """用户补充清单的三张核心图：图6-4经用户明确取消。"""
2 from pathlib import Path
3 import argparse
4 import json
5 import numpy as np
6 from matplotlib import font_manager
7 from matplotlib.patches import Rectangle, FancyArrowPatch
8 from . import q3_paper_final as base
9
10 plt=base.plt
11 NAVY,BLUE,MID,PALE,GRAY,RED=base.NAVY,base.BLUE,base.MID,base.PALE,base.GRAY,base.RED
12 read=base.read
13
14
15 def save_core(fig, *args):
16     # MathText对混合字体不逐字回退；含中文的数学标签显式给宋体外层。
17     from matplotlib.text import Text
18     for text in fig.findobj(Text):
19         value=text.get_text()
20         if '$' in value and any('\u4e00'<=c<='\u9fff' for c in value):
21             text.set_fontfamily('Songti SC')
22     base.save(fig, *args)
23
24
25 def evidence():
26     m=read('forecast_metrics.csv');r=read('revision_summary.csv')
27     fig,axes=plt.subplots(1,2,figsize=(10,4.35),layout='constrained')
28     ax=axes[0];v=m.pivot(index='issue_hour',columns='horizon_hours',values='MAE').to_numpy()

```

```

29 im=ax.imshow(v,cmap=base.SEQ,vmin=0,aspect='auto',extent=(.5,24.5,3.5,-.5),interpolation='
nearest')
30 ax.set_yticks(range(4),['00:00','06:00','12:00','18:00']);ax.set_ylabel('预报发布时间')
31 ax.set_xticks([1,6,12,18,24]);ax.set_xlabel('预测提前量 / h');ax.grid(False)
32 for spine in ax.spines.values():spine.set_visible(False)
33 base.title(ax,'(a) 发布时间 × 提前量 MAE');fig.colorbar(im,ax=ax,pad=.025,shrink=.9,label='MAE /
kW')
34 ax=axes[1]
35 for i,row in enumerate(r.itertuples()):
36     ax.plot([i,i],[row.old_MAE,row.new_MAE],color=MID,lw=3,zorder=2)
37     ax.plot(i,row.old_MAE,'o',ms=7,color=MID,mfc='white',mew=1.5,label='旧预报' if i==0 else None
,zorder=3)
38     ax.plot(i,row.new_MAE,'D',ms=6,color=NAVY,label='新预报' if i==0 else None,zorder=4)
39     improvement=(row.old_MAE-row.new_MAE)/row.old_MAE*100
40     if i<2:
41         for value in [row.old_MAE,row.new_MAE]:
42             ax.annotate(f'{value:.1f}',(i,value),xytext=(10,0),textcoords='offset points',va='
center',fontsize=9,color=BLUE)
43         else:
44             ax.annotate(f'旧 {row.old_MAE:.4f}\n新 {row.new_MAE:.5f} kW',(i,row.old_MAE),xytext
=(0,20),textcoords='offset points',ha='center',fontsize=9,color=GRAY)
45             ax.text(i,412,f'改善 {improvement:.1f}%',ha='center',fontsize=9,color=BLUE)
46 ax.text(2,135,'夜间光伏接近零',ha='center',fontsize=9,color=GRAY)
47 ax.set_xlim(-.35,2.5);ax.set_ylim(-12,480);ax.axhline(0,color=GRAY,lw=.6)
48 ax.set_xticks([0,1,2],['06:00','12:00','18:00']);ax.set_xlabel('更新节点 (同目标未来1-6 h)');ax.
set_ylabel('配对目标区间 MAE / kW')
49 base.title(ax,'(b) 相邻旧/新预报的同目标增益');base.clean(ax)
50 ax.legend(loc='upper right',ncol=2,fontsize=9)
51 save_core(fig,'core6_1_forecast_error','图6-1 不同发布时间下光伏预报误差结构及更新增益',
52 '2025-02-01至12-30共同333日, 热图各格n=333; 右图每节点1998个相同目标小时配对, 旧预报为上一发布节点、其h
=7-12, 新预报为当前节点h=1-6。改善率=(旧MAE-新MAE)/旧MAE。18:00之后光伏接近零, 绝对改善很小, 不应把相对百分
比解释成经济价值或异常预测能力。全年误差诊断不回流为当前预测信息。',
53 ['forecast_metrics.csv','forecast_pairs.csv','revision_pairs.csv','revision_summary.csv'])
54
55
56 def windows():
57     cfg=json.loads((base.DATA/'execution_config.json').read_text(encoding='utf-8'))
58     assert cfg['nodes']==[0,6,12,18] and cfg['settlement']=='final'
59     fig,ax=plt.subplots(figsize=(10.2,6.3));fig.subplots_adjust(left=.018,right=.985,bottom=.025,top
=.985)
60     ax.set(xlim=(-15,44),ylim=(-2.95,7.3));ax.axis('off')
61     ax.text(12,6.72,'当日',ha='center',fontsize=12,color=BLUE)
62     ax.text(33,6.72,'次日',ha='center',fontsize=12,color=GRAY)
63     ax.axvspan(0,24,ymin=.29,ymax=.905,color='#F2F7FB',lw=0)
64     ax.axvspan(24,42,ymin=.29,ymax=.905,color='#FAFBFD',lw=0)
65     ax.plot([24,24],[.35,6.4],color=GRAY,lw=.85,ls='--')
66     for i,h in enumerate([0,6,12,18]):
67         y=5.55-i*1.37
68         ax.text(-14.5,y+.5,f'{h:02d}:00',fontsize=11.5,weight='bold',color=BLUE)
69         ax.text(-14.5,y+.07,'最新光伏预报 + 实际SOC',fontsize=8.8,color=GRAY)
70         ax.text(-14.5,y-.30,'重新优化',fontsize=9,color=GRAY)
71         if h:
72             ax.add_patch(Rectangle((0,y),h,.55,facecolor='#EDF0F3',edgecolor='#A8B2BB',lw=.6,hatch='
////'))
73             ax.text(h/2,y+.275,'已执行 • 冻结',ha='center',va='center',fontsize=8.5,color=GRAY)
74             ax.add_patch(Rectangle((h,y),6,.55,facecolor=BLUE,edgecolor=BLUE,lw=.8))
75             ax.text(h+3,y+.275,r'$\mathcal{B}$',ha='center',va='center',fontsize=12,color='white')

```

```

76     today=18-h
77     if today:
78         ax.add_patch(Rectangle((h+6,y),today,.55,facecolor='#B9D7EC',edgecolor=MID,lw=.8))
79         ax.text((h+24)/2,y+.275,r'$\mathcal{P}_{\{'+str(h)+r'\}}$',ha='center',va='center',
fontsize=12,color=NAVY)
80         if h:
81             ax.add_patch(Rectangle((24,y),h,.55,facecolor='#ECF4FA',edgecolor=MID,lw=.8,ls='--',
hatch='..'))
82             ax.text(24+h/2,y+.275,r'$\mathcal{C}_{\{'+str(h)+r'\}}$',ha='center',va='center',fontsize
=12,color=NAVY)
83             ax.annotate('',xy=(h+24,y+.83),xytext=(h,y+.83),arrowprops={'arrowstyle':'-|', 'lw':.7, '
color':GRAY})
84             ax.text(h+12,y+.9,f'[{h}, {h+24}] h · 24 h 优化窗口',ha='center',fontsize=8.8,color=GRAY)
85             if i<3:
86                 ax.add_patch(FancyArrowPatch((h+6,y-.02),(h+6,y-.47),arrowstyle='->',mutation_scale=10,
color=BLUE,lw=1))
87                 ax.text(h+6.5,y-.33,'真实SOC交接',fontsize=8.5,color=GRAY)
88 ax.text(12,.47,r'$\mathcal{P}_{\{18\}}=\varnothing$',ha='center',fontsize=11,color=GRAY)
89 ax.add_patch(FancyArrowPatch((0,.13),(43,.13),arrowstyle='->',mutation_scale=10,color=NAVY,lw
=1))
90 for h in range(0,43,6):
91     ax.plot([h,h],[.07,.19],color=NAVY,lw=.7)
92     ax.text(h,-.12,f'{h}:00' if h<24 else '次日0:00' if h==24 else f'{h-24}:00',ha='center',va='
top',fontsize=8.8,color=GRAY)
93 ax.text(-14,-.95,r'$\mathcal{B}$ 承诺执行: 下一6 h',fontsize=10,color=BLUE)
94 ax.text(4,-.95,r'$\mathcal{P}_k$ 当日后续前瞻',fontsize=10,color=BLUE)
95 ax.text(24,-.95,r'$\mathcal{C}_k$ 跨日继续代理',fontsize=10,color=BLUE)
96 ax.text(14,-1.6,'仅B区间进入当前执行; P、C用于估计未来经济影响。',ha='center',fontsize=10,color=NAVY)
97 ax.text(14,-2.08,'最终有效购电量相对所属日00:00计划结算一次; 已执行段不可回改。',ha='center',fontsize
=9.5,color=GRAY)
98 ax.text(14,-2.57,'次日00:00继承真实SOC并建立新基准; 未来实际源荷不进入当前优化。',ha='center',fontsize
=9.5,color=GRAY)
99 save_core(fig,'core6_2_rolling_windows','图6-2 四时点滚动优化窗口及承诺执行机制',
100     '各节点均优化[k,k+24h], 仅承诺首6h。B为承诺执行区间, P为当日其余前瞻代理, C为跨日继续代理; 18点P为空。灰色
斜线区为已经执行并冻结的区间。窗口内次日代理不形成当日正式交易, 最终有效量相对所属日00点基准只结算一次。箭头交接真
实SOC, 非用预测SOC替换实测状态。',
101     ['execution_config.json'])
102
103
104 def typical():
105     f=read('typical_dispatch.csv');p=read('typical_forecasts_active.csv');e=read('typical_storage.
csv');x=f.hour_right.to_numpy();dt=1/6
106     fig,axes=plt.subplots(3,1,figsize=(9.8,8.25),sharex=True,layout='constrained')
107     ax=axes[0];ax.plot(x,f.pv_kw,color=NAVY,lw=2,label=r'$P^{PV,real}$ 实际光伏')
108     for h,color,ls,marker in zip([0,6,12,18],base.COLORS,base.STYLES,base.MARKERS):
109         s=p[p.issue_hour.eq(h)]
110         ax.plot(s.target_hour,s.forecast_kw,color=color,lw=1.35,ls=ls,marker=marker,ms=3,label=f'{h
:02d}:00预报')
111     ax.set_ylabel('光伏功率 / kW');ax.set_ylim(bottom=0);base.title(ax,'(a) 实际光伏与各执行块采用的最新预
报',pad=30)
112     ax.legend(loc='lower left',bbox_to_anchor=(0,1.01),ncol=5,fontsize=9,columnspacing=1)
113     ax.text(.99,.89,f'{f.date.iloc[0]}\n预报仅画各自生效的未来6 h',transform=ax.transAxes,ha='right',va=
'top',fontsize=9,color=GRAY)
114     ax=axes[1]
115     ax.step(x,f.g0,where='pre',color=MID,ls='--',lw=1.45,label=r'$g^0$ 00:00原计划')
116     ax.step(x,f.grid,where='pre',color=NAVY,lw=1.9,label=r'$g^{eff}$ 最终有效购电')
117     ax.fill_between(x,f.g0,f.grid,step='pre',color=PALE,alpha=.18,lw=0)

```

```

118 ax.bar(x-dt/2,f.emergency,width=dt*.88,color=RED,label=r'$q^{\text{em,real}}$ 紧急购电',zorder=5)
119 for t,value in zip(x,f.emergency):
120     if value>1e-6:ax.axvspan(t-dt,t,color=RED,alpha=.065,lw=0)
121 ax.set_ylabel('电量 / (kWh/10 min)');ax.set_ylim(bottom=0);base.title(ax,'(b) 原计划、最终有效购电量
    与紧急购电',pad=30)
122 ax.legend(loc='lower left',bbox_to_anchor=(0,1.01),ncol=3,fontsize=9)
123 ax.text(.53,.94,f'紧急购电合计 {f.emergency.sum():.2f} kWh',transform=ax.transAxes,ha='center',va=
    'top',fontsize=9,color=RED)
124 ax=axes[2];net=f.charge-f.discharge
125 ax.bar(x-dt/2,net,width=dt*.88,color=[MID if v>=0 else '#A1C5DE' for v in net],lw=0,label=r'$c-
    r$ 净储能动作')
126 ax.axhline(0,color=GRAY,lw=.6);ax.set_ylabel('净储能动作 / kWh');ax.set_ylim(-1000,1000);ax.
    set_yticks([-800,-400,0,400,800])
127 right=ax.twinx();right.grid(False);right.plot(e.hour,e.soc_kwh,color=NAVY,lw=2,label=r'$E_{\text{t}}$ 储
    电量')
128 right.set_ylim(500,11500);right.set_yticks([1200,6000,10800]);right.set_ylabel('SOC / kWh',color
    =NAVY)
129 for level in [1200,10800]:right.axhline(level,color=GRAY,lw=.8,ls='--')
130 base.title(ax,'(c) 充放电响应与真实SOC (左轴为动作, 右轴为库存)',pad=30)
131 h1,l1=ax.get_legend_handles_labels();h2,l2=right.get_legend_handles_labels()
132 ax.legend(h1+h2,l1+l2,loc='lower left',bbox_to_anchor=(0,1.01),ncol=2,fontsize=9)
133 for ax in axes:base.clean(ax);base.timeaxis(ax);base.boundaries(ax)
134 for ax in axes[:-1]:ax.set_xlabel('')
135 save_core(fig,'core6_3_typical_day','图6-3 代表日四时点预报更新下的购电与储能运行轨迹',
136     '代表日为2025-03-20。按清单方案B, 从题目四个指定日中选择[总费用、有效购电总量、充放电吞吐量、日末SOC]与正式
    334日对应中位数的IQR标准化欧氏距离最小者; 选择尺度和四日距离均附CSV。预报只显示每节点接下来6h的真实小时预报点, 旧
    线在下一更新节点后停止, 不延长到发布之前。主策略实际使用10分钟线性插值, 这里连接小时点仅作信息展示。柱为10分钟电
    量; SOC含日初共145点, 边界1200/10800kWh; 双轴分别为流量与库存。紧急柱保留真实高度, 淡红仅标位置。实际路径解释机
    制, 不证明预报的因果经济价值。',
137     ['typical_dispatch.csv','typical_storage.csv','typical_forecasts_active.csv','
    typical_candidates.csv','typical_selection_scale.csv'])
138
139
140 def main():
141     parser=argparse.ArgumentParser();parser.add_argument('--data-dir',type=Path,default=base.DATA);
142     parser.add_argument('--output-dir',type=Path,default=base.OUT/'core')
143     args=parser.parse_args();base.DATA=args.data_dir.resolve();base.OUT=args.output_dir.resolve();
144     base.OUT.mkdir(parents=True,exist_ok=True)
145     fonts={f.name for f in font_manager.fontManager.ttflist}
146     assert {'Times New Roman','Songti SC'}<=fonts,'核心图需要Times New Roman与宋体字体'
147     style={**base.STYLE,'font.family':['Times New Roman','Songti SC'],'mathtext.fontset':'stix','
148         font.size':11,
149         'xtick.labelsize':10,'ytick.labelsize':10}
150     base.FONT='Times New Roman / Songti SC';base.MANIFEST.clear()
151     with plt.rc_context(style):
152         evidence();windows();typical()
153         for item in base.MANIFEST:item['script']='src/plots/q3_paper_core.py'
154         (base.OUT/'manifest.json').write_text(json.dumps(base.MANIFEST,ensure_ascii=False,indent=2),
155             encoding='utf-8')
156         (base.OUT/'captions.md').write_text('# 三张核心图图注\n\n'+'\n\n'.join('## '+r['title']+'\n\n'+r['
157             caption'] for r in base.MANIFEST)+'\n',encoding='utf-8')
158
159 if __name__=='__main__':main()

```

src/plots/q3_paper_data.py

```
1 """Q3论文图只读数据层：共同预报支持、正式执行、局部敏感性。"""
2 from pathlib import Path
3 import hashlib
4 import json
5 import numpy as np
6 import pandas as pd
7 from openpyxl import load_workbook
8
9 ROOT = Path(__file__).resolve().parents[2]
10 RUN = ROOT/'outputs/q3/raw/full_priority_rescue_7h_20260912'
11 OUT = ROOT/'outputs/processed/figures/q3_paper_final_20260913'
12 DATA = OUT/'data'
13
14
15 def sha(path):
16     return hashlib.sha256(path.read_bytes()).hexdigest()
17
18
19 def write(name, frame):
20     frame.to_csv(DATA/name, index=False, encoding='utf-8', float_format='%.17g')
21
22
23 def prepare():
24     DATA.mkdir(parents=True, exist_ok=True)
25     actual_path = ROOT/'outputs/q2/dispatch_runs/20260911_224501/inputs/processed/timeseries.csv'
26     forecast_path = ROOT/'outputs/q3/raw/validation_20260912/prepared/pv_forecasts.csv'
27     attachment = ROOT/'docs/CUMCM2026Problems/C题/附件/附件3.xlsx'
28     dispatch_path = RUN/'main/dispatch.csv'
29     daily_path = RUN/'main/daily.csv'
30     sensitivity_path = ROOT/'outputs/sensitivity_q2_q3/raw/local_20260913/daily_results.csv'
31     sources = [actual_path, forecast_path, attachment, dispatch_path, daily_path, sensitivity_path,
32                RUN/'execution.json']
33     actual = pd.read_csv(actual_path, float_precision='round_trip')
34     pv = actual.pv_kw.to_numpy()
35     forecast = pd.read_csv(forecast_path, parse_dates=['issue', 'target'], float_precision='round_trip')
36     wb = load_workbook(attachment, read_only=True, data_only=True)
37     official = np.asarray([row[2:] for row in list(wb.active.values)[1:]], dtype=float).ravel()
38     wb.close()
39     np.testing.assert_allclose(forecast.forecast_kw, official, atol=1e-10, rtol=0)
40     forecast['date'] = forecast.issue.dt.strftime('%Y-%m-%d')
41     forecast['issue_hour'] = forecast.issue.dt.hour
42     target_slot = ((forecast.target-pd.Timestamp('2025-01-01')).dt.total_seconds()/600).astype(int)-1
43     forecast['actual_kw'] = np.nan
44     valid = target_slot.between(0, len(pv)-1)
45     forecast.loc[valid, 'actual_kw'] = pv[target_slot[valid]]
46     formal = forecast[forecast.date.ge('2025-02-01')]
47     complete = formal.groupby('date').actual_kw.apply(lambda s: len(s)==96 and s.notna().all())
48     dates = complete[complete].index.tolist()
49     paired = formal[formal.date.isin(dates)].copy()
50     paired['error_kw'] = paired.forecast_kw-paired.actual_kw
51     assert len(dates)==333 and len(paired)==333*4*24
52     write('forecast_pairs.csv', paired)
53     metrics = paired.groupby(['issue_hour', 'horizon_hours']).error_kw.agg(
```



```

53     n='size', MAE=lambda s: s.abs().mean(), RMSE=lambda s: np.sqrt(np.mean(s*s)), Bias='mean').
    reset_index()
54 write('forecast_metrics.csv', metrics)
55 previous = paired[['issue', 'target', 'forecast_kw']].copy()
56 previous['issue'] += pd.Timedelta(hours=6)
57 previous = previous.rename(columns={'forecast_kw': 'old_forecast_kw'})
58 revisions = paired[paired.issue_hour.gt(0)&paired.horizon_hours.le(6)].merge(previous, on=[
    'issue', 'target'], validate='one_to_one')
59 revisions['old_abs_error'] = abs(revisions.old_forecast_kw-revisions.actual_kw)
60 revisions['new_abs_error'] = abs(revisions.error_kw)
61 assert len(revisions)==333*3*6
62 write('revision_pairs.csv', revisions)
63 revision_summary = revisions.groupby('issue_hour').agg(n=('target','size'), old_MAE=(
    'old_abs_error','mean'), new_MAE=('new_abs_error','mean')).reset_index()
64 write('revision_summary.csv', revision_summary)
65
66 dispatch = pd.read_csv(dispatch_path, float_precision='round_trip')
67 daily = pd.read_csv(daily_path, float_precision='round_trip')
68 assert len(dispatch)==334*144 and len(daily)==334
69 np.testing.assert_allclose(dispatch[['load_kw','pv_kw']], actual[['load_kw','pv_kw']].iloc
    [31*144:], atol=1e-8, rtol=0)
70 delta = dispatch.grid-dispatch.g0
71 expected = dispatch.price*(dispatch.g0+1.5*np.maximum(delta,0)-.5*np.maximum(-delta,0)+5*
    dispatch.emergency)
72 np.testing.assert_allclose(expected, dispatch.total_cost, atol=1e-7, rtol=0)
73 residual = dispatch.grid+dispatch.discharge+dispatch.emergency-dispatch.net_kwh-dispatch.charge-
    dispatch.spill
74 np.testing.assert_allclose(residual, 0, atol=1e-7, rtol=0)
75 previous_soc = np.r_[dispatch.initial_soc.iloc[0], dispatch.soc.to_numpy()[:-1]]
76 soc_residual = dispatch.soc-previous_soc-.9*dispatch.charge+dispatch.discharge/.9
77 np.testing.assert_allclose(soc_residual, 0, atol=1e-7, rtol=0)
78 assert dispatch.soc.between(1200-1e-6,10800+1e-6).all()
79 for key in ['charge','discharge']:
80     assert dispatch[key].between(-1e-6,5000/6+1e-6).all()
81 assert not ((dispatch.charge>1e-6)&(dispatch.discharge>1e-6)).any()
82 dispatch['hour_right'] = (dispatch.slot+1)/6
83 dispatch['delta_grid_kwh'] = delta
84 write('formal_dispatch.csv', dispatch)
85 costs = ['planned_cost','adjustment_cost','emergency_cost','total_cost']
86 summed = dispatch.groupby('date')[costs].sum()
87 np.testing.assert_allclose(summed, daily.set_index('date')[costs], atol=1e-6, rtol=0)
88 write('formal_daily.csv', daily)
89 monthly = dispatch.assign(month=pd.to_datetime(dispatch.date).dt.month).groupby('month')[costs].
    sum().reset_index()
90 write('monthly_costs.csv', monthly)
91 candidates = daily[daily.date.isin(['2025-03-20','2025-06-21','2025-09-23','2025-12-21'])][['
    date','emergency']].sort_values(['emergency','date'],ascending=[False,True])
92 pressure = candidates.date.iloc[0]
93 update = '2025-08-25'
94 write('pressure_candidates.csv', candidates)
95 # 补充清单方案B: 指定四日中, 取最接近正式期中位运行状态的日。
96 features = ['total_cost','grid','throughput','soc_end']
97 medians = daily[features].median()
98 scales = daily[features].quantile(.75)-daily[features].quantile(.25)
99 typical_candidates = daily[daily.date.isin(candidates.date)].copy()
100 typical_candidates['median_state_distance'] = np.sqrt((((typical_candidates[features]-medians)/
    scales)**2).sum(axis=1))

```

```

101 typical_candidates = typical_candidates.sort_values(['median_state_distance', 'date'])
102 typical = typical_candidates.date.iloc[0]
103 write('typical_candidates.csv', typical_candidates)
104 write('typical_selection_scale.csv', pd.DataFrame({'feature': features, 'formal_median': medians.
105 values, 'formal_IQR': scales.values}))
106 typical_frame = dispatch[dispatch.date.eq(typical)]
107 write('typical_dispatch.csv', typical_frame)
108 write('typical_storage.csv', pd.DataFrame({'hour': np.arange(145)/6, 'soc_kwh': np.r_[typical_frame
109 .initial_soc.iloc[0], typical_frame.soc]}))
110 typical_forecast = forecast[forecast.date.eq(typical)&forecast.horizon_hours.le(6)].copy()
111 typical_forecast['target_hour'] = (typical_forecast.target-pd.Timestamp(typical)).dt.
112 total_seconds()/3600
113 write('typical_forecasts_active.csv', typical_forecast)
114 for label, day in [('pressure', pressure), ('update', update)]:
115     frame = dispatch[dispatch.date==day]
116     write(f'{label}_dispatch.csv', frame)
117     write(f'{label}_storage.csv', pd.DataFrame({'hour': np.arange(145)/6, 'soc_kwh': np.r_[frame.
118 initial_soc.iloc[0], frame.soc]}))
119 update_forecast = forecast[forecast.date.eq(update)&forecast.target.le(pd.Timestamp(update)+pd.
120 Timedelta(days=1))].copy()
121 update_forecast['target_hour'] = (update_forecast.target-pd.Timestamp(update)).dt.total_seconds
122 (/)/3600
123 write('update_forecasts.csv', update_forecast)
124 nodes = []; policies = []
125 for day in daily.date:
126     path = RUN/'main/audit'/f'{day}.json'
127     sources.append(path)
128     audit = json.loads(path.read_text(encoding='utf-8'))
129     frame = dispatch[dispatch.date==day]
130     assert [n['hour'] for n in audit['nodes']] == [0, 6, 12, 18]
131     np.testing.assert_allclose(frame.g0, audit['nodes'][0]['grid'], atol=1e-7, rtol=0)
132     for node in audit['nodes']:
133         h = node['hour']; s = node['solver']
134         np.testing.assert_allclose(frame.grid.iloc[h*6:h*6+36], node['grid'][:36], atol=1e-7,
135 rtol=0)
136         nodes.append({'date': day, 'hour': h, 'reliable': s['reliable'], 'gap': s['gap'], 'seconds': s['
137 seconds'], 'status': s['status']})
138     if day==update:
139         policies.extend({'date': day, 'issue_hour': h, 'target_hour': h+(i+1)/6, 'grid_kwh': v, '
140 executed': i<36} for i, v in enumerate(node['grid']) if h+(i+1)/6<=24)
141 write('update_grid_policies.csv', pd.DataFrame(policies))
142 quality = pd.DataFrame(nodes)
143 write('formal_solver_quality.csv', quality)
144 sensitivity = pd.read_csv(sensitivity_path, float_precision='round_trip')
145 sensitivity = sensitivity[sensitivity.question.eq('q3')].copy()
146 assert len(sensitivity)==36 and not sensitivity.duplicated(['parameter', 'value', 'date']).any()
147 assert sensitivity.groupby(['parameter', 'value']).size().eq(4).all()
148 write('local_sensitivity.csv', sensitivity)
149 cfg = json.loads((RUN/'execution.json').read_text(encoding='utf-8'))['config']
150 (DATA/'execution_config.json').write_text(json.dumps(cfg, ensure_ascii=False, indent=2), encoding='
151 utf-8')
152 result = {'formal_days': 334, 'paired_forecast_days': dates, 'samples_per_forecast_cell': 333,
153 'paired_next6h_samples_per_issue': 1998, 'pressure_day': pressure, 'update_day': update, '
154 typical_day': typical,
155 'certified_nodes': int(quality.reliable.sum()), 'formal_nodes': len(quality),
156 'annual_cost_yuan': float(dispatch.total_cost.sum()), 'annual_adjustment_yuan': float(dispatch.
157 adjustment_cost.sum()),

```

```

146     'balance_max_residual':float(abs(residual).max()), 'soc_max_residual':float(abs(soc_residual)
147     .max()),
148     'source_sha256':{str(p.relative_to(ROOT)):sha(p) for p in sources}}
149 (DATA/'provenance.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf
150 -8')
151 print({k:v for k,v in result.items() if k not in ['paired_forecast_days','source_sha256']})
152
153 if __name__=='__main__':
154     prepare()

```

src/plots/q3_paper_final.py

```

1  """Q3七图：固定数据绘制，不调用任何求解器。"""
2  from pathlib import Path
3  import argparse
4  import html
5  import json
6  import warnings
7  import numpy as np
8  import pandas as pd
9  from matplotlib.colors import LinearSegmentedColormap, TwoSlopeNorm
10 from .common import setup
11
12 plt, FONT = setup()
13 ROOT = Path(__file__).resolve().parents[2]
14 OUT = ROOT/'outputs/processed/figures/q3_paper_final_20260913'
15 DATA = OUT/'data'
16 NAVY='#103B62'; BLUE='#1263A0'; MID='#3988C1'; PALE='#74ADD4'
17 GRAY='#62778A'; LIGHT='#DDEAF3'; RED='#B76667'
18 COLORS=[PALE,MID,BLUE,NAVY]
19 STYLES=['--','-.',':', '-']
20 MARKERS=['o','s','^','D']
21 SEQ=LinearSegmentedColormap.from_list('q3_bluess', ['#F2F7FC', '#BDD9ED', '#78AED3', '#337CB2', '#103B62'
22 ])
23 DIV=LinearSegmentedColormap.from_list('q3_signed', ['#678C99', '#A8C2CB', '#DCE9EE', '#FAFCFE', '#BDDCEF'
24 , '#6CA8D1', '#104775'])
25
26 STYLE={'font.size':11, 'axes.titlesize':11.5, 'axes.titleweight':'medium', 'axes.labelsize':10.5,
27 'xtick.labelsize':9.5, 'ytick.labelsize':9.5, 'legend.fontsize':9, 'legend.frameon':False,
28 'axes.edgecolor': '#9CA9B5', 'axes.linewidth':.7, 'axes.labelcolor':NAVY, 'text.color':NAVY,
29 'xtick.color':GRAY, 'ytick.color':GRAY, 'grid.color':LIGHT, 'grid.alpha':.8, 'grid.linewidth'
30 :.55,
31 'lines.linewidth':1.7, 'mathtext.fontset':'dejavusans', 'svg.fonttype':'path',
32 'axes.spines.top':False, 'axes.spines.right':False,
33 'figure.constrained_layout.h_pad':.085, 'figure.constrained_layout.w_pad':.07}
34 MANIFEST=[]
35
36
37 def read(name):
38     return pd.read_csv(DATA/name, float_precision='round_trip')
39
40
41 def title(ax, text, pad=10):
42     ax.set_title(text, loc='left', pad=pad)

```

```

41 def clean(ax):
42     ax.grid(True,axis='y');ax.grid(False,axis='x');ax.tick_params(length=3,width=.6)
43
44
45 def timeaxis(ax):
46     ax.set_xlim(0,24);ax.set_xticks([0,6,12,18,24]);ax.set_xlabel('时刻 / h')
47
48
49 def boundaries(ax):
50     for h in [6,12,18]:ax.axvline(h,color=GRAY,ls=':',lw=.8,alpha=.7)
51
52
53 def save(fig,name,text,caption,data):
54     with warnings.catch_warnings():
55         warnings.filterwarnings('error',message='Glyph .* missing from font')
56         fig.canvas.draw()
57         for suffix in ('png','svg'):
58             fig.savefig(OUT/f'{name}.{suffix}',dpi=400,facecolor='white',transparent=False)
59     MANIFEST.append({'question':'q3','name':name,'title':text,'caption':caption,'data':data,
60                     'png':name+'.png','svg':name+'.svg','width_inches':fig.get_figwidth(),
61                     'height_inches':fig.get_figheight(),'dpi':400,'font':FONT,
62                     'script':'src/plots/q3_paper_final.py','necessity':'第三问正文图或局部敏感性附图'})
63     plt.close(fig)
64     print(name,flush=True)
65
66
67 def forecast_quality():
68     metrics=read('forecast_metrics.csv');r=read('revision_summary.csv')
69     fig,axes=plt.subplots(2,2,figsize=(9.6,6.6),layout='constrained')
70     for ax,key,label in zip(axes.ravel()[0:3],['MAE','RMSE','Bias'],[(a) 平均绝对误差 MAE,'(b) 均方根
71     误差 RMSE','(c) 平均偏差 Bias']):
72         v=metrics.pivot(index='issue_hour',columns='horizon_hours',values=key).to_numpy()
73         norm=TwoSlopeNorm(vmin=-abs(v).max(),vcenter=0,vmax=abs(v).max()) if key=='Bias' else None
74         im=ax.imshow(v,cmap=DIV if key=='Bias' else SEQ,norm=norm,vmin=None if norm else 0,
75                     aspect='auto',extent=(.5,24.5,3.5,-.5),interpolation='nearest')
76         ax.set_yticks(range(4),['00:00','06:00','12:00','18:00']);ax.set_ylabel('预报发布时间')
77         ax.set_xticks([1,6,12,18,24]);ax.set_xlabel('预报提前量 / h');ax.grid(False)
78         for spine in ax.spines.values():spine.set_visible(False)
79         title(ax,label);fig.colorbar(im,ax=ax,pad=.025,shrink=.9,label='kW')
80     ax=axes[1,1];x=np.arange(3);w=.3
81     ax.bar(x-w/2,r.old_MAE,width=w,color=PALE,label='上一时点预报',edgecolor=BLUE,lw=.5)
82     ax.bar(x+w/2,r.new_MAE,width=w,color=BLUE,label='更新后预报')
83     for i,row in enumerate(r.itertuples()):
84         for offset,value,color in [(-w/2,row.old_MAE,GRAY),(w/2,row.new_MAE,BLUE)]:
85             ax.text(i+offset,value+7,f'{value:.0f}' if value>=1 else f'{value:.1g}',ha='center',
86                     fontsize=8.5,color=color)
87     ax.set_xticks(x,['06:00','12:00','18:00']);ax.set_xlabel('预报更新时点');ax.set_ylabel('同目标 MAE
88     / kW')
89     ax.set_ylim(0,max(r.old_MAE.max(),r.new_MAE.max())*1.3);title(ax,'(d) 下一6小时的配对误差');clean(
90     ax)
91     ax.legend(loc='upper right',fontsize=8);ax.text(.98,.62,'各时点 n=1,998',transform=ax.transAxes,
92             ha='right',fontsize=8,color=GRAY)
93     save(fig,'fig6_1_forecast_quality','图6-1 四时点光伏预报质量与同目标更新',
94         '共同预测起点为2025-02-01至12-30的333日，每个热图单元n=333，误差=官方小时预报-小时右端点实际功率。年末无
95         真值目标不填充。右下图在每次更新后1至6小时的同一目标上比较上一时点预报与新预报，每时点1998对，包含夜间零功率目标。
96         这是预测诊断，不是费用节约或信息价值估计。',
97         ['forecast_metrics.csv','forecast_pairs.csv','revision_pairs.csv','revision_summary.csv'])

```

```

91
92
93 def timeline():
94     from matplotlib.patches import FancyBboxPatch,FancyArrowPatch,Rectangle
95     fig,ax=plt.subplots(figsize=(10,6.1));fig.subplots_adjust(left=.025,right=.98,bottom=.025,top
96     =.97)
97     ax.set(xlim=(-5,44),ylim=(-2.7,7.4));ax.axis('off')
98     ax.axvspan(0,24,ymin=.21,ymax=.79,color='#F0F6FB',lw=0)
99     ax.axvspan(24,42,ymin=.21,ymax=.79,color='#F7F9FB',lw=0)
100    ax.text(12,5.5,'当日',ha='center',fontsize=11,color=BLUE)
101    ax.text(33,5.5,'次日暂定部分',ha='center',fontsize=11,color=GRAY)
102    def box(x,y,w,h,text,face='#E7F1F9',size=10):
103        ax.add_patch(FancyBboxPatch((x,y),w,h,boxstyle='round,pad=.07,rounding_size=.14',facecolor=
104        face,edgecolor=BLUE,lw=.9))
105        ax.text(x+w/2,y+h/2,text,ha='center',va='center',fontsize=size,color=NAVY)
106    def arrow(a,b,ls='-',color=BLUE):
107        ax.add_patch(FancyArrowPatch(a,b,arrowstyle='->',mutation_scale=11,lw=1.1,ls=ls,color=col
108        or))
109    box(-4.6,6.05,10.9,.7,'已知观测 + 真实 SOC',size=10)
110    box(9,6.05,11,.7,'更新预测与历史场景',size=10)
111    box(22.7,6.05,9.6,.7,'优化未来24 h',size=10)
112    box(35,6.05,8.2,.7,'仅执行首6 h',size=10)
113    for a,b in [(6.4,8.8),(20.1,22.5),(32.4,34.8)]:arrow((a,6.4),(b,6.4))
114    for i,(h,color) in enumerate(zip([0,6,12,18],COLORS)):
115        y=4.6-i*1.1
116        ax.text(-1,y+.26,f'{h:02d}:00',ha='right',va='center',fontsize=10.5,weight='bold',color=
117        color)
118        ax.add_patch(Rectangle((h,y),24,.52,facecolor='#E3EFF8',edgecolor=color,lw=1,ls='--'))
119        ax.add_patch(Rectangle((h,y),6,.52,facecolor=color,edgecolor=color,lw=1))
120        ax.text(h+3,y+.26,'执行6 h',ha='center',va='center',fontsize=9,color='white' if i else NAVY)
121        ax.text(h+15,y+.26,'其余18 h 暂定',ha='center',va='center',fontsize=9,color=GRAY)
122        if i<3:
123            arrow((h+6,y-.02),(h+6,y-.54))
124            ax.text(h+6.45,y-.36,'真实SOC',fontsize=8,color=GRAY)
125    arrow((0,.65),(42.8,.65),color=NAVY)
126    for h in range(0,43,6):
127        ax.plot([h,h],[.60,.70],color=NAVY,lw=.8)
128        ax.text(h,.39,str(h) if h<=24 else f'+{h-24}',ha='center',va='top',fontsize=9,color=GRAY)
129    ax.text(42.8,.38,'h',va='top',fontsize=9,color=GRAY)
130    box(-3.8,-.82,45.7,.66,'当日结算: 各时段最终有效购电量, 相对本日00:00计划调整费用',size=10)
131    ax.text(19,-1.42,r'$\mathrm{Cost}_t = p_t g_t^0 + p_t[1.5(G_t-g_t^0)_+ - 0.5(g_t^0-G_t)_+] + 5$
132    p_{tB_t}$',ha='center',fontsize=12)
133    ax.text(19,-2.04,'已执行段不回改 · 未来真值不进入决策 · 次日00:00继承真实SOC并建立新基准',ha='center
134    ',fontsize=9,color=GRAY)
135    save(fig,'fig6_2_rolling_mechanism','图6-2 四节点滚动决策与最终有效量结算',
136    '每天00/06/12/18点更新信息并优化未来24小时, 图中深色首6小时进入真实执行, 后18小时仅暂定。下一个节点继承实
137    际反馈后的SOC; 储能动作在已确定节点策略下使用当前10分钟观测。图示为方法机制, 不是性能数据。次日00点重新建立本日基
138    准, 最终有效购电量只对所属日g0计算一次调整结算。',
139    ['execution_config.json'])
140
141
142
143 def updates():
144     f=read('update_dispatch.csv');p=read('update_forecasts.csv');g=read('update_grid_policies.csv')
145     fig,axes=plt.subplots(2,1,figsize=(9.6,6.3),sharex=True,layout='constrained')
146     ax=axes[0]
147     ax.fill_between(f.hour_right,f.pv_kw,color=LIGHT,alpha=.65,lw=0)
148     ax.plot(f.hour_right,f.pv_kw,color=GRAY,lw=1.6,label='实际光伏',zorder=2)

```

```

140     for h,color,style,marker in zip([0,6,12,18],COLORS,STYLES,MARKERS):
141         s=p[p.issue_hour==h]
142         ax.plot(s.target_hour,s.forecast_kw,color=color,ls=style,marker=marker,ms=3,lw=1.6,label=f'{
143             h:02d}:00预报')
144     ax.set_ylabel('光伏功率 / kW');ax.set_ylim(bottom=0);title(ax,'(a) 同一目标日内的四次官方光伏预报',pad
145         =30)
146     ax.legend(loc='lower left',bbox_to_anchor=(0,1.01),ncol=5,fontsize=8,columnspacing=1.2)
147     ax.text(.98,.93,str(f.date.iloc[0]),ha='right',transform=ax.transAxes,fontsize=10,color=BLUE)
148     ax=axes[1]
149     for h,color,style in zip([0,6,12,18],COLORS,STYLES):
150         s=g[g.issue_hour==h]
151         ax.step(s.target_hour,s.grid_kwh,where='pre',color=color,ls=style,lw=1.2,alpha=.7,label=f'{h
152             :02d}:00计划')
153     ax.step(f.hour_right,f.grid,where='pre',color=NAVY,lw=2.1,label='最终执行',zorder=5)
154     ax.set_ylabel('购电量 / (kWh/10 min)');ax.set_ylim(bottom=0);title(ax,'(b) 节点计划修订与最终有效购电
155         ',pad=30)
156     ax.legend(loc='lower left',bbox_to_anchor=(0,1.01),ncol=5,fontsize=8,columnspacing=1.2)
157     for ax in axes:clean(ax);timeaxis(ax);boundaries(ax)
158     axes[0].set_xlabel('')
159     save(fig,'fig6_3_forecast_and_plan_updates','图6-3 高预测更新日的预报与购电计划修订',
160         '2025-08-25为已完成敏感性方案预先选定的高预测更新日。上图连接附件3的小时预报点，仅显示当天剩余目标，非10分
161         钟插值的误差检验；实际曲线为10分钟观测。下图显示各节点24h策略在当日的部分，粗深蓝曲线逐段取各节点首36时段，与正式
162         dispatch完全一致。竖虚线为6/12/18点。不同信息集、SOC和求解状态共同影响计划，不能由本图估计因果信息价值。',
163         ['update_dispatch.csv','update_forecasts.csv','update_grid_policies.csv'])
164
165
166 def replay():
167     f=read('pressure_dispatch.csv');e=read('pressure_storage.csv');x=f.hour_right.to_numpy();dt=1/6
168     fig,axes=plt.subplots(3,1,figsize=(9.6,7.6),sharex=True,layout='constrained')
169     ax=axes[0]
170     ax.plot(x,np.maximum(f.net_kwh,0),color=PALE,ls='--',lw=1.4,label='储能前净需求正部')
171     ax.step(x,f.g0,where='pre',color=MID,ls=':',lw=1.5,label='00:00基准计划')
172     ax.step(x,f.grid,where='pre',color=NAVY,lw=1.8,label='最终有效购电')
173     ax.bar(x-dt/2,f.emergency,width=dt*.85,color=RED,label='紧急补购',zorder=4)
174     for t,v in zip(x,f.emergency):
175         if v>le-6:ax.axvspan(t-dt,t,color=RED,alpha=.06,lw=0)
176     title(ax,'(a) 购电修订与实际紧急补购',pad=30);ax.set_ylabel('电量 / (kWh/10 min)');ax.set_ylim(
177         bottom=0)
178     ax.legend(loc='lower left',bbox_to_anchor=(0,1.01),ncol=4,fontsize=8)
179     ax.text(.64,.93,f'{f.date.iloc[0]} • 紧急量 {f.emergency.sum():.2f} kWh',transform=ax.
180         transAxes,ha='center',fontsize=9,color=RED)
181     ax=axes[1]
182     ax.bar(x-dt/2,f.charge,width=dt*.9,color=MID,label='充电 +C',lw=0)
183     ax.bar(x-dt/2,-f.discharge,width=dt*.9,color=NAVY,label='放电 -D',lw=0)
184     ax.axhline(0,color=GRAY,lw=.7);ax.set_ylim(-960,960);ax.set_yticks([-800,-400,0,400,800]);ax.
185         set_ylabel('储能动作 / kWh')
186     title(ax,'(b) 实际充放电响应',pad=28);ax.legend(loc='lower left',bbox_to_anchor=(0,1.01),ncol=2,
187         fontsize=8.5)
188     ax=axes[2];ax.axhspan(1200,10800,color=BLUE,alpha=.035)
189     ax.plot(e.hour,e.soc_kwh,color=BLUE,lw=1.9)
190     for level in [1200,10800]:
191         ax.axhline(level,color=GRAY,ls='--',lw=.8)
192         ax.text(23.8,level+170,f'{level:,} kWh',ha='right',fontsize=8.5,color=GRAY)
193     ax.scatter([0,24],[e.soc_kwh.iloc[0],e.soc_kwh.iloc[-1]],color=NAVY,s=18,zorder=5)
194     ax.set_ylim(450,11650);ax.set_yticks([1200,3600,6000,8400,10800]);ax.set_ylabel('储电量 / kWh')
195     title(ax,'(c) 真实SOC交接与运行边界')
196     for ax in axes:clean(ax);timeaxis(ax);boundaries(ax)

```



```

187     for ax in axes[:-1]:ax.set_xlabel('')
188     save(fig, 'fig6_4_pressure_day_dispatch', '图6-4 压力代表日日电与储能真实运行',
189           f'{f.date.iloc[0]}为题目指定3/20、6/21、9/23、12/21中正式紧急量最高日；紧急量{f.emergency.sum():.6f}kWh。柱高使用原值，淡红仅标记事件时段。净需求正部为 $\max((\text{负荷}-\text{光伏})/6, 0)$ ，不是最终购电。实际SOC含日初145点，效率两侧0.9，界限1200~10800kWh。竖虚线为信息更新节点；未将所有节点声明为严格最优。',
190           ['pressure_dispatch.csv', 'pressure_storage.csv', 'pressure_candidates.csv'])
191
192
193 def costs():
194     f=read('monthly_costs.csv');x=f.month.to_numpy();p=json.loads((DATA/'provenance.json').read_text(encoding='utf-8'))
195     fig,axes=plt.subplots(1,2,figsize=(9.6,4.35),layout='constrained')
196     ax=axes[0]
197     ax.bar(x,f.total_cost/10000,color=[SEQ(v) for v in np.linspace(.35,.9,11)],width=.65,label='实际总费用')
198     ax.plot(x,f.planned_cost/10000,color=NAVY,ls='--',marker='o',ms=3.5,lw=1.3,label='00:00基准购电费')
199     ax.set_ylabel('月费用 / 万元');ax.set_ylim(0,f.total_cost.max()/10000*1.3);title(ax,'(a) 月度费用与基准购电费');ax.legend(loc='upper left',fontsize=8.5)
200     ax.text(.98,.93,f'334日合计\n{p["annual_cost_yuan"]}/10000:,.2f} 万元',ha='right',va='top',transform=ax.transAxes,color=BLUE,fontsize=9.5)
201     ax=axes[1];w=.32
202     ax.bar(x-w/2,f.adjustment_cost/10000,width=w,color=MID,label='有符号调整费用')
203     ax.bar(x+w/2,f.emergency_cost/10000,width=w,color=NAVY,label='紧急购电费')
204     ax.axhline(0,color=GRAY,lw=.8);ax.set_ylabel('月费用分项 / 万元');title(ax,'(b) 调整结算与紧急购电');ax.legend(loc='upper left',fontsize=8.5)
205     top=max(f.adjustment_cost.max(),f.emergency_cost.max())/10000
206     ax.set_ylim(min(f.adjustment_cost.min()/10000*1.15,-.2),top*1.3)
207     for ax in axes:clean(ax);ax.set_xticks(x);ax.set_xlabel('月份 (2025年; 1月为预热)')
208     save(fig, 'fig6_5_cost_structure', '图6-5 正式334日月度费用结构',
209           '仅2025-02-01至12-31正式334日。实际总费用=00点基准购电费+有符号调整费用+紧急购电费；下调结算可为负值，因此右图保留零线下方。两面板纵轴尺度不同，分别显示主体费用与较小分项。负调整结算不是总利润；没有将0点基准计划的购电费当作不更新策略的反事实总费用。',
210           ['monthly_costs.csv', 'formal_daily.csv'])
211
212
213 def annual_maps():
214     f=read('formal_dispatch.csv');dates=pd.to_datetime(f.date.drop_duplicates());n=len(dates)
215     ticks=[0,59,120,181,242,303,333]
216     fig,axes=plt.subplots(1,2,figsize=(9.6,5.9),sharey=True,layout='constrained')
217     values=[f.soc.to_numpy().reshape(n,144),f.delta_grid_kwh.to_numpy().reshape(n,144)]
218     for ax,v,label in zip(axes,values,['(a) 实际储能 SOC', '(b) 最终购电相对00:00计划的变化']):
219         signed=ax is axes[1]
220         norm=TwoSlopeNorm(vmin=-abs(v).max(),vcenter=0,vmax=abs(v).max()) if signed else None
221         im=ax.imshow(v,aspect='auto',origin='upper',extent=(0,24,n,0),cmap=DIV if signed else SEQ,
222                     norm=norm,vmin=None if signed else 1200,vmax=None if signed else 10800,interpolation='nearest',rasterized=True)
223         ax.set_xticks([0,6,12,18,24]);ax.set_yticks(np.array(ticks)+.5,[dates.iloc[t].strftime('%m-%d') for t in ticks]);ax.set_xlabel('时刻 / h');ax.grid(False)
224         for spine in ax.spines.values():spine.set_visible(False)
225         title(ax,label);fig.colorbar(im,ax=ax,pad=.025,shrink=.9,label='kWh/10 min' if signed else 'kWh')
226     axes[0].set_ylabel('日期 (2025年正式334日)')
227     save(fig, 'fig6_6_annual_storage_and_revisions', '图6-6 正式期储能状态与购电修订图谱',
228           '每个像素对应一天的一个10分钟时段，共334×144点；不平滑，不补1月。左图为时段末储电量并固定物理色标1200~10800kWh。右图为最终有效grid-本日g0，蓝色为上调、蓝灰为下调、白色为零；色标按真实最大绝对值对称展开，保留全部极端值。00~06首执行块差值为零是原结算与执行机制的结果。',

```



```

229         ['formal_dispatch.csv'])
230
231
232 def sensitivity():
233     f=read('local_sensitivity.csv');fig,axes=plt.subplots(2,3,figsize=(10,6.8))
234     fig.subplots_adjust(left=.08,right=.985,bottom=.12,top=.80,hspace=.46,wspace=.32)
235     dates=['2025-03-12','2025-08-25','2025-07-01','2025-07-02']
236     labels=['03-12 常规日','08-25 高更新日','07-01 高调整成本日','07-02 高风险日']
237     for j,(parameter,xlabel,levels,baseline) in enumerate([
238         ('S','场景数 S',[10,20,28],20),('lambda_E',r'终端价值系数  $\lambda_E$ ',[.8,1,1.2],1),('S_tail',
239         r'尾部场景数  $S_{tail}$ ',[0,2,4],2)]:
240         for ax in axes[:,j]:
241             ax.axvline(baseline,color=GRAY,ls=':',lw=1,alpha=.65);ax.set_xticks(levels);ax.
242             set_xlabel(xlabel);clean(ax)
243             for i,(day,label,color,style,marker) in enumerate(zip(dates,labels,COLORS,STYLES,MARKERS)):
244                 s=f[f.parameter.eq(parameter)&f.date.eq(day)].sort_values('value')
245                 for row,ykey in [(0,'delta_cost_pct'),(1,'final_soc')]:
246                     ax=axes[row,j];ax.plot(s.value,s[ykey],color=color,ls=style,lw=1.5,label=label)
247                     for point in s.itertuples():
248                         ax.plot(point.value,getattr(point,ykey),marker=marker,ms=5.6,color=color,
249                             markerfacecolor=color if point.reliable else 'white',markeredgewidth
250                             =1.2)
251                     axes[0,j].axhline(0,color=GRAY,lw=.7)
252                     for level in [1200,10800]:axes[1,j].axhline(level,color=GRAY,ls='--',lw=.7,alpha=.7)
253                     axes[1,j].set_ylim(700,11300);axes[1,j].set_yticks([1200,6000,10800])
254                     title(axes[0,j],f'({chr(97+j)}) '+'场景规模','终端库存价值','尾部代表性'][j])
255                     title(axes[1,j],f'({chr(100+j)}) 日末储电量')
256                 axes[0,0].set_ylabel('相对基准日费用变化 / %')
257                 axes[1,0].set_ylabel('日末储电量 / kWh')
258                 from matplotlib.lines import Line2D
259                 handles=[Line2D([],[],color=c,ls=s,marker=m,markerfacecolor='white',label=l) for c,s,m,l in zip(
260                     COLORS,STYLES,MARKERS,labels)]
261                 fig.legend(handles=handles,loc='upper center',bbox_to_anchor=(.54,.99),ncol=4,fontsize=9,
262                     handlelength=2.5)
263                 fig.text(.54,.905,'4代表日局部实验 • 空心: 存在未认证节点 / 实心: 该日全部节点达到3%证书',ha='center',
264                     ,fontsize=9,color=GRAY)
265                 fig.text(.54,.03,'日末库存随参数改变: 较低当日费用不能单独解释为跨日更优或全年稳健.',ha='center',fontsize
266                     =9,color=GRAY)
267                 save(fig,'figS3_1_local_sensitivity','附图S3-1 场景与终端价值的局部敏感性',
268                     '复用已完成Q3四代表日实验, 每列三水平均采用同日日初SOC和固定28条真实历史池, S按用户确认取10/20/28。尾部数0
269                     /2/4, 终端系数0.8/1/1.2。虚竖线为基准, 费用变化按同日基准分母计算; 点间连线只辅助阅读, 不代表中间参数已计算。空
270                     心点存在未认证节点, 实心为四节点均达到3%证书。三个因素共24变体+12基准记录, 22变体存在未认证节点, 四基准日亦未全部
271                     认证。终端库存不同, 不能直接外推全年经济性。',
272                     ['local_sensitivity.csv'])
273
274
275 def main():
276     global DATA,OUT
277     parser=argparse.ArgumentParser();parser.add_argument('--data-dir',type=Path,default=DATA);parser
278     .add_argument('--output-dir',type=Path,default=OUT)
279     args=parser.parse_args();DATA=args.data_dir.resolve();OUT=args.output_dir.resolve();OUT.mkdir(
280         parents=True,exist_ok=True)
281     with plt.rc_context(STYLE):
282         for fn in [forecast_quality,timeline,updates,replay,costs,annual_maps,sensitivity]:fn()
283         (OUT/'manifest.json').write_text(json.dumps(MANIFEST,ensure_ascii=False,indent=2),encoding='utf
284             -8')
285         (OUT/'captions.md').write_text('# 第三问科研图图注\n\n'+'\n\n'.join('## '+r['title']+'\n\n'+r['

```

```

caption'] for r in MANIFEST)+'\n',encoding='utf-8')
273 cards=''.join(f'<article><h2>{html.escape(r["title"])}</h2><a href="{r["png"]}"></a><p>{html.escape(r["caption"])}</p><a href="{r["svg"]}">SVG 矢量图</a> • <a href="{r["png"]}">PNG 原图</a></article>' for r in MANIFEST)
274 page='''<!doctype html><html lang="zh-CN"><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1"><title>第三问科研图 • 蓝色渐变</title><style>body{margin:0;background:#edf3f8;color:#103b62;font:16px/1.65 -apple-system,BlinkMacSystemFont,"PingFang SC",sans-serif}main{max-width:1160px;margin:auto;padding:30px 24px}article{background:#fff;padding:24px;margin:24px 0;border-radius:12px}h1{font-size:28px}h2{font-size:20px}img{width:100%;height:auto}p{color:#62778a;font-size:14px}a{color:#1263a0}</style><main><h1>第三问 • 预测更新与滚动调度</h1><p>6张正文图 + 1张敏感性附图 • 多强度蓝色 • 400dpi PNG / SVG • 已有真实结果</p>''' + cards + '</main></html>'
275 (OUT/'index.html').write_text(page,encoding='utf-8')
276
277
278 if __name__=='__main__':main()

```

src/plots/q3_paper_package.py

```

1 """三核心图、四扩展图及只读数据的独立ZIP交付。"""
2 from pathlib import Path
3 import csv
4 import hashlib
5 import html
6 import json
7 import shutil
8 import zipfile
9 from .q3_paper_data import ROOT,OUT,DATA
10
11 PACKAGE=ROOT/'outputs/processed/deliverables/问题三科研图_蓝色渐变完整版_20260913'
12 DOCS=ROOT/'docs/plots/q3_paper_final'
13
14
15 def prepare_package():
16     PACKAGE.mkdir(parents=True,exist_ok=True)
17     core=json.loads((OUT/'core/manifest.json').read_text(encoding='utf-8'))
18     extra={r['name']:r for r in json.loads((OUT/'manifest.json').read_text(encoding='utf-8'))}
19     selection=[(r,OUT/'core','核心图',name) for r,name in zip(core,['图6-1_预报误差与更新增益','图6-2_滚动窗口与承诺机制','图6-3_代表日预报购电储能'])]
20     for key,name in [('fig6_3_forecast_and_plan_updates','A1_高更新日计划修订'),('fig6_4_pressure_day_dispatch','A2_压力日储能分轴'),('fig6_5_cost_structure','A3_月度费用结构'),('fig6_6_annual_storage_and_revisions','A4_全年储能与购电修订')]:
21         selection.append((extra[key],OUT,'扩展备选图',name))
22     manifest=[];registry=[]
23     for row,folder,group,name in selection:
24         target=PACKAGE/group;target.mkdir(exist_ok=True)
25         item={**row,'title':row['title'] if group=='核心图' else '备选'+name.replace('_',' ')}
26         for suffix in ['png','svg']:
27             shutil.copy2(folder/row[suffix],target/f'{name}.{suffix}')
28             item[suffix]=f'{group}/{name}.{suffix}'
29         manifest.append(item)
30         registry.append({**row,'title':item['title'],'png':str((folder/row['png']).relative_to(OUT)),
31             'svg':str((folder/row['svg']).relative_to(OUT))})
32     (OUT/'delivery_manifest.json').write_text(json.dumps(registry,ensure_ascii=False,indent=2),encoding='utf-8')

```

```

33 (PACKAGE/'manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding='
    utf-8')
34 shutil.copytree(DATA,PACKAGE/'data',dirs_exist_ok=True)
35 for name in ['第三问科研图方案.md','第三问_必须绘制的核心图及绘图细节.md','图后分析与论文插入建议.md','
    acceptance.md']:
36     shutil.copy2(DOCS/name,PACKAGE/name)
37 shutil.copy2(OUT/'verification.json',PACKAGE/'verification.json')
38 sources=PACKAGE/'src/plots';sources.mkdir(parents=True,exist_ok=True)
39 for name in ['common.py','q3_paper_final.py','q3_paper_core.py','q3_paper_data.py','
    q3_paper_verify.py','q3_paper_package.py']:
40     shutil.copy2(ROOT/'src/plots'/name,sources/name)
41 for path in [PACKAGE/'src/___init___py',sources/'___init___py']:path.write_text('',encoding='utf-8
    ')
42 (PACKAGE/'requirements.txt').write_text('numpy==2.3.5\npandas==2.2.3\nmatplotlib==3.10.8\nPillow
    ==12.3.0\nopenpyxl==3.1.5\n',encoding='utf-8')
43 (PACKAGE/'captions.md').write_text('# 交付图注\n\n'+'\n\n'.join('## '+r['title']+'\n\n'+r[
    'caption'] for r in manifest)+'\n\n',encoding='utf-8')
44 readme='''# 第三问科研图 ZIP
45
46 包含3张正文核心图和4张扩展备选图，每张提供400dpi PNG及SVG。主色为多强度蓝色；核心图按补充文档改为宋体与Times New
    Roman。
47
48 **图6-4（FIV/OUV）经用户明确要求取消。** 当前没有全年配对反事实数据，本包没有用预测误差、调整费用或示意数据替代，也没
    有为此启动新实验。
49
50 ## 文件内容
51
52 | 路径 | 内容 |
53 |---|---|
54 | 核心图/ | 图6-1同目标预报误差增益、图6-2 B/P/C滚动窗口、图6-3同日预报一购电一储能三联图；正文优先使用 |
55 | 扩展备选图/ | A1高更新日实例、A2压力日分轴、A3月度费用、A4全年热图；可选用，不建议全塞正文 |
56 | 第三问科研图方案.md | 最终图组方案、客观选日规则、插入位置与统计范围 |
57 | 第三问_必须绘制的核心图及绘图细节.md | 用户提供的原始清单原样保存；其中图6-4被后续用户回复取消 |
58 | 图后分析与论文插入建议.md | 每张图的“看到什么—如何解释—支持什么”文字 |
59 | captions.md | 7张交付图的完整图注 |
60 | data/forecast_*.csv | 333共同日的31968条小时预报/真值配对与96格MAE/RMSE/Bias |
61 | data/revision_*.csv | 每节点1998条同目标旧/新配对与统计，18点近零绝对量保留 |
62 | data/typical_*.csv | 代表日144时段、145点SOC、当块小时预报及四日距离/标准化尺度 |
63 | data/pressure_*.csv、update_*.csv | 两个扩展实例日的真实运行、原始预报与节点政策 |
64 | data/formal_dispatch.csv | 正式334日×144段全量实际执行，含SOC、费用、g0与最终grid差值 |
65 | data/formal_daily.csv、monthly_costs.csv | 日度与月度结算汇总 |
66 | data/formal_solver_quality.csv | 1336节点求解状态；847认证、489限时可行 |
67 | data/local_sensitivity.csv | 已完成Q3三因素、四代表日的36条记录，含12条基准；按新清单以表交付，不额外堆敏感性图
    |
68 | data/execution_config.json、provenance.json | 正式模型配置、来源及341项源文件SHA-256 |
69 | src/plots/ | 绘图、数据准备、核验及打包源码，均UTF-8 |
70 | requirements.txt | 原绘图环境版本 |
71 | manifest.json、verification.json、acceptance.md | 图目与文件映射、数值/导出校验、逐条需求验收 |
72 | index.html | 解压后可直接打开的离线图集 |
73 | file_manifest.csv | 包内逐文件SHA-256，校验运输完整性 |
74
75 ## 口径与使用
76
77 正式费用及热图仅含2025-02-01至12-31共334日。预测比较采用共同333日起点；小时目标与真实小时右端点一致。核心图6-3日期3
    月20日由指定四日与全年中位运行状态的距离客观选择。
78
79 所有图展示已执行可行结果，存在489个未达到3%证书的节点，不宣称全节点严格最优。负调整费用是结算返还，不是总利润。18点夜间

```

光伏近零，预测相对改善不能直接解释为经济信息价值。局部S保持用户批准的10/20/28，终端库存差异不能忽略。

PNG可直接插入文档，SVG文字已转曲，可缩放；SVG热图矩阵按原像素嵌入，文字及线条为矢量。请在论文最终插入尺寸下检查字号，避免过度缩小多面板图。

从本包数据重绘

在解压目录安装requirements.txt所列库后运行（不调用求解器）：

```
```sh
python -m src.plots.q3_paper_core --data-dir data --output-dir redraw/core
python -m src.plots.q3_paper_final --data-dir data --output-dir redraw/extended
```
```

核心图重绘需要本机安装Times New Roman与宋体Songti SC；现成PNG/SVG无需字体。扩展脚本会复现最初设计的七张图，实际交付只选A1 - A4，映射见manifest；不据此恢复已取消的FIV/OUV图。

q3_paper_data和q3_paper_verify用于完整项目环境中的原始文件追溯，需原项目数据路径；本包足够离线重绘，无需重新优化。正式结果与之前交付ZIP均未覆盖。

```
'''
(PACKAGE/'README.md').write_text(readme,encoding='utf-8')
cards=''
for item in manifest:
    cards+=f'<article><h2>{html.escape(item["title"])}</h2><a href="{item["png"]}"></a><p>{html.escape(item["caption"])}</p><p><a
href="{item["svg"]}">SVG矢量文件</a> • <a href="{item["png"]}">PNG原图</a></p></article>'
page='''<!doctype html><html lang="zh-CN"><meta charset="utf-8"><meta name="viewport" content="
width=device-width,initial-scale=1"><title>第三问科研图 • 最终交付</title><style>body{margin:0;
color:#103b62;background:#edf3f8;font:16px/1.6 -apple-system,BlinkMacSystemFont,"PingFang SC",
sans-serif}main{max-width:1120px;margin:auto;padding:25px 18px}article{background:white;border-
radius:12px;padding:24px;margin:24px 0}img{width:100%;height:auto}h1{font-size:27px}h2{font-size
:20px}p{color:#62778a;font-size:14px}a{color:#1263a0}</style><main><h1>第三问科研图 • 蓝色渐变</h1
><p>3张正文核心图 + 4张扩展备选图 • 400dpi PNG / SVG</p><p>图6-4（FIV/OUV）已按用户要求取消。运行与预测统
计范围见各图图注。</p><p><a href="README.md">README</a> • <a href="第三问科研图方案.md">最终方案</a>
• <a href="图后分析与论文插入建议.md">图后分析</a></p>''' + cards + '</main></html>'
(PACKAGE/'index.html').write_text(page,encoding='utf-8')
print('Prepared',PACKAGE)

def archive():
    rows=[]
    for p in sorted(PACKAGE.rglob('*')):
        if p.is_file() and p.name!='file_manifest.csv':
            rows.append({'path':str(p.relative_to(PACKAGE)),'bytes':p.stat().st_size,'sha256':
            hashlib.sha256(p.read_bytes()).hexdigest()})
    with (PACKAGE/'file_manifest.csv').open('w',encoding='utf-8',newline='') as stream:
        w=csv.DictWriter(stream,fieldnames=['path','bytes','sha256']);w.writeheader();w.writerows(
        rows)
    zip_path=PACKAGE.with_suffix('.zip')
    files=[p for p in sorted(PACKAGE.rglob('*')) if p.is_file()]
    with zipfile.ZipFile(zip_path,'w',compression=zipfile.ZIP_DEFLATED,compresslevel=6) as z:
        for p in files:z.write(p,str(Path(PACKAGE.name)/p.relative_to(PACKAGE)))
    with zipfile.ZipFile(zip_path) as z:
        assert z.testzip() is None and len(z.namelist())==len(files)
        for p in files:
            assert hashlib.sha256(z.read(str(Path(PACKAGE.name)/p.relative_to(PACKAGE)))).digest()==
            hashlib.sha256(p.read_bytes()).digest()
```

```

120 receipt={'zip':str(zip_path),'bytes':zip_path.stat().st_size,'members':len(files),'figures':7,'
    PNG':7,'SVG':7,
121         'sha256':hashlib.sha256(zip_path.read_bytes()).hexdigest(),'CRC_and_member_SHA256':'passed'}
122 (DOCS/'zip_acceptance.json').write_text(json.dumps(receipt,ensure_ascii=False,indent=2),encoding
    ='utf-8')
123 print(receipt)
124
125
126 if __name__=='__main__':
127     import sys
128     if '--archive' in sys.argv:archive()
129     else:prepare_package()

```

src/plots/q3_paper_verify.py

```

1  """交付前独立复核：统计配对、结算、库存、原件哈希与图文件。"""
2  from pathlib import Path
3  import hashlib
4  import json
5  import xml.etree.ElementTree as ET
6  import numpy as np
7  import pandas as pd
8  from PIL import Image
9  from .q3_paper_data import ROOT,OUT,DATA
10
11
12 def check():
13     read=lambda name:pd.read_csv(DATA/name,float_precision='round_trip')
14     p=json.loads((DATA/'provenance.json').read_text(encoding='utf-8'))
15     for name,digest in p['source_sha256'].items():
16         assert hashlib.sha256((ROOT/name).read_bytes()).hexdigest()==digest,name
17     f=read('forecast_pairs.csv');m=read('forecast_metrics.csv');r=read('revision_pairs.csv')
18     assert f.groupby(['date','issue_hour']).size().eq(24).all() and f.date.nunique()==333
19     assert not f.duplicated(['issue','target']).any()
20     for row in m.itertuples():
21         s=f[(f.issue_hour==row.issue_hour)&(f.horizon_hours==row.horizon_hours)]
22         e=(s.forecast_kw-s.actual_kw).to_numpy()
23         np.testing.assert_allclose([abs(e).mean(),np.sqrt((e*e).mean()),e.mean()], [row.MAE,row.RMSE,
            row.Bias],atol=1e-10,rtol=0)
24     old=f[['issue','target','forecast_kw']].copy();old['issue']=(pd.to_datetime(old.issue)+pd.
        Timedelta(hours=6)).astype(str)
25     pair=r.merge(old,on=['issue','target'],suffixes=('', '_previous'),validate='one_to_one')
26     np.testing.assert_allclose(pair.old_forecast_kw,pair.forecast_kw_previous,atol=0,rtol=0)
27     assert r.groupby('issue_hour').size().eq(1998).all()
28     d=read('formal_dispatch.csv');source=pd.read_csv(ROOT/'outputs/q3/raw/
        full_priority_rescue_7h_20260912/main/dispatch.csv',float_precision='round_trip')
29     pd.testing.assert_frame_equal(d[source.columns],source,check_exact=False,atol=1e-10,rtol=0)
30     dg=d.grid-d.g0;adjust=d.price*(1.5*dg.clip(lower=0)+.5*dg.clip(upper=0))
31     np.testing.assert_allclose(d.adjustment_cost,adjust,atol=1e-8,rtol=0)
32     balance=d.grid+d.discharge+d.emergency-d.net_kwh-d.charge-d.spill
33     np.testing.assert_allclose(balance,0,atol=1e-8,rtol=0)
34     for date,s in d.groupby('date'):
35         e=np.r_[s.initial_soc.iloc[0],s.soc]
36         np.testing.assert_allclose(np.diff(e),.9*s.charge-s.discharge/.9,atol=1e-8,rtol=0)
37         assert (e>=1200-1e-6).all() and (e<=10800+1e-6).all()
38         assert np.array_equal(s.slot,np.arange(144))

```

```

39 daily=read('formal_daily.csv');np.testing.assert_allclose(daily.soc_start.iloc[1:],daily.soc_end
    .iloc[:-1],atol=1e-8,rtol=0)
40 costs=['planned_cost','adjustment_cost','emergency_cost','total_cost']
41 np.testing.assert_allclose(d.groupby('date')[costs].sum(),daily[costs],atol=1e-6,rtol=0)
42 baseline=daily.set_index('date')
43 sensitivity=read('local_sensitivity.csv')
44 for s in sensitivity.itertuples():
45     expected=(s.total_cost/baseline.loc[s.date,'total_cost']-1)*100
46     np.testing.assert_allclose(s.delta_cost_pct,expected,atol=1e-9,rtol=0)
47 typical=read('typical_dispatch.csv');c=read('typical_candidates.csv');scale=read('
    typical_selection_scale.csv').set_index('feature')
48 values=c[scale.index].to_numpy();dist=np.sqrt((((values-scale.formal_median.to_numpy())/scale.
    formal_IQR.to_numpy())**2).sum(axis=1))
49 np.testing.assert_allclose(c.median_state_distance,dist,atol=1e-10,rtol=0)
50 assert typical.date.nunique()==1 and typical.date.iloc[0]==c.date.iloc[dist.argmin()]
51 tf=read('typical_forecasts_active.csv');assert tf.groupby('issue_hour').size().eq(6).all()
52 assert ((tf.target_hour>tf.issue_hour)&(tf.target_hour<=tf.issue_hour+6)).all()
53 quality=read('formal_solver_quality.csv');assert len(quality)==1336 and quality.reliable.sum()
    ==847
54 images=[]
55 manifests=[(OUT/'manifest.json',None),(OUT/'core/manifest.json',None)]
56 for path,_ in manifests:
57     for item in json.loads(path.read_text(encoding='utf-8')):
58         image_path=path.parent/item['png'];svg=path.parent/item['svg']
59         with Image.open(image_path) as im:
60             assert im.format=='PNG' and all(abs(x-400)<.05 for x in im.info['dpi'])
61             expected=np.array([item['width_inches'],item['height_inches']])*400
62             assert np.max(abs(np.asarray(im.size)-expected))<=1.01
63             size=im.size
64             xml=ET.parse(svg);assert not xml.findall('.://{http://www.w3.org/2000/svg}text')
65             images.append({'name':item['name'],'pixels':size,'dpi':400,'svg_text':'glyph paths'})
66 for name in ['core_render.log','extended_render.log']:
67     log=(ROOT/'docs/plots/q3_paper_final'/name).read_text(encoding='utf-8')
68     assert all(x not in log.lower() for x in ['glyph','warning','dummy','traceback'])
69 report={'status':'passed','source_hashes_verified':len(p['source_sha256']),'forecast_cells':96,'
    paired_days':333,
70         'formal_days':334,'formal_slots':len(d),'formal_nodes':1336,'certified_nodes':847,'
    limited_nodes':489,
71         'typical_date':typical.date.iloc[0],'user_cancelled':'FIV/OUV图6-4: 未运行新实验',
72         'figures_rendered_and_checked':images,'max_energy_balance_residual':float(abs(balance).max()
    ),
73         'all_original_results_unchanged':True,'scope':'numerical/source/export verification; visual
    QA separately documented'}
74 (OUT/'verification.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='
    utf-8')
75 print({k:v for k,v in report.items() if k!='figures_rendered_and_checked'})
76
77
78 if __name__=='__main__':check()

```

src/plots/q4_paper_data.py

```

1 """Q4科研六图的真实数据与来源；不调用模型求解器。"""
2 from pathlib import Path
3 import hashlib,json
4 import numpy as np

```



```

5 import pandas as pd
6
7 ROOT=Path(__file__).resolve().parents[2]
8 OUT=ROOT/'outputs/processed/figures/q4_paper_final_20260913'
9 DATA=OUT/'data'
10 RUNS={'4-2':ROOT/'outputs/q4/raw/rescue_lp_q42_20260913_v2', '4-3':ROOT/'outputs/q4/raw/
    rescue_lp_q43_20260913_v3'}
11
12
13 def write(name,frame):
14     frame.to_csv(DATA/name,index=False,encoding='utf-8',float_format='%.17g')
15
16
17 def prepare():
18     DATA.mkdir(parents=True,exist_ok=True);frames={};daily={};sources=[];quality=[];flow=[];
    summaries=[]
19     for mode,path in RUNS.items():
20         for name in ['dispatch.csv','daily.csv','state.json','metrics.json','launch.json','result'+
    mode+'.xlsx']:sources.append(path/name)
21         f=pd.read_csv(path/'dispatch.csv',float_precision='round_trip');d=pd.read_csv(path/'daily.
    csv',float_precision='round_trip')
22         assert len(f)==48096 and len(d)==334 and d.date.iloc[0]=='2025-02-01' and d.date.iloc[-1]=='
    2025-12-31'
23         assert f.initial_soc.iloc[0]==6000 and f.groupby('date').size().eq(144).all()
24         np.testing.assert_allclose(f.called+f.discharge+f.emergency,f.net_kwh+f.charge+f.spill,atol
    =1e-7,rtol=0)
25         np.testing.assert_allclose(f.grid,f.called+f.unused,atol=1e-7,rtol=0)
26         np.testing.assert_allclose(f.soc-np.r_[6000,f.soc.to_numpy()[:-1]],.9*f.charge-f.discharge
    /.9,atol=1e-7,rtol=0)
27         delta=f.grid-f.g0;adjust=f.price*(1.5*np.maximum(delta,0)-.5*np.maximum(-delta,0))
28         np.testing.assert_allclose(f.adjustment_cost,adjust,atol=1e-7,rtol=0)
29         np.testing.assert_allclose(f.total_cost,f.price*f.g0+adjust+5*f.price*f.emergency,atol=1e-7,
    rtol=0)
30         f['hour_right']=(f.slot+1)/6;f['delta_grid_kwh']=delta
31         write(f'dispatch_{mode}.csv',f);write(f'daily_{mode}.csv',d)
32         write(f'riskday_{mode}.csv',f[f.date=='2025-07-01'])
33         day=f[f.date=='2025-07-01'];write(f'riskday_soc_{mode}.csv',pd.DataFrame({'hour':np.arange
    (145)/6,'soc_kwh':np.r_[day.initial_soc.iloc[0],day.soc]}))
34         for date in d.date:
35             audit_path=path/'audit'/f'{date}.json';sources.append(audit_path);a=json.loads(
    audit_path.read_text(encoding='utf-8'))
36             for node in [a] if mode=='4-2' else a['nodes']:
37                 s=node['solver'];assert s['status']=='Optimal' and s['integer_variables']==0 and s['
    reliable']
38                 sc=node['scenarios'];quality.append({'mode':mode,'date':date,'hour':node.get('hour'
    ,0),'status':s['status'],
39                 'seconds':s['seconds'],'gap':s['gap'],'S':sc['S'],'rho':sc['radius'],'ESS':sc.
    get('ESS'),
40                 'initial_soc':node['initial_soc'],'committed_slots':node['committed_slots']})
41             energy={'pv':float(f.pv_kw.sum()/6),'called':float(f.called.sum()),'emergency':float(f.
    emergency.sum()),
42                 'discharge':float(f.discharge.sum()),'load':float(f.load_kw.sum()/6),'charge':float(
    f.charge.sum()),'spill':float(f.spill.sum())}
43             np.testing.assert_allclose(sum(energy[k] for k in ['pv','called','emergency','discharge']),
    sum(energy[k] for k in ['load','charge','spill']),atol=1e-6,rtol=0)
44             flow.extend({'mode':mode,'side':'input' if key in ['pv','called','emergency','discharge']
    else 'output','key':key,'kwh':v} for key,v in energy.items())

```



```

45     summaries.append({'mode':mode,**{k:float(f[k].sum()) for k in ['grid','called','unused','
charge','discharge','emergency','spill','planned_cost','adjustment_cost','emergency_cost','
total_cost']}},
46                     'initial_soc':6000., 'final_soc':float(f.soc.iloc[-1]), 'annual_days':len(d)
47     })
48     frames[mode]=f;daily[mode]=d
49     first,second=frames['4-2'],frames['4-3']
50     for key in ['date','slot','price','price_forecast','load_kw','pv_kw']:
51         pd.testing.assert_series_equal(first[key],second[key],check_names=False)
52     write('solver_quality.csv',pd.DataFrame(quality));write('energy_flow.csv',pd.DataFrame(flow));
53     write('strategy_summary.csv',pd.DataFrame(summaries))
54     hourly=first.assign(hour=first.slot//6).groupby(['date','hour'])[['price','price_forecast']].
55     mean().reset_index()
56     write('hourly_prices.csv',hourly)
57     clock=[]
58     for hour,g in hourly.groupby('hour'):
59         clock.append({'hour':int(hour), 'n_days':len(g), 'actual_median':g.price.median(), 'actual_p10':
60         :g.price.quantile(.1),
61                     'actual_p90':g.price.quantile(.9), 'forecast_median':g.price_forecast.median()
62         })
63     write('price_clock.csv',pd.DataFrame(clock))
64     paired=daily['4-2'][['date','total_cost','soc_end']].merge(daily['4-3'][['date','total_cost','
65     soc_end']],on='date',suffixes=('_q42','_q43'),validate='one_to_one')
66     paired['saving_yuan']=paired.total_cost_q42-paired.total_cost_q43
67     write('daily_cost_pairs.csv',paired)
68     senspath=R00T/'outputs/processed/sensitivity_q4/daily_results.csv';sens=pd.read_csv(senspath,
69     float_precision='round_trip');sources.append(senspath)
70     assert len(sens)==30 and sens.all_optimal.all();write('local_sensitivity.csv',sens)
71     for mode in RUNS:
72         cols=['total_cost','planned_cost','adjustment_cost','emergency_cost'];f=frames[mode];d=daily
73         [mode]
74         np.testing.assert_allclose(f.groupby('date')[cols].sum(),d[cols],atol=1e-6,rtol=0)
75         err=first.price-first.price_forecast
76         config=json.loads((RUNS['4-3']/'state.json').read_text(encoding='utf-8'))['config']
77         source_theme=R00T/'docs/plots/q4_paper_final/ThemeDisplay.html';sources.append(source_theme)
78         audit={'runs':{m:str(p.relative_to(R00T)) for m,p in RUNS.items()}, 'days':334, '
79         slots_per_strategy':48096, 'optimal_nodes':len(quality),
80                 'source_sha256':{str(p.relative_to(R00T)):hashlib.sha256(p.read_bytes()).hexdigest() for
81                 p in sources}, 'riskday':'2025-07-01',
82                 'riskday_rule':'Reuse the high joint-risk day selected by the prior Q4 sensitivity
83                 protocol',
84                 'price_MAE':float(abs(err).mean()), 'price_RMSE':float(np.sqrt(np.mean(err*err))),
85                 'total_saving_yuan':float(paired.saving_yuan.sum()), 'saving_day_fraction':float((paired.
86                 saving_yuan>0).mean()),
87                 'config':config, 'fonts':'FandolSong-Regular + Latin Modern Roman + Computer Modern math;
88                 local TeX Live 2026',
89                 'palette':{'Blue Lotus':['#19309A','#4B69EF','#7E96FE','#B6D8F7','#FFB967'],'月白深藍':['#
90                 FFFEF0','#CDBFE2','#002254']}}
91     (DATA/'provenance.json').write_text(json.dumps(audit,ensure_ascii=False,indent=2),encoding='utf
92     -8')
93     print({k:v for k,v in audit.items() if k not in ['source_sha256','config','palette']})
94
95 if __name__=='__main__':prepare()

```

src/plots/q4_paper_final.py

```

1  """Q4六张图：蓝莲花渐变、TeX中文字体、真实运行数据。"""
2  from pathlib import Path
3  import argparse,json,warnings,html
4  import numpy as np
5  import pandas as pd
6  import matplotlib
7  matplotlib.use('Agg')
8  import matplotlib.pyplot as plt
9  from matplotlib import font_manager
10 from matplotlib.colors import LinearSegmentedColormap,LogNorm,to_rgb
11 from matplotlib.patches import Rectangle,FancyBboxPatch,FancyArrowPatch,PathPatch
12 from matplotlib.path import Path as MPath
13 from matplotlib.text import Text
14
15 ROOT=Path(__file__).resolve().parents[2]
16 OUT=ROOT/'outputs/processed/figures/q4_paper_final_20260913';DATA=OUT/'data'
17 INK='#002254';DARK='#19309A';BLUE='#4B69EF';PERI='#7E96FE';ICE='#B6D8F7';APRICOT='#FFB967';LILAC='#
    CDBFE2';CREAM='#FFFEF0';GRAY='#64748B'
18 SEQ=LinearSegmentedColormap.from_list('blue_lotus',['#FBFCFF',ICE,PERI,BLUE,DARK,INK])
19 STYLE={'font.family':['Latin Modern Roman','FandolSong'],'font.size':11,'axes.titlesize':12,'axes.
    labelsiz':10.5,
20         'xtick.labelsize':10,'ytick.labelsize':10,'legend.fontsize':9,'legend.frameon':False,'axes.
    titleweight':'regular',
21         'axes.spines.top':False,'axes.spines.right':False,'axes.edgecolor':'#A0AEC0','axes.linewidth'
    :.7,
22         'axes.labelcolor':INK,'text.color':INK,'xtick.color':GRAY,'ytick.color':GRAY,
23         'axes.axisbelow':True,'axes.unicode_minus':False,'grid.color':'#DDE5F1','grid.alpha':.75,'
    grid.linewidth':.55,
24         'mathtext.fontset':'cm','svg.fonttype':'path','figure.facecolor':'white','savefig.facecolor':
    'white',
25         'figure.constrained_layout.h_pad':.09,'figure.constrained_layout.w_pad':.07}
26 MANIFEST=[]
27
28
29 def fonts():
30     available=[ROOT/'assets/q4_fonts',Path('/usr/local/texlive/2026/texmf-dist/fonts/opentype/public
    ')]
31     for filename,sub in [('FandolSong-Regular.otf','fandol'),('lmroman10-regular.otf','lm')]:
32         paths=[available[0]/filename,available[1]/sub/filename]
33         p=next((p for p in paths if p.exists()),None)
34         if p is None:raise RuntimeError('缺少TeX字体: '+filename)
35         font_manager.fontManager.addfont(str(p))
36
37
38 def read(name):return pd.read_csv(DATA/name,float_precision='round_trip')
39 def title(ax,text,pad=10):ax.set_title(text,loc='left',pad=pad)
40 def clean(ax):ax.grid(True,axis='y');ax.grid(False,axis='x')
41 def timeaxis(ax):ax.set_xlim(0,24);ax.set_xticks([0,6,12,18,24])
42 def nodes(ax):
43     for h in [6,12,18]:ax.axvline(h,color=GRAY,ls=':',lw=.8,alpha=.65)
44
45
46 def gradient_rect(ax,x,y,width,height,color,orientation='vertical',alpha=1):
47     """矩形真实尺寸不变，仅将填色从浅色渐变到指定色。"""
48     lo,hi=sorted([y,y+height]);light=tuple(.75+.25*np.asarray(to_rgb(color)))

```

```

49 cm=LinearSegmentedColormap.from_list('bar_tint',[light,color])
50 values=np.linspace(0,1,256).reshape(-1,1) if orientation=='vertical' else np.linspace(0,1,256)[
    None]
51 ax.imshow(values,extent=(x,x+width,lo,hi),origin='lower',aspect='auto',cmap=cm,alpha=alpha,
    interpolation='bicubic',zorder=2)
52 ax.add_patch(Rectangle((x,lo),width,hi-lo,facecolor='none',edgecolor=color,lw=.7,zorder=3))
53
54
55 def save(fig,name,text,caption,data):
56     for t in fig.findobj(Text):
57         value=t.get_text()
58         if '$' in value and any('\u4e00'<=c<='\u9fff' for c in value):t.set_fontfamily('FandolSong')
59     with warnings.catch_warnings():
60         warnings.filterwarnings('error',message='Glyph .* missing from font')
61         fig.canvas.draw()
62         for suffix in ['png','svg']:fig.savefig(OUT/f'{name}.{suffix}',dpi=400,facecolor='white',
            transparent=False)
63     MANIFEST.append({'question':'q4','name':name,'title':text,'caption':caption,'data':data,'png':
        name+'.png','svg':name+'.svg',
64                     'width_inches':fig.get_figwidth(),'height_inches':fig.get_figheight(),'dpi'
        :400,
65                     'font':'FandolSong / Latin Modern Roman / Computer Modern math','script':'src/
        plots/q4_paper_final.py',
66                     'necessity':'问题四科研图，正文或补充选用'})
67     plt.close(fig);print(name,flush=True)
68
69
70 def mechanism():
71     fig,ax=plt.subplots(figsize=(10.1,6.4));fig.subplots_adjust(left=.018,right=.985,top=.97,bottom
        =.025)
72     ax.set(xlim=(0,1),ylim=(0,1));ax.axis('off')
73     def box(x,y,w,h,text,color=BLUE,size=10):
74         clip=FancyBboxPatch((x,y),w,h,boxstyle='round,pad=.008,rounding_size=.012',edgecolor=color,
            lw=.85,facecolor='none',zorder=3);ax.add_patch(clip)
75         cm=LinearSegmentedColormap.from_list('box',['#FAFBFF',tuple(.88+.12*np.asarray(to_rgb(color)
            ))])
76         im=ax.imshow(np.linspace(0,1,120)[None],extent=(x-.008,x+w+.008,y-.008,y+h+.008),origin='
            lower',aspect='auto',cmap=cm,zorder=1);im.set_clip_path(clip)
77         ax.text(x+w/2,y+h/2,text,ha='center',va='center',fontsize=size,color=INK,zorder=4)
78     def arrow(a,b,color=BLUE,ls='-'):
79         ax.add_patch(FancyArrowPatch(a,b,arrowstyle='->',mutation_scale=11,color=color,lw=1.05,ls=
            ls,zorder=4))
80     ax.text(.02,.955,'严格因果的联合不确定性',fontsize=13,color=DARK)
81     box(.025,.81,.24,.09,'0点冻结: 负荷与48h电价预测\n节点更新: 最新PV、已实现信息',size=9.5)
82     box(.335,.81,.27,.09,'同一历史日的净负荷—电价残差\n56日窗口 • 联合距离 • 尾部保留',size=9.5)
83     box(.675,.81,.29,.09,'Wasserstein DR0\n经验概率 + 标定半径 → 最坏期望',color=DARK,size=10)
84     arrow((.274,.855),(.32,.855));arrow((.61,.855),(.66,.855))
85     ax.text(.04,.726,'4-2 日前机制',fontsize=12,color=BLUE)
86     ax.text(.54,.726,'4-3 四时点机制',fontsize=12,color=DARK)
87     box(.03,.445,.43,.24,'',color=BLUE);box(.53,.445,.43,.24,'',color=DARK)
88     ax.text(.245,.642,'00:00 优化未来48 h',ha='center',fontsize=11)
89     ax.text(.745,.642,'00 / 06 / 12 / 18 各优化未来24 h',ha='center',fontsize=11)
90     for x,w,c,label in [(.065,.18,BLUE,'当日24 h 执行'),(.245,.18,ICE,'次日24 h 前瞻')]:
91         ax.add_patch(Rectangle((x,.558),w,.047,facecolor=c,edgecolor=BLUE,lw=.65,zorder=3));ax.text(
            x+w/2,.5815,label,ha='center',va='center',fontsize=8.7,color='white' if c==BLUE else INK,zorder
            =4)
92     for x,w,c,label in [(.565,.09,DARK,'首6 h'),(.655,.27,LILAC,'其余18 h 暂定')]:

```

```

93     ax.add_patch(Rectangle((x,.558),w,.047,facecolor=c,edgecolor=DARK,lw=.65,zorder=3));ax.text(
94     x+w/2,.5815,label,ha='center',va='center',fontsize=9,color='white' if c==DARK else INK,zorder=4)
95 ax.text(.245,.491,'预提交 g / c / r: 当日不再重优化',ha='center',fontsize=10)
96 ax.text(.745,.491,'冻结已执行块; 下一节点继承真实SOC',ha='center',fontsize=10)
97 arrow((.765,.8),(.745,.697),color=DARK)
98 arrow((.73,.788),(.245,.697),color=BLUE)
99 box(.08,.273,.32,.096,'结算账本: 额度 G 与本日基准 $g^0$ \n 实际电价 p · 单次净调整 · 5p 紧急费',color=
100 DARK,size=9.8)
101 box(.59,.273,.32,.096,'物理执行: 冻结 c / r \n 实际源荷只决定调用、紧急与弃电',color=BLUE,size=9.8)
102 ax.plot([.245,.245,.745,.745],[.435,.405,.405,.435],color=GRAY,lw=.9)
103 ax.text(.495,.421,'共同执行与核算',ha='center',fontsize=8.7,color=GRAY)
104 arrow((.495,.404),(.24,.382));arrow((.495,.404),(.75,.382),color=DARK)
105 box(.07,.08,.36,.11,r'$G=x+u$'+'\n 结算额度 = 实际调用 + 未调用',color=DARK,size=11)
106 box(.565,.08,.36,.11,r'$x+r+q^{em}=N+c+w$'+'\n 调用 + 放电 + 紧急 = 净负荷 + 充电 + 弃电',color=BLUE,
107 size=9.5)
108 arrow((.24,.263),(.24,.203));arrow((.75,.263),(.75,.203))
109 ax.text(.5,.025,'未调用额度 u 不进入物理能量流; 实际弃电 w 单独核算。所有时段均保留真实价格。',ha='center',
110 fontsize=9,color=GRAY)
111 save(fig,'fig7_1_joint_dro_mechanism','图7-1 严格因果DRO的双机制决策、结算与执行',
112 '价格与负荷预测在当天0点冻结, 4-3各节点更新PV及截至当前已实现的信息。4-2优化48h并执行首24h; 4-3优化24h并
113 执行首6h。g/c/r均为节点共享计划, 实际充放电按承诺计划执行, 不能画成事后全路径自适应。x=called、u=unused、w=
114 spill; 未调用额度不是弃电。两分支共享方法框架, 各自独立运行SOC链, 箭头不表示4-2结果作为4-3输入。',
115 ['provenance.json','solver_quality.csv'])
116
117 def price_clock():
118     clock=read('price_clock.csv');raw=read('dispatch_4-2.csv');meta=json.loads((DATA/'provenance.
119     json').read_text(encoding='utf-8'))
120     fig=plt.figure(figsize=(9.8,5.2),layout='constrained');gs=fig.add_gridspec(1,2,width_ratios
121     =[1,1.12])
122     ax=fig.add_subplot(gs[0,0],projection='polar');theta=(clock.hour.to_numpy()+.5)*2*np.pi/24
123     xx=np.r_[theta,theta[0]+2*np.pi]
124     close=lambda x:np.r_[x,x[0]]
125     ax.set_theta_zero_location('N');ax.set_theta_direction(-1)
126     ax.fill_between(xx,close(clock.actual_p10.to_numpy()),close(clock.actual_p90.to_numpy()),color=
127     ICE,alpha=.45,lw=0,label='实价10% - 90%跨日带')
128     ax.plot(xx,close(clock.actual_median.to_numpy()),color=DARK,lw=2,label='实际小时均价的中位数')
129     ax.plot(xx,close(clock.forecast_median.to_numpy()),color=BLUE,ls='--',lw=1.6,label='预测小时均价的
130     中位数')
131     ax.set_ylim(0,clock.actual_p90.max()*1.1);ax.set_xticks(np.arange(0,24,3)*2*np.pi/24,[f'{h:02d
132     }:00' for h in range(0,24,3)]);ax.set_rlabel_position(135)
133     ax.spines['polar'].set_color('#C8D5E8');ax.grid(True,color='#CDD8E9',lw=.65)
134     title(ax,'(a) 24小时电价时钟',pad=20)
135     ax.legend(loc='upper center',bbox_to_anchor=(.5,-.23),fontsize=8.8)
136     ax.text(.5,-.17,'半径: 电价 / (元/kWh); 每小时 n=334日',ha='center',transform=ax.transAxes,fontsize
137     =9,color=GRAY)
138     ax=fig.add_subplot(gs[0,1]);limit=max(raw.price.max(),raw.price_forecast.max())*1.025
139     hist=ax.hist2d(raw.price_forecast,raw.price,bins=65,range=[[0,limit],[0,limit]],norm=LogNorm(
140     vmin=1),cmap=SEQ,cmin=1)
141     ax.plot([0,limit],[0,limit],color=GRAY,ls='--',lw=1,label='预测=实际')
142     ax.set(xlim=(0,limit),ylim=(0,limit),xlabel='预测电价 / (元/kWh)',ylabel='实际电价 / (元/kWh)');ax.
143     set_aspect('equal');ax.grid(False)
144     fig.colorbar(hist[3],ax=ax,pad=.02,shrink=.80,label='每格样本数(对数色阶)')
145     title(ax,'(b) 全时段预测—实价密度')
146     ax.text(.04,.97,f'n = 48,096 \n MAE = {meta["price_MAE"]:.4f} \n RMSE = {meta["price_RMSE"]:.4f}',
147     transform=ax.transAxes,ha='left',va='top',fontsize=10,color=INK)
148     save(fig,'fig7_2_price_clock_and_density','图7-2 电价的日内时钟与严格因果预测密度',

```

```

135     '正式334日。左图先对每天每小时的6个10分钟值求均值，再跨日计算中位数与10%-90%分位带；分位带不是置信区间。以
    小时中点定位，首尾连接仅体现日周期统计。右图65×65等宽格统计全部48096点，色阶为对数计数，空格留空，没有裁去极值。
    实价与预测由两个正式策略共同使用，误差基于10分钟原值重算。',
136     ['price_clock.csv', 'hourly_prices.csv', 'dispatch_4-2.csv'])
137
138
139 def cost_bridge():
140     s=read('strategy_summary.csv').set_index('mode');f=read('daily_cost_pairs.csv');delta=s.loc['4-3']-s.loc['4-2']
141     vals=np.array([delta.planned_cost,delta.adjustment_cost,delta.emergency_cost])/10000
142     fig,axes=plt.subplots(1,2,figsize=(9.8,4.8),layout='constrained')
143     ax=axes[0];current=0
144     for i,v in enumerate(vals):
145         gradient_rect(ax,i-.29,current,.58,v,APRICOT if v>0 else BLUE)
146         ax.text(i,max(current,current+v)+4.5,f'{v:+.2f}',ha='center',fontsize=10,color=INK)
147         current+=v
148         if i<2:ax.plot([i+.29,i+.71],[current,current],color=GRAY,ls=':',lw=.9)
149     gradient_rect(ax,2.71,0,.58,current,DARK);ax.text(3,current-3.2,f'{current:+.2f}',ha='center',
    fontsize=10,color=DARK)
150     ax.axhline(0,color=GRAY,lw=.8);ax.set_xlim(-.55,3.55);ax.set_ylim(-42,36)
151     ax.set_xticks(range(4),['基准购电费','净调整费','紧急购电费','总费用变化']);ax.tick_params(axis='x',
    labelsz=9)
152     ax.set_ylabel('4-3 - 4-2 费用变化 / 万元');title(ax,'(a) 费用变化的分项桥图');clean(ax)
153     ax.text(.02,.96,'负值: 4-3支出较低',transform=ax.transAxes,fontsize=9,color=GRAY)
154     ax=axes[1];v=np.sort(f.saving_yuan.to_numpy()/1000);prob=np.arange(1,len(v)+1)/len(v)
155     ax.step(v,prob,where='post',color=DARK,lw=2);ax.fill_between(v,0,prob,step='post',color=ICE,
    alpha=.3)
156     ax.axvline(0,color=GRAY,ls='--',lw=1);ax.set_ylim(0,1.02);ax.set_yticks([0,.25,.5,.75,1],['0%','
    25%','50%','75%','100%'])
157     ax.set_xlabel('单日费用差: 4-2 - 4-3 / 千元');ax.set_ylabel('累计日期比例');title(ax,'(b) 334日配对费
    用差的完整分布');clean(ax)
158     fraction=(f.saving_yuan>0).mean();ax.text(.98,.14,f'4-3费用较低: {fraction:.1%}日期\n正式期合计少支出
    {(-delta.total_cost)/10000:.2f} 万元',transform=ax.transAxes,ha='right',fontsize=9.5,color=INK)
159     save(fig,'fig7_3_cost_change_bridge','图7-3 两套完整策略的费用变化与逐日配对分布',
160         f'共同334日的真实结算比较。4-2总费用{s.loc["4-2"],"total_cost":.6f}元，4-3为{s.loc["4-3",
    "total_cost":.6f}元；左图三分项之和精确等于总费用变化。右图保留所有正负日度差，正数表示4-3当日费用较低。两策略
    初始SOC均6000，最终SOC分别{s.loc["4-2"],"final_soc":.3f}/{s.loc["4-3"],"final_soc":.3f}kWh。前瞻长
    度、更新机会和库存轨迹均不同，结果不是纯FIV/OUV或同初态单日控制实验，不能只凭当日差排序节点价值。',
161         ['strategy_summary.csv','daily_cost_pairs.csv'])
162
163
164 def flow_diagram():
165     f=read('energy_flow.csv');s=read('strategy_summary.csv').set_index('mode')
166     fig,axes=plt.subplots(1,2,figsize=(10.5,6.2));fig.subplots_adjust(left=.015,right=.985,top=.94,
    bottom=.05,wspace=.045)
167     max_total=f[f.side=='input'].groupby('mode').kwh.sum().max();scale=.60/max_total
168     colors={'called':DARK,'pv':BLUE,'discharge':PERI,'emergency':APRICOT,'load':DARK,'charge':PERI,'
    spill':LILAC}
169     names={'called':'电网调用','pv':'实际光伏','discharge':'电池放电','emergency':'紧急购电','load':'负荷'
    , 'charge':'电池充电','spill':'实际弃电'}
170     def ribbon(ax,x0,x1,y0,y1,h,c):
171         mid=(x0+x1)/2
172         verts=[(x0,y0),(mid,y0),(mid,y1),(x1,y1),(x1,y1+h),(mid,y1+h),(mid,y0+h),(x0,y0+h),(x0,y0)]
173         codes=[MPPath.MOVETO,MPPath.CURVE4,MPPath.CURVE4,MPPath.CURVE4,MPPath.LINETO,MPPath.CURVE4,MPPath.
    CURVE4,MPPath.CURVE4,MPPath.CLOSEPOLY]
174         patch=PathPatch(MPath(verts,codes),facecolor='none',edgecolor=c,lw=.3,zorder=3);ax.add_patch
    (patch)

```

```

175     cm=LinearSegmentedColormap.from_list('ribbon',[tuple(.58+.42*np.asarray(to_rgb(c))),c])
176     im=ax.imshow(np.linspace(0,1,200)[None],extent=(x0,x1,min(y0,y1),max(y0,y1)+h),aspect='auto'
,origin='lower',cmap=cm,alpha=.80,zorder=2);im.set_clip_path(patch)
177 for ax,mode in zip(axes,['4-2','4-3']):
178     ax.set(xlim=(0,1),ylim=(0,1));ax.axis('off');row=f[f['mode']==mode].set_index('key');total=
row.loc[['called','pv','discharge','emergency'],'kwh'].sum()
179     ax.text(.5,.98,f'{mode}'+('日前48h前瞻' if mode=='4-2' else '四节点24h前瞻'),ha='center',
fontsize=13,color=DARK)
180     ax.text(.5,.94,'累计电量 / GWh · 带宽同尺度',ha='center',fontsize=9,color=GRAY)
181     top=.86;bus_top=top;input_top=top
182     for key in ['called','pv','discharge','emergency']:
183         value=row.loc[key,'kwh'];h=value*scale;input_bottom=input_top-h;bus_bottom=bus_top-h
184         ribbon(ax,.22,.47,input_bottom,bus_bottom,h,colors[key]);ax.add_patch(Rectangle((.207,
input_bottom),.013,h,facecolor=colors[key],lw=0))
185         ax.text(.193,input_bottom+h/2,f'{names[key]}\n{value/1e6:.3f}',ha='right',va='center',
fontsize=9,color=INK)
186         input_top=input_bottom-.026;bus_top=bus_bottom
187         bus_bottom=top-total*scale;ax.add_patch(Rectangle((.47,bus_bottom),.05,total*scale,facecolor
=ICE,edgecolor=BLUE,lw=.6,zorder=5))
188         ax.text(.495,top+.025,'能量母线',ha='center',fontsize=9,color=GRAY)
189         bus_top=top;output_top=top
190         for key in ['load','charge','spill']:
191             value=row.loc[key,'kwh'];h=value*scale;output_bottom=output_top-h;bus_bottom=bus_top-h
192             ribbon(ax,.52,.78,bus_bottom,output_bottom,h,colors[key]);ax.add_patch(Rectangle((.78,
output_bottom),.013,h,facecolor=colors[key],lw=0))
193             ax.text(.807,output_bottom+h/2,f'{names[key]}\n{value/1e6:.3f}',ha='left',va='center',
fontsize=9,color=INK)
194             output_top=output_bottom-.026;bus_top=bus_bottom
195             called=s.loc[mode,'called']/s.loc[mode,'grid'];ax.text(.5,.135,'结算额度的分拆（不属于物理流入）',
ha='center',fontsize=9.5,color=GRAY)
196             ax.add_patch(Rectangle((.12,.075),.76*called,.035,facecolor=BLUE,lw=0))
197             ax.add_patch(Rectangle((.12+.76*called,.075),.76*(1-called),.035,facecolor=LILAC,edgecolor=
GRAY,lw=.4,hatch='///'))
198             ax.text(.12,.042,f'调用 {called:.1%}',ha='left',fontsize=9,color=BLUE)
199             ax.text(.88,.042,f'未调用 {1-called:.1%}\n{s.loc[mode,"unused"]/1e6:.3f} GWh',ha='right',va='
top',fontsize=9,color=GRAY)
200 save(fig,'fig7_4_energy_flow_and_entitlement','图7-4 真实能量流与购电额度的调用分拆',
201     '全年正式334日累计AC侧电量, 1GWh=106kWh。两面带宽统一按绝对电量绘制, 输入=PV+电网调用+紧急+放电, 输出=
负荷+充电+实际弃电, 逐日逐槽亦满足平衡。所有流带仅连接公共能量母线, 不假定某一来源供给某一特定去向。下方条形各以该
策略结算grid为100%, 独立展示called与unused: unused没有进入物理母线, 也不与spill合并。充放电为AC侧真实计划
量, SOC及损耗另按0.9效率核算。',
202     ['energy_flow.csv','strategy_summary.csv'])
203
204
205 def panorama():
206     a=read('riskday_4-2.csv');b=read('riskday_4-3.csv');ea=read('riskday_soc_4-2.csv');eb=read('
riskday_soc_4-3.csv');x=b.hour_right.to_numpy();dt=1/6
207     fig,axes=plt.subplots(4,1,figsize=(10.2,9.3),sharex=True,layout='constrained')
208     ax=axes[0];ax.fill_between(x,0,b.price,color=ICE,alpha=.25);ax.plot(x,b.price,color=DARK,lw=1.9,
label='实际电价')
209     ax.plot(x,b.price_forecast,color=BLUE,ls='--',lw=1.35,label='00:00冻结预测');ax.set_ylabel('电价 /
(元/kWh)');ax.set_ylim(bottom=0)
210     title(ax,'(a) 7月1日: 实际电价与冻结预测',pad=28);ax.legend(loc='lower left',bbox_to_anchor=(0,1.01)
,ncol=2,fontsize=9)
211     ax=axes[1];ax.fill_between(x,0,b.called,step='pre',color=BLUE,alpha=.25,label='实际调用 x')
212     ax.fill_between(x,b.called,b.grid,step='pre',color=LILAC,alpha=.75,label='未调用 u')
213     ax.step(x,b.grid,where='pre',color=DARK,lw=1.65,label='最终结算额度 G');ax.step(x,b.g0,where='pre'

```



```

, color=PERI, ls='--', lw=1.25, label='00:00基准 $g^0$')
214 ax.bar(x-dt/2, b.emergency, width=dt*.9, color=APRICOT, label='紧急 q', lw=0, zorder=4); ax.set_ylabel('
4-3电量 / kWh'); ax.set_ylim(bottom=0)
215 title(ax, '(b) 调用、未用额度与紧急缺口', pad=28); ax.legend(loc='lower left', bbox_to_anchor=(0, 1.01),
ncol=5, fontsize=8.7, columnspacing=.9)
216 ax=axes[2]; ax.plot(x, a.charge-a.discharge, color=PERI, ls='--', lw=1.5, label='4-2 净充放电')
217 ax.plot(x, b.charge-b.discharge, color=DARK, lw=1.7, label='4-3 净充放电'); ax.fill_between(x, 0, b.
charge-b.discharge, color=BLUE, alpha=.10)
218 ax.axhline(0, color=GRAY, lw=.65); ax.set_ylim(-950, 950); ax.set_yticks([-800, 0, 800]); ax.set_ylabel('
净动作 / kWh'); title(ax, '(c) 节点预提交的储能动作', pad=28)
219 ax.legend(loc='lower left', bbox_to_anchor=(0, 1.01), ncol=2, fontsize=9)
220 ax=axes[3]; ax.plot(ea.hour, ea.soc_kwh, color=PERI, ls='--', lw=1.6, label='4-2 SOC'); ax.plot(eb.hour
, eb.soc_kwh, color=DARK, lw=1.9, label='4-3 SOC')
221 ax.fill_between(eb.hour, 1200, eb.soc_kwh, color=ICE, alpha=.2)
222 for level in [1200, 10800]: ax.axhline(level, color=GRAY, ls=':', lw=.8)
223 ax.set_ylim(450, 11600); ax.set_yticks([1200, 6000, 10800]); ax.set_ylabel('SOC / kWh'); title(ax, '(d)
独立连续库存链的真实SOC', pad=28)
224 ax.legend(loc='lower left', bbox_to_anchor=(0, 1.01), ncol=2, fontsize=9)
225 for ax in axes: timeaxis(ax); nodes(ax); clean(ax)
226 axes[-1].set_xlabel('时刻 / h')
227 save(fig, 'fig7_5_high_risk_panorama', '图7-5 高联合风险日的价—量—库存运行全景',
228 f'2025-07-01为之前Q4敏感性方案预先选定的高联合风险日。00点价格预测全天不更新。4-3面板实际调用/unused堆叠
正好等于最终grid; 紧急量单独按真实值展示。4-2与4-3日初SOC分别{a.initial_soc.iloc[0]:.3f}/{b.initial_soc.
iloc[0]:.3f}kWh, 来自各自前日, 不能当作相同初态单日受控实验。两策略当日紧急量分别{a.emergency.sum():.3f}/{b
.emergency.sum():.3f}kWh。g/c/r在承诺块内预先确定, 储能动作不按事后场景重选; 两侧效率均0.9。',
229 ['riskday_4-2.csv', 'riskday_4-3.csv', 'riskday_soc_4-2.csv', 'riskday_soc_4-3.csv'])
230
231
232 def sensitivity():
233 f=read('local_sensitivity.csv'); f=f[f.parameter=='rho']; fig, axes=plt.subplots(1, 2, figsize
=(10, 5.1)); fig.subplots_adjust(left=.08, right=.98, bottom=.17, top=.76, wspace=.25)
234 colors=[BLUE, DARK, PERI, '#9380BA']; linestyle=['-', '--', '-.', ':']; xlim=max(abs(f.delta_cost_pct).
max(), 1)*1.25; ylim=max(abs(f.delta_emergency_kwh).max(), 1)*1.24
235 for ax, mode in zip(axes, ['4-2', '4-3']):
236 group=f[f['mode']==mode]
237 for i, (date, g) in enumerate(group.groupby('date', sort=True)):
238 g=g.sort_values('setting'); color=colors[i]
239 ax.plot(g.delta_cost_pct, g.delta_emergency_kwh, color=color, ls=linestyle[i], lw=1.65,
label=date[5:]+ ' '+str(g.role.iloc[0]))
240 for row in g.itertuples():
241 marker={.75: 'v', 1.0: 'o', 1.25: '^'}[row.setting]
242 ax.plot(row.delta_cost_pct, row.delta_emergency_kwh, marker=marker, ms=7, color=color,
mfc='white' if row.setting==1 else color, mew=1)
243 ax.axhline(0, color=GRAY, lw=.7); ax.axvline(0, color=GRAY, lw=.7); ax.set_xlim(-xlim, xlim); ax.
set_ylim(-ylim, ylim); clean(ax)
244 title(ax, mode+' '+'(4个代表日 if mode=='4-2' else '2个代表日)'); ax.set_xlabel('相对同日基准成本
变化 / %'); ax.set_ylabel('紧急电量变化 / kWh')
245 ax.legend(loc='lower center', bbox_to_anchor=(.5, 1.15), ncol=2, fontsize=8.5, columnspacing=1)
246 fig.text(.5, .055, '▼ 0.75p* ○ 1.00p* (各日基准在原点) ▲ 1.25p* · 仅连接三个实测参数点'
, ha='center', fontsize=10, color=GRAY)
247 save(fig, 'fig7_6_radius_cost_risk_paths', '图7-6 鲁棒半径扰动的成本—紧急风险轨迹',
248 '复用已完成敏感性实验: 4-2四代表日、4-3两代表日。只绘p=0.75/1/1.25倍基准的18条含基准记录; 共同横纵轴尺
度, 纵轴为紧急量绝对变化, 避免零或很小分母夸大百分比。三角形分别标弱/强半径, 空心圆为同日基准, 线只是离散点的辅助连
接, 不是已计算连续前沿。全部新增节点Optimal; 不能从局部日推出全年参数稳健, 终端库存不同须结合原表解释。',
249 ['local_sensitivity.csv'])
250
251

```



```

252 def main():
253     global OUT, DATA
254     parser=argparse.ArgumentParser();parser.add_argument('--data-dir',type=Path,default=DATA);parser
        .add_argument('--output-dir',type=Path,default=OUT)
255     args=parser.parse_args();DATA=args.data_dir.resolve();OUT=args.output_dir.resolve();OUT.mkdir(
        parents=True,exist_ok=True);fonts()
256     with plt.rc_context(STYLE):
257         for fn in [mechanism,price_clock,cost_bridge,flow_diagram,panorama,sensitivity]:fn()
258     (OUT/'manifest.json').write_text(json.dumps(MANIFEST,ensure_ascii=False,indent=2),encoding='utf
        -8')
259     (OUT/'captions.md').write_text('# 问题四科研图图注\n\n'+'\n\n'.join('## '+r['title']+'\n\n'+r['
        caption'] for r in MANIFEST)+'\n',encoding='utf-8')
260     cards=''.join(f'<article><h2>{html.escape(r["title"])}</h2><a href="{r["png"]}"></a><p>{html.escape(r["caption"])}</p><a href="{r["svg
        "]}>SVG矢量图</a></article>' for r in MANIFEST)
261     page='''<!doctype html><html lang="zh-CN"><meta charset="utf-8"><meta name="viewport" content="
        width=device-width,initial-scale=1"><title>问题四 · 蓝莲花科研图</title><style>body{margin:0;
        background:linear-gradient(135deg,#fffef0,#edf3ff 45%,#ece8f7);color:#002254;font:16px/1.65 -
        apple-system,"Songti SC",serif}main{max-width:1140px;margin:auto;padding:28px 20px}article{
        background:white;padding:25px;margin:26px 0;border-radius:14px}img{width:100%;height:auto}h1{
        font-size:28px}h2{font-size:21px}p{font-size:14px;color:#64748b}a{color:#4b69ef}</style><main><
        h1>问题四 · 蓝莲花与月白深蓝</h1><p>六张科研图 · TeX Fandol宋体 / Latin Modern · 400dpi PNG / SVG ·
        334日正式结果</p>''' + cards + '</main></html>'
262     (OUT/'index.html').write_text(page,encoding='utf-8')
263
264
265 if __name__=='__main__':main()

```

src/plots/q4_paper_package.py

```

1  """问题四图组与方案的自包含数据ZIP。"""
2  from pathlib import Path
3  import csv,hashlib,json,shutil,zipfile
4  from .q4_paper_data import ROOT,OUT,DATA
5
6  DOCS=ROOT/'docs/plots/q4_paper_final'
7  PACKAGE=ROOT/'outputs/processed/deliverables/问题四科研图_蓝莲花渐变_LaTeX字体_20260913'
8
9
10 def prepare():
11     PACKAGE.mkdir(parents=True,exist_ok=True)
12     for path in OUT.iterdir():
13         if path.is_file() and path.suffix in ['.png','.svg','.json','.md','.html']:shutil.copy2(path
            ,PACKAGE/path.name)
14     shutil.copytree(DATA,PAGEAGE/'data',dirs_exist_ok=True)
15     for name in ['ThemeDisplayer.html','问题四科研图方案.md','图后分析与使用建议.md','acceptance.md']:
16         shutil.copy2(DOCS/name,PAGEAGE/name)
17     folder=PACKAGE/'src/plots';folder.mkdir(parents=True,exist_ok=True)
18     for name in ['q4_paper_data.py','q4_paper_final.py','q4_paper_verify.py','q4_paper_package.py']:
19         shutil.copy2(ROOT/'src/plots'/name,folder/name)
20     for p in [PACKAGE/'src/__init__.py',folder/'__init__.py']:p.write_text('',encoding='utf-8')
21     (PACKAGE/'requirements.txt').write_text('numpy==2.3.5\npandas==2.2.3\nmatplotlib==3.10.8\nPillow
        ==12.3.0\n',encoding='utf-8')
22     readme='''# 问题四科研图：蓝莲花渐变与LaTeX字体
23
24  包含6张完整科研图，各有400dpi PNG和SVG，另附做图方案、逐图图注、图后分析、数据及重绘代码。选取用户ThemeDisplayer中

```

的Blue Lotus与月白深蓝配色；中文直接采用TeX Live的Fandol宋体，英文/数字Latin Modern，数学Computer Modern，未使用微软雅黑。

图件

| 文件名（均有PNG/SVG） | 内容 | 正文建议 |
|------------------------------------|--------------------------------|---------|
| fig7_1_joint_dro_mechanism | 严格因果预测、联合误差DRO、双机制窗口、结算与物理执行流程 | 推荐正文 |
| fig7_2_price_clock_and_density | 24小时电价时钟与48096点预测密度 | 电价预测章节 |
| fig7_3_cost_change_bridge | 费用分项变化桥图与334日配对差ECDF | 推荐正文 |
| fig7_4_energy_flow_and_entitlement | 实际电量守恒流带及调用/未用额度拆分 | 物理语义或附录 |
| fig7_5_high_risk_panorama | 7月1日价格、采购、净充放电、SOC四行全景 | 推荐正文 |
| fig7_6_radius_cost_risk_paths | 0.75/1/1.25p*的局部成本—紧急风险轨迹 | 敏感性章节 |

文档与数据

- `问题四科研图方案.md`：配色、字体、图形选择、真实数据范围与验收标准。
- `captions.md`、`图后分析与使用建议.md`：完整图注、可用的图后分析及结论边界。
- `ThemeDisplay.html`：用户提供的配色页原件，图件只引用其中颜色。
- `index.html`：解压后直接打开的离线预览图集。
- `data/dispatch\_4-2.csv`、`dispatch\_4-3.csv`：两套正式334日×144段全量轨迹。
- `data/daily\_\*.csv`、`daily\_cost\_pairs.csv`、`strategy\_summary.csv`：每日结算、逐日费用差与总量。
- `data/hourly\_prices.csv`、`price\_clock.csv`：每小时6个时段均值及334日分位汇总。
- `data/energy\_flow.csv`：全年真实母线流入/流出AC侧电量；未调用额度另见strategy_summary。
- `data/riskday\_\*.csv`、`riskday\_soc\_\*.csv`：两策略7月1日原始轨迹及各自含日初的145点SOC。
- `data/local\_sensitivity.csv`：已做Q4半径与联合权重实验共30行含基准；图7-6只取18条半径记录。
- `data/solver\_quality.csv`：1670个正式最优节点，保留场景数、半径、ESS、初态和执行长度。
- `data/provenance.json`：来源、模型配置、统计口径、选日规则和682个原始文件SHA-256。
- `manifest.json`：各图与CSV/脚本的映射；`verification.json`：独立数值与导出检查；`acceptance.md`：逐项需求与视觉审查。
- `src/plots/`：UTF-8源代码；`requirements.txt`：绘图环境版本。
- `file\_manifest.csv`：包内每个文件的SHA-256，便于完整性核验。

口径

正式范围为2025年2月1日至12月31日，共334日。4-2使用完成并验收的v2结果，4-3使用v3结果，全部1670节点Optimal。两套策略初始SOC都为6000，但各自库存链不同、最终库存也不同，因此费用差是完整策略实际表现，不是纯信息价值FIV/OUV。

图7-4中未调用额度unused不是实际弃电spill，不进入物理能量母线；流图不虚构来源到具体设备的分配。日内时钟分位带是跨日分布范围，非置信区间。敏感性为少量代表日局部实验，不能外推全年参数稳健。

PNG可直接插入论文。SVG文字已转曲，不依赖本机字体；渐变色带等包含高分辨率栅格填充，轮廓/坐标/文字为矢量。图中透明色已按白底导出。请在最终插图尺寸检查字号，勿过度缩小多面板。

重绘

安装requirements所列库后，在解压目录执行：

```
```sh
python -m src.plots.q4_paper_final --data-dir data --output-dir redraw
```
```

脚本优先读取解压目录下`assets/q4\_fonts/FandolSong-Regular.otf`和`assets/q4\_fonts/lmroman10-regular.otf`，否则读取本机`/usr/local/texlive/2026/`同名字体。字体文件不包含在ZIP内；可使用自己的TeX Live字体。现成PNG/SVG不需要安装字体。

数据准备与原始文件验证脚本需要完整项目路径，离线重绘只需本包data目录；不调用求解器。此前所有结果和ZIP均未覆盖。

```

74 '''
75 (PACKAGE/'README.md').write_text(readme,encoding='utf-8')
76 print('Prepared',PACKAGE)
77
78
79 def archive():
80     files=[p for p in sorted(PACKAGE.rglob('*')) if p.is_file() and p.name!='file_manifest.csv']
81     with (PACKAGE/'file_manifest.csv').open('w',encoding='utf-8',newline='') as stream:
82         w=csv.DictWriter(stream,fieldnames=['path','bytes','sha256']);w.writeheader()
83         w.writerows({'path':str(p.relative_to(PACKAGE)),'bytes':p.stat().st_size,'sha256':hashlib.
84             sha256(p.read_bytes()).hexdigest()} for p in files)
85     files=[p for p in sorted(PACKAGE.rglob('*')) if p.is_file()]
86     target=PACKAGE.with_suffix('.zip')
87     with zipfile.ZipFile(target,'w',compression=zipfile.ZIP_DEFLATED,compresslevel=6) as z:
88         for p in files:z.write(p,str(Path(PACKAGE.name)/p.relative_to(PACKAGE)))
89     with zipfile.ZipFile(target) as z:
90         assert z.testzip() is None and len(z.namelist())==len(files)
91         for p in files:
92             assert hashlib.sha256(z.read(str(Path(PACKAGE.name)/p.relative_to(PACKAGE)))).digest()==
93                 hashlib.sha256(p.read_bytes()).digest()
94         result={'zip':str(target),'bytes':target.stat().st_size,'members':len(files),'PNG':6,'SVG':6,
95             'sha256':hashlib.sha256(target.read_bytes()).hexdigest(),'CRC_and_member_SHA256':'passed'}
96     (DOCS/'zip_acceptance.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding=
97         'utf-8');print(result)
98
99 if __name__=='__main__':
100     import sys
101     if '--archive' in sys.argv:archive()
102     else:prepare()

```

src/plots/q4_paper_verify.py

```

1  """Q4图组的独立数据、几何与导出核。"""
2  from pathlib import Path
3  import hashlib,json,xml.etree.ElementTree as ET
4  import numpy as np
5  import pandas as pd
6  from PIL import Image
7  from .q4_paper_data import ROOT,OUT,DATA,RUNS
8
9
10 def main():
11     meta=json.loads((DATA/'provenance.json').read_text(encoding='utf-8'))
12     for name,sha in meta['source_sha256'].items():assert hashlib.sha256((ROOT/name).read_bytes()).
13         hexdigest()==sha,name
14     read=lambda name:pd.read_csv(DATA/name,float_precision='round_trip')
15     max_balance=0.;max_soc=0.;originals={}
16     for mode,path in RUNS.items():
17         f=read('dispatch_'+mode+'.csv');raw=pd.read_csv(path/'dispatch.csv',float_precision='
18             round_trip')
19         pd.testing.assert_frame_equal(f[raw.columns],raw,check_exact=False,atol=1e-10,rtol=0)
20         assert f.groupby('date').size().eq(144).all() and len(f)==334*144
21         assert pd.to_datetime(f.timestamp).diff().dropna().eq(pd.Timedelta(minutes=10)).all()
22         r=f.called+f.discharge+f.emergency-f.net_kwh-f.charge-f.spill;max_balance=max(max_balance,

```

```

float(abs(r).max()))
21     np.testing.assert_allclose(r,0,atol=1e-7,rtol=0)
22     np.testing.assert_allclose(f.grid-f.called,f.unused,atol=1e-7,rtol=0)
23     np.testing.assert_allclose(f.unused+f.spill,f.total_surplus,atol=1e-7,rtol=0)
24     soc=f.soc-np.r_[6000,f.soc.to_numpy()[:-1]]-.9*f.charge+f.discharge/.9
25     max_soc=max(max_soc,float(abs(soc).max()));np.testing.assert_allclose(soc,0,atol=1e-7,rtol
=0)
26     assert f.soc.between(1200-1e-6,10800+1e-6).all() and (f.charge+f.discharge<=5000/6+1e-6).all
()
27     np.testing.assert_allclose(f.charge,f.charge_plan,atol=1e-7,rtol=0);np.testing.
assert_allclose(f.discharge,f.discharge_plan,atol=1e-7,rtol=0)
28     delta=f.grid-f.g0;cost=f.price*(f.g0+1.5*delta.clip(lower=0)+.5*delta.clip(upper=0)+5*f.
emergency)
29     np.testing.assert_allclose(f.total_cost,cost,atol=1e-7,rtol=0)
30     originals[mode]=f
31     f=originals['4-2'];clock=read('price_clock.csv')
32     actual=f.price.to_numpy().reshape(334,24,6).mean(axis=2);pred=f.price_forecast.to_numpy().
reshape(334,24,6).mean(axis=2)
33     np.testing.assert_allclose(clock.actual_median,np.median(actual,axis=0),atol=1e-12,rtol=0)
34     np.testing.assert_allclose(clock.actual_p10,np.quantile(actual,.1,axis=0),atol=1e-12,rtol=0)
35     np.testing.assert_allclose(clock.actual_p90,np.quantile(actual,.9,axis=0),atol=1e-12,rtol=0)
36     np.testing.assert_allclose(clock.forecast_median,np.median(pred,axis=0),atol=1e-12,rtol=0)
37     limit=max(f.price.max(),f.price_forecast.max())*1.025
38     density,xe,ye=np.histogram2d(f.price_forecast,f.price,bins=65,range=[[0,limit],[0,limit]])
39     assert density.sum()==48096
40     flow=read('energy_flow.csv');summaries=read('strategy_summary.csv').set_index('mode')
41     for mode in RUNS:
42         g=flow[flow['mode']==mode];np.testing.assert_allclose(g[g.side=='input'].kwh.sum(),g[g.side
=='output'].kwh.sum(),atol=1e-6,rtol=0)
43         np.testing.assert_allclose(summaries.loc[mode,'called']+summaries.loc[mode,'unused'],
summaries.loc[mode,'grid'],atol=1e-6,rtol=0)
44         delta=summaries.loc['4-3']-summaries.loc['4-2'];np.testing.assert_allclose(delta[['planned_cost'
,'adjustment_cost','emergency_cost']].sum(),delta.total_cost,atol=1e-7,rtol=0)
45         pairs=read('daily_cost_pairs.csv');np.testing.assert_allclose(pairs.saving_yuan.sum(),-delta.
total_cost,atol=1e-7,rtol=0)
46         s=read('local_sensitivity.csv');rho=s[s.parameter=='rho'];assert len(rho)==18 and rho.
all_optimal.all()
47         for (mode,day),g in rho.groupby(['mode','date']):
48             assert set(g.setting)=={.75,1,1.25};b=g[g.setting==1].iloc[0]
49             np.testing.assert_allclose(g.delta_cost_pct,(g.total_cost/b.total_cost-1)*100,atol=1e-9,rtol
=0)
50             np.testing.assert_allclose(g.delta_emergency_kwh,g.emergency_kwh-b.emergency_kwh,atol=1e-9,
rtol=0)
51         rows=[]
52         for r in json.loads((OUT/'manifest.json').read_text(encoding='utf-8')):
53             with Image.open(OUT/r['png']) as im:
54                 assert im.format=='PNG' and all(abs(x-400)<.1 for x in im.info['dpi'])
55                 assert np.max(abs(np.array(im.size)-np.array([r['width_inches'],r['height_inches']]
*400))<1.1
56                 shape=im.size
57                 xml=ET.parse(OUT/r['svg']);assert not xml.findall('.//{http://www.w3.org/2000/svg}text')
58                 rows.append({'name':r['name'],'pixels':shape,'dpi':400,'svg_glyph_paths':True})
59         assert len(rows)==6
60         log=(ROOT/'docs/plots/q4_paper_final/render.log').read_text(encoding='utf-8').lower()
61         assert not any(term in log for term in ['warning','glyph','dummy','traceback'])
62         result={'status':'passed','source_hashes':len(meta['source_sha256']),'formal_days_each':334,'
slots_each':48096,

```

```

63     'optimal_nodes':1670,'density_included_samples':int(density.sum()),'radius_records':18,'
max_balance_residual':max_balance,
64     'max_soc_residual':max_soc,'total_cost_delta_q43_minus_q42':float(delta.total_cost),'figures
':rows,
65     'all_source_files_unchanged':True,'caveats':'Two complete policies, different horizons and
inventory paths; not pure information value.')}
66 (OUT/'verification.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='
utf-8')
67 print({k:v for k,v in result.items() if k!='figures'})
68
69
70 if __name__=='__main__':main()

```

src/plots/replot_existing.py

```

1  """从已验收的CSV/JSON重绘原有Q1四组图或Q2十一组图，不重算统计或优化。"""
2  import argparse
3  import json
4  from pathlib import Path
5  import sys
6
7  ROOT = Path(__file__).resolve().parents[2]
8  sys.path.insert(0, str(ROOT))
9  from src.plots.common import digest, read_csv
10
11
12 def numeric(rows):
13     for row in rows:
14         for key, value in row.items():
15             try:
16                 row[key] = int(value) if value.lstrip('-').isdigit() else float(value)
17             except (ValueError, TypeError):
18                 pass
19     return rows
20
21
22 def replot(question, output):
23     generation = (ROOT / f'outputs/{question}_current').resolve(strict=True)
24     manifest = json.loads((generation / 'raw/run_manifest.json').read_text(encoding='utf-8'))
25     for rel, value in manifest['output_sha256'].items():
26         if digest(generation / rel) != value:
27             raise ValueError(f'来源产物哈希不符: {rel}')
28     output = Path(output).resolve()
29     # 原运行代及其任何别名都属于只读来源。
30     if output == generation or generation in output.parents:
31         raise ValueError('不能将重绘结果写入已验收运行代')
32     output.mkdir(parents=True, exist_ok=True)
33     source = generation / 'processed'
34     if question == 'q1':
35         from src.plots.q1_legacy import analysis_figures, create_figures
36         data = numeric(read_csv(generation / 'inputs/processed/timeseries.csv'))
37         solutions = {name: json.loads((generation / f'raw/solution_{name}.json').read_text(encoding=
'utf-8'))
38                     for name in ['main', 'eta085', 'eta095', 'roundtrip090']}
39         create_figures(data, solutions, numeric(read_csv(source / 'efficiency_comparison.csv')),
output)

```

```

40     a = {'main_marginals': [r for r in numeric(read_csv(source / 'marginal_values.csv')) if r['
scenario'] == 'main_l100_pv100'],
41         'baseline_rows': numeric(read_csv(source / 'baseline_comparison.csv')),
42         'scenarios': numeric(read_csv(source / 'sensitivity_scenarios.csv')),
43         'bottlenecks': numeric(read_csv(source / 'bottleneck_values.csv'))}
44     analysis_figures(a, output)
45 else:
46     import pandas as pd
47     from src.plots.q2 import create_figures
48     frame = pd.read_csv(source / 'timeseries.csv', parse_dates=['date', 'timestamp'],
49 float_precision='round_trip')
49     diagnostics = {}
50     for name in ['profiles', 'acf_pacf', 'spectrum', 'monthly_stability', '
transformed_monthly_moments', 'daily_statistics']:
51         diagnostics[name] = pd.read_csv(source / f'full_year_{name}.csv', float_precision='
round_trip',
52                                         parse_dates=['date'] if name == 'daily_statistics' else
None)
53     profiles = diagnostics['profiles']
54     numeric_groups = profiles.grouping.isin(['month', 'weekday'])
55     profiles.loc[numeric_groups, 'group'] = profiles.loc[numeric_groups, 'group'].astype(int)
56     figures = create_figures(frame, diagnostics, output)
57     (output / 'figure_manifest.json').write_text(json.dumps(figures, ensure_ascii=False, indent=2)
+ '\n', encoding='utf-8')
58     print(f'{question}重绘完成: {output}', flush=True)
59
60
61 if __name__ == '__main__':
62     parser = argparse.ArgumentParser(description=__doc__)
63     parser.add_argument('question', choices=['q1', 'q2'])
64     parser.add_argument('--output', type=Path, required=True)
65     args = parser.parse_args()
66     replot(args.question, args.output)

```

src/plots/run.py

```

1  """统一绘图命令；各题使用原固定依赖，避免Q1/Q2库版本互相覆盖。"""
2  import argparse
3  from pathlib import Path
4  import subprocess
5  import sys
6
7  ROOT = Path(__file__).resolve().parents[2]
8
9
10 def main():
11     parser = argparse.ArgumentParser(description=__doc__)
12     parser.add_argument('question', choices=['q1', 'q2', 'q2-dispatch', 'all'], nargs='?', default='q1'
13 )
14     parser.add_argument('--run-dir', type=Path, help='q2-dispatch的完整或部分真实运行目录')
15     args = parser.parse_args()
16     if args.question == 'q2-dispatch':
17         if args.run_dir is None:
18             parser.error('q2-dispatch需要--run-dir')
19         subprocess.run(['uv', 'run', '--with-requirements', 'src/q2/requirements.txt', 'python',
20 'src/plots/q2_dispatch.py', '--run-dir', str(args.run_dir)], cwd=ROOT, check=

```

```

20     True)
21     return
22 for question in (['q1','q2'] if args.question == 'all' else [args.question]):
23     command = ['uv','run','--with-requirements',f'src/{question}/requirements.txt','python','-u'
24 ]
25     command += ['src/plots/q1_paper.py'] if question == 'q1' else [
26         'src/plots/replot_existing.py','q2','--output','outputs/processed/figures/q2']
27     subprocess.run(command, cwd=R00T, check=True)
28     subprocess.run([sys.executable,'src/plots/catalog.py'], cwd=R00T, check=True)
29
30 if __name__ == '__main__':
31     main()

```

src/plots/verify.py

```

1  """论文图验收：磁盘回读、单位/时序/瀑布恒等式、来源及脚本集中化。"""
2  import ast
3  import json
4  from pathlib import Path
5  import sys
6  from xml.etree import ElementTree
7
8  import numpy as np
9  from PIL import Image
10
11 R00T = Path(__file__).resolve().parents[2]
12 sys.path.insert(0, str(R00T))
13 from src.plots.common import digest, read_csv
14 from src.q1.device_sensitivity import verify as verify_device
15
16
17 def close(left, right, tol=1e-6):
18     np.testing.assert_allclose(np.asarray(left,dtype=float), np.asarray(right,dtype=float), atol=tol
19     , rtol=0)
20
21 def main():
22     folder = R00T / 'outputs/processed/figures/q1'
23     provenance = json.loads((folder / 'provenance.json').read_text(encoding='utf-8'))
24     for path, value in provenance['code_sha256'].items():
25         if digest(R00T/path) != value: raise ValueError(f'绘图源码已变化，须重绘: {path}')
26     for path, value in provenance['output_sha256'].items():
27         if digest(folder/path) != value: raise ValueError(f'绘图产物被修改: {path}')
28     generation = R00T / provenance['source_generation']
29     original = read_csv(generation / 'processed/result1.csv')
30     power, energy, features = [read_csv(folder/name) for name in ['dispatch_power.csv','storage_145.csv','data_features.csv']]
31     if (len(original),len(power),len(energy),len(features)) != (144,144,145,144):
32         raise ValueError('144区间/145状态点不完整')
33     for i,(source,drawn,feature) in enumerate(zip(original,power,features)):
34         close([drawn['start_h'],drawn['end_h']], [i/6,(i+1)/6])
35         for key, field in [('G_kwh','optimal_grid_kw'),('C_kwh','charge_kw'),('D_kwh','discharge_kw')]:
36             close(float(drawn[field])/6,float(source[key]))
37         state = '充电' if float(source['C_kwh'])>1e-6 else '放电' if float(source['D_kwh'])>1e-6 else

```



```

'待机'
38     if drawn['state']!=state: raise ValueError('状态条与实际C/D不符')
39     close(float(drawn['no_ess_grid_kw']), max(float(source['load_kw'])-float(source['pv_kw']),0)
)
40     close(float(feature['surplus_pv_kw']), max(float(source['pv_kw'])-float(source['load_kw'])
,0))
41     close(float(energy[i+1]['energy_kwh']),float(source['E_kwh']))
42 close([float(energy[0]['energy_kwh']),float(energy[-1]['energy_kwh']),[6000,6000])
43 close([float(r['time_h']) for r in energy],np.arange(145)/6)
44 waterfall=read_csv(folder/'cost_waterfall.csv')
45 cost={r['baseline']:float(r['cost_yuan']) for r in read_csv(folder/'baseline_metrics.csv')}
46 close([float(r['bottom_yuan']) for r in waterfall],[0,cost['pv_only'],0,cost['full'],0])
47 close([float(r['height_yuan']) for r in waterfall],
48         [cost['no_ess'],cost['no_ess']-cost['pv_only'],cost['pv_only'],cost['pv_only']-cost['full']
],cost['full']))
49 if len(read_csv(folder/'efficiency_storage.csv')) != 580 or len(read_csv(folder/'
input_sensitivity.csv')) != 9:
50     raise ValueError('效率轨迹或九格敏感性数据不完整')
51 devices=verify_device([**r,**{k:float(r[k]) for k in ['load_kwh','pv_kwh','price_yuan_per_kwh'
]}} for r in original],
52                        ROOT/provenance['device_analysis'])
53 images=[]
54 for row in read_csv(folder/'figure_index.csv'):
55     with Image.open(folder/(row['figure']+'.png')) as im:
56         im.verify()
57     with Image.open(folder/(row['figure']+'.png')) as im:
58         if min(im.size)<1300 or abs(im.info['dpi'][0]-300)>.01:
59             raise ValueError('PNG尺寸或300dpi元数据错误')
60     images.append({'figure':row['figure'],'pixels':list(im.size),'dpi':list(im.info['dpi'])
})
61     svg=ElementTree.parse(folder/(row['figure']+'.svg'))
62     if not svg.findall('.//{http://www.w3.org/2000/svg}path'):
63         raise ValueError('SVG没有矢量路径')
64 if len(images)!=8: raise ValueError('论文图数不完整')
65 imports=[]
66 for path in (ROOT/'src').rglob('*.py'):
67     raw=path.read_bytes(); text=raw.decode('utf-8')
68     if raw.startswith(b'\xef\xbb\xbf'): raise ValueError(f'源码含BOM: {path}')
69     for node in ast.walk(ast.parse(text)):
70         names=[a.name for a in node.names] if isinstance(node,ast.Import) else [node.module or
'] if isinstance(node,ast.ImportFrom) else []
71         if any(n.split('.')[0] in ['matplotlib','seaborn','plotly','bokeh','altair'] for n in
names):
72             imports.append(str(path.relative_to(ROOT)))
73             if path.parent != ROOT/'src/plots': raise ValueError(f'绘图实现未集中: {path}')
74 result={'passed':True,'checks':['source_and_output_hashes','144_power_intervals','145
_energy_boundaries',
75     'kWh_to_kW','actual_action_states','surplus_shading','waterfall_identity','
efficiency_and_input_scenarios',
76     'all_133_device_solutions','png_300dpi','svg_paths',
all_project_plot_imports_centralized','utf8_no_bom'],
77     'devices':devices,'images':images,'plot_modules':sorted(set(imports))}
78 target=ROOT/'docs/plots/verification.json';target.parent.mkdir(exist_ok=True)
79 target.write_text(json.dumps(result,ensure_ascii=False,indent=2)+'\n',encoding='utf-8')
80 print(json.dumps(result,ensure_ascii=False),flush=True)
81
82

```

```

83 if __name__ == '__main__':
84     main()

```

B.7 问题一补充程序

src/问题 1/review_temporary_plan.py

```

1  """Local audit experiments for 临时方案.md; not a competition submission solver.
2
3  Run: uv run --with scipy --with openpyxl python src/问题1/review_temporary_plan.py
4  Audit convention: input timestamps label the END of a ten-minute interval;
5  power is constant within that interval; charge/discharge are bus-side energies.
6  The original template has a conflicting interval labeling, so these numbers
7  are conditional review evidence, not certified result1.xlsx answers.
8  """
9
10 import hashlib
11 import json
12 from pathlib import Path
13
14 import numpy as np
15 import scipy
16 from openpyxl import load_workbook
17 from scipy.optimize import Bounds, LinearConstraint, linprog, milp
18 from scipy.sparse import csc_matrix
19
20
21 ROOT = Path(__file__).resolve().parents[2]
22 BASE = ROOT / "docs/CUMCM2026Problems/C题/附件"
23 OUT = ROOT / "outputs/q1/raw"
24 DT = 1 / 6
25
26
27 def solve(prices, load, pv, initial=6000, eta=0.9, eta_d=None, lower=1200,
28           upper=10800, power=5000, secondary=None, integer=False):
29     n = len(prices)
30     eta_d = eta if eta_d is None else eta_d
31     size = 5 * n + 1
32     cost = np.zeros(size)
33     cost[:n] = prices
34     eq = np.zeros((2 * n + 1, size))
35     rhs = np.zeros(2 * n + 1)
36     for t in range(n):
37         eq[t, [t, n + t, 2 * n + t, 3 * n + t]] = [1, -1, 1, -1]
38         rhs[t] = load[t] - pv[t]
39         eq[n + t, [n + t, 2 * n + t, 4 * n + t, 4 * n + t + 1]] = [-eta, 1 / eta_d, -1, 1]
40     eq[-1, [4 * n, 5 * n]] = [1, -1]
41     bounds = (([0, None]) * n + [(0, power * DT)] * (2 * n)
42               + [(0, float(s)) for s in pv] + [(lower, upper)] * (n + 1))
43     if initial is not None:
44         bounds[4 * n] = (initial, initial)
45     first = linprog(cost, A_eq=eq, b_eq=rhs, bounds=bounds, method="highs")
46     if not first.success:
47         raise RuntimeError(first.message)
48     result = first
49     if secondary is not None:

```

```

50     objective = np.zeros(size)
51     if secondary == "throughput":
52         objective[n:3 * n] = 1
53     elif secondary == "initial_min":
54         objective[4 * n] = 1
55     elif secondary == "initial_max":
56         objective[4 * n] = -1
57     result = linprog(objective, A_eq=np.vstack([eq, cost]),
58                     b_eq=np.r_[rhs, first.fun], bounds=bounds, method="highs")
59     if not result.success:
60         raise RuntimeError(result.message)
61 if integer:
62     full_eq = np.pad(eq, ((0, 0), (0, n)))
63     mutex = np.zeros((2 * n, size + n))
64     for t in range(n):
65         mutex[t, n + t] = 1
66         mutex[t, size + t] = -power * DT
67         mutex[n + t, 2 * n + t] = 1
68         mutex[n + t, size + t] = power * DT
69     low = [b[0] for b in bounds] + [0] * n
70     high = [np.inf if b[1] is None else b[1] for b in bounds] + [1] * n
71     result = milp(np.r_[cost, np.zeros(n)], integrality=np.r_[np.zeros(size), np.ones(n)],
72                 bounds=Bounds(low, high), constraints=[
73         LinearConstraint(csc_matrix(full_eq), rhs, rhs),
74         LinearConstraint(csc_matrix(mutex), -np.inf,
75             np.r_[np.zeros(n), np.full(n, power * DT)])),
76                 options={"time_limit": 30, "mip_rel_gap": 1e-9})
77     if not result.success:
78         raise RuntimeError(result.message)
79 x = result.x[:size]
80 g, c, d, w, e = x[:n], x[n:2*n], x[2*n:3*n], x[3*n:4*n], x[4*n:]
81 finite_upper = np.array([np.inf if b[1] is None else b[1] for b in bounds])
82 dual = float(rhs @ first.eqlin.marginals + np.array([b[0] for b in bounds]) @ first.lower.
83     marginals
84     + finite_upper[np.isfinite(finite_upper)] @ first.upper.marginals[np.isfinite(
85     finite_upper)])
86 summary = {
87     "cost_yuan": float(cost @ x), "grid_kwh": float(g.sum()),
88     "charge_kwh": float(c.sum()), "discharge_kwh": float(d.sum()),
89     "curtail_kwh": float(w.sum()), "initial_kwh": float(e[0]), "final_kwh": float(e[-1]),
90     "state_min": float(e.min()), "state_max": float(e.max()),
91     "simultaneous_intervals": int(((c > 1e-7) & (d > 1e-7)).sum()),
92     "balance_residual": float(np.max(np.abs(eq @ x - rhs))),
93     "bound_violation": float(max(0, np.max(np.array([b[0] for b in bounds]) - x), np.max(x -
94     finite_upper))),
95     "cycle_residual": float(abs(d.sum() - eta*eta_d * c.sum())),
96     "global_energy_residual": float(abs(g.sum() - (load.sum()-pv.sum()+w.sum()+(1-eta*eta_d)*c.
97     sum()))),
98     "primary_lp_dual_gap": abs(float(first.fun)-dual),
99     "upper_state_raw_sum_including_fixed_initial": float(first.upper.marginals[4*n:].sum()),
100     "upper_capacity_marginal": float(first.upper.marginals[4*n+(initial is not None):].sum()),
101     "lower_capacity_marginal": float(first.lower.marginals[4*n+(initial is not None):].sum()),
102     "initial_bound_marginal": float(first.lower.marginals[4*n]+first.upper.marginals[4*n]),
103     "power_marginal_yuan_per_kw": float(DT * first.upper.marginals[n:3*n].sum()),
104 }
105 return summary, {"grid": g.tolist(), "charge": c.tolist(), "discharge": d.tolist(),
106     "curtail": w.tolist(), "state": e.tolist()}

```

```

103
104
105 def main():
106     ws = load_workbook(BASE / "附件1.xlsx", read_only=True, data_only=True).active
107     rows = list(ws.iter_rows(min_row=2, values_only=True))
108     p, load, pv = (np.array([r[i] for r in rows], dtype=float) for i in (1, 2, 3))
109     load, pv = load * DT, pv * DT
110     template = load_workbook(BASE / "附件5/result1.xlsx", read_only=True)["计划购电量"]
111     report = {"scipy_version": scipy.__version__, "assumptions": __doc__,
112              "source_sha256": {str(f.relative_to(ROOT)): hashlib.sha256(f.read_bytes()).hexdigest()
113                               for f in [ROOT / "临时方案.md", BASE / "附件1.xlsx", BASE / "附件5/
114 result1.xlsx"]},
115              "input": {"intervals": len(rows), "price_min": float(p.min()), "price_max": float(p.
116 max()),
117                        "load_kwh": float(load.sum()), "pv_kwh": float(pv.sum()),
118                        "pv_surplus_kwh": float(np.maximum(pv-load, 0).sum()),
119                        "template_first": template["A2"].value, "template_last": template["A145"].
120 value},
121              "cases": {}, "schedules": {}}
122     cases = {
123         "fixed6000_eta90": {}, "free_initial_eta90": {"initial": None},
124         "fixed6000_roundtrip90": {"eta": np.sqrt(0.9)},
125         "roundtrip90_split_c90_d100": {"eta": 0.9, "eta_d": 1.0},
126         "roundtrip90_split_c100_d90": {"eta": 1.0, "eta_d": 0.9},
127         "fixed6000_secondary": {"secondary": "throughput"}, "fixed6000_milp": {"integer": True},
128         "free_optimal_initial_min": {"initial": None, "secondary": "initial_min"},
129         "free_optimal_initial_max": {"initial": None, "secondary": "initial_max"},
130         "upper_plus_1": {"upper": 10801, "upper_minus_1": {"upper": 10799},
131         "nominal_plus_1_fixed6000": {"lower": 1200.1, "upper": 10800.9},
132         "power_plus_1": {"power": 5001, "power_minus_1": {"power": 4999},
133         "wrong_5000kwh_per_slot": {"power": 30000},
134         "wrong_allow_12000kwh": {"upper": 12000},
135     }
136     for name, options in cases.items():
137         report["cases"][name], report["schedules"][name] = solve(p, load, pv, **options)
138     for name in ["wrong_5000kwh_per_slot", "wrong_allow_12000kwh"]:
139         s = report["schedules"][name]
140         report["cases"][name]["against_original_limits"] = {
141             "max_charge_kw": max(s["charge"]) / DT, "max_discharge_kw": max(s["discharge"]) / DT,
142             "max_stored_kwh": max(s["state"])}
143     report["cases"]["cyclic_timestamp_shift_10min"], _ = solve(np.roll(p,1), np.roll(load,1), np.
144 roll(pv,1))
145     report["baseline"] = {"without_storage_cost": float(p @ np.maximum(load-pv, 0)),
146                          "without_storage_grid_kwh": float(np.maximum(load-pv, 0).sum())}
147     # A synthetic one-slot falsification test, not a claim about attachment 4 prices.
148     report["negative_price_counterexample"], _ = solve(np.array([-1.]), np.array([0.]), np.array
149 ([0.]),
150               initial=0, lower=0, upper=1, power=6)
151     # A synthetic feasible but dissipative optimum: free PV can be curtailed instead.
152     report["free_pv_cycle_example"] = {"pv_kwh": 0.19, "load_kwh": 0,
153                                       "charge_kwh": 1, "discharge_kwh": 0.81,
154                                       "grid_kwh": 0, "state_change_kwh": 0,
155                                       "alternative": "charge=discharge=0, curtail=0.19, same zero
156 cost"}
157     OUT.mkdir(parents=True, exist_ok=True)
158     (OUT / "temporary_plan_review_evidence.json").write_text(json.dumps(report, ensure_ascii=False,
159 indent=2), encoding="utf-8")

```

```
153     print(json.dumps({k:v for k,v in report.items() if k != "schedules"}, ensure_ascii=False, indent
154                   =2))
155
156 if __name__ == "__main__":
157     main()
```